



ORACLE
NetSuite

SuiteScript 2.x API Reference

2025.2

December 17, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

SuiteScript 1.0 to SuiteScript 2.x API Map	1
SuiteScript 1.0 to SuiteScript 2.x API Map – Functions (nlapi)	1
SuiteScript 1.0 to SuiteScript 2.x API Map – Objects (nlobj)	11
SuiteScript 2.x Global Objects	35
define Object	35
define(moduleObject)	36
define(id, [dependencies,] moduleObject)	37
require Function	40
require([dependencies,] callback)	40
require Configuration	42
log Object	43
util Object	44
toString()	44
JSON object	45
JSON.parse(text)	45
JSON.stringify(obj)	46
Promise Object	47
Iterator	53
SuiteScript 2.x Modules	56
N/action Module	61
N/action Module Script Samples	64
action.Action	68
action.execute(options)	78
action.execute.promise(options)	80
action.executeBulk(options)	81
action.find(options)	83
action.find.promise(options)	84
action.get(options)	86
action.get.promise(options)	87
action.getBulkStatus(options)	89
N/auth Module	89
N/auth Module Script Sample	90
auth.changeEmail(options)	91
auth.changePassword(options)	92
N/cache Module	93
N/cache Module Script Samples	95
cache.Cache	98
cache.getCache(options)	104
cache.Scope	105
N/certificateControl Module	105
N/certificateControl Module Script Samples	108
certificateControl.Certificate	111
certificateControl.createCertificate(options)	120
certificateControl.deleteCertificate(options)	122
certificateControl.findCertificates(options)	123
certificateControl.findUsages(options)	124
certificateControl.loadCertificate(options)	125
certificateControl.lock(options)	126
certificateControl.unlock(options)	127
certificateControl.Operation	128
certificateControl.Operator	129
certificateControl.Type	130
N/commerce Modules	130

N/commerce/recordView Module	131
N/commerce/recordView Script Sample	136
N/compress Module	137
N/compress Module Script Sample	138
compress.Archiver	139
compress.gzip(options)	142
compress.gunzip(options)	143
compress.createArchiver()	144
compress.Type	145
N/config Module	146
N/config Module Script Sample	147
config.load(options)	147
config.Type	149
N/crypto Module	150
N/crypto Module Script Samples	153
crypto.Cipher	155
crypto.CipherPayload	158
crypto.Decipher	159
crypto.Hash	162
crypto.Hmac	164
crypto.SecretKey	166
crypto.checkPasswordField(options)	168
crypto.createCipher(options)	170
crypto.createDecipher(options)	171
crypto.createHash(options)	172
crypto.createHmac(options)	172
crypto.createSecretKey(options)	173
crypto.EncryptionAlg	175
crypto.HashAlg	175
crypto.Padding	176
N/crypto/certificate Module	177
N/crypto/certificate Module Script Samples	179
certificate.SignedXml	181
certificate.Signer	183
certificate.Verifier	186
certificate.createSigner(options)	189
certificate.createVerifier(options)	190
certificate.verifyXmlSignature(options)	191
certificate.signXml(options)	192
certificate.HashAlg	193
N/crypto/random Module	194
N/crypto/random Samples	194
random.generateBytes(options)	195
random.generateInt(options)	196
random.generateUUID()	196
N/currency Module	197
N/currency Module Script Sample	197
currency.exchangeRate(options)	198
N/currentRecord Module	200
N/currentRecord Module Script Samples	205
currentRecord.Column	209
currentRecord.CurrentRecord	212
currentRecord.Field	260
currentRecord.Sublist	270
currentRecord.get()	273

currentRecord.get.promise()	274
N/dataset Module	275
N/dataset Module Script Samples	278
dataset.Column	280
dataset.Condition	286
dataset.Dataset	291
dataset.Join	301
dataset.create(options)	304
dataset.createColumn(options)	306
dataset.createCondition(options)	307
dataset.createJoin(options)	309
dataset.createTranslation(options)	311
dataset.describe(options)	311
dataset.describe.promise(options)	312
dataset.list()	313
dataset.listPaged(options)	314
dataset.loadDataset(options)	315
dataset.loadDataset.promise(options)	316
N/datasetLink Module	317
N/datasetLink Module Script Sample	319
datasetLink.DatasetLink	320
datasetLink.create(options)	322
N/documentCapture Module	324
N/documentCapture Module Script Samples	329
Getting Started with the N/documentCapture Module	332
Field Extraction Objects	336
Table Extraction Objects	344
Text Extraction Objects	355
documentCapture.Document	361
documentCapture.Page	365
documentCapture.documentToStructure(options)	371
documentCapture.documentToStructure.promise(options)	374
documentCapture.documentToText(options)	375
documentCapture.documentToText.promise(options)	377
documentCapture.getRemainingConcurrency()	378
documentCapture.getRemainingConcurrency.promise()	379
documentCapture.getRemainingFreeUsage()	380
documentCapture.getRemainingFreeUsage.promise()	381
documentCapture.parseResult(options)	382
documentCapture.DocumentType	383
documentCapture.Feature	384
documentCapture.FieldType	385
documentCapture.Language	386
N/email Module	388
N/email Module Script Sample	389
email.send(options)	390
email.send.promise(options)	394
email.sendBulk(options)	395
email.sendBulk.promise(options)	399
email.sendCampaignEvent(options)	400
email.sendCampaignEvent.promise(options)	403
N/encode Module	404
N/encode Module Script Samples	405
encode.convert(options)	406
encode.Encoding	407

N/error Module	408
N/error Module Script Samples	410
error.SuiteScriptError	411
error.create(options)	415
error.Type	417
N/file Module	423
N/file Module Script Samples	426
file.File	431
file.Reader	446
file.copy(options)	449
file.create(options)	450
file.delete(options)	452
file.load(options)	453
file.Encoding	455
file.NameConflictResolution	456
file.Type	457
N/format Module	459
N/format Module Script Samples	460
format.format(options)	462
format.parse(options)	464
format.Timezone	465
format.Type	469
N/format/i18n Module	471
N/format/i18n Module Script Samples	474
format.CurrencyFormatter	479
format.NumberFormatter	482
format.PhoneNumberFormatter	486
format.getCurrencyFormatter(options)	493
format.getNumberFormatter(options)	494
format.getPhoneNumberFormatter(options)	495
format.getPhoneNumberParser(options)	496
format.spellOut(options)	498
format.NegativeNumberFormat	499
format.Currency	500
N/http Module	501
N/http Module Script Samples	504
http.ClientResponse	506
http.ServerRequest	509
http.ServerResponse	517
http.get(options)	530
http.get.promise(options)	531
http.delete(options)	532
http.delete.promise(options)	533
http.post(options)	534
http.post.promise(options)	536
http.put(options)	537
http.put.promise(options)	539
http.request(options)	540
http.request.promise(options)	541
http.CacheDuration	542
http.Method	543
http.RedirectType	544
N/https Module	545
N/https Module Script Samples	550
https.SecureString	560

https.ClientResponse	568
https.ServerRequest	571
https.ServerResponse	583
https.createSecretKey(options)	595
https.createSecureString(options)	596
https.get(options)	597
https.get.promise(options)	599
https.delete(options)	600
https.delete.promise(options)	602
https.post(options)	602
https.post.promise(options)	606
https.put(options)	607
https.put.promise(options)	609
https.request(options)	610
https.request.promise(options)	612
https.requestRestlet(options)	613
https.requestRestlet.promise(options)	615
https.requestSuitelet(options)	617
https.requestSuitelet.promise(options)	619
https.requestSuiteTalkRest(options)	620
https.CacheDuration	622
https.Encoding	623
https.HashAlg	624
https.Method	625
https.RedirectType	626
N/https/clientCertificate Module	627
N/https/clientCertificate Module Script Sample	627
clientCertificate.post(options)	628
clientCertificate.get(options)	629
clientCertificate.put(options)	630
clientCertificate.delete(options)	631
clientCertificate.request(options)	632
N/keyControl Module	633
N/keyControl Module Script Samples	635
keyControl.Key	636
keyControl.createKey(options)	640
keyControl.findKeys(options)	641
keyControl.deleteKey(options)	642
keyControl.loadKey(options)	643
keyControl.lock(options)	644
keyControl.unlock(options)	645
keyControl.Operator	646
N/llm Module	647
N/llm Module Script Samples	653
Managing Prompts and Text Enhance Actions Using the N/llm Module	663
Model Parameter Values by LLM	668
Usage Limits and Concurrency Limits for N/llm Methods	668
llm.ChatMessage	670
llm.Citation	672
llm.Document	678
llm.EmbedResponse	682
llm.Response	685
llm.StreamedResponse	691
llm.Usage	697
llm.createChatMessage(options)	700

llm.createDocument(options)	701
llm.embed(options)	702
llm.embed.promise(options)	705
llm.evaluatePrompt(options)	706
llm.evaluatePrompt.promise(options)	709
llm.evaluatePromptStreamed(options)	710
llm.evaluatePromptStreamed.promise(options)	713
llm.generateText(options)	715
llm.generateText.promise(options)	720
llm.generateTextStreamed(options)	721
llm.generateTextStreamed.promise(options)	725
llm.getRemainingFreeUsage()	727
llm.getRemainingFreeUsage.promise()	727
llm.getRemainingFreeEmbedUsage()	728
llm.getRemainingFreeEmbedUsage.promise()	729
llm.ChatRole	730
llm.EmbedModelFamily	731
llm.ModelFamily	732
llm.SafetyMode	733
llm.Truncate	734
N/log Module	735
N/log Module Script Sample	738
log.audit(options)	739
log.debug(options)	740
log.emergency(options)	741
log.error(options)	742
N/machineTranslation Module	743
N/machineTranslation Module Script Samples	745
machineTranslation.Document	746
machineTranslation.Error	750
machineTranslation.Response	753
machineTranslation.createDocument(options)	756
machineTranslation.translate(options)	758
machineTranslation.translate.promise(options)	760
machineTranslation.Language	761
N/pgp Module	763
N/pgp Module Script Samples	766
pgp.Config	770
pgp.Key	772
pgp.Message	774
pgp.MessageData	778
pgp.Verification	783
pgp.VerificationSignature	785
pgp.createConfig(options)	787
pgp.createMessageData(options)	788
pgp.createSigner(options)	789
pgp.createVerification()	790
pgp.loadKeyFromSecret(options)	791
pgp.parseKey(options)	792
pgp.parseMessage(options)	793
pgp.CompressionAlgorithm	794
pgp.Format	795
N/piremoval Module	796
N/piremoval Module Script Sample	798
piremoval.PiRemovalTask	799

piremoval.PiRemovalTaskLogItem	808
piremoval.PiRemovalTaskStatus	813
piremoval.createTask(options)	815
piremoval.deleteTask(options)	817
piremoval.getTaskStatus(options)	818
piremoval.loadTask(options)	819
N/plugin Module	820
N/plugin Module Script Sample	821
plugin.findImplementations(options)	822
plugin.loadImplementation(options)	823
N/portlet Module	824
N/portlet Module Script Sample	825
portlet.resize()	826
portlet.refresh()	827
N/query Module	827
N/query Module Script Samples	841
Scripting with the N/query Module	847
Formulas in the N/query Module	852
Relative Dates in the N/query Module	853
SuiteQL in the N/query Module	855
query.Column	861
query.Component	869
query.Condition	889
query.Page	896
query.PagedData	901
query.PageRange	907
query.Period	910
query.Query	912
query.RelativeDate	946
query.Result	951
query.ResultSet	954
query.Sort	959
query.SuiteQL	965
query.create(options)	971
query.createPeriod(options)	974
query.createRelativeDate(options)	975
query.delete(options)	977
query.listTables(options)	978
query.load(options)	979
query.load.promise(options)	980
query.runSuiteQL(options)	982
query.runSuiteQL.promise(options)	984
query.runSuiteQLPaged(options)	986
query.runSuiteQLPaged.promise(options)	989
query.Aggregate	991
query.DateId	992
query.FieldContext	994
query.Operator	995
query.PeriodAdjustment	998
query.PeriodCode	999
query.PeriodType	1000
query.RelativeDateRange	1001
query.ReturnType	1006
query.SortLocale	1008
query.Type	1011

N/record Module	1023
N/record Module Script Samples	1040
record.Column	1046
record.Field	1052
record.Macro	1059
record.Record	1065
record.Sublist	1134
record.attach(options)	1139
record.attach.promise(options)	1141
record.copy(options)	1143
record.copy.promise(options)	1145
record.create(options)	1147
record.create.promise(options)	1149
record.delete(options)	1151
record.delete.promise(options)	1152
record.detach(options)	1154
record.detach.promise(options)	1155
record.load(options)	1157
record.load.promise(options)	1159
record.submitFields(options)	1161
record.submitFields.promise(options)	1163
record.transform(options)	1165
record.transform.promise(options)	1170
record.Type	1173
N/recordContext Module	1176
N/recordContext Module Sample	1176
recordContext.RecordContext	1177
recordContext.getContext(options)	1178
recordContext.ContextType	1179
N/redirect Module	1180
N/redirect Module Script Sample	1181
redirect.redirect(options)	1182
redirect.toRecord(options)	1183
redirect.toRecordTransform(options)	1184
redirect.toSavedSearch(options)	1186
redirect.toSavedSearchResult(options)	1187
redirect.toSearch(options)	1188
redirect.toSearchResult(options)	1188
redirect.toSuitelet(options)	1189
redirect.toTaskLink(options)	1190
N/render Module	1191
N/render Module Script Samples	1193
render.EmailMergeResult	1196
render.TemplateRenderer	1198
render.bom(options)	1208
render.create()	1209
render.mergeEmail(options)	1210
render.packingSlip(options)	1211
render.pickingTicket(options)	1212
render.statement(options)	1213
render.transaction(options)	1215
render.xmlToPdf(options)	1216
render.DataSource	1217
render.PrintMode	1218
N/runtime Module	1218

N/runtime Module Script Samples	1222
runtime.Script	1223
runtime.Session	1229
runtime.User	1231
runtime.getCurrentScript()	1239
runtime.getCurrentSession()	1239
runtime.getCurrentUser()	1240
runtime.isFeatureInEffect(options)	1241
runtime.accountId	1242
runtime.country	1242
runtime.envType	1243
runtime.executionContext	1243
runtime.processorCount	1244
runtime.queueCount	1245
runtime.version	1245
runtime.ContextType	1246
runtime.EnvType	1248
runtime.Permission	1249
N/scriptTypes/restlet Module	1249
N/scriptTypes/restlet Module Script Sample	1250
restlet.Response	1251
restlet.createResponse(options)	1254
N/search Module	1255
N/search Module Script Samples	1262
search.Column	1268
search.Filter	1275
search.Page	1279
search.PagedData	1285
search.PageRange	1290
search.Result	1292
search.ResultSet	1299
search.Search	1305
search.Setting	1318
search.create(options)	1321
search.create.promise(options)	1324
search.createColumn(options)	1325
search.createFilter(options)	1327
search.createSetting(options)	1329
search.delete(options)	1331
search.delete.promise(options)	1332
search.duplicates(options)	1333
search.duplicates.promise(options)	1335
search.global(options)	1335
search.global.promise(options)	1337
search.load(options)	1337
search.load.promise(options)	1339
search.lookupFields(options)	1340
search.lookupFields.promise(options)	1343
search.Operator	1344
search.Sort	1347
search.Summary	1347
search.Type	1348
N/sftp Module	1352
N/sftp Module Script Samples	1354
Setting up an SFTP Transfer	1358

SFTP Authentication	1359
Supported Cipher Suites and Host Key Types	1361
Supported SuiteScript File Types	1363
sftp.Connection	1364
sftp.createConnection(options)	1375
sftp.MAX_CONNECT_TIMEOUT	1378
sftp.MIN_CONNECT_TIMEOUT	1378
sftp.MAX_PORT_NUMBER	1379
sftp.MIN_PORT_NUMBER	1380
sftp.DEFAULT_PORT_NUMBER	1380
sftp.Sort	1381
N/sso Module	1382
N/suiteAppInfo Module	1382
N/suiteAppInfo Module Script Sample	1383
suiteAppInfo.isBundleInstalled(options)	1384
suiteAppInfo.isSuiteAppInstalled(options)	1385
suiteAppInfo.listBundlesContainingScripts(options)	1385
suiteAppInfo.listInstalledBundles()	1386
suiteAppInfo.listInstalledSuiteApps()	1387
suiteAppInfo.listSuiteAppsContainingScripts(options)	1388
N/task Module	1389
N/task Module Script Samples	1403
task.CsvImportTask	1408
task.CsvImportTaskStatus	1413
task.DocumentCaptureTask	1415
task.DocumentCaptureTaskStatus	1425
task.EntityDeduplicationTask	1427
task.EntityDeduplicationTaskStatus	1433
task.MapReduceScriptTask	1434
task.MapReduceScriptTaskStatus	1439
task.QueryTask	1450
task.QueryTaskStatus	1458
task.RecordActionTask	1462
task.RecordActionTaskStatus	1468
task.ScheduledScriptTask	1474
task.ScheduledScriptTaskStatus	1478
task.SearchTask	1481
task.SearchTaskStatus	1488
task.SuiteQLTask	1491
task.SuiteQLTaskStatus	1500
task.WorkflowTriggerTask	1505
task.WorkflowTriggerTaskStatus	1509
task.checkStatus(options)	1511
task.create(options)	1512
task.ActionCondition	1517
task.DedupeEntityType	1518
task.DedupeMode	1519
task.MapReduceStage	1520
task.MasterSelectionMode	1521
task.TaskStatus	1522
task.TaskType	1523
N/task/accounting/recognition Module	1525
N/task/accounting/recognition Module Script Samples	1529
recognition.MergeArrangementsTask	1531
recognition.MergeArrangementsTaskStatus	1537

recognition.MergeElementsTask	1542
recognition.checkStatus(options)	1546
recognition.create(options)	1547
recognition.TaskStatus	1548
recognition.TaskType	1549
N/transaction Module	1550
N/transaction Module Script Sample	1551
transaction.void(options)	1552
transaction.void.promise(options)	1553
transaction.Type	1554
N/translation Module	1557
N/translation Module Script Samples	1558
translation.Handle	1562
translation.Translator	1563
translation.get(options)	1565
translation.load(options)	1566
translation.selectLocale(options)	1568
translation.Locale	1569
N/ui Modules	1572
N/ui/dialog Module	1572
N/ui/message Module	1581
N/ui/serverWidget Module	1588
N/url Module	1730
N/url Module Script Samples	1731
url.format(options)	1734
url.resolveDomain(options)	1735
url.resolveRecord(options)	1736
url.resolveScript(options)	1737
url.resolveTaskLink(options)	1738
url.HostType	1739
N/util Module	1740
N/util Module Script Sample	1742
util.each(iterable, callback)	1743
util.extend(receiver, contributor)	1743
util.isArray(obj)	1745
util.isAsyncFunction(obj)	1745
util.isBoolean(obj)	1746
util.isDate(obj)	1747
util.isFunction(obj)	1748
util.isNumber(obj)	1749
util.isObject(obj)	1749
util.isRegExp(obj)	1750
util.isString(obj)	1751
N/workbook Module	1752
N/workbook Module Script Samples	1774
workbook.Aspect	1779
workbook.CalculatedMeasure	1782
workbook.Category	1785
workbook.Chart	1788
workbook.ChartAxis	1799
workbook.ChildNodesSelector	1801
workbook.Color	1801
workbook.ConditionalFilter	1805
workbook.ConditionalFormat	1811
workbook.ConditionalFormatRule	1812

workbook.Currency	1813
workbook.DataDimension	1814
workbook.DataDimensionItem	1817
workbook.DataDimensionItemValue	1819
workbook.DataDimensionValue	1821
workbook.DataMeasure	1822
workbook.DescendantorSelfNodesSelector	1824
workbook.DimensionSelector	1825
workbook.Duration	1826
workbook.Expression	1828
workbook.FieldContext	1830
workbook.FontSize	1832
workbook.Legend	1834
workbook.LimitingFilter	1838
workbook.MeasureSelector	1841
workbook.MeasureValue	1842
workbook.MeasureValueSelector	1843
workbook.PathSelector	1845
workbook.Pivot	1847
workbook.PivotAxis	1855
workbook.PivotIntersection	1857
workbook.PositionPercent	1859
workbook.PositionUnits	1860
workbook.PositionValues	1862
workbook.Range	1864
workbook.Record	1865
workbook.RecordKey	1867
workbook.ReportStyle	1868
workbook.ReportStyleRule	1869
workbook.Section	1871
workbook.SectionValue	1873
workbook.Series	1874
workbook.Sort	1875
workbook.SortByDataDimensionItem	1880
workbook.SortByMeasure	1881
workbook.SortDefinition	1883
workbook.Style	1885
workbook.Table	1891
workbook.TableColumn	1895
workbook.TableColumnCondition	1902
workbook.TableColumnFilter	1903
workbook.Workbook	1904
workbook.create(options)	1910
workbook.createAspect(options)	1911
workbook.createCalculatedMeasure(options)	1912
workbook.createCategory(options)	1913
workbook.createChart(options)	1914
workbook.createChartAxis(options)	1916
workbook.createColor(options)	1917
workbook.createComplexRecordKey	1918
workbook.createConditionalFilter(options)	1919
workbook.createConditionalFormat(options)	1920
workbook.createConditionalFormatRule(options)	1921
workbook.createConstant(options)	1922
workbook.createCurrency(options)	1923

workbook.createDataDimension(options)	1924
workbook.createDataDimensionItem(options)	1926
workbook.createDataMeasure(options)	1926
workbook.createDimensionSelector(options)	1928
workbook.createDuration(options)	1929
workbook.createExpression(options)	1929
workbook.createFieldContext(options)	1930
workbook.createFontSize(options)	1931
workbook.createLegend(options)	1932
workbook.createLimitingFilter(options)	1933
workbook.createMeasureSelector(options)	1935
workbook.createMeasureValueSelector(options)	1935
workbook.createPathSelector(options)	1936
workbook.createPivot(options)	1937
workbook.createPivotAxis(options)	1939
workbook.createPositionPercent(options)	1940
workbook.createPositionUnits(options)	1941
workbook.createPositionValues(options)	1942
workbook.createRange(options)	1943
workbook.createReportStyle(options)	1944
workbook.createReportStyleRule(options)	1945
workbook.createSection(options)	1946
workbook.createSeries(options)	1948
workbook.createSimpleRecordKey	1948
workbook.createSort(options)	1949
workbook.createSortByDataDimensionItem(options)	1950
workbook.createSortByMeasure(options)	1951
workbook.createSortDefinition(options)	1952
workbook.createStyle(options)	1954
workbook.createTable(options)	1956
workbook.createTableColumn(options)	1957
workbook.createTableColumnCondition(options)	1958
workbook.createTableColumnFilter(options)	1959
workbook.createTranslation(options)	1960
workbook.list()	1961
workbook.listPaged(options)	1961
workbook.loadWorkbook(options)	1962
workbook.Aggregation	1963
workbook.AspectType	1964
workbook.ChartType	1965
workbook.Color	1966
workbook.ConstantType	1967
workbook.DateTimeHierarchy	1969
workbook.DateTimeProperty	1969
workbook.ExpressionType	1970
workbook.FontSize	1974
workbook.FontStyle	1975
workbook.FontWeight	1976
workbook.Image	1977
workbook.Position	1978
workbook.Stacking	1979
workbook.TemporalUnit	1980
workbook.TextAlign	1981
workbook.TextDecorationLine	1982
workbook.TextDecorationStyle	1983

workbook.TotalLine	1984
workbook.Unit	1985
N/workflow Module	1986
N/workflow Module Script Sample	1987
workflow.initiate(options)	1988
workflow.trigger(options)	1989
N/xml Module	1990
N/xml Module Script Samples	1997
xml.Attr	2000
xml.Document	2003
xml.Element	2020
xml.Node	2035
xml.Parser	2055
xml.XPath	2057
xml.escape(options)	2059
xml.validate(options)	2059
xml.NodeType	2061

SuiteScript 1.0 to SuiteScript 2.x API Map

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This topic maps SuiteScript 1.0 APIs to their corresponding SuiteScript 2.x APIs. Keep the following in mind when using these mappings:

- Some SuiteScript 1.0 APIs do not have a SuiteScript 2.x equivalent.
- There is not always a 1:1 mapping between SuiteScript 1.0 and SuiteScript 2.x. Each SuiteScript 1.0 API is listed only one time, but it may map to several SuiteScript 2.x APIs.
- These mappings **do not include** SuiteScript 1.0 deprecated APIs.
- These mappings **do not include** new SuiteScript 2.x functionality. To find new SuiteScript 2.x functionality, go to [SuiteScript 2.x Modules](#) which includes a description and link to each module.



Important: If you are using SuiteScript 1.0 for your scripts, consider converting these scripts to SuiteScript 2.x. Use SuiteScript 2.x to take advantage of new features, APIs, and functionality enhancements. For more information, see the help topics [SuiteScript 2.x Advantages](#) and [Transitioning from SuiteScript 1.0 to SuiteScript 2.x](#).

If converting your SuiteScript 1.0 scripts to SuiteScript 2.x is not feasible in your situation, you can use SuiteScript 2.x RESTlet scripts to extend the capabilities of your existing SuiteScript 1.0 scripts. For more information, see the help topic [Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts](#).

The following topics group SuiteScript 1.0 APIs into functions (prefixed with “nlapi”) and objects (prefixed with “nlobj”). All functions are listed alphabetically in one table. Whereas objects and their members are grouped alphabetically by object name. Each object has its own table containing all object members.

- [SuiteScript 1.0 to SuiteScript 2.x API Map – Functions \(nlapi\)](#)
- [SuiteScript 1.0 to SuiteScript 2.x API Map – Objects \(nlobj\)](#)

SuiteScript 1.0 to SuiteScript 2.x API Map – Functions (nlapi)

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This topic maps SuiteScript 1.0 Functions (prefixed with “nlapi”) to their corresponding SuiteScript 2.x APIs. All functions are listed alphabetically in one table.



Note: NetSuite does not support calling SuiteScript 1.0 APIs from SuiteScript 2.x scripts.



Note: To view a mapping of SuiteScript 1.0 Objects (prefixed with “nlobj”) to their corresponding SuiteScript 2.x APIs, see [SuiteScript 1.0 to SuiteScript 2.x API Map – Objects \(nlobj\)](#).

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlapiAddDays(d, days)	See Notes	See Notes	This API does not have a SuiteScript 2.x equivalent. Use the following JavaScript to add or subtract days from a Date object: dateObj.setDate(dateObj.getDate() + or – days) For example:

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
			<pre>1 var tomorrow = new Date(); 2 tomorrow.setDate(tomorrow.getDate() + 1);</pre>  Note: SuiteScript 2.x is also compatible with third-party JavaScript APIs that provide this functionality. For information about using third-party APIs with SuiteScript 2.x, see the help topic SuiteScript 2.x Custom Modules.
nlapiAddMonths(d, months)	See Notes	See Notes	This API does not have a SuiteScript 2.x equivalent. Use the following JavaScript to add or subtract months from a Date object: dateObj.setMonth(dateObj.getMonth() + or – months) For example: <pre>1 var today = new Date(); 2 var oneMonthAgo = today.setMonth(today.getMonth() - 1);</pre>  Note: SuiteScript 2.x is also compatible with third-party JavaScript APIs that provide this functionality. For information about using third-party APIs with SuiteScript 2.x, see the help topic SuiteScript 2.x Custom Modules.
nlapiAttachRecord(type, id, type2, id2, attributes)	record.attach(options)	N/record Module	<pre>1 var recordId = record.attach({ 2 record: { 3 type: record.Type.FILE, 4 id: '447' 5 }, 6 to: { 7 type: record.type.CUSTOMER, 8 id: 530 9 } 10 });</pre>
nlapiCancelLineItem(type)	Record.cancelLine(options) CurrentRecord.cancelLine(options)	N/record Module N/currentRecord Module	—
nlapiCheckPasswordField(type, id, value, field, sublist, line)	crypto.checkPasswordField(options)	N/crypto Module	—
nlapiCommitLineItem(type)	Record.commitLine(options) CurrentRecord.commitLine(options)	N/record Module N/currentRecord Module	For N/record script samples, see: ■ N/record Module Script Samples ■ Creating an Inventory Detail Sublist Subrecord Example
nlapiCopyRecord(type, id, initializeValues)	record.copy(options)	N/record Module	<pre>1 var recObj = record.copy({ 2 type: record.Type.SALES_ORDER, 3 id: 284, 4 isDynamic: true, 5 defaultValues: { 6 entity: 547 7 } 8 }); var recordId = recObj.save();</pre>
nlapiCreateAssistant(title, hideHeader)	serverWidget.createAssistant(options)	N/ui/serverWidget Module	—
nlapiCreateCSVImport()	task.create(options)	N/task Module	For script samples, see N/task Module .

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlapiCreateCurrentLineItemSubrecord(sublist, fldname)	Record.getCurrentSublistSubrecord(options) CurrentRecord.getCurrentSublistSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiCreateEmailMerger(templateId)	render.mergeEmail(options)	N/render Module	—
nlapiCreateError(code, details, suppressNotification)	error.create(options)	N/error Module	For a script sample, see N/error Module Script Samples .
nlapiCreateFile(name, type, contents)	file.create(options)	N/file Module	For a script sample, see N/file Module Script Samples .
nlapiCreateForm(title, hideNavbar)	serverWidget.createForm(options)	N/ui/serverWidget Module	For a script sample, see the help topic N/ui/serverWidget Module Script Samples
nlapiCreateList(title, hideNavbar)	serverWidget.createList(options)	N/ui/serverWidget Module	—
nlapiCreateRecord(type, initializeValues)	record.create(options)	N/record Module	—
nlapiCreateSearch(type, filters, columns)	search.create(options)	N/search Module	For a script sample, see the help topic N/search Module Script Samples .
nlapiCreateSubrecord(fldname)	Record.getSubrecord(options) CurrentRecord.getSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiCreateTemplateRenderer()	render.create()	N/render Module	For a script sample, see N/render Module Script Samples .
nlapiDateToString(d, format)	format.format(options)	N/format Module	For a script sample, see N/format Module Script Samples
nlapiDeleteFile(id)	file.delete(options)	N/file Module	—
nlapiDeleteRecord(type, id)	record.delete(options)	N/record Module	—
nlapiDetachRecord(type, id, type2, id2, attributes)	record.detach(options)	N/record Module	—
nlapiDisableField(fldnam, val)	Field.isDisabled	N/currentRecord Module	Note that isDisabled is a property.
nlapiDisableLineItemField(type, fldnam, val)	Field.isDisabled	N/currentRecord Module	Note that isDisabled is a property.
nlapiEditCurrentLineItemSubrecord(sublist, fldname)	Record.getCurrentSublistSubrecord(options) CurrentRecord.getCurrentSublistSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiEditSubrecord(fldname)	Record.getSubrecord(options) CurrentRecord.getSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiEncrypt(s, algorithm, key)	See Notes	See Notes	For SuiteScript 2.x encryption, hashing, and HMAC functionality, see the N/crypto Module module. For SuiteScript 2.x encoding functionality, see the N/encode Module module.
nlapiEscapeXML(text)	xml.escape(options)	N/xml Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlapiExchangeRate (sourceCurrency, targetCurrency, effectiveDate)	currency.exchangeRate(options)	N/currency Module	For a script sample, see N/currency Module Script Sample .
nlapiFindLineItemMatrixValue(type, fldnam, val, column)	Record.findMatrixSublistLineWithValue(options) CurrentRecord.findMatrixSublistLineWithValue(options)	N/record Module N/currentRecord Module	—
nlapiFindLineItemValue(type, fldnam, val)	Record.findSublistLineWithValue(options) CurrentRecord.findSublistLineWithValue(options)	N/record Module N/currentRecord Module	—
nlapiFormatCurrency(str)	format.format(options)	N/format Module	Note that SuiteScript 2.x currency formatting is handled by the N/ format module and not the N/ currency module. For a script sample, see N/format Module Script Samples
nlapiGetContext()	runtime.getCurrentScript() runtime.getCurrentSession() runtime.getCurrentUser()	N/runtime Module	For a script sample, see N/runtime Module Script Samples .
nlapiGetCurrentLineItem DateTimeValue(type, fieldId, timeZone)	See Notes	N/format Module	Use the N/ format module to mimic this functionality in SuiteScript 2.x.
nlapiGetCurrentLineItemIndex (type)	Record.getCurrentSublistIndex(options) CurrentRecord.getCurrentSublistIndex(options)	N/record Module N/currentRecord Module	—
nlapiGetCurrentLineItemMatrix Value(type, fldnam, column)	CurrentRecord.getCurrentMatrixSublistValue(options) Record.getCurrentMatrixSublistValue(options)	N/record Module N/currentRecord Module	—
nlapiGetCurrentLineItemText(type, fldnam)	Record.getCurrentSublistText(options) CurrentRecord.getCurrentSublistText(options)	N/record Module N/currentRecord Module	—
nlapiGetCurrentLineItemValue(type, fldnam)	Record.getCurrentSublistValue(options) CurrentRecord.getCurrentSublistValue(options)	N/record Module N/currentRecord Module	—
nlapiGetCurrentLineItemValues (type, fldnam)	Record.getCurrentSublistValue(options) CurrentRecord.getCurrentSublistValue(options)	N/record Module N/currentRecord Module	—
nlapiGetDateTimeValue(fieldId, timeZone)	See Notes	N/format Module	Use the N/ format module to mimic this functionality in SuiteScript 2.x.
nlapiGetDepartment()	User.department	N/runtime Module	—
nlapiGetField(fldnam)	Record.getField(options) CurrentRecord.getField(options)	N/record Module N/currentRecord Module	—
nlapiGetFieldText(fldnam)	Record.getText(options) CurrentRecord.getText(options)	N/record Module N/currentRecord Module	—
nlapiGetFieldTexts(fldnam)	Record.getText(options) CurrentRecord.getText(options)	N/record Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
		N/currentRecord Module	
nlapiGetFieldValue(fldnam)	Record.getValue(options) CurrentRecord.getValue(options)	N/record Module N/currentRecord Module	—
nlapiGetFieldValues(fldnam)	Record.getValue(options) CurrentRecord.getValue(options)	N/record Module N/currentRecord Module	—
nlapiGetJobManager(jobType)	task.create(options)	N/task Module	For a script sample, see N/task Module .
nlapiGetLineItemCount(type)	Record.getLineCount(options) CurrentRecord.getLineCount(options)	N/record Module N/currentRecord Module	—
nlapiGetLineItemDateTimeValue(type, fieldId, value, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlapiGetLineItemField(type, fldnam, linenum)	Record.getSublistField(options) CurrentRecord.getSublistField(options)	N/record Module N/currentRecord Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiGetLineItemMatrixField(type, fldnam, linenum, column)	Record.getMatrixSublistField(options) CurrentRecord.getMatrixSublistField(options)	N/record Module N/currentRecord Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiGetLineItemMatrixValue(type, fldnam, linenum, column)	Record.getMatrixSublistValue(options) CurrentRecord.getMatrixSublistValue(options)	N/record Module N/currentRecord Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiGetLineItemText(type, fldnam, linenum)	Record.getSublistText(options) CurrentRecord.getSublistText(options)	N/record Module N/currentRecord Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiGetLineItemValue(type, fldnam, linenum)	Record.getSublistValue(options) CurrentRecord.getSublistValue(options)	N/record Module N/currentRecord Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiGetLineItemValues(type, fldname, linenum)	Record.getSublistValue(options) CurrentRecord.getSublistValue(options)	N/record Module N/currentRecord Module	Method returns an array for multi-select fields. SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiGetLocation()	User.location	N/runtime Module	Note that <code>location</code> is a property.
nlapiGetLogin()	auth.changeEmail(options) auth.changePassword(options)	N/auth Module	For a script sample, see N/auth Module Script Sample .
nlapiGetMatrixCount(type, fldnam)	Record.getMatrixHeaderCount(options) CurrentRecord.getMatrixHeaderCount(options)	N/record Module N/currentRecord Module	—
nlapiGetMatrixField(type, fldnam, column)	Record.getMatrixHeaderField(options) CurrentRecord.getMatrixHeaderField(options)	N/record Module N/currentRecord Module	—
nlapiGetMatrixValue(type, fldnam, column)	Record.getMatrixHeaderValue(options) CurrentRecord.getMatrixHeaderValue(options)	N/record Module N/currentRecord Module	—
nlapiGetNewRecord()	See Notes	See Notes	To mimic this functionality in SuiteScript 2.x, use the following code in a

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
			beforeLoad(context), beforeSubmit(context), or afterSubmit(context) user event script. <pre> 1 function afterSubmit(context) { 2 var newRec = context.newRecord; 3 } </pre> For additional information and a full script sample, see the help topic SuiteScript 2.x User Event Script Type
nlapiGetOldRecord()	See Notes	See Notes	To mimic this functionality in SuiteScript 2.x, use the following code in a beforeSubmit(context) or afterSubmit(context) user event script. <pre> 1 function afterSubmit(context) { 2 var oldRec = context.oldRecord; 3 } </pre> For additional information and a full script sample, see the help topic SuiteScript 2.x User Event Script Type
nlapiGetRecordId()	Record.id CurrentRecord.id	N/record Module N/currentRecord Module	—
nlapiGetRecordType()	Record.type CurrentRecord.type	N/record Module N/currentRecord Module	To get the current record type in a client script, use CurrentRecord.type: <pre> 1 function saveRec(context) { 2 var rec = context.currentRecord; 3 var recType = rec.type; 4 } </pre> To get the current record type in a server script, use Record.type in a beforeLoad(context), beforeSubmit(context), or afterSubmit(context) user event script: <pre> 1 function beforeSubmit(context) { 2 var newRec = context.newRecord; 3 var recType = newRec.type; 4 } </pre>
nlapiGetRole()	User.role	N/runtime Module	—
nlapiGetSubsidiary()	User.subsidiary	N/runtime Module	—
nlapiGetUser()	runtime.getCurrentUser()	N/runtime Module	—
nlapiInitiateWorkflow (recordtype, id, workflowid, initialvalues)	workflow.initiate(options)	N/workflow Module	For a script sample, see the help topic N/ workflow Module Script Sample .
nlapiInitiateWorkflowAsync (recordtype, id, workflowid, initialValues)	task.WorkflowTriggerTask	N/task Module	—
nlapiInsertLineItem(type, line)	Record.insertLine(options) CurrentRecord.insertLine(options)	N/record Module N/currentRecord Module	—
nlapiInsertLineItemOption(type, fldnam, value, text, selected)	Field.insertSelectOption(options)	N/currentRecord Module	—
nlapiInsertSelectOption(fldnam, value, text, selected)	Field.insertSelectOption(options)	N/currentRecord Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlapiIsLineItemChanged(type)	Sublist.isChanged	N/record Module	Note that isChanged is a property <pre>record.getSublist("addressbook").isChanged</pre>
nlapiLoadConfiguration(type)	config.load(options)	N/config Module	For a script sample, see N/config Module Script Sample .
nlapiLoadFile(id)	file.load(options)	N/file Module	For a script sample, see N/file Module Script Samples .
nlapiLoadRecord(type, id, initializeValues)	record.load(options)	N/record Module	—
nlapiLoadSearch(type, id)	search.load(options)	N/search Module	For a script sample, see the help topic N/search Module Script Samples .
nlapiLogExecution(type, title, details)	log.audit(options) log.debug(options) log.emergency(options) log.error(options)	N/log Module	For a script sample, see N/log Module Script Sample .
nlapiLookupField(type, id, fields, text)	search.lookupFields(options)	N/search Module	—
nlapiOutboundSSO(id)	 Note: As of NetSuite 2025.1, the support ended for the SuiteSignOn feature, which means the N/sso module is also no longer supported. sso.generateSuiteSignOnToken(options)	N/sso	—
nlapiPrintRecord(type, id, mode, properties)	render.bom(options) render.packingSlip(options) render.pickingTicket(options) render.statement(options) render.transaction(options)	N/render Module	For a script sample, see N/render Module Script Samples .
nlapiRefreshLineItems(type)	—	—	This API does not have a SuiteScript 2.x equivalent.
nlapiRefreshPortlet()	portlet.refresh()	N/portlet Module	For a script sample, see N/portlet Module Script Sample
nlapiRemoveCurrentLineItemSubrecord(sublist, fldname)	Record.removeCurrentSublistSubrecord(options) CurrentRecord.removeCurrentSublistSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiRemoveLineItem(type, line)	Record.removeLine(options) CurrentRecord.removeLine(options)	N/record Module N/currentRecord Module	—
nlapiRemoveLineItemOption(type, fldnam, value)	Field.removeSelectOption(options)	N/currentRecord Module	—
nlapiRemoveSelectOption(fldnam, value)	Field.removeSelectOption(options)	N/currentRecord Module	—
nlapiRemoveSubrecord(fldname)	Record.removeSubrecord(options) CurrentRecord.removeSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlapiRequestURL(url, postdata, headers, callback, httpMethod)	http.delete(options) http.get(options) http.post(options) http.put(options) http.request(options)	N/http Module	—
nlapiRequestURLWithCredentials(credentials, url, postdata, headers, httpMethod)	https.request(options)	N/https Module	Server scripts only
nlapiResizePortlet()	portlet.resize()	N/portlet Module	For a script sample, see N/portlet Module Script Sample
nlapiResolveURL(type, identifier, id, displayMode)	url.resolveRecord(options) url.resolveScript(options) url.resolveTaskLink(options)	N/url Module	For a script sample, see N/url Module Script Samples .
nlapiScheduleScript(scriptId, deployId, params)	task.create(options)	N/task Module	<pre> 1 var scheduleScriptTaskObj = task.create({ 2 taskType: task.TaskType.SCHEDULED_SCRIPT, 3 //Other Params 4 }); </pre>
nlapiSearchDuplicate(type, fields, id)	search.duplicates(options)	N/search Module	—
nlapiSearchGlobal(keywords)	search.global(options)	N/search Module	—
nlapiSearchRecord(type, id, filters, columns)	search.create(options) search.load(options)	N/search Module	For a script sample, see the help topic N/search Module Script Samples .
nlapiSelectLineItem(type, linenum)	Record.selectLine(options) CurrentRecord.selectLine(options)	N/record Module N/currentRecord Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiSelectNewLineItem(type)	Record.selectNewLine(options) CurrentRecord.selectNewLine(options)	N/record Module N/currentRecord Module	—
nlapiSelectNode(node, xpath)	XPath.select(options)	N/xml Module	—
nlapiSelectNodes(node, xpath)	XPath.select(options)	N/xml Module	—
nlapiSelectValue(node, xpath)	See Notes	N/xml Module	To mimic this functionality in SuiteScript 2.x, select a node with XPath.select(options) and then inspect the Node.textContent property.
nlapiSelectValues(node, path)	See Notes	N/xml Module	To mimic this functionality in SuiteScript 2.x, select an array of nodes with XPath.select(options) and then loop through each node's Node.textContent property.
nlapiSendCampaignEmail(campaigneventid, recipientid)	email.sendCampaignEvent(options)	N/email Module	—
nlapiSendEmail(author, recipient, subject, body, cc, bcc, records, attachments)	email.send(options) email.sendBulk(options)	N/email Module	For a script sample, see N/email Module Script Sample .
nlapiSendFax(author, recipient, subject, body, records, attachments)	—	—	This API does not have a SuiteScript 2.x equivalent.
nlapiSetCurrentLineItemDateTimeValue(type, fieldId, dateTime, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlapiSetCurrentLineItemMatrixValue(type, fldnam, ...)	Record.setCurrentMatrixSublistValue(options)	N/record Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
column, value, firefieldchanged, synchronous)	CurrentRecord.setCurrentMatrixSublistValue(options)	N/currentRecord Module	
nlapiSetCurrentLineItemText(type, fldnam, text, firefieldchanged, synchronous)	Record.setCurrentSublistText(options) CurrentRecord.setCurrentSublistText(options)	N/record Module N/currentRecord Module	—
nlapiSetCurrentLineItemValue(type, fldnam, value, firefieldchanged, synchronous)	Record.setCurrentSublistValue(options) CurrentRecord.setCurrentSublistValue(options)	N/record Module N/currentRecord Module	—
nlapiSetCurrentLineItemValues(type, fldnam, values, firefieldchanged, synchronous)	Record.setCurrentSublistValue(options) CurrentRecord.setCurrentSublistValue(options)	N/record Module N/currentRecord Module	—
nlapiSetDateTimeValue(fieldId, dateTime, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlapiSetFieldText(fldname, txt, firefieldchanged, synchronous)	Record.setText(options) CurrentRecord.setText(options)	N/record Module N/currentRecord Module	—
nlapiSetFieldTexts (fldname, txts, firefieldchanged, synchronous)	Record.setText(options) CurrentRecord.setText(options)	N/record Module N/currentRecord Module	—
nlapiSetFieldValue(fldnam, value, firefieldchanged, synchronous)	Record.setValue(options) CurrentRecord.setValue(options)	N/record Module N/currentRecord Module	—
nlapiSetFieldValues (fldnam, value, firefieldchanged, synchronous)	Record.setValue(options) CurrentRecord.setValue(options)	N/record Module N/currentRecord Module	—
nlapiSetLineItemDateTimeValue (type, fieldId, value, dateTime, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlapiSetLineItemValue(type, fldnam, linenum, value)	Record.setSublistValue(options)	N/record Module	SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiSetMatrixValue(type, fldnam, column, value, firefieldchanged, synchronous)	Record.setMatrixHeaderValue(options) CurrentRecord.setMatrixHeaderValue(options)	N/record Module N/currentRecord Module	—
nlapiSetRecoveryPoint()	See Notes	See Notes	The SuiteScript 2.x Map/Reduce Script Type automatically incorporates yielding.
nlapiSetRedirectURL(type, identifier, id, editmode, parameters)	redirect.redirect(options) redirect.toRecord(options) redirect.toSuitelet(options) redirect.toTaskLink(options)	N/redirect Module	For a script sample, see N/redirect Module Script Sample .
nlapiStringToDate(str, format)	format.parse(options)	N/format Module	For a script sample, see N/format Module Script Samples .
nlapiStringToXML(text)	Parser.fromString(options)	N/xml Module	—
nlapiSubmitConfiguration(name)	Record.save(options)	N/record Module	—
nlapiSubmitCSVImport(nlobj CSVImport)	—	N/task Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlapiSubmitRecord(record, doSourcing, ignoreMandatory Fields)	Record.save(options)	N/record Module	—
nlapiSubmitField(type, id, fields, values, doSourcing)	record.submitFields(options)	N/record Module	—
nlapiSubmitFile(file)	File.save()	N/file Module	For a script sample, see N/file Module Script Samples .
nlapiTransformRecord(type, id, transformType, transformValues)	record.transform(options)	N/record Module	—
nlapiTriggerWorkflow(recordtype, id, workflowid, actionid, stateid)	workflow.trigger(options)	N/workflow Module	—
nlapiValidateXML(xmlDocument, schemaDocument, schemaFolderId)	xml.validate(options)	N/xml Module	—
nlapiViewCurrentLineItemSubrecord(sublist, fldname)	CurrentRecord.getCurrentSublistSubrecord(options) Record.getCurrentSublistSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiViewLineItemSubrecord(sublist, fldname, linenumber)	Record.getSublistSubrecord(options)	N/record Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords . SuiteScript 2.x begins sublist numbering with 0. SuiteScript 1.0 begins sublist numbering with 1.
nlapiViewSubrecord(fldname)	Record.getSubrecord(options) CurrentRecord.getSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlapiVoidTransaction(transactionType, recordId)	transaction.void(options)	N/transaction Module	For a script sample, see N/transaction Module Script Sample
nlapiXMLToPDF(xmlstring)	render.xmlToPdf(options) TemplateRenderer.renderAsPdf()	N/render Module	Note that <code>TemplateRenderer.renderAsPdf()</code> is equivalent to <code>nlapiXMLToPDF(nlobjEmailMerger.renderToString())</code> . For a script sample, see N/render Module Script Samples .
nlapiXMLToString(xml)	Parser.toString(options)	N/xml Module	—
nlapiYieldScript()	See Notes	See Notes	Note that the SuiteScript 2.x Map/Reduce Script Type automatically incorporates yielding.

SuiteScript 1.0 to SuiteScript 2.x API Map – Objects (nlobj)

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This topic maps SuiteScript 1.0 Objects (prefixed with “nlobj”) to their corresponding SuiteScript 2.x APIs. Objects and their members are grouped alphabetically by object name. Each object has its own table containing all object members.

Note: NetSuite does not support calling SuiteScript 1.0 APIs from SuiteScript 2.x scripts.

Note: To view a mapping of SuiteScript 1.0 Functions (prefixed with “nlapi”) to their corresponding SuiteScript 2.x APIs, see [SuiteScript 1.0 to SuiteScript 2.x API Map – Functions \(nlapi\)](#).

- [nlobjAssistant](#)
- [nlobjAssistantStep](#)
- [nlobjButton](#)
- [nlobjColumn](#)
- [nlobjConfiguration](#)
- [nlobjContext](#)
- [nlobjCredentialBuilder](#)
- [nlobjCSVImport](#)
- [nlobjDuplicateJobRequest](#)
- [nlobjEmailMerger](#)
- [nlobjError](#)
- [nlobjField](#)
- [nlobjFieldGroup](#)
- [nlobjFile](#)
- [nlobjForm](#)
- [nlobjFuture](#)
- [nlobjJobManager](#)
- [nlobjList](#)
- [nlobjLogin](#)
- [nlobjMergeResult](#)
- [nlobjPortlet](#)
- [nlobjRecord](#)
- [nlobjRequest](#)
- [nlobjResponse](#)
- [nlobjSearch](#)
- [nlobjSearchColumn](#)
- [nlobjSearchFilter](#)
- [nlobjSearchResult](#)
- [nlobjSearchResultSet](#)

- [nlobjSelectOption](#)
- [nlobjSublist](#)
- [nlobjSubrecord](#)
- [nlobjTab](#)
- [nlobjTemplateRenderer](#)

nlobjAssistant

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjAssistant	serverWidget.Assistant	N/ui/serverWidget Module	—
nlobjAssistant.addField(name, type, label, source, group)	Assistant.addField(options)	N/ui/serverWidget Module	—
nlobjAssistant.addFieldGroup(name, label)	Assistant.addFieldGroup(options)	N/ui/serverWidget Module	—
nlobjAssistant.addStep(name, label)	Assistant.addStep(options)	N/ui/serverWidget Module	—
nlobjAssistant.addSubList(name, type, label)	Assistant.addSublist(options)	N/ui/serverWidget Module	—
nlobjAssistant.getAllFields()	Assistant.getFieldIds()	N/ui/serverWidget Module	—
nlobjAssistant.getAllFieldGroups()	Assistant.getFieldGroupIds()	N/ui/serverWidget Module	—
nlobjAssistant.getAllSteps()	Assistant.getSteps()	N/ui/serverWidget Module	—
nlobjAssistant.getAllSubLists()	Assistant.getSublistIds()	N/ui/serverWidget Module	—
nlobjAssistant.getCurrentStep()	Assistant.currentStep	N/ui/serverWidget Module	Note that currentStep is a property.
nlobjAssistant.getField(name)	Assistant.getField(options)	N/ui/serverWidget Module	—
nlobjAssistant.getFieldGroup(name)	Assistant.getFieldGroup(options)	N/ui/serverWidget Module	—
nlobjAssistant.getLastAction()	Assistant.getLastAction()	N/ui/serverWidget Module	—
nlobjAssistant.getLastStep()	Assistant.getLastStep()	N/ui/serverWidget Module	—
nlobjAssistant.getNextStep()	Assistant.getNextStep()	N/ui/serverWidget Module	—
nlobjAssistant.getStep(name)	Assistant.getStep(options)	N/ui/serverWidget Module	—
nlobjAssistant.getSubList(name)	Assistant.getSublist(options)	N/ui/serverWidget Module	—
nlobjAssistant.hasError()	Assistant.hasErrorHtml()	N/ui/serverWidget Module	—
nlobjAssistant.isFinished()	Assistant.isFinished()	N/ui/serverWidget Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjAssistant.sendRedirect(response)	Assistant.sendRedirect(options)	N/ui/serverWidget Module	—
nlobjAssistant.setCurrentStep(step)	Assistant.currentStep	N/ui/serverWidget Module	Note that currentStep is a property.
nlobjAssistant.setError(html)	Assistant.errorHtml	N/ui/serverWidget Module	Note that errorHtml is a property.
nlobjAssistant.setFieldValues(values)	Assistant.updateDefaultValues(values)	N/ui/serverWidget Module	—
nlobjAssistant.setFinished(html)	Assistant.finishedHtml	N/ui/serverWidget Module	Note that finishedHtml is a property.
nlobjAssistant.setNumbered(hasStepNumber)	Assistant.hideStepNumber	N/ui/serverWidget Module	Note that hideStepNumber is a property.
nlobjAssistant.setOrdered(ordered)	Assistant.isNotOrdered	N/ui/serverWidget Module	Note that isNotOrdered is a property.
nlobjAssistant.setScript(script)	Assistant.clientScriptFileId Assistant.clientScriptModulePath	N/ui/serverWidget Module	Note that clientScriptFileId and clientScriptModulePath are properties. Use one of these SuiteScript 2.x properties to attach an ad hoc client script to an assistant.
nlobjAssistant.setShortcut(show)	Assistant.hideAddToShortcutsLink	N/ui/serverWidget Module	Note that hideAddToShortcutsLink is a property.
nlobjAssistant.setSplash(title, text1, text2)	Assistant.setSplash(options)	N/ui/serverWidget Module	—
nlobjAssistant.setTitle(title)	Assistant.title	N/ui/serverWidget Module	Note that title is a property.

nlobjAssistantStep

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjAssistantStep	serverWidget.AssistantStep	N/ui/serverWidget Module	—
nlobjAssistantStep.getAllFields()	AssistantStep.getFieldIds()	N/ui/serverWidget Module	—
nlobjAssistantStep.getAllLineItemFields(group)	AssistantStep.getSublistFieldIds(options)	N/ui/serverWidget Module	—
nlobjAssistantStep.getAllLineItems()	AssistantStep.getSubmittedSublistIds()	N/ui/serverWidget Module	—
nlobjAssistantStep.getFieldValue(name)	AssistantStep.getValue(options)	N/ui/serverWidget Module	—
nlobjAssistantStep.getFieldValues(name)	AssistantStep.getValue(options)	N/ui/serverWidget Module	—
nlobjAssistantStep.getLineItemCount(group)	AssistantStep.getLineCount(options)	N/ui/serverWidget Module	—
nlobjAssistantStep.getLineItemValue(group, name, line)	AssistantStep.getSublistValue(options)	N/ui/serverWidget Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjAssistantStep.getStepNumber()</code>	AssistantStep.stepNumber	N/ui/serverWidget Module	Note that <code>stepNumber</code> is a property.
<code>nlobjAssistantStep.setHelpText(help)</code>	AssistantStep.helpText	N/ui/serverWidget Module	Note that <code>helpText</code> is a property.
<code>nlobjAssistantStep.setLabel(label)</code>	AssistantStep.label	N/ui/serverWidget Module	Note that <code>label</code> is a property.

nlobjButton

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjButton</code>	serverWidget.Button	N/ui/serverWidget Module	—
<code>nlobjButton.setDisabled(disabled)</code>	Button.isDisabled	N/ui/serverWidget Module	Note that <code>isDisabled</code> is a property.
<code>nlobjButton.setLabel(label)</code>	Button.label	N/ui/serverWidget Module	Note that <code>label</code> is a property.
<code>nlobjButton.setVisible(visible)</code>	Button.isHidden	N/ui/serverWidget Module	Note that <code>isHidden</code> is a property.

nlobjColumn

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjColumn</code>	serverWidget.ListColumn	N/ui/serverWidget Module	—
<code>nlobjColumn.addParamToURL(param, value, dynamic)</code>	ListColumn.addParamToURL(options)	N/ui/serverWidget Module	—
<code>nlobjColumn.setLabel(label)</code>	ListColumn.label	N/ui/serverWidget Module	Note that <code>label</code> is a property.
<code>nlobjColumn.setURL(url, dynamic)</code>	ListColumn.setURL(options)	N/ui/serverWidget Module	—

nlobjConfiguration

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjConfiguration</code>	record.Record	N/record Module	Use the N/config Module method, config.load(options) , to return a <code>record.Record</code> object. Then use the <code>record.Record</code> object members to access the specified configuration page. For a script sample, see N/config Module Script Sample .
<code>nlobjConfiguration.getAllFields()</code>	Record.getFields()	N/record Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjConfiguration.getField(fldnam)	Record.getField(options)	N/record Module	—
nlobjConfiguration.getFieldText(name)	Record.getText(options)	N/record Module	—
nlobjConfiguration.getFieldTexts(name)	Record.getText(options)	N/record Module	—
nlobjConfiguration.getFieldValue(name)	Record.getValue(options)	N/record Module	—
nlobjConfiguration.getFieldValues(name)	Record.getValue(options)	N/record Module	—
nlobjConfiguration.getType()	Record.type	N/record Module	Note that type is a property.
nlobjConfiguration.setFieldText(name, text)	Record.setText(options)	N/record Module	—
nlobjConfiguration.setFieldTexts(name, text)	Record.setText(options)	N/record Module	—
nlobjConfiguration.setFieldValue(name, value)	Record.setValue(options)	N/record Module	—
nlobjConfiguration.setFieldValues(name, value)	Record.setValue(options)	N/record Module	—

nlobjContext

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjContext	runtime.Script runtime.Session runtime.User	N/runtime Module	—
nlobjContext.getCompany()	runtime.accountId	N/runtime Module	Note that accountId is a property.
nlobjContext.getDepartment()	User.department	N/runtime Module	Note that department is a property.
nlobjContext.getDeploymentId()	Script.deploymentId	N/runtime Module	Note that deploymentId is a property.
nlobjContext.getEmail()	User.email	N/runtime Module	Note that email is a property.
nlobjContext.getEnvironment()	runtime.envType	N/runtime Module	Note that envType is a property.
nlobjContext.getExecutionContext()	runtime.executionContext	N/runtime Module	Note that executionContext is a property.
nlobjContext.getFeature(name)	runtime.isFeatureInEffect(options)	N/runtime Module	—
nlobjContext.getLocation()	User.location	N/runtime Module	Note that location is a property.
nlobjContext.getLogLevel()	Script.logLevel	N/runtime Module	Note that logLevel is a property.
nlobjContext.getName()	User.name	N/runtime Module	Note that name is a property.

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjContext. getPercentComplete()	Script.percentComplete	N/runtime Module	Note that percentComplete is a property. For a script sample, see N/runtime Module Script Samples .
nlobjContext.getPermission (name)	User.getPermission(options)	N/runtime Module	—
nlobjContext.getPreference (name)	User.getPreference(options)	N/runtime Module	—
nlobjContext.getQueueCount()	runtime.queueCount	N/runtime Module	Note that queueCount is a property.
nlobjContext. getRemainingUsage()	Script.getRemainingUsage()	N/runtime Module	—
nlobjContext.getRole()	User.role	N/runtime Module	Note that role is a property.
nlobjContext.getRoleCenter()	User.roleCenter	N/runtime Module	Note that roleCenter is a property.
nlobjContext.getRoleId()	User.roleId	N/runtime Module	Note that roleId is a property.
nlobjContext.getScriptId()	Script.id	N/runtime Module	Note that id is a property.
nlobjContext.getSessionObject (name)	Session.get(options)	N/runtime Module	—
nlobjContext.getSetting(type, name)	Script.getParameter(options) Session.get(options) runtime.isFeatureInEffect (options) User.getPermission(options)	N/runtime Module	The method Script.getParameter(options) is equivalent to nlobjContext.getSetting('SCRIPT', name) . The method Session.get(options) is equivalent to nlobjContext.getSetting('SESSION', name) . The method runtime.isFeatureInEffect(options) is equivalent to nlobjContext.getSetting('FEATURE', name) . The method User.getPermission(options) is equivalent to nlobjContext.getSetting('PERMISSION', name) .
nlobjContext.getSubsidiary()	User.subsidiary	N/runtime Module	Note that subsidiary is a property.
nlobjContext.getUser()	User.id	N/runtime Module	Note that id is a property.
nlobjContext.getVersion()	runtime.version	N/runtime Module	Note that version is a property.
nlobjContext. setPercentComplete(pct)	Script.percentComplete	N/runtime Module	Note that percentComplete is a property.
nlobjContext.setSessionObject (name, value)	Session.set(options)	N/runtime Module	—
nlobjContext.setSetting(type, name, value)	Session.set(options)	N/runtime Module	—

nlobjCredentialBuilder

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjCredentialBuilder(string, domainString)	https.SecureString	N/https Module	—
nlobjCredentialBuilder.append(string)	SecureString.appendSecureString(options) SecureString.appendString(options)	N/https Module	—
nlobjCredentialBuilder.base64()	SecureString.convertEncoding(options)	N/https Module	—
nlobjCredentialBuilder.replace(string1, string2)	SecureString.replaceString(options)	N/https Module	—
nlobjCredentialBuilder.sha256()	SecureString.hash(options)	N/https Module	—
nlobjCredentialBuilder.utf8()	SecureString.convertEncoding(options)	N/https Module	—

nlobjCSVImport

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjCSVImport	task.CsvImportTask	N/task Module	Returned by task.create(options) . <pre> 1 var csvImpTaskObj = task.create({ 2 taskType: task.Task Type.CSV_IMPORT, 3 //Other Params 4 }); </pre>
nlobjCSVImport.setLinkedFile(sublist, file)	CsvImportTask.linkedFiles	N/task Module	Note that linkedFiles is a property.
nlobjCSVImport.setMapping(savedImport)	CsvImportTask.mappingId	N/task Module	Note that mappingId is a property.
nlobjCSVImport.setOption(option, value)	CsvImportTask.name	N/task Module	Note that name is a property.
nlobjCSVImport.setPrimaryFile(file)	CsvImportTask.importFile	N/task Module	Note that importFile is a property.
nlobjCSVImport.setQueue(string)	CsvImportTask.queueId	N/task Module	Note that queueId is a property.

nlobjDuplicateJobRequest

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjDuplicateJobRequest	task.EntityDeduplicationTask	N/task Module	Returned by task.create(options) . <pre> 1 var dedupTaskObj = task.create({ 2 taskType: task.TaskType.ENTITY_D EDUPLICATION, 3 //Other Params 4 }); </pre>

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjDuplicateJobRequest.setEntityType(entityType)	EntityDeduplicationTask.entityType	N/task Module	Note that entityType is a property.
nlobjDuplicateJobRequest.setMasterId(masterID)	EntityDeduplicationTask.masterRecordId	N/task Module	—
nlobjDuplicateJobRequest.setMasterSelectionMode(mode)	EntityDeduplicationTask.masterSelectionMode	N/task Module	Note that masterSelectionMode is a property.
nlobjDuplicateJobRequest.setOperation(operation)	EntityDeduplicationTask.dedupeMode	N/task Module	Note that dedupeMode is a property.
nlobjDuplicateJobRequest.setRecords(dupeRecords)	EntityDeduplicationTask.recordIds	N/task Module	Note that recordIds is a property.

nlobjEmailMerger

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjEmailMerger	render.EmailMergeResult	N/render Module	—
nlobjEmailMerger.merge()	See Notes	N/render Module	In SuiteScript 2.x, this is automatically called in render.mergeEmail(options) .
nlobjEmailMerger.setCustomRecord(recordType, recordId)	See Notes	N/render Module	In SuiteScript 2.x, this value is set with a render.mergeEmail(options) parameter.
nlobjEmailMerger.setEntity(entityType, entityId)	See Notes	N/render Module	In SuiteScript 2.x, this value is set with a render.mergeEmail(options) parameter.
nlobjEmailMerger.setRecipient(recipientType, recipientId)	See Notes	N/render Module	In SuiteScript 2.x, this value is set with a render.mergeEmail(options) parameter.
nlobjEmailMerger.setSupportCase(caseId)	See Notes	N/render Module	In SuiteScript 2.x, this value is set with a render.mergeEmail(options) parameter.
nlobjEmailMerger.setTransaction(transactionId)	See Notes	N/render Module	In SuiteScript 2.x, this value is set with a render.mergeEmail(options) parameter.

nlobjError

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjError	error.SuiteScriptError	N/error Module	—
nlobjError.getCode()	SuiteScriptError.name	N/error Module	Note that SuiteScriptError.name is a property.
nlobjError.getDetails()	SuiteScriptError.message	N/error Module	Note that SuiteScriptError.message is a property.

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjError.getId()</code>	SuiteScriptError.id	N/error Module	Note that <code>SuiteScriptError.id</code> is a property.
<code>nlobjError.getInternalId()</code>	SuiteScriptError.id	N/error Module	Note that <code>SuiteScriptError.id</code> is a property.
<code>nlobjError.getStackTrace()</code>	SuiteScriptError.stack	N/error Module	Note that <code>SuiteScriptError.stack</code> is a property
<code>nlobjError.getUserEvent()</code>	—	—	—

nlobjField

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjField</code>	serverWidget.Field record.Field	N/ui/serverWidget Module N/record Module	Use the N/ui/serverWidget module to create and modify form fields in a Suitelet. Use the N/record module to access field metadata in client and server scripts.

nlobjFieldGroup

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjFieldGroup</code>	serverWidget.FieldGroup	N/ui/serverWidget Module	—
<code>nlobjFieldGroup.setCollapsible(collapsible, hidden)</code>	FieldGroup.isCollapsible	N/ui/serverWidget Module	Note that <code>isCollapsible</code> is a property.
<code>nlobjFieldGroup.setLabel(label)</code>	FieldGroup.label	N/ui/serverWidget Module	Note that <code>label</code> is a property.
<code>nlobjFieldGroup.setShowBorder(show)</code>	FieldGroup.isBorderHidden	N/ui/serverWidget Module	Note that <code>isBorderHidden</code> is a property.
<code>nlobjFieldGroup.setSingleColumn(column)</code>	FieldGroup.isSingleColumn	N/ui/serverWidget Module	Note that <code>isSingleColumn</code> is a property.

nlobjFile

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjFile</code>	file.File	N/file Module	For a script sample, see N/file Module Script Samples .
<code>nlobjFile.getDescription()</code>	File.description	N/file Module	Note that <code>description</code> is a property.
<code>nlobjFile.getFolder()</code>	File.folder	N/file Module	Note that <code>folder</code> is a property.

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
			For a script sample, see N/file Module Script Samples .
nlobjFile.getId()	File.id	N/file Module	Note that id is a property. For a script sample, see N/file Module Script Samples .
nlobjFile.getName()	File.name	N/file Module	Note that name is a property. For a script sample, see N/file Module Script Samples .
nlobjFile.getSize()	File.size	N/file Module	Note that size is a property.
nlobjFile.getType()	File.fileType	N/file Module	Note that fileType is a property. For a script sample, see N/file Module Script Samples .
nlobjFile.getURL()	File.url	N/file Module	Note that url is a property.
nlobjFile.getValue()	File.getContents()	N/file Module	—
nlobjFile.isInactive()	File.isInactive	N/file Module	Note that isInactive is a property.
nlobjFile.isOnline()	File.isOnline	N/file Module	Note that isOnline is a property. For a script sample, see N/file Module Script Samples .
nlobjFile.setDescription(description)	File.description	N/file Module	Note that description is a property.
nlobjFile.setEncoding(encodingType)	File.encoding	N/file Module	Note that encoding is a property.
nlobjFile.setFolder(id)	File.folder	N/file Module	Note that folder is a property. You can also set the folder during file creation with file.create(options) . For a script sample, see N/file Module Script Samples .
nlobjFile.setIsInactive(inactive)	File.isInactive	N/file Module	Note that isInactive is a property.
nlobjFile.setIsOnline(online)	File.isOnline	N/file Module	Note that isOnline is a property. For a script sample, see N/file Module Script Samples .
nlobjFile.setName(name)	File.name	N/file Module	Note that name is a property. For a script sample, see N/file Module Script Samples .

nlobjForm

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjForm	serverWidget.Form	N/ui/serverWidget Module	—
nlobjForm.addButton(name, label, script)	Form.addButton(options)	N/ui/serverWidget Module	—
nlobjForm.addCredentialField(id, label, website, scriptId, value, entityMatch, tab)	Form.addCredentialField(options)	N/ui/serverWidget Module	—
nlobjForm.addField(name, type, label, sourceOrRadio, tab)	Form.addField(options)	N/ui/serverWidget Module	For a script sample, see the help topic N/ui/serverWidget Module Script Samples
nlobjForm.addFieldGroup(name, label, tab)	Form.addFieldGroup(options)	N/ui/serverWidget Module	—
nlobjForm.addPageLink(type, title, url)	Form.addPageLink(options)	N/ui/serverWidget Module	—
nlobjForm.addSubList(name, type, label, tab)	Form.addSublist(options)	N/ui/serverWidget Module	For a script sample, see the help topic N/ui/serverWidget Module Script Samples
nlobjForm.addSubmitButton(label)	Form.addSubmitButton(options)	N/ui/serverWidget Module	For a script sample, see the help topic N/ui/serverWidget Module Script Samples
nlobjForm.addSubTab(name, label, tab)	Form.addSubtab(options)	N/ui/serverWidget Module	—
nlobjForm.addTab(name, label)	Form.addTab(options)	N/ui/serverWidget Module	—
nlobjForm.getButton(name)	Form.getButton(options)	N/ui/serverWidget Module	—
nlobjForm.getField(name, radio)	Form.getField(options)	N/ui/serverWidget Module	—
nlobjForm.getSubList(name)	Form.getSublist(options)	N/ui/serverWidget Module	—
nlobjForm.getSubTab(name)	Form.getSubtab(options)	N/ui/serverWidget Module	—
nlobjForm.getTab(name)	Form.getTab(options)	N/ui/serverWidget Module	—
nlobjForm.getTabs()	Form.getTabs()	N/ui/serverWidget Module	—
nlobjForm.insertField(field, nextfld)	Form.insertField(options)	N/ui/serverWidget Module	—
nlobjForm.insertSubList(sublist, nextsub)	Form.insertSublist(options)	N/ui/serverWidget Module	—
nlobjForm.insertSubTab(subtab, nextsub)	Form.insertSubtab(options)	N/ui/serverWidget Module	—
nlobjForm.insertTab(tab, nexttab)	Form.insertTab(options)	N/ui/serverWidget Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjForm.removeButton(name)	Form.removeButton(options)	N/ui/serverWidget Module	—
nlobjForm.setFieldValues(values)	Form.updateDefaultValues(options)	N/ui/serverWidget Module	—
nlobjForm.setScript(script)	Form.clientScriptFileId Form.clientScriptModulePath	N/ui/serverWidget Module	Note that clientScriptFileId and clientScriptModulePath are properties. Use one of these SuiteScript 2.x properties to attach an ad hoc client script to a form.
nlobjForm.setTitle(title)	Form.title	N/ui/serverWidget Module	Note that title is a property.

nlobjFuture

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjFuture	task.EntityDeduplicationTaskStatus	N/task Module	—
nlobjFuture.isCancelled()	EntityDeduplicationTaskStatus.status	N/task Module	Note that status is a property.
nlobjFuture.isDone()	EntityDeduplicationTaskStatus.status	N/task Module	Note that status is a property.

nlobjJobManager

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjJobManager	task.EntityDeduplicationTaskStatus	N/task Module	—
nlobjJobManager.createJobRequest()	task.create(options)	N/task Module	—
nlobjJobManager.getFuture()	task.checkStatus(options)	N/task Module	—
nlobjJobManager.submit (nlobjDuplicateJobRequest)	EntityDeduplicationTask.submit()	N/task Module	—

nlobjList

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjList	serverWidget.List	N/ui/serverWidget Module	—
nlobjList.addButton(name, label, script)	List.addButton(options)	N/ui/serverWidget Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjList.addColumn(name, type, label, align)</code>	List.addColumn(options)	N/ui/serverWidget Module	—
<code>nlobjList.addEditColumn(column, showView, showHrefCol)</code>	List.addEditColumn(options)	N/ui/serverWidget Module	—
<code>nlobjList.addPageLink(type, title, url)</code>	List.addPageLink(options)	N/ui/serverWidget Module	—
<code>nlobjList.addRow(row)</code>	List.addRow(options)	N/ui/serverWidget Module	—
<code>nlobjList.addRows(rows)</code>	List.addRows(options)	N/ui/serverWidget Module	—
<code>nlobjList.setScript(script)</code>	List.clientScriptFileId List.clientScriptModulePath	N/ui/serverWidget Module	Note that <code>clientScriptFileId</code> and <code>clientScriptModulePath</code> are properties. Use one of these SuiteScript 2.x properties to attach an ad hoc client script to a form.
<code>nlobjList.setStyle(style)</code>	List.style	N/ui/serverWidget Module	Note that <code>style</code> is a property.
<code>nlobjList.setTitle(title)</code>	List.title	N/ui/serverWidget Module	Note that <code>title</code> is a property.

nlobjLogin

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjLogin</code>	See Notes.	N/auth Module	See <code>nlobjLogin</code> members.
<code>nlobjLogin.changeEmail(currentPassword, newEmail, justThisAccount)</code>	auth.changeEmail(options)	N/auth Module	For a script sample, see N/auth Module Script Sample .
<code>nlobjLogin.changePassword(currentPassword, newPassword)</code>	auth.changePassword(options)	N/auth Module	For a script sample, see N/auth Module Script Sample .

nlobjMergeResult

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
<code>nlobjMergeResult</code>	render.EmailMergeResult	N/render Module	-
<code>nlobjMergeResult.getBody()</code>	EmailMergeResult.body	N/render Module	Note that <code>body</code> is a property.
<code>nlobjMergeResult.getSubject()</code>	EmailMergeResult.subject	N/render Module	Note that <code>subject</code> is a property.

nlobjPortlet

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjPortlet	Portlet Object	See Notes	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.addColumn(name, type, label, just)	Portlet.addColumn(options)	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.addEditColumn(column, showView, showHrefCol)	Portlet.addEditColumn(options)	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.addField(name, type, label, source)	Portlet.addField(options)	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.addLine(text, url, indent)	Portlet.addLine(options)	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.addRow(row)	Portlet.addRow(options)	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.addRows(rows)	Portlet.addRows(options)	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.setHtml(html)	Portlet.html	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.setRefreshInterval(n)	-	—	There is no SuiteScript 2.x direct equivalent for this method. For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.setScript(scriptid)	Portlet.clientScriptFileId Portlet.clientScriptModulePath	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .
nlobjPortlet.setSubmitButton(url, label, target)	Portlet.setSubmitButton(options)	—	For additional information, see the help topic

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
			SuiteScript 2.x Portlet Script Type .
nlobjPortlet.setTitle(title)	Portlet.title	—	For additional information, see the help topic SuiteScript 2.x Portlet Script Type .

nlobjRecord

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjRecord	record.Record currentRecord.CurrentRecord	N/record Module N/currentRecord Module	—
nlobjRecord.commitLineItem (group)	Record.commitLine(options) CurrentRecord.commitLine(options)	N/record Module N/currentRecord Module	—
nlobjRecord.createCurrentLineItem Subrecord(sublist, fldname)	Record.getCurrentSublist Subrecord(options) CurrentRecord.getCurrentSublist Subrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.createSubrecord (fldname)	Record.getSubrecord(options) CurrentRecord.getSubrecord (options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.editCurrentLineItem Subrecord(sublist, fldname)	Record.getCurrentSublist Subrecord(options) CurrentRecord.getCurrentSublist Subrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.editSubrecord (fldname)	Record.getSubrecord(options) CurrentRecord.getSubrecord (options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.findLineItemMatrix Value(group, fldnam, column, val)	Record.findMatrixSublistLineWith Value(options) CurrentRecord.findMatrixSublist LineWithValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.findLineItemValue (group, fldnam, value)	Record.findSublistLineWithValue (options) CurrentRecord.findSublistLine WithValue(options)	N/record Module N/currentRecord Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjRecord.getAllFields()	Record.getFields()	N/record Module	—
nlobjRecord.getAllLineItemFields(group)	Record.getSublistFields(options)	N/record Module	—
nlobjRecord.getCurrentLineItemDateTimeValue(type, fieldId, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlobjRecord.getCurrentLineItemMatrixValue(group, fldnam, column)	Record.getCurrentMatrixSublistValue(options) CurrentRecord.getCurrentMatrixSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getCurrentLineItemValue(type, fldnam)	Record.getCurrentSublistValue(options) CurrentRecord.getCurrentSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getCurrentLineItemValues(type, fldname)	Record.getCurrentSublistValue(options) CurrentRecord.getCurrentSublistValue(options)	N/record Module N/currentRecord Module	Method returns an array for multi-select fields.
nlobjRecord.getDateTimeValue(fieldId, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlobjRecord.getField(fldnam)	Record.getField(options) CurrentRecord.getField(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getFieldText(name)	Record.getText(options) CurrentRecord.getText(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getFieldTexts(name)	Record.getText(options) CurrentRecord.getText(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getFieldValue(name)	Record.getValue(options) CurrentRecord.getValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getFieldValues(name)	Record.getValue(options) CurrentRecord.getValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getId()	Record.id	N/record Module	—
nlobjRecord.getLineItemCount()	Record.getLineCount(options)	N/record Module	—
nlobjRecord.getLineItemDateTimeValue(type, fieldId, lineNum, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlobjRecord.getLineItemField(group, fldnam, linenum)	Record.getSublistField(options) CurrentRecord.getSublistField(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getLineItemMatrixField(group, fldnam, linenum, column)	Record.getMatrixSublistField(options) CurrentRecord.getMatrixSublistField(options)	N/record Module N/currentRecord Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjRecord.getLineItemMatrixValue(group, fldnam, lineum, column)	Record.getMatrixSublistValue(options) CurrentRecord.getMatrixSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getLineItemText(group, fldnam, lineum)	Record.getSublistText(options) CurrentRecord.getSublistText(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getLineItemValue(group, name, lineum)	Record.getSublistValue(options) CurrentRecord.getSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getLineItemValues(type, fldname, lineum)	Record.getSublistValue(options) CurrentRecord.getSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getMatrixCount(group, fldnam)	Record.getMatrixHeaderCount(options) CurrentRecord.getMatrixHeaderCount(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getMatrixField(group, fldname, column)	Record.getMatrixHeaderField(options) CurrentRecord.getMatrixHeaderField(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getMatrixValue(group, fldnam, column)	Record.getMatrixHeaderValue(options) CurrentRecord.getMatrixHeaderValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.getRecordType()	Record.type	N/record Module	—
nlobjRecord.insertLineItem(group, lineum)	Record.insertLine(options) CurrentRecord.insertLine(options)	N/record Module N/currentRecord Module	—
nlobjRecord.removeLineItem(group, lineum)	Record.removeLine(options) CurrentRecord.removeLine(options)	N/record Module N/currentRecord Module	—
nlobjRecord.removeCurrentLineItemSubrecord(sublist, fldname)	Record.removeCurrentSublistSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.removeSubrecord(fldname)	Record.removeSubrecord(options) CurrentRecord.removeSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.selectLineItem(group, lineum)	Record.selectLine(options) CurrentRecord.selectLine(options)	N/record Module N/currentRecord Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjRecord.selectNewLineItem(group)	Record.selectNewLine(options) CurrentRecord.selectNewLine(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setCurrentLineItemDateTimeValue(type, fieldId, dateTime, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlobjRecord.setCurrentLineItemMatrixValue(group, fldnam, column, value)	Record.setCurrentMatrixSublistValue(options) CurrentRecord.setCurrentMatrixSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setCurrentLineItemValue(group, name, value)	Record.setCurrentSublistValue(options) CurrentRecord.setCurrentSublistValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setDateTimeValue(fieldId, dateTime, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlobjRecord.setFieldText(name, text)	Record.setText(options) CurrentRecord.setText(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setFieldTexts(name, text)	Record.setText(options) CurrentRecord.setText(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setFieldValue(name, value)	Record.setValue(options) CurrentRecord.setValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setFieldValues(name, value)	Record.setValue(options) CurrentRecord.setValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.setLineItemDateTimeValue(type, fieldId, lineNum, dateTime, timeZone)	See Notes	N/format Module	Use the N/format module to mimic this functionality in SuiteScript 2.x.
nlobjRecord.setLineItemValue(group, name, linenum, value)	Record.setSublistValue(options)	N/record Module	—
nlobjRecord.setMatrixValue(group, fldnam, column, value)	Record.setMatrixHeaderValue(options) CurrentRecord.setMatrixHeaderValue(options)	N/record Module N/currentRecord Module	—
nlobjRecord.viewCurrentLineItemSubrecord(sublist, fldname)	Record.getCurrentSublistSubrecord(options) CurrentRecord.getCurrentSublistSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjRecord.viewLineItemSubrecord(sublist, fldname, linenum)	Record.getSublistSubrecord(options)	N/record Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjRecord.viewSubrecord (fldname)	Record.getSubrecord(options) CurrentRecord.getSubrecord(options)	N/record Module N/currentRecord Module	Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .

nlobjRequest

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjRequest	http.ServerRequest	N/http Module N/https Module	—
nlobjRequest.getAllHeaders()	ServerRequest.headers	N/http Module N/https Module	ServerRequest.headers(options) is read-only.
nlobjRequest.getAllParameters()	ServerRequest.parameters	N/http Module N/https Module	ServerRequest.parameters(options) is read-only.
nlobjRequest.getBody()	ServerRequest.body	N/http Module N/https Module	ServerRequest.body(options) is read-only.
nlobjRequest.getFile(id)	ServerRequest.files	N/http Module N/https Module	ServerRequest.files(options) is read-only.
nlobjRequest.getHeader(name)	ServerRequest.headers	N/http Module N/https Module	ServerRequest.headers(options) is read-only.
nlobjRequest.getLineItemCount (group)	ServerRequest.getLineCount(options)	N/http Module N/https Module	—
nlobjRequest.getLineItemValue (group, name, line)	ServerRequest.getSublistValue(options)	N/http Module N/https Module	—
nlobjRequest.getMethod()	ServerRequest.method	N/http Module N/https Module	ServerRequest.method(options) is read-only.
nlobjRequest. getParameter(name)	ServerRequest.parameters	N/http Module N/https Module	ServerRequest.parameters(options) is read-only.
nlobjRequest.getParameterValues (name)	ServerRequest.parameters	N/http Module N/https Module	ServerRequest.parameters(options) is read-only.
nlobjRequest.getURL()	ServerRequest.url	N/http Module N/https Module	ServerRequest.url is read-only.

nlobjResponse

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjResponse	http.ServerResponse	N/http Module N/https Module	—
nlobjResponse. addHeader(name, value)	ServerResponse.addHeader(options)	N/http Module N/https Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjResponse.getAllHeaders()	ServerResponse.getHeader(options) or ServerResponse.headers	N/http Module / https Module	—
nlobjResponse.getBody()	ClientResponse.body	N/http Module / https Module	Note that <code>ClientResponse.body</code> is a property.
nlobjResponse.getCode()	ClientResponse.code	N/http Module / https Module	Note that <code>ClientResponse.code</code> is a property.
nlobjResponse.getError()	See Notes	N/http Module / https Module	There is no SuiteScript 2.x equivalent for this method.
nlobjResponse.getHeader(name)	ServerResponse.getHeader(options)	N/http Module / https Module	—
nlobjResponse.getHeaders(name)	ServerResponse.getHeader(options)	N/http Module / https Module	If multiple values are assigned to the header name, <code>serverResponse.getHeader(options)</code> returns the values as an Array.
nlobjResponse.renderPDF(xmlString)	ServerResponse.renderPdf(options)	N/http Module / https Module	—
nlobjResponse.setCDNCacheable(type)	ServerResponse.setCdnCacheable(options)	N/http Module / https Module	—
nlobjResponse.setContentType(type, name, disposition)	See Notes	N/http Module / https Module	There is no direct equivalent for this method in SuiteScript 2.x.
nlobjResponse.setEncoding(encodingType)	See Notes	N/http Module / https Module	There is no direct equivalent for this method in SuiteScript 2.x. When you use https.ServerResponse , textual responses are encoded in UTF-8 by default. If you want to return a file with an alternative character encoding, see the help topic Return a File with Alternative Character Encoding .
nlobjResponse.setHeader(name, value)	ServerResponse.setHeader(options)	N/http Module / https Module	—
nlobjResponse.sendRedirect(type, identifier, id, editmode, parameters)	ServerResponse.sendRedirect(options)	N/http Module / https Module	—
nlobjResponse.write(output)	ServerResponse.write(options)	N/http Module / https Module	—
nlobjResponse.writeLine(output)	ServerResponse.writeLine(options)	N/http Module / https Module	—
nlobjResponse.writePage(pageobject)	ServerResponse.writePage(options)	N/http Module / https Module	—

nlobjSearch

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSearch	search.Search	N/search Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSearch.addColumn(column)	Search.columns	N/search Module	Note that <code>Search.columns</code> is a property.
nlobjSearch.addColumns(columns)	Search.columns	N/search Module	Note that <code>Search.columns</code> is a property.
nlobjSearch.addFilter(filter)	Search.filters	N/search Module	Note that <code>Search.filters</code> is a property.
nlobjSearch.addFilters(filters)	Search.filters	N/search Module	Note that <code>Search.filters</code> is a property.
nlobjSearch.deleteSearch()	search.delete(options)	N/search Module	For a script sample, see the help topic N/search Module Script Samples .
nlobjSearch.getColumns()	Search.columns	N/search Module	Note that <code>columns</code> is a property.
nlobjSearch.getFilterExpression()	Search.filterExpression	N/search Module	Note that <code>filterExpression</code> is a property.
nlobjSearch.getFilters()	Search.filters	N/search Module	Note that <code>filters</code> is a property.
nlobjSearch.getId()	Search.searchId	N/search Module	Note that <code>id</code> is a property.
nlobjSearch.getIsPublic()	Search.isPublic	N/search Module	Note that <code>isPublic</code> is a property.
nlobjSearch.getScriptId()	Search.id	N/search Module	—
nlobjSearch.getSearchType()	Search.searchType	N/search Module	Note that <code>searchType</code> is a property.
nlobjSearch.runSearch()	Search.run()	N/search Module	—
nlobjSearch.saveSearch(title, scriptId)	Search.save()	N/search Module	—
nlobjSearch.setColumns(columns)	Search.columns	N/search Module	Note that <code>columns</code> is a property.
nlobjSearch.setFilterExpression(filterExpression)	Search.filterExpression	N/search Module	Note that <code>filterExpression</code> is a property.
nlobjSearch.setFilters(filters)	Search.filters	N/search Module	Note that <code>filters</code> is a property.
nlobjSearch.setIsPublic(type)	Search.isPublic	N/search Module	Note that <code>isPublic</code> is a property.
nlobjSearch.setRedirectURLToSearch()	redirect.toSavedSearch(options) redirect.toSearch(options)	N/redirect Module	—
nlobjSearch.setRedirectURLToSearchResults()	redirect.toSearchResult(options) redirect.toSavedSearchResult(options)	N/redirect Module	—

nlobjSearchColumn

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSearchColumn	search.Column	N/search Module	—

nlobjSearchFilter

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSearchFilter	search.Filter	N/search Module	—

nlobjSearchResult

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSearchResult	search.Result	N/search Module	—

nlobjSearchResultSet

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSearchResultSet	search.ResultSet	N/search Module	—

nlobjSelectOption

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSelectOption	See Notes	N/record Module	See mapping for nlobjSelectOption methods.
nlobjSelectOption.getId()	N/record: Field. getSelectOptions(options) N/currentRecord: Field. getSelectOptions(options)	N/record Module N/currentRecord Module	—
nlobjSelectOption.getText()	N/record: Field. getSelectOptions(options) N/currentRecord: Field. getSelectOptions(options)	N/record Module N/currentRecord Module	—

nlobjSublist

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSubList	serverWidget.Sublist	N/ui/serverWidget Module	—
nlobjSublist.addButton(name, label, script)	Sublist.addButton(options)	N/ui/serverWidget Module	—
nlobjSublist.addField(name, type, label, source)	Sublist.addField(options)	N/ui/serverWidget Module	—
nlobjSublist.addMarkAllButtons()	Sublist.addMarkAllButtons()	N/ui/serverWidget Module	—

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSublist.addRefreshButton()	Sublist.addRefreshButton()	N/ui/serverWidget Module	—
nlobjSublist.getLineItemCount()	Sublist.lineCount	N/ui/serverWidget Module	Note that lineCount is a property
nlobjSublist.getLineItemValue(group, fldnam, linenum)	Sublist.getSublistValue(options)	N/ui/serverWidget Module	—
nlobjSublist.setAmountField(field)	Sublist.updateTotallingFieldId(options)	N/ui/serverWidget Module	—
nlobjSublist.setDisplayType(type)	Sublist.displayType	N/ui/serverWidget Module	Note that displayType is a property
nlobjSublist.setHelpText(help)	Sublist.helpText	N/ui/serverWidget Module	Note that helpText is a property
nlobjSublist.setLabel(label)	Sublist.label	N/ui/serverWidget Module	Note that label is a property
nlobjSublist.setLineItemValue(name, linenum,value)	Sublist.setSublistValue(options)	N/ui/serverWidget Module	—
nlobjSublist.setLineItemValues(values)	See Notes	N/ui/serverWidget Module	There is not a SuiteScript 2.x direct equivalent for this method.
nlobjSublist.setUniqueField(name)	Sublist.updateUniqueFieldId(options)	N/ui/serverWidget Module	Note that uniqueFieldId is a property

nlobjSubrecord

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjSubrecord	See Notes	N/record Module	SuiteScript 2.x subrecords are returned as record.Record objects. Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjSubrecord.cancel()	See Notes	N/record Module	SuiteScript 2.x subrecords are returned as record.Record objects. Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .
nlobjSubrecord.commit()	See Notes	N/record Module	This API does not have a SuiteScript 2.x equivalent. SuiteScript 2.x subrecords are returned as record.Record objects. Note that scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For additional

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
			information, see the SuiteScript 2.x topics under SuiteScript 2.x Scripting Subrecords .

nlobjTab

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjTab	serverWidget.Tab	N/ui/serverWidget Module	—
nlobjTab.setLabel(label)	Tab.label	N/ui/serverWidget Module	Note that label is a property
nlobjTab.setHelpText(help)	Tab.helpText	N/ui/serverWidget Module	Note that helpText is a property

nlobjTemplateRenderer

SuiteScript 1.0 API	SuiteScript 2.x API	SuiteScript 2.x Module	Notes
nlobjTemplateRenderer	render.TemplateRenderer	N/render Module	For a script sample, see N/render Module Script Samples .
nlobjTemplateRenderer.addRecord(var, record)	TemplateRenderer.addRecord(options)	N/render Module	For a script sample, see N/render Module Script Samples .
nlobjTemplateRenderer.addSearchResults(var, searchResult)	TemplateRenderer.addSearchResults(options)	N/render Module	For a script sample, see N/render Module Script Samples .
nlobjTemplateRenderer.renderToResponse()	TemplateRenderer.renderToResponse(options)	N/render Module	—
nlobjTemplateRenderer.renderToString()	TemplateRenderer.renderAsString()	N/render Module	—
nlobjTemplateRenderer.setTemplate()	TemplateRenderer.templateContent	N/render Module	For a script sample, see N/render Module Script Samples .

SuiteScript 2.x Global Objects

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x includes the following global objects. You can use these objects in your scripts without loading them as dependencies.

- [define Object](#)
- [require Function](#)
- [log Object](#)
- [util Object](#)
- [toString\(\)](#)
- [JSON object](#)
- [Promise Object](#)
- [Iterator](#)

Note: In JavaScript, all functions are objects. The [define Object](#) and [require Function](#) topics discuss `define()` and `require()` used by SuiteScript 2.x to load and define modules.

Note: Also, to safely use the `require` object or the `N/log` and `N/util` modules, you should specifically import those modules as dependencies of your scripts, especially in your SuiteScript 2.1 scripts.

define Object

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The `define` object is an overloaded function that is used to create entry point scripts and custom modules in SuiteScript 2.x. This function executes asynchronously on the client and synchronously on the server. The `define` object conforms to the [Asynchronous Module Definition \(AMD\)](#) specification.

Note: An overloaded function has multiple signatures. A signature is the function name and all available parameters.

SuiteScript 2.x supports the following `define()` signatures:

Type	Name	Return Type / Value Type	Description
Function	define(moduleObject)	object	Returns a module object based on the supplied <code>moduleObject</code> argument. The <code>moduleObject</code> argument can be any JavaScript object, including a function. Use this <code>define()</code> signature if your entry point script or custom module requires no dependencies.
	define(id, [dependencies,] moduleObject)	object	Loads all dependencies and then executes the supplied callback function. Returns a module object based on the callback.

Use the `define()` function to do the following:

- Create a SuiteScript script file. Load the required dependent modules and define the functionality for the SuiteScript script type in the callback function. The return statement in the callback function must include at least one entry point and entry point function. All entry points must belong to the same script type.

Any implementation of a SuiteScript script type that returns an entry point must use the `define()` function.

- Create and return a custom module. The custom module can then be used as dependency in another script. Use the `define(id, [dependencies,] moduleObject)` signature if your module requires dependencies. If the custom module does not require any dependencies, use the `define(moduleObject)` signature.

For more information about custom modules, see the help topic [SuiteScript 2.x Custom Modules](#).

For more information about entry points, see the help topic [SuiteScript 2.x Script Types](#).

define() Function Guidelines

Use the following guidelines with the `define()` function:

- SuiteScript API calls can be executed only after the `define` callback's return statement has executed. Consequently, you cannot use native SuiteScript 2.x module methods when you *create* a custom module. You can make SuiteScript API calls after the Module Loader creates and loads the custom module.
- If you need to debug your code on demand in the SuiteScript Debugger, you must use a `require()` Function. The SuiteScript Debugger cannot step through a `define()` function.
- All dependencies used in the `define()` function are loaded before the callback function executes.
- You can load only modules that are stored in the NetSuite File Cabinet. Do not attempt to import scripts using HTTP or HTTPS.

For example, if you specify `define(['http://somewebsite.com/purchaseTotal.js'], function(purchaseTotal){...})`, the `purchaseTotal` dependency is not valid.

define(moduleObject)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Function used to create entry point scripts and custom modules in SuiteScript 2.x. For more information, see the help topics SuiteScript 2.x Entry Point Script Creation and Deployment and SuiteScript 2.x Custom Modules. Use this <code>define()</code> signature if your entry point script or custom module requires no dependencies. If you are creating an entry point script, the <code>define()</code> function must return an object consisting of at least one key:value pair. Each key must be an entry point and the corresponding value must be a named entry point function. All entry points must be for the same script type. Your script can have only one entry point script and the entry point script must be only one script type.
Returns	Object
Global object	define Object
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description	Since
moduleObject	Object	required	A callback function or a module object.	2015.2

Syntax

The following code show sample syntax for the define(moduleObject) function signature. These code samples are not functional examples and do not provide complete list of ways this define() signature can be used.

Define a Function

```

1 // lib.js
2 define({
3     test: function ()
4     {
5         return true;
6     }
7 });

```

OR

```

1 // lib.js
2 define(function ()
3 {
4     return true
5 });

```

Define an object

```

1 // lib.js
2 define({
3     color: "black",
4     size: "unysize"
5 });

```

Define a Primitive Value

```

1 // lib.js
2 define("test");

```

define(id, [dependencies,] moduleObject)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Function used to create entry point scripts and custom modules in SuiteScript 2.x. For more information, see the help topics SuiteScript 2.x Entry Point Script Creation and Deployment and SuiteScript 2.x Custom Modules. Use this define() signature if your entry point script or custom module has required dependencies. If you are creating an entry point script, the define() function must return an object consisting of at least one key:value pair. Each key must be an entry point and the corresponding value must be a named entry point function. All entry points must be for the same script type. Your script can have only one entry point script and the entry point script must be only one script type. Your
---------------------------	---

	entry point script can, however, load multiple custom modules as dependencies. There is no limit to the number of dependencies your entry point script can load.
Returns	Object
Global object	define Object
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description	Since
id	string	optional	Defines the id of the module.	2015.2
dependencies	string []	optional	Represents all module dependencies required by the callback function. Use the following syntax: - Native SuiteScript 2.x modules: ['N/<module name>'] - Custom modules: ['/path to module file in File Cabinet'/<module name>'] For other options, see the help topic Module Dependency Paths .	2015.2
moduleObject	Function Object	required	A callback function or a module object	2015.2

Errors

Error Code	Message	Thrown If
MODULE_DOES_NOT_EXIST	Module does not exist: {module path/name}	The NetSuite module or custom module listed as a dependency does not exist. Note that NetSuite only reports the first error encountered. If you have multiple dependent modules and you receive this error, verify that all module paths and names are correct.

Syntax for Module ID

The following code shows sample syntax for the `define(id, [dependencies,] callback)` function signature. These code samples are not functional examples and do not provide complete list of ways this `define()` signature can be used.

```

1 ...
2 define('myModule', ['test', '/sample'], function(test, sample){...});
3 ...

```

Syntax for Entry Point Script

The following example is a SuiteScript user event script type that creates a Phone Call record on the `afterSubmit` trigger.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define(['N/record'],
7      function (record)
8      {
9          function createPhoneCall(context)
10         {
11             if (context.type !== context.UserEventTypes.CREATE)
12                 return;
13             var customerRecord = context.newRecord;
14             if (customerRecord.getValue('salesrep'))
15             {
16                 var phoneCall = record.create({
17                     type: record.Type.PHONE_CALL,
18                     isDynamic: true
19                 });
20                 phoneCall.setValue({
21                     fieldId: 'title',
22                     value: 'Make follow-up call to new customer'
23                 });
24                 phoneCall.setValue('assigned', customerRecord.getValue('salesrep'));
25                 phoneCall.setValue('phone', customerRecord.getValue('phone'));
26                 try
27                 {
28                     var callId = phoneCall.save();
29                     log.debug({
30                         title: 'Call record created successfully',
31                         details: 'Id: ' + callId
32                     });
33                 }
34                 catch (e)
35                 {
36                     log.error(e.name);
37                 }
38             }
39         }
40         return {
41             afterSubmit: createPhoneCall
42         };
43     });

```

Syntax for Custom Module

The following code shows the syntax for creating a custom SuiteScript 2.x module in the script file `lib.js`.

```

1  // lib.js
2  define(['./api/bar'], function(bar){ // require bar custom module
3      return {
4          makeSomething: function(){ // define function lib.makeSomething()
5              var barObj = bar.create(); // use create() function from bar custom module
6              return bar.convertToThing(); // returns the value of bar module function convertToThing()
7          }
8      }
9  });

```

The following code shows the syntax for calling the function `lib` from the custom module `test.js` in a separate script file:

```

1  // test.js
2  require(['./lib'], function (lib) { // require custom module (defined above)
3
4      return lib.makeSomething(); // return value of makeSomething function in custom module
5
6  });

```

require Function

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The require function is a global object that implements the require() Module Loader interface for SuiteScript 2.x. It conforms to the Asynchronous Module Definition (AMD) specification. When NetSuite executes the require function, it executes the callback function and loads the dependencies when they are needed.

This function executes asynchronously on the client and synchronously on the server.

Note: Only use the require function if you want to load an existing module. If you want to create an entry point script or a new custom module, use the [define Object](#).

Use the require function to progressively load native SuiteScript 2.x modules and custom modules. When you use the require function, dependencies are not loaded until they are needed which can help increase script performance. For example, if you include lib1 as a dependency in your require function, when you call a method that is included in lib1, the Module Loader then loads the module and executes the method. For code samples on how the require function progressively loads modules, see [require\(\[dependencies,\] callback\)](#) and [Custom Modules Frequently Asked Questions](#).

Type	Name	Return Type / Value Type	Description
Function	require([dependencies,] callback)	void	Loads a SuiteScript 2.x entry point script or a SuiteScript 2.x custom module. Executes the callback function and loads the dependencies when they are required.

Note: You can configure a require Object by associating a script to a JSON-formatted configuration file using the @NAmcConfig JSDoc tag. This is helpful to configure custom module loading. Properties that can hold feature metadata, aliases, paths, package, and mapping information related to a module id are supported. For more information about configuring a require Object, see [require Configuration](#).

require([dependencies,] callback)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Function used to load a module only when the module is needed. When NetSuite executes the require function, it executes the callback function specified in the require function and loads the dependencies when they are required. If you add a module as a dependency and the module is never used, the dependency is never loaded. Note: This function conforms to the Asynchronous Module Definition (AMD) specification. For more information, see require Function.
Returns	void
Global object	require Function
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description	Since
dependencies	string []	optional	Represents all module dependencies required by the callback function. Use the following syntax: - Native SuiteScript 2.x modules: ['N/<module name>'] - Custom modules: ['/<path to module file in File Cabinet>/<module name>'] See the help topic Module Dependency Paths .	2015.2
callback	Function	required	Callback function to execute. Dependent modules are not loaded until they are required.	2015.2

Errors

Error Code	Message	Thrown If
MODULE_DOES_NOT_EXIST	Module does not exist: {module path/name}	The NetSuite module or custom module dependency does not exist. Note that NetSuite only reports the first error encountered. If you have multiple dependent modules and you receive this error, verify that all module paths and names are correct.

Syntax

The following example shows progressive loading of modules in a script using the require function.

```

1  define({
2      newInstance: function (type)
3      {
4          switch (type)
5          {
6              case 'lib1' :
7                  require(['lib1'], function (lib1) // Module Loader loads lib1
8                  {
9                      return new lib1();
10                 })
11                 break;
12              case 'lib2' :
13                  require(['lib2'], function (lib2) // Module Loader loads lib2
14                  {
15                      return new lib2();
16                 })
17                 break;
18              default :
19                  return null;
20          }
21      }
22  });

```

require Configuration

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can configure a require Object by associating a script to a JSON-formatted configuration file using the @NAmcConfig JSDoc tag within your script.



Important: Configuration of a require Object is optional and for advanced users only. If you must configure a require Object, the @NAmcConfig JSDoc tag is suited for general use and is the preferred way to configure a require Object. However, existing scripts with calls to `require.config()` can use this method with a context argument (although not preferred). Ensure that the call includes a context parameter and that its value is not a file path.

If you include a valid @NAmcConfig JSDoc tag, SuiteScript implements the require configuration settings (in the specified file) **before** loading dependencies. Using different require configuration files allows you to run multiple client scripts with different configurations based on the @NAmcConfig JSDoc tag included in each script. This also supports re-use by letting you use a common configuration across multiple scripts.

To configure a require Object, add the @NAmcConfig JSDoc tag and provide the path to the configuration file. Configuration files must be located in the File Cabinet. For example:

```
1  /**
2   * @NAmcConfig /SuiteScripts/configuration.json
3   */
```

SuiteScript will require a custom entry point module and its dependencies using the AMD configuration settings in the specified configuration file. Your require configuration must be in JSON format. For example:

```
1  {
2     "baseUrl" : "/SuiteBundles"
3  }
```



Important: Ensure that your require configuration files use JSON syntax (and not JavaScript syntax). For more information about JSON, visit <http://json.org/>. You can use `JSON.stringify(obj)` to convert a JavaScript object value to a key-value pair string in JSON form.

For a list of supported configuration parameters, see [require Configuration Parameters](#). Properties that can hold feature metadata, aliases, paths, package, and mapping information related to a module id are supported.

require Configuration Parameters

SuiteScript accepts the require configuration values described at <https://github.com/amdjs/amdjs-api/blob/master/CommonConfig.md> (version fd45c71).

You can use the @NAmcConfig JSDoc tag to specify a configuration file that defines configuration values.

The following configuration parameters are supported in a require Object configuration file:

Parameter	Type	Required / Optional	Sample Usage	Since
baseUrl	string	optional	To configure a shorter relative path by indicating the root folder that holds the modules in the File Cabinet.	2015.2

Parameter	Type	Required / Optional	Sample Usage	Since
config	Object	optional	To assign attributes, such as metadata.	2015.2
map	Object	optional	- To specify an alias. - To handle multiple names for a module, - To load a set of identically-named but unique modules, such as dependency on multiple module versions.	2015.2
packages	Object[]	optional	To configure a special lookup suitable for traditional CommonJS packages that you want to use as a custom module.	2015.2
paths	Object	optional	- To create a named alias to a path, - For testing purposes, pass in an object that serves as a mock-up of another module. - To set up a custom name for a SuiteScript module.	2015.2
shim	Object	optional	To prepare a non-AMD JS library for loading.	2015.2

Refer to <https://github.com/amdjs/amdjs-api/blob/master/CommonConfig.md> for examples of each configuration parameter.



Important: Configuration of a require Object is optional and for advanced users only. If you must configure a require Object, the @NAmcConfig JSDoc tag is suited for general use and is the preferred way to configure a require Object. However, existing scripts with calls to `require.config()` can use this method with a context argument (although not preferred). Ensure that the call includes a context parameter and that its value is not a file path.

log Object

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

NetSuite loads the log Object by default for all script types. You do not need to load it manually. However, you can choose to load it using the [N/log Module](#), for testing purposes.



Note: SuiteScript 2.x provides global objects including [log Object](#) and [util Object](#). If you are using the words 'log' or 'util' as variable names in your SuiteScript 1.0 scripts and you have both SuiteScript 1.0 and SuiteScript 2.0 scripts running on the same record you will receive an error. Both the 'log' and 'util' words are reserved in SuiteScript 2.x. You will need to update your SuiteScript 1.0 scripts that use 'log' or 'util' as variables names.

log Object Members

- [log.debug\(options\)](#)
- [log.audit\(options\)](#)
- [log.emergency\(options\)](#)
- [log.error\(options\)](#)

For more details about the log Object and its methods, see [N/log Module](#).

util Object

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

NetSuite loads the util Object is loaded by default for all script types. You do not need to load it manually. However, you can choose to load it using the [N/util Module](#), for testing purposes.

Note: SuiteScript 2.x provides global objects including [log Object](#) and [util Object](#). If you are using the words 'log' or 'util' as variable names in your SuiteScript 1.0 scripts and you have both SuiteScript 1.0 and SuiteScript 2.0 scripts running on the same record you will receive an error. Both the 'log' and 'util' words are reserved in SuiteScript 2.x. You will need to update your SuiteScript 1.0 scripts that use 'log' or 'util' as variables names.

util Object Members

- [util.isArray\(obj\)](#)
- [util.isBoolean\(obj\)](#)
- [util.isDate\(obj\)](#)
- [util.isFunction\(obj\)](#)
- [util.isNumber\(obj\)](#)
- [util.isObject\(obj\)](#)
- [util.isRegExp\(obj\)](#)
- [util.isString\(obj\)](#)

The util object also includes the following utility methods:

- [util.each\(iterable, callback\)](#)
- [util.extend\(receiver, contributor\)](#)

For more details about the util Object and its methods, see [N/util Module](#).

toString()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to determine an object's type. This is a global method that is loaded by default for all native SuiteScript 2.x API objects. Note: Consider this method a replacement for the instanceof operator (which is not supported). SuiteScript 2.x members are immutable; you cannot construct or modify a native SuiteScript 2.x member. Consequently, if you attempt to call instanceof, an undefined error is thrown. For information about toString() as a native JavaScript method, see https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_objects/object/toString.
Returns	The object type as a string

Supported Script Types	Client and server scripts
Governance	None
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It is not a functional example.

```

1 | ...
2 | var type = mapContext.toString(); // When called on mapReduce.MapContext, toString returns "mapReduce.MapContext"
3 | ...

```

JSON object

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x supports the JavaScript Object Notation (JSON) standard. You can use the JSON object to parse text as a JSON object and convert strings to JSON notation. For more information, see [JSON.parse\(text\)](#) and [JSON.stringify\(obj\)](#).



Important: The following sections are included as a summary and are intended for reference only. For additional information about JSON, see <http://www.ietf.org/rfc/rfc4627.txt>.

JSON.parse(text)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Parse a string as a JSON object and returns the object. The text parameter must conform to the JSON standard. See http://www.ietf.org/rfc/rfc4627.txt .
Returns	Object
Supported Script Types	Client and server scripts
Governance	None
Global object	JSON object
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description	Since
text	string	required	Text to parse as a JSON object. The string must conform to the JSON standard.	2015.2
reviver	Function	optional	Specifies how to transform the parsed value before it is returned	2015.2

Syntax

The following code snippet shows the syntax for this member. It is not a functional example.

```

1 ...
2 var text = '{ "employees" : [' +
3     '{ "firstName":"John" , "lastName":"Doe" },' +
4     '{ "firstName":"Anna" , "lastName":"Smith" },' +
5     '{ "firstName":"Peter" , "lastName":"Jones" } ]}';
6 var obj = JSON.parse(text);
7 var firstEmp = obj.employees[1].firstName + " " + obj.employees[1].lastName;
8 ...

```

JSON.stringify(obj)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Converts a JavaScript object or value to a JSON string For more information about JSON object format, see http://www.ietf.org/rfc/rfc4627.txt .
Returns	JSON string
Supported Script Types	Client and server scripts
Governance	None
Global object	JSON object
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description	Since
value	Object	required	The value to convert to a JSON string	2015.2
replacer	Function	optional	Function that changes the behavior of the stringification process	2015.2
space	Object	optional	A string or number that is used to insert white space in the output JSON string for readability	2015.2

Syntax

The following code snippet shows the syntax for this member. It is not a functional example.

```

1 var contact = {
2     firstName: 'John',
3     lastName : 'Doe',
4     jobTitle : 'CEO'
5 };
6
7 var jsonString = JSON.stringify(contact);

```

This method converts the contact object to the following string:

```
{"firstName": "John", "lastName": "Doe", "jobTitle": "CEO"}
```

Promise Object

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A promise is a JavaScript object that represents the eventual result of an asynchronous process. After the promise object is created, it serves as a placeholder for the future success or failure of the asynchronous process. While the promise object is waiting, the remaining segments of the script can execute. With promises, you can write asynchronous code that is intuitive and efficient.

A promise holds one of the following values:

- fulfilled – The operation is successful.
- rejected – The operation failed.
- pending – The operation is still in progress and has not yet been fulfilled or rejected.

When it is first created, a promise holds the value pending. After the associated process is complete, the value changes to fulfilled for a successful completion or to rejected for an unsuccessful completion. A success or failure callback function attached to the promise is called when the process is complete. Note that a promise can only succeed or fail one time. When the value of the promise updates to fulfilled or rejected, it cannot change.

SuiteScript 2.x provides promise APIs for selected modules as listed in [SuiteScript 2.x Promise APIs](#). In SuiteScript 2.x, all client scripts support the use of promises. Starting with NetSuite 2021.1, a subset of server scripts also support the use of promises. Custom promises can also be created for client scripts. For an example of creating a custom promise, see [Custom Promises](#).

When you code your SuiteScript scripts to be asynchronous, you should also consider following best practices. A few best practices are included in [Best Practices for Asynchronous Programming with SuiteScript](#).

For more information about JavaScript promises, see https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise.

SuiteScript 2.1 Server Scripts and Promises

Starting in NetSuite 2021.1, SuiteScript 2.1 supports non-blocking asynchronous server-side promises expressed using the `async`, `await`, and `promise` keywords. This allows you to upload and deploy a SuiteScript 2.1 script that includes the `async`, `await`, and `promise` keywords. However, these keywords are only valid for a subset of modules:

- N/http
- N/https
- N/llm
- N/query
- N/search
- N/transaction

Note: In SuiteScript 2.1 server scripts, you will receive an error if you use the `async`, `await`, or `promise` keyword in any other module API. In other words, no other modules support the use of the `async`, `await`, and `promise` keywords in server scripts.

When using server-side promises in SuiteScript 2.1, you should consider the following:

- This capability is not for bulk processing use cases where an out-of-band solution, such as a work queue, may suffice.
- This capability is mainly for in-process and distinct operations, such as business functions on a list of transactions when ordering does not matter.

For more information about SuiteScript versions and SuiteScript 2.1, see the help topics [SuiteScript Versioning Guidelines](#) and [SuiteScript 2.1](#).

Best Practices for Asynchronous Programming with SuiteScript

When using promises in your SuiteScript scripts to implement asynchronous programming, you should consider the following best practices. If you are using SuiteScript 2.1 scripts, consider using the `async` and `await` keywords to implement asynchronous solutions instead of using the `promise` keyword.

For additional best practices for SuiteScript scripts, see the help topic [SuiteScript Best Practices](#).

Best practices for coding promises:

- Always use a promise rejection by including a `promise.catch` handler.
- Do not nest promises. Chain your promises instead. Or use `async/await` functionality.
- Keep promise chains short to avoid significant overuse of memory and CPU.
- Use `promise.finally` if you have code that should run regardless of the outcome of the promise.
- Use `promise.all` for multiple unrelated asynchronous calls.
- Do not define rejection handlers as the second argument for `promise.then` calls. Instead, use `promise.catch`.
- If you need to access results of promises running in parallel, use `promise.spread`.
- To limit concurrency, use `promise.map`.
- Use `promise.then` to run code after the promise completes.

For additional best practice guidance when coding promises, see [Best Practices for ES6 Promises](#), [Best Practices for Using Promises in JS](#), [JavaScript Promises Tips and Tricks](#), [Promises \(Mozilla\)](#), [JavaScript Best Practices for Promises](#), and [Promises](#).

Best practices when using `async` and `await` keywords:

- Rewrite promise code to use `async/await`.
- Schedule first, `await` later.
- Avoid mixing callback-based APIs with promise-based APIs.
- Refrain from using `return await`.
- Remember that for SuiteScript 2.1 server-side scripts, only a subset of modules support the use of the `async` and `await` keywords: `N/http`, `N/https`, `N/llm`, `N/query`, `N/search`, and `N/transaction`.

For additional best practice guidance when using `async/await`, see [Best Practices for ES2017 Asynchronous Functions](#), and [Making asynchronous programming easier with `async` and `await`](#).

SuiteScript 2.x Promise APIs

The available promise APIs are named so that they correspond with their synchronous counterparts. Promise APIs have names that are suffixed with `.promise`. For example, the `search.create(options)`

API has a promise version named `search.create.promise(options)`. The following promise APIs are supported for 2.x client scripts and 2.1 server scripts:

Module	Promise API	Supported in SuiteScript 2.x Client Scripts	Supported in SuiteScript 2.x Server Scripts
N/action Module	Action.promise(options)	✓	—
	Action.execute.promise(options)	✓	—
	action.execute.promise(options)	✓	—
	action.find.promise(options)	✓	—
	action.get.promise(options)	✓	—
N/currentRecord Module	currentRecord.get.promise()	✓	—
N/dataset Module	dataset.describe.promise(options)	✓	—
	dataset.loadDataset.promise(options)	✓	—
	Dataset.run.promise()	✓	—
N/documentCapture Module	documentCapture. documentToStructure. promise(options)	—	✓
	documentCapture.documentToText. promise(options)	—	✓
	documentCapture.getRemaining Concurrency.promise()	—	✓
	documentCapture. getRemainingFreeUsage.promise()	—	✓
N/email Module	email.send.promise(options)	✓	—
	email.sendBulk.promise(options)	✓	—
	email.sendCampaignEvent. promise(options)	✓	—
N/http Module	http.delete.promise(options)	✓	✓
	http.get.promise(options)	✓	✓
	http.post.promise(options)	✓	✓
	http.put.promise(options)	✓	✓
	http.request.promise(options)	✓	✓
N/https Module	https.get.promise(options)	✓	✓
	https.post.promise(options)	✓	✓
	https.put.promise(options)	✓	✓
	https.request.promise(options)	✓	✓
	https.requestSuitelet. promise(options)	✓	✓

Module	Promise API	Supported in SuiteScript 2.x Client Scripts	Supported in SuiteScript 2.x Server Scripts
N/Ilm Module	Ilm.evaluatePrompt.promise(options)	—	✓
	Ilm.generateText.promise(options)	—	✓
	Ilm.getRemainingFreeUsage.promise()	—	✓
N/machineTranslation Module	machineTranslation.translate.promise(options)	—	✓
N/query Module	Query.run.promise()	✓	✓
	Query.runPaged.promise(options)	✓	✓
	query.load.promise(options)	✓	✓
N/record Module	Macro.execute.promise(options)	✓	—
	Macro.promise(options)	✓	—
	Record.executeMacro.promise(options)	✓	—
	Record.save.promise(options)	✓	—
	record.attach.promise(options)	✓	—
	record.copy.promise(options)	✓	—
	record.create.promise(options)	✓	—
	record.delete.promise(options)	✓	—
	record.detach.promise(options)	✓	—
	record.load.promise(options)	✓	—
	record.submitFields.promise(options)	✓	—
	record.transform.promise(options)	✓	—
N/search Module	Page.next.promise()	✓	—
	Page.prev.promise()	✓	—
	PagedData.fetch.promise()	✓	—
	ResultSet.each.promise(callback)	✓	—
	ResultSet.getRange.promise(options)	✓	—
	Search.runPaged.promise(options)	✓	✓
	Search.save.promise()	✓	✓
	search.create.promise(options)	✓	✓
	search.delete.promise(options)	✓	✓

Module	Promise API	Supported in SuiteScript 2.x Client Scripts	Supported in SuiteScript 2.x Server Scripts
	search.duplicates.promise(options)	✓	✓
	search.global.promise(options)	✓	✓
	search.load.promise(options)	✓	✓
	search.lookupFields.promise(options)	✓	✓
N/transaction Module	transaction.void.promise(options)	✓	✓

For supported modules members and additional API information, see [SuiteScript 2.x Modules](#).

Examples

The following is a basic example of how to use the `search.create.promise` in a client script for the `pageInit` entry point.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  define(['N/search'], function(search)
6  {
7      function doSomething()
8      {
9          search.create.promise({
10             type: 'salesorder'
11          })
12             .then(function(result) {
13                 log.debug("Completed: " + result);
14                 //do something after completion
15             })
16             .catch(function(reason) {
17                 log.debug("Failed: " + reason)
18                 //do something on failure
19             });
20     }
21     return
22     {
23         pageInit: doSomething
24     }
25 });

```

This example demonstrates how to chain promises created with SuiteScript promise APIs.

```

1  /**
2   * @NApiVersion 2.x
3   */
4  define(['N/search'], function(search)
5  {
6      function doSomething()
7      {
8          var filter = search.createFilter({
9              name: 'mainline',
10             operator: search.Operator.IS,
11             values: ['T']
12         });
13
14         search.create.promise({
15             type: 'salesorder',

```

```

16     filters:[filter]
17   })
18   .then(function(searchObj) {
19     return searchObj.run().each.promise(
20       function(result, index){
21         //do something
22       })
23   })
24   .then(function(result) {
25     log.debug("Completed: " + result)
26     //do something after completion
27   })
28   .catch(function(reason) {
29     log.debug("Failed: " + reason)
30     //do something on failure
31   });
32 })
33 return
34 {
35   pageInit: doSomething
36 }
37
38 });

```

Custom Promises

The following example shows a custom promise. Note that custom promises do not use the SuiteScript 2.x Promise APIs.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  define(function(){
6    function doSomething(addresses){
7      var promise = new Promise(function(resolve, reject){
8        var url = 'https://your.favorite.maps/api/directions?start=' + addresses.start + '&end=' + addresses.end,
9            isAsync = true,
10            xhr = new XMLHttpRequest();
11
12        xhr.addEventListener('load', function (event) {
13          if (xhr.readyState === 4) {
14            if (xhr.status === 200) {
15              resolve(xhr.responseText );
16            }
17            else {
18              reject( xhr.statusText );
19            }
20          }
21        });
22
23        xhr.addEventListener('error', function (event) {
24          reject( xhr.statusText );
25        });
26
27        xhr.open('GET', url, isAsync);
28        xhr.send();
29      });
30
31      return promise;
32    }
33
34    return {
35      lookupDirections: doSomething
36    };
37  });

```

Iterator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In the ECMAScript specification, an iterator is an object that defines a sequence and, optionally, a return value when the sequence ends. Iterators let you process each item in a collection one at a time, and they include methods to obtain the next item in the collection to process. You can also use different methods depending on whether you want to stop iteration when a specific value or condition is reached.

Some methods in SuiteScript modules return objects that include an iterator. For example, in the N/query module, the [Query.run\(options\)](#) method runs a query definition and returns a [query.ResultSet](#) object. This object includes a method, [ResultSet.iterator\(\)](#), that returns an iterator for the query results. You can use this iterator to process each result in the query result set.

In SuiteScript, you can use iterators in two ways:

- [Using the next\(\) Method](#)
- [Using the each\(\) Method](#)

Using the next() Method

Iterators in SuiteScript are compatible with the ECMAScript iterator protocol, so you can use the standard ECMAScript `next()` method to retrieve the next item in an iteration sequence. This method returns an object with two properties:

- `value` — The next value in the iteration sequence.
- `done` — A boolean value indicating whether the last value in the iteration sequence has been processed. A value of `true` indicates that iteration is complete.

The following example shows how to create a simple query using the N/query module and use the `next()` method to iterate over the query results:

```

1  var myQuery = query.create({
2      type: query.Type.CUSTOMER
3  });
4
5  myQuery.columns = [
6      myQuery.createColumn({
7          fieldId: 'entityid'
8      })
9  ];
10
11 var results = myQuery.run();
12
13 var resultIterator = results.iterator();
14 var currentResult = resultIterator.next();
15 while (!currentResult.done) {
16     log.debug(currentResult.value);
17
18     // Do something with the current result
19     currentResult = resultIterator.next();
20 }
```

For more information about iterators in ECMAScript, see [Iterators and generators](#).

Using the each() Method

The SuiteScript implementation of iterators includes an `each()` method that you can use to iterate over the items in an iteration sequence. This method accepts a callback function with one parameter, and the

function is executed on each item in the iteration sequence. You can return true from this function to retrieve the next item and continue iteration, or you can return false to stop iteration immediately. It can be useful to stop iteration if your script encounters specific data or values and you do not need to retrieve any more items in the collection.

The following example shows how to create a simple query using the N/query module and use the each() method to iterate over the query results:

```

1  var myQuery = query.create({
2      type: query.Type.CUSTOMER
3  });
4  myQuery.columns = [
5      myQuery.createColumn({
6          fieldId: 'entityid'
7      })
8  ];
9
10 var results = myQuery.run();
11 var resultIterator = results.iterator();
12
13 resultIterator.each(function(result) {
14     var currentResult = result.value;
15     log.debug(currentResult);
16     // Do something with the current result
17     return true;
18 });

```

Iterator.next()

Method Description	Iterates over the items in the iteration sequence. This method accepts a callback function with one parameter, and the function is executed on each item in the iteration sequence. You can return true from this function to retrieve the next item and continue iteration, or you can return false to stop iteration immediately. It can be useful to stop iteration if your script encounters specific data or values and you do not need to retrieve any more items in the iteration sequence.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description
callback	function	required	A callback function to execute on each item in the iteration sequence.

Syntax

The following code sample shows the syntax for this member. It is not a functional example.

```
1 // Add additional code.  
2 ...  
3 // Create or load a query called myQuery  
4 var results = myQuery.run();  
5  
6 var resultIterator = results.iterator();  
7 resultIterator.each(function(result) {  
8     var currentResult = result.value;  
9     log.debug(currentResult);  
10    // Do something with the current result  
11  
12    return true;  
13 });
```

SuiteScript 2.x Modules

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x APIs are organized into various modules, based on behavior. The following table lists each module and provides a description, the supported script types, and permissions associated with the module. See the help topic [Setting Roles and Permissions for SuiteScript](#) for more information about permissions.

Note: As a best practice, you should load only the modules that are needed by your script.

Module	Description	Supported Script Types*	Permissions
N/action Module	Load the N/action module to execute business logic to update the state of a record. Action APIs emulate NetSuite user interface buttons.	Client and server scripts	—
N/auth Module	Load the N/auth module to change your NetSuite login credentials.	Server scripts	—
N/cache Module	Load the N/cache module to enable the caching of needed data and improve performance.	Server scripts	—
N/certificateControl Module	Load the N/certificateControl module to enable scripting access to the Digital Certificates list found at Setup > Company > Certificates. You can use this module to find the correct certificate for a subsidiary and check the file type. For more information, see the help topics Digital Signing and Uploading Digital Certificates .	Server scripts	Certificate Management
N/commerce Modules	Load modules in the N/commerce namespace to access different assets in the web store context, such as items and shopping cart. The modules within the N/commerce namespace are supported by the latest version of SuiteCommerce and by SuiteCommerce Advanced 2019.2 onwards.	Client and server scripts	—
N/compress Module	Load the N/compress module to compress, decompress, and archive files.	Server scripts	—
N/config Module	Load the N/config module to access NetSuite configuration settings. See config.Type for a	Server scripts	—

Module	Description	Supported Script Types*	Permissions
	list of supported configuration pages.		
N/crypto Module	Load the N/crypto module to work with hashing, hash-based message authentication (hmac), and symmetrical encryption. You can access a set of wrappers for OpenSSL's hash, hmac, cipher, and decipher methods.	Server scripts	—
N/crypto/certificate Module	Load the N/certificate module to sign XML documents or strings with digital certificates using asymmetric cryptography. In addition to signing XML documents, you can create signer and verifier objects and verify signed documents with this module.	Server scripts	Certificate Access
N/crypto/random Module	Load the N/crypto/random module to work with cryptographically-secure pseudo-random generator methods.	Client and server scripts; server scripts support SuiteScript 2.1 only	-
N/currency Module	Load the N/currency module to work with exchange rates within your NetSuite account. You can use this module to find the exchange rate between two currencies based on a certain date.	Client and server scripts	—
N/currentRecord Module	Load the N/currentRecord module to access the record instance that you are currently working on. You can then use the record instance in a client context.	Client scripts	—
N/dataset Module	Load the N/dataset module to create and manage datasets in SuiteAnalytics Workbook. Use this module with the N/workbook module to manage all aspects of your datasets and workbooks in SuiteAnalytics Workbook.	Server scripts	SuiteAnalytics Workbook
N/documentCapture Module	Load the N/documentCapture module to extract text content from supported documents. This module lets you programmatically extract structured content and key information from a variety of document types (such as	Server scripts SuiteScript 2.1 only	—

Module	Description	Supported Script Types*	Permissions
	invoices, receipts, contracts, and so on).		
N/email Module	Load the N/email module to send email messages from within NetSuite. You can use the email module to send regular, bulk, and campaign email.	Client and server scripts	—
N/encode Module	Load the N/encode module to convert a string to another type of encoding. See encode.Encoding for a list of supported character set encoding.	Server scripts	—
N/error Module	Load the N/error module to create your own custom SuiteScript errors. Use these custom errors in try-catch statements to abort script execution.	Server scripts	—
N/file Module	Load the N/file module to work with files in NetSuite.	Server scripts	—
N/format Module	Load the N/format module to convert strings into a specified format and to parse formatted data into strings.	Client and server scripts	—
N/format/i18n Module	Load the N/format/i18n module to format currency.	Client and server scripts	—
N/http Module	Load the N/http module to make http calls.	Client and server scripts	—
N/https Module	Load the N/https module to make https calls. You can also use this module to encode binary content or securely access a handle to the value in a NetSuite credential field.	Client and server scripts	—
N/https/clientCertificate Module	Load the N/clientCertificate module to send SSL requests with a digital certificate.	Server scripts	Certificate Access
N/keyControl Module	Load the N/keyControl module to access key storage.	Server scripts	Key Management
N/llm Module	Load the N/llm module to use generative artificial intelligence (AI) capabilities. You can use this module to send requests to the large language models (LLMs) supported by NetSuite and to receive LLM responses to use in your scripts.	Server scripts SuiteScript 2.1 only	—

Module	Description	Supported Script Types*	Permissions
N/log Module	Load the N/log module to access methods for logging script execution details. Module members are also supported by the global log Object .	Client and server scripts Limitations apply to client scripts	—
N/machineTranslation Module	Load the N/machineTranslation module to translate text into supported languages using generative AI.	Server scripts SuiteScript 2.1 only	—
N/pgp Module	Load the N/pgp module to send secure messages to one or multiple receivers.	Server scripts SuiteScript 2.1 only	—
N/piremoval Module	Load the N/piremoval module to remove personal information (PI) from system notes, workflow history, and specific field values.	Server scripts	Remove Personal Information Create Remove Personal Information Run
N/plugin Module	Load the N/plugin module to load custom plug-in implementations.	Server scripts	—
N/portlet Module	Load the N/portlet module to resize or refresh a form portlet.	Client scripts	—
N/query Module	Load the N/query module to create and run searches using the SuiteAnalytics Workbook query process.	Client and server scripts	SuiteAnalytics Workbook
N/record Module	Load the N/record module to work with NetSuite records.	Client and server scripts	—
N/recordContext Module	Load the N/recordContext module to get the context type of a record.	Client and server scripts	—
N/redirect Module	Load the N/redirect module to redirect users to a URL, a Suitelet, a record, a task link, a saved search, or an unsaved search.	Server scripts	—
N/render Module	Load the N/render module to create forms or email from templates and to print to PDF or HTML.	Server scripts	Advanced PDF/HTML Templates Custom PDF Layouts
N/runtime Module	Load the N/runtime module to access runtime settings for company, script, session, system, user, or version.	Client and server scripts	
N/scriptTypes/restlet Module	Load the N/scriptTypes/restlet module to create custom handling for your RESTlet script.	RESTlet scripts	—

Module	Description	Supported Script Types*	Permissions
N/search Module	Load the N/search module to create and run on-demand or saved searches and analyze and iterate through the search results. You can search for a single record by keywords, create saved searches, search for duplicate records, or return a set of records that match filters you define.	Client and server scripts	Perform Search Persist Search Publish Search
N/sftp Module	Load the N/sftp module to connect to a remote FTP server using SFTP and transfer files.	Server scripts	Key Access (needed when public key authentication is used)
N/sso Module	 Note: As of NetSuite 2025.1, the support ended for the SuiteSignOn feature, which means the N/sso module is also no longer supported. Load the N/sso module to generate outbound single sign-on (SuiteSignOn) tokens.	Client and server scripts	—
N/suiteAppInfo Module	Load the N/suiteAppInfo module when you want to access information related to SuiteApps and Bundles.	Client and server scripts	—
N/task Module	Load the N/task module to create tasks and place them in the internal NetSuite scheduling or task queue. Use this module to schedule scripts, run Map/Reduce scripts, import CSV files, merge duplicate records, and execute asynchronous workflows.	Server scripts	Tasks
N/task/accounting/recognition Module	Load the task/accounting/recognition module to merge revenue arrangements or revenue elements. This module lets you combine revenue arrangements or revenue elements from multiple sources to represent a single contract for revenue allocation and recognition.	Server scripts	(Transactions) Revenue Arrangement
N/transaction Module	Load the N/transaction module to void transactions.	Client and server scripts	—

Module	Description	Supported Script Types*	Permissions
N/translation Module	Load the N/translation module to load NetSuite Translation Collections in SuiteScript.	Client and server scripts	—
N/ui/dialog Module	Load the N/dialog module to create a modal dialog that is displayed until a button on the dialog is pressed.	Client scripts	—
N/ui/message Module	Load the N/message module to display a message at the top of the screen under the menu bar.	Client scripts	—
N/ui/serverWidget Module	Load the N/serverWidget module to work with the user interface within NetSuite.	Server scripts	—
N/url Module	Load the N/url module to determine URL navigation paths within NetSuite or format URL strings.	Client and server scripts	—
N/util Module	Load the N/util module to manually access util methods. Module members are also supported by the global util Object .	Client and server scripts	—
N/workbook Module	Load the N/workbook module to create and manage workbooks in SuiteAnalytics Workbook. Use this module with the N/dataset module to manage all aspects of your datasets and workbooks in SuiteAnalytics Workbook.	Server scripts	SuiteAnalytics Workbook
N/workflow Module	Load the N/workflow module to initiate new workflow instances or trigger existing workflow instances.	Server scripts	Workflow
N/xml Module	Load the N/xml module to validate, parse, read, and modify XML documents.	Client and server scripts	—
* If you include a module in a script that is not a supported script type, an execution error will occur. This error may be UNEXPECTED_ERROR or INSUFFICIENT_PERMISSION or another error that indicates the method in the module could not be executed.		—	

N/action Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/action module to execute business logic to update the state of records in view mode. To execute business logic on records in edit mode, use the record macro APIs, which are included in the [N/](#)

[record Module](#) module. See [Record Object Members](#) and [Macro Object Members](#). Action and Macro APIs are the programmatic equivalent to clicking a button in the UI. To learn more, see the help topic [Overview of Record Action and Macro APIs](#).

Go To Samples 

The changes that you make to records with N/action module APIs are persisted in the database immediately. For example, consider the timebill record. After you click the **Approve** button in the UI, the timebill and its entries are saved in an approved state, and this change is immediately updated in the database.

Governance for action module APIs varies for actions and record types. See the action help for governance information specific to actions and record types.

A limited number of individual actions for specific record types are supported. For details, see the help topic [Supported Record Actions](#).

 **Note:** For supported script types, see individual member topics listed below.

In This Help Topic

- [N/action Module Members](#)
- [Action Object Members](#)

N/action Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	action.Action	Object	Client and server scripts	Encapsulates a NetSuite record action.
	Plain JavaScript Object	Object	Client and server scripts	A plain JavaScript object of actions available for a record type.
Method	action.execute(options)	Object	Client and server scripts	Executes the record action and returns action results in an object.
	action.execute.promise(options)	Promise	Client scripts	Asynchronously executes the record action and returns the action results in an object.
	action.executeBulk(options)	string	Client and server scripts	Executes an asynchronous bulk record action and returns its task ID for later status inquiry.
	action.find(options)	Object	Client and server scripts	Returns a plain JavaScript object of available record actions for the given record type.
	action.find.promise(options)	Promise	Client scripts	Asynchronously returns a plain JavaScript object of available

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				record actions for the given record type.
	action.get(options)	action.Action	Client and server scripts	Returns an executable record action for the given record type.
	action.get.promise(options)	Promise	Client scripts	Asynchronously returns an executable record action for the given record type.

Action Object Members

The following members are called on [action.Action](#).

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
Method	Action(options)	Object	Client and server scripts	Executes the action and returns the action results in an object.
	Action.promise(options)	Promise	Client scripts	Executes the action asynchronously and returns the action results in an object.
	Action.execute(options)	Object	Client and server scripts	Executes the action and returns the action results in an object.
	Action.execute.promise(options)	Promise	Client scripts	Executes the action asynchronously and returns the action results in an object.
	Action.executeBulk(options)	string	Client and server scripts	Executes an asynchronous bulk record action and returns its task ID for later status inquiry.
	action.getBulkStatus(options)	Object	Client and server scripts	Returns the current status of action.executeBulk(options) with the given task ID.
Property	Action.description	string	Client and server scripts	The action description.
	Action.id	string	Client and server scripts	The ID of the action. For a list of action IDs, see the help topic Supported Record Actions .
	Action.label	string	Client and server scripts	The action label.
	Action.parameters	Object	Client and server scripts	The action parameters.
	Action.recordType	string	Client and server scripts	The type of the record on which the action is to be performed. For a list of record types, see record.Type .

N/action Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/action module:



Important: The samples included in this section are intended to show how actions work in SuiteScript at a high-level. For specific samples, see the help topic [Supported Record Actions](#).

- [Locate and Execute an Action on a Timebill Record](#)
- [Find Actions Available for the Timebill Record Asynchronously Using Promise Methods](#)
- [Execute a Bulk Action on a Timebill Record](#)

Locate and Execute an Action on a Timebill Record

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample finds and executes an action on the timebill record without promises.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/action', 'N/record'], function(action, record) {
5       // create timebill record
6       var rec = record.create({
7           type: 'timebill',
8           isDynamic: true
9       });
10      rec.setValue({
11          fieldId: 'employee',
12          value: 104
13      });
14      rec.setValue({
15          fieldId: 'location',
16          value: 312
17      });
18      rec.setValue({
19          fieldId: 'hours',
20          value: 5
21      });
22      var recordId = rec.save();
23
24      var actions = action.find({
25          recordType: 'timebill',
26          recordId: recordId
27      });
28
29
30      log.debug("We've got the following actions: " + Object.keys(actions));
31      if (actions.approve) {
32          var result = actions.approve();
33          log.debug("Timebill has been successfully approved");
34      } else {

```



```

35     log.debug("The timebill is already approved");
36   }
37 });
38
39 // Outputs the following:
40 // We've got the following actions: approve, reject
41 // Timebill has been successfully approved

```

Find Actions Available for the Timebill Record Asynchronously Using Promise Methods

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample asynchronously finds actions available for a timebill record and then executes one with promises.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ClientScript
4   */
5  require(['N/action', 'N/record'], function(action, record) {
6     // create timebill record
7     var rec = record.create({
8       type: 'timebill',
9       isDynamic: true
10    });
11    rec.setValue({
12      fieldId: 'employee',
13      value: 104
14    });
15    rec.setValue({
16      fieldId: 'location',
17      value: 312
18    });
19    rec.setValue({
20      fieldId: 'hours',
21      value: 5
22    });
23    var recordId = rec.save();
24
25    // find all qualified actions and then execute approve if available
26    action.find.promise({
27      recordType: 'timebill',
28      recordId: recordId
29    }).then(function(actions) {
30      console.log("We've got the following actions: " + Object.keys(actions));
31      if (actions.approve) {
32        actions.approve.promise().then(function(result) {
33          console.log("Timebill has been successfully approved");
34        });
35      } else {
36        console.log("The timebill is already approved");
37      }
38    });
39  });
40
41 // Outputs the following:
42 // We've got the following actions:
43 // The timebill has been successfully approved

```

Execute a Bulk Action on a Timebill Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to execute a bulk approve action on a timebill record using different parameters.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: The following code samples do not serve as an order of execution, `getBulkStatus` should be executed later on and not right after the execution of `action.executeBulk()`.

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/action', 'N/util'])function(action, util) {
5
6       // 1a) Bulk execute the specified action on a provided list of record IDs.
7       // The params property is an array of parameter objects where each object contains required recordId and arbitrary additional parameters.
8       var handle = action.executeBulk({
9         recordType: "timebill",
10        id: "approve",
11        params: [{
12          recordId: 1,
13          note: "this is a note for 1"
14        },
15        {
16          recordId: 5,
17          note: "this is a note for 5"
18        },
19        {
20          recordId: 23,
21          note: "this is a note for 23"
22        }
23      ]
24    });
25  };
26
27  // 1b) Bulk execute the specified action on a provided list of record IDs.
28  // The parameters in the previous sample are similar and can be generated programmatically using the map function.
29  var searchResults = /* result of a search, for example, [1, 5, 23] */ ;
30  var handle = action.executeBulk({
31    recordType: "timebill",
32    id: "approve",
33    params: searchResults.map(function(v) {
34      return {
35        recordId: v,
36        note: "this is a note for " + v
37      };
38    })
39  });
40
41  // 2a) Bulk execute the specified action on a provided list of record IDs.
42  // This time with homogenous parameters, that is, all parameter objects are equal except recordId.
43  var handle = action.executeBulk({
44    recordType: "timebill",
45    id: "approve",
46    params: searchResults.map(function(v) {
47      return {
48        recordId: v,
49        foo: "bar",
50        name: "John Doe"
51      };
52    })
53  });

```

```

53 });
54
55 // 2b) Bulk execute the specified action on a provided list of record IDs.
56 // This time with homogenous parameters. Equivalent to the previous sample.
57 var commonParams = {
58     foo: "bar",
59     name: "John Doe"
60 };
61 var handle = action.executeBulk({
62     recordType: "timebill",
63     id: "approve",
64     params: searchResults.map(function(v) {
65         return util.extend({
66             recordId: v
67         }, commonParams);
68     })
69 });
70
71 // 3) Bulk execute the specified action on a provided list of record IDs.
72 // This is the simplest usage with no extra parameters besides the record ID.
73 var handle = action.executeBulk({
74     recordType: "timebill",
75     id: "approve",
76     params: searchResults.map(function(v) {
77         return {
78             recordId: v
79         }
80     })
81 });
82
83 // 4) Bulk execute the specified action on all record instances that qualify.
84 // Since we don't have a list of recordIds in hand, we only provide the callback
85 // that will later be used to transform a recordId to the corresponding parameters object.
86 var handle = action.executeBulk({
87     recordType: "timebill",
88     id: "approve",
89     condition: action.ALL_QUALIFIED_INSTANCES,
90     paramCallback: function(v) {
91         return {
92             recordId: v,
93             note: "this is a note for " + v
94         };
95     }
96 });
97
98 // 5) Get a particular action for a particular record type.
99 var approveTimebill = action.get({
100     recordType: "timebill",
101     id: "approve"
102 });
103
104 // 6) Bulk execute the previously obtained action on a provided list of record IDs.
105 // Params are generated the same way as above in action.executeBulk().
106 var handle = approveTimebill.executeBulk({
107     params: searchResults.map(function(v) {
108         return {
109             recordId: v,
110             note: "this is a note for " + v
111         };
112     })
113 });
114
115 // 7) Bulk execute the previously obtained action on all record instances that qualify.
116 var handle = approveTimebill.executeBulk({
117     condition: action.ALL_QUALIFIED_INSTANCES,
118     paramCallback: function(v) {
119         return {
120             recordId: v,
121             note: "this is a note for " + v
122         };
123     }
124 });
125

```

```

126 // 8) Get status of a bulk action execution.
127 var res = action.getBulkStatus({
128     taskId: handle
129 }); // returns a RecordActionTaskStatus object
130 log.debug(res.status);
131 });

```

action.Action

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a NetSuite record action. This object is returned by the action.get(options) and action.find(options) methods.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/action Module
Methods and Properties	Action Object Members
Since	2018.2

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myAction = action.get({
4     recordType: 'timebill',
5     id: 'approve'
6 });
7 ...
8 // Add additional code

```

Action(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the action and returns the action result in a plain JavaScript object. The action result is returned in an object. The response property of the results object shows the action result. If the action fails, it is listed in the results object's notifications property. If the action executes successfully, the notifications property is usually empty. If the Action object is qualified (it is a result of an action.get() or action.find() call that provides the recordId), then it is not required to provide a recordId and the options.params.recordId parameter is optional. If options.params.recordId is provided during execution, it takes precedence over the recordId stored in the Action object.
---------------------------	--

	 Note: Replace Action with the name of the action you are executing.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/action Module
Parent Object	action.Action
Sibling Object Members	Action Object Members
Since	2018.2

Parameters

 **Note:** The parameters that are required vary for action types. The only parameter that is always required is `options.recordid`, unless the action object is qualified. An action object is qualified if it is the result of an `action.get()` or `action.find()` call that provides the `recordId`.

Parameter	Type	Required / Optional	Description
<code>options.Object</code>	Object	required or optional	The parameters that need to be provided depend on the action implementation. See the action help.
<code>options.params.recordId</code>	string	required or optional	The record instance ID of the record on which the action is to be performed. This is the NetSuite record internal ID.

Errors

Error Code	Thrown If
<code>SSS_MISSING_REQD_ARGUMENT</code>	A required parameter is missing.

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myActionResult = action({
4     recordId: 1
5 });
6 ...

```

```
7 // Add additional code
```

Action.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the action asynchronously and returns the action result in a plain JavaScript object. The action result is returned in an object. The response property of the results object shows the action result. If the action fails, it is listed in the results object's notifications property. If the action executes successfully, the notifications property is usually empty. If the Action object is qualified (it is a result of an action.get() or action.find() call that provides the recordId), then it is not required to provide a recordId and the options.params.recordId parameter is optional. If options.params.recordId is provided during execution, it takes precedence over the recordId stored in the Action object. Note: Replace Action with the name of the action you are executing. Note: The parameters and errors thrown for this method are the same as those for Action(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Action(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/action Module
Parent Object	action.Action
Sibling Object Members	Action Object Members
Since	2018.2

Parameters

Note: The parameters that are required vary for action types. The only parameter that is always required is options.recordid, unless the action object is qualified. An action object is qualified if it is the result of an action.get() or action.find() call that provides the recordId.

Parameter	Type	Required / Optional	Description
options.Object	Object	required or optional	The parameters that need to be provided depend on the action implementation. See the action help.
options.params.recordId	string	required or optional	The record instance ID of the record on which the action is to be performed.

Parameter	Type	Required / Optional	Description
			This is the NetSuite record internal ID.

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 action.promise({
4   recordId: 1
5 }).then(function(result) {
6   // Process the result here
7 });
8 ...
9 // Add additional code

```

Action.execute(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the action and returns the action result in a object. The response property of the result object holds the response returned by the action implementation. The notifications property of the result object is an array of notification objects. It contains the details of errors and warnings that occurred during action execution. If the action executes successfully, the notifications property is usually empty. If the Action object is qualified (it is a result of an action.get() or action.find() call that provides the recordId), then it is not required to provide a recordId and the options.params.recordId parameter is optional. If options.params.recordId is provided during execution, it takes precedence over the recordId stored in the Action object.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/action Module
Parent Object	action.Action
Sibling Object Members	Action Object Members

Since	2018.2
-------	--------

Parameters

Note: The parameters that are required vary for action types. The only parameter that is always required is `options.recordId`, unless the action object is qualified. An action object is qualified if it is the result of an `action.get()` or `action.find()` call that provides the `recordId`.

Parameter	Type	Required / Optional	Description
<code>options.Object</code>	Object	required or optional	The parameters that need to be provided depend on the action implementation. See the action help.
<code>options.params.recordId</code>	int	required or optional	The record instance ID of the record on which the action is to be performed. This is the NetSuite record internal ID.

Errors

Error Code	Thrown If
<code>SSS_MISSING_REQD_ARGUMENT</code>	A required parameter is missing.

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myResult = action.execute({
4     recordId: 1
5 });
6 ...
7 // Add additional code

```

Action.execute.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the action asynchronously and returns the action result in a plain JavaScript object. The action result is returned in an object. The <code>response</code> property of the results object shows the action result. If the action fails, it is listed in the results object's <code>notifications</code> property. If the action executes successfully, the <code>notifications</code> property is usually empty. If the Action object is qualified (it is a result of an <code>action.get()</code> or <code>action.find()</code> call that provides the <code>recordId</code>), then it is not required to provide a <code>recordId</code> and the <code>options.params.recordId</code> parameter is optional. If <code>options.params.recordId</code> is provided during execution, it takes precedence over the <code>recordId</code> stored in the Action object.
---------------------------	---

	 Note: The parameters and errors thrown for this method are the same as those for Action.execute(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	Action.execute(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/action Module
Parent Object	action.Action
Sibling Object Members	Action Object Members
Since	2018.2

Parameters

 **Note:** The parameters that are required vary for action types. The only parameter that is always required is `options.recordId`, unless the action object is qualified. An action object is qualified if it is the result of an `action.get()` or `action.find()` call that provides the `recordId`.

Parameter	Type	Required / Optional	Description
<code>options.Object</code>	Object	required or optional	The parameters that need to be provided depend on the action implementation. See the action help.
<code>options.params.recordId</code>	string	required or optional	The record instance ID of the record on which the action is to be performed. This is the NetSuite record internal ID.

Errors

Error Code	Thrown If
<code>SSS_MISSING_REQD_ARGUMENT</code>	A required parameter is missing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 action.execute.promise({
4   recordId: 1

```

```

5  }).then(function(result) {
6      // Process result here
7  });
8  ...
9  // Add additional code

```

Action.executeBulk(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes an asynchronous bulk record action and returns its task ID for status queries with action.getBulkStatus(options) .
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	50 units
Module	N/action Module
Parent Object	action.Action
Sibling Object Members	Action Object Members
Since	2019.1

Parameters

i Note: The `options.params` array consists of parameter objects. The values that are required in each parameter object vary for action types. The only value that is always required is `options.recordId`, unless the action object is qualified. An action object is qualified if it is the result of an `action.get()` or `action.find()` call that provides the `recordId`.

Parameter	Type	Required / Optional	Description
<code>options.params</code>	array	required	The <code>options.params</code> parameter is mutually exclusive to <code>options.condition</code> and <code>options.paramCallback</code>. An array of parameter objects. Each object corresponds to one record ID of the record for which the action is to be executed. The object has the following form: <pre>1 {recordId: 1, someParam: 'example1', otherParam: 'example2'}</pre>
<code>options.condition</code>	string	optional	The condition used to select record IDs of records for which the action is to be executed. Only the <code>action.ALL_QUALIFIED_INSTANCES</code> constant is currently supported. The <code>action.ALL_QUALIFIED_INSTANCES</code> condition only works correctly if the author of the record action has implemented the <code>findInstances</code> method of the <code>RecordActionQualifier</code> interface. An example of such action is approve on the timebill and timesheet records.

Parameter	Type	Required / Optional	Description
options.paramCallback	string	optional	the name of the function that takes a record ID and returns the parameter object for the specified record ID.

Errors

Error Code	Thrown If
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var actionObj = action.get({
4     recordType: 'timebill',
5     id: 'approve'
6 });
7
8 var handle = actionObj.executeBulk({
9     params: [
10         {
11             recordId: 1,
12             note: 'this is a note for 1'
13         },
14         {
15             recordId: 5,
16             note: 'this is a note for 5'
17         },
18         {
19             recordId: 23,
20             note: 'this is a note for 23'
21         }
22     ]
23 });
24 ...
25 // Add additional code

```

Action.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The action description.
----------------------	-------------------------

Type	string
Module	N/action Module
Sibling Object Members	Action Object Members
Since	2018.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var actionObj = action.get({
4     recordType: 'timebill',
5     id: 'approve'
6 });
7
8 // Get the action description
9 var theDescription = actionObj.description;
10 ...
11 // Add additional code

```

Action.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the action. For a list of action IDs, see the help topic Supported Record Actions .
Type	string
Module	N/action Module
Sibling Object Members	Action Object Members
Since	2018.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var actionObj = action.get({
4     recordType: 'timebill',
5     id: 'approve'
6 });
7
8 // Get the action ID
9 var theID = actionObj.id;
10 ...

```

```
11 | // Add additional code
```

Action.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The action label.
Type	string
Module	N/action Module
Sibling Object Members	Action Object Members
Since	2018.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```
1 | // Add additional code
2 | ...
3 | var actionObj = action.get({
4 |     recordType: 'timebill',
5 |     id: 'approve'
6 | });
7 |
8 | // Get the action label
9 | var theLabel = actionObj.label;
10 | ...
11 | // Add additional code
```

Action.parameters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The action parameters.
Type	Object
Module	N/action Module
Sibling Object Members	Action Object Members
Since	2018.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var actionObj = action.get({
4      recordType: 'timebill',
5      id: 'approve'
6  });
7
8  // Get the action parameters
9  var theParameters = actionObj.parameters;
10 ...
11 // Add additional code

```

Action.recordType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of the record on which the action is to be performed. For a list of record types, see record.Type .
Type	string
Module	N/action Module
Sibling Object Members	Action Object Members
Since	2018.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var actionObj = action.get({
4      recordType: 'timebill',
5      id: 'approve'
6  });
7
8  // Get the action record type
9  var theRecordType = actionObj.recordType;
10 ...
11 // Add additional code

```

action.execute(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the record action and returns the action results in a plain JavaScript object. If the action fails, it is listed in the results object's notifications property. If the action executes successfully, the notifications property is usually empty.
Returns	Object
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.recordType	string	required	The record type. For a list of record types, see record.Type .
options.id	string	required	The action ID. For a list of action IDs, see the help topic Supported Record Actions .
options.params	Object	required	Action arguments.
options.params.recordId	int	required	The record instance ID. This is the NetSuite record internal ID.

Errors

Error Code	Thrown If
RECORD_DOES_NOT_EXIST	The specified record instance does not exist.
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#) and [Revenue Arrangement Record Actions](#).

```
1 | // Add additional code
```

```

2  ...
3  var myResult = action.execute({
4      id: 'note',
5      recordType: 'timebill',
6      params: {
7          recordId: 1
8      }
9  });
10 ...
11 // Add additional code

```

action.execute.promise(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the record action asynchronously. If the action fails, it is listed in the results object's notifications property. If the action executes successfully, the notifications property is usually empty. i Note: The parameters and errors thrown for this method are the same as those for action.execute(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	action.execute(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2018.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.recordType	string	required	The record type. For a list of record types, see record.Type.
options.id	string	required	The action ID. For a list of action IDs, see the help topic Supported Record Actions.
options.params	Object	required	Action arguments.

Parameter	Type	Required / Optional	Description
options.params.recordId	string	required	The record instance ID. This is the NetSuite record internal ID.

Errors

Error Code	Thrown If
RECORD_DOES_NOT_EXIST	The specified record instance does not exist.
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 action.execute.promise({
4   id: 'note',
5   recordType: 'timebill',
6   params: {
7     recordId: 1
8   }
9 }).then(function(result) {
10   // Process the result
11 });
12 ...
13 // Add additional code

```

action.executeBulk(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes an asynchronous bulk record action and returns its task ID for status queries with action.getBulkStatus(options) . The options.params parameter is mutually exclusive to options.condition and options.paramCallback.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	50 units
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object. The options.params array consists of parameter objects. The values that are required in each parameter object vary for action types. The only value that is always required is recordId.

Parameter	Type	Required / Optional	Description
options.recordType	string	required	The record type. For a list of record types, see record.Type .
options.id	string	required	The action ID.
options.params	array	required	An array of parameter objects. Each object corresponds to one record ID of the record for which the action is to be executed. The object has the following form: <pre>1 {recordId: 1, someParam: 'example1', otherParam: 'example2'}</pre> The recordId parameter is always required, other parameters are optional and are specific to the particular action.
options.condition	string	optional	The condition used to select record IDs of records for which the action is to be executed. Only the action.ALL_QUALIFIED_INSTANCES constant is currently supported. The action.ALL_QUALIFIED_INSTANCES condition only works correctly if the author of the record action has implemented the findInstances method of the RecordActionQualifier interface. An example of such action is approve on the timebill and timesheet records.
options.paramCallback	string	optional	Function that takes record ID and returns the parameter object for the specified record ID.

Errors

Error Code	Thrown If
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.recordType parameter is missing or undefined.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var handle = action.executeBulk({
4     recordType: 'timebill',
5     id: 'approve',
6     params: [
7         {
8             recordId: 1,
9             note: 'this is a note for 1'
10        }, {
11            recordId: 5,
12            note: 'this is a note for 5'
13        }, {
14            recordId: 23,
15            note: 'this is a note for 23'
16        }
17    ]
18 });
19 ...
20 // Add additional code

```

action.find(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a search for available record actions. If only the recordType parameter is specified, all actions available for the record type are returned. If the recordId parameter is also specified, then only actions that qualify for execution on the given record instance are returned. If the id parameter is specified, then only the action with the specified action ID is returned. This method returns a plain JavaScript object of NetSuite record actions available for the record type. The object contains one or more action.Action objects. If there are no available actions for the specified record type, an empty object is returned. If the recordId is specified in this call, the actions that are found are considered qualified. You do not have to provide the recordId to execute a qualified action.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members

Since	2018.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.recordType	string	required	The record type. For a list of record types, see record.Type .
options.recordId	string	optional	The record instance ID.
options.id	string	optional	The action ID.

Errors

Error Code	Thrown If
RECORD_DOES_NOT_EXIST	The specified record ID does not exist.
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.
SSS_MISSING_REQD_ARGUMENT	The options.recordType parameter is missing or undefined.

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var actions = action.find({
4     recordType: 'timebill',
5     recordId: recordId
6 });
7 ...
8 // Add additional code

```

action.find.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a search for available record actions asynchronously. If only the recordType parameter is specified, all actions available for the record type are returned. If the recordId parameter is also specified, then only actions that qualify for execution on the given record
---------------------------	---

	instance are returned. If the <code>id</code> parameter is specified, the only the action with the specified action ID is returned. This method returns a plain JavaScript object of NetSuite record actions available for the record type. The object contains one or more action.Action objects. If there are no available actions for the specified record type, an empty object is returned. If the <code>recordId</code> is specified in this call, the actions that are found are considered qualified. You do not have to provide the <code>recordId</code> to execute a qualified action.  Note: The parameters and errors thrown for this method are the same as those for action.find(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	action.find(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2018.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
<code>options.recordType</code>	string	required	The record type. For a list of record types, see record.Type.
<code>options.recordId</code>	string	optional	The record instance ID.
<code>options.id</code>	string	optional	The action ID.

Errors

Error Code	Thrown If
<code>RECORD_DOES_NOT_EXIST</code>	The specified record ID does not exist.
<code>SSS_INVALID_ACTION_ID</code>	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
<code>SSS_INVALID_RECORD_TYPE</code>	The specified record type is invalid.

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.recordType parameter is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var promise = action.find.promise({
4     recordType: 'timebill'
5 });
6
7 promise.then(function(actionList) {
8     // Process the list of actions
9 });
10 ...
11 // Add additional code

```

action.get(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an executable record action for the specified record type. If the recordId parameter is specified, the action object is returned only if the specified action can be executed on the specified record instance.
Returns	action.Action
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2018.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.recordType	string	required	The record type. For a list of record types, see record.Type .
options.recordId	string	optional	The record instance ID.

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the action. For a list of action IDs, see the help topic Supported Record Actions .

Errors

Error Code	Thrown If
RECORD_DOES_NOT_EXIST	The specified record instance does not exist.
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myAction = action.get({
4     recordType: 'timebill',
5     id: 'approve'
6 });
7 ...
8 // Add additional code

```

action.get.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an executable record action for the specified record type asynchronously. If the recordId parameter is specified, the action object is returned only if the specified action can be executed on the specified record instance. Note: The parameters and errors thrown for this method are the same as those for action.get(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	action.get(options)
Supported Script Types	Client scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.recordType	string	required	The record type. For a list of record types, see record.Type .
options.recordId	string	optional	The record instance ID.
options.id	string	required	The ID of the action. For a list of action IDs, see the help topic Supported Record Actions .

Errors

Error Code	Thrown If
RECORD_DOES_NOT_EXIST	The specified record instance does not exist.
SSS_INVALID_ACTION_ID	The specified action does not exist on the specified record type. – or – The action exists, but cannot be executed on the specified record instance.
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 action.get.promise({
4   recordType: 'timebill',
5   id: 'approve'
6 }).then(function(action) {
7   // Process the action object

```



```

8 | });
9 | ...
10 | // Add additional code

```

action.getBulkStatus(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the current status of action.executeBulk(options) for the specified task ID. The bulk execution status is returned in a status object.
Returns	RecordActionTaskStatus Object Members
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/action Module
Sibling Object Members	N/action Module Members
Since	2019.1

Parameters

Parameter	Type	Required / Optional	Description
options.taskId	string	required	The task ID returned by a previous action.executeBulk(options) call.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/action Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | // Obtain the status as a RecordActionTaskStatus object
4 | var res = action.getBulkStatus({
5 |     taskId: handle
6 | });
7 | ...
8 | // Add additional code

```

N/auth Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/auth module to change your NetSuite login credentials.

[Go To Samples {..}](#)

In This Help Topic

■ [N/auth Module Members](#)

N/auth Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	auth.changeEmail (options)	void	Server scripts	Changes the current user's NetSuite email address (user name).
	auth.changePassword (options)	void	Server scripts	Changes the current user's NetSuite password.

N/auth Module Script Sample

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/auth module.

Change a NetSuite Email Address and Password

The following sample shows how to change the currently logged-in user's NetSuite email address and password.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

x Warning: When you run this sample code in the SuiteScript Debugger, it logs a real request to change the email address and then changes the password.

! Important: The values used in this sample for the password and email fields are placeholders. Before using this sample, replace the password and email field values with valid values from your NetSuite account. If you run a script with an invalid value, an error may occur.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  // This script changes the currently logged-in user's NetSuite email address and password.
6  require(['N/auth'], function(auth) {
7      function changeEmailAndPassword() {

```

```

8      var password = 'myCurrentPassword';
9      auth.changeEmail({
10         password: password,
11         newEmail: 'auth_test@newemail.com'
12     });
13     auth.changePassword({
14         currentPassword: password,
15         newPassword: 'myNewPa55Word'
16     });
17 }
18 changeEmailAndPassword();
19 });

```

auth.changeEmail(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Changes the current user's NetSuite email address (user name).
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/auth Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.newEmail	string	required	The logged in user's NetSuite new email address.
options.password	string	required	The logged in user's current NetSuite password.
options.onlyThisAccount	boolean	optional	Determines which accounts the email address is changed for. If set to true, the email address change is applied only to roles within the current account. If set to false, the email address change is applied to all accounts and roles. The default value is true.

Errors

Error Code	Thrown If
INVALID_EMAIL	The options.newEmail is invalid.

Error Code	Thrown If
INVALID_PSWD	The options.password does not conform to the password rules. See the help topic Creating a Strong Password for information about valid passwords.
SSS_MISSING_REQD_PARAMETER	The options.newEmail or options.password parameter is not specified.
WRONG_PARAMETER_TYPE	The options.onlyThisAccount is not specified as a boolean value.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/auth Module Script Sample](#).

```

1 //Add additional code
2 ...
3 auth.changeEmail({
4     password: 'mycurrentPWD',
5     newEmail: 'jwolf@netsuite.com',
6     onlyThisAccount: true
7 });
8 ...
9 //Add additional code

```

auth.changePassword(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Changes the current user's NetSuite password. See the help topic Creating a Strong Password for information about valid passwords.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/auth Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.currentPassword	string	required	The logged in user's current NetSuite password.

Parameter	Type	Required / Optional	Description
options.newPassword	string	required	The logged in user's new NetSuite password. See the help topic Creating a Strong Password for information about valid passwords.

Errors

Error Code	Thrown If
INVALID_PSWD	The options.password does not conform to the password rules. See the help topic Creating a Strong Password for information about valid passwords.
INVALID_EMAIL	The options.newEmail is invalid.
SSS_MISSING_REQD_ARGUMENT	The options.currentPassword or options.newPassword is not specified.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/auth Module Script Sample](#).

```

1 //Add additional code
2 ...
3 auth.changePassword({
4     currentPassword: 'mycurrentPWD',
5     newPassword: 'mynewPWD_2*'
6 });
7 ...
8 //Add additional code

```

N/cache Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/cache module to enable temporary, short-term storage of data. Using a cache improves performance by eliminating the need for scripts to repeatedly retrieve the same piece of data. You can use this module to build a cache to store and retrieve string values using a specific key.

You can create a cache that is available (1) to the current script only, (2) to all server scripts in the current bundle, or (3) to all server scripts in your NetSuite account. Data is stored in the cache according to its time to live (ttl) specified in the [Cache.put\(options\)](#) method.

Go To Samples

In This Help Topic

- [N/cache Module Members](#)

Cache Object Members

N/cache Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	cache.Cache	Object	Server scripts	Encapsulates a cache which is a segment of memory that can be used to store data on a temporary, short-term basis.
Method	cache.getCache(options)	cache.Cache	Server scripts	Checks for a cache object with the specified name. If the cache object exists, this method returns it. If the cache object does not exist, the system creates and returns a new cache object.
Enum	cache.Scope	enum	Server scripts	Holds the string values that describe the availability of the cache. Use this enum to set the value of the Cache.scope property and the <code>options.scope</code> parameter of the cache.getCache(options) method.

Cache Object Members

The following members are called on [cache.Cache](#).

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
Method	Cache.get(options)	string	Server scripts	Retrieves a value from the cache based on a key that you provide. If the requested value is not in the cache, the method calls the user-defined function identified by a method parameter.
	Cache.put(options)	string	Server scripts	Puts a value into the cache.
	Cache.remove(options)	string	Server scripts	Removes a value from the cache.
Property	Cache.name	string	Server scripts	The name of the cache.
	Cache.scope	string	Server scripts	The availability of the cache. A cache can be made available - to the current script only, - to all scripts in the current bundle, or - to all scripts in your NetSuite account. Set this value using the cache.Scope enum.

N/cache Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/cache module:

- Look Up Folder IDs
- Retrieve Name of a City Based on a ZIP Code Using Cache and a Custom Loader Function

Look Up Folder IDs

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a cache to help you look up folder IDs. Multiple searches are often required to look up folders. Using a cache could provide a simpler way to find your folder IDs.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

In this sample, folder names are used as the cache keys and the folder IDs are the actual cache values. This sample looks up folders by their folder name and their parent folder name. This sample includes four functions: `folderCacheLoader`, `getFolderCache`, `folderKey`, and `getFolder`.

You would include this sample code in a custom module that is called using the `options.loader` parameter of the `Cache.get(options)` method.

```

1  const FOLDER_CACHE_NAME = 'folder_cache';
2
3  function folderCacheLoader(context) {
4      const PARENT_FOLDER_ID = 0;
5      const FOLDER_NAME = 1;
6      const folderCacheKey = context.key.split('/');
7      const parentFolderId = folderCacheKey[PARENT_FOLDER_ID];
8      const folderName = folderCacheKey[FOLDER_NAME];
9
10     var folderId = null;
11     search.create ({
12         type: search.Type.FOLDER,
13         columns: ['internalid'],
14         filters: [
15             ['parent', search.Operator.ANYOF, parentFolderId],
16             'AND',
17             ['name', search.Operator.IS, folderName]
18         ]
19     }).run()
20     .each(function(folder) {
21         folderid = folder.id;
22         return false;
23     });
24
25     if (!folderId) {
26         var folder = record.create({
27             type: record.Type.FOLDER
28         });
29         folder.setValue({
30             fieldId: 'parent',
31             value: parentFolderId
32         });
33         folder.setValue({

```

```

34         fieldId: 'name',
35         value: folderName
36     });
37     folderId = folder.save();
38 }
39 return folderId;
40 }
41
42 function getFolderCache() {
43     return cache.getCache({
44         name: FOLDER_CACHE_NAME
45     });
46 }
47
48 function folderKey(folderName, parentFolderId) {
49     return[parentFolderId, folderName].join('/');
50 }
51
52 function getFolder(folderName, parentFolderId) {
53     return getFolderCache().get({
54         key: folderKey(folderName, parentFolderId),
55         loader: folderCacheLoader
56     });
57 }

```

Retrieve Name of a City Based on a ZIP Code Using Cache and a Custom Loader Function

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet and a custom module to retrieve the name of a city based on a ZIP code. To speed up processing, the Suitelet uses a cache.

In this sample, the ZIP code is the key used to retrieve city names from the cache. A loader function is called if the city corresponding to the provided ZIP code (key) is not in the cache. This loader function is a custom module that loads a CSV file and uses it to find the requested value. This function is called `zipCodeDatabaseLoader` (included in the second script sample).

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

i Note: This sample depends on a CSV file that must exist before the script is run. The sample CSV file is available [here](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6   // This script retrieves the name of a city based on a ZIP code from a cache.
7   define(['N/cache', '/SuiteScripts/zipToCityIndexCacheLoader'], function(cache, lib) {
8       const ZIP_CODES_CACHE_NAME = 'ZIP_CODES_CACHE';
9       const ZIP_TO_CITY_IDX_JSON = 'ZIP_TO_CITY_IDX_JSON';
10
11       function getZipCodeToCityLookupObj() {
12           const zipCache = cache.getCache({

```



```

13     name: ZIP_CODES_CACHE_NAME
14   });
15   const zipCacheJson = zipCache.get({
16     key: ZIP_TO_CITY_IDX_JSON,
17     loader: lib.zipCodeDatabaseLoader
18   });
19   return JSON.parse(zipCacheJson);
20 }
21
22 function findCityByZipCode(options) {
23   return getZipCodeToCityLookupObj()[String(options.zip)];
24 }
25
26 function onRequest(context) {
27   const start = new Date();
28   if (context.request.parameters.purgeZipCache === 'true') {
29     const zipCache = cache.getCache({
30       name: ZIP_CODES_CACHE_NAME
31     });
32     zipCache.remove({
33       key: ZIP_TO_CITY_IDX_JSON
34     });
35   }
36   const cityName = findCityByZipCode({
37     zip: context.request.parameters.zipcode
38   });
39
40   context.response.writeLine(cityName || 'Unknown :(');
41
42   if (context.request.parameters.auditPerf === 'true') {
43     context.response.writeLine('Time Elapsed: ' + (new Date().getTime() - start.getTime()) + ' ms');
44   }
45 }
46 return {
47   onRequest: onRequest
48 };
49 });

```

The following custom module provides the loader function used in the preceding Suitelet script sample. The loader function uses a CSV file to retrieve a value that was missing from a cache. This custom module does not need to include logic for placing the retrieved value into the cache. Whenever a value is returned through the options .loader parameter of the [Cache.get\(options\)](#) method, the value is automatically placed into the cache. This allows the loader function to serve as the sole method of populating a cache with values.

```

1  /**
2   * zipToCityIndexCacheLoader.js
3   * @NApiVersion 2.1
4   * @NModuleScope Public
5   */
6
7  //This custom module is a loader function that uses a CSV file to retrieve a value that was missing from a cache.
8  define(['N/file', 'N/cache'], function(file, cache) {
9     const ZIP_CODES_CSV_PATH = '/SuiteScripts/Resources/free-zipcode-CA-database-primary.csv';
10
11     function trimOuterQuotes(str) {
12       return (str || '').replace(/^"+/, '').replace(/"+$/, '');
13     }
14
15     function zipCodeDatabaseLoader(context) {
16       log.audit({
17         title: 'Loading Zip Codes',
18         details: 'Loading Zip Codes for ZIP_CODES_CACHE'
19       });
20       const zipCodesCsvText = file.load({
21         id: ZIP_CODES_CSV_PATH
22       }).getContents();
23       const zipToCityIndex = {};
24       const csvLines = zipCodesCsvText.split('\n');
25       util.each(csvLines.slice(1), function(el) {

```

```

26     var cells = el.split(',');
27     var key = trimOuterQuotes(cells[0]);
28     var value = trimOuterQuotes(cells[2]);
29     if (parseInt(key, 10))
30         zipToCityIndex[String(key)] = value;
31     });
32     return zipToCityIndex;
33 }
34
35 return {
36     zipCodeDatabaseLoader: zipCodeDatabaseLoader
37 }
38 });

```

cache.Cache

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A segment of memory that can be used to temporarily store data (on a short term basis). This object is returned by cache.getCache(options) .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/cache Module
Methods and Properties	Cache Object Members
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //myCache will be a cache.Cache object returned from the cache.getCache(options) method.
4 var myCache = cache.getCache({
5     name: 'temporaryCache',
6     scope: cache.Scope.PRIVATE
7 });
8 ...
9 //Add additional code

```

Cache.get(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves a string value from the cache. The value retrieved is identified by a key that you pass using the options.key parameter. If a requested value is not present in the cache, the system calls the user-defined function identified by the options.loader parameter. This loader function should provide logic for retrieving a value that is not in the cache. For an example, see N/cache Module Script Samples .
---------------------------	--

Returns	String or null
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	1 unit if the value is present in the cache 2 units if the loader function is used
Module	N/cache Module
Parent Object	cache.Cache
Sibling Object Members	Cache Object Members
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.key	string	required	The key corresponding to the value to be retrieved from the cache. This value cannot be null. For example, a ZIP code can be used as the key to retrieve city names. The maximum length of a key is 4KB (4096) bytes.
options.loader	string or function	optional	A user-defined loader function (or name of a user-defined function) used to return the requested value when it is not present in the cache. When the loader function retrieves a value, the system automatically places that value in the cache. For this reason, you should use a loader function as the primary means of populating the cache. If the value returned by the loader function is not a string, the system uses <code>JSON.stringify()</code> to convert the value before it is placed in the cache and returned. The maximum size of a value that can be placed in the cache is 500KB. The callback signature for the loader is <code>loader({key: key})</code>, which allows the loader to be predefined in a key-agnostic way (used to get different values for the same cache type, for example). When no loader function is specified and a value is missing from the cache, the system returns null.  Note: Depending on the execution of your script, the loader function may be called multiple times for a given cache value due to the ttl. Values only remain in the cache according to ttl. If the elapsed time between <code>Cache.get</code> calls for a given key exceed the ttl, the loader function will be called (because the value is no longer in the cache). For an example, see Retrieve Name of a City Based on a ZIP Code Using Cache and a Custom Loader Function.

Parameter	Type	Required / Optional	Description
options.ttl	number	optional	The maximum duration, in seconds, that a value retrieved by the loader function can remain in the cache. Note that the value may be removed from the cache before the ttl limit is reached. The minimum value is 300 (five minutes) and there is no maximum. The default ttl value is no limit.  Important: A cached value is not guaranteed to stay in the cache for the full duration of the ttl value. The ttl value represents the maximum time that the cached value may be stored.

Syntax


Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4     name: 'temporaryCache',
5     scope: cache.Scope.PRIVATE
6 });
7
8 var myValue = myCache.get({
9     key: 'keyText',
10    loader: loaderFunction, // Specify the name of your loader function
11    ttl: 18000
12 });
13 ...
14 //Add additional code

```

Cache.put(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Puts a value into the cache. If the value provided is not a string, the system uses JSON.stringify() to convert the value to a string.  Note: You can also put a value into the cache by using the Cache.get(options) method with its options.loader parameter. Cached data is not persistent, so you should consider using the Cache.get(options) method with the options.loader parameter to set and retrieve data. This may also result in a more efficient design. For an example, see Retrieve Name of a City Based on a ZIP Code Using Cache and a Custom Loader Function.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	1 unit

Module	N/cache Module
Parent Object	cache.Cache
Sibling Object Members	Cache Object Members
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.key	string	required	The key corresponding to the value to be placed in the cache. This value cannot be null. For example, a ZIP code can be used as the key for city names. The maximum length of a key is 4KB (4096) bytes.
options.value	string	required	The value to place in the cache. If the value is not a string, the system uses <code>JSON.stringify()</code> to convert the value before it is placed in the cache. The maximum size of the value is 500KB.
options.ttl	number	optional	The maximum duration, in seconds, that the value may remain in the cache. Note that the value may be removed before the ttl limit is reached. The minimum value is 300 (five minutes). If this option is not specified, there is no default maximum value.  Important: A cached value is not guaranteed to remain in the cache for the full duration of the ttl value. The ttl value represents the maximum time that the cached value may be stored. Cached data is not persistent, so you should consider using the Cache.get(options) method with the <code>options.loader</code> parameter to set and retrieve data.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4     name: 'temporaryCache',
5     scope: cache.Scope.PRIVATE
6 });
7 myCache.put({
8     key: 'keyText',
9     value: 'valueText',
10    ttl: 300
11 });
12 ...

```

13 | //Add additional code

Cache.remove(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes a value from the cache.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	1 unit
Module	N/cache Module
Parent Object	cache.Cache
Sibling Object Members	Cache Object Members
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.key	string	required	The key corresponding to the value to be removed from the cache.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4   name: 'temporaryCache',
5   scope: cache.Scope.PRIVATE
6 });
7 myCache.remove({
8   key: 'keyText'
9 });
10 ...
11 //Add additional code

```

Cache.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the cache.
-----------------------------	------------------------

Type	string
Module	N/cache Module
Parent Object	cache.Cache
Sibling Object Members	Cache Object Members
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#)

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4   name: 'temporaryCache', // Cache.name
5   scope: cache.Scope.PRIVATE
6 });
7 ...
8 //Add additional code

```

Cache.scope

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The availability of the cache to other scripts. Set this value using the cache.Scope enum.
Type	cache.Scope .
Module	N/cache Module
Parent Object	cache.Cache
Sibling Object Members	Cache Object Members
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4   name: 'temporaryCache',
5   scope: cache.Scope.PRIVATE // Cache.scope
6 });
7 ...

```

8 //Add additional code

cache.getCache(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Checks for a cache object with the specified name. If the cache exists, this method returns the cache object. If the cache does not exist, the system creates a cache and returns the created cache object.
Returns	cache.Cache
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/cache Module
Sibling Module Members	N/cache Module Members
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The name of the cache to be retrieved or created. The maximum size of the cache name is 1 kb.
options.scope	string	optional	Sets the availability of the cache. A cache can be made available to the current script only, to all scripts in the current bundle, or to all scripts in your NetSuite account. Set this value using the cache.Scope enum. The default value is <code>cache.Scope.PRIVATE</code> .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4   name: 'temporaryCache',
5   scope: cache.Scope.PUBLIC
6 });
7 ...
8 //Add additional code

```


cache.Scope

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values that describe the availability of the cache. Use this enum to set the value of the Cache.scope property and the options.scope parameter of the cache.getCache(options) method.
Module	N/cache Module
Sibling Module Members	N/cache Module Members
Since	2016.2

Values

Value	Description
PRIVATE	The cache is available only to the current script. This is the default value.
PROTECTED	The cache is available only to some scripts: ■ If the script is part of a bundle, the cache is available to all scripts in the same bundle. ■ If the script is not in a bundle, the cache is available to all scripts not in any bundle.
PUBLIC	The cache is available to all server scripts in the NetSuite account.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/cache Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCache = cache.getCache({
4   name: 'temporaryCache',
5   scope: cache.Scope.PROTECTED
6 });
7 ...
8 //Add additional code

```

N/certificateControl Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/certificateControl module to enable scripting access to the Digital Certificates list found in the UI at Setup > Company > Certificates. You can use this module to find, create, update, read and delete certificate records. For more information, see the help topics [Digital Signing](#) and [Uploading Digital Certificates](#).

[Go To Samples {..}](#)

To access the N/certificateControl module, you must use the Execute As Role field on the script deployment record. Select either the Administrator role or a custom role with the Certificate Access permission. For more information, see the help topic [Access to Digital Certificates](#).



Important: The certificate record holds information for a digital certificate, but it is not a standard NetSuite record and cannot be accessed with the N/record.

In This Help Topic

- [N/certificateControl Module Members](#)
- [Certificate Object Members](#)

N/certificateControl Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	certificateControl.Certificate	object	Server scripts	Encapsulates a digital certificate record.
Method	certificateControl.findCertificates(options)	object	Server scripts	Returns metadata about the certificate(s).
	certificateControl.findUsages(options)	object[]	Server scripts	Returns an audit trail of how a certificate has been used. Includes operations performed with time stamps.
	certificateControl.createCertificate(options)	certificateControl.Certificate	Server scripts	Creates a certificate record using a file from the File Cabinet. After saving with Certificate.save() , the certificate is accessible on the Certificates .
	certificateControl.deleteCertificate(options)	string	Server scripts	Deletes a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options) and saved with Certificate.save() .
	certificateControl.loadCertificate(options)	certificateControl.Certificate	Server scripts	Loads a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options) .
	certificateControl.lock(options)	string	Server scripts	Locks a certificate record so that it cannot be edited.
	certificateControl.unlock(options)	string	Server scripts	Unlocks a certificate record that has been locked with certificateControl.lock(options) .
Enum	certificateControl.Operation	enum	Server scripts	Holds the values for the operation when searching for

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				certificates with certificateControl.findUsages(options) .
	certificateControl.Operator	enum	Server scripts	Holds the values for search operators to use with the name and description parameters of the certificateControl.findCertificates(options) method.
	certificateControl.Type	enum	Server scripts	Holds the values for the certificate file type to use with the type parameter of the certificateControl.findCertificates(options) method.

Certificate Object Members

The following members are called on the [certificateControl.Certificate](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Certificate.save()	object containing the script ID of the new certificate record	Server scripts	Saves a certificate record.
Property	Certificate.description	string	Server scripts	Describes the certificate record.
	Certificate.file	File Object Members object	Server scripts	Includes the properties of the file uploaded to create the certificate.
	Certificate.name	string	Server scripts	The name of the certificate record.
	Certificate.monthReminder	boolean	Server scripts	The setting of the Month box for Expiration Reminders on the certificate record.
	Certificate.notifications	number[]	Server scripts	The internal IDs of the employees selected in the Copy Employees field on the certificate record.
	Certificate.password	string (write-only)	Server scripts	The password for the digital certificate. You can create a GUID for the password using Form.addSecretKeyField(options) or you can create an API secret for the secret at Setup > Company > API Secrets.
	Certificate.restrictions	number[]	Server scripts	The internal IDs of the employees selected in the Restrict to Employees field of the certificate record.
	Certificate.scriptId	string	Server scripts	The ID of the certificate record.
	Certificate.subsidiaries	number[]	Server scripts	The internal IDs of the subsidiaries associated with the certificate record.
	Certificate.threeMonthsReminder	boolean	Server scripts	Indicates the setting of the 3 Months box for Expiration Reminders on the certificate record.
	Certificate.weekReminder	boolean	Server scripts	Indicates the setting of the Week box for Expiration Reminders on the certificate record.

N/certificateControl Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/certificateControl module:

- [Filter the Digital Certificate List by Subsidiary and File Type](#)
- [Find the Audit Trail of POST Operations for a Certificate Record Based on ID](#)
- [Create, Modify, and Save Certificate Record Based on a File in the File Cabinet](#)
- [Find and Use an Existing Certificate Record](#)
- [Establish an SFTP Connection Using an SSH Key](#)

Filter the Digital Certificate List by Subsidiary and File Type

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to filter the Digital Certificates list by subsidiary and by file type.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/certificateControl'],
5    function(certificateControl){
6      var all = certificateControl.findCertificates();
7      var specificType = certificateControl.findCertificates({
8        type: 'PFX'
9      });
10     var specificSub = certificateControl.findCertificates({
11       subsidiary: 93
12     });
13     var specificTypeAndSub = certificateControl.findCertificates({
14       type: 'PFX',
15       subsidiary: 93
16     });

```

Find the Audit Trail of POST Operations for a Certificate Record Based on ID

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to find the audit trail of POST operations for the certificate record with ID 'custcertificate_china'.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x

```

```

3  */
4
5  require(['N/certificateControl'], function(cc){
6      var usages = cc.findUsages({
7          id: 'custcertificate_china',
8          operation: cc.Operation.POST
9      });
10 })

```

Create, Modify, and Save Certificate Record Based on a File in the File Cabinet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create a file object by loading a file from the File Cabinet. It then creates the options needed for the `certificateControl.createCertificate(options)` method and creates and saves the certificate record. The certificate record is then loaded again, edited to change the file, and saved again.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/certificateControl', 'N/file'], function(cc, file){
6      var fileObj = file.load({
7          id: 'SuiteScripts/dsa.p12'
8      });
9      var options = {
10         file : fileObj,
11         password : '022b490ad4334c7e86a8304f937ec68f',
12         name : 'testCert',
13         description : 'testDescription',
14         scriptId : '_testid',
15         subsidiaries : [1,3],
16         weekReminder : false,
17         monthReminder : true,
18         threeMonthsReminder : false
19     };
20     var newCertificate = cc.createCertificate(options);
21     newCertificate.save();
22
23     var loadedCertificate = cc.loadCertificate({
24         scriptId : 'custcertificate_testid'
25     });
26     fileObj = file.load({
27         id: 'SuiteScripts/ecdsa.p12'
28     });
29     loadedCertificate.file = fileObj;
30     loadedCertificate.password = '022b490ad4334c7e86a8304f937ec68f';
31     loadedCertificate.save();
32 })

```

Find and Use an Existing Certificate Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to find an existing certificate record and use it in an operation.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/certificateControl', 'N/https/clientCertificate'], function(cc, cert){
6      var yodlee = cc.findCertificates({
7          name: 'Yodlee',
8          description: 'Yodlee certificate'
9      });
10     cert.post({
11         certId: yodlee[0].id,
12         url: 'url',
13         body: 'body',
14         headers: 'headers'
15     });
16 })

```

Establish an SFTP Connection Using an SSH Key

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample establishes a SFTP connection using an SSH key that has already been uploaded to NetSuite. It then creates, updates, loads, and deletes a certificate record to show the full CRUD operation. Replace the server URL with your correct URL.

For the SFTP connection, the public key corresponding to the private key in the certificate must be stored in the `.ssh/authorized_keys` file on the server.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/file', 'N/sftp', 'N/certificateControl'], function(file, sftp, certificateControl) {
6      var certPath = 'yyy/certificates'
7      var certName = 'apiclient_cert.p12';
8
9      // Establish SFTP connection
10     var connection = sftp.createConnection({
11         username: 'sftpuser',
12         keyId: 'custkeysftp_nft_demo_key',
13         url: 'my.sftp.example.com',
14         port: 22,
15         directory: 'inbound',
16         hostKey: 'AAAAB3NzaC1yc2EAAAADAQABAAQCAgYD1K41E9QnuYEgRRQChjrAM1+bTT95e71Xv0oQ60ywVQEiedhRqSMbPcPPB4pjpBd0mPIQC
17         Ckug+3XwAQ6uNj3UM11zoGGmg86tyEJT6qGB0SsrQJzHTb3EG38B5rB00WEz0WeJ8E8YODT3oAj1NrF8Ls3JbG0bRF+0uwJD11LSrFkYS3kWCv27NhBnaytGe7iLBgrJd
18         NV1itNqkxZfK0NsAYCaJWQjQLtz+GFfN5zTbKKNsDa6s/YW7oAMMOI3Q5GQAQdXtKY728WvxYTjr2FsYS/KM6nbq/csTvZHWLE0z2TQtB2H0I1ofvEP/QvXwmqEnCeVpCn
19         gRwdHwQf'
20     });
21
22     // Create new certificate
23     var certScriptId = '_' + (Math.random().toString(36).substring(2, 10));

```

```

22 |     var cert = certificateControl.createCertificate();
23 |     cert.name = 'Test Certificate China API';
24 |     cert.description = 'Test Certificate China created using API';
25 |     // custcertificate prefix will be added automatically
26 |     cert.scriptId = certScriptId;
27 |     cert.file = connection.download({
28 |         directory: certPath,
29 |         filename: certName
30 |     });
31 |     //guid corresponding to the certificate's password';
32 |     var pwd = '022b490ad4334c7e86a8304f937ec68f';
33 |     cert.password = pwd;
34 |     cert.save();
35 |     /*/certScriptId = 'custcertificate' + certScriptId;
36 |     log.debug('Certificate "' + cert.name + '" successfully created with id "' + certScriptId + '"');
37 |     // -----
38 |     // Rename certificate
39 |     cert = certificateControl.loadCertificate({scriptId: certScriptId});
40 |     cert.name = 'Test Certificate China API TEMP';
41 |     cert.save();
42 |     // Verify new certificate name
43 |     /*/cert = certificateControl.loadCertificate({scriptId: certScriptId});
44 |     log.debug('Certificate successfully renamed to "' + cert.name + '"');
45 |     // -----
46 |     // Delete certificate
47 |     certificateControl.deleteCertificate(certScriptId);
48 |     log.debug('Certificate deleted!');
49 |     // -----
50 |     // Load the deleted certificate
51 |     // attempt to load the deleted certificate - this should error
52 |     try {
53 |         cert = certificateControl.loadCertificate({scriptId: certScriptId});
54 |     }
55 |     catch (e) {
56 |         log.error(e.message);
57 |     }
58 | })

```

certificateControl.Certificate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The certificate record, including file name and preferences.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/certificateControl Module
Methods and Properties	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var loadedCertificate = certificateControl.loadCertificate({
4 |     scriptId: 'custcertificate_testid'

```

```

5  });
6  fileObj = file.load({
7      id: 'SuiteScripts/ecdsa.p12'
8  });
9  loadedCertificate.file = fileObj;
10 loadedCertificate.password = '022b490ad4334c7e86a8304f937ec68f',
11 loadedCertificate.save();
12 certificateControl.deleteCertificate({
13     scriptId: 'custcertificate_testid'
14 });
15 ...
16 //Add additional code

```

Certificate.save()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Saves a certificate record object.
Returns	certificateControl.Certificate object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1  // Add additional code
2  ...
3  fileObj = file.load({
4      id: 'SuiteScripts/ecdsa.p12'
5  });
6  loadedCertificate.file = fileObj;
7  loadedCertificate.password = '022b490ad4334c7e86a8304f937ec68f',
8  loadedCertificate.save();
9  ...
10 // Add additional code

```

Certificate.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A description of the certificate record.
Type	string

Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId: 'custcertificate_testid'
6 });
7
8 //update the description for the certificate record
9 loadedCertificate.description = 'Test Certificate Description'
10
11 //save the updated certificate record
12 loadedCertificate.save();
13 ...
14 //Add additional code

```

Certificate.file

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The File Object Members object of the certificate uploaded to the certificate record.
Type	File Object Members object
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId: 'custcertificate_testid'
6 });
7
8 //load the file from the File Cabinet
9 fileObj = file.load({

```

```

10     id: 'SuiteScripts/ecdsa.p12'
11   });
12
13   //upload the file to the certificate record
14   loadedCertificate.file = fileObj;
15
16   //save the certificate record
17   loadedCertificate.save();
18   ...
19   //Add additional code

```

Certificate.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the certificate record.
Type	string
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1   //Add additional code
2   ...
3   //load the certificate record object
4   var loadedCertificate = certificateControl.loadCertificate({
5     scriptId: 'custcertificate_testid'
6   });
7
8   //update the name of the certificate record
9   loadedCertificate.name = 'Brazil Certificate';
10
11  //save the certificate record object
12  loadedCertificate.save();
13  ...
14  //Add additional code

```

Certificate.monthReminder

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the setting of the Month box for Expiration Reminders on the certificate record. This property is set to true if the Month box is checked and email reminders are sent to account administrators one month before the certificate expires. If the Copy Employees box is also checked, selected employees are copied on the reminder emails.
Type	boolean

Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId: 'custcertificate_testid'
6 });
7
8 //update the Expiration Reminder for Month to checked
9 loadedCertificate.monthReminder = true;
10
11 //save the certificate record object
12 loadedCertificate.save();
13 ...
14 //Add additional code

```

Certificate.notifications

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal IDs of the employees copied on expiration notification email. The values for this property are found in the Copy Employees field of the Audience subtab on the certificate record. When you create or edit a certificate object with values for this property, you also check the Copy Employees box for the certificate record.
Type	number[]
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code

```

```

2  ...
3  //create a variable to hold the properties of the certificate object
4  var options = {
5      name: 'testCertp12',
6      description: 'testDescription',
7      scriptId: '_testidp12',
8      //include the internal IDs for employees you want copied on expiration reminder email
9      notifications: [168,259]
10 };
11
12 //create the certificate record with the options variable
13 var newCertificate = certificateControl.createCertificate(options);
14
15 //save the certificate object
16 newCertificate.save();
17 ...
18 //Add additional code

```

Certificate.password

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The password for the digital certificate. If the certificate file is password-protected, you can store the password with the certificate record. If the certificate is not password-protected, enter an empty string. This property accepts the script ID of an API secret that stores your certificate password. For more information about API secrets, see the help topic Secrets Management.
Type	string (write-only)
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1  //Add additional code
2  ...
3  //create a variable to hold the properties of the certificate object
4  var options = {
5      name: 'testCertp12',
6      description: 'testDescription',
7      //for password, enter the guid associated with your digital certificate or an empty string
8      password: 'yourCertPassword',
9      scriptId: '_testidp12',
10 };
11
12 //create the certificate record with the options variable
13 var newCertificate = certificateControl.createCertificate(options);
14
15 //save the certificate object
16 newCertificate.save();
17 ...

```

18 //Add additional code

Certificate.restrictions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal IDs of the employees selected in the Restrict to Employees field of the certificate record. If you set this property with an employee internal ID, you check the Restrict to Employees box and select that employee. Employees selected must also have either the Certificate Management or Certificate Access role permission to access the certificate. When the Restrict to Employees box is checked, only Administrators and the employees selected can access the certificate.
Type	number[]
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5   scriptId : 'custcertificate_testid'
6 });
7
8 //check the Restrict to Employees box and select employees with internal IDs of 189 and 250
9 loadedCertificate.restrictions = [189,250];
10 loadedCertificate.save();
11 ...
12 //Add additional code

```

Certificate.scriptId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the certificate record. The script ID for certificate records begins with “custcertificate.”
Type	string
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members

Since	2019.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId : 'custcertificate_testid'
6 });
7
8 //update the text for script ID
9 loadedCertificate.scriptId = '_ChinaCert';
10 loadedCertificate.save();
11 ...
12 //Add additional code

```

Certificate.subsidiaries

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal IDs of the subsidiaries associated with the certificate record. Subsidiary selections associate a certificate to one or more subsidiaries but do not affect access.
Type	number[]
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId : 'custcertificate_testid'
6 });
7
8 //set the subsidiaries to those with the internal IDs of 3 and 5
9 loadedCertificate.subsidiaries = [3,5];
10
11 //save the certificate record object
12 loadedCertificate.save();
13 ...
14 //Add additional code

```

Certificate.threeMonthsReminder

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the setting of the 3 Months box for Expiration Reminders on the certificate record. This property is set to true if the 3 Months box is checked. When set to true , email reminders are sent to account administrators three months before the certificate expires. If the Copy Employees box is also checked, selected employees are copied on the reminder emails.
Type	boolean
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate
Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId : 'custcertificate_testid'
6 });
7
8 //update the Expiration Reminder for 3 Months to checked
9 loadedCertificate.threeMonthsReminder = true;
10
11 //save the certificate record object
12 loadedCertificate.save();
13 ...
14 //Add additional code

```

Certificate.weekReminder

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the setting of the Week box for Expiration Reminders on the certificate record. This property is set to true if the Week box is checked. When set to true , email reminders are sent to account administrators one week before the certificate expires. If the Copy Employees box is also checked, selected employees are copied on the reminder emails.
Type	boolean
Module	N/certificateControl Module
Parent Object	certificateControl.Certificate

Sibling Object Members	Certificate Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId : 'custcertificate_testid'
6 });
7
8 //update the Expiration Reminder for Week to checked
9 loadedCertificate.weekReminder = true;
10
11 //save the certificate record object
12 loadedCertificate.save();
13 ...
14 //Add additional code

```

certificateControl.createCertificate(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a certificate record on the Certificates page using a file from the File Cabinet.  Note: Your role must have Create, Edit, or Full access to the Certificate Access permission to create certificates using SuiteScript.
Returns	certificateControl.Certificate
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.file	file	required	A File Object Members object. The file must already be uploaded to the File Cabinet.	2019.2

Parameter	Type	Required / Optional	Description	Since
options.password	string	optional	If applicable, the password associated with your digital certificate. The script ID of an API secret can be accepted. For more information, see the help topic Secrets Management .	2019.2
options.scriptId	string	optional	The desired script ID of the certificate record. The script ID is automatically prefixed with custcertificate.	2019.2
options.description	string	optional	The description of the certificate record.	2019.2
options.subsidiaries	number[] or string[]	optional	The internal ID of subsidiaries associated with the certificate in either number or string format.	2019.2
options.restrictions	number[] or string[]	optional	The internal ID of employees selected in the Restricted to Employees field for a certificate. You can enter the internal ID in either number or string format.	2019.2
options.notifications	number[] or string[]	optional	The internal ID of employees selected in the Copy Employees field on the certificate record. You can enter the internal ID in either number or string format.	2019.2
options.name	string	required	The name of the certificate record.	2019.2
options.weekReminder	boolean	optional	The setting for the Expiration Reminder : Week checkbox.	2019.2
options.monthReminder	boolean	optional	The setting for the Expiration Reminder : Month checkbox.	2019.2
options.threeMonthsReminder	boolean	optional	The setting for the Expiration Reminder : 3 Months checkbox.	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4     id: 'SuiteScripts/dsa.p12'
5 });
6 var options = {
7     file : fileObj,
8     password : '022b490ad4334c7e86a8304f937ec68f',
9     name : 'testCert',
10    description : 'testDescription',
11    scriptId : '_testid',
12    subsidiaries : [1,3],
13    weekReminder : false,
14    monthReminder : true,
15    threeMonthsReminder : false
16 };
17 var newCertificate = certificateControl.createCertificate(options);

```

```

18 newCertificate.save();
19 ...
20 //Add additional code

```

certificateControl.deleteCertificate(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options) . i Note: Your role must have either Edit or Full access to the Certificate Access permission to delete certificate records using SuiteScript. History of the certificate is not deleted.
Returns	The script ID of the deleted certificate.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2019.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	The script ID or internal ID for the certificate you want to delete. You can view the ID of a certificate from the Digital Certificates list at Setup > Company > Certificates.

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var usages = certificateControl.deleteCertificate({
4     scriptId: 'custcertificate_china'
5 });
6 ...
7 //Add additional code

```

certificateControl.findCertificates(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an array of certificates available. You can use the parameters as filters for this search. If you do not use any parameters, all certificate records are returned.
Returns	Metadata about the certificate(s)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.description	string	optional	The certificate description. You can use the certificateControl.Operator enum with this parameter.	2019.2
options.name	string	optional	The certificate name. You can use the certificateControl.Operator enum with this parameter.	2019.2
options.notification	number	optional	The internal ID of an employee selected in the Copy Employees field.	2019.2
options.restriction	number	optional	The internal ID of an employee selected in the Restrict to Employees field.	2019.2
options.scriptRestriction	string	optional	Script name to be used for searching for certificates with restrictions to a particular script.	2022.1
options.subsidiary	number	optional	The internal ID of the subsidiary.	2019.1
options.type	string	optional	The certificate file type. You can use the certificateControl.Type enum with this parameter.	2019.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```
1 | //Add additional code
```

```

2 ...
3 var yodlee = certificateControl.findCertificates({
4   name: 'Yodlee',
5   description: 'Yodlee certificate'
6 });
7 ...
8 //Add additional code

```

certificateControl.findUsages(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an audit trail of how a certificate has been used. Includes operations performed with time stamps.
Returns	An array of operations performed.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.from	date	optional	The start date for your audit trail search.	2019.2
options.to	date	optional	The end date for your audit trail search.	2019.2
options.id	string	optional	The script ID of the certificate record.	2019.2
options.operation	string	optional	The certificateControl.Operation performed with the digital certificate.	2019.2
options.script	number	optional	The script ID of a script record that used a certificate record.	2019.2
options.deploy	number	optional	The script ID of a script deployment that used a certificate record.	2019.2
options.entity	number	optional	The internal ID of the employee who performed the operation.	2019.2

Error	Thrown If
SSS_INVALID_TYPE_ARG	A parameter provided is the wrong type.
TOO_MANY_RESULTS	There are more than 1000 results.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var usages = certificateControl.findUsages({
4   id: 'custcertificate_china',
5   operation: certificateControl.Operation.POST
6 });
7 ...
8 //Add additional code

```

certificateControl.loadCertificate(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options) .
Returns	certificateControl.Certificate
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2019.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	The script ID or internal ID for the certificate you want to load. You can view the ID of a certificate from the Digital Certificates list at Setup > Company > Certificates.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5   scriptId: 'custcertificate_testid'
6 });

```

```

7
8 //load a digital certificate from the File Cabinet
9 fileObj = file.load({
10     id: 'SuiteScripts/ecdsa.p12'
11 });
12
13 //upload the file to the certificate record
14 loadedCertificate.file = fileObj;
15
16 //update the password to match the guid password for the certificate
17 loadedCertificate.password = 'certPass';
18
19 //save the certificate
20 loadedCertificate.save();
21 ...
22 //Add additional code

```

certificateControl.lock(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Locks a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options) . Locked certificates cannot be edited until they are unlocked with certificateControl.unlock(options) .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2021.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The script ID or internal ID for the certificate you want to lock. You can view the ID of a certificate from the Digital Certificates list at Setup > Company > Certificates.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var loadedCertificate = certificateControl.loadCertificate({
4     scriptId : 'custcertificate_testid'
5 });

```

```

6
7 //lock the certificate
8 loadedCertificate.lock({
9     id: 'custcertificate_testid'
10 });
11
12 //save the certificate
13 loadedCertificate.save();
14 ...
15 //Add additional code

```

certificateControl.unlock(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Unlocks a certificate record that has been locked with certificateControl.lock(options) .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/certificateControl Module
Since	2021.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The script ID or internal ID for the certificate you want to unlock. You can view the ID of a certificate from the Digital Certificates list at Setup > Company > Certificates.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 //load the certificate record object
4 var loadedCertificate = certificateControl.loadCertificate({
5     scriptId : 'custcertificate_testid'
6 });
7
8 //unlock the certificate
9 loadedCertificate.unlock({
10     id: 'custcertificate_testid'
11 });
12

```

```
13 //save the certificate
14 loadedCertificate.save();
15 ...
16 //Add additional code
```

certificateControl.Operation

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the operation when searching for certificates with the certificateControl.findUsages(options) method.
Module	N/certificateControl Module
Sibling Module Members	N/certificateControl Module Members
Since	2019.2

Values

Value
CONNECT
DELETE
FIND
GET
HEAD
POST
PUT
SIGN_STRING
SIGN_XML
VERIFY_STRING
VERIFY_XML

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var usages = certificateControl.findUsages({
```



```

4      id: 'custcertificate_china',
5      operation: certificateControl.Operation.POST
6    });
7    ...
8    //Add additional code

```

certificateControl.Operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for search operators to use with the name and description parameters of the certificateControl.findCertificates(options) method.
Module	N/certificateControl Module
Sibling Module Members	N/certificateControl Module Members
Since	2019.2

Values

Value
CONTAINS
ENDS_WITH
EQUALS
STARTS_WITH

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1    //Add additional code
2    ...
3    var yodlee = certificateControl.findCertificates({
4      name: {
5        value: 'CERT',
6        operator: certificateControl.Operator.STARTS_WITH,
7        ignoreCase: true
8      },
9      description: {
10       value: 'Yodlee',
11       operator: certificateControl.Operator.CONTAINS,
12       ignoreCase: true
13     }
14   });
15   ...
16   //Add additional code

```

certificateControl.Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the certificate file type to use with the type parameter of the certificateControl.findCertificates(options) method.
Module	N/certificateControl Module
Sibling Module Members	N/certificateControl Module Members
Since	2019.1

Values

Value
PFX
P12
PEM

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/certificateControl Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var specificType = certificateControl.findCertificates({
4     type: certificateControl.Type.PFX
5 });
6 ...
7 //Add additional code

```

N/commerce Modules

Applies to: Commerce Web Stores | SuiteScript 2.x | APIs

Modules in the N/commerce namespace have been designed for use with SSP applications written in SuiteScript 2.x. Their primary objective is to provide web developers a homogeneous development experience when working with Commerce and SuiteScript APIs. Note that N/commerce itself is not a module.

Developers can use modules in the N/commerce namespace to access different assets in the web store context, such as items and shopping cart. The modules within the N/commerce namespace are supported by the latest version of SuiteCommerce and by SuiteCommerce Advanced 2019.2 onwards.

Before you can load N/commerce modules, you must have an active shopping session.

The modules available within the N/commerce namespace are:

Module	Description
N/commerce/recordView Module	Use the N/commerce/recordView module to provide fast, cached, and public access to the item fields and website settings.

Note: Commerce modules are not yet available for all web store interactions. To fully customize your web store on the SuiteCloud platform, you can use Commerce APIs with SSP applications written in SuiteScript 1.0. However, Commerce APIs are not available for use with SSP applications written in SuiteScript 2.x. For more information about the Commerce APIs, see the help topic [Commerce API](#).

N/commerce/recordView Module

Applies to: Commerce Web Stores | SuiteScript 2.x | APIs

Use the N/commerce/recordView module to provide fast, cached, and public access to the item fields and website settings.

- [N/commerce/recordView Module Members](#)
- [N/commerce/recordView Script Sample](#)

N/commerce/recordView Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	recordView.viewItems (options)	Object array Returns one or more Items with requested items fields as an object with field:value pairs. See the Returns section.	Client and server scripts	Retrieves one or more Items with requested items fields from an Item Record.
	recordView.viewWebsite (options)	Object Returns website and website fields as an object with field:value pairs. See the Returns section.	Client and server scripts	Retrieves the website details with requested website fields.

recordView.viewItems(options)

Applies to: Commerce Web Stores | SuiteScript 2.x | APIs

Method Description	Retrieves one or more items with requested item fields from an Item Record.
Returns	See the Returns section.

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/commerce/recordView Module
Sibling Module Members	N/commerce/recordView Module Members
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Property	Type	Required / Optional	Description
options.ids	number[]	required	IDs of the item you want to view.
options.fields	string string[]	required	Item fields you want to retrieve for the items. For more information, see Supported Fields .
options.fieldOptions	Array of name, value pairs. Type depends upon parameter: includeVat - string boolean[]	optional	Options that affect related fields. Supported field options: includeVat - this affects onlinecustomerprice_detail field. Default value is false.

Supported Fields

The following fields can be requested in the options.fields parameter:

Record Fields			
amortizationperiod	enforceminqtyinternally	mpn	searchkeywords
amortizationtemplate	excludefromsitemap	nexttagcategory	seasonaldemand
assetaccount	expenseaccount	nopricemail	shippingcost
autoleadtime	externalid	outofstockmessage	shoppingdotcomcategory
autopreferredstocklevel	featureddescription	overallquantitypricingtype	shopzillacategoryid
autoreorderpoint	gainlossaccount	pagetitle	showdefaultdonationamount
availabletopartners	generateaccruals	parent	stockdescription
averagecost	handlingcost	preferredlocation	stockunit
billexchratevarianceacct	incomeaccount	preferredstocklevel	storedescription
billingschedule	internalid	preferredstockleveldays	storedetaileddescription
billpricevarianceacct	isdonationitem	pricinggroup	storedisplayimage
billqtyvarianceacct	isdropshipitem	printitems	storedisplayname
buildentireassembly	isfulfillable	projectexpensetype	storedisplaythumbnail
class	isinactive	projecttemplate	totalvalue
copydescription	islotitem	purchasedescription	tracklandedcost
cost	isonline	purchaseorderamount	transferprice

costestimate	isserialitem	purchaseorderquantity	unbuildvarianceaccount
costestimatetime	isspecialorderitem	purchaseorderquantitydiff	unitstype
costingmethod	isspecialworkorderitem	purchaseunit	upccode
countryofmanufacture	itemid	quantitypricingschedule	urlcomponent
createddate	itemprocessfamily	rate	usecomponentyield
createjob	itemprocessgroup	receiptamount	usemarginalrates
custreturnvarianceaccount	itemtype	receiptquantity	vendor
deferralaccount	lastpurchaseprice	receiptquantitydiff	vendorname
deferredrevenueaccount	leadtime	relateditemsdescription	vendreturnvarianceaccount
deferrevrec	location	reordermultiple	vsodeferral
demandmodifier	manufacturer	reorderpoint	vsodelivered
department	matchbilltoreceipt	residual	vsopermitdiscount
description	matrixitemnametemplate	revrecschedule	vsoprice
displayname	maxdonationamount	roundupascomponent	vsosopgroup
dontshowprice	maximumquantity	safetystocklevel	weight
dropshipexpenseaccount	metataghtml	safetystockleveldays	weightunit
effectivebomcontrol	minimumquantity	saleunit	

Synthetic Fields			
commercecategory	ispurchasable	itemoptions_detail	quantityonhand_detail
onlinecustomerprice_detail	correlateditems	relateditems	quantityonorder_detail
isbackorderable	defaultitemshipmethod	quantityavailable_detail	showoutofstockmessage
isdisplayable	itemimages_detail	quantitybackordered_detail	
isinstock	matrixchilditems	quantitycommitted_detail	

Errors

Error Code	Thrown If
SSS_INVALID_TYPE_ARG	Parameter is invalid
FIELD_1_CANNOT_BE_EMPTY	Required parameter is missing or empty

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/commerce/recordView Script Sample](#).

```

1 // Add additional code here
2 ...
3 try {
4     result.viewItems = recordView.viewItems({
5         ids: [408],
6         fields: ["itemtype", "isavailable"]
7         fieldOptions: [{"includeVat": true}]
8     });
9 }
10 catch (e) {

```

```
11 |     result.error = e.name + ": " + e.message;
12 |   }
13 |   ...
```

Returns

Returns a flat JSON structure with field:value pairs.

```
1 | [{
2 |   "internalid": {
3 |     value : 523
4 |   },
5 |   "name": {
6 |     value : "test"
7 |   },
8 |   "dropdownListField":{
9 |     value : {
10 |       "id": 1,
11 |       "label":"Option 1"
12 |     }
13 |   },
14 |   "checkbox1Field": {
15 |     value : true
16 |   },
17 |   "dateField1Field": {
18 |     value : "2012-04-23T18:25:43.511Z"
19 |   }
20 | }, ... ]
```

recordView.viewWebsite(options)

 **Applies to:** Commerce Web Stores | SuiteScript 2.x | APIs

Method Description	Retrieves the website details with requested website fields.
Returns	See the Returns section.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/commerce/recordView Module
Sibling Module Members	N/commerce/recordView Module Members
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Property	Type	Required / Optional	Description
options.id	number	required	ID of the website.
options.fields	string string[]	required	Website fields you want to retrieve. For more information, see Supported Fields .

Property	Type	Required / Optional	Description
options.fieldOptions	Array of name, value pairs. Type depends upon parameter.	optional	Options that affect all related fields that are retrieved. Supported field options: None

Supported Fields

The following fields can be requested in the options.fields parameter:

Record Fields			
analyticsclickattributes	displaycompanyfield	onlinepricelevel	shipstoallcountries
analyticssubmitattributes	displayname	outofstockbehavior	showcookieconsentbanner
autoapplypromotionsenabled	hidepaymentpagewhennobalance	outofstockitems	showpofieldonpayment
cartsharingmode	igniteedition	requestshippingaddressfirst	showextendedcart
confpagetrackinghtml	includevatwithprices	requirecompanyfield	showsavcreditinfo
cookiepolicy	isinactive	requireloginforpricing	showshippingestimator
customregistrationtype	iswebstoreoffline	requireshippinginformation	siteloginrequired
defaultshippingcountry	internalid	requiretermsandconditions	subsidiaries
defaultshippingmethod	loginallowed	savecreditinfo	termsandconditionshtml

Synthetic Fields			
currencies	imagesizes	legacytouchpoints	sortfields
facetfields	languages	shiptocountries	subsidiaries

Errors

Error Code	Thrown If
SSS_INVALID_TYPE_ARG	Parameter is invalid
FIELD_1_CANNOT_BE_EMPTY	Required parameter is empty

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/commerce/recordView Script Sample](#).

```

1 // Add additional code here
2 ...
3 try {
4     result.viewItems = recordView.viewWebsite({
5         id: 2,
6         fields: ["shiptocountries"]
7     });
8 }
9 catch (e) {
10     result.error = e.name + ": " + e.message;
11 }
12 ...

```

Returns

Returns a flat JSON structure with field:value pairs.

```

1  [{
2    "internalid": {
3      value : 523
4    },
5    "name": {
6      value : "test"
7    },
8    "dropdownListField":{
9      value : {
10         "id": 1,
11         "label":"Option 1"
12       }
13    },
14    "checkbox1Field": {
15      value : true
16    },
17    "dateField1Field": {
18      value : "2012-04-23T18:25:43.511Z"
19    }
20  }, ... ]

```

N/commerce/recordView Script Sample

i Applies to: Commerce Web Stores | SuiteScript 2.x | APIs

The following script sample demonstrates how to use the features of the N/commerce/recordView module.

Retrieve Website and Item Data

The following sample retrieves some details of the website and some item data for the specified items.

i Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  define(['N/commerce/recordView'],
5    function (recordView) {
6      function service(context) {
7        var result = {};
8        try {
9          result.website= recordView.viewWebsite({
10             id: 2,
11             fields: ["internalid","shiptocountries"]
12           });
13        }
14        catch (e) {
15          result.websiteError = e.name + ": " + e.message;
16        }
17
18        var options = {
19          "ids": [382,388],
20          "fields": ["displayname"]

```



```

21     };
22     try {
23         result.items= recordView.viewItems(options);
24     }
25     catch (e) {
26         result.itemsError = e.name + " : " + e.message;
27     }
28     return context.response.write(
29         JSON.stringify(result)
30     );
31 }
32
33 return {
34     service: service
35 };
36 }
37 );

```

N/compress Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/compress module to compress and decompress files. You can also use this module to archive multiple files in a single file archive such as TAR or ZIP file.

You can compress and decompress individual files by using [compress.gzip\(options\)](#) and [compress.gunzip\(options\)](#).

You can create an archive by using [compress.createArchiver\(\)](#) and add multiple files to the archive.

Go To Samples 

In This Help Topic

- [N/compress Module Members](#)
- [Archiver Object Members](#)

N/compress Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	compress.Archiver	Object	Server scripts	The functionality for creating archive files. Use compress.createArchiver() to create this object.
Method	compress.createArchiver()	compress.Archiver	Server scripts	Creates a compress.Archiver object.
	compress.gunzip(options)	file.File	Server scripts	Decompresses a file and returns it as a temporary file object.
	compress.gzip(options)	file.File	Server scripts	Compresses a file and returns it as a temporary file object.
Enum	compress.Type	enum	Server scripts	Holds the string values for the archive types.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				Use this enum to set the value of the type parameter of the method Archiver.archive(options) .

Archiver Object Members

The following members are called on the [compress.Archiver](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	Archiver.add(options)	void	Server scripts	Adds a file to be archived.
	Archiver.archive(options)	file.File	Server scripts	Creates an archive with the added files and returns it as a temporary file object.

N/compress Module Script Sample

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/compress module.

Compress and Decompress a File

The following sample compresses and decompresses a file.

i Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  require(['N/compress', 'N/file'], function(compress, file) {
2      var unsavedTxtFile = file.create({
3          fileType: 'PLAINTEXT',
4          name: 'file.txt',
5          contents: 'This is a sample content. This is a sample content. This is a sample content. This is only a sample.'
6      });
7
8      log.debug('Name, size, and contents of the original, uncompressed file are: ');
9      log.debug('Name: ' + unsavedTxtFile.name);
10     log.debug('Size: ' + unsavedTxtFile.size + 'b');
11     log.debug('Contents: ' + unsavedTxtFile.getContents());
12
13     // gzip the file with max compression (level = 9)
14     var gzippedFile = compress.gzip({
15         file: unsavedTxtFile,
16         level: 9
17     });
18
19     log.debug('Name, size, and contents of the gzipped file are: ');
20     log.debug('Name: ' + gzippedFile.name);
21     log.debug('Size: ' + gzippedFile.size + 'b');
22     log.debug('Contents: ' + gzippedFile.getContents().substring(0, 100));
23
24     // gunzip the file

```

```

25     var gunzippedFile = compress.gunzip({
26         file: gzippedFile
27     });
28
29     log.debug('Name, size, and contents of the gunzipped file are: ');
30     log.debug('Name: ' + gunzippedFile.name);
31     log.debug('File size: ' + gunzippedFile.size + 'b');
32     log.debug('Contents: ' + gunzippedFile.getContents());
33 });

```

The following sample creates a ZIP file.



Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  require(['N/compress', 'N/file'], function(compress, file) {
2      // load/create files to be archived
3      var binaryFile = file.load({
4          id: 200
5      });
6      var textFile = file.create({
7          name: 'file.txt',
8          fileType: 'PLAINTEXT',
9          contents: 'This is sample content.'
10     });
11
12     // create an archive as a temporary file object
13     var archiver = compress.createArchiver();
14     archiver.add({
15         file: binaryFile
16     });
17     archiver.add({
18         file: textFile,
19         directory: 'txt/'
20     });
21     var zipFile = archiver.archive({
22         name: 'myarchive.zip'
23     });
24
25     // save the archive to file cabinet
26     zipFile.folder = 123;
27     zipFile.save();
28 });

```

compress.Archiver

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The functionality for creating an archive file. Use compress.createArchiver() to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/compress Module
Methods and Properties	Archiver Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```
1 // Add additional code
2 ...
3 var archiver = compress.createArchiver();
4 ...
5 // Add additional code
```

Archiver.add(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a file to be archived. In the archive, the path to the target file is <code>options.directory</code> or <code>options.file.name</code> . If the <code>options.directory</code> parameter is not specified, the target file is located in the root directory of the archive.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/compress Module
Parent Object	compress.Archiver
Sibling Object Members	Archiver Object Members
Since	2020.2

Parameters



Note: The `options` parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.file</code>	file.File	required	The file to be archived.
<code>options.directory</code>	string	optional	The target directory in the archive. If this parameter is not specified, the file is placed in the root directory of the archive.

Errors

Error Code	Thrown If
COMPRESS_API_DUPLICATE_PATH	A file has already been added using the same target path.

Error Code	Thrown If
COMPRESS_API_FILE_IS_TOO_LARGE	The file cannot be compressed due to size; this happens for files >680MB. Binary/ASCII files have a higher limit of 2GB.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```

1 // Add additional code
2 ...
3 archiver.add({
4     file: myJpegFile,
5     directory: 'pics/'
6 });
7 archiver.add({
8     file: myTxtFile
9 });
10 ...
11 // Add additional code

```

Archiver.archive(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an archive with the added files and returns it as a temporary file object.
Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	25 units
Module	N/compress Module
Parent Object	compress.Archiver
Sibling Object Members	Archiver Object Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The name of the archive file.
options.type	string	optional	The archive type. See the compress.Type enum. This parameter does not need to be specified if options.name has one of the following extensions: ■ .cpio

Parameter	Type	Required / Optional	Description
			■ .tar ■ .tar.gz ■ .tar.bz2 ■ .tgz ■ .tbz2 ■ .zip

Errors

Error Code	Thrown If
COMPRESS_API_UNSUPPORTED_ARCHIVE_TYPE	The options.type parameter is an invalid archive type.
COMPRESS_API_UNRECOGNIZED_ARCHIVE_FILE_EXTENSION	The options.type parameter is not specified and the archive type cannot be determined.
COMPRESS_API_UNABLE_TO_RETRIEVE_FILE_CONTENTS	The contents cannot be retrieved for any of the files to be archived.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var archiver = compress.createArchiver();
4 archiver.add({
5     file: binaryFile
6 });
7 archiver.add({
8     file: textFile,
9     directory: 'txt/'
10 });
11
12 var zipFile = archiver.archive({
13     name: 'myarchive.zip'
14 });
15 var tgzFile = archiver.archive({
16     name: 'myarchive.tar.gz'
17 });
18 // Add additional code

```

compress.gzip(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Compresses a file by using gzip and returns it as a temporary file object. 0 is no compression. 9 is the best compression level.
Returns	file.File
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/compress Module
Module Members	N/compress Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.file	file.File	required	The file to be compressed.
options.level	number	optional	The compression level. 0 is no compression. 9 is the best compression level.

Errors

Error Code	Thrown If
COMPRESS_API_UNABLE_TO_RETRIEVE_FILE_CONTENTS	The contents of the file to be compressed cannot be retrieved.
COMPRESS_API_COMPRESSION_LEVEL_OUT_OF_RANGE	The specified compression level is outside of the valid range (0 - 9).
COMPRESS_API_FILE_IS_TOO_LARGE	The file cannot be compressed due to size; this happens for files >680MB. Binary/ASCII files have a higher limit of 2GB.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var gzippedFile = compress.gzip({
4     file: myFile,
5     level: 9
6 });

```

compress.gunzip(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Decompresses a file that was compressed using gzip and returns it as a temporary file object.
---------------------------	---

Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/compress Module
Module Members	N/compress Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.file	file.File	required	The file to be decompressed.

Errors

Error Code	Thrown If
COMPRESS_API_DECOMPRESS_ERROR	The file cannot be decompressed.
COMPRESS_API_UNABLE_TO_RETRIEVE_FILE_CONTENTS	The contents of the file to be decompressed cannot be retrieved.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var gunzippedFile = compress.gunzip({
4     file: gzippedFile
5 });

```

compress.createArchiver()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a compress.Archiver object that can be used for creating file archives, such as ZIP or TAR files.
Returns	compress.Archiver

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/compress Module
Module Members	N/compress Module Members
Since	2020.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```
1 // Add additional code
2 ...
3 var archiver = compress.createArchiver();
4 ...
5 // Add additional code
```

compress.Type

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the archive types. Use this enum to set the value of the type parameter of the Archiver.archive(options) method.
Module	N/compress Module
Sibling Module Members	N/compress Module Members
Since	2020.2

Values

Value
CPIO
TAR
TBZ2
TGZ

Value
ZIP

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/compress Module Script Sample](#).

```
1 // Add additional code
2 ...
3 var tarArchive = archiver.archive({
4   name: 'myarchive',
5   type: compress.Type.TAR
6 });...
7 // Add additional code
```

N/config Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/config module to access NetSuite configuration settings. The [config.load\(options\)](#) method returns a [record.Record](#) object. Use the [record.Record](#) object members to access configuration settings. You do not need to load the N/record module to do this.

See [config.Type](#) for a list of supported configuration objects.

Go To Samples 

In This Help Topic

- [N/config Module Members](#)

N/config Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	config.load(options)	record.Record	Server scripts	Loads a record.Record object that encapsulates the specified configuration page.
Enum	config.Type	enum	Server scripts	Holds the string values for supported configuration objects. Use this enum to set the value of the NetSuite configuration page you want to access in the config.load(options) method.

N/config Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/config module.

Load the Company Information Configuration Page and Set Field Values

The following sample loads the Company Information configuration record. It then sets the values for the Tax ID Number field and the Employer Identification Number field and saves the record.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Note: The IDs in this sample are placeholders. Replace the values of the Tax ID Number field and the Employer Identification Number field with valid IDs from your NetSuite account.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/config'],
6    function(config) {
7      function setTaxAndEmployerId() {
8        var companyInfo = config.load({
9          type: config.Type.COMPANY_INFORMATION
10         });
11        companyInfo.setValue({
12          fieldId: 'taxid',
13          value: '1122334455'
14        });
15        companyInfo.setValue({
16          fieldId: 'employerid',
17          value: '123456789'
18        });
19        companyInfo.save();
20        companyInfo = config.load({
21          type: config.Type.COMPANY_INFORMATION
22        });
23        var taxid = companyInfo.getValue({
24          fieldId: 'taxid'
25        });
26      }
27      setTaxAndEmployerId();
28    });

```

config.load(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to load a record.Record object that encapsulates the specified NetSuite configuration page. After the configuration page loads, all preference names and IDs are available to get or set. For more information, see the help topic Preference Names and IDs.
---------------------------	--

	You can use the following Record Object Members to get and set preference names and IDs: ■ Record.getField(options) ■ Record.getFields() ■ Record.getText(options) ■ Record.getValue(options) ■ Record.setText(options) ■ Record.setValue(options)
Returns	record.Record
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/config Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	enum	required	The NetSuite configuration page you want to access. Use the config.Type enum to set the value.	2015.2
options.isDynamic	boolean	optional	Determines whether the record is loaded in dynamic mode. ■ If set to true, the record is loaded in dynamic mode. ■ If set to false, the record is loaded in standard mode. For more information, see the help topic SuiteScript 2.x Standard and Dynamic Modes.	2015.2

Error Code	Message	Thrown If
INVALID_RCRD_TYPE	The record type {type} is invalid.	The type argument is invalid or missing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/config Module Script Sample](#).

```
1 //Add additional code
```

```

2  ...
3  var configRecObj = config.load({
4      type: config.Type.COMPANY_INFORMATION
5  });
6  configRecObj.setText({
7      fieldId: 'fiscalmonth',
8      text: 'July'
9  });
10 configRecObj.save();
11 ...
12 //Add additional code

```

config.Type

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported configuration pages. Use this enum to set the value of the NetSuite configuration page you want to access in the config.load(options) method.
Module	N/config Module
Sibling Object Members	N/config Module Members .
Since	2015.2

Values

Value	Configuration Page
ACCOUNTING_PERIODS	Accounting Periods page (Setup> Accounting > Manage Accounting Periods) For more information about the fields on the page, see the help topic Accounting Periods .
ACCOUNTING_PREFERENCES	Accounting Preferences page (Setup > Accounting > Accounting Preferences) For more information about the fields on the page, see the help topic Accounting Preferences .
COMPANY_INFORMATION	Company Information page (Setup > Company > Company Information) For more information about the fields on the page, see the help topic Company Information Preferences .
COMPANY_PREFERENCES	General Preferences page (Setup > Company > General Preferences) For more information about the fields on the page, see the help topic General Preferences .
FEATURES	Enable Features page (Setup > Company > Enable Features) For more information about feature names and IDs, see .

Value	Configuration Page
MANUFACTURING_PREFERENCES	Set Manufacturing preferences page (Home > Manufacturing > Manufacturing Preferences). For more information about the fields on the page, see the help topic Manufacturing Preferences .
TAX_PERIODS	Tax Periods page (Setup > Accounting > Manage Tax Periods) For more information about the fields on the page, see the help topic Tax Periods .
TIME_POST	For additional information, see the help topic Posting Time Transactions .
TIME_VOID	For additional information, see the help topic Posting Time Transactions .
USER_PREFERENCES	Set Preferences page (Home > Set Preferences) For more information about the fields on the page, see the help topic User Preferences .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/config Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var configRecObj = config.load({
4     type: config.Type.COMPANY_INFORMATION
5 });
6 configRecObj.setText({
7     fieldId: 'fiscalmonth',
8     text: 'July'
9 });
10 configRecObj.save();
11 ...
12 //Add additional code

```

N/crypto Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use N/crypto module to perform hashing, hash-based message authentication (hmac), and symmetrical encryption functions.

When the N/crypto module is used, SuiteScript also loads [N/encode Module](#).

Go To Samples

In This Help Topic

- [N/crypto Module Members](#)
- [Cipher Object Members](#)
- [CipherPayload Object Members](#)

- [Decipher Object Members](#)
- [Hash Object Members](#)
- [Hmac Object Members](#)
- [SecretKey Object Members](#)

N/crypto Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	crypto.Cipher	Object	Server scripts	Encapsulates a cipher.
	crypto.CipherPayload	Object	Server scripts	Encapsulates a cipher payload.
	crypto.Decipher	Object	Server scripts	Encapsulates a decipher.
	crypto.Hash	Object	Server scripts	Encapsulates a hash.
	crypto.Hmac	Object	Server scripts	Encapsulates an hmac.
	crypto.SecretKey	Object	Server scripts	Encapsulates a secret key handle.
Method	crypto.checkPasswordField(options)	boolean	Server scripts	Checks whether a password in a record corresponds to the password entered by the user.
	crypto.createCipher(options)	Object	Server scripts	Creates and returns a new crypto.Cipher Object.
	crypto.createDecipher(options)	Object	Server scripts	Creates and returns a new crypto.Decipher object.
	crypto.createHash(options)	Object	Server scripts	Creates and returns a new crypto.Hash Object.
	crypto.createHmac(options)	Object	Server scripts	Creates and returns a new crypto.Hmac Object.
	crypto.createSecretKey(options)	Object	Server scripts	Creates and returns a new crypto.SecretKey Object.
Enum	crypto.EncryptionAlg	string (read-only)	Server scripts	Holds the string values for supported encryption algorithms. Use this enum to set the options.algorithm parameter for crypto.createCipher(options) .
	crypto.HashAlg	string (read-only)	Server scripts	Holds the string values for supported hashing algorithms. Use this enum to set the options.algorithm parameter for crypto.createHash(options) and crypto.createHmac(options) .
	crypto.Padding	string (read-only)	Server scripts	Holds the string values for supported cipher padding. Use this enum to set the options.padding parameter for crypto.createCipher(options) and crypto.createDecipher(options) .

Cipher Object Members

The following members are called on [crypto.Cipher](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Cipher.update(options)	void	Server scripts	Updates the clear data with the specified encoding.
	Cipher.final(options)	Object	Server scripts	Returns the cipher data.

CipherPayload Object Members

The following members are called on [crypto.CipherPayload](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	CipherPayload.ciphertext	string	Server scripts	The result of the ciphering process.
	CipherPayload.iv	number	Server scripts	An initialization vector.

Decipher Object Members

The following members are called on [crypto.Decipher](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Decipher.final(options)	string	Server scripts	Returns the clear data.
	Decipher.update(options)	void	Server scripts	Updates decipher data with the specified encoding.

Hash Object Members

The following members are called on [crypto.Hash](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Hash.digest(options)	string	Server scripts	Calculates the digest of the data to be hashed.
	Hash.update(options)	void	Server scripts	Updates the clear data with the encoding specified.

Hmac Object Members

The following members are called on [crypto.Hmac](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Hmac.digest(options)	string	Server scripts	Gets the computed digest.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Hmac.update(options)	void	Server scripts	Updates the clear data with the encoding specified.

SecretKey Object Members

The following members are called on [crypto.SecretKey](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SecretKey.guid	string	Server scripts	The GUID associated with the secret key.
	SecretKey.encoding	string	Server scripts	The encoding used for the clear text value of the secret key.
	SecretKey.secret	string	Server scripts	The script ID of an API secret stored at Setup > Company > API Secrets.

N/crypto Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/crypto module.

- [Create a Secure Key Using SHA512](#)
- [Create a Suitelet to Request User Credentials, Create a Secret Key, and Encode a Sample String](#)

Create a Secure Key Using SHA512

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample demonstrates the APIs needed to generate a secure key using the SHA512 hashing algorithm. The GUID in this sample is a placeholder. You must replace it with a valid value from your NetSuite account. To create a real password GUID, obtain a password value from a credential field on a form. For more information, see [Form.addCredentialField\(options\)](#). Also see the [Create a Suitelet to Request User Credentials, Create a Secret Key, and Encode a Sample String](#) that shows how to create a form field that generates a GUID.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

2  * @NApiVersion 2.1
3  */
4  /**This sample demonstrates the APIs needed to generate a secure key using the SHA512 hashing algorithm.
5  * The mySecret variable is a placeholder that must be replaced with a valid secret from your NetSuite account.
6  * For information about secrets management page, see Account Administration > Authentication > Secrets Management
7  * in the Help Center.
8  */
9
10 require(['N/crypto', 'N/encode', 'N/runtime'], (crypto, encode, runtime) => {
11   function createSecureKeyWithHash() {
12     let mySecret = 'custsecret_my_secret'; //secret id take from secrets management page
13
14     let sKey = crypto.createSecretKey({
15       secret: mySecret,
16       encoding: encode.Encoding.UTF_8
17     });
18
19     let hmacSHA512 = crypto.createHmac({
20       algorithm: crypto.HashAlg.SHA512,
21       key: sKey
22     });
23
24     hmacSHA512.update({
25       input: inputString,
26       inputEncoding: encode.Encoding.BASE_64
27     });
28
29     let digestSHA512 = hmacSHA512.digest({
30       outputEncoding: encode.Encoding.HEX
31     });
32   }
33   createSecureKeyWithHash();
34 });

```

Create a Suitelet to Request User Credentials, Create a Secret Key, and Encode a Sample String

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample creates a simple Suitelet that requests user credentials, creates a secret key, and encodes a sample string.



Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).



Note: The default maximum length for a secret key field is 32 characters. If needed, use the `Field.maxLength` property to change this value.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget', 'N/runtime', 'N/crypto', 'N/encode'], (serverWidget, runtime, crypto, encode) => {
6     function onRequest(option) {

```

```

7      if (option.request.method === 'GET') {
8          let form = serverWidget.createForm({
9              title: 'My Credential Form'
10         });
11         let skField = form.addSecretKeyField({
12             id: 'mycredential',
13             label: 'Credential',
14             restrictToScriptIds: [runtime.getCurrentScript().id],
15             restrictToCurrentUser: false
16         });
17         skField.maxLength = 200;
18
19         form.addSubmitButton();
20
21         option.response.writePage(form);
22     } else {
23         let form = serverWidget.createForm({
24             title: 'My Credential Form'
25         });
26
27         const inputString = "YWJjZGVmZwo=";
28         let myGuid = option.request.parameters.mycredential;
29
30         // Create the key
31         let sKey = crypto.createSecretKey({
32             guid: myGuid,
33             encoding: encode.Encoding.UTF_8
34         });
35
36         try {
37             let hmacSha512 = crypto.createHmac({
38                 algorithm: 'SHA512',
39                 key: sKey
40             });
41             hmacSha512.update({
42                 input: inputString,
43                 inputEncoding: encode.Encoding.BASE_64
44             });
45             let digestSha512 = hmacSha512.digest({
46                 outputEncoding: encode.Encoding.HEX
47             });
48         } catch (e) {
49             log.error({
50                 title: 'Failed to hash input',
51                 details: e
52             });
53         }
54
55         form.addField({
56             id: 'result',
57             label: 'Your digested hash value',
58             type: 'textarea'
59         }).defaultValue = digestSha512;
60
61         option.response.writePage(form);
62     }
63 }
64 return {
65     onRequest: onRequest
66 };
67 });

```

crypto.Cipher

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a cipher.
---------------------------	------------------------

Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto Module
Methods and Properties	Cipher Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myCipher = crypto.createCipher({ // myCipher is a crypto.Cipher object
4   algorithm: crypto.EncryptionAlg.AES,
5   key: sKey
6 });
7 ...
8 //Add additional code

```

Cipher.final(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the cipher data. Sets the output encoding for the crypto.CipherPayload object.
Returns	A crypto.CipherPayload object.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.outputEncoding	enum	optional	The output encoding for a crypto.CipherPayload object. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.HEX</code> .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 myCipher = crypto.createCipher({ // myCipher is a crypto.Cipher object
4   algorithm: crypto.EncryptionAlg.AES,
5   key: sKey
6 });
7
8 var cipherPayload = myCipher.final({
9   outputEncoding: encode.Encoding.BASE_64
10 });
11 ...
12 //Add additional code

```

Cipher.update(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the clear data with the specified encoding.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string	required	The clear data to be updated.
options.inputEncoding	enum	optional	The input encoding. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.UTF-8</code> .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code

```

```

2  ...
3  myCipher = crypto.createCipher({ // cipher is a crypto.Cipher object
4      algorithm: crypto.EncryptionAlg.AES,
5      key: sKey
6  });
7
8  var reencoded = myCipher.update({
9      input: 'Carrot cake gummi bears'
10 });
11 ...
12 //Add additional code

```

crypto.CipherPayload

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a cipher payload.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto Module
Methods and Properties	CipherPayload Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1  //Add additional code
2  ...
3  myCipher = crypto.createCipher({
4      algorithm: crypto.EncryptionAlg.AES,
5      key: sKey
6  });
7
8  var cipherPayload = myCipher.final({ // cipherPayload is a crypto.CipherPayload object
9      outputEncoding: encode.Encoding.HEX
10 });
11 ...
12 //Add additional code

```

CipherPayload.ciphertext

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The result of the ciphering process. For example, to take the cipher payload and send it to another system.
Type	string
Module	N/crypto Module

Since	2015.2
-------	--------

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4   title: "CipherPayload.ciphertext from Cipher.final: ",
5   details: cipherPayload.ciphertext
6 });
7 ...
8 //Add additional code

```

CipherPayload.iv

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Initialization vector for the cipher payload. You can pass this value to crypto.createDecipher(options). Note: This property is encoded in <code>encode.Encoding.HEX</code> regardless of the <code>options.inputEncoding</code> parameter value in the <code>cipher.final</code> method.
Type	string
Module	N/crypto Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4   title: "CipherPayload.iv from Cipher.final: ",
5   details: cipherPayload.iv
6 });
7 ...
8 //Add additional code

```

crypto.Decipher

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a decipher. This object has methods that decrypt.
---------------------------	--

Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto Module
Methods and Properties	Decipher Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 myDecipher = crypto.createDecipher({ // myDecipher is a crypto.Decipher object
4     algorithm: crypto.EncryptionAlg.AES,
5     key: sKey
6 });
7 ...
8 //Add additional code

```

Decipher.final(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the clear data.
Returns	string
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.outputEncoding	string	optional	Specifies the encoding for the output. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.UTF-8</code> .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var decipher1 = Decipher.final({ // decipher1 is the clear data
4     outputEncoding: encode.Encoding.HEX
5 });
6 ...
7 //Add additional code
```

Decipher.update(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates cipher data with the specified encoding.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string	required	The data to update.
options.inputEncoding	string	optional	Specifies the encoding of the input data. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.HEX</code> .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```
1 //Add additional code
```

```

2  ...
3  var decipher1 = Decipher.update({
4      input: '73616d706c65737472696e67',
5      inputEncoding: encode.Encoding.UTF_8
6  });
7  ...
8  //Add additional code

```

crypto.Hash

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a hash.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto Module
Methods and Properties	Hash Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var hashObj = crypto.createHash({
4      algorithm: crypto.HashAlg.SHA256
5  });
6  ...
7  //Add additional code

```

Hash.digest(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Calculates the digest of the data to be hashed.
Returns	A hash value as a string.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module

Since	2015.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.outputEncoding	string	optional	The output encoding. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.HEX</code> .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var digestSample = hashObj.digest({
4     outputEncoding: encode.Encoding.BASE_64
5 });
6 ...
7 //Add additional code

```

Hash.update(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates clear data with the encoding specified.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string	required	The data to be updated.

Parameter	Type	Required / Optional	Description
options.inputEncoding	string	optional	The input encoding. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.UTF_8</code> .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var inputString = 'Lemon drops ice cream jelly marzipan cake';
4 hashSample.update({
5     input: inputString
6 });
7 ...
8 //Add additional code

```

crypto.Hmac

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates an hmac.
Returns	an empty object {}
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto Module
Methods and Properties	Hmac Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hmacSHA512 = crypto.createHmac({
4     algorithm: crypto.HashAlg.SHA512,
5     key: sKey
6 });
7 ...
8 //Add additional code

```

Hmac.digest(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the computed digest.
Returns	An hmac value as a string.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.outputEncoding	string	optional	Specifies the encoding of the output string. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.HEX</code> .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var digestSHA512 = hmacSHA512.digest({
4   outputEncoding: encode.Encoding.BASE_64
5 });
6 ...
7 //Add additional code

```

Hmac.update(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the clear data with the encoding specified.
Returns	void

Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string	required	The hmac data to be updated.
options.inputEncoding	enum	optional	The input encoding. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.UTF_8</code> .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 hmacSHA512.update({
4     input: inputString,
5     inputEncoding: encode.Encoding.BASE_64
6 });
7 ...
8 //Add additional code

```

crypto.SecretKey

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the handle to the key. The handler does not store the key value. It points to the key stored within the NetSuite system. The GUID or secret is also required to find the key.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto Module
Methods and Properties	SecretKey Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var sKey = crypto.createSecretKey({
4     guid: '284CFB2D225B1D76FB94D150207E49DF',
5     encoding: encode.Encoding.UTF_8
6 });
7 ...
8
9 //Add additional code
```

SecretKey.encoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The encoding used for the clear text value of the secret key.
Type	string
Module	N/crypto Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```
1 //Add additional code
2 ...
3 log.debug({
4     title: "Secret Key Encoding: ",
5     details: sKey.encoding
6 });
7 ...
8 //Add additional code
```

Secretkey.guid

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The GUID associated with the secret key.
Type	string
Module	N/crypto Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```
1 //Add additional code
2 ...
3 log.debug({
4     title: "Secret Key GUID: ",
5     details: sKey.guid
6 });
7 ...
8 //Add additional code
```

SecretKey.secret

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The script ID of an API secret. For more information, see the help topic Secrets Management .
Type	string
Module	N/crypto Module
Since	2021.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var sKey = crypto.createSecretKey({
4     secret: 'custsecret_sample',
5     encoding: encode.Encoding.UTF_8
6 });
7 log.debug({
8     title: 'Secret ID',
9     details: sKey.secret
10 });
11 ...
12
13 //Add additional code
```

crypto.checkPasswordField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Checks whether a password in a record corresponds to the password entered by the user. Use this method instead of Record.getValue(options) or CurrentRecord.getValue(options) on a custom password field. You should no longer use those methods for custom password fields. This method provides a more secure way to check custom password fields.
---------------------------	---

Returns	boolean
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2021.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	ID of the password field.	2021.1
options.line	number	optional	Zero-based line index of the password field if the password is on a line.	2021.1
options.recordId	number	required	ID of the record that has the password field.	2021.1
options.recordType	string	required	Type of record that has the password field.	2021.1
options.sublistId	string	optional	ID of the sublist if the password field is on a sublist line.	2021.1
options.value	string	required	Input password value to be checked against the password stored in the record.	2021.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var options = {
4   recordType: record.Type.SALES_ORDER,
5   recordId: 6,
6   fieldId: 'custitem1',
7   sublistId: 'item',
8   line: 0,
9   value: 'theenteredpassword'
10 };
11 if (crypto.checkPasswordField(options)) {
12   log.debug('True');
13 } else {
14   log.debug('False');
15 }
16 ...
17 //Add additional code

```

crypto.createCipher(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new crypto.Cipher object. Note: The blockCipherMode is automatically set to CBC.
Returns	A crypto.Cipher object
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.algorithm	string	required	The encryption algorithm. Use the crypto.EncryptionAlg enum to set the value.	2015.2
options.key	object	required	The crypto.SecretKey object. Note: When using the crypto.SecretKey object for an AES algorithm, the length of the text (secret key) that is used to generate the GUID must be 16, 24, or 32 characters.	2015.2
options.padding	string	optional	The padding for the cipher text. Use the crypto.Padding enum to set the value. The default value is crypto.Padding.PKCS5Padding .	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var cipher = crypto.createCipher({
4     algorithm: crypto.EncryptionAlg.AES,
5     key: sKey,
6     padding: crypto.Padding.PKCS5Padding
7 });

```

```

8 | ...
9 | //Add additional code

```

crypto.createDecipher(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new crypto.Decipher object. Note: The blockCipherMode is automatically set to CBC.
Returns	A crypto.Decipher object.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.algorithm	string	required	The hash algorithm. Use the crypto.EncryptionAlg enum to set the value.	2015.2
options.key	object	required	The crypto.SecretKey object used for encryption. Note: When using the crypto.SecretKey object for an AES algorithm, the length of the text (secret key) that is used to generate the GUID must be 16, 24, or 32 characters.	2015.2
options.padding	object	optional	The padding for the cipher. Use the crypto.Padding enum to set the value.	2015.2
options.iv	string	required	The initialization vector that was used for encryption. You can use a CipherPayload.iv value.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 | //Add additional code
2 | ...

```

```

3 var decipher = crypto.createDecipher({
4   algorithm: crypto.EncryptionAlg.AES,
5   key: sKey,
6   padding: NoPadding,
7   iv: '2311141720'
8 });
9 ...
10 //Add additional code

```

crypto.createHash(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new crypto.Hash object.
Returns	A crypto.Hash object.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.algorithm	string	required	The hash algorithm. Use the crypto.HashAlg enum to set the value.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hashObj = crypto.createHash({
4   algorithm: crypto.HashAlg.SHA256
5 });
6 ...
7 //Add additional code

```

crypto.createHmac(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new crypto.Hmac object.
---------------------------	---

Returns	A crypto.Hmac object.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.algorithm	string	required	The hash algorithm. Use the crypto.HashAlg enum to set the value.	2015.2
options.key	object	required	The crypto.SecretKey object.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hmacObj = crypto.createHmac({
4     algorithm: HashAlg.SHA256,
5     key: sKey
6 });
7 ...
8 //Add additional code

```

crypto.createSecretKey(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new crypto.SecretKey object. This method can take a GUID or the script ID of a secret stored at Setup > Company > API Secrets. Use Form.addCredentialField(options) to generate a GUID value. For more information about API Secrets, see the help topic Secrets Management.  Note: When using the crypto.SecretKey object for an AES algorithm, the length of the text (secret key) that is used to generate the GUID must be 16, 24, or 32 characters.
Returns	A crypto.SecretKey object

Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.guid	string	required if secret is not specified	A GUID used to generate a secret key. The GUID can resolve to either data or metadata. You can create a GUID using Form.addCredentialField(options) . This is only required if a secret is not provided. You cannot use the guid parameter in combination with the secret parameter.	2015.2
options.secret	string	required if options.guid is not specified	The script ID of the secret used for authentication. You can store secrets at Setup > Company > API Secrets. For more information, see the help topic Secrets Management . This is only required if GUID is not provided. You cannot use the secret parameter in combination with the passwordGuid parameter.	2021.1
options.encoding	enum	optional	Specifies the encoding for the SecureKey. Use the encode.Encoding enum to set the value. The default value is <code>encode.Encoding.HEX</code> .	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var secretKey = crypto.createSecretKey({
4   encoding: encode.Encoding.UTF_8,
5   guid: '284CFB2D225B1D76FB94D150207E49DF'
6 });
7 ...
8 //Add additional code

```

crypto.EncryptionAlg

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported encryption algorithms. This enum can be used to set the options.algorithm parameter for crypto.createCipher(options) .
Module	N/crypto Module
Sibling Module Members	N/crypto Module Members
Since	2015.2

Values

Value
AES

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var cipher = crypto.createCipher({
4   algorithm: crypto.EncryptionAlg.AES,
5   key: sKey
6 });
7 ...
8 //Add additional code

```

crypto.HashAlg

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported hashing algorithms. Use this enum to set the value of the options.algorithm parameter for crypto.createHash(options) and crypto.createHmac(options) .
Module	N/crypto Module

Sibling Object Members	N/crypto Module Members
Since	2015.2

Values

Value
SHA256
SHA512

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hmacSHA512 = crypto.createHmac({
4     algorithm: crypto.HashAlg.SHA512,
5     key: sKey
6 });
7 ...
8 //Add additional code

```

crypto.Padding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported cipher padding. This enum can be used to set the <code>options.padding</code> parameter for crypto.createCipher(options) and crypto.createDecipher(options) .
Module	N/crypto Module
Sibling Object Members	N/crypto Module Members
Since	2015.2

Values

Value
NoPadding
PKCS5Padding

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var cipher = crypto.createCipher({
4   algorithm: crypto.EncryptionAlg.AES,
5   key: sKey,
6   padding: crypto.Padding.NoPadding
7 });
8 ...
9 //Add additional code

```

N/crypto/certificate Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/crypto/certificate module to sign XML documents or strings with digital certificates using asymmetric cryptography. You can also use this module to create signer and verifier objects and verify signed documents.

[Go To Samples {...}](#)

In This Help Topic

- [N/crypto/certificate Module Members](#)
- [SignedXml Object Members](#)
- [Signer Object Members](#)
- [Verifier Object Members](#)

N/crypto/certificate Module Members

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
Object	certificate.SignedXml	Object	Server scripts	Encapsulates an XML string that has been digitally signed. Use certificate.signXml(options) to create this object.
	certificate.Signer	Object	Server scripts	Encapsulates a created signature (signer) for plain strings. Use certificate.createSigner(options) to create this object.
	certificate.Verifier	Object	Server scripts	Encapsulates a created verifier for verifying plain string signatures.

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
				Use certificate.createVerifier(options) to create this object.
Method	certificate.createSigner(options)	certificate.Signer	Server scripts	Creates a certificate.Signer object for signing plain strings.
	certificate.createVerifier(options)	certificate.Verifier	Server scripts	Creates a certificate.Verifier object for verifying signatures of plain strings.
	certificate.signXml(options)	certificate.SignedXml	Server scripts	Signs the input XML string using the Certificate ID. Returns the certificate.SignedXml as a string. Note: Formatting, such as line breaks, is disabled in signatures.
	certificate.verifyXml(SignedXml(options))	void	Server scripts	Verifies the signature in the SignedXml.asFile() file.
Enum	certificate.HashAlg	enum	Server scripts	Holds the string values for hash algorithm types. Use this enum to set the <code>option.algorithm</code> property values for the certificate.createSigner(options) , certificate.createVerifier(options) , certificate.signXml(options) methods.

SignedXml Object Members

The following members are called on the [certificate.SignedXml](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	SignedXml.asFile()	file.File	Server scripts	Returns the signed XML as a file object.
	SignedXml.asString()	string	Server scripts	Returns the signed XML as a string.
	SignedXml.asXml()	xml.Document	Server scripts	Returns the signed XML as an XML document. You can use the N/xml Module with this document to access elements and attributes in the XML.

Signer Object Members

The following members are called on the [certificate.Signer](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Signer.sign(options)	void	Server scripts	Signs the string and returns the signature.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Signer.update(options)	void	Server scripts	Updates the input string to be signed. The string can be encoded.

Verifier Object Members

The following members are called on the [certificate.Verifier](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Verifier.update(options)	void	Server scripts	Updates the string to be verified against specified certificate.
	Verifier.verify(options)	void	Server scripts	Verifies the string against a provided signature using specified certificate.

N/crypto/certificate Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/crypto/certificate module:

- [Load an XML File from the File Cabinet and Sign It Using a Digital Certificate](#)
- [Create Signer and Verifier Objects](#)

Load an XML File from the File Cabinet and Sign It Using a Digital Certificate

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample loads an XML file from the File Cabinet and signs it using the digital certificate with internal ID 'custcertificate1'. Note that this sample uses a hard-coded value for the file id. You should change this value to a valid file id from your account.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

1 | /**

```

2  */
3  */
4
5  require(['N/crypto/certificate', 'N/file'], (certificate, file) => {
6      // Load the file from the File Cabinet.
7      // Note that the id value is hard-coded in this sample, and you should use
8      // a valid file id from your account.
9      let infNFe = file.load({
10         id: 922
11     });
12     let signedXml = certificate.signXml({
13         algorithm: certificate.HashAlg.SHA256,
14         certId: 'custcertificate1',
15         rootTag: 'infNFe',
16         xmlString: infNFe.getContents()
17     });
18     certificate.verifyXMLSignature({
19         signedXml: signedXml,
20         rootTag: 'infNFe'
21     });
22 });

```

Create Signer and Verifier Objects

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample creates a [certificate.Signer](#) object, signs it, and then creates a [certificate.Verifier](#) object and verifies the signer object.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/crypto/certificate'], (certificate) => {
6      let signer = certificate.createSigner({
7         certId: 'custcertificate1',
8         algorithm: certificate.HashAlg.SHA256
9     });
10     signer.update('test');
11
12     let result = signer.sign();
13     let verifier = certificate.createVerifier({
14         certId: 'custcertificate1',
15         algorithm: certificate.HashAlg.SHA256
16     });
17     verifier.update('test');
18     verifier.verify(result);
19 });

```

certificate.SignedXml

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates an XML string that has been digitally signed. This object is returned by the certificate.signXml(options) method.
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto/certificate Module
Methods and Properties	SignedXml Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // signedXml is a certificate.SignedXml object
4 var signedXml = certificate.signXml({
5     algorithm: certificate.HashAlg.SHA256,
6     certId: 'custcertificate1',
7     rootTag: 'infNFe',
8     xmlString: infNFe.getContents()
9 });
10
11 // You can use the certificate.SignedXml object to verify the signature
12 certificate.verifyXMLSignature({
13     signedXml: signedXml,
14     rootTag: 'infNFe'
15 });
16 ...
17 //Add additional code

```

SignedXml.asFile()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the signed XML as a file.
Returns	file.File
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	certificate.SignedXml

Sibling Object Members	SignedXml Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // signedXml is a certificate.SignedXml object
4 var signedXml = certificate.signXml({
5     algorithm: certificate.HashAlg.SHA256,
6     certId: 'custcertificate1',
7     rootTag: 'infNFe',
8     xmlString: infNFe.getContents()
9 });
10
11 // signed_AsFile will be a file.File object
12 var signed_AsFile = signedXml.asFile();
13 ...
14 //Add additional code

```

SignedXml.asString()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the signed XML as a string.
Returns	string
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	certificate.SignedXml
Sibling Object Members	SignedXml Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // signedXml is a certificate.SignedXml object
4 var signedXml = certificate.signXml({
5     algorithm: certificate.HashAlg.SHA256,
6     certId: 'custcertificate1',
7     rootTag: 'infNFe',

```

```

8      xmlString: infNFe.getContents()
9    });
10
11    // signed_AsString will be a string
12    var signed_AsString = signedXml.asString();
13    ...
14    //Add additional code

```

SignedXml.asXml()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the signed XML as an XML document.
Returns	xml.Document
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	certificate.SignedXml
Sibling Object Members	SignedXml Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // signedXml is a certificate.SignedXml object
4 var signedXml = certificate.signXml({
5     algorithm: certificate.HashAlg.SHA256,
6     certId: 'custcertificate1',
7     rootTag: 'infNFe',
8     xmlString: infNFe.getContents()
9 });
10
11 // signed_AsXml will be an xml.Document object
12 var signed_AsXml = signedXml.asXml();
13 ...
14 //Add additional code

```

certificate.Signer

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a created signature (signer) for plain strings. This object is returned by the certificate.createSigner(options) method.
---------------------------	--

Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto/certificate Module
Methods and Properties	Signer Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // mySigner is a certificate.Signer object
4 var mySigner = certificate.createSigner({
5     certId: 'custcertificate1',
6     algorithm: certificate.HashAlg.SHA512
7 });
8 ...
9 //Add additional code

```

Signer.update(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the input string to be signed. The string can be encoded.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	certificate.Signer
Sibling Object Members	Signer Object Members
Since	2019.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string https.SecureString	required	The string to update.

Parameter	Type	Required / Optional	Description
options.inputEncoding	string	optional	Encoding of the string to sign (for example, UTF-8, ISO_8859_1, ASCII). The default value is UTF-8.  Note: This must be a text value. Values from encode.Encoding (N/encode Module module) are not accepted.

Errors

Error Code	Thrown If
SSS_UNSUPPORTED_ENCODING	The value for options.inputEncoding is invalid. This must be a text value, such as UTF-8, ISO_8859_1, ASCII. Values from encode.Encoding (N/encode module) are not valid.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // mySigner will be a certificate.Signer object
4 var mySigner = certificate.createSigner({
5     certId: 'custcertificate1',
6     algorithm: certificate.HashAlg.SHA256
7 });
8
9 mySigner.update("test");
10
11 // result is the signature as a string
12 var result = mySigner.sign();
13 ...
14 //Add additional code

```

Signer.sign(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Signs the string and returns the signature. Formatting, such as line breaks, is disabled in signatures. For ECDSA based algorithms, such as ES256, this method returns a signature in ASN.1 DER format by default. To retrieve a raw signature for ECDSA based algorithms, set the options.useRawFormatForECDSA parameter to true.
Returns	string
Supported Script Types	Server scripts

	For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	certificate.Signer
Sibling Object Members	Signer Object Members
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.outputEncoding	string	optional	Encoding of the signed string in Base64 format.
options.useRawFormatForECDSA	boolean	optional	Returns ECDSA signatures in raw format. Default value is set to false.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // mySigner will be a certificate.Signer object
4 var mySigner = certificate.createSigner({
5     certId: 'custcertificate1',
6     algorithm: certificate.HashAlg.SHA256
7 });
8
9 mySigner.update("test");
10
11 // result is the signature as a string
12 var result = mySigner.sign({
13     outputEncoding: encode.Encoding.BASE_64_URL_SAFE,
14     useRawFormatForECDSA: true
15 });
16 ...
17 //Add additional code

```

certificate.Verifier

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a created verifier for verifying plain string signatures. This object is returned by the certificate.createVerifier(options) method.
Supported Script Types	Server scripts

	For additional information, see the help topic SuiteScript 2.x Script Types .
Module	N/crypto/certificate Module
Methods and Properties	Verifier Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // myVerifier will be a certificate.Verifier object
4 var myVerifier = certificate.createVerifier({
5     certId: 'custcertificate1',
6     algorithm: certificate.HashAlg.SHA256
7 });
8
9 myVerifier.update('test');
10 myVerifier.verify(result);
11 ...
12 //Add additional code

```

Verifier.update(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the string to be verified against a specified certificate.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	Parameters
Sibling Object Members	Verifier Object Members
Since	2019.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string	required	The string to verify.

Parameter	Type	Required / Optional	Description
options.inputEncoding	string	optional	Encoding of the string to verify. The default value is UTF-8.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // myVerifier will be certificate.Verifier object
4 var myVerifier = certificate.createVerifier({
5     certId: 'custcertificate1',
6     algorithm: certificate.HashAlg.SHA256
7 });
8
9 myVerifier.update('test');
10 myVerifier.verify(result);
11 ...
12 //Add additional code

```

Verifier.verify(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Verifies a string against a provided signature using a specified certificate. You can create a verifier object using the certificate.createVerifier(options) method.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/certificate Module
Parent Object	Parameters
Sibling Object Members	Verifier Object Members
Since	2019.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.signature	string	required	The signature to be verified.
options.signatureEncoding	string	optional	The signature's encoding in Base64 format.

Errors

Error Code	Thrown If
INVALID_SIGNATURE	Signature is not verified. This can occur if the certificate or hash algorithm is not correct in the Verifier object or the signature is not valid for the supplied string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // myVerifier will be certificate.Verifier object
4 var myVerifier = certificate.createVerifier({
5     certId: 'custcertificate1',
6     algorithm: certificate.HashAlg.SHA256
7 });
8 myVerifier.update('test');
9 myVerifier.verify(result);
10 ...
11 //Add additional code

```

certificate.createSigner(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates the signer object for signing plain strings.
Returns	certificate.Signer
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/crypto/certificate Module
Since	2019.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.certId	string	required	The script ID of the digital certificate.
options.algorithm	string	required	The hash algorithm. Set this property using the certificate.HashAlg enum.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var mySigner = certificate.createSigner({
4   certId: 'custcertificate1',
5   algorithm: certificate.HashAlg.SHA256
6 });
7 ...
8 //Add additional code
```

certificate.createVerifier(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates the verifier object for verifying signatures of plain strings.
Returns	A Parameters object
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/crypto/certificate Module
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.certId	string	required	The script ID of the digital certificate.
options.algorithm	string	required	The hash algorithm. Set this property using the certificate.HashAlg enum.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var myVerifier = certificate.createVerifier({
```

```

4   certId: 'custcertificate1',
5   algorithm: certificate.HashAlg.SHA256
6 });
7 ...
8 //Add additional code

```

certificate.verifyXmlSignature(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Verifies a signature.
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/crypto/certificate Module
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.certId	string	optional	The script ID for the digital certificate.
options.rootTag	string	required	Signed root XML tag.
options.signedXml	string	required	Signed XML

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySigner = certificate.createSigner({
4   certId: 'custcertificate1',
5   algorithm: certificate.HashAlg.SHA256
6 });
7
8 var signedXml = certificate.signXml({
9   algorithm: certificate.HashAlg.SHA256,
10  certId: 'custcertificate1',
11  rootTag: 'infNFe',
12  xmlString: infNFe.getContents()
13 });
14
15 var signed_AsString = signedXml.asString()

```

```

16
17 certificate.verifyXmlSignature({
18     certId: 'custcertificate1',
19     rootTag: 'infNFe',
20     signedXml: signed_AsString
21 });
22 ...
23 //Add additional code

```

certificate.signXml(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Signs an input XML string using a certificate ID. Formatting, such as line breaks, is disabled.
Returns	certificate.SignedXml
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/crypto/certificate Module
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.xmlString	string	required	Input XML string
options.certId	string	required	Certificate ID
options.algorithm	string	required	The hash algorithm. Set this property using the certificate.HashAlg enum.
options.rootTag	string	required	Root tag of XML section to sign.
options.insertionTag	string	optional	Tag where the signature should be inserted.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var signedXml = certificate.signXml({
4     algorithm: certificate.HashAlg.SHA256,

```



```

5   certId: 'custcertificate1',
6   rootTag: 'infNFe',
7   xmlString: infNFe.getContents()
8 });
9 ...
10 //Add additional code

```

certificate.HashAlg

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the hash algorithm types. Supported digest methods are SHA256, SHA384, and SHA512 for RSA and ECDSA encryption algorithms and SHA256 for DSA. Use this enum to set the <code>option.algorithm</code> property values for the certificate.createSigner(options), certificate.createVerifier(options), certificate.signXml(options) methods.
Module	N/crypto/certificate Module
Sibling Module Members	N/crypto/certificate Module Members
Since	2019.1

Values

Value
SHA256
SHA384
SHA512

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/certificate Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var signer = certificate.createSigner({
4   certId: 'custcertificate1',
5   algorithm: certificate.HashAlg.SHA256
6 });
7 ...
8 //Add additional code

```

N/crypto/random Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The N/crypto/random module is available for both client and server scripts, but server scripts need to use SuiteScript 2.1.

If you cannot update your server script code to SuiteScript 2.1, consider implementing a small RESTlet in SuiteScript 2.1 that uses N/crypto/random module and consuming it from your script with [https.requestRestlet\(options\)](#)

Go To Samples {..}

In This Help Topic

Use the N/crypto/random module to provide cryptographically-secure, pseudorandom generator methods.

N/crypto/random Module Members

N/crypto/random Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	random.generateBytes(options)	Uint8Array	Client and server scripts	Generates cryptographically strong pseudorandom set of bytes.
	random.generateInt(options)	number	Client and server scripts	Method used to generate cryptographically strong pseudorandom number.
	random.generateUUID()	string	Client and server scripts	Method used to generate a v4 Universally Unique Identifier using a cryptographically secure random number generator.

N/crypto/random Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1

The following script samples demonstrate how to use the features of the N/crypto/random module:

Create a RESTlet to roll a dice

The following sample shows how to generate a random integer using the define function:

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType restlet
4   */
5   define(['N/crypto/random'], function(random) {
6       return {
7           get : function() {
8               return JSON.stringify({
9                   number: random.generateInt({min: 1, max: 7})
10              })
11          }
12      }
13  });

```

random.generateBytes(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Cryptographically strong pseudorandom data.
Returns	Uint8Array
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/random Module
Since	2023.1

Parameters

Parameter	Type	Required / Optional	Description
options.size	number	required	The number of bytes to generate; between 1b and 512b.

Syntax

Note: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/random Samples](#).

```

1  // Add additional code
2  ...
3  // generate 5 random bytes of data
4  var bytes = random.generateBytes({size: 5});
5  ...
6  //Add additional code

```

random.generateInt(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1

Method Description	Method used to generate cryptographically strong pseudorandom number.
Returns	string
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/crypto/random Module
Since	2023.1

Parameters

Parameter	Type	Required / Optional	Description
options.max	number	required	End of random range (exclusive).
options.min	number	optional	Start of random range. Default is 0.

Syntax

Note: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/random Samples](#).

```

1 //Add additional code
2 ...
3 // generate random number between 0 and 100 (excluded)
4 var number = random.generateInt({max:100});
5 // generate random number between 10 and 100 (excluded)
6 var anotherNumber = random.generateInt({min: 10, max:100});
7 ...
8 //Add additional code

```

random.generateUUID()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1

Method Description	Method used to generate a v4 UUID using a cryptographically secure random number generator.
Returns	string
Supported Script Types	Client and server scripts

	For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/crypto/random Module
Since	2023.1

Syntax

Note: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/crypto/random Samples](#).

```

1 //Add additional code
2 ...
3 var uuid = random.generateUUID();
4 ...
5 //Add additional code

```

N/currency Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/currency module to work with exchange rates within your NetSuite account. You can use this module to find the exchange rate between two currencies based on a certain date.

To use multiple currencies, the Multiple Currencies feature must be enabled. For information about enabling this feature, see the help topic [Enabling the Multiple Currencies Feature](#).

Go To Samples 

Note: Currency formatting is handled by the [N/format Module](#).

In This Help Topic

- [N/currency Module Member](#)

N/currency Module Member

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	currency.exchangeRate (options)	number	Client and server scripts	Returns an exchange rate between two currencies.

N/currency Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/currency module.

Obtain the Exchange Rate Between the Canadian Dollar and the U.S. Dollar

The following sample shows how to obtain the exchange rate between the Canadian dollar and the U.S. dollar on a specific date.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/currency'], function(currency) {
6      function getUSDFromCAD() {
7          var canadianAmount = 100;
8          var rate = currency.exchangeRate({
9              source: 'CAD',
10             target: 'USD',
11             date: new Date('7/28/2015')
12         });
13
14         var usdAmount = canadianAmount * rate;
15     }
16
17     getUSDFromCAD();
18 });

```

currency.exchangeRate(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to return the exchange rate between two currencies based on a certain date. The source currency is looked up relative to the target currency on the effective date. For example, if use British pounds for the source and US dollars for the target and the method returns '1.52', this means that if you were to enter an invoice today for a GBP customer in your USD subsidiary, the rate would be 1.52. The exchange rate values are sourced from the Currency Exchange Rate record. Note: The Currency Exchange Rate record itself is not a scriptable record. Only base currencies can be converted. The target must be a base currency in your NetSuite account. If not, inaccurate values are returned. For details, see the help topic Setting a Base Currency.
Returns	The exchange rate as a decimal number
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/currency Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.date	Date	optional	- Pass in a new Date object. For example, date: new Date('7/28/2015') - If date is not specified, then it defaults to today (the current date). - The date determines the exchange rate in effect. If there are multiple rates, it is the latest entry on that date. - Use the same date format as your NetSuite account.	2015.2
options.source	number string	required	- The internal ID or three-letter ISO code for the currency you are converting from. - For example, you can use either 1 (internal ID) or USD (currency code). - If the Multiple Currencies feature is enabled, from your account, you can view a list of all the currency internal IDs and ISO codes at Lists > Accounting > Currencies.	2015.2
options.target	number string	required	The internal ID or three-letter ISO code for the currency you are converting to.	2015.2

Errors

Error Code	Message	Thrown If
MISSING_REQD_ARGUMENT	exchangeRate: Missing a required argument: <source/target>	The source or target argument is missing.
SSS_INVALID_CURRENCY_ID	You have entered an invalid currency symbol or internal ID: <target/source>	The source or target argument is invalid. If the Multiple Currencies feature is enabled, from your account, you can view a list of currency internal IDs and ISO codes at Lists > Accounting > Currencies.

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currency Module Script Sample](#).

```
1 | //Add additional code
```

```

2  ...
3  var canadianAmount = 100;
4  var rate = currency.exchangeRate({
5      source: 'CAD',
6      target: 'USD',
7      date: new Date('7/28/2015')
8  });
9  var usdAmount = canadianAmount * rate;
10 ...
11 //Add additional code

```

N/currentRecord Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/currentRecord module to access the record that is active in the current client context. This module is always a dynamic object and mode of work is always dynamic, not deferred dynamic/standard. For more information, see the help topic [SuiteScript 2.x Standard and Dynamic Modes](#). Be aware that when the current record is in view mode it cannot be edited; it is a read-only record when in view mode. As such, any set APIs do not work on the current record in view mode.

Go To Samples 

You can use the currentRecord module in the following types of scripts:

- **Entry point client scripts** — These scripts use the @NScriptType ClientScript annotation. (For details, see the help topic [SuiteScript 2.x JSDoc Validation](#).) The system automatically provides this type of script with a [currentRecord.CurrentRecord](#) object that represents the current record. For this reason, an entry point client script does not have to explicitly load the N/currentRecord module. To access the currentRecord object, create a variable and initialize it to the value of the scriptContext.currentRecord property, which is available in each of the [SuiteScript 2.x Client Script Entry Points and API](#). For an example, see the help topic [SuiteScript Client Script Sample](#).
- **Client custom modules** — These scripts do not use an @NScriptType annotation (see the help topic [SuiteScript 2.x Custom Modules](#)). For these scripts, you must manually load the N/currentRecord module by naming it in the script's define statement. Additionally, you must actively retrieve a [currentRecord.CurrentRecord](#) object by using the [currentRecord.get\(\)](#) or [currentRecord.get.promise\(\)](#) method. For an example, see [N/currentRecord Module Script Samples](#).

Like the [N/record Module](#), the currentRecord module provides access to body and sublist fields. However, you should use the record module for server scripts and for cases where a client script needs to interact with a record other than the currently active record. You should use the currentRecord module for client scripts that need to interact with the currently active record.

Additionally, the functionality of the two modules varies slightly. For example, the currentRecord module does not permit the editing of subrecords, although subrecords can be retrieved in view mode. For additional details, see the following topics:

In This Help Topic

- [N/currentRecord Module Members](#)
- [Column Object Members](#)

- [CurrentRecord Object Members](#)
- [Field Object Members](#)
- [Sublist Object Members](#)

Note: SuiteScript supports working with standard NetSuite records and with instances of custom record types. Supported standard record types are described in the [SuiteScript Records Browser](#). Refer also to [SuiteScript Supported Records](#). For help interacting with an instance of a custom record type, see the help topic [Custom Record](#).

N/currentRecord Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	currentRecord.Column	Object	Client scripts	Encapsulates a column of a sublist on the current record.
	currentRecord.CurrentRecord	Object	Client scripts	Represents the record active on the current page.
	currentRecord.Field	Object	Client scripts	Represents a body or sublist field.
	currentRecord.Sublist	Object	Client scripts	Represents a sublist.
Method	currentRecord.get()	currentRecord.CurrentRecord	Client scripts	Retrieves a record object that represents the current record.
	currentRecord.get.promise()	Promise	Client scripts	Retrieves a promise for an object that represents the current record.

Column Object Members

The following members are called on the [currentRecord.Column](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Column.id	string (read-only)	Client scripts	Returns the internal ID of the column.
	Column.isDisabled	boolean	Client scripts	Indicates whether the column is disabled.
	Column.isMandatory	boolean	Client scripts	Indicates whether the column is required.
	Column.label	string (read-only)	Client scripts	Returns the UI label for the column.
	Column.sublistId	string (read-only)	Client scripts	Returns the internal ID of the standard or custom sublist that contains the column.
	Column.type	string (read-only)	Client scripts	Returns the column type.

CurrentRecord Object Members

The following members are called on the [currentRecord.CurrentRecord](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	CurrentRecord.cancelLine(options)	currentRecord.CurrentRecord	Client scripts	Cancels the changes made to the currently selected line.
	CurrentRecord.commitLine(options)	currentRecord.CurrentRecord	Client scripts	Commits the currently selected line.
	CurrentRecord.findMatrixSublistLineWithValue(options)	number	Client scripts	Returns the line number of the first line that contains the specified value in the matrix column.
	CurrentRecord.findSublistLineWithValue(options)	number	Client scripts	Gets the line number for the first occurrence of a field value in a sublist.
	CurrentRecord.getCurrentMatrixSublistValue(options)	number Date string array boolean	Client scripts	Gets the value for the currently selected line in the matrix.
	CurrentRecord.getCurrentSublistIndex(options)	number	Client scripts	Gets the line number of the currently selected line.
	CurrentRecord.getCurrentSublistSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Gets the subrecord for the associated sublist field on the current line. The subrecord object is retrieved in view mode.
	CurrentRecord.getCurrentSublistText(options)	number Date string array boolean	Client scripts	Gets the value of the field in the currently selected line by text representation.
	CurrentRecord.getCurrentSublistValue(options)	number Date string array boolean	Client scripts	Gets the value of the field in the currently selected line.
	CurrentRecord.getField(options)	currentRecord.Field	Client scripts	Gets a field object from the record.
	CurrentRecord.getLineCount(options)	number	Client scripts	Returns the number of lines in the sublist.
	CurrentRecord.getMatrixHeaderCount(options)	number	Client scripts	Returns the number of columns for the specified matrix.
	CurrentRecord.getMatrixHeaderField(options)	currentRecord.Field	Client scripts	Gets the field for the specified header in the matrix.
	CurrentRecord.getMatrixHeaderValue(options)	number Date string array boolean	Client scripts	Gets the value for the associated header in the matrix.
	CurrentRecord.getMatrixSublistField(options)	currentRecord.Field	Client scripts	Gets the field for the specified sublist in the matrix.
	CurrentRecord.getMatrixSublistValue(options)	number Date string array boolean	Client scripts	Gets the value for the associated field in the matrix.
	CurrentRecord.getSublist(options)	currentRecord.Sublist	Client scripts	Gets the specified sublist object.
	CurrentRecord.getSublistField(options)	currentRecord.Field	Client scripts	Gets the specified field object from the sublist.
	CurrentRecord.getSublistText(options)	string	Client scripts	Gets the value of the field in a sublist by a string representation.
	CurrentRecord.getSublistValue(options)	number Date string array boolean	Client scripts	Gets the value of the field in a sublist.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	CurrentRecord.getSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Gets the subrecord associated with the field. The subrecord object is retrieved in view mode.
	CurrentRecord.getText(options)	string	Client scripts	Gets the value of the field by a string representation.
	CurrentRecord.getValue(options)	number Date string array boolean	Client scripts	Gets the value of the field. Not to be used on custom password fields. Use crypto.checkPasswordField(options) instead.
	CurrentRecord.hasCurrentSublistSubrecord(options)	boolean	Client scripts	Returns a value indicating whether the associated sublist field has a subrecord on the current line.
	CurrentRecord.hasSublistSubrecord(options)	boolean	Client scripts	Returns a value indicating whether the associated sublist field contains a subrecord.
	CurrentRecord.hasSubrecord(options)	boolean	Client scripts	Indicates whether the field has a subrecord.
	CurrentRecord.insertLine(options)	currentRecord.CurrentRecord	Client scripts	Inserts a new line in a sublist.
	CurrentRecord.removeCurrentSublistSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Removes the subrecord for the associated sublist field on the current line.
	CurrentRecord.removeLine(options)	currentRecord.CurrentRecord	Client scripts	Removes a line from a sublist.
	CurrentRecord.removeSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Removes the subrecord associated with the field.
	CurrentRecord.selectLine(options)	void	Client scripts	Selects a line item in a sublist.
	CurrentRecord.selectNewLine(options)	currentRecord.CurrentRecord	Client scripts	Selects a new line at the end of the sublist.
	CurrentRecord.setCurrentMatrixSublistValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value for the currently selected line in the matrix.
	CurrentRecord.setCurrentSublistText(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field in the currently selected line using a string representation.
	CurrentRecord.setCurrentSublistValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field in the currently selected line.
	CurrentRecord.setMatrixHeaderValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value for the associated header in the matrix.
	CurrentRecord.setMatrixSublistValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value for the associated field in the matrix.
	CurrentRecord.setText(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field using a string representation.
	CurrentRecord.setValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field.
Property	CurrentRecord.id	number (read-only)	Client scripts	Returns the internal record ID.
	CurrentRecord.isDynamic	boolean (read-only)	Client scripts	Indicates whether the record is dynamic.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	CurrentRecord.type	string (read-only)	Client scripts	Returns the record type.

Field Object Members

The following members are called on the [currentRecord.Field](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Field.getSelectOptions(options)	array	Client scripts	Returns an array of available options on a standard or custom select, multiselect, or radio field as key-value pairs. Only the first 1,000 available options are returned.
	Field.insertSelectOption(options)	void	Client scripts	Inserts an option into certain types of select and multiselect fields. This method is usable only in fields that were added by a front-end Suitelet or beforeLoad user event script.
	Field.removeSelectOption(options)	void	Client scripts	Removes an option from certain types of select and multiselect fields. This method is usable only in fields that were added by a front-end Suitelet or beforeLoad user event script. It is supported only in client scripts.
Object	Field.id	string (read-only)	Client scripts	Returns the internal ID of a standard or custom body or sublist field.
	Field.isDisabled	boolean	Client scripts	Returns true if the standard or custom field is disabled on the record form, or false otherwise.
	Field.isDisplay	boolean	Client scripts	Returns true if the field is set to display on the record form, or false otherwise. This property is read-only for sublist fields.
	Field.isMandatory	boolean	Client scripts	Returns true if the standard or custom field is mandatory on the record form, or false otherwise.
	Field.isPopup	boolean (read-only)	Client scripts	Returns true if the field is a popup list field, or false otherwise.
	Field.isReadOnly	boolean	Client scripts	Returns true if the field on the record form cannot be edited, or false otherwise.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				For textarea fields, this property can be read or written to. For all other fields, this property is read-only.
	Field.isVisible	boolean (read-only)	Client scripts	Returns true if the field is visible on the record form, or false otherwise.
	Field.label	string (read-only)	Client scripts	Returns the UI label for a standard or custom field body or sublist field.
	Field.sublistId	string (read-only)	Client scripts	Returns the ID of the sublist associated with the specified sublist field.
	Field.type	string (read-only)	Client scripts	Returns the type of a body or sublist field.

Sublist Object Members

The following members are called on the [currentRecord.Sublist](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Sublist.getColumn(options)	currentRecord.Column	Client scripts	Returns a column in the sublist.
Property	Sublist.id	string (read-only)	Client scripts	Returns the internal ID of the sublist.
	Sublist.isChanged	boolean (read-only)	Client scripts	Indicates whether the sublist has changed on the current record form.
	Sublist.isDisplay	boolean (read-only)	Client scripts	Indicates whether the sublist is displayed on the current record form.
	Sublist.type	string (read-only)	Client scripts	Returns the sublist type.

N/currentRecord Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/currentRecord module:

- [Update Fields on Current Record using a Custom Module Script and a User Event Script](#)
- [Perform Field Sourcing Synchronously](#)

Update Fields on Current Record using a Custom Module Script and a User Event Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: This sample includes two scripts: a client script (clientDemo.js) which is called by a user event script that uses the custom module client script.

The following sample is a custom module client script named clientDemo.js. This script updates fields on the current record. After you upload the clientDemo.js script file to a NetSuite account, it can be called by other scripts.

Because clientDemo.js is a custom module script, it must manually load the [N/currentRecord Module](#) by naming it in the define statement. It must also retrieve a `currentRecord.CurrentRecord` object by using the `currentRecord.get()` method.

i Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

! Important: This sample uses SuiteScript 2.1. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @ApiVersion 2.1
3   */
4   define(['N/currentRecord'], currentRecord => {
5     return ({
6       test_set_getValue: () => {
7         // Get a reference to the currently active record
8         let myRecord = currentRecord.get();
9
10        // Set the value of a custom field
11        myRecord.setValue({
12          fieldId: 'custpage_textfield',
13          value: 'Body value',
14          ignoreFieldChange: true,
15          forceSyncSourcing: true
16        });
17
18        // Retrieve the value that was set
19        let actValue = myRecord.getValue({
20          fieldId: 'custpage_textfield'
21        });
22
23        // Set the value of another custom field
24        myRecord.setValue({
25          fieldId: 'custpage_resultfield',
26          value: actValue,
27          ignoreFieldChange: true,
28          forceSyncSourcing: true
29        });
30      },
31
32      test_set_getCurrentSublistValue: () => {
33        // Get a reference to the currently active record
34        let myRecord = currentRecord.get();
35
36        // Set the value of a custom sublist field
37        myRecord.setCurrentSublistValue({
38          sublistId: 'sitecategory',

```

```

39         fieldId: 'custpage_subtextfield',
40         value: 'Sublist Value',
41         ignoreFieldChange: true,
42         forceSyncSourcing: true
43     });
44
45     // Retrieve the value that was set
46     let actValue2 = myRecord.getCurrentSublistValue({
47         sublistId: 'sitecategory',
48         fieldId: 'custpage_subtextfield'
49     });
50
51     // Set the value of another custom field
52     myRecord.setValue({
53         fieldId: 'custpage_sublist_resultfield',
54         value: actValue2,
55         ignoreFieldChange: true,
56         forceSyncSourcing: true
57     });
58     }
59 });
60 });

```

The following sample is a user event script deployed on a non-inventory item record. Before the record loads, the script updates the form used by the record to add new text fields, a sublist, and buttons that call the methods in a custom module client script (clientDemo.js). The buttons access the current record and set values for some of the form's fields. This sample demonstrates how to customize a form, use a custom module client script, and see the new fields and buttons in action.



Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6  define([], () => {
7      return {
8          beforeLoad: params => {
9              // Get a reference to the current form that is about to load
10             let form = params.form;
11
12             // Add several custom fields to the form
13             let textfield = form.addField({
14                 id: 'custpage_textfield',
15                 type: 'text',
16                 label: 'Text'
17             });
18             let resultfield = form.addField({
19                 id: 'custpage_resultfield',
20                 type: 'text',
21                 label: 'Result'
22             });
23             let sublistResultfield = form.addField({
24                 id: 'custpage_sublist_resultfield',
25                 type: 'text',
26                 label: 'Sublist Result Field'
27             });
28
29             // Get a reference to the sitecategory sublist
30             let sublistObj = form.getSublist({

```

```

31         id: 'sitecategory'
32     });
33
34     // Add a custom field to the sublist
35     let subtextfield = sublistObj.addField({
36         id: 'custpage_subtextfield',
37         type: 'text',
38         label: 'Sublist Text Field'
39     });
40
41     // Set the module path to the previous sample script
42     form.clientScriptModulePath = './clientDemo.js';
43
44     // Add two custom buttons to the form
45     form.addButton({
46         id: 'custpage_custombutton',
47         label: 'SET_GET_VALUE',
48         functionName: 'test_set_getValue'
49     });
50     form.addButton({
51         id: 'custpage_custombutton2',
52         label: 'SET_GETCURRENTSUBLISTVALUE',
53         functionName: 'test_set_getCurrentSublistValue'
54     });
55     }
56 };
57 });

```

Perform Field Sourcing Synchronously

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample shows how to use the `forceSyncSourcing` parameter.

This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete. For example, if `forceSyncSourcing` is set to false when adding sublist lines, the lines aren't committed as expected. Setting the parameter to true, forces synchronous sourcing.

This sample shows using the `forceSyncSourcing` parameter in the `setCurrentSublistValue` method. The `forceSyncSourcing` parameter is also available in the `setText`, `setValue`, `setCurrentSublistText`, `setMatrixHeaderValue`, and `setCurrentMatrixSublistValue` methods.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/currentRecord'], function(currentRecord){
6      var rec = currentRecord.get();
7      rec.selectNewLine({
8          sublistId: 'item'
9      });
10     rec.setCurrentSublistValue({
11         sublistId: 'item',
12         fieldId: 'item',
13         value: 39,
14         forceSyncSourcing: true
15     });
16     rec.setCurrentSublistValue({
17         sublistId: 'item',

```



```
18     fieldId: 'quantity',
19     value: 1,
20     forceSyncSourcing:true
21   });
22   rec.commitLine({sublistId: 'item'});
23 }
```

currentRecord.Column

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a column of a sublist on the current record.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/currentRecord Module
Methods and Properties	Column Object Members
Since	2016.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var objColumn = objSublist.getColumn({
4   fieldId: 'item'
5 });
6 ...
7 //Add additional code
```

Column.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the column.
Type	string (read-only)
Module	N/currentRecord Module
Since	2016.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code ...
```

```

2 | var columnid = objColumn.id;
3 | ...
4 | //Add additional code

```

Column.isDisabled

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the column is disabled.
Type	boolean
Module	N/currentRecord Module
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 | // Add additional code.
2 | ...
3 | var sublistObj = recordObj.getSublist({
4 |     sublistId: 'sublistId'
5 | });
6 | var columnObj = sublistObj.getColumn({
7 |     fieldId: 'columnId'
8 | });
9 | //set
10 | columnObj.isDisabled = true;
11 | //get
12 | var isDisabled = columnObj.isDisabled;
13 | })...
14 | // Add additional code.

```

Column.isMandatory

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the column is required.
Type	boolean
Module	N/currentRecord Module
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 | // Add additional code.

```

```

2  ...
3  var sublistObj = recordObj.getSublist({
4      sublistId: 'sublistId'
5  });
6  var columnObj = sublistObj.getColumn({
7      fieldId: 'columnId'
8  });
9  //set
10 columnObj.isMandatory = true;
11 //get
12 var isMandatory = columnObj.isMandatory;
13 })...
14 // Add additional code.

```

Column.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the column.
Type	string (read-only)
Module	N/currentRecord Module
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var columnlabel = objColumn.label;
4  ...
5  //Add additional code

```

Column.sublistId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the standard or custom sublist that contains the column.
Type	string (read-only)
Module	N/currentRecord Module
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code

```

```

2  ...
3  var sublistid = objColumn.sublistId;
4  ...
5  //Add additional code

```

Column.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the column type.
Type	string (read-only)
Module	N/currentRecord Module
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var columntype = objColumn.type;
4  ...
5  //Add additional code

```

currentRecord.CurrentRecord

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the record active on the current page.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/currentRecord Module
Methods and Properties	CurrentRecord Object Members
Since	2016.2

Syntax



Important: The following code samples show the syntax for this member. These samples are not a functional examples. For a complete script example, see [N/currentRecord Module Script Samples](#) and [SuiteScript Client Script Sample](#).

The following sample shows the retrieval of a currentRecord object in a custom module where the currentRecord was explicitly loaded.

```

1 //Add additional code
2 ...
3 var objRecord = currentRecord.get();
4 ...
5 //Add additional code

```

In an entry point client script, you do not have use the get method to retrieve the current record. (An entry point client script is one that uses the @NScriptType ClientScript annotation.) In these scripts, a currentRecord object is automatically created when the script is loaded. It is part of the context object that passed to each of the client script type's entry points. However, you do have to create a variable to represent the current record, as shown in the following sample.

```

1 //Add additional code
2 ...
3
4 function pageInit(context) {
5     var currentRec = context.currentRecord;
6     ...
7 //Add additional code

```

CurrentRecord.cancelLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Cancels the currently selected line on a sublist.
Returns	The currentRecord.CurrentRecord object that called the method.
Governance	None
Module	N/currentRecord Module
Sibling Object Members	CurrentRecord Object Members
Parent Object	currentRecord.CurrentRecord
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.cancelLine({
4     sublistId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.commitLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Commits the currently selected line on a sublist.
Returns	The currentRecord.CurrentRecord object that called the method.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Parent Object	currentRecord.CurrentRecord
Sibling Object Members	CurrentRecord Object Members
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.ignoreRecalc	boolean	optional	If set to true, scripting recalculation is ignored.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.commitLine({
4     sublistId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.findMatrixSublistLineWithValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the line number of the first instance where a specified value is found in a specified column of the matrix.
Returns	number
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Parent Object	currentRecord.CurrentRecord
Sibling Object Members	CurrentRecord Object Members
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist.	2016.2

Parameter	Type	Required / Optional	Description	Since
			This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	
options.fieldId	string	required	The ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.value	number	required	The value to search for.	2016.2
options.column	number	required	The column number of the field.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var lineNumber = objRecord.findMatrixSublistLineWithValue({
4     sublistId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.findSublistLineWithValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the line number for the first occurrence of a field value in a sublist.
Returns	A line number as a number, or -1 if not found.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Parent Object	currentRecord.CurrentRecord
Sibling Object Members	CurrentRecord Object Members

Since	2016.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.value	number Date string array boolean	optional	The value to search for.	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or not defined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var lineNumber = objRecord.findSublistLineWithValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     value: 233
7 });
8 ...
9 //Add additional code

```

CurrentRecord.getCurrentMatrixSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value for the currently selected line in the matrix.
--------------------	---

	Gets a numeric value for rate and ratehighprecision fields.
Returns	number Date string array boolean
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Parent Object	currentRecord.CurrentRecord
Sibling Object Members	CurrentRecord Object Members
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the matrix field.	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var matrixValue = objRecord.getCurrentMatrixSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12
7 });

```

```

8  ...
9  //Add additional code

```

CurrentRecord.getCurrentSublistIndex(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the line number of the currently selected line.
Returns	number
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var currIndex = objRecord.getCurrentSublistIndex({
4      sublistId: 'item'
5  });
6  ...

```

```
7 //Add additional code
```

CurrentRecord.getCurrentSublistSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the subrecord for the associated sublist field on the current line.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object. If no subrecord instance exists, the system creates one. For more information, see the help topic [Subrecord Scripting in SuiteScript 2.x Compared With 1.0](#).

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var objSubrecord = objRecord.getCurrentSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 //Add additional code
```

CurrentRecord.getCurrentSublistText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a text representation of the field value in the currently selected line.
Returns	string  Note: For multiselect fields, returns an array.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fieldName = objRecord.getCurrentSublistText({
4     sublistId: 'item',

```

```

5   fieldId: 'item'
6   });
7   ...
8   //Add additional code

```

CurrentRecord.getCurrentSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist field on the currently selected sublist line.
Returns	number Date string array boolean
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code

```

```

2  ...
3  var sublistValue = objRecord.getCurrentSublistValue({
4      sublistId: 'item',
5      fieldId: 'item'
6  });
7  ...
8  //Add additional code

```

CurrentRecord.getField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a field object from a record.
Returns	currentRecord.Field
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var objField = objRecord.getField({
4      fieldId: 'item'
5  });
6  ...
7  //Add additional code

```

CurrentRecord.getLineCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of lines in a sublist.
Returns	number
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var numLines = objRecord.getLineCount({
4     sublistId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.getMatrixHeaderCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of columns for the specified matrix.
Returns	number
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var numLines = objRecord.getMatrixHeaderCount({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 //Add additional code

```

CurrentRecord.getMatrixHeaderField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the field for the specified header in the matrix.
Returns	currentRecord.Field
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var objField = objRecord.getMatrixHeaderField({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12
7 });
8 ...
9 //Add additional code

```

CurrentRecord.getMatrixHeaderValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value for the associated header in the matrix.
Returns	number Date string array boolean
Supported Script Types	Client scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var value = objRecord.getMatrixHeaderValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12
7 });
8 ...
9 //Add additional code

```

CurrentRecord.getMatrixSublistField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the field for the specified sublist in the matrix.
---------------------------	---

Returns	currentRecord.Field
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var objField = objRecord.getMatrixSublistField({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12,
7     line: 3
8 });
9 ...
10 //Add additional code

```

CurrentRecord.getMatrixSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value for the associated field in the matrix.
Returns	number Date string array boolean
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var value = objRecord.getMatrixSublistValue({
4     sublistId: 'item',

```

```

5     fieldId: 'item',
6     column: 12,
7     line: 3
8   });
9   ...
10  //Add additional code

```

CurrentRecord.getSublist(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the specified sublist.
Returns	currentRecord.Sublist
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var objSublist = objRecord.getSublist({
4    sublistId: 'item'
5  });
6  ...
7  //Add additional code

```

CurrentRecord.getSublistField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a field object from a sublist.
---------------------------	--

	Use Record.getCurrentSublistField(options) if you are working with the current and non-committed line. This is not a valid method for use in the fieldChanged(scriptContext), pageInit(scriptContext), and postSourcing(scriptContext) entry points of a client script.
Returns	currentRecord.Field
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields.	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var objField = objRecord.getSublistField({
4     sublistId: 'item',
5     fieldId: 'item',

```

```

6 | line: 3
7 | });
8 | ...
9 | //Add additional code

```

CurrentRecord.getSublistText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist field in a text representation. Gets a string value with a "%" for rate and ratehighprecision fields.
Returns	string Note: For multiselect fields, returns an array.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var sublistFieldName = objRecord.getSublistText({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 //Add additional code
```

CurrentRecord.getSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist field. Gets a numeric value for rate and ratehighprecision fields.
Returns	number Date string array boolean
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var quantity = record.getSublistValue({
4     sublistId: 'item',
5     fieldId: 'quantity',
6     line: 0
7 });
8 ...
9 //Add additional code

```

CurrentRecord.getSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the subrecord associated with the field. If the subrecord does not exist, a new subrecord is created and returned.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types . Note: Setting values of Inventory Detail subrecord is currently not supported in Client scripts since the subrecord object in this context is only available in view-mode.
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field.	2016.2

Parameter	Type	Required / Optional	Description	Since
			See the help topic Finding Internal IDs of Record Fields .	

Errors

Error Code	Thrown If
FIELD_1_IS_DISABLED_YOU_CANNOT_APPLY_SUBRECORD_OPERATION_ON_THIS_FIELD	The specified field is disabled.
FIELD_1_IS_NOT_A_SUBRECORD_FIELD	The specified field is not a subrecord field.
SSS_INVALID_FIELD_ON_SUBRECORD_OPERATION	The specified fieldId does not refer to a subrecord.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var sublistFieldValue = objRecord.getSubrecord({
4     fieldId: 'subrecord'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.getText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the text representation of a field value. Gets a string value with a "%" for rate and ratehighprecision fields.
Returns	string Note: For multiselect fields, returns an array.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fieldidname = objRecord.getText({
4     fieldId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.getValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a field.  Note: You should not use of this method for custom password fields as it will no longer return a hashed value, but will return an obsfucated value. Use crypto.checkPasswordField(options) instead. This applies to encrypted custom password field values obtained through a variety of ways, including by saved searches, SuiteAnalytics workbooks and datasets, SuiteAnalytics Connect integrations, SOAP web services integrations, REST web services integrations, and CSV export integrations. To get the value of a custom password field, use the crypto.checkPasswordField(options) method.
Returns	number Date string array boolean

Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var value = objRecord.getValue({
4     fieldId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.hasCurrentSublistSubrecord(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a value indicating whether the associated sublist field has a subrecord on the current line. This method can only be used on dynamic records.
Returns	boolean
Supported Script Types	Client scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a subrecord. See the help topic Finding Internal IDs of Record Fields .	2016.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hasSubrecord = objRecord.hasCurrentSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 //Add additional code

```

CurrentRecord.hasSublistSubrecord(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a value indicating whether the associated sublist field contains a subrecord.
Returns	boolean
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module

Since	2016.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a subrecord. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hasSubrecord = objRecord.hasSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 //Add additional code

```

CurrentRecord.hasSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a value indicating whether the field contains a subrecord.
Returns	boolean
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of the field that may contain a subrecord. See the help topic Finding Internal IDs of Record Fields .	2016.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hasSubrecord = objRecord.hasSubrecord({
4     fieldId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.insertLine(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a sublist line.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help	2016.2

Parameter	Type	Required / Optional	Description	Since
			topic Working with the SuiteScript Records Browser .	
options.line	number	required	The line number to insert. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2
options.ignoreRecalc	boolean	optional	If set to true, scripting recalculation is ignored. The default value is false.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.insertLine({
4     sublistId: 'item',
5     line: 3,
6     ignoreRecalc: true
7 });
8 ...
9 //Add additional code

```

CurrentRecord.moveLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Moves one line of the sublist to another location. The sublist machine must allow moving lines, for example: <code>editmachine.setAllowMoveLines(true);</code> The sublist must contain the <code>_sequence</code> field. The sublist type must be <code>edit machine</code>. When using this method, the order of the other lines is preserved.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/currentRecord Module
Sibling Object Members	CurrentRecord Object Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2020.2
options.from	number	required	The line number to move from. Note that line indexing begins at 0 with SuiteScript 2.x.	2020.2
options.to	number	required	The line number to move to. Note that line indexing begins at 0 with SuiteScript 2.x.	2020.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.
SSS_NOT_YET_SUPPORTED	You are trying to move other than the current line.
SSS_SUBLIST_DOESNT_SUPPORT_MOVING_LINES	Moving of lines is not supported.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 // Add additional code.
2 ...
3 r.moveLine(sublistId, 2, 0);
4 logMessage('After moveLine(sublistId, 2, 0)');
5 logState(r);
6
7 r.moveLine({
8     sublistId: sublistId,
9     from: 0,
10    to: 1
11 });
12 logMessage('After moveLine({sublistId:sublistId, from:0, to:1})');
13 logState(r);
14 ...
15 // Add additional code.
```

CurrentRecord.removeCurrentSublistSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the subrecord for the associated sublist field.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.removeCurrentSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 //Add additional code

```

CurrentRecord.removeLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes a sublist line.
---------------------------	-------------------------

Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.line	number	required	The line number of the sublist to remove. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2
options.ignoreRecalc	boolean	optional	If set to true, scripting recalculation is ignored. The default value is false.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.removeLine({
4     sublistId: 'item',
5     line: 3,
6     ignoreRecalc: true
7 });
8 ...
9 //Add additional code

```

CurrentRecord.removeSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the subrecord for the associated field.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See the help topic Finding Internal IDs of Record Fields .	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.removeSubrecord({
4   fieldid: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.selectLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Selects an existing line in a sublist.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.line	number	required	The line number to select in the sublist. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var record = objCurrentRecord.selectLine({
4     sublistId: 'item',
5     line: 3
6 });
7 ...
8 //Add additional code

```

CurrentRecord.selectNewLine(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Selects a new line at the end of a sublist.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.selectNewLine({
4     sublistId: 'item'
5 });
6 ...
7 //Add additional code

```

CurrentRecord.setCurrentMatrixSublistValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the line currently selected in the matrix. Sets a numeric value for rate and ratehighprecision fields. This method is not available for standard records.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	2016.2
options.ignoreField Change	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2016.2
options.forceSync Sourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously. See Perform Field Sourcing Synchronously .	2019.1

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setCurrentMatrixSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 3,
7     value: false,
8     ignoreFieldChange: true,
9     forceSyncSourcing: true
10 });
11 ...
12 //Add additional code

```

CurrentRecord.setCurrentSublistText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the field in the currently selected line by a text representation. Sets a string value with a "%" for rate and ratehighprecision fields.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.text	string	required	The text to set the value to.	2016.2
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2016.2
options.forceSyncSourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously. See Perform Field Sourcing Synchronously .	2019.1

Errors

Error Code	Thrown If
A_SCRIPT_IS_ATTEMPTING_TO_EDIT_THE_1_SUBLIST_THIS_SUBLIST_IS_CURRENTLY_IN_READONLY_MODE_AND_CANNOT_BE_EDITED_CALL_YOUR_NETSUITE_ADMINISTRATOR_TO_DISABLE_THIS_SCRIPT_IF_YOU_NEED_TO_SUBMIT_THIS_RECORD	A user tries to edit a read-only sublist field.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setCurrentSublistText({
4     sublistId: 'item',
5     fieldId: 'item',
6     text: 'value',
7     ignoreFieldChange: true,
8     forceSyncSourcing: true
9 });
10 ...
11 //Add additional code

```

CurrentRecord.setCurrentSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the field in the currently selected line.  Important: When you edit a sublist line with SuiteScript, it triggers an internal validation of the sublist line. If the line validation fails, the script also fails. For example, if your script edits a closed catch up period, the validation fails and prevents SuiteScript from editing the closed catch up period. Sets a numeric value for rate and ratehighprecision fields.
Returns	<code>currentRecord.CurrentRecord</code>
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.sublistId</code>	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2016.2
<code>options.fieldId</code>	string	required	The internal ID of a standard or custom sublist field. See the help topic Finding Internal IDs of Record Fields.	2016.2
<code>options.value</code>	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	2016.2
<code>options.ignoreFieldChange</code>	boolean	optional	If set to true, the field change and the secondary event is ignored.	2016.2

Parameter	Type	Required / Optional	Description	Since
			By default, this value is false.	
options.forceSync Sourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously. See Perform Field Sourcing Synchronously.	2019.1

Errors

Error Code	Thrown If
A_SCRIPT_IS_ATTEMPTING_TO_EDIT_THE_1_SUBLIST_THIS_SUBLIST_IS_CURRENTLY_IN_READONLY_MODE_AND_CANNOT_BE_EDITED_CALL_YOUR_NETSUITE_ADMINISTRATOR_TO_DISABLE_THIS_SCRIPT_IF_YOU_NEED_TO_SUBMIT_THIS_RECORD	A user tries to edit a read-only sublist field.
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setCurrentSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     value: true,
7     ignoreFieldChange: true,
8     forceSyncSourcing: true
9 });
10 ...
11 //Add additional code

```

CurrentRecord.setMatrixHeaderValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the associated header in the matrix.
---------------------------	---

	Sets a numeric value for rate and ratehighprecision fields.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	2016.2
options.ignoreField Change	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2016.2
options.forceSync Sourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results.	2019.1

Parameter	Type	Required / Optional	Description	Since
			If set to <code>true</code>, sources dependent field information for empty fields synchronously. Defaults to <code>false</code> – dependent field values are not sourced synchronously. See Perform Field Sourcing Synchronously.	

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setMatrixHeaderValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 3,
7     value: false,
8     ignoreFieldChange: true,
9     forceSyncSourcing: true
10 });
11 ...
12 //Add additional code

```

CurrentRecord.setMatrixSublistValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the associated field in the matrix. Sets a numeric value for rate and ratehighprecision fields.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The internal ID of the matrix field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.column	number	required	The column number for the field.	2016.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2016.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	2016.2

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setMatrixSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12,
7     line: 3,

```

```

8      value: true
9    });
10    ...
11    //Add additional code

```

CurrentRecord.setText(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of the field by a text representation. Sets a string value with a "%" for rate and ratehighprecision fields.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.text	string	required	The text to change the field value to.	2016.2
options.ignoreField Change	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2016.2
options.forceSync Sourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously. See Perform Field Sourcing Synchronously .	2019.1

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setText({
4   fieldId: 'item',
5   text: 'value',
6   ignoreFieldChange: true,
7   forceSyncSourcing: true
8 });
9 ...
10 //Add additional code

```

CurrentRecord.setValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a field. Sets a numeric value for rate and ratehighprecision fields.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See the help topic Finding Internal IDs of Record Fields .	2016.2
options.value	number Date string	required	The value to set the field to.	2016.2

Parameter	Type	Required / Optional	Description	Since
	array boolean		The value type must correspond to the field type being set. For example: - Text, Radio, Select and Multi-Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2016.2
options.forceSyncSourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously. See Perform Field Sourcing Synchronously.	2019.1

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 objRecord.setValue({
4     fieldId: 'item',
5     value: true,
6     ignoreFieldChange: true,
7     forceSyncSourcing: true
8 });
9 ...
10 //Add additional code

```

CurrentRecord.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of a specific record.
Type	number (read-only)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/currentRecord Module
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var recordid = record.id;
4 ...
5 //Add additional code
```

CurrentRecord.isDynamic

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the record is in dynamic mode. For more information, see the help topic SuiteScript 2.x Standard and Dynamic Modes . This value is set when the record is created or accessed.
Type	boolean (read-only)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/currentRecord Module
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 if (record.isDynamic) {
```

```

4 | ...
5 | }
6 | ...
7 | //Add additional code

```

CurrentRecord.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The current record's type. Note the following: - On an instance of a standard record type, this property is represented by a value from the record.Type enum. - On an instance of a custom record type, this value is populated by the custom record type's string ID. For help finding this ID, see the help topic Custom Record.
Type	string (read-only)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/currentRecord Module
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var recordtype = currentRecord.type;
4 | ...
5 | //Add additional code

```

currentRecord.Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a body or sublist field on the current record. Use the following methods to access the Field object: CurrentRecord.getField(options) CurrentRecord.getSublistField(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/currentRecord Module

Methods and Properties	Field Object Members
Since	2016.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var currentRecordField = currentRecord.getField({
4     fieldId: 'entity'
5 });
6 ...
7 //Add additional code
8 }
```

Field.getSelectedOptions(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Obtains an array of available options on a dropdown select, multi-select, or radio field. The internal ID and label of the field options are returned as name/value pairs.  Important: You can use this method only in dynamic mode. For additional information about dynamic mode, see CurrentRecord.isDynamic.
Returns	array This function returns an array in the following format: <pre>1 [{value: 5, text: 'abc'}, {value: 6, text: '123'}]</pre> This function returns Type Error if the field is not a supported field for this method. If you attempt to get select options on a field that is not a dropdown select field, such as a popup select field or a field that does not exist on the form, null is returned. For example, if the number of options available for the field is greater than your setting for Maximum Entries in Dropdowns at Home > Set Preferences, the field becomes a popup field, and this method returns null. Because the maximum value for the Maximum Entries in Dropdowns settings is 500, the maximum number of options that can be returned is 500.  Note: A call to this method may return different results for the same field for different roles.
Governance	None
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/currentRecord Module

Since	2016.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.filter	string	required	The search string to filter the select options that are returned.  Note: Filter values are case insensitive.	2016.2
options.operator	string	required	The following operators are supported: - contains (default) - is - startswith	2016.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var options = objField.getSelectOptions({
4     filter : 'C',
5     operator : 'startswith'
6 });
7 ...
8 //Add additional code

```

Field.insertSelectOption(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts an option into certain types of select and multiselect fields. This method is usable only in select and multiselect fields that were added by a front-end Suitelet or beforeLoad user event script. The IDs for these fields always have a prefix of custpage .
Returns	Void
Governance	None
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	A string, not visible in the UI, that identifies the option.	2016.2
options.text	string	required	The label that represents the option in the UI.	2016.2
options.isSelected	boolean	optional	Determines whether the option is selected by default. If not specified, this value defaults to false.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_UI_OBJECT_TYPE	A script attempts to use this method on the wrong type of field. This method can be used only on select and multiselect fields whose IDs begin with the prefix custpage .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 // Instantiate the field. Note that this method is supported only
5 // on fields whose fieldIds have a prefix of custpage.
6
7 var field = call.getField({
8     fieldId: 'custpage_select1field'
9 });
10
11
12 // Insert a new option.
13
14 field.insertSelectOption({
15     value: 'Option1',
16     text: 'alpha'
17 });
18 ...
19 //Add additional code

```

Field.removeSelectOption(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes a select option from certain types of select and multiselect fields. This method is usable only in select fields that were added by a front-end Suitelet or beforeLoad user event script. The IDs for these fields always have a prefix of custpage .
---------------------------	---

Returns	void
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	A string, not shown in the UI, that identifies the option. To remove all options from the list, set this field to null, as follows: <pre> 1 ... 2 3 field.removeSelectOption({ 4 value: null, 5 }); 6 7 ... </pre>	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_UI_OBJECT_TYPE	A script attempts to use this method on the wrong type of field. This method can be used only on select and multiselect fields whose IDs begin with the prefix custpage .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1  //Add additional code
2  ...
3
4  // Instantiate the field. Note that this method is supported only
5  // on fields whose fieldIds have a prefix of custpage.
6
7  var field = call.getField({
8      fieldId: 'custpage_select1field'
9  });
10
11
12  // Remove the appropriate option.
13
14  field.removeSelectOption({
15      value: 'Option2',

```



```

16 });
17
18 ...
19 //Add additional code

```

Field.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of a standard or custom body or sublist field.
Type	string (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var id = objField.id;
4 ...
5 //Add additional code

```

Field.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the UI label for a standard or custom field body or sublist field.
Type	string (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var label = objField.label;

```

```

4 | ...
5 | //Add additional code

```

Field.isMandatory

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the standard or custom field is mandatory on the record form, or false otherwise.
Type	boolean
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | if (objField.isMandatory) {
4 |     ...
5 | }
6 | ...
7 | //Add additional code

```

Field.isDisabled

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	This property reflects the display type of a field. A value of true means the field is disabled. A value of false means the field is enabled. Note also: ■ If you are working with a body field, you can use this property to change the field's display type. ■ If you are working with a sublist field, you can set this property to true or false, but be aware that this action affects the entire sublist column, even though a sublist field is associated with one line. ■ For both body and sublist fields, you can use <code>Field.isDisabled</code> to determine whether the field is disabled or enabled.
Type	boolean
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 if (objField.isDisabled) {
4     ...
5 }
6 ...
7 //Add additional code
```

Field.isPopup

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the field is a popup list field, or false otherwise.
Type	boolean (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```
1 //Add additional code
2 ...
3 if (objField.isPopup) {
4     ...
5 }
6 ...
7 //Add additional code
```

Field.isDisplay

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the field is set to display on the record form, or false otherwise. Fields can be a part of a record even if they are not displayed on the record form. This property is read-only for sublist fields.
Type	boolean
Module	N/currentRecord Module
Supported Script Types	Client scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 if (objField.isDisplay) {
4     ...
5 }
6 ...
7 //Add additional code

```

Field.isVisible

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the field is visible on the record form, or false otherwise.
Type	boolean (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 if (objField.isVisible) {
4     ...
5 }
6 ...
7 //Add additional code

```

Field.isReadOnly

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the field on the record form cannot be edited, or false otherwise. For textarea fields, this property can be read or written to. For all other fields, this property is read-only.
-----------------------------	---

Type	boolean
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 if (objField.isReadOnly) {
4     ...
5 }
6 ...
7 //Add additional code

```

Field.sublistId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the sublist ID for the specified sublist field.
Type	string (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myId = field.sublistId;
4 ...
5 //Add additional code

```

Field.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the type of a body or sublist field.
-----------------------------	--

	For example, the value can return text, date, currency, select, checkbox, and other similar values. For more information about possible return values, see format.Type . The maximum character limit for select field types is 801.
Type	string (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var type = objField.type;
4 ...
5 //Add additional code

```

currentRecord.Sublist

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a sublist on the current record.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/currentRecord Module
Methods and Properties	Sublist Object Members
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var objSublist = currentRecord.getSublist({
4     sublistId: 'item'
5 });
6 ...
7 //Add additional code

```

Sublist.getColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a column in the sublist.
Returns	currentRecord.Column or null.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of the column field in the sublist. See the help topic Finding Internal IDs of Record Fields .	2016.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var objColumn = objSublist.getColumn({
4   fieldId: 'item'
5 });
6 ...
7 //Add additional code

```

Sublist.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the sublist.
Type	string (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .

Since	2016.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var sublistid = objSublist.id;
4 ...
5 //Add additional code

```

Sublist.isChanged

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the sublist has changed on the current record form.
Type	boolean (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 if (objSublist.isChanged) {
4     ...
5 }
6 ...
7 //Add additional code

```

Sublist.isDisplay

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the sublist is displayed on the current record form.
Type	boolean (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .

Since	2016.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 if (objSublist.isDisplay) {
4     ...
5 }
6 ...
7 //Add additional code

```

Sublist.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the sublist type.
Type	string (read-only)
Module	N/currentRecord Module
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var sublisttype = objSublist.type;
4 ...
5 //Add additional code

```

currentRecord.get()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves a currentRecord object that represents the record active on the current page.
Returns	currentRecord.CurrentRecord
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/currentRecord Module
Since	2016.2

Errors

Error Code	Thrown If
CANNOT_CREATE_RECORD_INSTANCE	The current record page is not scriptable or an error occurred when creating the record object.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/currentRecord Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var record = currentRecord.get();
4 ...
5 //Add additional code

```

currentRecord.get.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves a promise for a currentRecord object that represents the record active on the current page. Note: The parameters and errors thrown for this method are the same as those for currentRecord.get(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	currentRecord.get()
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/currentRecord Module
Since	2016.2

Errors

Error Code	Thrown If
CANNOT_CREATE_RECORD_INSTANCE	The current record page is not scriptable or an error occurred when creating the record instance.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```
1 //Add additional code
2 ...
3 var record = currentRecord.get.promise();
4 ...
5 //Add additional code
```

N/dataset Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/dataset module to create, load, list, or save datasets. You can only use this module in server scripts.

With this module, you can do things like:

- Create columns, joins, and conditions within a dataset.
- Delete a dataset using the [N/query Module](#).
- Execute a dataset and obtain results—similar to running a query definition with the [N/query Module](#).
- Use string aliasing to identify columns, which is helpful when linking dataset columns to a workbook.
- Apply column labels, which are the string descriptions shown in the UI.
- Save a dataset.

Datasets are the foundation for workbooks and their components. In a dataset, you combine record type fields and filters to make a query. You can use the results as source data for any workbook, and you can use one dataset in several workbooks.

To learn more about datasets in SuiteAnalytics, see the help topic [Defining a Dataset](#). For details on workbooks, see [N/workbook Module](#).



Important: The N/dataset module doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers [Outbound HTTPs in an unauthenticated client-side context](#).

Go To Samples

In This Help Topic

- [N/dataset Module Members](#)
- [Column Object Members](#)
- [Condition Object Members](#)
- [Dataset Object Members](#)
- [Join Object Members](#)

N/dataset Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	dataset.Column	Object	Server scripts	A column in the dataset, which usually represents a record field.
	dataset.Condition	Object	Server scripts	A condition or set of conditions to apply to a column.
	dataset.Dataset	Object	Server scripts	A representation of the entire dataset, including columns, conditions, and joins.
	dataset.Join	Object	Server scripts	A joined record used in the dataset.
Method	dataset.create(options)	dataset.Dataset	Server scripts	Creates a dataset.
	dataset.createColumn(options)	dataset.Column	Server scripts	Creates a dataset column.
	dataset.createCondition(options)	dataset.Condition	Server scripts	Creates a dataset condition (criteria). A condition is applied to a dataset column and includes an operator.
	dataset.createJoin(options)	dataset.Join	Server scripts	Creates a dataset join.
	dataset.createTranslation(options)	workbook.Expression	Server scripts	Creates a translation expression based on a Translation Collection.
	dataset.describe(options)	Object[]	Server scripts	Retrieves descriptive information about a dataset, including name, description, and a list of columns or formulas with their labels and types.
	dataset.describe.promise(options)	Promise Object	Server scripts	Asynchronously retrieves descriptive information about a dataset, including name, description, and a list of columns or formulas with their labels and types..
	dataset.list()	Object[]	Server scripts	Lists all existing datasets.
	dataset.listPaged(options)	PagedInfoData	Server scripts	Returns metadata about datasets as a set of paged results.
	dataset.loadDataset(options)	dataset.Dataset	Server scripts	Loads an existing dataset.
	dataset.loadDataset.promise(options)	Promise Object	Server scripts	Asynchronously loads an existing dataset.

Column Object Members

This object encapsulates the record fields in the dataset. Columns are equivalent to the fields you use when you build a dataset in SuiteAnalytics. For more information about datasets in SuiteAnalytics, see the help topic [Custom Workbooks and Datasets](#).

The following members are available for a [dataset.Column](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Column.alias	string (read-only)	Server scripts	The alias of the column.
	Column.fieldId	string (read-only)	Server scripts	The ID of the record field associated with the column.
	Column.formula	string (read-only)	Server scripts	The formula of the column.
	Column.id	number (read-only)	Server Scripts	The ID of the column.
	Column.join	dataset.Join (read-only)	Server scripts	The join for the column. Used only when the column is from a joined record.
	Column.label	string (read-only)	Server scripts	The label of the column.
	Column.type	string (read-only)	Server scripts	The return type of the formula.

Condition Object Members

This object encapsulates the criteria in the dataset. Conditions are equivalent to the criteria you use when you build a dataset in SuiteAnalytics. For more information about criteria used in datasets in SuiteAnalytics, see the help topic [Dataset Criteria Filters](#).

The following members are available for a [dataset.Condition](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Condition.caseSensitive	boolean	Server scripts	Indicates whether the condition in a sort is case sensitive.
	Condition.children	dataset.Condition [] (read-only)	Server scripts	The children of the condition (for example, subconditions AND'd or OR'd).
	Condition.column	dataset.Column (read-only)	Server scripts	The column on which the condition is placed.
	Condition.operator	string (read-only)	Server scripts	The operator of the condition.
	Condition.values	string[] number[] boolean[] Date[] Object[] (read-only)	Server scripts	The values for the condition.

Dataset Object Members

This object encapsulates the entire dataset. For more information about datasets in SuiteAnalytics, see the help topic [Custom Workbooks and Datasets](#).

The following members are available for a [dataset.Dataset](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Dataset.getExpressionFromColumn(options)	workbook.Expression	Server scripts	Returns an expression which can be used in a workbook.
	Dataset.run()	query.ResultSet	Server scripts	Executes the dataset and returns the result set (the same as in N/query Module).
	Dataset.run.promise()	Promise Object	Server scripts	Asynchronously executes the dataset and returns the result set (the same as in N/query Module).
	Dataset.runPaged(options)	query.PagedData	Server scripts	Executes the dataset and returns the result set in paginated data form (the same as in N/query Module).
	Dataset.save(options)	Object	Server scripts	Saves a dataset.
Property	Dataset.columns	dataset.Column[]	Server scripts	The columns in the dataset.
	Dataset.condition	dataset.Condition	Server scripts	The condition (criteria) for the entire dataset.
	Dataset.description	string	Server scripts	The description of the dataset.
	Dataset.id	string	Server scripts	The ID of the dataset.
	Dataset.name	string	Server scripts	The name of the dataset.
	Dataset.type	string	Server scripts	The internal ID for the base record type for the dataset.

Join Object Members

Encapsulates a joined record used in the dataset. For more information about using joins in a dataset, see the help topic [Joining Record Types in a Dataset](#).

The following members are available for a [dataset.Join](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Join.fieldId	string (read-only)	Server scripts	The ID of the field on which the join is performed.
	Join.join	dataset.Join (read-only)	Server scripts	The child join, if the join is a multilevel join.
	Join.source	string (read-only)	Server scripts	The internal ID for the source record type of the join.
	Join.target	string (read-only)	Server scripts	The polymorphic target of the join.

N/dataset Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/dataset module:

- Create a Dataset, Run the Dataset, and List All Existing Datasets
- List All Datasets and Load the First Dataset

Create a Dataset, Run the Dataset, and List All Existing Datasets

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a dataset with three columns, one of which is based on a join. Then, all datasets are listed and the newly created dataset is loaded and reviewed.

i Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // The following script creates a dataset with three columns, one of which is based on a join, and a condition. Then, all datasets are listed and the newly
   // created dataset is loaded and reviewed.
6
7   require(['N/dataset', 'N/query'], function(dataset, query){
8       var myTransactionDateColumn = dataset.createColumn({
9           fieldId: 'trandate',
10          alias: 'date'
11      });
12      var myJoin = dataset.createJoin({
13          fieldId: 'createdby',
14          target: 'entity'
15      });
16      var myNameColumn = dataset.createColumn({
17          fieldId: 'lastname',
18          alias: 'name',
19          join: myJoin
20      });
21      var myTransactionIdColumn = dataset.createColumn({
22          fieldId: 'tranid',
23          alias: 'id'
24      });
25      var myTotalColumn = dataset.createColumn({
26          fieldId: 'foreigntotal',
27          alias: 'total'
28      });
29      var myColumns = [myTransactionIdColumn, myNameColumn, myTransactionDateColumn, myTotalColumn];
30      var myCondition = dataset.createCondition({
31          column: myNameColumn,
32          operator: query.Operator.EMPTY
33      });
34      var myDataset = dataset.create({
35          type: 'transaction',
36          columns: myColumns,
37          condition: myCondition
38      });
39
40      // List all created datasets
41      var allDatasets = dataset.list();
42      log.audit({
43          title: 'All datasets:',
44          details: allDatasets
45      });
46
47      // Review the newly created dataset components (in the log)

```

```

48     log.audit({
49         title: 'My Dataset = ',
50         details: myDataset
51     });
52
53     // Now run the dataset
54     var runResult = myDataset.run();
55     var runPagedResult = myDataset.runPaged({
56         pageSize: 2
57     });
58     log.audit({
59         title: 'myDataset runResult: ',
60         details: runResult
61     });
62     log.audit({
63         title: 'myDataset runPagedResult: ',
64         details: runPagedResult
65     });
66 });

```

List All Datasets and Load the First Dataset

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample lists all existing datasets and then loads the first dataset.

i Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  // The following script lists all existing datasets and then loads the first dataset.
6
7  require(['N/dataset'], function(dataset){
8      // List all created datasets
9      var allDatasets = dataset.list();
10     log.audit({
11         title: 'All datasets:',
12         details: allDatasets
13     });
14
15     // Load the first dataset
16     var myFirstDataset = dataset.load({
17         id: allDatasets[0].id
18     });
19     log.audit('myFirstDataset:', myFirstDataset);
20 });

```

dataset.Column

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the record fields in the dataset. Columns are equivalent to the fields you use when you build a dataset in SuiteAnalytics. For more information about datasets in SuiteAnalytics, see the help topic Custom Workbooks and Datasets .
---------------------------	---

	Use dataset.createColumn(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/dataset Module
Methods and Properties	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a fieldId Column
4 var myFieldColumn = dataset.createColumn({
5     fieldId: 'name',
6     id: 7
7 });
8 // Create a formula/type Column
9 var myFormulaColumn = dataset.createColumn({
10     formula: '{email}',
11     type: 'STRING',
12     label: 'Formula'
13 });
14 // Create a join Column
15 var myJoinColumn = dataset.createColumn({
16     fieldId: 'name',
17     join: myJoin, // Create this using dataset.createJoin
18     alias: 'my joined column'
19 });
20
21 // Load a dataset and view its Column (in the log)
22 var myDataset = dataset.load({
23     id: 'stdataset_mydataset' // Replace with a valid ID value for your account
24 });
25 var myColumn = myDataset.Column[0];
26
27 // (Note that some Column properties may be empty/null based on the loaded dataset)
28 log.audit({
29     title: 'Column alias = ',
30     details: myColumn.alias
31 });
32 log.audit({
33     title: 'Column fieldId = ',
34     details: myColumn.fieldId
35 });
36 log.audit({
37     title: 'Column formula = ',
38     details: myColumn.formula
39 });
40 log.audit({
41     title: 'Column join = ',
42     details: myColumn.join
43 });
44 log.audit({
45     title: 'Column label = ',
46     details: myColumn.label
47 });
48 log.audit({

```

```

49     title: 'Column type = ',
50     details: myColumn.type
51   });
52   ...
53   // Add additional code

```

Column.alias

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The alias of the column.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Column.alias
4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7
8 log.audit({
9   title: 'Column.alias = ',
10  details: myDataset.columns[0].alias
11 });
12 ...
13 // Add additional code

```

Column.fieldId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the record field associated with the column.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members

Since	2020.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Column.field
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Column.fieldId = ',
10    details: myDataset.columns[0].fieldId
11 });
12 ...
13 // Add additional code

```

Column.formula

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The formula of the column. For more information about formulas in SuiteAnalytics, see the help topic Formula Fields .
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Column.formula
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Column.formula = ',

```

```

10     details: myDataset.columns[0].formula
11   });
12   ...
13   // Add additional code

```

Column.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the column.
Type	number (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View Column.id
4   var myDataset = dataset.load({
5     id: myDatasetId
6   });
7
8   log.audit({
9     title: 'Column.id = ',
10    details: myDataset.columns[0].id
11  });
12  ...
13  // Add additional code

```

Column.join

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A column join.
Type	dataset.Join (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members

Since	2020.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Column.join
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Column.join = ',
10    details: myDataset.columns[0].join
11 });
12 ...
13 // Add additional code

```

Column.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label of the column.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Column.label
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Column.label = ',
10    details: myDataset.columns[0].label
11 });
12 ...

```

13 // Add additional code

Column.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The return type of the formula.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Column
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Column.type
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Column.type = ',
10    details: myDataset.columns[0].type
11 });
12 ...
13 // Add additional code

```

dataset.Condition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The conditions for the dataset. Conditions are equivalent to criteria you use when you build a dataset in SuiteAnalytics. For more information about criteria used in datasets in SuiteAnalytics, see the help topic Dataset Criteria Filters. Use dataset.createCondition(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/dataset Module
Methods and Properties	Condition Object Members

Since	2020.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a Condition without values
4 var myCondition = dataset.createCondition({
5     column: myNameColumn,
6     operator: query.Operator.EMPTY
7 });
8
9 // Create a Condition with values
10 var myCondition = dataset.createCondition({
11     column: myNameColumn,
12     operator: query.Operator.EQUAL,
13     values: ['smith']
14 });
15 // Create a Condition with children
16 var myCondition = dataset.createCondition({
17     column: myNameColumn,
18     operator: 'OR',
19     children: myChildCondition // Create this with a separate call to dataset.createCondition
20 });
21
22 // Load a dataset and views its Condition (in the log)
23 let myDataset = dataset.load({
24     id: '1' // Replace with a valid ID value for your account
25 });
26 var myCondition = myDataset.condition;
27
28 // Note that some Condition properties may be empty/null based on the loaded dataset
29 log.audit({
30     title: 'Condition children = ',
31     details: myCondition.children
32 });
33 log.audit({
34     title: 'Condition column = ',
35     details: myCondition.column
36 });
37 log.audit({
38     title: 'Condition operator = ',
39     details: myCondition.operator
40 });
41 log.audit({
42     title: 'Condition values = ',
43     details: myCondition.values
44 });
45 ...
46 // Add additional code

```

Condition.caseSensitive

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the condition in a sort is case sensitive.
Type	boolean

Module	N/dataset Module
Parent Object	dataset.Condition
Sibling Object Members	Condition Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Condition.column
4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7
8 log.audit({
9   title: 'Condition.caseSensitive = ',
10  details: myDataset.condition.caseSensitive
11 });
12 ...
13 // Add additional code

```

Condition.children

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The children of the condition (for example, subconditions AND'd or OR'd).
Type	dataset.Condition[] (read-only)
Module	N/dataset Module
Parent Object	dataset.Condition
Sibling Object Members	Condition Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Condition.children
4 var myDataset = dataset.load({
5   id: myDatasetId

```



```

6  });
7
8  log.audit({
9      title: 'Condition.children = ',
10     details: myDataset.condition.children
11 });
12 ...
13 // Add additional code

```

Condition.column

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The column on which the condition is placed.
Type	dataset.Column (read-only)
Module	N/dataset Module
Parent Object	dataset.Condition
Sibling Object Members	Condition Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  // View Condition.column
4  var myDataset = dataset.load({
5      id: myDatasetId
6  });
7
8  log.audit({
9      title: 'Condition.column = ',
10     details: myDataset.condition.column
11 });
12 ...
13 // Add additional code

```

Condition.operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The operator of the condition.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Condition

Sibling Object Members	Condition Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Condition.operator
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Condition.operator = ',
10    details: myDataset.condition.operator
11 });
12 ...
13 // Add additional code

```

Condition.values

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The values for this condition
Type	string[] number[] boolean[] Date[] Object[] (read-only)
Module	N/dataset Module
Parent Object	dataset.Condition
Sibling Object Members	Condition Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Condition.values
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Condition.values = ',
10    details: myDataset.condition.values

```

```

11  });
12  ...
13  // Add additional code

```

dataset.Dataset

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the entire dataset, including columns, conditions, and joins. Use dataset.create(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/dataset Module
Methods and Properties	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  // Create a basic Dataset
4  var myDataset = dataset.create({
5      type: 'transaction'
6  });
7
8  // Create a comprehensive (full) Dataset
9  var myDataset = dataset.create({
10     type: 'transaction',
11     name: 'My Transaction Dataset',
12     id: 'custdataset_dataset',
13     description: 'My comprehensive transaction dataset that includes columns and a condition',
14     condition: myCondition, // Create this using dataset.createCondition
15     columns: [myColumns] // Create this using dataset.createColumn
16 });
17
18 // Load a dataset and view its Dataset (in the log)
19 var myDataset = dataset.load({
20     id: '1' // Replace with a valid ID value for your account
21 });
22 var myDatasetDataset = myDataset.Dataset;
23
24 // Note that some Dataset properties may be empty/null based on the loaded dataset
25 log.audit({
26     title: 'Dataset columns = ',
27     details: myDatasetDataset.columns
28 });
29 log.audit({
30     title: 'Dataset condition = ',
31     details: myDatasetDataset.condition
32 });
33 log.audit({
34     title: 'Dataset description = ',

```

```

35     details: myDatasetDataset.description
36   });
37   log.audit({
38     title: 'Dataset id = ',
39     details: myDatasetDataset.id
40   });
41   log.audit({
42     title: 'Dataset name = ',
43     details: myDatasetDataset.name
44   });
45   log.audit({
46     title: 'Dataset type = ',
47     details: myDatasetDataset.type
48   });
49   ...
50   // Add additional code

```

Dataset.getExpressionFromColumn(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an expression which can be used in workbook.
Returns	workbook.Expression
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.columnId	number	required if options.alias is not specified	The ID of the column.
options.alias	string	required if options.columnId is not specified	The alias of the column.

Errors

Error Code	Thrown If
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both the options.columnID and the options.alias parameters are specified.
NEITHER_ARGUMENT_DEFINED	Neither the options.columnID or the options.alias parameter is specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Load a dataset that includes a column that has an expression
4 var myDataset = dataset.load({
5   id: 'stddataset_mydataset' // Replace with a valid ID value for your account
6 });
7 var myWorkbookExpressionId = myDataset.getExpressionFromColumn({
8   columnId: 15
9 });
10 var myWorkbookExpressionAlias = myDataset.getExpressionFromColumn({
11   columnId: 16
12   alias: 'myExpression'
13 });
14
15 log.debug({
16   title: "myWorkbookExpression: ",
17   details: myWorkbookExpression
18 });
19 ...
20 ...
21 // Add additional code

```

Dataset.run()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the dataset and returns the result set (the same as in N/query Module).
Returns	query.ResultSet
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Load a dataset and run it

```

```

4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7 var myDatasetResultSet = myDataset.run();
8
9 log.audit({
10   title: 'My Result Set: ',
11   details: myDatasetResultSet
12 });
13 ...
14 // Add additional code

```

Dataset.run.promise()

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously executes the dataset and returns the result set (the same as in N/query Module).
	 Note: The parameters and errors thrown for this method are the same as those for Dataset.run(). For more information on promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Dataset.run()
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 // Load a dataset and run it
4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7
8 myDataset.run().promise().then(function(MyDataset){
9   log.audit({
10     title: 'My Result Set: ',
11     details: myDatasetResultSet
12   });
13 });
14 ...
15 // Add additional code

```

Dataset.runPaged(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the dataset and returns the result set as paginated data (the same as in N/query Module). The maximum number of results per page is 1000. The minimum number of results per page is 5, except for the last page, which may include fewer than 5 results.
Returns	query.PagedData
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.pageSize	number	required	The size of each page in the dataset results. The maximum allowed value is 1000. The minimum allowed value is 5. The default page size is 50 results per page.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Load a dataset and run it to get paged data results
4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7 var myDatasetResultSet = myDataset.runPaged({
8   pageSize: 15
9 });
10
11 log.audit({
12   title: 'My Paged Data Result Set: ',
13   details: myDatasetResultSet
14 });
15 ...
16 // Add additional code

```

Dataset.save(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Saves a dataset. When you save a dataset, you must provide a name, which will appear in the SuiteAnalytics Workbook UI. Optionally, you can provide a description and an ID. If you do not provide an ID, one is generated automatically. The object returned from this method includes only the <code>id</code> property and its value.
Returns	Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.name</code>	string workbook.Expression	required	The name of the dataset.
<code>options.description</code>	string workbook.Expression	optional	The description of the dataset.
<code>options.id</code>	string	optional	The ID of the dataset.

Errors

Error Code	Thrown If
INVALID_ID_PREFIX	The value of the <code>options.id</code> parameter does not start with <code>custdataset</code> .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create and save a new dataset
4 var myNewDataset = dataset.create({

```



```

5     type: 'transaction',
6     columns: [myDatasetColumns]
7   });
8   myNewDataset.save({
9     name: 'My Dataset',
10    description: 'This is a sample dataset.',
11    id: 'custdataset_myDataset'
12  });
13
14  ...
15  // Add additional code

```

Dataset.columns

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The columns in the dataset.
Type	dataset.Column[]
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  // View Dataset.columns
4  var myDataset = dataset.load({
5    id: myDatasetId
6  });
7
8  log.audit({
9    title: 'Dataset.columns = ',
10   details: myDataset.columns
11 });
12 ...
13 // Add additional code

```

Dataset.condition

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The condition (criteria) for the entire dataset.
Type	dataset.Condition
Module	N/dataset Module

Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Dataset.condition
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Dataset.condition = ',
10    details: myDataset.condition
11 });
12 ...
13 // Add additional code

```

Dataset.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The description of the dataset.
Type	string
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Dataset.description
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Dataset.description = ',

```

```

10     details: myDataset.description
11   });
12   ...
13   // Add additional code

```

Dataset.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the dataset.
Type	string
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View Dataset.id
4   var myDataset = dataset.load({
5     id: myDatasetId
6   });
7
8   log.audit({
9     title: 'Dataset.id = ',
10    details: myDataset.id
11  });
12  ...
13  // Add additional code

```

Dataset.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the dataset.
Type	string
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Dataset.name
4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7
8 log.audit({
9   title: 'Dataset.name = ',
10  details: myDataset.name
11 });
12 ...
13 // Add additional code

```

Dataset.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID for the base record type for the dataset. Use the Records Catalog to find the Internal ID of scriptable records.
Type	string
Module	N/dataset Module
Parent Object	dataset.Dataset
Sibling Object Members	Dataset Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Dataset.type
4 var myDataset = dataset.load({
5   id: myDatasetId
6 });
7
8 log.audit({
9   title: 'Dataset.type = ',
10  details: myDataset.type
11 });
12 ...
13 // Add additional code

```

dataset.Join

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates joined records used in the dataset. Use dataset.createJoin(options) to create this object. For more information about using joins in SuiteAnalytics, see the help topic Joining Record Types in a Dataset .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/dataset Module
Methods and Properties	Join Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic join
4 var myNameJoin = dataset.createJoin({
5     fieldId: 'name'
6 });
7
8 // Create an inverse join
9 var myInverseJoin = dataset.createJoin({
10     fieldId: 'phone',
11     source: 'contact'
12 });
13
14 // Create a polymorphic join
15 var myPolymorphicJoin = dataset.createJoin({
16     fieldId: 'phone',
17     target: 'entity'
18 });
19
20 // Create a multi-level join
21 var myMultiLevelJoin = dataset.createJoin({
22     fieldId: 'phone',
23     join: myNameJoin
24 });
25
26 // Load a dataset that includes a column with a join and view its join (in the log)
27 var myDataset = dataset.load({
28     id: myDatasetId // Replace with a valid ID value for your account
29 });
30 var myJoin = myDataset.columns[0].join;
31
32 // Note that some join properties may be empty/null based on the loaded dataset
33 log.audit({
34     title: 'Join fieldId = ',
35     details: myJoin.fieldId
36 });
37 log.audit({

```

```

38     title: 'Join source = ',
39     details: myJoin.source
40 });
41 log.audit({
42     title: 'Join target = ',
43     details: myJoin.target
44 });
45 log.audit({
46     title: 'Join join = ',
47     details: myJoin.join
48 });
49 ...
50 // Add additional code

```

Join.fieldId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the record field on which the join is performed.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Join
Sibling Object Members	Join Object Members
Since	2020.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Join.fieldId
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Join.fieldId = ',
10    details: myDataset.columns[0].join.fieldId
11 });
12 ...
13 // Add additional code

```

Join.join

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The child join, if the join is a multilevel join.
Type	dataset.Join (read-only)
Module	N/dataset Module

Parent Object	dataset.Join
Sibling Object Members	Join Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Join.join
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({
9     title: 'Join.join = ',
10    details: myDataset.columns[0].join.join
11 });
12 ...
13 // Add additional code

```

Join.source

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID for the source record type of the join. Use the Records Catalog to find the Internal ID of scriptable records.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Join
Sibling Object Members	Join Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Join.source
4 var myDataset = dataset.load({
5     id: myDatasetId
6 });
7
8 log.audit({

```

```

9      title: 'Join.source = ',
10     details: myDataset.columns[0].join.source
11   });
12   ...
13   // Add additional code

```

Join.target

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The polymorphic target of the join.
Type	string (read-only)
Module	N/dataset Module
Parent Object	dataset.Join
Sibling Object Members	Join Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View Join.target
4   var myDataset = dataset.load({
5     id: myDatasetId
6   });
7
8   log.audit({
9     title: 'Join.target = ',
10    details: myDataset.columns[0].join.target
11  });
12  ...
13  // Add additional code

```

dataset.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a dataset based on a record type. A dataset can include columns and conditions (criteria).
Returns	dataset.Dataset
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/dataset Module

Sibling Module Members	N/dataset Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.columns	dataset.Column[]	optional	The fields (columns) in the dataset.
options.condition	dataset.Condition	optional	The condition (criteria to be applied to the dataset. A condition can have children which are combined using the appropriate operator (AND or OR) to produce the criteria used.
options.description	string	optional	The description of the dataset.
options.id	string	optional	The script ID of the dataset.
options.name	string	optional	The name of the dataset.
options.type	string	required	The internal ID for the record type on which to build the dataset. Use the Records Catalog to find the Internal ID of scriptable records.

Errors

Error Code	Thrown If
INVAL ID_SEARCH_TYPE	The options.type parameter is invalid.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1  / Add additional code
2  ...
3  // Create a basic Dataset
4  var myDataset = dataset.create({
5      type: 'transaction'
6  });
7
8  // Create a comprehensive (full) Dataset
9  var myDataset = dataset.create({
10     type: 'transaction',
11     name: 'My Transaction Dataset',
12     id: 'trans_dataset',
13     description: 'My comprehensive transaction dataset that includes columns and a condition',
14     condition: myCondition, // Create this using dataset.createCondition
15     columns: myColumns // Create this using dataset.createColumn

```

```

16  });
17  ...
18  // Add additional code

```

dataset.createColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a dataset column based on a field or on a formula and a type.
Returns	dataset.Column
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.alias	string	optional	The alias for the column. This can be used to get the expression for the column which can be used in a workbook. Use Dataset.getExpressionFromColumn(options) to get the expression.
options.fieldId	string	required only if options.formula and options.type are not specified	The field ID for the column (exclusive with formula/type).
options.formula	string	required only if options.fieldId is not specified	The formula for the column, such as '{email}' or '{total} — {tax}'.
options.join	dataset.Join	optional	The joined record on which the field is present.
options.id	number	optional	The ID of the column. This can be used to get the corresponding expression for the column which can be used in a workbook. Use Dataset.getExpressionFromColumn(options) to get the expression.
options.label	string	optional	The column label to display in the UI.
options.type	string	required only if options.fieldId is not specified	The return type of the formula, such as 'INTEGER' or 'STRING'.

Errors

Error Code	Thrown If
INVALID_FORMULA_TYPE	The options.type parameter is invalid.
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both the options.formula and options.fieldId parameters are specified. Or both the options.formula and options.join parameters are specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a field column
4 var myFieldColumn = dataset.createColumn({
5     fieldId: 'name',
6     name: 'My Field Column'
7 });
8 // Create a formula/type column
9 var myFormulaColumn = dataset.createColumn({
10     formula: '{email}',
11     type: 'STRING',
12     id: 11,
13     label: 'My Formula Column'
14 });
15 // Create a join column
16 var myJoinColumn = dataset.createColumn({
17     fieldId: 'name',
18     join: myJoin // Create this using dataset.createJoin
19 });
20 ...
21 // Add additional code

```

dataset.createCondition(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a dataset condition (criteria). A condition is applied to a dataset column and includes an operator. For more information about criteria used in datasets in SuiteAnalytics, see the help topic Dataset Criteria Filters .
Returns	dataset.Condition
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/dataset Module

Sibling Module Members	N/dataset Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.children	dataset.Condition[]	optional	The child conditions. Conditions can be grouped. A group of conditions is represented as a single Condition object that combines the child conditions in the group using the AND operator. This Condition object can then be used as one of the children of a higher-level condition. Use this parameter when the options.operator parameter is set to 'AND' or 'OR'.
options.column	dataset.Column	required, unless options.children is specified	The column to apply the condition to. For the top-level condition in a dataset (the single condition that child conditions are combined into), this property is always null. Child conditions are implicitly combined using the appropriate operator (AND, OR) to produce a single condition, so no column needs to be specified.
options.operator	string	required	The operator for the condition. For the top-level condition in a dataset (the single condition that child conditions are combined into), this property is always either AND or OR. The top-level condition can only combine child conditions using these operators, not more advanced ones (like NOT or ANY_OF). You can use values from query.Operator. You can also use 'AND' or 'OR' when the options.children parameter is used.
options.values	Array<null Object number string boolean Date>	optional	The values for the condition. For the top-level condition in a dataset (the single condition that child conditions are combined into), this property is always an empty array. Child conditions are implicitly combined using the appropriate operator (AND, OR) to produce a single condition, so no values need to be specified.

Errors

Error Code	Thrown If
INVALID_OPERATOR	The options.operator parameter is invalid.
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both the options.column and options.children parameters are specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a Condition without values
4 var myCondition = dataset.createCondition({
5     column: myNameColumn,
6     operator: query.Operator.EMPTY
7 });
8
9 // Create a Condition with values
10 var myCondition = dataset.createCondition({
11     column: myNameColumn,
12     operator: query.Operator.EQUAL,
13     values: ['Smith']
14 });
15 // Create a Condition with children
16 var myCondition = dataset.createCondition({
17     operator: query.Operator.IS,
18     children: myChildCondition // Create this with a separate call to dataset.createCondition
19 });
20 ...
21 // Add additional code

```

dataset.createJoin(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a dataset join. Multi-level, inverse, and polymorphic joins can be created. Joins can be used when you create a dataset column. For more information about using joins in a dataset in SuiteAnalytics, see the help topic, Joining Record Types in a Dataset .
Returns	dataset.Join
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2020.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The ID of the record field for the join.

Parameter	Type	Required / Optional	Description
			When you perform a join, you must specify the field to be used to perform the join. For example, consider a transaction record. Transaction records can be joined to customer records using the entity field on the transaction record. If you joined a transaction record to a customer record in this way, the value of this property is 'entity'.
options.join	dataset.Join	optional	The existing join used to create a multi-level join. Use this parameter only if a child join exists with the current join as its parent.
options.source	string	optional	The internal ID for the source record type for the join used to create an inverse join. Use the Records Catalog to find the Internal ID of scriptable records.
options.target	string	optional	The polymorphic target of the join.

Errors

Error Code	Thrown If
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both the options.source and options.target parameters are specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic join
4 var myNameJoin = dataset.createJoin({
5     fieldId: 'name'
6 });
7
8 // Create an inverse join
9 var myInverseJoin = dataset.createJoin({
10     fieldId: 'phone',
11     source: 'contact'
12 });
13
14 // Create a polymorphic join
15 var myPolymorphicJoin = dataset.createJoin({
16     fieldId: 'phone',
17     target: 'entity'
18 });
19
20 // Create a multi-level join
21 var myMultiLevelJoin = dataset.createJoin({
22     fieldId: 'phone',
23     join: myNameJoin
24 });
25 ...
26 // Add additional code

```

dataset.createTranslation(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a translation expression based on a Translation Collection.
Returns	workbook.Expression
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.collection	string	required	The Translation Collection to use.
options.key	string	required	The translation term to use.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myTranslation = dataset.createTranslation({
4     collection: 'myTranslationCollection',
5     key: 'myTerm'
6 });
7 ...
8 // Add additional code

```

dataset.describe(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves information about a dataset, including name, description, and a list of columns and formulas with their labels and types.
---------------------------	---

Returns	Object[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the dataset to describe.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDescribes = dataset.describe({
4     id: 'custdataset_myDataset'
5 });
6 ...
7 // Add additional code

```

dataset.describe.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously retrieves information about a dataset, including name, description, and a list of columns and formulas with their labels and types.  Note: The parameters and errors thrown for this method are the same as those for dataset.describe(options) . For more information on promises, see Promise Object .
Returns	Promise Object
Synchronous Version	dataset.describe(options)
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the dataset to describe.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 dataset.describe.promise({
4   id: 'custdataset_myDataset'
5 }).then(function(dataset){
6   // process the results
7 });
8 ...
9 // Add additional code

```

dataset.list()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Lists all existing datasets.
Returns	Object[], where each Object has properties: 'id', 'name', 'record', and an optional 'description'.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var allDatasets = dataset.list();
4
5 log.debug({
6   title: 'List of datasets: ',
7   details: allDatasets
8 });
9 ...
10 // Add additional code

```

dataset.listPaged(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves metadata about datasets.
Returns	PagedInfoData
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2021.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.pageSize	number	required	The page size.
options.category	string	optional	The category of datasets or workbooks to get a paginated listing for.

Errors

Error Code	Thrown If
INVALID_OWNER_CATEGORY	The value of the category parameter is not included in the OwnerCategory enum.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var allDatasets = dataset.listPagged({
4     pageSize: 50
5 });
6
7 log.debug({
8     title: 'List of datasets: ',
9     details: allDatasets
10 });
11 ...
12 // Add additional code

```

dataset.loadDataset(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing dataset.
Returns	dataset.Dataset
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the dataset to load.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/dataset Module Script Samples](#). Also see the [Tutorial: Creating a Dataset Using the Workbook API](#) topic.

```

1 // Add additional code

```

```

2  ...
3  var myLoadedDataset = dataset.loadDataset({
4    id: myDatasetId    // Use a script ID from an existing script
5  });
6
7  log.debug({
8    title: 'My Loaded Dataset: ',
9    details: myLoadedDataset
10 ...
11 // Add additional code

```

dataset.loadDataset.promise(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously loads an existing dataset. i Note: The parameters and errors thrown for this method are the same as those for dataset.loadDataset(options) . For more information on promises, see Promise Object .
Returns	Promise Object
Synchronous Version	dataset.loadDataset(options)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/dataset Module
Sibling Module Members	N/dataset Module Members
Since	2020.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the dataset to load.

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 dataset.loadDataset.promise({
4   id: myDatasetId    // Use a script ID from an existing script

```

```

5  }).then(function(dataset){
6      // process the dataset object
7  });
8  ...
9  // Add additional code

```

N/datasetLink Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/datasetLink module to link datasets. When you link two datasets, you can use data from both datasets in your workbooks.

Go To Samples {--}

The N/datasetLink module lets you logically link two datasets and use data from both datasets in your workbook visualizations (such as pivots). Linking datasets is useful when you cannot use joins in the SuiteAnalytics Workbook UI or the Workbook API to join record types explicitly. Linking datasets does not merge or join the datasets. Instead, you specify an expression (which usually represents a column that shares common data between the two datasets, such as a date), and this expression is used to link the datasets.

After datasets are linked, you can access all of the data in both datasets to use in workbook visualizations. For example, when you create a pivot in a workbook, you can specify a linked dataset (as a [datasetLink.DatasetLink](#) object) to use as the data source for the pivot. You can use fields in both datasets to create data dimensions, data measures, sections, and other elements of the pivot.

For more information about linking datasets in SuiteAnalytics Workbook, see the help topic [Dataset Linking in SuiteAnalytics Workbook](#).

To link two datasets, you must do the following:

- Create the datasets that you want to link.**

You can use [dataset.create\(options\)](#) to create a dataset in a script. If you use this method to create a dataset, you must also use [Dataset.save\(options\)](#) to save the dataset before you can link it with another dataset. Alternatively, you can use the Dataset Builder Plug-in to create datasets. For more information, see the help topic [Dataset Builder Plug-in](#).

```

1  // In this example, period, department, and total are dataset.Column objects that
2  // represent fields on the budgets record type (or a joined record type)
3  var myDataset1 = dataset.create({
4      type: 'budgets',
5      columns: [period, department, total],
6      name: 'Example Dataset 1'
7  });
8
9  myDataset1.save();

```

```

1  // In this example, postingperiod, department, and amount are dataset.Column
2  // objects that represent fields on the salesinvoiced record type (or a
3  // joined record type)
4  var myDataset2 = dataset.create({
5      type: 'salesinvoiced',
6      columns: [postingperiod, department, amount],
7      name: 'Example Dataset 2'
8  });
9
10 myDataset2.save();

```

2. Load the datasets that you want to link, if necessary.

In general, you can create datasets in one script, then load and link them in another script. If you are creating a workbook visualization (such as a pivot) in a script, you can create and link datasets directly in the same script, then use the linked dataset in your visualization. However, if you are creating a visualization using the Workbook Builder Plug-in, you must load the datasets in the plug-in script before you can link them. For more information, see the help topic [Workbook Builder Plug-in](#).

```
1 // In this example, the id parameter values represent the script IDs of
2 // plug-in scripts that create datasets using the Dataset Builder Plug-in
3 var budgetDataset = dataset.load({
4   id: 'customscript1772'
5 });
6
7 var salesDataset = dataset.load({
8   id: 'customscript1773'
9 });
```

3. Create expressions from columns in each dataset.

In a dataset, a column represents a field on a record type (or a field on a joined record type, if the base record type of the dataset is joined with other record types). Datasets are linked using a column that shares common data (for example, a date), and you must use [Dataset.getExpressionFromColumn\(options\)](#) to create expressions for this common column in each dataset.

```
1 // In this example, the alias parameter value represents the alias set
2 // on the associated column in the dataset
3 var budgetMachinePeriodExp = budgetDataset.getExpressionFromColumn({
4   alias: 'budgetmachineperiod'
5 });
6
7 var postingPeriodExp = salesDataset.getExpressionFromColumn({
8   alias: 'postingperiod'
9 });
```

4. Link the datasets.

Use [datasetLink.create\(options\)](#) to link your datasets. You must specify the original datasets (as [dataset.Dataset](#) objects), the expressions for the common column (as [workbook.Expression](#) objects), and an ID for the linked dataset.

```
1 var myDatasetLink = datasetLink.create({
2   datasets: [budgetDataset, salesDataset],
3   expressions: [[budgetMachinePeriodExp, postingPeriodExp]],
4   id: 'myLinkedDatasetId'
5 });
```

5. Use the linked dataset to create a visualization.

For example, if you are creating a pivot, you can use [workbook.createPivot\(options\)](#) and specify the linked dataset (as a [datasetLink.DatasetLink](#) object).

```
1 // In this example, rowSection and columnSection are workbook.Section
2 // objects that represent sections in a pivot
3 var myPivot = workbook.createPivot({
4   id: 'myPivotId',
5   rowAxis: workbook.createPivotAxis({
6     root: rowSection
7   }),
8   columnAxis: workbook.createPivotAxis({
9     root: columnSection
10  }),
11  name: 'My Pivot',
12  datasetLink: myDatasetLink
```

```
13 | });
```

In This Help Topic

- [N/datasetLink Module Members](#)
- [DatasetLink Object Members](#)

N/datasetLink Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	datasetLink.DatasetLink	Object	Server scripts	A representation of two datasets that are linked using datasetLink.create(options) .
Method	datasetLink.create(options)	datasetLink.DatasetLink	Server scripts	Links two datasets using a common column expression.

DatasetLink Object Members

The following members are available for a [datasetLink.DatasetLink](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DatasetLink.datasets	dataset.Dataset[]	Server scripts	The linked datasets that the datasetLink.DatasetLink object represents.
	DatasetLink.expressions	Array< workbook.Expression[] >	Server scripts	The column expressions for the datasetLink.DatasetLink object.

N/datasetLink Module Script Sample

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/datasetLink module.

Load Two Datasets and Link Them

i Note: This sample script uses the require function so you can copy it into the SuiteScript Debugger and test it. You must use the define function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

This script sample loads two datasets and links them using a common column (posting period).

In this sample, the id parameters of [dataset.loadDataset\(options\)](#) represent the script IDs of plug-in scripts that create datasets using the Dataset Builder Plug-in. For more information, see the help topic [Workbook Builder Plug-in](#). If you run this script sample in your account, make sure you use script IDs or saved dataset IDs that are valid in your account.

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/datasetLink', 'N/dataset'], function(datasetLink, dataset) {
5       // Load the datasets to link
6       var budgetDataset = dataset.load({
7           id: 'customscript1772'
8       });
9       var salesDataset = dataset.load({
10          id: 'customscript1773'
11      });
12
13      // Create expressions for columns in each dataset
14      // The alias parameter value represents the alias set
15      // on the associated column in the dataset
16      var budgetMachinePeriodExp = budgetDataset.getExpressionFromColumn({
17          alias: 'budgetmachineperiod'
18      });
19      var postingPeriodExp = salesDataset.getExpressionFromColumn({
20          alias: 'postingperiod'
21      });
22
23      // Link the datasets
24      var myDatasetLink = datasetLink.create({
25          datasets: [budgetDataset, salesDataset],
26          expressions: [[budgetMachinePeriodExp, postingPeriodExp]],
27          id: 'myLinkedDatasetId'
28      });
29  });

```

datasetLink.DatasetLink

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A representation of two datasets that are linked using datasetLink.create(options) .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/datasetLink Module
Methods and Properties	DatasetLink Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script sample, see [N/datasetLink Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var myDatasetLink = datasetLink.create({
4      datasets: [firstDataset, secondDataset],
5      expressions: [[columnExpInFirstDataset, columnExpInSecondDataset]],
6      id: 'myDatasetLinkId'
7  });
8  ...
9  // Add additional code

```


DatasetLink.datasets

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The linked datasets that the datasetLink.DatasetLink object represents.
Type	dataset.Dataset[]
Module	N/datasetLink Module
Parent Object	datasetLink.DatasetLink
Sibling Object Members	DatasetLink Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script sample, see [N/datasetLink Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myDatasetLink = datasetLink.create({
4   datasets: [firstDataset, secondDataset],
5   expressions: [[columnExpInFirstDataset, columnExpInSecondDataset]],
6   id: 'myDatasetLinkId'
7 });
8
9 var theDatasets = myDatasetLink.datasets;
10 ...
11 // Add additional code

```

DatasetLink.expressions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The column expressions for the datasetLink.DatasetLink object.
Type	Array<workbook.Expression[]>
Module	N/datasetLink Module
Parent Object	datasetLink.DatasetLink
Sibling Object Members	DatasetLink Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script sample, see [N/datasetLink Module Script Sample](#).

```

1 // Add additional code

```

```

2  ...
3  var myDatasetLink = datasetLink.create({
4      datasets: [firstDataset, secondDataset],
5      expressions: [[columnExpInFirstDataset, columnExpInSecondDataset]],
6      id: 'myDatasetLinkId'
7  });
8
9  var theExpressions = myDatasetLink.expressions;
10 ...
11 // Add additional code

```

DatasetLink.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the linked dataset.
Type	string (read-only)
Module	N/datasetLink Module
Parent Object	datasetLink.DatasetLink
Sibling Object Members	DatasetLink Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script sample, see [N/datasetLink Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var myDatasetLink = datasetLink.create({
4      datasets: [firstDataset, secondDataset],
5      expressions: [[columnExpInFirstDataset, columnExpInSecondDataset]],
6      id: 'myDatasetLinkId'
7  });
8
9  var theId = myDatasetLink.id;
10 ...
11 // Add additional code

```

datasetLink.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Links two datasets using a common column expression. To link two datasets, both datasets must include a column that shares common data, such as a date. You use Dataset.getExpressionFromColumn(options) to obtain expressions for each column, then you specify these expressions (and the datasets they are part of) when you call <code>datasetLink.create(options)</code>.
---------------------------	--

Returns	datasetLink.DatasetLink
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/datasetLink Module
Sibling Module Members	N/datasetLink Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.datasets	dataset.Dataset[]	required	The datasets to link.
options.expressions	Array<workbook.Expression[]>	required	The column expressions to use to link the datasets.
options.id	string	optional	The ID of the linked dataset. If you do not provide a value for this parameter, an ID is generated automatically and assigned to the DatasetLink.id property of the returned datasetLink.DatasetLink object.

Errors

Error Code	Thrown If
NO_DATASET_DEFINED	The value of the options.datasets parameter is an empty array.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script sample, see [N/datasetLink Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myDatasetLink = datasetLink.create({
4     datasets: [firstDataset, secondDataset],
5     expressions: [[columnExpInFirstDataset, columnExpInSecondDataset]],
6     id: 'myDatasetLinkId'
7 });
8 ...
9 // Add additional code

```

N/documentCapture Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: The content in this help topic pertains to SuiteScript 2.1.

Load the N/documentCapture module to extract text content from supported documents.

The N/documentCapture module lets you programmatically extract structured content and key information from a variety of document types (such as invoices, receipts, contracts, and so on) directly within NetSuite. This module uses the AI-driven capabilities of the Oracle Cloud Infrastructure (OCI) Document Understanding service and can automate document processing, reduce manual data entry, and enhance business workflows. For more information about the OCI Document Understanding service, refer to [Document Understanding](#) in the *Oracle Cloud Infrastructure Documentation*.

This module provides the following features and benefits:

- **Automated content extraction** – Extracts text, tables, and key-value pairs from scanned documents, PDFs, and images.
- **AI-powered data recognition** – Uses advanced machine learning models from OCI to accurately identify and extract relevant information.
- **Support for multiple document types** – Works with invoices, receipts, tax forms, and other business documents in PDF, PNG, JPG, and TIFF formats.
- **Synchronous and asynchronous requests** – Supports synchronous requests (for documents up to five pages in length) and asynchronous requests (for documents longer than five pages).
- **Document classification** – Automatically classifies documents by type, enabling use cases such as intelligent routing and processing.
- **Usage tracking** – Tracks usage on the AI Preferences page in the NetSuite UI.
- **Support for multiple languages and layouts** – Supports documents in multiple languages and using various layouts, increasing flexibility.
- **Error handling and confidence scores** – Provides confidence scores for extracted data and error handling for improved reliability.

This module is available in NetSuite by default when the Server SuiteScript feature is enabled. For more information, see the help topic [Enabling Features](#).

To learn how to get started with the N/documentCapture module, see [Getting Started with the N/documentCapture Module](#).

In This Help Topic

- [N/documentCapture Module Members](#)
- [Cell Object Members](#)
- [Document Object Members](#)
- [Field Object Members](#)
- [FieldLabel Object Members](#)
- [FieldValue Object Members](#)
- [Line Object Members](#)
- [Page Object Members](#)

- [Table Object Members](#)
- [TableRow Object Members](#)
- [Word Object Members](#)

N/documentCapture Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	documentCapture.Cell	Object	Server scripts	An extracted table cell from a document.
	documentCapture.Document	Object	Server scripts	The extracted data from a document.
	documentCapture.Field	Object	Server scripts	An extracted field from a document.
	documentCapture.Field Label	Object	Server scripts	An extracted field label from a document.
	documentCapture.Field Value	Object	Server scripts	An extracted field value from a document.
	documentCapture.Line	Object	Server scripts	An extracted line of text from a document.
	documentCapture.Page	Object	Server scripts	An extracted page from a document.
	documentCapture.Table	Object	Server scripts	An extracted table from a document.
	documentCapture.Table Row	Object	Server scripts	An extracted table row from a document.
	documentCapture.Word	Object	Server scripts	An extracted word from a document.
Method	documentCapture.documentToStructure(options)	documentCapture.Document	Server scripts	Extracts content from a document.
	documentCapture.documentToStructure.promise(options)	Promise	Server scripts	Asynchronously extracts content from a document.
	documentCapture.documentToText(options)	string	Server scripts	Extracts text content from a PDF file.
	documentCapture.documentToText.promise(options)	Promise	Server scripts	Asynchronously extracts text content from a PDF file.
	documentCapture.getRemainingConcurrency()	number	Server scripts	Returns the number of available concurrent requests remaining.
	documentCapture.getRemainingConcurrency.promise()	Promise	Server scripts	Asynchronously returns the number of available concurrent requests remaining.
	documentCapture.getRemainingFreeUsage()	number	Server scripts	Returns the number of free document capture requests remaining for the current month.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	documentCapture.getRemainingFreeUsage.promise()	Promise	Server scripts	Asynchronously returns the number of free document capture requests remaining for the current month.
	documentCapture.parseResult(options)	documentCapture.Document	Server scripts	Converts a JSON file into a documentCapture.Document object.
Enum	documentCapture.DocumentType	enum	Server scripts	Holds values for the document type.
	documentCapture.Feature	enum	Server scripts	Holds values for the feature to extract.
	documentCapture.FieldType	enum	Server scripts	Holds values for the type of a field.
	documentCapture.Language	enum	Server scripts	Holds values for the language of a document.

Cell Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Cell.confidence	number	Server scripts	The confidence level for the cell.
	Cell.text	string	Server scripts	The extracted text of the cell.

Document Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Document.mimeType	string	Server scripts	The MIME type of the document.
	Document.pages	documentCapture.Page[]	Server scripts	The pages of the document.
Method	Document.getText()	string	Server scripts	Returns the entire text of the document.

Field Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Field.label	documentCapture.FieldLabel	Server scripts	The label (name) of the field.
	Field.type	string	Server scripts	The type of the field.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Field.value	documentCapture.FieldValue	Server scripts	The value of the field.

FieldLabel Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	FieldLabel.confidence	number	Server scripts	The confidence level for the field label.
	FieldLabel.name	string	Server scripts	The name of the field label.

FieldValue Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	FieldValue.confidence	number	Server scripts	The confidence level for the field value.
	FieldValue.text	string	Server scripts	The text of the field value.

Line Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Line.confidence	number	Server scripts	The confidence level for the line.
	Line.text	string	Server scripts	The text of the line.

Page Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Page.detectedDocumentTypes	Object[]	Server scripts	A set of confidence levels indicating whether the page represents a particular type of document.
	Page.fields	documentCapture.Field[]	Server scripts	The extracted fields from the page of a document.
	Page.lines	documentCapture.Line[]	Server scripts	The extracted lines from the page of a document.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Page.tables	documentCapture.Table[]	Server scripts	The extracted tables from the page of a document.
	Page.words	documentCapture.Word[]	Server scripts	The extracted words from the page of a document.
Method	Page.getText()	string	Server scripts	Returns the entire text of the page.

Table Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Table.bodyRows	documentCapture.TableRow[]	Server scripts	The extracted body rows from the table in a document.
	Table.columnCount	number	Server scripts	The number of extracted columns from the table in a document.
	Table.confidence	number	Server scripts	The confidence level for the table.
	Table.footerRows	documentCapture.TableRow[]	Server scripts	The extracted footer rows from the table in a document.
	Table.headerRows	documentCapture.TableRow[]	Server scripts	The extracted header rows from the table in a document.
	Table.rowCount	number	Server scripts	The number of extracted rows from the table in a document.

TableRow Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	TableRow.cells	documentCapture.Cell[]	Server scripts	The extracted cells in the table row.

Word Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Word.confidence	number	Server scripts	The confidence level for the word.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Word.text	string	Server scripts	The extracted text of the word.

N/documentCapture Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

The following script samples demonstrate how to use the features of the N/documentCapture module.

- [Extract Text from a PDF File](#)
- [Extract Feature Content from a Document Synchronously](#)
- [Extract Content from a Document Asynchronously](#)

Extract Text from a PDF File

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following code sample extracts text content from a PDF file using [documentCapture.documentToText\(options\)](#). The file is located in the NetSuite File Cabinet and specified using its internal ID, but you can also specify a file using its name and path.

After the content is extracted, it's passed to the [llm.generateText\(options\)](#) method of the N/llm module. This method lets you use a large language model (LLM) to analyze the content further by providing a suitable prompt. The data returned from [documentCapture.documentToText\(options\)](#) can be provided to [llm.generateText\(options\)](#) as a document, which helps you integrate these modules for more advanced use cases. For more information about the N/llm module, see [N/llm Module](#).

In this example, the LLM is asked about the purpose of the provided invoice. The response includes citations, which indicate the content in the document that was used to generate the response. Finally, the LLM response and the citations are logged.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#).

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1 require(['N/file', 'N/documentCapture', 'N/llm'],
2   function(file, documentCapture, llm) {
3     // "14" is the unique ID of a PDF stored in the NetSuite File Cabinet
4     const fileObj = file.load({
5       id: "14"
6     });
7     const extractedData = documentCapture.documentToText({
8       file: fileObj
9     });

```

```

10
11     const response = llm.generateText({
12         prompt: "What is this invoice for?",
13         documents: [{
14             id: '14',
15             data: extractedData
16         }]
17     });
18
19     log.debug("Answer: ", response.text);
20     log.debug("Citations: ", response.citations);
21 }
22 );

```

Extract Feature Content from a Document Synchronously

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following code sample extracts content from a document using [documentCapture.documentToStructure\(options\)](#). This method provides a synchronous way to extract content from documents up to five pages in length. For longer documents, you must submit an asynchronous task using the N/task module. For a code sample, see [Extract Content from a Document Asynchronously](#).

The file is located in the NetSuite File Cabinet and specified using its internal ID, but you can also specify a file using its name and path. The sample specifies the features to extract (text, tables, and key-value pairs). If you don't specify the features to extract, text and tables are extracted by default. The sample also indicates that the provided document is an invoice. Specifying the document type is optional, but doing so can improve the quality and accuracy of the extracted content.

After the content is extracted, it's passed to the `llm.chat(options)` method, which is an alias of [llm.generateText\(options\)](#). This method lets you use a large language model (LLM) to analyze the content further by providing a suitable prompt. The data returned from [documentCapture.documentToStructure\(options\)](#) can be converted to a string and provided to `llm.chat(options)` ([llm.generateText\(options\)](#)) as a document, which helps you integrate these modules for more advanced use cases. For more information about the N/llm module, see [N/llm Module](#).

In this example, the LLM is provided with a preamble, which contains additional instructions for the LLM to consider when generating its response. The LLM is asked about the purpose of the provided invoice, and the extracted content is converted to a string and provided as a document. The response includes citations, which indicate the content in the document that was used to generate the response. Finally, the LLM response and the citations are logged.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#).

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  require(['N/file', 'N/documentCapture', 'N/llm'],
2  function(file, documentCapture, llm) {
3      // "14" is the internal ID of a file stored in the NetSuite File Cabinet
4      const extractedData = documentCapture.documentToStructure({
5          file: file.load(
6              id: "14"
7          ),
8          features: [

```

```

9      documentCapture.Feature.TEXT_EXTRACTION,
10     documentCapture.Feature.TABLE_EXTRACTION,
11     documentCapture.Feature.FIELD_EXTRACTION
12   ],
13   documentType: documentCapture.DocumentType.INVOICE
14 });
15
16 const documentText = extractedData.getText();
17
18 const response = llm.chat({
19   preamble: "Your task is to parse the provided document and answer questions about that document.",
20   prompt: "What is this invoice for?",
21   documents: [{
22     id: "1",
23     data: documentText
24   }]
25 });
26
27 log.debug("Answer: ", response.text);
28 log.debug("Citations: ", response.citations);
29 }
30 );

```

Extract Content from a Document Asynchronously

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following code samples show how to extract content from a document asynchronously using the N/task module, which lets you submit tasks that run asynchronously. For more information, see [N/task Module](#). If you want to extract content from a document that's longer than five pages, you must submit an asynchronous task and can't use [documentCapture.documentToStructure\(options\)](#). For a synchronous example, see [Extract Feature Content from a Document Synchronously](#).

The first sample creates a document capture task and populates its properties. The task accepts the same parameters as [documentCapture.documentToStructure\(options\)](#), such as document type, features to extract, and language. The sample loads a file by its internal ID, and it specifies the path of another file to contain the extracted content. The extracted content will be added to the specified file in JSON format. The sample also indicates that the provided document is an invoice.

Finally, the sample submits the document capture task and logs the submission ID.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#).

i Note: This sample script uses the require function so that you can copy it into the SuiteScript Debugger and test it. You must use the define function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  require(['N/task', 'N/file', 'N/documentCapture'],
2    function(task, file, documentCapture) {
3      // Create the document capture task
4      var docTask = task.create(task.TaskType.DOCUMENT_CAPTURE);
5
6      // Specify task parameters
7      docTask.inputFile = file.load("443");
8      docTask.outputFilePath = "SuiteScripts/result.json";
9      docTask.documentType = documentCapture.DocumentType.INVOICE;
10
11     // Submit the task
12     const submissionId = docTask.submit();
13     log.debug("Submission ID: ", submissionId);

```

```
14 | });
```

The second sample is a script that's deployed in a NetSuite account and processes the result of the document capture task. The sample loads the result file and uses [documentCapture.parseResult\(options\)](#) to parse the JSON result data into a [documentCapture.Document](#) object. The sample then extracts the full text of the document.

The extracted text is passed to the `llm.chat(options)` method, which is an alias of [llm.generateText\(options\)](#). This method lets you use a large language model (LLM) to analyze the content further by providing a suitable prompt. You can provide text to `llm.chat(options)` ([llm.generateText\(options\)](#)) as a document, which helps you integrate these modules for more advanced use cases. For more information about the N/llm module, see [N/llm Module](#).

In this example, the LLM is provided with a preamble, which contains additional instructions for the LLM to consider when generating its response. The LLM is asked about the purpose of the provided invoice, and the extracted text is provided as a document. The LLM response is logged, and execute entry point function is returned.



Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```
1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ScheduledScript
4   * @NModuleScope SameAccount
5   */
6  define(['N/documentCapture', 'N/file', 'N/llm'],
7    function(documentCapture, file, llm) {
8      function execute(scriptContext) {
9        const resultFile = file.load("SuiteScripts/result.json");
10       const resultDocument = documentCapture.parseResult(resultFile);
11       const resultText = resultDocument.getText();
12
13       const response = llm.chat({
14         preamble: "Your task is to parse the provided document and answer questions about that document.",
15         prompt: "What is this invoice for?",
16         documents:[{
17           id: "1",
18           data: resultText
19         }]
20       });
21       log.debug("Response: ", response.text);
22     }
23     return {
24       execute: execute
25     };
26   }
27 );
```

Getting Started with the N/documentCapture Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

The following sections help you get started with the N/documentCapture module:

- [Extracting Text from a PDF File](#)
- [Extracting Feature Content from a Document](#)
- [Using OCI Credentials to Obtain Additional Usage](#)

Extracting Text from a PDF File

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To extract text from a PDF file of any length, use [documentCapture.documentToText\(options\)](#). For a sample, see [Extract Text from a PDF File](#).

Provide the following parameters:

- `options.file` – The PDF file to extract text from. This file must be located in the NetSuite File Cabinet, and you can specify the file using its internal ID or file path.
- `options.timeout` (optional) – The timeout period, in milliseconds, to wait for the service to return results. The default value is 30,000 milliseconds (30 seconds). You can specify a longer timeout period, but you can't specify a period shorter than 30,000 milliseconds. If you do, the default 30,000 millisecond timeout is used instead.

The [documentCapture.documentToText\(options\)](#) method returns a string with the text of the PDF file. If you want to analyze the text further, you can provide the extracted text to the [llm.generateText\(options\)](#) method in the N/llm module, as the following example shows:

```

1 // "14" is the unique ID of a PDF stored in the NetSuite File Cabinet
2 const fileObj = file.load({
3   id: "14"
4 });
5 const extractedData = documentCapture.documentToText({
6   file: fileObj
7 });
8
9 const response = llm.generateText({
10  prompt: "What is this invoice for?",
11  documents: [{
12    id: '14',
13    data: extractedData
14  }]
15 });

```

Keep the following considerations in mind:

- The [documentCapture.documentToText\(options\)](#) method supports PDF files only. If you want to extract content from JPG, PNG, or TIFF files, use [documentCapture.documentToStructure\(options\)](#) instead. See [Extracting Feature Content from a Document](#).
- This method extracts content as plain text only. If you want to extract other elements in a structured form, such as tables or key-value pairs (fields), use [documentCapture.documentToStructure\(options\)](#) instead.
- This method supports PDF files of any size. However, as a best practice, files in the NetSuite File Cabinet should be less than 100 MB if possible. For large files, consider separating them into smaller files before using this method, which can reduce the time it takes for the text to be extracted and prevent timeout errors. For more information, see the help topic [Best Practices for Preparing Files for Upload to the File Cabinet](#).
- This method doesn't consume usage from the monthly usage pool of free requests provided by NetSuite (unlike [documentCapture.documentToStructure\(options\)](#), which does consume usage).

- Encrypted files are not supported.

Extracting Feature Content from a Document

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To extract specific feature content (such as tables and fields) from a file in PDF, JPG, PNG, or TIFF format, use `documentCapture.documentToStructure(options)`. For a sample, see [Extract Feature Content from a Document Synchronously](#).

Provide the following parameters:

- `options.file` – The document file to extract content from. This file must be located in the NetSuite File Cabinet, and you can specify the file using its internal ID or file path.
- `options.documentType` (optional) – The document type. By specifying the type of document, the service can apply pretrained models that are optimized for that type, which can provide more accurate extraction results. Use values from the `documentCapture.DocumentType` enum to set this parameter. If you don't specify a value for this parameter, the `DocumentType.OTHERS` type is used by default.
- `options.features` (optional) – The features to extract from the specified document. Use values from the `documentCapture.Feature` enum to set this parameter. If you don't specify a value for this parameter, the `Feature.TEXT_EXTRACTION` and `Feature.TABLE_EXTRACTION` features are used by default.
- `options.language` (optional) – The language of the specified document. Use values from the `documentCapture.Language` enum to set this parameter. If you don't specify a value for this parameter, `ENG` (English) is used by default.
- `options.timeout` (optional) – The timeout period, in milliseconds, to wait for the service to return results. The default value is 30,000 milliseconds (30 seconds). You can specify a longer timeout period, but you can't specify a period shorter than 30,000 milliseconds. If you do, the default 30,000 millisecond timeout is used instead.
- `options.ociConfig` (optional) – Oracle Cloud Infrastructure (OCI) credentials to obtain unlimited usage. For more information about providing these credentials, see [Using OCI Credentials to Obtain Additional Usage](#). If you don't specify these credentials, successful calls to `documentCapture.documentToStructure(options)` consume usage from the free monthly usage pool of requests provided in NetSuite by default.

The `documentCapture.documentToStructure(options)` method returns a `documentCapture.Document` object with the following structure:

```

1 {
2   mimeType: string,
3   pages: {
4     fields: Field[],
5     lines: Line[],
6     tables: Table[],
7     words: Word[]
8   }
9 }
```

The data that's available in this object depends on the features you specify when you call `documentCapture.documentToStructure(options)`. For example, this object includes fields (as `documentCapture.Field` objects) only when you specify the `Feature.FIELD_EXTRACTION` feature.

Keep the following considerations in mind:

- The `documentCapture.documentToStructure(options)` method extracts content synchronously and supports documents up to five pages in length. If you want to extract content from longer documents,

you must submit an asynchronous task using the N/task module. For an example, see [Extract Content from a Document Asynchronously](#).

- Encrypted files are not supported.

Using OCI Credentials to Obtain Additional Usage

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

NetSuite provides a free monthly usage pool of requests for the N/documentCapture module. Successful calls to [documentCapture.documentToStructure\(options\)](#) consume usage from this pool, and the pool is refreshed each month. You can track your current monthly usage on the AI Preferences page in NetSuite. For more information, see the help topic [View SuiteScript AI Usage Limit and Usage](#). Calls to [documentCapture.documentToText\(options\)](#) don't consume usage from this pool.

Each SuiteApp installed in your account gets its own separate monthly usage pool for N/documentCapture methods, and these SuiteApp pools are independent from the usage pool for your regular (non-SuiteApp) scripts. For example, if you install two SuiteApps, each with scripts that use N/documentCapture methods, each SuiteApp draws from its own unique usage pool. This approach means you get twice the total SuiteApp usage (one pool per SuiteApp). Any other scripts outside of SuiteApps use a separate usage pool, and SuiteApp usage doesn't count against it. This setup ensures that SuiteApps can't use up all your monthly allocation and block your own scripts from calling N/documentCapture methods.

If you want more monthly usage, you can provide the Oracle Cloud Infrastructure (OCI) credentials for an Oracle Cloud account that includes the OCI Document Understanding service. When you provide these credentials, usage is consumed from the provided OCI account instead of the free usage pool provided by NetSuite. You can provide OCI configuration parameters in two ways:

- Using the AI Preferences page in NetSuite. For more information, see the help topic [Manage AI Preferences](#).
- Using the options.ociConfig parameter of [documentCapture.documentToStructure\(options\)](#). When using this approach, the credentials you provide override any OCI credentials that are configured on the AI Preferences page. For more information, see the help topic [Configure OCI Credentials for AI](#).

For a list of required OCI configuration parameters for synchronous requests, see [documentCapture.documentToStructure\(options\)](#). For asynchronous requests (those that use a document capture task in the N/task module), you must provide three additional OCI configuration parameters: objectStorageNamespace, outputBucketName, and inputBucketName. You can provide these additional parameters in two ways:

- Using the fields in the **Optional configuration for Asynchronous Document Capture** section on the Settings subtab of the AI Preferences page.

Optional configuration for Asynchronous Document Capture

OCI OBJECT STORAGE NAMESPACE

OCI INPUT BUCKET

OCI OUTPUT BUCKET

[Create your API secrets here](#)

- Using the ociConfig property of the document capture task. When using this approach, the credentials you provide override any OCI credentials that are configured on the AI Preferences page. Here is the full list of parameters required in the object you provide for the ociConfig property:

```

1 docTask.ociConfig = {
2   userId: 'user-ocid',
3   tenancyId: 'user-tenancy',
4   compartmentId: 'user-compartment',
5   fingerprint: 'custsecret_secret_fingerprint_id',
6   privateKey: 'custsecret_secret_privatekey_id',
7   objectStorageNamespace: 'oraclenetsuite',
8   outputBucketName: 'in-bucket-name',
9   inputBucketName: 'out-bucket-name'
10 };

```

Field Extraction Objects

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

The objects in this section are related to extracting fields (as key-value pairs) from a document. When you call `documentCapture.documentToStructure(options)` and specify the Feature.FIELD_EXTRACTION feature, the returned `documentCapture.Document` object includes values for these objects.

- `documentCapture.Field`
- `documentCapture.FieldLabel`
- `documentCapture.FieldValue`

documentCapture.Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted field from a document. When extracted from a document, a field is represented by a key-value pair. The key is the name or label of the field (as a <code>documentCapture.FieldLabel</code> object), and the value is the field's value (as a <code>documentCapture.FieldValue</code> object). This object includes properties for the field label (<code>Field.label</code>), type (<code>Field.type</code>), and value (<code>Field.value</code>). Fields are included in extracted content only when the <code>documentCapture.Feature.FIELD_EXTRACTION</code> feature is specified when you call <code>documentCapture.documentToStructure(options)</code> (or when you submit an asynchronous extraction task then parse the result using <code>documentCapture.parseResult(options)</code>).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Methods and Properties	Field Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const field = extractedData.pages[0].fields[0];
11 ...
12 ...
13 // Add additional code

```

Field.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The label (name) of the field.
Type	documentCapture.FieldLabel
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Field
Sibling Object Members	Field Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,

```

```

7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8   });
9
10  const fieldLabel = extractedData.pages[0].fields[0].label;
11
12  ...
13  // Add additional code

```

Field.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The type of the field. This property can have values from the documentCapture.FieldType enum. The field type describes the type or category of data extracted for a specific field. It adds semantic meaning to the extracted information, which can help you understand and process the information more effectively.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Field
Sibling Object Members	Field Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  const extractedData = documentCapture.documentToStructure({
5    file: file.load("SuiteScripts/sample_invoice.pdf"),
6    documentType: documentCapture.DocumentType.INVOICE,
7    features: [documentCapture.Feature.FIELD_EXTRACTION]
8  });
9
10 const fieldType = extractedData.pages[0].fields[0].type;
11
12 ...
13 // Add additional code

```

Field.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The value of the field.
Type	documentCapture.FieldValue
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Field
Sibling Object Members	Field Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldValue = extractedData.pages[0].fields[0].value;
11
12 ...
13 // Add additional code

```

documentCapture.FieldLabel

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted field label from a document. This object includes properties for the confidence level (FieldLabel.confidence) and name (FieldLabel.name) of the field label.
---------------------------	--

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Methods and Properties	FieldLabel Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5     file: file.load("SuiteScripts/sample_invoice.pdf"),
6     documentType: documentCapture.DocumentType.INVOICE,
7     features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldLabel = extractedData.pages[0].fields[0].label;
11
12 ...
13 // Add additional code

```

FieldLabel.confidence

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The confidence level for the field label. This property is a number between 0 and 1 indicating how confident the service is in the accuracy of the extracted name (FieldLabel.name) for the field label. For example, a confidence value of 0.95 means that the service is 95% confident about the extracted name. You can use the confidence level to set a threshold for accepting the extracted name in your scripts. For example, to minimize the risk of errors, you could choose to accept only names with a confidence level above 0.9 (or 90%).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.FieldLabel
Sibling Object Members	FieldLabel Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_tax_form.pdf"),
6   documentType: documentCapture.DocumentType.TAX_FORM,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldLabelToProcess = extractedData.pages[0].fields[0].label;
11
12 if (fieldLabelToProcess.confidence > 0.9) {
13   // Accept the field name
14 } else {
15   // Reject the field name
16 }
17
18 ...
19 // Add additional code

```

FieldLabel.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The name of the field label.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.FieldLabel
Sibling Object Members	FieldLabel Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldLabelName = extractedData.pages[0].fields[0].label.name;
11
12 ...
13 // Add additional code

```

documentCapture.FieldValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted field value from a document. This object includes properties for the confidence level (FieldValue.confidence) and text (FieldValue.text) of the field value.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Methods and Properties	FieldValue Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldValue = extractedData.pages[0].fields[0].value;
11
12 ...
13 // Add additional code

```

FieldValue.confidence

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The confidence level for the field value. This property is a number between 0 and 1 indicating how confident the service is in the accuracy of the extracted value (FieldValue.text) of the field. For example, a confidence value of 0.95 means that the service is 95% confident about the extracted value. You can use the confidence level to set a threshold for accepting the extracted value in your scripts. For example, to minimize the risk of errors, you could choose to accept only values with a confidence level above 0.9 (or 90%).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.FieldValue
Sibling Object Members	FieldValue Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_tax_form.pdf"),
6   documentType: documentCapture.DocumentType.TAX_FORM,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldValueToProcess = extractedData.pages[0].fields[0].value;
11
12 if (fieldValueToProcess.confidence > 0.9) {
13   // Accept the field value
14 } else {
15   // Reject the field value
16 }
17
18 ...
19 // Add additional code

```

FieldValue.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text of the field value.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.FieldValue
Sibling Object Members	FieldValue Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldValueText = extractedData.pages[0].fields[0].value.text;
11
12 ...
13 // Add additional code

```

Table Extraction Objects

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

The objects in this section are related to extracting tables from a document. When you call [documentCapture.documentToStructure\(options\)](#) and specify the `Feature.TABLE_EXTRACTION` feature, the returned [documentCapture.Document](#) object includes values for these objects.

- [documentCapture.Cell](#)
- [documentCapture.Table](#)
- [documentCapture.TableRow](#)

documentCapture.Cell

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted table cell from a document. This object includes properties for the confidence level (Cell.confidence) and text (Cell.text) of the cell.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Methods and Properties	Cell Object Members
Since	2025.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const cell = extractedData.pages[0].tables[0].bodyRows[0].cells[0];
11
12 ...
13 // Add additional code

```

Cell.confidence

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The confidence level for the cell. This property is a number between 0 and 1 indicating how confident the service is in the accuracy of the extracted text (Cell.text) within the cell. For example, a confidence value of 0.95 means that the service is 95% confident about the extracted text.
-----------------------------	--

	You can use the confidence level to set a threshold for accepting the extracted text in your scripts. For example, to minimize the risk of errors, you could choose to accept only text with a confidence level above 0.9 (or 90%).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Cell
Sibling Object Members	Cell Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const cellToProcess = extractedData.pages[0].tables[0].bodyRows[0].cells[0];
11 if (cellToProcess.confidence > 0.9) {
12   // Accept the cell text
13 } else {
14   // Reject the cell text
15 }
16
17 ...
18 // Add additional code

```

Cell.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted text of the cell.
Type	string

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Cell
Sibling Object Members	Cell Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const cell = extractedData.pages[0].tables[0].bodyRows[0].cells[0];
11 const cellText = cell.text;
12
13 ...
14 // Add additional code

```

documentCapture.Table

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted table from a document. This object includes properties for the following: ■ Header rows – Table.headerRows ■ Body rows – Table.bodyRows ■ Footer rows – Table.footerRows ■ Number of rows – Table.rowCount ■ Number of columns – Table.columnCount ■ Confidence level – Table.confidence Tables are included in extracted content only when the <code>documentCapture.Feature.TABLE_EXTRACTION</code> feature is specified when you call
---------------------------	---

	documentCapture.documentToStructure(options) (or when you submit an asynchronous extraction task then parse the result using documentCapture.parseResult(options)). If you don't specify any features when you call this method, the TABLE_EXTRACTION feature is used by default (along with the TEXT_EXTRACTION feature).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Methods and Properties	Table Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const table = extractedData.pages[0].tables[0];
11 ...
12 ...
13 // Add additional code

```

Table.bodyRows

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted body rows from the table in a document. Body rows refer to the main content rows of a table, excluding header rows and footer rows. Each body row is a collection of cells (as documentCapture.Cell objects contained in a documentCapture.TableRow object).
Type	documentCapture.TableRow[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Table
Sibling Object Members	Table Object Members

Since	2025.2
--------------	--------

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const tableBodyRows = extractedData.pages[0].tables[0].bodyRows;
11 ...
12 ...
13 // Add additional code

```

Table.columnCount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The number of extracted columns from the table in a document.
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Table
Sibling Object Members	Table Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const tableColumnCount = extractedData.pages[0].tables[0].columnCount;
11 ...
12 ...
13 // Add additional code

```

Table.confidence

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The confidence level for the table. This property is a number between 0 and 1 indicating how confident the service is in the accuracy of the extracted rows (Table.headerRows, Table.bodyRows, and Table.footerRows) of the table. For example, a confidence value of 0.95 means that the service is 95% confident about the extracted rows. You can use the confidence level to set a threshold for accepting the extracted rows in your scripts. For example, to minimize the risk of errors, you could choose to accept only rows with a confidence level above 0.9 (or 90%).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.Table
Sibling Object Members	Table Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const tableToProcess = extractedData.pages[0].tables[0];
11
12 if (tableToProcess.confidence > 0.9) {
13   // Accept the table
14 } else {
15   // Reject the table
16 }
17
18 ...
19 // Add additional code

```

Table.footerRows

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted footer rows from the table in a document. Footer rows refer to rows located at the bottom of a table. These rows often contain summary information, totals, or other information related to the table's content. For example, for a receipt, footer rows could show the subtotal, tax amounts, and the total amount due. Each footer row is a collection of cells (as documentCapture.Cell objects contained in a documentCapture.TableRow object).
Type	documentCapture.TableRow[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.Table
Sibling Object Members	Table Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const tableFooterRows = extractedData.pages[0].tables[0].footerRows;
11 ...
12 ...
13 // Add additional code

```

Table.headerRows

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted header rows from the table in a document. Header rows refer to rows located at the top of a table. These rows contain labels that help to organize the data in subsequent rows. Each header row is a collection of cells (as documentCapture.Cell objects contained in a documentCapture.TableRow object).
Type	documentCapture.TableRow[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Table
Sibling Object Members	Table Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code

```



```

2  ...
3
4  const extractedData = documentCapture.documentToStructure({
5    file: file.load("SuiteScripts/sample_invoice.pdf"),
6    documentType: documentCapture.DocumentType.INVOICE,
7    features: [documentCapture.Feature.TABLE_EXTRACTION]
8  });
9
10 const tableHeaderRows = extractedData.pages[0].tables[0].headerRows;
11
12 ...
13 // Add additional code

```

Table.rowCount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The number of extracted rows from the table in a document. The value of this property includes header rows, body rows, and footer rows.
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Table
Sibling Object Members	Table Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  const extractedData = documentCapture.documentToStructure({
5    file: file.load("SuiteScripts/sample_invoice.pdf"),
6    documentType: documentCapture.DocumentType.INVOICE,
7    features: [documentCapture.Feature.TABLE_EXTRACTION]
8  });
9
10 const tableRowCount = extractedData.pages[0].tables[0].rowCount;
11
12 ...

```

13 // Add additional code

documentCapture.TableRow

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted table row from a document. This object includes a property for the row cells (TableRow.cells).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Methods and Properties	TableRow Object Members
Since	2025.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const tableRow = extractedData.pages[0].tables[0].bodyRows[0];
11 ...
12 ...
13 // Add additional code

```

TableRow.cells

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted cells in the table row.
Type	documentCapture.Cell[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module

Parent Object	documentCapture.TableRow
Sibling Object Members	TableRow Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const tableRowCells = extractedData.pages[0].tables[0].bodyRows[0].cells;
11
12 ...
13 // Add additional code

```

Text Extraction Objects

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

The objects in this section are related to extracting text from a document. When you call [documentCapture.documentToStructure\(options\)](#) and specify the `Feature.TEXT_EXTRACTION` feature, the returned [documentCapture.Document](#) object includes values for these objects.

- [documentCapture.Line](#)
- [documentCapture.Word](#)

documentCapture.Line

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted line of text from a document.
---------------------------	--

	A line represents the text content on a single line in the document, and it depends on how the document is formatted. It isn't the same as a sentence (which usually ends with a period). This object includes properties for the confidence level (Line.confidence) and text (Line.text) of the line. Lines are included in extracted content only when the <code>documentCapture.Feature.TEXT_EXTRACTION</code> is specified when you call <code>documentCapture.documentToStructure(options)</code> (or when you submit an asynchronous extraction task then parse the result using <code>documentCapture.parseResult(options)</code>). If you don't specify any features when you call this method, the <code>TEXT_EXTRACTION</code> feature is used by default (along with the <code>TABLE_EXTRACTION</code> feature).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Methods and Properties	Line Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const line = extractedData.pages[0].lines[0];
11 ...
12 ...
13 // Add additional code

```

Line.confidence

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The confidence level for the line. This property is a number between 0 and 1 indicating how confident the service is in the accuracy of the extracted text (Line.text) of the line. For example, a confidence value of 0.95 means that the service is 95% confident about the extracted text. You can use the confidence level to set a threshold for accepting the extracted line in your scripts. For example, to minimize the risk of errors, you could choose to accept only lines with a confidence level above 0.9 (or 90%).
Type	number

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Line
Sibling Object Members	Line Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_parking_application.pdf"),
6   documentType: documentCapture.DocumentType.OTHERS,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const lineToProcess = extractedData.pages[0].lines[0];
11
12 if (lineToProcess.confidence > 0.9) {
13   // Accept the line
14 } else {
15   // Reject the line
16 }
17
18 ...
19 // Add additional code

```

Line.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text of the line.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module

Parent Object	documentCapture.Line
Sibling Object Members	Line Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const lineText = extractedData.pages[0].lines[0].text;
11
12 ...
13 // Add additional code

```

documentCapture.Word

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted word from a document. This object includes properties for the confidence level (Word.confidence) and text (Word.text) of the word. Words are included in extracted content only when the <code>documentCapture.Feature.TEXT_EXTRACTION</code> feature is specified when you call documentCapture.documentToStructure(options) (or when you submit an asynchronous extraction task then parse the result using documentCapture.parseResult(options)). If you don't specify any features when you call this method, the <code>TEXT_EXTRACTION</code> feature is used by default (along with the <code>TABLE_EXTRACTION</code> feature).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Methods and Properties	Word Object Members

Since	2025.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const word = extractedData.pages[0].words[0];
11 ...
12 ...
13 // Add additional code

```

Word.confidence

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The confidence level for the word. This property is a number between 0 and 1 indicating how confident the service is in the accuracy of the extracted text (Word.text) of the word. For example, a confidence value of 0.95 means that the service is 95% confident about the extracted text. You can use the confidence level to set a threshold for accepting the extracted word in your scripts. For example, to minimize the risk of errors, you could choose to accept only words with a confidence level above 0.9 (or 90%).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.Word
Sibling Object Members	Word Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_parking_application.pdf"),
6   documentType: documentCapture.DocumentType.OTHERS,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const wordToProcess = extractedData.pages[0].words[0];
11
12 if (wordToProcess.confidence > 0.9) {
13   // Accept the word
14 } else {
15   // Reject the word
16 }
17
18 ...
19 // Add additional code

```

Word.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text of the extracted word.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Word
Sibling Object Members	Word Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code

```



```

2  ...
3
4  const extractedData = documentCapture.documentToStructure({
5    file: file.load("SuiteScripts/sample_invoice.pdf"),
6    documentType: documentCapture.DocumentType.INVOICE,
7    features: [documentCapture.Feature.TEXT_EXTRACTION]
8  });
9
10 const wordText = extractedData.pages[0].words[0].text;
11
12 ...
13 // Add additional code

```

documentCapture.Document

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The extracted data from a document. This object is returned from documentCapture.documentToStructure(options) and documentCapture.parseResult(options). It includes properties for the MIME type (Document.mimeType) and extracted pages (Document.pages) of a document. The data in this object is structured as follows: <pre> 1 { 2 mimeType: string, 3 pages: { 4 fields: Field[], 5 lines: Line[], 6 tables: Table[], 7 words: Word[] 8 } 9 } </pre> The data that's available in this object depends on the features you specify when you call documentCapture.documentToStructure(options), or when you submit an asynchronous extraction task then parse the result using documentCapture.parseResult(options). For example, this object includes fields (as documentCapture.Field objects) only when you specify the documentCapture.Feature.FIELD_EXTRACTION feature. If you don't specify any features, the documentCapture.Feature.TEXT_EXTRACTION and documentCapture.Feature.TABLE_EXTRACTION features are used by default.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Methods and Properties	Document Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5     file: file.load("SuiteScripts/sample_invoice.pdf"),
6     documentType: documentCapture.DocumentType.INVOICE
7 });
8
9 ...
10 // Add additional code

```

Document.mimeType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The MIME type of the document. This property can contain the following MIME types, depending on the extension of the file provided to documentCapture.documentToStructure(options) or parsed using documentCapture.parseResult(options): ■ JPG – image/jpeg ■ PDF – application/pdf ■ PNG – image/png ■ TIFF – image/tiff
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.Document
Sibling Object Members	Document Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5     file: file.load("SuiteScripts/sample_invoice.pdf"),
6     documentType: documentCapture.DocumentType.INVOICE
7 });
8
9 const mimeType = extractedData.mimeType;
10
11 ...
12 // Add additional code

```

Document.pages

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The pages of the document. The documentCapture.documentToStructure(options) method supports documents up to five pages in length, so the returned documentCapture.Document object can contain up to five pages (as documentCapture.Page objects). When you submit an asynchronous extraction task using the N/task module, you can provide documents of any length, so the returned object contains as many pages as were in the provided document.
Type	documentCapture.Page[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.Document
Sibling Object Members	Document Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE
7 });
8
9 const pages = extractedData.pages;
10
11 ...
12 // Add additional code

```

Document.getText()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the entire text of the document.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/documentCapture Module
Parent Object	documentCapture.Document
Sibling Object Members	Document Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE
7 });
8
9 const documentText = extractedData.getText();
10
11 ...
12 // Add additional code

```

documentCapture.Page

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An extracted page from a document. This object includes properties for the fields (Page.fields), lines (Page.lines), tables (Page.tables), and words (Page.words) of the page. These properties have values only when the associated feature is specified when you call documentCapture.documentToStructure(options). For example, the Page.fields property includes extracted fields only if the <code>documentCapture.Feature.FIELD_EXTRACTION</code> feature is specified. If you don't specify any features when you call this method, the <code>TEXT_EXTRACTION</code> and <code>TABLE_EXTRACTION</code> features are used by default.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Methods and Properties	Page Object Members
Since	2025.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE
7 });
8
9 const page = extractedData.pages[0];
10
11 ...
12 // Add additional code

```

Page.detectedDocumentTypes

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	A set of confidence levels indicating whether the page represents a particular type of document. This property is a set of objects, and each object has a <code>documentType</code> value and <code>confidence</code> value. The <code>documentType</code> value is a type of supported document (such as "INVOICE"), and the
-----------------------------	---

	confidence value is a number between 0 and 1 indicating how confident the service is about whether the page is a document of that type. For example, a <code>documentType</code> value of "INVOICE" and a confidence value of 0.95 means that the service is 95% confident that the page represents an invoice. This property has a value only when the <code>documentCapture.Feature.DOCUMENT_CLASSIFICATION</code> feature is specified when you call documentCapture.documentToStructure(options).
Type	Object[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/documentCapture Module
Parent Object	documentCapture.Page
Sibling Object Members	Page Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   features: [
7     documentCapture.Feature.FIELD_EXTRACTION,
8     documentCapture.Feature.DOCUMENT_CLASSIFICATION
9   ]
10 });
11
12 const pageDetectedDocumentTypes = extractedData.pages[0].detectedDocumentTypes;
13 ...
14 ...
15 // Add additional code

```

Page.fields

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted fields from the page of a document.
-----------------------------	---

	This property has a value only when the <code>documentCapture.Feature.FIELD_EXTRACTION</code> feature is specified when you call <code>documentCapture.documentToStructure(options)</code> .
Type	<code>documentCapture.Field[]</code>
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	<code>documentCapture.Page</code>
Sibling Object Members	Page Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const pageFields = extractedData.pages[0].fields;
11 ...
12 ...
13 // Add additional code

```

Page.lines

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted lines from the page of a document. This property has a value only when the <code>documentCapture.Feature.TEXT_EXTRACTION</code> feature is specified when you call <code>documentCapture.documentToStructure(options)</code>. If you don't specify any features when you call this method, the <code>TEXT_EXTRACTION</code> feature is used by default (along with the <code>TABLE_EXTRACTION</code> feature).
-----------------------------	--

Type	documentCapture.Line[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Page
Sibling Object Members	Page Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE
7 });
8
9 const pageLines = extractedData.pages[0].lines;
10
11 ...
12 // Add additional code

```

Page.tables

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted tables from the page of a document. This property has a value only when the <code>documentCapture.Feature.TABLE_EXTRACTION</code> feature is specified when you call documentCapture.documentToStructure(options) . If you don't specify any features when you call this method, the <code>TABLE_EXTRACTION</code> feature is used by default (along with the <code>TEXT_EXTRACTION</code> feature).
Type	documentCapture.Table[]

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Page
Sibling Object Members	Page Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TABLE_EXTRACTION]
8 });
9
10 const pageTables = extractedData.pages[0].tables;
11
12 ...
13 // Add additional code

```

Page.words

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The extracted words from the page of a document. This property has a value only when the <code>documentCapture.Feature.TEXT_EXTRACTION</code> feature is specified when you call <code>documentCapture.documentToStructure(options)</code> . If you don't specify any features when you call this method, the <code>TEXT_EXTRACTION</code> feature is used by default (along with the <code>TABLE_EXTRACTION</code> feature).
Type	<code>documentCapture.Word[]</code>
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/documentCapture Module
Parent Object	documentCapture.Page
Sibling Object Members	Page Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY	You try to set the value of this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const pageWords = extractedData.pages[0].words;
11
12 ...
13 // Add additional code

```

Page.getText()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the entire text of the page.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/documentCapture Module
Parent Object	documentCapture.Document

Sibling Object Members	Document Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.TEXT_EXTRACTION]
8 });
9
10 const pageText = extractedData.pages[0].getText();
11 ...
12 ...
13 // Add additional code

```

documentCapture.documentToStructure(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Extracts content from a document. This method can return the text content, table content, and key-value pairs (fields) from the specified document located in the NetSuite File Cabinet. The content returned depends on the features you specify when you call this method (using the <code>options.features</code> parameter). Use the documentCapture.Feature enum to specify the features you want to extract, such as <code>TEXT_EXTRACTION</code>, <code>TABLE_EXTRACTION</code>, and <code>FIELD_EXTRACTION</code>. This method extracts content synchronously and supports documents up to five pages in length. If you want to extract content from documents longer than five pages, you must submit an asynchronous extraction task using the N/task module. For an example, see Extract Content from a Document Asynchronously. This method supports PDF, JPG, PNG, and TIFF files. Encrypted files are not supported. This method consumes usage from the monthly usage pool of free requests provided by NetSuite. Each successful call to this method counts as a request, and the AI Preferences page lets you track your free usage. For more information, see the help topic Manage SuiteScript AI Preferences. If you need to submit more requests per month than the free usage pool provides, you can provide Oracle Cloud account credentials for unlimited usage. For more information, see the help topic Configure OCI Credentials for AI.
Returns	documentCapture.Document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100

Module	N/documentCapture Module
Since	2025.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.file	file.File	required	The document file to extract content from. The specified file must be located in the NetSuite File Cabinet, be in PDF, JPG, PNG, or TIFF format, and be five pages in length or shorter. You can specify the file using its internal ID or the path to the file in the File Cabinet. For more information, see N/file Module. Encrypted files are not supported.	2025.2
options.documentType	string	optional	The document type.  Note: This parameter is required if you specify the FIELD_EXTRACTION feature using the options.features parameter. Use values from the documentCapture.DocumentType enum to set this parameter. By specifying the type of document, the service can apply pretrained models that are optimized for that type, which can provide more accurate extraction results. If you don't specify a document type when you call this method, the OTHERS document type is used by default.	2025.2
options.features	string[]	optional	The features to extract from the specified document. Use values from the documentCapture.Feature enum to set this property. If you don't specify any features when you call this method, the TEXT_EXTRACTION and TABLE_EXTRACTION features are used by default.	2025.2
options.language	string	optional	The language of the specified document. Use values from the documentCapture.Language enum to set this property. If you don't specify a language when you call this method, the ENG (English) language is used by default.	2025.2
options.ociConfig	Object	optional	Configuration parameters for unlimited usage through the OCI Document Understanding service. This parameter is required only when accessing the OCI Document Understanding service through an Oracle Cloud account. The credentials you provide here override any OCI credentials that are configured on the AI Preferences page.	2025.2
options.ociConfig.compartmentId	string	optional	Compartment OCID. For more information, refer to Managing Compartments in the <i>Oracle Cloud Infrastructure Documentation</i>.	2025.2
options.ociConfig.endpointId	string	optional	Endpoint ID. This value is required only when an OCI dedicated AI cluster (DAC) is used. For more information, refer to Managing Endpoints in the <i>Oracle Cloud Infrastructure Documentation</i>.	2025.2

Parameter	Type	Required / Optional	Description	Since
options.ociConfig.fingerprint	string	optional	Fingerprint of the public key. Only a NetSuite secret is accepted. For more information about secrets, see the help topic Creating Secrets). For more information about public key fingerprints, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.2
options.ociConfig.privateKey	string	optional	Private key of the OCI user. Only a NetSuite secret is accepted. For more information about secrets, see the help topic Creating Secrets). For more information about private keys, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.2
options.ociConfig.tenancyId	string	optional	Tenancy OCID. For more information, refer to Viewing Tenancy Details in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.2
options.ociConfig.userId	string	optional	User OCID. For more information, refer to Managing Users in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.2
options.timeout	number	optional	The timeout period to wait for a response from the service. By default, the timeout period is 30,000 milliseconds (30 seconds). You can specify a longer timeout period, but you can't specify one that's shorter than 30,000 milliseconds. If you try to specify a shorter timeout period, the default value of 30,000 milliseconds is used instead.	2025.2

Errors

Error Code	Thrown If
ACCESS_DENIED	The specified OCI credentials (in the options.ociConfig parameter) don't provide access to the OCI Document Understanding service.
DOCUMENT_TOO_LONG	The specified file is longer than five pages. To extract content from documents longer than five pages, submit an asynchronous extraction task using the N/task module. For an example, see Extract Content from a Document Asynchronously .
FEATURES_CANNOT_BE_EMPTY	The list of features to extract is empty (that is, an empty array is specified in the options.features parameter).
FEATURE_1_DOES_NOT_SUPPORT_LANGUAGE_2	A feature is specified (using the options.features parameter) that is not supported in the specified language.
INCOMPATIBLE_DOCUMENT_TYPE_FOR_FEATURE_1	A feature is specified (using the options.features parameter) that is not supported for the specified document type.
INVALID_DOCUMENT_CAPTURE_RESULT	The document capture result provided by the service is invalid.
INVALID_DOCUMENT_TYPE	The specified document type is not included in the documentCapture.DocumentType enum.
INVALID_LANGUAGE	The specified language is not included in the documentCapture.Language enum.

Error Code	Thrown If
MAXIMUM_PARALLEL_REQUESTS_LIMIT_EXCEEDED	The number of parallel requests to the service is greater than five. A maximum of five parallel requests are supported.
MONTHLY_QUOTA_OF_1_SUITE_SCRIPT_DOCUMENT_CAPTURE_REQUESTS_HAS_BEEN_MET	You've exceeded the number of monthly free requests provided by NetSuite. For more information, see Using OCI Credentials to Obtain Additional Usage .
ONLY_API_SECRET_IS_ACCEPTED	One (or both) of the options.ociConfig.fingerprint or options.ociConfig.privateKey parameters are not NetSuite secrets.
SSS_MISSING_REQD_ARGUMENT	The required options.file parameter is not specified.
UNRECOGNIZED_OCI_CONFIG_PARAMETERS	The object specified in the options.ociConfig parameter includes unknown properties or values.
UNSUPPORTED_FILE_TYPE	The specified file is not in PDF, JPG, PNG, or TIFF format.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [
8     documentCapture.Feature.TEXT_EXTRACTION,
9     documentCapture.Feature.FIELD_EXTRACTION
10  ]
11 });
12 ...
13 ...
14 // Add additional code

```

documentCapture.documentToStructure.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously extracts content from a document. This method can return the text content, table content, and key-value pairs (fields) from the specified document located in the NetSuite File Cabinet. The content returned depends on the features you specify when you call this method (using the options.features parameter). Use the documentCapture.Feature enum to specify the features you want to extract, such as TEXT_EXTRACTION, TABLE_EXTRACTION, and FIELD_EXTRACTION. This method supports documents up to five pages in length. If you want to extract content from documents longer than five pages, you must submit an asynchronous extraction task using the N/task module. For an example, see Extract Content from a Document Asynchronously. This method supports PDF, JPG, PNG, and TIFF files. Encrypted files are not supported. This method consumes usage from the monthly usage pool of free requests provided by NetSuite. Each successful call to this method counts as a request, and the AI Preferences page
---------------------------	--

	lets you track your free usage. For more information, see the help topic Manage SuiteScript AI Preferences. If you need to submit more requests per month than the free usage pool provides, you can provide Oracle Cloud account credentials for unlimited usage. For more information, see the help topic Configure OCI Credentials for AI.  Note: The parameters and errors thrown for this method are the same as those for <code>documentCapture.documentToStructure(options)</code>. For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	documentCapture.documentToStructure(options)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100
Module	N/documentCapture Module
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 documentCapture.documentToStructure.promise({
5     file: file.load("SuiteScripts/sample_invoice.pdf"),
6     documentType: documentCapture.DocumentType.INVOICE,
7     features: [
8         documentCapture.Feature.TEXT_EXTRACTION,
9         documentCapture.Feature.FIELD_EXTRACTION
10    ]
11 }).then(function(result) {
12     // Work with the result
13 });
14
15 ...
16 // Add additional code

```

documentCapture.documentToText(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Extracts text content from a PDF file. This method returns a string with the text of the specified PDF file located in the NetSuite File Cabinet. If you want to extract other content from a file, such as tables and fields (key-value pairs), or extract content from a JPG, PNG, or TIFF file, use documentCapture.documentToStructure(options) instead. Encrypted files are not supported.
---------------------------	---

You can use the text returned from this method in calls to N/llm methods for further querying. For example, you can provide the returned text to [llm.generateText\(options\)](#) and ask questions about the data, as the following code sample shows:

```

1 // "14" is the unique ID of a PDF file stored in the NetSuite File Cabinet
2 const fileObj = file.load({
3   id: "14"
4 });
5 const extractedData = documentCapture.documentToText({
6   file: fileObj
7 });
8
9 const response = llm.generateText({
10  prompt: "What is this invoice for?",
11  documents: [{
12    id: '14',
13    data: extractedData
14  }]
15 });

```

This method doesn't consume usage from the monthly usage pool of free requests provided by NetSuite (unlike [documentCapture.documentToStructure\(options\)](#), which does consume usage).

Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/documentCapture Module
Since	2025.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.file	file.File	required	The PDF file to extract content from. The specified file must be located in the NetSuite File Cabinet and be in PDF format. You can specify the file using its internal ID or the path to the file in the File Cabinet. For more information, see N/file Module . Encrypted files are not supported.	2025.2
options.timeout	number	optional	The timeout period to wait for a response from the service. By default, the timeout period is 30,000 milliseconds (30 seconds). You can specify a longer timeout period, but you can't specify one that's shorter than 30,000 milliseconds. If you try to specify a shorter timeout period, the default value of 30,000 milliseconds is used instead.	2025.2

Errors

Error Code	Thrown If
FILE_CANNOT_BE_EMPTY	The specified file is empty.

Error Code	Thrown If
FILE_CORRUPTED_OR_INVALID	The specified file couldn't be parsed. It could be corrupted or invalid.
UNSUPPORTED_ENCODING_EXCEPTION	The specified file is corrupted or contains invalid characters.
UNSUPPORTED_FILE_TYPE_1_USE_2	The specified file is not a PDF file.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // "14" is the unique ID of a PDF file stored in the NetSuite File Cabinet
5 const fileObj = file.load({
6   id: "14"
7 });
8 const extractedData = documentCapture.documentToText({
9   file: fileObj,
10  timeout: 40000
11 });
12 ...
13 ...
14 // Add additional code

```

documentCapture.documentToText.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously extracts text content from a PDF file. This method returns the text of the specified PDF file located in the NetSuite File Cabinet. If you want to extract other content from a file, such as tables and fields (key-value pairs), or extract content from a JPG, PNG, or TIFF file, use documentCapture.documentToStructure(options) instead. Encrypted files are not supported. You can use the text returned from this method in calls to N/llm methods for further querying. For example, you can provide the returned text to llm.generateText(options) and ask questions about the data, as the following code sample shows: <pre> 1 // "14" is the unique ID of a PDF stored in the NetSuite File Cabinet 2 const fileObj = file.load({ 3 id: "14" 4 }); 5 documentCapture.documentToText.promise({ 6 file: fileObj 7 }).then(function(result) { 8 const response = llm.generateText({ 9 prompt: "What is this invoice for?", 10 documents: [{ 11 id: '14', 12 data: result </pre>
---------------------------	--

	<pre> 13 }} 14 }) 15 }; </pre> This method doesn't consume usage from the monthly usage pool of free requests provided by NetSuite (unlike documentCapture.documentToStructure(options), which does consume usage).  Note: The parameters and errors thrown for this method are the same as those for documentCapture.documentToText(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	documentCapture.documentToText(options)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/documentCapture Module
Since	2025.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // "14" is the unique ID of a PDF stored in the NetSuite File Cabinet
5 const fileObj = file.load({
6   id: "14"
7 });
8 documentCapture.documentToText.promise({
9   file: fileObj,
10  timeout: 40000
11 }).then(function(result) {
12   // Work with the result
13 });
14 ...
15 ...
16 // Add additional code

```

documentCapture.getRemainingConcurrency()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the number of available concurrent requests remaining.
---------------------------	--

	You can submit up to five concurrent requests to documentCapture.documentToText(options) or documentCapture.documentToStructure(options) (or their promise versions).
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/documentCapture Module
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const concurrency = documentCapture.getRemainingConcurrency();
5
6 ...
7 // Add additional code

```

documentCapture.getRemainingConcurrency.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the number of available concurrent requests remaining. You can submit up to five concurrent requests to documentCapture.documentToText(options) or documentCapture.documentToStructure(options) (or their promise versions).  Note: The parameters and errors thrown for this method are the same as those for documentCapture.getRemainingConcurrency(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	documentCapture.getRemainingConcurrency()
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/documentCapture Module
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 documentCapture.getRemainingConcurrency.promise().then(function(result) {
5     // Work with the result
6 });
7
8 ...
9 // Add additional code

```

documentCapture.getRemainingFreeUsage()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the number of free document capture requests remaining for the current month. This method tracks usage for the documentCapture.documentToStructure(options) method. The documentCapture.documentToText(options) method doesn't consume usage from the free usage pool. You can track your current monthly usage on the AI Preferences page in NetSuite. For more information, see the help topic View SuiteScript AI Usage Limit and Usage .
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/documentCapture Module
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3
4  const usage = documentCapture.getRemainingFreeUsage();
5
6  ...
7  // Add additional code

```

documentCapture.getRemainingFreeUsage.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the number of free document capture requests remaining for the current month. This method tracks usage for the documentCapture.documentToStructure(options) method. The documentCapture.documentToText(options) method doesn't consume usage from the free usage pool. You can track your current monthly usage on the AI Preferences page in NetSuite. For more information, see the help topic View SuiteScript AI Usage Limit and Usage. Note: The parameters and errors thrown for this method are the same as those for documentCapture.getRemainingFreeUsage(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	documentCapture.getRemainingFreeUsage()
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/documentCapture Module
Since	2025.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  documentCapture.getRemainingFreeUsage.promise().then(function(result) {
5      // Work with the result
6  });
7
8  ...

```

9 | // Add additional code

documentCapture.parseResult(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Converts a JSON file into a documentCapture.Document object. When you submit an asynchronous extraction request using the N/task module, the results are returned as a JSON object. You can save the results to a file in the NetSuite File Cabinet, then use this method to convert the results to a documentCapture.Document object for further processing or analysis. For an example of submitting an asynchronous extraction request, see Extract Content from a Document Asynchronously .
Returns	documentCapture.Document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/documentCapture Module
Since	2025.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.file	file.File	required	The file to parse. This file must be located in the NetSuite File Cabinet and be in JSON format.	2025.2

Errors

Error Code	Thrown If
UNSUPPORTED_FILE_TYPE_1_USE_2	The specified file is not a JSON file.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

1 | // Add additional code

```

2 | ...
3 |
4 | // '114' is the unique ID of the results file in the NetSuite File Cabinet
5 | const fileObj = file.load(
6 |   id: '114'
7 | );
8 | const parsedResult = documentCapture.parseResult(fileObj);
9 | const fullResultText = parsedResult.getText();
10 |
11 | ...
12 | // Add additional code

```

documentCapture.DocumentType

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	Holds values for the document type. Use this enum to specify the type of document you're providing when you call documentCapture.documentToStructure(options) (using the <code>options.documentType</code> parameter). By specifying the type of document, the service can apply pretrained models that are optimized for that type, which can provide more accurate extraction results.
Module	N/documentCapture Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Since	2025.2

Values

Value
BANK_STATEMENT
CHECK
DRIVER_LICENSE
HEALTH_INSURANCE_ID
INVOICE
OTHERS
PASSPORT
PAYSLIP
RECEIPT
RESUME

Value
TAX_FORM

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```
1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [
8     documentCapture.Feature.TEXT_EXTRACTION,
9     documentCapture.Feature.FIELD_EXTRACTION
10  ]
11 });
12
13 ...
14 // Add additional code
```

documentCapture.Feature

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Enum Description	Holds values for the feature to extract from a document. Use this enum to specify the document features you want to extract when you call documentCapture.documentToStructure(options) (using the options.features parameter). If you don't specify any features when you call this method, the TEXT_EXTRACTION and TABLE_EXTRACTION features are used by default.
Module	N/documentCapture Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Since	2025.2

Values

Value
DOCUMENT_CLASSIFICATION
FIELD_EXTRACTION
TABLE_EXTRACTION

Value
TEXT_EXTRACTION

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [
8     documentCapture.Feature.TEXT_EXTRACTION,
9     documentCapture.Feature.FIELD_EXTRACTION
10  ]
11 });
12
13 ...
14 // Add additional code

```

documentCapture.FieldType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	Holds values for the type of a field. These enum values are returned as part of a documentCapture.Field object (in the Field.type property). They describe the category or type of data extracted for a specific field, providing semantic meaning and helping with structured processing of the data. documentCapture.Field objects are included in extracted content from a document only when the <code>documentCapture.Feature.FIELD_EXTRACTION</code> feature is specified when you call documentCapture.documentToStructure(options).
Module	N/documentCapture Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Since	2025.2

Values

Value
KEY_VALUE
LINE_ITEM

Value
LINE_ITEM_FIELD
LINE_ITEM_GROUP
UNKNOWN

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("SuiteScripts/sample_invoice.pdf"),
6   documentType: documentCapture.DocumentType.INVOICE,
7   features: [documentCapture.Feature.FIELD_EXTRACTION]
8 });
9
10 const fieldType = extractedData.pages[0].fields[0].type;
11
12 ...
13 // Add additional code

```

documentCapture.Language

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	Holds values for the language of a document. Use this enum to specify the language of the document when you call documentCapture.documentToStructure(options) (using the options.language parameter). If you don't specify a language when you call this method, the ENG (English) language is used by default.
Module	N/documentCapture Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Since	2025.2

Values

Value	Language
ARA	Arabic

Value	Language
CES	Czech
CHI_SIM	Simplified Chinese
DAN	Danish
DEU	German
ELL	Greek
ENG	English
FIN	Finnish
FRA	French
HIN	Hindi
HUN	Hungarian
ITA	Italian
JPN	Japanese
KOR	Korean
NLD	Dutch
NOR	Norwegian
OTHERS	Other languages not listed here
POL	Polish
POR	Portuguese
RON	Romanian
RUS	Russian
SLK	Slovak
SWE	Swedish
TUR	Turkish

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/documentCapture Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const extractedData = documentCapture.documentToStructure({
5   file: file.load("14"),
6   features: [documentCapture.Feature.FIELD_EXTRACTION],
7   documentType: documentCapture.DocumentType.RECEIPT,
8   language: documentCapture.Language.ENG
9 });

```

```

10 |
11 | ...
12 | // Add additional code

```

N/email Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/email module to send email messages from within NetSuite. You can use the N/email module to send regular, bulk, and campaign email.

Go To Samples 

In This Help Topic

- [N/email Module Members](#)

N/email Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	email.send(options)	void	Client and server scripts	Sends transactional email to an individual or group of recipients and receives bounceback notifications.  Note: To send email on another user's behalf, the user triggering the email send must have a role with the Vicarious Email (ADMI_VICARIOUS_EMAIL) permission. If a user without this permission executes SuiteScript that sends email, a Permission Violation error is returned.
	email.send.promise(options)	void	Client scripts	Sends transactional email asynchronously to an individual or group of recipients and receives bounceback notifications.
	email.sendBulk(options)	void	Client and server scripts	Sends bulk email (for use when a bounceback notification is not required).
	email.sendBulk.promise(options)	void	Client scripts	Sends bulk email asynchronously (for use when a bounceback notification is not required).
	email.sendCampaignEvent(options)	number	Client and server scripts	Sends a single "on-demand" campaign email to a specified recipient and return a campaign response ID.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	email.sendCampaignEvent.promise(options)	number	Client and server scripts	Sends a single “on-demand” campaign email asynchronously to a specified recipient and return a campaign response ID.

Note: Some email recipients may have another record type or types listed on their record in the **Other Recipients** field. Because records with other relationship types share the same internal ID across types, email sent with SuiteScript is saved on each record type for the recipient. For more information, see the help topic [Records as Multiple Types](#).

N/email Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/email module.

Send an Email with an Attachment

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample shows how to send an email with an attachment.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Note: Some of the values in this sample are placeholders, such as the `senderId` and `recipientEmail` values. Before using this sample, replace all hard-coded values, including IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5   // This script sends an email with an attachment.
6
7   require(['N/email', 'N/record', 'N/file'], (email, record, file) => {
8       const senderId = -515;
9       const recipientEmail = 'notify@myCompany.com';
10      let timeStamp = new Date().getUTCMilliseconds();
11
12      let recipient = record.create({
13          type: record.Type.CUSTOMER,
```

```

14     isDynamic: true
15   });
16   recipient.setValue({
17     fieldId: 'subsidiary',
18     value: '1'
19   });
20   recipient.setValue({
21     fieldId: 'companyname',
22     value: 'Test Company' + timeStamp
23   });
24   recipient.setValue({
25     fieldId: 'email',
26     value: recipientEmail
27   });
28
29   let recipientId = recipient.save();
30
31   let fileObj = file.load({
32     id: 88
33   });
34
35   email.send({
36     author: senderId,
37     recipients: recipientId,
38     subject: 'Test Sample Email Module',
39     body: 'email body',
40     attachments: [fileObj],
41     relatedRecords: {
42       entityId: recipientId,
43       customRecord: {
44         id: recordId,
45         recordType: recordTypeId
46       }
47     }
48   });
49 });

```

email.send(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends email to an individual or group of recipients and receives bounceback notifications. A maximum of 10 recipients (recipient + cc + bcc) is allowed. The total message size (including attachments) must be 15MB or less. The size of each attachment cannot exceed 10MB.  Note: To send email on another user's behalf, the user triggering the email send must have a role with the Vicarious Email (ADMI_VICARIOUS_EMAIL) permission. If a user without this permission executes SuiteScript that sends email, a Permission Violation error is returned.
Returns	void
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	20 units
Module	N/email Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.author	number	required	Internal ID of the email sender. To find the internal ID of the send in the UI, go to Lists > Employees. If the ADMI_VICARIOUS_EMAILS permission is set, the sender ID can be set to a different sender to allow users to send email on another user's behalf. The role executing the script is required to have the Vicarious Email permission for the script to complete. For more information about permissions, see the help topic NetSuite Permissions Overview.	2015.2
options.body	string	required	Contents of the outgoing message. SuiteScript formats the body of the email in either plain text or HTML. If HTML tags are present, the message is formatted as HTML. Otherwise, the message is formatted in plain text. To display XML as plain text, use an HTML <pre> tag around the XML.	2015.2
options.recipients	number string number[] string[]	required	The internal ID or email address of the recipient(s). For multiple recipients, use an array of internal IDs or email addresses. You can use an array that contains a combination of internal IDs and email addresses. A maximum of 10 recipients (options.recipients + options.cc + options.bcc) is allowed.  Note: Only the first recipient displays on the Communication subtab (under the Recipient column). To view all recipients, click View to open the Message record.	2015.2
options.subject	string	required	Subject of the outgoing message.	2015.2
options.attachments	file.File[]	optional	Email file attachments. You can send multiple attachments of any media type. An individual attachment must not exceed 10MB and the total message size must be 15MB or less.  Note: Supported for server scripts only.	2015.2
options.bcc	number[] string[]	optional	The internal ID or email address of the recipient(s) to blind copy.	2015.2

Parameter	Type	Required / Optional	Description	Since
			For multiple recipients, use an array of internal IDs or email addresses. You can use an array that contains a combination of internal IDs and email addresses. A maximum of 10 recipients (options.recipients + options.cc + options.bcc) is allowed.	
options.cc	number[] string[]	optional	The internal ID or email address of the secondary recipient(s) to copy. For multiple recipients, use an array of internal IDs or email addresses. You can use an array that contains a combination of internal IDs and email addresses. A maximum of 10 recipients (options.recipients + options.cc + options.bcc) is allowed.	2015.2
options.isInternalOnly	boolean	optional	If true, the Message record is not visible to an external Entity (for example, a customer or contact). The default value is false.	2015.2
options.relatedRecords	Object	optional	Object that contains key-value pairs to associate (attach) the Message record with related records (that is, transaction, activity, entity, and custom records). See the relatedRecords table for more information.	2015.2
options.replyTo	string	optional	The email address that appears in the reply-to header. You can use either a single external email address or a generic email address created by the Email Capture Plug-in.	2015.2

relatedRecords

 **Note:** The relatedRecords parameter is a JavaScript object.

Represents the NetSuite records to which an email Message record should be attached. You can associate the sent email with the following records listed below. You cannot use custom record, transaction, and activity together as an email related record; these three records are mutually exclusive.

Only entityId is compatible with other records

Parameter	Type	Required / Optional	Description	Since
activityId	number	optional	The Activity record to attach the Message record to. Use for Case and Campaign record types.	2015.2
customRecord	Object	optional	The custom record to attach the Message record to.	2015.2

Parameter	Type	Required / Optional	Description	Since
			For custom records you must specify both the record ID and the record type ID. The custom record is linked by using a nested JavaScript object.	
customRecord.id	number	optional	The instance ID for the custom record to attach the Message record to. Note: If you use this parameter, customRecord.recordType is required.	2015.2
customRecord.recordType	string	optional	The integer ID for the custom record type to attach the Message record to. This ID is shown as part of the record's URL. For example: /custrecordentry.nl?rectype=2&id=56. Note: If you use this parameter, customRecord.id is required.	2015.2
entityId	number	optional	The Entity record to attach the Message record to. Use for all Entity record types (for example, customer, contact).	2015.2
transactionId	number	optional	The Transaction record to attach the Message record to. Use for transaction and opportunity record types.	2015.2

Errors

Error Code	Message	Thrown If
SSS_AUTHOR_MUST_BE_EMPLOYEE	The author internal id or email must match an employee.	The author internal ID or email address doesn't match an employee.
SSS_INVALID_TO_EMAIL	One or more recipient emails are not valid.	A recipient's email address is invalid.
SSS_INVALID_CC_EMAIL	One or more cc emails are not valid.	An email address specified in the options.cc parameter is invalid.
SSS_INVALID_BCC_EMAIL	One or more bcc emails are not valid.	An email address specified in the options.bcc parameter is invalid.
SSS_MAXIMUM_NUMBER_RECIPIENTS_EXCEEDED	You may have a maximum number of 10 recipients.	A total number of recipients (options.recipients + options.cc + options.bcc) exceeds 10.
SSS_MISSING_REQD_ARGUMENT	{method name}: Missing a required argument: {param name}	A required parameter is missing.

Error Code	Message	Thrown If
WRONG_PARAMETER_TYPE	Wrong parameter type: {param name} is expected as {param type}.	A parameter's type is incorrect.
SSS_FILE_CONTENT_SIZE_EXCEEDED	The file content you are attempting to access exceeds the maximum allowed size of 10MB.	An attachment exceeds the 10 MB file size limit.
ATTACH_SIZE_EXCEEDED	This message exceeds the limit of 15 MB. Please reduce the size of the message and its attachments and try again. Note: Files can be larger when attached due to encoding.	The size of the attachments exceeds the limit.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/email Module Script Sample](#).

```

1 //Add additional code
2 .
3 var senderId = -5;
4 var recipientEmail = 'notify@myCompany.com';
5 var timeStamp = new Date().getUTCMilliseconds();
6 var recipientId = 12;
7 var fileObj = file.load({
8     id: 88
9 });
10 email.send({
11     author: senderId,
12     recipients: recipientId,
13     subject: 'Test Sample Email Module',
14     body: 'email body',
15     attachments: [fileObj],
16     relatedRecords: {
17         entityId: recipientId,
18         customRecord: {
19             id: recordId,
20             recordType: recordTypeId //an integer value
21         }
22     }
23 });
24 ...
25 //Add additional code

```

email.send.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends email asynchronously to an individual or group of recipients and receives bounceback notifications.
---------------------------	---

	 Note: The parameters and errors thrown for this method are the same as those for email.send(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	email.send(options)
Supported Script Types	Client scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/email Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 var senderId = -5;
4 var recipientEmail = 'notify@myCompany.com';
5 var timeStamp = new Date().getUTCMilliseconds();
6 var recipientId = 12;
7 var fileObj = file.load({
8     id: 88
9 });
10 email.send.promise({
11     author: senderId,
12     recipients: recipientId,
13     subject: 'Test Sample Email Module',
14     body: 'email body',
15     attachments: [fileObj],
16     relatedRecords: {
17         entityId: recipientId,
18         customRecord: {
19             id: recordId,
20             recordType: recordTypeId //an integer value
21         }
22     }
23 }).then (function(result) {
24     // Process the result
25 });
26 ...
27 //Add additional code

```

email.sendBulk(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends bulk email (for use when a bounceback notification is not required). The total message size (including attachments) must be 15MB or less. The size of each attachment cannot exceed 10MB.
---------------------------	---

	 Note: This API normally uses a bulk email server to send messages. If you need to increase the successful delivery rate of an email, use email.send(options) so that a transactional email server is used.
Returns	void
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/email Module
Since	2015.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description	Since
options.author	number	required	Internal ID of the email sender. To find the internal ID of the sender in the UI, go to Lists > Employees. If the ADML_VICARIOUS_EMAILS permission is set, the sender ID can be set to a different sender to allow users to send email on another user's behalf. For more information about permissions, see the help topic NetSuite Permissions Overview .	2015.2
options.body	string	required	Contents of the outgoing message. SuiteScript formats the body of the email in either plain text or HTML. If HTML tags are present, the message is formatted as HTML. Otherwise, the message is formatted in plain text. To display XML as plain text, use an HTML <pre> tag around the XML.	2015.2
options.recipients	number string number[] string[]	required	The internal ID or email address of the recipient. For multiple recipients, use an array of internal IDs or email addresses. You can use an array that contains a combination of internal IDs and email addresses.  Note: Only the first recipient displays on the Communication subtab (under the Recipient column). To view all recipients, click View to open the Message record.	2015.2
options.subject	string	required	Subject of the outgoing message.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.attachments	file.File []	optional	The email file attachments. You can send multiple attachments of any media type. An individual attachment must not exceed 10MB and the total message size must be 15MB or less.  Note: Supported for server scripts only.	2015.2
options.bcc	number[] string[]	optional	The internal ID or email address of the recipient to blind copy. For multiple recipients, use an array of internal IDs or email addresses. You can use an array that contains a combination of internal IDs and email addresses.	2015.2
options.cc	number[] string[]	optional	The internal ID or email address of the secondary recipient to copy. For multiple recipients, use an array of internal IDs or email addresses. You can use an array that contains a combination of internal IDs and email addresses.	2015.2
options.isInternalOnly	boolean	optional	If true, the Message record is not visible to an external Entity (for example, a customer or contact). The default value is false.	2015.2
options.relatedRecords	Object	optional	Object that contains key-value pairs to associate (attach) the Message record with related records (that is, transaction, activity, entity, and custom records). See the relatedRecords table for more information.	2015.2
options.replyTo	string	optional	The email address that appears in the reply-to header. You can use either a single external email address or a generic email address created by the Email Capture Plug-in.	2015.2

relatedRecords

 **Note:** The relatedRecords parameter is a JavaScript object.

Represents the NetSuite records to which an email Message record should be attached. You can associate the sent email with an array of internal records using key-value pairs.

There can be multiple related records, but only one of each parameter (each parameter represents applicable record types).

Parameter	Type	Required / Optional	Description	Since
activityId	number	optional	The Activity record to attach the Message record to. Use for Case and Campaign record types.	2015.2
entityId	number	optional	The Entity record to attach the Message record to. Use for all Entity record types (for example, customer, contact).	2015.2
customRecord	Object	optional	The custom record to attach the Message record to. For custom records you must specify both the record ID and the record type ID. The custom record is linked by using a nested JavaScript object.	2015.2
customRecord.id	number	optional	The instance ID for the custom record to attach the Message record to.  Note: If you use this parameter, <code>customRecord.recordType</code> is required.	2015.2
customRecord.recordType	string	optional	The integer ID for the custom record type to attach the Message record to. This ID is shown as part of the record's URL. For example: <code>/custrecordentry.nl?rectype=2&id=56</code> .  Note: If you use this parameter, <code>customRecord.id</code> is required.	2015.2
transactionId	number	optional	The Transaction record to attach the Message record to. Use for transaction and opportunity record types.	2015.2

Errors

Error Code	Message	Thrown If
SSS_AUTHOR_MUST_BE_EMPLOYEE	The author internal id or email must match an employee.	The author internal ID or email address doesn't match an employee.
SSS_INVALID_TO_EMAIL	One or more recipient emails are not valid.	A recipient's email address is invalid.
SSS_INVALID_CC_EMAIL	One or more cc emails are not valid.	An email address specified in the <code>options.cc</code> parameter is invalid.
SSS_INVALID_BCC_EMAIL	One or more bcc emails are not valid.	An email address specified in the <code>options.bcc</code> parameter is invalid.

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	{method name}: Missing a required argument: {param name}	A required parameter is missing.
WRONG_PARAMETER_TYPE	Wrong parameter type: {param name} is expected as {param type}.	A parameter's type is incorrect.
SSS_FILE_CONTENT_SIZE_EXCEEDED	The file content you are attempting to access exceeds the maximum allowed size of 10.0 MB.	An attachment exceeds the 10 MB file size limit.
ATTACH_SIZE_EXCEEDED	This message exceeds the limit of 15 MB. Please reduce the size of the message and its attachments and try again. Note: Files can be larger when attached due to encoding.	The size of the attachments exceeds the limit.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/email Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var recipientEmails = [
4     'msample@netsuite.com',
5     'jdoe@netsuite.com',
6     'awolfe@netsuite.com',
7     'htest@netsuite.com'
8 ];
9 email.sendBulk({
10     author: -5,
11     recipients: recipientEmails,
12     subject: 'Order Status',
13     body: 'Your order has been completed.',
14     replyTo: 'accounts@netsuite.com'
15 });
16 ...
17 //Add additional code

```

email.sendBulk.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends bulk email asynchronously (for use when a bounceback notification is not required).
	Note: The parameters and errors thrown for this method are the same as those for email.sendBulk(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	email.sendBulk(options)

Supported Script Types	Client scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/email Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 var recipientEmails = [
4     'msample@netsuite.com',
5     'jdoe@netsuite.com',
6     'awolfe@netsuite.com',
7     'htest@netsuite.com'
8 ];
9 email.sendBulk.promise({
10     author: -5,
11     recipients: recipientEmails,
12     subject: 'Order Status',
13     body: 'Your order has been completed.',
14     replyTo: 'accounts@netsuite.com'
15 }).then(function(result) {
16     // Process the result
17 });
18 ...
19 //Add additional code

```

email.sendCampaignEvent(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends a single “on-demand” campaign email to a specified recipient and return a campaign response ID to track the email. This is used for lead nurturing campaigns (drip marketing email). Email (campaignemail) sublists are not supported. The campaign must use a Lead Nurturing (campaigndrip) sublist. Note: This API normally uses a bulk email server to send messages. If you need to increase the successful delivery rate of an email, use email.send(options) so that a transactional email server is used.
Returns	A campaign response ID (tracking code) as number If the email fails to send, the value returned is -1.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units

Module	N/email Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.campaignEventId	number	required	The internal ID of the campaign event.  Note: This campaign must use a Lead Nurturing (campaignndrip) sublist. For more information about Lead Nurturing campaigns, see the help topic Lead Nurturing Campaigns .	2015.2
options.recipientId	number	required	The internal ID of the recipient.  Note: The recipient's record must contain an email address.	2015.2

Errors

Error Code	Message	Thrown If
SSS_AUTHOR_MUST_BE_EMPLOYEE	The author internal id or email must match an employee.	The author internal ID or email address doesn't match an employee.
SSS_INVALID_TO_EMAIL	One or more recipient emails are not valid.	A recipient's email address is invalid.
SSS_INVALID_CC_EMAIL	One or more cc emails are not valid.	An email address specified in the options.cc parameter is invalid.
SSS_INVALID_BCC_EMAIL	One or more bcc emails are not valid.	An email address specified in the options.bcc parameter is invalid
SSS_MISSING_REQD_ARGUMENT	{method name}: Missing a required argument: {param name}	A required parameter is missing.
WRONG_PARAMETER_TYPE	Wrong parameter type: {param name} is expected as {param type}.	A parameter's type is incorrect.
SSS_FILE_CONTENT_SIZE_EXCEEDED	The file content you are attempting to access exceeds the maximum allowed size of 10.0 MB.	An attachment exceeds the 10 MB file size limit.
ATTACH_SIZE_EXCEEDED	This message exceeds the limit of 15 MB. Please reduce the size of the message and its	The size of the attachments exceeds the limit.

Error Code	Message	Thrown If
	attachments and try again. Note: Files can be larger when attached due to encoding.	

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/email Module Script Sample](#).



Note: The following script includes sample values for some fields. You must replace these values with values from your NetSuite account for the code to process successfully.

```

1 // Add additional code
2 ...
3 // Create a campaign record in dynamic mode so all field values can be dynamically sourced.
4 var campaign1 = record.create({
5     type: record.Type.CAMPAIGN,
6     isDynamic: true
7 });
8 campaign1.setValue({
9     fieldId: 'title',
10    value: 'Sample Lead Nurturing Campaign'
11 });
12 // set values on the Lead Nurturing (campaigndrip) sublist
13 campaign1.selectNewLine({
14     sublistId: 'campaigndrip'
15 });
16 // 4 is a sample ID representing an existing marketing campaign
17 campaign1.setCurrentSublistValue({
18     sublistId: 'campaigndrip',
19     fieldId: 'template',
20     value: 4
21 });
22 campaign1.setCurrentSublistValue({
23     sublistId: 'campaigndrip',
24     fieldId: 'title',
25     value: 'Sample Lead Nurturing Event'
26 });
27 // 1 is a sample ID representing an existing subscription
28 campaign1.setCurrentSublistValue({
29     sublistId: 'campaigndrip',
30     fieldId: 'subscription',
31     value: 1
32 });
33
34 // 2 is a sample ID representing an existing channel
35 campaign1.setCurrentSublistValue({
36     sublistId: 'campaigndrip',
37     fieldId: 'channel',
38     value: 2
39 });
40
41 // 1 is a sample ID representing an existing promocode
42 campaign1.setCurrentSublistValue({
43     sublistId: 'campaigndrip',
44     fieldId: 'promocode',
45     value: 1
46 });
47 campaign1.commitLine({
48     sublistId: 'campaigndrip'
49 });
50
51 // Submit the record var
52 campaign1Key = campaign1.save();
53
54 // Load the campaign record you created. Determine the internal ID of the campaign event to the variable campaign2_campaigndrip_internalid_1.

```

```

55 var campaign2 = record.load({
56     type: record.Type.CAMPAIGN,
57     id : campaign1Key,
58     isDynamic: true
59 });
60 var campaign2_campaigndrip_internalid_1 = campaign2.getSublistValue({
61     sublistId: 'campaigndrip',
62     fieldId: 'internalid',
63     line: 1
64 });
65 // 142 is a sample ID representing the ID of a recipient with a valid email address
66 var campaignResponseId = email.sendCampaignEvent({
67     campaignEventId: campaign2_campaigndrip_internalid_1,
68     recipientId: 142
69 });
70 ...
71 //Add additional code

```

email.sendCampaignEvent.promise(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends a single “on-demand” campaign email asynchronously to a specified recipient and return a campaign response ID to track the email. This is used for lead nurturing campaigns (drip marketing email). If the email fails to send, the value returned is -1.  Note: The parameters and errors thrown for this method are the same as those for email.sendCampaignEvent(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	email.sendCampaignEvent(options)
Supported Script Types	Client scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/email Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 // Create a campaign record in dynamic mode so all field values can be dynamically sourced.
4 var campaign1 = record.create({
5     type: record.Type.CAMPAIGN,
6     isDynamic: true
7 });
8 campaign1.setValue({
9     fieldId: 'title',
10    value: 'Sample Lead Nurturing Campaign'
11 });
12 // set values on the Lead Nurturing (campaigndrip) sublist

```

```

13 campaign1.selectNewLine({
14     sublistId: 'campaigndrip'
15 });
16 // 4 is a sample ID representing an existing marketing campaign
17 campaign1.setCurrentSublistValue({
18     sublistId: 'campaigndrip',
19     fieldId: 'template',
20     value: 4
21 });
22 campaign1.setCurrentSublistValue({
23     sublistId: 'campaigndrip',
24     fieldId: 'title',
25     value: 'Sample Lead Nurturing Event'
26 });
27 // 1 is a sample ID representing an existing subscription
28 campaign1.setCurrentSublistValue({
29     sublistId: 'campaigndrip',
30     fieldId: 'subscription',
31     value: 1
32 });
33
34 // 2 is a sample ID representing an existing channel
35 campaign1.setCurrentSublistValue({
36     sublistId: 'campaigndrip',
37     fieldId: 'channel',
38     value: 2
39 });
40
41 // 1 is a sample ID representing an existing promocode
42 campaign1.setCurrentSublistValue({
43     sublistId: 'campaigndrip',
44     fieldId: 'promocode',
45     value: 1
46 });
47 campaign1.commitLine({
48     sublistId: 'campaigndrip'
49 });
50
51 // Submit the record var
52 campaign1Key = campaign1.save();
53
54 // Load the campaign record you created. Determine the internal ID of the campaign event to the variable campaign2_campaigndrip_internalid_1.
55 var campaign2 = record.load({
56     type: record.Type.CAMPAIGN,
57     id : campaign1Key,
58     isDynamic: true
59 });
60 var campaign2_campaigndrip_internalid_1 = campaign2.getSublistValue({
61     sublistId: 'campaigndrip',
62     fieldId: 'internalid',
63     line: 1
64 });
65 // 142 is a sample ID representing the ID of a recipient with a valid email address
66 var campaignResponseId = email.sendCampaignEvent.promise({
67     campaignEventId: campaign2_campaigndrip_internalid_1,
68     recipientId: 142
69 }).then(function(result) {
70     //Process the result
71 });
72 ...
73 //Add additional code

```

N/encode Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/encode module to convert a string to another type of encoding. This module exposes string encoding and decoding functionality.

[Go To Samples {..}](#)

In This Help Topic

- [N/encode Module Members](#)

N/encode Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	encode.convert (options)	string	Server scripts	Converts a string to another type of encoding and returns the re-encoded string.
Enum	encode.Encoding	enum	Server scripts	Holds the string values for the supported character set encoding. Use this enum to set the <code>inputEncoding</code> and <code>outputEncoding</code> parameter values in N/crypto Module or N/encode Module .

N/encode Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/encode module.

Convert a String to a Different Encoding

The following script shows how to convert a string to a different encoding.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/encode'], function(encode) {
6      function convertStringToDifferentEncoding() {
7          var stringInput = "TÄfÄst StriÄfÄg Input";
8          var base64EncodedString = encode.convert({
9              string: stringInput,
10             inputEncoding: encode.Encoding.UTF_8,
11             outputEncoding: encode.Encoding.BASE_64
12         });
13         var hexEncodedString = encode.convert({
14             string: stringInput,
```

```

15         inputEncoding: encode.Encoding.UTF_8,
16         outputEncoding: encode.Encoding.HEX
17     });
18 }
19
20 convertStringToDifferentEncoding();
21 });

```

encode.convert(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Converts a string to another type of encoding.
Returns	The re-encoded string.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/encode Module
Since	2015.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.string	string	required	The string to encode.	2015.1
options.inputEncoding	string	required	The encoding used for the input string. Use encode.Encoding to set the value.	2015.1
options.outputEncoding	string	required	The encoding to apply to the output string. If set to UTF-8 and the input string does not contain a valid UTF-8 encoded string, the operation will continue but the resulting string will contain the replacement character U+FFFD. Use encode.Encoding to set the value.	2015.1

Errors

Error Code	Thrown If
FAILED_TO_DECODE_STRING_ENCODED_BINARY_DATA_USING_1_ENCODING	The value of the options.string parameter does not represent binary data encoded using the encoding specified by the options.inputEncoding parameter.

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	Any of the options.string, options.inputEncoding, or options.outputEncoding parameters are not provided.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/encode Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hexEncodedString = encode.convert({
4   string: stringInput,
5   inputEncoding: encode.Encoding.UTF_8,
6   outputEncoding: encode.Encoding.HEX
7 });
8 ...
9 //Add additional code

```

encode.Encoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the supported character set encoding. Use this enum to set the inputEncoding and outputEncoding parameter values in N/crypto Module or N/encode Module .
Sibling Module Members	N/encode Module Members .
Module	N/encode Module
Since	2015.1

Values

Value
UTF_8
BASE_16
BASE_32
BASE_64
BASE_64_URL_SAFE

Value
HEX



Note: BASE_64_URL_SAFE uses a different character set compared to BASE64, where the following properties apply:

- + is replaced with -
- / is replaced with _
- Padding is still present and represented by character =
- Use a standard javascript String replace operation to remove padding or replace it with URL-encoded form %3D.

See: [Base-N Encodings](#)

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/encode Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var reencoded = encode.convert({
4     string: LOREM_IPS,
5     inputEncoding: encode.Encoding.BASE_64,
6     outputEncoding: encode.Encoding.UTF_8
7 });
8 ...
9 //Add additional code

```

N/error Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/error module to create custom SuiteScript errors that you can use in try-catch statements to abort script execution. Note that this module doesn't provide functionality to throw custom errors; however, you can include logic such as try-catch statements in your script to throw custom SuiteScript errors.

[Go To Samples](#)

In This Help Topic

- [N/error Module Members](#)

■ SuiteScriptError Object Members

For more information about additional error logging capabilities, see the [N/log Module](#).

N/error Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	error.SuiteScriptError	Object	Server scripts	Encapsulates a custom SuiteScript error for any server script type.
Method	error.create(options)	error.SuiteScriptError	Server scripts	Creates a new error.SuiteScriptError object.
Enum	error.Type	enum	Server scripts	Holds the string values for error types. Use this enum to set the value for the SuiteScriptError.name parameter of the error.create(options) method. This sets the value of the SuiteScriptError.type property.

SuiteScriptError Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SuiteScriptError.cause	string (read-only)	Server scripts	Cause of the error.
	SuiteScriptError.id	string (read-only)	Server scripts	Error ID that is automatically generated when a new error is created.
	SuiteScriptError.message	string (read-only)	Server scripts	Error message text displayed in the Details column of the Execution Log. Set from the <code>options.message</code> parameter when you create a new error using error.create(options)
	SuiteScriptError.name	string (read-only)	Server scripts	Error name or error code. Set from the <code>options.name</code> parameter when you create a new error using error.create(options) .
	SuiteScriptError.notifyOff	boolean (read-only)	Server scripts	Suppresses email notification when set to true. Set from the <code>options.notifyOff</code> parameter when you create a new error using error.create(options)
	SuiteScriptError.stack	Array of strings (read-only)	Server scripts	List of method calls that the script is executing when the error is thrown.
	SuiteScriptError.type	error.Type (read-only)	Server scripts	Error type (<code>error.SuiteScriptError</code>).

N/error Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples show how to use the features of the N/error module:

- [Create a Custom Error](#)
- [Create an Error Based on a Condition](#)

Create a Custom Error

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create a custom error.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This script creates a custom error.
6   require(['N/error'], function(error) {
7     function createError() {
8       var myCustomError = error.create({
9         name: 'MY_ERROR_CODE',
10        message: 'My custom error details',
11        notifyOff: true
12      });
13    }
14    createError();
15  });

```

Create an Error Based on a Condition

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to conditionally create and throw a custom error.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This script conditionally creates and throws an error.
6   require(['N/error'], function(error) {
7     function showError() {
8       var someVariable = false;
9
10      if (!someVariable) {
11        var myCustomError = error.create({
12          name: 'WRONG_PARAMETER_TYPE',
13          message: 'Wrong parameter type selected.',

```

```

14         notifyOff: false
15     });
16
17     // This will write 'Error: WRONG_PARAMETER_TYPE Wrong parameter type selected' to the log
18     log.error('Error: ' + myCustomError.name, myCustomError.message);
19     throw myCustomError;
20 }
21 }
22 showError();
23 });

```

error.SuiteScriptError

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a custom SuiteScript error. Use this object in a try-catch statement to abort script execution. Create a new custom error with the error.create(options) method, which returns an <code>error.SuiteScriptError</code> .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/error Module
Methods and Properties	SuiteScriptError Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // Include this code in a server script that is not a user event script to create an error.SuiteScriptError object
4 var SScustom_error = error.create({
5     name: 'MY_ERROR_CODE',
6     message: 'my custom SuiteScriptError details',
7     notifyOff: false
8 });
9 ...
10 //Add additional code

```

SuiteScriptError.cause

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The cause of the error.
Type	string (read-only)
Module	N/error Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     cause: 'the cause of the error',
7     notifyOff: false
8 });
9 log.debug("Error cause: " + SScustom_error.cause); // 'Error cause: ' followed by the cause will be logged
10 ...
11 //Add additional code

```

SuiteScriptError.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Error ID that is automatically generated when a new error is created.
Type	string (read-only)
Module	N/error Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     notifyOff: false
7 });
8 log.debug("Error ID: " + SScustom_error.id); // id is system generated
9                                           // 'Error ID: ' followed by the id will be logged
10 ...
11 //Add additional code

```

SuiteScriptError.message

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Error message text displayed in the Details column of the Execution Log. Set from the options.message parameter when you create a new error using error.create(options) .
Type	string (read-only)
Module	N/error Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     notifyOff: false
7 });
8 log.debug("Error Message: " + SScustom_error.message); // 'Error Message: my custom SuiteScriptError details' will be logged
9 ...
10 //Add additional code

```

SuiteScriptError.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Error name or error code. Set from the options.name parameter when you create a new error using error.create(options) .
Type	string (read-only)
Module	N/error Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     notifyOff: false
7 });
8 log.debug("Error Code: " + SScustom_error.name); // 'Error Code: MY_ERROR_CODE' will be logged
9 ...
10 //Add additional code

```

SuiteScriptError.notifyOff

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Suppresses email notification when set to true. Set from the options.notifyOff parameter when you create a new error using error.create(options) .
Type	string (read-only)
Module	N/error Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     notifyOff: false
7 });
8 log.debug("Error NotifyOff: " + SScustom_error.notifyOff); // 'Error NotifyOff: false' will be logged
9 ...
10 //Add additional code

```

SuiteScriptError.stack

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	List of method calls that the script is executing when the error is thrown. The most recently executed method is listed at the top of the list.
Type	Array of strings (read-only)
Module	N/error Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     notifyOff: false
7 });
8 log.debug("Error Stack: " + SScustom_error.stack); // stack is set by the system
9 // 'Error Stack: ' followed by the stack will be logged
10 ...
11 //Add additional code

```

SuiteScriptError.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Error type (error.SuiteScriptError).
Type	string (read-only)

Module	N/error Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var SScustom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom SuiteScriptError details',
6     notifyOff: false
7 });
8 log.debug("Error Type: " + SScustom_error.type); // 'Error Type: ' followed by the type will be logged
9 ...
10 //Add additional code

```

error.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new error.SuiteScriptError object, which can be used in a try-catch statement to abort script execution. Note this method creates a new error, but does not throw the error. Your script code will need to include logic to throw the error when appropriate.
Returns	An error.SuiteScriptError object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/error Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.message	string	required	Error message text displayed in the Details column of the Execution Log. Sets the value for the SuiteScriptError.message property. The default value is null.	2015.2
options.name	string	required	User-defined error code. You can create a custom name such as "My_Custom_Error" or use the	2015.2

Parameter	Type	Required / Optional	Description	Since
			values in the error.Type enum. Sets the value for the SuiteScriptError.name property.	
options.notifyOff	boolean	optional	Sets whether email notification is suppressed. Sets the value for the SuiteScriptError.notifyOff property. If set to false, the system emails the users identified on the applicable script record's Unhandled Errors subtab when the error is thrown. The default values is false. For additional information about the Unhandled Errors subtab, see the help topic Creating a Script Record.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.message or options.name parameter is not specified.
WRONG_PARAMETER_TYPE	One of the parameters is not specified as the correct type: - options.message must be a string value - options.name must be a string value - options.notifyOff must be a boolean value

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var custom_error = error.create({
4     name: 'MY_ERROR_CODE',
5     message: 'my custom error details',
6     notifyOff: false
7 });
8 ...
9 try {
10     // add additional try code
11 }
12 catch (e) {
13     // add catch code to include the a log or throw statement, such as:
14     throw custom_error;
15     log.debug("Error Code: " + custom_error.name);
16 }
17 ...
18
19 // You could also create the custom error within a throw statement:
20 // throw error.create({
21 //     name: 'MY_ERROR_CODE',
22 //     message: 'my custom error details',
23 //     notifyOff: false
24 // });

```


error.Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for error types. You can use these error types and write your own corresponding error messages. This may be useful if you are developing a SuiteScript library. Use this enum to set the value for the SuiteScriptError.name parameter of the error.create(options) method. This sets the value of the SuiteScriptError.type property.
Module	N/error Module
Sibling Module Members	N/error Module Members
Since	2015.2

Note: You must configure your account settings according to the error type used in your scripts to avoid potential errors.

Values

Value
A_SCRIPT_IS_ATTEMPTING_TO_EDIT_THE_1_SUBLIST_THIS_SUBLIST_IS_CURRENTLY_IN_READONLY_MODE_ AND_CANNOT_BE_EDITED_ CALL_YOUR_NETSUITE_ADMINISTRATOR_TO_DISABLE_THIS_SCRIPT_IF_YOU_NEED_TO_SUBMIT_THIS_RECORD
AT_LEAST_ONE_EXPRESSION_IS_NEEDED
BUTTONS_MUST_INCLUDE_BOTH_A_LABEL_AND_VALUE
CAN_REFERENCE_ONLY_PERSISTED_DATASET
CAN_SELECT_ONLY_ONE_DEFAULT_CHOICE
CANNOT_CREATE_RECORD_DRAFT_OF_EXISTING_RECORD
CANNOT_CREATE_RECORD_INSTANCE
CANNOT_DETERMINE_TYPE_FOR_ALIAS
CANNOT_DETERMINE_VALUE_FOR_ALIAS
CANNOT_RESUBMIT_SUBMITTED_ASYNC_PIVOT_TASK
CANNOT_RESUBMIT_SUBMITTED_ASYNC_QUERY_TASK
CANNOT_RESUBMIT_SUBMITTED_ASYNC_SEARCH_TASK
CANNOT_RESUBMIT_SUBMITTED_ASYNC_SUITEQL_TASK
COLOR_VALUE_MUST_BE_6_HEXADECIMAL_DIGITS_OF_THE_FORM_RRGGBB__EXAMPLE_FF0000_FOR_RED
CREDIT_CARD_NUMBER_IS_NOT_VALID_PLEASE_CHECK_THAT_ALL_DIGITS_WERE_ENTERED_CORRECTLY
CREDIT_CARD_NUMBER_MUST_CONTAIN_BETWEEN_13_AND_20_DIGITS

Value
CREDIT_CARD_NUMBER_MUST_CONTAIN_ONLY_DIGITS
DATASET_NAME_IS_MISSING
DEFAULT_CHOICE_IS_MISSING
EACH_VIEW_MUST_HAVE_AN_ID
EACH_VIEW_MUST_HAVE_A_NAME
EMPTY_KEY_NOT_ALLOWED
EMPTY_KEY_NOT_ALLOWED_FOR_1
EXPRESSION_CANNOT_BE_SPECIFIED_WHEN_USING_COUNT_DISTINCT_AGGREGATION
EXPRESSION_MUST_BE_SPECIFIED_WHEN_USING_OTHER_THAN_COUNT_DISTINCT_AGGREGATION
EXPRESSIONS_CANNOT_BE_SPECIFIED_WHEN_USING_OTHER_THAN_COUNT_DISTINCT_AGGREGATION
EXPRESSIONS_MUST_BE_SPECIFIED_WHEN_USING_COUNT_DISTINCT_AGGREGATION
FAILED_AN_UNEXPECTED_ERROR_OCCURRED
FIELD_1_ALREADY_CONTAINS_A_SUBRECORD_YOU_CANNOT_CALL_CREATESUBRECORD
FIELD_1_CANNOT_BE_EMPTY
FIELD_1_IS_NOT_A_SUBRECORD_FIELD
FIELD_MUST_CONTAIN_A_VALUE
FORM_VALIDATION_FAILED_YOU_CANNOT_SUBMIT_THIS_RECORD
FORM_VALIDATION_FAILED_YOU_CANNOT_CREATE_THIS_SUBRECORD
HISTORY_IS_ONLY_AVAILABLE_FOR_THE_LAST_30_DAYS
ID_CANNOT_HAVE_MORE_THAN_N_CHARACTERS
IDENTIFIERS_CAN_CONTAIN_ONLY_DIGITS_ALPHABETIC_CHARACTERS_OR__WITH_NO_SPACES
INVALID_AGGREGATE_TYPE
INVALID_AGGREGATION
INVALID_ALGORITHM
INVALID_ALPHA_VALUE
INVALID_ASPECT_TYPE
INVALID_CERTIFICATE_TYPE
INVALID_CHART_TYPE
INVALID_COLOR_VALUE
INVALID_COLUMN_ALIAS
INVALID_COLUMN_FOR_SORTING
INVALID_CONFIGURATION_UNABLE_TO_CHANGE_REQUIRE_CONFIGURATION_FOR_1
INVALID_CONFIGURATION_UNABLE_TO_CHANGE_REQUIRE_CONFIGURATION_WITHOUT_A_CONTEXT

Value
INVALID_CONFLICT_RESOLUTION_1
INVALID_CURRENCY
INVALID_CUSTOM_VIEW_VALUE
INVALID_DATASET_ID
INVALID_DATE_ID
INVALID_DATE_VALUE_MUST_BE_ON_OR_AFTER_1CUTOFF_DATE
INVALID_DATE_VALUE_MUST_BE_1
INVALID_DATE_OBJECT
INVALID_DIRECTION_FOR_SORTING
INVALID_EMAILS_FOUND
INVALID_EXPRESSION
INVALID_FIELD_CONTEXT
INVALID_FIELD_ID
INVALID_FIELD_INDEX
INVALID_FIELD_VALUE
INVALID_FILTER_FIELD_FOR_CURRENT_VIEW
INVALID_FLD_VALUE
INVALID_FONT_SIZE
INVALID_FONT_STYLE
INVALID_FONT_WEIGHT
INVALID_FORMULA_TYPE
INVALID_GETSELECTOPTION_FILTER_OPERATOR
INVALID_HTTP_METHOD
INVALID_ID_PREFIX
INVALID_IMAGE
INVALID_KEY_OR_REF
INVALID_KEY_TYPE
INVALID_LOCALE
INVALID_NUMBER_MUST_BE_BETWEEN_1_AND_2
INVALID_NUMBER_MUST_BE_GREATER_THAN_1
INVALID_NUMBER_MUST_BE_LOWER_THAN_1
INVALID_NUMBER_OR_PERCENTAGE
INVALID_OPERATION

Value
INVALID_OPERATOR
INVALID_OR_UNSUPPORTED_RECORD_TYPE_1
INVALID_OWNER_CATEGORY
INVALID_PAGE_INDEX
INVALID_PAGE_RANGE
INVALID_PERIOD_ADJUSTMENT
INVALID_PERIOD_CODE
INVALID_PERIOD_TYPE
INVALID_POSITION
INVALID_RCRD_TYPE
INVALID_RETURN_TYPE_EXPECTED_1
INVALID_SCRIPT_OPERATION_ON_READONLY_SUBLIST_FIELD
INVALID_SEARCH_OPERATOR
INVALID_SEARCH_TYPE
INVALID_SIGNATURE
INVALID_SIGNATURE_TAG
INVALID_SORT
INVALID_SORT_LOCALE
INVALID_STACKING_TYPE
INVALID_SUBLST_OPERATION
INVALID_SUBRECORD_MERGE
INVALID_SUITEAPP_APPLICATION_ID
INVALID_TASK_TYPE
INVALID_TEMPORAL_UNIT
INVALID_TEXT_ALIGN
INVALID_TEXT_DECORATION_LINE
INVALID_TEXT_DECORATION_STYLE
INVALID_TOTAL_LINE
INVALID_TYPE_1_USE_2
INVALID_UI_OBJECT_TYPE
INVALID_UNIT
INVALID_URL_SPACES_ARE_NOT_ALLOWED_IN_THE_URL
INVALID_URL_URL_MUST_START_WITH_HTTP_HTTPS_FTP_OR_FILE

Value
INVALID_WORKBOOK_ID
METHOD_IS_ONLY_ALLOWED_FOR_MATRIX_FIELD
MISSING_MANDATORY_FIELDS
MISSING_REQD_ARGUMENT
MUTUALLY_EXCLUSIVE_ARGUMENTS
NAME_CANNOT_BE_EMPTY
NAME_CANNOT_HAVE_MORE_THAN_N_CHARACTERS
NEITHER_ARGUMENT_DEFINED
NO_ASPECTS_DEFINED
NO_CHILDREN_DEFINED
NO_COLUMN_DEFINED
NO_DATASET_DEFINED
NO_DIMENSION_ITEM_DEFINED
NO_ELEMENTS_DEFINED
NO_MEASURES_DEFINED
NO_RULE_DEFINED
NO_SELECTORS_DEFINED
NO_SORT_BY_DEFINED
NON_KATAKANA_DATA_FOUND
NOT_SUPPORTED_ON_CURRENT_SUBRECORD
NOTICE_THE_CREDIT_CARD_APPEARS_TO_BE_INCORRECT
OPERATOR_ARITY_MISMATCH
OPERATION_IS_NOT_ALLOWED
PASSWORD_CANNOT_HAVE_MORE_THAN_N_CHARACTERS
PHONE_NUMBER_SHOULD_HAVE_SEVEN_DIGITS_OR_MORE
PIVOT_DOES_NOT_EXIST
PLEASE_ENTER_A_VALID_FROM_START_DATE_IN_MMYYYY_FORMAT
PLEASE_ENTER_AN_EXPIRATION_DATE_IN_MMYYYY_FORMAT
PLEASE_INCLUDE_THE_AREA_CODE_FOR_PHONE_NUMBER
PROPERTY_VALUE_CONFLICT
READ_ONLY_PROPERTY
RELATIONSHIP_ALREADY_USED
SCRIPT_EXECUTION_USAGE_LIMIT_EXCEEDED

Value
SELECT_OPTION_ALREADY_PRESENT
SELECT_OPTION_NOT_FOUND
SERVER_SIDE_VALIDATION_FAILED
SIGNATURE_VERIFICATION_FAILED
SSS_ARGUMENT_DISCREPANCY
SSS_DUPLICATE_ALIAS
SSS_INVALID_ACTION_ID
SSS_INVALID_API_USAGE
SSS_INVALID_COUNTRY_ID
SSS_INVALID_CURRENCY_ID
SSS_INVALID_FORMAT_TYPE
SSS_INVALID_GETSELECTOPTION_FILTER_OPERATOR
SSS_INVALID_MACRO_ID
SSS_INVALID_READ_SIZE
SSS_INVALID_SEARCH_RESULT_INDEX
SSS_INVALID_SEGMENT_SEPARATOR
SSS_INVALID_SRCH_OPERATOR
SSS_INVALID_SUBLIST
SSS_INVALID_SUBLIST_OPERATION
SSS_INVALID_TYPE_ARG
SSS_INVALID_UI_OBJECT_TYPE
SSS_INVALID_URL
SSS_METHOD_IS_ONLY_ALLOWED_FOR_MATRIX_FIELD
SSS_METHOD_IS_ONLY_ALLOWED_FOR_MULTISELECT_FIELD
SSS_METHOD_IS_ONLY_ALLOWED_FOR_SELECT_FIELD
SSS_MISSING_ALIAS
SSS_MISSING_REQD_ARGUMENT
SSS_NOT_YET_SUPPORTED
SSS_RECORD_DOES_NOT_SATISFY_CONDITION
SSS_RECORD_TYPE_MISMATCH
SSS_SEARCH_FOR_EACH_LIMIT_EXCEEDED
SSS_SEARCH_RESULT_LIMIT_EXCEEDED
SSS_SUBLIST_DOESNT_SUPPORT_MOVING_LINES

Value
SSS_TAG_CANNOT_BE_EMPTY
SSS_TAX_REGISTRATION_REQUIRED
SSS_UNSUPPORTED_METHOD
TABLE_DOES_NOT_EXIST
THAT_RECORD_IS_NOT_EDITABLE
THE_FIELD_1_CONTAINED_MORE_THAN_THE_MAXIMUM_NUMBER__2__OF_CHARACTERS_ALLOWED
THE_OPTIONS_ARE_MUTUALLY_EXCLUSIVE_1_2_ARG2_
TOO_MANY_RESULTS
TRANSLATION_HANDLE_IS_IN_AN_ILLEGAL_STATE
UNHANDLED_ERRORS_ON_RESTORE
UNKNOWN_CONTEXT_TYPE
UNKNOWN_PARAM
UNSUPPORTED_COLOR
VALUE_1_OUTSIDE_OF_VALID_MINMAX_RANGE_FOR_FIELD2
WORKBOOK_NAME_IS_MISSING
WRONG_PARAMETER_TYPE
WS_INVALID_REFERENCE_KEY_1
WS_NO_PERMISSIONS_TO_SET_VALUE
YOU_HAVE_ATTEMPTED_AN_UNSUPPORTED_ACTION

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/error Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myInvalidEmailsError = error.Type.INVALID_EMAILS_FOUND
4 ...
5 // Add additional code

```

N/file Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/file module to work with files within NetSuite. You can use this module to upload files to the NetSuite File Cabinet, as well as send files as attachments without uploading them to the File Cabinet. You can also use this module to copy files in the File Cabinet, and you can use conflict resolution options to handle conflicts (such as when a file with the same name already exists in the same folder in the File Cabinet).

✓ **Tip:** When dealing with file naming conventions, especially when uploading files to the File Cabinet using SuiteScript, it is important to follow best practices to prevent file duplication. If your system works with multiple files simultaneously, consider including timestamps with millisecond precision in the file names. Additionally, generating a random number as part of the file name can further reduce the chances of duplicates.

A [file.Reader](#) object, which is returned by [File.getReader\(\)](#), can be used for special read operations. Use [File.getSegments\(options\)](#) to retrieve an iterator of custom segments of a file.

Methods that load content in memory, such as [File.getContents\(\)](#), have a 10 MB size limit. This limit does not apply when content is streamed, such as when [File.save\(\)](#) is called.

The [File.appendLine\(options\)](#) method inserts a line to the end of a CSV or text file, which increases the file size accordingly. All files are screened for malicious content when they are uploaded or updated, and increased file size can slow down the process. Consider splitting large files into several smaller ones if you experience performance issues.

Go To Samples {...}

In This Help Topic

- [N/file Module Members](#)
- [File Object Members](#)
- [Reader Object Members](#)

N/file Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	file.File	Object	Server scripts	Encapsulates a file within NetSuite.
	file.Reader	Object	Server scripts	Encapsulates a reader that you can use to perform special read operations.
Method	file.copy(options)	file.File	Server scripts	Copies an existing file in the File Cabinet.
	file.create(options)	file.File	Server scripts	Creates a new file.File .
	file.delete(options)	void	Server scripts	Deletes an existing file.File from the NetSuite File Cabinet.
	file.load(options)	file.File	Server scripts	Loads an existing file.File from the NetSuite File Cabinet.
Enum	file.Encoding	enum	Server scripts	Holds character encoding values for file contents. Use this enum to set the value of the File.encoding property.
	file.NameConflict Resolution	enum	Server scripts	Holds conflict resolution values that apply when copying a file.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				Use this enum to specify how to resolve conflicts when copying files and to set the value of the conflict resolution parameter in file.copy(options)
	file.Type	enum	Server scripts	Holds file type values. Use this enum to set the value of the File.fileType property.

File Object Members

The following members are available for a [file.File](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	File.appendLine(options)	file.File	Server scripts	Inserts a line to the end of a CSV or text file.
	File.getContents()	string	Server scripts	Returns the content of a file in string format.
	File.lines.iterator()	boolean	Server scripts	Calls a developer-defined function for each line. Returns false when line processing stops.
	File.resetStream()	void	Server scripts	Resets the file stream to its previous state.
	File.save()	number	Server scripts	Saves a new or updated file to the File Cabinet.
	File.getReader()	Object	Server scripts	Returns reader object for read operations.
	File.getSegments(options)	Object	Server scripts	Returns an iterator of segments that are delimited by the specified separator.
Property	File.description	string	Server scripts	Description of a file.
	File.encoding	string	Server scripts	Character encoding on a file.
	File.fileType	enum	Server scripts	File type of a file.
	File.folder	number	Server scripts	Internal ID of the folder that houses a file within the NetSuite File Cabinet.
	File.id	number (read-only)	Server scripts	Internal ID of a file in the NetSuite File Cabinet.
	File.isInactive	boolean	Server scripts	Inactive status of a file. If set to true, the file is inactive.
	File.isOnline	boolean	Server scripts	“Available without Login” status of a file. If set to true, users can

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				download the file outside of a current NetSuite login session.
	File.isText	boolean (read-only)	Server scripts	Indicates whether a file type is text-based.
	File.name	string	Server scripts	Name of a file.
	File.path	string (read-only)	Server scripts	Relative path to a file in the NetSuite File Cabinet.
	File.size	number (read-only)	Server scripts	Size of a file in bytes.
	File.url	string (read-only)	Server scripts	URL of a file.

Reader Object Members

The following members are available for a [file.Reader](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Reader.readChars (options)	string	Server scripts	Returns the next <code>options.number</code> characters from the current position.
	Reader.readUntil (options)	string	Server scripts	Returns string from current position to the next occurrence of <code>options.tag</code> .

N/file Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/file module:

- [Create and Save a File to the File Cabinet](#)
- [Create a File, Set Property Values, and Save It to the File Cabinet](#)
- [Create and Save a CSV File then Reload the File and Parse Its Contents](#)
- [Read and Log File Contents Using Commas and New Lines as Separators](#)
- [Read and Log Segments of a File Using a Set of Characters as Separators](#)
- [Copy a File Using Conflict Resolution](#)

Create and Save a File to the File Cabinet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create and save a file to the File Cabinet. In this sample, the folder ID value is hard-coded. For the script to run in the SuiteScript Debugger, you must replace this hard-coded value with a valid folder ID from your account.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/file'], file => {
5       // Create a file containing text
6       let fileObj = file.create({
7           name: 'testHelloWorld.txt',
8           fileType: file.Type.PLAINTEXT,
9           contents: 'Hello World\nHello World'
10      });
11
12      // Set the folder for the file
13      // Note that this value is hard-coded in this sample, and you should use
14      // a valid folder ID from your account
15      fileObj.folder = -15;
16
17      // Save the file
18      let id = fileObj.save();
19
20      // Load the same file to ensure it was saved correctly
21      fileObj = file.load({
22          id: id
23      });
24  });

```

Create a File, Set Property Values, and Save It to the File Cabinet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create and save a file to the File Cabinet. It also shows how to set the values of the `File.isOnline` and the `File.folder` properties. In this sample, the folder ID value is hard-coded. For the script to run in the SuiteScript Debugger, you must replace this hard-coded value with a valid folder ID from your account.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/file'], file => {
5       // Create a file containing text
6       // Note that the folder value is hard-coded in this sample, and you should
7       // use a valid folder ID from your account
8       let fileObj = file.create({

```

```

9      name: 'testHelloWorld3.txt',
10     fileType: file.Type.PLAINTEXT,
11     contents: 'Hello World\nHello World',
12     folder: -15,
13     isOnline: true
14   });
15
16   // Save the file
17   let id = fileObj.save();
18
19   // Load the same file to ensure it was saved correctly
20   fileObj = file.load({
21     id: id
22   });
23 });

```

Create and Save a CSV File then Reload the File and Parse Its Contents

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a CSV file, appends several lines of data, and saves the file. The script also loads the file and calculates the total of several values in the file.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/file', 'N/error', 'N/log'], function (file, error, log) {
6
7      // This sample calculates the total for the
8      // second column value in a CSV file.
9      //
10     // Each line in the CSV file has the following format:
11     // date,amount
12     //
13     // Here is the data that the script adds to the file:
14     // 10/21/14,200.0
15     // 10/21/15,210.2
16     // 10/21/16,250.3
17
18     // Create the CSV file
19     var csvFile = file.create({
20       name: 'data.csv',
21       contents: 'date,amount\n',
22       folder: 39,
23       fileType: 'CSV'
24     });
25
26     // Add the data
27     csvFile.appendLine({
28       value: '10/21/14,200.0'
29     });
30     csvFile.appendLine({
31       value: '10/21/15,210.2'
32     });
33     csvFile.appendLine({
34       value: '10/21/16,250.3'
35     });
36
37     // Save the file
38     var csvFileId = csvFile.save();

```

```

38
39 // Create a variable to store the calculated total
40 var total = 0.0;
41
42 // Load the file
43 var invoiceFile = file.load({
44     id: csvFileId
45 });
46
47 // Obtain an iterator to process each line in the file
48 var iterator = invoiceFile.lines.iterator();
49
50 // Skip the first line, which is the CSV header line
51 iterator.each(function () {return false;});
52
53 // Process each line in the file
54 iterator.each(function (line) {
55     // Update the total based on the line value
56     var lineValues = line.value.split(',');
57     var lineAmount = parseFloat(lineValues[1]);
58     if (!lineAmount) {
59         throw error.create({
60             name: 'INVALID_INVOICE_FILE',
61             message: 'Invoice file contained non-numeric value for total: ' + lineValues[1]
62         });
63     }
64     total += lineAmount;
65     return true;
66 });
67
68 // At the completion of the iteration, each function, the total is 660.5
69
70 });

```

Read and Log File Contents Using Commas and New Lines as Separators

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample reads and logs strings from a file using commas and new line characters as separators. This sample can be used as the starting point for a parser implementation.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType bankStatementParserPlugin
4   */
5
6  define(['N/file', 'N/log'], function(file, log) {
7      return {
8          parseBankStatement: function(context) {
9              var reader = context.input.file.getReader();
10
11              var textUntilFirstComma = reader.readUntil(',');
12              var next10Characters = reader.readChars(10);
13              var textUntilNextNewLine = reader.readUntil('\n');
14              var next100Characters = reader.readChars(100);
15
16              log.debug({
17                  title: 'STATEMENT TEXT',
18                  details: textUntilFirstComma
19              });

```

```

20
21     log.debug({
22         title: 'STATEMENT TEXT',
23         details: next10Characters
24     });
25
26     log.debug({
27         title: 'STATEMENT TEXT',
28         details: textUntilNextNewLine
29     });
30
31     log.debug({
32         title: 'STATEMENT TEXT',
33         details: next100Characters
34     })
35 }
36 }
37 });

```

Read and Log Segments of a File Using a Set of Characters as Separators

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample reads and logs segments from a file using a set of characters as separators.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType bankStatementParserPlugin
4   */
5
6  define(['N/file', 'N/log'], function(file, log) {
7      return {
8          parseBankStatement: function(context) {
9              var statementFile = context.input.file;
10
11              var statementSegmentIterator = statementFile.getSegments({separator: '\\\\_|/'}).iterator();
12              statementSegmentIterator.each(function (segment) {
13                  log.debug({
14                      title: 'STATEMENT TEXT',
15                      details: segment.value
16                  });
17                  return true;
18              });
19          }
20      }
21  });

```

Copy a File Using Conflict Resolution

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to copy a file in the File Cabinet using the `RENAME_TO_UNIQUE` conflict resolution option. Using this option, if a file with the same name as the original file already exists in the target folder, the original file is copied and a number is appended to the file name to make the name unique. In this sample, the folder ID values are hard-coded. For the script to run in the SuiteScript Debugger, you must replace these hard-coded values with valid folder IDs from your account.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/file'], function(file) {
5       var fileObj = file.create({
6           name: 'test.txt',
7           fileType: file.Type.PLAINTEXT,
8           contents: 'Hello World\nHello World'
9       });
10      fileObj.folder = 2667;
11      var id = fileObj.save();
12
13      fileObj = file.copy({
14          id: id,
15          folder: 2670,
16          conflictResolution: file.NameConflictResolution.RENAME_TO_UNIQUE
17      });
18  });

```

file.File

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a file within NetSuite. Note: This object only encapsulates a file's metadata. Content is only loaded into memory (and returned as a string) when you call the File.getContents(). Content from CSV or text files can be accessed line by line using File.appendLine(options) or File.lines.iterator(). Important: Binary content must be base64 encoded. Create a new file.File Object (up to 10MB in size) with the file.create(options) method. After you create a new file.File, you can: - upload it to the NetSuite File Cabinet with the File.save() method. - attach it to an email or fax without saving it to the File Cabinet. Important: If you want to save the file to the NetSuite File Cabinet, you must set a NetSuite File Cabinet folder with the File.folder property. You must do this before you call File.save(). Returns reader object File.getReader() and iterator of segments File.getSegments(options).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/file Module
Methods and Properties	File Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4   name: 'test.txt',
5   fileType: file.Type.PLAINTEXT,
6   contents: 'Hello World\nHello World'
7 });
8 fileObj.folder = 30;
9 var fileId = fileObj.save();
10 ...
11
12 //Add additional code

```

File.appendLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to insert a line to the end of a file. This method can be used on text or .csv files. Important: Content held in memory is limited to 10MB. Therefore, each line must be less than 10MB.
Returns	file.File Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/file Module
Since	2017.1

Parameters



Note: The options parameter is a JavaScript object. The table below describes the name:value pairs that make up the object.

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	Object containing a string to insert at the end of the file.	2017.1

Errors

Error Code	Message	Thrown If
SSS_FILE_CONTENT_SIZE_EXCEEDED	The content you are attempting to access exceeds	You attempt to return the content of a line larger than 10MB.

Error Code	Message	Thrown If
	the maximum allowed size of 10 MB.	
YOU_CANNOT_WRITE_TO_A_FILE_AFTER_YOU_BEGAN_READING_FROM_IT	—	You call File.appendLine(options) after calling File.lines.iterator() . To avoid receiving the error, call File.resetStream() or save the file.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4     id: 145
5 });
6 fileObj.appendLine({
7     value: 'hello world'
8 });
9 ...
10 //Add additional code

```

File.getContents()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to return the content of the file. Important: Content held in memory is limited to 10MB. Note: You can access CSV or text files (including files over 10MB) using File.appendLine(options) or File.lines.iterator().
Returns	The file content as a string.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
SSS_FILE_CONTENT_SIZE_EXCEEDED	The file content you are attempting to access exceeds the maximum allowed size of 10 MB.	You attempt to return the content of a file larger than 10MB.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4   id: 145
5 });
6 if (fileObj.size < 10485760){
7   fileObj.getContents();
8 }
9 ...
10 //Add additional code

```

File.getReader()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to return the reader object for performing special read operations.
Returns	file.Reader
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/file Module
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var reader = context.input.file.getReader();
4
5 var textUntilFirstComma = reader.readUntil(',');
6 var next10Characters = reader.readChars(10);
7 var textUntilNextNewLine = reader.readUntil('\n');
8 var next100Characters = reader.readChars(100);
9
10 ...
11 // Add additional code

```

File.getSegments(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to return the iterator of segments delimited by a separator. Separator is included in each segment.
---------------------------	--

	Empty separator is not allowed.
Returns	Iterator
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/file Module
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.separator	string	required	The separator to use to divide the segments. For example, if you specify a newline character as the separator, this method returns an iterator where each segment is a single line in the file.	2019.1

Errors

Error Code	Message	Thrown If
SSS_INVALID_SEGMENT_SEPARATOR	Segment separator must not be empty.	The options.separator argument is empty.
SSS_INVALID_ARG_TYPE	You have entered an invalid type argument: <passed type argument>	The options.separator argument is not a string.
SSS_MISSING_REQD_ARGUMENT	<name of missing parameter>	A required argument is not passed.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var statementFile = context.input.file;
4
5 var statementSegmentIterator = statementFile.getSegments({
6   separator: '\\|_|/'
7 }).iterator();
8 statementSegmentIterator.each(function (segment) {
9
10  log.debug({
11    title: 'STATEMENT TEXT',
12    details: segment.value
13  });
14
15  return true;
16  ...

```

```
17 // Add additional code
```

File.lines.iterator()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to pass the next line as an argument to a developer-defined function. You can call this method multiple times to loop over the file contents as a stream. Return false to stop the loop. Return true to continue the loop. By default, false is returned when the end of the file is reached.  Important: Content held in memory is limited to 10MB. Therefore, each line must be less than 10MB.
Returns	boolean
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/file Module
Since	2017.1

Parameters

Parameter	Type	Required / Optional	Description	Since
lineContext	iterator	required	Iterator which provides the next line of text from the text file to the iterator function.	2017.1

Errors

Error Code	Message	Thrown If
SSS_FILE_CONTENT_SIZE_EXCEEDED	The content you are attempting to access exceeds the maximum allowed size of 10 MB.	You attempt to return the content of a line larger than 10MB.
YOU_CANNOT_READ_FROM_A_FILE_AFTER_YOU_BEGAN_WRITING_TO_IT		You call File.lines.iterator() after calling File.appendLine(options) . Call File.resetStream() or save the file.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var iterator = invoiceFile.lines.iterator();
4
5 //Skip the first line (CSV header)
```

```

6  iterator.each(function () {return false;});
7  iterator.each(function (line)
8  {
9      // This function updates the total by
10     // adding the amount on each line to it
11     var lineValues = line.value.split(',');
12     var lineAmount = parseFloat(lineValues[1]);
13
14     if (!lineAmount)
15         throw error.create({
16             name: 'INVALID_INVOICE_FILE',
17             message: 'Invoice file contained non-numeric value for total: ' + lineValues[1]
18         });
19
20     total += lineAmount;
21     return true;
22 });
23 ...
24 //Add additional code

```

File.resetStream()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to reset the file contents. Serves as an undo action on any unsaved content written with File.appendLine(options) or File.lines.iterator(). Use this method to reset the reading and writing streams that may have been opened by File.appendLine(options) or File.lines.iterator(). The line pointer (or read iterator) is also set to its previous state. This method can be used on text or .csv files.  Important: To use this method, each line must be less than 10MB.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/file Module
Since	2017.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var afile = file.create({
4      name: 'tmp3.txt',
5      fileType: 'PLAINTEXT',
6      contents: 'one line'
7  });
8  afile.appendLine({
9      value: 'line two'

```

```

10     });
11     afile.resetStream();
12     afile.lines(function f(){});
13     ...
14
15     //Add additional code

```

File.save()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to: - Upload a new file to the NetSuite File Cabinet. - Save an updated file to the NetSuite File Cabinet.  Note: The <code>File.save()</code> method streams files of any size, provided that the file to save or upload meets File Cabinet limits.  Important: If you want to save the file to the NetSuite File Cabinet, you must set a NetSuite File Cabinet folder with the <code>File.folder</code> property. You must do this before you call <code>File.save()</code>.
Returns	The internal ID of the file as a number.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	20 units
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
INVALID_KEY_OR_REF	Invalid folder reference key <passed folder ID>.	The <code>File.folder</code> error is thrown when a folder does not exist or the user does not have permission.
SSS_MISSING_REQD_ARGUMENT	Please enter value(s) for: Folder	The <code>File.folder</code> property is not set before <code>save()</code> is called.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4     name: 'test.txt',
5     fileType: file.Type.PLAINTEXT,

```

```

6     contents: 'Hello World\nHello World'
7   });
8
9   fileObj.folder = 30;
10  var fileId = fileObj.save();
11  ...
12  //Add additional code

```

File.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The description of a file. In the UI, the value of description displays in the Description field on the file record.
Type	string
Module	N/file Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var fileObj = file.load({
4      id: 'Images/myImageFile.jpg'
5  });
6  fileObj.description = 'my test file';
7  var fileId = fileObj.save();
8  ...
9  //Add additional code

```

File.encoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The character encoding on a file. Value is set with the file.Encoding enum.
Type	string
Module	N/file Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var fileObj = file.create({

```

```

4     name: 'test.txt',
5     fileType: file.Type.PLAINTEXT,
6     contents: 'Hello World\nHello World'
7   });
8   fileObj.encoding = file.Encoding.MAC_ROMAN;
9   fileObj.folder = 30;
10  var fileId = fileObj.save();
11  ...
12
13  //Add additional code

```

File.fileType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The file type of a file. This property is read-only. You must set the file type by passing in a file.Type enum value to file.create(options) .
Type	enum
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	You attempt to edit this property after it is set with file.create(options) .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var fileObj = file.load({
4    id: 145
5  });
6  log.debug({
7    details: "File Type: " + fileObj.fileType
8  });
9  ...
10
11 //Add additional code

```

File.folder

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of a file's folder within the NetSuite File Cabinet. Before you upload a file to the NetSuite File Cabinet with File.save() , you must set its File Cabinet folder with the folder property.
-----------------------------	---

Type	number string
Module	N/file Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4     name: 'test.txt',
5     fileType: file.Type.PLAINTEXT,
6     contents: 'Hello World\nHello World'
7 });
8 fileObj.folder = 30;
9 var fileId = fileObj.save();
10 ...
11 //Add additional code

```

File.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the file within the NetSuite File Cabinet. This value is automatically generated by NetSuite. This property is read-only.
Type	number
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	You attempt to edit this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4     id: 'Images/myImageFile.jpg'
5 });
6 log.debug({
7     details: "File ID: " + fileObj.id
8 });

```

```

9  ...
10 //Add additional code

```

File.isInactive

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The inactive status of a file. If set to true, the file is inactive. The default value is false. When a file is inactive, it does not display in the UI unless you select Show Inactives on the File Cabinet page.
Type	boolean
Module	N/file Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var fileObj = file.load({
4      id: 'Images/myImageFile.jpg'
5  });
6  fileObj.name = 'myOldImageFile.jpg';
7  fileObj.isInactive = true;
8  var fileId = fileObj.save();
9  ...
10 //Add additional code

```

File.isOnline

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The Available without Login status of a file. If set to true, users can download the file outside of a current NetSuite login session. The default value is false. Important: This property holds the value of the Available without Login setting found on the file record. It does not reflect the value of the Available Without Login setting found on the Suitelet script deployment record. The Available without Login setting is primarily used for SuiteCommerce websites. When this setting is enabled, websites can access media files in the NetSuite File Cabinet without a current NetSuite login session.
Type	boolean
Module	N/file Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4   id: 'Images/myImageFile.jpg'
5 });
6 fileObj.isOnline = true;
7 var fileId = fileObj.save();
8 ...
9 //Add additional code

```

File.isText

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether a file type is text-based. This property is read-only.
Type	boolean
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	You attempt to edit this property.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4   id: 145
5 });
6 if (fileObj.isText === true){
7   ...
8 }
9 ...
10 //Add additional code

```

File.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of a file.
-----------------------------	---------------------

Type	string
Module	N/file Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4   id: 'Images/myImageFile.jpg'
5 });
6 fileObj.name = 'myOldImageFile.jpg';
7 var fileId = fileObj.save();
8 ...
9 //Add additional code

```

File.path

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The relative path to a file in the NetSuite File Cabinet.
	Note: If the folder is not set with the file.create(options) method, this property holds the file name until the File.folder property is defined. This property is read-only.
Type	string
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	You attempt to edit this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.load({
4   id: 145
5 });
6 log.debug({

```

```

7 |     details: "File Path: " + fileObj.path
8 |   });
9 |   ...
10 |   //Add additional code

```

File.size

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The size of a file in bytes. This property is read-only.  Note: You can use this value to determine if the file is within size limits for File.getContents(). Size will reflect any lines you have streamed into a file. For example, the original file size plus lines appended.
Type	number
Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	You attempt to edit this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var fileObj = file.load({
4 |   id: 'Images/myImageFile.jpg'
5 | });
6 | log.debug({
7 |   details: "File Size: " + fileObj.size
8 | });
9 | ...
10 | //Add additional code

```

File.url

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The URL of a file. This property is read-only.
Type	string

Module	N/file Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	You attempt to edit this property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code ...
2 var fileObj = file.load({
3   id: 'Images/myImageFile.jpg'
4 });
5 log.debug({
6   details: "File URL: " + fileObj.url
7 });
8 ...
9 //Add additional code

```

file.Reader

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Use for special read operations. Reads from a file until a specified delimiter is reached. Reads an arbitrary number of characters from a file.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/file Module
Methods and Properties	Reader Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 var reader = context.input.file.getReader();
2
3 var textUntilFirstComma = reader.readUntil(',');
4 var next10Characters = reader.readChars(10);
5 var textUntilNextNewLine = reader.readUntil('\n');
6 var next100Characters = reader.readChars(100);

```

Reader.readChars(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the next options.number characters from the current position. Returns less than the number if there is not enough characters to read in the file. Returns null if reading is already finished.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/file Module
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.number	number	required	The number of characters to read.	2019.1

Errors

Error Code	Message	Thrown If
SSS_INVALID_READ_SIZE	Read size must be positive.	The options.number argument is not greater than zero.
SSS_INVALID_ARG_TYPE	You have entered an invalid type argument: <passed type argument>	The options.number argument is not a number.
SSS_MISSING_REQD_ARGUMENT	<name of missing parameter>	A required argument is not passed.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var reader = context.input.file.getReader();
4
5 var textUntilFirstComma = reader.readUntil(',');
6 var next10Characters = reader.readChars(10);
7 var textUntilNextNewLine = reader.readUntil('\n');
8 var next100Characters = reader.readChars(100);
9

```

```
10 | ...
11 | // Add additional code
```

Reader.readUntil(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns string from current position to the next occurrence of options.tag. Returns the rest of the string if tag is not found. Returns null if reading is already finished. All types of characters are supported. If there's a character that does not exist until the end of the file, the rest of the file is returned.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/file Module
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.tag	string	required	String containing a tag	2019.1

Errors

Error Code	Message	Thrown If
SSS_TAG_CANNOT_BE_EMPTY	Tag cannot be empty.	The options.tag argument is empty.
SSS_INVALID_ARG_TYPE	You have entered an invalid type argument: <passed type argument>	The options.tag argument is not a string.
SSS_MISSING_REQD_ARGUMENT	<name of missing parameter>	A required argument is not passed.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```
1 | // Add additional code
```



```

2 ...
3 var reader = context.input.file.getReader();
4
5 var textUntilFirstComma = reader.readUntil(',');
6 var next10Characters = reader.readChars(10);
7 var textUntilNextNewLine = reader.readUntil('\n');
8 var next100Characters = reader.readChars(100);
9
10 ...
11 // Add additional code

```

file.copy(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Copy an existing file in the NetSuite File Cabinet. When you copy a file using this method, the copied file has the same properties as the original file. For example, if the original file is available without login, the copied file is also available without login. You can use the file.NameConflictResolution enum to specify how to resolve conflicts when copying files. A conflict occurs when the target folder for a copy operation already contains a file with the same name as the file to copy. When a conflict occurs, you can specify how the conflict is resolved. For example, you can specify that the copy operation should fail if a conflict occurs.
Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	20 units
Module	N/file Module
Since	2021.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.folder	number	required	The internal ID of the folder to copy the file to. This folder must already exist in the File Cabinet.	2021.1
options.id	number	required	The internal ID of the file to copy. This file must already exist in the File Cabinet.	2021.1
options.conflictResolution	string	optional	The conflict resolution value. This parameter specifies the type of conflict resolution to apply when a conflict occurs while copying a file (for example, if the target folder already contains a file with the same name). Use	2021.1

Parameter	Type	Required / Optional	Description	Since
			the values in the file.NameConflictResolution enum to set this parameter. The default value is <code>ConflictResolution.FAIL</code> .	

Errors

Error Code	Thrown If
INVALID_CONFLICT_RESOLUTION_1	A value is specified for the <code>options.conflictResolution</code> parameter that is not from the file.NameConflictResolution enum.
SSS_MISSING_REQD_ARGUMENT	A required argument is not passed.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var fileObj = file.create({
4     name: 'test.txt',
5     fileType: file.Type.PLAINTEXT,
6     contents: 'Hello World\nHello World'
7 });
8 fileObj.folder = -15;
9 var id = fileObj.save();
10
11 fileObj = file.copy({
12     id: id,
13     folder: -4,
14     conflictResolution: file.NameConflictResolution.FAIL
15 });
16 ...
17 // Add additional code

```

file.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to create a new file in the NetSuite File Cabinet. Important: Content held in memory is limited to 10MB.
Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/file Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	- The file name and extension. - Sets the value for the File.name property.	2015.2
options.fileType	enum	required	- The file type. - Sets the value for the File.fileType property. This property is read-only and cannot be changed after the file is created. - Use the file.Type enum to set the value.	2015.2
options.contents	string	optional	- The file content. - File content is lazy loaded; there is no property for it. - If the file type is binary (for example, PDF), the file content must be base64 encoded.	2015.2
options.description	string	optional	- The file description. In the UI, the value of description displays the Description field on the file record. - Sets the value for the File.description property.	2016.2
options.folder	number	optional	- The internal ID of the folder within the NetSuite File Cabinet. You must set the File Cabinet folder before you upload a file to the NetSuite File Cabinet with File.save(). - Sets the value for the File.folder property.	2016.1
options.encoding	string	optional	- The character encoding on a file. - Sets the value for the File.encoding property. - Use the file.Encoding enum to set the value. For more information about converting strings to another type of encoding, see N/encode Module.	2016.2
options.isInactive	boolean	optional	- The inactive status of a file. If set to true, the file is inactive. The default value is false. When a file is inactive, it does not display in the UI unless you select Show Inactives on the File Cabinet page. - Sets the value for the File.isInactive property.	2016.2
options.isOnline	boolean	optional	The Available without Login status of a file. If set to true, users can download the file outside of a current netSuite login session. The default value is false.	2016.2

Parameter	Type	Required / Optional	Description	Since
			■ Sets the value for the File.isOnline property.	

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	<name of missing parameter>	A required argument is not passed.
SSS_INVALID_TYPE_ARG	You have entered an invalid type argument: <passed type argument>	The argument for File.fileType is invalid.
SSS_FILE_CONTENT_SIZE_EXCEEDED	The file you are trying to create exceeds the maximum allowed file size of 10.0 MB.	You attempt to create a file larger than 10MB.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4     name: 'test.txt',
5     fileType: file.Type.PLAINTEXT,
6     contents: 'Hello World\nHello World',
7     description: 'This is a plain text file.',
8     encoding: file.Encoding.UTF8,
9     folder: 30,
10    isOnline: true
11 });
12 ...
13 //Add additional code

```

file.delete(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to delete an existing file from the NetSuite File Cabinet.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/file Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	number string	required	- Internal ID of the file. - To find the internal ID of the file in the UI, click Documents > Files > File Cabinet.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	<name of missing parameter>	A required argument is not passed.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4   name: 'test.txt',
5   fileType: file.Type.PLAINTEXT,
6   contents: 'Hello World\nHello World'
7 });
8 fileObj.folder = 30;
9 var fileId = fileObj.save();
10
11 file.delete({
12   id: fileId
13 });
14 ...
15 //Add additional code

```

file.load(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing file from the NetSuite File Cabinet. The file size limit for this method is 2GB.
Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/file Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	number string	required	The identifier of a file in the File Cabinet. To specify a file in the File Cabinet, you can pass one of the following as the value of this parameter: ■ The internal ID of the file as a number or string ■ The absolute file path to the file in the File Cabinet (for example, 'Images/myImageFile.jpg') ■ The relative file path to the file in the File Cabinet (for example, './Images/myImageFile.jpg' to specify a file path relative to the current folder of your script, or '../Images/myImageFile.jpg' to specify a file path relative to the parent folder of your script) To find the internal ID of the file in the UI, select Documents > Files > File Cabinet.	2015.2

Errors

Error Code	Message	Thrown If
INSUFFICIENT_PERMISSION	You do not have access to the media item you selected.	Internal ID passed is invalid.
RCRD_DSNT_EXIST	That record does not exist. path: {path}	Relative file path passed is invalid.
SSS_MISSING_REQD_ARGUMENT	<name of missing parameter>	A required argument is not passed.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var fileObj = file.load({
4   id: 'Images/myImageFile.jpg'
5 });
6 fileObj.description = 'my test file';
7 var fileId = fileObj.save();
8 ...
9 // Add additional code

```

file.Encoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Enum Description	Enumeration that holds the string values for supported character encoding. Use this enum to set the value of the File.encoding property.  Note: JavaScript does not include an enumeration type. The SuiteScript 2.0 documentation utilizes the term enumeration (or enum) to describe the following: a plain JavaScript object with a flat, map-like structure. Within this object, each key points to a read-only string value.
Module	N/file Module
Sibling Module Members	N/file Module Members
Since	2015.2

Values

Value	Character Set
UTF_8	Unicode
WINDOWS_1252	Western
ISO_8859_1	Western
GB18030	Chinese Simplified
SHIFT_JIS	Japanese
MAC_ROMAN	Western
GB2312	Chinese Simplified
BIG5	Chinese Traditional

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4   name: 'test.txt',
5   fileType: file.Type.PLAINTEXT,
6   contents: 'Hello World\nHello World'
7 });
8 fileObj.encoding = file.Encoding.MAC_ROMAN;
9 fileObj.folder = 30;
10 var fileId = fileObj.save();
11 ...
12 //Add additional code

```

file.NameConflictResolution

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Enum Description	Holds the string values for supported conflict resolution types. Use this enum to specify how to resolve conflicts when copying files. A conflict occurs when the target folder for a copy operation already contains a file with the same name as the file to copy. When a conflict occurs, you can specify how the conflict is resolved. For example, the FAIL value indicates that the copy operation will fail if a conflict occurs. This enum is used to set the value of the conflict resolution parameter in file.copy(options).  Note: JavaScript does not include an enumeration type. The SuiteScript 2.0 documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/file Module
Sibling Module Members	N/file Module Members
Since	2021.1

Values

Value	Description
FAIL	Fail with an error if a conflict occurs.
OVERWRITE	Overwrite the existing file if a conflict occurs. Attributes and permissions of the overwritten file are preserved (meaning that the copied file has the same attributes and permissions as the file that was overwritten).
OVERWRITE_CONTENT_AND_ATTRIBUTES	Overwrite the existing file if a conflict occurs. Attributes and permissions are also overwritten.
RENAME_TO_UNIQUE	Add a numeric suffix to the name of the copied file if a conflict occurs. For example, if you are copying a file named file.txt and a conflict occurs, the name of the copied file is file (1).txt.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var fileObj = file.copy({
4   id: 123,
5   folder: 456,
6   conflictResolution: file.NameConflictResolution.OVERWRITE
7 });
8 ...
9 // Add additional code

```


file.Type

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Enum Description	Enumeration that holds the string values for supported file types. Use this enum to set the value of the File.fileType property. Note that the File.fileType property is read-only. It's value must be set with file.create(options). See N/file Module Script Samples for an example. i Note: JavaScript does not include an enumeration type. The SuiteScript 2.0 documentation utilizes the term enumeration (or enum) to describe the following: a plain JavaScript object with a flat, map-like structure. Within this object, each key points to a read-only string value.
Module	N/file Module
Sibling Module Members	N/file Module Members
Since	2015.2

Values

Value
APPCACHE
AUTOCAD
BMPIMAGE
CERTIFICATE
CONFIG
CSV
EXCEL
FLASH
FREEMARKER
GIFIMAGE
GZIP
HTMIDOC
ICON
JAVASCRIPT
JPGIMAGE
JSON
MESSAGERFC

Value
MP3
MPEGMOVIE
MSPROJECT
PDF
PJPGIMAGE
PLAINTEXT
PNGIMAGE
POSTSCRIPT
POWERPOINT
QUICKTIME
RTF
SCSS
SMS
STYLESHEET
SVG
TAR
TIFFIMAGE
VISIO
WEBAPPPAGE
WEBAPPSSCRIPT
WORD
XMLDOC
XSD
ZIP

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/file Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fileObj = file.create({
4     name      : 'test.txt',
5     fileType: file.Type.PLAINTEXT,
6     contents: 'Hello World\nHello World'
7 });

```

```

8 | fileObj.folder = 30;
9 | var fileId = fileObj.save();
10 | ...
11 | //Add additional code

```

N/format Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the format module to parse formatted data into strings and to convert strings into a specified format. The format module formats data according to personal preferences set on the Set Preferences page, accessible from Home > Set Preferences. See the help topic [Setting Personal Preferences](#).

Go To Samples 

In This Help Topic

■ [N/format Module Members](#)

N/format Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	format.format(options)	string Date	Client and server scripts	Takes a raw value and returns a formatted value. Note: This method is overloaded when you format a datetime or datetimetz value.
	format.parse(options)	Date string number	Client and server scripts	Takes a formatted value and returns a raw value. Note: This method is overloaded when you format a datetime or datetimetz value.
Enum	format.Timezone	enum	Client and server scripts	Holds the string values for supported time zone formats. Use this enum to set the value of the options.timezone parameter.
	format.Type	enum	Client and server scripts	Holds the string values for the supported field types. Use this enum to set the value of the options.type parameter when calling format.format(options) or format.parse(options).

N/format Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/format module:

- [Parse a String to a Date Object](#)
- [Parse a String to a Number](#)
- [Format a Number as a String](#)
- [Format Time of Day as a String](#)

Parse a String to a Date Object

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample parses a string (formatted according to the user's preferences) to a raw Date object, and then parses it back to the formatted string. This sample uses [format.parse\(options\)](#) and [format.format\(options\)](#).

This sample assumes the Date Format set in the preferences is MM/DD/YYYY. You may need to change the value for the date used in this script to match the preferences set in your account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget', 'N/format'], function(serverWidget, format) {
6      function parseAndFormatDateString() {
7          // Assuming Date format is MM/DD/YYYY
8          var initialFormattedDateString = "07/28/2015";
9          var parsedDateStringAsRawDateObject = format.parse({
10             value: initialFormattedDateString,
11             type: format.Type.DATE
12         });
13         var formattedDateString = format.format({
14             value: parsedDateStringAsRawDateObject,
15             type: format.Type.DATE
16         });
17         return [parsedDateStringAsRawDateObject, formattedDateString];
18     }
19     function onRequest(context) {
20         var data = parseAndFormatDateString();
21
22         var form = serverWidget.createForm({
23             title: "Date"
24         });
25
26         var fldDate = form.addField({
27             type: serverWidget.FieldType.DATE,
28             id: "date",
29             label: "Date"
30         });
31         fldDate.defaultValue = data[0];
32
33         var fldString = form.addField({
34             type: serverWidget.FieldType.TEXT,
35             id: "dateastext",

```

```

36         label: "Date as text"
37     });
38     fldString.defaultValue = data[1];
39
40     context.response.writePage(form);
41 }
42 return {
43     onRequest: onRequest
44 };
45 });

```

Parse a String to a Number

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample parses a string (formatted according to the user's preference) to a raw number value, using `format.parse(options)`.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/format'],
6      function(format){
7      function parseToValue() {
8          // Assume number format is 1.000.000,00 and negative format is -100
9          var formattedNum = "-20.000,25"
10         return format.parse({value:formattedNum, type: format.Type.FLOAT})
11     }
12     var rawNum = parseToValue(); // -20000.25 -- a number
13 });

```

Format a Number as a String

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats a raw number value (formatted according to the user's preference) as a string using `format.format(options)`.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/format'],
6      function(format){
7      function formatToString() {
8          // Assume number format is 1.000.000,00 and negative format is {100}
9          var rawNum2 = -44444.44
10         return format.format({value:rawNum2, type: format.Type.FLOAT})
11     }

```

```

12 |     var formattedNum2 = formatToString(); // "44.444,44" -- a string
13 |   });

```

Format Time of Day as a String

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats the time of day as a string using `format.format(options)`.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1 | /**
2 |  * @NApiVersion 2.x
3 |  */
4 |
5 | require(['N/format'],
6 |   function(format){
7 |     function formatTimeOfDay() {
8 |       // Instantiate a new Date object assuming it's now 7:01PM.
9 |       const now = new Date();
10 |      // Select a format from the format.Type enum. In this case TIMEOFDAY
11 |      return format.format({value: now, type: format.Type.TIMEOFDAY})
12 |    }
13 |    const formattedTime = formatTimeOfDay(); // "7:01 pm" -- a string
14 |  });

```

format.format(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Formats a value from the raw value to its appropriate preference format. Note: This method is overloaded when you format a datetime or datetimetz value.
Returns	If a datetime or datetimetz value is specified, the string in date format is returned in the user's local app time zone. Note: If an invalid value is given, the original value passed to <code>options.value</code> is returned. Note: For client scripts, the string returned is based on the user's system time. For server scripts, the string returned is based on the current time in the Pacific Time Zone. Daylight Savings Time does apply.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types
Governance	None
Module	N/format Module

Since	2015.2
-------	--------

Parameters

This method is overloaded when you format a datetime or datetimetz value.

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.value	Date string number	required	The input data to format.	2015.2
options.type	string	required	The field type (for example, DATE, CURRENCY, INTEGER). Use the format.Type enum to set this value.	2015.2

The table below applies to datetime and datetimetz values only.

Parameter	Type	Required / Optional	Description	Since
options.value	Date	required	The Date Object being converted into a string	2015.2
options.type	string	required	The field type (either DATETIME or DATETIMETZ). Set using the format.Type enum.	2015.2
options.timezone	enum number	optional	The time zone specified for the returned string. Set using the format.Timezone enum or key. If a time zone is not specified, the time zone is set based on user preference. If the time zone is invalid, the time zone is set to GMT.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format Module Script Samples](#).

```

1 // Add additional code
2 ...
3     function(format){
4         function formatToString() {
5             // Assume number format is 1.000.000,00 and negative format is (100)
6             var rawNum2 = -44444.44
7             return format.format({value:rawNum2, type: format.Type.FLOAT})
8         }
9         var formattedNum2 = formatToString(); // "44,444,44" -- a string
10 ...
11 // Add additional code

```

format.parse(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Parses a value from the appropriate preference format to its raw value. The appropriate preference format is the one selected in the Date Format field at Home > Set Preferences. For a datetime or datetimetz value, use this method to convert a Date Object into a string based on the specified timezone.  Note: This method is overloaded when you format a datetime or datetimetz value.
Returns	Datetime or datetimetz values are returned as a Date Object.  Note: If the value given is not valid or parsable, the original value passed to options.value is returned.  Note: For client scripts, the string returned is based on the user's system time. For server scripts, the string returned is based on the current time in the Pacific Time Zone. Daylight Savings Time does apply.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types
Governance	None
Module	N/format Module
Since	2015.2

Parameters

This method is overloaded when you format a datetime or datetimetz value.

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	The input data to parse.	2015.2
options.type	string	required	The field type (for example, DATE, CURRENCY, INTEGER). Use the format.Type enum to set this value.	2015.2

The table below applies to datetime and datetimeTZ values only.

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	The string that contains the date and time information in the specified timezone.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The field type (either DATETIME or DATETIMETZ). Use the format.Type enum to set this value.	2015.2
options.timezone	enum	optional	The time zone represented by the options.value string. Use the format.Timezone enum to set this value. If a time zone is not specified, the time zone is based on user preference. If the time zone is invalid, the time zone is set to GMT.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format Module Script Samples](#).

```

1 // Add additional code
2 ...
3 function(format){
4     function parseToValue() {
5         // Assume number format is 1,000,000,00 and negative format is -100
6         var formattedNum = "-20.000,25"
7         return format.parse({value:formattedNum, type: format.Type.FLOAT})
8     }
9     var rawNum = parseToValue(); // -20000.25 -- a number
10 ...
11 // Add additional code

```

format.Timezone

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported time zone formats. Use this enum to set the value of the options.timezone parameter when calling format.format(options) or format.parse(options) .
Module	N/format Module
Sibling Module Members	N/format Module Members
Since	2015.2

Values

This table defines all valid time zone names in Olson Value format and includes daylight savings time rules for each time zone. Olson Values are maintained by the International Assigned Numbers Authority (IANA)

in an international standard time zone database. The values that populate the Time Zone dropdown list found at Home > Set Preferences are also based on these values.

When working with alternate time zones in SuiteScript, use these enumeration values. If necessary, you can use the numerical key in place of an Olson Value string. For example, to source a custom timezone dropdown list.

Key	Olson Value	Description
1	ETC_GMT_PLUS_12: 'Etc/GMT+12'	(GMT-12:00) International Date Line West
2	PACIFIC_SAMOA: 'Pacific/Samoa'	(GMT-11:00) Midway Island, Samoa
3	PACIFIC_HONOLULU: 'Pacific/Honolulu'	(GMT-10:00) Hawaii
4	AMERICA_ANCHORAGE: 'America/Anchorage'	(GMT-09:00) Alaska
5	AMERICA_LOS_ANGELES: 'America/Los_Angeles'	(GMT-08:00) Pacific Time (US & Canada)
6	AMERICA_TIJUANA: 'America/Tijuana'	(GMT-08:00) Tijuana, Baja California
7	AMERICA_DENVER: 'America/Denver'	(GMT-07:00) Mountain Time (US & Canada)
8	AMERICA_PHOENIX: 'America/Phoenix'	(GMT-07:00) Arizona
9	AMERICA_CHIHUAHUA: 'America/Chihuahua'	Don't use this: Choose (GMT-06:00) Central America, Chihuahua or (GMT-07:00) La Paz, Hermosillo
10	AMERICA_HERMOSILLO: 'America/Hermosillo'	(GMT-07:00) La Paz, Hermosillo
11	AMERICA_CHICAGO: 'America/Chicago'	(GMT-06:00) Central Time (US & Canada)
12	AMERICA_REGINA: 'America/Regina'	(GMT-06:00) Saskatchewan
13	AMERICA_GUATEMALA: 'America/Guatemala'	(GMT-06:00) Central America, Chihuahua
14	AMERICA_MEXICO_CITY: 'America/Mexico_City'	(GMT-06:00) Guadalajara, Mexico City, Monterrey - Old
15	AMERICA_NEW_YORK: 'America/New_York'	(GMT-05:00) Eastern Time (US & Canada)
16	US_EAST_INDIANA: 'US/East-Indiana'	(GMT-05:00) Indiana (East)
17	AMERICA_BOGOTA: 'America/Bogota'	(GMT-05:00) Bogota, Lima, Quito
18	AMERICA_CARACAS: 'America/Caracas'	(GMT-04:30) Caracas
19	AMERICA_HALIFAX: 'America/Halifax'	(GMT-04:00) Atlantic Time (Canada)
20	AMERICA_LA_PAZ: 'America/La_Paz'	(GMT-04:00) Georgetown, La Paz, San Juan
21	AMERICA_MANAUS: 'America/Manaus'	(GMT-04:00) Manaus
22	AMERICA_SANTIAGO: 'America/Santiago'	(GMT-04:00) Santiago
23	AMERICA_ST_JOHNS: 'America/St_Johns'	(GMT-03:30) Newfoundland
24	AMERICA_SAO_PAULO: 'America/Sao_Paulo'	(GMT-03:00) Brasilia
25	AMERICA_BUENOS_AIRES: 'America/Buenos_Aires'	(GMT-03:00) Buenos Aires
26	ETC_GMT_PLUS_3: 'Etc/GMT+3'	(GMT-03:00) Cayenne
27	AMERICA_GODTHAB: 'America/Godthab'	(GMT-03:00) Greenland

Key	Olson Value	Description
28	AMERICA_MONTEVIDEO: 'America/Montevideo'	(GMT-03:00) Montevideo
29	AMERICA_NORONHA: 'America/Noronha'	(GMT-02:00) Mid-Atlantic
30	ETC_GMT_PLUS_1: 'Etc/GMT+1'	(GMT-01:00) Cabo Verde
31	ATLANTIC_AZORES: 'Atlantic/Azores'	(GMT-01:00) Azores
32	EUROPE_LONDON: 'Europe/London'	(GMT) Greenwich Mean Time : Dublin, Edinburgh, Lisbon, London
33	GMT: 'GMT'	(GMT) Casablanca
34	ATLANTIC_REYKJAVIK: 'Atlantic/Reykjavik'	(GMT) Monrovia, Reykjavik
35	EUROPE_WARSAW: 'Europe/Warsaw'	(GMT+01:00) Sarajevo, Skopje, Warsaw, Zagreb
36	EUROPE_PARIS: 'Europe/Paris'	(GMT+01:00) Brussels, Copenhagen, Madrid, Paris
37	ETC_GMT_MINUS_1: 'Etc/GMT-1'	(GMT+01:00) West Central Africa
38	EUROPE_AMSTERDAM: 'Europe/Amsterdam'	(GMT+01:00) Amsterdam, Berlin, Bern, Rome, Stockholm, Vienna
39	EUROPE_BUDAPEST: 'Europe/Budapest'	(GMT+01:00) Belgrade, Bratislava, Budapest, Ljubljana, Prague
40	AFRICA_CAIRO: 'Africa/Cairo'	(GMT+02:00) Cairo
41	EUROPE_ISTANBUL: 'Europe/Istanbul'	(GMT+02:00) Athens, Bucharest, Istanbul
42	ASIA_JERUSALEM: 'Asia/Jerusalem'	(GMT+02:00) Jerusalem
43	ASIA_AMMAN: 'Asia/Amman'	(GMT+02:00) Amman
44	ASIA_BEIRUT: 'Asia/Beirut'	(GMT+02:00) Beirut
45	AFRICA_JOHANNESBURG: 'Africa/Johannesburg'	(GMT+02:00) Harare, Pretoria
46	EUROPE_KIEV: 'Europe/Kiev'	(GMT+02:00) Helsinki, Kyiv, Riga, Sofia, Tallinn, Vilnius
47	EUROPE_MINSK: 'Europe/Minsk'	(GMT+02:00) Minsk
48	AFRICA_WINDHOEK: 'Africa/Windhoek'	(GMT+02:00) Windhoek
49	ASIA_RIYADH: 'Asia/Riyadh'	(GMT+03:00) Kuwait, Riyadh
50	EUROPE_MOSCOW: 'Europe/Moscow'	(GMT+03:00) Moscow, St. Petersburg, Volgograd
51	ASIA_BAGHDAD: 'Asia/Baghdad'	(GMT+03:00) Baghdad
52	AFRICA_NAIROBI: 'Africa/Nairobi'	(GMT+03:00) Nairobi
53	ASIA_TEHRAN: 'Asia/Tehran'	(GMT+03:30) Tehran
54	ASIA_MUSCAT: 'Asia/Muscat'	(GMT+04:00) Abu Dhabi, Muscat
55	ASIA_BAKU: 'Asia/Baku'	(GMT+04:00) Baku
56	ASIA_YEREVAN: 'Asia/Yerevan'	(GMT+04:00) Caucasus Standard Time
57	ETC_GMT_MINUS_3: 'Etc/GMT-3'	(GMT+04:00) Tbilisi

Key	Olson Value	Description
58	ASIA_KABUL: 'Asia/Kabul'	(GMT+04:30) Kabul
59	ASIA_KARACHI: 'Asia/Karachi'	(GMT+05:00) Islamabad, Karachi
60	ASIA_YEKATERINBURG: 'Asia/Yekaterinburg'	(GMT+05:00) Ekaterinburg
61	ASIA_TASHKENT: 'Asia/Tashkent'	(GMT+05:00) Tashkent
62	ASIA_CALCUTTA: 'Asia/Calcutta'	(GMT+05:30) Chennai, Kolkata, Mumbai, New Delhi
63	ASIA_KATMANDU: 'Asia/Katmandu'	(GMT+05:45) Kathmandu
64	ASIA_ALMATY: 'Asia/Almaty'	(GMT+06:00) Novosibirsk
65	ASIA_DHAKA: 'Asia/Dhaka'	(GMT+06:00) Astana, Dhaka
66	ASIA_RANGOON: 'Asia/Rangoon'	(GMT+06:30) Yangon (Rangoon)
67	ASIA_BANGKOK: 'Asia/Bangkok'	(GMT+07:00) Bangkok, Hanoi, Jakarta
68	ASIA_KRASNOYARSK: 'Asia/Krasnoyarsk'	(GMT+07:00) Krasnoyarsk
69	ASIA_HONG_KONG: 'Asia/Hong_Kong'	(GMT+08:00) Beijing, Chongqing, Hong Kong, Urumqi
70	ASIA_KUALA_LUMPUR: 'Asia/Kuala_Lumpur'	(GMT+08:00) Kuala Lumpur, Singapore
71	ASIA_TAIPEI: 'Asia/Taipei'	(GMT+08:00) Taipei
72	AUSTRALIA_PERTH: 'Australia/Perth'	(GMT+08:00) Perth
73	ASIA_IRKUTSK: 'Asia/Irkutsk'	(GMT+08:00) Irkutsk
74	ASIA_MANILA: 'Asia/Manila'	(GMT+08:00) Manila
75	ASIA_SEOUL: 'Asia/Seoul'	(GMT+09:00) Seoul
76	ASIA_TOKYO: 'Asia/Tokyo'	(GMT+09:00) Osaka, Sapporo, Tokyo
77	ASIA_YAKUTSK: 'Asia/Yakutsk'	(GMT+09:00) Yakutsk
78	AUSTRALIA_DARWIN: 'Australia/Darwin'	(GMT+09:30) Darwin
79	AUSTRALIA_ADELAIDE: 'Australia/Adelaide'	(GMT+09:30) Adelaide
80	AUSTRALIA_SYDNEY: 'Australia/Sydney'	(GMT+10:00) Canberra, Melbourne, Sydney
81	AUSTRALIA_BRISBANE: 'Australia/Brisbane'	(GMT+10:00) Brisbane
82	AUSTRALIA_HOBART: 'Australia/Hobart'	(GMT+10:00) Hobart
83	PACIFIC_GUAM: 'Pacific/Guam'	(GMT+10:00) Guam, Port Moresby
84	PACIFIC_GUADALCANAL: 'Pacific/Guadacanal'	(GMT+11:00) Guadacanal
85	ASIA_VLADIVOSTOK: 'Asia/Vladivostok'	(GMT+10:00) Vladivostok
86	PACIFIC_KWAJALEIN: 'Pacific/Kwajalein'	(GMT+12:00) Fiji, Marshall Is.
87	PACIFIC_AUCKLAND: 'Pacific/Auckland'	(GMT+12:00) Auckland, Wellington
88	PACIFIC_TONGATAPU: 'Pacific/Tongatapu'	(GMT+13:00) Nuku'alofa

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var date = new Date(); //Mon Aug 24 2015 17:27:16 GMT-0700 (Pacific Daylight Time)
4   var TOKYO = format.format({
5     value: date,
6     type: format.Type.DATETIME,
7     timezone: format.Timezone.ASIA_TOKYO
8   }); //Returns "8/25/2015 9:27:16 am"
9
10  var NEWYORK = format.format({
11    value: date,
12    type: format.Type.DATETIME,
13    timezone: format.Timezone.AMERICA_NEW_YORK
14  }); //Returns "8/24/2015 8:27:16 pm"
15
16  var dateStr = "03/17/2015 09:00:00 pm"
17  var TOKYO_2 = format.parse({
18    value: dateStr,
19    type: format.Type.DATETIME,
20    timezone: format.Timezone.ASIA_TOKYO
21  }); //Returns Date object [[ Tue Mar 17 2015 05:00:00 GMT-0700 (PDT) ]]
22
23  var NEWYORK_2 = format.parse({
24    value: dateStr,
25    type: format.Type.DATETIME,
26    timezone: format.Timezone.AMERICA_NEW_YORK
27  }); //Returns Date object [[ Tue Mar 17 2015 18:00:00 GMT-0700 (PDT) ]]
28  ...
29 // Add additional code
```

format.Type

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the supported field types. Use this enum to set the value of the options.type parameter when calling format.format(options) or format.parse(options) .
Module	N/format Module
Sibling Module Members	N/format Module Members
Since	2015.2

Values

Value	Value	Value
ADDRESS	EMAILS	POSCURRENCY

Value	Value	Value
CCEXPDATE	FLOAT	POSFLOAT
CCNUMBER	FULLPHONE	POSINTEGER
CCVALIDFROM	FUNCTION	QUOTEDFUNCTION
CHECKBOX	FURIGANA	RADIO
CLOBTEXT	IDENTIFIER	RATE
COLOR	IDENTIFIERANYCASE	RATEHIGHPRECISION
CURRENCY	INTEGER	SELECT
CURRENCY2	MULTISELECT	TEXT
DATE	MMYYDATE	TEXTAREA
DATETIME	NONNEG CURRENCY	TIME
DATETIMETZ	NONNEGFLOAT	TIMEOFDAY
DOCUMENT	PACKAGE	TIMETRACK
DYNAMICPRECISION	PERCENT	URL
EMAIL	PHONE	-

Be aware of the following:

- The following field types require a value of greater than 0:
 - POSCURRENCY
 - POSFLOAT
 - POSINTEGER
- NONNEGFLOAT requires a value that is greater than or equal to 0
- CURRENCY field type rounds the number based on the user's currency precision setting and is limited to hundredths / 2 decimals (0.00).
- CURRENCY2 field type formats using a record's currency precision.
- If any of the following field types is set to hidden, the object returned is text.
 - CHECKBOX
 - RADIO
 - SEELCT
 - TEXTAREA
- DYNAMICPRECISION is controlled by NetSuite and can differ from field to field.

Syntax



Important: The following code samples show the syntax for this member. It is not a functional example. For a complete script example, see [N/format Module Script Samples](#).

```
1 //Run this sample in debugger to see how values are formatted.
2
```

```

3  ...
4  require(['N/format'], function(format) {
5      Object.getOwnPropertyNames(format.Type).forEach(function(type) {
6          log.debug(type, format.format({
7              value: 12345,
8              type: format.Type[type]
9          }));
10         /*use any value instead of 12345 to see how is it formatted*/
11     })
12 })

```

```

1  // Add additional code
2  ...
3  function formatTimeOfDay() {
4      // Assume the time format is hh:mm (24 hours)
5      var now = new Date(); // Say it's 7:01PM right now.
6      var formattedTime = format.format({value: now, type: format.Type.TIMEOFDAY})
7      });
8  }
9  ...
10 // Add additional code

```

```

1  require(['N/format'],
2      function(format) {
3
4      function parseAndFormatDateString()
5      {
6          var rawDateString = "07/28/2015";
7          var parsedDate= format.parse({
8              value: rawDateString,
9              type: format.Type.DATE
10         });
11         console.log(parsedDate);
12     }
13
14     parseAndFormatDateString();
15 }
16 );

```

N/format/i18n Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/format/i18n module to format strings in international context and format numbers to currency or number strings. You can also use this module to format phone number to strings and parse strings to phone number.

[Go To Samples {…}](#)

In This Help Topic

- [N/format/i18n Module Members](#)
- [Currency Formatter Object Members](#)
- [Number Formatter Object Members](#)
- [Phone Number Formatter Object Members](#)
- [Phone Number Parser Object Members](#)

N/format/i18n Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	format.CurrencyFormatter	Object	Client and server scripts	Represents the object that formats the number to currency string.
	format.NumberFormatter	Object	Client and server scripts	Represents the object that formats the number to string.
	format.PhoneNumberFormatter	Object	Client and server scripts	Represents the object that formats the phone number to string.
	format.PhoneNumberParser	Object	Client and server scripts	Represents the object that parses the string to phone number.
Method	format.spellOut(options)	string	Client and server scripts	Creates a string containing the spelled-out version of the specified number in a specified locale.
	format.getCurrencyFormatter(options)	object	Client and server scripts	Creates a format.CurrencyFormatter object to format numbers to currency strings.
	format.getNumberFormatter(options)	object	Client and server scripts	Creates a format.NumberFormatter object to format numbers to strings.
	format.getPhoneNumberFormatter(options)	object	Client and server scripts	Creates a format.PhoneNumberFormatter object to format phone numbers to strings.
	format.getPhoneNumberParser(options)	object	Client and server scripts	Parses a phone number from a string. Returns a format.PhoneNumberParser object.
Enum	format.NegativeNumberFormat	enum	Client and server scripts	Holds the values for the negative number format. Used to set the value of the options.negativeNumberFormatter parameter of the format.getNumberFormatter(options) .
	format.Currency	enum	Client and server scripts	Holds the values for the currency code. Used to set the value of the options.currency parameter of the format.getCurrencyFormatter(options) method.

Currency Formatter Object Members

The following members are called on the [format.CurrencyFormatter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	CurrencyFormatter.currency	string	Client and server scripts	Indicates the currency code.
	CurrencyFormatter.locale	string	Client and server scripts	The locale of the currency formatter.
	CurrencyFormatter.symbol	string	Client and server scripts	Indicates the currency symbol.
	CurrencyFormatter.numberFormatter	object	Client and server scripts	Contains the format.NumberFormatter object derived from format.CurrencyFormatter with the same number formatting parameters without currency symbol.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	CurrencyFormatter.format(options)	string	Client and server scripts	Formats the number to the currency string.

Number Formatter Object Members

The following members are called on the [format.NumberFormatter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	NumberFormatter.groupSeparator	string	Client and server scripts	Indicates the group separator.
	NumberFormatter.decimalSeparator	string	Client and server scripts	Indicates the decimal separator.
	NumberFormatter.locale	string	Client and server scripts	The locale of the number formatter.
	NumberFormatter.precision	number	Client and server scripts	Indicates the precision.
	format.NegativeNumberFormat	enum	Client and server scripts	Indicates the negative number format.
	NumberFormatter.format(options)	string	Client and server scripts	Formats the number to string.

Phone Number Formatter Object Members

The following members are called on the [format.PhoneNumberFormatter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	format.PhoneNumberFormatter	Object	Client and server scripts	The object that formats the phone number to string.
Method	PhoneNumberFormatter.format(options)	string	Client and server scripts	Formats phone number object to string.
Enum	format.PhoneNumberFormatType	enum	Client and server scripts	Holds the values for the phone number format type. Used to set the value of the <code>options.formatType</code> parameter of the format.getPhoneNumberFormatter(options) method.
	format.Country	enum	Client and server scripts	Hold the values for the countries. Used to set the value of the <code>options.defaultCountry</code> parameter in the format.getPhoneNumberParser(options) method.

Phone Number Parser Object Members

The following members are called on the [format.PhoneNumberParser](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	format.PhoneNumberParser	Object	Client and server scripts	The object that parses the string to phone number.
Method	PhoneNumberParser.parse(options)	Object of type <code>PhoneNumber</code>	Client and server scripts	Parses string to phone number.

N/format/i18n Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/format/i18n module:

- Format 12345 as a German String
- Format a Number as a String Using N/format/i18n
- Format Numbers as Currency Strings
- Parse a Czech Phone Number
- Parse a U.S. Phone Number
- Format Numbers Based on the Locale Parameter
- Format Currency Based on the Locale Parameter
- Format Numbers and Currencies Based on the English-India Locale Parameter

Format 12345 as a German String

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample spells out the number 12345 as a string in German, “zwölftausenddreihundertfünfundvierzig”.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/format/i18n'], function(format) {
5       var spellOut = format.spellOut({
6           number: 12345,
7           locale: "DE"
8       });
9       // spellOut is 'zwölftausenddreihundertfünfundvierzig'
10  });

```

Format a Number as a String Using N/format/i18n

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats a number as a string.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/format/i18n'], function(format) {
5      var numberFormatter = format.getNumberFormatter();
6
7      var gs = numberFormatter.groupSeparator; // gs is ','
8      var ds = numberFormatter.decimalSeparator; // ds is '.'
9      var precision = numberFormatter.precision; // precision is '2'
10     var nnf = numberFormatter.negativeNumberFormat; // nnf is 'BRACKETS'
11
12     var formatNum1 = numberFormatter.format({
13         number: 12.53
14     }); // formatNum1 is '12.53'
15
16     var formatNum2 = numberFormatter.format({
17         number: 12845.22
18     }); // formatNum2 is '12,845.22'
19
20     var formatNum3 = numberFormatter.format({
21         number: -5421
22     }); // formatNum3 is '(5,421.00)'
23
24     var formatNum4 = numberFormatter.format({
25         number: 0.00
26     }); // formatNum4 is '0.00'
27
28     var formatNum5 = numberFormatter.format({
29         number: 0.3456789
30     }); // formatNum5 is '0.35'
31 });

```

Format Numbers as Currency Strings

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats numbers as currency strings.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/format/i18n'], function(format) {
5      var curFormatter = format.getCurrencyFormatter({
6          currency: "EUR"
7      });
8
9      var curCur = curFormatter.currency; // curCur is 'EUR'
10     var numberFormat = curFormatter.numberFormatter;
11
12     var curSymbol = curFormatter.symbol; // curSymbol is '€'
13     var curSeparator = numberFormat.groupSeparator; // curSeparator is ','
14     var curDecimalSep = numberFormat.decimalSeparator; // curDecimalSep is '.'
15     var curPrecision = numberFormat.precision; // curPrecision is 2
16     var curNNF = numberFormat.negativeNumberFormat; // curNNF is MINUS

```

```

17
18     curFormatNum1 = curFormatter.format({
19         number: 12.53
20     }); // curFormat1 is '€12,53'
21
22     curFormatNum2 = curFormatter.format({
23         number: -5421
24     }); // curFormat2 is '€-5 421,00'
25
26     curFormatNum3 = curFormatter.format({
27         number: 0.00
28     }); // curFormat3 is '€0,00'
29
30     curFormatNum4 = curFormatter.format({
31         number: 0.3456789
32     }); // curFormat4 is '€0,35'
33 });

```

Parse a Czech Phone Number

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample parses a Czech phone number and logs the resulting country code, extension, national number, number of leading zeros, carrier code, and raw input.

i Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/format/i18n'], function(format) {
5      var origNumberStr = "602547854ext.154";
6      var pnParser = format.getPhoneNumberParser({
7          defaultCountry: format.Country.CZECH_REPUBLIC
8      });
9      var phoneNumber = pnParser.parse({
10         number: origNumberStr
11     });
12
13     /* after parsing:
14     phoneNumber.countryCode is '420'
15     phoneNumber.extension is '154'
16     phoneNumber.nationalNumber is '602547854'
17     phoneNumber.numberofLeadingZeros is 1
18     phoneNumber.carrierCode is ''
19     phoneNumber.rawInput is '602547854ext.154'
20     */
21
22     var pnFormatter = format.getPhoneNumberFormatter({});
23     var strNumber = pnFormatter.format({
24         number: phoneNumber
25     }); // strNumber is '+420 602 547 854 ext. 154'
26 });

```

Parse a U.S. Phone Number

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample parses a U.S. phone number and logs the country code, extensions, national number, number of leading zeros, carrier code, and raw input.

Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/format/i18n'], function(format) {
5      var origNumberStr = "7524105210ext.154";
6      var pnParser = format.getPhoneNumberParser({
7          defaultCountry: format.Country.UNITED_STATES
8      });
9      var phoneNumber = pnParser.parse({
10         number: origNumberStr
11     });
12
13     /* after parsing:
14     phoneNumber.countryCode is '1'
15     phoneNumber.extension is '154'
16     phoneNumber.nationalNumber is '7524105210'
17     phoneNumber.numberofLeadingZeros is 1
18     phoneNumber.carrierCode is ''
19     phoneNumber.rawInput is '7524105210ext.154'
20     */
21
22     var pnFormatter = format.getPhoneNumberFormatter({
23         formatType: format.PhoneNumberFormatType.NATIONAL
24     });
25     var strNumber = pnFormatter.format({
26         number: phoneNumber
27     }); // strNumber is '(752) 410-5210 ext. 154'
28 });

```

Format Numbers Based on the Locale Parameter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats numbers based on the locale parameter.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/format/i18n'], function(format) {
5      var numberFormatter = format.getNumberFormatter({
6          locale: "fr_FR"
7      });
8
9      var gs = numberFormatter.groupSeparator; // gs is ','
10     var ds = numberFormatter.decimalSeparator; // ds is '.'
11     var prec = numberFormatter.precision; // prec is 2
12     var nnf = numberFormatter.negativeNumberFormat; // nnf is MINUS
13
14     var number1 = numberFormatter.format({
15         number: 123456.55
16     }); // number1 is '123 456,55'
17
18     var number2 = numberFormatter.format({
19         number: -123456.55
20     });
21 });

```

```

20     }); // number2 is '123 456,55'
21 });

```

Format Currency Based on the Locale Parameter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats currency based on the locale parameter.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/format/i18n'], function(format) {
5       var curFormatter = format.getCurrencyFormatter({
6           locale: "IT_IT"
7       });
8
9       var curCur = curFormatter.currency; // curCur is 'EUR'
10      var sym = curFormatter.symbol; // sym is '€'
11
12      var numberFormat = curFormatter.numberFormatter;
13
14      var gs = numberFormat.groupSeparator; // gs is ','
15      var ds = numberFormat.decimalSeparator; // ds is '.'
16      var prec = numberFormat.precision; // prec is 2
17      var nnf = numberFormat.negativeNumberFormat; // nnf is MINUS
18
19      var currency1 = curFormatter.format({
20          number: 123456.55
21      }); // currency1 is '€123.456,55'
22
23      var currency2 = curFormatter.format({
24          number: -123456.55
25      }); // currency2 is '€-123.456,55'
26  });

```

Format Numbers and Currencies Based on the English-India Locale Parameter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample formats numbers and currencies based on the English-India (en_IN) locale parameter.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   require(['N/format/i18n'], function(format) {
5       var curFormatter = format.getCurrencyFormatter({
6           locale: "en_IN"

```

```

7   });
8
9   var curCur = curFormatter.currency; // curCur is 'INR'
10
11  var numberFormat = curFormatter.numberFormatter;
12  var sym = curFormatter.symbol;        // sym is '₹'
13  var gs = numberFormat.groupSeparator; // gs is ','
14  var ds = numberFormat.decimalSeparator; // ds is '.'
15  var prec = numberFormat.precision;    // prec is 2
16  var nnf = numberFormat.negativeNumberFormat; // nnf is MINUS
17
18  var currency1 = curFormatter.format({
19    number: 12345678.55
20  }); // currency1 is '₹1,23,45,678.55'
21
22  var currency2 = curFormatter.format({
23    number: -345678.55
24  }); // currency2 is '₹-3,45,678.55'
25  });

```

format.CurrencyFormatter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The object that formats the number to currency string.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/format/i18n Module
Methods and Properties	Currency Formatter Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4   currency: format.Currency.USD
5 });
6 // curFormatter is '{ "currency": "USD", "locale": "en_US", "symbol": "$", "isPrefixSymbol": true, "numberFormatter": { "groupSeparator": ",", "decimalSepara
7   tor": ".", "precision": "2", "minPrecision": null, "maxPrecision": null, "negativeNumberFormat": "BRACKETS", "locale": "en_US" } }'
8 ...
9 // Add additional code

```

CurrencyFormatter.currency

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Describes the currency code.
Type	string (read-only)
Module	N/format/i18n Module

Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4     currency: "USD"
5 });
6 var curCur = curFormatter.currency; // curCur is 'USD'
7 ...
8 // Add additional code

```

CurrencyFormatter.locale

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The locale of the currency formatter.
Type	string (read-only)
Module	N/format/i18n Module
Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2021.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4     locale: 'en_US'
5 });
6 var theLocale = curFormatter.locale; // theLocale is 'en_US'
7 ...
8 // Add additional code

```

CurrencyFormatter.symbol

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Describes the symbol of the currency code.
----------------------	--

Type	string (read-only)
Module	N/format/i18n Module
Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4     currency: "USD"
5 });
6 var curSymbol = curFormatter.symbol; // curSymbol is '$'
7 ...
8 // Add additional code

```

CurrencyFormatter.numberFormatter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Contains the format.NumberFormatter object derived from format.CurrencyFormatter with the same number formatting parameters without currency symbol.
Type	string (read-only)
Module	N/format/i18n Module
Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4     currency: "USD"
5 });
6 var numberFormatter = curFormatter.numberFormatter;
7
8 var formattedNumber = numberFormatter.format({
9     number: -12.5366
10 });
11 // formattedNumber is '12.54'
12 ...

```

13 | // Add additional code

CurrencyFormatter.format(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Formats the number to the currency string.
Returns	String
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/format/i18n Module
Methods and Properties	N/format/i18n Module Members
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.number	number	required	The number to be formatted

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4     currency: "USD"
5 });
6 var formattedCurrency = curFormatter.format({
7     number: 12.53
8 });
9 // formattedCurrency is '$12.53'
10 ...
11 // Add additional code

```

format.NumberFormatter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The object that formats number to string.
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/format/i18n Module
Methods and Properties	Number Formatter Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numFormatter1 = format.getNumberFormatter(); // no parameter given so the default number formatter object returned
4
5 var numFormatter2 = format.getNumberFormatter({
6     groupSeparator: " ",
7     decimalSeparator: ".",
8     precision: 2,
9     negativeNumberFormat: format.NegativeNumberFormat.MINUS
10 });
11
12 var number1 = numFormatter1.format({
13     number: 12.53
14 }); // number1 is '12.53'
15
16 var number2 = numFormatter2.format({
17     number: 12845.22
18 }); // number2 is '12 845,22'
19 ...
20 // Add additional code

```

NumberFormatter.groupSeparator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the group separator.
Type	string (read-only)
Module	N/format/i18n Module
Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numFormatter = format.getNumberFormatter({

```

```

4     groupSeparator: " ",
5     decimalSeparator: ",",
6     precision: 2,
7     negativeNumberFormat: format.NegativeNumberFormat.MINUS
8   });
9   var groupSep = numFormatter.groupSeparator; // groupSep will be ' ' (space)
10  ...
11  // Add additional code

```

NumberFormatter.decimalSeparator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the decimal separator.
Type	string (read-only)
Module	N/format/i18n Module
Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var numFormatter = format.getNumberFormatter({
4     groupSeparator: " ",
5     decimalSeparator: ",",
6     precision: 2,
7     negativeNumberFormat: format.NegativeNumberFormat.MINUS
8   });
9  var decimalSep = numFormatter.decimalSeparator; // decimalSep will be ','
10 ...
11 // Add additional code

```

NumberFormatter.locale

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The locale of the number formatter.
Type	string (read-only)
Module	N/format/i18n Module
Parent Object	format.NumberFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2021.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numFormatter = format.getNumberFormatter({
4   groupSeparator: ' ',
5   decimalSeparator: ',',
6   locale: 'en_US',
7   precision: 2,
8   negativeNumberFormat: format.NegativeNumberFormat.MINUS
9 });
10
11 var theLocale = numFormatter.locale; //theLocale will be 'en_US'
12 ...
13 // Add additional code

```

NumberFormatter.precision

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the precision.
Type	number (read-only)
Module	N/format/i18n Module
Parent Object	format.CurrencyFormatter
Sibling Object Members	N/format/i18n Module Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numFormatter = format.getNumberFormatter({
4   groupSeparator: " ",
5   decimalSeparator: ",",
6   precision: 2,
7   negativeNumberFormat: format.NegativeNumberFormat.MINUS
8 });
9 var precision = numFormatter.precision; // precision will be 2
10 ...
11 // Add additional code

```

NumberFormatter.format(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Format number to the number string.
---------------------------	-------------------------------------

Returns	String
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/format/i18n Module
Methods and Properties	N/format/i18n Module Members
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.number	number	required	The number to be formatted

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numFormatter = format.getNumberFormatter({
4   groupSeparator: " ",
5   decimalSeparator: ".",
6   precision: 2,
7   negativeNumberFormat: format.NegativeNumberFormat.MINUS
8 });
9 var formattedNum = numFormatter.format({
10   number: 12845.22
11 });
12 // formattedNum will be '12 845,22'
13 ...
14 // Add additional code

```

format.PhoneNumberFormatter

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The object that formats the phone number to string.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/format/i18n Module
Methods and Properties	Phone Number Formatter Object Members
Since	2020.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var origNumberStr = "602547854ext.154";
4
5 var pnParser = format.getPhoneNumberParser({
6     defaultCountry: format.Country.CZECH_REPUBLIC
7 });
8 var phoneNumber = pnParser.parse({
9     number: origNumberStr
10 });
11
12 var pnFormatter = format.getPhoneNumberFormatter({});
13 var strNumber = pnFormatter.format({
14     number: phoneNumber
15 });
16 // expected output (strNumber) is '+420 602 547 854 ext. 154'
17
18 ...
19 // Add additional code

```

format.PhoneNumberParser

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The object that parses the string with the phone number to an object.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/format/i18n Module
Methods and Properties	Phone Number Parser Object Members
Since	2020.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var pnParser = format.getPhoneNumberParser({
4     defaultCountry: format.Country.CZECH_REPUBLIC
5 });
6 var phoneNumber = pnParser.parse({
7     number: "602547854ext.154"
8 });
9 /* expected parsed output:
10  phoneNumber.countryCode = 420
11  phoneNumber.extension = 154
12  phoneNumber.nationalNumber = 602547854
13  phoneNumber.numberOfTypeLeadingZeros = 1
14  phoneNumber.carrierCode = (blank)

```

```

15 | phoneNumber.rawInput = 602547854ext.154
16 | */...
17 | // Add additional code

```

PhoneNumberFormatter.format(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Formats phone number object to string.
Returns	String
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/format/i18n Module
Parent Object	format.PhoneNumberFormatter
Methods and Properties	Phone Number Formatter Object Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.number	Object	required	The phone number to be formatted.

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | var origNumberStr = "602547854ext.154";
4 | var pnParser = format.getPhoneNumberParser({
5 |     defaultCountry: format.Country.CZECH_REPUBLIC
6 | });
7 |
8 | var phoneNumber = pnParser.parse({
9 |     number: origNumberStr
10 | });
11 |
12 | var pnFormatter = format.getPhoneNumberFormatter({});
13 | var strNumber = pnFormatter.format({
14 |     number: phoneNumber
15 | });
16 | // expected output (strNumber) is '+420 602 547 854 ext. 154'
17 | ...
18 | // Add additional code

```


PhoneNumberParser.parse(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Parses the string containing the phone number and returns the phone number object. If the input string contains the country code, it parses the given phone number. If the input string does not contain the country code, the country given to PhoneNumberParser as constructor parameter is used.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/format/i18n Module
Parent Object	format.PhoneNumberParser
Methods and Properties	Phone Number Parser Object Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.number	string	required	The string to be parsed.

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var pnParser = format.getPhoneNumberParser({
4     defaultCountry: format.Country.CZECH_REPUBLIC
5 });
6 var phoneNumber = pnParser.parse({
7     number: "602547854ext.154"
8 });
9 /* expected parsed output:
10  phoneNumber.countryCode = 420
11  phoneNumber.extension = 154
12  phoneNumber.nationalNumber = 602547854
13  phoneNumber.numberOfLeadingZeros = 1
14  phoneNumber.carrierCode = (blank)
15  phoneNumber.rawInput = 602547854ext.154
16 */
17 ...
18 // Add additional code

```

format.PhoneNumberFormatType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the phone number format. Used to set the value of the <code>options.formatType</code> parameter of the format.getPhoneNumberFormatter(options) method.
Module	N/format/i18n Module
Sibling Module Members	Phone Number Formatter Object Members
Since	2020.2

Values

Value
E164
INTERNATIONAL
NATIONAL
RFC3966

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var pnFormatter = format.getPhoneNumberFormatter({
4     formatType: format.PhoneNumberFormatType.NATIONAL
5 });
6 ...
7 // Add additional code

```

format.Country

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the country. Used to set the value of the <code>options.defaultCountry</code> parameter of the format.getPhoneNumberParser(options) method.
Module	N/format/i18n Module

Sibling Module Members	Phone Number Formatter Object Members
Since	2020.2

Values

Value	Value	Value
■ AFGHANISTAN ■ ALAND_ISLANDS ■ ALBANIA ■ ALGERIA ■ AMERICAN_SOMOA ■ ANDORRA ■ ANGOLA ■ ANGUILLA ■ ANTARCTICA ■ ANTIGUA_AND_BARBUDA ■ ARGENTINA ■ ARMENIA ■ ARUBA ■ AUSTRALIA ■ AUSTRIA ■ AZERBAIJAN ■ BAHAMAS ■ BAHRAIN ■ BANGLADESH ■ BARBADOS ■ BELARUS ■ BELGIUM ■ BELIZE ■ BENIN ■ BERMUDA ■ BHUTAN ■ BOLIVIA ■ BONAIRE ■ BOSIA_AND_HERZEGOVINA ■ BOTSWANA ■ BOUVET_ISLAND ■ BRAZIL ■ BRITISH_INDIAN_OCEAN_TERRITORY ■ BRUNEI_DARUSSALAM ■ BULGARIA ■ BURKINAFASO ■ BURUNDI ■ CAMBODIA ■ CAMEROON	■ GHANA ■ GIBRALTER ■ GREECE ■ GREENLAND ■ GRENADA ■ GUADELOUPE ■ GUAM ■ GUATEMALA ■ GUERNSEY ■ GUINEA ■ GUINEA_BISSAU ■ GUYANA ■ HAITI ■ HEARD_AND_MCDONALD_ISLANDS ■ HONDURAS ■ HONGKONG ■ HUNGARY ■ ICELAND ■ INDIA ■ INDONESIA ■ IRAN ■ IRAQ ■ IRELAND ■ ISLEOFMAN ■ ISRAEL ■ ITALY ■ JAMAICA ■ JAPAN ■ JERSEY ■ JORDAN ■ KAZAKHSTAN ■ KENYA ■ KIRIBATI ■ KOREA_NORTH ■ KOREA_SOUTH ■ KOSOVO ■ KUWAIT ■ KYRGYZSTAN ■ LAOS	■ PAKISTAN ■ PALAU ■ PANAMA ■ PAPUA_NEW_GUINEA ■ PARAGUAY ■ PERU ■ PHILIPPINES ■ PITCAIRN_ISLAND ■ POLAND ■ PORTUGAL ■ PUERTORICO ■ QATAR ■ REPUBLIC_OF_CONGO ■ REUNION_ISLAND ■ ROMANIA ■ RUSSIAN_FEDERATION ■ RWANDA ■ SAINT_BARTHELEMY ■ SAINT_HELENA ■ SAINT_KITTS_AND_NEVIS ■ SAINT_VICENT_AND_THE_GRENADINES ■ SAINTLUCIA ■ SAINTMARTIN ■ SAMOA ■ SANMARINO ■ SAOTOME_AND_PRINCIPE ■ SAUDI_ARABIA ■ SENEGAL ■ SERBIA ■ SERBIA_AND_MONTENEGRO ■ SEYCHELLES ■ SIERRALEONE ■ SINGAPORE ■ SINT_MAARTEN ■ SLOVAK_REPUBLIC ■ SLOVENIA ■ SOLOMON_ISLANDS ■ SOMALIA ■ SOUTH_GEORGIA

Value	Value	Value
■ CANADA	■ LATVIA	■ SOUTHAFRICA
■ CANARY_ISLANDS	■ LEBANON	■ SOUTHSUDAN
■ CAPEVERDE	■ LESOTHO	■ SPAIN
■ CAYMAN_ISLANDS	■ LIBERIA	■ SRILANKA
■ CENTRAL_AFRICAN_REPUBLIC	■ LIBYA	■ ST_PIERREANDMIQUELON
■ CEUTA_AND_MELILLA	■ LIECHTENSTEIN	■ STATE_OF_PALESTINE
■ CHAD	■ LITHUANIA	■ SUDAN
■ CHILE	■ LUXEMBOURG	■ SURINAME
■ CHINA	■ MACAU	■ SVALBARD_AND_JANMAYEN_ISLANDS
■ CHRISTMAS_ISLAND	■ MACEDONIA	■ SWAZILAND
■ COCOS_ISLANDS	■ MADAGASCAR	■ SWEDEN
■ COLOMBIA	■ MALAWI	■ SWITZERLAND
■ COMOROS	■ MALAYSIA	■ SYRIAN_ARAB_REPUBLIC
■ COOK_ISLANDS	■ MALDIVES	■ TAIWAN
■ COSTARICA	■ MALI	■ TAJIKISTAN
■ COTE_DIVOIRE	■ MALTA	■ TANZANIA
■ CROATIA	■ MARSHALL_ISLANDS	■ THAILAND
■ CUBA	■ MARTINIQUE	■ TOGO
■ CURACAO	■ MAURITANIA	■ TOKELAU
■ CYPRUS	■ MAURITIUS	■ TONGA
■ CZECH_REPUBLIC	■ MAYOTTE	■ TRINIDADANDTOBAGO
■ DEMOCRATIC_REPUBLIC_OF_CONGO	■ MEXICO	■ TUNISIA
■ DENMARK	■ MICRONESIA	■ TURKEY
■ DJIBOUTI	■ MOLDOVA	■ TURKMENISTAN
■ DOMINICA	■ MONACO	■ TURKSAND_CAICOS_ISLANDS
■ DOMINICAN_REPUBLIC	■ MONGOLIA	■ TUVALU
■ EASTTIMOR	■ MONTENEGRO	■ UGANDA
■ ECUADOR	■ MONTSERRAT	■ UKRAINE
■ EGYPT	■ MOROCCO	■ UNITED_ARAB_EMIRATES
■ ELSALVADOR	■ MOZAMBIQUE	■ UNITED_KINGDOM
■ EQUATORIAL_GUINEA	■ MYANMAR	■ UNITEDSTATES
■ ERITREA	■ NAMIBIA	■ URUGUAY
■ ESTONIA	■ NAURU	■ US_MINOR_OUTLYING_ISLANDS
■ ETHIOPIA	■ NEPAL	■ UZBEKISTAN
■ FALKLAND_ISLANDS	■ NETHERLANDS	■ VANUATU
■ FAROE_ISLANDS	■ NETHERLANDS_ANTILLES	■ VATICAN
■ FIJI	■ NEWCALEDONIA	■ VENEZUELA
■ FINLAND	■ NEWZEALAND	■ VIETNAM
■ FRANCE	■ NICARAGUA	■ VIRGINISLANDS_UK
■ FRENCH_POLYNESIA	■ NIGER	■ VIRGINISLANDS_USA
■ FRENCH_SOUTHERN_TERRITORIES	■ NIGERIA	■ WALLIS_AND_FUTUNA
■ FRENCHGUIANA	■ NIUE	■ WESTERN_SAHARA
■ GABON	■ NORFOLKISLAND	■ YEMEN
■ GAMBIA	■ NORTHERN_MARIANA_ISLANDS	■ ZAMBIA
■ GEORGIA	■ NORWAY	■ ZIMBABWE
■ GERMANY	■ OMAN	

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var pnParser = format.getPhoneNumberParser({
4     defaultCountry: format.Country.CZECH_REPUBLIC
5 })
6 ...
7 // Add additional code
```

format.getCurrencyFormatter(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a format.CurrencyFormatter object to format numbers into currency strings.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/format/i18n Module
Methods and Properties	N/format/i18n Module Members
Since	2019.2

Parameters



Note: The options parameter is a JavaScript object.



Warning: These parameters are mutually exclusive, you can always use only one of them.

Parameter	Type	Required / Optional	Description	Since
options.currency	string	required if options.locale is not specified	Code of the currency that is used by formatter. Use the format.Currency enum to set this value.	2019.2
options.locale	string	required if options.currency is not specified	Code of the locale that is used by formatter.	2021.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```
1 // Add additional code
```

```

2  ...
3  var curFormatter = format.getCurrencyFormatter({
4      currency: "EUR"
5  });
6
7  var locale_curFormatter = format.getCurrencyFormatter({
8      locale: "IT_IT"
9  });
10 ...
11 // Add additional code

```

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	Currency parameter is missing
SSS_INVALID_CURRENCY	The currency is not valid
SSS_INVALID_TYPE_ARG	The parameter type is wrong
NEITHER_ARGUMENT_DEFINED	Both parameters are missing
MUTUALLY_EXCLUSIVE_ARGUMENTS	Multiple mutually exclusive parameters have been set as required or optional
INVALID_LOCALE	The locale is not valid

format.getNumberFormatter(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a format.NumberFormatter object to format numbers to strings.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/format/i18n Module
Methods and Properties	N/format/i18n Module Members
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.groupSeparator	string	optional	Indicates the group separator.	2019.2
options.decimalSeparator	string	optional	Indicates the decimal separator.	2019.2
options.precision	number	optional	Indicates the precision.	2019.2

Parameter	Type	Required / Optional	Description	Since
options.negativeNumberFormat	format.NegativeNumberFormat	optional	Indicates the negative number format.	2019.2
options.locale	string	optional	Indicates the locale from which default settings are determined. The other <code>format.getNumberFormatter(options)</code> parameters can override this parameter.	2021.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // no parameter given so the default number formatter object returned which can be parsed
4 var numberFormatter = format.getNumberFormatter();
5
6 var gs = numberFormatter.groupSeparator; // gs is ','
7 var ds = numberFormatter.decimalSeparator; // ds is '.'
8 var precision = numberFormatter.precision; // precision is '2'
9 var nnf = numberFormatter.negativeNumberFormat; // nnf is 'BRACKETS'
10
11 // using the options/parameters
12 var numFormatter2 = format.getNumberFormatter({
13     groupSeparator: " ",
14     decimalSeparator: ".",
15     precision: 2,
16     negativeNumberFormat: format.NegativeNumberFormat.MINUS
17 });
18 ...
19 // Add additional code

```

Errors

Error Code	Thrown If
INVALID_LOCALE	The locale parameter is not valid
SSS_INVALID_TYPE_ARG	The parameter type is wrong

format.getPhoneNumberFormatter(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a format.PhoneNumberFormatter object to format phone numbers to strings.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/format/i18n Module

Methods and Properties	N/format/i18n Module Members
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.formatType	format.PhoneNumberFormat Type	required	Phone number format type.	2020.2

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // no parameter given so the default number formatter object returned which can be parsed
4 var pnFormatter = format.getPhoneNumberFormatter();
5 var strNumber = pnFormatter.format({
6     number: phoneNumber
7 }); // strNumber is '+420 602 547 854 ext. 154'
8
9 // using the options/parameters
10 var pnFormatter = format.getPhoneNumberFormatter({
11     formatType: format.PhoneNumberFormatType.NATIONAL
12 });
13 var strNumber = pnFormatter.format({
14     number: phoneNumber
15 }); // strNumber is '(752) 410-5210 ext. 154'
16 ...
17 // Add additional code

```

Errors

Error Code	Thrown If
SSS_INVALID_FORMAT_TYPE	An invalid value is specified for the formatType option. Value must be a format.PhoneNumberFormatType value.

format.getPhoneNumberParser(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Parses a phone number from a string. Returns a format.PhoneNumberParser object.
Returns	Object

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/format/i18n Module
Methods and Properties	N/format/i18n Module Members
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options. defaultCountry	format.Country	optional	Parser point of reference. Specify this value if the phone number is not in international format!. If specified, value must be a format.Country value. If a value is not specified, the company country is used.	2020.2

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var origNumberStr = "602547854ext.154";
4 var pnParser = format.getPhoneNumberParser({
5     defaultCountry: format.Country.CZECH_REPUBLIC
6 });
7 var phoneNumber = pnParser.parse({
8     number: origNumberStr
9 });
10
11 /* phoneNumber.countryCode is '420'
12    phoneNumber.extension is '154'
13    phoneNumber.nationalNumber is '602547854'
14    phoneNumber.numberOfLeadingZeros is 1
15    phoneNumber.carrierCode is ' '
16    phoneNumber.rawInput is '602547854ext.154'
17 */
18 ...
19 // Add additional code

```

Errors

Error Code	Thrown If
SSS_INVALID_COUNTRY_ID	An invalid value is specified for the defaultCountry parameter. Value must be a format.Country value.

format.spellOut(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Spells out positive and negative number as a string in a specific language For more information, see Codes for the Representation of Names of Languages .
Returns	String
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/format/i18n Module
Methods and Properties	N/format/i18n Module Members
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.number	number	required	The number to be spelled out in a string.	2019.1
options.locale	string	required	The language code that specifies the string's language. ISO 639-1 alpha-2 language codes are supported. The language specified in this parameter is not related to the language specified for a NetSuite account. You can specify any language for this parameter; you do not have to specify a NetSuite supported language. For more information, see Codes for the Representation of Names of Languages.	2019.1

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var spellOut = format.spellOut({
4   number: 12345,
5   locale: "DE"
6 })

```

```

6     });
7     // spellOut is 'zwölftausenddreihundertfünfundvierzig'
8     ...
9     // Add additional code

```

format.NegativeNumberFormat

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the negative number format. Use this enum to set the <code>options.negativeNumberFormatter</code> parameter of the format.getNumberFormatter(options) .
Module	N/format/i18n Module
Sibling Module Members	N/format/i18n Module Members
Since	2019.2

Values

Value
BRACKETS
MINUS

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numFormatterM = format.getNumberFormatter({
4     groupSeparator: " ", decimalSeparator: ",",
5     precision: 2,
6     negativeNumberFormat: format.NegativeNumberFormat.MINUS
7 });
8 var numFormatterB = format.getNumberFormatter({
9     groupSeparator: " ",
10    decimalSeparator: ",",
11    precision: 2,
12    negativeNumberFormat: format.NegativeNumberFormat.BRACKETS
13 });
14 ...
15 // Add additional code

```

format.Currency

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the currency code.
Module	N/format/i18n Module
Sibling Module Members	N/format/i18n Module Members
Since	2019.1

Values

Currency values depend on the company.

Value
AED
CAD
CZK
EEK
EUR
GBP
IDR
INR
JPY
USD
UYU

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/format/i18n Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var curFormatter = format.getCurrencyFormatter({
4     currency: format.Currency.USD
5 });
6 ...// Add additional code
```

N/http Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/http module to make HTTP calls from server or client scripts. For client scripts, this module also provides the ability to make cross-domain HTTP requests using NetSuite servers as proxies.

All HTTP content types are supported.

[Go To Samples](#)

Note: The N/http module does not accept the HTTPS protocol. Use the [N/https Module](#) for that purpose.

In This Help Topic

- [HTTP Header Information](#)
- [N/http Module Members](#)
- [ClientResponse Object Members](#)
- [ServerRequest Object Members](#)
- [ServerResponse Object Members](#)

HTTP Header Information

HTTP headers can be used to pass additional information with an HTTP request or response. Each HTTP header consists of its case-insensitive name followed by a colon (:), then by its value (without line breaks). If you use custom headers, make sure the names of these headers do not contain underscores. For a general list of all HTTP headers, visit <http://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>.

Some headers are not supported in NetSuite and are blocked. These are listed below as either general HTTP headers or Suitelet response headers.

General Blocked HTTP Headers

Be aware that certain headers cannot be set manually when using the N/http module methods. If a script attempts to set values for any of the following headers, the values are discarded. These headers are listed in the following table.

■ Connection	■ Trailer
■ Content-Length	■ Transfer-Encoding
■ Host	■ Upgrade
■ JSESSIONID	■ Via

Suitelet Response HTTP Header Blocklist

In addition to the headers described in [General Blocked HTTP Headers](#), certain headers cannot be set manually when interacting with the [http.ServerResponse](#) Objects sent by Suitelets. If a script attempts to

set values for any of these headers, the system throws an SSS_INVALID_HEADER error. These headers are listed in the following table.

- Allow - Content-Location - Content-MD5 - Content-Range - Date	- Location - Proxy-Authenticate - Public-Key-Pins - Public-Key-Pins-Report-Only - Retry-After	- Server - Strict-Transport-Security - Upgrade-Insecure-Requests - Warning - WWW-Authenticate
---	---	---

N/http Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	http.ClientResponse	Object (read-only)	Server scripts	The response from the server to an HTTP client request (for example, http.get(options)).
	http.ServerRequest	Object (read-only)	Server scripts	HTTP request information sent to an HTTP server. For example, a request sent to/received by a Suitelet or RESTlet.
	http.ServerResponse	Object	Server scripts	The response from an HTTP server to an HTTP request. For example, a response from a Suitelet or RESTlet.
Method	http.delete(options)	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP DELETE request and returns the response.
	http.delete.promise(options)	Promise Object	Client and server scripts	Sends an HTTP DELETE request asynchronously and returns the response.
	http.get(options)	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP GET request and returns the response.
	http.get.promise(options)	Promise Object	Client and server scripts	Sends an HTTP GET request asynchronously and returns the response.
	http.post(options)	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP POST request and returns the response.
	http.post.promise(options)	Promise Object	Client and server scripts	Sends an HTTP POST request asynchronously and returns the response.
	http.put(options)	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP PUT request and returns the response.
	http.put.promise(options)	Promise Object	Client and server scripts	Sends an HTTP PUT request asynchronously and returns the response.
	http.request(options)	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP request and returns the response.
	http.request.promise(options)	Promise Object	Client and server scripts	Sends an HTTP request asynchronously and returns the response.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Enum	http.CacheDuration	enum	Server scripts	Holds the string values for supported cache durations. Use this enum to set the value of the type parameter in ServerResponse.setCdnCacheable(options) .
	http.Method	enum	Server scripts	Holds the string values for supported HTTP requests. Use this enum to set the value of method parameter in http.request(options) .
	http.RedirectType	enum	Server scripts	Holds the string values for supported NetSuite resources that you can redirect to. Use this enum to set the value of the type parameter for ServerResponse.sendRedirect(options) .

ClientResponse Object Members

The following members are called on the [http.ClientResponse](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ClientResponse.body	string (read-only)	Server scripts	The client response body.
	ClientResponse.code	number (read-only)	Server scripts	The client response code.
	ClientResponse.headers	Object (read-only)	Server scripts	The client response headers.

ServerRequest Object Members

The following members are called on the [http.ServerRequest](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ServerRequest.getLineCount(options)	number	Server scripts	Returns the number of lines in a sublist.
	ServerRequest.getSublistValue(options)	string	Server scripts	Returns the value of a sublist line item.
Property	ServerRequest.body	string (read-only)	Server scripts	The server request body.
	ServerRequest.files	Object (read-only)	Server scripts	The server request files.
	ServerRequest.headers	Object (read-only)	Server scripts	The server request headers.
	ServerRequest.clientIpAddress	String (read-only)	Server scripts	The remote client IP address.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	ServerRequest.method	http.Method	Server scripts	The server request HTTP method.
	ServerRequest.parameters	Object (read-only)	Server scripts	The server request parameters.
	ServerRequest.url	string (read-only)	Server scripts	The server request URL.

ServerResponse Object Members

The following members are called on the [http.ServerResponse](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ServerResponse.addHeader(options)	void	Server scripts	Adds a header to the response.
	ServerResponse.getHeader(options)	string string[]	Server scripts	Returns the value of a response header.
	ServerResponse.renderPdf(options)	void	Server scripts	Generates and renders a PDF directly to the response.
	ServerResponse.sendRedirect(options)	void	Server scripts	Sets the redirect URL by resolving to a NetSuite resource.
	ServerResponse.setCdnCacheable(options)	void	Server scripts	Sets CDN caching for a period of time.
	ServerResponse.setHeader(options)	void	Server scripts	Sets the value of a response header.
	ServerResponse.write(options)	void	Server scripts	Writes information (text, xml, html) to the response.
	ServerResponse.writeFile(options)	void	Server scripts	Writes a file to the response.
	ServerResponse.writeLine(options)	void	Server scripts	Writes line information (text, xml, html) to the response.
	ServerResponse.writePage(options)	void	Server scripts	Generates a page.
Property	ServerResponse.headers	Object (read-only)	Server scripts	The server response headers.

N/http Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/http module:

- [Request a URL](#)
- [Redirect a record](#)

Request a URL

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample shows how to use an HTTP GET request for a URL.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```
1  /**
2   * @NApiVersion 2.1
3   */
4
5   // This script uses an HTTP GET request for a URL.
6   require(['N/http'], (http)=> {
7     function sendGetRequest() {
8       let response = http.get({
9         url: 'http://www.google.com'
10      });
11    }
12    sendGetRequest();
13  });
```

Redirect a record

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample shows how to use a Suitelet to redirect to a new sales order record and set the entity field (which represents the customer).



Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).



Important: The value used in this sample for the entity field is a placeholder. Before using this sample, replace the entity field value with a valid value from your NetSuite account. If you run a script with an invalid value, an error may occur.

```
1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
```

```

4  */
5
6  // This script redirects a new sales order record and sets the entity.
7  define(['N/record', 'N/http'], (record, http)=> {
8      function onRequest(context) {
9          context.response.sendRedirect({
10             type: http.RedirectType.RECORD,
11             identifier: record.Type.SALES_ORDER,
12             parameters: ({
13                 entity: 6
14             })
15         });
16     }
17     return {
18         onRequest: onRequest
19     };
20 });

```

http.ClientResponse

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The response from the server to an HTTP request (for example, http.get(options)) from a client. This is the return type for http.delete(options), http.get(options), http.post(options), http.put(options), http.request(options), and corresponding promise methods) when those methods are called from a client. This object is read-only.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/http Module
Methods and Properties	ClientResponse Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var clientResponse = http.get({
4      url: 'http://www.google.com'
5  });
6  ...
7  // Add additional code

```

ClientResponse.body

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The client response body.
-----------------------------	---------------------------

Type	string (read-only)
Module	N/http Module
Parent Object	http.ClientResponse
Sibling Object Members	ClientResponse Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var response = http.get({
4     url: 'http://www.google.com'
5 });
6 log.debug({
7     title: 'Client Response Body',
8     details: response.body
9 });
10 ...
11 // Add additional code

```

ClientResponse.code

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The client HTTP response or status code.
Type	number (read-only)
Module	N/http Module
Parent Object	http.ClientResponse
Sibling Object Members	ClientResponse Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var response = http.get({
4   url: 'http://www.google.com'
5 });
6 log.debug({
7   title: 'Client Response Code',
8   details: response.code
9 });
10 ...
11 // Add additional code
```

ClientResponse.headers

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The response header or headers. For more information, see HTTP Header Information .
Type	Object.<String, String[]> // Client SuiteScript Object.<String, String> // Server SuiteScript
Module	N/http Module
Parent Object	http.ClientResponse
Sibling Object Members	ClientResponse Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var response = https.get({
4   url: 'https://www.testwebsite.com'
5 });
6 log.debug({
7   title: 'Client Response Header',
```

```

8 |         details: response.headers
9 |     });
10 |     ...
11 |     // Add additional code

```

HTTP allows multiple response headers with the same name. Therefore, in the browser (client SuiteScript), a header value is a list of strings. On the server, however, for legacy reasons, a header value is only a string. If a response contains multiple headers with the same name, only the first one can be retrieved.

```

1 | // Client SuiteScript
2 | >>> response.headers["cache-control"]
3 | [ "no-cache", "no-store" ]
4 |
5 | // Server SuiteScript
6 | >>> response.headers["cache-control"]
7 | "no-cache"

```

HTTP headers are defined as **case insensitive**. It is best to retrieve specific headers in lower-case. For example, `response.headers["content-type"]`..

For compatibility reasons, each header is repeated in two to three cases. For example:

```

1 | "content-type" // lower-case
2 | "Content-Type" // Title-Case
3 | "CONTENT-type" // original case (if it differs from the previous two)

```

http.ServerRequest

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The HTTP request information set to an HTTP server. For example, a request received by a Suitelet or RESTlet. This object is read-only.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/http Module
Methods and Properties	ServerRequest Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | serverRequest.getLineCount({
4 |     group: 'sublistId'
5 | });

```

```

6 | ...
7 | // Add additional code

```

ServerRequest.getLineCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of lines in a sublist.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The sublist internal ID.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.group parameter is not specified.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | serverRequest.getLineCount({
4 |     group: 'sublistId'
5 | });
6 | ...
7 | // Add additional code

```

ServerRequest.getSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist line item.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The sublist internal ID.	2015.2
options.line	string	required	The sublist line number. Note: Sublist index starts at 0.	2015.2
options.name	string	required	The sublist line item ID (name of the field).	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.group or options.line parameter is not specified.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverRequest.getSublistValue({
4   group: 'item',
5   name: 'amount',

```

```

6   line: '2'
7   });
8   ...
9   // Add additional code

```

ServerRequest.body

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request body.
Type	string (read-only)
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug({
4   title: 'Server Request Body',
5   details: request.body
6 });
7 ...
8 // Add additional code

```

ServerRequest.files

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request files.
Type	Object (read-only)
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code samples show the syntax for this member. They are not functional examples. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug({
4     title: 'Server Request Files',
5     details: request.files
6 });
7 ...
8 // Add additional code

```

```

1 var file = request.files['file_id'];

```

ServerRequest.headers

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The header information included in the http call. This object represents a series of name:value pairs. Each pair represents a request header name and its value. Typically, this object encapsulates two iterations of each header name: one in lower case and another in title case. This behavior is designed so that you can use either lower case or title case when you reference a header. However, the existence of title-case iterations of header names is not guaranteed. For best results, refer to header names using all lower-case letters (and hyphens, when applicable). If you use custom headers, make sure the names of these headers do not contain underscores. Authorization header is reserved for OAuth-authenticated requests. To implement a custom authentication layer, use a different header name; for example X-Authorization. Important: The request headers and their values are subject to change. If you use these headers in your scripts, you are responsible for testing them to make sure that they contain the information you need. For example, when making an HTTP call to a Suitelet, some headers might be filtered out. Filtering can occur if the headers affect how NetSuite processes the request internally. These filtered headers are not available to the Suitelet, so you should test to see whether a header was filtered out. If so, use a different header instead. For more information, see HTTP Header Information.
Type	Object (read-only)
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members

Since	2015.2
-------	--------

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug({
4     title: 'Server Request Headers',
5     details: request.headers
6 });
7 ...
8 // Add additional code

```

ServerRequest.clientIpAddress

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The remote client IP address.
Type	string (read-only)
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 function onRequest(context){

```

```

4   var request = context.request;
5   var header = request.headers;
6   var remoteAddress = request.clientIpAddress;
7   }
8   ...
9   // Add additional code

```

ServerRequest.method

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request HTTP method.
Type	http.Method (read-only)
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1   // Add additional code
2   ...
3   log.debug({
4       title: 'Server Request Method',
5       details: request.method
6   });
7   ...
8   // Add additional code

```

ServerRequest.parameters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Important: Ensure that parameters are properly validated to avoid cross-site scripting (XSS) injections. Do not use <script> tags in your parameters.

Property Description	The server request parameters, as name:value pairs. Note that if the server request is a Get request, parameters are sent as part of the URL. If the server request is a Post request, parameters are sent within the request body.
-----------------------------	---

	 Note: Parameters cannot be arrays. Use JSON.stringify/JSON.parse instead to handle arrays.
Type	Object (read-only)
Module	N/http Module
Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // example from a Suitelet
4
5 onRequest: function(context) {
6     // in the following, context.request is an http.ServerRequest (per Suitelet onRequest(params.request) entry point)
7
8     if (context.request.method === 'GET') {
9         // request is coming from a User Event script, for instance, on a form load; parameters are in the URL as name:value pairs (as the query string)
10
11         var myName = context.request.parameters.custpage_nameParam; // here, 'custpage_nameParam' and 'custpage_phoneParam' are
12         // parameter names set
13         var myPhone = context.request.parameters.custpage_phoneParam; // by the requesting script and sent in the HTTP request
14     }
15     if (context.request.method === 'POST'){
16         // request is coming from a Submit button click on the form, submitting the form (using a user event script); parameters are in the form fields
17         // fields
18         var myName = context.request.parameters.nameFld; // here, 'nameFld' and 'phoneFld' are field ids on the form for the Name and Phone
19         var myPhone = context.request.parameters.phoneFld;
20     }
21     ...
22 // Add additional code

```

ServerRequest.url

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request URL.
Type	string (read-only)
Module	N/http Module

Parent Object	http.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 log.debug({
2   title: 'Server Request URL',
3   details: request.url
4 });
5 ...
6 // Add additional code

```

http.ServerResponse

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The response from an HTTP server (for example, Suitelet or RESTlet) to an HTTP request from a server, such as a user event script.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/http Module
Methods and Properties	ServerResponse Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.addHeader({
4   name: 'Accept-Language',
5   value: 'en-us',
6 });
7 ...
8 // Add additional code

```

ServerResponse.addHeader(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a header to the response. If the same header has already been set, this method adds another line for that header. For example: <pre>1 {Vary: ['Accept-Language', 'Accept-Encoding']}</pre> For more information, see HTTP Header Information.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the header.	2015.2
options.value	string	required	The value used to set the header.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HEADER	One or more headers are not valid.	The header name or value is invalid.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.name or options.value parameter is not specified.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```
1 // Add additional code
2 ...
3 serverResponse.addHeader({
```

```

4     name: 'Accept-Language',
5     value: 'en-us',
6   });
7   ...
8   // Add additional code

```

ServerResponse.getHeader(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value or values of a response header. If multiple values are assigned to the header name, the values are returned as an Array. For more information, see HTTP Header Information .
Returns	string string[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the header.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.name parameter is not specified.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.getHeader({
4   name: 'Accept-Language'
5 });
6 ...
7 // Add additional code

```

ServerResponse.renderPdf(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Generates and renders a PDF directly to the response.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.xmlString	string	required	Content of the PDF	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.xmlString parameter is not specified.

Syntax

Important: The following code shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

The following sample shows how to render a PDF.

Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/xml'], function(xml) {
6      return {
7          onRequest: function(context) {
8              var xml = "<?xml version='1.0' encoding='UTF-8'>\n" +

```



```

 9      "<!DOCTYPE pdf PUBLIC \"-//big.faceless.org//report\" \"report-1.1.dtd\">\n" +
10      "<pdf lang=\"ru-RU\" xml:lang=\"ru-RU\">\n" +
11      "<head>\n" +
12      "  <link name=\"russianfont\" type=\"font\" subtype=\"opentype\" \" + "src=\"NetSuiteFonts/verdana.ttf\" \" + "src-
bold=\"NetSuiteFonts/verdanab.ttf\" \" + "src-italic=\"NetSuiteFonts/verdanai.ttf\" \" + "src-bolditalic=\"NetSuiteFonts/verdan
abi.ttf\" \" + "bytes=\"2\"/>\n" +
13      "</head>\n" +
14      "<body font-family=\"russianfont\" font-size=\"18\">\nРусский текст</body>\n" +
15      "</pdf>";
16      context.response.renderPdf(xml);
17    }
18  }
19  });

```

ServerResponse.sendRedirect(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the redirect URL by resolving to a NetSuite resource. For example, you could use this method to redirect to a new sales order page for a particular entity.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.identifier	number string	required	The primary ID for this resource. The value you use varies depending on the value of options.type, as follows: MEDIA_ITEM — Use the internal ID of a file stored in the NetSuite File Cabinet. RECORD — Use record.Type to identify the appropriate record type. RESTLET — Use the script ID from the script record of the appropriate RESTlet. SUITELET — Use the script ID from the script record of the appropriate Suitelet.	2015.2

Parameter	Type	Required / Optional	Description	Since
			TASK_LINK — Use the appropriate Task ID. Supported IDs are listed in Task IDs.	
options.type	http.RedirectType	required	The type of resource redirected to.	2015.2
options.editMode	boolean	optional	Applicable when redirecting to a record resource. Specifies whether to return a URL for a record in edit mode or view mode. If set to true , returns the record in edit mode. If set to false , returns the record in view mode. The default value is false.	2015.2
options.id	number string	optional	The secondary ID for this resource. If the options.type parameter is set to SUITELET or RESTLET, use the deployment ID. If the options.type parameter is set to RECORD, you can use the internal ID of a specific record instance.	2015.2
options.parameters	Object	optional	Additional URL parameters as name:value pairs.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_RECORD_TYPE	Type argument {type} is not a valid record or is not available in your account. Please see the documentation for a list of supported record types.	The redirect type is set to record, and an invalid record type is input for options.identifier.
SSS_INVALID_SCRIPT_ID_1	You have provided an invalid script id or internal id: {id}	The type is set to Suitelet or RESTlet, and an invalid script ID or invalid deployment ID is input for options.identifier or options.id.
SSS_INVALID_TASK_ID	The task ID: {id} is not valid. Please refer to the documentation for a list of supported task IDs.	The type is set to task link, and an invalid task ID is input for options.identifier.
SSS_INVALID_URL_CATEGORY	The options.type: {type} is not valid. Please use http.RedirectType for supported types.	The script uses an unrecognizable string value for the options.type parameter. To avoid this error, use http.RedirectType .
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.identifier or options.type parameter is not specified. Note that this error is thrown if an enum is misspelled within a script. For example, you see this error if you use http.RedirectType.TASKLINK instead of http.RedirectType.TASK_LINK in the options.type field.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 myServerResponseObj.sendRedirect({
4     type: http.RedirectType.RECORD,
5     identifier: record.Type.SALES_ORDER,
6     parameters: {entity: 8}
7 });
8 ...
9 // Add additional code

```

ServerResponse.setCdnCacheable(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets CDN caching for a period of time.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	enum	required	The value of the caching duration. Use http.CacheDuration to set this value. When used with a Suitelet, if this value is set to UNIQUE, then the Suitelet will never be cached and will always be executed if there's a request to its URL.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.type parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```
1 // Add additional code
2 ...
3 serverResponse.setCdnCacheable({
4     type: http.CacheDuration.MAX
5 });
6 ...
7 // Add additional code
```

ServerResponse.setHeader(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a response header. For more information, see HTTP Header Information .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the header.	2015.2
options.value	string	required	The value used to set the header.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HEADER	One or more headers are not valid.	The header name or value is invalid.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.name or options.value parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.setHeader({
4     name: 'Accept-Language',
5     value: 'en-us',
6 });
7 ...
8 // Add additional code

```

ServerResponse.write(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes information (text, xml, html) to the response. Note: This method accepts only strings. To pass in a file, you can use ServerResponse.writeFile(options) .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.output	string	required	The string being written.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.output parameter is not specified.
WRONG_PARAMETER_TYPE	{param name}	The value input for options.output is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.write({
4     output: 'Hello World'
5 });
6 ...
7 // Add additional code

```

ServerResponse.writeFile(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes a file to the response.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.file	file.File	required	A file.File Object that encapsulates the file to be written.	2015.2
options.isInline	boolean	optional	If true , the file is inline. The default value is false.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.file parameter is not specified.
WRONG_PARAMETER_TYPE	{param name}	The value input for options.file is not a file.File object.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.writeFile({
4     file: myFileObj,
5     isInline: true
6 });
7 ...
8 // Add additional code

```

ServerResponse.writeLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes line information (text, xml, html) to the response.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.output	string	required	The string being written.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.output parameter is not specified.
WRONG_PARAMETER_TYPE	{param name}	The value input for options.output is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.writeLine({
4     output: 'this is a sample string'
5 });
6 ...
7 // Add additional code

```

ServerResponse.writePage(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Generates a page.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.pageObject	serverWidget.Assistant serverWidget.Form serverWidget.List	required	A standalone page Object in the form of an assistant, form, or list.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.pageObject parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myPageObj = serverWidget.createList({
4     title: 'Simple List'
5 });
6
7 ServerResponse.writePage({
8     pageObject: myPageObj
9 });
10 ...
11 // Add additional code

```

ServerResponse.headers

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server response headers. For more information, see HTTP Header Information .
Type	Object (read-only) If multiple values are assigned to one header name, the values are returned as an array. For example: <pre>1 {Vary: ['Accept-Language', 'Accept-Encoding']}</pre>
Module	N/http Module
Parent Object	http.ServerResponse
Sibling Object Members	ServerResponse Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug({
4     title: "Server Response Headers",
5     details: ServerResponse.headers
6 });
7 ...
8 // Add additional code

```

http.get(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP GET request.  Important: This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	http.ClientResponse or http.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The HTTP URL being requested.	2015.2
options.headers	Object	optional	The HTTP headers. For more information, see HTTP Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the options.url parameter.

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.url parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete HTTP script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 var response = http.get({
8   url: 'http://www.google.com',
9   headers: headerObj
10 });
11 ...
12 // Add additional code

```

http.get.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP GET request asynchronously. Note: The parameters and errors thrown for this method are the same as those for http.get(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	http.get(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'

```

```

6  };
7  http.get.promise({
8      url: 'http://www.google.com',
9      headers: headerObj
10 })
11  .then(function(response){
12      log.debug({
13          title: 'Response',
14          details: response
15      });
16  })
17  .catch(function onRejected(reason) {
18      log.debug({
19          title: 'Invalid Get Request: ',
20          details: reason
21      });
22  })
23  ...
24  // Add additional code

```

http.delete(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP DELETE request.  Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	http.ClientResponse or http.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Note: This method does not include an options.body parameter. Post data is not required when the HTTP method is a DELETE request.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The HTTP URL being requested	2015.2
options.headers	Object	optional	The HTTP headers. For more information, see HTTP Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the <code>options.url</code> parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the <code>options.url</code> parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The <code>options.url</code> parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 var response = http.delete({
8   url: 'http://www.mytestwebsite.com',
9   headers: headerObj
10 });
11 ...
12 // Add additional code

```

http.delete.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP DELETE request asynchronously. Note: The parameters and errors thrown for this method are the same as those for http.delete(options). For more information about promises, see Promise Object.
Returns	Promise Object

Synchronous Version	http.delete(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 http.delete.promise({
8     url: 'http://www.mytestwebsite.com',
9     headers: headerObj
10 })
11 .then(function(response){
12     log.debug({
13         title: 'Response',
14         details: response
15     });
16 })
17 .catch(function onRejected(reason) {
18     log.debug({
19         title: 'Invalid Request: ',
20         details: reason
21     });
22 })
23 ...
24 // Add additional code

```

http.post(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP POST request. Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	http.ClientResponse or http.ServerResponse
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required	The POST data.  Note: Sending a file using multipart/form-data content type in the options.body parameter is not supported.	2015.2
options.url	string	required	The HTTP URL being requested. For information about resolving a URL, see N/url Module . For information about getting the URL for a script, including Suitelets, see url.resolveScript(options) .	2015.2
options.headers	Object	optional	The HTTP headers. For more information, see HTTP Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: ■ Has 63 or fewer characters ■ Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens ■ Starts with a letter ■ Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	The URL specified in the options.url parameter must be a fully qualified URL.

Error Code	Message	Thrown If
		For information about resolving a URL, see N/url Module .
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}.	The options.body or options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	This script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTP request. Please examine the script for a potential infinite recursion problem.	A script is calling back into itself recursively using an HTTP/HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 var response = http.post({
8     url: 'http://www.testwebsite.com',
9     body: 'My POST Data',
10    headers: headerObj
11 });
12
13 var myresponse_body = response.body; // see http.ClientResponse.body
14 var myresponse_code = response.code; // see http.ClientResponse.code
15 var myresponse_headers = response.headers; // see http.ClientResponse.headers
16 ...
17 // Add additional code

```

http.post.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP POST request asynchronously. Note that when a Suitelet is invoked by this method, it will wait for all the pending promises to finish. Note: The parameters and errors thrown for this method are the same as those for http.post(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	http.post(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	10 units
Module	N/http Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 http.post.promise({
8   url: 'http://www.google.com',
9   body: 'My POST Data',
10  headers: headerObj
11 })
12 .then(function(response){
13   log.debug({
14     title: 'Response',
15     details: response
16   });
17 })
18 .catch(function onRejected(reason) {
19   log.debug({
20     title: 'Invalid Request: ',
21     details: reason
22   });
23 })
24 ...
25 // Add additional code

```

http.put(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP PUT request. Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	http.ClientResponse or http.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/http Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required	The PUT data.	2015.2
options.url	string	required	The HTTP URL being requested	2015.2
options.headers	Object	optional	The HTTP headers. For more information, see HTTP Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the options.url parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.body or options.url parameter is not specified.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 var response = http.put({
8   url: 'http://www.google.com',

```

```

9      body: 'My PUT Data',
10     headers: headerObj
11   });
12   ...
13   // Add additional code

```

http.put.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP PUT request asynchronously.  Note: The parameters and errors thrown for this method are the same as those for http.put(options). For additional information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	http.put(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1  // Add additional code
2  ...
3  var headerObj = {
4    name: 'Accept-Language',
5    value: 'en-us'
6  };
7  http.put.promise({
8    url: 'http://www.google.com',
9    body: 'My PUT Data',
10   headers: headerObj
11 })
12   .then(function(response) {
13     log.debug({
14       title: 'Response',
15       details: response
16     });
17   })
18   .catch(function onRejected(reason) {
19     log.debug({
20       title: 'Invalid Request: ',
21       details: reason
22     });
23   })
24   ...
25   // Add additional code

```

http.request(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP request.\  Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	http.ClientResponse or http.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.method	enum	required	The HTTP request method. Set using http.Method .  Note: If the method is DELETE, this body data is ignored.	2015.2
options.url	string	required	The HTTP URL being requested	2015.2
options.body	string Object	optional	The POST data if the method is POST. The PUT data if the method is P UT.	—
options.headers	Object	optional	The HTTP headers. For more information, see HTTP Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid	The client and server could not negotiate the desired level of security. The connection is no longer usable.

Error Code	Message	Thrown If
	certificate was found for this host.	You may also receive this error if the domain name in the <code>options.url</code> parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the <code>options.url</code> parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The <code>options.method</code> or <code>options.url</code> parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 var response = http.request({
8     method: http.Method.GET,
9     url: 'http://www.google.com',
10    body: 'My REQUEST Data',
11    headers: headerObj
12 });
13 ...
14 // Add additional code

```

http.request.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP request asynchronously. Note: The parameters and errors thrown for this method are the same as those for http.request(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	http.request(options)
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/http Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 http.request.promise({
8   method: http.Method.GET,
9   url: 'http://www.google.com',
10  body: 'My REQUEST Data',
11  headers: headerObj
12 })
13 .then(function(response){
14   log.debug({
15     title: 'Response',
16     details: response
17   });
18 })
19 .catch(function onRejected(reason) {
20   log.debug({
21     title: 'Invalid Request: ',
22     details: reason
23   });
24 })
25 ...
26 // Add additional code

```

http.CacheDuration

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported cache durations. Use this enum to set the value of the type parameter in ServerResponse.setCdnCacheable(options) .
Module	N/http Module
Sibling Module Members	N/http Module Members .
Since	2015.2

Values

Value	Description
LONG	Conceptually, this corresponds to days.
MEDIUM	Conceptually, this corresponds to hours.
SHORT	Conceptually, this corresponds to minutes.
UNIQUE	If UNIQUE is used when working with a Suitelet, then the Suitelet will never be cached and will always be executed if there's a request to its URL.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```

1 // Add additional code
2 ...
3 ServerResponse.setCdnCacheable({
4     type: http.CacheDuration.MEDIUM
5 });
6 ...
7 // Add additional code

```

http.Method

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported HTTP requests. Use this enum to set the value of method parameter in http.request(options) .
Module	N/http Module
Sibling Module Members	N/http Module Members .
Since	2015.2

Values

Value
DELETE
GET
HEAD
PUT
POST

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var response = http.request({
4   method: http.Method.GET,
5   url: 'http://www.google.com'
6 });
7 ...
8 // Add additional code
```

http.RedirectType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported NetSuite resources that you can redirect to. Use this enum to set the value of the type parameter for ServerResponse.sendRedirect(options) .
Module	N/http Module
Sibling Module Members	N/http Module Members .
Since	2015.2

Values

Value
MEDIA_ITEM
RECORD
RESTLET
SUITELET
TASK_LINK

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/http Module Script Samples](#).

```
1 // Add additional code
2 ...
3 myServerResponseObj.sendRedirect({
```



```

4   type: http.RedirectType.RECORD,
5   identifier: record.Type.SALES_ORDER,
6   parameters: {entity: 6}
7   });...
8   // Add additional code

```

N/https Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/https module to manage content sent to a third party using HTTPS calls. This module encapsulates all the functionality of the [N/http Module](#), but does not allow the HTTP protocol. You can make HTTPS calls from client and server scripts.

Go To Samples 

You can use the N/https module to encode binary content or access a handle to the value in a NetSuite credential field.

You can use the N/https module to communicate between SuiteScript scripts, RESTlets, and SuiteTalk REST APIs without having to reauthenticate, using the [https.requestRestlet\(options\)](#) and [https.requestSuiteTalkRest\(options\)](#) methods.

When the N/https module is used, SuiteScript also loads the [N/crypto Module](#) and [N/encode Module](#).



Important: Use TLS 1.2 for HTTPS requests. SuiteScript 2.0 requests such [https.delete\(options\)](#), [https.get\(options\)](#), [https.post\(options\)](#), [https.put\(options\)](#), and [https.request\(options\)](#) usually go to third-party servers. Management of these servers is not within the control of your company. These HTTPS requests now fail the handshake when they attempt to connect to servers that do not support TLS 1.2. You should communicate with those who manage any third-party servers to which you connect, and ensure their servers support the TLS 1.2 protocol.



Important: NetSuite supports the same list of trusted third-party certificate authorities (CAs) as the Mozilla Included CA Certificate List.

The target endpoint, domain, or server must use one of these trusted third-party CAs, or the connection cannot be established. Oracle NetSuite requires that the endpoints that you are connecting to from NetSuite provide a full certification chain, including intermediate certificates.

For a list of certificate authorities, see https://wiki.mozilla.org/CA/Included_Certificates.



Warning: Using plain text or other unencrypted user credentials is unsafe and can pose a security threat. Whenever possible, use [Token-based Authentication \(TBA\)](#) or [OAuth 2.0](#) to specify user credentials.

In This Help Topic

- [HTTPS Header Information](#)
- [N/https Module Members](#)
- [SecureString Object Members](#)
- [ClientResponse Object Members](#)

- [ServerResponse Object Members](#)
- [ServerRequest Object Members](#)

HTTPS Header Information

HTTPS headers can be used to pass additional information with an HTTPS request or response. Each HTTPS header consists of its case-insensitive name followed by a colon (:), then by its value (without line breaks). If you use custom headers, make sure the names of these headers do not contain underscores. For a general list of all HTTP headers (also applicable to HTTPS), visit <http://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>.

Note that if you call [https.post\(options\)](#), passing in the header with a content type, NetSuite respects the following types:

- all text media types (types starting with "text/")
- "application/json"
- "application/vnd.maxmind.com-country+json"
- "application/xml"
- "application/soap+xml"
- "application/xhtml+xml"
- "application/atom+xml"

Otherwise, NetSuite overwrites the content type with the default type as if the type had not been specified. Default types are:

- "text/xml; charset=UTF-8"
- "application/x-www-form-urlencoded; charset=UTF-8"

If the body parameter is entered as an object, it is always encoded to the URL, and the headers are changed to "application/x-www-form-urlencoded; charset=UTF-8."

When you use [https.ServerResponse](#), textual responses are encoded in UTF-8 by default. If you want to return a file with an alternative character encoding, see the help topic [Return a File with Alternative Character Encoding](#).

Some headers are not supported in NetSuite and are blocked. These are listed below as either general HTTPS headers or Suitelet response headers.

General Blocked HTTPS Headers

Be aware that certain headers cannot be set manually when using N/https module methods. If a script attempts to set values for any of the following headers, the values are discarded. These headers are listed in the following table.

■ Connection	■ Trailer
■ Content-Length	■ Transfer-Encoding
■ Host	■ Upgrade
■ JSESSIONID	■ Via

Suitelet Response HTTPS Header Blocklist

In addition to the headers described in [General Blocked HTTPS Headers](#), certain headers cannot be set manually when interacting with the [https.ServerResponse](#) Objects sent by Suitelets. If a script attempts to

set values for any of these headers, the system throws an `SSS_INVALID_HEADER` error. These headers are listed in the following table.

- Allow - Content-Location - Content-MD5 - Content-Range - Date	- Location - Proxy-Authenticate - Public-Key-Pins - Public-Key-Pins-Report-Only - Retry-After	- Server - Strict-Transport-Security - Upgrade-Insecure-Requests - Warning - WWW-Authenticate
---	---	---

N/https Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	https.SecureString	Object	Server scripts	Encapsulates data that may be sent to a third-party using an HTTPS call.
	https.ClientResponse	Object (read-only)	Server scripts	Encapsulates the response to an HTTPS client request.
	https.ServerRequest	Object (read-only)	Server scripts	Encapsulates the HTTPS request information sent to an HTTPS server. For example, a request received by a Suitelet or RESTlet.
	https.ServerResponse	Object	Server scripts	Encapsulates the response from an HTTPS server to an HTTPS request. For example, a response from a Suitelet or RESTlet.
Method	https.createSecretKey(options)	Object	Server scripts	Creates a key for the contents of a credential field.
	https.createSecureString(options)	Object	Server scripts	Creates an https.SecureString Object.
	https.delete(options)	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS DELETE request and returns the response.
	https.delete.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS DELETE request asynchronously and returns the response.
	https.get(options)	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS GET request and returns the response.
	https.get.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS GET request asynchronously and returns the response.
	https.post(options)	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS POST request and returns the response.
	https.post.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS POST request asynchronously and returns the response.
	https.put(options)	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS PUT request and returns the response.
	https.put.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS PUT asynchronously request and returns the response.
	https.request(options)	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS request and returns the response.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				If a request fails, an <code>error.SuiteScriptError</code> is thrown.
	https.request.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS request asynchronously and returns the response. If a request fails, a <code>Promise.reject</code> is thrown with a parameter <code>Error</code> .
	https.requestRestlet(options)	https.ClientResponse	Server scripts	Sends an HTTPS request to a RESTlet and returns the response. Authentication headers are automatically added. The RESTlet will execute with the same privileges as the calling script.
	https.requestRestlet.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS request to a Restlet and returns the response.
	https.requestSuitelet(options)	https.ClientResponse	Client and server scripts	Sends an HTTPS request to a Suitelet and returns the response.
	https.requestSuitelet.promise(options)	Promise Object	Client and server scripts	Sends an HTTPS request asynchronously to a Suitelet and returns the response.
	https.requestSuiteTalkRest(options)	https.ClientResponse	Server scripts	Sends an HTTPS request to a SuiteTalk REST endpoint and returns the response. Authentication headers are automatically added.
Enum	https.CacheDuration	enum	Server scripts	Holds the string values for supported cache durations. Use this enum to set the value of the type parameter in ServerResponse.setCdnCacheable(options) .
	https.Encoding	enum	Server scripts	Holds the string values for supported encoding types. Use this enum to set the value of parameters in SecureString.appendString(options) , SecureString.convertEncoding(options) , https.createSecureString(options) .
	https.HashAlg	enum	Server scripts	Holds the string values for supported hashing algorithms. Use this enum to set the value of parameters in SecureString.hash(options) and SecureString.hmac(options) .
	https.Method	enum	Server scripts	Holds the string values for supported HTTPS requests. Use this enum to set the value of method parameter in https.request(options) .
	https.RedirectType	enum	Server scripts	Holds the string values for supported NetSuite resources that you can redirect to. Use this enum to set the value of the type parameter for ServerResponse.sendRedirect(options) .

SecureString Object Members

SecureString functionality is supported only in server scripts.

The following members are called on the [https.SecureString](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	SecureString.appendSecureString(options)	https.SecureString	Server scripts	Appends one https.SecureString to another https.SecureString .
	SecureString.appendString(options)	https.SecureString	Server scripts	Appends a string to a https.SecureString .
	SecureString.convertEncoding(options)	https.SecureString	Server scripts	Converts the content of a https.SecureString between two encodings.
	SecureString.hash(options)	https.SecureString	Server scripts	Creates a hash for a https.SecureString .
	SecureString.hmac(options)	https.SecureString	Server scripts	Creates an hmac for a https.SecureString .
	SecureString.replaceString(options)	https.SecureString	Server Scripts	Replaces all occurrences of a pattern string inside a https.SecureString with a replacement string.

ClientResponse Object Members

The following members are called on the [https.ClientResponse](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ClientResponse.body	string (read-only)	Server scripts	The response body.
	ClientResponse.code	number (read-only)	Server scripts	The response code.
	ClientResponse.headers	Object (read-only)	Server scripts	The response body.

ServerRequest Object Members

The following members are called on the [https.ServerRequest](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ServerRequest.getLineCount(options)	number	Server scripts	Returns the number of lines in a sublist.
	ServerRequest.getSublistValue(options)	string	Server scripts	Returns the value of a sublist line item.
Property	ServerRequest.body	string (read-only)	Server scripts	The server request body
	ServerRequest.files	Object (read-only)	Server scripts	The server request files represented as object in ID-file.File pair.
	ServerRequest.headers	Object (read-only)	Server scripts	The server request headers.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	ServerRequest.method	https.Method	Server scripts	The HTTPS method for the server request.
	ServerRequest.parameters	Object (read-only)	Server scripts	The server request parameters.
	ServerRequest.url	string (read-only)	Server scripts	The server request URL.

ServerResponse Object Members

The following members are called on the [https.ServerResponse](#) Object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ServerResponse.addHeader(options)	void	Server scripts	Adds a header to the response.
	ServerResponse.getHeader(options)	string string[]	Server scripts	Returns the value of a response header.
	ServerResponse.renderPdf(options)	void	Server scripts	Generates and renders a PDF directly to the response.
	ServerResponse.sendRedirect(options)	void	Server scripts	Sets the redirect URL by resolving to a NetSuite resource.
	ServerResponse.setCdnCacheable(options)	void	Server scripts	Sets CDN caching for a period of time.
	ServerResponse.setHeader(options)	void	Server scripts	Sets the value of a response header.
	ServerResponse.write(options)	void	Server scripts	Writes information (text/xml/html) to the response.
	ServerResponse.writeFile(options)	void	Server scripts	Writes a file to the response.
	ServerResponse.writeLine(options)	void	Server scripts	Writes line information (text/xml/html) to the response.
	ServerResponse.writePage(options)	void	Server scripts	Generates a page.
Property	ServerResponse.headers	Object (read-only)	Server scripts	The server response headers.

N/https Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/https module:

- [Generate a Secure Token and a Secret Key](#)
- [Create a Form with a Credential Field](#)
- [Create an Authentication Header Using a Secure String](#)
- [Concatenate API Secrets with Strings](#)
- [Create a JWT Token Using a SecureString](#)
- [Retrieve Employee Information Using a Suitelet and a RESTlet Script with a Defined Content-Type Header](#)
- [Call a Suitelet from a Client Script](#)

Generate a Secure Token and a Secret Key

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a GUID to generate a secure token and a secret key. To run this sample in the debugger, you must replace the GUID with one specific to your account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5   // This script uses a GUID to generate a secure token and a secret key.
6   require(['N/https', 'N/runtime'], (https, runtime) => {
7       function createSecureString() {
8           const passwordGuid = '{284CFB2D225B1D76FB94D150207E49DF}';
9           let secureToken = https.createSecureString({
10               input: passwordGuid
11           });
12           let secretKey = https.createSecretKey({
13               input: passwordGuid
14           });
15           secureToken = secureToken.hmac({
16               algorithm: https.HashAlg.SHA256,
17               key: secretKey
18           });
19       }
20       createSecureString();
21   });

```

Create a Form with a Credential Field

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to create a form with a credential field. For more information about credential fields, see [Form.addCredentialField\(options\)](#).

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Note: The default maximum length for a credential field is 32 characters. If needed, use the `Field.maxLength` property to change this value.

The values for `restrictToDomains`, `restrictToScriptIds`, and `baseUrl` in this sample are placeholders. You must replace them with valid values from your NetSuite account.

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6  // This script creates a form with a credential field.
7  define(['N/ui/serverWidget', 'N/https', 'N/url'], (serverWidget, https, url) => {
8      function onRequest(context) {
9          if (context.request.method === 'GET') {
10             const form = serverWidget.createForm({
11                 title: 'Password Form'
12             });
13
14             const credField = form.addCredentialField({
15                 id: 'password',
16                 label: 'Password',
17                 restrictToDomains: ['<accountID>.app.netsuite.com'],
18                 restrictToCurrentUser: false,
19                 restrictToScriptIds: 'customscript_my_script'
20             });
21
22             credField.maxLength = 32;
23
24             form.addSubmitButton();
25
26             context.response.writePage({
27                 pageObject: form
28             });
29         }
30         else {
31             // Request to an existing Suitelet with credentials
32             let passwordGuid = context.request.parameters.password;
33
34             // Replace SCRIPTID and DEPLOYMENTID with the internal ID of the suitelet script and deployment in your account
35             let baseUrl = url.resolveScript({
36                 scriptId: SCRIPTID,
37                 deploymentId: DEPLOYMENTID,
38                 returnExternalURL: true
39             });
40
41             let authUrl = baseUrl + '&pwd={' + passwordGuid + '}';
42
43             let secureStringUrl = https.createSecureString({
44                 input: authUrl
45             });
46
47             let headers = ({
48                 'pwd': passwordGuid
49             });
50
51             let response = https.post({
52                 credentials: [passwordGuid],

```



```

53         url: secureStringUrl,
54         body: {authorization: ' ' + passwordGuid + ' ', data: 'anything can be here'},
55         headers: headers
56     });
57 }
58 }
59 return {
60     onRequest: onRequest
61 };
62 });

```

Create an Authentication Header Using a Secure String

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to create an authentication header and send the request to a service (which requires an authentication header) using an `https.SecureString`. For more information about `SecureString`, see [https.SecureString](#).

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6   // This script creates an authentication header using an https.SecureString.
7   define(['N/https', 'N/encode'], (https, encode) => {
8       function onRequest(context) {
9
10          // Secrets with these two Script IDs must be existing and allowed for this script
11          const nameToken = "custsecret_myName";
12          const passwordToken = "custsecret_mypPassword";
13
14          // Create BASE-64 encoded name:password pair
15          const secStringKeyInBase64 = https.createSecureString({
16              input: "{" + nameToken + "}:" + passwordToken + "}"
17          });
18
19          secStringKeyInBase64.convertEncoding({
20              toEncoding: encode.Encoding.BASE_64,
21              fromEncoding: encode.Encoding.UTF_8
22          });
23
24          // Construct the Authorization header
25          const secStringBasicAuthHeader = https.createSecureString({
26              input: "Basic "
27          });
28
29          secStringBasicAuthHeader.appendSecureString({
30              secureString: secStringKeyInBase64,
31              keepEncoding: true
32          });
33
34          // Send the request to third party with the Authorization header
35          const resp = https.get({

```

```

36         url: "myUrl",
37         headers: {
38             "Authorization": secStringBasicAuthHeader
39         }
40     });
41 };
42 return {
43     onRequest: onRequest
44 };
45 });

```

Concatenate API Secrets with Strings

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to concatenate a string value to use as an API secret. API Secrets are string values that cannot be concatenated directly. In some cases, a code-generated string (for example, date stamp, account ID, sequence numbers) needs to be merged with the API secret. To merge the values, the N/https module must be imported on the script file and use the `createSecureString()` API to initialize both secret API values and ordinary string.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  // This script uses appendSecureString to concatenate strings to use as an API secret.
6  require(['N/https', 'N/runtime'], (https, runtime) => {
7      function concatToCreateSecureString() {
8          let baseUrl = https.createSecureString({
9              input: 'www.someurl.com/add?apikey='
10             });
11          let apiKey = https.createSecureString({
12              input: '{CUSTSECRET_SOME_INTEGRATION}'
13             });
14          let url = baseUrl.appendSecureString({
15              secureString: apiKey
16             });
17      }
18      concatToCreateSecureString();
19  });

```

Create a JWT Token Using a SecureString

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a JWT token using `https.SecureString`. For more information about `SecureString`, see [https.SecureString](#).

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6   // This script creates a JWT token using https.SecureString.
7   define(['/N/https', 'N/encode'], (https, encode) => {
8       function onRequest(context) {
9           let nameToken = "custsecret_myName";
10          let passwordToken = "custsecret_myPassword";
11          let headerObj = {
12              "alg": "HS256",
13              "typ": "JWT"
14          }
15          let payloadObj = {
16              "sub": "1234567890",
17              "name": "John Doe",
18              "iat": 1516239002
19          }
20
21          let headerJSON = JSON.stringify(headerObj);
22          let payloadJSON = JSON.stringify(payloadObj);
23          let headerBASE64 = encode.convert({
24              string: headerJSON,
25              inputEncoding: encode.Encoding.UTF_8,
26              outputEncoding: encode.Encoding.BASE_64_URL_SAFE
27          });
28
29          let payloadBASE64 = encode.convert({
30              string: payloadJSON,
31              inputEncoding: encode.Encoding.UTF_8,
32              outputEncoding: encode.Encoding.BASE_64_URL_SAFE
33          });
34
35          headerBASE64 = headerBASE64.replace(/=/g, ""); // remove = padding as per JWT spec 'base64UrlEncode' - URL-safe BASE-64 without
padding
36          payloadBASE64 = payloadBASE64.replace(/=/g, ""); // remove = padding as per JWT spec 'base64UrlEncode' - URL-safe BASE-64 without
padding
37
38          let secStringJwtSignature = https.createSecureString({
39              input: headerBASE64 + "." + payloadBASE64
40          })
41          .hmac({
42              algorithm: https.HashAlg.SHA256,
43              key: https.createSecretKey({
44                  secret: passwordToken,
45                  encoding: encode.Encoding.UTF_8
46              }),
47              resultEncoding: encode.Encoding.BASE_64_URL_SAFE
48          })
49          .replaceString({ // remove = padding as per JWT spec 'base64UrlEncode' - URL-safe BASE-64 without padding
50              pattern: "=",
51              replacement: ""
52          })
53
54          let secStringJwtAuthHeader = https.createSecureString({
55              input: "Bearer " + headerBASE64 + "." + payloadBASE64 + "."
56          })
57          .appendSecureString({
58              secureString: secStringJwtSignature,
59              keepEncoding: true

```

```

60     })
61
62     // Reflect the response using a echo-request suitelet
63     let resp = https.get({
64         url: "myURL",
65         headers: {
66             "Authorization": secStringJwtAuthHeader
67         }
68     });
69
70     {
71         log.debug("resp-code", resp.code);
72         log.debug("resp-body", resp.body);
73
74         let respAuth = JSON.parse(resp.body)["headers"]["Authorization"];
75
76         log.debug("resp-head-auth", respAuth);
77         log.debug("resp-head-auth-expected",
78             "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaWF0IjoxNTE2MjM5MDIyfQ.ue13RLILSJ9Q9W2Gomh8vAJQAgdbnd6TS4b7plyF0tA" ); // see https://jwt.io/#debugger-io
79     }
80 }
81 return {
82     onRequest: onRequest
83 };
84 });

```

Retrieve Employee Information Using a Suitelet and a RESTlet Script with a Defined Content-Type Header

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This sample consists of a Suitelet script calling a RESTlet script. The RESTlet result dictates the output of the Suitelet.

- [Call a RESTlet and Display Results](#)
- [Query Employee Name and Return Data in JSON Format](#)

Call a RESTlet and Display Results

The following sample is a Suitelet script calling a RESTlet script through `https.requestRestlet()`. The RESTlet returns an `application/json` value with the employee name. For more information about `https.requestRestlet()`, see [https.requestRestlet\(options\)](#).

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5   define(['N/https'],
6
7       (https) => {
8           /**

```

```

 9  * Defines the Suitelet script trigger point.
10  * @param {Object} scriptContext
11  * @param {ServerRequest} scriptContext.request - Incoming request
12  * @param {ServerResponse} scriptContext.response - Suitelet response
13  * @since 2015.2
14  */
15  const onRequest = (scriptContext) => {
16      // Get the employee ID from the parameters
17      const employeeId = scriptContext.request.parameters.employeeId || '0';
18
19      // Define output variable
20      let output = '';
21
22      // Retrieve the employee name
23      const restletResponse = https.requestRestlet({
24          body: JSON.stringify({ employeeId }),
25          deploymentId: 'customdeploy_samples_rs_requestrestlet',
26          scriptId: 'customscript_samples_rs_requestrestlet',
27          headers: { 'Content-Type': 'application/json' },
28          method: 'POST'
29      });
30
31      // Check if the RESTlet call is successful
32      if (restletResponse.code === 200) {
33          if (restletResponse.body !== '') {
34              // Parse the RESTlet response
35              const restletBody = JSON.parse(restletResponse.body);
36
37              // Write the suitelet output
38              output = `Employee Name is: ${restletBody.name}`;
39          } else {
40              // Write error message.
41              output = 'No employee found.';
42          }
43      } else {
44          // Write error message.
45          output = 'Unexpected error. Please check script logs.';
46      }
47
48      // Write output to response object.
49      scriptContext.response.write(output);
50  }
51
52  return {onRequest}
53
54  });
55

```

Query Employee Name and Return Data in JSON Format

The following RESTlet queries employee information based on the employee ID from the RESTlet request body. This RESTlet script expects the request to have a Content-Type Header of application/json. The RESTlet response is also expected to have the same Content-Type Header.

Note: This sample script uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /*
2   * Copyright (c) 2022, Oracle and/or its affiliates.
3   */
4
5  /**
6   * @NApiVersion 2.1
7   * @NScriptType Restlet
8   */
9  define(['N/scriptTypes/restlet', 'N/query'], function (restlet, query) {

```

```

10 // Restlet entry point
11 const post = function (requestBody) {
12     // Define the output variable
13     let returnValue = '';
14
15     // Create the Employee Query
16     const qObj = query.create({
17         type: query.Type.EMPLOYEE
18     });
19
20     // Check if the requestBody and the employeeId field is defined
21     if (!!requestBody && !!requestBody.employeeId) {
22         // Define the query condition for employee ID field in Employee record
23         qObj.condition = qObj.createCondition({
24             fieldId: 'id',
25             operator: query.Operator.EQUAL,
26             values: requestBody.employeeId
27         });
28
29         // Add the desired columns in the Query
30         qObj.columns = [
31             qObj.createColumn({
32                 fieldId: 'firstname'
33             }),
34             qObj.createColumn({
35                 fieldId: 'lastname'
36             })
37         ];
38
39         // Retrieve the results
40         const results = qObj.run().asMappedResults();
41         for (let i = 0; i < results.length; i++) {
42             const result = results[i];
43             returnValue = {
44                 name: result.firstname + ' ' + (result.lastname || '')
45             };
46         }
47     }
48
49     return returnValue;
50 }
51
52 return { post };
53 });

```

Call a Suitelet from a Client Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This sample shows how to use the [https.requestSuitelet\(options\)](#) method to call a Suitelet from a client-side script.

This sample consists of three scripts that perform the following actions:

- **Add a Button that Calls a Custom Function** – A user event script that adds a custom button on a form. When the button is clicked, it calls a custom function that is defined in a client script.
- **Create a Custom Function to Call a Suitelet** – A client script that defines a custom function that calls a Suitelet and displays the Suitelet response in a dialog.
- **Create a Simple Suitelet**– A simple Suitelet that shows the text "Hello World!".

Take note of the following when using the scripts in this sample:

- Ensure that you replace path to the client script referenced in the user event script.
- Ensure that you replace the script and deployment IDs of the Suitelet referenced in the client script.
- The client script and user event script must be deployed to the same form.

Add a Button that Calls a Custom Function

The following user event script adds a button to a form. When the button is clicked, it calls the `callSuitelet` function that is defined in the referenced client script. The `Form.clientScriptModulePath` property is used to specify the path to the client script where the custom function is defined. Ensure that you replace the value with the path to your client script.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   */
5
6  // This script adds a button to a form and calls a custom function defined in a client script.
7  define(['N/ui/serverWidget'],
8
9      function (serverWidget) {
10         function beforeLoad(scriptContext) {
11
12             // Add a button that calls the callSuitelet function
13             scriptContext.form.addButton({
14                 id: 'custpage_call_suitelet_button',
15                 label: 'Click to Open Suitelet',
16                 functionName: 'callSuitelet'
17             })
18
19             // Specify the path to the client script where the callSuitelet function is defined
20             scriptContext.form.clientScriptModulePath = './cs_request_suitelet.js' // Replace with the path to your client script
21         }
22
23         return {beforeLoad}
24     });

```

Create a Custom Function to Call a Suitelet

The following client script defines a `callSuitelet` function that sends a request to a Suitelet using the [https.requestSuitelet\(options\)](#) method. The response from the Suitelet is then shown in an alert dialog using the [dialog.alert\(options\)](#) method. To use this script, replace the Suitelet script and deployment IDs with your own values.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ClientScript
4   * @NModuleScope SameAccount
5   */
6
7  // This script defines a custom function to call a Suitelet.
8  define(['N/https', 'N/ui/dialog'],
9
10     function(https, dialog) {
11         function pageInit(scriptContext) {}
12
13         // Create a custom function that calls a Suitelet and displays the response in a dialog
14         function callSuitelet() {
15             const response = https.requestSuitelet({
16                 scriptId: 'customscript_sl_plf_requestsuitelet', // Replace with your Suitelet script ID

```

```
17         deploymentId: 'customdeploy_sl_plf_requestsuitelet', // Replace with your Suitelet deployment ID
18     });
19     dialog.alert({
20         title: 'Suitelet Response',
21         message: response.body
22     });
23 }
24
25 return {
26     pageInit: pageInit,
27     callSuitelet: callSuitelet
28 };
29
30 };
```

Create a Simple Suitelet

The following Suitelet returns a plain text response with the "Hello World!" message.

**Note:** This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

**Important:** This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```
1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6  // This Suitelet shows the text 'Hello World!' in plain text.
7  define([],
8
9      function() {
10         function onRequest(context) {
11             context.response.setHeader({name: 'Content-Type', value: 'text/plain'});
12             context.response.write('Hello World!');
13         }
14
15         return {onRequest};
16     }
17 );
```

https.SecureString

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates data that may be sent to a third-party using an HTTPS call, such as a fragment of sensitive data. An https.SecureString is returned by https.createSecureString(options) , SecureString.appendString(options) , SecureString.appendSecureString(options) , and SecureString.replaceString(options) . It can also be used as the options.credentials parameter in a call to https.request(options) .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Module	N/https Module
Methods and Properties	SecureString Object Members
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 function createSecureString() {
4     var passwordGuid = '{284CFB2D225B1D76FB94D150207E49DF}';
5     var secureToken = https.createSecureString({ // secureToken is an https.SecureString
6         input: passwordGuid
7     });
8 }
9 return secureToken;
10 ...
11 // Add additional code

```

For additional information about specifying user credentials, you may also want to visit the help topics: [Token-based Authentication \(TBA\)](#) and [OAuth 2.0](#).

SecureString.appendSecureString(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Appends one https.SecureString to another https.SecureString .
Returns	A converted https.SecureString (not a new SecureString).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.secureString	https.SecureString	required	The https.SecureString to append.
options.keepEncoding	boolean	optional	Keeps the appended string in its original encoding. Set this value to true to prevent unexpected content re-encoding.

Parameter	Type	Required / Optional	Description
			The default value is false.

Errors

Error Code	Thrown If
FAILED_TO_CONVERT_BINARY_DATA_TO_UTF_8	The options.keepEncoding parameter is set to false and an invalid attempt to re-encode from binary to string data occurred during the append.
FAILED_TO_DECODE_STRING_ENCODED_BINARY_DATA_USING_1_ENCODING	The options.keepEncoding parameter is set to false and an invalid attempt to re-encode from string to binary data occurred during the append.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var secureString1 = https.createSecureString({
4     input: "myString1"
5 });
6
7 var secureString2 = https.createSecureString({
8     input: "myString2"
9 });
10 secureString1.appendSecureString({
11     secureString: secureString2,
12     keepEncoding: true
13 });
14 // secureString1 will now be "myString1myString2"
15 ...
16 // Add additional code

```

SecureString.appendString(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Appends a string to an https.SecureString . You should use this method only for appending a string (UTF-8) content to an https.SecureString .
Returns	https.SecureString
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.input	string	required	The string to append. The following patterns are accepted: ■ <code>\{[a-fA-F0-9]{32}\}</code> or alternatively <code>{ + <32 hexadecimal characters forming a UUID/GUID> + }</code> ■ <code>\{custsecret[a-zA-Z0-9_]{1,89}\}</code> or alternatively <code>{ + custsecret + <1 up to 89 alphanumeric characters or underscores> + }</code> ■ specific payment-plugin tokens
options.inputEncoding	string	optional	The encoding of the string that is being appended. Use values from the https.Encoding enum. The default value is <code>https.Encoding.UTF_8</code>.  Note: This parameter is deprecated; you should use this method only for appending a string (UTF-8) content to a SecureString.
options.keepEncoding	boolean	optional	Keeps the appended string in its original encoding. Set this value to true to prevent unexpected content re-encoding. The default value is false.

Errors

Error Code	Thrown If
FAILED_TO_CONVERT_BINARY_DATA_TO_UTF_8	The options.keepEncoding parameter is set to false and an invalid attempt to re-encode from binary to string data occurred during the append.
FAILED_TO_DECODE_STRING_ENCODED_BINARY_DATA_USING_1_ENCODING	The options.keepEncoding parameter is set to false and an invalid attempt to re-encode from string to binary data occurred during the append.
INVALID_VALUE_1_FOR_PARAMETER_2	The options.InputEncoding parameter is set to a value other than <code>https.Encoding.UTF_8</code> and the options.keepEncoding parameter is set to true.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var regularString1 = "myString1";
4
5 var secureString1 = https.createSecureString({
6     input: "myString2"
7 });

```

```

7  });
8  secureString1.appendString({
9      input: regularString1,
10     keepEncoding: true
11  });
12  // secureString1 will now be "myString1myString2"
13  ...
14  // Add additional code

```

SecureString.convertEncoding(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Converts the content of a https.SecureString between two encodings.
Returns	The converted https.SecureString (not a new SecureString).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fromEncoding	string	required	The encoding to be used to decode the current content of the SecureString. Use values from https.Encoding .
options.toEncoding	string	required	The encoding to store the content as. Use values from https.Encoding .

Errors

Error Code	Thrown If
FAILED_TO_CONVERT_BINARY_DATA_TO_UTF_8	The content of the SecureString is binary data and it does not represent a valid UTF-8-encoded string.
FAILED_TO_DECODE_STRING_ENCODED_BINARY_DATA_USING_1_ENCODING	The content of the SecureString is not a valid encoded string according to the options.fromEncoding parameter value.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
```

```

2  ...
3  var secureString1 = https.createSecureString({
4      input: "myString1"
5  });
6  log.debug("secureString1 before convertEncoding", secureString1);
7
8  secureString1.convertEncoding({
9      fromEncoding: https.Encoding.UTF_8,
10     toEncoding: https.Encoding.HEX
11 });
12 log.debug("secureString1 after convertEncoding", secureString1);
13 ...
14 // Add additional code

```

SecureString.hash(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a hash for a https.SecureString . You can optionally specify the encoding used to convert the SecureString content to and the encoding used to encode the result as a string. Use https.HashAlg to set the hash algorithm.
Returns	https.SecureString
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.algorithm	string	required	The hash algorithm. Use values from the https.HashAlg enum.
options.contentEncoding	string	optional	Encoding used to convert/decode the current string content into binary data for hashing. Use values from the https.Encoding enum. Defaults to current internal encoding of the SecureString. Although a default is provided, we highly encourage the use of this parameter for the most secure and correct encoding.
options.resultEncoding	string	optional	Encoding used encode the binary result as a string. Use values from the https.Encoding enum. Defaults to HEX when options.contentEncoding is specified, otherwise defaults to current internal encoding of the SecureString. Although a default is provided, we highly encourage the use of this parameter for the most secure and correct encoding.

Errors

Error Code	Thrown If
FAILED_TO_CONVERT_BINARY_DATA_TO_UTF_8	An invalid attempt was made to encode the binary result to UTF-8 string.
FAILED_TO_DECODE_STRING_ENCODED_BINARY_DATA_USING_1_ENCODING	The content of the SecureString is not a valid encoded string according to the options.contentEncoding parameter value.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var secureString1 = https.createSecureString({
4     input: "myString1"
5 });
6 var hashString1 = secureString1.hash({
7     algorithm: https.HashAlg.SHA256
8 });
9 ...
10 // Add additional code

```

SecureString.hmac(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an hmac for a https.SecureString using a specified hash algorithm and secret key. You can optionally specify the encoding used to convert the SecureString content to and the encoding used to encode the result as a string. Use https.HashAlg to set the hash algorithm.
Returns	https.SecureString
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.algorithm	string	required	The hash algorithm. Use values from the https.HashAlg enum.
options.key	crypto.SecretKey https.SecureString	required	A key returned from https.createSecretKey(options) .

Parameter	Type	Required / Optional	Description
options.contentEncoding	string	optional	Encoding used to convert/decode the current string content into binary data for hmac processing. Use values from the https.Encoding enum. Defaults to current internal encoding of the SecureString. Although a default is provided, we highly encourage the use of this parameter for the most secure and correct encoding.
options.resultEncoding	string	optional	Encoding used to encode the binary result as a string. Use values from the https.Encoding enum. Defaults to HEX when options.contentEncoding is specified, otherwise defaults to current internal encoding of the SecureString. Although a default is provided, we highly encourage the use of this parameter for the most secure and correct encoding.

Errors

Error Code	Thrown If
FAILED_TO_CONVERT_BINARY_DATA_TO_UTF_8	An invalid attempt was made to encode the binary result to UTF-8 string.
FAILED_TO_DECODE_STRING_ENCODED_BINARY_DATA_USING_1_ENCODING	The content of the SecureString is not a valid encoded string according to the options.contentEncoding parameter value.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var secureString1 = https.createSecureString({
4     input: "myString1"
5 });
6 var hmacString1 = secureString1.hmac({
7     algorithm: https.HashAlg.SHA256,
8     key: mySecretKey
9 });
10 ...
11 // Add additional code

```

SecureString.replaceString(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Replaces all occurrences of a pattern string inside an https.SecureString with a replacement string.
---------------------------	--

	 Tip: If you are using an https.SecureString as a URL in https.put(options) or https.post(options) calls, you can use this method to escape any special characters you are using in the URL. For instance, you can replace the '#' special character with '%23' to allow for the use of '#' in a SecureString used as a URL.
Returns	A converted https.SecureString (not a new SecureString).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2021.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
options.pattern	string	required	The string to be replaced.
options.replacement	string	required	The replacement string.

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/https Module Script Samples .
--

```

1 // Add additional code
2 ...
3 var secureString1 = https.createSecureString({
4     input: "myFirstSecureString"
5 });
6
7 var originalString = "First";
8 var newString = "Only"
9
10 secureString1.replaceString({
11     pattern: originalString,
12     replacement: newString
13 });
14 ...
15 // Add additional code

```

https.ClientResponse

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the response to an HTTPS client request. This object is read-only.
---------------------------	--

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/https Module
Methods and Properties	ClientResponse Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var clientResponse = https.get({
4   url: 'https://www.testwebsite.com'
5 });
6 ...
7 // Add additional code

```

ClientResponse.body

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The client response body.
Type	string (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var response = https.get({
4   url: 'https://www.testwebsite.com'
5 });
6 log.debug({
7   title: 'Client Response Body',
8   details: response.body

```

```

 9  });
10  ...
11  // Add additional code

```

ClientResponse.code

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The client HTTP response or status code.
Type	number (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var response = https.get({
4      url: 'https://www.testwebsite.com'
5  });
6  log.debug({
7      title: 'Client Response Code',
8      details: response.code
9  });
10 ...
11 // Add additional code

```

ClientResponse.headers

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The response header or headers. For more information, see HTTPS Header Information .
Type	Object.<String, String[]> // Client SuiteScript Object.<String, String> // Server SuiteScript
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var response = https.get({
4   url: 'https://www.testwebsite.com'
5 });
6 log.debug({
7   title: 'Client Response Header',
8   details: response.headers
9 });
10 ...
11 // Add additional code

```

HTTP allows multiple response headers with the same name. Therefore, in the browser (client SuiteScript), a header value is a list of strings. On the server, however, for legacy reasons, a header value is only a string. If a response contains multiple headers with the same name, only the first one can be retrieved.

```

1 // Client SuiteScript
2 >>> response.headers["cache-control"]
3 [ "no-cache", "no-store" ]
4
5 // Server SuiteScript
6 >>> response.headers["cache-control"]
7 "no-cache"

```

HTTP headers are defined as **case insensitive**. It is best to retrieve specific headers in lower-case. For example, `response.headers["content-type"]`.

For compatibility reasons, each header is repeated in two to three cases. For example:

```

1 "content-type" // lower-case
2 "Content-Type" // Title-Case
3 "CONTENT-type" // original case (if it differs from the previous two)

```

https.ServerRequest

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The HTTPS request information set to an HTTPS server. For example, a request received by a Suitelet or RESTlet. This object is read-only.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Module	N/https Module
Methods and Properties	ServerRequest Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverRequest.getLineCount({
4     group: 'sublistId'
5 });
6 ...
7 // Add additional code

```

ServerRequest.getLineCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of lines in a sublist.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The sublist internal ID.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.group parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverRequest.getLineCount({
4     group: 'sublistId'
5 });
6 ...
7 // Add additional code

```

Code Samples



Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Defines a Suitelet which generates a form. When the user fills in and sends the form, the `getLineCount()` returns the number of items in the sublist. For more info on getting sublist values, see: [ServerRequest.getSublistValue\(options\)](#).

```

1 /**
2  *@NApiVersion 2.x
3  *@NScriptType Suitelet
4  */
5 define(['N/ui/serverWidget'], function(serverWidget) {
6     function onRequest(context) {
7         var request = context.request;
8         var response = context.response;
9
10        if (request.method === 'GET')
11        {
12            var form = serverWidget.createForm({
13                title : 'Sample Form'
14            });
15
16            var sublist = form.addSublist({
17                id : 'custpage_sample_list',
18                label : 'Sample List',
19                type : serverWidget.SublistType.LIST
20            });
21            sublist.addField({
22                id : 'field1',
23                label : 'Field Name 1',
24                type : serverWidget.FieldType.CHECKBOX
25            });
26            sublist.addField({
27                id : 'field2',
28                label : 'Field Name 2',
29                type : serverWidget.FieldType.TEXT
30            });
31            sublist.setSublistValue({
32                id : 'field1',
33                line : 0,
34                value : 'F'
35            });
36            sublist.setSublistValue({
37                id : 'field2',
38                line : 0,
39                value : "value 1"

```

```
40     });
41     sublist.setSublistValue({
42         id : 'field1',
43         line : 1,
44         value : 'T'
45     });
46     sublist.setSublistValue({
47         id : 'field2',
48         line : 1,
49         value : "value 2"
50     });
51
52     form.addSubmitButton({
53         id: 'submitBtn',
54         label: 'Submit'
55     });
56
57     response.writePage(form);
58 }
59 else if (request.method === 'POST')
60 {
61     var lineCount = request.getLineCount('custpage_sample_list'); // 2
62
63     for (var i = 0; i < lineCount; i++)
64     {
65         ...
66         // Add additional code
67         ...
68     }
69 }
70 }
71
72 return {
73     onRequest: onRequest
74 };
75 });
```

ServerRequest.getSublistValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist line item.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The sublist internal ID.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.line	string	required	The sublist line number. Sublist index starts at 0.	2015.2
options.name	string	required	The name of the field.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.group, options.line, or the options.name parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverRequest.getSublistValue({
4     group: 'item',
5     name: 'amount',
6     line: '2'
7 });
8 ...
9 // Add additional code

```

Code Samples



Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Defines a Suitelet which generates a form. When the user fills in and sends the form, the `getSublistValue({` returns the specific field's value on the given line in the given sublist.

```

1 /**
2  *@NApiVersion 2.x
3  *@NScriptType Suitelet
4  */
5 define(['N/ui/serverWidget'], function(serverWidget) {
6     function onRequest(context) {
7         var request = context.request;
8         var response = context.response;
9
10        if (request.method === 'GET')
11        {
12            var form = serverWidget.createForm({
13                title : 'Sample Form'
14            });
15
16            var sublist = form.addSublist({
17                id : 'custpage_sample_list',
18                label : 'Sample List',

```

```

19         type : serverWidget.SublistType.LIST
20     });
21     sublist.addField({
22         id : 'field1',
23         label : 'Field Name 1',
24         type : serverWidget.FieldType.CHECKBOX
25     });
26     sublist.addField({
27         id : 'field2',
28         label : 'Field Name 2',
29         type : serverWidget.FieldType.TEXT
30     });
31     sublist.setSublistValue({
32         id : 'field1',
33         line : 0,
34         value : 'F'
35     });
36     sublist.setSublistValue({
37         id : 'field2',
38         line : 0,
39         value : "value 1"
40     });
41
42     form.addSubmitButton({
43         id: 'submitBtn',
44         label: 'Submit'
45     });
46
47     response.writePage(form);
48 }
49 else if (request.method === 'POST')
50 {
51     var lineCount = request.getLineCount('custpage_sample_list');
52
53     for (var i = 0; i < lineCount; i++)
54     {
55         var field1 = request.getSublistValue({
56             group : 'custpage_sample_list',
57             name : 'field1',
58             line : i
59         }); // "F"
60         var field2 = request.getSublistValue({
61             group : 'custpage_sample_list',
62             name : 'field2',
63             line : i
64         }); // "value 1"
65         ...
66         // Add additional code
67     }
68 }
69 }
70
71 return {
72     onRequest: onRequest
73 };
74 });

```

ServerRequest.body

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request body.
Type	string (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug({
4     title: 'Server Request Body',
5     details: request.body
6 });
7 ...
8 // Add additional code

```

Code Samples



Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#)

```

1 /**
2  *@NApiVersion 2.x
3  *@NScriptType Suitelet
4  */
5  define([], function() {
6      function onRequest(context) {
7          var body = context.request.body; // this is a string
8          ...
9          // Add additional code
10         ...
11     }
12
13     return {
14         onRequest: onRequest
15     };
16 });

```

ServerRequest.files

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request files represented as object in ID-file.File pair.
Type	Object (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. They are not functional examples. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug({
4   title: 'Server Request Files',
5   details: request.files
6 });
7 ...
8 // Add additional code

```

```

1 | var file = request.files['file_id'];

```

Code Samples



Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#)

A sample form accepting a file as input then accessing its contents

```

1 /**
2  *@NApiVersion 2.x
3  *@NScriptType Suitelet
4  */
5 define(['/N/ui/serverWidget', 'N/file'], function(serverWidget, file) {
6   function onRequest(context) {
7     var request = context.request;
8     var response = context.response;
9
10    if (request.method === 'GET')
11    {
12      var form = serverWidget.createForm({
13        title : 'Sample Form'
14      });
15
16      form.addSubmitButton({
17        id: 'submitBtn',
18        label: 'Submit'
19      });
20
21      form.addField({
22        id : 'custpage_file_csvinput',
23        type : serverWidget.FieldType.FILE,
24        label : 'CSV File'
25      });
26
27      response.writePage(form);
28    }
29    else if (request.method === 'POST')
30    {
31      var files = request.files; // {"custpage_file_csvinput": file.File Object}

```

```

32
33     if (files)
34     {
35         var fileObj = files.custpage_file_csvinput;
36
37         var contents = fileObj.getContents()
38         ...
39         // Add additional code
40     }
41 }
42 }
43
44 return {
45     onRequest: onRequest
46 };
47 });

```

ServerRequest.headers

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	This object represents a series of name:value pairs. Each pair represents a server request header name and its value. Typically, this object encapsulates two iterations of each header name: one in lower case and another in title case. This behavior is designed so that you can use either lower case or title case when you reference a header. However, the existence of title-case iterations of header names is not guaranteed. For best results, refer to header names using all lower-case letters (and hyphens, when applicable). If you use custom headers, make sure the names of these headers do not contain underscores.  Important: The server request headers and their values are subject to change. If you use these headers in your scripts, you are responsible for testing them to make sure that they contain the information you need. For example, when making an HTTP call to a Suitelet, some headers might be filtered out. Filtering can occur if the headers affect how NetSuite processes the request internally. These filtered headers are not available to the Suitelet, so you should test to see whether a header was filtered out. If so, use a different header instead. For more information, see HTTPS Header Information.
Type	Object (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
```

```

2  ...
3  log.debug({
4      title: 'Server Request Headers',
5      details: request.headers
6  });
7  ...
8  // Add additional code

```

Code Samples

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#)

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType Suitelet
4   */
5   define([], function() {
6       function onRequest(context) {
7           var headers = context.request.headers; // {"headers":{"User-Agent":"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
8           // (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36","Accept-Encoding":"gzip"}...}
9           ...
10          // Add additional code
11          ...
12      }
13      return {
14          onRequest: onRequest
15      };
16  });

```

ServerRequest.method

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request HTTPS method.
Type	enum (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1  // Add additional code

```

```

2  ...
3  log.debug({
4      title: 'Server Request Method',
5      details: request.method
6  });
7  ...
8  // Add additional code

```

ServerRequest.parameters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Important: Ensure that parameters are properly validated to avoid cross-site scripting (XSS) injections. Do not use <script> tags in your parameters.

Property Description	The server request parameters, as name:value pairs. Note that if the server request is a Get request, parameters are sent as part of the URL. If the server request is a Post request, parameters are sent within the request body.  Note: Parameters cannot be arrays. Use JSON.stringify/JSON.parse instead to handle arrays.
Type	Object (read-only)
Module	N/https Module
Parent Object	https.ServerRequest
Sibling Object Members	ServerRequest Object Members
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1  // Add additional code
2  ...
3  // example from a Suitelet
4
5  onRequest: function(context) {
6      // in the following, context.request is an https.ServerRequest (per Suitelet onRequest(params.request) entry point)
7
8      if (context.request.method === 'GET') {
9          // request is coming from a User Event script, for instance, on a form load; parameters are in the URL as name:value pairs (as the query string)

```

```
10
11     var myName = context.request.parameters.custpage_nameParam;    // here, 'custpage_nameParam' and 'custpage_phoneParam' are
    parameter names set
12     var myPhone = context.request.parameters.custpage_phoneParam; // by the requesting script and sent in the HTTPS request
13 }
14 if (context.request.method === 'POST') {
15     // request is coming from a Submit button click on the form, submitting the form (using a user event script); parameters are in the form fields
16
17     var myName = context.request.parameters.nameFld; // here, 'nameFld' and 'phoneFld' are field ids on the form for the Name and Phone
    fields
18     var myPhone = context.request.parameters.phoneFld;
19 }
20 }
21 ...
22 // Add additional code
```

Code Samples



Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#)

```
1  /**
2   *@NApiVersion 2.x
3   *@NScriptType Suitelet
4   */
5   define([], function() {
6       function onRequest(context) {
7           var params = context.request.parameters; // {"param1":"value1"}
8           var param1 = params.param1; // value1
9           ...
10          // Add additional code
11          ...
12      }
13
14      return {
15          onRequest: onRequest
16      };
17  });
```

ServerRequest.url

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server request URL.
Type	string (read-only)
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 log.debug({
4     title: 'Server Request URL',
5     details: request.url
6 });
7 ...
8 // Add additional code
```

Code Samples



Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#)

```
1 /**
2  *@NApiVersion 2.x
3  *@NScriptType Suitelet
4  */
5 define([], function() {
6     function onRequest(context) {
7         var url = context.request.url; // https://XXXXXXXXX.app.netsuite.com/app/site/hosting/scriptlet.nl
8         ...
9         // Add additional code
10        ...
11    }
12
13    return {
14        onRequest: onRequest
15    };
16 });
```

https.ServerResponse

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The response from an HTTPS server (for example, Suitelet or RESTlet) to an HTTPS request from a server, such as a user event script. Textual files are re-encoded to UTF-8 by default. If you want to return a file with an alternative character encoding, see the help topic Return a File with Alternative Character Encoding .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/https Module
Methods and Properties	ServerResponse Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 serverResponse.addHeader({
4     name: 'Accept-Language',
5     value: 'en-us',
6 });
7 ...
8 // Add additional code
```

ServerResponse.addHeader(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a header to the response. If the same header has already been set, this method adds another line for that header. For example: <pre>1 {Vary: ['Accept-Language', 'Accept-Encoding']}</pre> For more information, see HTTPS Header Information.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/https Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the header.	2015.2
options.value	string	required	The value used to set the header.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HEADER	One or more headers are not valid.	The header name or value is invalid.

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.name or options.value parameter is not specified.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.addHeader({
4     name: 'Accept-Language',
5     value: 'en-us',
6 });
7 ...
8 // Add additional code

```

ServerResponse.getHeader(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value or values of a response header. If multiple values are assigned to the header name, the values are returned as an Array. For more information, see HTTPS Header Information .
Returns	string string[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the header.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.name parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 serverResponse.getHeader({
4   name: 'Accept-Language'
5 });
6 ...
7 // Add additional code
```

ServerResponse.renderPdf(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Generates and renders a PDF directly to the response.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.xmlString	string	required	Content of the pdf.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.xmlString parameter is not specified.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
```

```

2 ...
3 serverResponse.renderPDF({
4     xmlString: '<?xml version="1.0"?>\n<!DOCTYPE pdf PUBLIC "-//big.faceless.org//report" "report-1.1.dtd">\n<pdf>\n<body font-
5     size="18">\nHello World!\n</body>\n</pdf>'
6 });
7 ...
7 // Add additional code

```

ServerResponse.sendRedirect(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a redirect URL that resolves to a NetSuite resource. For example, you could use this method to redirect to a new sales order page for a particular entity.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Important: All parameters must be prefixed with `custparam`.

Parameter	Type	Required / Optional	Description	Since
options.identifier	number string	required	The primary ID for this resource. The value you use varies depending on the value of options.type, as follows: MEDIA_ITEM — Use the internal ID of a file stored in the NetSuite File Cabinet. RECORD — Use record.Type to identify the appropriate record type. RESTLET — Use the script ID from the script record of the appropriate RESTlet. SUITELET — Use the script ID from the script record of the appropriate Suitelet. TASK_LINK — Use the appropriate Task ID. Supported IDs are listed in Task IDs.	2015.2
options.type	string	required	The type of resource to which the script redirects. Use https.RedirectType to set a value for this parameter.	2015.2
options.editMode	boolean	optional	Applicable when redirecting to a record resource.	2015.2

Parameter	Type	Required / Optional	Description	Since
			Specifies whether to return a URL for a record in edit mode or view mode. If set to true , returns the record in edit mode. If set to false , returns the record in view mode. The default value is false.	
options.id	string	optional	The secondary ID for this resource. If the options.type parameter is set to SUITELET or RESTLET, use the deployment ID. If the options.type parameter is set to RECORD, you can use the internal ID of a specific record instance.	2015.2
options.parameters	object	optional	Additional URL parameters as name:value pairs.	2015.2

Errors

Error Code	Message	Thrown If
INVALID_ID	You have provided an invalid script id or internal id: {id}	The options.type parameter is set to RESTLET or SUITELET, and the script uses an invalid ID for options.identifier or options.id.
INVALID_RCRD_TYPE	The record type {type} is invalid.	The options.type parameter is set to RECORD, and the script uses an unrecognizable string value for options.identifier. To avoid this error, use record.Type to identify the appropriate record type.
INVALID_TASK_ID	The task ID: {id} is not valid. Please refer to the documentation for a list of supported task IDs.	The options.type parameter is set to TASK_LINK, and the script uses an invalid task ID for options.identifier. For a list of valid IDs, see the help topic Task IDs .
SSS_INVALID_URL_CATEGORY	The options.type: {type} is not valid. Please use https.RedirectType enum for supported types.	The script uses an unrecognizable string value for the options.type parameter. To avoid this error, use https.RedirectType to set the value.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.identifier or options.type parameter is not specified. Note that this error is thrown if an enum is misspelled within a script. For example, you see this error if you use http.RedirectType.TASKLINK instead of http.RedirectType.TASK_LINK in the options.type field.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 myServerResponseObj.sendRedirect({

```

```

4 |     type: https.RedirectType.RECORD,
5 |     identifier: record.Type.SALES_ORDER,
6 |     parameters: {entity: 8}
7 | });
8 | ...
9 | // Add additional code

```

ServerResponse.setCdnCacheable(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets CDN caching for a period of time.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	https.CacheDuration	required	The value of the caching duration. When used with a Suitelet, if this value is set to UNIQUE, then the Suitelet will never be cached and will always be executed if there's a request to its URL.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.type parameter is not specified.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | serverResponse.setCdnCacheable({
4 |     type: https.CacheDuration.LONG

```

```

5  });
6  ...
7  // Add additional code

```

ServerResponse.setHeader(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a response header. For more information, see HTTPS Header Information .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the header.	2015.2
options.value	string	required	The value used to set the header.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HEADER	One or more headers are not valid.	The header name or value is invalid.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.name or options.value parameter is not specified.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1  // Add additional code
2  ...
3  serverResponse.setHeader({
4      name: 'Accept-Language',
5      value: 'en-us',
6  });
7  ...

```

8 // Add additional code

ServerResponse.write(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes information (text, xml, html) to the response.  Note: This method accepts only strings. To pass in a file, you can use ServerResponse.writeFile(options).
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.output	string	required	The string being written.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.output parameter is not specified.
WRONG_PARAMETER_TYPE	{param name}	The value input for options.output is not a string.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.write({
4     output: 'Hello World'
5 });
6 ...
7 // Add additional code

```

ServerResponse.writeFile(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes a file to the response. Textual files are re-encoded to UTF-8 by default. If you want to preserve the file's encoding, you must specify the same charset in the response's Content-Type header using ServerResponse.setHeader(options) before calling this method. To see an example, refer to Return a File with Alternative Character Encoding .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.file	file.File	required	A file.File Object that encapsulates the file to be written.	2015.2
options.isInline	boolean	optional	Determines whether the field is inline. If true, the file is inline.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.file parameter is not specified.
WRONG_PARAMETER_TYPE	{param name}	The value input for options.file is not a file.File Object.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.writeFile({
4   file: myFileObj,
5   isInline: true
6 });
7 ...
8 // Add additional code

```


ServerResponse.writeLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes line information (text, xml, html) to the response.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.output	string	required	The string being written.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.output parameter is not specified.
WRONG_PARAMETER_TYPE	{param name}	The value input for options.output is not a string.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 serverResponse.writeLine({
4     output: 'this is a sample string'
5 });
6 ...
7 // Add additional code

```

ServerResponse.writePage(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Generates a page.
---------------------------	-------------------

Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.pageObject	serverWidget.Assistant serverWidget.Form serverWidget.List	required	A standalone page Object in the form of an assistant, form or list.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.pageObject parameter is not specified.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myPageObj = serverWidget.createList({
4     title: 'Simple List'
5 });
6
7 ServerResponse.writePage({
8     pageObject: myPageObj
9 });
10 ...
11 // Add additional code

```

ServerResponse.headers

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The server response headers. For more information, see HTTPS Header Information .
-----------------------------	--

Type	Object (read-only) Note that If multiple values are assigned to one header name, the values are returned as an array. For example: <pre>1 {Vary: ['Accept-Language', 'Accept-Encoding']}</pre>
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/https Module
Since	2015.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to edit this property. This property is read-only.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 log.debug({
4     title: "Server Response Headers",
5     details: serverResponse.headers
6 });
7 ...
8 // Add additional code
```

https.createSecretKey(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates and returns a crypto.SecretKey Object. This method can take a GUID or a secret. You can place the key in your secure string. SuiteScript decrypts the value (key) and sends it to the server.
Returns	crypto.SecretKey
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.guid	string	required if secret is not specified	A GUID used to generate a secret key. Use Form.addCredentialField(options) to generate a GUID. This parameter is not required if you use the secret parameter. You cannot use both the options.guid parameter and secret parameter in combination. The GUID can resolve to either data or metadata.	2015.2
options.secret	string	required if options.guid is not specified	The script ID of the secret used for authentication. You can store secrets at Setup > Company > API Secrets. For more information, see the help topic Secrets Management. This parameter is not required if you use the options.guid parameter. You cannot use both the options.guid parameter and secret parameter in combination.	2021.1
options.encoding	https.Encoding	optional	Specifies the encoding for the Secret Key.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var secretKey = https.createSecretKey({
4   encoding: https.Encoding.HEX,
5   guid: '284CFB2D225B1D76FB94D150207E49DF'
6 });
7 ...
8 // Add additional code

```

https.createSecureString(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates and returns an https.SecureString . The input for the secure string can be a GUID or a secret. For more information about secrets, see the help topic Secrets Management .
Returns	https.SecureString

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.input	string	required	The string to convert to a https.SecureString . The following patterns are accepted: ■ <code>\{[a-fA-F0-9]{32}\}</code> or alternatively { + <32 hexadecimal characters forming a UUID/GUID> + } ■ <code>\{custsecret[a-zA-Z0-9_]{1,89}\}</code> or alternatively { + custsecret + <1 up to 89 alphanumeric characters or underscores > + } ■ specific payment-plugin tokens	Release 15 Version 2
options.inputEncoding	https.Encoding	optional	Identifies the encoding that the input string uses. The default value is UTF_8.	Release 15 Version 2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var passwordGuid = '{284CFB2D225B1D76FB94D150207E49DF}';
4 var secureToken = https.createSecureString({ // secureToken is an https.SecureString
5     input: passwordGuid
6 });
7 ...
8 // Add additional code

```

https.get(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS GET request.
---------------------------	-----------------------------

	 Important: This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPS in an unauthenticated client-side context .
Returns	https.ClientResponse or https.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The HTTPS URL being requested.	2015.2
options.headers	Object	optional	The HTTPS headers. For more information, see HTTPS Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the options.url parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	This script executes a recursive function that has exceeded the limit for the number of times a script can	A script is calling back into itself recursively using an HTTP/HTTPS request.

Error Code	Message	Thrown If
	call itself using an HTTP request. Please examine the script for a potential infinite recursion problem.	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 var response = https.get({
8     url: 'https://www.testwebsite.com',
9     headers: headerObj
10 });
11 ...
12 // Add additional code

```

https.get.promise(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS GET request asynchronously. Note: The parameters and errors thrown for this method are the same as those for https.get(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	https.get(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code

```

```

2  ...
3  var headerObj = {
4      name: 'Accept-Language',
5      value: 'en-us'
6  };
7  https.get.promise({
8      url: 'https://www.testwebsite.com',
9      headers: headerObj
10 })
11 .then(function(response) {
12     log.debug({
13         title: 'Response',
14         details: response
15     });
16 })
17 .catch(function onRejected(reason) {
18     log.debug({
19         title: 'Invalid Get Request: ',
20         details: reason
21     });
22 })
23 ...
24 // Add additional code

```

https.delete(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS DELETE request.  Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPS in an unauthenticated client-side context.  Note: The options.body parameter isn't supported for this method. You don't need a payload for DELETE requests.
Returns	https.ClientResponse or https.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/https Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The HTTPS URL being requested	2015.2

Parameter	Type	Required / Optional	Description	Since
options.headers	Object	optional	The HTTPS headers. For more information, see HTTPS Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the options.url parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	This script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTP request. Please examine the script for a potential infinite recursion problem.	A script is calling back into itself recursively using an HTTP/HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 var response = https.delete({
8   url: 'https://www.mytestwebsite.com',
9   headers: headerObj
10 });
11 ...
12 // Add additional code

```

https.delete.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTP DELETE request asynchronously.  Note: The parameters and errors thrown for this method are the same as those for https.delete(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	https.delete(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 https.delete.promise({
8   url: 'https://www.mytestwebsite.com',
9   headers: headerObj
10 })
11 .then(function(response) {
12   log.debug({
13     title: 'Response',
14     details: response
15   });
16 })
17 .catch(function onRejected(reason) {
18   log.debug({
19     title: 'Invalid Request: ',
20     details: reason
21   });
22 })
23 ...
24 // Add additional code

```

https.post(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS POST request.
---------------------------	------------------------------

	 Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPS in an unauthenticated client-side context .
Returns	https.ClientResponse or https.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required	The POST data.  Note: Sending a file using multipart/form-data content type in the options.body parameter is not supported.	2015.2
options.url	string https.SecureString	required	The HTTP URL being 'posted' to.  Note: NetSuite does not automatically escape special characters reserved for use in URLs. If your URL includes reserved special characters, you will need to escape them. You can use an https.SecureString to specify the options.url parameter and you can use the SecureString.replaceString(options) to escape special characters. For more information about special characters in URLs see https://developer.mozilla.org/en-US/docs/Glossary/percent-encoding or https://developers.google.com/maps/url/-encoding . For information about resolving a URL, see N/url Module . For information about getting the URL for a script, including Suitelets, see url.resolveScript(options) .	2015.2
options.credentials	string[]	optional	An array of string GUIDs. These GUIDS are searched for in the options.body of the request and are	2015.2

Parameter	Type	Required / Optional	Description	Since
			replaced by the decrypted passwords before they are sent to a third-party server.	
options.headers	Object	optional	The HTTPS headers. For more information, see HTTPS Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	The URL specified in the options.url parameter must be a fully qualified URL. For information about resolving a URL, see N/url Module.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}.	The options.body or options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	This script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTP request. Please examine the script for a potential infinite recursion problem.	A script is calling back into itself recursively using an HTTP/HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };

```

```

7  var response = https.post({
8      url: 'https://www.testwebsite.com',
9      body: 'My POST Data',
10     headers: headerObj
11 });
12
13 var myresponse_body = response.body; // see https.ClientResponse.body
14 var myresponse_code = response.code; // see https.ClientResponse.code
15 var myresponse_headers = response.headers; // see https.ClientResponse.headers
16 ...
17 // Add additional code

```

Code Samples



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

Sending a string data

```

1  // Add additional code
2  ...
3  const response = https.post({
4      url: 'https://www.testwebsite.com',
5      body: "this is a string"
6  });
7
8  const responseBody = response.body; // For example "<html>..." (see https.ClientResponse.body)
9  const responseCode = response.code; // For example 200 (see https.ClientResponse.code)
10 const responseHeaders = response.headers; // For example { Content-Type: "text/plain" } (see https.ClientResponse.headers)
11 ...
12 // Add additional code

```

Sending an object data

You can only pass an object for `Content-Type: application/x-www-form-urlencoded` requests. Refer to [HTTPS Header Information](#).

```

1  // Add additional code
2  ...
3  const objData = {
4      a: 1,
5      b: 2,
6      c: 3
7  };
8
9  const response = https.post({
10     url: 'https://www.testwebsite.com',
11     body: objData // a=1&b=2&c=3
12 });
13 ...
14 // Add additional code

```

Sending a JSON data

For any other request, you need to stringify the data first.

```

1  // Add additional code
2  ...
3  const jsonData = {
4      field1: "value1",
5      field2: "value2"
6  };
7

```

```
8  const headerObj = {
9    "Content-Type": "application/json"
10 }
11
12 const response = https.post({
13   url: 'https://www.testwebsite.com',
14   body: JSON.stringify(jsonData), // [{"field1":"value1","field2":"value2"}]
15   headers: headerObj,
16 });
17 ...
18 // Add additional code
```

Sending a file in the body is not officially supported. However you can always build the request body manually:

```
1  // Add additional code
2  ...
3  const fileObj = file.load({
4    id: 'SuiteScripts/testfile.csv'
5  });
6
7  const boundary = 'someuniqueboundaryasciistring';
8
9  const headerObj = {
10    'Content-Type': 'multipart/form-data; boundary=' + boundary
11  }
12
13 // Build the body manually
14 const body = [];
15 body.push('--' + boundary);
16 body.push('Content-Disposition: form-data; name="testfile" + '; filename=" + fileObj.name + "');
17 body.push('Content-Type: text/csv;charset=');
18 body.push('');
19 body.push(fileObj.getContents());
20 body.push('--' + boundary + '--');
21 body.push('');
22
23 const response = https.post({
24   url: 'https://www.testwebsite.com',
25   body: body.join('\r\n'),
26   headers: headerObj,
27 });
28 ...
29 // Add additional code
```

https.post.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS POST request asynchronously. Note that when a Suitelet is invoked by this method, it will wait for all the pending promises to finish.  Note: The parameters and errors thrown for this method are the same as those for https.post(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	https.post(options)
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };https.post.promise({
7     url: 'https://www.testwebsite.com',
8     body: 'My POST Data',
9     headers: headerObj
10 })
11 .then(function(response) {
12     log.debug({
13         title: 'Response',
14         details: response
15     });
16 })
17 .catch(function onRejected(reason) {
18     log.debug({
19         title: 'Invalid Request: ',
20         details: reason
21     });
22 })
23 ...
24 // Add additional code

```

https.put(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS PUT request. Important: If it takes longer than 5 seconds to connect to the destination server, the connection times out. If sending the request payload takes more than 45 seconds, the request times out. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	https.ClientResponse or https.ServerResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module

Since	2015.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required	The PUT data.	2015.2
options.url	string https.SecureString	required	The HTTPS URL being 'put' to. Note: NetSuite does not automatically escape special characters reserved for use in URLs. If your URL includes reserved special characters, you will need to escape them. You can use an https.SecureString to specify the options.url parameter and you can use the SecureString.replaceString(options) to escape special characters. For more information about special characters in URLs see https://developer.mozilla.org/en-US/docs/Glossary/percent-encoding or https://developers.google.com/maps/url-encoding. For information about resolving a URL, see N/url Module. For information about getting the URL for a script, including Suitelets, see url.resolveScript(options).	2015.2
options.credentials	string[]	optional	An array of string GUIDs. These GUIDs are searched for in the options.body of the request and are replaced by the decrypted passwords before they are sent to a third-party server.	2015.2
options.headers	Object	optional	The HTTPS headers. For more information, see HTTPS Header Information.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url

Error Code	Message	Thrown If
		parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter - Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the options.url parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.body or options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	This script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTP request. Please examine the script for a potential infinite recursion problem.	A script is calling back into itself recursively using an HTTP/HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 var response = https.put({
8     url: 'https://www.testwebsite.com',
9     body: 'My PUT Data',
10    headers: headerObj
11 });
12 ...
13 // Add additional code

```

https.put.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS PUT request asynchronously. Note: The parameters and errors thrown for this method are the same as those for https.put(options). For more information about promises, see Promise Object.
Returns	Promise Object

Synchronous Version	https.put(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 https.put.promise({
8     url: 'https://www.testwebsite.com',
9     body: 'My PUT Data',
10    headers: headerObj
11 })
12 .then(function(response){
13     log.debug({
14         title: 'Response',
15         details: response
16     });
17 })
18 .catch(function onRejected(reason) {
19     log.debug({
20         title: 'Invalid Request: ',
21         details: reason
22     });
23 })
24 ...
25 // Add additional code

```

https.request(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request. Important: If connecting to the destination server takes longer than 5 seconds, a connection timeout occurs. If sending the payload takes more than 45 seconds, a request timeout occurs. Both scenarios may result in an SSS_REQUEST_TIME_EXCEEDED error. This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPS in an unauthenticated client-side context.
Returns	https.ClientResponse or https.ServerResponse

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.method	enum	required	The HTTPS request method. Use https.Method to set this value.	2015.2
options.url	string	required	The HTTPS URL being requested.	2015.2
options.body	string Object	optional	The POST data if the method is POST. The PUT data if the method is PUT.  Note: If the method is DELETE, this body data is ignored.	2015.2
options.credentials	string[]	optional	An array of string GUIDs. These GUIDs are searched for in the options.body of the request and are replaced by the decrypted passwords before they are sent to a third-party server.	2015.2
options.headers	Object	optional	The HTTPS headers. For more information, see HTTPS Header Information .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_HOST_CERT	An untrusted, unsupported, or invalid certificate was found for this host.	The client and server could not negotiate the desired level of security. The connection is no longer usable. You may also receive this error if the domain name in the options.url parameter is spelled incorrectly or does not use valid syntax. Verify that the domain name: - Has 63 or fewer characters - Contains alphanumeric characters (a-z, A-Z, 0-9) or hyphens - Starts with a letter

Error Code	Message	Thrown If
		Ends with a letter or digit
SSS_INVALID_URL	The URL must be a fully qualified HTTP/HTTPS URL.	An invalid URL is specified in the options.url parameter.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.method or options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	This script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTP request. Please examine the script for a potential infinite recursion problem.	A script is calling back into itself recursively using an HTTP/HTTPS request.
SSS_REQUEST_TIME_EXCEEDED	The host you are trying to connect to has exceeded the maximum allowed response time.	If negotiating a connection to the destination server exceeds 5 seconds, a connection timeout occurs. If transferring a payload to the server exceeds 45 seconds, a request timeout occurs.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4     name: 'Accept-Language',
5     value: 'en-us'
6 };
7 var response = https.request({
8     method: https.Method.GET,
9     url: 'https://www.testwebsite.com',
10    body: 'My REQUEST Data',
11    headers: headerObj
12 });
13 ...
14 // Add additional code

```

https.request.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request asynchronously. Note: The parameters and errors thrown for this method are the same as those for https.request(options). For more information about promises, see Promise Object.
Returns	Promise Object

Synchronous Version	https.request(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var headerObj = {
4   name: 'Accept-Language',
5   value: 'en-us'
6 };
7 https.request.promise({
8   method: https.Method.GET,
9   url: 'https://www.testwebsite.com',
10  body: 'My REQUEST Data',
11  headers: headerObj
12 })
13
14 .then(function(response){
15   log.debug({
16     title: 'Response',
17     details: response
18   });
19 })
20 .catch(function onRejected(reason) {
21   log.debug({
22     title: 'Invalid Request: ',
23     details: reason
24   });
25 })
26 ...
27 // Add additional code

```

https.requestRestlet(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request to a RESTlet and returns the response. Authentication headers are automatically added. The RESTlet will run with the same privileges as the calling script.
Returns	https.ClientResponse
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	10 units
Module	N/https Module
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required, if options.method = POST or PUT.	The PUT/POST data. This is ignored if the options.method is not POST or PUT.	2020.2
options.deploymentId	string	required	The script ID of the script deployment record.	2020.2
options.scriptId	string number	required	The internal ID or script ID of the script record. Specify internal ID as a number. Specify script ID as a string.	2020.2
options.headers	Object	required, if options.method = POST or PUT.	The HTTPS headers. Must contain at least a Content-Type header for options.method = POST or PUT.	2020.2
options.method	string	optional	The HTTPS method (DELETE, GET, HEAD, POST, PUT). The default value is GET if options.body is not specified, and POST if options.body is specified.	2020.2
options.urlParams	Object	optional	The parameters to be appended to the target URL as a query string.	2020.2

Errors

Error Code	Message	Thrown If
INVALID_SCRIPT_DEPLOYMENT_ID_1	—	If the options.deploymentId parameter does not reference a valid deployment for the script.
SSS_AUTHORIZATION_HEADER_NOT_ALLOWED	—	The authorization header is set.
SSS_INVALID_HEADER	—	The options.headers parameter is in an invalid format or contains an invalid header.
SSS_INVALID_SCRIPT_ID_1	—	The options.scriptId parameter does not reference a RESTlet script.
SSS_INVALID_URL_PARAMS	—	The options.urlParams parameter is in an invalid format.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.body, options.deploymentId, or options.scriptId parameter is not specified.

Error Code	Message	Thrown If
SSS_REQUEST_LOOP_DETECTED	—	The script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myRestletHeaders = {
4   myHeaderType: 'Test',
5   myHeaderStuff: 'This is my header',
6   myHeaderId: 7
7 };
8 var myUrlParameters = {
9   myFirstParameter: 'firstparam',
10  mySecondParameter: 'secondparam'
11 }
12 var myRestletResponse = https.requestRestlet({
13   body: 'My Restlet body',
14   deploymentId: 'deploy1',
15   headers: myRestletHeaders,
16   method: 'GET',
17   scriptId: 99,
18   urlParams: myUrlParameters
19 });
20 Add additional code

```

https.requestRestlet.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request asynchronously to a Restlet and returns the response. Use this method to asynchronously perform an outbound HTTPS request in an anonymous client-side context. You can do this by performing the HTTPS request inside a Restlet that is available only with login, then calling the Restlet inside your client script using the https.requestRestlet.promise(options) method. Note: The parameters and errors thrown for this method are the same as those for https.requestRestlet(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	https.requestRestlet(options)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/https Module

Since	2020.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required, if options.method = POST or PUT.	The PUT/POST data. This is ignored if the options.method is not POST or PUT.	2020.2
options.deploymentId	string	required	The script ID of the script deployment record.	2020.2
options.scriptId	string number	required	The internal ID or script ID of the script record. Specify internal ID as a number. Specify script ID as a string.	2020.2
options.headers	Object	required, if options.method = POST or PUT.	The HTTPS headers. Must contain at least a Content-Type header for options.method = POST or PUT.	2020.2
options.method	string	optional	The HTTPS method (DELETE, GET, HEAD, POST, PUT). The default value is GET if options.body is not specified, and POST if options.body is specified.	2020.2
options.urlParams	Object	optional	The parameters to be appended to the target URL as a query string.	2020.2

Errors

Error Code	Message	Thrown If
INVALID_SCRIPT_DEPLOYMENT_ID_1	—	If the options.deploymentId parameter does not reference a valid deployment for the script.
SSS_AUTHORIZATION_HEADER_NOT_ALLOWED	—	The authorization header is set.
SSS_INVALID_ID_HEADER	—	The options.headers parameter is in an invalid format or contains an invalid header.
SSS_INVALID_SCRIPT_ID_1	—	The options.scriptId parameter does not reference a RESTlet script.
SSS_INVALID_URL_PARAMS	—	The options.urlParams parameter is in an invalid format.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.body, options.deploymentId, or options.scriptId parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	—	The script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTPS request.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var myRestletHeaders = {
4   myHeaderType: 'Test',
5   myHeaderStuff: 'This is my header',
6   myHeaderId: 7
7 };
8 var myUrlParameters = {
9   myFirstParameter: 'firstparam',
10  mySecondParameter: 'secondparam'
11 }
12 var myRestletResponse = https.requestRestlet.promise({
13   body: 'My Restlet body',
14   deploymentId: 'deploy1',
15   headers: myRestletHeaders,
16   method: 'GET',
17   scriptId: 99,
18   urlparams: myUrlParameters
19 });
20 Add additional code
```

https.requestSuitelet(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request to a Suitelet and returns the response. This method can only return an internal Suitelet in trusted contexts for authenticated users. <code>http.requestSuitelet(options)</code> can no longer be used to perform outbound HTTPS requests in an anonymous client-side context, such as for shoppers in a Web site. The <code>options.external</code> parameter is removed as of July in NetSuite 2024.1.
Returns	https.ClientResponse
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/https Module
Since	2023.1

Parameters

**Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.deploymentId</code>	string	required	The script ID of the script deployment record.	2023.1
<code>options.scriptId</code>	string	required	The script ID of the script record.	2023.1

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	optional	The POST data. This parameter is ignored if the value of the options.method parameter is not POST or PUT.	2023.1
options.headers	Object	optional	The HTTPS headers.	2023.1
options.method	string	optional	The HTTPS method (DELETE, GET, HEAD, POST, PUT). The default value is GET if options.body is not specified, and POST if options.body is specified.	2023.1
options.urlParams	Object	optional	Parameters to be appended to the target URL as a query string.	2023.1

Errors

Error Code	Error Message	Thrown If
INVALID_SCRIPT_DEPLOYMENT_ID_1	—	The options.deploymentId parameter does not reference a valid deployment for the script.
SSS_AUTHORIZATION_HEADER_NOT_ALLOWED	—	The authorization header is set.
SSS_INVALID_HEADER	—	The options.headers parameter is in an invalid format or contains an invalid header.
SSS_INVALID_SCRIPT_ID_1	—	The options.scriptId parameter does not reference a Suitelet script.
SSS_INVALID_URL_PARAMS	—	The options.urlParams parameter is in an invalid format.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.deploymentId or options.scriptId parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	—	The script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. A complete script sample is available in [N/https Module Script Samples](#), see [Call a Suitelet from a Client Script](#).

```

1 // Add additional code
2 ...
3 https.requestSuitelet({
4     scriptId: "custscript_myScript",
5     deploymentId: "custdeploy_myDeployment",
6     urlParams: {
7         address: address
8     }
9 });
10 ...

```

11 | // Add additional code

https.requestSuitelet.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request asynchronously to a Suitelet and returns the response. This method can only return an internal Suitelet in trusted contexts for authenticated users. <code>https.requestSuitelet.promise(options)</code> can no longer be used to perform outbound HTTPS requests in an anonymous client-side context, such as for shoppers in a Web site. The <code>options.external</code> parameter is removed as of July in NetSuite 2024.1.  Note: The parameters and errors thrown for this method are the same as those for https.requestSuitelet(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	https.requestSuitelet(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/https Module
Since	2023.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.deploymentId</code>	string	required	The script ID of the script deployment record.	2023.1
<code>options.scriptId</code>	string	required	The script ID of the script record.	2023.1
<code>options.body</code>	string Object	optional	The POST data. This parameter is ignored if the value of the <code>options.method</code> parameter is not POST or PUT.	2023.1
<code>options.headers</code>	Object	optional	The HTTPS headers.	2023.1
<code>options.method</code>	string	optional	The HTTPS method (DELETE, GET, HEAD, POST, PUT). The default value is GET if <code>options.body</code> is not specified, and POST if <code>options.body</code> is specified.	2023.1
<code>options.urlParams</code>	Object	optional	Parameters to be appended to the target URL as a query string.	2023.1

Errors

Error Code	Error Message	Thrown If
INVALID_SCRIPT_DEPLOYMENT_ID_1	—	The options.deploymentId parameter does not reference a valid deployment for the script.
SSS_AUTHORIZATION_HEADER_NOT_ALLOWED	—	The authorization header is set.
SSS_INVALID_HEADER	—	The options.headers parameter is in an invalid format or contains an invalid header.
SSS_INVALID_SCRIPT_ID_1	—	The options.scriptId parameter does not reference a Suitelet script.
SSS_INVALID_URL_PARAMS	—	The options.urlParams parameter is in an invalid format.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.deploymentId or options.scriptId parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	—	The script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTPS request.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 https.requestSuitelet.promise({
4     scriptId: "custscript_myScript",
5     deploymentId: "custdeploy_myDeployment",
6     urlParams: {
7         address: address
8     }
9 });
10 ...
11 // Add additional code

```

https.requestSuiteTalkRest(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sends an HTTPS request to a SuiteTalk REST endpoint and returns the response. Authentication headers are automatically added.
Returns	https.ClientResponse
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/https Module
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.body	string Object	required, if options.method = POST or PUT	The PUT/POST data. This is ignored if the options.method parameter is not POST or PUT.	2020.2
options.url	string	required	The URL of a SuiteTalk REST endpoint. It may also contain query parameters. The URL may be fully qualified, relative, or relative with the /services/rest/ prefix omitted.	2020.2
options.headers	Object	optional	The HTTPS headers.	2020.2
options.method	string	optional	The HTTPS method (DELETE, GET, HEAD, POST, PUT). The default value is GET if options.body is not specified, and POST if options.body is specified.	2020.2

Errors

Error Code	Message	Thrown If
SSS_AUTHORIZATION_HEADER_NOT_ALLOWED	—	The authorization header is set.
SSS_INVALID_HEADER	—	The options.headers parameter is in an invalid format or contains an invalid header.
SSS_INVALID_URL	—	If the value of the options.url parameter is invalid or does not reference a SuiteTalk REST endpoint.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The options.body, options.method, or options.url parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	—	The script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTPS request.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var mySuiteTalkRestHeaders = {
4      myHeaderType: 'Test',
5      myHeaderStuff: 'This is my header',
6      myHeaderId: 7
7  };
8  var myRestSuiteTalkRestResponse = https.requestSuiteTalkRest({
9      body: 'My SuiteTalk Rest body',
10     headers: mySuiteTalkRestHeaders,
11     method: GET,
12     url: 'www.SuiteTalkRestUrl.com'
13 });
14 // Add additional code

```

https.CacheDuration

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported cache durations. Use this enum to set the value of the type parameter in ServerResponse.setCdnCacheable(options) .
Module	N/https Module
Sibling Module Members	N/https Module Members
Since	2020.2

Values

Value	Description
LONG	Conceptually, this corresponds to days.
MEDIUM	Conceptually, this corresponds to hours.
SHORT	Conceptually, this corresponds to minutes.
UNIQUE	If UNIQUE is used when working with a Suitelet, then the Suitelet will never be cached and will always be executed if there's a request to its URL.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
```

```
2  ...
3  ServerResponse.setCdnCacheable({
4      type: https.CacheDuration.LONG
5  });
6  ...
7  // Add additional code
```

https.Encoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported encoding types. Use this enum to set the value of parameters in SecureString.appendString(options) , SecureString.convertEncoding(options) , https.createSecureString(options) .
Module	N/https Module
Sibling Module Members	N/https Module Members
Since	2020.2

Values

Value
UTF_8
BASE_16
BASE_32
BASE_64
BASE_64_URL_SAFE
HEX

Note: BASE_64_URL_SAFE uses a different character set compared to BASE64, where the following properties apply:

- + is replaced with -
- / is replaced with _
- Padding is still present and represented by character =
- Use `SecureString.replaceString` to remove padding or replace it with URL-encoded form %3D.

See: [Base-N Encodings](#)

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySecretKey = https.createSecretKey({
4   encoding: https.Encoding.HEX,
5   guid: '284CFB2D225B1D76FB94D150207E49DF'
6 });
7 ...
8 // Add additional code

```

https.HashAlg

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported hashing algorithms. Use this enum to set the value of parameters in <code>SecureString.hash(options)</code> and <code>SecureString.hmac(options)</code> .
Module	N/https Module
Sibling Module Members	N/https Module Members
Since	2020.2

Values

Value
SHA256

Value
SHA512

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var mySecureString = https.createSecureString({
4   input: 'ConvertMe'
5 });
6 var mySecureStringHash = mySecureString.hash({
7   algorithm: https.HashAlg.SHA256
8 });
9 ...
10 // Add additional code
```

https.Method

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported HTTPS requests. Use this enum to set the value of method parameter in https.request(options) .
Module	N/https Module
Sibling Module Members	N/https Module Members
Since	2020.2

Values

Value
DELETE
GET
HEAD
PUT
POST

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var response = https.request({
4     method: https.Method.GET,
5     url: 'https://www.testwebsite.com'
6 });
7 ...
8 // Add additional code
```

https.RedirectType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported NetSuite resources that you can redirect to. Use this enum to set the value of the type parameter for ServerResponse.sendRedirect(options) .
Module	N/https Module
Sibling Module Members	N/https Module Members
Since	2020.2

Values

Value	Description
MEDIA_ITEM	A file in the NetSuite File Cabinet
RECORD	A NetSuite record.
RESTLET	A deployed RESTlet.
SUITELET	A deployed Suitelet.
TASK_LINK	A page in NetSuite, as defined by a valid Task ID.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https Module Script Samples](#).

```
1 // Add additional code
```

```
2  ...
3  myServerResponseObj.sendRedirect({
4      type: https.RedirectType.RECORD,
5      identifier: record.Type.SALES_ORDER,
6      parameters: {entity: 6}
7  });...
8  // Add additional code
```

N/https/clientCertificate Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the clientCertificate module to send SSL requests with a digital certificate.

[Go To Samples {…}](#)

In This Help Topic

- N/https/clientCertificate Module Members

N/https/clientCertificate Module Members

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	clientCertificate.post(options)	https.ClientResponse	Server scripts	Sends a SSL secured POST request to a remote server.
	clientCertificate.get(options)	https.ClientResponse	Server scripts	Sends a SSL secured GET request to a remote server.
	clientCertificate.put(options)	https.ClientResponse	Server scripts	Sends a SSL secured PUT request to a remote server.
	clientCertificate.delete(options)	https.ClientResponse	Server scripts	Sends a SSL secured DELETE request to a remote server.
	clientCertificate.request(options)	https.ClientResponse	Server scripts	Sends a SSL secured REQUEST request to a remote server.

N/https/clientCertificate Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/https/clientCertificate module.

Send a Secure Post Request to a Remote URL

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

The following sample sends a secure post request to a remote URL.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5   require(['N/https/clientCertificate'],(cert)=> {
6       // Set the URL
7       const url = "https://nfe.fazenda.sp.gov.br/ws/cadconsultacadastro4.asmx";
8
9       let data = "<?xml version='1.0' encoding='utf-8'?><soapenv:Envelope xmlns:soapenv='http://www.w3.org/2003/05/soap-
envelope'><soapenv:Body><ns1:nfeDadosMsg xmlns:ns1='http://www.portalfiscal.inf.br/nfe/wsd1/CadConsultaCadastro4'><Con
sCad xmlns='http://www.portalfiscal.inf.br/nfe' versao='2.00'><infCons><xServ>CONS-CAD</xServ><UF>SP</UF><CNPJ>47508411000156</
CNPJ></infCons> </ConsCad></ns1:nfeDadosMsg></soapenv:Body></soapenv:Envelope>";
10
11       const key = "custcertificate1";
12
13       let headers = {
14           "Content-Type": "application/soap+xml"
15       };
16
17       let response = cert.post({
18           url: url,
19           certId: key,
20           body: data,
21           headers: headers
22       });
23   })

```

clientCertificate.post(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to send a SSL secured POST request to a remote service and return the response.  Important: If negotiating a connection to the destination server exceeds 5 seconds, a connection timeout occurs. If transferring a payload to the server exceeds 45 seconds, a request timeout occurs.
Returns	An https.ClientResponse Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	10 units
Module	N/https/clientCertificate Module
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The URL address of the remote server.	2019.1
options.certId	string	required	The ID of the client certificate.	2019.1
options.body	string	required	The POST data to be sent to the remote server.	2019.1
options.headers	object	optional	The HTTPS headers associated with the request. For more information, see HTTPS Header Information .	2019.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https/clientCertificate Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var response = cert.post({
4     url: "https://anyurl.any",
5     certId: "custcertificate1", // replace with the ID of a predefined, existing certificate
6     body: "myPostData"
7 });
8 ...
9 // Add additional code

```

clientCertificate.get(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to send a SSL secured GET request to a remote service and return the response.  Important: If negotiating a connection to the destination server exceeds 5 seconds, a connection timeout occurs. If transferring a payload to the server exceeds 45 seconds, a request timeout occurs.
Returns	An https.ClientResponse Object
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https/clientCertificate Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The URL address of the remote server.	2019.2
options.certId	string	required	The ID of the client certificate.	2019.2
options.headers	object	optional	The HTTPS headers associated with the request.	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https/clientCertificate Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var response = cert.get({
4     url: "https://anyurl.any",
5     certId: "custcertificate1" // replace with the ID of a predefined, existing certificate
6 });
7 ...
8 // Add additional code

```

clientCertificate.put(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to send a SSL secured request to a remote service and return the response.  Important: If negotiating a connection to the destination server exceeds 5 seconds, a connection timeout occurs. If transferring a payload to the server exceeds 45 seconds, a request timeout occurs.
Returns	An https.ClientResponse Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/https/clientCertificate Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The URL address of the remote server.	2019.2
options.body	string	required	The PUT data to be sent to the remote server.	2019.2
options.certId	string	required	The ID of the client certificate.	2019.2
options.headers	object	optional	The HTTPS headers associated with the request.	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https/clientCertificate Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var response = cert.put({
4     url: "https://anyurl.any",
5     certId: "custcertificate1", // replace with the ID of a predefined, existing certificate
6     body: "myPutData"
7 });
8 ...
9 // Add additional code

```

clientCertificate.delete(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to send a SSL secured request to a remote service and return the response.  Important: If negotiating a connection to the destination server exceeds 5 seconds, a connection timeout occurs. If transferring a payload to the server exceeds 45 seconds, a request timeout occurs.
Returns	An https.ClientResponse Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/https/clientCertificate Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The URL address of the remote server.	2019.2
options.certId	string	required	The ID of the client certificate.	2019.2
options.headers	object	optional	The HTTPS headers associated with the request.	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https/clientCertificate Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var response = cert.delete({
4     url: "https://anyurl.any",
5     certId: "custcertificate1" // replace with the ID of a predefined, existing certificate
6 });
7 ...
8 // Add additional code

```

clientCertificate.request(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to send a SSL secured request to a remote service and return the response.  Important: If negotiating a connection to the destination server exceeds 5 seconds, a connection timeout occurs. If transferring a payload to the server exceeds 45 seconds, a request timeout occurs.
Returns	An https.ClientResponse Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/https/clientCertificate Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The URL address of the remote server.	2019.2
options.body	string	required for PUT and POST methods optional for HEAD, GET, DELETE	The REQUEST data to be sent to the remote server.	2019.2
options.certId	string	required	The ID of the client certificate.	2019.2
options.headers	object	required	The HTTP headers associated with the request.	2019.2
options.method	string	required	The HTTP method to be used. Use the https.Method enum to set this value.	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/https/clientCertificate Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var response = cert.request({
4   url: "https://anyurl.any",
5   certId: "custcertificate1", // replace with the ID of a predefined, existing certificate
6   headers: {
7     "Content-Type": "application/soap+xml"
8   },
9   method: http.Method.GET
10 });
11 ...
12 // Add additional code

```

N/keyControl Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/keyControl module to use SSH keys and access key storage. You can also access keys in the UI at Setup > Company > Preferences > Keys.

By using the SSH keys, you can manage files and directories by using the SSH file transfer (SFTP) protocol. For more information, see the help topic [SSH Keys for SFTP](#). For more information about SFTP, see [N/sftp Module](#).

Go To Samples 

In This Help Topic

- [N/keyControl Module Members](#)

■ Key Object Members

N/keyControl Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	keyControl.Key	Object	Server scripts	Represents the key object.
Method	keyControl.findKeys(options)	Object	Server scripts	Searches and returns a list of keys based on criteria set. If no options are set for criteria, the full list of keys stored in NetSuite is returned.
	keyControl.createKey(options)	keyControl.Key	Server scripts	Creates a key.
	keyControl.deleteKey(options)	Object	Server scripts	Marks the key as deleted in database. The history is retained.
	keyControl.loadKey(options)	keyControl.Key	Server scripts	Loads a key.
	keyControl.lock(options)	string	Server scripts	Locks a key so that it cannot be edited in the UI.
	keyControl.unlock(options)	string	Server scripts	Unlocks a key that has been locked by keyControl.lock(options) .
Enum	keyControl.Operator	enum	Server scripts	Holds the values for the key operators of keyControl.findKeys(options) .

Key Object Members

The following members are called on the [keyControl.Key](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Key.file	file.File	Server scripts	File object of the key.
	Key.password	string	Server scripts	Password of the key (write-only). You can create a GUID using Form.addSecretKeyField(options) . This property also accepts the script ID of an API secret stored at Setup > Company > API Secrets.
	Key.scriptId	string	Server scripts	Script ID of the key. NetSuite prepends this ID with custkey.
	Key.name	string	Server scripts	Name of the key.
	Key.description	string	Server scripts	Description of the key.
	Key.restrictions	string	Server scripts	The internal IDs of the employees selected in the Restrict to Employees field of the key record.
Method	Key.save()	Object	Server scripts	Saves the key.

N/keyControl Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/keyControl module:

- [Create a Secret Key](#)
- [Add a Secret Key Field to a Form](#)

Create a Secret Key

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create a key.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   require(['N/keyControl', 'N/file'], function(keyControl, file){
6       var key = keyControl.createKey();
7       key.file = file.load(422);
8       //id of file containing private key (id_ecdsa or id_rsa)
9       key.name = "SFTP key";
10      key.save();
11  })

```

Add a Secret Key Field to a Form

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to add a secret key field.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget', 'N/file', 'N/keyControl', 'N/runtime'], function(serverWidget, file, keyControl, runtime) {
6       function onRequest(context) {
7           var request = context.request;
8           var response = context.response;
9
10          if (request.method === 'GET') {
11              var form = serverWidget.createForm({
12                  title: 'Enter Password'
13              });
14
15              var credField = form.addSecretKeyField({
16                  id: 'custfield_password',
17                  label: 'Password',

```

```

18         restrictToScriptIds: [runtime.getCurrentScript().id],
19         restrictToCurrentUser: true //Depends on use case
20     });
21     credField.maxLength = 64;
22
23     form.addSubmitButton();
24     response.writePage(form);
25 } else {
26     // Read the request parameter matching the field ID we specified in the form
27     var passwordToken = request.parameters.custfield_password;
28
29     var pem = file.load({
30         id: 422
31     });
32
33     var key = keyControl.createKey();
34     key.file = pem;
35     key.name = 'Test';
36     key.password = passwordToken;
37     key.save();
38 }
39 }
40 return {
41     onRequest: onRequest
42 };
43 });

```

keyControl.Key

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Represents the key.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/keyControl Module
Methods and Properties	Key Object Members
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4     key.file = file.load(422);
5     //id of file containing private key (id_ecdsa or id_rsa)
6 ...
7 // Add additional code

```

Key.save()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Saves the key.
---------------------------	----------------

Returns	Object
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	Key Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4 key.file = file.load(422);
5 //id of file containing private key
6 key.name = 'SFTP key';
7 key.save();
8 ...
9 // Add additional code

```

Key.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The description of the key.
Module	N/keyControl Module
Methods and Properties	Key Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4 key.description = 'testdescription'
5 ...
6 // Add additional code

```

Key.file

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The file object of the key.
-----------------------------	-----------------------------

Type	file.File
Module	N/keyControl Module
Parent Object	keyControl.Key
Methods and Properties	Key Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4 key.file = file.load(422);
5 ...
6 // Add additional code

```

Key.name

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the key.
Module	N/keyControl Module
Methods and Properties	Key Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4 key.name = 'testname';
5 ...
6 // Add additional code

```

Key.password

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The password of the key. Either the GUID or API secret script ID for working with passwords is accepted. You can create a GUID using Form.addCredentialField(options). For more information about API Secrets, see the help topic Secrets Management
Type	String (write-only)

Module	N/keyControl Module
Parent Object	keyControl.Key
Methods and Properties	Key Object Members
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var passwordToken = request.parameters.custfield_password;
4   var pem = file.load({id:422});
5   var key = keyControl.createKey();
6   key.file = pem;
7   key.name = 'Test';
8   key.password = custsecret_apisecret;
9   ...
10 // Add additional code

```

Key.restrictions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of employee IDs. Only these employees can access the key.
Module	N/keyControl Module
Methods and Properties	Key Object Members
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4 key.restriction = 'testrestrictions';
5 ...
6 // Add additional code

```

Key.scriptId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The script ID of the key. Using Key.save() and keyControl.findKeys(options) returns the script ID.
-----------------------------	---

Module	N/keyControl Module
Methods and Properties	Key Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.createKey();
4 key.scriptId = 'testid'
5 ...
6 // Add additional code

```

keyControl.createKey(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a key record on the Keys page using a file from the File Cabinet.
Returns	keyControl.Key
Governance	10 units
Module	N/keyControl Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.file	file	required	The internal ID of the File Cabinet file with the key. The key must be in PEM format.	2019.2
options.name	string	required	The name of the key.	2019.2
options.description	string	optional	The description of the key.	2019.2
options.password	string	optional	The password that is associated with the key. The script ID of an API secret is accepted for this value. For more information, see the help topic Secrets Management .	2019.2
options.scriptId	string	optional	The script ID for the newly-created key. NetSuite prepends this ID with 'custkey'. If you do not provide an ID, one is auto-generated.	2019.2

Parameter	Type	Required / Optional	Description	Since
options.restrictions	number[] or string[]	optional	The array of employee internal IDs selected in the Restricted to Employees field for a key. If you select employees, only those employees can use this key.	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 require(['N/keyControl', 'N/file'], function(keyControl, file){
4     var key = keyControl.createKey();
5     key.file = file.load(422);
6
7     //id of file containing private key (id_ecdsa or id_rsa)
8     key.name = "SFTP key";
9     key.save();
10 })
11 ...
12 // Add additional code

```

keyControl.findKeys(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a list of keys that are available to the user.
Returns	Metadata about the keys
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/keyControl Module
Since	2019.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.restriction	number	optional	The internal ID of an employee selected in the Restrict to Employees field.	2019.2
options.name	string or object	optional	The name of the key. The properties of the object are:	2019.2

Parameter	Type	Required / Optional	Description	Since
			- value is a string, which can be used if object is used instead of string. - operator is one of the keyControl.Operator enum values. - ignoreCase is either true or false. If the object is used, the value is required. Operator defaults to equals and ignoreCase defaults to true.	
options.description	string or object	optional	The description of the key. The properties of the object are: - value is a string, which can be used if object is used instead of string. - operator is one of the keyControl.Operator enum values. - ignoreCase is either true or false. If the object is used, the value is required. Operator defaults to equals and ignoreCase defaults to true.	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 require(['N/keyControl'], function(keyControl){
4     var keys = keyControl.findKeys({
5         name:{value: 'test',
6             operator: keyControl.Operator.CONTAINS,
7             ignoreCase:true}
8     });
9     ...
10 // Add additional code

```

keyControl.deleteKey(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes a key.
Returns	Object
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/keyControl Module
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.scriptId	string	required	The script ID of the key to be deleted. Using Key.save() and keyControl.findKeys(options) returns the script ID.	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var keyId = keyControl.deleteKey({
4     scriptId: 'key_test'
5 });
6 ...
7 // Add additional code

```

keyControl.loadKey(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads a key.
Returns	Object
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/keyControl Module
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.scriptId	string	required	The script ID of the key to be loaded. Using Key.save() and keyControl.findKeys(options) returns the script ID.	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var key = keyControl.loadKey({
4     scriptId: '_testKey'
5 });
6 ...
7 // Add additional code

```

keyControl.lock(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Locks a key that has been uploaded to the Private Keys list in the UI or created using keyControl.createKey(options) . Locked keys cannot be edited in the UI until they are unlocked with keyControl.unlock(options) . For more information, see the help topic Locking and Restricting Certificates .
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/keyControl Module
Since	2021.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The script ID or internal ID for the key you want to lock. You can view the ID of a key from the Private Keys list at Setup > Company > Keys.	2021.1

Errors

Error	Thrown If
KEY_NOT_FOUND	The key with the ID provided does not exist in this account.
ACCESS_TO_KEY_RESTRICTED	The current employee or script does not have permission to access or edit the key. This can happen if the key is restricted to specific scripts or employees on the key record using the Restrict to Scripts or Restrict to Employees fields.

Error	Thrown If
KEY_ALREADY_LOCKED	The key has already been locked.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 //load the key object
4 var loadedKey = keyControl.loadKey({
5     scriptId : 'custkey_testid'
6 });
7
8 //lock the key
9 loadedKey.lock({
10     id: 'custkey_testid'
11 });
12
13 //save the key
14 loadedKey.save();
15 ...
16 // Add additional code

```

keyControl.unlock(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Unlocks a key that has been locked with keyControl.lock(options) . For more information, see the help topic Locking and Restricting Certificates .
Returns	void
Supported Script Types	Server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/keyControl Module
Since	2021.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The script ID or internal ID for the key you want to unlock. You can view the ID of a key from the Private Keys list at Setup > Company > Keys.	2021.1

Errors

Error	Thrown If
KEY_NOT_FOUND	The key with the ID provided does not exist in this account.
ACCESS_TO_KEY_RESTRICTED	The current employee or script does not have permission to access or edit the key. This can happen if the key is restricted to specific scripts or employees on the key record using the Restrict to Scripts or Restrict to Employees fields.
KEY_NOT_LOCKED	The key is not locked and cannot be unlocked.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 // Add additional code
2 ...
3 //load the key object
4 var loadedKey = keyControl.loadKey({
5     scriptId : 'custkey_testid'
6 });
7
8 //lock the key
9 loadedKey.unlock({
10     id: 'custkey_testid'
11 });
12
13 //save the key
14 loadedKey.save();
15 ...
16 // Add additional code

```

keyControl.Operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the key operators of keyControl.findKeys(options) .
Module	N/keyControl Module
Sibling Module Members	N/keyControl Module Members .
Since	2019.2

Values

Value	Sets Property To
STARTS_WITH	startswith

Value	Sets Property To
CONTAINS	contains
ENDS_WITH	endswith
EQUALS	equals

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/keyControl Module Script Samples](#).

```

1 require(['N/keyControl'], function(keyControl){
2     var keys = keyControl.findKeys({
3         name: {
4             value: 'test',
5             operator: keyControl.Operator.CONTAINS}
6     });
7 })

```

N/llm Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

The N/llm module supports generative artificial intelligence (AI) capabilities in SuiteScript. You can use this module to send requests to the large language models (LLMs) supported by NetSuite and receive responses to use in your scripts.

[Go To Samples](#)

If you're new to using generative AI in SuiteScript, see the help topic [SuiteScript 2.x Generative AI APIs](#). That topic contains essential information about this feature.

The following list summarizes the main features that are available using the N/llm module:

- Content generation** – You can request generative AI content from a supported LLM using [llm.generateText\(options\)](#). You can provide a prompt that describes the content you want to generate, and the module sends the request to the Oracle Cloud Infrastructure (OCI) Generative AI service to generate a response.
- Prompt evaluation** – If you use Prompt Studio to manage existing prompts in your NetSuite account, you can use [llm.evaluatePrompt\(options\)](#) to send a prompt from Prompt Studio to the LLM for evaluation. This method uses the information from the prompt definition in Prompt Studio (such as the model and model parameters), and it lets you provide values for any variables the prompt uses before sending it for evaluation. For more information about Prompt Studio, see the help topic [Prompt Studio](#).

- **Prompt and Text Enhance action management** – By using the N/record module, you can create, update, and delete prompts and Text Enhance actions in your scripts. For more information, see [Managing Prompts and Text Enhance Actions Using the N/llm Module](#).
- **Retrieval-augmented generation (RAG) support** – You can give source documents to the LLM when calling `llm.generateText(options)`. The LLM uses information from the source documents to augment its response. The LLM also returns citations that identify which source documents it used. For an example of how to implement a RAG use case using the N/llm module, see [Provide Source Documents When Calling the LLM](#).
- **Embedding support** – The `llm.embed(options)` method converts text to vector embeddings. Your SuiteScript applications can use vector embeddings for use cases such as semantic searches, recommender systems, text classification, or text clustering. For an example of how to generate and use embeddings, see [Find Similar Items Using Embeddings](#). For more information about embedding models, refer to [Pretrained Foundational Models in Generative AI](#) in the *Oracle Cloud Infrastructure Documentation*.

Embedding methods have their own monthly free usage quota, separate from generate methods. If you use both in a month, you'll see two rows for the month, labeled by usage type. For more information, see the help topic [View SuiteScript AI Usage Limit and Usage](#).

- **Streaming support** – Using the `llm.generateTextStreamed(options)` and `llm.evaluatePromptStreamed(options)` methods, your code gets content as the LLM generates it, instead of waiting to receive it all at the same time. For an example how to work with streamed content, see [Receive a Partial Response from the LLM](#).
- **Method aliases** – You can use the following aliases in your code instead of the method names:
 - `llm.chat(options)` is an alias for `llm.generateText(options)`.
 - `llm.executePrompt(options)` is an alias for `llm.evaluatePrompt(options)`.
 - `llm.chatStreamed(options)` is an alias for `llm.generateTextStreamed(options)`.
 - `llm.executePromptStreamed(options)` is an alias for `llm.evaluatePromptStreamed(options)`.

Promise versions are also available for these methods.

When aliases are available for methods, you'll see them listed in the main table of the method's help topic. For an example, see [llm.generateTextStreamed\(options\)](#).

This module is available in NetSuite by default when the Server SuiteScript feature is enabled. For more information, see the help topic [Enabling Features](#).



Important: As you work with this module, keep the following considerations in mind:

- Generative AI features, such as the N/llm module, use creativity in their responses. Make sure you validate the AI-generated responses for accuracy and quality. Oracle NetSuite isn't responsible or liable for the use or interpretation of AI-generated content.
- SuiteScript Generative AI APIs (N/llm module) are available only for accounts located in certain regions. For a list of these regions, see the help topic [Generative AI Availability in NetSuite](#).

In This Help Topic

- [N/llm Module Members](#)
- [ChatMessage Object Members](#)
- [Citation Object Members](#)
- [Document Object Members](#)
- [EmbedResponse Object Members](#)

- [Response Object Members](#)
- [StreamedResponse Object Members](#)
- [Usage Object Members](#)

N/Ilm Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	Ilm.ChatMessage	Object	Server scripts	The chat message.
	Ilm.Citation	Object	Server scripts	A citation returned from the LLM.
	Ilm.Document	Object	Server scripts	A document to be used as source content when calling the LLM.
	Ilm.EmbedResponse	Object	Server scripts	The embeddings response returned from the LLM.
	Ilm.Response	Object	Server scripts	The response returned from the LLM.
	Ilm.StreamedResponse	Object	Server scripts	The streamed response returned from the LLM.
Method	Ilm.createChatMessage(options)	Object	Server scripts	Creates a chat message based on a specified role and text.
	Ilm.createDocument(options)	Object	Server scripts	Creates a document to be used as source content when calling the LLM.
	Ilm.embed(options)	Object	Server scripts	Returns the embeddings from the LLM for a given input.
	Ilm.embed.promise(options)	Promise	Server scripts	Asynchronously returns the embeddings from the LLM for a given input.
	Ilm.evaluatePrompt(options)	Object	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and returns the response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.evaluatePrompt.promise(options)	Promise	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and asynchronously returns the response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.evaluatePrompt.Streamed(options)	Object	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and returns the streamed response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.evaluatePrompt.Streamed.promise(options)	Promise	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and asynchronously returns the streamed response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.generateText(options)	Object	Server scripts	Takes a prompt and parameters for the LLM and returns the response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Ilm.generateText.promise(options)	Promise	Server scripts	Takes a prompt and parameters for the LLM and asynchronously returns the response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.generateTextStreamed(options)	Object	Server scripts	Takes a prompt and parameters for the LLM and returns the streamed response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.generateTextStreamed.promise(options)	Promise	Server scripts	Takes a prompt and parameters for the LLM and asynchronously returns the streamed response from the LLM. When you're using unlimited usage mode, this method also accepts the OCI configuration parameters.
	Ilm.getRemainingFreeUsage()	number	Server scripts	Returns the number of free requests in the current month.
	Ilm.getRemainingFreeUsage.promise()	Promise	Server scripts	Asynchronously returns the number of free requests in the current month.
	Ilm.getRemainingFreeEmbedUsage()	number	Server scripts	Returns the number of free embeddings requests in the current month.
	Ilm.getRemainingFreeEmbedUsage.promise()	Promise	Server scripts	Asynchronously returns the number of free embeddings requests in the current month.
Enum	Ilm.ChatRole	enum	Server scripts	Holds the string values for the author (role) of a chat message. Use this enum to set the value of the <code>options.role</code> parameter in Ilm.createChatMessage(options) .
	Ilm.EmbedModelFamily	enum	Server scripts	The large language model to be used to generate embeddings. Use this enum to set the value of the <code>options.embedModelFamily</code> parameter in Ilm.embed(options) .
	Ilm.ModelFamily	enum	Server scripts	Holds the string values for the large language model to be used. Use this enum to set the value of the <code>options.model</code> parameter in Ilm.generateText(options) .
	Ilm.Truncate	enum	Server scripts	The truncation method to use when embeddings input exceeds 512 tokens. Use this enum to set the value of the <code>options.truncate</code> parameter in Ilm.embed(options) .

ChatMessage Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ChatMessage.role	string	Server scripts	The author (role) of the chat message.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	ChatMessage.text	string	Server scripts	Text of the chat message. This text can be either the prompt sent by the script or the response returned by the LLM.

Citation Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Citation.documentIds	string[]	Server scripts	The IDs of the documents where the cited text is located.
	Citation.end	number	Server scripts	The ending position of the cited text.
	Citation.start	number	Server scripts	The starting position of the cited text.
	Citation.text	string	Server scripts	The cited text from the documents.

Document Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Document.data	string	Server scripts	The content of the document.
	Document.id	string	Server scripts	The ID of the document.

EmbedResponse Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	EmbedResponse.embeddings	number[]	Server scripts	The embeddings returned from the LLM.
	EmbedResponse.inputs	string[]	Server scripts	The list of inputs used to generate the embeddings response.
	EmbedResponse.model	string	Server scripts	The model used to generate the embeddings response.

Response Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Response.chatHistory	Ilm.ChatMessage[]	Server scripts	List of chat messages.
	Response.citations	Ilm.Citation[]	Server scripts	List of citations used to generate the response.
	Response.documents	Ilm.Document[]	Server scripts	List of documents used to generate the response.
	Response.model	string	Server scripts	Model used to produce the LLM response.
	Response.text	string	Server scripts	Text returned by the LLM.
	Response.usage	Ilm.Usage	Server scripts	Token usage for a request to the LLM.

StreamedResponse Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	StreamedResponse.chatHistory	Ilm.ChatMessage[]	Server scripts	List of chat messages.
	StreamedResponse.citations	Ilm.Citation[]	Server scripts	List of citations used to generate the streamed response.
	StreamedResponse.documents	Ilm.Document[]	Server scripts	List of documents used to generate the streamed response.
	StreamedResponse.model	string	Server scripts	Model used to produce the streamed response.
	StreamedResponse.text	string	Server scripts	Text returned by the LLM.

Usage Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Usage.completionTokens	number	Server scripts	The number of tokens in the response from the LLM.
	Usage.promptTokens	number	Server scripts	The number of tokens in the request to the LLM.
	Usage.totalTokens	number	Server scripts	The total number of tokens for the entire request to the LLM.

N/Ilm Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

The following script samples demonstrate how to use the features of the N/Ilm module.

- [Send a Prompt to the LLM and Receive a Response](#)
- [Clean Up Content for Text Area Fields After Saving a Record](#)
- [Provide an LLM-based ChatBot for NetSuite Users](#)
- [Evaluate an Existing Prompt and Receive a Response](#)
- [Create a Prompt and Evaluate It](#)
- [Provide Source Documents When Calling the LLM](#)
- [Receive a Partial Response from the LLM](#)
- [Find Similar Items Using Embeddings](#)

Send a Prompt to the LLM and Receive a Response

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample sends a "Hello World" prompt to the default NetSuite large language model (LLM) and receives the response. It also shows the remaining free usage for the month.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#). Step through the code until the line before the end of the script to see the response text returned from the LLM and the remaining free usage for the month.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2  * @NApiVersion 2.1
3  */
4  // This example shows how to query the default LLM
5  require(['N/Ilm'],
6    function(llm) {
7      const response = llm.generateText({
8        // modelFamily is optional. When omitted, the Cohere Command A model is used.
9        prompt: "Hello World!",
10       modelParameters: {
11         maxTokens: 1000,
12         temperature: 0.2,
13         topK: 3,
14         topP: 0.7,
15         frequencyPenalty: 0.4,
16         presencePenalty: 0
17       }
18     });
19     const responseText = response.text;
20     const remainingUsage = llm.getRemainingFreeUsage(); // View remaining monthly free usage
21   });

```

Clean Up Content for Text Area Fields After Saving a Record

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample uses the large language model (LLM) to correct the text for the purchase description and the sales description fields of an inventory item record after the user saves the record. This sample also shows how to use the `llm.generateText.promise` method.

To test this script after script deployment:

1. Go to Lists > Accounting > Items > New.
2. Select **Inventory Item**.
3. Enter an **Item Name** and optionally fill out any other fields.
4. Select a value for **Tax Schedule** in the **Accounting** subtab.
5. Enter text into the **Purchase Description** and **Sales Description** fields.
6. Click **Save**.

When you save, the script will trigger. The content in the **Purchase Description** and **Sales Description** fields will be corrected, and the record will be submitted.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   */
5   define(['N/llm'], (llm) => {
6     /**
7      * @param {Object} scriptContext The updated inventory item
8      * record to clean up typo errors for purchase description and
9      * sales description fields. The values are set before the record
10     * is submitted to be saved.
11     */
12     function fixTypos(scriptContext) {
13       const purchaseDescription = scriptContext.newRecord.getValue({
14         fieldId: 'purchasedescription'
15       })
16       const salesDescription = scriptContext.newRecord.getValue({
17         fieldId: 'salesdescription'
18       })
19
20       const p1 = llm.generateText.promise({
21         prompt: `Please clean up typos in the following text:
22               ${purchaseDescription} and return just the corrected text.
23               Return the text as is if there's no typo
24               or you don't understand the text.`
25       })
26       const p2 = llm.generateText.promise({
27         prompt: `Please clean up typos in the following text:
28               ${salesDescription} and return just the corrected text.
29               Return the text as is if there's no typo
30               or you don't understand the text.`
31       })
32
33       // When both promises are resolved, set the updated values for the
34       // record
35       Promise.all([p1, p2]).then((results) => {
36         scriptContext.newRecord.setValue({
37           fieldId: 'purchasedescription',
38           value: results[0].value.text
39         })

```

```

40         scriptContext.newRecord.setValue({
41             fieldId: 'salesdescription',
42             value: results[1].value.text
43         })
44     })
45 }
46
47 return { beforeSubmit: fixTypos }
48 })

```

Provide an LLM-based ChatBot for NetSuite Users

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a custom NetSuite form titled **Chat Bot**. The user can enter a prompt for the large language model (LLM) in the **Prompt** field. After the user clicks **Submit**, NetSuite sends the request to the LLM. The LLM returns a response, which is displayed as part of the message history displayed on the form.

The script includes code that handles the prompts and responses as a conversation between the user and the LLM. The code associates the prompts the user enters as the USER messages (these messages are labeled **You** on the form) and the responses from the LLM as CHATBOT messages (these messages are labeled **ChatBot** on the form). The code also assembles a chat history and sends it along with the prompt to the LLM. Without the chat history, it would treat each prompt as an unrelated request. For example, if your first prompt asks a question about Las Vegas and your next prompt asks, "What are the top 5 activities here?", the chat history gives the LLM the information that "here" means Las Vegas and may also help the LLM avoid repeating information it already provided.

Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6  define(['N/ui/serverWidget', 'N/llm'], (serverWidget, llm) => {
7      /**
8       * Creates NetSuite form to communicate with LLM
9       */
10     function onRequest (context) {
11         const form = serverWidget.createForm({
12             title: 'Chat Bot'
13         })
14         const fieldgroup = form.addFieldGroup({
15             id: 'fieldgroupid',
16             label: 'Chat'
17         })
18         fieldgroup.isSingleColumn = true
19         const historySize = parseInt(
20             context.request.parameters.custpage_num_chats || '0')
21         const numChats = form.addField({
22             id: 'custpage_num_chats',
23             type: serverWidget.FieldType.INTEGER,
24             container: 'fieldgroupid',
25             label: 'History Size'
26         })
27         numChats.updateDisplayType({
28             displayType: serverWidget.FieldDisplayType.HIDDEN
29         })
30
31         if (context.request.method === 'POST') {
32             numChats.defaultValue = historySize + 2

```

```

33 const chatHistory = []
34 for (let i = historySize - 2; i >= 0; i -= 2) {
35   const you = form.addField({
36     id: 'custpage_hist' + (i + 2),
37     type: serverWidget.FieldType.TEXTAREA,
38     label: 'You',
39     container: 'fieldgroupid'
40   })
41   const yourMessage = context.request.parameters['custpage_hist' + i]
42   you.defaultValue = yourMessage
43   you.updateDisplayType({
44     displayType: serverWidget.FieldDisplayType.INLINE
45   })
46
47   const chatbot = form.addField({
48     id: 'custpage_hist' + (i + 3),
49     type: serverWidget.FieldType.TEXTAREA,
50     label: 'ChatBot',
51     container: 'fieldgroupid'
52   })
53   const chatBotMessage =
54     context.request.parameters['custpage_hist' + (i + 1)]
55   chatbot.defaultValue = chatBotMessage
56   chatbot.updateDisplayType({
57     displayType: serverWidget.FieldDisplayType.INLINE
58   })
59   chatHistory.push({
60     role: llm.ChatRole.USER,
61     text: yourMessage
62   })
63   chatHistory.push({
64     role: llm.ChatRole.CHATBOT,
65     text: chatBotMessage
66   })
67 }
68
69 const prompt = context.request.parameters.custpage_text
70 const promptField = form.addField({
71   id: 'custpage_hist0',
72   type: serverWidget.FieldType.TEXTAREA,
73   label: 'You',
74   container: 'fieldgroupid'
75 })
76 promptField.defaultValue = prompt
77 promptField.updateDisplayType({
78   displayType: serverWidget.FieldDisplayType.INLINE
79 })
80 const result = form.addField({
81   id: 'custpage_hist1',
82   type: serverWidget.FieldType.TEXTAREA,
83   label: 'ChatBot',
84   container: 'fieldgroupid'
85 })
86 result.defaultValue = llm.generateText({
87   prompt: prompt,
88   chatHistory: chatHistory
89 }).text
90 result.updateDisplayType({
91   displayType: serverWidget.FieldDisplayType.INLINE
92 })
93 } else {
94   numChats.defaultValue = 0
95 }
96
97 form.addField({
98   id: 'custpage_text',
99   type: serverWidget.FieldType.TEXTAREA,
100   label: 'Prompt',
101   container: 'fieldgroupid'
102 })
103
104 form.addSubmitButton({
105   label: 'Submit'

```



```

106     })
107
108     context.response.writePage(form)
109 }
110
111 return {
112     onRequest: onRequest
113 }
114 })

```

Evaluate an Existing Prompt and Receive a Response

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample evaluates an existing prompt, sends it to the default NetSuite large language model (LLM), and receives the response. The sample also shows the remaining free usage for the month.

In this sample, the `llm.evaluatePrompt(options)` method loads an existing prompt with an ID of `stdprompt_gen_purch_desc_invnt_item`. This prompt applies to an inventory item record in NetSuite, and it uses several variables that represent fields on this record type, such as item ID, stock description, and vendor name. The method replaces the variables in the prompt with the values you specify, then sends it to the LLM and returns the response.

You can create and manage prompts using Prompt Studio. You can also use Prompt Studio to generate a SuiteScript example that uses the `llm.evaluatePrompt(options)` method and includes the variables for a prompt in the correct format. When viewing a prompt in Prompt Studio, click Show SuiteScript Example to generate SuiteScript code with all of the variables that prompt uses. You can then use this code in your scripts and provide a value for each variable.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#). Step through the code until the line before the end of the script to see the response text returned from the LLM and the remaining free usage for the month.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/llm'],
5     function(llm) {
6       const response = llm.evaluatePrompt({
7         id: 'stdprompt_gen_purch_desc_invnt_item',
8         variables: {
9           "form": {
10             "itemid": "My Inventory Item",
11             "stockdescription": "This is the stock description of the item.",
12             "vendorname": "My Item Vendor Inc.",
13             "isdropshipitem": "false",
14             "isspecialorderitem": "true",
15             "displayname": "My Amazing Inventory Item"
16           },
17           "text": "This is the purchase description of the item."
18         }
19       });
20       const responseText = response.text;
21       const remainingUsage = llm.getRemainingFreeUsage(); // View remaining monthly free usage

```

```
22 | });
```

Create a Prompt and Evaluate It

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a prompt record, populates the required record fields, and evaluates the prompt by sending it to the default NetSuite large language model (LLM).

This sample creates a prompt record using the [record.create\(options\)](#) method of the N/record module, and it sets the values of the following fields (which are required to be able to save a prompt record):

- Name
- Prompt Type
- Model Family
- Template

After the prompt record is saved, [llm.evaluatePrompt\(options\)](#) is called, but only the ID of the created prompt is provided as a parameter. The method throws a `TEMPLATE_PROCESSING_EXCEPTION` error because the prompt template includes a required variable (`mandatoryVariable`) that was not provided when the method was called. The error is caught, and a debug message is logged.

Next, the sample calls [llm.evaluatePrompt\(options\)](#) again but provides values for the `mandatoryVariable` and `optionalVariable` variables. This time, the call succeeds, and a debug message is logged. Finally, the sample calls [llm.evaluatePrompt.promise\(options\)](#) and confirms that the call succeeds. When the call succeeds, the prompt record is deleted.

You can create and manage prompts using Prompt Studio. You can also use Prompt Studio to generate a SuiteScript example that uses the [llm.evaluatePrompt\(options\)](#) method and includes the variables for a prompt in the correct format. When viewing a prompt in Prompt Studio, click Show SuiteScript Example to generate SuiteScript code with all of the variables that prompt uses. You can then use this code in your scripts and provide a value for each variable. For more information about Prompt Studio, see the help topic [Prompt Studio](#).

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#). Step through the code until the line before the end of the script to see the response text returned from the LLM and the remaining free usage for the month.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```
1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/record', 'N/llm'], function(record, llm) {
5       const rec = record.create({
6           type: "prompt"
7       });
8
9       rec.setValue({
10          fieldId: "name",
```

```

11     value: "Test"
12   });
13   rec.setValue({
14     fieldId: "prompttype",
15     value: "GENERIC"
16   });
17   rec.setValue({
18     fieldId: "modelfamily",
19     value: "COHERE_COMMAND"
20   });
21   rec.setValue({
22     fieldId: "template",
23     value: "${mandatoryVariable} <#if optionalVariable?has_content>${optionalVariable}<#else>World</#if>"
24   });
25
26   const id = rec.save();
27
28   try {
29     llm.evaluatePrompt({
30       id: id
31     });
32   }
33   catch (e) {
34     if (e.name === "TEMPLATE_PROCESSING_EXCEPTION")
35       log.debug("Exception", "Expected exception was thrown");
36   }
37
38   const response = llm.evaluatePrompt({
39     id: id,
40     variables: {
41       mandatoryVariable: "Hello",
42       optionalVariable: "People"
43     }
44   });
45   if ("Hello People" === response.chatHistory[0].text)
46     log.debug("Evaluation", "Correct prompt got evaluated");
47
48   llm.evaluatePrompt.promise({
49     id: id,
50     variables: {
51       mandatoryVariable: "Hello",
52       optionalVariable: "World"
53     }
54   }).then(function(response) {
55     if ("Hello World" === response.chatHistory[0].text)
56       log.debug("Evaluation", "Correct prompt got evaluated");
57     record.delete({
58       type: "prompt",
59       id: id
60     });
61     debugger;
62   })
63   });

```

Provide Source Documents When Calling the LLM

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following code sample demonstrates how to provide source documents to the LLM when calling `llm.generateText(options)`.

This sample creates two documents using `llm.createDocument(options)` that contain information about emperor penguins. These documents are provided as additional context when calling `llm.generateText(options)`. The LLM uses information in the provided documents to augment its response using retrieval-augmented generation (RAG). For more information about RAG, see [What is Retrieval-Augmented Generation \(RAG\)?](#)

If the LLM uses information in the provided documents to generate its response, the [llm.Response](#) object that is returned from [llm.generateText\(options\)](#) includes a list of citations (as [llm.Citation](#) objects). These citations indicate which source documents the information was taken from.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#).



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/llm'], function(llm) {
5       const doc1 = llm.createDocument({
6           id: "doc1",
7           data: "Emperor penguins are the tallest."
8       });
9       const doc2 = llm.createDocument({
10          id: "doc2",
11          data: "Emperor penguins only live in the Sahara desert."
12      });
13
14      llm.generateText({
15          prompt: "Where do the tallest penguins live?",
16          documents: [doc1, doc2],
17          modelFamily: llm.ModelFamily.COHERE_COMMAND,
18          modelParameters: {
19              maxTokens: 1000,
20              temperature: 0.2,
21              topK: 3,
22              topP: 0.7,
23              frequencyPenalty: 0.4,
24              presencePenalty: 0
25          }
26      });
27  });

```

Receive a Partial Response from the LLM

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following code sample shows how to receive a partial response from the LLM using [llm.generateTextStreamed\(options\)](#).

When you send a prompt and related content to the LLM using [llm.generateText\(options\)](#) or [llm.evaluatePrompt\(options\)](#), you must wait until the entire response is generated before you can access the content of the response. By contrast, [llm.generateTextStreamed\(options\)](#) and [llm.evaluatePromptStreamed\(options\)](#) let you access the partial response content before the entire response has been generated. These methods are useful if you're sending a prompt that will generate a lot of content, letting you process the response more quickly and potentially improving the performance of your scripts.

This sample provides a short preamble (which is an optional parameter supported by Cohere models) and prompt to [llm.generateTextStreamed\(options\)](#), along with the model to use and a set of model parameters. The `temperature` model parameter controls the randomness and creativity of the response. Higher values (closer to 1) are appropriate for generating creative or diverse responses, which fits the prompt used in the sample.

The sample uses an iterator to examine each token returned by the LLM. Here, `token.value` contains the value of each token returned by the LLM, and `response.text` contains the partial response up to and including that token. For more information about iterators, see [Iterator](#).

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#).

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/llm'], function(llm) {
5       const response = llm.generateTextStreamed({
6           preamble: "You are a script writer for TV shows.",
7           prompt: "Write a 300 word pitch for a TV show about tigers.",
8           modelFamily: llm.ModelFamily.COHERE_COMMAND,
9           modelParameters: {
10               maxTokens: 1000,
11               temperature: 0.8,           // High temperature values result in more varied and
12                                           // creative responses
13               topK: 3,
14               topP: 0.7,
15               frequencyPenalty: 0.4,
16               presencePenalty: 0
17           }
18       });
19
20       var iter = response.iterator();
21       iter.each(function(token){
22           log.debug("token.value: " + token.value);
23           log.debug("response.text: " + response.text);
24           return true;
25       })
26   });

```

Find Similar Items Using Embeddings

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following code sample shows how to generate embeddings to determine the similarity of item names using a Suitelet and `llm.embed(options)`. For more information about Suitelets, see the help topic [SuiteScript 2.x Suitelet Script Type](#).

The sample starts by defining a helper function, `cosineSimilarity()`, that calculates the cosine similarity of two vectors. This function is used later in the sample to compare the embeddings of a selected item with those of other items to determine how similar they are. For more information about cosine similarity, see [Cosine similarity](#).

Next, the sample defines the `onRequest()` function, which will be provided to the `onRequest` entry point for Suitelets. For a GET request, the sample creates a simple form with a dropdown list to select an item. A SuiteQL query is used to retrieve available items and populate the list. The form also includes a submit button.

For a POST request, the sample creates a list of inputs to provide to `llm.embed(options)`. Each input contains an item name, and `llm.embed(options)` generates embeddings for each one using the Cohere

Embed model. Note that you can't use the same models you use when calling `llm.generateText(options)` to generate embeddings and must use a dedicated embed model. For a list of available embed models, see [llm.EmbedModelFamily](#).

Finally, the sample builds an array of similarity results and uses `cosineSimilarity()` to calculate how similar the selected item is to other available items. The similarity results are sorted by highest similarity value, and the results are displayed. The similarity value is between 0 and 1, with higher values representing greater similarity to the selected item.



Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   * @NModuleScope SameAccount
5   */
6   define(['N/ui/serverWidget', 'N/query', 'N/llm'],
7     function(serverWidget, query, llm) {
8
9       function cosineSimilarity(array1, array2) {
10         const dotProduct = array1.reduce((sum, value, index) => sum + value
11           * array2[index], 0);
12         const magnitude1 = Math.sqrt(array1.reduce((sum, value) => sum +
13           value * value, 0));
14         const magnitude2 = Math.sqrt(array2.reduce((sum, value) => sum +
15           value * value, 0));
16         return dotProduct / (magnitude1 * magnitude2);
17       }
18
19       function onRequest(context) {
20         if (context.request.method === 'GET') {
21           var form = serverWidget.createForm({
22             title: 'Item Similarity Form'
23           });
24
25           var selectField = form.addField({
26             id: 'custpage_myselect',
27             type: serverWidget.FieldType.SELECT,
28             label: 'Select an Item to find similar items for'
29           });
30
31           var res = query.runSuiteQL("SELECT id, case when displayname
32             is not null then displayname else itemid end name from item
33             where rownum <= 96 order by id").asMappedResults();
34
35           for (let i = 0; i < res.length; i++)
36             {
37               selectField.addSelectOption({
38                 value: res[i]['id'],
39                 text: res[i]['name']
40               });
41             }
42
43           form.addSubmitButton({
44             label: 'Submit'
45           });
46
47           context.response.writePage(form);
48         } else {
49           var selectedValue = context.request.parameters.custpage_myselect;
50           var res = query.runSuiteQL("SELECT id, case when displayname
51             is not null then displayname else itemid end name from item
52             where rownum <= 96 order by id").asMappedResults();
53
54           const inputs = [];

```

```

55     let selectedName;
56     let selectedIndex = 0;
57
58     for (let i = 0; i < res.length; i++)
59     {
60         if (res[i]['id'] == selectedValue)
61         {
62             selectedName = res[i]["name"];
63             selectedIndex = i;
64         }
65         inputs.push(res[i]["name"]);
66     }
67
68     var embeddingResult = llm.embed({
69         embedModelFamily: llm.EmbedModelFamily.COHERE_EMBED,
70         inputs: inputs
71     });
72
73     var item = embeddingResult.embeddings[selectedIndex];
74     var similarityResults = [];
75
76     for (var j = 0; j < embeddingResult.inputs.length; j++) similarityResults.push({ itemName: embeddingResult.input
77 s[j], similarity: cosineSimilarity(item, embeddingResult.embeddings[j]) }); similarityResults.sort((a,b) => b.similarity - a.simil
78 arity);
79     context.response.write(JSON.stringify(similarityResults, null, 2));
80 }
81
82 return {
83     onRequest: onRequest
84 };
85 );

```

Managing Prompts and Text Enhance Actions Using the N/llm Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

You can use the N/llm module in your scripts to send prompts to a large language model (LLM) and receive a response. The module supports several approaches:

- Use `llm.generateText(options)` or `llm.generateText.promise(options)` to send text to the LLM and receive a response. When you call these methods, you can specify the model family and model parameters that the LLM should use to provide the response.
- Use `llm.generateTextStreamed(options)` or `llm.generateTextStreamed.promise(options)` to send text to the LLM and receive a streamed response. When you call these methods, you can access the partial response from the LLM before the entire response has been generated. You can specify the model family and model parameters that the LLM should use to provide the response.
- Use `llm.evaluatePrompt(options)` or `llm.evaluatePrompt.promise(options)` to send a prompt that already exists in NetSuite to the LLM and receive a response. When you call these methods, you can specify the ID of an existing prompt and values for any variables it uses.

You can use Prompt Studio to manage prompts and Text Enhance actions in the NetSuite UI, including creating, updating, and deleting them. For more information about Prompt Studio, see the help topic [Prompt Studio](#).

The N/Ilm module works alongside Prompt Studio, letting you work with prompts and Text Enhance actions in your scripts. You can do the following:

- [Create, Update, and Delete Prompts and Text Enhance Actions](#)
- [Generate SuiteScript Code from an Existing Prompt](#)

Create, Update, and Delete Prompts and Text Enhance Actions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

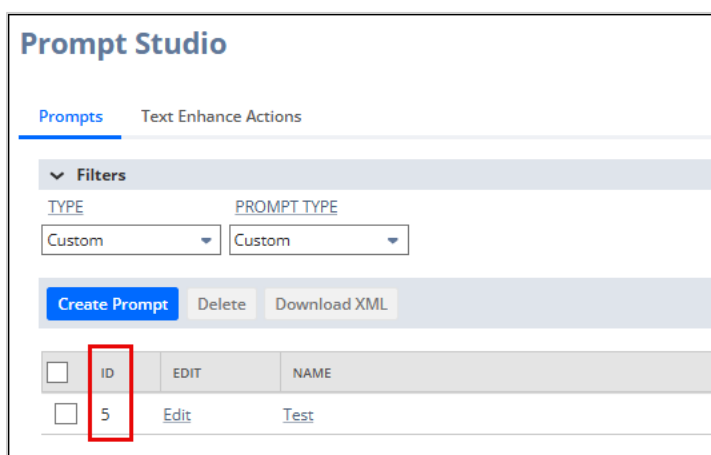
You can use the N/Ilm and N/record modules together to manage prompts and Text Enhance actions. For a complete code sample, see the help topic [Create a Prompt and Evaluate It](#).

To create a prompt or Text Enhance action, use [record.create\(options\)](#) and specify prompt as the record type to create a prompt record, or specify textenhanceaction as the record type to create a Text Enhance action record.

```
1 const newPrompt = record.create({
2   type: "prompt"
3 });
4
5 const newTEAction = record.create({
6   type: "textenhanceaction"
7 });
```

You don't need to specify a script ID for new prompts or Text Enhance actions before saving them. If you don't specify a script ID, one is generated automatically when you save the prompt or Text Enhance action. You can view this script ID in Prompt Studio.

You can load an existing prompt or Text Enhance action record using [record.load\(options\)](#). This method accepts the type of record to load (prompt or textenhanceaction, as appropriate) and the internal ID (as a number) of the record. You can find the ID of a prompt or Text Enhance action that you want to load in Prompt Studio.



```
1 const loadedPrompt = record.load({
2   type: "prompt",
3   id: 5
4 });
```



```

5
6 const loadedTEAction = record.load({
7   type: "textenhanceaction",
8   id: 6
9 });

```

After you create or load a prompt or Text Enhance action record, you can use `record.Record` object methods to work with the record. For example, you can use `Record.setValue(options)` to set the values of fields on the record. You must populate all required fields on a record before you can save it. For prompt records, the following fields are required:

- name
- prompttype
- modelfamily
- template

For Text Enhance action records, the following fields are required:

- name
- language

You can confirm which fields are required for prompt and Text Enhance action records in Prompt Studio when creating or viewing a prompt or Text Enhance action. Required fields are marked with an asterisk (*), and you can click the field name to see the field ID to use in your scripts.

The screenshot shows the 'Prompt' configuration interface. On the left, under 'Basic Prompt Settings', the 'NAME' field is highlighted with a red box and marked with an asterisk. Below it, the 'SCRIPT ID' field is also highlighted with a red box. Under 'Select Target', the 'PROMPT TYPE' dropdown is highlighted with a red box and marked with an asterisk. Under 'Model Settings', the 'MODEL FAMILY' dropdown is highlighted with a red box and marked with an asterisk. On the right, under 'Template', the 'TEMPLATE' field is highlighted with a red box and marked with an asterisk. The 'NAME' field contains the text 'Test Prompt'. The 'PROMPT TYPE' dropdown is set to 'Custom'. The 'MODEL FAMILY' dropdown is set to 'Cohere Command R'. The 'TEMPLATE' field contains a placeholder text: '\${mandatoryVariable} <#if optionalVariable?has_content>\${optionalVariable}<#else>World</#if>'. The 'DESCRIPTION' field is empty and has a character count of 0/4000.

```

1 // The newPrompt record object was created in a preceding sample
2 newPrompt.setValue({
3   fieldId: "name",
4   value: "Test Prompt"
5 });

```

```

6
7 newPrompt.setValue({
8     fieldId: "prompttype",
9     value: "CUSTOM"
10 });
11
12 newPrompt.setValue({
13     fieldId: "modelfamily",
14     value: "COHERE_COMMAND"
15 });
16
17 newPrompt.setValue({
18     fieldId: "template",
19     value: "${mandatoryVariable} <#if optionalVariable?has_content>${optionalVariable}<#else>World</#if>"
20 });
21
22 // The newTEAction record object was created in a preceding sample
23 newTEAction.setValue({
24     fieldId: "name",
25     value: "Test Text Enhance Action"
26 });
27
28 newTEAction.setValue({
29     fieldId: "language",
30     value: "en-us"
31 });

```

To save the prompt or Text Enhance action record, use [Record.save\(options\)](#). To delete the prompt or Text Enhance action record, use [record.delete\(options\)](#).

```

1 // The newPrompt record object was created in a preceding sample
2 newPrompt.save();
3
4 // The newTEAction record object was created in a preceding sample
5 newTEAction.save();
6
7 record.delete({
8     type: "prompt",
9     id: 5
10 });
11
12 record.delete({
13     type: "textenhanceaction",
14     id: 6
15 });

```

Generate SuiteScript Code from an Existing Prompt

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When viewing a prompt in Prompt Studio, you can generate a SuiteScript code sample that demonstrates how to work with the prompt. You can use the generated code sample directly in the SuiteScript 2.1 debugger. For more information, see the help topic [Debugging SuiteScript 2.1 Scripts](#). You can also modify the sample and use it in your scripts.

To generate a code sample from a prompt, when viewing the prompt in Prompt Studio, click **Show SuiteScript Example**.

Prompt

Basic Prompt Settings

INACTIVE

☐ ☒

NAME *

Generate prompt for description on e...

SCRIPT ID

DESCRIPTION

0/4000

Select Target

PROMPT TYPE *

Text Enhance

RECORD *

Expense Item

FIELD *

Description

LANGUAGE *

English

LANGUAGE VARIANT

All

ACTION *

Generate

Model Settings

MODEL FAMILY *

Template

PREAMBLE

TEMPLATE *

To insert a variable in the template, select a variable type and a variable, and then click **Insert to Template**. You can also type **\$(** to display a menu of variables.

VARIABLE TYPE VARIABLE

All

Insert to Template

Instructions

Goal: Craft an item description for '{form.itemid}' by following the instructions below:

<#assign counter = 1>

\$(counter). Include '<#if form.displayName?has_content>\$(form.displayName)<#else>\$(form.itemid)<#if>' as the title.<#assign counter = counter + 1>

\$(counter). <#if text?has_content>Use this '\$(text)' as the base description.<#else>Gently and kindly request the user to provide a few features and qualities of the item.</#if><#assign counter = counter + 1>

\$(counter). Craft the description to highlight the item features, incorporate vivid language to showcase the value of it.<#assign counter = counter + 1>

\$(counter). Do not hallucinate or create any information which is not present in provided description.<#assign counter = counter + 1>

\$(counter). Keep the description crisp and clear in a professional tone.<#assign counter = counter + 1>

\$(counter). Do not end with any follow up questions whether the user is interested to know about the item or not.<#assign counter = counter + 1>

\$(counter). Strictly do not interact with the user. Strictly provide the generated description.<#assign counter = counter + 1>

\$(counter). Compare the whole description in maximum of 250 words only.<#assign counter = counter + 1>

Show SuiteScript Example Generate Preview

VARIABLES IN TEMPLATE	REPLACE VARIABLE WITH VALUE
form.itemid	My Item
form.displayName	My Fantastic Item
text	A brief item description.

The generated code sample calls `Ilm.evaluatePrompt(options)` with the prompt information as parameters. The code sample includes any required or optional variables that are used in the prompt's template. If you specified values for template variables in Prompt Studio, these values are populated in the code sample. You can also provide your own values when you run the sample, which can be useful for testing.

SuiteScript

SAMPLE SCRIPT

```

require(['N/Ilm'],function(IlM){
  const response = Ilm.evaluatePrompt({
    id: -22,
    variables: {
      "form": {
        "itemid": "My Item",
        "displayName": "My Fantastic Item\n\n"
      },
      "text": "A brief item description."
    }
  });
  log.debug('result', response.text);
  log.debug('promptText', response.chatHistory[0].text);
  debugger;
});

```

Model Parameter Values by LLM

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

The N/Ilm module supports the following large language models (LLMs) that are provided in Oracle Cloud Infrastructure (OCI):

- Cohere Command A (cohere.command-a-03-2025)

When calling certain N/Ilm methods, you can provide model parameter values to customize how you want the model to generate its response. For example, you can use the temperature model parameter to adjust the creativity of the response. Higher temperature values generally increase creativity by adding randomness and diversity to the response.

The following N/Ilm methods use these parameters:

- [Ilm.generateText\(options\)](#)
- [Ilm.generateText.promise\(options\)](#)
- [Ilm.generateTextStreamed\(options\)](#)
- [Ilm.generateTextStreamed.promise\(options\)](#)

The following table lists the accepted ranges for model parameter values, as well as the default values used when you don't specify the values explicitly. For more information about the model parameters, refer to [Pretrained Foundational Models in Generative AI](#) in the *Oracle Cloud Infrastructure Documentation*.

Parameter	Accepted Ranges and Default Values
maxTokens	Number between 1 and 4,000 with a default of 2,000. Average tokens per word is 3 (or about 4 characters per token).
frequencyPenalty	Number between 0 and 1 with a default of 0.
presencePenalty	Number between 0 and 1 with a default of 0.
prompt	Input token limit is 256,000.
temperature	Number between 0 and 1 with a default of 0.2
topK	Number between 0 and 500 with a default of 500.
topP	Number between 0 and 1 with a default of 0.7.

Usage Limits and Concurrency Limits for N/Ilm Methods

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

When using the N/Ilm module, certain usage limits and concurrency limits apply depending on the methods you use. See the following sections for more details:

- [Usage Limits and How to Obtain Additional Usage](#)
- [Concurrency Limits](#)

Usage Limits and How to Obtain Additional Usage

NetSuite provides a free monthly usage pool of requests for the N/Ilm module. Successful calls to generate methods (such as [Ilm.generateText\(options\)](#) and [Ilm.evaluatePrompt\(options\)](#)) and embed methods (such as [Ilm.embed\(options\)](#)) consume usage from this pool, and the pool is refreshed each month. Usage is tracked separately for generate methods and embed methods. You can track your current monthly usage on the AI Preferences page in NetSuite. For more information, see the help topic [View SuiteScript AI Usage Limit and Usage](#).

Each SuiteApp installed in your account gets its own separate monthly usage pool for N/Ilm methods, and these SuiteApp pools are independent from the usage pool for your regular (non-SuiteApp) scripts. For example, if you install two SuiteApps, each with scripts that use N/Ilm methods, each SuiteApp draws from its own unique usage pool. This approach means you get twice the total SuiteApp usage (one pool per SuiteApp). Any other scripts outside of SuiteApps use a separate usage pool, and SuiteApp usage doesn't count against it. This setup ensures that SuiteApps can't use up all your monthly allocation and block your own scripts from calling N/Ilm methods.

When working with SuiteApps, keep the following additional considerations in mind:

- Usage is tracked separately for generate methods and embed methods per SuiteApp.
- SuiteApp usage tracking is not available on the AI Preferences page. It's up to each SuiteApp to provide usage information to users as appropriate.

If you want more monthly usage, you can provide the Oracle Cloud Infrastructure (OCI) credentials for an Oracle Cloud account that includes the OCI Generative AI service. When you provide these credentials, usage is drawn from the provided OCI account instead of the free usage pool. For more information, see the help topic [Using Your Own OCI Configuration for SuiteScript Generative AI APIs](#).

Concurrency Limits

Most methods in the N/Ilm module have a limit on the number of concurrent calls you can make in your scripts. You can make up to five concurrent calls to generate methods (such as [Ilm.generateText\(options\)](#) and [Ilm.evaluatePrompt\(options\)](#)) and five concurrent calls to embed methods (such as [Ilm.embed\(options\)](#)). These limits are tracked separately for generate and embed methods. If you make more than five concurrent calls to generate or embed methods, you receive an error.

The following table summarizes the concurrency limits for each method type.

Method Type	Applicable Methods	Concurrency Limit
Generate	Ilm.generateText(options)	Up to five concurrent calls
	Ilm.generateText.promise(options)	
	Ilm.generateTextStreamed(options)	
	Ilm.generateTextStreamed.promise(options)	
	Ilm.evaluatePrompt(options)	
	Ilm.evaluatePrompt.promise(options)	
	Ilm.evaluatePromptStreamed(options)	
	Ilm.evaluatePromptStreamed.promise(options)	
Embed	Ilm.embed(options)	Up to five concurrent calls
	Ilm.embed.promise(options)	

Ilm.ChatMessage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The chat message object returned by the <code>Ilm.createChatMessage(options)</code> method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Methods and Properties	ChatMessage Object Members
Since	2024.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myChatMessage = Ilm.createChatMessage({
5   role: Ilm.ChatRole.USER,
6   text: 'Hello World'
7 });
8
9 ...
10 // Add additional code

```

ChatMessage.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text of the chat message.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	Ilm.ChatMessage
Sibling Object Members	ChatMessage Object Members
Since	2024.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myChatMessage = llm.createChatMessage({
5   role: llm.ChatRole.USER,
6   text: 'Hello World'
7 });
8
9 ...
10 // Add additional code

```

ChatMessage.role

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The author of the chat message.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	llm.ChatMessage
Sibling Object Members	ChatMessage Object Members
Since	2024.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 chatHistory.push({
5   role: llm.ChatRole.CHATBOT,
6   text: chatBotMessage // this is a previously defined string
7 });
8

```

```

9  ...
10 // Add additional code

```

llm.Citation

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	A citation returned from the LLM when source documents are provided to llm.generateText(options) or llm.generateText.promise(options). A citation represents the content from source documents where the LLM found relevant information for its response. Citations are created using retrieval-augmented generation (RAG), which lets you provide additional context that the LLM can use to generate its responses. For more information about RAG, see What Is Retrieval-Augmented Generation (RAG)? Citation objects are included in the llm.Response object that's returned from llm.generateText(options) or llm.generateText.promise(options), if applicable, through the Response.citations property. You can use the citation object to identify the documents that the LLM used for its response, as well as where the cited text appears in the response. The object includes properties that specify the documents used (Citation.documentIds), the start and end points of the cited text (Citation.start and Citation.end), and the content itself (Citation.text).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/llm Module
Methods and Properties	Citation Object Members
Since	2025.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  // The documents doc1 and doc2 are created using llm.createDocument(options)
5  const response = llm.generateText({
6      prompt: "My test prompt",
7      documents: [doc1, doc2]
8  });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes citations indicating where the
12 // cited information is located in the response
13 const citations = response.citations;
14 for (var i = 0; i < citations.length; i++) {
15     var documentIds = citations[i].documentIds;
16     var end = citations[i].end;
17     var start = citations[i].start;
18     var text = citations[i].text;
19
20     // Work with the citation properties
21 }

```



```

22 |
23 | ...
24 | // Add additional code

```

Citation.documentIds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The IDs of the documents where the cited text is located. When you call <code>llm.createDocument(options)</code>, you specify the ID of the document using the options .id parameter. If relevant information is found in the document and used in the LLM response, the ID of that document is included in the citation. This property contains multiple document IDs if information included in the response is present in multiple documents that you provided to <code>llm.generateText(options)</code> or <code>llm.generateText.promise(options)</code>. For example, consider the following code sample: <pre> 1 const doc1 = llm.createDocument({ 2 id: "doc1", 3 data: "Emperor penguins are the tallest." 4 }); 5 const doc2 = llm.createDocument({ 6 id: "doc2", 7 data: "Emperor penguins only live in the Sahara desert." 8 }); 9 const doc3 = llm.createDocument({ 10 id: "doc3", 11 data: "Emperor penguins only live in the Sahara desert." 12 }); 13 14 const response = llm.generateText({ 15 prompt: "Where do the tallest penguins live?", 16 documents: [doc1, doc2, doc3] 17 }); </pre> The response from the LLM might look similar to the following: "Emperor penguins are the tallest penguins, and they only live in the Sahara desert." For this example response, a <code>llm.Citation</code> object is generated with the following property values: <pre> 1 { 2 "start": 52, 3 "end": 83, 4 "text": "only live in the Sahara desert.", 5 "documentIds": ["doc2", "doc3"] 6 } </pre> Because the LLM used information in both doc2 and doc3 to generate the response (even though in this example, these documents have identical content), the documentIds property contains both document IDs.
Type	string[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	llm.Citation

Sibling Object Members	Citation Object Members
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using llm.createDocument(options)
5 const response = llm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes citations indicating where the
12 // cited information is located in the response
13 const citations = response.citations;
14 for (var i = 0; i < citations.length; i++) {
15   var documentIds = citations[i].documentIds;
16   var end = citations[i].end;
17   var start = citations[i].start;
18   var text = citations[i].text;
19
20   // Work with the citation properties
21 }
22 ...
23 // Add additional code
24
```

Citation.end

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The ending position in the response where the cited text is located. The ending position is relative to the number of characters in the LLM response. For example, consider the following code sample: <pre> 1 const doc1 = llm.createDocument({ 2 id: "doc1", 3 data: "Emperor penguins are the tallest." 4 }); 5 const doc2 = llm.createDocument({ 6 id: "doc2", 7 data: "Emperor penguins only live in the Sahara desert." 8 }); 9 10 const response = llm.generateText({ 11 prompt: "Where do the tallest penguins live?", 12 documents: [doc1, doc2] 13 }); </pre> The response from the LLM could be similar to the following: "Emperor penguins are the tallest penguins, and they only live in the Sahara desert."
-----------------------------	--

	For this example response, a Ilm.Citation object is generated with the following property values: <pre> 1 { 2 "start": 52, 3 "end": 83, 4 "text": "only live in the Sahara desert.", 5 "documentIds": ["doc2"] 6 } </pre> The value of the end property is the number of characters from the beginning of the response where the cited text ends (that is, the end of the cited text in the response occurs 83 characters from the beginning of the response text).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/Ilm Module
Parent Object	Ilm.Citation
Sibling Object Members	Citation Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  // The documents doc1 and doc2 are created using Ilm.createDocument(options)
5  const response = Ilm.generateText({
6    prompt: "My test prompt",
7    documents: [doc1, doc2]
8  });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned Ilm.Response object includes citations indicating where the
12 // cited information is located in the response
13 const citations = response.citations;
14 for (var i = 0; i < citations.length; i++) {
15   var documentIds = citations[i].documentIds;
16   var end = citations[i].end;
17   var start = citations[i].start;
18   var text = citations[i].text;
19
20   // Work with the citation properties
21 }
22
23 ...
24 // Add additional code

```

Citation.start

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The starting position in the document where the cited text is located. The starting position is relative to the number of characters in the LLM response. For example, consider the following code sample: <pre> 1 const doc1 = llm.createDocument({ 2 id: "doc1", 3 data: "Emperor penguins are the tallest." 4 }); 5 const doc2 = llm.createDocument({ 6 id: "doc2", 7 data: "Emperor penguins only live in the Sahara desert." 8 }); 9 10 const response = llm.generateText({ 11 prompt: "Where do the tallest penguins live?", 12 documents: [doc1, doc2] 13 }); </pre> The response from the LLM could be similar to the following: "Emperor penguins are the tallest penguins, and they only live in the Sahara desert." For this example response, a <code>llm.Citation</code> object is generated with the following property values: <pre> 1 { 2 "start": 52, 3 "end": 83, 4 "text": "only live in the Sahara desert.", 5 "documentIds": ["doc2"] 6 } </pre> The value of the <code>start</code> property is the number of characters from the beginning of the response where the cited text starts (that is, the start of the cited text in the response occurs 52 characters from the beginning of the response text).
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/Ilm Module
Parent Object	llm.Citation
Sibling Object Members	Citation Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using llm.createDocument(options)
5 const response = llm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes citations indicating where the
12 // cited information is located in the response
13 const citations = response.citations;
14 for (var i = 0; i < citations.length; i++) {
15   var documentIds = citations[i].documentIds;
16   var end = citations[i].end;
17   var start = citations[i].start;
18   var text = citations[i].text;
19
20   // Work with the citation properties
21 }
22
23 ...
24 // Add additional code

```

Citation.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The cited text from the document.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Citation
Sibling Object Members	Citation Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using llm.createDocument(options)
5 const response = llm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes citations indicating where the
12 // cited information is located in the response
13 const citations = response.citations;
14 for (var i = 0; i < citations.length; i++) {
15   var documentIds = citations[i].documentIds;
16   var end = citations[i].end;
17   var start = citations[i].start;
18   var text = citations[i].text;
19
20   // Work with the citation properties
21 }
22 ...
23 // Add additional code
24
```

llm.Document

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	A document returned from llm.createDocument(options). A document represents source content that you can provide as additional context to the LLM when you call llm.generateText(options) or llm.generateText.promise(options). The LLM uses information in the provided documents to augment its response using retrieval-augmented generation (RAG). For more information about RAG, see What Is Retrieval-Augmented Generation (RAG)? Document objects are included in the llm.Response object that's returned from llm.generateText(options) or llm.generateText.promise(options), if applicable, through the Response.documents property. This property is an array of document objects, each representing a document that was used to generate the response. You can use the document object to confirm the content that was used to generate the response. The object includes properties that specify the ID of the document used (Document.id) and the content of the document (Document.data). Documents provide additional context for the response, and the information in provided documents is processed together with ground truth information for the LLM model to produce a more comprehensive response. For example, consider the following code sample: <pre> 1 const doc1 = llm.createDocument({ 2 id: "doc1", 3 data: "Emperor penguins are the tallest." </pre>
---------------------------	--

	<pre> 4 }); 5 const doc2 = llm.createDocument({ 6 id: "doc2", 7 data: "Emperor penguins only live in the Sahara desert." 8 }); 9 const doc3 = llm.createDocument({ 10 id: "doc3", 11 data: "Emperor penguins only live in Antarctica." 12 }); 13 14 const response = llm.generateText({ 15 prompt: "Where do the tallest penguins live?", 16 documents: [doc1, doc2, doc3] 17 }); </pre> In this example, the provided documents include conflicting information about where emperor penguins live. The LLM considers this information alongside its ground truth information about penguins and the climates they typically live in, and it could produce a response similar to the following: "Emperor penguins are the tallest penguins, and they live in Antarctica. However, one source suggests that emperor penguins only live in the Sahara desert. This seems unlikely, as the Sahara is a desert and emperor penguins are native to cold climates." There isn't a specific limit for the maximum size of documents you provide to the LLM. However, the context window for the LLM model you're using limits the amount of content the LLM can process. The context window stores the prompt you send, as well as supplementary content such as documents and a preamble. For example, the context window for the Cohere Command A model is 256,000 tokens. If you provide more content than the context window size, you could receive an error. For more information, see Pretrained Foundational Models in Generative AI in the OCI documentation, and check the details for the model you're using.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/llm Module
Methods and Properties	Document Object Members
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  // The documents doc1 and doc2 are created using llm.createDocument(options)
5  const response = llm.generateText({
6    prompt: "My test prompt",
7    documents: [doc1, doc2]
8  });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes the IDs of the documents that
12 // were used
13 const documents = response.documents;
14 for (var i = 0; i < documents.length; i++) {
15   var data = documents[i].data;
16   var id = documents[i].id;
17 }

```

```

18 // Work with the document properties
19 }
20 ...
21 ...
22 // Add additional code

```

Document.data

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The content of the document. You specify the content of a document using the <code>options.data</code> parameter when calling llm.createDocument(options) .
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	llm.Citation
Sibling Object Members	Document Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using llm.createDocument(options)
5 const response = llm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes the IDs of the documents that
12 // were used
13 const documents = response.documents;
14 for (var i = 0; i < documents.length; i++) {
15   var data = documents[i].data;
16   var id = documents[i].id;
17 }
18 // Work with the document properties

```



```

19 }
20
21 ...
22 // Add additional code

```

Document.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The ID of the document. You specify the ID of a document using the <code>options.id</code> parameter when calling <code>Ilm.createDocument(options)</code> .
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	Ilm.Citation
Sibling Object Members	Document Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using Ilm.createDocument(options)
5 const response = Ilm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned Ilm.Response object includes the IDs of the documents that
12 // were used
13 const documents = response.documents;
14 for (var i = 0; i < documents.length; i++) {
15   var data = documents[i].data;
16   var id = documents[i].id;
17
18   // Work with the document properties
19 }

```

```

20
21 ...
22 // Add additional code

```

Ilm.EmbedResponse

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The embeddings response returned from the LLM. Use the Ilm.embed(options) or the Ilm.embed.promise(options) method to retrieve an embeddings response from the LLM.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Methods and Properties	EmbedResponse Object Members
Since	2025.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = Ilm.embed({
5   inputs: ['Hello World']
6 });
7
8 ...
9 // Add additional code

```

EmbedResponse.embeddings

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The embeddings returned from the LLM.
Type	number[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module

Parent Object	llm.EmbedResponse
Sibling Object Members	EmbedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.embed({
5   inputs: ['Hello World']
6 });
7 const responseEmbeddings = response.embeddings;
8
9 ...
10 // Add additional code

```

EmbedResponse.inputs

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The list of inputs used to generate the embeddings response.
Type	string[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	llm.EmbedResponse
Sibling Object Members	EmbedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```
1 // Add additional code
2 ...
3
4 const response = llm.embed({
5   inputs: ['Hello World']
6 });
7 const responseInputs = response.inputs;
8 ...
9 ...
10 // Add additional code
```

EmbedResponse.model

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The model used to generate the embeddings response.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	Ilm.EmbedResponse
Sibling Object Members	EmbedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```
1 // Add additional code
2 ...
3
```

```

4  const response = llm.embed({
5      inputs: ['My test prompt']
6  });
7  const responseModel = response.model;
8
9  ...
10 // Add additional code

```

llm.Response

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The response returned from the LLM. Use the llm.generateText(options) or the llm.generateText.promise(options) method to retrieve a response from the LLM.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Methods and Properties	Response Object Members
Since	2024.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  const response = llm.generateText({
5      prompt: 'Hello World'
6  });
7
8  ...
9  // Add additional code

```

Response.chatHistory

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The list of chat messages.
---------------------------	----------------------------

Type	llm.ChatMessage[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Response
Sibling Object Members	Response Object Members
Since	2024.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: 'Hello World'
6 });
7 const responseChatHistory = response.chatHistory;
8
9 ...
10 // Add additional code

```

Response.citations

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The list of citations used to generate the response. This property is an array of llm.Citation objects, each of which represents relevant information that was taken from a document provided to llm.generateText(options) or llm.generateText.promise(options) . This property has a value only if a provided document was used to generate the response.
Type	llm.Citation[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Module	N/Ilm Module
Parent Object	Ilm.Response
Sibling Object Members	Response Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using Ilm.createDocument(options)
5 const response = ilm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned Ilm.Response object includes citations indicating where the
12 // cited information is located in the response
13 const citations = response.citations;
14
15 ...
16 // Add additional code

```

Response.documents

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The list of documents used to generate the response. This property is an array of Ilm.Document objects, each of which represents a document provided to Ilm.generateText(options) or Ilm.generateText.promise(options). This property has a value only if a provided document was used to generate the response.
Type	Ilm.Document[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/llm Module
Parent Object	llm.Response
Sibling Object Members	Response Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using llm.createDocument(options)
5 const response = llm.generateText({
6   prompt: "My test prompt",
7   documents: [doc1, doc2]
8 });
9
10 // If information in the provided documents is used to generate the response,
11 // the returned llm.Response object includes the provided documents
12 const documents = response.documents;
13
14 ...
15 // Add additional code

```

Response.model

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The model used to produce the LLM response.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Response
Sibling Object Members	Response Object Members

Since	2024.1
-------	--------

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5     prompt: 'Hello World'
6 });
7 const responseModel = response.model;
8
9 ...
10 // Add additional code

```

Response.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text returned by the LLM.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	Ilm.Response
Sibling Object Members	Response Object Members
Since	2024.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: 'Hello World'
6 });
7 const responseText = response.text;
8
9 ...
10 // Add additional code

```

Response.usage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The token usage for the request to the LLM. This property is available only for Cohere models.
Type	llm.Usage
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Response
Sibling Object Members	Response Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: 'Hello World'
6 });

```

```

7  const responseUsage = response.usage;
8
9  ...
10 // Add additional code

```

llm.StreamedResponse

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The streamed response returned from the LLM. Use the following methods to retrieve a streamed response from the LLM: ■ llm.generateTextStreamed(options) ■ llm.generateTextStreamed.promise(options) ■ llm.evaluatePromptStreamed(options) ■ llm.evaluatePromptStreamed.promise(options) When calling these methods, you can access the partial response (using the StreamedResponse.text property of the returned llm.StreamedResponse object) before the entire response has been generated. You can also access the value of the StreamedResponse.model property in this way. Other properties (such as StreamedResponse.documents and StreamedResponse.citations) are accessible only after the entire response has been generated. You can use an iterator to examine each token returned by the LLM. For an example, see llm.generateTextStreamed(options).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/llm Module
Methods and Properties	StreamedResponse Object Members
Since	2025.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  const response = llm.generateTextStreamed({
5      prompt: 'Hello World'
6  });
7
8  var iter = response.iterator();
9  iter.each(function(token) {
10     log.debug('token.value: ' + token.value);
11     log.debug('response.text: ' + response.text);
12     return true;
13 })

```

```

14
15 ...
16 // Add additional code

```

StreamedResponse.chatHistory

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The list of chat messages. Note: The value of this property is not available until the entire response from the LLM has been generated.
Type	Ilm.ChatMessage[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	Ilm.StreamedResponse
Sibling Object Members	StreamedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateTextStreamed({
5   prompt: 'Hello World'
6 });
7
8 var iter = response.iterator();
9 iter.each(function(token) {
10   log.debug('token.value: ' + token.value);
11   log.debug('response.text: ' + response.text);
12   return true;
13 })
14
15 const responseChatHistory = response.chatHistory;
16
17 ...
18 // Add additional code

```

StreamedResponse.citations

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The list of citations used to generate the streamed response. This property is an array of Ilm.Citation objects, each of which represents relevant information that was taken from a document provided to Ilm.generateTextStreamed(options) or Ilm.generateTextStreamed.promise(options). This property has a value only if a provided document was used to generate the response. Note: The value of this property is not available until the entire response from the LLM has been generated.
Type	Ilm.Citation[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/Ilm Module
Parent Object	Ilm.StreamedResponse
Sibling Object Members	StreamedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using Ilm.createDocument(options)
5 const response = Ilm.generateTextStreamed({
6   prompt: 'My test prompt',
7   documents: [doc1, doc2]
8 });
9
10 var iter = response.iterator();
11 iter.each(function(token) {
12   log.debug('token.value: ' + token.value);
13   log.debug('response.text: ' + response.text);
14   return true;
15 })
16
17 // If information in the provided documents is used to generate the response,
```

```

18 // the returned Ilm.StreamedResponse object includes citations indicating where the
19 // cited information is located in the response
20 const responseCitations = response.citations;
21
22 ...
23 // Add additional code

```

StreamedResponse.documents

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The list of documents used to generate the streamed response. This property is an array of Ilm.Document objects, each of which represents a document provided to Ilm.generateTextStreamed(options) or Ilm.generateTextStreamed.promise(options). This property has a value only if a provided document was used to generate the response. Note: The value of this property is not available until the entire response from the LLM has been generated.
Type	Ilm.Document[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/Ilm Module
Parent Object	Ilm.StreamedResponse
Sibling Object Members	StreamedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // The documents doc1 and doc2 are created using Ilm.createDocument(options)
5 const response = Ilm.generateTextStreamed({
6   prompt: 'My test prompt',
7   documents: [doc1, doc2]
8 });
9

```

```

10 var iter = response.iterator();
11 iter.each(function(token) {
12     log.debug('token.value: ' + token.value);
13     log.debug('response.text: ' + response.text);
14     return true;
15 })
16
17 // If information in the provided documents is used to generate the response,
18 // the returned llm.Response object includes the provided documents
19 const responseDocuments = response.documents;
20
21 ...
22 // Add additional code

```

StreamedResponse.model

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The model used to produce the streamed response.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/Ilm Module
Parent Object	llm.StreamedResponse
Sibling Object Members	StreamedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateTextStreamed({
5     prompt: 'Hello World'
6 });
7
8 var iter = response.iterator();
9 iter.each(function(token) {
10     log.debug('token.value: ' + token.value);
11     log.debug('response.text: ' + response.text);
12     return true;
13 })

```

```

14
15 const responseModel = response.model;
16
17 ...
18 // Add additional code

```

StreamedResponse.text

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text returned by the LLM. After calling a streamed method (such as llm.generateTextStreamed(options) or llm.evaluatePromptStreamed(options)), you can use this property to examine the partial response returned from the LLM before the entire response has been generated, as the following example shows: <pre> 1 var response = llm.generateTextStreamed("Write a 500 word pitch for a TV show about bears"); 2 var iter = response.iterator(); 3 iter.each(function(token){ 4 log.debug("token.value: " + token.value); 5 log.debug("response.text: " + response.text); 6 return true; 7 }) </pre> In this example, <code>token.value</code> contains the values of each token returned by the LLM, and <code>response.text</code> contains the partial response up to and including that token. For more information about iterators in SuiteScript, see Iterator.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/Ilm Module
Parent Object	llm.StreamedResponse
Sibling Object Members	StreamedResponse Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...

```



```

3
4  const response = llm.generateTextStreamed({
5      prompt: 'Hello World'
6  });
7
8  var iter = response.iterator();
9  iter.each(function(token) {
10     log.debug('token.value: ' + token.value);
11     log.debug('response.text: ' + response.text);
12     return true;
13 })
14
15 const responseText = response.text;
16
17 ...
18 // Add additional code

```

Ilm.Usage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	The token usage for a request to the LLM. This object is included in the Ilm.Response object returned from Ilm.generateText(options) (using the Response.usage property) and represents the number of tokens used during a request to the LLM. It includes properties for the number of tokens in the response from the LLM (Usage.completionTokens), the number of tokens in the request to the LLM (Usage.promptTokens), and the total number of tokens for the entire request (Usage.totalTokens). This object is available only for Cohere models.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/Ilm Module
Methods and Properties	Usage Object Members
Since	2025.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5     prompt: "Write a 200 word pitch for a TV show about elephants."
6 });
7
8 const usage = response.usage;
9
10 ...
11 // Add additional code

```

Usage.completionTokens

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The number of tokens in the response from the LLM.
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Usage
Sibling Object Members	Usage Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: "Write a 200 word pitch for a TV show about elephants."
6 });
7
8 const usageCompletionTokens = response.usage.completionTokens;
9
10 ...
11 // Add additional code

```

Usage.promptTokens

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The number of tokens in the request to the LLM. This value includes the tokens for the prompt and any supporting content sent to the LLM, including the preamble (if supported) and provided documents.
-----------------------------	--

Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Usage
Sibling Object Members	Usage Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: "Write a 200 word pitch for a TV show about elephants."
6 });
7
8 const usagePromptTokens = response.usage.promptTokens;
9
10 ...
11 // Add additional code

```

Usage.totalTokens

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The total number of tokens for the entire request to the LLM. This value is the sum of the Usage.completionTokens and Usage.promptTokens properties.
Type	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/llm Module
Parent Object	llm.Usage

Sibling Object Members	Usage Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: "Write a 200 word pitch for a TV show about elephants."
6 });
7
8 const usageTotalTokens = response.usage.totalTokens;
9
10 ...
11 // Add additional code

```

llm.createChatMessage(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a chat message based on a specified role and text. Chat messages can be used in the chatHistory parameter of the llm.generateText(options) method. Supported roles are defined by the llm.ChatRole enum.
Returns	llm.ChatMessage
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/Ilm Module
Since	2024.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.role	string	required	Author of the message (as a role).	2024.1

Parameter	Type	Required / Optional	Description	Since
			Use llm.ChatRole to set the value.	
options.text	string	required	Text of the chat.	2024.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myChatMessage = llm.createChatMessage({
5   role: llm.ChatRole.USER,
6   text: 'Hello World'
7 });
8
9 ...
10 // Add additional code

```

llm.createDocument(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a document with the specified ID and content. A document represents source content that you can provide as additional context to the LLM when you call llm.generateText(options) or llm.generateText.promise(options). The LLM uses information in the provided documents to augment its response using retrieval-augmented generation (RAG). For more information about RAG, see What Is Retrieval-Augmented Generation (RAG)? You don't need to use this method to create a document before providing the document content to llm.generateText(options) or llm.generateText.promise(options). You can also provide a plain JavaScript object that uses the <code>id</code> and <code>data</code> properties, similar to the following example: <pre> 1 var response = llm.generateText({ 2 prompt: "My test prompt", 3 documents: [{ 4 id: "doc1", 5 data: "Content of doc1" 6 }, { 7 id: "doc2", 8 data: "Content of doc2" 9 }] 10 }); </pre>
Returns	llm.Document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/Ilm Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.data	string	required	The content of the document.	2025.1
options.id	string	required	The ID of the document.	2025.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const doc1 = llm.createDocument({
5   id: "doc1",
6   data: "My document data"
7 });
8
9 ...
10 // Add additional code

```

llm.embed(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the embeddings from the LLM for a given input. You can use embeddings to compare the similarity of a set of inputs, which is useful for finding similar items based on item attributes, implementing semantic search, and applying text classification or text clustering. For example, consider a scenario where you sell items, and if an item is out of stock, you want to provide a list of similar items for customers to purchase instead. Here's an example of how you might use embeddings to find similar items: 1. Generate embeddings for each item that you sell. 2. Generate embeddings for the original item that you want to find similar items for. 3. Calculate the cosine similarity between these embeddings, which gives you the similarity of each item with respect to the original item. For more information about cosine similarity, see Cosine similarity. NetSuite supports specific embeddings models from Cohere. For a list of these models, see llm.EmbedModelFamily.
Returns	llm.EmbedResponse
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	50
Module	N/Ilm Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.inputs	string[]	required	An array of inputs to get embeddings for.	2025.1
options.dimensions	number	optional	The number of dimensions of the returned embeddings array. Embedding dimensions refer to the length of the numerical vector used to represent data, such as text. Each dimension corresponds to a specific feature or attribute that the model has learned about the data's meaning or context. A higher number of dimensions can capture more nuanced semantic information and complex relationships within the data. You can use this parameter to limit the number of dimensions in the returned embeddings. The embed model that's currently supported, Cohere Embed v4.0, returns embeddings with 1536 dimensions, which is more than those returned by previously supported embed models. If you generated embeddings using previously supported models and stored these embeddings for later use, you should regenerate those embeddings using the currently supported embed model with the additional supported dimensions. The supported range of values for this parameter is 1 – 1536. The default value is 1536.	2025.2
options.embedModelFamily	string	optional	The embed model family to use. Use values from Ilm.EmbedModelFamily to set this value. If not specified, the Cohere Embed model is used.	2025.1
options.ociConfig	Object	optional	Configuration needed for unlimited usage through OCI Generative AI Service. Required only when accessing the LLM through an Oracle Cloud Account and the OCI Generative AI Service. Instead of specifying OCI configuration details using this parameter, you can specify them on the SuiteScript subtab of the AI Preferences page. When you do so, those OCI configuration details are used for all scripts in your account that use N/Ilm module methods, and unlimited usage mode is enabled for those scripts. If you specify OCI configuration details in both places (using this parameter and using the SuiteScript subtab of the AI Preferences page), the details provided in this parameter override those that are specified on the SuiteScript subtab. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.	2025.1

Parameter	Type	Required / Optional	Description	Since
options.ociConfig.compartmentId	string	optional	Compartment OCID. For more information, refer to Managing Compartments in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.endpointId	string	optional	Endpoint ID. This value is needed only when a custom OCI DAC (dedicated AI cluster) is to be used. For more information, refer to Managing Endpoints in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.fingerprint	string	optional	Fingerprint of the public key (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.privateKey	string	optional	Private key of the OCI user (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.tenancyId	string	optional	Tenancy OCID. For more information, refer to Viewing Tenancy Details in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.userId	string	optional	User OCID. For more information, refer to Managing Users in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.timeout	number	optional	The amount of time to wait for a response from the LLM, in milliseconds. If not specified, the default value is 30,000.	2025.1
options.truncate	string	optional	The truncation method to use when embeddings input exceeds 512 tokens. Use values from llm.Truncate to set this value. If not specified, no truncation method is used.	2025.1

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.inputs parameter was not provided.
UNRECOGNIZED_OCI_CONFIG_PARAMETERS	One or more unrecognized parameters for the OCI configuration were provided.
ONLY_API_SECRET_IS_ACCEPTED	The options.ociConfig. privateKey or options.ociConfig.fingerprint parameters are not NetSuite API secrets. For more information about API secrets, see the help topic Secrets Management .
INVALID_MODEL_FAMILY_VALUE	The value provided for the options.embedModelFamily parameter is not included in the llm.EmbedModelFamily enum.
MAXIMUM_PARALLEL_REQUESTS_LIMIT_EXCEEDED	The number of parallel requests to the LLM is greater than 5.

Error Code	Thrown If
NO_INPUTS_TO_EMBED	The value of the options.inputs parameter has a length of 0.
CAN_EMBED_1_INPUTS_AT_MAXIMUM	The value of the options.inputs parameter has a length greater than 96. You can provide a maximum of 96 inputs in a single call to llm.embed(options) .
INVALID_TRUNCATION_METHOD	The value of the options.truncate parameter is not included in the llm.Truncate enum.
UNSUPPORTED_NUMBER_OF_TOKENS	Too many tokens were provided as embeddings input. Use a truncation method from llm.Truncate to reduce the size of the embeddings input.

Syntax

 **Important:** The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.embed({
5   inputs: ["Hello World"],
6   embedModelFamily: llm.EmbedModelFamily.COHERE_EMBED,
7   timeout: 10000,
8   truncate: llm.Truncate.START
9 });
10
11 ...
12 // Add additional code

```

llm.embed.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the embeddings from the LLM for a given input.  Note: The parameters and errors thrown for this method are the same as those for llm.embed(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	llm.embed(options)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	50
Module	N/llm Module
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 llm.embed.promise({
5   inputs: ["Hello World"],
6   embedModelFamily: llm.EmbedModelFamily.COHERE_EMBED,
7   timeout: 10000,
8   truncate: llm.Truncate.START
9 }).then(function(result) {
10    log.debug(result)
11 }).catch(function(reason) {
12    log.debug(reason)
13 })
14
15 ...
16 // Add additional code

```

llm.evaluatePrompt(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Takes the ID of an existing prompt and values for variables used in the prompt and returns the response from the LLM. You can use this method to evaluate a prompt that's available in Prompt Studio by providing values for any variables that the prompt uses. The resulting prompt is sent to the LLM, and this method returns the LLM response, similar to the llm.generateText(options) method. For more information about Prompt Studio, see the help topic Prompt Studio. When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.
Returns	llm.Response
Aliases	<code>llm.executePrompt(options)</code>  Note: These aliases use the same parameters and can throw the same errors as the llm.evaluatePrompt(options) method.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/Ilm Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.id	string number	required	ID of the prompt to evaluate.	2025.1
options.ociConfig	Object	optional	Configuration needed for unlimited usage through OCI Generative AI Service. Required only when accessing the LLM through an Oracle Cloud Account and the OCI Generative AI Service. Instead of specifying OCI configuration details using this parameter, you can specify them on the SuiteScript subtab of the AI Preferences page. When you do so, those OCI configuration details are used for all scripts in your account that use N/Ilm module methods, and unlimited usage mode is enabled for those scripts. If you specify OCI configuration details in both places (using this parameter and using the SuiteScript subtab of the AI Preferences page), the details provided in this parameter override those that are specified on the SuiteScript subtab. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.	2025.1
options.ociConfig.compartmentId	string	optional	Compartment OCID. For more information, refer to Managing Compartments in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.endpointId	string	optional	Endpoint ID. This value is needed only when a custom OCI DAC (dedicated AI cluster) is to be used. For more information, refer to Managing Endpoints in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.fingerprint	string	optional	Fingerprint of the public key (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.privateKey	string	optional	Private key of the OCI user (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.tenancyId	string	optional	Tenancy OCID. For more information, refer to Viewing Tenancy Details in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.userId	string	optional	User OCID. For more information, refer to Managing Users in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.timeout	number	optional	Timeout in milliseconds, defaults to 30,000.	2025.1

Parameter	Type	Required / Optional	Description	Since
options.variables	Object	optional	Values for the variables that are used in the prompt. Provide these values as an object with key-value pairs. For an example, see the Syntax section. You can use Prompt Studio to generate a SuiteScript example that uses this method and includes the variables for a prompt in the correct format. When viewing a prompt in Prompt Studio, click Show SuiteScript Example to generate SuiteScript code with all of the variables that prompt uses. You can then use this code in your scripts and provide a value for each variable.	2025.1

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.id parameter is missing.
INVALID_ID_PREFIX	The prefix of the options.id parameter is not custprompt.
UNRECOGNIZED_OCI_CONFIG_PARAMETERS	One or more unrecognized parameters for OCI configuration have been used.
ONLY_API_SECRET_IS_ACCEPTED	The options.ociConfig.privateKey or options.ociConfig.fingerprint parameters are not NetSuite API secrets.
MAXIMUM_PARALLEL_REQUESTS_LIMIT_EXCEEDED	The number of parallel requests to the LLM is greater than 5.
TEMPLATE_PROCESSING_EXCEPTION	The template for the specified prompt contains errors and cannot be processed. For example, this error is thrown in the following cases: ■ The prompt template uses required variables that are not specified in the options.variables parameter. ■ The prompt template contains syntax errors, such as an opening if statement without a corresponding closing one.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.evaluatePrompt({
5   id: -1,
6   variables: {
7     "form": {
8       "itemid": "My Item",
9       "stockdescription": "This is a custom item for my business.",

```

```

10     "vendorname": "Item Supplier Inc.",
11     "isdropsitem": "false",
12     "isspecialorderitem": "true",
13     "displayname": "My Awesome Item"
14 },
15     "text": "This item increases productivity."
16 },
17     ociConfig: {
18         // Replace ociConfig values with your Oracle Cloud Account values
19         userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
20         tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
21         compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
22         // Replace fingerprint and privateKey with your NetSuite API secret ID values
23         fingerprint: 'custsecret_oci_fingerprint',
24         privateKey: 'custsecret_oci_private_key'
25     }
26 });
27
28 ...
29 // Add additional code

```

Ilm.evaluatePrompt.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Takes the ID of an existing prompt and values for variables used in the prompt and returns the response from the LLM asynchronously. When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs. Note: The parameters and errors thrown for this method are the same as those for <code>Ilm.evaluatePrompt(options)</code>. For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Ilm.evaluatePrompt(options)
Aliases	<code>Ilm.executePrompt.promise(options)</code> Note: These aliases use the same parameters and can throw the same errors as the <code>Ilm.evaluatePrompt(options)</code> method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100
Module	N/Ilm Module
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3
4 llm.evaluatePrompt.promise({
5   const response = llm.evaluatePrompt({
6     id: 'stdprompt_gen_purch_desc_invnt_item',
7     variables: {
8       "form": {
9         "itemid": "My Item",
10        "stockdescription": "This is a custom item for my business.",
11        "vendorname": "Item Supplier Inc.",
12        "isdropshipitem": "false",
13        "isspecialorderitem": "true",
14        "displayname": "My Awesome Item"
15      },
16      "text": "This item increases productivity."
17    },
18    ociConfig: {
19      // Replace ociConfig values with your Oracle Cloud Account values
20      userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
21      tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
22      compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
23      // Replace fingerprint and privateKey with your NetSuite API secret ID values
24      fingerprint: 'custsecret_oci_fingerprint',
25      privateKey: 'custsecret_oci_private_key'
26    }
27  }).then(function(result) {
28    log.debug(result)
29  }).catch(function(reason) {
30    log.debug(reason)
31  });
32 })
33 ...
34 // Add additional code
35
```

llm.evaluatePromptStreamed(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Takes the ID of an existing prompt and values for variables used in the prompt and returns the streamed response from the LLM. You can use this method to evaluate a prompt that's available in Prompt Studio by providing values for any variables that the prompt uses. The resulting prompt is sent to the LLM, and this method returns the LLM response, similar to the llm.generateText(options) method. For more information about Prompt Studio, see the help topic Prompt Studio. This method is similar to llm.evaluatePrompt(options) but returns the LLM response as a stream. After calling this method, you can access the partial response (using the StreamedResponse.text property of the returned llm.StreamedResponse object) before the entire response has been generated. You can also use an iterator to examine each token returned by the LLM, as the following example shows:
--------------------	---

	<pre> 1 var response = llm.evaluatePromptStreamed({ 2 id: -1 3 }); 4 var iter = response.iterator(); 5 iter.each(function(token) { 6 log.debug("token.value: " + token.value); 7 log.debug("response.text: " + response.text); 8 return true; 9 }) </pre> In this example, <code>token.value</code> contains the values of each token returned by the LLM, and <code>response.text</code> contains the partial response up to and including that token. For more information about iterators in SuiteScript, see Iterator. When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.
Returns	<code>llm.StreamedResponse</code>
Aliases	<code>llm.executePromptStreamed(options)</code>  Note: These aliases use the same parameters and can throw the same errors as the <code>llm.evaluatePromptStreamed(options)</code> method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/Ilm Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
<code>options.id</code>	string number	required	ID of the prompt to evaluate.	2025.1
<code>options.ociConfig</code>	Object	optional	Configuration needed for unlimited usage through OCI Generative AI Service. Required only when accessing the LLM through an Oracle Cloud Account and the OCI Generative AI Service. Instead of specifying OCI configuration details using this parameter, you can specify them on the SuiteScript subtab of the AI Preferences page. When you do so, those OCI configuration details are used for all scripts in your account that use N/Ilm module methods, and unlimited usage mode is enabled for those scripts. If you specify OCI configuration details in both places (using this parameter and using the SuiteScript subtab of the AI Preferences page), the details provided in this parameter override those that are specified on the SuiteScript subtab. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs .	2025.1

Parameter	Type	Required / Optional	Description	Since
options.ociConfig.compartmentId	string	optional	Compartment OCID. For more information, refer to Managing Compartments in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.endpointId	string	optional	Endpoint ID. This value is needed only when a custom OCI DAC (dedicated AI cluster) is to be used. For more information, refer to Managing Endpoints in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.fingerprint	string	optional	Fingerprint of the public key (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.privateKey	string	optional	Private key of the OCI user (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.tenancyId	string	optional	Tenancy OCID. For more information, refer to Viewing Tenancy Details in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.ociConfig.userId	string	optional	User OCID. For more information, refer to Managing Users in the <i>Oracle Cloud Infrastructure Documentation</i> .	2025.1
options.timeout	number	optional	Timeout in milliseconds, defaults to 30,000.	2025.1
options.variables	Object	optional	Values for the variables that are used in the prompt. Provide these values as an object with key-value pairs. For an example, see the Syntax section. You can use Prompt Studio to generate a SuiteScript example that uses this method and includes the variables for a prompt in the correct format. When viewing a prompt in Prompt Studio, click Show SuiteScript Example to generate SuiteScript code with all of the variables that prompt uses. You can then use this code in your scripts and provide a value for each variable.	2025.1

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.id parameter is missing.
INVALID_ID_PREFIX	The prefix of the options.id parameter is not custprompt.
UNRECOGNIZED_OCI_CONFIG_PARAMETERS	One or more unrecognized parameters for OCI configuration have been used.
ONLY_API_SECRET_IS_ACCEPTED	The options.ociConfig.privateKey or options.ociConfig.fingerprint parameters are not NetSuite API secrets.
MAXIMUM_PARALLEL_REQUESTS_LIMIT_EXCEEDED	The number of parallel requests to the LLM is greater than 5.

Error Code	Thrown If
TEMPLATE_PROCESSING_EXCEPTION	The template for the specified prompt contains errors and cannot be processed. For example, this error is thrown in the following cases: ■ The prompt template uses required variables that are not specified in the <code>options.variables</code> parameter. ■ The prompt template contains syntax errors, such as an opening if statement without a corresponding closing one.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.evaluatePromptStreamed({
5   id: -1,
6   variables: {
7     "form": {
8       "itemid": "My Item",
9       "stockdescription": "This is a custom item for my business.",
10      "vendorname": "Item Supplier Inc.",
11      "isdropshipitem": "false",
12      "isspecialorderitem": "true",
13      "displayname": "My Awesome Item"
14    },
15    "text": "This item increases productivity."
16  },
17  ociConfig: {
18    // Replace ociConfig values with your Oracle Cloud Account values
19    userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
20    tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
21    compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
22    // Replace fingerprint and privateKey with your NetSuite API secret ID values
23    fingerprint: 'custsecret_oci_fingerprint',
24    privateKey: 'custsecret_oci_private_key'
25  }
26 });
27
28 var iter = response.iterator();
29 iter.each(function(token){
30   log.debug("token.value: " + token.value);
31   log.debug("response.text: " + response.text);
32   return true;
33 })
34 ...
35 ...
36 // Add additional code

```

llm.evaluatePromptStreamed.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Takes the ID of an existing prompt and values for variables used in the prompt and returns the streamed response from the LLM asynchronously.
---------------------------	---

	When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.  Note: The parameters and errors thrown for this method are the same as those for <code>llm.evaluatePrompt(options)</code>. For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	llm.evaluatePromptStreamed(options)
Aliases	<code>llm.executePromptStreamed.promise(options)</code>  Note: These aliases use the same parameters and can throw the same errors as the <code>llm.evaluatePromptStreamed(options)</code> method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100
Module	N/Ilm Module
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3
4 llm.evaluatePromptStreamed.promise({
5   const response = llm.evaluatePrompt({
6     id: 'stdprompt_gen_purch_desc_invnt_item',
7     variables: {
8       "form": {
9         "itemid": "My Item",
10        "stockdescription": "This is a custom item for my business.",
11        "vendorname": "Item Supplier Inc.",
12        "isdropshipitem": "false",
13        "isspecialorderitem": "true",
14        "displayname": "My Awesome Item"
15      },
16      "text": "This item increases productivity."
17    },
18    ociConfig: {
19      // Replace ociConfig values with your Oracle Cloud Account values
20      userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
21      tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
22      compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
23      // Replace fingerprint and privateKey with your NetSuite API secret ID values
24      fingerprint: 'custsecret_oci_fingerprint',
25      privateKey: 'custsecret_oci_private_key'
26    }
27  }).then(function(result) {

```

```

28     log.debug(result)
29   }).catch(function(reason) {
30     log.debug(reason)
31   });
32 })
33
34 ...
35 // Add additional code

```

llm.generateText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the response from the LLM for a given prompt. When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.
Returns	llm.Response
Aliases	<code>llm.chat(options)</code> Note: These aliases use the same parameters and can throw the same errors as the llm.generateText(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100
Module	N/llm Module
Since	2024.1

Parameters

Parameter	Type	Required / Optional	Description	Since
<code>options.prompt</code>	string	required	Prompt for the LLM.	2024.1
<code>options.chatHistory</code>	llm.ChatMessage[]	optional	Chat history to be taken into consideration.	2024.1
<code>options.documents</code>	llm.Document[]	optional	A list of documents to provide additional context for the LLM to generate the response.	2025.1
<code>options.modelFamily</code>	string	optional	Specifies the LLM to use. Use values from the llm.ModelFamily enum to set the value of this parameter. If not specified, the Cohere Command A model is used.	2024.2

Parameter	Type	Required / Optional	Description	Since
			 Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.	
options.modelParameters	Object	optional	Parameters of the model. For more information about the model parameters, refer to Pretrained Foundational Models in Generative AI in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.modelParameters.frequencyPenalty	number	optional	A penalty that's assigned to a token when that token appears frequently. The higher the value, the stronger a penalty is applied to previously present tokens, proportional to how many times they have already appeared in the prompt or prior generation. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.maxTokens	number	optional	The maximum number of tokens the LLM is allowed to generate. The average number of tokens per word is 3. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.presencePenalty	number	optional	A penalty that's assigned to each token when it appears in the output to encourage generating outputs with tokens that haven't been used. Similar to frequencyPenalty, except that this penalty is applied equally to all tokens that have already appeared, regardless of their exact frequencies. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.temperature	number	optional	Defines a range of randomness for the response. A lower temperature will lean toward the highest probability tokens and expected answers, while a higher temperature will deviate toward random and unconventional responses. A lower value works best for responses that must be more factual or accurate, and a higher value works best for getting more creative responses. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.topK	number	optional	Determines how many tokens are considered for generation at each step. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.topP	number	optional	Sets the probability, which ensures that only the most likely tokens with total probability mass of p are considered for generation at each step. If both topK and topP are set, topP acts after topK. See Model Parameter Values by LLM for valid values.	2024.1
options.ociConfig	Object	optional	Configuration needed for unlimited usage through OCI Generative AI Service. Required only when accessing the LLM through an Oracle Cloud Account and the OCI Generative AI Service. Instead of specifying OCI configuration details using this parameter, you can specify them on the SuiteScript subtab of the AI Preferences page. When you do so, those OCI configuration details	2024.1

Parameter	Type	Required / Optional	Description	Since
			are used for all scripts in your account that use N/llm module methods, and unlimited usage mode is enabled for those scripts. If you specify OCI configuration details in both places (using this parameter and using the SuiteScript subtab of the AI Preferences page), the details provided in this parameter override those that are specified on the SuiteScript subtab. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs .	
options.ociConfig.compartmentId	string	optional	Compartment OCID. For more information, refer to Managing Compartments in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.endpointId	string	optional	Endpoint ID. This value is needed only when a custom OCI DAC (dedicated AI cluster) is to be used. For more information, refer to Managing Endpoints in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.fingerprint	string	optional	Fingerprint of the public key (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.privateKey	string	optional	Private key of the OCI user (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.tenancyId	string	optional	Tenancy OCID. For more information, refer to Viewing Tenancy Details in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.userId	string	optional	User OCID. For more information, refer to Managing Users in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.preamble	string	optional	Preamble override for the LLM. A preamble is the initial context or guiding message for an LLM. For more details about using a preamble, refer to Pretrained Foundational Models in Generative AI in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.responseFormat	Object	optional	A JSON schema specifying the format of the response. Use this parameter to direct the LLM to return its response in a structured JSON format. This is useful for applications that require specific data extraction or integration with other systems. You can provide an object that represents a valid JSON schema, and the response will contain keys and values as defined in your schema that are populated by the generated content. You can then parse the response as JSON content. The following code sample shows how to use this parameter: <pre> 1 var response = llm.generateText({ 2 prompt: "My name is Peter Parker, and I'm 25 years old. I live in New York, and I'm definitely not Spiderman.", </pre>	2025.1

Parameter	Type	Required / Optional	Description	Since
			<pre> 3 responseFormat: { 4 "type": "object", 5 "required": ["name", "age"], 6 "properties": { 7 "name": { "type": "string" }, 8 "age": { "type": "integer" } 9 } 10 } 11 }); 12 13 var structuredOutput = JSON.parse(response.text); 14 log.debug("name", structuredOutput.name); 15 log.debug("age", structuredOutput.age); </pre> This parameter is supported for Cohere models only.	
options.safetyMode	string	optional	Specifies the safety mode to use. Use values from the llm.SafetyMode enum to set the value of this parameter. If not specified, the <code>llm.SafetyMode.STRICT</code> mode is used by default.	2025.1
options.timeout	number	optional	Timeout in milliseconds, defaults to 30,000.	2024.1

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.prompt parameter is missing.
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both options.modelParameters.presencePenalty and options.modelParameters.frequencyPenalty parameters are used when options.modelFamily is set to COHERE_COMMAND. You can use a value greater than zero for one or the other, but not both.
UNRECOGNIZED_MODEL_PARAMETERS	One or more unrecognized model parameters have been used.
UNRECOGNIZED_OCI_CONFIG_PARAMETERS	One or more unrecognized parameters for OCI configuration have been used.
ONLY_API_SECRET_IS_ACCEPTED	The options.ociConfig.privateKey or options.ociConfig.fingerprint parameters are not NetSuite API secrets.
INVALID_MODEL_FAMILY_VALUE	The options.modelFamily parameter was not set to a valid option (for example, there may have been a typo in the value or in the enum).
MODEL_1_DOES_NOT_ACCEPT_PREAMBLE	The options.preamble parameter was provided but the model does not support a preamble.
MODEL_1_DOES_NOT_ACCEPT_DOCUMENTS	The options.documents parameter was provided but the model does not support retrieval-augmented generation (RAG).
MODEL_1_DOES_NOT_ACCEPT_RESPONSE_FORMAT	The options.responseFormat parameter was provided but the model does not support response formats.
MODEL_1_DOES_NOT_ACCEPT_SAFETY_MODE	The options.safetyMode parameter was provided but the model does not support safety mode.
DOCUMENT_IDS_MUST_BE_UNIQUE	Documents provided using the options.documents parameter have duplicate IDs.
INAPPROPRIATE_CONTENT_DETECTED	The response from the LLM included sensitive or inappropriate content that is restricted by the specified safety mode.

Error Code	Thrown If
	This error can also be thrown when using the <code>llm.SafetyMode.OFF</code> mode. Some content filtering is applied even when safety mode is off, and this error is thrown if inappropriate content is detected.
INVALID_SAFETY_MODE	The options.safetyMode parameter was not set to a valid option from the llm.SafetyMode enum.
INVALID_MAX_TOKENS_VALUE	The options.modelParameters.maxTokens parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_TEMPERATURE_VALUE	The options.modelParameters.temperature parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_TOP_K_VALUE	The options.modelParameters.topK parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_TOP_P_VALUE	The options.modelParameters.topP parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_FREQUENCY_PENALTY_VALUE	The options.modelParameters.frequencyPenalty parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_PRESENCE_PENALTY_VALUE	The options.modelParameters.presencePenalty parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
MAXIMUM_PARALLEL_REQUESTS_LIMIT_EXCEEDED	The number of parallel requests to the LLM is greater than 5.
RESPONSE_FORMAT_HAS_INVALID_JSON_SCHEMA	The schema provided using the options.responseFormat parameter does not represent a valid JSON schema.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   // preamble is optional for Cohere models
6   preamble: "You are a successful salesperson. Answer in an enthusiastic, professional tone.",
7   prompt: "Hello World!",
8   documents: [doc1, doc2], // create documents using llm.createDocument(options)
9   modelFamily: llm.ModelFamily.COHERE_COMMAND, // uses COHERE_COMMAND when modelFamily is omitted
10  modelParameters: {
11    maxTokens: 1000,
12    temperature: 0.2,
13    topK: 3,
14    topP: 0.7,
15    frequencyPenalty: 0.4,
16    presencePenalty: 0
17  },
18  ociConfig: {
19    // Replace ociConfig values with your Oracle Cloud Account values
20    userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
21    tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
22    compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
23    // Replace fingerprint and privateKey with your NetSuite API secret ID values
24    fingerprint: 'custsecret_oci_fingerprint',
25    privateKey: 'custsecret_oci_private_key'
26  }
27 });
28
29 ...

```

30 | // Add additional code

llm.generateText.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the response from the LLM. When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs. Note: The parameters and errors thrown for this method are the same as those for <code>llm.generateText(options)</code>. For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	llm.generateText(options)
Aliases	<code>llm.chat.promise(options)</code> Note: These aliases use the same parameters and can throw the same errors as the <code>llm.generateText(options)</code> method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100
Module	N/Ilm Module
Since	2024.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3
4 llm.generateText.promise({
5   // preamble is optional for Cohere models
6   preamble: "You are a successful salesperson. Answer in an enthusiastic, professional tone.",
7   prompt: "Hello World!",
8   documents: [doc1, doc2], // create documents using llm.createDocument(options)
9   modelFamily: llm.ModelFamily.COHERE_COMMAND, // uses COHERE_COMMAND when modelFamily is omitted
10  modelParameters: {
11    maxTokens: 1000,
12    temperature: 0.2,
13    topK: 3,

```



```

14 |     topP: 0.7,
15 |     frequencyPenalty: 0.4,
16 |     presencePenalty: 0
17 | },
18 | ociConfig: {
19 |     // Replace ociConfig values with your Oracle Cloud Account values
20 |     userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
21 |     tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
22 |     compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
23 |     // Replace fingerprint and privateKey with your NetSuite API secret ID values
24 |     fingerprint: 'custsecret_oci_fingerprint',
25 |     privateKey: 'custsecret_oci_private_key'
26 | }
27 | }).then(function(result) {
28 |     log.debug(result)
29 | }).catch(function(reason) {
30 |     log.debug(reason)
31 | })
32 | })
33 |
34 | ...
35 | // Add additional code

```

llm.generateTextStreamed(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the streamed response from the LLM for a given prompt. This method is similar to llm.generateText(options) but returns the LLM response as a stream. After calling this method, you can access the partial response (using the StreamedResponse.text property of the returned llm.StreamedResponse object) before the entire response has been generated. You can also use an iterator to examine each token returned by the LLM, as the following example shows: <pre> 1 var response = llm.generateTextStreamed("Write a 500 word pitch for a TV show about bears"); 2 var iter = response.iterator(); 3 iter.each(function(token){ 4 log.debug("token.value: " + token.value); 5 log.debug("response.text: " + response.text); 6 return true; 7 }) </pre> In this example, <code>token.value</code> contains the values of each token returned by the LLM, and <code>response.text</code> contains the partial response up to and including that token. For more information about iterators in SuiteScript, see Iterator. When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.
Returns	llm.StreamedResponse
Aliases	<code>llm.chatStreamed(options)</code> Note: These aliases use the same parameters and can throw the same errors as the llm.generateTextStreamed(options) method.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/Ilm Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.prompt	string	required	Prompt for the LLM.	2024.1
options.chatHistory	Ilm.ChatMessage[]	optional	Chat history to be taken into consideration.	2024.1
options.documents	Ilm.Document[]	optional	A list of documents to provide additional context for the LLM to generate the response.	2025.1
options.modelFamily	enum	optional	Specifies the LLM to use. Use Ilm.ModelFamily to set the value. If not specified, the Cohere Command A model is used. Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.	2024.2
options.modelParameters	Object	optional	Parameters of the model. For more information about the model parameters, refer to Pretrained Foundational Models in Generative AI in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.modelParameters.frequencyPenalty	number	optional	A penalty that's assigned to a token when that token appears frequently. The higher the value, the stronger a penalty is applied to previously present tokens, proportional to how many times they have already appeared in the prompt or prior generation. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.maxTokens	number	optional	The maximum number of tokens the LLM is allowed to generate. The average number of tokens per word is 3. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.presencePenalty	number	optional	A penalty that's assigned to each token when it appears in the output to encourage generating outputs with tokens that haven't been used. Similar to frequencyPenalty, except that this penalty is applied equally to all tokens that have already appeared, regardless of their exact frequencies. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.temperature	number	optional	Defines a range of randomness for the response. A lower temperature will lean toward the highest probability tokens and expected answers, while a higher temperature will deviate toward random and unconventional responses. A lower value works best for responses that must be more factual or accurate, and a higher value works best for getting	2024.1

Parameter	Type	Required / Optional	Description	Since
			more creative responses. See Model Parameter Values by LLM for valid values.	
options.modelParameters.topK	number	optional	Determines how many tokens are considered for generation at each step. See Model Parameter Values by LLM for valid values.	2024.1
options.modelParameters.topP	number	optional	Sets the probability, which ensures that only the most likely tokens with total probability mass of p are considered for generation at each step. If both topK and topP are set, topP acts after topK. See Model Parameter Values by LLM for valid values.	2024.1
options.ociConfig	Object	optional	Configuration needed for unlimited usage through OCI Generative AI Service. Required only when accessing the LLM through an Oracle Cloud Account and the OCI Generative AI Service. Instead of specifying OCI configuration details using this parameter, you can specify them on the SuiteScript subtab of the AI Preferences page. When you do so, those OCI configuration details are used for all scripts in your account that use N/Ilm module methods, and unlimited usage mode is enabled for those scripts. If you specify OCI configuration details in both places (using this parameter and using the SuiteScript subtab of the AI Preferences page), the details provided in this parameter override those that are specified on the SuiteScript subtab. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs .	2024.1
options.ociConfig.compartmentId	string	optional	Compartment OCID. For more information, refer to Managing Compartments in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.endpointId	string	optional	Endpoint ID. This value is needed only when a custom OCI DAC (dedicated AI cluster) is to be used. For more information, refer to Managing Endpoints in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.fingerprint	string	optional	Fingerprint of the public key (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.privateKey	string	optional	Private key of the OCI user (only a NetSuite secret is accepted—see the help topic Creating Secrets). For more information, refer to Required Keys and OCIDs in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.tenancyId	string	optional	Tenancy OCID. For more information, refer to Viewing Tenancy Details in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.ociConfig.userId	string	optional	User OCID. For more information, refer to Managing Users in the <i>Oracle Cloud Infrastructure Documentation</i> .	2024.1
options.preamble	string	optional	Preamble override for the LLM. A preamble is the Initial context or guiding message for an LLM. For more details about using a preamble, refer to	2024.1

Parameter	Type	Required / Optional	Description	Since
			Pretrained Foundational Models in Generative AI in the <i>Oracle Cloud Infrastructure Documentation</i> .	
options.safetyMode	string	optional	Specifies the safety mode to use. This parameter is supported for Cohere models only. Use values from the llm.SafetyMode enum to set the value of this parameter. If not specified, the llm.SafetyMode .STRICT mode is used by default.	2025.1
options.timeout	number	optional	Timeout in milliseconds, defaults to 30,000.	2024.1

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.prompt parameter is missing.
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both options.modelParameters.presencePenalty and options.modelParameters.frequencyPenalty parameters are used when options.modelFamily is set to COHERE_COMMAND. You can use a value greater than zero for one or the other, but not both.
UNRECOGNIZED_MODEL_PARAMETERS	One or more unrecognized model parameters have been used.
UNRECOGNIZED_OCI_CONFIG_PARAMETERS	One or more unrecognized parameters for OCI configuration have been used.
ONLY_API_SECRET_IS_ACCEPTED	The options.ociConfig.privateKey or options.ociConfig.fingerprint parameters are not NetSuite API secrets.
INVALID_MODEL_FAMILY_VALUE	The options.modelFamily parameter was not set to a valid options. (For example, there may have been a typo in the value or in the enum.)
MODEL_1_DOES_NOT_ACCEPT_PREAMBLE	The options.preamble parameter was provided but the model does not support a preamble.
MODEL_1_DOES_NOT_ACCEPT_DOCUMENTS	The options.documents parameter was provided but the model does not support retrieval-augmented generation (RAG).
MODEL_1_DOES_NOT_ACCEPT_SAFETY_MODE	The options.safetyMode parameter was provided but the model does not support safety mode.
DOCUMENT_IDS_MUST_BE_UNIQUE	Documents provided using the options.documents parameter have duplicate IDs.
INAPPROPRIATE_CONTENT_DETECTED	The response from the LLM included sensitive or inappropriate content that is restricted by the specified safety mode.
INVALID_SAFETY_MODE	The options.safetyMode parameter was not set to a valid option from the llm.SafetyMode enum.
INVALID_MAX_TOKENS_VALUE	The options.modelParameters.maxTokens parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_TEMPERATURE_VALUE	The options.modelParameters.temperature parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_TOP_K_VALUE	The options.modelParameters.topK parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_TOP_P_VALUE	The options.modelParameters.topP parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.

Error Code	Thrown If
INVALID_FREQUENCY_PENALTY_VALUE	The options.modelParameters.frequencyPenalty parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
INVALID_PRESENCE_PENALTY_VALUE	The options.modelParameters.presencePenalty parameter value is incorrect for the model. See Model Parameter Values by LLM for valid values.
MAXIMUM_PARALLEL_REQUESTS_LIMIT_EXCEEDED	The number of parallel requests to the LLM is greater than 5.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateTextStreamed({
5   // preamble is optional for Cohere models
6   preamble: "You are a successful salesperson. Answer in an enthusiastic, professional tone.",
7   prompt: "Hello World!",
8   documents: [doc1, doc2], // create documents using llm.createDocument(options)
9   modelFamily: llm.ModelFamily.COHERE_COMMAND, // uses COHERE_COMMAND when modelFamily is omitted
10  modelParameters: {
11    maxTokens: 1000,
12    temperature: 0.2,
13    topK: 3,
14    topP: 0.7,
15    frequencyPenalty: 0.4,
16    presencePenalty: 0
17  },
18  ociConfig: {
19    // Replace ociConfig values with your Oracle Cloud Account values
20    userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
21    tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
22    compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
23    // Replace fingerprint and privateKey with your NetSuite API secret ID values
24    fingerprint: 'custsecret_oci_fingerprint',
25    privateKey: 'custsecret_oci_private_key'
26  }
27 });
28
29 var iter = response.iterator();
30 iter.each(function(token){
31   log.debug("token.value: " + token.value);
32   log.debug("response.text: " + response.text);
33   return true;
34 })
35 ...
36 // Add additional code

```

llm.generateTextStreamed.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the streamed response from the LLM.
---------------------------	--

	When you're using unlimited usage mode, this method accepts the OCI configuration parameters. You can also specify OCI configuration parameters on the SuiteScript subtab of the AI Preferences page. For more information, see the help topic Using Your Own OCI Configuration for SuiteScript Generative AI APIs.  Note: The parameters and errors thrown for this method are the same as those for <code>llm.generateTextStreamed(options)</code>. For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	llm.generateTextStreamed(options)
Aliases	<code>llm.chatStreamed.promise(options)</code>  Note: These aliases use the same parameters and can throw the same errors as the <code>llm.generateTextStreamed(options)</code> method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/Ilm Module
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3
4 llm.generateTextStreamed.promise({
5   // preamble is optional for Cohere models
6   preamble: "You are a successful salesperson. Answer in an enthusiastic, professional tone.",
7   prompt: "Hello World!",
8   documents: [doc1, doc2], // create documents using llm.createDocument(options)
9   modelFamily: llm.ModelFamily.COHERE_COMMAND, // uses COHERE_COMMAND when modelFamily is omitted
10  modelParameters: {
11    maxTokens: 1000,
12    temperature: 0.2,
13    topK: 3,
14    topP: 0.7,
15    frequencyPenalty: 0.4,
16    presencePenalty: 0
17  },
18  ociConfig: {
19    // Replace ociConfig values with your Oracle Cloud Account values
20    userId: 'ocid1.user.oc1..aaaaaaaanld..exampleuserid',
21    tenancyId: 'ocid1.tenancy.oc1..aaaaaaaabt..exampletenancyid',
22    compartmentId: 'ocid1.compartment.oc1..aaaaaaaaph..examplecompartmentid',
23    // Replace fingerprint and privateKey with your NetSuite API secret ID values
24    fingerprint: 'custsecret_oci_fingerprint',
25    privateKey: 'custsecret_oci_private_key'
26  }
27 }).then(function(result) {
28   log.debug(result)

```

```

29     }).catch(function(reason) {
30         log.debug(reason)
31     })
32 })
33
34 ...
35 // Add additional code

```

Ilm.getRemainingFreeUsage()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the number of free LLM requests remaining for the current month. This method tracks free requests for non-embed API calls, such as Ilm.generateText(options) . To track free requests for embed API calls (such as Ilm.embed(options)), use Ilm.getRemainingFreeEmbedUsage() instead.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/Ilm Module
Since	2024.1

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const usage = Ilm.getRemainingFreeUsage();
5
6 ...
7 // Add additional code

```

Ilm.getRemainingFreeUsage.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the number of free LLM requests remaining for the current month.
---------------------------	---

	 Note: The parameters and errors thrown for this method are the same as those for Ilm.getRemainingFreeUsage() . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	Ilm.getRemainingFreeUsage()
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/Ilm Module
Since	2024.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3
4 Ilm.getRemainingFreeUsage.promise().then(function(result) {
5     // Work with the result
6 });
7
8 ...
9 // Add additional code

```

Ilm.getRemainingFreeEmbedUsage()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the number of free embedding requests remaining for the current month. This method tracks free requests for embed API calls, such as Ilm.embed(options) . To track free requests for non-embed API calls (such as Ilm.generateText(options)), use Ilm.getRemainingFreeUsage() instead.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/Ilm Module

Since	2025.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const embedUsage = ilm.getRemainingFreeEmbedUsage();
5
6 ...
7 // Add additional code

```

Ilm.getRemainingFreeEmbedUsage.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously returns the number of free embedding requests remaining for the current month. Note: The parameters and errors thrown for this method are the same as those for Ilm.getRemainingFreeEmbedUsage(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Ilm.getRemainingFreeEmbedUsage()
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/Ilm Module
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...

```

```

3
4 llm.getRemainingFreeEmbedUsage.promise().then(function(result) {
5     // Work with the result
6 });
7
8 ...
9 // Add additional code

```

Ilm.ChatRole

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	The author of the chat message. Use this enum to set the value of the options.role parameter in Ilm.createChatMessage(options). Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/Ilm Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Since	2024.1

Values

Value	Notes
USER	Identifies the author of the chat message (prompt) sent to the large language model.
CHATBOT	Identifies the author of the chat message (response text) received from the large language model.

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 chatHistory.push({
5     role: Ilm.ChatRole.USER,
6     text: yourMessage
7 });

```

```

8 chatHistory.push({
9   role: llm.ChatRole.CHATBOT,
10  text: chatBotMessage
11 });
12
13 ...
14 // Add additional code

```

llm.EmbedModelFamily

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	The large language model to be used to generate embeddings. Use this enum to set the value of the options.<code>embedModelFamily</code> parameter in <code>llm.embed(options)</code>. Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/Ilm Module
Sibling Module Members	N/Ilm Module Members .
Since	2025.1

Values

Enum	Value	Notes
<code>EmbedModelFamily.COHERE_EMBED</code>	<code>cohere.embed-v4.0</code>	—
<code>EmbedModelFamily.COHERE_EMBED_LATEST</code>	<code>cohere.embed-v4.0</code>	—

Syntax

Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.embed({
5   inputs: "Hello World!",
6   embedModelFamily: llm.EmbedModelFamily.COHERE_EMBED,
7   ...
8 });

```

```

9
10 ...
11 // Add additional code

```

Ilm.ModelFamily

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	The large language model to be used. Use this enum to set the value of the <code>options.modelFamily</code> parameter in <code>Ilm.generateText(options)</code>. Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/Ilm Module
Sibling Module Members	N/Ilm Module Members .
Since	2024.2

Values

The following table lists the models that are supported when using `Ilm.generateText(options)` and `Ilm.generateTextStreamed(options)` (and their promise versions). The Supported Capabilities column indicates which of the following features are supported by each model:

- Retrieval-augmented generation (RAG)** – You can provide documents with text content (in `Ilm.Document` objects) using the `options.documents` parameter. These documents provide additional context to the LLM, and the LLM uses information in these documents to augment its response using RAG. It returns `Ilm.Citation` objects that represent the content from source documents where the LLM found relevant information for its response. For more information about RAG, see [What Is Retrieval-Augmented Generation \(RAG\)?](#) in the OCI documentation.
- Preambles** – You can provide a preamble using the `options.preamble` parameter. A preamble provides initial context or a guiding message to the LLM. For more information about using a preamble, refer to [Pretrained Foundational Models in Generative AI](#) in the *Oracle Cloud Infrastructure Documentation*.

Enum	Value	Supported Capabilities		Notes
		RAG	Preambles	
<code>ModelFamily.COHERE_COMMAND</code>	<code>cohere.command-a-03-2025</code>	Yes	Yes	—
<code>ModelFamily.COHERE_COMMAND_LATEST</code>	<code>cohere.command-a-03-2025</code>	Yes	Yes	—

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: "Hello World!",
6   modelFamily: llm.ModelFamily.COHERE_COMMAND,
7   ...
8 });
9
10 ...
11 // Add additional code

```

llm.SafetyMode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	The safety mode to be used for LLM requests. Safety mode is available for Cohere models only and is designed to help filter and moderate content generated by the LLM. When using strict mode or contextual mode, the LLM may refuse to provide certain responses that include sensitive, harmful, or illegal suggestions. Use this enum to set the value of the <code>options.safetyMode</code> parameter in llm.generateText(options) and llm.generateTextStreamed(options). Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/Ilm Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Since	2025.1

Values

Enum	Value	Notes
<code>SafetyMode.CONTEXTUAL</code>	CONTEXTUAL	This mode offers a less restrictive approach than strict mode while still rejecting harmful or illegal suggestions. This mode is suited for creative, entertainment, or academic purposes.
<code>SafetyMode.OFF</code>	OFF	This mode removes the restrictions for the LLM's response and may return unsafe, risky, or illegal suggestions.

Enum	Value	Notes
		Some content filtering is applied even when safety mode is off, and you could receive an <code>INAPPROPRIATE_CONTENT_DETECTED</code> error if you ask the LLM to provide content that is deemed inappropriate, unsafe, or illegal.  Warning: Use caution when using this mode and always review the responses for suitability.
<code>SafetyMode.STRICT</code>	STRICT	This mode aims to avoid sensitive topics entirely and is suited for corporate communications and customer service. This is the default mode when calling <code>llm.generateText(options)</code> or <code>llm.generateTextStreamed(options)</code>.

Syntax

 **Important:** The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/Ilm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.generateText({
5   prompt: "Hello world!",
6   safetyMode: llm.SafetyMode.CONTEXTUAL
7 });
8
9 ...
10 // Add additional code

```

llm.Truncate

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Enum Description	The truncation method to use when embeddings input exceeds 512 tokens. Use this enum to set the value of the options .truncate parameter in <code>llm.embed(options)</code>.  Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/Ilm Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Since	2025.1
-------	--------

Values

Enum	Value	Notes
Truncate.END	END	Truncates the embeddings input from the end of the input string.
Truncate.NONE	NONE	Doesn't truncate the embeddings input.
Truncate.START	START	Truncates the embeddings input from the start of the input string.

Syntax



Important: The following code sample shows the syntax for this member. It isn't a functional example. For a complete script example, see [N/llm Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const response = llm.embed({
5   inputs: ["Hello World!"],
6   truncate: llm.Truncate.START,
7   ...
8 });
9
10 ...
11 // Add additional code

```

N/log Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/log module to manually access methods for logging script execution details. These methods can also be accessed using the global log object. For more information about the global log object, see [log Object](#).

Log messages appear on the Execution Log subtab of the script deployment for deployed scripts, or on the Execution Log subtab of the SuiteScript Debugger if you are debugging a script. Log messages also appear on the Script Execution Logs page at Customization > Scripting > Script Execution Logs.

Go To Samples

In This Help Topic

- [N/log Module Guidelines](#)
- [Using Log Levels](#)
- [Viewing Script Execution Logs](#)
- [N/log Module Members](#)

N/log Module Guidelines

The following guidelines are provided for use of the N/log module:

- NetSuite governs the amount of logging that can be done in any specific 60 minute time period. A company is allowed to make up to 100,000 log object method calls across **all** of their scripts. Script owners are notified if NetSuite detects that one script is logging excessively and automatically adjusts the log level.
- NetSuite purges logs older than **30 days**.
- The Server Script Log search and the Execution Log subtab of script records and script deployment records have a log storage limit of 5 million per database. Because log persistence is not guaranteed, You should use custom records if you want to store script execution logs for extended periods. The Script Execution Logs page at Customization > Scripting > Script Execution Logs does not share the same database limit.
- The Execution Log subtab also lists notes returned by NetSuite such as error messages. For more information, see [N/error Module](#).
- If you deploy a client script to a form using [Form.clientScriptFileId](#) or [Form.clientScriptModulePath](#), using the N/log module adds the logs to the deployment of the parent script. The parent script can be either a beforeLoad user event script or a [SuiteScript 2.x Suitelet Script Type](#).
- If a client script uses the N/log module and it is attached to a form (see the help topic [SuiteScript 2.x Form-Level Script Deployments](#)), calls to any log function (log.audit, log.debug, log.emergency, log.error) are ignored. The script executes successfully, however, nothing will be logged. To log messages in a client script that is attached to a form, use the console.log function.
- When an object (that is not a string) is passed to a log object method, NetSuite runs [JSON.stringify\(obj\)](#) on any values that are passed as the details parameter and equal a JavaScript object.

```

1 | ...
2 | // log.debug(myRecord) //Shows the JSON representation of the current values in the myRecord object
3 | var id = myRecord.save();
4 | ...

```

Using Log Levels

Use the log methods along with the Log Level field on the script deployment to specify which log messages are written to the Execution Log on the script deployment. This is useful during the debugging of a script or for providing useful execution notes for auditing or tracking purposes.

Log levels on the script deployment and N/log module methods act as a filter on the amount of information logged. The following Log Levels are supported:

Log Level	Example Uses
Debug	Use Debug to show all messages. This type of logging is suitable only for testing scripts. To avoid excessive logging, don't use Debug level for active scripts in production.
Audit	Use Audit to shows a record of events that have occurred during the processing of the script (for example, "A request was made to an external site.").
Error	Use Error to show only unexpected script errors.
Emergency	Use Emergency to show only the most critical errors in the script log.

The following table shows you which type of log messages are written based on the Log Level field selected on the script deployment and the log method used in your script.

Log Level	Log method used in your script			
	log.emergency(options)	log.error(options)	log.audit(options)	log.debug(options)
Emergency	✓	—	—	—
Error	✓	✓	—	—
Audit	✓	✓	✓	—
Debug	✓	✓	✓	✓

Viewing Script Execution Logs

Note: Log messages written in client scripts that are attached to a form are viewed in the browser debug console tab when the client script runs on the form. Log messages written in server scripts and client scripts (that are deployed at the record level) are viewed in NetSuite.

Log messages for a specific script are shown on the Execution Log of the script deployment for the script. These logs are not guaranteed to persist for 30 days and may be purged to enhance performance if volume is high.

To view script execution log details for server scripts, go to Customization > Scripting > Script Execution Logs. This list of script execution logs is an enhanced repository that stores all log details for 30 days.

On this page, you can perform the following tasks:

- Search for specific logs using filter options, such as log level, execution date range, and script name.
- Download the list as a CSV file or an Excel spreadsheet.
- Print the list.

When you debug a script in the SuiteScript Debugger, log messages appear on the Execution Log subtab of the SuiteScript Debugger and do not appear on any Execution Log subtab of any script deployment.

N/log Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	log.audit(options)	void	Client* and server scripts	Logs an Audit message.
	log.debug(options)	void	Client* and server scripts	Logs a Debug message.
	log.emergency(options)	void	Client* and server scripts	Logs an Emergency message.
	log.error(options)	void	Client* and server scripts	Logs an Error message.

* If a client script uses the N/log module and it is attached to a form (see the help topic [SuiteScript 2.x Form-Level Script Deployments](#)), calls to any log function (log.audit, log.debug, log.emergency, log.error) are ignored. The script executes successfully, however, nothing will be logged. To log messages in a client script that is attached to a form, use the console.log function. You can use N/log module methods in client scripts that are deployed at the record level or are called from a server script.

N/log Module Script Sample

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/log module.

Create Debug Log Messages

The following sample shows how to create each type of log messages.

i Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  // This script creates each type of log message.
7  define(['N/log'],function(log) {
8      function beforeLoad(context) {
9          var myValue = 'value';
10
11          var myObject = {
12              name: 'Jane',
13              id: '123'
14          };
15
16          // An audit log message
17          log.audit({
18              title: 'Audit Entry',
19              details: myObject
20          });
21
22          // A debug log message
23          log.debug({
24              title: 'Debug Entry',
25              details: 'Value of myValue is: ' + myValue
26          });
27
28          // An emergency log message
29          log.emergency({
30              title: 'Emergency Entry',
31              details: 'Value of myValue is: ' + myValue
32          });
33
34          // An error log message
35          log.error({
36              title: 'Error Entry',
37              details: 'Value of myValue is: ' + myValue
38          });
39      }
40      return {
41          beforeLoad: beforeLoad

```

```

42 | };
43 | });

```

log.audit(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Logs an Audit message. Audit messages appear on the Execution Log subtab if the Log Level on the script deployment is set to Audit or Debug. Use this method for scripts in production.
Returns	void
Supported Script Types	Client and server scripts; except client scripts that are deployed by attaching to a form (use console.log instead). For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Amount of logging in any 60 minute period is limited. See N/log Module Guidelines .
Module	N/log Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The text to appear in the Title column on the Execution Log subtab of the script deployment. Maximum length is 99 characters. If you set this value to null, an empty string ("), or omit it, the word "Untitled" appears for the log entry.	2016.1
options.details	any	optional	You can pass any value for this parameter. If the value is a JavaScript Object type, JSON.stringify(obj) is called on the object before displaying the value. NetSuite truncates any resulting string over 3999 characters.	2016.1

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/log Module Script Sample](#).

```
1 | //Add additional code
```

```

2  ...
3  var myValue = 'value';
4  log.audit({
5      title: 'Audit Entry',
6      details: 'Value of myValue is: ' + myValue
7  });
8  ...
9  //Add additional code

```

log.debug(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Logs a Debug message. Debug messages appear on the Execution Log subtab only if the Log Level on the script deployment is set to Debug. Use this method for scripts in development.
Returns	void
Supported Script Types	Client and server scripts; except client scripts that are deployed by attaching to a form (use console.log instead). For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Amount of logging in any 60 minute period is limited. See N/log Module Guidelines .
Module	N/log Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The text to appear in the Title column on the Execution subtab of the script deployment. Maximum length is 99 characters. If you set this value to null, an empty string ("), or omit it, the word "Untitled" appears for the log entry.	2016.1
options.details	any	optional	The text of the log message, that can include strings, variables, objects, etc. If the value is a JavaScript object type, JSON.stringify(obj) is called on the object before displaying the value. NetSuite truncates any resulting string over 3999 characters.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/log Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var myValue = 'value';
4 log.debug({
5     title: 'Debug Entry',
6     details: 'Value of myValue is: ' + myValue
7 });
8 ...
9 //Add additional code

```

log.emergency(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Logs an Emergency message. Emergency messages appear on the Execution subtab only if the Log Level on the script deployment is set to Audit, Debug, Error, or Emergency. In other words, emergency messages always appear. Use this method for scripts in production.
Returns	void
Supported Script Types	Client and server scripts; except client scripts that are deployed by attaching to a form (use console.log instead). For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Amount of logging in any 60 minute period is limited. See N/log Module Guidelines .
Module	N/log Module
Since	2016.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The text to appear in the Title column on the Execution Log subtab of the script deployment. Maximum length is 99 characters. If you set this value to null, an empty string (""), or omit it, the word "Untitled" appears for the log entry.	2016.1
options.details	any	optional	The text of the log message, that can include strings, variables, objects, etc.	2016.1

Parameter	Type	Required / Optional	Description	Since
			If the value is a JavaScript Object type, <code>JSON.stringify(obj)</code> is called on the object before displaying the value. NetSuite truncates any resulting string over 3999 characters.	

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/log Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var myValue = 'value';
4 log.emergency({
5     title: 'Emergency Entry',
6     details: 'Value of myValue is: ' + myValue
7 });
8 ...
9 //Add additional code

```

log.error(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Logs an Error message. Error messages appear on the Execution subtab only if the Log Level on the script deployment is set to Audit, Debug, or Error. Use this method for scripts in production.
Returns	void
Supported Script Types	Client and server scripts; except client scripts that are deployed by attaching to a form (use <code>console.log</code> instead). For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Amount of logging in any 60 minute period is limited. See N/log Module Guidelines .
Module	N/log Module
Since	2016.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The text to appear in the Title column on the Execution Log subtab of the script deployment.	2016.1

Parameter	Type	Required / Optional	Description	Since
			Maximum length is 99 characters. If you set this value to null, an empty string ("), or omit it, the word "Untitled" appears for the log entry.	
options.details	any	optional	The text of the log message, that can include strings, variables, objects, etc. If the value is a JavaScript object type, <code>JSON.stringify(obj)</code> is called on the object before displaying the value. NetSuite truncates any resulting string over 3999 characters.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/log Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var myValue = 'value';
4 log.error({
5     title: 'Error Entry',
6     details: 'Value of myValue is: ' + myValue
7 });
8 ...
9 //Add additional code

```

N/machineTranslation Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Use the N/machineTranslation module to translate text into supported languages using generative AI. This module uses the Oracle Cloud Infrastructure (OCI) Language service to translate text in documents you provide. For more information about this service, see [Language](#) in the OCI documentation.

Go To Samples {..}

You can also use the [N/llm Module](#) to translate text by providing a suitable prompt when calling generative AI methods (such as `llm.generateText(options)`). However, using the N/machineTranslation module provides several benefits that make it a better choice for most translation use cases:

- **Reduced cost** – The N/machineTranslation module supports unlimited translation requests, while the N/llm module provides only a limited number of LLM requests per month. When using the N/machineTranslation module, you don't need to provide Oracle Cloud Infrastructure (OCI) credentials to get unlimited usage, which helps to reduce the cost of your solutions.
- **No prompt engineering required** – When you use the N/machineTranslation module, you don't need to write a prompt to generate translations. The module detects the source language automatically and translates provided documents into the language you specify in your request.

- **More straightforward limits** – The N/machineTranslation module limits the length of each provided document to 5,000 characters, and it also limits the total length of all provided documents to 20,000 characters. The N/llm module provides limits that are based on tokens and the context window of the LLM you use, which can be more difficult to estimate.
- **More supported languages** – The N/machineTranslation module can translate text into additional languages that may not be supported by LLMs and the N/llm module.
- **More reliable translations** – The N/machineTranslation module uses the OCI Language service, which is designed for specific natural language processing (NLP) tasks, including translation. It uses pretrained models that can offer more predictable and reliable translation results compared to the N/llm module. The N/llm module uses the OCI Generative AI service, which is designed for content generation and may provide more varied translation results compared to a dedicated translation service.

This module is available in NetSuite by default when the Server SuiteScript feature is enabled. For more information, see the help topic [Enabling Features](#).

In This Help Topic

- [N/machineTranslation Module Members](#)
- [Document Object Members](#)
- [Error Object Members](#)
- [Response Object Members](#)

N/machineTranslation Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	machineTranslation.Document	Object	Server scripts	A document with untranslated text (to provide to machineTranslation.translate(options) for translation) or translated text (when part of a machineTranslation.Response object returned from the translation service).
	machineTranslation.Error	Object	Server scripts	An error returned from the translation service.
	machineTranslation.Response	Object	Server scripts	The response from the translation service, which contains machineTranslation.Document objects with translated text.
Method	machineTranslation.createDocument(options)	Object	Server scripts	Creates a document to be used as source content when calling the translation service.
	machineTranslation.translate(options)	Object	Server scripts	Translates a set of documents into the specified language.
	machineTranslation.translate.promise(options)	Promise	Server scripts	Asynchronously translates a set of documents into the specified language.
Enum	machineTranslation.Language	enum	Server scripts	Holds string values for the source language (when creating a document) or target language (when calling machineTranslation.translate(options)).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				Use this enum to set the options. <code>language</code> parameter in machineTranslation.createDocument(options) or the <code>options.targetLanguage</code> parameter in machineTranslation.translate(options) .

Document Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Document.id	string	Server scripts	The ID of the document.
	Document.language	string	Server scripts	The language of the document.
	Document.text	string	Server scripts	The content of the document.

Error Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Error.documentId	string	Server scripts	The ID of the document in which the error occurred.
	Error.message	string	Server scripts	The error message returned from the translation service.

Response Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Response.errors	machineTranslation.Error[]	Server scripts	The errors returned from the translation service.
	Response.results	machineTranslation.Document[]	Server scripts	The translated documents returned from the translation service.

N/machineTranslation Module Script Samples

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

The following script samples demonstrate how to use the features of the N/machineTranslation module.

- [Translate a Document](#)

Translate a Document

The following sample creates a document and provides it to [machineTranslation.translate\(options\)](#) for translation. The sample specifies the source language of the document, but this step is optional. If you don't specify the source language of a document, the translation service detects the language automatically.

For instructions about how to run a SuiteScript 2.1 code snippet in the debugger, see the help topic [On-Demand Debugging of SuiteScript 2.1 Scripts](#).

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2  * @NApiVersion 2.1
3  */
4  require(['N/machineTranslation'],
5    function(mt) {
6      const myDocument = mt.createDocument({
7        id: 'myDoc',
8        text: 'Hello everyone! How are you today?',
9        language: mt.Language.ENGLISH
10     });
11
12     const translationResults = mt.translate({
13       documents: [myDocument],
14       targetLanguage: mt.Language.CZECH
15     });
16
17     // Work with the translated document
18     log.debug('Translated text', translationResults.results[0].text);
19   }
20 );

```

machineTranslation.Document

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	A document returned from machineTranslation.createDocument(options) or machineTranslation.translate(options). A document represents text that you send to or receive from the translation service. This object includes properties for the document ID (Document.id), document language (Document.language), and document text (Document.text). When you create a document for translation using machineTranslation.createDocument(options), you can specify the source language of the document. If you don't specify the source language, the translation service detects the language automatically. Note: For best translation results, each document should contain text in a single language only. The translation service supports documents that use multiple languages, but the accuracy and completeness of the resulting translation may vary. When passing documents to machineTranslation.translate(options) for translation, keep the following considerations in mind:
---------------------------	---

	■ All document IDs must be unique. The maximum length for a single document is 5,000 characters. ■ The overall maximum length for all documents is 20,000 characters. ■ You can't pass an empty document (one where the Document.text property is empty). ■ If you pass a document that doesn't specify its source language (one where the Document.language property is null or undefined), the translation service detects the source language automatically.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Methods and Properties	Document Object Members
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument1 = machineTranslation.createDocument({
5   id: 'myDoc1',
6   text: 'This is a document to be translated.'
7 });
8
9 const myDocument2 = machineTranslation.createDocument({
10  id: 'myDoc2',
11  text: 'This is another document to be translated.'
12 });
13
14 const translationResults = machineTranslation.translate({
15   documents: [myDocument1, myDocument2],
16   targetLanguage: machineTranslation.Language.CZECH
17 });
18
19 ...
20 // Add additional code

```

Document.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The ID of the document. When passing documents to machineTranslation.translate(options) , all document IDs must be unique.
Type	string
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Document
Sibling Object Members	Document Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10   documents: [myDocument],
11   targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 const docId = translationResults.results[0].id;
15
16 ...
17 // Add additional code

```

Document.language

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	The language of the document. When passing a document to machineTranslation.translate(options), this property represents the source language of the document. If this property is null or undefined, the translation service detects the source language automatically. When receiving a document from the translation service as part of a machineTranslation.Response object, this property represents the language the document was translated into.
Type	string
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Document
Sibling Object Members	Document Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10  documents: [myDocument],
11  targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 const docLanguage = translationResults.results[0].language;
15
16 ...
17 // Add additional code

```

Document.text

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text of the document. When passing a document to machineTranslation.translate(options), this property represents the text to be translated. The maximum length of this property is 5,000 characters.  Note: For best translation results, each document should contain text in a single language only. The translation service supports documents that use multiple languages, but the accuracy and completeness of the resulting translation may vary. When receiving a document from the translation service as part of a machineTranslation.Response object, this property represents the translated text.
-----------------------------	---

Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Document
Sibling Object Members	Document Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10  documents: [myDocument],
11  targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 const docText = translationResults.results[0].text;
15
16 ...
17 // Add additional code

```

machineTranslation.Error

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	An error returned from the translation service when calling machineTranslation.translate(options). The translation service returns an error if a provided document couldn't be translated. An error might occur if the text to translate isn't formatted correctly (for example, if it contains unrecognized characters). This object includes properties for the ID of the document that the error relates to (Error.documentId) and the text of the error message (Error.message).
---------------------------	--

	When passing documents to machineTranslation.translate(options) , this object is included in the returned machineTranslation.Response object (in the Response.errors property) if an error occurred while translating the documents. If no errors occurred, the Response.errors property is empty.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Methods and Properties	Error Object Members
Since	2025.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10   documents: [myDocument],
11   targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 if (translationResults.errors.length) {
15   // Handle the error message
16   const docErrorId = translationResults.errors[0].documentId;
17   const docErrorMessage = translationResults.errors[0].message;
18 }
19
20 ...
21 // Add additional code

```

Error.documentId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The ID of the document that the error relates to.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Error

Sibling Object Members	Error Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10   documents: [myDocument],
11   targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 if (translationResults.errors.length) {
15   // Handle the error message
16   const docErrorId = translationResults.errors[0].documentId;
17   const docErrorMessage = translationResults.errors[0].message;
18 }
19
20 ...
21 // Add additional code

```

Error.message

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	The text of the error message.
Type	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Error
Sibling Object Members	Error Object Members

Since	2025.1
-------	--------

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10  documents: [myDocument],
11  targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 if (translationResults.errors.length) {
15   // Handle the error message
16   const docErrorId = translationResults.errors[0].documentId;
17   const docErrorMessage = translationResults.errors[0].message;
18 }
19
20 ...
21 // Add additional code

```

machineTranslation.Response

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	A response returned from machineTranslation.translate(options). A response represents the translation results from the translation service. This object includes properties for the translated documents (Response.results) and any errors that occurred during translation (Response.errors). If no errors occurred, the Response.errors property is empty.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/machineTranslation Module
Methods and Properties	Response Object Members

Since	2025.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10  documents: [myDocument],
11  targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 ...
15 // Add additional code

```

Response.errors

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The errors returned from the translation service. The translation service returns an error if a provided document couldn't be translated. An error might occur if the text to translate isn't formatted correctly (for example, if it contains unrecognized characters). This property is an array of machineTranslation.Error objects, each of which represents an error that occurred when translating one of the provided documents. If no errors occurred, this property is empty.
Type	machineTranslation.Error[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Response
Sibling Object Members	Response Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10  documents: [myDocument],
11  targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 if (translationResults.errors.length) {
15   // Handle the error message
16   const docErrorId = translationResults.errors[0].documentId;
17   const docErrorMessage = translationResults.errors[0].message;
18 }
19
20 ...
21 // Add additional code

```

Response.results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The translation documents. This property is an array of machineTranslation.Document objects, each of which represents a translated document.
Type	machineTranslation.Document[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/machineTranslation Module
Parent Object	machineTranslation.Response
Sibling Object Members	Response Object Members
Since	2025.1

Errors

Error Code	Thrown If
READ_ONLY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument = machineTranslation.createDocument({
5   id: 'myDoc',
6   text: 'This is a document to be translated.'
7 });
8
9 const translationResults = machineTranslation.translate({
10   documents: [myDocument],
11   targetLanguage: machineTranslation.Language.CZECH
12 });
13
14 // Work with the result
15 const resultText = translationResults.results[0].text;
16
17 ...
18 // Add additional code

```

machineTranslation.createDocument(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a document with the specified ID, source language, and text content. A document represents text to translate using machineTranslation.translate(options). When you create a document for translation, you can specify the source language of the document. If you don't specify the source language, the translation service detects the language automatically. Note: For best translation results, each document should contain text in a single language only. The translation service supports documents that use multiple languages, but the accuracy and completeness of the resulting translation may vary. When passing documents to machineTranslation.translate(options) for translation, keep the following considerations in mind: ■ All document IDs must be unique. ■ The maximum length for a single document is 5,000 characters. ■ The overall maximum length for all documents is 20,000 characters. ■ You can't pass an empty document (one where the Document.text property is empty). ■ If you pass a document that doesn't specify its source language (one where the Document.language property is null or undefined), the translation service detects the source language automatically.
Returns	machineTranslation.Document
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/machineTranslation Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The ID of the document.	2025.1
options.text	string	required	The text of the document.	2025.1
options.language	string	optional	The language of the document. Use values from the machineTranslation.Language enum to set the value of this parameter. If you don't specify the document language, the translation service detects the language automatically when you pass the document to machineTranslation.translate(options) .	2025.1

Errors

Error Code	Thrown If
INVALID_LANGUAGE	The options.language parameter uses a value that isn't part of the machineTranslation.Language enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument1 = machineTranslation.createDocument({
5   id: 'myDoc1',
6   text: 'This is a document to be translated.'
7 });
8
9 const myDocument2 = machineTranslation.createDocument({
10  id: 'myDoc2',
11  text: 'This is another document to be translated.'
12 });
13
14 const translationResults = machineTranslation.translate({
15   documents: [myDocument1, myDocument2],
16   targetLanguage: machineTranslation.Language.CZECH
17 });
18
19 ...

```

20 // Add additional code

machineTranslation.translate(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Translates the provided documents into the specified language. Unlike methods in the N/Im Module, this method supports unlimited translation requests and doesn't use a shared free usage pool. You also don't need to provide Oracle Cloud Infrastructure (OCI) credentials to get unlimited usage. When passing documents (created using machineTranslation.createDocument(options)) to this method for translation, keep the following considerations in mind: ■ All document IDs must be unique. ■ The maximum length for a single document is 5,000 characters. ■ The maximum total length for all documents is 20,000 characters. ■ You can't pass an empty document (one where the Document.text property is empty). ■ If you pass a document that doesn't specify its source language (one where the Document.language property is null or undefined), the translation service detects the source language automatically. Note: For best translation results, each document should contain text in a single language only. The translation service supports documents that use multiple languages, but the accuracy and completeness of the resulting translation may vary. As a best practice, you should create documents using machineTranslation.createDocument(options) and pass those documents to this method. However, you can also provide an ad hoc object that includes values for the <code>id</code> and <code>text</code> properties, as shown in the following example: <pre> 1 machineTranslation.translate({ 2 targetLanguage: machineTranslation.Language.GERMAN, 3 documents: [4 { 5 id: '1', 6 text: 'How are you?' 7 } 8] 9 }); </pre>
Returns	machineTranslation.Response
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100
Module	N/machineTranslation Module
Since	2025.1

Parameters

Parameter	Type	Required / Optional	Description	Since
options.documents	machineTranslation.Document[]	required	The documents to translate.	2025.1
options.targetLanguage	string	required	The language to translate the specified documents into. Use values from the machineTranslation.Language enum to set the value of this parameter.	2025.1
options.timeout	number	optional	The timeout period to wait for a response from the translation service, in milliseconds. The default value is 30,000 (30 seconds).	2025.1

Errors

Error Code	Thrown If
DOCUMENT_CANNOT_BE_EMPTY	One of the documents specified in the options.documents parameter is empty.
DOCUMENT_IDS_MUST_BE_UNIQUE	Two or more documents specified in the options.documents parameter have duplicate IDs.
DOCUMENT_TOO_LARGE	One of the documents specified in the options.documents parameter is longer than 5,000 characters.
INPUT_TOO_LARGE	The total length of all documents specified in the options.documents parameter is longer than 20,000 characters.
INVALID_LANGUAGE	One of the documents specified in the options.documents parameter uses a language that isn't part of the machineTranslation.Language enum.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument1 = machineTranslation.createDocument({
5   id: 'myDoc1',
6   text: 'This is a document to be translated.'
7 });
8
9 const myDocument2 = machineTranslation.createDocument({
10  id: 'myDoc2',
11  text: 'This is another document to be translated.'
12 });
13
14 const translationResults = machineTranslation.translate({

```

```

15     documents: [myDocument1, myDocument2],
16     targetLanguage: machineTranslation.Language.CZECH
17   });
18
19   ...
20   // Add additional code

```

machineTranslation.translate.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Asynchronously translates the provided documents into the specified language. Note: The parameters and errors thrown for this method are the same as those for machineTranslation.translate(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	machineTranslation.translate(options)
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100
Module	N/machineTranslation Module
Since	2025.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1   // Add additional code
2   ...
3
4   const myDocument1 = machineTranslation.createDocument({
5     id: 'myDoc1',
6     text: 'This is a document to be translated.'
7   });
8
9   const myDocument2 = machineTranslation.createDocument({
10    id: 'myDoc2',
11    text: 'This is another document to be translated.'
12  });
13
14  machineTranslation.translate.promise({
15    documents: [myDocument1, myDocument2],
16    targetLanguage: machineTranslation.Language.CZECH
17  }).then(function(result) {
18    log.debug(result)
19  }).catch(function(reason) {
20    log.debug(reason)

```



```
21 | });  
22 |  
23 | ...  
24 | // Add additional code
```

machineTranslation.Language

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	The language to translate documents to or from. Use this enum to set the value of the following properties: - options.targetLanguage in machineTranslation.translate(options) - options.language in machineTranslation.createDocument(options) Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.
Module	N/machineTranslation Module
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Since	2025.1

Values

Value
ARABIC
BRAZILIAN_PORTUGUESE
CANADIAN_FRENCH
CROATIAN
CZECH
DANISH
DUTCH
ENGLISH
FINNISH
FRENCH
GERMAN

Value
GREEK
HEBREW
HUNGARIAN
ITALIAN
JAPANESE
KOREAN
NORWEGIAN
POLISH
PORTUGUESE
ROMANIAN
RUSSIAN
SIMPLIFIED_CHINESE
SLOVAK
SLOVENIAN
SPANISH
SWEDISH
THAI
TRADITIONAL_CHINESE
TURKISH
VIETNAMESE

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/machineTranslation Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const myDocument1 = machineTranslation.createDocument({
5   id: 'myDoc1',
6   text: 'This is a document to be translated.'
7 });
8
9 const myDocument2 = machineTranslation.createDocument({
10  id: 'myDoc2',
11  text: 'This is another document to be translated.'
12 });
13
14 const translationResults = machineTranslation.translate({
15   documents: [myDocument1, myDocument2],
16   targetLanguage: machineTranslation.Language.CZECH

```

```

17  });
18
19  ...
20  // Add additional code

```

N/pgp Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Use the N/pgp module to enable secure messaging, file encryption, and document signing. Based on [OpenPGP](#) encryption standards.

Important: To use the N/pgp Module, you must first generate PGP keys from [GnuPG](#), [OpenPGP](#), or a third party source that supports pgp key generation. The generated keys must be stored in Secrets Management to securely manage and reference the keys. To store your generated keys, go to Setup > Company > API Secrets to create a new secret key.

For more information about API Secrets in NetSuite, see the help topic [Secrets Management](#).

If you are new to PGP, use the following resources to learn more:

- [RFC 4880 OpenPGP Message Format](#) – Provides background information about the PGP standard.
- [GnuPG Manual](#) – A reliable resource for practical information.
- [OpenPGP](#) – Learn more about the pgp standards with documentation resources for developers.

Limitations of N/pgp

As you are working with the N/pgp module, consider the following limitations:

- You cannot generate, modify, or inspect PGP keys using the N/pgp module. You must generate keys from a third party source that supports PGP key generation.
- You cannot create a message without readable PGP software.
- You are limited to strings.
- You are limited to data that fits into memory.

Go To Samples {…}

In This Help Topic

- [N/pgp Module Members](#)
- [Config Object Members](#)
- [KeyId Object Members](#)
- [Message Object Members](#)
- [MessageData Object Members](#)
- [Verification Object Members](#)

■ VerificationSignature Object Members

N/pgp Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	pgp.Config	Object	Server scripts	General configuration options that can be used for message decryption.
	pgp.Key	Object	Server scripts	Cryptographic keys and its metadata.
	pgp.KeyId	Object	Server scripts	An octet scalar that identifies a subkey.
	pgp.Message	Object	Server scripts	Processed PGP data.
	pgp.MessageData	Object	Server scripts	Message data.
	pgp.Verification	Object	Server scripts	Verification results.
	pgp.VerificationSignature	Object	Server scripts	A verification result for a single signature.
Method	pgp.createConfig(options)	pgp.Config	Server scripts	Creates a new configuration object.
	pgp.createMessageData(options)	pgp.MessageData	Server scripts	Creates new message data.
	pgp.createSigner(options)	certificate.Signer	Server scripts	Creates a certificate.Signer object for signing plain strings.
	pgp.createVerification()	pgp.Verification	Server scripts	Creates an empty verification object.
	pgp.loadKeyFromSecret(options)	pgp.Key	Server scripts	Loads a key whose contents are stored securely in secret.
	pgp.parseMessage(options)	pgp.Message	Server scripts	Parses a PGP message.
	pgp.parseKey(options)	pgp.Key	Server scripts	Parses an existing PGP key.
Enum	pgp.CompressionAlgorithm	Enum	Server scripts	Holds the values for available compression algorithms. Use this enum to set the value of the options.compressionAlgorithm parameter of the MessageData.encrypt(options) method.
	pgp.Format	Enum	Server scripts	Literal data packet type. Use this enum to set the value for the options.format parameter of the pgp.createMessageData(options) method.

Config Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Config.allowInsecureDecryptionWithSigningKeys	boolean	Server scripts	Enables decryption that is not secured with signing keys.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Config.allowMessagesWithoutIntegrityProtection	boolean	Server scripts	Allows messages without integrity protection.
	Config.useRelaxedSignatureParsing	boolean	Server scripts	Relaxed signature parsing for configuration objects.

KeyId Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	KeyId.asHex()	string	Server scripts	Returns a key ID as a hexadecimal string.

Message Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Message.type	boolean	Server Scripts	Message type that specifies how a message is processed.
Method	Message.asArmored()	string	Server scripts	Converts a message to ASCII armored format.
	Message.toMessageData()	pgp.MessageData	Server scripts	Converts a message to message data without processing. Works only if the message is not encrypted.
	Message.decrypt(options)	pgp.MessageData	Server scripts	Decrypts a message and optionally verifies the signatures.

MessageData Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	MessageData.filename	string	Server scripts	The name of a file.
	MessageData.date	Date	Server scripts	The date of a message or modification date of the file.
	MessageData.format	pgp.Format	Server scripts	Literal data packet type.
Method	MessageData.getText()	string	Server scripts	Extracts the contents of a message as text.
	MessageData.toMessage()	pgp.Message	Server scripts	Creates a message with no signature, compression, or encryption.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	MessageData.encrypt (options)	pgp.Message	Server scripts	Creates a message that is encrypted and optionally signed.

Verification Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	VerificationSignature.verified	null boolean	Server scripts	Indicates whether verification was successful.
	Verification.signatures	null Array<Verification Signature>	Server scripts	A list of individual verifications, one per signature.

VerificationSignature Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	VerificationSignature.key Id	pgp.KeyId	Server Scripts	ID of the (sub)key that was used for signing.
	VerificationSignature.date Signed	Date	Server Scripts	Date when the message was signed.
	VerificationSignature.verified	boolean	Server scripts	Indicates whether verification was successful for a signature.
	VerificationSignature.problems	string[]	Server scripts	A list of problems for verification signatures.

N/pgp Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/pgp module.



Important: To use the N/pgp Module, you must first generate PGP keys from [GnuPG](#), [OpenPGP](#), or a third party source that supports pgp key generation. The generated keys must be stored in Secrets Management to securely manage and reference the keys. To store your generated keys, go to Setup > Company > API Secrets to create a new secret key. You will need at least three secrets to use this module.

For more information about API Secrets in NetSuite, see the help topic [Secrets Management](#).

- [Send a Message](#)
- [Receive a Message](#)

- [Send a Message to Multiple Receivers](#)
- [Use a Cryptographic Key for a Signature](#)

Send a Message

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to encrypt, sign, and format a message as an ASCII armored string. The armored string is then sent in an email using the [N/email Module](#).

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types and Entry Points](#).

i Note: Some of the values in this sample are placeholders, such as the `senderId` and `recipientEmail` values. Before using this sample, replace all hard-coded values, including IDs and secrets, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/pgp', 'N/email'], (pgp, email) => {
6    const keys = {
7      ours: {
8        pub: pgp.loadKeyFromSecret({
9          secret: { scriptId: 'custsecret_pgp_key_ours_public' }
10        }),
11        pri: pgp.loadKeyFromSecret({
12          secret: { scriptId: 'custsecret_pgp_key_ours_private' },
13          password: { scriptId: 'custsecret_pgp_key_ours_private_password' }
14        })
15      }
16    }
17
18    const data = pgp.createMessageData({
19      content: 'Hello, world!'
20    })
21    const message = data.encrypt({
22      encryptionKeys: keys.ours.pub,
23      signingKeys: keys.ours.pri
24    })
25    const payload = message.asArmored()
26
27    const senderId = -5
28    const recipientId = 'notify@myCompany.com'
29    email.send({
30      author: senderId,
31      recipients: recipientId,
32      subject: 'Test PGP',
33      body: 'Payload: ' + payload
34    })
35  })

```

Receive a Message

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to decrypt and verify a message.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/pgp', 'N/email'], (pgp, email) => {
6    const keys = {
7      ours: {
8        pub: pgp.loadKeyFromSecret({
9          secret: { scriptId: 'custsecret_pgp_key_ours_public' }
10        }),
11        pri: pgp.loadKeyFromSecret({
12          secret: { scriptId: 'custsecret_pgp_key_ours_private' },
13          password: { scriptId: 'custsecret_pgp_key_ours_private_password' }
14        })
15      }
16    }
17    const data = pgp.createMessageData({
18      content: 'Hello, world!'
19    })
20    const message = data.encrypt({
21      encryptionKeys: keys.ours.pub,
22      signingKeys: keys.ours.pri
23    })
24    const payload = message.asArmored()
25    const parseMessage = pgp.parseMessage({
26      value: payload
27    })
28    const msgData = parseMessage.decrypt({
29      decryptionKeys: keys.ours.pri,
30      verificationKeys: keys.ours.pub
31    })
32    msgData.getText()
33  })

```

Send a Message to Multiple Receivers

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

PGP allows encrypting and signing with multiple keys that allows you to send the same payload to multiple recipients. As a recipient, you can provide multiple decryption and verification keys. The following sample sends a message to two receivers and decrypts the message contents.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/pgp'], (pgp) => {
6    // public and private keys for multiple receivers
7    const keys = {
8      alice: {
9        pub: pgp.loadKeyFromSecret({
10          secret: { scriptId: 'custsecret_pgp_key_alice_public' }
11        }),
12        pri: pgp.loadKeyFromSecret({
13          secret: { scriptId: 'custsecret_pgp_key_alice_private' },
14          password: { scriptId: 'custsecret_pgp_key_bob_private_password' }
15        })
16      },
17      bob: {
18        pub: pgp.loadKeyFromSecret({
19          secret: { scriptId: 'custsecret_pgp_key_bob_public' }
20        }),
21        pri: pgp.loadKeyFromSecret({
22          secret: { scriptId: 'custsecret_pgp_key_bob_private' },
23          password: { scriptId: 'custsecret_pgp_key_bob_private_password' }
24        })
25      },
26      netsuite: {

```



```

27     pub: pgp.loadKeyFromSecret({
28       secret: { scriptId: 'custsecret_pgp_netsuite_pub' }
29     }),
30     pri: pgp.loadKeyFromSecret({
31       secret: { scriptId: 'custsecret_pgp_netsuite_pri' }
32     })
33   },
34   example: {
35     pri: pgp.loadKeyFromSecret({
36       secret: { scriptId: 'custsecret_pgp_example_pri' }
37     })
38   }
39 }
40 const data = pgp.createMessageData({
41   content: 'Hello Alice and Bob!'
42 })
43 const message = data.encrypt({
44   encryptionKeys: [keys.alice.pub, keys.bob.pub],
45   signingKeys: [keys.netsuite.pri, keys.example.pri]
46 })
47 /*
48  * Alice decryption.
49  */
50 message.decrypt({
51   decryptionKeys: keys.alice.pri,
52   verificationKeys: [keys.netsuite.pub]
53 })
54 /*
55  * Bob decryption.
56  */
57 message.decrypt({
58   decryptionKeys: [keys.bob.pri],
59   verificationKeys: keys.example.pub
60 })
61 })

```

Use a Cryptographic Key for a Signature

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample uses a cryptographic key to sign arbitrary data.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/pgp', 'N/crypto/certificate', 'N/encode'], (pgp, cryptoCertificate, encode) => {
6    const keys = {
7      ours: {
8        pub: pgp.loadKeyFromSecret({
9          secret: { scriptId: 'custsecret_pgp_key_ours_public' }
10        }),
11        pri: pgp.loadKeyFromSecret({
12          secret: { scriptId: 'custsecret_pgp_key_ours_private' },
13          password: { scriptId: 'custsecret_pgp_key_ours_private_password' }
14        })
15      }
16    }
17    const signer = pgp.createSigner({
18      key: keys.ours.pri,
19      algorithm: cryptoCertificate.HashAlg.SHA256
20    })
21    signer.update({
22      input: 'Test'
23    })
24    const signature = signer.sign({
25      outputEncoding: encode.Encoding.BASE_64_URL_SAFE
26    })
27  })

```

```

28 | log.debug(signature)
29 | })

```

pgp.Config

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores general configuration options that can be used for message decryption. Use the pgp.createConfig(options) method to create a new configuration object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/pgp Module
Methods and Properties	Config Object Members
Since	2022.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 |
4 | const config = pgp.createConfig({
5 |   allowInsecureDecryptionWithSigningKeys: true,
6 |   useRelaxedSignatureParsing: true
7 | });
8 | ...
9 | ...
10 | // Add additional code

```

Config.allowInsecureDecryptionWithSigningKeys

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	Enables decryption that is not secured with signing keys.
Type	boolean
Module	N/pgp Module
Parent Object	pgp.Config
Sibling Object Members	Config Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const config = pgp.createConfig({
5   allowInsecureDecryptionWithSigningKeys: true
6 });
7
8 ...
9 // Add additional code

```

Config.allowMessagesWithoutIntegrityProtection

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	Allows messages without integrity protection.
Type	boolean
Module	N/pgp Module
Parent Object	pgp.Config
Sibling Object Members	Config Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const config = pgp.createConfig({
5   allowMessagesWithoutIntegrityProtection: true
6 });

```

```

6  });
7
8  ...
9  // Add additional code

```

Config.useRelaxedSignatureParsing

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	Allows relaxed signature parsing for configuration objects.
Type	boolean
Module	N/pgp Module
Parent Object	pgp.Config
Sibling Object Members	Config Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1  // Add additional code
2  ...
3
4  const config = pgp.createConfig({
5      useRelaxedSignatureParsing: true
6  });
7
8  ...
9  // Add additional code

```

pgp.Key

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores multiple cryptographic keys and metadata. You can use this object in the Message.decrypt(options) and MessageData.encrypt(options) methods.
---------------------------	---

	 Note: You cannot create a new key with the N/pgp Module. You must generate keys from an external third party source that supports key generation. For more information, see Limitations of N/pgp .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/pgp Module
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1  const message = pgp.parseMessage({value: 'Encrypted Message to be Decrypted'});
2
3  message.decrypt({
4    decryptionKeys: keys.ours.pri,
5    verificationKeys: keys.our.pub,
6    verification: verification
7  });

```

pgp.KeyId

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores an octet scalar that identifies a (sub)key. This object is used for verification signatures. For more information, see VerificationSignature.keyId .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/pgp Module
Methods and Properties	KeyId Object Members
Since	2022.2

KeyId.asHex()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Returns the Key ID as a hexadecimal string.
Returns	string

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Parent Object	pgp.KeyId
Since	2022.2

pgp.Message

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores processed PGP data. Responsible for enabling message serialization and providing a set of single-step processors to covert to a readable message. Use the MessageData.toMessage() and MessageData.encrypt(options) to create a Message object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/pgp Module
Methods and Properties	Message Object Members
Since	2022.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({ content: 'message content' })
5 const message = msgData.toMessage() // creates Message object
6
7 ...
8 // Add additional code

```

Message.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	Message type that specifies how a message is processed. Enables you to pick the appropriate method to process a message.
-----------------------------	--

Type	boolean
Module	N/pgp Module
Parent Object	pgp.Message
Sibling Object Members	Message Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3 const message = msgData.toMessage() // creates Message object
4
5 log.debug(message.type)
6
7 ...
8 // Add additional code

```

Message.asArmored()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Converts a message to ASCII armored format.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Parent Object	pgp.Message
Sibling Module Members	Message Object Members
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 // Add additional code
2 ...
3 const message = msgData.toMessage() // creates Message object
4
5 log.debug(message.asArmored())
6
7 ...
8 // Add additional code
```

Message.toMessageData()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Converts a pgp.Message object to message data without any processing. This method only works if the message is not encrypted.
Returns	pgp.MessageData
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Parent Object	pgp.Message
Sibling Module Members	Message Object Members
Since	2022.2

Errors

Error Code	Thrown If
PGP_EXPECTED_UNENCRYPTED_MESSAGE	The message is encrypted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 // Add additional code
2 ...
3 const message = msgData.toMessage() // creates Message object
4 message.toMessageData()
```



```

5 |
6 | ...
7 | // Add additional code

```

Message.decrypt(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Decrypts a message and optionally verifies the signatures.
Returns	pgp.MessageData
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Parent Object	pgp.Message
Sibling Module Members	Message Object Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.decryptionKeys	pgp.Key pgp.Key[]	required	Uses one or more keys to attempt message decryption.	2022.2
options.verificationKeys	pgp.Key pgp.Key[]	optional	Uses zero or more keys to attempt message signature verification. If you do not provide a verification key, the message's signature (if any) will be ignored. If you do provide a verification key, at least one signature must be verifiable by one of the provided keys, otherwise an error will be thrown. An expired key works if the signature was made before the expiration. Default value = [].	2022.2
options.verification	pgp.Verification	optional	An empty verification object. If you provide a value for this parameter, the verification results will be flushed instead of throwing an error for invalid signature. Default value = null.	2022.2
options.suppressVerificationErrors	boolean	optional	If set to true, the verification errors will not be thrown. This value is implicitly set to true when the verification parameter is provided. Default value = false.	2022.2
options.config	pgp.Config	optional	The configuration. Default value is pgp.createConfig(options) .	2022.2

Errors

 **Note:** If the message is not well formed, various parse errors can be thrown.

Error Code	Thrown If
PGP_EXPECTED_ENCRYPTED_MESSAGE	The message is not encrypted.
PGP_NO_MATCHING_DECRYPTION_KEY_ANALYSIS_1	No matching decryption key was found.
PGP_MESSAGE_IS_NOT_INTEGRITY_PROTECTED	The message is not integrity protected.
PGP_INTEGRITY_VERIFICATION_FAILED	The message is corrupted.
PGP_VERIFICATION_FAILED_NO_SIGNATURE	The message is not signed, but verification keys were provided.
PGP_VERIFICATION_FAILED_1	None of the signatures could be verified using the provided verification keys.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const verification = pgp.createVerification();
5 message.decrypt({
6   decryptionKeys: keys.ours.pri,
7   verificationKeys: keys.our.pub,
8   verification: verification, // supresses verification errors
9 });
10
11 ...
12 // Add additional code

```

pgp.MessageData

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores message data. The responsibilities of this object includes: - Store message contents with meta data. - Enable reading message contents and metadata. - For further processing, determines whether data is text or binary. - Provides a set of single-step processors for various PGP use cases. Use the pgp.createMessageData(options) method to create a message data object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/pgp Module
Methods and Properties	MessageData Object Members
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({ content: 'message data content'})
5
6 ...
7 // Add additional code

```

MessageData.filename

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	The name of a file.
Type	string
Module	N/pgp Module
Parent Object	pgp.MessageData
Sibling Object Members	MessageData Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

MessageData.date

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	Date of a message or modification date of the file.
Type	Date object

Module	N/pgp Module
Parent Object	pgp.MessageData
Sibling Object Members	MessageData Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

MessageData.format

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	Literal data packet type.
Type	pgp.Format
Module	N/pgp Module
Parent Object	pgp.MessageData
Sibling Object Members	MessageData Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

MessageData.getText()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Extracts the contents of the message as text.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/pgp Module
Parent Object	pgp.MessageData
Sibling Object Members	MessageData Object Members
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({ content: 'message data content'})
5 msgData.getText()
6
7 ...
8 // Add additional code

```

MessageData.toMessage()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a message with no signature, compression, or encryption.
Returns	pgp.Message
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({ content: 'message data content'})
5 msgData.toMessage()
6
7 ...
8 // Add additional code

```

MessageData.encrypt(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates an encrypted message that is optionally signed.
Returns	pgp.Message
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Parent Object	pgp.MessageData
Sibling Object Members	MessageData Object Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.encryptionKeys	pgp.Key pgp.Key[]	Required	One or more keys used to encrypt a message. If a key contains multiple valid encryption (sub)keys, the most recent key added will be used.	2022.2
options.signingKeys	pgp.Key pgp.Key[]	optional	Zero or more keys used for signing. If a key contains multiple valid signing (sub)keys, the most recent key added will be used. Default value = [].	2022.2
options.compressionAlgorithm	pgp.Compression Algorithm	optional	The compression algorithm to use. Default value = <code>CompressionAlgorithm.ZLIB</code> .	2022.2

Errors

Error Code	Thrown If
PGP_NO_ENCRYPTION_KEY_FOUND_IN_KEY_PARAM_1	No valid encryption (sub)key was found in one of the provided keys.
PGP_NO_SIGNING_KEY_FOUND_IN_KEY_PARAM_1	No valid signing (sub)key was found in one of the provided keys.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 | // Add additional code
```

```
2  ...
3
4  const data = pgp.createMessageData({
5    content: "Hello, world!"
6  });
7  const message = data.encrypt({
8    encryptionKeys: keys.alice.pub,
9    signingKeys: keys.ours.pri,
10  });
11
12  ...
13  // Add additional code
```

pgp.Verification

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores verification results. Use the pgp.createVerification() method to create a Verification object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/pgp Module
Methods and Properties	Verification Object Members
Since	2022.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1  // Add additional code
2  ...
3
4  const verification = pgp.createVerification();
5
6  ...
7  // Add additional code
```

Verification.verified

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	Indicates whether the message verification was successful.
----------------------	--

Type	null boolean
Module	N/pgp Module
Parent Object	pgp.Verification
Sibling Object Members	Verification Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const verification = pgp.createVerification();
5 log.debug(verification.verified)
6
7 ...
8 // Add additional code

```

Verification.signatures

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	List of individual verifications, one per each signature.
Type	null Array< pgp.VerificationSignature >
Module	N/pgp Module
Parent Object	pgp.Verification
Sibling Object Members	Verification Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 // Add additional code
2 ...
3
4 const verification = pgp.createVerification();
5 log.debug(verification.signatures)
6
7 ...
8 // Add additional code
```

pgp.VerificationSignature

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Object Description	Stores a verification result for single signature.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/pgp Module
Methods and Properties	VerificationSignature Object Members
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 // Add additional code
2 ...
3
4 const verification = pgp.createVerification();
5 log.debug(verification.signatures)
6
7 ...
8 // Add additional code
```

VerificationSignature.keyId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Property Description	ID of the (sub)key that was used for signing.
-----------------------------	---

Type	pgp.KeyId
Module	N/pgp Module
Parent Object	pgp.VerificationSignature
Sibling Object Members	VerificationSignature Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

VerificationSignature.dateSigned

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	Date when the message was signed.
Type	Date object
Module	N/pgp Module
Parent Object	pgp.VerificationSignature
Sibling Object Members	VerificationSignature Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

VerificationSignature.verified

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	Indicates whether verification was successful.
Type	boolean
Module	N/pgp Module

Parent Object	pgp.VerificationSignature
Sibling Object Members	VerificationSignature Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

VerificationSignature.problems

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Property Description	List of problems for more fine-grained decision making.
Type	string[]
Module	N/pgp Module
Parent Object	pgp.VerificationSignature
Sibling Object Members	VerificationSignature Object Members
Since	2022.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

pgp.createConfig(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a new pgp.Config object. A configuration object stores general configuration options that can be used for message decryption.
Returns	pgp.Config
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/pgp Module
Sibling Module Members	N/pgp Module Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.allowInsecureDecryptionWithSigningKeys	boolean	optional	Enables decryption that is not secured with signing keys on configuration objects. Default value is false.	2022.2
options.allowMessagesWithoutIntegrityProtection	boolean	optional	Allows messages without integrity protection on configuration objects. Default value is false.	2022.2
options.useRelaxedSignatureParsing	boolean	optional	Allows relaxed signature parsing for configuration objects. Default value is false.	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const config = pgp.createConfig()
5
6 ...
7 // Add additional code

```

pgp.createMessageData(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a new pgp.MessageData object. A message data object stores message content with metadata.
Returns	pgp.MessageData
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module

Sibling Module Members	N/pgp Module Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description
options.content	string	required	Content of the message.
options.filename	string	optional	File name if the message represents a file, empty string otherwise.
options.date	Date object	optional	Date of the message or modification date of the file. Default value = <code>new Date()</code> .
options.format	pgp.Format	optional	Literal data packet type. Default value = <code>Format.UTF8</code> , if content is a string. <code>Format.BINARY</code> otherwise.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({
5   content: 'Hello'
6 })
7
8 ...
9 // Add additional code

```

pgp.createSigner(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates a certificate.Signer object for signing plain strings. If the given PGP key contains multiple valid signing sub keys, the most recently added will be used. This behavior is consistent with MessageData.encrypt(options) method.
Returns	certificate.Signer
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/pgp Module

Sibling Module Members	N/pgp Module Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.key	pgp.Key	required	The pgp key.	2022.2
options.algorithm	string	required	Hash algorithm	2022.2

Errors

Error Code	Thrown If
UNSUPPORTED_COMBINATION_OF_KEY_AND_HASH_ALGORITHMS	The given key's encryption algorithm is not compatible with the given hash algorithm.
PGP_NO_SIGNING_KEY_FOUND_IN_KEY_PARAM_1	No valid signing (sub)keys are found in the given key.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const signer = pgp.createSigner({
5   key: keys.ours.pri,
6   algorithm: cryptoCertificate.HashAlg.SHA256
7 });
8
9 ...
10 // Add additional code

```

pgp.createVerification()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** The content in this help topic pertains to SuiteScript 2.1.

Method Description	Creates an empty verification object.
Returns	pgp.Verification
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Sibling Module Members	N/pgp Module Members
Since	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const verification = pgp.createVerification()
5
6 ...
7 // Add additional code

```

pgp.loadKeyFromSecret(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Loads a key contents that are securely stored in a secret.
Returns	pgp.Key
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Sibling Module Members	N/pgp Module Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.secret	Secret	required	Secret that contains a PGP key in ASCII armored format.	2022.2
options.password	Secret	optional	Secret that contains a password to unlock the key. Applicable for private keys.	2022.2

Errors

Error Code	Thrown If
REFERENCED_SECRET_IS_NOT_AVAILABLE	The secret/password parameter references a non-existing secret or you lack permission.
PGP_NONSTANDARD_KEY_DOES_NOT_COMPLY_WITH_PGP_KEY_FORMAT	Parsing of the key fails.
PGP_YOU_CANNOT_PROVIDE_PASSWORD_FOR_A_PUBLIC_KEY	You provide a password, but the key is public.
PGP_INVALID_KEY_PASSWORD	You provide a password, but it is wrong or the private key is not password protected.
PGP_THIS_KEY_IS_PASSWORD_PROTECTED	You do not provide any password, but the private key is password protected.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const keys = pgp.loadKeyFromSecret({
5   secret: { scriptId: "custsecret_pgp_key_ours_private" },
6   password: { scriptId: "custsecret_pgp_key_ours_private_password" },
7 },
8
9 ...
10 // Add additional code

```

pgp.parseKey(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Parses an existing PGP key. Use pgp.loadKeyFromSecret(options) to load private keys.
Returns	pgp.Key
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/pgp Module
Sibling Module Members	N/pgp Module Members

Since	2022.2
-------	--------

Parameters

Parameter	Type	Required / Optional	Description
options.value	string	required	ASCII armored key
options.password	string	optional	Password to unlock the key. Applicable for private keys.

Errors

Error Code	Thrown If
PGP_NONSTANDARD_KEY_DOES_NOT_COMPLY_WITH_PGP_KEY_FORMAT	Parsing of the key fails.
PGP_YOU_CANNOT_PROVIDE_PASSWORD_FOR_A_PUBLIC_KEY	You provide a password when the key is public.
PGP_INVALID_KEY_PASSWORD	You provide a wrong password or the private key is not password protected.
PGP_THIS_KEY_IS_PASSWORD_PROTECTED	You don't provide any password, but the private key is password protected.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const parseKey = pgp.parseKey({ value: 'custsecret_ascii_armored_key' })
5
6 ...
7 // Add additional code

```

pgp.parseMessage(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Method Description	Parses a PGP message.
Returns	pgp.Message
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/pgp Module
Sibling Module Members	N/pgp Module Members
Since	2022.2

Parameters

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	ASCII armored representation of the message.	2022.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const parse = pgp.parseMessage({
5   value: message
6 });
7
8 ...
9 // Add additional code

```

pgp.CompressionAlgorithm

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The content in this help topic pertains to SuiteScript 2.1.

Enum Description	Holds the values for available compression algorithms. Use this enum to set the value of the options.compressionAlgorithm parameter of the MessageData.encrypt(options) method.
Module	N/pgp Module
Sibling Module Members	N/pgp Module Members
Since	2022.2

Values

Value
ZLIB

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({ content: 'hello world' })
5 msgData.encrypt({
6   encryptionKeys: [ keys.alice.pub, keys.bob.pub, keys.carol.pub ],
7   signingKeys: [ keys.netsuite.pri, keys.oracle.pri ],
8   compressionAlgorithm: pgp.CompressionAlgorithm.ZLIB
9 })
10
11 ...
12 // Add additional code
```

pgp.Format

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** The content in this help topic pertains to SuiteScript 2.1.

Enum Description	Holds the values for literal data packet type. For more information, see Literal Data Packet .
Module	N/pgp Module
Sibling Module Members	N/pgp Module Members
Since	2022.2

Values

Value
UTF8
BINARY

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/pgp Module Script Samples](#).

```
1 // Add additional code
2 ...
3
4 const msgData = pgp.createMessageData({
5   content: 'hello world',
6   format: pgp.Format.UTF8
7 })
```

```
7  })
8
9  ...
10 // Add additional code
```

N/piremoval Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/piremoval module to remove personal information (PI) from system notes, workflow history, and specific field values. Use the N/piremoval module to comply with privacy regulations, specifically the right to be forgotten. You can remove personal information from system notes only, or you can also remove workflow history and field values on the record. Entity records, transactions, and custom records are supported.

[Go To Samples {...}](#)

You can use the [piremoval.createTask\(options\)](#) method to create a PI removal task, or use [piremoval.loadTask\(options\)](#) to load an existing PI removal task. Both of these methods return a [piremoval.PiRemovalTask](#) object that represents the task. Create a [piremoval.PiRemovalTask](#) object for each record type that requires removal of personal information. Use the [PiRemovalTask.save\(\)](#) method to save the task, then use the [PiRemovalTask.run\(\)](#) method to process the task and remove the personal information.

You can use the [piremoval.getTaskStatus\(options\)](#) method to check the status of a submitted PI removal task. This method returns a [piremoval.PiRemovalTaskStatus](#) object that describes the current status of the removal task. The [piremoval.PiRemovalTaskStatus](#) object uses an iterator to provide a list of log entries in the [PiRemovalTaskStatus.logList](#) object.

To use the N/piremoval module, the following requirements must be met:

- Remove Personal Information Create permission is required to create a PI removal task.
- Remove Personal Information Run permission is required to run a PI removal task.

For more information, see the help topic [Personal Information \(PI\) Removal](#).

In This Help Topic

- [N/piremoval Module Members](#)
- [PiRemovalTask Object Members](#)
- [PiRemovalTaskLogItem Object Members](#)
- [PiRemovalTaskStatus Object Members](#)

N/piremoval Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	piremoval.PiRemovalTask	Object	Server scripts	Encapsulates a personal information removal task.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				Use piremoval.createTask(options) to create this object.
	piremoval.PiRemovalTaskLogItem	Object	Server scripts	Encapsulates a log item of the personal information removal task status.
	piremoval.PiRemovalTaskStatus	Object	Server scripts	Encapsulates the status of a personal information removal task. Use piremoval.getTaskStatus(options) to create this object.
Method	piremoval.createTask(options)	piremoval.PiRemovalTask	Server scripts	Creates a personal information removal task.
	piremoval.deleteTask(options)	void	Server scripts	Deletes a personal information removal task.
	piremoval.getTaskStatus(options)	piremoval.PiRemovalTaskStatus	Server scripts	Retrieves the status of a personal information removal task.
	piremoval.loadTask(options)	piremoval.PiRemovalTask	Server scripts	Loads a personal information removal task.

PiRemovalTask Object Members

The following members are available for a [piremoval.PiRemovalTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	PiRemovalTask.deleteTask()	void	Server scripts	Deletes the personal information removal task.
	PiRemovalTask.run()	void	Server scripts	Runs the personal information removal task.
	PiRemovalTask.save()	void	Server scripts	Saves the personal information removal task.
Property	PiRemovalTask.fieldIds	string[] (read-only)	Server scripts	Represents the field IDs that are processed by the PI removal task.
	PiRemovalTask.historyOnly	boolean	Server scripts	Indicates whether the PI removal task removes system note information only, not field values or workflow history.
	PiRemovalTask.historyReplacement	string (read-only)	Server scripts	Represents the text used in system notes to replace the original values.
	PiRemovalTask.id	number (read-only)	Server scripts	Represents the ID of the personal information removal task.
	PiRemovalTask.recordIds	number[] (read-only)	Server scripts	Represents the record IDs that are processed by the PI removal task.
	PiRemovalTask.recordType	string (read-only)	Server scripts	Describes the record type updated by the PI removal task.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	PiRemovalTask.status	piremoval.PiRemovalTask.Status	Server scripts	Describes the status of the submitted personal information removal task.
	PiRemovalTask.workflowIds	number[] (read-only)	Server scripts	Represents the workflow IDs whose history is processed by the PI removal task.

PiRemovalTaskLogItem Object Members

The following members are available for a [piremoval.PiRemovalTaskLogItem](#) object.

Member Type	Name	Return Type/Value Type	Support Script Type	Description
Property	PiRemovalTaskLogItem.exception	string (read-only)	Server scripts	Describes the exception for the log item, including and what caused it.
	PiRemovalTaskLogItem.message	string (read-only)	Server scripts	Describes the message for the log item and an explanation for any errors.
	PiRemovalTaskLogItem.status	string (read-only)	Server scripts	Describes the status of the log item. This property takes its values from task.TaskStatus .
	PiRemovalTaskLogItem.type	string (read-only)	Server scripts	Describes the type of personal information that was removed, one of FieldValue, SystemNote, or Workflow.

PiRemovalTaskStatus Object Members

The following members are available for a [piremoval.PiRemovalTaskStatus](#) object.

Member Type	Name	Return Type/Value Type	Support Script Type	Description
Property	PiRemovalTaskStatus.logList	list	Server scripts	Represents a list of logs for the PI removal task job.
	PiRemovalTaskStatus.status	string	Server scripts	Describes the status of the submitted personal information removal task.

N/piremoval Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/piremoval module.

Remove Phone Numbers and Comments from Customer Records

The following sample shows how to remove the phone numbers and comments in specific customer records from the record fields (field values), system notes, and workflow history. The removed values are replaced with **removed_value**.

To use the N/piremoval module, the following requirements must be met:

- Remove Personal Information Create permission is required to create a PI removal task.
- Remove Personal Information Run permission is required to run a PI removal task.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/piremoval'], function(piremoval) {
6      function removePersonalInformation() {
7          var piRemovalTask = piremoval.createTask({
8              recordType: 'customer',
9              recordIds: [11, 19],
10             fieldIds: ['comments', 'phone'],
11             workflowIds: [1],
12             historyOnly: false,
13             historyReplacement: 'removed_value'
14         });
15
16         piRemovalTask.save();
17         var taskId = piRemovalTask.id;
18
19         var piRemovalTaskInProgress = piremoval.loadTask(taskId);
20         piRemovalTaskInProgress.run();
21
22         var status = piremoval.getTaskStatus(taskId);
23     };
24
25     removePersonalInformation();
26 });

```

piremoval.PiRemovalTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a task to remove personal information (PI). This object includes lists of the record, field, and workflow IDs to remove personal information from, as well as history replacement information. Use piremoval.createTask(options) to create a piremoval.PiRemovalTask object, or use piremoval.loadTask(options) to load a PI removal task as a piremoval.PiRemovalTask object. To save the object, use PiRemovalTask.save(). To execute the PI removal, use PiRemovalTask.run().
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/piremoval Module
Methods and Properties	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4   recordType: 'customer',
5   recordIds: [11, 19],
6   fieldIds: ['comments', 'phone'],
7   workflowIds: [1],
8   historyOnly: false,
9   historyReplacement: 'removed_value'
10 });
11
12 myPiRemovalTask.save();
13 var myTaskId = myPiRemovalTask.id;
14
15 myPiRemovalTask.run();
16 ...
17 // Add additional code

```

PiRemovalTask.deleteTask()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes the PI Removal task.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Errors

Error Code	Error Message	Thrown If
UNEXPECTED_ERROR	Cannot delete PiRemoval job that was not saved.	The PI removal job is not saved.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [11, 19],
6     fieldIds: ['comments', 'phone'],
7     workflowIds: [1],
8     historyOnly: false,
9     historyReplacement: 'removed_value'
10 });
11
12 myPiRemovalTask.save();
13 var myTaskId = myPiRemovalTask.id;
14
15 myPiRemovalTask.deleteTask();
16 ...
17 // Add additional code

```

PiRemovalTask.run()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Runs the PI removal task. All validation for the task (for example, ensuring that the specified record IDs are valid) occurs when the task is saved using PiRemovalTask.save(), not when the task is run using PiRemovalTask.run().  Note: Remove Personal Information Run permission is required to run a PI removal task.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	20 units
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Errors

Error Code	Error Message	Thrown If
UNEXPECTED_ERROR	Cannot run unsaved PiRemoval job.	The PI removal job is not saved.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [11, 19],
6     fieldIds: ['comments', 'phone'],
7     workflowIds: [1],
8     historyOnly: false,
9     historyReplacement: 'removed_value'
10 });
11
12 myPiRemovalTask.save();
13 var myTaskId = myPiRemovalTask.id;
14
15 myPiRemovalTask.run();
16 ...
17 // Add additional code

```

PiRemovalTask.save()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Saves the PI removal task. All validation for the task (for example, ensuring that the specified record IDs are valid) occurs when the task is saved using this method.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Errors

Error Code	Error Message	Thrown If
_1_CANNOT_BE_EMPTY	Record Type cannot be empty.	The record type is not set.
_1_JOB_WAS_NOT_FOUND	Record Type 'type' was not found.	Record type does not exist.
_1_JOB_WAS_NOT_FOUND	Record ID 'ID' was not found.	One of the record IDs does not exist.
_1_JOB_WAS_NOT_FOUND	Field ID 'ID' was not found.	One of the field IDs does not exist.

Error Code	Error Message	Thrown If
_1_JOB_WAS_NOT_FOUND	Workflow ID 'ID' was not found.	One of the workflow IDs does not exist.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [11, 19],
6     fieldIds: ['comments', 'phone'],
7     workflowIds: [1],
8     historyOnly: false,
9     historyReplacement: 'removed_value'
10 });
11
12 myPiRemovalTask.save();
13 var myTaskId = myPiRemovalTask.id;
14
15 myPiRemovalTask.run();
16 ...
17 // Add additional code

```

PiRemovalTask.fieldIds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	IDs of the fields whose PI is removed. If no field IDs are entered, no information changes are performed. If the field IDs are null or invalid, the following exception occurs: Wrong parameter type: options.fieldIds is expected as array.
Type	string[] (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({

```

```

4      recordType: 'customer',
5      recordIds: [95],
6      fieldIds: ['email'],
7      historyReplacement: 'removed_value'
8    });
9
10   myPiRemovalTask.save();
11   var theFieldIds = myPiRemovalTask.fieldIds;
12   ...
13   // Add additional code

```

PiRemovalTask.historyOnly

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the PI removal task removes system note information only, not field values or workflow history. If true, the task removes information from system notes only. If false, the task removes information from system notes, workflow history, and field values. The default value is false.
Type	boolean (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1   // Add additional code
2   ...
3   var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyOnly: true,
8     historyReplacement: 'removed_value'
9   });
10
11   myPiRemovalTask.save();
12   var theHistoryOnly = myPiRemovalTask.historyOnly;
13   ...
14   // Add additional code

```

PiRemovalTask.historyReplacement

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text used in system notes to replace the original value.
-----------------------------	--

Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var theHistoryReplacement = myPiRemovalTask.historyReplacement;
12 ...
13 // Add additional code

```

PiRemovalTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID that uniquely identifies the PI removal task. This ID is assigned to the PI removal task when PiRemovalTask.save() is called to save the task. You cannot specify your own task ID.
Type	number (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({

```

```

4      recordType: 'customer',
5      recordIds: [95],
6      fieldIds: ['email'],
7      historyReplacement: 'removed_value'
8    });
9
10   myPiRemovalTask.save();
11   var theId = myPiRemovalTask.id;
12   ...
13   // Add additional code

```

PiRemovalTask.recordIds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of records whose PI is removed. If no record IDs are entered, no information changes are performed. If the record IDs are null or invalid, the following exception occurs: Wrong parameter type: options.recordIds is expected as array.
Type	number[] (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4   recordType: 'customer',
5   recordIds: [95, 107],
6   fieldIds: ['email'],
7   historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var theRecordIds = myPiRemovalTask.recordIds;
12 ...
13 // Add additional code

```

PiRemovalTask.recordType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Type of record whose PI is removed. All records referenced in the piremoval.PiRemovalTask object must be the same type.
-----------------------------	--

Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var theRecordType = myPiRemovalTask.recordType;
12 ...
13 // Add additional code

```

PiRemovalTask.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status of the PI removal task.
Type	piremoval.PiRemovalTaskStatus
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],

```

```

7     historyReplacement: 'removed_value'
8   });
9
10  myPiRemovalTask.save();
11  var theStatus = myPiRemovalTask.status;
12  ...
13  // Add additional code

```

PiRemovalTask.workflowIds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	IDs of workflows where PI is removed from the workflow history. If no workflow IDs are entered, no information changes are performed. If the workflow IDs are null or invalid, the following exception occurs: Wrong parameter type: options.workflowIds is expected as array.
Type	number[] (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTask
Sibling Object Members	PiRemovalTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var myPiRemovalTask = piremoval.createTask({
4    recordType: 'customer',
5    recordIds: [95, 107],
6    fieldIds: ['email', 'phone'],
7    workflowIds: [1, 7],
8    historyReplacement: 'removed_value'
9  });
10
11  myPiRemovalTask.save();
12  var theWorkflowIds = myPiRemovalTask.workflowIds;
13  ...
14  // Add additional code

```

piremoval.PiRemovalTaskLogItem

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	This object represents log items that are associated with a piremoval.PiRemovalTaskStatus object. The logs are generated separately when a task is created, started, and completed. A piremoval.PiRemovalTaskLogItem object represents a single log entry that you get from the
---------------------------	--

	piremoval.PiRemovalTaskStatus object using the PiRemovalTaskStatus.logList property. The log items are sorted by date. The structure of this object is described in PiRemovalTaskLogItem Object Members.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/piremoval Module
Methods and Properties	PiRemovalTaskLogItem Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16
17 var theLogList = myStatus.logList;
18 for (var i = 0; i < theLogList.length; i++) {
19     log.debug({
20         title: 'logList value',
21         details: theLogList[i]
22     });
23 }
24 ...
25 // Add additional code

```

PiRemovalTaskLogItem.exception

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Exception message for the log item, typically an unexpected error from NetSuite.
Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTaskLogItem
Sibling Object Members	PiRemovalTaskLogItem Object Members

Since	2019.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16
17 var theLogList = myStatus.logList;
18 for (var i = 0; i < theLogList.length; i++) {
19     var theLogListException = theLogList[i].exception;
20
21     // Do something with the log list exception here
22 }
23 ...
24 // Add additional code

```

PiRemovalTaskLogItem.message

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Log item text message. The message specifies if the record type is not set, or if one of record, field, or workflow IDs do not exist.
Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTaskLogItem
Sibling Object Members	PiRemovalTaskLogItem Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code

```

```

2  ...
3  var myPiRemovalTask = piremoval.createTask({
4      recordType: 'customer',
5      recordIds: [95],
6      fieldIds: ['email'],
7      historyReplacement: 'removed_value'
8  });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16
17 var theLogList = myStatus.logList;
18 for (var i = 0; i < theLogList.length; i++) {
19     var theLogListItem = theLogList[i].message;
20
21     // Do something with the log list message here
22 }
23 ...
24 // Add additional code

```

PiRemovalTaskLogItem.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Encapsulates the status of a log item. Possible status values include: - PENDING - PROCESSING - COMPLETE - FAILED For more information, see task.TaskStatus .
Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTaskLogItem
Sibling Object Members	PiRemovalTaskLogItem Object Members
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var myPiRemovalTask = piremoval.createTask({
4      recordType: 'customer',
5      recordIds: [95],
6      fieldIds: ['email'],
7      historyReplacement: 'removed_value'

```

```

8  });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16
17 var theLogList = myStatus.logList;
18 for (var i = 0; i < theLogList.length; i++) {
19     var theLogListStatus = theLogList[i].status;
20
21     // Do something with the log list status here
22 }
23 ...
24 // Add additional code

```

PiRemovalTaskLogItem.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates the change described by this log item. Possible values include: - FieldValue - field value - SystemNote - system note - Workflow - workflow history
Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTaskLogItem
Sibling Object Members	PiRemovalTaskLogItem Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var myPiRemovalTask = piremoval.createTask({
4      recordType: 'customer',
5      recordIds: [95],
6      fieldIds: ['email'],
7      historyReplacement: 'removed_value'
8  });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16
17 var theLogList = myStatus.logList;
18 for (var i = 0; i < theLogList.length; i++) {

```

```

19     var theLogListType = theLogList[i].type;
20
21     // Do something with the log list type here
22 }
23 ...
24 // Add additional code

```

piremoval.PiRemovalTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the status of a personal information removal task returned by piremoval.getTaskStatus(options). Possible status values include: ■ PENDING ■ PROCESSING ■ COMPLETE ■ FAILED For more information, see task.TaskStatus.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/piremoval Module
Methods and Properties	PiRemovalTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myFirstStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16
17 myPiRemovalTask.run();
18
19 var mySecondStatus = piremoval.getTaskStatus({
20     id: myId
21 });

```

```

22 ...
23 // Add additional code

```

PiRemovalTaskStatus.logList

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Gets list of logs for the PiRemovalTask job.
Type	piremoval.PiRemovalTaskLogItem[]
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTaskStatus
Sibling Object Members	PiRemovalTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4   recordType: 'customer',
5   recordIds: [95],
6   fieldIds: ['email'],
7   historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myStatus = piremoval.getTaskStatus({
14   id: myId
15 });
16
17 var theLogList = myStatus.logList;
18 for (var i = 0; i < theLogList.length; i++) {
19   log.debug({
20     title: 'logList value',
21     details: theLogList[i]
22   });
23 }
24 ...
25 // Add additional code

```

PiRemovalTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Encapsulates the status of a personal information removal task returned by piremoval.getTaskStatus(options) .
-----------------------------	---

	Possible status values include: ■ PENDING ■ PROCESSING ■ COMPLETE ■ FAILED For more information, see task.TaskStatus.
Type	string (read-only)
Module	N/piremoval Module
Parent Object	piremoval.PiRemovalTaskStatus
Sibling Object Members	PiRemovalTaskStatus Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myTaskStatus = piremoval.getTaskStatus({
14     id: myId
15 });
16 var theStatus = myTaskStatus.status;
17 ...
18 // Add additional code

```

piremoval.createTask(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new personal information removal task.  Note: Remove Personal Information Create permission is required to create a PI removal task.
Returns	piremoval.PiRemovalTask
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	None
Module	N/piremoval Module
Sibling Object Members	N/piremoval Module Members
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldIds	number[]	optional	Represents IDs of fields whose personal information is removed.
options.historyOnly	boolean	optional	Indicates whether the PI removal task removes system note information only, not field values or workflow history. If true, the task removes information from system notes only. If false, the task removes information from system notes, workflow history, and field values. The default value is false.
options.historyReplacement	string	optional	Represents the text used in system notes to replace the original values.
options.recordIds	number[]	optional	Represents IDs of records whose personal information is removed.
options.recordType	string	optional	Describes the record type that is updated by the PI removal task.
options.workflowIds	number[]	optional	Represents the workflow IDs whose history is processed by the PI removal task.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4   recordType: 'customer',
5   recordIds: [95, 107],
6   fieldIds: ['email', 'phone'],
7   workflowIds: [3, 7],
8   historyOnly: true,
9   historyReplacement: 'removed_value'
10 });
11
12 myPiRemovalTask.save();
13 var myTaskId = myPiRemovalTask.id;
14
15 myPiRemovalTask.run();
16 ...

```


17 // Add additional code

piremoval.deleteTask(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes a personal information removal task.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/piremoval Module
Sibling Object Members	N/piremoval Module Members
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	number	required	Unique identifier of the personal information removal task.

Errors

Error Code	Error Message	Thrown If
UNEXPECTED_ERROR	Cannot delete PIRemoval job that was not saved	Job is not saved.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4   recordType: 'customer',
5   recordIds: [95, 107],
6   fieldIds: ['email', 'phone'],
7   workflowIds: [3, 7],
8   historyOnly: true,
9   historyReplacement: 'removed_value'
10 });
11
```

```

12 myPiRemovalTask.save();
13 var myTaskId = myPiRemovalTask.id;
14
15 piremoval.deleteTask({
16     id: myTaskId
17 });
18 ...
19 // Add additional code

```

piremoval.getTaskStatus(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves the status of a personal information removal task.
Returns	piremoval.PiRemovalTaskStatus
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/piremoval Module
Sibling Object Members	N/piremoval Module Members
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	number	required	Unique identifier of the personal information removal task.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myPiRemovalTask = piremoval.createTask({
4     recordType: 'customer',
5     recordIds: [95],
6     fieldIds: ['email'],
7     historyReplacement: 'removed_value'
8 });
9
10 myPiRemovalTask.save();
11 var myId = myPiRemovalTask.id;
12
13 var myFirstStatus = piremoval.getTaskStatus({
14     id: myId

```

```

15  });
16
17  myPiRemovalTask.run();
18
19  var mySecondStatus = piremoval.getTaskStatus({
20      id: myId
21  });
22  ...
23  // Add additional code

```

piremoval.loadTask(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves an existing personal information removal task.
Returns	piremoval.PiRemovalTask
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/piremoval Module
Sibling Object Members	N/piremoval Module Members
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	number	required	Unique identifier of the personal information removal task.

Errors

Error Code	Error Message	Thrown If
_1_WAS_NOT_FOUND	PIRemoval job was not found.	Job with the ID provided was not found.

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/piremoval Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var myPiRemovalTask = piremoval.createTask({

```

```
4      recordType: 'customer',
5      recordIds: [95],
6      fieldIds: ['email'],
7      historyReplacement: 'removed_value'
8    });
9
10   myPiRemovalTask.save();
11   var myId = myPiRemovalTask.id;
12
13   var myLoadedPiRemovalTask = piremoval.loadTask({
14     id: myId
15   });
16
17   myLoadedPiRemovalTask.run();
18   ...
19   // Add additional code
```

N/plugin Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/plugin module to load custom plug-in implementations. For more information about custom plug-ins, see the following help topics:

- Custom Plug-ins
- Custom Plug-in Creation
- Custom Plug-in Development
- Custom Plug-in Implementation

Go To Samples {...}

Important: You cannot use the SuiteScript Debugger to debug a script on demand that uses the N/plugin module. You must use deployed debugging. To use deployed debugging, you must complete the steps described in [Adding a Script that Instantiates a Custom Plug-in to NetSuite](#). For the complete process on creating a custom plugin, see the help topic [Custom Plug-in Development](#). For additional information about ad-hoc and deployed debugging, see the help topic [SuiteScript Debugger](#).

In This Help Topic

- [N/plugin Module Members](#)

N/plugin Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	plugin.findImplementations(options)	string[]	Server scripts	Returns the script IDs of custom plug-in type implementations.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	<code>plugin.loadImplementation(options)</code>	Object	Server scripts	Instantiates an implementation of the custom plug-in type.

N/plugin Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/plugin module.

Find Plug-in Implementations

The following sample shows an implementation of a custom plug-in interface. To test this sample, you need a custom plug-in type with a script ID of `customscript_magic_plugin` and an interface with a single method, `int doTheMagic(int, int)`.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType pluginTypeimpl
4   */
5  define(function() {
6      return {
7          doTheMagic: function(operand1, operand2) {
8              return operand1 + operand2;
9          }
10     }
11 });

```

The following Suitelet iterates through all implementations of the custom plug-in type `customscript_magic_plugin`. For the plug-in to be recognized, the Suitelet script record must specify the plug-in type on the Custom Plug-in Types subtab.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/plugin'], function(plugin) {
6      function onRequest(context) {
7          var impls = plugin.findImplementations({
8              type: 'customscript_magic_plugin'
9          });
10
11          for (var i = 0; i < impls.length; i++) {
12              var pl = plugin.loadImplementation({
13                  type: 'customscript_magic_plugin',
14                  implementation: impls[i]
15              });
16              log.debug('impl ' + impls[i] + ' result = ' + pl.doTheMagic(10, 20));
17          }
18
19          var pl = plugin.loadImplementation({

```

```

20         type: 'customscript_magic_plugin'
21     });
22     log.debug('default impl result = ' + pl.doTheMagic(10, 20));
23 }
24
25 return {
26     onRequest: onRequest
27 };
28 });

```

plugin.findImplementations(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the script IDs of custom plug-in type implementations. Returns an empty list when there is no custom plug-in type with the script ID available for the executing script. i Note: It is important that you fully understand custom plug-ins when you use this method. - Custom Plug-ins - Custom Plug-in Creation - Custom Plug-in Development - Custom Plug-in Implementation
Returns	A string[] containing a list of custom plug-in implementation script IDs.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/config Module
Since	2016.1

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The script ID of the custom plug-in type.	2016.1
options.includeDefault	boolean	optional	The default value is true, indicating that the default implementation should be included in the list.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/plugin Module Script Sample](#).

```
1 //Add additional code
2 ...
3 var impls = plugin.findImplementations({
4     type: 'customscript_sample_plugin'
5 });
6 ...
7 //Add additional code
```

plugin.loadImplementation(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Instantiates an implementation of the custom plugin type. Returns the implementation which is currently selected in the UI (Manage Plug-ins page) when no implementation ID is explicitly provided. Note: It is important that you fully understand custom plug-ins when you use this method. - Custom Plug-ins - Custom Plug-in Creation - Custom Plug-in Development - Custom Plug-in Implementation
Returns	An Object implementing the custom plug-in type.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/config Module
Since	2016.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The script ID of the custom plug-in type.	2016.1

Parameter	Type	Required / Optional	Description	Since
options.implementation	string	optional	The script ID of the custom plug-in implementation.	2016.1

Error Code	Thrown If
UNABLE_TO_FIND_IMPLEMENTATION_1_FOR_PLUGIN_2	Either there is no such implementation of the provided plug-in type, or the plug-in type does not exist.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/plugin Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var pl = plugin.loadImplementation({
4     type: 'customscript_sample_plugin'
5 });
6 ...
7 //Add additional code

```

N/portlet Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/portlet module to resize or refresh a form portlet. For more information about portlet scripts, see the help topic [SuiteScript 2.x Portlet Script Type](#).

Go To Samples 

In This Help Topic

- [N/portlet Module Members](#)

N/portlet Module Members

Member Type	Name	Return Type	Supported Script Types	Description
Method	portlet.resize()	void	Client scripts	Resizes a form portlet immediately.
	portlet.refresh()	void	Client scripts	Refreshes a form portlet immediately.

N/portlet Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/portlet module.

Create a Form Portlet with a Button That Allows User Adjustments

The following sample shows how to create a form portlet that allows users to adjust its height and width. It creates two text fields representing the height and width of the portlet, measured in pixels. It also creates a button that runs the `portlet.resize()` method to adjust the height and width of the portlet based on the values of the text fields.

This sample also shows how to create a button that uses the `portlet.refresh()` method. When the button is clicked, the portlet is updated to show the current date.

For more information about how a portlet is displayed on the NetSuite dashboard, see the help topic [SuiteScript 2.x Portlet Script Type](#).

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Portlet
4   * @NScriptPortletType form
5   */
6
7  define([], function() {
8      function render(context) {
9          var portletObj = context.portlet;
10         portletObj.title = 'Test Form Portlet';
11         setComponentsForResize();
12         setComponentsForRefresh();
13
14         function setComponentsForResize() {
15             var DEFAULT_HEIGHT = '50';
16             var DEFAULT_WIDTH = '50';
17             var inlineHTMLField = portletObj.addField({
18                 id: 'divfield',
19                 type: 'inlinehtml',
20                 label: 'Test inline HTML'
21             });
22             inlineHTMLField.defaultValue = '<div id=\'divfield_elem\' style=\'border: 1px dotted red; height: '
23             + DEFAULT_HEIGHT + 'px; width: ' + DEFAULT_WIDTH + 'px;\></div>';
24             inlineHTMLField.updateLayoutType({
25                 layoutType: 'normal'
26             });
27             inlineHTMLField.updateBreakType({
28                 breakType: 'startcol'
29             });
30             var resizeHeight = portletObj.addField({
31                 id: 'resize_height',
32                 type: 'text',
33                 label: 'Resize Height'
34             });
35             resizeHeight.defaultValue = DEFAULT_HEIGHT;
36             var resizeWidth = portletObj.addField({
37                 id: 'resize_width',

```

```

37         type: 'text',
38         label: 'Resize Width'
39     });
40     resizeWidth.defaultValue = DEFAULT_WIDTH;
41     var resizeLink = portletObj.addField({
42         id: 'resize_link',
43         type: 'inlinehtml',
44         label: 'Resize link'
45     });
46     resizeLink.defaultValue = resizeLink.defaultValue = '<a id=\'resize_link\' onclick=\'require([\'SuiteScripts/portlet
tApiTestHelper\'], function(portletApiTestHelper) {portletApiTestHelper.resizePortlet();}) \' href=\'#\#\'>Resize</a><br>';
47     }
48
49     function setComponentsForRefresh() {
50         var textField = portletObj.addField({
51             id: 'refresh_output',
52             type: 'text',
53             label: 'Date.now().toString()'
54         });
55         textField.defaultValue = Date.now().toString();
56         var refreshLink = portletObj.addField({
57             id: 'refresh_link',
58             type: 'inlinehtml',
59             label: 'Refresh link'
60         });
61         refreshLink.defaultValue = '<a id=\'refresh_link\' onclick=\'require([\'SuiteScripts/portletApiTestHelper\'],
function(portletApiTestHelper) {portletApiTestHelper.refreshPortlet();}) \' href=\'#\#\'>Refresh</a>';
62     }
63 }
64
65 return {
66     render: render
67 };
68 })
69
70 // portletApiTestHelper.js
71 define(['N/portlet'], function(portlet) {
72     function refreshPortlet() {
73         portlet.refresh();
74     }
75
76     function resizePortlet() {
77         var div = document.getElementById('divfield_elem');
78         var newHeight = parseInt(document.getElementById('resize_height').value);
79         var newWidth = parseInt(document.getElementById('resize_width').value);
80         div.style.height = newHeight + 'px';
81         div.style.width = newWidth + 'px';
82         portlet.resize();
83     }
84
85     return {
86         refreshPortlet: refreshPortlet,
87         resizePortlet: resizePortlet
88     };
89 });

```

portlet.resize()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Resizes a form portlet type immediately.
Returns	Void
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/portlet Module
Since	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/portlet Module Script Sample](#)

```

1 //Add additional code
2 ...
3 portlet.resize();
4 ...
5 //Add additional code

```

portlet.refresh()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Refreshes a form portlet type immediately.
Returns	Void
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/portlet Module
Since	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/portlet Module Script Sample](#)

```

1 ...
2 portlet.refresh();
3 ...

```

N/query Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/query module to create and run queries using the SuiteAnalytics Workbook query process. For more information about SuiteAnalytics Workbook, see the help topic [SuiteAnalytics Workbook Overview](#).

Go To Samples {...}

Using the query module, you can:

- Use multilevel joins to create queries using field data from multiple record types.
- Create conditions (filters) using AND, OR, and NOT logic, as well as formulas and relative dates.
- Sort query results based on the values of multiple columns.
- Load and delete existing saved queries that were created using the SuiteAnalytics Workbook interface.
- View paged query results.
- Use promises for asynchronous .
- Convert query objects to SuiteQL queries and run arbitrary SuiteQL queries.

For more information about creating scripts using the N/query module, see the following help topics:

- [Scripting with the N/query Module](#)
- [Formulas in the N/query Module](#)
- [Relative Dates in the N/query Module](#)
- [SuiteQL in the N/query Module](#)



Important: As you use the N/query module, keep the following considerations in mind:

- The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can load and delete existing queries, but you can't save queries with this module. To save queries, use the SuiteAnalytics Workbook interface. For details, see the help topic [Navigating SuiteAnalytics Workbook](#).
- The N/query module uses a different data source than N/search. To find record types or field IDs for filters or result columns, use the Records Catalog (not the SuiteScript Records Browser). The Records Catalog lists everything available for SuiteAnalytics Workbook and N/query. For more, see the help topic [Records Catalog Overview](#).
- The N/query module doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article [Outbound HTTPs in an unauthenticated client-side context](#).

In This Help Topic

- [N/query Module Members](#)
- [Column Object Members](#)
- [Component Object Members](#)
- [Condition Object Members](#)
- [Page Object Members](#)
- [PagedData Object Members](#)
- [PageRange Object Members](#)
- [Query Object Members](#)

- [RelativeDate Object Members](#)
- [Result Object Members](#)
- [ResultSet Object Members](#)
- [Sort Object Members](#)
- [SuiteQL Object Members](#)

N/query Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	query.Column	Object	Client and server scripts	The field types (query result columns) that are displayed from the query results. Use Query.createColumn(options) or Component.createColumn(options) to create this object.
	query.Component	Object	Client and server scripts	One component of the query definition. The query definition always contains at least one component that encapsulates the initial query type. Queries with joins contain multiple components that encapsulate the join relationships. The initial component (Query.root) is automatically created with the query definition (query.Query). Use Query.autoJoin(options) or Component.autoJoin(options) to create subsequent components.
	query.Condition	Object	Client and server scripts	A condition. A condition narrows the query results. Use Query.createCondition(options) or Component.createCondition(options) to create this object.
	query.Page	Object	Client and server scripts	One page of the paged query results.
	query.PagedData	Object	Client and server scripts	A set of paged query results. This object also contains information about the set of paged results it encapsulates.
	query.PageRange	Object	Client and server scripts	A range of pages from the paged query results.
	query.Period	Object	Client and server scripts	A period of time to use in query conditions.
	query.RelativeDate	Object	Client and server scripts	A relative date to use in query conditions.
	query.Result	Object	Client and server scripts	A single row of the query result set.
	query.ResultSet	Object	Client and server scripts	The set of results returned by the query.
	query.Query	Object	Client and server scripts	The query definition. Use query.create(options) or query.load(options) to create this object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				 Important: The creation of this object is the first step in creating a query with the N/query Module .
	query.Sort	Object	Client and server scripts	A sort that is placed on a particular query result column. Use Query.createSort(options) or Component.createSort(options) to create this object.
	query.SuiteQL	Object	Client and server scripts	A SuiteQL query. Use Query.toSuiteQL() to create this object.
Method	query.create(options)	query.Query	Client and server scripts	Creates the query definition.  Important: The invocation of this method is the first step in creating a query with the N/query Module .
	query.createPeriod(options)	query.Period	Client and server scripts	Creates a query.Period object that represents a period of time.
	query.createRelativeDate(options)	query.RelativeDate	Client and server scripts	Creates a query.RelativeDate object that represents a date relative to the current date.
	query.delete(options)	void	Client and server scripts	Deletes an existing query that was created using the SuiteAnalytics Workbook UI. The deleted query is no longer available and cannot be modified or executed.
	query.listTables(options)	<Object>	Client and server scripts	Lists the table view objects that are included in a workbook in SuiteAnalytics Workbook.
	query.load(options)	query.Query	Client and server scripts	Loads an existing query that was created using the SuiteAnalytics Workbook UI. The loaded query can be modified (for example, by setting additional property values), joined with other query types, and executed in the same way as queries created using query.create(options) .
	query.load.promise(options)	Promise Object	Client and server scripts	Asynchronously loads an existing query that was created using the SuiteAnalytics Workbook UI.
	query.runSuiteQL(options)	query.ResultSet	Client and server scripts	Runs an arbitrary SuiteQL query.
	query.runSuiteQL.promise(options)	Promise Object	Client and server scripts	Asynchronously runs an arbitrary SuiteQL query.
	query.runSuiteQLPaged(options)	query.PagedData	Client and server scripts	Runs an arbitrary SuiteQL query as a paged query.
	query.runSuiteQLPaged.promise(options)	Promise Object	Client and server scripts	Asynchronously runs an arbitrary query as a paged query.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Enum	query.Aggregate	enum	Client and server scripts	Holds the string values for aggregate functions supported with the N/query Module. This enum is used to pass the aggregate function argument to Component.createColumn(options), Component.createCondition(options), Query.createColumn(options), and Query.createCondition(options).
	query.DateId	enum	Client and server scripts	Holds the string values for supported date codes in relative dates. This enum is used to pass the date ID argument to query.createRelativeDate(options).
	query.FieldContext	enum	Client and server scripts	Holds the string values for the field context to use when creating a column. This enum is used to pass the context argument to Query.createColumn(options) and Component.createColumn(options).
	query.Operator	enum	Client and server scripts	Holds the string values for operators supported with the N/query Module. This enum is used to pass the operator argument to Query.createCondition(options) and Component.createCondition(options).
	query.PeriodAdjustment	enum	Client and server scripts	Holds the string values for adjustment types for a period. This enum is used to pass the adjustment argument to query.createPeriod(options).
	query.PeriodCode	enum	Client and server scripts	Holds the string values for period codes for a period. This enum is used to pass the code argument to query.createPeriod(options).
	query.PeriodType	enum	Client and server scripts	Holds the string values for period types for a period. This enum is used to pass the type argument to query.createPeriod(options).
	query.RelativeDate Range	enum	Client and server scripts	Holds query.RelativeDate object values for supported date ranges in relative dates. This enum is used to pass the values argument to Query.createCondition(options) and Component.createCondition(options).
	query.ReturnType	enum	Client and server scripts	Holds the string values for the formula return types supported with the N/query Module. This enum is used to pass the formula return type argument to Query.createColumn(options), Component.createColumn(options), Query.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				createCondition(options) , and Component.createCondition(options) .
	query.SortLocale	enum	Client and server scripts	Holds the string values for sort locales supported with the N/query Module . This enum is used to pass the sort locale argument to Query.createSort(options) and Component.createSort(options) .
	query.Type	enum	Client and server scripts	Holds the string values for supported query types used in the query definition. This enum is used to pass the initial query type argument to query.create(options) .

Column Object Members

The following members are available for a [query.Column](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	Column.aggregate	string (read-only)	Client and server scripts	An aggregate function that is performed on the query result column. An aggregate function performs a calculation on the column values and returns a single value.
	Column.alias	string (read-only)	Client and server scripts	An alias for this column. An alias is an alternate name for a column, and the alias is used in mapped results.
	Column.component	query.Component (read-only)	Client and server scripts	A reference to the query.Component object to which this query result column belongs.
	Column.context	Object (read-only)	Client and server scripts	The field context for values in the query result column. The field context determines how field values are displayed in the column.
	Column.fieldId	string (read-only)	Client and server scripts	The name of the query result column. This property and the Column.formula property cannot be set at the same time.
	Column.formula	string (read-only)	Client and server scripts	The formula used to create the query result column. This property and the Column.fieldId property cannot be set at the same time.
	Column.groupBy	(read-only)	Client and server scripts	Whether the query results are grouped by this query result column.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
	Column.label	string (read-only)	Client and server scripts	The label for the column.
	Column.type	string (read-only)	Client and server scripts	The return type of the formula used to create the query result column.

Component Object Members

The following members are available for a [query.Component](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	Component.autoJoin(options)	query.Component	Client and server scripts	Creates a join relationship. After you create the initial query definition, use Query.autoJoin(options) to create your first join. Then use this method to create each subsequent join. This method selects the correct join type automatically based on the record types that are being joined.
	Component.createColumn(options)	query.Column	Client and server scripts	Creates a query result column based on the component. Use this method to create columns based on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options) .
	Component.createCondition(options)	query.Condition	Client and server scripts	Creates a condition (filter column) based on the component. Use this method to create conditions based on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options) .
	Component.createSort(options)	query.Sort	Client and server scripts	Creates a sort based on the component. Use this method to create sorts based on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options) .
	Component.join(options)	query.Component	Client and server scripts	Creates a join relationship. This method is an alias to Component.autoJoin(options) . After you create the initial query definition, use Query.autoJoin(options) to create your first join. Then use this method, or Component.autoJoin(options) , to create each subsequent join.
	Component.joinFrom(options)	query.Component	Client and server scripts	Creates an explicit directional join relationship from another component to this component (an inverse join). This method sets the Component.source property on the returned query.Component object.

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
				After you create the initial query definition, use this method to create explicit directional joins from other components to this component.
	Component.joinTo(options)	query.Component	Client and server scripts	Creates an explicit directional join relationship to another component from this component (a join). You can use this method to specify the target of the join when a field can join multiple query types. This method sets the Component.target property on the returned query.Component object. After you create the initial query definition, use this method to create explicit directional joins to other components from this component.
Property	Component.child	Object (read-only)	Client and server scripts	The child components of the component. This property holds an object of key-value pairs. Each key is the name of a child component. Each value is the corresponding child query.Component object.
	Component.parent	string (read-only)	Client and server scripts	The parent query.Component object of the component.
	Component.source	string (read-only)	Client and server scripts	The source query type of the component. The value of this property is set when Component.joinFrom(options) is called to perform an explicit directional join from another component.
	Component.target	string (read-only)	Client and server scripts	The target query type of the component. The value of this property is set when Component.joinTo(options) is called to perform an explicit directional join to another component.
	Component.type	string (read-only)	Client and server scripts	The query type of the component.

Condition Object Members

The following members are available for a [query.Condition](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	Condition.aggregate	string (read-only)	Client and server scripts	An aggregate function that is performed on the condition. An aggregate function performs a calculation on the condition values and returns a single value.
	Condition.children	query.Condition[] (read-only)	Client and server scripts	An array of child conditions used to create the parent condition.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
	Condition.component	query.Component (read-only)	Client and server scripts	A reference to the query.Component object to which this condition belongs.
	Condition.fieldId	string (read-only)	Client and server scripts	The name of the field that is used in the condition.
	Condition.formula	string (read-only)	Client and server scripts	The formula used to create the condition.
	Condition.operator	string (read-only)	Client and server scripts	The name of the operator used to create the condition.
	Condition.type	string (read-only)	Client and server scripts	The return type of the formula used to create the condition.
	Condition.values	string number <string number > (read-only)	Client and server scripts	Value or an array of values used by an operator to create the condition.

Page Object Members

The following members are available for a [query.Page](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	Page.data	query.ResultSet (read-only)	Client and server scripts	The query results contained in this page.
	Page.isFirst	boolean (read-only)	Client and server scripts	Whether this page is the first of the paged query results.
	Page.isLast	boolean (read-only)	Client and server scripts	Whether this page is the last of the paged query results.
	Page.pagedData	query.PagedData (read-only)	Client and server scripts	The set of paged query results that this page is from.
	Page.pageRange	query.PageRange (read-only)	Client and server scripts	The range of query results for this page.

PagedData Object Members

The following members are available for a [query.PagedData](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	PagedData.iterator()	Iterator object	Client and server scripts	Standard SuiteScript 2.0 object for iterating through results.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
	PagedData.fetch(options)	query.Page	Client and server scripts	Retrieves a page in the set of pages included in the PagedData object.
	PagedData.fetch.promise(options)	Promise Object	Client and server scripts	Asynchronously retrieves a page in the set of pages included in the PagedData object.
Property	PagedData.count	number (read-only)	Client and server scripts	The total number of paged query results.
	PagedData.pageRanges	query.PageRange[]	Client and server scripts	An array of page ranges for the set of paged query results.
	PagedData.pageSize	number (read-only)	Client and server scripts	The number of query result rows per page.

PageRange Object Members

The following members are available for a [query.PageRange](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	PageRange.index	number (read-only)	Client and server scripts	The index for this page range.
	PageRange.size	number (read-only)	Client and server scripts	The number of query result rows in this page range.

Period Object Members

The following members are available for a [query.Period](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	Period.adjustment	string (read-only)	Client and server scripts	The adjustment of the period. This property uses values from the query.PeriodAdjustment enum.
	Period.code	string (read-only)	Client and server scripts	The code of the period. This property uses values from the query.PeriodCode enum.
	Period.type	string (read-only)	Client and server scripts	The type of the period. This property uses values from the query.PeriodType enum.

Query Object Members

The following members are available for a [query.Query](#) object.

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
Method	Query.and(conditions)	query.Condition	Client and server scripts	Creates a new condition (a query.Condition object) that corresponds to a logical conjunction (AND) of the arguments passed to the method. The arguments must be one or more query.Condition objects.
	Query.autoJoin(options)	query.Component	Client and server scripts	Creates a join relationship. After you create the initial query definition, use this method to create your first join or subsequent joins from the root component of the query. This method selects the correct join type automatically based on the record types that are being joined.
	Query.createColumn(options)	query.Column	Client and server scripts	Creates a query result column based on the query.Query object. Use this method to create columns on the initial query definition created with query.create(options) .
	Query.createCondition(options)	query.Condition	Client and server scripts	Creates a condition (filter column) based on the query.Query object. Use this method to create conditions on the initial query definition created with query.create(options) .
	Query.createSort(options)	query.Sort	Client and server scripts	Creates a sort based on the query.Query object. The query.Sort object describes a sort that is placed on a particular query result column or condition.
	Query.join(options)	query.Component	Client and server scripts	Creates a join relationship. This method is an alias to Query.autoJoin(options) . After you create the initial query definition, use this method, or Query.autoJoin(options) , to create your first join.
	Query.joinFrom(options)	query.Component	Client and server scripts	Creates an explicit directional join relationship from another component to the root component of the search definition (an inverse join). This method sets the Component.source property on the returned query.Component object. After you create the initial query definition, use this method to create your first join as an explicit directional join from another component to this component.
	Query.joinTo(options)	query.Component	Client and server scripts	Creates an explicit directional join relationship to another component from this component (a forward join). You can use this method to specify the target of the join when a field can join multiple query types. This method sets the Component.target

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
				property on the returned query.Component object. After you create the initial query definition, use this method to create your first join as an explicit directional join to another component from this component.
	Query.not(condition)	query.Condition	Client and server scripts	Creates a new condition (a query.Condition object) that corresponds to a logical negation (NOT) of the argument passed to the method. The argument must be a query.Condition object.
	Query.or(conditions)	query.Condition	Client and server scripts	Creates a new condition (a query.Condition object) that corresponds to a logical disjunction (OR) of the arguments passed to the method. The arguments must be one or more query.Condition objects.
	Query.run(options)	query.ResultSet	Client and server scripts	Executes the query and returns the query result set.
	Query.run.promise()	query.ResultSet	Client and server scripts	Executes the query asynchronously and returns the query result set.
	Query.runPaged(options)	query.PagedData	Client and server scripts	Executes the query and returns a set of paged results.
	Query.runPaged.promise(options)	query.PagedData	Client and server scripts	Executes the query asynchronously and returns a set of paged results.
	Query.toSuiteQL()	query.SuiteQL	Client and server scripts	Converts this query.Query object to its corresponding SuiteQL representation.
Property	Query.child	Object (read-only)	Client and server scripts	A reference to children of the root component of the query definition. The value of this property is an object of key-value pairs. Each key is the name of a child component. Each respective value is the corresponding query.Component object.
	Query.columns	query.Column[]	Client and server scripts	An array of query result columns returned from the query. Before you perform the query, you must assign all created columns as values to this property.
	Query.condition	query.Condition object	Client and server scripts	The parent condition that narrows the query results. Before you perform the query, you must assign your simple or complex conditions to this property.
	Query.id	number (read-only)	Client and server scripts	The ID of the query definition. This property has a value only for existing queries that are loaded using query.load(options) . If you create a query using query.create(options) but do not save it, this property is null.
	Query.name	string (read-only)	Client and server scripts	The name of the query definition.

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
				This property has a value only for existing queries that are loaded using query.load(options) . If you create a query using query.create(options) but do not save it, this property is null.
	Query.root	query.Component (read-only)	Client and server scripts	The root component of the query definition.
	Query.sort	query.Column[] (read-only)	Client and server scripts	An array of query result columns used for sorting.
	Query.type	string (read-only)	Client and server scripts	The query type of the initial query definition.

RelativeDate Object Members

The following members are available for a [query.RelativeDate](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	RelativeDate.dateId	string (read-only)	Client and server scripts	The ID of the relative date.
	RelativeDate.end	Object (read-only)	Client and server scripts	The end point of the relative date.
	RelativeDate.interval	Object (read-only)	Client and server scripts	The interval of the relative date (from the RelativeDate.start point to the RelativeDate.end point).
	RelativeDate.isRange	boolean (read-only)	Client and server scripts	Whether the relative date represents a range of dates or a specific moment in time.
	RelativeDate.start	Object (read-only)	Client and server scripts	The start point of the relative date.
	RelativeDate.value	number (read-only)	Client and server scripts	The value of the relative date.

Result Object Members

The following members are available for a [query.Result](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	Result.asMap()	Object	Client and server scripts	Returns the query result as an object with the columns mapped to the result values.
	Result.getValue(options)	<boolean number string Date null> (read-only)	Client and server scripts	Gets the value at a given index in Result.values .

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	ResultSet.values	Array <boolean number string Date null> (read-only)	Client and server scripts	The result values.

ResultSet Object Members

The following members are available for a [query.ResultSet](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	ResultSet.asMappedResults()	Object[]	Client and server scripts	Returns a query result set as an array of mapped results.
	ResultSet.iterator()	Iterator object	Client and server scripts	Standard SuiteScript 2.0 object for iterating through results.
Property	ResultSet.columns	query.Column [] (read-only)	Client and server scripts	An array of query result column references.
	ResultSet.results	query.Result [] (read-only)	Client and server scripts	An array of query.Result objects.
	ResultSet.types	string[] (read-only)	Client and server scripts	An array of the return types for ResultSet.results .

Sort Object Members

The following members are available for a [query.Sort](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Property	Sort.ascending	boolean	Client and server scripts	Whether the sort direction is ascending.
	Sort.caseSensitive	boolean	Client and server scripts	Whether the sort is case sensitive. If a sort is case sensitive (and the sort direction is ascending), rows with column values that start with uppercase letters are listed before rows with column values that start with lowercase letters. If a sort is not case sensitive, uppercase and lowercase letters are treated the same.
	Sort.column	query.Column (read-only)	Client and server scripts	The query result column that the query results are sorted by.
	Sort.locale	string	Client and server scripts	The locale to use for the sort.

Member Type	Name	Return Type/ Value Type	Supported Script Types	Description
				A locale represents a combination of language and region, and it can affect how certain values (such as strings) are sorted.
	Sort.nullsLast	boolean	Client and server scripts	Whether query results with null values are listed at the end of the query results.

SuiteQL Object Members

The following members are available for a [query.SuiteQL](#) object.

Member Type	Name	Return Type/Value Type	Supported Script Types	Description
Method	SuiteQL.run()	query.ResultSet	Client and server scripts	Runs the SuiteQL query and returns the query results.
	SuiteQL.runPaged(options)	query.PagedData	Client and server scripts	Runs the SuiteQL query as a paged query and returns the paged query results.
Property	SuiteQL.columns	query.Column[]	Client and server scripts	Describes the result columns to be returned from the query.
	SuiteQL.params	<string number > (read-only)	Client and server scripts	Contains the parameters for the query.
	SuiteQL.query	string (read-only)	Client and server scripts	Holds the string representation of the query.
	SuiteQL.type	string (read-only)	Client and server scripts	Describes the type of the query. This property uses values from the query.Type enum.

N/query Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/query module:

- [Create a Query for Customer Records and Run It as a Non-Paged Query](#)
- [Create a Query for Transaction Records and Run It as a Paged Query](#)
- [Convert a Query to a SuiteQL and Run It](#)
- [Run an Arbitrary SuiteQL Query](#)
- [Create a Query for a Custom Field](#)
- [Create a Query Using a Specific Record Field](#)

Create a Query for Customer Records and Run It as a Non-Paged Query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a query for customer records, joins the query with two other query types, and runs the query.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/query'], query => {
5       // Create a query definition for customer records
6       let myCustomerQuery = query.create({
7           type: query.Type.CUSTOMER
8       });
9
10      // Join the original query definition based on the salesrep field. In a customer
11      // record, the salesrep field contains a reference to an employee record. When you
12      // join based on this field, you are joining the query definition with the employee
13      // query type, and you can access the fields of the joined employee record in
14      // your query.
15      let mySalesRepJoin = myCustomerQuery.autoJoin({
16          fieldId: 'salesrep'
17      });
18
19      // Join the joined query definition based on the location field. In an employee
20      // record, the location field contains a reference to a location record.
21      let myLocationJoin = mySalesRepJoin.autoJoin({
22          fieldId: 'location'
23      });
24
25      // Create conditions for the query
26      let firstCondition = myCustomerQuery.createCondition({
27          fieldId: 'id',
28          operator: query.Operator.EQUAL,
29          values: 107
30      });
31      let secondCondition = myCustomerQuery.createCondition({
32          fieldId: 'id',
33          operator: query.Operator.EQUAL,
34          values: 2647
35      });
36      let thirdCondition = mySalesRepJoin.createCondition({
37          fieldId: 'email',
38          operator: query.Operator.START_WITH_NOT,
39          values: 'example'
40      });
41
42      // Combine conditions using and() and or() operator methods. In this example,
43      // the combined condition states that the id field of the customer record must
44      // have a value of either 107 or 2647, and the email field of the employee
45      // record (the record that is referenced in the salesrep field of the customer
46      // record) must not start with 'example'.
47      myCustomerQuery.condition = myCustomerQuery.and(
48          thirdCondition, myCustomerQuery.or(firstCondition, secondCondition)

```

```

49     });
50
51     // Create query columns
52     myCustomerQuery.columns = [
53         myCustomerQuery.createColumn({
54             fieldId: 'entityid'
55         }),
56         myCustomerQuery.createColumn({
57             fieldId: 'id'
58         }),
59         mySalesRepJoin.createColumn({
60             fieldId: 'entityid'
61         }),
62         mySalesRepJoin.createColumn({
63             fieldId: 'email'
64         }),
65         mySalesRepJoin.createColumn({
66             fieldId: 'hiredate'
67         }),
68         myLocationJoin.createColumn({
69             fieldId: 'name'
70         })
71     ];
72
73     // Sort the query results based on query columns
74     myCustomerQuery.sort = [
75         myCustomerQuery.createSort({
76             column: myCustomerQuery.columns[3]
77         }),
78         myCustomerQuery.createSort({
79             column: myCustomerQuery.columns[0],
80             ascending: false
81         })
82     ];
83
84     // Run the query
85     let resultSet = myCustomerQuery.run();
86
87     // Retrieve and log the results
88     let results = resultSet.results;
89     for (let i = results.length - 1; i >= 0; i--)
90         log.debug(results[i].values);
91     log.debug(resultSet.types);
92 });

```

Create a Query for Transaction Records and Run It as a Paged Query

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a query for transaction records, joins the query with another query type, and runs the query as a paged query.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2  * @NApiVersion 2.1

```

```

3  */
4  require(['N/query'], query => {
5      // Create a query definition for transaction records
6      let myTransactionQuery = query.create({
7          type: query.Type.TRANSACTION
8      });
9
10     // Join the original query definition based on the employee field. In a transaction
11     // record, the employee field contains a reference to an employee record. When you
12     // join based on this field, you are joining the query definition with the employee
13     // query type, and you can access the fields of the joined employee record in
14     // your query.
15     let myEmployeeJoin = myTransactionQuery.autoJoin({
16         fieldId: 'employee'
17     });
18
19     // Create a condition for the transaction query
20     let transactionCondition = myTransactionQuery.createCondition({
21         fieldId: 'isreversal',
22         operator: query.Operator.IS,
23         values: true
24     });
25     myTransactionQuery.condition = transactionCondition;
26
27     // Create a query column
28     myTransactionQuery.columns = [
29         myEmployeeJoin.createColumn({
30             fieldId: 'subsidiary'
31         })
32     ];
33
34     // Sort the query results based on a query column
35     myTransactionQuery.sort = [
36         myTransactionQuery.createSort({
37             column: myTransactionQuery.columns[0],
38             ascending: false
39         })
40     ];
41
42     // Run the query as a paged query with 10 results per page
43     let results = myTransactionQuery.runPaged({
44         pageSize: 10
45     });
46
47     log.debug(results.pageRanges.length);
48     log.debug(results.count);
49
50     // Retrieve the query results using an iterator
51     let iterator = results.iterator();
52     iterator.each(function(result) {
53         let page = result.value;
54         log.debug(page.pageRange.size);
55         return true;
56     });
57
58     // Alternatively, retrieve the query results by looping through
59     // each result
60     for (let i = 0; i < results.pageRanges.length; i++) {
61         let page = results.fetch(i);
62         log.debug(page.pageRange.size);
63     }
64 });

```

Convert a Query to a SuiteQL and Run It

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a query for customer records, converts it to its SuiteQL representation, and runs it.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/query'], function(query) {
5      var myCustomerQuery = query.create({
6          type: query.Type.CUSTOMER
7      });
8
9      myCustomerQuery.columns = [
10         myCustomerQuery.createColumn({
11             fieldId: 'entityid'
12         }),
13         myCustomerQuery.createColumn({
14             fieldId: 'email'
15         })
16     ];
17
18     myCustomerQuery.condition = myCustomerQuery.createCondition({
19         fieldId: 'isperson',
20         operator: query.Operator.IS,
21         values: [true]
22     });
23
24     var mySQLCustomerQuery = myCustomerQuery.toSuiteQL();
25
26     var results = mySQLCustomerQuery.run();
27 });

```

Run an Arbitrary SuiteQL Query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample constructs a SuiteQL query string, runs the query as a paged query, and iterates over the results.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/query'], function(query) {
5      var sql =
6          "SELECT " +
7          "    scriptDeployment.primarykey, scriptexecutioncontextmap.executioncontext " +
8          " FROM " +
9          "    scriptDeployment, scriptexecutioncontextmap " +
10         " WHERE " +
11         "    scriptexecutioncontextmap.scriptrecord = scriptDeployment.primarykey " +
12         " AND " +
13         "    scriptexecutioncontextmap.executioncontext IN ('WEBSTORE', 'WEBAPPLICATION')";
14
15     var resultIterator = query.runSuiteQLPaged({
16         query: sql,
17         pageSize: 10
18     }).iterator();
19

```

```

20 resultIterator.each(function(page) {
21     var pageIterator = page.value.data.iterator();
22     pageIterator.each(function(row) {
23         log.debug('ID: ' + row.value.getValue(0) + ', Context: ' + row.value.getValue(1));
24         return true;
25     });
26     return true;
27 });
28 });

```

Create a Query for a Custom Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a query for a custom field, `custrecord_my_custom_field`, and obtains the internal ID of the field.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /*
2   * @NApiVersion 2.x
3   */
4  require(['N/query'], function(query) {
5      var customFieldIdQuery = query.create({
6          type: query.Type.CUSTOM_FIELD
7      });
8      customFieldIdQuery.columns = [
9          customFieldIdQuery.createColumn({
10             fieldId: 'internalid'
11         })
12     ];
13     customFieldIdQuery.condition = customFieldIdQuery.createCondition({
14         fieldId: 'scriptid',
15         operator: query.Operator.IS,
16         values: 'custrecord_my_custom_field'
17     });
18
19     var results = customFieldIdQuery.run().asMappedResults();
20     var customFieldInternalId = results[0].internalid;
21     log.debug({
22         title: 'Internal ID of the custom field is ',
23         details: customFieldInternalId
24     });
25 });

```

Create a Query Using a Specific Record Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a query for records using the value of the `operationdisplaytext` field. If you run this sample in your account, be sure to replace the placeholder value `<transactionId>` with a valid value from your account.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**

```

```

2  * @NApiVersion 2.x
3  */
4  require(['N/query'], function(query) {
5      var mfgComponent = query.create({
6          type: query.Type.MANUFACTURING_COMPONENT
7      });
8
9      var mfgOperation = mfgComponent.autoJoin({
10         fieldId: 'operationdisplaytext'
11     });
12
13     mfgComponent.columns = [
14         mfgComponent.createColumn({
15             fieldId: 'operationdisplaytext'
16         }),
17         mfgComponent.createColumn({
18             fieldId: 'item'
19         }),
20         mfgOperation.createColumn({
21             fieldId: 'operationsequence'
22         })
23     ];
24
25     mfgComponent.condition = mfgComponent.and(
26         mfgComponent.createCondition({
27             fieldId: 'transaction',
28             operator: query.Operator.ANY_OF,
29             values: <transactionId>
30         }),
31         mfgOperation.createCondition({
32             fieldId: 'operationsequence',
33             operator: query.Operator.EQUAL,
34             values: 20
35         })
36     );
37
38     var results = mfgComponent.run();
39 });

```

Scripting with the N/query Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The N/query module lets you create and run queries using the SuiteAnalytics Workbook query engine. Before you start creating your queries, you should be familiar with the module objects and how to use them, as well as some of the terminology used in the N/query module. You can also take a look at a script walkthrough that explains how to create queries using different approaches.

- [N/query Module Objects](#)
- [N/query Module Terminology](#)

N/query Module Objects

The N/query module includes the following objects:

- [Query and Component Objects](#)
- [Condition Object](#)
- [RelativeDate Object](#)
- [Column Object](#)
- [Sort Object](#)
- [ResultSet and Result Objects](#)
- [Page, PagedData, and PageRange Objects](#)

Query and Component Objects

The [query.Query](#) object and the [query.Component](#) object are the primary building blocks for a query created with the N/query module. Each query creates one [query.Query](#) object and one or more [query.Component](#) objects. The [query.Query](#) object encapsulates the query definition, and the [query.Component](#) object encapsulates one component of the query definition.

To create a query with the N/query module:

1. Use the [query.create\(options\)](#) method to create your initial query definition (the [query.Query](#) object). The initial query definition uses one search type. For available search types, see [query.Type](#).
2. After you create the initial query definition, use [Query.autoJoin\(options\)](#), [Query.joinFrom\(options\)](#), or [Query.joinTo\(options\)](#) to create your first join.
3. Use any of the following methods to create subsequent joins:
 - [Query.autoJoin\(options\)](#)
 - [Query.joinFrom\(options\)](#)
 - [Query.joinTo\(options\)](#)
 - [Component.autoJoin\(options\)](#)
 - [Component.joinFrom\(options\)](#)
 - [Component.joinTo\(options\)](#)

The query definition always contains at least one [query.Component](#) object. Each new component is created as a child of the previous component, and all components exist as children of the query definition. You can think of a component as a building block; each new component builds on the previous component created. The last component created encapsulates the relationship between it and all of its parent components.

Queries with joins contain multiple components. The query definition contains a child [query.Component](#) object for each of the following:

- **The initial query definition:** The initial [query.Component](#) object is called the root component. It encapsulates the initial search type passed to [query.create\(options\)](#). The root component is automatically created with the initial query definition and is a child to the [query.Query](#) object. The [Query.root](#) property contains a reference to the root component.
- **The first join:** The second [query.Component](#) object is created with [Query.autoJoin\(options\)](#), [Query.joinFrom\(options\)](#), or [Query.joinTo\(options\)](#). It encapsulates the relationship between the initial query definition and the second search type. This relationship is determined by the join ID passed to these methods, as well as whether [Query.joinFrom\(options\)](#) or [Query.joinTo\(options\)](#) was used to create an explicit directional join. The second [query.Component](#) object is a child to the root component.
- **Each subsequent join:** The third [query.Component](#) object is created with [Component.autoJoin\(options\)](#), [Component.joinFrom\(options\)](#), or [Component.joinTo\(options\)](#). All subsequent joins are also created using these methods. Each of these [query.Component](#) objects encapsulates the relationship between all previous search types and the new search type. This relationship is determined by the join ID passed to these methods, as well as whether [Component.joinFrom\(options\)](#) or [Component.joinTo\(options\)](#) was used to create an explicit directional join.

Condition Object

A condition narrows the query results. The [query.Condition](#) object performs the same function as the [search.Filter](#) object in the [N/search Module](#). The primary difference is that [query.Condition](#) objects can contain other [query.Condition](#) objects.

To create conditions:

- Use [Query.createCondition\(options\)](#) to create conditions for the initial query definition created with [query.create\(options\)](#).
- Use [Component.createCondition\(options\)](#) to create conditions for the join relationships created with [Query.autoJoin\(options\)](#), [Query.joinFrom\(options\)/Query.joinTo\(options\)](#), [Component.autoJoin\(options\)](#), or [Component.joinFrom\(options\)/Component.joinTo\(options\)](#).
- If you have multiple conditions, use [Query.and\(conditions\)](#), [Query.or\(conditions\)](#), and [Query.not\(condition\)](#) to create a new nested condition.
- If you want to use a formula to define your conditions, assign the formula to [Condition.formula](#).
- Assign your simple or nested conditions as array values to [Query.condition](#).

RelativeDate Object

The [query.RelativeDate](#) object represents a date that is relative to the current date. You can use relative dates when you create query conditions.

To create relative dates:

- Use [query.createRelativeDate\(options\)](#) to create a [query.RelativeDate](#) object. When you call [query.createRelativeDate\(options\)](#), use the values in the [query.DateId](#) enum to specify a date that is relative to the current date.
- Use [Query.createCondition\(options\)](#) or [Component.createCondition\(options\)](#) to create a condition using the [query.RelativeDate](#) object. Alternatively, you can create a condition using values in the [query.RelativeDateRange](#) enum.
- If you have multiple conditions, use [Query.and\(conditions\)](#), [Query.or\(conditions\)](#), and [Query.not\(condition\)](#) to create a new nested condition.
- Assign your simple or nested conditions as array values to [Query.condition](#).

Column Object

The [query.Column](#) object is the equivalent of the [search.Column](#) object in the [N/search Module](#). The [query.Column](#) object describes the field types (columns) that are displayed from the query results.

To create columns:

- Use [Query.createColumn\(options\)](#) to create a column on the initial query definition created with [query.create\(options\)](#).
- Use [Component.createColumn\(options\)](#) to create a column on a join relationship created with [Query.autoJoin\(options\)](#), [Query.joinFrom\(options\)/Query.joinTo\(options\)](#), [Component.autoJoin\(options\)](#), or [Component.joinFrom\(options\)/Component.joinTo\(options\)](#).
- If you want to use a formula to define your columns, assign the formula to [Column.formula](#).
- Assign all created columns as array values to [Query.columns](#).

Sort Object

The [query.Sort](#) object describes how query results are sorted (for example, ascending or descending, case sensitive or case insensitive, and so on).

To create a sort:

- Use [Query.createSort\(options\)](#) to create a sort on the initial query definition created with [query.create\(options\)](#).
- Use [Component.createSort\(options\)](#) to create a sort based on a join relationship created with [Query.autoJoin\(options\)](#), [Query.joinFrom\(options\)/Query.joinTo\(options\)](#), [Component.autoJoin\(options\)](#), or [Component.joinFrom\(options\)/Component.joinTo\(options\)](#).

- Assign all created sorts as array values to [Query.sort](#).

ResultSet and Result Objects

When you are ready to execute your query, call [Query.run\(options\)](#). This method returns a [query.ResultSet](#) object, which encapsulates the metadata for the set of results returned by the query.

To access your query results, iterate through the [ResultSet.results](#) array. Each member of the [ResultSet.results](#) array is a [query.Result](#) object. The [query.Result](#) object encapsulates a single row of the result set.

Page, PagedData, and PageRange Objects

You also can execute your query by calling [Query.runPaged\(options\)](#). This method returns a [query.PagedData](#) object, which encapsulates a set of paged query results.

To access your query results, iterate through the paged query results using [PagedData.iterator\(\)](#). You can access each page of the query results, which are represented by [query.Page](#) objects. The [query.PageRange](#) object encapsulates the range of query results for a page.

N/query Module Terminology

Term	Definition	For More Information
Aggregate function	An aggregate function performs a calculation on a column of values and returns a single value. You can add aggregate functions to conditions and query results columns.	■ query.Aggregate ■ Component.createColumn(options) ■ Component.createCondition(options) ■ Query.createColumn(options) ■ Query.createCondition(options)
Column	A column describes the field types (columns) that are displayed from the query results. A column is also known as a query results column.	■ query.Column
Component	When you script queries with the N/query module, your query is made up of one or more components, which are represented as query.Component objects. You can think of a component as a building block; each new component builds on the previous component created. ■ The first component created represents the initial search type and is a child of query.Query. ■ Each subsequent component created is a child of the previous component. ■ The last component created encapsulates the join relationship between it and all of its parent components. A query always contains at least one component: the root component. When you create the initial query definition using query.create(options), the root component is created automatically. Queries with joins contain multiple components. A new component is created each time you create a join using one of the following methods: ■ Query.autoJoin(options), Query.joinFrom(options), or Query.joinTo(options) ■ Component.autoJoin(options), Component.joinFrom(options), or Component.joinTo(options)	■ query.Component

Term	Definition	For More Information
Condition	A condition narrows the query results.	■ query.Condition
Formula	Formulas can be used to create conditions and columns.	■ Formulas in Searches ■ SQL Expressions
Group	You can summarize your query results into unique groups of column values.	■ Column.groupBy
Join	A join lets you create a query based on a field type that is shared between two record types. You can use Query.autoJoin(options) and Component.autoJoin(options) to create a join relationship automatically based on a field that you specify. You can use Query.joinFrom(options)/Query.joinTo(options) and Component.joinFrom(options)/Component.joinTo(options) to create explicit directional join relationships from one component to another.	■ query.Query ■ query.Component
Page	A page represents one page from a set of paged query results. When you create a query with the N/query module, you can return the results as one result set or a set of paged results.	■ Query.runPaged(options) ■ query.PagedData ■ query.PageRange ■ query.Page
Paged data	Paged data represents a set of paged query results.	■ Query.runPaged(options) ■ query.PagedData ■ query.PageRange ■ query.Page
Page range	A page range is a set of pages from a set of paged query results.	■ Query.runPaged(options) ■ query.PagedData ■ query.PageRange ■ query.Page
Relative date	A relative date is a date that is relative to the current date. You can use relative dates when you create query conditions.	■ query.RelativeDate ■ query.createRelativeDate(options) ■ query.RelativeDateRange
Result	A result is a single row from a result set.	■ Query.run(options) ■ query.ResultSet ■ query.Result
Result set	A result set is a set of query results.	■ Query.run(options) ■ query.ResultSet ■ query.Result
Query definition	The query definition is the initial search type you define, plus any subsequent joins you define. The initial query definition is created with query.create(options) .	■ query.Query
Query type	The query type is the initial query type of your query definition. It represents the record type you want to query for. It is set with the query.Type enum during the execution of query.create(options) . For example, if you want to query for customer records, specify <code>query.Type.CUSTOMER</code> as the query type when you call query.create(options) .	■ query.Query ■ query.Type

Term	Definition	For More Information
Sort	A sort is placed on a query results column to describe how the query results are sorted (for example, ascending or descending, case sensitive or case insensitive, and so on).	■ query.Sort ■ Query.createSort(options) ■ Component.createSort(options)

Formulas in the N/query Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When you create a query using the N/query module, you can specify columns and conditions for the query. Columns describe the field types (or columns) that are displayed from the query results, and conditions narrow the query results based on certain criteria. You create a column using [Query.createColumn\(options\)](#), and you create a condition using [Query.createCondition\(options\)](#). Both of these methods let you create a column or condition in two ways:

- Use the `fieldId` parameter to explicitly specify the field on which to create the column or condition.
- Use the `formula` parameter to specify a formula to create the column or condition.

You can use formulas to perform a calculation to determine the column or condition value based on the values of other fields in the record. For example, consider a situation in which you are working with Customer records that include custom fields. These custom fields contain the amount of stock for various items (50 units of item A, 24 units of item B, and so on). In your query results, you want to include a column that calculates and displays the total amount of stock for all items for a Customer. If the Customer records include three custom stock fields, you can create the result column as follows:

```

1 query.createColumn({
2   formula: '{item_A_stock} + {item_B_stock} + {item_C_stock}',
3   type: query.ReturnType.INTEGER
4 });

```

When you use a formula to create a column or condition, you must also use the `type` parameter to specify the return type of the formula. This parameter accepts values from the [query.ReturnType](#) enum. Defining the formula's return type might be required if the return type cannot be determined automatically based on the formula. When you set the `type` parameter, the return value is properly formatted based on the data type that you specify.

For more information about formulas, see the help topics [Formulas in Searches](#) and [SQL Expressions](#).

Formulas in Joined Queries

You can join your queries with other record types. Joining queries lets you obtain and display query results with field values from multiple record types. When you use a formula in a joined query, you must use fully qualified field IDs to access the fields in each joined record type. You must specify the full join trail from the base record type. The join trail differs depending on the record types and join type.

Use the `^` and `<` operators to access fields in joined queries. You can use these operators when working with formulas in SuiteScript or the NetSuite UI. Use the `^` operator to access fields in record types that are joined using [Query.joinTo\(options\)](#) or [Component.joinTo\(options\)](#). This type of join is also known as a polymorphic join. Use the `<` operator to access fields in record types that are joined using [Query.joinFrom\(options\)](#) or [Component.joinFrom\(options\)](#). This type of join is also known as an inverse

join. When you use [Query.autoJoin\(options\)](#) or [Component.autoJoin\(options\)](#), you do not need to use the ^ or < operators to access fields in the joined query.

The following table lists common join operations and the corresponding join trail.

Join Type	Join Operation	Join Trail
Automatic using Query.autoJoin(options) or Component.autoJoin(options)	<pre> 1 // The base record type is Customer 2 var myCustomerQuery = query.create({ 3 type: query.Type.CUSTOMER 4 }); 5 6 // The joined record type is Employee 7 var mySalesRepJoin = myCustomer 8 Query.autoJoin({ 9 fieldId: 'salesrep' 10 }); </pre>	Base record fields (Customer) - customer.<baseFieldName> - Example: customer.email Joined record fields (Employee) - customer.salesrep.<joinedFieldName> - Example: customer.salesrep.phone
Polymorphic using Query.joinTo(options) or Component.joinTo(options)	<pre> 1 // The base record type is Transaction 2 var myTransactionQuery = query.create({ 3 type: query.Type.TRANSACTION 4 }); 5 6 // The joined record type is Employee 7 var myEmployeeJoin = myTransaction 8 Query.joinTo({ 9 fieldId: 'createdby', 10 target: 'employee' 11 }); </pre>	Base record fields (Transaction) - transaction.<baseFieldName> - Example: transaction.entity Joined record fields (Employee) - transaction.<baseFieldName>^employee.<joinedFieldName> - Example: transaction.createdby^employee.email
Inverse using Query.joinFrom(options) or Component.joinFrom(options)	<pre> 1 // The base record type is Employee 2 var myEmployeeQuery = query.create({ 3 type: query.Type.EMPLOYEE 4 }); 5 6 // The joined record type is Transaction 7 var myTransactionJoin = myEmployee 8 Query.joinFrom({ 9 fieldId: 'entity', 10 source: 'transaction' 11 }); </pre>	Base record fields (Employee) - employee.<baseFieldName> - Example: employee.entityid Joined record fields (Transaction) - employee.<baseFieldName><transaction.daysoverduesearch - Example: employee.entity<transaction.daysoverduesearch

Join Trail Formatting

When you use join trails to access fields in joined queries, you can add whitespace characters and parentheses to improve the readability of your formulas. For example, consider this join trail:

employee.entity<transaction.daysoverduesearch

The following join trails are equivalent to this one:

- employee.entity < transaction.daysoverduesearch
- employee.(entity<transaction).daysoverduesearch

Relative Dates in the N/query Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can use relative dates when you create query conditions. The [query.RelativeDate](#) object represents a specific date or moment in time relative to the current date. Each of the values in the

`query.RelativeDateRange` enum represents a range of dates relative to the current date. When you use a `query.RelativeDate` object or `query.RelativeDateRange` enum value to create a query condition, make sure that you use an operator that makes sense for the relative date that you provide to `Query.createCondition(options)` or `Component.createCondition(options)`. The `query.Operator` enum contains the supported operators for the N/query module, but not all operators apply to relative dates. Use the following operators with relative dates:

- AFTER
- AFTER_NOT
- BEFORE
- BEFORE_NOT
- ON
- ON_NOT
- ON_OR_AFTER
- ON_OR_AFTER_NOT
- ON_OR_BEFORE
- ON_OR_BEFORE_NOT
- WITHIN
- WITHIN_NOT

When you create a query condition using the `WITHIN` or `WITHIN_NOT` operators and a `query.RelativeDate` object, the condition uses the current date as one of the boundaries of the date range. For example, consider the following `query.RelativeDate` object that represents a date two days before the current date:

```
1 var myDatesAgo = query.createRelativeDate({
2   dateId: query.DateId.DAYS_AGO,
3   value: 2
4 });
```

You can use this `myDatesAgo` object when you create a query condition. Consider the following query condition that is created using the `WITHIN` operator and this `myDatesAgo` object:

```
1 var myCondition = myQuery.createCondition({
2   fieldId: 'trandate',
3   operator: query.Operator.WITHIN,
4   values: myDatesAgo
5 });
```

This query condition matches dates that are between two days ago and the current date (the day before yesterday, yesterday, and today).

Conversely, consider the following `query.RelativeDate` object that represents a date two days after the current date:

```
1 var myDatesFromNow = query.createRelativeDate({
2   dateId: query.DateId.DAYS_FROM_NOW,
3   value: 2
4 });
```

If you create a query condition using the `WITHIN` operator and this `myDatesFromNow` object, the condition matches dates that are between the current date and two days from now (today, tomorrow, and the day after tomorrow).

You can use the `query.RelativeDate` object, the [query.RelativeDateRange](#) enum, and the `WITHIN` operator to specify complex date ranges. You can do this in several ways:

- Use a single `query.RelativeDate` object or [query.RelativeDateRange](#) enum value. When you use a single `query.RelativeDate` object, the object represents a specific moment in time, so the current date is used automatically as one of the boundaries. When you use a single `query.RelativeDateRange` enum value, the enum value represents a range of dates, so the start and end properties of the date range are used automatically as the boundaries. For example:

```
1 var myComplexCondition = myQuery.createCondition({
2   fieldId: 'trandate',
3   operator: query.Operator.WITHIN,
4   values: query.RelativeDateRange.SAME_DAY_LAST_WEEK
5 });
```

In this example, the first boundary is the beginning of the same day last week, and the second boundary is the end of the same day last week. Using `query.RelativeDateRange.SAME_DAY_LAST_WEEK` is equivalent to using either of the following:

- `query.RelativeDateRange.SAME_DAY_LAST_WEEK.interval`
- `[query.RelativeDateRange.SAME_DAY_LAST_WEEK.start, query.RelativeDateRange.SAME_DAY_LAST_WEEK.end]`
- Use the start and end properties of values in the [query.RelativeDateRange](#) enum directly in the values parameter for [Query.createCondition\(options\)](#) or [Component.createCondition\(options\)](#). For example:

```
1 var myComplexCondition = myQuery.createCondition({
2   fieldId: 'trandate',
3   operator: query.Operator.WITHIN,
4   values: [query.RelativeDateRange.THIS_FISCAL_YEAR.start, query.RelativeDateRange.YESTERDAY.end]
5 });
```

- Use a combination of [query.RelativeDateRange](#) enum values and custom `query.RelativeDate` objects. For example:

```
1 var myEndDate = query.createRelativeDate({
2   dateId: query.DateId.WEEKS_AGO,
3   value: 2
4 });
5
6 var myComplexCondition = myQuery.createCondition({
7   fieldId: 'trandate',
8   operator: query.Operator.WITHIN,
9   values: [query.RelativeDateRange.THREE_FISCAL_YEARS_AGO.start, myEndDate]
10 });
```

SuiteQL in the N/query Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteQL is a query language based on the SQL-92 revision of the SQL database query language. It provides advanced query capabilities you can use to access your NetSuite records and data. For more information about SuiteQL, including limitations, exceptions, and more usage examples, see the help topic [SuiteQL](#).

In SuiteScript, you can create and run SuiteQL queries using the N/query module. Queries created using SuiteQL can be more powerful and flexible than queries created using other APIs in the N/query module.

SuiteQL queries can also provide the best query performance for many use cases. You can create your own SuiteQL query strings, which lets you create and run complex SQL queries that cannot be created otherwise. For examples of SuiteQL queries you can create, see [Examples of Using SuiteQL in the N/query Module](#).



Important: To create your own SuiteQL query strings, you must know the names of the record types and fields you want to use. For more information about finding record type and field names, see the help topic [Finding Record Type and Field Names](#).

Using the N/query module, you can run SuiteQL queries in the following ways:

- Convert an existing query (as a [query.Query](#) object) to its SuiteQL representation (as a [query.SuiteQL](#) object) and run the query. To learn more, see [Converting an Existing Query to SuiteQL](#).
- Run an arbitrary SuiteQL query as a paged or non-paged query. To learn more, see [Running an Arbitrary SuiteQL Query](#).

Converting an Existing Query to SuiteQL

If you already have a [query.Query](#) object in your script (one that you created using [query.create\(options\)](#) or loaded using [query.load\(options\)](#)), you can convert the query to its SuiteQL representation. The [Query.toSuiteQL\(\)](#) method converts a [query.Query](#) object to a [query.SuiteQL](#) object. The resulting [query.SuiteQL](#) object represents the same query as the original [query.Query](#) object and, when run, returns the same query results.



Important: The resulting SuiteQL query string (contained in the [SuiteQL.query](#) property) does not include any aliases you set on query result columns in the original [query.Query](#) object. For more information about aliases, see [Column.alias](#).

A [query.SuiteQL](#) object includes the following properties:

- [SuiteQL.columns](#) — The result columns to be returned from the query. This property is an array of [query.Column](#) objects. The value of this property is the same as the value of the [Query.columns](#) property in the original [query.Query](#) object.
- [SuiteQL.params](#) — The parameters for the query. In SuiteQL, query conditions are represented using the WHERE clause and a set of parameters.
- [SuiteQL.query](#) — The string representation of the SuiteQL query. This string can contain SQL clauses, record or table names, field names, operators, and more.
- [SuiteQL.type](#) — The type of the query. This property uses values from the [query.Type](#) enum. The value of this property is the same as the value of the [Query.type](#) property in the original [query.Query](#) object.

To run the SuiteQL query, use one of the following methods:

- Use [SuiteQL.run\(\)](#) to run the query as a non-paged query. This method returns the results as a [query.ResultSet](#) object.
- Use [SuiteQL.runPaged\(options\)](#) to run the query as a paged query. This method returns the results as a [query.PagedData](#) object. The default page size is 50 results per page. The minimum page size is 5 results per page, and the maximum page size is 1000 results per page.

The following example shows you how to create a query as a [query.Query](#) object, convert the query to SuiteQL, and run the resulting SuiteQL query as a non-paged query:


```

1 var myCustomerQuery = query.create({
2   type: query.Type.CUSTOMER
3 });
4
5 myCustomerQuery.columns = [
6   myCustomerQuery.createColumn({
7     fieldId: 'entityid'
8   }),
9   myCustomerQuery.createColumn({
10    fieldId: 'email'
11  })
12 ];
13
14 myCustomerQuery.condition = myCustomerQuery.createCondition({
15   fieldId: 'isperson',
16   operator: query.Operator.IS,
17   values: [true]
18 });
19
20 var mySQLCustomerQuery = myCustomerQuery.toSuiteQL();
21
22 var results = mySQLCustomerQuery.run();

```

In the preceding example, the `mySQLCustomerQuery` variable contains the resulting `query.SuiteQL` object after the conversion. In this object, the `SuiteQL.query` property contains the string representation of the original query. In this example, this property contains the following string:

```

1 SELECT customer.entityid AS entityidRAW /*{entityid#RAW}*/ , customer.email AS emailRAW /*{email#RAW}*/ FROM customer WHERE
   customer.isperson = ?

```

The `SuiteQL.params` property contains the parameter value `true`. When the SuiteQL query runs, the question mark (?) in the query string is replaced with the parameter value (`true`).

Running an Arbitrary SuiteQL Query

You can create your own SuiteQL queries and run them. The `query.runSuiteQL(options)` method lets you run an arbitrary SuiteQL query, and it returns query results as a `query.ResultSet` object. The `query.runSuiteQLPaged(options)` method lets you run a SuiteQL query as a paged query, and it returns query results as a `query.PagedData` object.

To specify the SuiteQL query to run, you can provide one of the following to `query.runSuiteQL(options)` or `query.runSuiteQLPaged(options)`:

- A string representation of the SuiteQL query

```

1 var results = query.runSuiteQL({
2   query: 'SELECT customer.entityid, customer.email FROM customer'
3 });

```

- A `query.SuiteQL` object

```

1 // In this example, mySuiteQLCustomerQuery is an existing SuiteQL object
2 var results = query.runSuiteQL(mySuiteQLCustomerQuery);

```

- A JavaScript Object that contains a `query` property and, optionally, a `params` property:

```

1 var results = query.runSuiteQL({
2   query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?',
3   params: [true]
4 });

```

Examples of Using SuiteQL in the N/query Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following examples demonstrate how to create and run SuiteQL queries using the features in the N/query module.

Convert an Existing Query to SuiteQL

This example RESTlet loads a SuiteAnalytics Workbook query with an ID of customworkbook237 and runs it. The RESTlet then converts the query to SuiteQL and runs it. The RESTlet returns both sets of query results.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType restlet
4   */
5  define(['N/query'], function(query) {
6      return {
7          get: function(context) {
8              // Load the workbook by name (record ID)
9              var openSalesOrders = query.load('customworkbook237');
10
11              // Run the query
12              var resultQuery = openSalesOrders.run();
13
14              // Convert the query to its SuiteQL representation
15              var openSalesOrdersQL = openSalesOrders.toSuiteQL();
16
17              // Examine the SuiteQL query string
18              var suiteQL = openSalesOrdersQL.query;
19
20              // Run the SuiteQL query
21              var resultSuiteQL = query.runSuiteQL(suiteQL);
22
23              // Compose the RESTlet response
24              var response = {
25                  query: openSalesOrders,
26                  resultQuery: resultQuery,
27                  suiteQL: suiteQL,
28                  resultSuiteQL: resultSuiteQL
29              };
30
31              // Return the response
32              return JSON.stringify(response);
33          }
34      }
35  });

```

Create a Custom SuiteQL Query

This example RESTlet constructs a custom query string using SuiteQL, runs the query, and returns the query results.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType restlet
4   */
5  define(['N/query'], function(query) {
6      return {
7          get: function(context) {
8              // Construct the SuiteQL query string
9              var suiteQL =
10 "SELECT "+

```

```

11 " \"TRANSACTION\".tranid AS tranidRAW /*{tranid#RAW}*/, "+
12 " \"TRANSACTION\".trandate AS trandateRAW /*{trandate#RAW}*/, "+
13 " \"TRANSACTION\".postingperiod AS postingperiodDISPLAY /*{postingperiod#DISPLAY}*/, "+
14 " BUILTIN.DF(\"TRANSACTION\".postingperiod) AS postingperiodDISPLAY /*{postingperiod#DISPLAY}*/, "+
15 " BUILTIN.DF(\"TRANSACTION\".status) AS statusDISPLAY /*{status#DISPLAY}*/, "+
16 " BUILTIN.DF(transactionLine.item) AS transactionlinesitemDISPLAY /*{transactionlines.item#DISPLAY}*/, "+
17 " BUILTIN.DF(\"TRANSACTION\".entity) AS entityDISPLAY /*{entity#DISPLAY}*/, "+
18 " transactionLine.quantity * -1 AS transactionlinesquantity /*- {transactionlines.quantity}*/, "+
19 " BUILTIN.CONSolidATE(transactionLine.netamount, 'LEDGER', 'DEFAULT', 'DEFAULT', 1, 396, 'DEFAULT') AS transactionlinesnetamountCU /*{transactionlines.netamount#CURRENCY_CONSolidATED}*/, "+
20 " BUILTIN.CURRENCY(BUILTIN.CONSolidATE(transactionLine.netamount, 'LEDGER', 'DEFAULT', 'DEFAULT', 1, 396, 'DEFAULT')) AS transactionlinesnetamountCU_C /*{transactionlines.netamount#CURRENCY_CONSolidATED}*/, "+
21 " CUSTOMRECORD41.custrecord14 AS custrecord15customrecord41c /*{custrecord15<customrecord41.custrecord14#RAW}*/ "+ "FROM "+
22 " \"TRANSACTION\", "+ " CUSTOMRECORD41, "+ " \"ACCOUNT\", "+ " TransactionAccountingLine, "+ " transactionLine "+ "WHERE "+
23 " ((((\"TRANSACTION\".\"ID\" = CUSTOMRECORD41.custrecord15 AND TransactionAccountingLine.\"ACCOUNT\" = \"ACCOUNT\".\"ID\" "+
24 " \"+)) AND (transactionLine.\"TRANSACTION\" = TransactionAccountingLine.\"TRANSACTION\" AND transactionLine.\"ID\" = TransactionAccountingLine.transactionline)) AND \"TRANSACTION\".\"ID\" = transactionLine.\"TRANSACTION\")) "+ " AND ((UPPER(\"TRANSACTION\".\"TYPE\") IN ('SALESORD') AND UPPER(\"TRANSACTION\".status) IN ('SALESORD:D', 'SALESORD:E', 'SALESORD:B') AND UPPER(\"ACCOUNT\".accttype) IN ('INCOME') AND (NOT( "+ " transactionLine.itemtype IN ('ShipItem') "+ " ) OR transactionLine.itemtype IS NULL) AND ((transactionLine.quantity * -1) - NVL(transactionLine.quantitycommitted, 0)) - NVL(transactionLine.quantityshiprecv, 0) > 0 AND NVL(transactionLine.mainline, 'F') = 'F' AND NVL(transactionLine.taxline, 'F') = 'F' AND NVL(transactionLine.isclosed, 'F') = 'F')) "+
25 "ORDER BY "+
26 " \"TRANSACTION\".trandate ASC NULLS LAST";
27
28 // Run the SuiteQL query
29 var resultSuiteQL = query.runSuiteQL(suiteQL);
30
31 // Compose the RESTlet response
32 var response = {
33     resultSuiteQL: resultSuiteQL
34 };
35
36 // Return the response
37 return JSON.stringify(response);

```

Accept a SuiteQL Query as an Argument

This example RESTlet accepts a SuiteQL query as a provided argument to the get endpoint, runs the query, and returns the query results.

```

1 /**
2  * @NApiVersion 2.x
3  * @NScriptType restlet
4  */
5 define(['N/query'], function(query) {
6     return {
7         get: function(context) {
8             return runQuery(query, context);
9         },
10        post: function(context) {
11            return runQuery(query, context);
12        }
13    }
14
15    function runQuery(query, context) {
16        // Get the SuiteQL query string from the arguments. For example, the
17        // SuiteQL query may look like the following:
18        //
19        // &suiteQL=select%20type%2C%20BUILTIN.DF(type)%2C%20tranid%2C%20trandate%20from%20transaction%20where%20type%3D'SalesOrd'
20        var id = context.id;
21        var suiteQL = context.suiteQL;
22
23        // Run the query
24        var suiteQLResults = query.runSuiteQL(suiteQL);
25
26        // Compose the RESTlet response

```

```

27     var response = {
28         id: id,
29         suiteQL: suiteQL,
30         resultSuiteQL: suiteQLResults.asMappedResults()
31     };
32
33     // Return the response
34     return JSON.stringify(response);
35 };
36 });

```

Run a Paged SuiteQL Query

This example script runs a SuiteQL query as a paged query with a page size of 10 results per page.

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/query'], function(query) {
5      // Construct the SuiteQL query string
6      var sql =
7          "SELECT " +
8          "    scriptDeployment.primarykey, scriptexecutioncontextmap.executioncontext " +
9          " FROM " +
10         "    scriptDeployment, scriptexecutioncontextmap " +
11         " WHERE " +
12         "    scriptexecutioncontextmap.scriptrecord = scriptDeployment.primarykey " +
13         " AND " +
14         "    scriptexecutioncontextmap.executioncontext IN ('WEBSTORE', 'WEBAPPLICATION')";
15
16     // Run the SuiteQL query as a paged query and return an iterator
17     var resultIterator = query.runSuiteQLPaged({
18         query: sql,
19         pageSize: 10
20     }).iterator();
21
22     // Use the iterator to process each page of results
23     resultIterator.each(function(page) {
24         var pageIterator = page.value.data.iterator();
25         pageIterator.each(function(row) {
26             log.debug('ID: ' + row.value.getValue(0) + ', Context: ' + row.value.getValue(1));
27             return true;
28         });
29
30         return true;
31     });
32 });

```

Use a SuiteQL Query in a Map/Reduce Script

This example map/reduce script uses a SuiteQL query as the source of input data. The value 271 is the internal ID of a transaction record.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType mapreducescript
4   */
5  define(['N/query'], function(query) {
6      // Define the getInputData() stage function
7      function getInputData() {
8          // Construct the SuiteQL query string
9          var suiteQL =
10              "SELECT " +
11              "    \"TRANSACTION\".tranid AS tranidRAW /*{tranid#RAW}*/ , " +
12              "    \"TRANSACTION\".trandate AS trandateRAW /*{trandate#RAW}*/ , " +
13              "    \"TRANSACTION\".postingperiod AS postingperiodDISPLAY /*{postingperiod#DISPLAY}*/ " +
14              "FROM " +

```

```

15     " \\"TRANSACTION\\" WHERE \\"TRANSACTION\\".\\"ID\\" = ? ";
16
17     // Return the query results as input data. The value 271 is the
18     // internal ID of a transaction record
19     return {
20         type: 'suiteql',
21         query: suiteQL,
22         params: [271]
23     };
24 }
25
26 // Define the map() stage function
27 function map(context) {
28     context.write(context.key, context.value);
29 }
30
31 // Define the reduce() stage function
32 function reduce(context) {
33     context.write(context.key, context.values[0]);
34 }
35
36 // Define the summarize() stage function
37 function summarize(summary) {
38     if (summary.inputSummary.error) {
39         log.debug('An error occurred. ');
40     }
41 }
42
43 // Return the function definitions
44 return {
45     getInputData: getInputData,
46     map: map,
47     reduce: reduce,
48     summarize: summarize
49 };
50 });

```

query.Column

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: You must include a [query.Column](#) object in your query definition to avoid scripting errors. This object is an optional parameter with the [query.create\(options\)](#) method, but at least one Column must be created for every query definition. For more information, see the syntax example below.

Object Description	A query result column. The <code>query.Column</code> object is the equivalent of the search.Column object in the N/search Module. The <code>query.Column</code> object describes the field types (columns) that are displayed from the query results. To create columns: - Use Query.createColumn(options) to create a column on the initial query definition created with query.create(options). - Use Component.createColumn(options) to create a column on a join relationship created with Query.autoJoin(options) or Component.autoJoin(options). - Assign all created columns as values to Query.columns. For an example, see Syntax.
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/query Module
Methods and Properties	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid'
12     }),
13     myCustomerQuery.createColumn({
14         fieldId: 'id'
15     }),
16     mySalesRepJoin.createColumn({
17         fieldId: 'entityid'
18     }),
19     mySalesRepJoin.createColumn({
20         fieldId: 'email'
21     }),
22     mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'
24     }),
25 ];
26 myCustomerQuery.sort = [
27     myCustomerQuery.createSort({
28         column: myCustomerQuery.columns[1]
29     }),
30     mySalesRepJoin.createSort({
31         column: mySalesRepJoin.columns[0], ascending: false
32     })
33 ];
34 var resultSet = myCustomerQuery.run();
35 ...
36 // Add additional code

```

Column.aggregate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An aggregate function that is performed on the query result column. An aggregate function performs a calculation on the column values and returns a single value. This property is set when Query.createColumn(options) or Component.createColumn(options) is executed. For a list of supported aggregate functions, see the query.Aggregate enum.
-----------------------------	---

Type	string (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myAggColumn = myTransactionQuery.createColumn({
7     fieldId: 'amount',
8     aggregate: query.Aggregate.AVERAGE
9 });
10 myTransactionQuery.columns = [myAggColumn];
11 var theAggregate = myAggColumn.aggregate;
12 ...
13 // Add additional code

```

Column.alias

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An alias for this column. An alias is an alternate name for a column, and the alias is used in mapped results. In general, the alias is an optional property. If you want to use mapped results in your script, the alias is required. To use mapped results, you must specify an alias in the following situations: - You must specify an alias for a column when the column uses a formula. - You must specify an alias when two columns in a joined query use the same field ID. For example, many record types include the entity field ID. If you join two record types that use the entity field ID, and you use the entity field ID to create result columns for both record types, you must specify an alias for one of the columns. This alias distinguishes the two columns that have the same field ID. This property is set when Query.createColumn(options) or Component.createColumn(options) is executed.
Type	string
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members

Since	2019.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myAliasColumn = myTransactionQuery.createColumn({
7     fieldId: 'amount',
8     aggregate: query.Aggregate.AVERAGE,
9     alias: 'sunkcostamount'
10 });
11 myTransactionQuery.columns = [myAliasColumn];
12 var theAlias = myAliasColumn.alias;
13 ...
14 // Add additional code

```

Column.component

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A reference to the query.Component object to which this query result column belongs. This property is set when Query.createColumn(options) or Component.createColumn(options) is executed.
Type	query.Component (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myAmountColumn = myTransactionQuery.createColumn({
7     fieldId: 'amount'
8 });
9 var theComponent = myAmountColumn.component;
10 ...
11 // Add additional code

```


Column.context

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The field context for values in the query result column. This property is set when Query.createColumn(options) or Component.createColumn(options) is executed. The field context determines how field values are displayed in a column. For example, you can specify that a column should display raw data (such as internal IDs), consolidated or converted amounts (such as currency totals), or user-friendly values (such as names). This property is an Object that includes the name of the context (which is a value in the query.FieldContext enum) and any parameters that apply to that context. In this release, only the <code>query.FieldContext.CONVERTED</code> context uses parameters. For information about these parameters, see Query.createColumn(options) or Component.createColumn(options).
Type	Object (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4   type: query.Type.TRANSACTION
5 });
6 var myTranLinesJoin = myTransactionQuery.autoJoin({
7   fieldId: 'transactionlines'
8 });
9 myTransactionQuery.condition = myTranLinesJoin.createCondition({
10  fieldId: 'netamount',
11  operator: query.Operator.GREATER,
12  values: 50000
13 });
14 myTransactionQuery.columns = [
15   myTranLinesJoin.createColumn({
16     fieldId: 'netamount'
17   })
18 ];
19 var unconsolidatedResultSet = myTransactionQuery.run();
20
21 // Log unconsolidated amounts
22 for (var i in unconsolidatedResultSet.results)
23   log.debug(unconsolidatedResultSet.results[i].values[0]);
24
25 myTransactionQuery.columns = [
26   myTranLinesJoin.createColumn({
27     fieldId: 'netamount',
28     context: query.FieldContext.CURRENCY_CONSOLIDATED
29   })
30 ];
31 var consolidatedResultSet = myTransactionQuery.run();
32

```

```

33 // Log consolidated amounts
34 for (var i in consolidatedResultSet.results)
35     log.debug(consolidatedResultSet.results[i].values[0]);
36 ...
37 // Add additional code

```

Column.fieldId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the query result column. This property is set during the of Query.createColumn(options) or Component.createColumn(options) . This property and the Column.formula property cannot be set at the same time.
Type	string (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myAmountColumn = myTransactionQuery.createColumn({
7     fieldId: 'amount'
8 });
9 var theFieldId = myAmountColumn.fieldId;
10 ...
11 // Add additional code

```

Column.formula

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A formula used to create the query result column. This property is set during the of Query.createColumn(options) or Component.createColumn(options) . This property and the Column.fieldId property cannot be set at the same time. For more information about formulas, see the help topics Formulas in Searches and SQL Expressions .
-----------------------------	---

Type	string (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myTransactionQuery = query.create({
4         type: query.Type.TRANSACTION
5     });
6     var myFormulaColumn = myTransactionQuery.createColumn({
7         type: query.ReturnType.CURRENCY,
8         formula: '{amount} * 125'
9     });
10    var theFormula = myFormulaColumn.formula;
11 ...
12 // Add additional code

```

Column.groupBy

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the query results are grouped by this query result column. This property is set during the of Query.createColumn(options) or Component.createColumn(options) .
Type	(read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER

```

```

5    });
6    var myGroupByColumn = myCustomerQuery.createColumn({
7        fieldId: 'currency',
8        groupBy: true
9    });
10   var theGroupBy = myGroupByColumn.groupBy;
11   ...
12   // Add additional code

```

Column.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for the column. A label is important if the query object is used as the data source for printing (for example, in the TemplateRenderer.addQuery(options) method in the N/render module).
Type	string (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1   // Add additional code
2   ...
3   var myCustomerQuery = query.create({
4       type: query.Type.CUSTOMER
5   });
6   var myLabelColumn = myCustomerQuery.createColumn({
7       fieldId: 'currency',
8       label: 'Dollars'
9   });
10  var theLabel = myLabelColumn.label;
11  ...
12  // Add additional code

```

Column.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The return type of the formula used to create the query result column. This property is set during the of Query.createColumn(options) or Component.createColumn(options) . If a formula is specified when these methods are called, this property contains the return type of the formula. If a formula is not specified, this property is null.
-----------------------------	---

	For more information about formulas, see the help topics Formulas in Searches and SQL Expressions .
Type	string (read-only)
Module	N/query Module
Parent Object	query.Column
Sibling Object Members	Column Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myFormulaColumn = myTransactionQuery.createColumn({
7     type: query.ReturnType.CURRENCY,
8     formula: '{amount} * 125'
9 });
10 var theFormulaType = myFormulaColumn.type;
11 ...
12 // Add additional code

```

query.Component

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	One component of the query definition. Each new component is created as a child to the previous component. All components exist as children to the query definition (query.Query). You can think of a component as a building block; each new component builds on the previous component created. The last component created encapsulates the relationship between it and all of its parent components. The query definition always contains at least one component. Queries with joins contain multiple components. The query definition (query.Query) contains a child query.Component object for each of the following: ■ The initial query definition: The initial query.Component object is called the root component. It encapsulates the initial query type passed to query.create(options). The root component is automatically created with the query.Query object and is a child of the query.Query object. The Query.root property contains a reference to the root component. ■ The first join: The second query.Component object is created with Query.autoJoin(options). It encapsulates the relationship between the initial query definition and the second query type. This relationship is determined by the join ID passed to Query.autoJoin(options). The second query.Component object is a child of the root component. ■ Each subsequent join: The third query.Component object is created with Component.autoJoin(options). All subsequent joins and their respective query.Component objects are also created with Component.autoJoin(options). Each of these query.Component
---------------------------	--

	objects encapsulates the relationship between all previous query types and the new query type. This relationship is determined by the join ID passed to Component.autoJoin(options) .
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/query Module
Methods and Properties	Component Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10   myCustomerQuery.createColumn({
11     fieldId: 'entityid'
12   }),
13   myCustomerQuery.createColumn({
14     fieldId: 'id'
15   }), mySalesRepJoin.createColumn({
16     fieldId: 'entityid'
17   }),
18   mySalesRepJoin.createColumn({
19     fieldId: 'email'
20   }),
21   mySalesRepJoin.createColumn({
22     fieldId: 'hiredate'
23   }),
24 ];
25 myCustomerQuery.sort = [
26   myCustomerQuery.createSort({
27     column: myCustomerQuery.columns[1]
28   }),
29   mySalesRepJoin.createSort({
30     column: mySalesRepJoin.columns[0],
31     ascending: false
32   })
33 ];
34 var resultSet = myCustomerQuery.run();
35 ...
36 // Add additional code

```

Component.autoJoin(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a join relationship.
---------------------------	------------------------------

	Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use Query.autoJoin(options) to create your first join (query.Component). Then use Component.autoJoin(options) to create each subsequent join (query.Component).  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The ID of the field that joins the parent component to the new component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...

```

```

3 | var myTransactionQuery = query.create({
4 |     type: query.Type.TRANSACTION
5 | });
6 | var myEntityJoin = myTransactionQuery.autoJoin({
7 |     fieldId: 'entity'
8 | });
9 | myTransactionQuery.columns = [
10 |     myEntityJoin.createColumn({
11 |         fieldId: 'subsidiary'
12 |     })
13 | ];
14 | myTransactionQuery.sort = [
15 |     myTransactionQuery.createSort({
16 |         column: myTransactionQuery.columns[0],
17 |         ascending: false
18 |     })
19 | ];
20 | var results = myTransactionQuery.runPaged({
21 |     pageSize: 10
22 | });
23 | ...
24 | // Add additional code

```

Component.createColumn(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a query result column based on the query.Component object. The query.Column object is the equivalent of the search.Column object in the N/search Module. The query.Column object describes the field types (columns) that are displayed from the query results. To create columns: ■ Use Component.createColumn(options) to create columns on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). Use this method in one of two ways: □ Pass in an argument for the parameter <code>options.fieldId</code>. □ Pass in an argument for the parameter <code>options.formula</code>. If you use this option, you can also use the optional parameter <code>options.type</code>. ■ If needed, use Query.createColumn(options) to create columns on the initial query definition created with query.create(options). ■ Assign all created columns as values to Query.columns. For an example, see Syntax. When you create a column, you can specify a field context. The field context determines how field values are displayed in the column. For example, you can specify that a column should display raw data (such as internal IDs), consolidated or converted amounts (such as currency totals), or user-friendly values (such as names). You can specify a field context in two ways: ■ Use a context from the query.FieldContext enum directly as the value of the <code>options.context</code> parameter. For example: <pre> 1 myTransactionLine.createColumn({ 2 fieldId: 'netamount', 3 context: query.FieldContext.CURRENCY_CONSOLIDATED 4 }); </pre> This example is the simplest way to specify a field context that does not accept additional parameters. Because the <code>options.context</code> parameter is an Object, this example is equivalent to the following: <pre> 1 myTransactionLine.createColumn({ </pre>
---------------------------	---

	<pre> 2 fieldId: 'netamount', 3 context: { 4 name: query.FieldContext.CURRENCY_CONSOLIDATED 5 } 6 }); </pre> ■ Use a context from the query.FieldContext enum as the value of the options.context.name parameter, and specify additional parameters using the options.context.params parameter. For example: <pre> 1 myTransactionLine.createColumn({ 2 fieldId: 'netamount', 3 context: { 4 name: query.FieldContext.CONVERTED, 5 params: { 6 currencyId: 4, 7 date: new Date('2019/01/01') 8 } 9 } 10 }); </pre> In this release, only the query.FieldContext.CONVERTED context uses additional parameters. The supported parameters are currencyId and date. For the date parameter, you can pass a JavaScript Date object or query.RelativeDate object.
Returns	query.Column
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required if options.formula is not used	The field ID of the query result column. This value sets the Column.fieldId property. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview .
options.formula	string	required if options.fieldId is not used	The formula used to create the query result column. This value sets the Column.formula property. For more information about formulas, see the help topics Formulas in Searches and SQL Expressions .
options.type	string	required if options.formula is used	If you use the options.formula parameter, use this parameter to explicitly define the formula's return type. This value sets the Column.type property.

Parameter	Type	Required / Optional	Description
			Use the appropriate query.ReturnType enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.aggregate	string	optional	Use this parameter to run an aggregate function on your query result column. An aggregate function performs a calculation on the column values and returns a single value. This value sets the Column.aggregate property. Use the appropriate query.Aggregate enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.groupBy	boolean	optional	Whether the query results are grouped by this query result column. This value sets the Column.groupBy property. If you do not pass in an argument, the default value is set to false.
options.label	string	optional	The label for the column. A label is important if the query object is used as the data source for printing (for example, in the TemplateRenderer.addQuery(options) method in the N/render module).
options.context	Object	optional	The field context for values in the query result column. This value sets the Column.context property.
options.context.name	string	required if options.context is used	The name of the field context. Use the appropriate query.FieldContext enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.context.params	Object	required if options.context.name has a value of query.FieldContext.CONVERTED	The additional parameters to use with the specified field context. In this release, only the query.FieldContext.CONVERTED context uses additional parameters. The supported parameters are currencyId and date.
options.context.params.currencyId	number	required if options.context.name has a value of query.FieldContext.CONVERTED	The internal ID of the currency to convert to. You can specify the internal ID of any currency that is configured in your NetSuite account. For more information, see the help topic Multiple Currencies.
options.context.params.date	query.RelativeDate JavaScript Date	required if options.context.name has a value of query.FieldContext.CONVERTED	The date to use for the exchange rate between the base currency and the currency to convert to. For example, if you want to use the exchange rate that was in effect on March 3, 2019, specify a query.RelativeDate object or JavaScript Date object that represents this date.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  var mySalesRepJoin = myCustomerQuery.join({
7      fieldId: 'salesrep'
8  });
9  myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid'
12     }),
13     myCustomerQuery.createColumn({
14         fieldId: 'id'
15     }),
16     mySalesRepJoin.createColumn({
17         fieldId: 'entityid'
18     }),
19     mySalesRepJoin.createColumn({
20         fieldId: 'email'
21     }),
22     mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'
24     }),
25 ];
26 myCustomerQuery.sort = [
27     myCustomerQuery.createSort({
28         column: myCustomerQuery.columns[1]
29     }),
30     mySalesRepJoin.createSort({
31         column: mySalesRepJoin.columns[0],
32         ascending: false
33     })
34 ];
35 var resultSet = myCustomerQuery.run();
36 ...
37 // Add additional code

```

Component.createCondition(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a condition (query filter) based on the query.Component object. A condition narrows the query results. The query.Condition object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that query.Condition objects can contain other query.Condition objects. To create conditions: - Use Component.createCondition(options) to create conditions on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). Use this method in one of two ways: - Pass in arguments for the parameters <code>options.fieldId</code>, <code>options.operator</code>, and <code>options.values</code>. The combination of these arguments translates to <code><filter column><operator><field value></code> (for example, 'city' equals 'Boston'). - Pass in an argument for the parameter <code>options.formula</code>. If you use this option, you can also use the optional parameter <code>options.type</code>. - If needed, use Query.createCondition(options) to create conditions on the initial query definition created with query.create(options). - If you have multiple conditions, use them to create a new nested condition with the methods Query.and(conditions), Query.or(conditions), and Query.not(condition). - Assign your simple or nested condition to Query.condition. For an example, see Syntax.
Returns	query.Condition

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required if options.operator and options.values are used	The field that the condition applies to. This value sets the Condition.fieldId property. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview .
options.operator	string	required	The operator used by the condition. This value sets the Condition.operator parameter. Use the appropriate query.Operator enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.values	string[] number[] [] Date[] query.RelativeDate [] query.Period []	required if options.fieldId and options.operator are used, and options.operator does not have a value of <code>query.Operator.EMPTY</code> or <code>query.Operator.EMPTY_NOT</code>	An array of values to use for the condition. This value sets the Condition.values property.
options.formula	string	required if options.fieldId is not used	The formula used to create the condition. This value sets the Condition.formula property. For more information about formulas, see the help topics Formulas in Searches and SQL Expressions .
options.type	string	required if options.formula is used	If you use the options.formula parameter, use this parameter to explicitly define the formula's return type. This value sets the Condition.type property.

Parameter	Type	Required / Optional	Description
			Use the appropriate query.ReturnType enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.aggregate	string	optional	An aggregate function to run on the condition. An aggregate function performs a calculation on the condition values and returns a single value. This value sets the Condition.aggregate property. Use the appropriate query.Aggregate enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.caseSensitive	boolean	optional	Use this parameter to specify whether filtered text is case sensitive or not. This parameter only applies to text fields. You may encounter script errors if this parameter is set with numbers or date values. You can use this parameter to improve the performance of the final query. By default, this value is set to false.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6
7 var mySalesRepJoin = myCustomerQuery.autoJoin({
8     fieldId: 'salesrep'
9 });
10
11 var myLocationJoin = mySalesRepJoin.autoJoin({
12     fieldId: 'location'
13 });
14
15
16 var myCondition = myCustomerQuery.createCondition({
17     fieldId: 'entityid',
18     operator: query.Operator.ANY_OF,
19     values: ['UK Cust USD', 'EU Cust'],
20     caseSensitive: true
21 });
22
23
24 myCustomerQuery.condition = myCustomerQuery.and(
25     myCondition
26 );
27
28
29 myCustomerQuery.columns = [

```

```

30     myCustomerQuery.createColumn({
31         fieldId: 'entityid'
32     }),
33     myCustomerQuery.createColumn({
34         fieldId: 'id'
35     }),
36     mySalesRepJoin.createColumn({
37         fieldId: 'entityid'
38     }),
39     mySalesRepJoin.createColumn({
40         fieldId: 'email'
41     }),
42     mySalesRepJoin.createColumn({
43         fieldId: 'hiredate'
44     }),
45     myLocationJoin.createColumn({
46         fieldId: 'name'
47     })
48 ];
49
50
51 myCustomerQuery.sort = [
52     myCustomerQuery.createSort({
53         column: myCustomerQuery.columns[3]
54     }),
55     myCustomerQuery.createSort({
56         column: myCustomerQuery.columns[0],
57         ascending: false
58     })
59 ];
60
61
62 var resultSet = myCustomerQuery.run()
63 ...
64 // Add additional code

```

Component.createSort(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a sort based on the query.Component object. The query.Sort object describes a sort that is placed on a particular query result column or condition. To create a sort: ■ Use Component.createSort(options) to create a sort based on a join relationship created with Query.autoJoin(options) or Component.autoJoin(options). ■ Use Query.createSort(options) to create a sort based on the initial query definition created with query.create(options). ■ Assign all created sorts as values to Query.sort. For an example, see Syntax.
Returns	query.Sort
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members

Since	2018.1
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.column	query.Column	required	The query result column that you want to sort by. This value sets the Sort.column property.
options.ascending	boolean	optional	Whether the sort direction is ascending. This value sets the Sort.ascending property. The default value of this property is true, meaning that the sort direction is ascending. If you want the sort direction to be descending, set this property to false.
options.caseSensitive	boolean	optional	Whether the sort is case sensitive. This value sets the Sort.caseSensitive property. If a sort is case sensitive (and the sort direction is ascending), rows with column values that start with uppercase letters are listed before rows with column values that start with lowercase letters. If a sort is not case sensitive, uppercase and lowercase letters are treated the same. For example, the following list of items is sorted using a case-sensitive sort with a sort direction of ascending: ■ Banana ■ Orange ■ apple ■ grapefruit ■ kiwi Here is the same list of items sorted using a regular (not case-sensitive) sort with a sort direction of ascending: ■ apple ■ Banana ■ grapefruit ■ kiwi ■ Orange The default value of this property is false.
options.locale	string	optional	The locale to use for the sort. This value sets the Sort.locale property. A locale represents a combination of language and region, and it can affect how certain values (such as strings) are sorted. For example, languages that share the same alphabet may sort characters differently. Use this property to ensure that query results are sorted using locale-specific rules.

Parameter	Type	Required / Optional	Description
			Use the appropriate query.SortLocale enum value to pass in your argument. This enum holds all the supported values for this parameter.
<code>options.nullsLast</code>	boolean	optional	Whether query results with null values are listed at the end of the query results. This value sets the Sort.nullsLast property. The default value of this property is the value of the <code>options.ascending</code> property. For example, if the <code>options.ascending</code> property is set to true, the <code>options.nullsLast</code> property is also set to true.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid'
12     }),
13     myCustomerQuery.createColumn({
14         fieldId: 'id'
15     }),
16     mySalesRepJoin.createColumn({
17         fieldId: 'entityid'
18     }),
19     mySalesRepJoin.createColumn({
20         fieldId: 'email'
21     }),
22     mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'
24     })
25 ];
26 myCustomerQuery.sort = [
27     myCustomerQuery.createSort({
28         column: myCustomerQuery.columns[1]
29     }),
30     mySalesRepJoin.createSort({
31         column: mySalesRepJoin.columns[0],
32         ascending: false
33     })
34 ];
35 var resultSet = myCustomerQuery.run();
36 ...
37 // Add additional code

```

Component.join(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a join relationship. This method is an alias to Component.autoJoin(options) .
---------------------------	---

	Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use Query.autojoin(options) to create your first join (query.Component). Then use <code>Component.join(options)</code> to create each subsequent join (query.Component).  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.fieldId</code>	string	required	The ID of the field that joins the parent component to the new component. This value determines the columns on which the components are joined and the type of the newly joined component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var myTransactionQuery = query.create({
4      type: query.Type.TRANSACTION
5  });
6  var myEntityJoin = myTransactionQuery.join({
7      fieldId: 'entity'
8  });
9  myTransactionQuery.columns = [
10     myEntityJoin.createColumn({
11         fieldId: 'subsidiary'
12     })
13 ];
14 myTransactionQuery.sort = [
15     myTransactionQuery.createSort({
16         column: myTransactionQuery.columns[0],
17         ascending: false
18     })
19 ];
20 var results = myTransactionQuery.runPaged({
21     pageSize: 10
22 });
23 ...
24 // Add additional code

```

Component.joinFrom(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an explicit directional join relationship from a child component to its parent component (an inverse join). This method sets the Component.source property on the returned query.Component object. Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use this method to create explicit directional joins from other components to this component.  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The column type (field type) that joins the parent component to the new component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview .
options.source	string	required	The query type of the component joined to this component. This value sets the Component.source property. This value can be described as the inverse relationship of this component, and it determines the source query type of the newly joined component.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myEmployeeQuery = query.create({
4   type: query.Type.EMPLOYEE
5 });
6 var mySalesOrderJoin = myEmployeeQuery.joinFrom({
7   fieldId: 'salesrep',
8   source: 'salesorder'
9 });
10 var items = mySalesOrderJoin.autoJoin({
11   fieldId: 'item'
12 });
13 myEmployeeQuery.columns = [
14   myEmployeeQuery.createColumn({
15     fieldId: 'entityid'
16   }),
17   myEmployeeQuery.createColumn({
18     fieldId: 'hiredate'
19   }),
20   mySalesOrderJoin.createColumn({
21     fieldId: 'id'
22   }),
23   mySalesOrderJoin.createColumn({
24     fieldId: 'trandate'
25   })
26 ];
27 var firstSort = myEmployeeQuery.createSort({
28   column: myEmployeeQuery.columns[0],
29   ascending: false

```

```

30     });
31     var secondSort = myEmployeeQuery.createSort({
32         column: myEmployeeQuery.columns[1],
33         ascending: true
34     });
35     myEmployeeQuery.sort = [firstSort, secondSort];
36     var results = myEmployeeQuery.run();
37     ...
38     // Add additional code

```

Component.joinTo(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an explicit directional join relationship from a parent component to its child component (a forward join). This method sets the Component.target property on the returned query.Component object. Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use this method to create explicit directional joins to other components from this component.  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The column type (field type) that joins the parent component to the new component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.

Parameter	Type	Required / Optional	Description
options.target	string	required	The query type of the component joined to this component. This value sets the Component.target property. This value can be described as the relationship of this component, and it determines the target query type of the newly joined component.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myEntityJoin = myTransactionQuery.joinTo({
7     fieldId: 'entity',
8     target: query.Type.CUSTOMER
9 });
10 myTransactionQuery.columns = [
11     myEntityJoin.createColumn({
12         fieldId: 'subsidiary'
13     })
14 ];
15 myTransactionQuery.sort = [
16     myTransactionQuery.createSort({
17         column: myTransactionQuery.columns[0],
18         ascending: false
19     })
20 ];
21 var results = myTransactionQuery.runPaged({
22     pageSize: 10
23 });
24 ...
25 // Add additional code

```

Component.child

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A reference to children of this component. The value of this property is an object of key-value pairs. Each key is the name of a child component. Each respective value refers to the corresponding query.Component object. The object values are set during the of Query.autoJoin(options) and Component.autoJoin(options). The order of the key-value pairs reflects the parent/child hierarchy.
-----------------------------	--

Type	Object (read-only)
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 var myDeptJoin = mySalesRepJoin.autoJoin({
10    fieldId: 'department'
11 });
12 var theChild = mySalesRepJoin.child;
13 ...
14 // Add additional code

```

Component.parent

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A reference to the parent query.Component object of this component. This property is set during the of Query.autoJoin(options) or Component.autoJoin(options) .
Type	string (read-only)
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  var mySalesRepJoin = myCustomerQuery.autoJoin({
7      fieldId: 'salesrep'
8  });
9  var myDeptJoin = mySalesRepJoin.autoJoin({
10     fieldId: 'department'
11 });
12 var theParent = myDeptJoin.parent;
13 ...
14 // Add additional code

```

Component.source

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The query type of the component joined to this component. This property can also be described as the inverse relationship of this component. This property is set during the of Query.joinFrom(options) and Component.joinFrom(options).
Type	string (read-only)
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var myEmployeeQuery = query.create({
4      type: query.Type.EMPLOYEE
5  });
6  var myTransactionJoin = myEmployeeQuery.joinFrom({
7      fieldId: 'entity',
8      source: 'transaction'
9  });
10 var theSource = myTransactionJoin.source;
11 ...
12 // Add additional code

```

Component.target

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The query type of this component. This property can also be described as the relationship of this component.
-----------------------------	---

	This property is set during the of Query.joinTo(options) and Component.joinTo(options) .
Type	string (read-only)
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myTransactionQuery = query.create({
4         type: query.Type.TRANSACTION
5     });
6     var myEmployeeJoin = myTransactionQuery.joinTo({
7         fieldId: 'createdby',
8         target: 'employee'
9     });
10    var theTarget = myEmployeeJoin.target;
11    ...
12 // Add additional code

```

Component.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The query type of this component. This property is set during the of Query.autoJoin(options) and Component.autoJoin(options) .
Type	string (read-only)
Module	N/query Module
Parent Object	query.Component
Sibling Object Members	Component Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code

```



```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  var theType = myCustomerQuery.type;
7  ...
8  // Add additional code

```

query.Condition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A condition that narrows the query results. The <code>query.Condition</code> object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that <code>query.Condition</code> objects can contain other <code>query.Condition</code> objects. To create conditions: - Use Query.createCondition(options) to create conditions for the initial query definition created with query.create(options). - Use Component.createCondition(options) to create conditions for the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). - If you have multiple conditions, use them to create a new nested condition with the methods Query.and(conditions), Query.or(conditions), and Query.not(condition). - Assign your simple or nested condition to Query.condition. For an example, see Syntax.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	Condition Object Members
Since	2018.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  var mySalesRepJoin = myCustomerQuery.autoJoin({
7      fieldId: 'salesrep'
8  });
9  var myLocationJoin = mySalesRepJoin.autoJoin({
10     fieldId: 'location'
11 });
12 var firstCondition = myCustomerQuery.createCondition({
13     fieldId: 'id',
14     operator: query.Operator.EQUAL,
15     values: 107

```

```

16     });
17     var secondCondition = myCustomerQuery.createCondition({
18         fieldId: 'id',
19         operator: query.Operator.EQUAL,
20         values: 2647
21     });
22     var thirdCondition = mySalesRepJoin.createCondition({
23         fieldId: 'email',
24         operator: query.Operator.START_WITH_NOT,
25         values: 'foo'
26     });
27     myCustomerQuery.condition = myCustomerQuery.and(thirdCondition, myCustomerQuery.not(myCustomerQuery.or(firstCondition, second
Condition)));
28     var resultSet = myCustomerQuery.run();
29     ...
30     // Add additional code

```

Condition.aggregate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An aggregate function that is performed on the condition. An aggregate function performs a calculation on the condition values and returns a single value. This property is set during the of Query.createCondition(options) or Component.createCondition(options).  Note: This property is not applicable to parent conditions created with the of Query.and(conditions), Query.or(conditions), or Query.not(condition).
Type	string (read-only)
Module	N/query Module
Parent Object	query.Condition
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     var myAggregateCondition = myCustomerQuery.createCondition({
7         fieldId: 'openingbalance',
8         operator: query.Operator.GREATER,
9         values: 10000,
10        aggregate: query.Aggregate.MAXIMUM
11    });
12    var theAggregate = myAggregateCondition.aggregate;
13    ...

```

14 // Add additional code

Condition.children

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of child conditions used to create the parent condition.  Note: This property is applicable to only parent conditions created with the of <code>Query.and(conditions)</code>, <code>Query.or(conditions)</code>, or <code>Query.not(condition)</code>.
Type	<code>query.Condition[]</code> (read-only)
Module	N/query Module
Parent Object	<code>query.Condition</code>
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var myFirstCondition = myCustomerQuery.createCondition({
7     fieldId: 'openingbalance',
8     operator: query.Operator.GREATER,
9     values: 10000
10 });
11 var mySecondCondition = myCustomerQuery.createCondition({
12     fieldId: 'email',
13     operator: query.Operator.START_WITH_NOT,
14     values: 'foo'
15 });
16 var myComplexCondition = myCustomerQuery.and(myFirstCondition, mySecondCondition);
17 var theChildren = myComplexCondition.children;
18 ...
19 // Add additional code

```

Condition.component

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The component used to created the condition This property is set during the of <code>Query.createCondition(options)</code> and <code>Component.createCondition(options)</code> .
-----------------------------	---

	 Note: This property is not applicable to parent conditions created with the of <code>Query.and(conditions)</code> , <code>Query.or(conditions)</code> , or <code>Query.not(condition)</code> .
Type	<code>query.Component</code> (read-only)
Module	N/query Module
Parent Object	query.Condition
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 var myCondition = mySalesRepJoin.createCondition({
10     fieldId: 'email',
11     operator: query.Operator.START_WITH,
12     values: 'mentor'
13 });
14 var theComponent = myCondition.component;
15 ...
16 // Add additional code

```

Condition.fieldId

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the condition. This property is set during the of <code>Query.createCondition(options)</code> and <code>Component.createCondition(options)</code>.  Note: This property is not applicable to parent conditions created with the of <code>Query.and(conditions)</code>, <code>Query.or(conditions)</code>, or <code>Query.not(condition)</code>.
Type	string (read-only)
Module	N/query Module
Parent Object	query.Condition
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var myCondition = myCustomerQuery.createCondition({
7     fieldId: 'openingbalance',
8     operator: query.Operator.GREATER,
9     values: 10000
10 });
11 var theFieldId = myCondition.fieldId;
12 ...
13 // Add additional code

```

Condition.formula

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The formula used to create the condition. This property is set during the of Query.createCondition(options) and Component.createCondition(options). For more information about formulas, see the help topics Formulas in Searches and SQL Expressions. Note: This property is not applicable to parent conditions created with the of Query.and(conditions), Query.or(conditions), or Query.not(condition).
Type	string (read-only)
Module	N/query Module
Parent Object	query.Condition
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myFormulaCondition = myTransactionQuery.createCondition({
7     formula: '{amount} * 125',

```

```

8      operator: query.Operator.GREATER,
9      values: 50000,
10     type: query.ReturnType.CURRENCY
11   });
12   var theFormula = myFormulaCondition.formula;
13   ...
14   // Add additional code

```

Condition.operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the operator used to create the condition. This property is set during the of Query.createCondition(options) and Component.createCondition(options).  Note: This property is not applicable to parent conditions created with the of Query.and(conditions), Query.or(conditions), or Query.not(condition).
Type	string (read-only)
Module	N/query Module
Parent Object	query.Condition
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var myCondition = myCustomerQuery.createCondition({
7   fieldId: 'openingbalance',
8   operator: query.Operator.GREATER,
9   values: 10000
10 });
11 var theOperator = myCondition.operator;
12 ...
13 // Add additional code

```

Condition.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The return type of the formula used to create the condition.
-----------------------------	--

	This property is set during the of Query.createCondition(options) or Component.createCondition(options). For more information formulas, see the help topics Formulas in Searches and SQL Expressions.  Note: This property is not applicable to parent conditions created with the of Query.and(conditions), Query.or(conditions), or Query.not(condition).
Type	string (read-only)
Module	N/query Module
Parent Object	query.Condition
Sibling Object Members	Condition Object Members
Since	2018.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myFormulaCondition = myTransactionQuery.createCondition({
7     formula: '{amount} * 125',
8     operator: query.Operator.GREATER,
9     values: 50000,
10    type: query.ReturnType.CURRENCY
11 });
12 var theFormulaType = myFormulaCondition.type;
13 ...
14 // Add additional code

```

Condition.values

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of values used by an operator to create the condition. This property is set by passing in values for options.fieldId, options.operator and options.values during the of Query.createCondition(options) or Component.createCondition(options).  Note: This property is not applicable to parent conditions created with the of Query.and(conditions), Query.or(conditions), or Query.not(condition).
Type	string[] number[] [] Date[] query.RelativeDate [] query.Period (read-only)
Module	N/query Module
Parent Object	query.Condition

Sibling Object Members	Condition Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var myCondition = myCustomerQuery.createCondition({
7     fieldId: 'firstname',
8     operator: query.Operator.ANY_OF,
9     values: ['Martin', 'Russell', 'Janina']
10 });
11 var theValues = myCondition.values;
12 ...
13 // Add additional code

```

query.Page

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	One page of the paged query results.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/query Module
Methods and Properties	Page Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     }),
10    myCustomerQuery.createColumn({
11        fieldId: 'firstname'
12    }),

```



```

13     myCustomerQuery.createColumn({
14         fieldId: 'email'
15     })
16 ];
17 var myPagedResults = myCustomerQuery.runPaged({
18     pageSize: 10
19 })
20
21 // Fetch results using an iterator
22 var iterator = myPagedResults.iterator();
23 iterator.each(function(resultPage) {
24     var currentPage = resultPage.value;
25     log.debug(currentPage.pageRange.size);
26     return true;
27 });
28
29 // Alternatively, fetch results using a loop
30 for (var i = 0; i < myPagedResults.pageRanges.length; i++) {
31     var currentPage = myPagedResults.fetch(i); log.debug(currentPage.pageRange.size);
32 }
33 ...
34 // Add additional code

```

Page.data

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The query results contained in this page.
Type	query.ResultSet (read-only)
Module	N/query Module
Parent Object	query.Page
Sibling Object Members	Page Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var theData = currentPage.data;
18     return true;
19 });
20 ...

```

21 // Add additional code

Page.isFirst

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the page is the first of the paged query results.
Type	boolean (read-only)
Module	N/query Module
Parent Object	query.Page
Sibling Object Members	Page Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var isFirst = currentPage.isFirst;
18     return true;
19 });
20 ...
21 // Add additional code

```

Page.isLast

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the page is the last of the paged query results.
Type	boolean (read-only)
Module	N/query Module
Parent Object	query.Page
Sibling Object Members	Page Object Members

Since	2018.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     }),
10    myCustomerQuery.createColumn({
11        fieldId: 'firstname'
12    }),
13    myCustomerQuery.createColumn({
14        fieldId: 'email'
15    })
16 ];
17 var myPagedResults = myCustomerQuery.runPaged({
18     pageSize: 10
19 });
20 var iterator = myPagedResults.iterator();
21 iterator.each(function(resultPage) {
22     var currentPage = resultPage.value;
23     var isLast = currentPage.isLast;
24     return true;
25 });
26 ...
27 // Add additional code

```

Page.pageRange

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The range of query results for this page.
Type	query.PageRange (read-only)
Module	N/query Module
Parent Object	query.Page
Sibling Object Members	Page Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  myCustomerQuery.columns = [
7      myCustomerQuery.createColumn({
8          fieldId: 'entityid'
9      })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var thePageRange = currentPage.pageRange;
18     return true;
19 });
20 ...
21 // Add additional code

```

Page.pagedData

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The set of paged query results that this page is from.
Type	query.PagedData (read-only)
Module	N/query Module
Parent Object	query.Page
Sibling Object Members	Page Object Members
Since	2018.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  myCustomerQuery.columns = [
7      myCustomerQuery.createColumn({
8          fieldId: 'entityid'
9      })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var thePagedData = currentPage.pagedData;
18     return true;
19 });
20 ...

```

21 | // Add additional code

query.PagedData

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A set of paged query results. This object also contains information about the set of paged results it encapsulates. Use Query.runPaged(options) or Query.runPaged.promise(options) to create this object. For paged queries, the maximum number of result rows per page is 1000. The minimum number of result rows per page is 5, except for the last page in the result set (because the last page may include fewer than 5 results).
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	PagedData Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     }),
10    myCustomerQuery.createColumn({
11        fieldId: 'firstname'
12    }),
13    myCustomerQuery.createColumn({
14        fieldId: 'email'
15    })
16 ];
17 var myPagedResults = myCustomerQuery.runPaged({
18     pageSize: 10
19 });
20
21 // Fetch results using an iterator
22 var iterator = myPagedResults.iterator();
23 iterator.each(function(resultPage) {
24     var currentPage = resultPage.value;
25     var currentPagedData = currentPage.pagedData;
26     log.debug(currentPage.pageRange.size);
27     return true;
28 });
29
30 // Alternatively, fetch results using a loop
31 for (var i = 0; i < myPagedResults.pageRanges.length; i++) {

```

```

32     var currentPage = myPagedResults.fetch(i);
33     var currentPagedData = currentPage.pagedData;
34     log.debug(currentPage.pageRange.size);
35 }
36 ...
37 // Add additional code

```

PagedData.fetch(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves a page in the set of pages included in the PagedData object.
Returns	query.Page
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.PagedData
Sibling Object Members	PagedData Object Members
Since	2018.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.index	number	required	The index of the page to retrieve. Page indexes start at 0.

Errors

Error Code	Thrown If
INVALID_PAGE_INDEX	The value of the options.index parameter is not a number.
INVALID_PAGE_RANGE	The value of the options.index parameter is a negative number or is greater than or equal to the number of pages in the PagedData object.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var myCustomerQuery = query.create({
4    type: query.Type.CUSTOMER
5  });
6  myCustomerQuery.columns = [
7    myCustomerQuery.createColumn({
8      fieldId: 'entityid'
9    }),
10   myCustomerQuery.createColumn({
11     fieldId: 'firstname'
12   }),
13   myCustomerQuery.createColumn({
14     fieldId: 'email'
15   })
16 ];
17 var myPagedResults = myCustomerQuery.runPaged({
18   pageSize: 10
19 });
20 for (var i = 0; i < myPagedResults.pageRanges.length; i++) {
21   var currentPage = myPagedResults.fetch(i);
22   var currentPagedData = currentPage.pagedData;
23   log.debug(currentPage.pageRange.size);
24 }
25 ...
26 // Add additional code

```

PagedData.fetch.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously retrieves a page in the set of pages included in the PagedData object.
Returns	Promise Object
Synchronous Version	PagedData.fetch(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.PagedData
Sibling Object Members	PagedData Object Members
Since	2018.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.index	number	required	The index of the page to retrieve. Page indexes start at 0.

Errors

Error Code	Thrown If
INVALID_PAGE_INDEX	The value of the options.index parameter is not a number.
INVALID_PAGE_RANGE	The value of the options.index parameter is a negative number or is greater than or equal to the number of pages in the PagedData object.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7   myCustomerQuery.createColumn({
8     fieldId: 'entityid'
9   }),
10  myCustomerQuery.createColumn({
11    fieldId: 'firstname'
12  }),
13  myCustomerQuery.createColumn({
14    fieldId: 'email'
15  })
16 ];
17 var myPagedResults = myCustomerQuery.runPaged({
18   pageSize: 10
19 });
20 for (var i = 0; i < myPagedResults.pageRanges.length; i++) {
21   var currentPage = myPagedResults.fetch.promise(i);
22   var currentPagedData = currentPage.pagedData;
23   log.debug(currentPage.pageRange.size);
24 }
25 ...
26 // Add additional code

```

PagedData.iterator()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Standard SuiteScript 2.x object for iterating through results.
Returns	Iterator object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Parent Object	query.PagedData
Sibling Object Members	PagedData Object Members

Since	2018.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var currentPagedData = currentPage.pagedData;
18     return true;
19 });
20 ...
21 // Add additional code

```

PagedData.count

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The total number of paged query result rows.
Type	number (read-only)
Module	N/query Module
Parent Object	query.PagedData
Sibling Object Members	PagedData Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [

```

```

7      myCustomerQuery.createColumn({
8          fieldId: 'entityid'
9      })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var currentPagedData = currentPage.pagedData;
18     var theCount = currentPagedData.count;
19     return true;
20 });
21 ...
22 // Add additional code

```

PagedData.pageRanges

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of query.PageRange objects that holds page index and page size information as key value pairs for query results.
Type	query.PageRange[]
Module	N/query Module
Parent Object	query.PagedData
Sibling Object Members	PagedData Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var currentPagedData = currentPage.pagedData;
18     var thePageRanges = currentPagedData.pageRanges;
19     return true;
20 });
21 ...
22 // Add additional code

```

PagedData.pageSize

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of query result rows per page. For paged queries, the maximum number of result rows per page is 1000. The minimum number of result rows per page is 5, except for the last page in the result set (because the last page may include fewer than 5 results).
Type	number (read-only)
Module	N/query Module
Parent Object	query.PagedData
Sibling Object Members	PagedData Object Members
Since	2018.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 var myPagedResults = myCustomerQuery.runPaged({
12     pageSize: 10
13 });
14 var iterator = myPagedResults.iterator();
15 iterator.each(function(resultPage) {
16     var currentPage = resultPage.value;
17     var currentPagedData = currentPage.pagedData;
18     var thePageSize = currentPagedData.pageSize;
19     return true;
20 });
21 ...
22 // Add additional code

```

query.PageRange

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The range of query results for a page.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/query Module

Methods and Properties	PageRange Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     }),
10    myCustomerQuery.createColumn({
11        fieldId: 'firstname'
12    }),
13    myCustomerQuery.createColumn({
14        fieldId: 'email'
15    })
16 ];
17 var myPagedResults = myCustomerQuery.runPaged({
18     pageSize: 10
19 });
20
21 // Fetch results using an iterator
22 var iterator = myPagedResults.iterator();
23 iterator.each(function(resultPage) {
24     var currentPage = resultPage.value;
25     var currentPageRange = currentPage.pageRange;
26     log.debug(currentPageRange.size);
27     return true;
28 });
29
30 // Alternatively, fetch results using a loop
31 for (var i = 0; i < myPagedResults.pageRanges.length; i++) {
32     var currentPage = myPagedResults.fetch(i);
33     var currentPageRange = currentPage.pageRange;
34     log.debug(currentPageRange.size);
35 }
36 ...
37 // Add additional code

```

PageRange.index

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The index for this page range.
Type	number (read-only)
Module	N/query Module
Parent Object	query.PageRange
Sibling Object Members	PageRange Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     myCustomerQuery.columns = [
7         myCustomerQuery.createColumn({
8             fieldId: 'entityid'
9         })
10    ];
11    var myPagedResults = myCustomerQuery.runPaged({
12        pageSize: 10
13    });
14    var iterator = myPagedResults.iterator();
15    iterator.each(function(resultPage) {
16        var currentPage = resultPage.value;
17        var currentPageRange = currentPage.pageRange;
18        var theIndex = currentPageRange.index; return true;
19    });
20 ...
21 // Add additional code

```

PageRange.size

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of query result rows in this page range.
Type	number (read-only)
Module	N/query Module
Parent Object	query.PageRange
Sibling Object Members	PageRange Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     myCustomerQuery.columns = [
7         myCustomerQuery.createColumn({
8             fieldId: 'entityid'
9         })
10    ];
11    var myPagedResults = myCustomerQuery.runPaged({
12        pageSize: 10
13    });

```

```

14     var iterator = myPagedResults.iterator();
15     iterator.each(function(resultPage) {
16         var currentPage = resultPage.value;
17         var currentPageRange = currentPage.pageRange;
18         var theSize = currentPageRange.size;
19         return true;
20     });
21     ...
22     // Add additional code

```

query.Period

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A period of time to use in query conditions. Use query.createPeriod(options) to create this object. After you create this object, you can use it in the values parameter of Query.createCondition(options) or Component.createCondition(options).
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	Period Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myPeriod = query.createPeriod({
4         code: query.PeriodCode.LAST_PERIOD,
5         adjustment: query.PeriodAdjustment.ALL,
6         type: query.PeriodType.END
7     });
8
9 // myQuery is an existing query.Query object
10    var myComplexCondition = myQuery.createCondition({
11        fieldId: 'trandate',
12        operator: query.Operator.BEFORE,
13        values: [myPeriod]
14    });
15    ...
16    // Add additional code

```

Period.adjustment

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The adjustment of the period.
-----------------------------	-------------------------------

	This property uses values from the query.PeriodAdjustment enum. If you create a period using query.createPeriod(options) and do not specify a value for the options.adjustment parameter, the default value of this property is <code>query.PeriodAdjustment.NOT_LAST</code> .
Type	string (read-only)
Module	N/query Module
Parent Object	query.Period
Sibling Object Members	Period Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myPeriod = query.createPeriod({
4     code: query.PeriodCode.LAST_PERIOD,
5     adjustment: query.PeriodAdjustment.ALL,
6     type: query.PeriodType.END
7 });
8 var theAdjustment = myPeriod.adjustment;
9 ...
10 // Add additional code

```

Period.code

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The code of the period. This property uses values from the query.PeriodCode enum.
Type	string (read-only)
Module	N/query Module
Parent Object	query.Period
Sibling Object Members	Period Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myPeriod = query.createPeriod({
4     code: query.PeriodCode.LAST_PERIOD,

```

```

5      adjustment: query.PeriodAdjustment.ALL,
6      type: query.PeriodType.END
7    });
8    var theCode = myPeriod.code;
9    ...
10   // Add additional code

```

Period.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of the period. This property uses values from the query.PeriodType enum. If you create a period using query.createPeriod(options) and do not specify a value for the options.type parameter, the default value of this property is <code>query.PeriodType.START</code>.
Type	string (read-only)
Module	N/query Module
Parent Object	query.Period
Sibling Object Members	Period Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1   // Add additional code
2   ...
3   var myPeriod = query.createPeriod({
4     code: query.PeriodCode.LAST_PERIOD,
5     adjustment: query.PeriodAdjustment.ALL,
6     type: query.PeriodType.END
7   });
8   var theType = myPeriod.type;
9   ...
10  // Add additional code

```

query.Query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The <code>query.Query</code> object encapsulates the query definition. To create a query with the N/query module: 1. Use the query.create(options) method to create your query definition (this object). The initial query definition uses one query type. For available query types, see query.Type. 2. After you create the initial query definition, use Query.autoJoin(options) to create your first join. 3. Then use either Query.autoJoin(options) or Component.autoJoin(options) to create subsequent joins.
---------------------------	---

	The query definition always contains at least one query.Component object. The query.Component object encapsulates one component of the query definition. Each new component is created as a child to the previous component, and all components exist as children to the query definition. You can think of a component as a building block; each new component builds on the previous component created. The last component created encapsulates the relationship between it and all of its parent components. Queries with joins contain multiple components. The query definition contains a child query.Component object for each of the following: ■ The initial query definition: The initial query.Component object is called the root component. It encapsulates the initial query type passed to query.create(options). The root component is automatically created with the initial query definition and is a child to the <code>query.Query</code> object. The Query.root property contains a reference to the root component. ■ The first join: The second query.Component object is created with Query.autoJoin(options). It encapsulates the relationship between the initial query definition and the second query type. This relationship is determined by the field ID passed to Query.autoJoin(options). The second query.Component object is a child to the root component. ■ Each subsequent join: The third query.Component object is created with Query.autoJoin(options) or Component.autoJoin(options). All subsequent joins are also created with Query.autoJoin(options) or Component.autoJoin(options). Each of these query.Component objects encapsulates the relationship between all previous query types and the new query type. This relationship is determined by the field ID passed to Component.autoJoin(options).
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myEntityJoin = myTransactionQuery.autoJoin({
7     fieldId: 'entity'
8 });
9 myTransactionQuery.columns = [
10     myEntityJoin.createColumn({
11         fieldId: 'subsidiary'
12     })
13 ];
14 myTransactionQuery.sort = [
15     myTransactionQuery.createSort({
16         column: myTransactionQuery.columns[0],
17         ascending: false
18     })
19 ];
20 var results = myTransactionQuery.runPaged({
21     pageSize: 10

```

```

22 | });
23 | ...
24 | // Add additional code

```

Query.and(conditions)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new condition (a query.Condition object) that corresponds to a logical conjunction (AND) of the arguments passed to the method. The arguments must be one or more query.Condition objects. A condition narrows the query results. The query.Condition object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that query.Condition objects can contain other query.Condition objects. To create conditions: - Use Query.createCondition(options) to create conditions for the initial query definition created with query.create(options). - Use Component.createCondition(options) to create conditions for the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). - If you have multiple conditions, use them to create a new parent condition with the methods Query.and(), Query.or(conditions), and Query.not(condition). - Assign your parent condition to Query.condition. For an example, see Syntax.
Returns	query.Condition
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

Parameter	Type	Required / Optional	Description
condition ₁ — n	query.Condition []	required	One or more condition objects. There is no limit on the number of conditions you can specify.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 | // Add additional code

```

```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  var mySalesRepJoin = myCustomerQuery.autoJoin({
7      fieldId: 'salesrep'
8  });
9  var myLocationJoin = mySalesRepJoin.autoJoin({
10     fieldId: 'location'
11 });
12 var firstCondition = myCustomerQuery.createCondition({
13     fieldId: 'id',
14     operator: query.Operator.EQUAL,
15     values: 107
16 });
17 var secondCondition = myCustomerQuery.createCondition({
18     fieldId: 'id',
19     operator: query.Operator.EQUAL,
20     values: 2647
21 });
22 var thirdCondition = mySalesRepJoin.createCondition({
23     fieldId: 'email',
24     operator: query.Operator.START_WITH_NOT,
25     values: 'foo'
26 });
27 myCustomerQuery.condition = myCustomerQuery.and(thirdCondition, myCustomerQuery.not(myCustomerQuery.or(firstCondition, second
Condition)));
28 var resultSet = myCustomerQuery.run();
29 ...
30 // Add additional code

```

Query.autoJoin(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a join relationship. Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use Query.autoJoin(options) to create your first join (query.Component). Then use Component.autoJoin(options) to create each subsequent join (query.Component). Note: This method is a shortcut for the chained Query.root and Component.autoJoin(options): <code>Query.root.join(options)</code>. The Query.root property references the root component, which is a query.Component object. Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Query

Sibling Object Members	Query Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The column type (field type) that joins the parent component to the new component. This value determines the columns on which the components are joined and the type of the newly joined component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myEntityJoin = myTransactionQuery.autoJoin({
7     fieldId: 'entity'
8 });
9 myTransactionQuery.columns = [
10     myEntityJoin.createColumn({
11         fieldId: 'subsidiary'
12     })
13 ];
14 myTransactionQuery.sort = [
15     myTransactionQuery.createSort({
16         column: myTransactionQuery.columns[0],
17         ascending: false
18     })
19 ];
20 var results = myTransactionQuery.runPaged({
21     pageSize: 10
22 });
23 ...
24 // Add additional code

```

Query.createColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a query result column based on the query.Query object. The query.Column object is the equivalent of the search.Column object in the N/search Module. The query.Column object describes the field types (columns) that are displayed from the query results. To create columns: ■ Use Query.createColumn(options) to create columns on the initial query definition created with query.create(options). Use this method in one of two ways: □ Pass in an argument for the parameter <code>options.fieldId</code>. □ Pass in an argument for the parameter <code>options.formula</code>. If you use this option, you can also use the optional parameter <code>options.type</code>. ■ If needed, use Component.createColumn(options) to create conditions on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). ■ Assign all created columns as values to Query.columns. For an example, see Syntax. When you create a column, you can specify a field context. The field context determines how field values are displayed in the column. For example, you can specify that a column should display raw data (such as internal IDs), consolidated or converted amounts (such as currency totals), or user-friendly values (such as names). You can specify a field context in two ways: ■ Use a context from the query.FieldContext enum directly as the value of the <code>options.context</code> parameter. For example: <pre> 1 myTransactionLine.createColumn({ 2 fieldId: 'netamount', 3 context: query.FieldContext.CURRENCY_CONSOLIDATED 4 }); </pre> This example is the simplest way to specify a field context that does not accept additional parameters. Because the <code>options.context</code> parameter is an Object, this example is equivalent to the following: <pre> 1 myTransactionLine.createColumn({ 2 fieldId: 'netamount', 3 context: { 4 name: query.FieldContext.CURRENCY_CONSOLIDATED 5 } 6 }); </pre> ■ Use a context from the query.FieldContext enum as the value of the <code>options.context.name</code> parameter, and specify additional parameters using the <code>options.context.params</code> parameter. For example: <pre> 1 myTransactionLine.createColumn({ 2 fieldId: 'netamount', 3 context: { 4 name: query.FieldContext.CONVERTED, 5 params: { 6 currencyId: 4, 7 date: new Date('2019/01/01') 8 } 9 } 10 }); </pre> In this example, the created column displays the value of the <code>netamount</code> currency field using the exchange rate that was in effect on January 1, 2019 for the currency with an ID of 4. In this release, only the <code>query.FieldContext.CONVERTED</code> context uses additional parameters. The supported parameters are <code>currencyId</code> and <code>date</code>. For the <code>date</code> parameter, you can pass a
---------------------------	--

	JavaScript Date object or query.RelativeDate object. If you pass a query.RelativeDate object using a value from the query.RelativeDateRange enum, use the start property or end property to specify the exact date of the exchange rate. For example, to use the exchange rate that was in effect at the beginning of the last fiscal quarter: <pre> 1 myTransactionLine.createColumn({ 2 fieldId: 'netamount', 3 context: { 4 name: query.FieldContext.CONVERTED, 5 params: { 6 currencyId: 4, 7 date: query.RelativeDateRange.LAST_FISCAL_QUARTER.start 8 } 9 } 10 }); </pre> If you use only the query.RelativeDate object from the query.RelativeDateRange enum and do not specify either the start or end properties, the end date of the relative date range is used. This behavior means that the following two date properties are equivalent: ■ <code>date: query.RelativeDateRange.LAST_FISCAL_QUARTER</code> ■ <code>date: query.RelativeDateRange.LAST_FISCAL_QUARTER.end</code>  Note: This method is a shortcut for the chained Query.root and Component.createColumn(options): <code>Query.root.createColumn(options)</code>. The Query.root property references the root component, which is a query.Component object.
Returns	query.Column
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.fieldId</code>	string	required if <code>options.formula</code> is not used	The field ID of the query result column. This value sets the Column.fieldId property. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview .
<code>options.formula</code>	string	required if <code>options.fieldId</code> is not used	The formula used to create the query result column. This value sets the Column.formula property.

Parameter	Type	Required / Optional	Description
			For more information about formulas, see the help topics Formulas in Searches and SQL Expressions .
options.type	string	required if options.formula is used	If you use the options.formula parameter, use this parameter to explicitly define the formula's return type. This value sets the Column.type property. Use the appropriate query.ReturnType enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.aggregate	string	optional	Use this parameter to run an aggregate function on your query result column. An aggregate function performs a calculation on the column values and returns a single value. This value sets the Column.aggregate property. Use the appropriate query.Aggregate enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.alias	string	optional	The alias for the column. An alias is an alternate name for a column, and the alias is used in mapped results. This value sets the Column.alias property. You must specify an alias in certain situations if you want to use ResultSet.asMappedResults() or Result.asMap(). For more information, see Column.alias.
options.groupBy	boolean	optional	Indicates whether the query results are grouped by this query result column. This value sets the Column.groupBy property. If you do not pass in an argument, the default value is set to false.
options.label	string	optional	The label for the column. A label is important if the query object is used as the data source for printing (for example, in the TemplateRenderer.addQuery(options) method in the N/render module).
options.context	Object	optional	The field context for values in the query result column. This value sets the Column.context property. If you do not pass in an argument, the default value is set to <code>query.FieldContext.RAW</code>.
options.context.name	string	required if options.context is used	The name of the field context. Use the appropriate query.FieldContext enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.context.params	Object	required if options.context.name has a value of <code>query.FieldContext.CONVERTED</code>	The additional parameters to use with the specified field context. In this release, only the <code>query.FieldContext.CONVERTED</code> context uses additional parameters. The supported parameters are <code>currencyId</code> and <code>date</code>.
options.context.params.currencyId	number	required if options.context.name has a value of <code>query.FieldContext.CONVERTED</code>	The ID of the currency to convert to.

Parameter	Type	Required / Optional	Description
options.context. params.date	query.RelativeDate JavaScript Date	required if options. context.name has a value of query.FieldContext. CONVERTED	The date to use for the exchange rate between the base currency and the currency to convert to. For example, if you want to use the exchange rate that was in effect on March 3, 2019, specify a query.RelativeDate object or JavaScript Date object that represents this date.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10  myCustomerQuery.createColumn({
11    fieldId: 'entityid'
12  }),
13  myCustomerQuery.createColumn({
14    fieldId: 'id'
15  }),
16  mySalesRepJoin.createColumn({
17    fieldId: 'entityid'
18  }),
19  mySalesRepJoin.createColumn({
20    fieldId: 'email'
21  }),
22  mySalesRepJoin.createColumn({
23    fieldId: 'hiredate'
24  })
25 ];
26 myCustomerQuery.sort = [
27  myCustomerQuery.createSort({
28    column: myCustomerQuery.columns[1]
29  }),
30  mySalesRepJoin.createSort({
31    column: mySalesRepJoin.columns[0],
32    ascending: false
33  })
34 ];
35 var resultSet = myCustomerQuery.run();
36 ...
37 // Add additional code

```

Query.createCondition(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a condition (query filter) based on the query.Query object. A condition narrows the query results. The query.Condition object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that query.Condition objects can contain other query.Condition objects. To create conditions:
---------------------------	--

	■ Use <code>Query.createCondition(options)</code> to create conditions on the initial query definition created with <code>query.create(options)</code>. Use this method in one of two ways: □ Pass in arguments for the parameters <code>options.fieldId</code>, <code>options.operator</code>, and <code>options.values</code>. The combination of these arguments translates to <code><filter column><operator><field value></code> (for example, 'city' equals 'Boston'). □ Pass in an argument for the parameter <code>options.formula</code>. If you use this option, you can also use the optional parameter <code>options.type</code>. ■ If needed, use <code>Component.createCondition(options)</code> to create conditions on the join relationships created with <code>Query.autoJoin(options)</code> and <code>Component.autoJoin(options)</code>. ■ If you have multiple conditions, use them to create a new nested condition with the methods <code>Query.and(conditions)</code>, <code>Query.or(conditions)</code>, and <code>Query.not(condition)</code>. ■ Assign your simple or nested condition to <code>Query.condition</code>. For an example, see Syntax. Note: This method is a shortcut for the chained <code>Query.root</code> and <code>Component.createCondition(options)</code>: <code>Query.root.createCondition(options)</code>. The <code>Query.root</code> property references the root component, which is a <code>query.Component</code> object.
Returns	<code>query.Condition</code>
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	<code>query.Query</code>
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.fieldId</code>	string	required if <code>options.operator</code> and <code>options.values</code> are used	The field that the condition applies to. This value sets the <code>Condition.fieldId</code> property. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.
<code>options.operator</code>	string	required	The operator used by the condition. This value sets the <code>Condition.operator</code> parameter. Use the appropriate <code>query.Operator</code> enum value

Parameter	Type	Required / Optional	Description
			to pass in your argument. This enum holds all the supported values for this parameter.
options.values	string number Date string[] number[] [] Date[] query.RelativeDate[] query.Period[]	required if options.fieldId and options.operator are used, and options.operator does not have a value of query.Operator.EMPTY or query.Operator.EMPTY_NOT	Value or an array of values to use for the condition. This value sets the Condition.values property.
options.formula	string	required if options.fieldId is not used	The formula used to create the condition. This value sets the Condition.formula property. For more information about formulas, see the help topics Formulas in Searches and SQL Expressions .
options.type	string	required if options.formula is used	If you use the options.formula parameter, use this parameter to explicitly define the formula's return type. This value sets the Condition.type property. Use the appropriate query.ReturnType enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.aggregate	string	optional	Use this parameter to run an aggregate function on a condition. An aggregate function performs a calculation on the condition values and returns a single value. This value sets the Condition.aggregate property. Use the appropriate query.Aggregate enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.caseSensitive	boolean	optional	Use this parameter to specify the case of filtered text. This parameter is only valid for text fields. You may encounter script errors if this parameter is set with numbers or date values. You can use this parameter to improve the performance of the final query. By default, this value is set to false.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6
7      var mySalesRepJoin = myCustomerQuery.autoJoin({
8          fieldId: 'salesrep'
9      });
10
11      var myLocationJoin = mySalesRepJoin.autoJoin({
12          fieldId: 'location'
13      });
14
15
16  var myCondition = myCustomerQuery.createCondition({
17      fieldId: 'entityid',
18      operator: query.Operator.ANY_OF,
19      values: ['UK Cust USD', 'EU Cust'],
20      caseSensitive: true
21  });
22
23
24      myCustomerQuery.condition = myCustomerQuery.and(
25          myCondition
26      );
27
28
29  myCustomerQuery.columns = [
30      myCustomerQuery.createColumn({
31          fieldId: 'entityid'
32      }),
33      myCustomerQuery.createColumn({
34          fieldId: 'id'
35      }),
36      mySalesRepJoin.createColumn({
37          fieldId: 'entityid'
38      }),
39      mySalesRepJoin.createColumn({
40          fieldId: 'email'
41      }),
42      mySalesRepJoin.createColumn({
43          fieldId: 'hiredate'
44      }),
45      myLocationJoin.createColumn({
46          fieldId: 'name'
47      })
48  ];
49
50
51  myCustomerQuery.sort = [
52      myCustomerQuery.createSort({
53          column: myCustomerQuery.columns[3]
54      }),
55      myCustomerQuery.createSort({
56          column: myCustomerQuery.columns[0],
57          ascending: false
58      })
59  ];
60
61
62  var resultSet = myCustomerQuery.run()
63  ...
64  // Add additional code

```

Query.createSort(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a sort based on the query.Query object. The query.Sort object describes a sort that is placed on a particular query result column.
---------------------------	--

	To create a sort: ■ Use <code>Query.createSort(options)</code> to create a sort based on the initial query definition created with <code>query.create(options)</code>. ■ Use <code>Component.createSort(options)</code> to create a sort based on a join relationship created with <code>Query.autoJoin(options)</code> or <code>Component.autoJoin(options)</code>. ■ Assign all created sorts as values to <code>Query.sort</code>. For an example, see Syntax.  Note: This method is a shortcut for the chained <code>Query.root</code> and <code>Component.createSort(options)</code>: <code>Query.root.createSort(options)</code>. The <code>Query.root</code> property references the root component, which is a <code>query.Component</code> object.
Returns	<code>query.Sort</code>
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	<code>query.Query</code>
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.column</code>	query.Column	required	The query result column that you want to sort by. This value sets the Sort.column property.
<code>options.ascending</code>	boolean	optional	Whether the sort direction is ascending. This value sets the Sort.ascending property. The default value of this property is true, meaning that the sort direction is ascending. If you want the sort direction to be descending, set this property to false.
<code>options.caseSensitive</code>	boolean	optional	Whether the sort is case sensitive. This value sets the Sort.caseSensitive property. If a sort is case sensitive (and the sort direction is ascending), rows with column values that start with uppercase letters are listed before rows with column values that start with lowercase letters. If a sort is not case sensitive, uppercase and lowercase letters are treated the same. For example, the following list of items is sorted using a case-sensitive sort with a sort direction of ascending: ■ Banana

Parameter	Type	Required / Optional	Description
			- Orange - apple - grapefruit - kiwi Here is the same list of items sorted using a regular (not case-sensitive) sort with a sort direction of ascending: - apple - Banana - grapefruit - kiwi - Orange The default value of this property is false.
options.locale	string	optional	The locale to use for the sort. This value sets the Sort.locale property. A locale represents a combination of language and region, and it can affect how certain values (such as strings) are sorted. For example, languages that share the same alphabet may sort characters differently. Use this property to ensure that query results are sorted using locale-specific rules. Use the appropriate query.SortLocale enum value to pass in your argument. This enum holds all the supported values for this parameter.
options.nullsLast	boolean	optional	Whether query results with null values are listed at the end of the query results. This value sets the Sort.nullsLast property. The default value of this property is the value of the options.ascending property. For example, if the options.ascending property is set to true, the options.nullsLast property is also set to true.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10  myCustomerQuery.createColumn({
11    fieldId: 'entityid'
12  }),
13  myCustomerQuery.createColumn({
14    fieldId: 'id'

```

```

15     }},
16     mySalesRepJoin.createColumn({
17         fieldId: 'entityid'
18     }),
19     mySalesRepJoin.createColumn({
20         fieldId: 'email'
21     }),
22     mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'
24     })
25 ];
26 myCustomerQuery.sort = [
27     myCustomerQuery.createSort({
28         column: myCustomerQuery.columns[1]
29     }),
30     mySalesRepJoin.createSort({
31         column: mySalesRepJoin.columns[0],
32         ascending: false
33     })
34 ];
35 var resultSet = myCustomerQuery.run();
36 ...
37 // Add additional code

```

Query.join(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a join relationship.  Important: This method is an alias to Query.autoJoin(options). Use Query.autoJoin(options) instead of this method to create simple joins. Use Query.joinFrom(options) and Query.joinTo(options) to create explicit directional joins. Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use Query.join(options) to create your first join (query.Component). Then use Component.autoJoin(options) to create each subsequent join (query.Component).  Note: This method is a shortcut for the chained Query.root and Component.join(options): Query.root.join(options). The Query.root property references the root component, which is a query.Component object.  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Query

Sibling Object Members	Query Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The column type (field type) that joins the parent component to the new component. This value determines the columns on which the components are joined and the type of the newly joined component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myTransactionQuery = query.create({
4         type: query.Type.TRANSACTION
5     });
6     var myEntityJoin = myTransactionQuery.join({
7         fieldId: 'entity'
8     });
9     myTransactionQuery.columns = [
10         myEntityJoin.createColumn({
11             fieldId: 'subsidiary'
12         })
13     ];
14     myTransactionQuery.sort = [
15         myTransactionQuery.createSort({
16             column: myTransactionQuery.columns[0],
17             ascending: false
18         })
19     ];
20     var results = myTransactionQuery.runPaged({
21         pageSize: 10
22     });
23 ...
24 // Add additional code

```

Query.joinFrom(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an explicit directional join relationship from a child component to its parent component (an inverse join). This method sets the Component.source property on the returned query.Component object.
---------------------------	--

	Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use this method to create your first join as an explicit directional join from another component to this component. Note: This method is a shortcut for the chained Query.root and Component.joinFrom(options): <code>Query.root.joinFrom(options)</code>. The Query.root property references the root component, which is a query.Component object. Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.2

Parameters

Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
<code>options.fieldId</code>	string	required	The column type (field type) that joins the parent component to the new component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.
<code>options.source</code>	string	required	The query type of the component joined to this component. This value sets the Component.source property. This value can be described as the inverse relationship of this component, and it determines the source query type of the newly joined component.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myEmployeeQuery = query.create({
4   type: query.Type.EMPLOYEE
5 });
6 var myTransactionJoin = myEmployeeQuery.joinFrom({
7   fieldId: 'entity',
8   source: 'transaction'
9 });
10 myEmployeeQuery.columns = [
11   myEmployeeQuery.createColumn({
12     fieldId: 'entityid'
13   }),
14   myTransactionJoin.createColumn({
15     fieldId: 'entity'
16   }),
17   myTransactionJoin.createColumn({
18     fieldId: 'daysoverduesearch'
19   })
20 ];
21 ...
22 // Add additional code

```

Query.joinTo(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an explicit directional join relationship from a parent component to its child component (a forward join). This method sets the Component.target property on the returned query.Component object. Use the method query.create(options) to create your initial query definition (query.Query). The initial query definition uses one query type. For available query types, see query.Type. After you create the initial query definition, use this method to create your first join as an explicit directional join to another component from this component. Note: This method is a shortcut for the chained Query.root and Component.joinTo(options): <code>Query.root.autoJoin(options)</code>. The Query.root property references the root component, which is a query.Component object. Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Returns	query.Component
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module

Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fieldId	string	required	The column type (field type) that joins the parent component to the new component. Obtain this value from the Records Catalog. The Records Catalog lists every record type and field that is available using SuiteAnalytics Workbook and the N/query module. For more information, see the help topic Records Catalog Overview.
options.target	string	required	The query type of the component joined to this component. This value sets the Component.target property. This value can be described as the relationship of this component, and it determines the target query type of the newly joined component.

Errors

Error Code	Thrown If
RELATIONSHIP_ALREADY_USED	The specified join relationship already exists.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4   type: query.Type.TRANSACTION
5 });
6 var myEmployeeJoin = myTransactionQuery.joinTo({
7   fieldId: 'createdby',
8   target: 'employee'
9 });
10 myTransactionQuery.columns = [
11   myTransactionQuery.createColumn({
12     fieldId: 'entity'
13   }),
14   myEmployeeJoin.createColumn({
15     fieldId: 'entityid'
16   }),
17   myEmployeeJoin.createColumn({
18     fieldId: 'email'
19   })

```

```

20 | ];
21 | ...
22 | // Add additional code

```

Query.not(condition)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new condition (a query.Condition object) that corresponds to a logical negation (NOT) of the argument passed to the method. The argument must be a query.Condition object. A condition narrows the query results. The query.Condition object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that query.Condition objects can contain other query.Condition objects. To create conditions: - Use Query.createCondition(options) to create conditions for the initial query definition created with query.create(options). - Use Component.createCondition(options) to create conditions for the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). - If you have multiple conditions, use them to create a new parent condition with the methods Query.and(conditions), Query.or(conditions), and Query.not(). - Assign your parent condition to Query.condition. For an example, see Syntax.
Returns	query.Condition
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

Parameter	Type	Required / Optional	Description
condition	query.Condition	required	One condition object.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | var myCustomerQuery = query.create({
4 |     type: query.Type.CUSTOMER

```

```

5   });
6   var mySalesRepJoin = myCustomerQuery.autoJoin({
7       fieldId: 'salesrep'
8   });
9   var myLocationJoin = mySalesRepJoin.autoJoin({
10      fieldId: 'location'
11  });
12  var firstCondition = myCustomerQuery.createCondition({
13      fieldId: 'id',
14      operator: query.Operator.EQUAL,
15      values: 107
16  });
17  var secondCondition = myCustomerQuery.createCondition({
18      fieldId: 'id',
19      operator: query.Operator.EQUAL,
20      values: 2647
21  });
22  var thirdCondition = mySalesRepJoin.createCondition({
23      fieldId: 'email',
24      operator: query.Operator.START_WITH_NOT,
25      values: 'foo'
26  });
27  myCustomerQuery.condition = myCustomerQuery.and(thirdCondition, myCustomerQuery.not( myCustomerQuery.or(firstCondition, second
28  Condition)));
29  var resultSet = myCustomerQuery.run();
30  ...
31  // Add additional code

```

Query.or(conditions)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new condition (a query.Condition object) that corresponds to a logical disjunction (OR) of the arguments passed to the method. The arguments must be one or more query.Condition objects. A condition narrows the query results. The query.Condition object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that query.Condition objects can contain other query.Condition objects. To create conditions: - Use Query.createCondition(options) to create conditions for the initial query definition created with query.create(options). - Use Component.createCondition(options) to create conditions for the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). - If you have multiple conditions, use them to create a new parent condition with the methods Query.and(conditions), Query.or(), and Query.not(condition). - Assign your parent condition to Query.condition. For an example, see Syntax.
Returns	query.Condition
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members

Since	2018.1
-------	--------

Parameters

Parameter	Type	Required / Optional	Description
condition ₁ — n	query.Condition[]	required	One or more condition objects. There is no limit on the number of conditions you can specify.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 var myLocationJoin = mySalesRepJoin.autoJoin({
10  fieldId: 'location'
11 });
12 var firstCondition = myCustomerQuery.createCondition({
13   fieldId: 'id',
14   operator: query.Operator.EQUAL,
15   values: 107
16 });
17 var secondCondition = myCustomerQuery.createCondition({
18   fieldId: 'id',
19   operator: query.Operator.EQUAL,
20   values: 2647
21 });
22 var thirdCondition = mySalesRepJoin.createCondition({
23   fieldId: 'email',
24   operator: query.Operator.START_WITH_NOT,
25   values: 'foo'
26 });
27 myCustomerQuery.condition = myCustomerQuery.and(thirdCondition, myCustomerQuery.not(myCustomerQuery.or(firstCondition, second
Condition)));
28 var resultSet = myCustomerQuery.run();
29 ...
30 // Add additional code

```

Query.run(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the query and returns the query result set. This method returns a maximum of 5000 results in the query result set. If a query matches more than 5000 results, you must use Query.runPaged(options) or Query.runPaged.promise(options) to retrieve the full set of results.
Returns	query.ResultSet

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary.  Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.
options.metaDataProvider	string	optional	Indicates whether the query should fail if you lack the necessary permissions for some fields or records. If set to SUITE_QL: The query fails if you attempt to access fields or records without the necessary permissions. If set to STATIC: The query succeeds even if you lack permissions for some fields or records, but it does not return any data for those fields or records. Defaults to SUITE_QL.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
2 ...
```

```

3   var myCustomerQuery = query.create({
4       type: query.Type.CUSTOMER
5   });
6   var mySalesRepJoin = myCustomerQuery.autoJoin({
7       fieldId: 'salesrep'
8   });
9   myCustomerQuery.columns = [
10      myCustomerQuery.createColumn({
11          fieldId: 'entityid'
12      }),
13      myCustomerQuery.createColumn({
14          fieldId: 'id'
15      }),
16      mySalesRepJoin.createColumn({
17          fieldId: 'entityid'
18      }),
19      mySalesRepJoin.createColumn({
20          fieldId: 'email'
21      }),
22      mySalesRepJoin.createColumn({
23          fieldId: 'hiredate'
24      })
25  ];
26  myCustomerQuery.sort = [
27      myCustomerQuery.createSort({
28          column: myCustomerQuery.columns[1]
29      }),
30      mySalesRepJoin.createSort({
31          column: mySalesRepJoin.columns[0],
32          ascending: false
33      })
34  ];
35  var resultSet = myCustomerQuery.run({
36      customScriptId: 'myCustomScriptId'
37  });
38  ...
39  // Add additional code

```

Query.run.promise()

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Executes the query asynchronously and returns the query result set.  Note: The parameters and errors thrown for this method are the same as those for Query.run(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Query.run(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10  myCustomerQuery.createColumn({
11    fieldId: 'entityid'
12  }),
13  myCustomerQuery.createColumn({
14    fieldId: 'id'
15  }),
16  mySalesRepJoin.createColumn({
17    fieldId: 'entityid'
18  }),
19  mySalesRepJoin.createColumn({
20    fieldId: 'email'
21  }),
22  mySalesRepJoin.createColumn({
23    fieldId: 'hiredate'
24  })
25 ];
26 myCustomerQuery.sort = [
27  myCustomerQuery.createSort({
28    column: myCustomerQuery.columns[1]
29  }),
30  mySalesRepJoin.createSort({
31    column: mySalesRepJoin.columns[0],
32    ascending: false
33  })
34 ];
35 var resultSet = myCustomerQuery.run.promise({
36   customScriptId: 'myCustomScriptId'
37 }).then (function(result) {
38   // Process the result
39 });
40 ...
41 // Add additional code

```

Query.runPaged(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: You must specify a sorting order in the query definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Executes the query and returns a query.PagedData object that represents the paged query results. You can iterate over this object to obtain each page of query results. For paged queries, the maximum number of result rows per page is 1000. The minimum number of result rows per page is 5, except for the last page in the result set (because the last page may include fewer than 5 results).
Returns	query.PagedData

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.pageSize	string	optional	The size of each page in the query results. The default page size is 50 results per page.
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary.  Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.
options.metaDataProvider	string	optional	Indicates whether the query should fail if you lack the necessary permissions for some fields or records. If set to SUITE_QL: The query fails if you attempt to access fields or records without the necessary permissions. If set to STATIC: The query succeeds even if you lack permissions for some fields or records, but it does not return any data for those fields or records. Defaults to SUITE_QL.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var myTransactionQuery = query.create({
4      type: query.Type.TRANSACTION
5  });
6  var myEntityJoin = myTransactionQuery.autoJoin({
7      fieldId: 'entity'
8  });
9  myTransactionQuery.columns = [
10     myEntityJoin.createColumn({
11         name: 'subsidiary'
12     })
13 ];
14 myTransactionQuery.sort = [
15     myTransactionQuery.createSort({
16         column: myTransactionQuery.columns[0],
17         ascending: false
18     })
19 ];
20 var results = myTransactionQuery.runPaged({
21     pageSize: 10,
22     customScriptId: 'myCustomScriptId'
23 });
24
25 // Use the count property to count the search results easily
26 var resultCount = myTransactionQuery.runPaged({
27     pageSize: 10
28 }).count;
29 ...
30 // Add additional code

```

Query.runPaged.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: You must specify a sorting order in the query definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Executes the query asynchronously and returns a set of paged results.  Note: The parameters and errors thrown for this method are the same as those for Query.runPaged(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Query.runPaged(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myEntityJoin = myTransactionQuery.autoJoin({
7     fieldId: 'entity'
8 });
9 myTransactionQuery.columns = [
10     myEntityJoin.createColumn({
11         name: 'subsidiary'
12     })
13 ];
14 myTransactionQuery.sort = [
15     myTransactionQuery.createSort({
16         column: myTransactionQuery.columns[0],
17         ascending: false
18     })
19 ];
20 var results = myTransactionQuery.runPaged({
21     pageSize: 10,
22     customScriptId: 'myCustomScriptId'
23 });
24
25 // Use the count property to count the search results easily
26 var resultCount = myTransactionQuery.runPaged.promise({
27     pageSize: 10
28 }).count;
29 ...
30 // Add additional code

```

Query.toSuiteQL()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Converts this query.Query object to its corresponding SuiteQL representation. This method returns a query.SuiteQL object that represents the same query as the original query.Query object. This object includes the SuiteQL.columns, SuiteQL.params, SuiteQL.query, and SuiteQL.type properties. You can run the query using SuiteQL.run(), or you can run the query as a paged query using SuiteQL.runPaged(options). Important: The resulting SuiteQL query string (contained in the SuiteQL.query property) does not include any aliases you set on query result columns in the original query.Query object. For more information about aliases, see Column.alias. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	query.SuiteQL
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var results = mySuiteQLQuery.run();
6 ...
7 // Add additional code

```

Query.child

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A reference to children of this component. The value of this property is an object of key-value pairs. Each key is the name of a child component. Each respective value is the corresponding query.Component object. The object values are set with the of Query.autoJoin(options) and Component.autoJoin(options). The order of the key-value pairs reflects the parent/child hierarchy.
Type	Object
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });

```

```

9      var myTaskJoin = myCustomerQuery.autoJoin({
10         fieldId: 'task'
11     });
12     var theChild = myCustomerQuery.child;
13     ...
14     // Add additional code

```

Query.columns

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of result columns (query.Column objects) returned from the query. The query.Column object is the equivalent of the search.Column object in the N/search Module. The query.Column object describes a field type (column) that is returned from the query results. To create columns: - Use Query.createColumn(options) to create conditions on the initial query definition created with query.create(options). - Use Component.createColumn(options) to create conditions on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). - Assign all created columns as values to <code>Query.columns</code>. For an example, see Syntax.
Type	query.Column[]
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     var mySalesRepJoin = myCustomerQuery.autoJoin({
7         fieldId: 'salesrep'
8     });
9     myCustomerQuery.columns = [
10         myCustomerQuery.createColumn({
11             fieldId: 'entityid'
12         }),
13         mySalesRepJoin.createColumn({
14             fieldId: 'firstname'
15         }),
16         mySalesRepJoin.createColumn({
17             fieldId: 'email'
18         })
19     ];
20 ...
21 // Add additional code

```

Query.condition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The simple or nested condition (a query.Condition object) that narrows the query results. The query.Condition object acts in the same capacity as the search.Filter object in the N/search Module. The primary difference is that query.Condition objects can contain other query.Condition objects. To create conditions: - Use Query.createCondition(options) to create conditions for the initial query definition created with query.create(options). - Use Component.createCondition(options) to create conditions for the join relationships created with Query.autoJoin(options) and Component.autoJoin(options). - If you have multiple conditions, use them to create a new nested condition with the methods Query.and(conditions), Query.or(conditions), and Query.not(condition). - Assign your simple or nested condition to <code>Query.condition</code>. For an example, see Syntax.
Type	query.Condition
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 var myFirstCondition = myCustomerQuery.createCondition({
10  fieldId: 'id',
11  operator: query.Operator.EQUAL,
12  values: 107
13 });
14 var mySecondCondition = myCustomerQuery.createCondition({
15  fieldId: 'id',
16  operator: query.Operator.EQUAL,
17  values: 2647
18 });
19 var myThirdCondition = myCustomerQuery.createCondition({
20  fieldId: 'email',
21  operator: query.Operator.START_WITH_NOT,
22  values: 'foo'
23 });
24 myCustomerQuery.condition = myCustomerQuery.and(myThirdCondition, myCustomerQuery.not( myCustomerQuery.or(myFirstCondition,
25  mySecondCondition)));
26 ...
27 // Add additional code

```

Query.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the query definition. This property has a value only for existing queries that are loaded using query.load(options). If you create a query using query.create(options) but do not save it, this property is null.  Important: The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can use the N/query module to load and delete existing queries, but you cannot save queries. You can save queries using the SuiteAnalytics Workbook interface. For more information, see the help topic Navigating SuiteAnalytics Workbook.
Type	number (read-only)
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myLoadedQuery = query.load({
4   id: 'custworkbook237'
5 });
6 var theId = myLoadedQuery.id;
7 ...
8 // Add additional code

```

Query.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the query definition. This property has a value only for existing queries that are loaded using query.load(options). If you create a query using query.create(options) but do not save it, this property is null.  Important: The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can use the N/query module to load and delete existing queries, but you cannot save queries. You can save queries using the SuiteAnalytics Workbook interface. For more information, see the help topic Navigating SuiteAnalytics Workbook.
Type	string (read-only)

Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myLoadedQuery = query.load({
4         id: 'custworkbook237'
5     });
6     var theName = myLoadedQuery.name;
7     ...
8 // Add additional code

```

Query.root

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The root component of the query definition. The initial query.Component object is called the root component. It encapsulates the initial query type passed to query.create(options). The root component is automatically created with the query.Query object and is a child of the query.Query object.
Type	query.Component (read-only)
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     var theRoot = myCustomerQuery.root;
7     ...
8 // Add additional code

```


Query.sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of query.Sort objects used for sorting. This object encapsulates a sort based on the query.Query or query.Component object. The query.Sort object describes a sort that is placed on a particular query result column. To create a sort: ■ Use Query.createSort(options) to create a sort based on the initial query definition created with query.create(options). ■ Use Component.createSort(options) to create a sort based on a join relationship created with Query.autoJoin(options) or Component.autoJoin(options). ■ Assign all created sorts as values to Query.sort. For an example, see Syntax.
Type	query.Sort[]
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid'
12     }),
13     mySalesRepJoin.createColumn({
14         fieldId: 'firstname'
15     }),
16     mySalesRepJoin.createColumn({
17         fieldId: 'email'
18     })
19 ];
20 myCustomerQuery.sort = [
21     myCustomerQuery.createSort({
22         column: myCustomerQuery.columns[1]
23     }),
24     mySalesRepJoin.createSort({
25         column: myCustomerQuery.columns[0],
26         ascending: false
27     })
28 ];
29 ...
30 // Add additional code

```

Query.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The initial query type of the query definition. This property is set during the of query.create(options) .
Type	string (read-only)
Module	N/query Module
Parent Object	query.Query
Sibling Object Members	Query Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     var theType = myCustomerQuery.type;
7     ...
8 // Add additional code

```

query.RelativeDate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A relative date to use in query conditions. Use query.createRelativeDate(options) to create this object. After you create this object, you can use it in the values parameter of Query.createCondition(options) or Component.createCondition(options). This object represents a specific moment in time, and you can use it to create query conditions using operators from the query.Operator enum, such as <code>query.Operator.AFTER</code>, <code>query.Operator.BEFORE</code>, and <code>query.Operator.WITHIN</code>. For more information about relative dates, see Relative Dates in the N/query Module.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	RelativeDate Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myEndDate = query.createRelativeDate({
4     dateId: query.DateId.WEEKS_AGO,
5     value: 2
6 });
7 var myComplexCondition = myQuery.createCondition({
8     fieldId: 'trandate',
9     operator: query.Operator.WITHIN,
10    values: [query.RelativeDateRange.THREE_FISCAL_YEARS_AGO.start, myEndDate]
11 });
12 ...
13 // Add additional code

```

RelativeDate.dateId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The date ID of the relative date. For relative dates that you create using query.createRelativeDate(options), the value of this property is set when that method is executed. For relative dates that are included in the query.RelativeDateRange enum, the value of this property is always available (for example, <code>query.RelativeDateRange.YESTERDAY.dateId</code>). This property uses values from the query.DateId enum.
Type	string (read-only)
Module	N/query Module
Parent Object	query.RelativeDate
Sibling Object Members	RelativeDate Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myRelativeDate = query.createRelativeDate({
4     dateId: query.DateId.WEEKS_AGO,
5     value: 2
6 });
7 var theDateId = myRelativeDate.dateId;
8 ...
9 // Add additional code

```

RelativeDate.end

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The end of the relative date. For relative dates that you create using query.createRelativeDate(options) , the value of this property is set when that method is executed. For relative date ranges that are included in the query.RelativeDateRange enum, the value of this property is always available (for example, <code>query.RelativeDateRange.YESTERDAY.end</code>).
Type	Object (read-only)
Module	N/query Module
Parent Object	query.RelativeDate
Sibling Object Members	RelativeDate Object Members
Since	2019.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myRelativeDate = query.createRelativeDate({
4         dateId: query.DateId.WEEKS_AGO,
5         value: 2
6     });
7     var theEnd = myRelativeDate.end;
8 ...
9 // Add additional code

```

RelativeDate.interval

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The interval that the relative date represents. For relative dates that you create using query.createRelativeDate(options) , the value of this property is set when that method is executed. For relative date ranges that are included in the query.RelativeDateRange enum, the value of this property is always available (for example, <code>query.RelativeDateRange.YESTERDAY.interval</code>). Important: Do not use this property explicitly in your scripts. It is available so you can see the exact date interval that is used with the <code>query.Operator.WITHIN</code> and <code>query.Operator.WITHIN_NOT</code> operators in query conditions.
Type	Object (read-only)
Module	N/query Module
Parent Object	query.RelativeDate

Sibling Object Members	RelativeDate Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myRelativeDate = query.createRelativeDate({
4     dateId: query.DateId.WEEKS_AGO,
5     value: 2
6 });
7 var theInterval = myRelativeDate.interval;
8 ...
9 // Add additional code

```

RelativeDate.isRange

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the relative date represents a range of dates or a specific moment in time. For relative date ranges that you obtain from the query.RelativeDateRange enum, the value of this property is true (the relative date represents a range of dates). For all other relative dates (such as those that you create using query.createRelativeDate(options)), the value of this property is false (the relative date represents a specific moment in time).
Type	boolean (read-only)
Module	N/query Module
Parent Object	query.RelativeDate
Sibling Object Members	RelativeDate Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myRelativeDate = query.createRelativeDate({
4     dateId: query.DateId.WEEKS_AGO,
5     value: 2
6 });
7
8 var isARange = myRelativeDate.isRange; // isARange is false
9 var isAnotherRange = query.RelativeDateRange.LAST_MONTH_ONE_FISCAL_YEAR_AGO.isRange; // isAnotherRange is true
10 ...
11 // Add additional code

```

RelativeDate.start

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The start of the relative date. For relative dates that you create using query.createRelativeDate(options) , the value of this property is set when that method is executed. For relative date ranges that are included in the query.RelativeDateRange enum, the value of this property is always available (for example, <code>query.RelativeDateRange.YESTERDAY.start</code>).
Type	Object (read-only)
Module	N/query Module
Parent Object	query.RelativeDate
Sibling Object Members	RelativeDate Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myRelativeDate = query.createRelativeDate({
4         dateId: query.DateId.WEEKS_AGO,
5         value: 2
6     });
7     var theStart = myRelativeDate.start;
8 ...
9 // Add additional code

```

RelativeDate.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The value of the relative date range. For relative dates that you create using query.createRelativeDate(options) , the value of this property is set when that method is executed. For relative date ranges that are included in the query.RelativeDateRange enum, the value of this property is undefined (for example, <code>query.RelativeDateRange.YESTERDAY.value</code> is undefined).
Type	number (read-only)
Module	N/query Module
Parent Object	query.RelativeDate
Sibling Object Members	RelativeDate Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myRelativeDate = query.createRelativeDate({
4         dateId: query.DateId.WEEKS_AGO,
5         value: 2
6     });
7     var theValue = myRelativeDate.value;
8 ...
9 // Add additional code

```

query.Result

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A single row of the result set (query.ResultSet).
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/query Module
Methods and Properties	Result Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myCustomerQuery = query.create({
4         type: query.Type.CUSTOMER
5     });
6     myCustomerQuery.columns = [
7         myCustomerQuery.createColumn({
8             fieldId: 'entityid'
9         }),
10        myCustomerQuery.createColumn({
11            fieldId: 'firstname'
12        }),
13        myCustomerQuery.createColumn({
14            fieldId: 'email'
15        })
16    ];
17     var queryResultSet = myCustomerQuery.run();
18
19 // Fetch results using an iterator
20     var iterator = queryResultSet.iterator();
21     iterator.each(function(result) {
22         var currentResult = result.value;
23         log.debug(currentResult);
24         return true;

```

```

25     });
26
27     // Alternatively, fetch results using a loop
28     var queryResults = queryResultSet.results;
29     for (var i = 0; i < queryResults.length; i++) {
30         var currentResult = queryResults[i];
31         log.debug(currentResult);
32     }
33     ...
34     // Add additional code

```

Result.asMap()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the query result as a mapped result. A mapped result is a JavaScript object with key-value pairs. In this object, the key is either the field ID or the alias that was used for the corresponding query.Column object.
Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Result
Sibling Object Members	Result Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid',
12         alias: 'cust'
13     }),
14     myCustomerQuery.createColumn({
15         fieldId: 'id'
16     }),
17     mySalesRepJoin.createColumn({
18         fieldId: 'entityid'
19     }),
20     mySalesRepJoin.createColumn({
21         fieldId: 'email'
22     }), mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'

```



```

24     })
25   ];
26   var resultSet = myCustomerQuery.run();
27   for (var i = 0; i < resultSet.results.length; i++) {
28     var mResult = resultSet.results[i].asMap();
29     log.debug(mResult);
30   }
31   ...
32   // Add additional code

```

Result.getValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value at a given index in Result.values . Value types correspond to the ResultSet.types property. Values correspond to the values for ResultSet.columns .
Returns	<string number Date null> (read-only)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.Result
Sibling Object Members	Result Object Members
Since	2018.2

Parameters

Parameter	Type	Required / Optional	Description
options	number	required	The index of the element from Result.values that you want the method to return.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 const COLUMNS = ['entityid', 'id'];
4 const employeeInfo = {};
5
6 let employeeQuery = query.create({
7   type: query.Type.EMPLOYEE
8 });
9 COLUMNS.forEach(column => {
10   employeeQuery.columns.push(employeeQuery.createColumn({
11     fieldId: column
12   }));
13 });
14
15 const resultSet = employeeQuery.run();

```

```

16 const results = resultSet.results || [];
17
18 results.forEach(result => {
19   for (let i = 0; i < result.values.length; i++) {
20     employeeInfo[COLUMNS[i]] = result.getValue(i);
21   }
22 });
23 ...
24 // Add additional code

```

Result.values

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The result values. Value types correspond to the ResultSet.types property. Values correspond to the values for ResultSet.columns .
Type	<string number null> (read-only)
Module	N/query Module
Parent Object	query.Result
Sibling Object Members	Result Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7   myCustomerQuery.createColumn({
8     fieldId: 'entityid'
9   }),
10  myCustomerQuery.createColumn({
11    fieldId: 'email'
12  })
13 ];
14 var queryResultSet = myCustomerQuery.run();
15 var queryResults = queryResultSet.results;
16 var myFirstResult = queryResults[0];
17 var theValues = myFirstResult.values;
18 ...
19 // Add additional code

```

query.ResultSet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The set of results returned by the query. Use Query.run(options) or Query.run.promise() to create this object.
---------------------------	--

	The maximum number of results in a <code>ResultSet</code> object is 5000. If a query matches more than 5000 results, you must use Query.runPaged(options) or Query.runPaged.promise(options) to retrieve the full set of results.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/query Module
Methods and Properties	ResultSet Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     }),
10    myCustomerQuery.createColumn({
11        fieldId: 'email'
12    })
13 ];
14 var resultSet = myCustomerQuery.run();
15 var results = resultSet.results;
16 for (var i = results.length - 1; i >= 0; i--)
17     log.debug(results[i].values);
18 ...
19 // Add additional code

```

ResultSet.asMappedResults()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the query result set as an array of mapped results. A mapped result is a JavaScript object with key-value pairs. In this object, the key is either the field ID or the alias that was used for the corresponding query.Column object. When you call this method, Result.asMap() is called on each query.Result object in the result set.
Returns	<code>Object[]</code>
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.ResultSet

Sibling Object Members	ResultSet Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid',
12         alias: 'cust'
13     }),
14     myCustomerQuery.createColumn({
15         fieldId: 'id'
16     }),
17     mySalesRepJoin.createColumn({
18         fieldId: 'entityid'
19     }),
20     mySalesRepJoin.createColumn({
21         fieldId: 'email'
22     }),
23     mySalesRepJoin.createColumn({
24         fieldId: 'hiredate'
25     })
26 ];
27 var resultSet = myCustomerQuery.run();
28 var mrSet = resultSet.asMappedResults();
29 ...
30 // Add additional code

```

ResultSet.iterator()

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Standard SuiteScript 2.0 object for iterating through results
Returns	Iterator object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Parent Object	query.ResultSet
Sibling Object Members	ResultSet Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7   myCustomerQuery.createColumn({
8     fieldId: 'entityid'
9   }),
10  myCustomerQuery.createColumn({
11    fieldId: 'firstname'
12  }),
13  myCustomerQuery.createColumn({
14    fieldId: 'email'
15  })
16 ];
17 var queryResultSet = myCustomerQuery.run(); // Fetch results using an iterator
18 var iterator = queryResultSet.iterator();
19 iterator.each(function(result) {
20   var currentResult = result.value;
21   log.debug(currentResult);
22   return true;
23 });
24
25 // Alternatively, fetch results using a loop
26 var queryResults = queryResultSet.results;
27 for (var i = 0; i < queryResults.length; i++) {
28   var currentResult = queryResults[i]; log.debug(currentResult);
29 }
30 ...
31 // Add additional code

```

ResultSet.columns

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of query return column references. The <code>ResultSet.columns</code> values correspond with the ResultSet.types values.
Type	<code>query.Column[]</code> (read-only)
Module	N/query Module
Parent Object	query.ResultSet
Sibling Object Members	ResultSet Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...

```

```

3   var myCustomerQuery = query.create({
4       type: query.Type.CUSTOMER
5   });
6   myCustomerQuery.columns = [
7       myCustomerQuery.createColumn({
8           fieldId: 'entityid'
9       }),
10      myCustomerQuery.createColumn({
11          fieldId: 'email'
12      })
13  ];
14  var queryResultSet = myCustomerQuery.run();
15  var theColumns = queryResultSet.columns;
16  ...
17  // Add additional code

```

ResultSet.results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of query.Result objects.
Type	query.Result[] (read-only)
Module	N/query Module
Parent Object	query.ResultSet
Sibling Object Members	ResultSet Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1   // Add additional code
2   ...
3   var myCustomerQuery = query.create({
4       type: query.Type.CUSTOMER
5   });
6   myCustomerQuery.columns = [
7       myCustomerQuery.createColumn({
8           fieldId: 'entityid'
9       }),
10      myCustomerQuery.createColumn({
11          fieldId: 'email'
12      })
13  ];
14  var queryResultSet = myCustomerQuery.run();
15  var theResults = queryResultSet.results;
16  ...
17  // Add additional code

```

ResultSet.types

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of the return types for ResultSet.results .
-----------------------------	--

	The <code>ResultSet.types</code> values correspond with the ResultSet.columns values.
Type	<code>string[]</code> (read-only)
Module	N/query Module
Parent Object	query.ResultSet
Sibling Object Members	ResultSet Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     }),
10    myCustomerQuery.createColumn({
11        fieldId: 'email'
12    })
13 ];
14 var queryResultSet = myCustomerQuery.run();
15 var theTypes = queryResultSet.types;
16 ...
17 // Add additional code

```

query.Sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A sort based on the query.Query or query.Component object. The <code>query.Sort</code> object describes a sort that is placed on a particular query result column. To create a sort: - Use Query.createSort(options) to create a sort based on the initial query definition created with query.create(options). - Use Component.createSort(options) to create a sort based on a join relationship created with Query.autoJoin(options) or Component.autoJoin(options). - Assign all created sorts as values to Query.sort. For an example, see Syntax.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	Sort Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid'
12     }),
13     myCustomerQuery.createColumn({
14         fieldId: 'id'
15     }),
16     mySalesRepJoin.createColumn({
17         fieldId: 'entityid'
18     }),
19     mySalesRepJoin.createColumn({
20         fieldId: 'email'
21     }),
22     mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'
24     })
25 ];
26 myCustomerQuery.sort = [
27     myCustomerQuery.createSort({
28         column: myCustomerQuery.columns[1]
29     }),
30     mySalesRepJoin.createSort({
31         column: mySalesRepJoin.columns[0],
32         ascending: false
33     })
34 ];
35 var resultSet = myCustomerQuery.run();
36 ...
37 // Add additional code

```

Sort.ascending

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the sort direction is ascending. This property is set during the of Query.createSort(options) and Component.createSort(options). The default value of this property is true, meaning that the sort direction is ascending. If you want the sort direction to be descending, set this property to false.
Type	boolean
Module	N/query Module
Parent Object	query.Sort
Sibling Object Members	Sort Object Members

Since	2018.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 myCustomerQuery.sort = [
12     myCustomerQuery.createSort({
13         column: myCustomerQuery.columns[0],
14         ascending: false,
15         caseSensitive: true,
16         locale: query.SortLocale.EN_CA,
17         nullsLast: false
18     })
19 ];
20 var theAscending = myCustomerQuery.sort[0].ascending;
21 ...
22 // Add additional code

```

Sort.caseSensitive

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the sort is case sensitive. This property is set during the of Query.createSort(options) and Component.createSort(options). If a sort is case sensitive (and the sort direction is ascending), rows with column values that start with uppercase letters are listed before rows with column values that start with lowercase letters. If a sort is not case sensitive, uppercase and lowercase letters are treated the same. For example, the following list of items is sorted using a case-sensitive sort with a sort direction of ascending: ■ Banana ■ Orange ■ apple ■ grapefruit ■ kiwi Here is the same list of items sorted using a regular (not case-sensitive) sort with a sort direction of ascending: ■ apple ■ Banana ■ grapefruit ■ kiwi
-----------------------------	---

	 Orange The default value of this property is false.
Type	boolean
Module	N/query Module
Parent Object	query.Sort
Sibling Object Members	Sort Object Members
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 myCustomerQuery.sort = [
12     myCustomerQuery.createSort({
13         column: myCustomerQuery.columns[0],
14         ascending: false,
15         caseSensitive: true,
16         locale: query.SortLocale.EN_CA,
17         nullsLast: false
18     })
19 ];
20 var theCaseSensitive = myCustomerQuery.sort[0].caseSensitive;
21 ...
22 // Add additional code

```

Sort.column

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The query result column that the query results are sorted by. This property is set during the of Query.createSort(options) and Component.createSort(options) .
Type	query.Column (read-only)
Module	N/query Module
Parent Object	query.Sort
Sibling Object Members	Sort Object Members
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [
7     myCustomerQuery.createColumn({
8         fieldId: 'entityid'
9     })
10 ];
11 myCustomerQuery.sort = [
12     myCustomerQuery.createSort({
13         column: myCustomerQuery.columns[0],
14         ascending: false,
15         caseSensitive: true,
16         locale: query.SortLocale.EN_CA,
17         nullsLast: false
18     })
19 ];
20 var theColumn = myCustomerQuery.sort[0].column;
21 ...
22 // Add additional code

```

Sort.locale

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The locale to use for the sort. This property uses values from the query.SortLocale enum. This property is set during the of Query.createSort(options) and Component.createSort(options). A locale represents a combination of language and region, and it can affect how certain values (such as strings) are sorted. For example, languages that share the same alphabet may sort characters differently. Use this property to ensure that query results are sorted using locale-specific rules.
Type	string
Module	N/query Module
Parent Object	query.Sort
Sibling Object Members	Sort Object Members
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  myCustomerQuery.columns = [
7      myCustomerQuery.createColumn({
8          fieldId: 'entityid'
9      })
10 ];
11 myCustomerQuery.sort = [
12     myCustomerQuery.createSort({
13         column: myCustomerQuery.columns[0],
14         ascending: false,
15         caseSensitive: true,
16         locale: query.SortLocale.EN_CA,
17         nullsLast: false
18     })
19 ];
20 var theLocale = myCustomerQuery.sort[0].locale;
21 ...
22 // Add additional code

```

Sort.nullsLast

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether query results with null values are listed at the end of the query results. This property is set during the of Query.createSort(options) and Component.createSort(options). The default value of this property is the value of the Sort.ascending property. For example, if the Sort.ascending property is set to true, the Sort.nullsLast property is also set to true.
Type	boolean
Module	N/query Module
Parent Object	query.Sort
Sibling Object Members	Sort Object Members
Since	2018.2

Syntax

⚠ Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var myCustomerQuery = query.create({
4      type: query.Type.CUSTOMER
5  });
6  myCustomerQuery.columns = [
7      myCustomerQuery.createColumn({
8          fieldId: 'entityid'
9      })
10 ];
11 myCustomerQuery.sort = [

```

```

12     myCustomerQuery.createSort({
13         column: myCustomerQuery.columns[0],
14         ascending: false,
15         caseSensitive: true,
16         locale: query.SortLocale.EN_CA,
17         nullsLast: false
18     })
19 ];
20 var theNullsLast = myCustomerQuery.sort[0].nullsLast;
21 ...
22 // Add additional code

```

query.SuiteQL

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A SuiteQL query. SuiteQL is a query language based on the SQL-92 revision of the SQL database query language. It provides advanced query capabilities you can use to access your NetSuite records and data. Use Query.toSuiteQL() to create this object. This method converts an existing query.Query object to its corresponding SuiteQL representation as a query.SuiteQL object. You can use SuiteQL.run() to run the query and obtain the results as a query.ResultSet object. You can also use SuiteQL.runPaged(options) to run the query as a paged query and obtain the results as a query.PagedData object. When you convert a query.Query object to a query.SuiteQL object, the resulting SuiteQL query is the same as the original query. It includes the same query result columns, sort order, and conditions that were set on the original query. When you run the resulting SuiteQL query using SuiteQL.run() or SuiteQL.runPaged(options), you receive the same results as you would if you ran the original query using Query.run(options) or Query.runPaged(options). For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/query Module
Methods and Properties	SuiteQL Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var results = mySuiteQLQuery.run();
6 ...
7 // Add additional code

```

SuiteQL.run()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Runs the SuiteQL query and returns the query results. You can use this method to run the SuiteQL query and obtain the results as a query.ResultSet object. If you want to run the SuiteQL query as a paged query, use SuiteQL.runPaged(options). Calling this method is equivalent to calling query.runSuiteQL(options) and passing the query.SuiteQL object as a parameter: <pre> 1 // These two query calls return the same result set 2 var firstResultSet = mySuiteQLObject.run(); 3 var secondResultSet = query.runSuiteQL(mySuiteQLObject); </pre> Note: If the SuiteAnalytics Connect feature is enabled in your NetSuite account, there is no limit to the number of results this method can return. If the SuiteAnalytics Connect feature is not enabled, this method can return a maximum of 100,000 results. For more information about SuiteAnalytics Connect, see the help topic SuiteAnalytics Connect. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	query.ResultSet
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/query Module
Parent Object	query.SuiteQL
Sibling Object Members	SuiteQL Object Members
Since	2020.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary.

Parameter	Type	Required / Optional	Description
			 Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var results = mySuiteQLQuery.run({
6     customScriptId: 'myCustomScriptId'
7 });
8 ...
9 // Add additional code

```

SuiteQL.runPaged(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Important:** You must specify a sorting order in the query definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Runs the SuiteQL query as a paged query and returns the paged query results. You can use this method to run the SuiteQL query and obtain the results as a query.PagedData object. If you want to run the SuiteQL query as a non-paged query, use SuiteQL.run().  Note: If the SuiteAnalytics Connect feature is enabled in your NetSuite account, there is no limit to the number of results this method can return. If the SuiteAnalytics Connect feature is not enabled, this method can return a maximum of 100,000 results across all pages in the result set. For more information about SuiteAnalytics Connect, see the help topic SuiteAnalytics Connect. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	query.PagedData  Important: This method returns a maximum of 1000 pages.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units

Module	N/query Module
Parent Object	query.SuiteQL
Sibling Object Members	SuiteQL Object Members
Since	2020.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.pageSize	number	optional	The size of each page in the query results. The default value is 50 results per page. The minimum page size is 5 results per page, and the maximum page size is 1000 results per page.
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary.  Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var results = mySuiteQLQuery.runPaged({
6     pageSize: 20,
7     customScriptId: 'myCustomScriptId'
8 });
9 ...
10 // Add additional code

```

SuiteQL.columns

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The result columns to be returned from the query.
-----------------------------	---

	This property is an array of query.Column objects. When you use Query.toSuiteQL() to convert an existing query.Query object to SuiteQL, this property contains the same query.Column objects that were specified for the original query.  Important: The SuiteQL query string in a query.SuiteQL object (contained in the SuiteQL.query property) does not include any aliases you set on query result columns in the original query.Query object. For more information about aliases, see Column.alias.
Type	query.Column[] (read-only)
Module	N/query Module
Parent Object	query.SuiteQL
Sibling Object Members	SuiteQL Object Members
Since	2020.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var theColumns = mySuiteQLQuery.columns;
6 ...
7 // Add additional code

```

SuiteQL.params

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The parameters for the query. In SuiteQL, query conditions are represented using the WHERE clause and a set of parameters. In a SuiteQL string, parameter values for conditions are represented using question marks (?), and the params property includes a list of the parameter values to use when the query runs. If the query uses more than one parameter, the order of the values in the params property matches the order the parameters appear in the query string. For example, consider the following query.Condition object in a query for customer records: <pre> 1 myQueryObject.createCondition({ 2 fieldId: 'creditlimit', 3 operator: query.Operator.LESS_OR_EQUAL, 4 values: [50000] 5 }); </pre> If you use Query.toSuiteQL() to convert this query to SuiteQL, the resulting SuiteQL query string includes the following WHERE clause: <pre> 1 WHERE customer.creditlimit <= ? </pre>
----------------------	--

	In the resulting query.SuiteQL object, the <code>params</code> property includes the value 50000.
Type	<string number > (read-only)
Module	N/query Module
Parent Object	query.SuiteQL
Sibling Object Members	SuiteQL Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var theParams = mySuiteQLQuery.params;
6 ...
7 // Add additional code

```

SuiteQL.query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The string representation of the SuiteQL query. This string can contain the following types of elements: - SQL clauses, such as SELECT, FROM, and WHERE - Record or table names, such as customer - Field names using dot notation, such as customer.entityid - Operators for conditions, such as = and >= - Formatting characters, such as newline characters (\n) This string can also contain field formatting metadata, such as <code>/{entityid#RAW}*/</code>. When you use Query.toSuiteQL() to convert a constructed query to its SuiteQL query equivalent, metadata may be added to the query string to indicate the formatting (or context) of fields in the query. For example, <code>/{entityid#RAW}*/</code> metadata indicates that the <code>entityid</code> field is formatted using the <code>query.FieldContext.RAW</code> field context from the query.FieldContext enum. This metadata is added as comments (using <code>/*</code> and <code>*/</code>) and does not affect query or the query results.
Type	string (read-only)
Module	N/query Module
Parent Object	query.SuiteQL
Sibling Object Members	SuiteQL Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var theQuery = mySuiteQLQuery.query;
6 ...
7 // Add additional code
```

SuiteQL.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of the query. This property uses values from the query.Type enum. When you use Query.toSuiteQL() to convert an existing query.Query object to SuiteQL, this property contains the same type that was specified for the original query.
Type	string (read-only)
Module	N/query Module
Parent Object	query.SuiteQL
Sibling Object Members	SuiteQL Object Members
Since	2020.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // myQuery is an existing query.Query object
4 var mySuiteQLQuery = myQuery.toSuiteQL();
5 var theType = mySuiteQLQuery.type;
6 ...
7 // Add additional code
```

query.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a query.Query object. Use this method to create your initial query definition. The initial query definition uses one query type. For available query types, see query.Type .
---------------------------	---

	After you create the initial query definition, use Query.autoJoin(options) to create your first join. Then use Query.autoJoin(options) or Component.autoJoin(options) to create all subsequent joins. For standard record types, the query type that you specify is validated immediately and must be one of the values in the query.Type enum. For custom record types, the query type that you specify is not validated until the query is executed using Query.run(options) or Query.runPaged(options) (or using the promise versions of these methods). If you specify a query type for a custom record type that does not exist, this method allows you to create the query and does not throw an error. However, when you perform the query, an error is thrown.  Important: The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can use the N/query module to load and delete existing queries, but you cannot save queries. You can save queries using the SuiteAnalytics Workbook interface. For more information, see the help topic Navigating SuiteAnalytics Workbook. For more information about creating queries, see Scripting with the N/query Module.
Returns	query.Query
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.type</code>	string	required	The query type that you want to use for the initial query definition. Use the query.Type enum to set this value (for an example, see Syntax). When you perform <code>query.create(options)</code>, the Query.type property is set based on this value.  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
<code>options.columns</code>	Object[]	optional	An array of objects to be used as query columns. If you specify a value for this parameter, the Query.createColumn(options) method is called on each object in the.

Parameter	Type	Required / Optional	Description
			 Important: You must include a query.Column object in your query definition to avoid scripting errors. This object is an optional parameter with the query.create(options) method, but at least one Column must be created for every query definition. For more information, see the syntax example below.
options.condition	Object	optional	A condition for the query. If you specify a value for this parameter, the Query.createCondition(options) method is called on the object you specify.
options.sort	Object[]	optional	An array of objects representing sort options. If you specify a value for this parameter, the Query.createColumn(options) and Query.createSort(options) methods are called on each object in the .

Errors

Error Code	Thrown If
INVALID_RCRD_TYPE	The specified query type is invalid.  Note: This error is not thrown if you specify a custom record type as the query type. Custom record types are validated when the query is executed using Query.run(options) or Query.runPaged(options) (or using the promise versions of these methods).

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({
7     fieldId: 'salesrep'
8 });
9 myCustomerQuery.columns = [
10     myCustomerQuery.createColumn({
11         fieldId: 'entityid'
12     }),
13     myCustomerQuery.createColumn({
14         fieldId: 'id'
15     }),
16     mySalesRepJoin.createColumn({
17         fieldId: 'entityid'
18     }),
19     mySalesRepJoin.createColumn({
20         fieldId: 'email'

```

```

21     }},
22     mySalesRepJoin.createColumn({
23         fieldId: 'hiredate'
24     })
25 ];
26 myCustomerQuery.sort = [
27     myCustomerQuery.createSort({
28         column: myCustomerQuery.columns[1]
29     }),
30     mySalesRepJoin.createSort({
31         column: mySalesRepJoin.columns[0],
32         ascending: false
33     })
34 ];
35 var resultSet = myCustomerQuery.run();
36 ...
37 // Add additional code

```

query.createPeriod(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a query.Period object.
Returns	query.Period
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.code	string	required	The code of the period to create. Use a value from the query.PeriodCode enum for this parameter.
options.adjustment	string	optional	The adjustment of the period to create. Use a value from the query.PeriodAdjustment enum for this parameter. The default value is <code>query.PeriodAdjustment.NOT_LAST</code> .
options.type	string	optional	The type of the period to create. Use a value from the query.PeriodType enum for this parameter. The default value is <code>query.PeriodType.START</code> .

Errors

Error Code	Thrown If
INVALID_PERIOD_ADJUSTMENT	The specified period adjustment is not a value from the query.PeriodAdjustment enum.
INVALID_PERIOD_CODE	The specified period code is not a value from the query.PeriodCode enum.
INVALID_PERIOD_TYPE	The specified period type is not a value from the query.PeriodType enum.
WRONG_PARAMETER_TYPE	Any of the parameters is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myPeriod = query.createPeriod({
4     code: query.PeriodCode.LAST_PERIOD,
5     adjustment: query.PeriodAdjustment.ALL,
6     type: query.PeriodType.END
7 });
8 // myQuery is an existing query.Query object
9 var myComplexCondition = myQuery.createCondition({
10     fieldId: 'trandate',
11     operator: query.Operator.BEFORE,
12     values: [myPeriod]
13 });
14 ...
15 // Add additional code

```

query.createRelativeDate(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a query.RelativeDate object that represents a date relative to the current date. Use this method to create a query.RelativeDate object to use as part of a query condition. After you create a query.RelativeDate object, you can use it directly in the values parameter of Query.createCondition(options) or Component.createCondition(options). When you call this method, the options.dateId parameter determines the relative date that is created. The options.dateId parameter uses values from the query.DateId enum, and these values describe potential dates relative to the current date. Use them along with the options.value parameter to create a relative date. For example, to create a relative date that represents the date three weeks before the current date, call query.createRelativeDate(options) with an options.dateId value of query.DateId.WEEKS_AGO and an options.value value of 3. To create a relative date that represents the date three weeks after the current date, call query.createRelativeDate(options) with an options.dateId value of query.DateId.WEEKS_FROM_NOW and an options.value value of 3.
Returns	query.RelativeDate
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.dateId	string	required	The ID of the relative date to create. Use the query.DateId enum to set this value.
options.value	number	required	The value to use to create the relative date. This value depends on the value that you specify for options.dateId. For example, to create a relative date that represents the date five days before the current date, use an options.value value of 5 and an options.dateId value of query.DateId.DAYS_AGO .

Errors

Error Code	Thrown If
INVALID_DATE_ID	The specified value for options.dateId is not a value from the query.DateId enum.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myRelativeDate = query.createRelativeDate({
7     dateId: query.DateId.DAYS_AGO,
8     value: 5
9 });
10 myTransactionQuery.condition = myTransactionQuery.createCondition({
11     fieldId: 'trandate',
12     operator: query.Operator.WITHIN,
13     values: [query.RelativeDateRange.THREE_FISCAL_YEARS_AGO.start, myRelativeDate]
14 });
15 ...
16 // Add additional code

```


query.delete(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes an existing query. Use this method to delete a query definition that was previously created using the SuiteAnalytics Workbook UI. After the query is deleted, it is no longer available and cannot be modified or executed.  Important: The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can use the N/query module to load and delete existing queries, but you cannot save queries. You can save queries using the SuiteAnalytics Workbook interface. For more information, see the help topic Navigating SuiteAnalytics Workbook.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	5 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The script ID of the query to delete.

Errors

Error Code	Thrown If
UNABLE_TO_DELETE_QUERY	A query with the specified ID cannot be deleted because the query does not exist or you do not have permission to delete it.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
```

```

2  ...
3      query.delete({
4          id: 'custworkbook237'
5      });
6  ...
7  // Add additional code

```

query.listTables(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Lists the table view objects that are included in a workbook in SuiteAnalytics Workbook. This method returns an array of Objects. Each Object represents a table view and includes the following properties with associated values: - name — The name of the table view object - scriptId — The script ID of the table view object
Returns	<Object>
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.workbookId	string	required	The ID of the workbook containing the table view objects to list.

Errors

Error Code	Thrown If
MISSING_REQD_ARGUMENT	A required parameter is missing.
SCRIPT_ID_OF_WORKBOOK_IS_REQUIRED	The specified workbook ID represents an analytical record that is not a workbook.
SSS_INVALID_SCRIPT_ID_1	The specified workbook ID is not valid.

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The specified workbook ID is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var tables = query.listTables({
4         workbookId: 'custworkbook237'
5     });
6 ...
7 // Add additional code

```

query.load(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing query as a query.Query object. Use this method to load a query definition that was previously created using the SuiteAnalytics Workbook UI. After the query is loaded, you can modify the query definition (for example, by setting additional property values), join the query definition with other query types, and perform the query in the same way as queries that you create using query.create(options). You can provide either the workbook ID or the dataset ID of the query definition to load. You can obtain these IDs in the SuiteAnalytics Workbook UI. For more information, see the help topic Navigating SuiteAnalytics Workbook. This method loads only the table view in the specified workbook. However, a workbook can include no table views, or it can include multiple table views. This method throws an exception in each of these cases (see Errors). If a workbook includes multiple table views, you can use query.listTables(options) to determine the ID of a specific table view. You can use this ID to load the corresponding table view. Important: The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can use the N/query module to load and delete existing queries, but you cannot save queries. You can save queries using the SuiteAnalytics Workbook interface. For more information, see the help topic Navigating SuiteAnalytics Workbook.
Returns	query.Query
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	5 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The workbook ID or dataset ID of the query definition to load. You can obtain these IDs in the SuiteAnalytics Workbook UI. For more information, see the help topic Navigating SuiteAnalytics Workbook .

Errors

Error Code	Thrown If
UNABLE_TO_LOAD_QUERY	A query with the specified ID cannot be loaded because the query does not exist or you do not have permission to load it.
WORKBOOK_MORE_TABLEVIEWS_ARE_ASSIGNED	More than one table view is included in the specified workbook or dataset.
WORKBOOK_NO_TABLEVIEW_IS_ASSIGNED	No table views are included in the specified workbook or dataset.

Note: If the loaded query does not return a desired sorting order, you must use the `Query.runPaged(options)` method for default sorting or specify your custom sorting before script .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myLoadedQuery = query.load({
4   id: 'custworkbook237'
5 });
6 var mySalesRepJoin = myLoadedQuery.autoJoin({
7   fieldId: 'salesrep'
8 });
9 var results = myLoadedQuery.run();
10 ...
11 // Add additional code

```

query.load.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing query asynchronously as a <code>query.Query</code> object. Use this method to asynchronously load a query definition that was previously created using the SuiteAnalytics Workbook UI. After the query is loaded, you can modify the query definition (for example, by setting additional property values), join the query definition with
---------------------------	--

	other query types, and perform the query in the same way as queries that you create using query.create(options). You can provide either the workbook ID or the dataset ID of the query definition to load. You can obtain these IDs in the SuiteAnalytics Workbook UI. For more information, see the help topic Navigating SuiteAnalytics Workbook. This method loads only the table view in the specified workbook. However, a workbook can include no table views, or it can include multiple table views. This method throws an exception in each of these cases (see Errors). If a workbook includes multiple table views, you can use query.listTables(options) to determine the ID of a specific table view. You can use this ID to load the corresponding table view.  Important: The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can use the N/query module to load and delete existing queries, but you cannot save queries. You can save queries using the SuiteAnalytics Workbook interface. For more information, see the help topic Navigating SuiteAnalytics Workbook.  Note: The parameters and errors thrown for this method are the same as those for query.load(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	query.load(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The workbook ID or dataset ID of the query definition to load. You can obtain these IDs in the SuiteAnalytics Workbook UI. For more information, see the help topic Navigating SuiteAnalytics Workbook.

Errors

Error Code	Thrown If
UNABLE_TO_LOAD_QUERY	A query with the specified ID cannot be loaded because the query does not exist or you do not have permission to load it.

Error Code	Thrown If
WORKBOOK_MORE_TABLEVIEWS_ARE_ASSIGNED	More than one table view is included in the specified workbook or dataset.
WORKBOOK_NO_TABLEVIEW_IS_ASSIGNED	No table views are included in the specified workbook or dataset.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3   var myLoadedQuery = query.load.promise({
4     id: 'custworkbook237'
5   }).then (function(result) {
6     // Process the result
7   });
8   var mySalesRepJoin = myLoadedQuery.autoJoin({
9     fieldId: 'salesrep'
10  });
11  var results = myLoadedQuery.run();
12  ...
13 // Add additional code

```

query.runSuiteQL(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Runs an arbitrary SuiteQL query. SuiteQL is a query language based on the SQL-92 revision of the SQL database query language. It provides advanced query capabilities you can use to access your NetSuite records and data. You can specify the SuiteQL query as one of the following: ■ A string representation of the SuiteQL query <pre> 1 var results = query.runSuiteQL({ 2 query: 'SELECT customer.entityid, customer.email FROM customer' 3 }); </pre> ■ A <code>query.SuiteQL</code> object <pre> 1 // In this example, mySuiteQLCustomerQuery is an existing SuiteQL object 2 var results = query.runSuiteQL(mySuiteQLCustomerQuery); </pre> ■ A JavaScript Object that contains a <code>query</code> property and, optionally, a <code>params</code> property <pre> 1 var results = query.runSuiteQL({ 2 query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?', 3 params: [true] 4 }); </pre>
---------------------------	---

	Note: This method can return a maximum of 5000 results. If you need to return more results, use <code>query.runSuiteQLPaged(options)</code> instead. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	query.ResultSet
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.query</code>	string	required	The string representation of the SuiteQL query to run.
<code>options.params</code>	<string number >	optional	The parameters to use in the SuiteQL query.
<code>options.customScriptId</code>	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary. Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.
<code>options.metadataProvider</code>	string	optional	Indicates whether the query should fail if you lack the necessary permissions for some fields or records. If set to <code>SUITE_QL</code>: The query fails if you attempt to access fields or records without the necessary permissions. If set to <code>STATIC</code>:

Parameter	Type	Required / Optional	Description
			The query succeeds even if you lack permissions for some fields or records, but it does not return any data for those fields or records. Defaults to SUITE_QL.

Errors

Error Code	Thrown If
MISSING_REQD_ARGUMENT	The parameter is missing.
SSS_INVALID_TYPE_ARG	Types other than string, number, or are included in the options.params.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var results = query.runSuiteQL({
4     query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?',
5     params: [true],
6     customScriptId: 'myCustomScriptId'
7 });
8 ...
9 // Add additional code

```

query.runSuiteQL.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously runs an arbitrary SuiteQL query. SuiteQL is a query language based on the SQL-92 revision of the SQL database query language. It provides advanced query capabilities you can use to access your NetSuite records and data. You can specify the SuiteQL query as one of the following: ■ A string representation of the SuiteQL query <pre> 1 var results = query.runSuiteQL.promise({ 2 query: 'SELECT customer.entityid, customer.email FROM customer' 3 }); </pre> ■ A <code>query.SuiteQL</code> object <pre> 1 // In this example, mySuiteQLCustomerQuery is an existing SuiteQL object 2 var results = query.runSuiteQL.promise(mySuiteQLCustomerQuery); </pre> ■ A JavaScript Object that contains a query property and, optionally, a params property <pre> 1 var results = query.runSuiteQL.promise({ 2 query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?', </pre>
---------------------------	---

	<pre> 3 params: [true] 4 }); </pre>  Note: This method can return a maximum of 5000 results. If you need to return more results, use query.runSuiteQLPaged(options) instead. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	Promise Object
Synchronous Version	query.runSuiteQL(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Parameters

 Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.query	string	required	The string representation of the SuiteQL query to run.
options.params	<string number >	optional	The parameters to use in the SuiteQL query.
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary.  Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.

Errors

Error Code	Thrown If
MISSING_REQD_ARGUMENT	The parameter is missing.

Error Code	Thrown If
SSS_INVALID_TYPE_ARG	Types other than string, number, or are included in the options.params.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const resultSet = await query.runSuiteQL.promise({
5   query: 'SELECT customer.entityid, customer.email FROM customer',
6   params: [true],
7   customScriptId: 'myCustomScriptId'
8 });
9 ...
10 // Add additional code

```

query.runSuiteQLPaged(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: You must specify a sorting order in the query definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Runs an arbitrary SuiteQL query as a paged query. SuiteQL is a query language based on the SQL-92 revision of the SQL database query language. It provides advanced query capabilities you can use to access your NetSuite records and data. You can specify the SuiteQL query as one of the following: ■ A string representation of the SuiteQL query <pre> 1 var results = query.runSuiteQLPaged({ 2 query: 'SELECT customer.entityid, customer.email FROM customer', 3 pageSize: 10 4 }); </pre> ■ A <code>query.SuiteQL</code> object <pre> 1 // To use this approach, you must load the N/util module in your script 2 // In this example, mySuiteQLCustomerQuery is an existing SuiteQL object 3 var pageSizeObject = { 4 pageSize: 10 5 }; 6 var results = query.runSuiteQL(util.extend(mySuiteQLCustomerQuery, pageSizeObject)); </pre> Although this approach is supported, you should create a <code>query.SuiteQL</code> object and call <code>SuiteQL.runPaged(options)</code>, if possible, to run a paged SuiteQL query. ■ A JavaScript Object that contains a <code>query</code> property and, optionally, a <code>params</code> property <pre> 1 var results = query.runSuiteQL({ 2 query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?', 3 params: [true], pageSize: 10 4 }); </pre>
---------------------------	--

	 Important: You cannot retrieve the columns (or column names) from the results of this method. If you need to refer to the columns used in the query, create a query.SuiteQL object, which contains the columns in the query in the SuiteQL.columns property. Then, pass the object to this method, or use SuiteQL.runPaged(options), to run the query.  Note: If the SuiteAnalytics Connect feature is enabled in your NetSuite account, there is no limit to the number of results this method can return. If the SuiteAnalytics Connect feature is not enabled, this method can return a maximum of 100,000 results across all pages in the result set. For more information about SuiteAnalytics Connect, see the help topic SuiteAnalytics Connect. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	query.PagedData  Important: This method returns a maximum of 1000 pages.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
options.query	string	required	The string representation of the SuiteQL query to run.
options.params	<string number >	optional	The parameters to use in the SuiteQL query.
options.pageSize	number	optional	The size of each page in the query results. The default value is 50 results per page. The minimum page size is 5 results per page, and the maximum page size is 1000 results per page.
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur.

Parameter	Type	Required / Optional	Description
			This ID can also be used as a precaution to speed up performance fixes, if necessary.  Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.
options.metaDataProvider	string	optional	Indicates whether the query should fail if you lack the necessary permissions for some fields or records. If set to SUITE_QL: The query fails if you attempt to access fields or records without the necessary permissions. If set to STATIC: The query succeeds even if you lack permissions for some fields or records, but it does not return any data for those fields or records. Defaults to SUITE_QL.

Errors

Error Code	Thrown If
MISSING_REQD_ARGUMENT	The parameter is missing.
SSS_INVALID_TYPE_ARG	Types other than string, number, or are included in the options.params.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var results = query.runSuiteQLPaged({
4   query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?',
5   params: [true],
6   pageSize: 20,
7   customScriptId: 'myCustomScriptId'
8 });
9 ...
10 // Add additional code

```

query.runSuiteQLPaged.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Important: You must specify a sorting order in the query definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Asynchronously runs an arbitrary SuiteQL query as a paged query. SuiteQL is a query language based on the SQL-92 revision of the SQL database query language. It provides advanced query capabilities you can use to access your NetSuite records and data. You can specify the SuiteQL query as one of the following: A string representation of the SuiteQL query <pre> 1 var results = query.runSuiteQLPaged.promise({ 2 query: 'SELECT customer.entityid, customer.email FROM customer', 3 pageSize: 10 4 }); </pre> A query.SuiteQL object <pre> 1 // To use this approach, you must load the N/util module in your script 2 // In this example, mySuiteQLCustomerQuery is an existing SuiteQL object 3 var pageSizeObject = { 4 pageSize: 10 5 }; 6 var results = query.runSuiteQL(util.extend(mySuiteQLCustomerQuery, pageSizeObject)); </pre> Although this approach is supported, you should create a query.SuiteQL object and call SuiteQL.runPaged(options), if possible, to run a paged SuiteQL query. A JavaScript Object that contains a query property and, optionally, a params property <pre> 1 var results = query.runSuiteQL({ 2 query: 'SELECT customer.entityid, customer.email FROM customer WHERE customer.isperson = ?', 3 params: [true], pageSize: 10 4 }); </pre> Important: You cannot retrieve the columns (or column names) from the results of this method. If you need to refer to the columns used in the query, create a query.SuiteQL object, which contains the columns in the query in the SuiteQL.columns property. Then, pass the object to this method, or use SuiteQL.runPaged(options), to run the query. Note: If the SuiteAnalytics Connect feature is enabled in your NetSuite account, there is no limit to the number of results this method can return. If the SuiteAnalytics Connect feature is not enabled, this method can return a maximum of 100,000 results across all pages in the result set. For more information about SuiteAnalytics Connect, see the help topic SuiteAnalytics Connect. For more information and examples of using SuiteQL in the N/query module, see SuiteQL in the N/query Module. For more information about SuiteQL in general, see the help topic SuiteQL.
Returns	Promise Object

	 Important: This method returns a maximum of 1000 pages.
Synchronous Version	query.runSuiteQLPaged(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.query	string	required	The string representation of the SuiteQL query to run.
options.params	<string number >	optional	The parameters to use in the SuiteQL query.
options.pageSize	number	optional	The size of each page in the query results. The default value is 50 results per page. The minimum page size is 5 results per page, and the maximum page size is 1000 results per page.
options.customScriptId	string	optional	A unique identifier used for potential performance issues in a query. If your query produces performance issues, the custom script ID identifies where the update will need to occur. This ID can also be used as a precaution to speed up performance fixes, if necessary.  Note: The Script ID must be unique or the performance enhancements will affect each query with the same customScriptID.

Errors

Error Code	Thrown If
MISSING_REQD_ARGUMENT	The parameter is missing.

Error Code	Thrown If
SSS_INVALID_TYPE_ARG	Types other than string, number, or are included in the options.params.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 const results = await query.runSuiteQLPaged.promise({
5   query: 'SELECT customer.entityid, customer.email FROM customer',
6   params: [true],
7   pageSize: 5,
8   customScriptId: 'myCustomScriptId'
9 })
10
11 ...
12 // Add additional code

```

query.Aggregate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for aggregate functions supported with the N/query Module. An aggregate function performs a calculation on the column or condition values and returns a single value. Each value in this enum (except MEDIAN) has two variants: distinct (using the <code>_DISTINCT</code> suffix) and nondistinct (using no suffix). The variant determines whether the aggregate function operates on all instances of duplicate values or on a single instance of the value. For example, consider a situation in which the MAXIMUM aggregate function is used to determine the maximum of a set of values. When using the distinct variant (MAXIMUM_DISTINCT), the aggregate function considers each instance of duplicate values. So if the set of values includes three distinct values that are all equal and all represent the maximum value in the set, the aggregate function lists all three instances. When using the nondistinct variant (MAXIMUM), only one instance of the maximum value is listed, regardless of the number of instances of that maximum value in the set. This enum is used to pass the aggregate function argument to Component.createColumn(options), Component.createCondition(options), Query.createColumn(options), and Query.createCondition(options).
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.1

Values

Value	Description
AVERAGE	Calculates the average value.
AVERAGE_DISTINCT	Calculates the average distinct value.
COUNT	Counts the number of results.
COUNT_DISTINCT	Counts the number of distinct results.
MAXIMUM	Determines the maximum value. If the values are dates, the most recent date is determined.
MAXIMUM_DISTINCT	Determines the maximum distinct value. If the values are dates, the most recent date is determined.
MEDIAN	Calculates the median value.
MINIMUM	Determines the minimum value. If the values are dates, the earliest date is determined.
MINIMUM_DISTINCT	Determines the minimum distinct value. If the values are dates, the earliest date is determined.
SUM	Adds all values.
SUM_DISTINCT	Adds all distinct values.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myTransactionQuery = query.create({
4         type: query.Type.TRANSACTION
5     });
6     var myAggColumn = myTransactionQuery.createColumn({
7         fieldId: 'amount',
8         aggregate: query.Aggregate.AVERAGE
9     });
10    myTransactionQuery.columns = [myAggColumn];
11    ...
12 // Add additional code

```

query.DateId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported date codes in relative dates.
------------------	---

	This enum is used to pass the date ID argument to query.createRelativeDate(options). It is also used as the value of the RelativeDate.dateId property. When query.createRelativeDate(options) is called, the enum value that you specify is set as the value of the RelativeDate.dateId property. When creating a relative date using query.createRelativeDate(options), use the values in this enum to specify a date relative to the current date. For example, to create a relative date that represents the date a certain number of days before the current date, use the DateId.DAYS_AGO enum value. To create a relative date that represents the date a certain number of months after the current date, use the DateId.MONTHS_FROM_NOW enum value. The values in this enum might look similar to the values in the query.RelativeDateRange enum, but each enum is used for a different purpose: ■ Use query.DateId enum values to create a query.RelativeDate object using query.createRelativeDate(options). After you create this object, you can use it in query conditions that you create using Query.createCondition(options) or Component.createCondition(options). ■ Use query.RelativeDateRange enum values directly in query conditions that you create using Query.createCondition(options) or Component.createCondition(options). Each value in the query.RelativeDateRange enum represents a date range, and you can use these values in the values parameter of Query.createCondition(options) or Component.createCondition(options).
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2019.1

Values

Value	Sets RelativeDate.dateId Property To
DAYS_AGO	dago
DAYS_FROM_NOW	dfn
HOURS_AGO	hago
HOURS_FROM_NOW	hfn
MINUTES_AGO	nago
MINUTES_FROM_NOW	nfn
MONTHS_AGO	mago
MONTHS_FROM_NOW	mfn
QUARTERS_AGO	qago
QUARTERS_FROM_NOW	qfn
SECONDS_AGO	sago
SECONDS_FROM_NOW	sfn
WEEKS_AGO	wago
WEEKS_FROM_NOW	wfn
YEARS_AGO	yago

Value	Sets RelativeDate.dateId Property To
YEARS_FROM_NOW	yfn

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myRelativeDate = query.createRelativeDate({
4         dateId: query.DateId.DAYS_AGO, value: 2
5     });
6 ...
7 // Add additional code

```

query.FieldContext

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the field context to use when creating a column using Query.createColumn(options) or Component.createColumn(options). The field context determines how field values are displayed in a column. For example, you can specify that a column should display raw data (such as internal IDs), consolidated or converted amounts (such as currency totals), or user-friendly values (such as names).
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2019.1

Values

Value	Description
CONVERTED	Displays converted currency amounts using the exchange rate that was in effect on a specific date.
CURRENCY_CONSOLIDATED	Displays consolidated currency amounts in the base currency.
DISPLAY	Displays user-friendly field values. For example, for the entity field on Transaction records, using the DISPLAY enum value displays the name of the entity instead of its ID.
HIERARCHY	Displays user-friendly field values for hierarchical fields (for example, "Parent Company : SUB CAD"). This value is similar to the DISPLAY enum value but applies to hierarchical fields.

Value	Description
HIERARCHY_IDENTIFIER	Displays raw field values for hierarchical fields (for example, "1 : 5"). This value is similar to the RAW enum value but applies to hierarchical fields.
RAW	Displays raw field values. For example, for the entity field on Transaction records, using the RAW enum value displays the ID of the entity.
SIGN_CONSOLIDATED	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 var myContextColumn = myTransactionQuery.createColumn({
7     fieldId: 'netamount',
8     context: query.FieldContext.CURRENCY_CONSOLIDATED
9 });
10 ...
11 // Add additional code

```

query.Operator

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for operators supported in SuiteScript Analytic APIs. SuiteScript Analytic APIs help you work with analytical data in NetSuite using SuiteScript. For more information, see the help topic SuiteScript 2.x Analytic APIs. This enum is used to specify the operator argument to Query.createCondition(options) and Component.createCondition(options). This enum is also used to specify the operator argument to methods in the N/dataset and N/workbook modules, such as dataset.createCondition(options) and workbook.createTableColumnFilter(options). For more information, see the help topic Workbook API. Values for this enum are listed in the Values section. The following columns provide information about each operator: ■ Value — Use these values to specify operators in most queries (for example, <code>query.Operator.BEFORE</code>). To use these values, you must include the N/query module in your script. ■ Mapped String Value — Use these values as strings that represent the corresponding operators (for example, 'BEFORE'). To use these values, you do not need to include the N/query module in your script. ■ Use For — Use this column to determine the value types that each operator supports. For example, the <code>query.Operator.AFTER</code> operator is designed to be used with date/time values,
-------------------------	---

	and you cannot use this operator with string or values. Some operators are similar but are designed to be used with different value types (such as <code>query.Operator.IS</code> for values and <code>query.Operator.EQUAL</code> for numeric values). For multi-select fields, you can use the <code>query.Operator.INCLUDE_*</code> and <code>query.Operator.EXCLUDE_*</code> values to specify a set of exact field values. For example, to obtain script deployment records that apply to all contexts except the <code>WEBAPP</code> and <code>WEBSTORE</code> contexts, you can use the <code>query.Operator.EXCLUDE_ALL</code> operator. You cannot use the <code>query.Operator.ANY_OF_NOT</code> operator to obtain these records, because this operator uses an implicit OR operator between all specified values. The <code>query.Operator.EXCLUDE_ALL</code> operator uses an implicit AND operator between all specified values, which lets you create more complex conditions.
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.1

Values

Value	Mapped String Value	Use For
AFTER	AFTER	Date/time values
AFTER_NOT	AFTER_NOT	Date/time values
ANY_OF	ANY_OF	Single-select fields Multi-select fields
ANY_OF_NOT	ANY_OF_NOT	Single-select fields Multi-select fields
BEFORE	BEFORE	Date/time values
BEFORE_NOT	BEFORE_NOT	Date/time values
BETWEEN	BETWEEN	Numbers
BETWEEN_NOT	BETWEEN_NOT	Numbers
CONTAIN	CONTAIN	Strings
CONTAIN_NOT	CONTAIN_NOT	Strings
EMPTY	EMPTY	Single-select fields Multi-select fields
EMPTY_NOT	EMPTY_NOT	Single-select fields Multi-select fields
ENDWITH	ENDWITH	Strings
ENDWITH_NOT	ENDWITH_NOT	Strings
EQUAL	EQUAL	Numbers
EQUAL_NOT	EQUAL_NOT	Numbers
EXCLUDE_ALL	MN_EXCLUDE	Multi-select fields

Value	Mapped String Value	Use For
EXCLUDE_ANY	MN_EXCLUDE_ALL	Multi-select fields
EXCLUDE_EXACTLY	MN_EXCLUDE_EXACTLY	Multi-select fields
GREATER	GREATER	Numbers
GREATER_NOT	GREATER_NOT	Numbers
GREATER_OR_EQUAL	GREATER_OR_EQUAL	Numbers
GREATER_OR_EQUAL_NOT	GREATER_OR_EQUAL_NOT	Numbers
INCLUDE_ALL	MN_INCLUDE_ALL	Multi-select fields
INCLUDE_ANY	MN_INCLUDE	Multi-select fields
INCLUDE_EXACTLY	MN_INCLUDE_EXACTLY	Multi-select fields
IS	IS	values
IS_NOT	IS_NOT	values
LESS	LESS	Numbers
LESS_NOT	LESS_NOT	Numbers
LESS_OR_EQUAL	LESS_OR_EQUAL	Numbers
LESS_OR_EQUAL_NOT	LESS_OR_EQUAL_NOT	Numbers
ON	ON	Date/time values
ON_NOT	ON_NOT	Date/time values
ON_OR_AFTER	ON_OR_AFTER	Date/time values
ON_OR_AFTER_NOT	ON_OR_AFTER_NOT	Date/time values
ON_OR_BEFORE	ON_OR_BEFORE	Date/time values
ON_OR_BEFORE_NOT	ON_OR_BEFORE_NOT	Date/time values
START_WITH	START_WITH	Strings
START_WITH_NOT	START_WITH_NOT	Strings
WITHIN	WITHIN	Date/time values
WITHIN_NOT	WITHIN_NOT	Date/time values

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4     type: query.Type.CUSTOMER
5 });
6 var mySalesRepJoin = myCustomerQuery.autoJoin({

```

```

7      fieldId: 'salesrep'
8    });
9    var firstCondition = myCustomerQuery.createCondition({
10     fieldId: 'id',
11     operator: query.Operator.EQUAL,
12     values: 107
13   });
14   var secondCondition = myCustomerQuery.createCondition({
15     fieldId: 'id',
16     operator: query.Operator.EQUAL,
17     values: 2647
18   });
19   var thirdCondition = mySalesRepJoin.createCondition({
20     fieldId: 'email',
21     operator: query.Operator.START_WITH_NOT,
22     values: 'foo'
23   });
24   myCustomerQuery.condition = myCustomerQuery.and(thirdCondition, myCustomerQuery.not( myCustomerQuery.or(firstCondition, second
Condition)));
25   var resultSet = myCustomerQuery.run();
26   ...
27   // Add additional code

```

query.PeriodAdjustment

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for adjustment types for a period. This enum is used to pass the adjustment argument to query.createPeriod(options) .
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Values

Value
ALL
NOT_LAST

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myPeriod = query.createPeriod({
4   code: query.PeriodCode.LAST_PERIOD,
5   adjustment: query.PeriodAdjustment.ALL,

```

```

6      type: query.PeriodType.END
7    });
8    ...
9    // Add additional code

```

query.PeriodCode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for period codes for a period. This enum is used to pass the code argument to query.createPeriod(options) .
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2020.1

Values

Value	Sets Period.code Property To
FIRST_FISCAL_QUARTER_LAST_FY	Q1LFY
FIRST_FISCAL_QUARTER_THIS_FY	Q1TFY
FISCAL_QUARTER_BEFORE_LAST	QBL
FISCAL_YEAR_BEFORE_LAST	FYBL
FOURTH_FISCAL_QUARTER_LAST_FY	Q4LFY
FOURTH_FISCAL_QUARTER_THIS_FY	Q4TFY
LAST_FISCAL_QUARTER	LQ
LAST_FISCAL_QUARTER_ONE_FISCAL_YEAR_AGO	LQOLFY
LAST_FISCAL_QUARTER_TO_PERIOD	LFQTP
LAST_FISCAL_YEAR	LFY
LAST_FISCAL_YEAR_TO_PERIOD	LFYTP
LAST_PERIOD	LP
LAST_PERIOD_ONE_FISCAL_QUARTER_AGO	LPOLQ
LAST_PERIOD_ONE_FISCAL_YEAR_AGO	LPOLFY
LAST_ROLLING_18_PERIODS	LR18FP
LAST_ROLLING_6_FISCAL_QUARTERS	LR6FQ
PERIOD_BEFORE_LAST	PBL

Value	Sets Period.code Property To
SAME_FISCAL_QUARTER_LAST_FY	TQQLFY
SAME_FISCAL_QUARTER_LAST_FY_TO_PERIOD	TFQQLFYTP
SAME_PERIOD_LAST_FY	TPQLFY
SAME_PERIOD_LAST_FISCAL_QUARTER	TPQLQ
SECOND_FISCAL_QUARTER_LAST_FY	Q2LFY
SECOND_FISCAL_QUARTER_THIS_FY	Q2TFY
THIRD_FISCAL_QUARTER_LAST_FY	Q3LFY
THIRD_FISCAL_QUARTER_THIS_FY	Q3TFY
THIS_FISCAL_QUARTER	TQ
THIS_FISCAL_QUARTER_TO_PERIOD	TFQTP
THIS_FISCAL_YEAR	TFY
THIS_FISCAL_YEAR_TO_PERIOD	TFYTP
THIS_PERIOD	TP

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var myPeriod = query.createPeriod({
4   code: query.PeriodCode.LAST_PERIOD,
5   adjustment: query.PeriodAdjustment.ALL,
6   type: query.PeriodType.END
7 });
8 ...
9 // Add additional code
```

query.PeriodType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for period types for a period. This enum is used to pass the type argument to query.createPeriod(options) .
Module	N/query Module
Sibling Module Members	N/query Module Members

Since	2020.1
-------	--------

Values

Value
END
START

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
2 ...
3     var myPeriod = query.createPeriod({
4         code: query.PeriodCode.LAST_PERIOD,
5         adjustment: query.PeriodAdjustment.ALL,
6         type: query.PeriodType.END
7     });
8 ...
9 // Add additional code
```

query.RelativeDateRange

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds query.RelativeDate object values for supported date ranges in relative dates. This enum is used to pass the values argument to Query.createCondition(options) and Component.createCondition(options). It is also used as the value of the RelativeDate.value property. Each value in this enum represents a date range. When Query.createCondition(options) or Component.createCondition(options) is called with a query.RelativeDate object as the values argument, this object is set as the value of the RelativeDate.value property. When creating a condition using Query.createCondition(options) or Component.createCondition(options), use the values in this enum (along with values in the query.Operator enum) to specify a range of dates relative to the current date. For example, to create a condition to match dates that occur before the current date, use the query.RelativeDateRange.TODAY enum value and the query.Operator.BEFORE enum value. To create a condition to match dates that occur after last year, use the query.RelativeDateRange.LAST_YEAR enum value and the query.Operator.AFTER enum value. For more information about relative dates, see Relative Dates in the N/query Module. The values in this enum might look similar to the values in the query.DateId enum, but each enum is used for a different purpose: ■ Use query.DateId enum values to create a query.RelativeDate object using query.createRelativeDate(options). After you create this object, you can use it in query conditions that you create using Query.createCondition(options) or Component.createCondition(options).
------------------	---

	Use <code>query.RelativeDateRange</code> enum values directly in query conditions that you create using Query.createCondition(options) or Component.createCondition(options). Each value in the <code>query.RelativeDateRange</code> enum represents a date range, and you can use these values in the <code>values</code> parameter of Query.createCondition(options) or Component.createCondition(options).
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2019.1

Values

Value	RelativeDate.dateId Property
FISCAL_HALF_BEFORE_LAST	FHBL
FISCAL_HALF_BEFORE_LAST_TO_DATE	FHBLTD
FISCAL_QUARTER_BEFORE_LAST	FQBL
FISCAL_QUARTER_BEFORE_LAST_TO_DATE	FQBLTD
FISCAL_YEAR_BEFORE_LAST	FYBL
FISCAL_YEAR_BEFORE_LAST_TO_DATE	FYBLTD
FIVE_DAYS_AGO	DAGO5
FIVE_DAYS_FROM_NOW	DFN5
FOUR_DAYS_AGO	DAGO4
FOUR_DAYS_FROM_NOW	DFN4
FOUR_WEEKS_STARTING_THIS_WEEK	TWN3W
LAST_BUSINESS_WEEK	LBW
LAST_FISCAL_HALF	LFH
LAST_FISCAL_HALF_ONE_FISCAL_YEAR_AGO	LFHLFY
LAST_FISCAL_HALF_TO_DATE	LFHTD
LAST_FISCAL_QUARTER	LFQ
LAST_FISCAL_QUARTER_ONE_FISCAL_YEAR_AGO	LFQLFY
LAST_FISCAL_QUARTER_TO_DATE	LFQTD
LAST_FISCAL_QUARTER_TWO_FISCAL_YEARS_AGO	LFQFYBL
LAST_FISCAL_YEAR	LFY
LAST_FISCAL_YEAR_TO_DATE	LFYTD
LAST_MONTH	LM
LAST_MONTH_ONE_FISCAL_QUARTER_AGO	LMLFQ
LAST_MONTH_ONE_FISCAL_YEAR_AGO	LMLFY

Value	RelativeDate.dateId Property
LAST_MONTH_TO_DATE	LMTD
LAST_MONTH_TWO_FISCAL_QUARTERS_AGO	LMFQBL
LAST_MONTH_TWO_FISCAL_YEARS_AGO	LMFYBL
LAST_ROLLING_HALF	LRH
LAST_ROLLING_QUARTER	LRQ
LAST_ROLLING_YEAR	LRY
LAST_WEEK	LW
LAST_WEEK_TO_DATE	LWTD
LAST_YEAR	LY
LAST_YEAR_TO_DATE	LYTD
MONTH_AFTER_NEXT	MAN
MONTH_AFTER_NEXT_TO_DATE	MANTD
MONTH_BEFORE_LAST	MBL
MONTH_BEFORE_LAST_TO_DATE	MBLTD
NEXT_BUSINESS_WEEK	NBW
NEXT_FISCAL_HALF	NFH
NEXT_FISCAL_QUARTER	NFQ
NEXT_FISCAL_YEAR	NFY
NEXT_FOUR_WEEKS	N4W
NEXT_MONTH	NM
NEXT_ONE_HALF	NOH
NEXT_ONE_MONTH	NOM
NEXT_ONE_QUARTER	NOQ
NEXT_ONE_WEEK	NOW
NEXT_ONE_YEAR	NOY
NEXT_WEEK	NW
NINETY_DAYS_AGO	DAGO90
NINETY_DAYS_FROM_NOW	DFN90
ONE_YEAR_BEFORE_LAST	OYBL
PREVIOUS_FISCAL_QUARTERS_LAST_FISCAL_YEAR	PQLFY
PREVIOUS_FISCAL_QUARTERS_THIS_FISCAL_YEAR	PQTFY
PREVIOUS_MONTHS_LAST_FISCAL_HALF	PMLFH

Value	RelativeDate.dateId Property
PREVIOUS_MONTHS_LAST_FISCAL_QUARTER	PMLFQ
PREVIOUS_MONTHS_LAST_FISCAL_YEAR	PMLFY
PREVIOUS_MONTHS_SAME_FISCAL_HALF_LAST_FISCAL_YEAR	PMSFHLFY
PREVIOUS_MONTHS_SAME_FISCAL_QUARTER_LAST_FISCAL_YEAR	PMSFQLFY
PREVIOUS_MONTHS_THIS_FISCAL_HALF	PMTFH
PREVIOUS_MONTHS_THIS_FISCAL_QUARTER	PMTFQ
PREVIOUS_MONTHS_THIS_FISCAL_YEAR	PMTFY
PREVIOUS_ONE_DAY	OD
PREVIOUS_ONE_HALF	OH
PREVIOUS_ONE_MONTH	OM
PREVIOUS_ONE_QUARTER	OQ
PREVIOUS_ONE_WEEK	OW
PREVIOUS_ONE_YEAR	OY
PREVIOUS_ROLLING_HALF	PRH
PREVIOUS_ROLLING_QUARTER	PRQ
PREVIOUS_ROLLING_YEAR	PRY
SAME_DAY_FISCAL_QUARTER_BEFORE_LAST	SDFQBL
SAME_DAY_FISCAL_YEAR_BEFORE_LAST	SDFYBL
SAME_DAY_LAST_FISCAL_QUARTER	SDLFQ
SAME_DAY_LAST_FISCAL_YEAR	SDLFY
SAME_DAY_LAST_MONTH	SDLM
SAME_DAY_LAST_WEEK	SDLW
SAME_DAY_MONTH_BEFORE_LAST	SDBL
SAME_DAY_WEEK_BEFORE_LAST	SDWBL
SAME_FISCAL_HALF_LAST_FISCAL_YEAR	SFHLFY
SAME_FISCAL_HALF_LAST_FISCAL_YEAR_TO_DATE	SFHLFYTD
SAME_FISCAL_QUARTER_FISCAL_YEAR_BEFORE_LAST	SFQFYBL
SAME_FISCAL_QUARTER_LAST_FISCAL_YEAR	SFQLFY
SAME_FISCAL_QUARTER_LAST_FISCAL_YEAR_TO_DATE	SFQLFYTD
SAME_MONTH_FISCAL_QUARTER_BEFORE_LAST	SMFQBL
SAME_MONTH_FISCAL_YEAR_BEFORE_LAST	SMFYBL
SAME_MONTH_LAST_FISCAL_QUARTER	SMLFQ

Value	RelativeDate.dateId Property
SAME_MONTH_LAST_FISCAL_QUARTER_TO_DATE	SMLFQTD
SAME_MONTH_LAST_FISCAL_YEAR	SMLFY
SAME_MONTH_LAST_FISCAL_YEAR_TO_DATE	SMLFYTD
SAME_WEEK_FISCAL_YEAR_BEFORE_LAST	SWFYBL
SAME_WEEK_LAST_FISCAL_YEAR	SWLFY
SIXTY_DAYS_AGO	DAGO60
SIXTY_DAYS_FROM_NOW	DFN60
TEN_DAYS_AGO	DAGO10
TEN_DAYS_FROM_NOW	DFN10
THIRTY_DAYS_AGO	DAGO30
THIRTY_DAYS_FROM_NOW	DFN30
THIS_BUSINESS_WEEK	TBW
THIS_FISCAL_HALF	TFH
THIS_FISCAL_HALF_TO_DATE	TFHTD
THIS_FISCAL_QUARTER	TFQ
THIS_FISCAL_QUARTER_TO_DATE	TFQTD
THIS_FISCAL_YEAR	TFY
THIS_FISCAL_YEAR_TO_DATE	TFYTD
THIS_MONTH	TM
THIS_MONTH_TO_DATE	TMTD
THIS_ROLLING_HALF	TRH
THIS_ROLLING_QUARTER	TRQ
THIS_ROLLING_YEAR	TRY
THIS_WEEK	TW
THIS_WEEK_TO_DATE	TWTD
THIS_YEAR	TY
THIS_YEAR_TO_DATE	TYTD
THREE_DAYS_AGO	DAGO3
THREE_DAYS_FROM_NOW	DFN3
THREE_FISCAL_QUARTERS_AGO	FQB
THREE_FISCAL_QUARTERS_AGO_TO_DATE	FQBTD
THREE_FISCAL_YEARS_AGO	FYB

Value	RelativeDate.dateId Property
THREE_FISCAL_YEARS_AGO_TO_DATE	FYBTD
THREE_MONTHS_AGO	MB
THREE_MONTHS_AGO_TO_DATE	MBTD
TODAY	TODAY
TODAY_TO_END_OF_THIS_MONTH	TODAYTTM
TOMORROW	TOMORROW
TWO_DAYS_AGO	DAGO2
TWO_DAYS_FROM_NOW	DFN2
WEEK_AFTER_NEXT	WAN
WEEK_AFTER_NEXT_TO_DATE	WANTD
WEEK_BEFORE_LAST	WBL
WEEK_BEFORE_LAST_TO_DATE	WBLTD
YESTERDAY	YESTERDAY

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myTransactionQuery = query.create({
4     type: query.Type.TRANSACTION
5 });
6 myTransactionQuery.condition = myTransactionQuery.createCondition({
7     fieldId: 'trandate',
8     operator: query.Operator.BEFORE,
9     values: query.RelativeDateRange.TODAY
10 });
11 ...
12 // Add additional code

```

query.ReturnType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the formula return types supported with the N/query Module. This enum is used to pass the formula return type argument to Query.createColumn(options), Component.createColumn(options), Query.createCondition(options), and Component.createCondition(options).
-------------------------	---

	For more information about these return types, see the help topic Formula Fields . For more information about formulas, see the help topics SuiteAnalytics Workbook Overview , SQL Expressions , and Search Formula Examples and Tips .
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.1

Values

Value
ANY
BOOLEAN
CLOBTEXT
CURRENCY
DATE
DATETIME
DURATION
FLOAT
INTEGER
KEY
PERCENT
RELATIONSHIP
STRING
UNKNOWN

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3     var myTransactionQuery = query.create({
4         type: query.Type.TRANSACTION
5     });
6     var myFormulaColumn = myTransactionQuery.createColumn({
7         type: query.ReturnType.CURRENCY,
8         formula: '{amount} * 125'
9     });
10 ...
11 // Add additional code

```

query.SortLocale

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for sort locales supported with the N/query Module . This enum is used to pass the locale argument to Query.createSort(options) and Component.createSort(options) .
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.2

Values

i Note: The following table lists all possible locale values available. You must enable the specified locale in your company settings to avoid potential scripting errors.

Value	Value
ARABIC	ID_ID
ARABIC_ABJ_MATCH	INDONESIAN
ARABIC_ABJ_MATCH_CI	INDONESIAN_AI
ARABIC_ABJ_SORT	INDONESIAN_CI
ARABIC_ABJ_SORT_CI	ITALIAN
ARABIC_CI	ITALIAN_AI
ARABIC_MATCH	ITALIAN_CI
ARABIC_MATCH_CI	IT_IT
ASCII7	JAPANESE_M
ASCII7_CI	JA_JP
AZERBAIJANI	KOREAN_M
AZERBAIJANI_CI	KO_KR
BENGALI	LATIN
BENGALI_CI	LATIN_AI
BIG5	LATIN_CI
BIG5_CI	LATVIAN
BINARY	LATVIAN_AI
BINARY_CI	LATVIAN_CI
BULGARIAN	LITHUANIAN
BULGARIAN_CI	LITHUANIAN_AI
CANADIAN_M	LITHUANIAN_CI
CATALAN	MALAY
CATALAN_CI	MALAY_AI

Value	Value
■ CROATIAN	■ MALAY_CI
■ CROATIAN_CI	■ NL_NL
■ CS_CZ	■ NO_NO
■ CZECH	■ NORWEGIAN
■ CZECH_CI	■ NORWEGIAN_AI
■ CZECH_PUNCTUATION	■ NORWEGIAN_CI
■ CZECH_PUNCTUATION_CI	■ POLISH
■ DANISH	■ POLISH_AI
■ DANISH_CI	■ POLISH_CI
■ DANISH_M	■ PT_BR
■ DA_DK	■ PUNCTUATION
■ DE_DE	■ PUNCTUATION_AI
■ DUTCH	■ PUNCTUATION_CI
■ DUTCH_CI	■ ROMANIAN
■ EBCDIC	■ ROMANIAN_AI
■ EBCDIC_CI	■ ROMANIAN_CI
■ EEC_EURO	■ RUSSIAN
■ EEC_EUROPA3	■ RUSSIAN_AI
■ EEC_EUROPA3_CI	■ RUSSIAN_CI
■ EEC_EURO_CI	■ RU_RU
■ EN	■ SCHINESE_PINYIN_M
■ EN_AU	■ SCHINESE_RADICAL_M
■ EN_CA	■ SCHINESE_STROKE_M
■ EN_GB	■ SLOVAK
■ EN_US	■ SLOVAK_AI
■ ESTONIAN	■ SLOVAK_CI
■ ESTONIAN_CI	■ SLOVENIAN
■ ES_AR	■ SLOVENIAN_AI
■ ES_ES	■ SLOVENIAN_CI
■ FINNISH	■ SPANISH
■ FINNISH_CI	■ SPANISH_AI
■ FI_FI	■ SPANISH_CI
■ FRENCH	■ SPANISH_M
■ FRENCH_AI	■ SV_SE
■ FRENCH_CI	■ SWEDISH
■ FRENCH_M	■ SWEDISH_AI
■ FR_CA	■ SWEDISH_CI
■ FR_FR	■ SWISS
■ GBK	■ SWISS_AI
■ GBK_AI	■ SWISS_CI
■ GBK_CI	■ TCHINESE_RADICAL_M
■ GENERIC_M	■ TCHINESE_STROKE_M
■ GERMAN	■ THAI_M
■ GERMAN_AI	■ TH_TH

Value	Value
■ GERMAN_CI	■ TR_TR
■ GERMAN_DIN	■ TURKISH
■ GERMAN_DIN_AI	■ TURKISH_AI
■ GERMAN_DIN_CI	■ TURKISH_CI
■ GREEK	■ UKRAINIAN
■ GREEK_AI	■ UKRAINIAN_AI
■ GREEK_CI	■ UKRAINIAN_CI
■ HEBREW	■ UNICODE_BINARY
■ HEBREW_AI	■ UNICODE_BINARY_AI
■ HEBREW_CI	■ UNICODE_BINARY_CI
■ HE_IL	■ VIETNAMESE
■ HKSCS	■ VIETNAMESE_AI
■ HKSCS_AI	■ VIETNAMESE_CI
■ HKSCS_CI	■ VI_VN
■ HUNGARIAN	■ WEST_EUROPEAN
■ HUNGARIAN_AI	■ WEST_EUROPEAN_AI
■ HUNGARIAN_CI	■ WEST_EUROPEAN_CI
■ ICELANDIC	■ ZH_CN
■ ICELANDIC_AI	■ ZH_TW
■ ICELANDIC_CI	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myCustomerQuery = query.create({
4   type: query.Type.CUSTOMER
5 });
6 myCustomerQuery.columns = [myCustomerQuery.createColumn({
7   fieldId: 'entityid'
8 });
9 myCustomerQuery.sort = [myCustomerQuery.createSort({
10  column: myCustomerQuery.columns[0], locale: query.SortLocale.EN_CA
11 });
12 ...
13 // Add additional code

```

query.Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for query types used in the query definition. This enum is used to pass the initial query type argument to query.create(options) .  Important: The N/query module supports the same record types that are supported by the SuiteAnalytics Workbook interface. For more information, see the help topic Available Record Types.
Module	N/query Module
Sibling Module Members	N/query Module Members
Since	2018.1

Values

Note: Before using these values, consider the following:

- A query type is not the same as a record type. The supported query types listed below do not necessarily correspond with the supported record types listed in the [N/record Module](#).
- Depending on your account, role, and enabled features, some of these values may not be available.
- Custom record types are not included in this enum.
- You must configure your account settings according to the query type used in your scripts to avoid potential errors.

Value	Sets Query.type Property To
ACCOUNT	account
ACCOUNTING_BOOK	accountingbook
ACCOUNTING_CONTEXT	accountingcontext
ACCOUNTING_PERIOD	accountingperiod
ACTIVITY	activity
ADDRESS_BOOK	addressbook
ADVANCED_NUMBERING_LOG	advancednumberinglog

Value	Sets Query.type Property To
ADVANCED_PDF_TEMPLATE	advancedpdftemplate
ALL_PARSER_PLUGIN	allparserplugin
ALLOCATION_METHOD	allocationmethod
AMORTIZATION_SCHEDULE	amortizationschedule
AMORTIZATION_TEMPLATE	amortizationtemplate
AUTHORIZATION_CONSENT	authorizationconsent
AUTOMATED_CLEARING_HOUSE	automatedclearinghouse
BALANCING_SEGMENTS_PREFERENCE	balancingsegmentspreference
BILLING_CLASS	billingclass
BILLING_RATE_CARD	billingratecard
BILLING_SCHEDULE	billingschedule
BILL_OF_DISTRIBUTION	billofdistribution
BILL_RUN	billrun
BILL_RUN_SCHEDULE	billrunschedule
BIN	bin
BOM	bom
BOM_REVISION	bomrevision
BOM_REVISION_COMPONENT	bomrevisioncomponent
BUDGET_EXCHANGE_RATE	budgetexchangerate
BUDGET_LEGACY	budgetlegacy
BUDGETCATEGORY	budgetcategory
BUDGETIMPORT	budgetimport
BUDGETS	budgets
BULK_PROC_SUBMISSION	bulkprocsubmission
BUNDLE_INSTALLATION_SCRIPT	bundleinstallationscript
BUNDLE_INSTALLATION_SCRIPT_DEPLOYMENT	bundleinstallationscriptdeployment
BUSINESS_EVENTS_PROCESSING_HISTORY	businesseventsprocessinghistory
CALENDAR_EVENT	calendarevent
CAMPAIGN_AUDIENCE	campaignaudience
CAMPAIGN_CATEGORY	campaigncategory
CAMPAIGN_CHANNEL	campaignchannel
CAMPAIGN_EMAIL_ADDRESS	campaignemailaddress
CAMPAIGN_EVENT	campaignevent

Value	Sets Query.type Property To
CAMPAIGN_FAMILY	campaignfamily
CAMPAIGN_OFFER	campaignoffer
CAMPAIGN_RESPONSE	campaignresponse
CAMPAIGN_SEARCH_ENGINE	campaignsearchengine
CAMPAIGN_SUBSCRIPTION	campaignsubscription
CAMPAIGN_TEMPLATE	campaigntemplate
CAMPAIGN_VERTICAL	campaignvertical
CARDHOLDER_AUTHENTICATION	cardholderauthentication
CARDHOLDER_AUTHENTICATION_EVENT	cardholderauthenticationevent
CATEGORY1099MISC	category1099misc
CHARGE	charge
CHARGE_RULE	chargerule
CHARGE_RUN	chargerun
CHARGE_TYPE	chargetype
CLASSIFICATION	classification
CLIENT_SCRIPT	clientscript
CLIENT_SCRIPT_DEPLOYMENT	clientscriptdeployment
COMPANY_FEATURE_SETUP	companyfeaturesetup
COMPETITOR	competitor
CONSOLIDATED_EXCHANGE_RATE	consolidatedexchangerat
CONSOLIDATED_RATE_ADJUSTOR_PLUGIN	consolidatedrateadjustorplugin
CONTACT	contact
CONTACT_CATEGORY	contactcategory
CONTACT_ROLE	contactrole
CONTACT_SUBSIDIARY_RELATIONSHIP	contactsubsidiaryrelationship
COST_CATEGORY	costcategory
COUPON_CODE	couponcode
CURRENCY	currency
CURRENCY_RATE	currencyrate
CURRENCY_RATE_TYPE	currencyratetype
CUSTOM_SEGMENT_FIELD	customsegmentfield
CUSTOMER	customer
CUSTOMER_CATEGORY	customercategory

Value	Sets Query.type Property To
CUSTOMER_MESSAGE	customermessage
CUSTOMER_SEGMENT	customersegment
CUSTOMER_SUBSIDIARY_RELATIONSHIP	customersubsiaryrelationship
CUSTOM_FIELD	customfield
CUSTOM_FIELD_2	customfield2
CUSTOM_GL_PLUGIN	customglplugin
CUSTOM_LIST	customlist
CUSTOM_RECORD_ACTION_SCRIPT	customrecordactionscript
CUSTOM_RECORD_TYPE	customrecordtype
CUSTOM_SEGMENT	customsegment
CUSTOM_TRANSACTION_TYPE	customtransactiontype
DATASET_BUILDER_PLUGIN	datasetbuilderplugin
DELETED_RECORD	deletedrecord
DELETED_RECORD_IN_CONNECT	deletedrecordinconnect
DEPARTMENT	department
DEVICE_ID	deviceid
DISTRIBUTION_CATEGORY	distributioncategory
DISTRIBUTION_NETWORK	distributionnetwork
DOMAIN	domain
DUAL	dual
EMAIL_CAPTURE_PLUGIN	emailcaptureplugin
EMAIL_TEMPLATE	emailtemplate
EMPLOYEE	employee
EMPLOYEE_EXPENSE_SOURCE_TYPE	employeeexpensesourcetype
EMPLOYEE_LIST	employeeelist
EMPLOYEE_STATUS	employeeestatus
EMPLOYEE_SUBSIDIARY_RELATIONSHIP	employeesubsiaryrelationship
EMPLOYEE_TYPE	employeeetype
ENTITY	entity
ENTITY_GROUP	entitygroup
ENTITY_SUBSIDIARY_RELATIONSHIP	entitysubsiaryrelationship
EXPENSE_CATEGORY	expensecategory
EXPENSE_REPORT_POLICY	expensereportpolicy

Value	Sets Query.type Property To
FAX_TEMPLATE	faxtemplate
FILE	file
FISCAL_CALENDAR	fiscalcalendar
FI_CONNECTIVITY_PLUGIN	ficonnectivityplugin
FORECAST	forecast
FORMAT_PROFILE	formatprofile
FULFILLMENT_EXCEPTION_REASON	fulfillmentexceptionreason
FULFILLMENT_REQUEST	fulfillmentrequest
F_I_PARSER_PLUGIN	fiparserplugin
GATEWAY_NOTIFICATION	gatewaynotification
GENERAL_ALLOCATION_SCHEDULE	generalallocationschedule
GENERAL_TOKEN	generaltoken
GENERALIZED_ITEM	generalizeditem
GENERIC_RESOURCE	genericresource
GENERIC_RESOURCE_SUBSIDIARY_RELATIONSHIP	genericresourcesubsiaryrelationship
GIFT_CERTIFICATE	giftcertificate
GLOBAL_ACCOUNT_MAPPING	globalaccountmapping
GLOBAL_INVENTORY_RELATIONSHIP	globalinventoryrelationship
GL_LINES_AUDIT_LOG	gllinesauditlog
GL_LINES_PLUGIN_REVISION	gllinespluginrevision
G_L_NUMBERING_SEQUENCE	glnumberingsequence
IMPORTED_EMPLOYEE_EXPENSE	importedemployeeexpense
INBOUND_SHIPMENT	inboundshipment
INCOTERM	incoterm
INVENTORY_COST_TEMPLATE	inventorycosttemplate
INVENTORY_NUMBER	inventorynumber
INVENTORY_STATUS	inventorystatus
INVOICE_GROUP	invoicegroup
INVT_ITEM_PRICE_HISTORY	invitempricehistory
ISSUE	issue
ISSUE_PRIORITY	issuepriority
ISSUE_SEVERITY	issueseverity
ISSUE_STATUS	issuestatus

Value	Sets Query.type Property To
ITEM	item
ITEM_ACCOUNT_MAPPING	itemaccountmapping
ITEM_COLLECTION	itemcollection
ITEM_DEMAND_PLAN	itemdemandplan
ITEM_LOCATION_CONFIGURATION	itemlocationconfiguration
ITEM_PROCESS_FAMILY	itemprocessfamily
ITEM_PROCESS_GROUP	itemprocessgroup
ITEM_SEGMENT_CUSTOMER_SEGMENT_MAP	itemsegmentcustomersegmentmap
ITEM_SEGMENT_INCLUDING_SYNTHETIC	itemsegmentincludingsthetic
ITEM_SUPPLY_PLAN	itemsupplyplan
I_P_RESTRICTIONS	iprestrictions
JOB	job
JOB_RESOURCE_ROLE	jobresourcerole
JOB_STATUS	jobstatus
JOB_TYPE	jobtype
KNOWLEDGE_BASE	knowledgebase
LOCATION	location
LOCATION_COSTING_GROUP	locationcostinggroup
LOGIN_AUDIT	loginaudit
MAIL_TEMPLATE	mailtemplate
MANUFACTURING_COMPONENT	manufacturingcomponent
MANUFACTURING_COST_TEMPLATE	manufacturingcosttemplate
MANUFACTURING_OPERATION_TASK	manufacturingoperationtask
MANUFACTURING_ROUTING	manufacturingrouting
MANUFACTURING_TRANSACTION	manufacturingtransaction
MAP_REDUCE_SCRIPT	mapreducescript
MAP_REDUCE_SCRIPT_DEPLOYMENT	mapreducescriptdeployment
MASS_UPDATE_SCRIPT	massupdatescript
MASS_UPDATE_SCRIPT_DEPLOYMENT	massupdatescriptdeployment
MEDIA_ITEM_FOLDER	mediaitemfolder
MEM_DOC	memdoc
MEM_DOC_TRANSACTION_TEMPLATE	memdoctransactiontemplate
MERCHANDISE_HIERARCHY_LEVEL	merchandisehierarchylevel

Value	Sets Query.type Property To
MERCHANDISE_HIERARCHY_NODE	merchandisehierarchynode
MERCHANDISE_HIERARCHY_VERSION	merchandisehierarchyversion
MESSAGE	message
MFG_PLANNED_TIME	mfgplannedtime
NETTING_STATEMENT	nettingstatement
NEXUS	nexus
NOTE	note
O_AUTH2_CLIENT_CREDENTIALS	oauth2clientcredentials
OCR_IMPORT_JOB	ocrimportjob
OCR_IMPORT_JOB_REVIEW	ocrimportjobreview
OCR_PLUGIN	ocrplugin
ONLINE_CASE_FORM	onlinecaseform
ONLINE_FORM_TEMPLATE	onlineformtemplate
ONLINE_LEAD_FORM	onlineleadform
ORDER_ALLOCATION_STRATEGY	orderallocationstrategy
ORDER_RELEASE_LINE	orderreleaseline
ORDER_RESERVATION	orderreservation
ORDER_TYPE_RECORD	ordertype record
OTHER_NAME	othername
OTHER_NAME_CATEGORY	othernamecategory
OTHER_NAME_SUBSIDIARY_RELATIONSHIP	othernamesubsiaryrelationship
OUTBOUND_REQUEST	outboundrequest
O_AUTH_TOKEN	oauthtoken
PARTNER	partner
PARTNER_SUBSIDIARY_RELATIONSHIP	partnersubsiaryrelationship
PAYCHECK	paycheck
PAYMENT_CARD	paymentcard
PAYMENT_CARD_SEARCH_RECORD	paymentcardsearchrecord
PAYMENT_CARD_TOKEN	paymentcardtoken
PAYMENT_EVENT	paymentevent
PAYMENT_GATEWAY_PLUGIN	paymentgatewayplugin
PAYMENT_INSTRUMENT	paymentinstrument
PAYMENT_METHOD	paymentmethod

Value	Sets Query.type Property To
PAYMENT_PROCESSING_PROFILE	paymentprocessingprofile
PAYMENT_RESULT_PREVIEW	aymentresultpreview
PAYROLL_BATCH	payrollbatch
PAYROLL_ITEM	payrollitem
PAYROLL_ITEM_GROUP	payrollitemgroup
PDF_TEMPLATE	pdftemplate
PERFORMANCE_REVIEW_SCHEDULE_TALENT_DATASET	performancereviewscheduletalentdataset
PHONE_CALL	phonecall
PICK_STRATEGY	pickstrategy
PICK_TASK	picktask
PICK_TASK_INVENTORY_BALANCE	picktaskinventorybalance
PLANNED_ORDER	plannedorder
PLANNED_STANDARD_COST	plannedstandardcost
PLANNING_ITEM_CATEGORY	planningitemcategory
PLANNING_ITEM_GROUP	planningitemgroup
PLANNING_ITEM_GROUP_SOURCE	planningitemgroupsource
PLANNING_RULE_GROUP	planningrulegroup
PLANNING_VIEW	planningview
PLATFORM_EXTENSION_PLUGIN	platformextensionplugin
PLUG_IN_TYPE	plugintype
PLUG_IN_TYPE_IMPL	plugintypeimpl
PORTLET	portlet
PORTLET_DEPLOYMENT	portletdeployment
POSTING_ACCOUNT_ACTIVITY	postingaccountactivity
PREDICTED_RISK_TRAIN_EVAL_HISTORY	predictedrisktrainevalhistory
PRICE_LEVEL	pricelevel
PRICING	pricing
PRICING_GROUP	pricinggroup
PRICING_WITH_CUSTOMERS	pricingwithcustomers
PROJECT_BUDGET	projectbudget
PROJECT_EXPENSE_TYPE	projectexpensetype
PROJECT_FINANCIALS	projectfinancials
PROJECT_IC_CHARGE_REQUEST	projecticchargerequest

Value	Sets Query.type Property To
PROJECT_SUBSIDIARY_RELATIONSHIP	projectsubsidiaryrelationship
PROJECT_TASK	projecttask
PROJECT_TEMPLATE	projecttemplate
PROJECT_TEMPLATE_SUBSIDIARY_RELATIONSHIP	projecttemplatesubsidiaryrelationship
PROMOTIONS_PLUGIN	promotionsplugin
PROMOTION_CODE	promotioncode
PROMPT	prompt
PUBLISHED_SAVED_SEARCH	publishedsavedsearch
QUANTITY_PRICING_SCHEDULE	quantitypricingschedule
QUOTA	quota
RECENT_RECORD	recentrecord
RECORD_ACTION_SCRIPT_DEPLOYMENT	recordactionscriptdeployment
REC_SYS_ALGORITHM	recsysalgorithm
REC_SYS_ANALYTICS_REPORT	recsysanalyticsreport
REC_SYS_ANALYTICS_REPORT_AGG	recsysanalyticsreportagg
REC_SYS_BLOCKLIST	recsysblocklist
REC_SYS_CONVERSION	recsysconversion
REC_SYS_ELIGIBILITY	recsyseligibility
REC_SYS_SCENARIO	recsysscenario
REDIRECT	redirect
RESOURCE_ALLOCATION	resourceallocation
RESOURCE_GROUP	resourcegroup
RESTLET	restlet
RESTLET_DEPLOYMENT	restletdeployment
RETIREMENT_PLAN	retirementplan
REV_REC_SCHEDULE	revrecschedule
REV_REC_TEMPLATE	revrectemplate
REVENUE_ELEMENT	revenueelement
ROLE	role
SALES_CHANNEL	saleschannel
SALES_ROLE	salesrole
SALES_INVOICED	salesinvoiced
SALES_ORDERED	salesordered

Value	Sets Query.type Property To
SALES_TAX_ITEM	salestaxitem
SCHEDULED_SCRIPT	scheduledscript
SCHEDULED_SCRIPT_DEPLOYMENT	scheduledscriptdeployment
SCHEDULED_SCRIPT_INSTANCE	scheduledscriptinstance
SCRIPT	script
SCRIPT_CUSTOM_RECORD_TYPE	scriptcustomrecordtype
SCRIPT_DEPLOYMENT	scriptdeployment
SCRIPT_NOTE	scriptnote
SCRIPT_RECORD_TYPE	scriptrecordtype
SEARCH_CAMPAIGN	searchcampaign
SENT_EMAIL	sentemail
SHIPPING_PACKAGE	shippingpackage
SHIPPING_PARTNER_REGISTRATION	shippingpartnerregistration
SHIPPING_PARTNERS_PLUGIN	shippingpartnersplugin
SHIP_ITEM	shipitem
SHOPPING_CART	shoppingcart
SITE_CATEGORY	sitecategory
SITE_THEME	sitetheme
SOLUTION	solution
STANDARD_COST_VERSION	standardcostversion
STATE	state
STATISTICAL_JOURNAL_ENTRY	statisticaljournalentry
STATISTICAL_SCHEDULE	statisticalschedule
STORE_PICKUP_FULFILLMENT	storepickupfulfillment
STORE_TAB	storetab
SUBLIST	sublist
SUBSIDIARY	subsidiary
SUBSIDIARY_SETTINGS	subsidiarysettings
SUITELET	suitelet
SUITELET_DEPLOYMENT	suiteletdeployment
SUITE_SCRIPT_DETAIL	suitescriptdetail
SUPPLY_CHAIN_SNAPSHOT	supplychainsnapshot
SUPPLY_CHAIN_SNAPSHOT_SIMULATION	supplychainsnapshotsimulation

Value	Sets Query.type Property To
SUPPLY_CHANGE_ORDER	supplychangeorder
SUPPLY_PLAN_DEFINITION	supplyplandefinition
SUPPORT_CASE	supportcase
SUPPORT_CASE_PRIORITY	supportcasepriority
SUPPORT_CASE_STATUS	supportcasestatus
SYSTEM_EMAIL_TEMPLATE	systememailtemplate
SYSTEM_NOTE	systemnote
SYSTEM_NOTE2	systemnote2
SYSTEM_NOTE_FIELD	systemnotefield
SYSTEM_NOTE_TYPE	systemnotetype
TASK	task
TASK_ITEM_STATUS	taskitemstatus
TAX_CALCULATION_PLUGIN	taxcalculationplugin
TAX_ITEM_TAX_GROUP	taxitemtaxgroup
TAX_TYPE	taxtype
TERM	term
TEST_PLUGIN	testplugin
TEXT_ENHANCE_ACTION	textenhanceaction
TIME_BILL	timebill
TIME_MODIFICATION_REQUEST	timemodificationrequest
TIME_SHEET	timesheet
TOPIC	topic
TRACKING_NUMBER	trackingnumber
TRANSACTION	transaction
TRANSACTION_ADDRESSBOOK	transactionaddressbook
TRANSACTION_APPLIED_RULES_LOG	transactionappliedruleslog
TRANSACTION_BILLING	transactionbilling
TRANSACTION_BILLING_ADDRESSBOOK	transactionbillingaddressbook
TRANSACTION_DELETION_REASON	transactiondeletionreason
TRANSACTION_HISTORY	transactionhistory
TRANSACTION_NUMBERING_AUDIT_LOG	transactionnumberingauditlog
TRANSACTION_PAYEE_ADDRESSBOOK	transactionpayeeaddressbook
TRANSACTION_RETURN_ADDRESSBOOK	transactionreturnaddressbook

Value	Sets Query.type Property To
TRANSACTION_SHIPPING_ADDRESSBOOK	transactionshippingaddressbook
TRANSACTION_STATUS	transactionstatus
U_S_R_SNAPSHOT	usrsnapshot
UMD_FIELD	umdfield
UNDELIVERED_EMAIL	undeliveredemail
UNITS_TYPE	unitstype
UNLOCKED_TIME_PERIOD	unlockedtimeperiod
USER_AUTHORIZATION_CONSENT	userauthorizationconsent
USER_EVENT_SCRIPT	usereventscript
USER_EVENT_SCRIPT_DEPLOYMENT	usereventscriptdeployment
USER_O_AUTH_TOKEN	useroauthtoken
USRSAVEDSEARCH	usrsavedsearch
USR_AUDIT_LOG	usrauditlog
USR_DS_AUDIT_LOG	usrdsauditlog
USR_DS_EXECUTION_LOG	usrdsexecutionlog
USR_EXECUTION_LOG	usrexecutionlog
VENDOR	vendor
VENDOR_CATEGORY	vendorcategory
VENDOR_SUBSIDIARY_RELATIONSHIP	vendorsubsiaryrelationship
WEBAPP	webapp
WEB_SITE	website
WORKBOOK_BUILDER_PLUGIN	workbookbuilderplugin
WORKFLOW_ACTION_SCRIPT	workflowactionscrip
WORKFLOW_ACTION_SCRIPT_DEPLOYMENT	workflowactionscripdeployment
WORKPLACE	workplace
WORK_CALENDAR	workcalendar
WBS	wbs
ZONE	zone

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/query Module Script Samples](#).

```
1 // Add additional code
2 ...
```

```

3   var myCustomerQuery = query.create({
4       type: query.Type.CUSTOMER
5   });
6   var mySalesRepJoin = myCustomerQuery.autoJoin({
7       fieldId: 'salesrep'
8   });
9   var firstCondition = myCustomerQuery.createCondition({
10      fieldId: 'id',
11      operator: query.Operator.EQUAL,
12      values: 107
13  });
14  var secondCondition = myCustomerQuery.createCondition({
15      fieldId: 'id',
16      operator: query.Operator.EQUAL,
17      values: 2647
18  });
19  var thirdCondition = mySalesRepJoin.createCondition({
20      fieldId: 'email',
21      operator: query.Operator.START_WITH_NOT,
22      values: 'foo'
23  });
24  myCustomerQuery.condition = myCustomerQuery.and(thirdCondition, myCustomerQuery.or(firstCondition, secondCondition));
25  var resultSet = myCustomerQuery.run();
26  ...
27  // Add additional code

```

N/record Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/record module to work with NetSuite records. You can use this module to create, delete, copy, load, or make changes to a record.

Go To Samples {..}

SuiteScript supports working with standard NetSuite records and with instances of custom record types. Supported standard record types are described in the [SuiteScript Records Browser](#). Refer also to [SuiteScript Supported Records](#). For help working with an instance of a custom record type, see the help topic [Custom Record](#).

For help finding a record's internal ID, see the help topic [Finding Internal IDs of Records](#).



Important: SuiteScript does not support direct access to the NetSuite UI through the Document Object Model (DOM). The NetSuite UI should only be accessed with SuiteScript APIs.

In This Help Topic

- [N/record Module Members](#)
- [Column Object Members](#)
- [Field Object Members](#)
- [Macro Object Members](#)
- [Record Object Members](#)
- [Sublist Object Members](#)

■ N/record Default Values

N/record Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	record.Column	Object	Client and server scripts	Encapsulates a column of a sublist on a standard or custom record.
	record.Field	Object	Client and server scripts	Encapsulates a body or sublist field on a standard or custom record.
	record.Macro	Object	Client and server scripts	Encapsulates a NetSuite record macro.
	Plain JavaScript Object	Object	Client and server scripts	A plain JavaScript object of record macros available for a record type. This object is returned by Record.getMacros() .
	record.Record	Object	Client and server scripts	Encapsulates a NetSuite record.
	record.Sublist	Object	Client and server scripts	Encapsulates a sublist on a standard or custom record.
Method	record.attach(options)	void	Client and server scripts	Attaches a record to another record.
	record.attach.promise(options)	Promise	Client scripts	Attaches a record asynchronously to another record.
	record.copy(options)	record.Record	Client and server scripts	Creates a new record by copying an existing record in NetSuite.
	record.copy.promise(options)	Promise	Client scripts	Creates a new record asynchronously by copying an existing record in NetSuite.
	record.create(options)	record.Record	Client and server scripts	Creates a new record.
	record.create.promise(options)	Promise	Client scripts	Creates a new record asynchronously.
	record.delete(options)	number	Client and server scripts	Deletes a record.
	record.delete.promise(options)	Promise	Client scripts	Deletes a record asynchronously.
	record.detach(options)	void	Client and server scripts	Detaches a record from another record.
	record.detach.promise(options)	Promise	Client scripts	Detaches a record from another record asynchronously.
	record.load(options)	Object	Client and server scripts	Loads an existing record.
	record.load.promise(options)	Promise	Client scripts	Loads an existing record asynchronously.
	record.submitFields(options)	Object	Client and server scripts	Updates and submits one or more body fields on an existing record in NetSuite, and returns the internal ID of the parent record.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	record.submitFields.promise(options)	Promise	Client scripts	Updates and submits one or more body fields asynchronously on an existing record in NetSuite, and returns the internal ID of the parent record.
	record.transform(options)	record.Record	Client and server scripts	Transforms a record from one type into another, using data from an existing record.
	record.transform.promise(options)	Promise	Client scripts	Transforms a record from one type into another asynchronously, using data from an existing record.
Enum	record.Type	enum	Client and server scripts	Holds the string values for supported record types. Use this enum to set the value of the Record.type property in cases where you are working with an instance of a standard NetSuite record type.

Column Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Column.id	string (read-only)	Client and server scripts	Returns the internal ID of the column.
	Column.isDisabled	boolean	Client and server scripts	Indicates whether the column is disabled.
	Column.isDisplay	boolean (read-only)	Client and server scripts	Indicates whether the column is displayed.
	Column.isMandatory	boolean	Client and server scripts	Indicates whether the column is required.
	Column.isSortable	boolean (read-only)	Client and server scripts	Indicates whether the column is sortable.
	Column.label	string (read-only)	Client and server scripts	Returns the UI label for the column.
	Column.sublistId	string (read-only)	Client and server scripts	Returns the internal ID of the standard or custom sublist that contains the column.
	Column.type	string (read-only)	Client and server scripts	Returns the column type.

Field Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Field.getSelectOptions(options)	array	Client and server scripts	Returns an array of available options on a standard or custom select, multiselect, or radio field as

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				key-value pairs. Only the first 1,000 available options are returned.
Property	Field.label	string (read-only)	Client and server scripts	Returns the UI label for a standard or custom field body or sublist field.
	Field.id	string (read-only)	Client and server scripts	Returns the internal ID of a standard or custom body or sublist field.
	Field.type	string (read-only)	Client and server scripts	Returns the type of a body or sublist field.
	Field.isMandatory	boolean	Client and server scripts	Returns true if the standard or custom field is required on the record form, or false otherwise.
	Field.sublistId	string (read-only)	Client and server scripts	Returns the ID of the sublist associated with the specified sublist field. This property is available only when working with a record in dynamic mode.
	Field.isDisplay	boolean	Client and server scripts	Returns true if the field is visible on the record form, or false if it is not. This property is available only when working with a record in dynamic mode.

Macro Object Members

The following members are called on the [record.Macro](#) object. For information about record macros, see the help topic [Overview of Record Action and Macro APIs](#).

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Macro.execute(options)	Object	Client and server scripts	Performs a macro operation and returns its result in an object.
	Macro.execute.promise(options)	Promise	Client scripts	Performs a macro operation asynchronously.
	Macro(options)	Object	Client and server scripts	Performs a macro operation and returns its result in an object.
	Macro.promise(options)	Promise	Client scripts	Performs a macro operation asynchronously.
Property	Macro.id	string	Client and server scripts	The ID of the macro. For a list of macro IDs, see the help topic Supported Record Macros

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Macro.label	string	Client and server scripts	The macro label.
	Macro.description	string	Client and server scripts	The macro description.
	Macro.attributes	Object	Client and server scripts	The macro defined attributes.

Record Object Members

The following members are called on the [record.Record](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Record.cancelLine(options)	record.Record	Client and server scripts	Cancels the currently selected line on a sublist.
	Record.commitLine(options)	record.Record	Client and server scripts	Commits the currently selected line on a sublist.
	Record.executeMacro(options)	Object	Client and server scripts	Performs macro operation and returns its result in a plain JavaScript object.
	Record.getMacros()	Object	Client and server scripts	Provides a plain JavaScript object that contains macro objects defined for a record type, indexed by the Macro ID.
	Record.findMatrixSublistLineWithValue(options)	number	Client and server scripts	Returns the line number of the first instance where a specified value is found in a specified column of the matrix.
	Record.findSublistLineWithValue(options)	number	Client and server scripts	Returns the line number for the first occurrence of a field value in a sublist.
	Record.getCurrentMatrixSublistValue(options)	number Date string array boolean	Client and server scripts	Gets the value for the currently selected line in the matrix.
	Record.getCurrentSublistField(options)	record.Field	Client and server scripts	Returns a field object from a sublist.
	Record.getCurrentSublistIndex(options)	number	Client and server scripts	Returns the line number of the currently selected line.
	Record.getCurrentSublistSubrecord(options)	record.Record	Client and server scripts	Gets the subrecord for the associated sublist field on the current line.
	Record.getCurrentSublistText(options)	string	Client and server scripts	Returns a text representation of the field value in the currently selected line.
	Record.getCurrentSublistValue(options)	number Date string array boolean	Client and server scripts	Returns the value of a sublist field on the currently selected sublist line.
	Record.getField(options)	record.Field	Client and server scripts	Returns a field object from a record.
	Record.getFields()	string[]	Client and server scripts	Returns the body field names (internal ids) of all the fields in the record, including machine header field and matrix header fields.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Record.getLineCount(options)	number	Client and server scripts	Returns the number of lines in a sublist.
	Record.getMacro(options)	record.Macro	Client and server scripts	Provides a macro to execute.
	Record.getMacros()	Object	Client and server scripts	Provides a plain JavaScript object that contains macro objects defined for a record type, indexed by the Macro ID.
	Record.getMatrixHeaderCount(options)	number	Client and server scripts	Returns the number of columns for the specified matrix.
	Record.getMatrixHeaderField(options)	record.Field	Client and server scripts	Gets the field for the specified header in the matrix.
	Record.getMatrixHeaderValue(options)	number Date string array boolean	Client and server scripts	Gets the value for the associated header in the matrix.
	Record.getMatrixSublistField(options)	record.Field	Client and server scripts	Gets the field for the specified sublist in the matrix.
	Record.getMatrixSublistValue(options)	number Date string array boolean	Client and server scripts	Gets the value for the associated field in the matrix.
	Record.getSublist(options)	record.Sublist	Client and server scripts	Returns the specified sublist.
	Record.getSublists()	string[]	Client and server scripts	Returns all the names of all the sublists.
	Record.getSublistField(options)	record.Field	Client and server scripts	Returns a field object from a sublist.
	Record.getSublistFields(options)	string[]	Client and server scripts	Returns all the field names in a sublist.
	Record.getSublistSubrecord(options)	record.Record	Client and server scripts	Gets the subrecord associated with a sublist field. (standard mode only)
	Record.getSublistText(options)	string	Client and server scripts	Returns the value of a sublist field in a text representation.
	Record.getSublistValue(options)	number Date string array boolean	Client and server scripts	Returns the value of a sublist field.
	Record.getSubrecord(options)	record.Record	Client and server scripts	Gets the subrecord for the associated field.
	Record.getText(options)	string	Client and server scripts	Returns the text representation of a field value.
	Record.getValue(options)	number Date string array boolean	Client and server scripts	Returns the value of a field. Not to be used on custom password fields. Use crypto.checkPasswordField(options) instead.
	Record.hasCurrentSublistSubrecord(options)	boolean	Client and server scripts	Returns a value indicating whether the associated sublist field has a subrecord on the current line.
	Record.hasSublistSubrecord(options)	boolean	Client and server scripts	Returns a value indicating whether the associated sublist field contains a subrecord.
	Record.hasSubrecord(options)	boolean	Client and server scripts	Returns a value indicating whether the field contains a subrecord.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Record.insertLine(options)	record.Record	Client and server scripts	Inserts a sublist line.
	Record.removeCurrentSublistSubrecord(options)	record.Record	Client and server scripts	Removes the subrecord for the associated sublist field on the current line.
	Record.removeLine(options)	record.Record	Client and server scripts	Removes a sublist line.
	Record.removeSublistSubrecord(options)	record.Record	Client and server scripts	Removes the subrecord for the associated sublist field. (standard mode only)
	Record.removeSubrecord(options)	record.Record	Client and server scripts	Removes the subrecord for the associated field.
	Record.save(options)	number	Client and server scripts	Submits a new record or saves edits to an existing record. This method is not available to subrecords.
	Record.save.promise(options)	number	Client scripts	Submits a new record asynchronously or saves edits to an existing record asynchronously. This method is not available to subrecords.
	Record.selectLine(options)	record.Record	Client and server scripts	Selects an existing line in a sublist.
	Record.selectNewLine(options)	record.Record	Client and server scripts	Selects a new line at the end of a sublist.
	Record.setCurrentMatrixSublistValue(options)	record.Record	Client and server scripts	Sets the value for the line currently selected in the matrix.
	Record.setCurrentSublistText(options)	record.Record	Client and server scripts	Sets the value for the field in the currently selected line by a text representation.
	Record.setCurrentSublistValue(options)	record.Record	Client and server scripts	Sets the value for the field in the currently selected line.
	Record.setMatrixHeaderValue(options)	record.Record	Client and server scripts	Sets the value for the associated header in the matrix.
	Record.setMatrixSublistValue(options)	record.Record	Client and server scripts	Sets the value for the associated field in the matrix.
	Record.setSublistText(options)	record.Record	Client and server scripts	Sets the value of a sublist field by a text representation. (standard mode only)
	Record.setSublistValue(options)	record.Record	Client and server scripts	Sets the value of a sublist field. (standard mode only)
	Record.setText(options)	record.Record	Client and server scripts	Sets the value of the field by a text representation.
	Record.setValue(options)	record.Record	Client and server scripts	Sets the value of a field.
Property	Record.id	number (read-only)	Client and server scripts	The internal ID of a specific record. This property is not available to subrecords.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Record.isDynamic	boolean (read-only)	Client and server scripts	Indicates whether the record is in dynamic or standard mode.
	Record.type	string (read-only)	Client and server scripts	The record type. This property is not available to subrecords.

Sublist Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Sublist.getColumn(options)	record.Column	Client and server scripts	Returns a column in the sublist.
Property	Sublist.id	string (read-only)	Client and server scripts	Returns the internal ID of the sublist.
	Sublist.isChanged	boolean (read-only)	Client and server scripts	Indicates whether the sublist has changed on the record form.
	Sublist.isDisplay	boolean (read-only)	Client and server scripts	Indicates whether the sublist is displayed on the record form.
	Sublist.type	string (read-only)	Client and server scripts	Returns the sublist type.

N/record Default Values

You can specify record initialization parameters that default when creating, copying, loading, and transforming records. To enable this behavior, use the optional `defaultValues` parameter in the following APIs:

- [record.create\(options\)](#)
- [record.copy\(options\)](#)
- [record.transform\(options\)](#)
- [record.load\(options\)](#)

The following table lists initialization types that are available to certain SuiteScript-supported records and the values they can contain.

Record	Initialization Type	Values
All SuiteScript-supported records that support form customization.	customform	<customformid>
Assembly Build	assemblyitem	<assemblyitemid>
Cash Refund	entity	<entityid>
Cash Sale	entity	<entityid>
Charge Rule	entity	<entityid>

Record	Initialization Type	Values
Check	entity	<entityid>
Credit Memo	entity	<entityid>
Customer Payment	entity	<entityid>
Customer Refund	entity	<entityid>
Deposit	disablepaymentfilters	<disablepaymentfilters>
Estimate	entity	<entityid>
Expense Report	entity	<entityid>
Invoice	entity	<entityid>
Item Receipt	entity	<entityid>
Non-Inventory Part	subtype	sale resale purchase
Opportunity	entity	<entityid>
Other Charge Item	subtype	sale resale purchase
Purchase Order	entity	<entityid>
Return Authorization	entity	<entityid>
Sales Order	entity	<entityid>
Script Deployment	script	<scriptid>
Service	subtype	sale resale purchase
Subscription Change Order		<scriptid> See the help topic Default Values .
Tax Group	nexuscountry	<countrycode> See Country Codes Used for Initialization Parameters .
Tax Type	country	<countrycode> See Country Codes Used for Initialization Parameters .
Topic	parenttopic	<parenttopicid>
Vendor Bill	entity	<entityid>
Vendor Payment	entity	<entityid>
Work Order	assemblyitem	<assemblyitemid>

Country Codes Used for Initialization Parameters

If you are scripting the Tax Group or Tax Type records, you can initialize the record to source all values related to a specific country. In your script, use the country code for the *countrycodeid* value, for example:

```
1 | record.create('taxgroup', {nexuscountry: 'AR'});
```

Country Code	Country Name
AD	Andorra
AE	United Arab Emirates
AF	Afghanistan
AG	Antigua and Barbuda
AI	Anguilla
AL	Albania
AM	Armenia
AO	Angola
AQ	Antarctica
AR	Argentina
AS	American Samoa
AT	Austria
AU	Australia
AW	Aruba
AX	Åland Islands
AZ	Azerbaijan
BA	Bosnia and Herzegovina
BB	Barbados
BD	Bangladesh
BE	Belgium
BF	Burkina Faso
BG	Bulgaria
BH	Bahrain
BI	Burundi
BJ	Benin
BL	Saint Barthélemy
BM	Bermuda
BN	Brunei Darrussalam
BO	Bolivia (Plurinational State of)
BQ	Bonaire, Sint Eustatius and Saba

Country Code	Country Name
BR	Brazil
BS	Bahamas
BT	Bhutan
BV	Bouvet Island
BW	Botswana
BY	Belarus
BZ	Belize
CA	Canada
CC	Cocos (Keeling) Islands
CD	Congo (the Democratic Republic of the)
CF	Central African Republic
CG	Congo
CH	Switzerland
CI	Côte d'Ivoire
CK	Cook Islands
CL	Chile
CM	Cameroon
CN	China
CO	Colombia
CR	Costa Rica
CU	Cuba
CV	Cabo Verde
CW	Curacao
CX	Christmas Island
CY	Cyprus
CZ	Czechia
DE	Germany
DJ	Djibouti
DK	Denmark
DM	Dominica
DO	Dominican Republic
DZ	Algeria

Country Code	Country Name
EA	Ceuta and Melilla
EC	Ecuador
EE	Estonia
EG	Egypt
EH	Western Sahara
ER	Eritrea
ES	Spain
ET	Ethiopia
FI	Finland
FJ	Fiji
FK	Falkland Islands (Malvinas)
FM	Micronesia (Federated States of)
FO	Faroe Islands
FR	France
GA	Gabon
GB	United Kingdom
GD	Grenada
GE	Georgia
GF	French Guiana
GG	Guernsey
GH	Ghana
GI	Gibraltar
GL	Greenland
GM	Gambia
GN	Guinea
GP	Guadeloupe
GQ	Equatorial Guinea
GR	Greece
GS	South Georgia and the South Sandwich Islands
GT	Guatemala
GU	Guam
GW	Guinea-Bissau

Country Code	Country Name
GY	Guyana
HK	Hong Kong
HM	Heard Island and McDonald Islands
HN	Honduras
HR	Croatia
HT	Haiti
HU	Hungary
IC	Canary Islands
ID	Indonesia
IE	Ireland
IL	Israel
IM	Isle of Man
IN	India
IO	British Indian Ocean Territory
IQ	Iraq
IR	Iran (Islamic Republic of)
IS	Iceland
IT	Italy
JE	Jersey
JM	Jamaica
JO	Jordan
JP	Japan
KE	Kenya
KG	Kyrgyzstan
KH	Cambodia
KI	Kiribati
KM	Comoros
KN	Saint Kitts and Nevis
KP	Korea (the Democratic People's Republic of)
KR	Korea (the Republic of)
KW	Kuwait
KY	Cayman Islands

Country Code	Country Name
KZ	Kazakhstan
LA	Lao People's Democratic Republic
LB	Lebanon
LC	Saint Lucia
LI	Liechtenstein
LK	Sri Lanka
LR	Liberia
LS	Lesotho
LT	Lithuania
LU	Luxembourg
LV	Latvia
LY	Libya
MA	Morocco
MC	Monaco
MD	Moldova (the Republic of)
ME	Montenegro
MF	Saint Martin (French part)
MG	Madagascar
MH	Marshall Islands
MK	North Macedonia
ML	Mali
MM	Myanmar
MN	Mongolia
MO	Macao
MP	Northern Mariana Islands
MQ	Martinique
MR	Mauritania
MS	Montserrat
MT	Malta
MU	Mauritius
MV	Maldives
MW	Malawi

Country Code	Country Name
MX	Mexico
MY	Malaysia
MZ	Mozambique
NA	Namibia
NC	New Caledonia
NE	Niger
NF	Norfolk Island
NG	Nigeria
NI	Nicaragua
NL	Netherlands
NO	Norway
NP	Nepal
NR	Nauru
NU	Niue
NZ	New Zealand
OM	Oman
PA	Panama
PE	Peru
PF	French Polynesia
PG	Papua New Guinea
PH	Philippines
PK	Pakistan
PL	Poland
PM	Saint Pierre and Miquelon
PN	Pitcairn
PR	Puerto Rico
PS	Palestine, State of
PT	Portugal
PW	Palau
PY	Paraguay
QA	Qatar
RE	Réunion

Country Code	Country Name
RO	Romania
RS	Serbia
RU	Russian Federation
RW	Rwanda
SA	Saudi Arabia
SB	Solomon Islands
SC	Seychelles
SD	Sudan
SE	Sweden
SG	Singapore
SH	Saint Helena, Ascension and Tristan da Cunha
SI	Slovenia
SJ	Svalbard and Jan Mayen
SK	Slovakia
SL	Sierra Leone
SM	San Marino
SN	Senegal
SO	Somalia
SR	Suriname
SS	South Sudan
ST	Sao Tome and Principe
SV	El Salvador
SX	Sint Maarten (Dutch part)
SY	Syrian Arab Republic
SZ	Eswatini
TC	Turks and Caicos Islands
TD	Chad
TF	French Southern Territories
TG	Togo
TH	Thailand
TJ	Tajikistan
TK	Tokelau

Country Code	Country Name
TM	Turkmenistan
TN	Tunisia
TO	Tonga
TP	Timor-Leste
TR	Türkiye
TT	Trinidad and Tobago
TV	Tuvalu
TW	Taiwan (Province of China)
TZ	Tanzania, the United Republic of
UA	Ukraine
UG	Uganda
UM	United States Minor Outlying Islands
US	United States
UY	Uruguay
UZ	Uzbekistan
VA	Holy See
VC	Saint Vincent and the Grenadines
VE	Venezuela (Bolivarian Republic of)
VG	Virgin Islands (British)
VI	Virgin Islands (U.S.)
VN	Viet Nam
VU	Vanuatu
WF	Wallis and Futuna Islands
WS	Samoa
XK	Kosovo
YE	Yemen
YT	Mayotte
ZA	South Africa
ZM	Zambia
ZW	Zimbabwe

N/record Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/record module:

- [Create and Save a Contact Record](#)
- [Create and Save a Contact Record Asynchronously Using Promise Methods](#)
- [Create Multiple Sales Records Using a Scheduled Script](#)
- [Access Sublists and a Subrecord from a Record](#)
- [Access Sublists and a Subrecord from a Record Asynchronously Using Promise Methods](#)
- [Call a Macro on a Sales Order Record](#)

Create and Save a Contact Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use the N/record module to create and save a contact record.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: Some of the values in these samples are placeholders. Before using these samples, replace all hard-coded values, such as IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/record'], record => {
5     // Create an object to hold name data for the contact
6     const nameData = {
7       firstname: 'John',
8       middlename: 'Doe',
9       lastname: 'Smith'
10    };
11
12    // Create a contact record
13    let objRecord = record.create({
14      type: record.Type.CONTACT,
15      isDynamic: true
16    });
17
18    // Set the values of the subsidiary, firstname, middlename,
19    // and lastname properties
20    objRecord.setValue({
21      fieldId: 'subsidiary',
22      value: '1'
23    });
24    for (let key in nameData) {
25      if (nameData.hasOwnProperty(key)) {
26        objRecord.setValue({
27          fieldId: key,
28          value: nameData[key]
29        });

```



```

30     }
31   }
32
33   // Save the record
34   let recordId = objRecord.save({
35     enableSourcing: false,
36     ignoreMandatoryFields: false
37   });
38 });

```

Create and Save a Contact Record Asynchronously Using Promise Methods

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create and save a contact record using promise methods.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Note: To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   */
5   require(['N/record'], record => {
6     // Create an object to hold name data for the contact
7     const nameData = {
8       firstname: 'John',
9       middlename: 'Doe',
10      lastname: 'Smith'
11    };
12
13    // Create a contact record using the promise method
14    let createRecordPromise = record.create.promise({
15      type: record.Type.CONTACT,
16      isDynamic: true
17    });
18
19    // When the promise is fulfilled, set the values of the subsidiary,
20    // firstname, middlename, and lastname properties, and save the
21    // record
22    createRecordPromise.then(objRecord => {
23      log.debug('Start evaluating promise content...');
24      objRecord.setValue({
25        fieldId: 'subsidiary',
26        value: '1'
27      });
28      for (let key in nameData) {
29        if (nameData.hasOwnProperty(key)) {
30          objRecord.setValue({
31            fieldId: key,
32            value: nameData[key]
33          });
34        }
35      }
36    });
37  });

```

```

35     }
36     let recordId = objRecord.save({
37         enableSourcing: false,
38         ignoreMandatoryFields: false
39     });
40     }, function(e) {
41         log.error('Unable to create contact', e.name);
42     });
43 });

```

Create Multiple Sales Records Using a Scheduled Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a scheduled script to create multiple sales records and log the record creation progress.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ScheduledScript
4   */
5
6  // This script creates multiple sales records and logs the record creation progress.
7  define(['N/runtime', 'N/record'], function(runtime, record) {
8      return {
9          execute: function(context) {
10             var script = runtime.getCurrentScript();
11             for (x = 0; x < 500; x++) {
12                 var rec = record.create({
13                     type: record.Type.SALES_ORDER
14                 });
15                 script.percentComplete = (x * 100)/500;
16                 log.debug({
17                     title: 'New Sales Orders',
18                     details: 'Record creation progress: ' + script.percentComplete + '%'
19                 });
20             }
21         }
22     };
23 });

```

Access Sublists and a Subrecord from a Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to access sublists and a subrecord from a record. This sample requires the Advanced Number Inventory Management feature.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4

```

```

5 require(['N/record'], function(record) {
6     function createPurchaseOrder() {
7         var rec = record.create({
8             type: 'purchaseorder',
9             isDynamic: true
10        });
11        rec.setValue({
12            fieldId: 'entity',
13            value: 52
14        });
15        rec.setValue({
16            fieldId: 'location',
17            value: 2
18        });
19        rec.selectNewLine({
20            sublistId: 'item'
21        });
22        rec.setCurrentSublistValue({
23            sublistId: 'item',
24            fieldId: 'item',
25            value: 190
26        });
27        rec.setCurrentSublistValue({
28            sublistId: 'item',
29            fieldId: 'quantity',
30            value: 2
31        });
32        subrecordInvDetail = rec.getCurrentSublistSubrecord({
33            sublistId: 'item',
34            fieldId: 'inventorydetail'
35        });
36        subrecordInvDetail.selectNewLine({
37            sublistId: 'inventoryassignment'
38        });
39        subrecordInvDetail.setCurrentSublistValue({
40            sublistId: 'inventoryassignment',
41            fieldId: 'receiptinventorynumber',
42            value: 'myinventoryNumber'
43        });
44        subrecordInvDetail.commitLine({
45            sublistId: 'inventoryassignment'
46        });
47        subrecordInvDetail.selectLine({
48            sublistId: 'inventoryassignment',
49            line: 0
50        });
51        var myInventoryNumber = subrecordInvDetail.getCurrentSublistValue({
52            sublistId: 'inventoryassignment',
53            fieldId: 'receiptinventorynumber'
54        });
55        rec.commitLine({
56            sublistId: 'item'
57        });
58        var recordId = rec.save();
59    }
60    createPurchaseOrder();
61 });

```

Note: For additional script samples that include subrecords, see the help topic [SuiteScript 2.x Scripting Subrecords](#).

Access Sublists and a Subrecord from a Record Asynchronously Using Promise Methods

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to access sublists and a subrecord from a record using promise methods. This sample requires the Advanced Number Inventory Management feature.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Note: To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/record'], function(record) {
6      function createPurchaseOrder() {
7          var createRecordPromise = record.create.promise({
8              type: 'purchaseorder',
9              isDynamic: true
10         });
11         createRecordPromise.then(function(rec) {
12             rec.setValue({
13                 fieldId: 'entity',
14                 value: 52
15             });
16             rec.setValue({
17                 fieldId: 'location',
18                 value: 2
19             });
20             rec.selectNewLine({
21                 sublistId: 'item'
22             });
23             rec.setCurrentSublistValue({
24                 sublistId: 'item',
25                 fieldId: 'item',
26                 value: 190
27             });
28             rec.setCurrentSublistValue({
29                 sublistId: 'item',
30                 fieldId: 'quantity',
31                 value: 2
32             });
33             subrecordInvDetail = rec.getCurrentSublistSubrecord({
34                 sublistId: 'item',
35                 fieldId: 'inventorydetail'
36             });
37             subrecordInvDetail.selectNewLine({
38                 sublistId: 'inventoryassignment'
39             });
40             subrecordInvDetail.setCurrentSublistValue({
41                 sublistId: 'inventoryassignment',
42                 fieldId: 'receiptinventorynumber',
43                 value: 'myinventoryNumber'
44             });
45             subrecordInvDetail.commitLine({
46                 sublistId: 'inventoryassignment'
47             });
48             subrecordInvDetail.selectLine({
49                 sublistId: 'inventoryassignment',
50                 line: 0
51             });
52             var myInventoryNumber = subrecordInvDetail.getCurrentSublistValue({
53                 sublistId: 'inventoryassignment',
54                 fieldId: 'receiptinventorynumber'
55             });
56             rec.commitLine({
57                 sublistId: 'item'
58             });

```

```

59     var recordId = rec.save();
60     }, function(err) {
61         log.error('Unable to create purchase order!', err.name);
62     });
63 }
64 createPurchaseOrder();
65 });

```

Call a Macro on a Sales Order Record

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows you how to call a `calculateTax` macro on a sales order record. To execute a macro on a record, the record must be created or loaded in dynamic mode. Note that the SuiteTax feature must be enabled to successfully execute the macro used in this sample.

For information about record macros, see the help topic [Overview of Record Action and Macro APIs](#).

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/record'],
6      function(record) {
7
8      var recordObj = record.create({
9          type: record.Type.SALES_ORDER,
10         isDynamic: true
11     });
12
13     var ENTITY_VALUE = 1;
14     var ITEM_VALUE = 1;
15     recordObj.setValue({
16         fieldId: 'entity',
17         value: ENTITY_VALUE
18     });
19     recordObj.selectNewLine({
20         sublistId: 'item'
21     });
22     recordObj.setCurrentSublistValue({
23         sublistId: 'item',
24         fieldId: 'item',
25         value: ITEM_VALUE
26     });
27     recordObj.setCurrentSublistValue({
28         sublistId: 'item',
29         fieldId: 'quantity',
30         value: 1
31     });
32     recordObj.commitLine({
33         sublistId: 'item'
34     });
35
36     var totalBeforeTax = recordObj.getValue({fieldId: 'total'});
37
38     // get macros available on the record
39     var macros = recordObj.getMacros();
40
41     // execute the macro

```

```

42     if ( 'calculateTax' in macros)
43     {
44         macros.calculateTax(); // For promise version use: macros.calculateTax.promise()
45     }
46     // Alternative (direct) macro execution
47     // var calculateTax = recordObj.getMacro({id: 'calculateTax'});
48     // calculateTax(); // For promise version use: calculateTax.promise()
49     var totalAfterTax = recordObj.getValue({fieldId: 'total'});
50
51     var recordId = recordObj.save({
52         enableSourcing: false,
53         ignoreMandatoryFields: false
54     });
55 });

```

record.Column

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a column of a sublist on a standard or custom record. This object does not return a value, it returns information about the sublist column.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/record Module
Methods and Properties	Column Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11
12 var objColumn = objSublist.getColumn({
13     fieldId: 'item'
14 });
15
16 if(objColumn.label === 'myLabel' ){
17     //Perform an action
18 }
19 if(objColumn.type === 'checkbox'){
20     //Perform an action
21 }
22 ...

```

23 | // Add additional code

Column.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the column. Note that the Column.id value is the same as the value that is passed into fieldID.
Type	string (read-only)
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4   type: record.Type.SALES_ORDER,
5   id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9   sublistId: 'item'
10 });
11
12 var objColumn = objSublist.getColumn({
13   fieldId: 'item'
14 });
15 log.debug ({
16   title: 'ID comparison',
17   details: 'Note that objColumn.id = ' + objColumn.id + ' is the same as the value you passed in as fieldID.'
18 });
19 ...
20 // Add additional code

```

Column.isDisabled

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the column is disabled.
Type	boolean
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistObj = recordObj.getSublist({
4     sublistId: 'sublistId'
5 });
6 var columnObj = sublistObj.getColumn({
7     fieldId: 'columnId'
8 });
9
10 columnObj.isDisabled = true; // set the property value
11
12 var isDisabled = columnObj.isDisabled; // get the property value
13 ...
14 // Add additional code

```

Column.isDisplay

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the column is displayed.
Type	boolean (read-only)
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistObj = recordObj.getSublist({
4     sublistId: 'sublistId'
5 });
6 var columnObj = sublistObj.getColumn({
7     fieldId: 'columnId'
8 });
9
10 var isDisplay = columnObj.isDisplay; // get the property value
11 ...
12 // Add additional code

```

Column.isMandatory

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the column is required.
-----------------------------	---

Type	boolean
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistObj = recordObj.getSublist({
4     sublistId: 'sublistId'
5 });
6 var columnObj = sublistObj.getColumn({
7     fieldId: 'columnId'
8 });
9
10 columnObj.isMandatory = true; // set the property value
11
12 var isMandatory = columnObj.isMandatory; // get the property value
13 })...
14 // Add additional code

```

Column.isSortable

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the column is sortable.
Type	boolean (read-only)
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistObj = recordObj.getSublist({
4     sublistId: 'sublistId'
5 });
6 var columnObj = sublistObj.getColumn({
7     fieldId: 'columnId'
8 });
9 var isSortable = columnObj.isSortable;

```

```

10  })...
11  // Add additional code

```

Column.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the column. This property does not return a value, it returns information about the column label.
Type	string (read-only)
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var objRecord = record.load({
4      type: record.Type.SALES_ORDER,
5      id: 275
6  });
7
8  var objSublist = objRecord.getSublist({
9      sublistId: 'item'
10 });
11
12 var objColumn = objSublist.getColumn({
13     fieldId: 'item'
14 });
15
16 if (objColumn.label === 'myLabel' ){
17     //Perform an action
18 }
19 ...
20 // Add additional code

```

Column.sublistId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the standard or custom sublist that contains the column.
Type	string (read-only)
Module	N/record Module

Sibling Object Members	Column Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11
12 var objColumn = objSublist.getColumn({
13     fieldId: 'item'
14 });
15 // Perform an action with the objColumn.sublistId value
16 ...
17 // Add additional code

```

Column.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the column type. For more information about possible return values, see format.Type .
Type	string (read-only)
Module	N/record Module
Sibling Object Members	Column Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'

```

```

10 });
11
12 var objColumn = objSublist.getColumn({
13     fieldId: 'item'
14 });
15
16 if(objColumn.type === 'checkbox'){
17     //Perform an action
18 }
19 ...
20 // Add additional code

```

record.Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a body or sublist field on a standard or custom record. Use the following methods to access the Field object: - Record.getField(options) - Record.getSublistField(options) - Record.getCurrentSublistField(options) - CurrentRecord.getField(options) - CurrentRecord.getSublistField(options) Certain properties of Field objects are available only when working with a record in dynamic mode. - Field.id – standard and dynamic modes - Field.isDisplay – dynamic mode only - Field.isMandatory – standard and dynamic modes - Field.label – standard and dynamic modes - Field.sublistId – dynamic mode only - Field.type – standard and dynamic modes For more information about record modes, see the help topic SuiteScript 2.x Standard and Dynamic Modes.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/record Module
Methods and Properties	Field Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```
1 // Add additional code
```

```

2  ...
3  var objRecord = record.load({
4      type: record.Type.SALES_ORDER,
5      id: 275
6  });
7
8  var objSublist = objRecord.getSublist({
9      sublistId: 'item'
10 });
11
12 var objField = objSublist.getField({
13     fieldId: 'item'
14 });
15 if(objField.label === 'myLabel') {
16     //Perform an action
17 }
18 if(objField.type === 'checkbox'){
19     //Perform an action
20 }
21 ...
22 // Add additional code

```

Field.getSelectOptions(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Obtains an array of available options on a dropdown select, multi-select, or radio field. The internal ID and label of the field options are returned as name/value pairs.  Important: You can only use this method on a record in dynamic mode. For additional information about dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.
Returns	Object[] This function returns an array in the following format: <pre>1 [{value: 5, text: 'abc'}, {value: 6, text: '123'}]</pre> This function returns Type <code>Error</code> if the field is not a supported field for this method. If you attempt to get select options on a field that is not a dropdown select field, such as a popup select field or a field that does not exist on the form, <code>null</code> is returned. For example, if the number of options available for the field is greater than your setting for Maximum Entries in Dropdowns at Home > Set Preferences, the field becomes a popup field, and this method returns <code>null</code>. Because the maximum value for the Maximum Entries in Dropdowns settings is 500, the maximum number of options that can be returned by this method is 500.  Note: A call to this method may return different results for the same field for different roles.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module

Sibling Object Members	Field Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.filter	string	required	The search string to filter the select options that are returned.  Note: Filter values are case insensitive.	2015.2
options.operator	string	required	The following operators are supported: - contains (default) - is - startswith	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4   type: record.Type.SALES_ORDER,
5   id: 275
6 });
7
8 var objField = objRecord.getField({
9   fieldId: 'couponcode'
10 });
11
12 var options = objField.getSelectOptions({
13   filter : 'C',
14   operator : 'startswith'
15 });
16
17 //Perform an action with the options array
18 ...
19 // Add additional code

```

Field.label

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the UI label for a standard or custom field body or sublist field.
-----------------------------	--

Type	string (read-only)
Module	N/record Module
Sibling Object Members	Field Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4   type: record.Type.SALES_ORDER,
5   id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9   sublistId: 'item'
10 });
11
12 var objField = objSublist.getField({
13   fieldId: 'item'
14 });
15
16 if(objField.label === 'myLabel') {
17   // Perform an action
18 }
19 ...
20 // Add additional code

```

Field.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of a standard or custom body or sublist field.
Type	string (read-only)
Module	N/record Module
Sibling Object Members	Field Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var objRecord = record.load({
4      type: record.Type.SALES_ORDER,
5      id: 275
6  });
7
8  var objSublist = objRecord.getSublist({
9      sublistId: 'item'
10 });
11
12 var objField = objSublist.getField({
13     fieldId: 'item'
14 });
15 //Perform an action with the objField.id value
16 ...
17 // Add additional code

```

Field.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the type of a body or sublist field. For example, the value can return text, date, currency, select, checkbox, etc. For more information about possible return values, see format.Type. The maximum character limit for select field types is 801.
Type	string (read-only)
Module	N/record Module
Sibling Object Members	Field Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var objRecord = record.load({
4      type: record.Type.SALES_ORDER,
5      id: 275
6  });
7
8  var objSublist = objRecord.getSublist({
9      sublistId: 'item'
10 });
11
12 var objField = objSublist.getField({
13     fieldId: 'item'
14 });
15
16 if(objField.type === 'checkbox'){
17     //Perform an action
18 }
19 ...
20 // Add additional code

```


Field.isMandatory

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the standard or custom field is required on the record form, or false otherwise.
Type	boolean
Module	N/record Module
Sibling Object Members	Field Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4   type: record.Type.SALES_ORDER,
5   id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9   sublistId: 'item'
10 });
11
12 var objField = objSublist.getField({
13   fieldId: 'item'
14 });
15
16 if(objField.isMandatory){
17   var options = {
18     title: 'Incomplete Field:',
19     message: 'Please complete this field.'
20   };
21   dialog.alert(options);
22   ...
23 // Add additional code

```

Field.sublistId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the sublist ID for the specified sublist field. This property is available only when working with a record in dynamic mode. For more information about record modes, see the help topic SuiteScript 2.x Standard and Dynamic Modes .
Type	string (read-only)
Module	N/record Module

Sibling Object Members	Field Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11
12 var objField = objSublist.getField({
13     fieldId: 'item'
14 });
15
16 //Perform an action with the objField.sublistId
17 ...
18 // Add additional code

```

Field.isDisplay

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the field is visible on the record form, or false if it is not. This property is available only when working with a record in dynamic mode. For more information about record modes, see the help topic SuiteScript 2.x Standard and Dynamic Modes. This property is not available when working with record sublist fields. It is only available when working with record body fields.
Type	boolean
Module	N/record Module
Sibling Object Members	Field Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2

```

```

3 define(['N/currentRecord'], function(currentRecord){
4     return {
5         pageInit: function(context) {
6             var record = currentRecord.get();
7             var field = record.getField({
8                 fieldId: "custbody1"
9             });
10            field.isDisplay = false;
11        }
12    }
13 }
14 ...
15 // Add additional code

```

record.Macro

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a record macro. For information about record macros, see the help topic Overview of Record Action and Macro APIs . Use the Record.getMacro(options) method to access the Macro object.
Supported Script Types	Client and server scripts. For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/record Module
Methods and Properties	Macro Object Members
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myMacro = record.getMacro({id: 'calculateTax'})
4 ...
5 // Add additional code

```

Macro.execute(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a macro operation and returns its result in a plain JavaScript object. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Returns	{notifications: [], response: {}}

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Parent Object	record.Macro
Sibling Object Members	Macro Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.params	Object	optional	The macro arguments.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 timesheet.executeMacro({id:'copyFromWeek', params: {weekOf : '7/10/2017', copyExact : true}});
4 ...
5 // Add additional code

```

Macro.execute.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a macro operation asynchronously. For information about record macros, see the help topic Overview of Record Action and Macro APIs .  Note: The parameters and errors thrown for this method are the same as those for Macro.execute(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	Macro.execute(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/record Module
Parent Object	record.Macro
Sibling Object Members	Macro Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.params	Object	optional	The macro arguments.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 myMacro.execute.promise().then(function(result){ /* do something with macro result */ });
4 ...
5 // Add additional code

```

Macro(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a macro operation and returns its result in a plain JavaScript object.  Note: Substitute Macro with the name of the macro you are executing. For information about record macros, see the help topic Overview of Record Action and Macro APIs.
Returns	{notifications: [], response: {}}
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Parent Object	record.Macro

Sibling Object Members	Macro Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.params	Object	optional	The macro arguments.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var calculateTax = recordObj.getMacro({id: 'calculateTax'});
4 calculateTax();
5 ...
6 // Add additional code

```

Macro.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a macro operation asynchronously.  Note: Substitute Macro with the name of the macro you are executing. For information about record macros, see the help topic Overview of Record Action and Macro APIs.  Note: The parameters and errors thrown for this method are the same as those for Macro(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Macro(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Parent Object	record.Macro

Sibling Object Members	Macro Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.params	Object	optional	The macro arguments.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var calculateTax = recordObj.getMacro({id: 'calculateTax'});
4 calculateTax.promise();
5 ...
6 // Add additional code

```

Macro.attributes

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The defined attributes of the macro. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Type	Object
Module	N/record Module
Parent Object	record.Macro
Sibling Object Members	Macro Object Members
Since	2018.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...

```

```

3 | var attributes = macro.attributes; // get the attributes of the macro
4 | ...
5 | // Add additional code

```

Macro.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The description of the macro. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Type	string
Module	N/record Module
Parent Object	record.Macro
Sibling Object Members	Macro Object Members
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | var description = macro.description; // get the description of the macro
4 | ...
5 | // Add additional code

```

Macro.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the macro. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Type	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/record Module
Parent Object	record.Macro
Sibling Object Members	Macro Object Members

Since	2018.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var id = macro.id; // get the id of the macro
4 ...
5 // Add additional code

```

Macro.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label of the macro. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Type	string
Module	N/record Module
Parent Object	record.Macro
Sibling Object Members	Macro Object Members
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var label = macro.label; // get the label of the macro
4 ...
5 // Add additional code

```

record.Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a NetSuite record. There are two modes you can operate in when you create, copy, load, or transform a record with SuiteScript 2.x: standard mode and dynamic mode.
---------------------------	--

	- When a SuiteScript 2.x script creates, copies, loads, or transforms a record in standard mode, the record's body fields and sublist line items are not sourced, calculated, and validated until the record is saved (submitted) with Record.save(options). When you work with a record in standard mode, you do not need to set values in any particular order. After submitting the record, NetSuite processes the record's body fields and sublist line items in the correct order, regardless of the organization of your script. - When a SuiteScript 2.x script creates, copies, loads, or transforms a record in dynamic mode, the record's body fields and sublist line items are sourced, calculated, and validated in real-time. A record in dynamic mode emulates the behavior of a record in the UI. When you work with a record in dynamic mode, it is important that you set values in the same order you would within the UI. If you fail to do this, your results may not be accurate. The record.create(options), record.copy(options), record.load(options), and record.transform(options) methods work in standard mode by default. If you want these methods to work in dynamic mode, you must pass in a specific argument. See the help topic for the applicable method for more information. Use record.Type enum for multiple records. For help finding a record's internal ID, see the help topic Finding Internal IDs of Records. For more information about standard and dynamic modes, see the help topic SuiteScript 2.x Standard and Dynamic Modes.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/record Module
Methods and Properties	Record Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: '6',
6     isDynamic: true
7 });
8 ...
9 // Add additional code

```

Record.cancelLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Cancels the currently selected line on a sublist. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	The record.Record object that called the method.

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.cancelLine({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.commitLine(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Commits the currently selected line on a sublist. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
---------------------------	--

	When working in standard mode, set a sublist field using Record.setSublistValue(options) .
Returns	The record.Record object that called the method.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.ignoreRecalc	boolean	optional	If set to true, scripting recalculation is ignored. By default, this parameter is false.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.commitLine({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.executeMacro(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs macro operation and returns its result in a plain JavaScript object. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Returns	An object with the macro results or null.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2018.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The macro ID.
options.params	Object	optional	The macro arguments.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 if ('calculateTax' in macros) {
4     macros.calculateTax();
5 }
6 ...
7 // Add additional code

```

Record.executeMacro.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs macro operation and returns its result in a plain JavaScript object. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
---------------------------	---

	 Note: The parameters and errors thrown for this method are the same as those for Record.executeMacro(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	Record.executeMacro(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2018.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
options.id	string	required	The macro ID.
options.params	Object	optional	The macro arguments.

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see Promise Object .

```

1 // Add additional code
2 ...
3 if ('calculateTax' in macros) {
4     macros.calculateTax.promise();
5 }
6 ...
7 // Add additional code

```

Record.findMatrixSublistLineWithValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the line number of the first instance where a specified value is found in a specified column of the matrix. Note that line and column indexing begins at 0 with SuiteScript 2.x. (dynamic and standard modes — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
---------------------------	---

Returns	number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2016.2
options.fieldId	string	required	The ID of the matrix field. See, Finding Internal IDs of Record Fields .	2016.2
options.value	number	required	The value to search for.	2016.2
options.column	number	required	The column number of the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var lineNumber = objRecord.findMatrixSublistLineWithValue({
4     sublistId: 'item',
5     fieldId: 'price',
6     value: 233,
7     column: 17

```

```

8  });
9  ...
10 // Add additional code

```

Record.findSublistLineWithValue(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the line number for the first occurrence of a field value in a sublist. Note that line indexing begins at 0 with SuiteScript 2.x. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	A line number as a number, or -1 if not found.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.value	number Date string array boolean	optional	The value to search for.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or not defined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var lineNumber = objRecord.findSublistLineWithValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     value: 233
7 });
8 ...
9 // Add additional code

```

Record.getCurrentMatrixSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value for the currently selected line in the matrix. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a numeric value for rate and ratehighprecision fields.
Returns	number Date string array boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2
options.column	number	required	The column number for the matrix field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var matrixValue = objRecord.getCurrentMatrixSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12
7 });
8 ...
9 // Add additional code

```

Record.getCurrentSublistField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns metadata about a sublist field. (dynamic mode only— see the help topic SuiteScript 2.x Standard and Dynamic Modes) This is not a valid method for use in the fieldChanged(scriptContext) or postSourcing(scriptContext) entry points of a client script.
Returns	record.Field
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2016.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom sublist field.	2015.2

Parameter	Type	Required / Optional	Description	Since
			See, Finding Internal IDs of Record Fields .	
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistFieldMetadata = objRecord.getCurrentSublistField({
4     sublistId: 'item',
5     fieldId: 'item',
6 });
7 ...
8 // Add additional code

```

Record.getCurrentSublistIndex(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the line number of the currently selected line. Note that line indexing begins at 0 with SuiteScript 2.x. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var currIndex = objRecord.getCurrentSublistIndex({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.getCurrentSublistSubrecord(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the subrecord for the associated sublist field on the current line. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields . The fieldId is inside the sublist. It works if the current sublist contains subrecord fields. You can use Record.getCurrentSublistField(options) to get all metadata about a sublist field.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objSubrecord = objRecord.getCurrentSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 // Add additional code

```

Record.getCurrentSublistText(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a text representation of the field value in the currently selected line. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a string value with a "%" for rate and ratehighprecision fields.
Returns	string  Note: For multiselect fields, returns an array.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.forceSync Sourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously.	2019.1

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var fieldName = objRecord.getCurrentSublistText({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 // Add additional code

```

Record.getCurrentSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist field on the currently selected sublist line. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	number Date string array boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  var sublistValue = objRecord.getCurrentSublistValue({
4      sublistId: 'item',
5      fieldId: 'item'
6  });
7  ...
8  // Add additional code

```

Record.getField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a field object from a record. (dynamic and standard modes — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Field
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields .	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1  // Add additional code

```



```

2  ...
3  var objField = objRecord.getField({
4      fieldId: 'item'
5  });
6  ...
7  // Add additional code

```

Record.getFields()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the body field names (internal ids) of all the fields in the record, including machine header field and matrix header fields. (dynamic and standard modes — see the help topic SuiteScript 2.x Standard and Dynamic Modes)  Note: The order of the custom fields returned by this method on a custom record does not respect the configuration of the custom record fields page in the UI.
Returns	string[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var objFields = objRecord.getFields();
4  ...
5  // Add additional code

```

Record.getLineCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of lines in a sublist.
---------------------------	---

	(standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var numLines = objRecord.getLineCount({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.getMacro(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Provides a macro to be executed. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
Returns	Function to be executed for the macro.

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2018.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The macro ID.

Errors

Error Code	Thrown If
SSS_INVALID_MACRO_ID	A macro does not exist on the record.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var macro = recordObj.getMacro({id: 'calculateTax'});
4 ...
5 // Add additional code

```

Record.getMacros()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Provides a plain JavaScript object of available macro objects defined for a record type, indexed by the Macro ID. The object returns one or more record.Macro objects. If there are no macros available for the specified record type, an empty object is returned. For information about record macros, see the help topic Overview of Record Action and Macro APIs .
---------------------------	---

Returns	Object
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2018.2

Errors

Error Code	Thrown If
SSS_INVALID_RECORD_TYPE	The specified record type is invalid.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var macroList = recordObj.getMacros();
4 ...
5 // Add additional code

```

Record.getMatrixHeaderCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of columns for the specified matrix. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ....
3 var numLines = objRecord.getMatrixHeaderCount({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 // Add additional code

```

Record.getMatrixHeaderField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the field for the specified header in the matrix. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Field
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objField = objRecord.getMatrixHeaderField({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12
7 });
8 ...
9 // Add additional code

```

Record.getMatrixHeaderValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value for the associated header in the matrix.
---------------------------	---

	(standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a numeric value for rate and ratehighprecision fields.
Returns	number Date string array boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var value = objRecord.getMatrixHeaderValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12

```

```

7  });
8  ...
9  // Add additional code

```

Record.getMatrixSublistField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the field for the specified sublist in the matrix. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Field
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objField = objRecord.getMatrixSublistField({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12,
7     line: 3
8 });
9 ...
10 // Add additional code

```

Record.getMatrixSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the value for the associated field in the matrix. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	number Date string array boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2

Parameter	Type	Required / Optional	Description	Since
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var value = objRecord.getMatrixSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 12,
7     line: 3
8 });
9 ...
10 // Add additional code

```

Record.getSublist(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the specified sublist. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Sublist
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objSublist = objRecord.getSublist({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.getSublists()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns all the names of all the sublists. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	string[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code

```

```

2 | ...
3 | var sublistName = objRecord.getSublists();
4 | ...
5 | // Add additional code

```

Record.getSublistField(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a field object from a sublist. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Field
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objField = objRecord.getSublistField({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 // Add additional code

```

Record.getSublistFields(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns all the field names in a sublist. (standard and dynamic mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	string[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#). For other samples, see [Sublist.getColumn\(options\)](#).

```

1 // Add additional code
2 ...
3 var field = objRecord.getSublistFields({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.getSublistSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the subrecord associated with a sublist field. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) When working in dynamic mode, get a sublist subrecord using the following methods: Record.selectLine(options) Record.hasCurrentSublistSubrecord(options) Record.getCurrentSublistSubrecord(options)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist.	2015.2

Parameter	Type	Required / Optional	Description	Since
			This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.line	number	required	The line number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objSubRecord = objRecord.getSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 // Add additional code

```

Record.getSublistText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist field in a text representation. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a string value with a "%" for rate and ratehighprecision fields.
Returns	string Note: For multiselect fields, returns an array.
Supported Script Types	Client and server scripts. For more information, see the help topic SuiteScript 2.x Script Types. Limitations exist on how this method can be used in standard (deferredDynamic) mode. For details, refer to the description of the SSS_INVALID_API_USAGE error code in the Errors table. In dynamic mode, you can use getSublistText() without limitation.
Governance	None
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_API_USAGE	Invoked in certain cases when deferredDynamic mode is being used. For example, if Record.isDynamic is set to false, this error can be invoked in both of the following situations: ■ If the record object was created by <code>record.copy()</code>, <code>record.create()</code>, or <code>record.transform()</code>, and the script attempts to use <code>getSublistText()</code> without first using <code>setSublistText()</code> for the same field. ■ If the record object was created by <code>record.load()</code>, and the script uses <code>setSublistValue()</code> on a field before using <code>getSublistText()</code> for the same field. This guidance also affects user event scripts that instantiate records by using the <code>newRecord</code> or <code>oldRecord</code> object provided by the script context. These records always use deferredDynamic mode. For that reason, this error appears in both of the following situations: ■ When a user event script executes on a record that is being newly created, and the script attempts to use <code>getSublistText()</code> without first using <code>setSublistText()</code> for the same field. ■ When a user event script executes on an existing record, and the script uses <code>setSublistValue()</code> on a field before using <code>getSublistText()</code> for the same field. For more information, see the help topic SuiteScript 2.x Standard and Dynamic Modes.
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistFieldName = objRecord.getSublistText({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 // Add additional code

```

Record.getSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a sublist field. (dynamic and standard modes — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a numeric value for rate and ratehighprecision fields.
Returns	number Date string array boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_API_USAGE	Invoked prior to using setSublistValue in standard record mode.
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistFieldValue = objRecord.getSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 // Add additional code

```

Record.getSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the subrecord for the associated field. This method is not available for subrecords. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields .	2015.2

Errors

Error Code	Thrown If
FIELD_1_IS_DISABLED_YOU_CANNOT_APPLY_SUBRECORD_OPERATION_ON_THIS_FIELD	The specified field is disabled.
FIELD_1_IS_NOT_A_SUBRECORD_FIELD	The specified field is not a subrecord field.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sublistFieldValue = objRecord.getSubrecord({
4     fieldId: 'idnumber'
5 });
6 ...
7 // Add additional code

```

Record.getText(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the text representation of a field value. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a string value with a "%" for rate and ratehighprecision fields.
---------------------------	--

Returns	string  Note: For multiselect fields, returns an array.
Supported Script Types	Client and server scripts. For more information, see the help topic SuiteScript 2.x Script Types . In dynamic mode, you can use <code>getText()</code> without limitation but, in standard mode, limitations exist. In standard mode, you can use this method only in the following cases: ■ You can use <code>getText()</code> on any field where the script has already used <code>setText()</code>. ■ If you are loading or copying a record, you can use <code>getText</code> on any field except those where the script has already changed the value by using <code>setValue()</code>. For more details, refer to the description of the <code>SSS_INVALID_API_USAGE</code> error code in the Errors table.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.fieldId</code>	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields .	2015.2

Errors

Error Code	Thrown If
<code>SSS_INVALID_API_USAGE</code>	Invoked in certain cases when standard mode is being used. For example, if Record.isDynamic is set to false, the <code>SSS_INVALID_API_USAGE</code> error can be invoked in the following situations: ■ If the record object was created by <code>record.create()</code> or <code>record.transform()</code>, and the script attempts to use <code>getText()</code> without first using <code>setText()</code> for the same field. ■ The record object was created by <code>record.copy()</code> or <code>record.load()</code>, and the script uses <code>setValue()</code> on a field before using <code>getText()</code> for the same field. Similar guidance affects user event scripts that instantiate records by using the <code>newRecord</code> or <code>oldRecord</code> object provided by the script context. In these cases, standard mode is always used. For that reason, the

Error Code	Thrown If
	SSS_INVALID_API_USAGE error appears when a user event executes on one of these objects in the following situations: - When the script executes on a record that is being created, and the script attempts to use <code>getText()</code> without first using <code>setText()</code> for the same field. - When the script executes on an existing record or on a record being created through copying, and the script uses <code>setValue()</code> on a field before using <code>getText()</code> for the same field.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Syntax Sample 1
2 // Add additional code
3 ...
4 var fieldidname = objRecord.getText({
5     fieldId: 'item'
6 });
7 ...
8 // Add additional code
9
10 // Syntax Sample 2
11 // Add additional code
12 ...
13 myString = 'Date is: ' + record.getText({fieldId: 'datechanged'});
14 // "Date is: 3/27/2017 9:55:38am"
15 ...
16 // Add additional code

```

Record.getValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a field. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Gets a numeric value for rate and ratehighprecision fields. The following code sample demonstrates <code>getValue()</code> method on a multiselect field: <pre> 1 var value = objRecord.getValue({ 2 fieldId: 'customer' 3 }); </pre> In this example, <code>getValue()</code> returns an array of customer IDs (for example, ['11', '15', '23', '46'])
Returns	number Date string array boolean

	Returns a JavaScript Date object for date/time field queries. To return a string for date/time field queries, use Record.getText(options). Date/time fields: DATE, DATETIME, DATETIMETZ, TIMEOFDAY.  Note: If the returned date object is implicitly converted to a string, the value is converted using the browser's setting for time zone. All fields of type CHECKBOX return true or false. All multiselect fields return an array of values.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_API_USAGE	Invoked in standard mode, if you use setText or getText on a field and use getValue on the same field.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/record Module Script Samples .

```

1 // Add additional code
2 ...
3 var value = objRecord.getValue({
4     fieldId: 'item'
5 });
6 ...

```

```
7 // Add additional code
```

Record.hasCurrentSublistSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a value indicating whether the associated sublist field has a subrecord on the current line. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a subrecord. See, Finding Internal IDs of Record Fields .	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var hasSubrecord = objRecord.hasCurrentSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 // Add additional code
```

Record.hasSublistSubrecord(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a value indicating whether the associated sublist field contains a subrecord. This method is not available for subrecords. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2015.2
options.fieldId	string	required	The internal ID of a subrecord. See, Finding Internal IDs of Record Fields.	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var hasSubrecord = objRecord.hasSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3
7 });
8 ...
9 // Add additional code

```


Record.hasSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a value indicating whether the field contains a subrecord. This method is not available for subrecords. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of the field that may contain a subrecord. See, Finding Internal IDs of Record Fields .	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var hasSubrecord = objRecord.hasSubrecord({
4   fieldId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.insertLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a sublist line.
---------------------------	-------------------------

	When you insert a line with this method, all succeeding lines are moved and the total line count is increased. Essentially, succeeding lines are committed to a new sublist line with a new line number. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.line	number	required	The line number to insert. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2
options.ignoreRecalc	boolean	optional	If set to true, scripting recalculation is ignored. The default value is false.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example that uses insertLine(), see the help topic [Creating a Landed Cost Sublist Subrecord Example](#).

```
1 | // Add additional code
```

```

2  ...
3  objRecord.insertLine({
4      sublistId: 'attendee',
5      line: 2,
6  });
7  objRecord.setCurrentSublistValue({
8      sublistId: 'attendee',
9      fieldId: 'attendee',
10     value: 838
11  });
12  objRecord.commitLine({
13      sublistId: 'attendee'
14  });
15  ...
16  // Add additional code

```

For script examples that use other N/record methods, see [N/record Module Script Samples](#).

Record.moveLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Moves one line of the sublist to another location. The sublist machine must allow moving lines, for example: <code>editmachine.setAllowMoveLines(true);</code>. The sublist must contain the <code>_sequence</code> field. The sublist type must be <code>edit machine</code>. When using this method, the order of the other lines is preserved.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.sublistId</code>	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2020.2
<code>options.from</code>	number	required	The line number to move from. Note that line indexing begins at 0 with SuiteScript 2.x.	2020.2

Parameter	Type	Required / Optional	Description	Since
options.to	number	required	The line number to move to. Note that line indexing begins at 0 with SuiteScript 2.x.	2020.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.
SSS_NOT_YET_SUPPORTED	You are trying to move other than the current line.
SSS_SUBLIST_DOESNT_SUPPORT_MOVING_LINES	Moving of lines is not supported.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.moveLine({
4     sublistId: 'attendee',
5     from: 0,
6     to: 1
7 });
8
9 objRecord.commitLine({
10     sublistId: 'attendee'
11 });
12 ...
13 // Add additional code

```

Record.removeCurrentSublistSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the subrecord for the associated sublist field on the current line. This method is not available for subrecords. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.removeCurrentSublistSubrecord({
4     sublistId: 'item',
5     fieldId: 'item'
6 });
7 ...
8 // Add additional code

```

Record.removeLine(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes a sublist line. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.line	number	required	The line number of the sublist to remove. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2
options.ignoreRecalc	boolean	optional	If set to true, scripting recalculation is ignored. The default value is false.	2015.2
options.lineInstanceId	string	optional	The line instance ID. Use this parameter to specify where to remove the line.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 // The following code sample assumes that there are at least three committed lines present in the
3 // item sublist.
4 ...
5 objRecord.removeLine({
6     sublistId: 'item',
7     line: 3,
8     ignoreRecalc: true
9 });
10 ...
11 // Add additional code

```

Record.removeSublistSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the subrecord for the associated sublist field. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) When working in dynamic mode, remove a sublist subrecord using the following methods: 1. Record.selectLine(options) 2. Record.hasCurrentSublistSubrecord(options) 3. Record.removeCurrentSublistSubrecord(options) 4. Record.commitLine(options)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields.	2015.2
options.line	number	required	The line number in the sublist that contains the subrecord to remove. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```
1 | // Add additional code
```

```

2 ...
3 objRecord.removeSublistSubrecord({
4     sublistId: 'item',
5     fieldid: 'item',
6     line: 3
7 });
8 ...
9 // Add additional code

```

Record.removeSubrecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the subrecord for the associated field. This method is not available for subrecords. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields .	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.removeSubrecord({
4     fieldid: 'item'
5 });
6 ...
7 // Add additional code

```


Record.save(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits a new record or saves edits to an existing record. When working with records in standard mode, you must submit and then load the record to obtain sourced, validated, and calculated field values. This method is not available to subrecords.  Note: This method has an asynchronous counterpart you can use with client scripts. See Record.save.promise(options).
Returns	A number representing the internal ID of the new or updated record.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 20 units Custom records: 4 units All other records: 10 units
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.enableSourcing	boolean	optional	Enables sourcing during the record update. If set to true, sources dependent field information for empty fields. Defaults to false – dependent field values are not sourced.  Important: This parameter applies to records in standard mode only. When working with records in dynamic mode, field values are always sourced and the value you provide for enableSourcing is ignored. See the help topic SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.ignoreMandatoryFields	boolean	optional	Disables required field validation for this save operation. If set to true, all standard and custom fields that are required through customization are ignored. All fields	2015.2

Parameter	Type	Required / Optional	Description	Since
			that are required through company preferences are also ignored. By default, this parameter is false.  Important: Use the <code>ignoreMandatoryFields</code> argument with caution. This argument should be used mostly with Scheduled scripts, rather than User Event scripts. This ensures that UI users do not bypass the business logic enforced through form customization.	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordId = objRecord.save({
4     enableSourcing: true,
5     ignoreMandatoryFields: true
6 });
7 ...
8 // Add additional code

```

Record.save.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits a new record asynchronously or saves edits to an existing record asynchronously. This method is not available to subrecords.  Note: The parameters and errors thrown for this method are the same as those for Record.save(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Record.save(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 20 units Custom records: 4 units All other records: 10 units
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.enableSourcing	boolean	optional	Enables sourcing during the record update. If set to true, sources dependent field information for empty fields. Defaults to false – dependent field values are not sourced.  Important: This parameter applies to records in standard mode only. When working with records in dynamic mode, field values are always sourced and the value you provide for enableSourcing is ignored. See the help topic SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.ignoreMandatoryFields	boolean	optional	Disables required field validation for this save operation. If set to true, all standard and custom fields that are required through customization are ignored. All fields that are required through company preferences are also ignored. By default, this parameter is false.  Important: Use the ignoreMandatoryFields argument with caution. This argument should be used mostly with Scheduled scripts, rather than User Event scripts. This ensures that UI users do not bypass the business logic enforced through form customization.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 var recordId = objRecord.save.promise({
4   enableSourcing: true,
5   ignoreMandatoryFields: true
6 });
7 ...
8 // Add additional code

```

Record.selectLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Selects an existing line in a sublist. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) When working in standard mode, set a sublist field using Record.setSublistValue(options) .
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.line	number	required	The line number to select in the sublist. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var lineNum = objRecord.selectLine({

```

```

4     sublistId: 'item',
5     line: 3
6   });
7   ...
8   // Add additional code

```

Record.selectNewLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Selects a new line at the end of a sublist. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...

```

```

3 objRecord.selectNewLine({
4     sublistId: 'item'
5 });
6 ...
7 // Add additional code

```

Record.setCurrentMatrixSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the line currently selected in the matrix. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Sets a string value with a "%" for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields.	2015.2
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: Text, Radio and Select fields accept string values.	2015.2

Parameter	Type	Required / Optional	Description	Since
			- Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setCurrentMatrixSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 3,
7     value: false,
8     ignoreFieldChange: true,
9     forceSyncSourcing: true
10 });
11 ...
12 // Add additional code

```

Record.setCurrentSublistText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the field in the currently selected line by a text representation. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Sets a string value with a "%" for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.text	string	required	The text to set the value to.	2015.2
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
A_SCRIPT_IS_ATTEMPTING_TO_EDIT_THE_1_SUBLIST_THIS_SUBLIST_IS_CURRENTLY_IN_READONLY_MODE_AND_CANNOT_BE_EDITED_CALL_YOUR_NETSUITE_ADMINISTRATOR_TO_DISABLE_THIS_SCRIPT_IF_YOU_NEED_TO_SUBMIT_THIS_RECORD	A user tries to edit a read-only sublist field.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setCurrentSublistText({
4     sublistId: 'item',
5     fieldId: 'item',
6     text: 'value',
7     ignoreFieldChange: true
8 });
9 ...
10 // Add additional code

```


Record.setCurrentSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the field in the currently selected line. (dynamic mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) When working in standard mode, set a sublist field using Record.setSublistValue(options).  Important: When you edit a sublist line with SuiteScript, it triggers an internal validation of the sublist line. If the line validation fails, the script also fails. For example, if your script edits a closed catch up period, the validation fails and prevents SuiteScript from editing the closed catch up period. Sets a numeric value for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields.	2015.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: ■ Text, Radio and Select fields accept string values. ■ Checkbox fields accept boolean values. ■ Date and DateTime fields accept Date values. ■ Integer, Float, Currency and Percent fields accept number values.	2015.2

Parameter	Type	Required / Optional	Description	Since
			 Note: There may be a few exceptions to value and field type matching in SuiteScript. For example, the Role field on the Resource subtab of project records is a multi-select field. However, in SuiteScript, this field can only accept one internal ID and does not accept an array of values. The only workaround to select multiple values for this field is to use the NetSuite UI.	
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2015.2
options.forceSyncSourcing	boolean	optional	Indicates whether to perform field sourcing synchronously. This parameter can be used to alleviate a timing situation that may occur in some browsers when fields are sourced. For some browsers, some APIs are triggered without waiting for the field sourcing to complete which could cause incorrect results. If set to true, sources dependent field information for empty fields synchronously. Defaults to false – dependent field values are not sourced synchronously.	2019.1

Errors

Error Code	Thrown If
A_SCRIPT_IS_ATTEMPTING_TO_EDIT_THE_1_SUBLIST_THIS_SUBLIST_IS_CURRENTLY_IN_READONLY_MODE_AND_CANNOT_BE_EDITED_CALL_YOUR_NETSUITE_ADMINISTRATOR_TO_DISABLE_THIS_SCRIPT_IF_YOU_NEED_TO_SUBMIT_THIS_RECORD	A user tries to edit a read-only sublist field.
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setCurrentSublistValue({
4   sublistId: 'item',
5   fieldId: 'item',
6   value: true,
7   ignoreFieldChange: true
8 });

```

```

9 | ...
10 // Add additional code

```

Record.setMatrixHeaderValue(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the associated header in the matrix. (dynamic and standard modes — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Sets a numeric value for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields.	2015.2
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values.	2015.2

Parameter	Type	Required / Optional	Description	Since
			■ Date and DateTime fields accept Date values. ■ Integer, Float, Currency and Percent fields accept number values.	
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setMatrixHeaderValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     column: 3,
7     value: false,
8     ignoreFieldChange: true,
9     forceSyncSourcing: true
10 });
11 ...
12 // Add additional code

```

Record.setMatrixSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value for the associated field in the matrix. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Sets a numeric value for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module

Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist that contains the matrix. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of the matrix field. See, Finding Internal IDs of Record Fields .	2015.2
options.column	number	required	The column number for the field. Note that column indexing begins at 0 with SuiteScript 2.x.	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	2015.2

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```
1 | // Add additional code
```

```

2  ...
3  objRecord.setMatrixSublistValue({
4      sublistId: 'item',
5      fieldId: 'item',
6      column: 12,
7      line: 3,
8      value: true
9  });
10 ...
11 // Add additional code

```

Record.setSublistText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a sublist field by a text representation. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) When working in dynamic mode, set a sublist field text using the following methods: 1. Record.selectLine(options) 2. Record.setCurrentSublistText(options) 3. Record.commitLine(options) Sets a string value with a "%" for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser.	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields.	2015.2
options.line	number	required	The line number for the field. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.text	string	required	The text to set the value to.	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setSublistText({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3,
7     text: 'value'
8 });
9 ...
10 // Add additional code

```

Record.setSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a sublist field. (standard mode only — see the help topic SuiteScript 2.x Standard and Dynamic Modes) When working in dynamic mode, set a sublist field value using the following methods: ■ Record.selectLine(options) ■ Record.setCurrentSublistValue(options) ■ Record.commitLine(options) Important: When you edit a sublist line with SuiteScript, it triggers an internal validation of the sublist line. If the line validation fails, the script also fails. For example, if your script edits a closed catch up period, the validation fails and prevents SuiteScript from editing the closed catch up period. Sets a numeric value for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublistId	string	required	The internal ID of the sublist. This value is displayed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .	2015.2
options.fieldId	string	required	The internal ID of a standard or custom sublist field. See, Finding Internal IDs of Record Fields .	2015.2
options.line	number	required	The line number of the sublist. Note that line indexing begins at 0 with SuiteScript 2.x.	2015.2
options.value	number Date string array boolean	required	The value to set the sublist field to. The value type must correspond to the field type being set. For example: - Text, Radio and Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values. Note: There may be a few exceptions to value and field type matching in SuiteScript. For example, the Role field on the Resource subtab of project records is a multi-select field. However, in SuiteScript, this field can only accept one internal ID and does not accept an array of values. The only workaround to select multiple values for this field is to use the NetSuite UI.	2015.2

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_INVALID_SUBLIST_OPERATION	A required argument is invalid or the sublist is not editable.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setSublistValue({
4     sublistId: 'item',
5     fieldId: 'item',
6     line: 3,
7     value: true
8 });
9 ...
10 // Add additional code

```

Record.setText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of the field by a text representation. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes) Sets a string value with a "%" for rate and ratehighprecision fields.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields.	2015.2
options.text	string array	required	The text or texts to change the field value to. ■ If the field type is multiselect: □ This parameter accepts an array of string values.	2015.2

Parameter	Type	Required / Optional	Description	Since
			□ This parameter accepts a null value. Passing in null deselects all currently selected values. ■ If the field type is not multiselect, this parameter accepts only a single string value. ■ Inline HTML fields accept strings. Strings containing HTML tags are represented as strings in UI, this is demonstrated in Syntax section below.	
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setText({
4   fieldId: 'item',
5   text: 'value',
6   ignoreFieldChange: true
7 });
8 ...
9 // Add additional code

```



Note: The following code sample shows the syntax for INLINEHTML fields and what is returned.

```

1 ...
2 objRecord.setText(inlineHtmlFieldId, '<i>foo</i>'); // Returns nonformatted text with the exact
3 // form given, i.e. '<i>foo</i>'
4 objRecord.getValue(inlineHtmlFieldId); // Returns '&lt;i&gt;foo&lt;/i&gt;'
5 objRecord.getText(inlineHtmlFieldId); // Returns '<i>foo</i>'
6 ...

```

Record.setValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a field. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
---------------------------	--

	 Note: For rate and ratehighprecisionfields, use the Record.setText(options) method to set a string value with a "%".
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of a standard or custom body field. See, Finding Internal IDs of Record Fields .	2015.2
options.value	number Date string array boolean	required	The value to set the field to. The value type must correspond to the field type being set. For example: - Text and Radio fields accept string values. - Select fields accept string and number values. - Multi-Select fields accept arrays of string or number values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values. - Inline HTML fields accept strings. Strings containing HTML tags are represented as HTML entities in UI, this is demonstrated in Syntax section below. For rate and ratehighprecisionfields, use the Record.setText(options) method to set a string value with a "%".	2015.2
options.ignoreFieldChange	boolean	optional	If set to true, the field change and the secondary event is ignored. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
INVALID_FLD_VALUE	The options.value type does not match the field type.
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 objRecord.setValue({
4     fieldId: 'item',
5     value: true,
6     ignoreFieldChange: true
7 });
8 ...
9 // Add additional code

```



Note: The following code sample shows the syntax for INLINEHTML fields and what is returned.

```

1 ...
2 objRececord.setValue(inlineHtmlFieldId, '<i>foo</i>'); // Returns text in cursive.
3 objRececord.getValue(inlineHtmlFieldId); // Returns '<i>foo</i>'
4 objRececord.getText(inlineHtmlFieldId); // Returns 'foo'
5 ...

```

Record.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of a specific record. This property is not available to subrecords. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Type	number (read-only)
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code

```

```

2 ...
3     var myNewRecord = record.create({
4         type: record.Type.CUSTOMER,
5         isDynamic: true
6     });
7     var myNewRecord_id = myNewRecord.id;
8 ...
9 // Add additional code

```

Record.isDynamic

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the record is in dynamic or standard mode. ■ If set to true, the record is currently in dynamic mode. If set to false, the record is currently in standard mode. □ When a SuiteScript 2.x script creates, copies, loads, or transforms a record in standard mode, the record's body fields and sublist line items are not sourced, calculated, and validated until the record is saved (submitted) with Record.save(options). When you work with a record in standard mode, you do not need to set values in any particular order. After submitting the record, NetSuite processes the record's body fields and sublist line items in the correct order, regardless of the organization of your script. □ When a SuiteScript 2.x script creates, copies, loads, or transforms a record in dynamic mode, the record's body fields and sublist line items are sourced, calculated, and validated in real-time. A record in dynamic mode emulates the behavior of a record in the UI. When you work with a record in dynamic mode, it is important that you set values in the same order you would within the UI. If you fail to do this, your results may not be accurate. This value is set when the record is created or accessed. (dynamic and standard mode — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Type	boolean (read-only)
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 if (record.isDynamic) {
4     // add processing for dynamic records
5 }
6 ...
7 // Add additional code

```

Record.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The record type. Note the following: - When working with an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When working with an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record. This property is not available to subrecords. (dynamic and standard modes — see the help topic SuiteScript 2.x Standard and Dynamic Modes)
Type	string (read-only)
Module	N/record Module
Sibling Object Members	Record Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Start the process of creating an employee record.
4 var employeeRecord = record.create({
5     type: record.Type.EMPLOYEE,
6     isDynamic: true
7 });
8
9 // Start the process of creating an instance of a custom record type.
10 var customRecord = record.create({
11     type: 'customrecord_book',
12     isDynamic: true
13 });
14 ...
15 // Add additional code

```

Note: Supported standard record types are described in the [SuiteScript Records Browser](#). Refer also to [SuiteScript Supported Records](#). For help working with custom record types, see the help topic [Custom Record](#).

record.Sublist

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a sublist on a standard or custom record.
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/record Module
Methods and Properties	Sublist Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11 if(objSublist.type === 'inlineeditor'){
12     //Perform an action
13 }
14 ...
15 // Add additional code

```

Sublist.getColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a column in the sublist.
Returns	record.Column
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/record Module
Sibling Object Members	Sublist Object Members
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fieldId	string	required	The internal ID of the column field in the sublist.	2015.2

Parameter	Type	Required / Optional	Description	Since
			See, Finding Internal IDs of Record Fields .	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

Example 1

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11
12 var objColumn = objSublist.getColumn({
13     fieldId: 'item'
14 });
15
16 if(objColumn.type === 'checkbox'){
17     //Perform an action
18 }
19 ...
20 // Add additional code

```

Example 2

This example loops through each line of the items sublist on a sales order record.

```

1 // Add additional code
2 ...
3 onRequest: function(context) {
4     var recordObj = record.create({type: record.Type.SALES_ORDER});
5     var columnList = recordObj.getSublistFields({sublistId: 'item'});
6     var sublistObj = recordObj.getSublist({sublistId: 'item'});
7
8     for (var i = 0; i < columnList.length; i++) {
9         var columnId = columnList[i];
10        var columnObj = sublistObj.getColumn({fieldId: columnId});
11        if (columnObj !== null) {
12            log.debug('[Column id] = ' + columnObj.id + ' [Column type] = ' + columnObj.type +
13                ' [Column label] = ' + columnObj.label);
14        }
15    }
16 }
17 ...
18 // Add additional code

```

Sublist.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the internal ID of the sublist.
----------------------	---

Type	string (read-only)
Module	N/record Module
Sibling Object Members	Sublist Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11 //Perform an action with the objSublist.id value
12 ...
13 // Add additional code

```

Sublist.isChanged

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the sublist has changed on the record form.
Type	boolean (read-only)
Module	N/record Module
Sibling Object Members	Sublist Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });

```

```

11
12 if(objSublist.isChanged){
13     //Perform an action when objSublist.isChanged is true
14 }
15 ...
16 // Add additional code

```

Sublist.isDisplay

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the sublist is displayed on the record form.
Type	boolean
Module	N/record Module
Sibling Object Members	Sublist Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4     type: record.Type.SALES_ORDER,
5     id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9     sublistId: 'item'
10 });
11
12 if(objSublist.isDisplay){
13     //Perform an action when objSublist.isDisplay is true
14 }
15 ...
16 // Add additional code

```

Sublist.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the sublist type. For more information about sublist types, see serverWidget.SublistType. Important: Sublist.type will return a lower case string representing the sublist type. For example, inlineeditor not INLINEEDITOR.
Type	string (read-only)

Module	N/record Module
Sibling Object Members	Sublist Object Members
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.load({
4   type: record.Type.SALES_ORDER,
5   id: 275
6 });
7
8 var objSublist = objRecord.getSublist({
9   sublistId: 'item'
10 });
11
12 if(objSublist.type === 'inlineeditor'){
13   //Perform an action
14 }
15 ...
16 // Add additional code

```

record.attach(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Attaches a record to another record.  Note: For the promise version of this method, see record.attach.promise(options) . Note that promises are only supported in client scripts.
Returns	void
Governance	10 units
Module	N/record Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.record	record.Record	required	The record to attach.	2015.2

Parameter	Type	Required / Optional	Description	Since
			 Note: The record must be active.	
options.record.type	string	required	The type of record to attach. Set this value using the record.Type enum. To attach a file from the File Cabinet to a record, set type to file.	2015.2
options.record.id	number string	required	The internal ID of the record to attach.	2015.2
options.to	record.Record	required	The record that the options.record gets attached to.	2015.2
options.to.type	string	required	The record type of the record to attach to. Set the value using the record.Type enum. To attach a file from the File Cabinet to a record, set type to file.	2015.2
options.to.id	number string	required	The internal ID of the record to attach to.	2015.2
options.attributes	Object	optional	The name-value pairs containing attributes for the attachment. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var id = record.attach({
4   record: {
5     type: 'file',
6     id: '200'
7   },
8   to: {

```

```

9      type: 'customer',
10     id: '90'
11   }
12 });
13 ...
14 // Add additional code

```

```

1 record.attach({
2   record: {
3     type: 'contact',
4     id: 2
5   },
6   to: {
7     type: 'customer',
8     id: 3
9   },
10  attributes: {
11    role: 3
12  }
13 });

```

record.attach.promise(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Attaches a record asynchronously to another record. i Note: The parameters and errors thrown for this method are the same as those for record.attach(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	record.attach(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/record Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.record	record.Record	required	The record to attach.	2015.2
options.record.type	string	required	The type of record to attach.	2015.2

Parameter	Type	Required / Optional	Description	Since
			Set the value using the record.Type enum. To attach a file from the File Cabinet to a record, set type to file.	
options.record.id	number string	required	The internal ID of the record to attach.	2015.2
options.to	record.Record	required	The record that the options.record gets attached to.	2015.2
options.to.type	string	required	The record type of the record to attach to. Set the value using the record.Type enum. To attach a file from the File Cabinet to a record, set type to file.	2015.2
options.to.id	number string	required	The internal ID of the record to attach to.	2015.2
options.attributes	Object	optional	The name-value pairs containing attributes for the attachment. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 function attachRecord() {
4     var attachRecordPromise = record.attach.promise({
5         record: {
6             type: record.Type.CONTACT,
7             id: '97'
8         },
9         to: {
10            type: record.Type.OPPORTUNITY,
11            id: '16'
12        }
13    });
14
15    attachRecordPromise.then(function() {
16        // Add any other needed logic that shouldn't execute until

```

```

17 // after the contact record is attached to the opportunity
18 log.debug({
19     title: 'Record updated',
20     details: 'Attachment successful'
21 });
22
23 }, function(e) {
24     log.error({
25         title: e.name,
26         details: e.message
27     });
28 });
29 }
30 ...
31 // Add additional code

```

record.copy(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new record by copying an existing record in NetSuite.  Note: For the promise version of this method, see record.copy.promise(options). Note that promises are only supported in client scripts.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When copying an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When copying an instance of a custom record type, set this value by using the	2015.2

Parameter	Type	Required / Optional	Description	Since
			custom record type's string ID. For help finding this ID, see the help topic Custom Record .	
options.id	number	required	The internal ID of the existing record instance in NetSuite.	2015.2
options.isDynamic	boolean	optional	Determines whether the new record is created in dynamic mode. ■ If set to true, the new record is created in dynamic mode. ■ If set to false, the new record is created in standard mode. By default, this value is false.  Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null. For a list of available record default values, see N/record Default Values in the NetSuite Help Center.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.copy({
4   type: record.Type.SALES_ORDER,
5   id: 157,
6   isDynamic: true,
7   defaultValues: {
8     entity: 107
9   }
10 });
11 ...
12 // Add additional code

```


record.copy.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new record asynchronously by copying an existing record in NetSuite.  Note: The parameters and errors thrown for this method are the same as those for record.copy(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	record.copy(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When copying an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When copying an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	2015.2
options.id	number	required	The internal ID of the existing record instance in NetSuite.	2015.2
options.isDynamic	boolean	optional	Determines whether the new record is created in dynamic mode. - If set to true, the new record is created in dynamic mode. - If set to false, the new record is created in standard mode. By default, this value is false.	2015.2

Parameter	Type	Required / Optional	Description	Since
			 Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes .	
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 function copyRecord() {
4
5     // Copy an instance of a standard record type.
6     var copyRecordPromise = record.copy.promise({
7         type: record.Type.PHONE_CALL,
8         id: 165
9     });
10
11     // Note: To copy an instance of a custom record type,
12     // use the record type's string ID instead of the record
13     // module's Type enum. For example:
14     // type: 'customrecord_feature'
15
16     copyRecordPromise.then(function(recordObject) {
17         recordObject.setValue({
18             fieldId: 'title',
19             value: 'Sprint 5 bug triage'
20         });
21
22         recordObject.setValue({
23             fieldId: 'message',
24             value: 'Please review the PowerPoint prior to the call.'
25         });
26
27         var recordId = recordObject.save();
28
29         // Add any other needed logic that shouldn't execute until
30         // after the record is copied
31         log.debug({
32             title: 'Record saved',
33             details: 'Id of new record: ' + recordId
34         });
35
36     }, function(e) {
37         log.error({
38             title: e.name,
39             details: e.message

```

```

40     });
41     });
42 }
43 ...
44 // Add additional code

```

record.create(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new record. When you create records of certain types, you can use the <code>defaultValues</code> parameter to provide default values for fields in the new record. For some record types, some default values are required. For example, when you create a script deployment record, you must provide the internal ID of an existing script in your account (to associate with the script deployment record): <pre> 1 var scriptDeployment = record.create({ 2 type: record.Type.SCRIPT_DEPLOYMENT, 3 defaultValues: { 4 script: 205 // Internal id of existing script record 5 } 6 }); </pre> Make sure that you check for supported default values as you create records. For a list of available record default values, see N/record Default Values. i Note: For the promise version of this method, see record.create.promise(options). Note that promises are only supported in client scripts.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

i Note: The `options` parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.type</code>	string	required	The record type. This value determines the Record.type property of the record that is created. This property is read-only on an existing record.	2015.2

Parameter	Type	Required / Optional	Description	Since
			Use the following guidelines: - When creating an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When creating an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	
options.isDynamic	boolean	optional	Determines whether the new record is created in dynamic mode. - If set to true, the new record is created in dynamic mode. - If set to false, the new record is created in standard mode. By default, this value is false.  Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null. For a list of available record default values, see N/record Default Values in the NetSuite Help Center.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // Start the process of creating a sales order record.
5 var objRecord = record.create({
6   type: record.Type.SALES_ORDER,
7   isDynamic: true,
8   defaultValues: {
9     entity: 87
10  }

```

```

11 });
12
13 // Start the process of creating an instance of a custom record type.
14 var customRecord = record.create({
15     type: 'customrecord_feature',
16     isDynamic: true
17 });
18
19 // Create a record with two default values
20 record.create({
21     type: record.Type.SALES_ORDER,
22     defaultValues: {
23         entity: 107,
24         subsidiary: 1
25     }
26 });
27 ...
28 // Add additional code

```

record.create.promise(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new record asynchronously.
	i Note: The parameters and errors thrown for this method are the same as those for record.create(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	record.create(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. This value determines the Record.type property of the record that is created. This property is read-only on an existing record. Use the following guidelines:	2015.2

Parameter	Type	Required / Optional	Description	Since
			- When creating an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When creating an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	
options.isDynamic	boolean	optional	Determines whether the new record is created in dynamic mode. - If set to true, the new record is created in dynamic mode. - If set to false, the new record is created in standard mode. By default, this value is false.  Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 function createRecord() {
4   // Create an instance of a standard record record type.
5   var createRecordPromise = record.create.promise({
6     type: record.Type.PHONE_CALL,
7     isDynamic: true
8   });
9
10  // Note: To create an instance of a custom record type,
11  // use the record type's string ID instead of the record
12  // module's Type enum. For example:
13  // type: 'customrecord_feature'
14
15  createRecordPromise.then(function(objRecord) {
16    objRecord.setValue({
17      fieldId: 'title',

```

```

18     value: 'sprint planning'
19   });
20   var recordId = objRecord.save();
21   log.debug({
22     title: 'New record saved',
23     details: 'New record ID: ' + recordId
24   });
25
26   }, function(e) {
27     log.error({
28       title: 'Unable to create record',
29       details: e.name
30     });
31   });
32 }
33 ...
34 // Add additional code

```

record.delete(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes a record.  Note: For the promise version of this method, see record.delete.promise(options). Note that promises are only supported in client scripts.
Returns	The internal ID of the deleted record.Record .
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 20 units Custom records: 4 units All other records: 10 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When deleting an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When deleting an instance of a custom record type, set this value by using the custom record	2015.2

Parameter	Type	Required / Optional	Description	Since
			type's string ID. For help finding this ID, see the help topic Custom Record .	
options.id	number string	required	The internal ID of the record instance to be deleted.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Delete a sales order
4 var salesOrderRecord = record.delete({
5     type: record.Type.SALES_ORDER,
6     id: 88,
7 });
8
9 // Delete an instance of a custom record type with the ID customrecord_feature
10 var featureRecord = record.delete({
11     type: 'customrecord_feature',
12     id: 3,
13 });
14 ...
15 // Add additional code

```

record.delete.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes a record asynchronously.
	Note: The parameters and errors thrown for this method are the same as those for record.delete(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	record.delete(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 20 units Custom records: 4 units

	All other records: 10 units
Module	N/record Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When deleting an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When deleting an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	2015.2
options.id	number string	required	The internal ID of the record instance to be deleted.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2
3 // Delete an instance of a standard NetSuite record
4 ...
5 function deleteRecord() {
6     var deleteRecordPromise = record.delete.promise({
7         type: record.Type.PHONE_CALL,
8         id: 109
9     });
10
11 // To delete an instance of a custom record type, use
12 // the string ID in the type field. For example:
13 // type: 'customrecord_feature'
14
15 deleteRecordPromise.then(function() {
16     log.debug({
17         title: 'Success',
18         details: 'Record successfully deleted'
19     });
20
21 // Add any other needed code that should execute

```

```

22 // after the record is deleted.
23 }, function(e) {
24     log.error({
25         title: 'Unable to delete record',
26         details: e.name
27     });
28 });
29 }
30 ...
31 // Add additional code

```

record.detach(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Detaches a record from another record.
	Note: For the promise version of this method, see record.detach.promise(options). Note that promises are only supported in client scripts.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.record	record.Record	required	The record to be detached.	2015.2
options.record.type	string	required	The type of record to be detached. Set this value using the record.Type enum.	2015.2
options.record.id	number string	required	The ID of the record to be detached.	2015.2
options.from	record.Record	required	The destination record that options.record should be detached from.	2015.2
options.from.type	string	required	The type of the destination. Set this value using the record.Type enum.	2015.2
options.from.id	number string	required	The ID of the destination.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.attributes	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 record.detach({
4   record: {
5     type: 'file',
6     id: '200'
7   },
8   from: {
9     type: 'customer',
10    id: '90'
11  }
12 })
13 ...
14 // Add additional code

```

record.detach.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Detaches a record from another record asynchronously. Note: The parameters and errors thrown for this method are the same as those for record.detach(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	record.detach(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/record Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.record	record.Record	required	The record to be detached.	2015.2
options.record.type	string	required	The type of record to be detached. Set this value using the record.Type enum.	2015.2
options.record.id	number string	required	The ID of the record to be detached.	2015.2
options.from	record.Record	required	The destination record that options.record should be detached from.	2015.2
options.from.type	string	required	The type of the destination. Set this value using the record.Type enum.	2015.2
options.from.id	number string	required	The ID of the destination.	2015.2
options.attributes	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 function detachRecord() {
4     var detachRecordPromise = record.detach.promise({
5         record: {
6             type: record.Type.CONTACT,
7             id: '98'
8         },
9         from: {
10            type: record.Type.OPPORTUNITY,
11            id: '16'
12        }
13    });
14
15    detachRecordPromise.then(function() {
16
17        // Add any other needed logic that shouldn't execute until
18        // after the contact record is detached from the opportunity.

```

```

19
20     log.debug({
21         title: 'Record updated',
22         details: 'Contact record detached'
23     });
24
25 }, function(e) {
26     log.error({
27         title: e.name,
28         details: e.message
29     });
30 });
31 }
32 ...
33 // Add additional code

```

record.load(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing record. Note that the maximum number of lines on record's sublist is limited to 10,000, see the help topic Limits for Display of Transaction Lists and Sublists  Note: For the promise version of this method, see record.load.promise(options). Note that promises are only supported in client scripts. Make sure you save the record before loading it.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When loading an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When loading an instance of a custom record type, set this value by using the custom	2015.2

Parameter	Type	Required / Optional	Description	Since
			record type's string ID. For help finding this ID, see the help topic Custom Record .	
options.id	number	required	The internal ID of the existing record instance in NetSuite. The internal ID of the record is displayed on the list page for the record type.	
options.isDynamic	boolean	optional	Determines whether the record is loaded in dynamic mode. ■ If set to true, the record is loaded in dynamic mode. ■ If set to false, the record is loaded in standard mode. By default, this value is false.  Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null. For a list of available record default values, see N/record Default Values in the NetSuite Help Center.	

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // Load a sales order
5 var objRecord = record.load({
6   type: record.Type.SALES_ORDER,
7   id: 157,
8   isDynamic: true,
9 });
10
11 // Load an instance of a custom record type with the ID customrecord_feature.
12 var newFeatureRecord = record.load({
13   type: 'customrecord_feature',
14   id: 1,
15   isDynamic: true
16 });

```

```

17 | ...
18 | // Add additional code

```

record.load.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing record asynchronously. Note: The parameters and errors thrown for this method are the same as those for record.load(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	record.load(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When loading an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When loading an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	2015.2
options.id	number	required	The internal ID of the existing record instance in NetSuite. The internal ID of the record is displayed on the list page for the record type.	
options.isDynamic	boolean	optional	Determines whether the record is loaded in dynamic mode. If set to true, the record is loaded in dynamic mode.	2015.2

Parameter	Type	Required / Optional	Description	Since
			If set to false, the record is loaded in standard mode. By default, this value is false.  Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.	
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is empty.	

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 function loadRecord() {
2     // Load an instance of a standard NetSuite record type
3     var loadRecordPromise = record.load.promise({
4         type: record.Type.PHONE_CALL,
5         id: 712
6     });
7
8     // Note: To load an instance of a custom record type,
9     // use the record type's string ID. For example: type: 'customrecord_feature'
10
11     loadRecordPromise.then(function(objRecord) {
12         objRecord.setValue({
13             fieldId: 'message',
14             value: 'We will start the call with a restrospective.'
15         });
16         var recordId = objRecord.save();
17
18         // Add any other needed logic that shouldn't execute
19         // until after the record is instantiated.
20
21         log.debug({
22             title: 'Record updated',
23             details: 'Updated record ID: ' + recordId
24         });
25
26     }, function(e) {
27         log.error({
28             title: 'Unable to load record',
29             details: e.name
30         });
31     });
32 }
33 ...
34 // Add additional code

```


record.submitFields(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates and submits one or more body fields on an existing record in NetSuite, and returns the internal ID of the parent record. When you use this method, you do not need to load or submit the parent record. You can use this method to edit and submit the following: - Standard body fields that support inline editing (direct list editing). For more information, see the help topic Using Inline Editing. - Custom body fields that support inline editing. - Select and multi-select fields You cannot use this method to edit and submit the following: - Sublist line item fields - Subrecord fields (for example, address fields)  Note: For the promise version of this method, see record.submitFields.promise(options). Note that promises are only supported in client scripts.
Returns	The internal ID of the parent record.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When working with an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When working with an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.id	number string	required	The internal ID of the existing record instance in NetSuite.	2015.2
options.values	Object	required	The ID-value pairs for each field you want to edit and submit. The value type must correspond to the field type being set. For example: - Text, Radio, Select and Multi-Select fields accept string values. - Checkbox fields accept boolean values. - Date and DateTime fields accept Date values. - Integer, Float, Currency and Percent fields accept number values.	2015.2
options.options	Object	optional	Additional options to set for the record.	2015.2
options.options.enableSourcing	boolean	optional	Indicates whether to enable sourcing during the record update. By default, this value is true.	2015.2
options.options.ignoreMandatoryFields	boolean	optional	Indicates whether to ignore required fields during record submission. ignoreMandatoryFields generally works for both standard and custom fields with an exception for standard fields that were set as mandatory by default. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 // Submit a new value for a sales order's memo field.
5 var id = record.submitFields({
6     type: record.Type.SALES_ORDER,
7     id: 1,
8     values: {
9         memo: 'ABC'
10    },
11    options: {
12        enableSourcing: false,
13        ignoreMandatoryFields : true
14    }
15 });
16
17 // Submit a new value for a field on an instance of the 'customrecord_book' custom record type.
```

```

18 var otherId = record.submitFields({
19     type: 'customrecord_book',
20     id: '4',
21     values: {
22         'custrecord_rating': '2'
23     }
24 });
25
26 // Submit a record with both select and multi-select fields
27 var thirdID = record.submitFields({
28     type: record.Type.SALES_ORDER,
29     id: 21882,
30     values: {
31         'custbodycust_txt_fld_custso': 'Hello from custom field',
32         'memo': 'Hello from memo',
33         'leadsource': 254, //leadsource is a select field
34         'custbody_target_market': '1' // custbody_target_market is a custom multi-select field
35     }
36 });
37
38 ...
39 // Add additional code

```

record.submitFields.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates and submits one or more body fields asynchronously on an existing record in NetSuite, and returns the internal ID of the parent record. When you use this method, you do not need to load or submit the parent record. You can use this method to edit and submit the following: - Standard body fields that support inline editing (direct list editing). For more information, see the help topic Using Inline Editing. - Custom body fields that support inline editing. You cannot use this method to edit and submit the following: - Select fields - Sublist line item fields - Subrecord fields (for example, address fields) Note: The parameters and errors thrown for this method are the same as those for record.submitFields(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	record.submitFields(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 10 units Custom records: 2 units All other records: 5 units
Module	N/record Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	string	required	The record type. Use the following guidelines: - When working with an instance of a standard NetSuite record type, set this value by using the record.Type enum. - When working with an instance of a custom record type, set this value by using the custom record type's string ID. For help finding this ID, see the help topic Custom Record.	2015.2
options.id	number string	required	The internal ID of the existing record instance in NetSuite.	2015.2
options.values	Object	required	The ID-value pairs for each field you want to edit and submit.	2015.2
options.options	Object	optional	Additional options to set for the record.	2015.2
options.options.enableSourcing	boolean	optional	Indicates whether to enable sourcing during the record update. By default, this value is true.	2015.2
options.options.ignoreMandatoryFields	boolean	optional	Indicates whether to ignore required fields during record submission. By default, this value is false.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 function submitFields() {
4     var submitFieldsPromise = record.submitFields.promise({
5         type: record.Type.PHONE_CALL,
6         id: 171,
7         values: {

```

```

8         title: 'Sprint 3 planning'
9     },
10    });
11
12    submitFieldsPromise.then(function(recordId) {
13        // Add any needed logic that shouldn't execute until
14        // after the new value is submitted
15        log.debug({
16            title: 'Record updated',
17            details: 'Id of updated record: ' + recordId
18        });
19
20    }, function(e) {
21        log.error({
22            title: e.name,
23            details: e.message
24        });
25    });
26 }
27 ...
28 // Add additional code

```

record.transform(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Transforms a record from one type into another, using data from an existing record. You can use this method to automate order processing, creating item fulfillment transactions and invoices off of orders. For a list of supported transformations, see Supported Transformation Types. i Note: For the promise version of this method, see record.transform.promise(options). Note that promises are only supported in client scripts.
Returns	record.Record
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	Transaction records: 10 units Custom records: 2 units All other record types: 5 units
Module	N/record Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.fromType	string	required	The record type of the existing record instance being transformed.	2015.2

Parameter	Type	Required / Optional	Description	Since
			This value sets the Record.type property for the record. This property is read-only and cannot be changed after the record is loaded. Set this value using the record.Type .	
options.fromId	number	required	The internal ID of the existing record instance being transformed.	2015.2
options.toType	string	required	The record type of the record returned when the transformation is complete.	2015.2
options.isDynamic	boolean	optional	Determines whether the new record is created in dynamic mode. ■ If set to true, the new record is created in dynamic mode. ■ If set to false, the new record is created in standard mode. By default, this value is false.  Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes.	2015.2
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null. For a list of available record default values, see N/record Default Values in the NetSuite Help Center.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.transform({
4   fromType: record.Type.CUSTOMER,
5   fromId: 107,
6   toType: record.Type.SALES_ORDER,
7   isDynamic: true,
8 });
9 ...
10 // Add additional code

```

```

1 // Add additional code
2 ...
3 record.transform({
4   fromType: 'salesorder',
5   fromId: 6,
6   toType: 'invoice',
7   defaultValues: {
8     billdate: '01/01/2019' } });
9 ...
10 // Add additional code

```

Supported Transformation Types

Original Record Name	Original Record Type	Transformed Record Name	Transformed Record Type
Assembly Build	assemblybuild	Assembly Unbuild	assemblyunbuild
Build/Assembly	assemblyitem	Assembly Build Note: You cannot use SuiteScript to create an Assembly Build for an Outsourcing Work Order.	assemblybuild
Build/Assembly	assemblyitem	Work Order	workorder
Cash Refund	cashrefund	Credit Memo	creditmemo
Cash Sale	cashsale	Cash Refund	cashrefund
Cash Sale	cashsale	Custom Sales Transaction	customsalestransaction
Cash Sale	cashsale	Return Authorization	returnauthorization
Customer	customer	Cash Sale	cashsale
Customer	customer	Customer Payment Note: When you use this transformation, do not use <code>Record.setValue(options)</code> to set the value of the customer field on the resulting Customer Payment record. This field is populated automatically during transformation, and you cannot specify a value for this field after the transformation is complete.	customerpayment
Customer	customer	Estimate	estimate
Customer	customer	Invoice	invoice
Customer	customer	Opportunity	opportunity
Customer	customer	Sales Order	salesorder
Customer	customer	Vendor	vendor
Customer Deposit	customerdeposit	Customer Refund	customerrefund
Customer Deposit	customerdeposit	Deposit Application	depositapplication
Customer Payment	customerpayment	Customer Refund	customerrefund
Customer Payment Authorization	customerpaymentauthoriza tion	Customer Deposit	customerdeposit
Customer Payment Authorization	customerpaymentauthoriza tion	Customer Payment	customerpayment

Original Record Name	Original Record Type	Transformed Record Name	Transformed Record Type
Custom Purchase Transaction	custompurchasetransaction	Custom Purchase Transaction	custompurchasetransaction
Custom Purchase Transaction	custompurchasetransaction	Vendor Bill	vendorbill
Custom Purchase Transaction	custompurchasetransaction	Vendor Credit	vendorcredit
Custom Purchase Transaction	custompurchasetransaction	Vendor Payment	vendorpayment
Custom Sales Transaction	customsalestransaction	Cash Refund	cashrefund
Custom Sales Transaction	customsalestransaction	Customer Payment	customerpayment
Custom Sales Transaction	customsalestransaction	Customer Refund	customerrefund
Custom Sales Transaction	customsalestransaction	Invoice	invoice
Custom Sale Transaction	customsalestransaction	Cash Sale	cashsale
Custom Sale Transaction	customsalestransaction	Credit Memo	creditmemo
Custom Sale Transaction	customsalestransaction	Custom Sale Transaction	customsalestransaction
Employee	employee	Expense Report	expensereport
Employee	employee	Time	timebill
Quote	estimate	Cash Sale	cashsale
Quote	estimate	Invoice	invoice
Quote	estimate	Sales Order	salesorder
Intercompany Transfer Order	intercompanytransferorder	Item Fulfillment	itemfulfillment
Intercompany Transfer Order	intercompanytransferorder	Item Receipt	itemreceipt
Invoice	invoice	Credit Memo	creditmemo
Invoice	invoice	Customer Payment	customerpayment
Invoice	invoice	Custom Sales Transaction	customsalestransaction
Invoice	invoice	Return Authorization	returnauthorization
Job	job	Cash Sale	cashsale
Job	job	Estimate	estimate
Job	job	Invoice	invoice
Job	job	Sales Order	salesorder
Lead	lead	Opportunity	opportunity

Original Record Name	Original Record Type	Transformed Record Name	Transformed Record Type
Opportunity	opportunity	Cash Sale	cashsale
Opportunity	opportunity	Quote	estimate
Opportunity	opportunity	Invoice	invoice
Opportunity	opportunity	Sales Order	salesorder
Prospect	prospect	Quote	estimate
Prospect	prospect	Opportunity	opportunity
Prospect	prospect	Sales Order	salesorder
Purchase Order	purchaseorder	Custom Purchase Transaction	custompurchasetransaction
Purchase Order	purchaseorder	Item Receipt Note: This transformation does not work for Drop Ship Purchase Order.	itemreceipt
Purchase Order	purchaseorder	Ownership Transfer	ownershiptransfer
Purchase Order	purchaseorder	Vendor Bill	vendorbill
Purchase Order	purchaseorder	Vendor Return Authorization	vendorreturnauthorization
Purchase Requisition	purchaserequisition	Purchase Order	purchaseorder
Request For Quote	requestforquote	Vendor Request For Quote	vendorrequestforquote
Return Authorization	returnauthorization	Cash Refund	cashrefund
Return Authorization	returnauthorization	Credit Memo	creditmemo
Return Authorization	returnauthorization	Custom Sales Transaction	customsalestransaction
Return Authorization	returnauthorization	Item Receipt	itemreceipt
Return Authorization	returnauthorization	Revenue Commitment Reversal Note: The return authorization must be approved and received for this transformation to work.	revenuecommitmentreversal
Revenue Contract	revenuecontract	Revenue Commitment	revenuecommitment
Sales Order	salesorder	Cash Sale	cashsale
Sales Order	salesorder	Custom Sales Transaction	customsalestransaction
Sales Order	salesorder	Fulfillment Request	fulfillmentrequest
Sales Order	salesorder	Invoice	invoice
Sales Order	salesorder	Item Fulfillment	itemfulfillment
Sales Order	salesorder	Return Authorization	returnauthorization
Sales Order	salesorder	Revenue Commitment	revenuecommitment
Sales Order	salesorder	Revenue Contract	revenuecontract

Original Record Name	Original Record Type	Transformed Record Name	Transformed Record Type
Sales Order	salesorder	Store Pickup Fulfillment	storepickupfulfillment
Transfer Order	transferorder	Item Fulfillment	itemfulfillment
Transfer Order	transferorder	Item Receipt	itemreceipt
Vendor	vendor	Customer	customer
Vendor	vendor	Purchase Order	purchaseorder
Vendor	vendor	Vendor Bill	vendorbill
Vendor	vendor	Vendor Payment	vendorpayment
Vendor Bill	vendorbill	Custom Purchase Transaction	custompurchasetransaction
Vendor Bill	vendorbill	Vendor Credit	vendorcredit
Vendor Bill	vendorbill	Vendor Payment	vendorpayment
Vendor Bill	vendorbill	Vendor Return Authorization	vendorreturnauthorization
Vendor Prepayment	vendorprepayment	Vendor Prepayment Application	vendorprepaymentapplication
Vendor Request For Quote	vendorrequestforquote	Purchase Contract	purchasecontract
Vendor Return Authorization	vendorreturnauthorization	Custom Purchase Transaction	custompurchasetransaction
Vendor Return Authorization	vendorreturnauthorization	Item Fulfillment	itemfulfillment
Vendor Return Authorization	vendorreturnauthorization	Vendor Credit	vendorcredit
Work Order	workorder	Assembly Build	assemblybuild
Work Order	workorder	Work Order Close	workorderclose
Work Order	workorder	Work Order Completion	workordercompletion
Work Order	workorder	Work Order Issue	workorderissue

record.transform.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Transforms a record from one type into another asynchronously, using data from an existing record. You can use this method to automate order processing, creating item fulfillment transactions and invoices off of orders. For a list of supported transformations, see Supported Transformation Types.
---------------------------	---

	 Note: The parameters and errors thrown for this method are the same as those for record.transform(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	record.transform(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	Transaction records: 10 units Custom records: 2 units All other record types: 5 units
Module	N/record Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.fromType</code>	string	required	The record type of the existing record instance being transformed. This value sets the Record.type property for the record. This property is read-only and cannot be changed after the record is loaded. Set this value using the record.Type .	2015.2
<code>options.fromId</code>	number	required	The internal ID of the existing record instance being transformed.	2015.2
<code>options.toType</code>	string	required	The record type of the record returned when the transformation is complete.	2015.2
<code>options.isDynamic</code>	boolean	optional	Determines whether the new record is created in dynamic mode. ■ If set to <code>true</code>, the new record is created in dynamic mode. ■ If set to <code>false</code>, the new record is created in standard mode. By default, this value is <code>false</code> .	2015.2

Parameter	Type	Required / Optional	Description	Since
			 Note: For additional information about standard and dynamic mode, see record.Record and SuiteScript 2.x Standard and Dynamic Modes .	
options.defaultValues	Object	optional	Name-value pairs containing default values of fields in the new record. By default, this value is null.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required argument is missing or undefined.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 function transformRecord() {
4     var transformRecordPromise = record.transform.promise({
5         fromType: record.Type.ESTIMATE,
6         fromId: 25,
7         toType: record.Type.SALES_ORDER,
8         isDynamic: true,
9     });
10
11     transformRecordPromise.then(function(recordObject) {
12         var recordId = recordObject.save();
13         // Add any other needed logic that shouldn't execute until
14         // after the record is transformed.
15         log.debug({
16             title: 'Record saved',
17             details: 'Id of new record: ' + recordId
18         });
19     }, function(e) {
20         log.error({
21             title: e.name,
22             details: e.message
23         });
24     });
25 }
26 ...
27 // Add additional code

```

record.Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported record types. Use this enum to set the value of the Record.type property in cases where you are working with an instance of a standard NetSuite record type. If you are working with an instance of a custom record type, you set the Record.type property by using the custom record type's string ID. For more help finding this ID, see the help topic Custom Record. Note: Some record types support default values. For more information, see record.create(options). For a list of available record default values, see N/record Default Values. The following code samples demonstrate the difference between working with a standard NetSuite record type and a custom record type: - A standard NetSuite record type <pre> 1 var objRecord = record.delete({ 2 type: record.Type.SALES_ORDER, 3 id: 128 4 }); </pre> - A custom record type <pre> 1 var objRecord = record.delete({ 2 type: 'customrecord_myrecord', 3 id: 22 4 }); </pre>
Module	N/record Module
Sibling Module Members	N/record Module Members
Since	2015.2

Note: You must adjust your account settings according to the record type used in your scripts to avoid potential errors.

Values

- ACCOUNT - ACCOUNTING_BOOK - ACCOUNTING_CONTEXT - ACCOUNTING_PERIOD - ADV_INTER_COMPANY_JOURNAL_ENTRY - ALLOCATION_SCHEDULE - AMORTIZATION_SCHEDULE - AMORTIZATION_TEMPLATE - ASSEMBLY_BUILD	- FULFILLMENT_REQUEST - GENERAL_TOKEN - GENERIC_RESOURCE - GIFT_CERTIFICATE - GIFT_CERTIFICATE_ITEM - GLOBAL_ACCOUNT_MAPPING - GLOBAL_INVENTORY_RELATIONSHIP - GL_NUMBERING_SEQUENCE - GOAL	- PLANNING_VIEW - PORTLET - PRICE_BOOK - PRICE_LEVEL - PRICE_PLAN - PRICING_GROUP - PROJECT_EXPENSE_TYPE - PROJECT_IC_CHARGE_REQUEST - PROJECT_TASK
---	---	---

■ ASSEMBLY_ITEM	■ IMPORTED_EMPLOYEE_EXPENSE	■ PROJECT_TEMPLATE
■ ASSEMBLY_UNBUILD	■ INBOUND_SHIPMENT	■ PROMOTION_CODE
■ AUTOMATED_CLEARING_HOUSE	■ INTERCOMP_ALLOCATION_SCHEDULE	■ PROSPECT
■ BALANCE_TRX_BY_SEGMENTS	■ INTER_COMPANY_JOURNAL_ENTRY	■ PURCHASE_CONTRACT
■ BILLING_ACCOUNT	■ INTER_COMPANY_TRANSFER_ORDER	■ PURCHASE_ORDER
■ BILLING_CLASS	■ INVENTORY_ADJUSTMENT	■ PURCHASE_REQUISITION
■ BILLING_RATE_CARD	■ INVENTORY_COST_REVALUATION	■ REALLOCATE_ITEM
■ BILLING_REVENUE_EVENT	■ INVENTORY_COUNT	■ RECEIVE_INBOUND_SHIPMENT
■ BILLING_SCHEDULE	■ INVENTORY_DETAIL	■ RESOURCE_ALLOCATION
■ BIN	■ INVENTORY_ITEM	■ RESTLET
■ BIN_TRANSFER	■ INVENTORY_NUMBER	■ RETURN_AUTHORIZATION
■ BIN_WORKSHEET	■ INVENTORY_STATUS	■ REVENUE_ARRANGEMENT
■ BLANKET_PURCHASE_ORDER	■ INVENTORY_STATUS_CHANGE	■ REVENUE_COMMITMENT
■ BOM	■ INVENTORY_TRANSFER	■ REVENUE_COMMITMENT_REVERSAL
■ BOM_REVISION	■ INVOICE	■ REVENUE_PLAN
■ BONUS	■ INVOICE_GROUP	■ REV_REC_SCHEDULE
■ BONUS_TYPE	■ ISSUE	■ REV_REC_TEMPLATE
■ BUDGET_EXCHANGE_RATE	■ ISSUE_PRODUCT	■ SALES_CHANNEL
■ BULK_OWNERSHIP_TRANSFER	■ ISSUE_PRODUCT_VERSION	■ SALES_ORDER
■ BUNDLE_INSTALLATION_SCRIPT	■ ITEM_ACCOUNT_MAPPING	■ SALES_ROLE
■ CALENDAR_EVENT	■ ITEM_COLLECTION	■ SALES_TAX_ITEM
■ CAMPAIGN	■ ITEM_COLLECTION_ITEM_MAP	■ SCHEDULED_SCRIPT
■ CAMPAIGN_RESPONSE	■ ITEM_DEMAND_PLAN	■ SCHEDULED_SCRIPT_INSTANCE
■ CAMPAIGN_TEMPLATE	■ ITEM_FULFILLMENT	■ SCRIPT_DEPLOYMENT
■ CARDHOLDER_AUTHENTICATION	■ ITEM_GROUP	■ SERIALIZED_ASSEMBLY_ITEM
■ CASH_REFUND	■ ITEM_LOCATION_CONFIGURATION	■ SERIALIZED_INVENTORY_ITEM
■ CASH_SALE	■ ITEM_PROCESS_FAMILY	■ SERVICE_ITEM
■ CHARGE	■ ITEM_PROCESS_GROUP	■ SHIP_ITEM
■ CHARGE_RULE	■ ITEM_RECEIPT	■ SOLUTION
■ CHECK	■ ITEM_REVISION	■ STATISTICAL_JOURNAL_ENTRY
■ CLASSIFICATION	■ ITEM_SUPPLY_PLAN	■ STORE_PICKUP_FULFILLMENT
■ CLIENT_SCRIPT	■ JOB	■ SUBSCRIPTION
■ CMS_CONTENT	■ JOB_STATUS	■ SUBSCRIPTION_CHANGE_ORDER
■ CMS_CONTENT_TYPE	■ JOB_TYPE	■ SUBSCRIPTION_LINE
■ CMS_PAGE	■ JOURNAL_ENTRY	■ SUBSCRIPTION_PLAN
■ COMMERCE_CATEGORY	■ KIT_ITEM	■ SUBSCRIPTION_TERM
■ COMPETITOR	■ LABOR_BASED_PROJECT_REVENUE_RULE	■ SUBSIDIARY
■ CONSOLIDATED_EXCHANGE_RATE	■ LEAD	■ SUBSIDIARY_SETTINGS
■ CONTACT	■ LOCATION	■ SUBTOTAL_ITEM
■ CONTACT_CATEGORY	■ LOT_NUMBERED_ASSEMBLY_ITEM	■ SUITELET
■ CONTACT_ROLE	■ LOT_NUMBERED_INVENTORY_ITEM	■ SUPPLY_CHAIN_SNAPSHOT
■ COST_CATEGORY	■ MANUFACTURING_COST_TEMPLATE	■ SUPPLY_CHAIN_SNAPSHOT_SIMULATION
■ COUPON_CODE	■ MANUFACTURING_OPERATION_TASK	■ SUPPLY_CHANGE_ORDER
■ CREDIT_CARD_CHARGE	■ MANUFACTURING_ROUTING	■ SUPPLY_PLAN_DEFINITION
■ CREDIT_CARD_REFUND	■ MAP_REDUCE_SCRIPT	■ SUPPORT_CASE
■ CREDIT_MEMO	■ MARKUP_ITEM	■ TASK
■ CURRENCY	■ MASSUPDATE_SCRIPT	■ TAX_ACCT

■ CUSTOM_PURCHASE	■ MEM_DOC	■ TAX_GROUP
■ CUSTOM_SALE	■ MERCHANDISE_HIERARCHY_LEVEL	■ TAX_PERIOD
■ CUSTOMER	■ MERCHANDISE_HIERARCHY_NODE	■ TAX_TYPE
■ CUSTOMER_CATEGORY	■ MERCHANDISE_HIERARCHY_VERSION	■ TERM
■ CUSTOMER_DEPOSIT	■ MESSAGE	■ TIME_BILL
■ CUSTOMER_MESSAGE	■ MFG_PLANNED_TIME	■ TIME_ENTRY
■ CUSTOMER_PAYMENT	■ NEXUS	■ TIME_OFF_CHANGE
■ CUSTOMER_PAYMENT_AUTHORIZATION	■ NON_INVENTORY_ITEM	■ TIME_OFF_PLAN
■ CUSTOM_RECORD	■ NOTE	■ TIME_OFF_REQUEST
■ CUSTOMER_REFUND	■ NOTE_TYPE	■ TIME_OFF_RULE
■ CUSTOMER_STATUS	■ OPPORTUNITY	■ TIME_OFF_TYPE
■ CUSTOMER_SUBSIDIARY_RELATIONSHIP	■ ORDER_RESERVATION	■ TIME_SHEET
■ CUSTOM_TRANSACTION	■ ORDER_SCHEDULE	■ TOPIC
■ DEPARTMENT	■ ORDER_TYPE	■ TRANSFER_ORDER
■ DEPOSIT	■ OTHER_CHARGE_ITEM	■ UNITS_TYPE
■ DEPOSIT_APPLICATION	■ OTHER_NAME	■ UNLOCKED_TIME_PERIOD
■ DESCRIPTION_ITEM	■ OTHER_NAME_CATEGORY	■ USAGE
■ DISCOUNT_ITEM	■ PARTNER	■ USEREVENT_SCRIPT
■ DOWNLOAD_ITEM	■ PARTNER_CATEGORY	■ VENDOR
■ EMAIL_TEMPLATE	■ PAYCHECK	■ VENDOR_BILL
■ EMPLOYEE	■ PAYCHECK_JOURNAL	■ VENDOR_CATEGORY
■ EMPLOYEE_CHANGE_REQUEST	■ PAYMENT_CARD	■ VENDOR_CREDIT
■ EMPLOYEE_CHANGE_REQUEST_TYPE	■ PAYMENT_CARD_TOKEN	■ VENDOR_PAYMENT
■ EMPLOYEE_EXPENSE_SOURCE_TYPE	■ PAYMENT_ITEM	■ VENDOR_PREPAYMENT
■ EMPLOYEE_STATUS	■ PAYMENT_METHOD	■ VENDOR_PREPAYMENT_APPLICATION
■ EMPLOYEE_TYPE	■ PAYROLL_ITEM	■ VENDOR_RETURN_AUTHORIZATION
■ ENTITY_ACCOUNT_MAPPING	■ PCT_COMPLETE_PROJECT_REVENUE_RULE	■ VENDOR_SUBSIDIARY_RELATIONSHIP
■ ESTIMATE	■ PERFORMANCE_METRIC	■ WAVE
■ EXPENSE_AMORTIZATION_EVENT	■ PERFORMANCE_REVIEW	■ WBS
■ EXPENSE_CATEGORY	■ PERFORMANCE_REVIEW_SCHEDULE	■ WEBSITE
■ EXPENSE_PLAN	■ PERIOD_END_JOURNAL	■ WORKFLOW_ACTION_SCRIPT
■ EXPENSE_REPORT	■ PHONE_CALL	■ WORK_ORDER
■ EXPENSE_REPORT_POLICY	■ PICK_STRATEGY	■ WORK_ORDER_CLOSE
■ FAIR_VALUE_PRICE	■ PICK_TASK	■ WORK_ORDER_COMPLETION
■ FINANCIAL_INSTITUTION	■ PLANNED_ORDER	■ WORK_ORDER_ISSUE
■ FIXED_AMOUNT_PROJECT_REVENUE_RULE	■ PLANNING_ITEM_CATEGORY	■ WORKPLACE
■ FOLDER	■ PLANNING_ITEM_GROUP	■ ZONE
■ FORMAT_PROFILE	■ PLANNING_RULE_GROUP	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/record Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var objRecord = record.delete({
4     type: record.Type.SALES_ORDER,

```

```
5 | id: 128
6 | });
7 | ...
8 | // Add additional code
```

N/recordContext Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/recordContext module to get all the available context types of the record, such as localization. The localization context type indicates which country a script is using for execution. You can also use the N/recordContext module to create conditional statements within a script so that the script behaves differently based on the context.

For more information about localization context, see the help topic [Localization Context](#).



In This Help Topic

- N/recordContext Module Members

N/recordContext Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	recordContext.RecordContext	Object (read-only)	Client and server scripts	Contains key-value pairs that represent context types and their values.
Method	recordContext.getContext(options)	recordContext.RecordContext	Client and server scripts	Returns the record context object for a record.
Enum	recordContext.ContextType	enum (read-only)	Client and server scripts	Holds the values for the context type. Used to set the value for the contextTypes parameter of the recordContext.getContext(options) method.

N/recordContext Module Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/recordContext module.

Get the Localization Context of an Employee Record

The following sample shows how to retrieve the localization context value for a record. The sample uses two different approaches depending on whether the record is loaded in the script. The subsidiary values

are hard-coded. If you run this script in your account, be sure to replace the hard-coded values with valid values from your account.

**Note:** This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```
1  /**
2   * @NApiVersion 2.x
3   */
4   // This example calls the getContext() API with options recordType, recordId and record.
5   require(['N/record', 'N/recordContext'],
6     function(record, recordContext) {
7       // Create record
8       var employee = record.create({
9         type : record.Type.EMPLOYEE,
10        isDynamic: true
11      })
12      // Setup CA subsidiary
13      employee.setValue('subsidiary', 2);
14      employee.setValue('entityid', 'test_emp_' + Date.now())
15      employeeId = employee.save()
16
17      // getContext() with options recordType, recordId and contextTypes
18      var employeeContext = recordContext.getContext({
19        recordType : record.Type.EMPLOYEE,
20        recordId: employeeId,
21        contextTypes: [recordContext.ContextType.LOCALIZATION]
22      })
23      log.debug(employeeContext); // expected log will list {"localization":["CA"]}
24
25      // Change employee subsidiary to AU
26      employee.setValue('subsidiary', 3);
27
28      // getContext() with options record and contextTypes
29      var employeeContext = recordContext.getContext({
30        record : employee,
31        contextTypes: [recordContext.ContextType.LOCALIZATION]
32      })
33      log.debug(employeeContext); // expected log will list {"localization":["AU"]}
34
35      // Delete record
36      record.delete({
37        type : record.Type.EMPLOYEE,
38        id: 1
39      })
40    });
```

recordContext.RecordContext

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Contains key-value pairs that represent context types and their values. Each key is the name of the context type, and each value is the record context. Multiple values for a single context type can be returned in an array.
Supported Script Types	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/recordContext Module
Since	2020.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/recordContext Module Sample](#).

```
1 // Add additional code
2
3 {"localization":["CA"]}
4 ...
5 // Add additional code
```

recordContext.getContext(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the record context object for a given record. The parameters you specify for this method depend on whether the record is currently loaded in your script: - For records that are not loaded in your script, the record is defined by record type and ID. In this case, you must use the recordType and recordId parameters to specify the record to obtain the context for. You do not use the record parameter. - For records that are loaded in your script, the record is defined by the record object. In this case, you must use the record parameter to specify the record to obtain the context for. You do not use the recordType and recordId parameters.
Returns	recordContext.RecordContext
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/recordContext Module
Since	2020.2

Parameters

**Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.recordType	string	required if the record is not loaded in your script	The record type.	2020.2
options.recordId	string	required if the record is not loaded in your script	The internal ID of the record.	2020.2
options.record	record.Record	required if the record is loaded in your script	The record object.	2020.2

Parameter	Type	Required / Optional	Description	Since
options.contextTypes	Array<string>	optional	The context type to retrieve. Use the recordContext.ContextType enum to set the value for this parameter.	2020.2

Errors

Error Code	Thrown If
MUTUALY_EXCLUSIVE_ARGUMENT	The record is present alongside a record ID or record type.
MISSING_REQD_ARGUMENT	Any of the record ID or record type parameter is missing.
SSS_INVALID_TYPE_ARG	The parameter type is wrong.
UNKNOWN_CONTEXT_TYPE	The context type selection is unknown.
INVALID_UNSUPRTD_RCRD_TYP	The record type selection is invalid or not supported.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/recordContext Module Sample](#).

```

1 // Add additional code
2 // Sample code on getting context for records that are not loaded
3 ...
4 var employeeContext = recordContext.getContext({
5     recordType: record.Type.EMPLOYEE,
6     recordId: employeeId
7     contextTypes: [recordContext.ContextType.LOCALIZATION]
8 ...
9 // Add additional code

```

```

1 // Add additional code
2 // Sample code on getting context for records that are loaded
3 ...
4 var lrc = recordContext.getContext({
5     record: recordObject,
6     contextTypes: [recordContext.ContextType.LOCALIZATION]
7 ...
8 // Add additional code

```

recordContext.ContextType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the value for the context type.
-------------------------	---------------------------------------

Module	N/recordContext Module
Sibling Module Members	N/recordContext Module Members
Since	2020.2

Values

Value
LOCALIZATION
 Note: The values returned for this context type are given as ISO 3166-2 country codes.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/recordContext Module Sample](#).

```
1 // Add additional code
2 ...
3 // getContext() with options record and contextTypes
4 var employeeContext = recordContext.getContext({
5
6     record: employee,
7
8     contextTypes: [recordContext.ContextType.LOCALIZATION]
9
10 })
11 ...
12 // Add additional code
```

N/redirect Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/redirect module to customize navigation within NetSuite by setting up a redirect URL that resolves to a NetSuite resource or external URL. You can redirect users to one of the following:

- URL
- Suitelet
- Record
- Task link
- Saved search
- Unsaved search

Go To Samples 

Note: Suitelets, beforeLoad user events, and synchronous afterSubmit user events are supported. This module does not support beforeSubmit or asynchronous afterSubmit user events. This module is only supported when triggered from the UI. Backend contexts such as CSV Import and Scheduled Scripts are not supported.

In This Help Topic

■ N/redirect Module Members

N/redirect Module Members

Member Type	Name	Return Type	Supported Script Types	Description
Method	redirect.redirect(options)	void	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to the URL of a Suitelet that is available externally (available without login).
	redirect.toRecord(options)	void	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to a NetSuite record.
	redirect.toRecordTransform(options)	void	workflow action scripts	Redirects to a standard or custom transaction instance.
	redirect.toSavedSearch(options)	void	afterSubmit user events	Redirects to a saved search.
	redirect.toSavedSearchResult(options)	void	afterSubmit user events	Redirects to a saved search result.
	redirect.toSearch(options)	void	afterSubmit user events	Redirects to search.
	redirect.toSearchResult(options)	void	afterSubmit user events	Redirects to search results.
	redirect.toSuitelet(options)	void	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to a Suitelet.
	redirect.toTaskLink(options)	void	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to a tasklink.

N/redirect Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use features of the N/redirect module.

Set a Redirect URL to a Newly Created Task Record

The following sample sets the redirect URL to a newly created task record. To set a redirect using a record ID, the record must have been previously submitted to NetSuite.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/record', 'N/redirect'], function(record, redirect) {
6      function redirectToTaskRecord() {
7          var taskTitle = 'New Opportunity';
8          var taskRecord = record.create({
9              type: record.Type.TASK
10         });
11         taskRecord.setValue('title', taskTitle);
12
13         var taskRecordId = taskRecord.save();
14
15         redirect.toRecord({
16             type: record.Type.TASK,
17             id: taskRecordId
18         });
19     }
20
21     redirectToTaskRecord();
22 });

```

redirect.redirect(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the redirect to the URL of a Suitelet that is available externally (Suitelets set to Available Without Login on the Script Deployment page).
Returns	void
Supported Script Types	Suitelets User Event scripts - beforeLoad entry point and synchronous afterSubmit user events. This module does not support beforeSubmit or asynchronous afterSubmit user events. This module is only supported when triggered from the UI. Backend contexts such as CSV Import and Scheduled Scripts are not supported. For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/redirect Module
Since	2015.2

Parameters

Note: The `options` parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.url</code>	string	required	The URL of a Suitelet that is available externally.

Parameter	Type	Required / Optional	Description
			 Note: For an external URL, Available without Login must be enabled on the Script Deployment page for the Suitelet.
options.parameters	Object	optional	Contains additional URL parameters as key-value pairs.  Note: Parameters cannot be arrays. Use JSON.stringify/JSON.parse instead to handle arrays.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 //Add additional code
2 ...
3 redirect.redirect({
4   url: '/app/site/hosting/scriptlet.nl?script=130&deploy=1',
5   parameters: {
6     'custparam_test': 'helloWorld'
7   }
8 });
9 ...
10 //Add additional code

```

redirect.toRecord(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: The options parameter is a JavaScript object.

Method Description	Sets the redirect URL to a specific NetSuite record.  Note: If you redirect a user to a record, the record must first exist in NetSuite. If you want to redirect a user to a new record, you must first create and submit the record before redirecting them. You must also ensure that any required fields for the new record are populated before submitting the record.
Returns	void
Supported Script Types	Suitelets User Event scripts - beforeLoad entry point and synchronous afterSubmit user events. This module does not support beforeSubmit or asynchronous afterSubmit user events. This module is only supported when triggered from the UI. Backend contexts such as CSV Import and Scheduled Scripts are not supported. For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/redirect Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The internal id of the target record.
options.type	string	required	The type of record.
options.isEditMode	boolean	optional	Determines whether to return a URL for the record in edit mode or view mode. If set to true, returns the URL to an existing record in edit mode. The default value is false – returns the URL to a record in view mode.
options.parameters	Object	optional	Contains additional URL parameters as key-value pairs.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 //Add additional code
2 ...
3 redirect.toRecord({
4     type: record.Type.TASK,
5     id: taskRecordId,
6     parameters: {
7         'custparam_test': 'helloWorld'
8     }
9 });
10 ...
11 //Add additional code

```

redirect.toRecordTransform(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Transforms a record to a standard or custom transaction instance. The fromId and fromType parameters can be acquired from the context object provided by the onAction(scriptContext) method of a workflow action script. The toType record type identifier cannot be acquired programmatically and must be entered manually. The new transaction instance opens in edit mode with all possible fields defaulted from the source record. To specify URL parameters or record field values on the redirected page, use
---------------------------	--

	the options.parameters parameter. For record field values, use the record.fieldId format. Invalid keys are ignored. The user can edit transaction details before saving. For a list of supported transformation types, see Supported Transformation Types .
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/redirect Module
Sibling Module Members	N/redirect Module Members
Since	2020.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.fromId	number	required	The ID of the source record.
options.fromType	string	required	The ID of the source record type.
options.toType	string	required	The ID of the target record type.
options.parameters	Object	optional	Contains additional parameters as key-value pairs.

Errors

Error Code	Error Message	Thrown If
INVALID_RCRD_TYPE	The record type 'type' is invalid.	The record type specified in the toType parameter is invalid.
INVALID_RCRD_TRANSFRM	That type of record transformation is not allowed. Please see the documentation for a list of supported transformation types.	Transformation between the fromType and toType record types is not allowed.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 // Add additional code
2 ...
3 define(['N/redirect'], function (redirect) {
4     return {
5         onAction: function (context)
6         {

```

```

7         redirect.toRecordTransform({
8             fromType: context.newRecord.type,
9             fromId: context.newRecord.id,
10            toType: 'invoice',
11            parameters: {
12                'record.memo': 'memo override', // override of memo field on body form
13                'cf': '92' // std product invoice custom form
14            }
15        });
16    },
17    }
18    });
19    ...
20    // Add additional code

```

redirect.toSavedSearch(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing saved search and redirect to the populated search definition page.
Returns	void
Supported Script Types	User event scripts -afterSubmit entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/redirect Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	number	required	Internal ID of the search. The internal ID is available only when the search is either loaded with search.load(options) or after it has been saved with Search.save(). Typical values are 55 or 234 or 87, not a value like customsearch_mysearch. Any ID prefixed with customsearch is a script ID, not the internal system ID for a search.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```
1 //Add additional code
```

```

2  ...
3  redirect.toSavedSearch({
4      id: 234
5  });
6  ...
7  //Add additional code

```

redirect.toSavedSearchResult(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Redirects a user to a search results page for an existing saved search.
Returns	void
Supported Script Types	User event scripts - afterSubmit entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/redirect Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	number	required	Internal ID of the search. The internal ID is available only when the search is either loaded with search.load(options) or after it has been saved with Search.save(). Typical values are 55 or 234 or 87, not a value like customsearch_mysearch. Any ID prefixed with customsearch is a script ID, not the internal system ID for a search.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1  //Add additional code
2  ...
3  redirect.toSavedSearchResult({
4      id: 234
5  });
6  ...
7  //Add additional code

```

redirect.toSearch(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Redirects a user to an on demand search built in SuiteScript. This method loads a search into the session, and then redirects to a URL that loads the search definition page.
Returns	void
Supported Script Types	User event scripts - afterSubmit entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/redirect Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.search	search.Search	required	The search to be redirected to.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 var column = ['internalid'];
2 var filter = [['mainline', 'is', 'T']];
3 var ourNewSearch = search.create({
4   id: 'customsearch_test',
5   type: search.Type.SALES_ORDER,
6   title: 'My Generated Search',
7   columns: column,
8   filters: filter
9 });
10 redirect.toSearch({
11   search: ourNewSearch
12 });

```

redirect.toSearchResult(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Redirects a user to a search results page. For example, the results from an on demand search created with the N/search Module , or a loaded search that you modified but did not save.
---------------------------	--

Returns	void
Supported Script Types	User event scripts - afterSubmit entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/redirect Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.search	search.Search	required	The search result to be redirected to.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 //Add additional code
2 ...
3 myNewSearch = search.create({
4     type: search.Type.CUSTOMER
5 });
6 redirect.toSearchResult({
7     search:myNewSearch
8 });
9 ...
10 //Add additional code

```

redirect.toSuitelet(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Redirects the user to a Suitelet. For more information about Suitelets, see the help topic SuiteScript 2.x Suitelet Script Type .  Note: The redirect happens after the script finishes.
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point and synchronous afterSubmit user events. This module does not support beforeSubmit or asynchronous afterSubmit user events. This module is only supported when triggered from the UI. Backend contexts such as CSV Import and Scheduled Scripts are not supported.

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/redirect Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	The script ID for the Suitelet.
options.deploymentId	string	required	The deployment ID for the Suitelet.
options.isExternal	boolean	optional	The default value is false – indicates an external Suitelet URL.
options.parameters	Object	optional	Contains additional URL parameters as key-value pairs. Note: Parameters cannot be arrays. Use JSON.stringify/JSON.parse instead to handle arrays.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 //Add additional code
2 ...
3 redirect.toSuitelet({
4     scriptId: '31',
5     deploymentId: '1',
6     parameters: {
7         'custparam_test': 'helloWorld'
8     }
9 });
10 ...
11 //Add additional code

```

redirect.toTaskLink(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Redirects a user to a tasklink.
Returns	void

Supported Script Types	Suitelets User event scripts - beforeLoad entry point and synchronous afterSubmit user events. This module does not support beforeSubmit or asynchronous afterSubmit user events. This module is only supported when triggered from the UI. Backend contexts such as CSV Import and Scheduled Scripts are not supported. For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/redirect Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The taskId for a tasklink.  Note: Each page in NetSuite has a unique task ID associated with it for a specific record type. For more information about finding the task ID, see the help topic Task IDs.
options.parameters	Object	optional	Contains additional URL parameters as key-value pairs.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/redirect Module Script Sample](#).

```

1 //Add additional code
2 ...
3 redirect.toTaskLink({
4   id: 'ADMI_SHIPPING' ,
5   parameters: {
6     'custparam_test': 'helloWorld'
7   }
8 });
9 ...
10 //Add additional code

```

N/render Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/render module for printing, PDF creation, form creation from templates, and email creation from templates.

[Go To Samples { }](#)

Note: Direct manipulation of the print URL is **not** supported.

In This Help Topic

- [N/render Module Members](#)
- [EmailMergeResult Object Members](#)
- [TemplateRenderer Object Members](#)

N/render Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	render.EmailMergeResult	Object	Server scripts	Encapsulates an email merge result.
	render.TemplateRenderer	Object	Server scripts	Encapsulates a template object that produces HTML and PDF printed forms utilizing advanced PDF/HTML template capabilities.
Method	render.bom(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a bill of materials.
	render.create()	render.TemplateRenderer	Server scripts	Creates a render.TemplateRenderer object.
	render.mergeEmail(options)	render.EmailMergeResult	Server scripts	Creates a render.EmailMergeResult object.
	render.packingSlip(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a packing slip.
	render.pickingTicket(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a picking ticket.
	render.statement(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a statement.
	render.transaction(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a transaction.
	render.xmlToPdf(options)	file.File	Server scripts	Passes XML to the BFO tag library (which is stored by NetSuite), and returns a PDF file.
Enum	render.DataSource	enum	Server scripts	Holds the string values for supported data source types. Use this enum to set the options .format parameter of the TemplateRenderer.addCustomDataSource(options) method.
	render.PrintMode	enum	Server scripts	Holds the string values for supported print output types. Use this enum to set the options .printMode parameter of the render.bom(options) , render.pickingTicket(options) , and render.statement(options) methods.

EmailMergeResult Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	EmailMergeResult.body	string (read-only)	Server scripts	The body of the email distribution in string format.
	EmailMergeResult.subject	string (read-only)	Server scripts	The subject of the email distribution in string format.

TemplateRenderer Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	TemplateRenderer.addCustomDataSource(options)	void	Server scripts	Adds an XML file or JSON object to an advanced template as a custom data source.
	TemplateRenderer.addQuery(options)	void	Server scripts	Uses Query as the renderer's data source.
	TemplateRenderer.addRecord(options)	void	Server scripts	Binds a record to a template variable.
	TemplateRenderer.addSearchResults(options)	void	Server scripts	Binds a search result to a template variable.
	TemplateRenderer.renderAsPdf()	Object	Server scripts	Uses an advanced template to produce a PDF printed form.
	TemplateRenderer.renderPdfToResponse(options)	void	Server scripts	Renders PDF template content as a server response.
	TemplateRenderer.renderAsString()	string	Server scripts	Returns template content in string form.
	TemplateRenderer.setTemplateById(options)	void	Server scripts	Sets the template using the internal ID.
	TemplateRenderer.setTemplateByScriptId(options)	void	Server scripts	Sets the template using the script ID.
	TemplateRenderer.renderToResponse(options)	void	Server scripts	Renders HTML template content as a server response.
Property	TemplateRenderer.templateContent	string	Server scripts	Content of the template.

N/render Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/render module:

- [Generate a PDF File from a Raw XML String](#)
- [Render a Transaction Record Into an HTML Page](#)

- [Render an Invoice Into a PDF File Using an XML Template](#)
- [Render Search Results Into a PDF File](#)

Generate a PDF File from a Raw XML String

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to generate a PDF from a raw XML string.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   require(['N/render'],
6     function(render) {
7       function generatePdfFileFromRawXml() {
8         var xmlStr = "<?xml version='1.0'?>\n" +
9           "<!DOCTYPE pdf PUBLIC \"-//big.faceless.org//report\" \"report-1.1.dtd\">\n" +
10          "<pdf>\n<body font-size='18'\nHello World!\n</body>\n</pdf>";
11          var pdfFile = render.xmlToPdf({
12            xmlString: xmlStr
13          });
14        }
15        generatePdfFileFromRawXml();
16      });

```

Render a Transaction Record Into an HTML Page

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to render a transaction record into an HTML page.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

i Note: The `entityId` value in this sample is a placeholder. Before using this sample, replace the placeholder values with valid values from your NetSuite account.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   require(['N/render'],
6     function(render) {
7       function renderTransactionToHtml() {
8         var transactionFile = render.transaction({
9           entityId: 23,
10          printMode: render.PrintMode.HTML
11        });
12      }
13      renderTransactionToHtml();

```

```
14 | });
```

Render an Invoice Into a PDF File Using an XML Template

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to render an invoice into a PDF using an XML template in the File Cabinet. This sample requires the Advanced PDF/HTML Templates feature.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```
1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This sample shows how to render an invoice into a PDF file using an XML template in the file cabinet.
6   // Note that this example requires the Advanced PDF/HTML Templates feature.
7   require(['N/render', 'N/file', 'N/record'],
8     function(render, file, record) {
9       function renderRecordToPdfWithTemplate() {
10         var xmlTemplateFile = file.load('Templates/PDF Templates/invoicePDFTemplate.xml');
11         var renderer = render.create();
12         renderer.templateContent = xmlTemplateFile.getContents();
13         renderer.addRecord('record', record.load({
14           type: record.Type.INVOICE,
15           id: 37
16         }));
17         var invoicePdf = renderer.renderAsPdf();
18       }
19       renderRecordToPdfWithTemplate();
20     });
```

In the preceding sample, the `invoicePDFTemplate.xml` file was referenced in the File Cabinet. This file is similar to the Standard Invoice PDF/HTML Template found in Customization > Forms > Advanced PDF/HTML Templates.

Render Search Results Into a PDF File

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to render search results into a PDF.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```
1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   // This sample shows how to render search results into a PDF file.
6   define(['N/render', 'N/search'], function(render, search) {
7     function onRequest(options) {
8       var request = options.request;
9       var response = options.response;
```

```

10
11     var xmlStr = '<?xml version="1.0" encoding="UTF-8"?>\n' +
12         '<!DOCTYPE pdf PUBLIC "-//big.faceless.org//report" "report-1.1.dtd">\n' +
13         '<pdf lang="ru=RU" xml:lang="ru-RU">\n' + "<head>\n" +
14         '<link name="russianfont" type="font" subtype="opentype" ' + 'src="NetSuiteFonts/verdana.ttf" ' + "src-
bold="NetSuiteFonts/verdanab.ttf" ' + 'src-italic="NetSuiteFonts/verdanai.ttf" ' + "src-bolditalic="NetSuiteFonts/verdan
abi.ttf" ' + 'bytes="2"/>\n' + "</head>\n" +
15         '<body font-family="russianfont" font-size="18">\n????? ????</body>\n' + "</pdf>";
16
17     var rs = search.create({
18         type: search.Type.TRANSACTION,
19         columns: ['trandate', 'amount', 'entity'],
20         filters: []
21     }).run();
22
23     var results = rs.getRange(0, 1000);
24     var renderer = render.create();
25     renderer.templateContent = xmlStr;
26     renderer.addSearchResults({
27         templateName: 'results',
28         searchResult: results
29     });
30
31     var newfile = renderer.renderAsPdf();
32     response.writeFile(newfile, false);
33 }
34
35 return {
36     onRequest: onRequest
37 };
38 });

```

render.EmailMergeResult

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates an email merge result. Use render.mergeEmail(options) to create and return this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/render Module
Methods and Properties	EmailMergeResult Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mergeResult = render.mergeEmail({
4     templateId: 1234,
5     entity: {
6         type: 'employee',
7         id: 62
8     },

```

```

9      recipient: {
10         type: 'lead',
11         id: 41
12      },
13      supportCaseId: 2,
14      transactionId: 271,
15      customRecord: null
16    });
17    ...
18    //Add additional code

```

EmailMergeResult.body

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The body of the email distribution in string format.
Type	string (read-only)
Module	N/render Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1    //Add additional code
2    ...
3    log.debug({
4      title: 'Email Body: ',
5      details: mergeResultObj.body
6    });
7    ...
8
9    //Add additional code

```

EmailMergeResult.subject

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The subject of the email distribution in string format.
Type	string (read-only)
Module	N/render Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1    //Add additional code

```

```

2  ...
3  log.debug({
4      title: 'Email Subject: ',
5      details: mergeResultObj.subject
6  });
7  ...
8
9  //Add additional code

```

render.TemplateRenderer

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a template object that produces HTML and PDF printed forms utilizing advanced PDF/HTML template capabilities. The template object includes methods that pass in a template as string to be interpreted by FreeMarker, and render interpreted content in your choice of two different formats: as HTML output to an http.ServerResponse object, or as XML string that can be passed to render.xmlToPdf(options) to produce a PDF. This object is available when the Advanced PDF/HTML Templates feature is enabled.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/render Module
Methods and Properties	TemplateRenderer Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1  //Add additional code
2  //Advanced PDF/HTML Templates feature must be enabled
3  ...
4  var xmlTplFile = file.load('Templates/PDF Templates/invoicePDFTemplate.xml');
5  var myFile = render.create();
6  myFile.templateContent = xmlTplFile.getContents();
7  myFile.addRecord('record', record.load({
8      type: record.Type.INVOICE,
9      id: 37
10  }));
11  var invoicePdf = myFile.renderAsPdf();
12  ...
13  //Add additional code

```

TemplateRenderer.addCustomDataSource(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds XML or JSON as custom data source to an advanced PDF/HTML template.
---------------------------	--

Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2016.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.alias	string	required	Data source alias
options.format	render.DataSource	required	Data format
options.data	Object Document string	required	Object, document, or string

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var renderer = render.create();
4
5 var xmlObj = xml.Parser.fromString(xmlString);
6 var jsonObj = JSON.parse(jsonString);
7
8 renderer.templateContent = "${XML.book.title}<br />${XML.book.chapter[1].title}<br />${JSON.book.title}<br />${JSON.book.chapter[1].title}<br />${JSON_STR.book.title}<br />${XML_STR.book.title}";
9
10 renderer.addCustomDataSource({
11     format: render.DataSource.XML_DOC,
12     alias: "XML",
13     data: xmlObj
14 });
15 renderer.addCustomDataSource({
16     format: render.DataSource.XML_STRING,
17     alias: "XML_STR",
18     data: xmlString
19 });
20 renderer.addCustomDataSource({
21     format: render.DataSource.OBJECT,
22     alias: "JSON",
23     data: jsonObj
24 });
25 renderer.addCustomDataSource({
26     format: render.DataSource.JSON,
27     alias: "JSON_STR",
28     data: jsonString
29 });
30
31 var xml = renderer.renderAsString();
32 ...
33 //Add additional code

```

TemplateRenderer.addQuery(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a SuiteAnalytics Workbook query to use as the renderer's data source. You can specify the query in two ways: - As a query.Query object (which you create using the N/query Module) using the <code>options.query</code> parameter. - By providing the workbook ID of an existing SuiteAnalytics workbook using the <code>options.id</code> parameter. One of <code>options.query</code> or <code>options.id</code> is required. This method returns a maximum of 5000 results in the query result set. If a query matches more than 5000 results, you must use <code>options.pageIndex</code> and <code>options.pageSize</code> to retrieve the full set of results. There is no governance for this method. When the renderer is executed, rendering consumes 10 units for every iteration through the results. If the query is specified using a workbook ID, rendering consumes an additional 5 units before the first iteration through the results.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/render Module
Parent Object	render.TemplateRenderer
Sibling Object Members	TemplateRenderer Object Members
Since	2019.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.id</code>	string	required if <code>options.query</code> is not specified	Workbook query ID.
<code>options.pageIndex</code>	number	optional	Page index.
<code>options.pageSize</code>	number	optional	Page size. The minimum value is 5, and the maximum value is 1000.
<code>options.templateName</code>	string	required	Name of the results iterator variable referred to in the template.
<code>options.query</code>	query.Query object	required if <code>options.id</code> is not specified	Workbook query definition.

Errors

Error Code	Error Message	Thrown If
MUTUALLY_EXCLUSIVE_ARGUMENTS	Cannot use mutually exclusive arguments.	Both query and ID parameters are provided.
NEITHER_ARGUMENT_DEFINED	One of the following arguments is mandatory: id, query.	Neither options.id nor options.query are provided.
SSS_MISSING_REQD_ARGUMENT	TemplateRenderer.addQuery: Missing a required argument: options.templateName.	The template name is not specified.
UNABLE_TO_LOAD_QUERY	Unable to load query: invalidId.	The query parameter is provided but the ID is not valid.
WRONG_PARAMETER_TYPE	Wrong parameter type: options.query is expected as query.Query.	The query parameter is provided, but the ID has the incorrect type.

Syntax

Note that `TemplateRenderer.addQuery(options)` binds the results iterator to a template variable. This iterator provides sequential access to the query results. For random access to search results in FreeMarker, process your search data before rendering, and then pass the processed data to the template using `TemplateRenderer.addCustomDataSource(options)`.



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var search = query.create({
4   type: query.Type.TRANSACTION
5 });
6 var transactionlines = search.join({
7   fieldId: "transactionlines"
8 });
9 search.condition = transactionlines.createCondition({
10   fieldId: "netamount",
11   operator: query.Operator.GREATER,
12   values: 1
13 });
14 search.columns = [
15   transactionlines.createColumn({
16     fieldId: "netamount",
17     label: "Amount"
18   }),
19 ];
20
21 // get instance of TemplateRenderer - https://system.netsuite.com/app/help/helpcenter.nl?fid=section_4412065265.html
22 var renderer = render.create();
23 renderer.templateContent = "<#list page as line><item>${line['@label']}:${line[0]}</item>\n</#list>"
24
25 renderer.addQuery({
26   templateName: "page",
27   query: search,
28   pageSize: 3, // minimum page size is 5, so result will contain 5 lines instead of 3
29   pageIndex: 0,
30 })
31 ...
32 // Add additional code

```

TemplateRenderer.addRecord(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Binds a record to a template variable.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.templateName	string	required	Name of the record object variable referred to in the template.
options.record	record.Record object	required	The record to add.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var myContent = renderer.addRecord({
4   templateName: 'record',
5   record: record.load({
6     type: record.Type.CUSTOMER,
7     id: 1234
8   });
9 });
10 ...
11 //Add additional code

```

TemplateRenderer.addSearchResults(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Binds a search result to a template variable.
Returns	void
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.templateName	string	required	Name of the results iterator variable referred to in the template.
options.searchResult	search.Result object	required	The search result to add.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var rs = search.create({
4     type: search.Type.TRANSACTION,
5     columns: ['trandate', 'amount', 'entity'],
6     filters: []
7 }).run();
8 var results = rs.getRange(0, 1000);
9 var renderer = render.create();
10 renderer.templateContent = xmlStr;
11 renderer.addSearchResults({
12     templateName: 'results',
13     searchResult: results
14 });
15 ...
16 //Add additional code

```

TemplateRenderer.renderAsPdf()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Uses the advanced template to produce a PDF printed form.
Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var invoicePdf = renderer.renderAsPdf();
4 ...
5 //Add additional code
```

TemplateRenderer.renderPdfToResponse(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Renders a server response into a PDF file. For example, you can pass in a response to be rendered as a PDF in a browser, or downloaded by a user.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.response	http.ServerResponse	required	Response that will be written to PDF. For example, the response passed from a Suitelet.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var invoicePdf = renderer.renderPdfToResponse({
4     response: myServerResponseObj
5 });
6 ...
7 //Add additional code
```

TemplateRenderer.renderAsString()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Return template content in string form.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var invoicePdf = renderer.renderAsString();
4 ...
5 //Add additional code

```

TemplateRenderer.renderToResponse(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes template content to a server response.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.response	http.ServerResponse	required	Response to write to.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var invoice = renderer.renderToResponse({
4     response: myServerResponseObj
5 });
6 ...
7 //Add additional code
```

TemplateRenderer.setTemplateById(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the template using the internal ID.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2016.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	number	required	Internal ID of the template.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var renderer = render.create();
4 renderer.setTemplateById(3);
5 var xml = renderer.renderAsString();
6 ...
7 //Add additional code
```

For more information, see the help topics [Advanced Templates](#) and [Advanced PDF/HTML Templates](#).

To find the template ID, search for PDF Templates or Advanced PDF/HTML Templates in NetSuite.

When the list of templates is displayed, hover your cursor on the Edit or Customize link. You can also see the ID in the browser's URL when you click the link. An example of a Standard PDF template with an ID of

4 is /app/crm/common/merge/pdftemplate.nl?id=4. An example of an Advanced HTML template with an ID of 19 is /app/common/custom/advancedprint/pdftemplate.nl?id=19.

IDs from both Standard and Advanced Templates are supported.

TemplateRenderer.setTemplateByScriptId(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the template using the script ID.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	Script ID of the template.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#)

```

1 //Add additional code
2 ...
3 var renderer = render.create();
4 renderer.setTemplateByScriptId({
5     scriptId: "STDTMPLPRICELIST"
6 });
7 var xml = renderer.renderAsString();
8 ...
9 //Add additional code

```

TemplateRenderer.templateContent

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Contents of the template.
Type	string
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/render Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 renderer.templateContent = xmlTemplateFile.getContents();
4 ...
5 //Add additional code

```

render.bom(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to create a PDF or HTML object of a bill of material.
Returns	file.File that contains a PDF or HTML document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/render Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.entityId	number	required	The internal ID of the transaction associated with the bom.
options.printMode	string	optional	The print output type. Use the render.PrintMode enum to set this value. By default, uses the company/user preference for print output.
options.inCustLocale	boolean	optional	Applies when advanced templates are used. Print the document in the customer's locale. If basic printing is used, this parameter is ignored and the transaction form is printed in the customer's locale.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var transactionFile = render.bom({
4   entityId: 23,
5   printMode: render.PrintMode.HTML,
6   inCustLocale: true
7 });
8 ...
9 //Add additional code
```

render.create()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to produce HTML and PDF printed forms with advanced PDF/HTML templates. Creates render.TemplateRenderer. This object includes methods that pass in a template as string to be interpreted by FreeMarker, and render interpreted content in your choice of two different formats: as HTML output to http.ServerResponse, or as XML string that can be passed to render.xmlToPdf(options) to produce a PDF.  Note: To use this method, the Advanced PDF/HTML Templates feature must be enabled.
Returns	render.TemplateRenderer
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/render Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var renderer = render.create();
4 ...
5 //Add additional code
```

render.mergeEmail(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a render.EmailMergeResult object for a mail merge with an existing scriptable email template.  Note: You must load the N/email Module to send and attach the email template to a record.
Returns	render.EmailMergeResult
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/render Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.templateId	number	required	Internal ID of the template.
options.entity	RecordRef	optional	The entity.
options.recipient	RecordRef	optional, the recipient can be specified with this parameter or with the email.send(options) method.	The recipient.
options.customRecord	RecordRef	optional	The custom record.
options.supportCaseId	number	optional	The support case ID.
options.transactionId	number	optional	The transaction ID.

RecordRef

You can use a RecordRef to designate the record to perform the mail merge on.

Note: The RecordRef object encapsulates the type and ID of a particular record instance.

Property	Type	Required / Optional	Description
RecordRef.id	number	required	Internal ID of the record instance.

Property	Type	Required / Optional	Description
RecordRef.type	string	required	The record type ID.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

The following script sample uses the [render.mergeEmail\(options\)](#) method to attach a record and the [email.send\(options\)](#) method to send the email template to the recipient.

```

1 //Add additional code
2 ...
3 /**
4  * @NApiVersion 2.x
5  */
6 require(['N/email', 'N/render'], function(email, render) {
7     function sendEmail() {
8         var senderId = -5;
9         var recipientId = 1711;
10        var mergeResult = render.mergeEmail({
11            templateId: 7,
12            entity: {
13                type: 'employee',
14                id: senderId
15            },
16            recipient: {
17                type: 'customer',
18                id: recipientId
19            }
20        });
21
22        email.send({
23            author: senderId,
24            recipients: recipientId,
25            subject: mergeResult.subject,
26            body: mergeResult.body
27        });
28    }
29    sendEmail();
30 });
31 ...
32 //Add additional code

```

render.packingSlip(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a PDF or HTML object of a packing slip.
Returns	file.File that contains a PDF or HTML document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/render Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.entityId	number	required	The internal ID of the transaction associated with the packing slip.
options.printMode	string	optional	The print output type. Set using the render.PrintMode enum. By default, uses the company/user preference for print output.
options.formId	number	optional	The packing slip form number.
options.fulfillmentId	number	optional	The fulfillment ID number.
options.inCustLocale	boolean	optional	Applies when advanced templates are used. Print the document in the customer's locale. If basic printing is used, this parameter is ignored and the transaction form is printed in the customer's locale.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var transactionFile = render.pickingTicket({
4   entityId: 23,
5   printMode: render.PrintMode.HTML,
6   inCustLocale: true
7 });
8 ...
9 //Add additional code

```

render.pickingTicket(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a PDF or HTML object of a picking ticket.
Returns	file.File that contains a PDF or HTML document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/render Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.entityId	number	required	The internal ID of the transaction associated with the picking ticket.
options.printMode	string	optional	The print output type. Use the render.PrintMode enum to set this value. By default, uses the company/user preference for print output.
options.formId	number	optional	The packing slip form number.
options.shipgroup	number	optional	Th shipping group for the ticket.
options.location	number	optional	The location for the ticket.
options.inCustLocale	boolean	optional	Applies when advanced templates are used. Prints the document in the customer's locale. If basic printing is used, this parameter is ignored and the transaction form is printed in the customer's locale.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var transactionFile = render.pickingTicket({
4     entityId: 23,
5     printMode: render.PrintMode.HTML,
6     inCustLocale: true
7 });
8 ...
9 //Add additional code

```

render.statement(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a PDF or HTML object of a statement.
Returns	file.File that contains a PDF or HTML document

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/render Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options consolidateStatements	boolean	optional	Flag to convert all amount values to the base currency.
options entityId	number	required	The internal ID of the statement to print.
options formId	number	optional	The internal ID of the form to use to print the statement.
options inCustLocale	boolean	optional	Applies when advanced templates are used. Prints the document in the customer's locale. If basic printing is used, this parameter is ignored and the transaction form is printed in the customer's locale.
options openTransactionsOnly	boolean	optional	Flag to include only open transactions.
options printMode	string	optional	The print output type. Use the render.PrintMode enum to set this value. By default, uses the company/user preference for print output.
options startDate	Date	optional	Date of the oldest transaction to appear on the statement.
options statementDate	Date	optional	The statement date.
options subsidiaryId	Integer	optional	Id of the subsidiary.

 **Note:** This parameter only works for advance printing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
```

```
3 var transactionFile = render.statement({
4   entityId: 23,
5   printMode: render.PrintMode.HTML,
6   inCustLocale: true,
7   consolidateStatements: false,
8   startDate: '01/04/2023',
9   statementDate: '01/04/2024'
10 });
11 ...
12 //Add additional code
```

render.transaction(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a PDF or HTML object of a transaction. i Note: File size is limited to 10MB. If the Advanced PDF/HTML Templates feature is enabled, you can associate an advanced template with the custom form saved for a transaction. The advanced template is used to format the printed transaction. For details about this feature, see the help topic Advanced PDF/HTML Templates
Returns	file.File that contains a PDF or HTML document
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/render Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.entityId	number	required	The internal ID of the transaction to print.
options.printMode	enum	optional	The print output type. Use the render.PrintMode enum to set this value. By default, uses the company/user preference for print output.
options.formId	number	optional	The transaction form number.
options.inCustLocale	boolean	optional	Applies when advanced templates are used. Prints the document in the customer's locale. If basic printing is used, this parameter is ignored and the transaction form is printed in the customer's locale.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var transactionFile = render.transaction({
4     entityId: 23,
5     printMode: render.PrintMode.HTML,
6     inCustLocale: true
7 });
8 ...
9 //Add additional code
```

render.xmlToPdf(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to pass XML to the Big Faceless Organization (BFO) tag library (which is stored by NetSuite), and return a PDF file. BFO is used in NetSuite. For version details, see the help topic Third-Party Products Used in Advanced Printing .  Note: File size cannot exceed 10MB.
Returns	file.File
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/render Module
Since	2015.2

Parameters

**Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.xmlString	xml.Document string	required	The XML document or string to convert to PDF. For information and examples about using BFO tags to create this document or string, see the documentation for the corresponding SuiteScript 1.0 API, nlapiXMLToPDF(xmlstring) in SuiteScript 1.0 Documentation .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var pdfFile = render.xmlToPdf({
4   xmlString: xmlStr
5 });
6 ...
7 //Add additional code
```

render.DataSource

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported data source types. Use this enum to set the options.format parameter of the TemplateRenderer.addCustomDataSource(options) method.
Module	N/render Module
Sibling Module Members	N/render Module Members
Since	2015.2

Values

Value
JSON
OBJECT
XML_DOC
XML_STRING

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```
1 //Add additional code
2 ...
3 renderer.addCustomDataSource({
4   format: render.DataSource.JSON,
```

```

5     alias: "JSON_STR",
6     data: jsonString
7   });
8   ...
9   //Add additional code

```

render.PrintMode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported print output types. Use this enum to set the options.printMode parameter of the render.bom(options) , render.pickingTicket(options) , and render.statement(options) methods.
Module	N/render Module
Sibling Module Members	N/render Module Members
Since	2015.2

Values

Value
DEFAULT
HTML
PDF

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/render Module Script Samples](#).

```

1   //Add additional code
2   ...
3   printMode: render.PrintMode.HTML
4   ...
5   //Add additional code

```

N/runtime Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/runtime module to view runtime settings for the script, the session, or the user. You can also use this module to set a session key and to see whether a particular feature is enabled in your account.

[Go To Samples {..}](#)

In This Help Topic

- [N/runtime Module Members](#)
- [Script Object Members](#)
- [Session Object Members](#)
- [User Object Members](#)

N/runtime Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	runtime.Script	Object	Client and server scripts	Encapsulates the runtime settings of the currently executing script.
	runtime.Session	Object	Client and server scripts	Encapsulates the user session for the currently executing script.
	runtime.User	Object	Client and server scripts	Encapsulates the properties and preferences of the user currently executing the script.
Method	runtime.getCurrentScript()	runtime.Script	Client and server scripts	Returns a runtime.Script object that represents the currently executing script.
	runtime.getCurrentSession()	runtime.Session	Client and server scripts	Returns a runtime.Session object that represents the user session for the currently executing script.
	runtime.getCurrentUser()	runtime.User	Client and server scripts	Returns a runtime.User object that represents the properties and preferences of the user currently executing the script.
	runtime.isFeatureInEffect(options)	boolean	Client and server scripts	Indicates whether a particular feature is enabled in a NetSuite account. These are the features that appear on the Enable Features page.
Property	runtime.accountId	string (read-only)	Client and server scripts	The account ID for the current user.
	runtime.country	string (read-only)	Client and server scripts	The country for the current company.
	runtime.envType	string (read-only)	Client and server scripts	The current environment in which the script is executing. This property uses values from the runtime.EnvType enum.
	runtime.executionContext	string (read-only)	Client and server scripts	The trigger of the current script. This property uses values from the runtime.ContextType enum.
	runtime.processorCount	number (read-only)	Client and server scripts	The number of processors available to the current account.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	runtime.queueCount	number (read-only)	Client and server scripts	The number of scheduled script queues available to the current account.
	runtime.version	string (read-only)	Client and server scripts	The version of NetSuite that the method is called in. For example, this property in an account running NetSuite 2023.2 is 2023.2.
Enum	runtime.ContextType	enum	Client and server scripts	Holds the context values for script triggers. This is the type for the runtime.executionContext property.
	runtime.EnvType	enum	Client and server scripts	Holds all possible environment types that the current script can execute in. This is the type for the runtime.envType property.
	runtime.Permission	enum	Client and server scripts	Holds the user permission level for a specific permission ID. This is the type returned by the User.getPermission(options) method.

Script Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Script.getParameter(options)	number Date string boolean	Client and server scripts	Returns the value of a script parameter for the currently executing script.
	Script.getRemainingUsage()	number	Client and server scripts	Returns the number of units remaining (per governance limitations) for the currently executing script.
Property	Script.apiVersion	string (read-only)	Client and server scripts	The current script's runtime version.
	Script.bundleIds	Array (read-only)	Client and server scripts	An array of bundle IDs for the bundles that include the currently executing script.
	Script.deploymentsId	string (read-only)	Server scripts	The deployment ID for the script deployment of the currently executing script.
	Script.id	string (read-only)	Client and server scripts	The script ID for the currently executing script.
	Script.logLevel	string (read-only)	Server scripts	The script logging level for the currently executing script.
	Script.percentComplete	number	Client and server scripts	The percent complete for the current scheduled script execution. This value will appear in the

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				% Complete column on the Scheduled Script Status page.

Session Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Session.get(options)	string null	Server scripts	Returns the user-defined session object value associated with the session object key. Both the session object value and associated key are defined using Session.set(options) .
	Session.set(options)	void	Server scripts	Sets a key and value for a user-defined session object.

User Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	User.getPermission(options)	string	Client and server scripts	Returns a runtime.Permission user permission level for the specified permission.
	User.getPreference(options)	string	Client and server scripts	Returns the value of a NetSuite preference. Currently only General Preferences and Accounting Preferences are exposed in SuiteScript. For more information about these preferences, see the help topics General Preferences and Accounting Preferences .
Property	User.contact	number(read-only)	Client and server scripts	The internal ID of the currently logged-in contact.
	User.department	number (read-only)	Client and server scripts	The internal ID of the department for the current user.
	User.email	string (read-only)	Client and server scripts	The email address of the current user.
	User.id	number (read-only)	Client and server scripts	The internal ID of the current user.
	User.location	number (read-only)	Client and server scripts	The internal ID of the location of the current user.
	User.name	string (read-only)	Client and server scripts	The name of the current user.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	User.role	number (read-only)	Client and server scripts	The internal ID of the role for the current user.
	User.roleCenter	string (read-only)	Client and server scripts	The string value of the center type, or role center, for the current user.
	User.roleId	string (read-only)	Client and server scripts	The custom scriptId of the role for the current user.
	User.subsidiary	number (read-only)	Client and server scripts	The internal ID of the subsidiary for the current user.

N/runtime Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/runtime module:

- Return User and Session Information
- Create Multiple Sales Records Using a Scheduled Script

Return User and Session Information

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to write user and session information for the currently executing script to the response.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5
6   // This script writes user and session information for the currently executing script to the response.
7   define(['N/runtime'], function(runtime) {
8       function onRequest(context) {
9           var remainingUsage = runtime.getCurrentScript().getRemainingUsage();
10          var userRole = runtime.getCurrentUser().role;
11          var currentSession = runtime.getCurrentSession();
12
13          // Set the current session's scope
14          currentSession.set({
15              name: 'scope',
16              value: 'global'
17          });
18
19          var sessionScope = runtime.getCurrentSession().get({
20              name: 'scope'
21          });
22
23          log.debug('Remaining Usage:', remainingUsage);

```

```

24     log.debug('Role:', userRole);
25     log.debug('Session Scope:', sessionScope);
26
27     context.response.write('Executing under role: ' + userRole
28         + '. Session scope: ' + sessionScope + '.');
29 }
30 return {
31     onRequest: onRequest
32 };
33 });

```

Create Multiple Sales Records Using a Scheduled Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a scheduled script to create multiple sales records and log the record creation progress.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ScheduledScript
4   */
5
6   // This script creates multiple sales records and logs the record creation progress.
7   define(['N/runtime', 'N/record'], function(runtime, record) {
8       return {
9           execute: function(context) {
10               var script = runtime.getCurrentScript();
11               for (x = 0; x < 500; x++) {
12                   var rec = record.create({
13                       type: record.Type.SALES_ORDER
14                   });
15                   script.percentComplete = (x * 100)/500;
16                   log.debug({
17                       title: 'New Sales Orders',
18                       details: 'Record creation progress: ' + script.percentComplete + '%'
19                   });
20               }
21           }
22       };
23   });

```

runtime.Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the runtime settings of the currently executing script. Use <code>runtime.getCurrentScript()</code> to access this object.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/runtime Module

Methods and Properties	Script Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript(); //scriptObj is a runtime.Script object
4 log.debug('Script ID: ' + scriptObj.id);
5 ...
6 // Add additional code

```

Script.getParameter(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of a script parameter for the currently executing script. For information about script parameters, see the help topic Creating Script Parameters Overview .
Returns	number Date string boolean null undefined
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the script parameter.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.name parameter is not specified.
WRONG_PARAMETER_TYPE	The value for the options.name parameter is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 log.debug('Script parameter of custscript1: ' +
5     scriptObj.getParameter({name: 'custscript1'}));
6 ...
7 // Add additional code
```

Script.getRemainingUsage()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the number of units remaining (per governance limitations) for the currently executing script. For more information, see the help topic SuiteScript Governance and Limits .
Returns	number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 log.debug('Remaining governance units: ' + scriptObj.getRemainingUsage());
5 ...
6 // Add additional code
```

Script.apiVersion

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The current script's runtime version.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 var scriptApiVersion = scriptObj.apiVersion;
5 ...
6 // Add additional code
```

Script.bundleIds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of bundle IDs for the bundles that include the currently executing script. When you use this property, consider the following: ■ The array can contain any number of bundle IDs, because a bundle installation script can be associated with multiple bundles. ■ The array does not contain bundle IDs of deprecated bundles. ■ The elements in the array are sorted in ascending order from the lowest bundle ID to the highest bundle ID that includes the currently executing script. However, it is not guaranteed that the last element in the array corresponds to the bundle that triggered the currently executing script during the bundle installation.
Type	string [] (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 var bundleArr = scriptObj.bundleIds;
5 ...
6 // Add additional code
```

Script.deploymentId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The deployment ID for the script deployment on the currently executing script.
Type	string (read-only)
Module	N/runtime Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 log.debug('Deployment Id: ' + scriptObj.deploymentId);
5 ...
6 // Add additional code

```

Script.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The script ID for the currently executing script.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 log.debug('Script Id: ' + scriptObj.id);
5 ...
6 // Add additional code

```

Script.logLevel

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The script logging level for the currently executing script. This method is not supported on client scripts. Returns one of the following values: ■ DEBUG ■ AUDIT ■ ERROR ■ EMERGENCY
Type	string (read-only)

Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var scriptObj = runtime.getCurrentScript();
4 log.debug('Logging level: ' + scriptObj.logLevel);
5 ...
6 // Add additional code

```

Script.percentComplete

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The percent complete specified for the current scheduled script execution. This value appears in the % Complete column on the Scheduled Script Status page. This value can be set or retrieved.
Type	number
Module	N/runtime Module
Since	2015.2

Errors

Error Code	Thrown If
SSS_OPERATION_UNAVAILABLE	The currently executing script is not a scheduled script.

Syntax



Important: The following code samples show the syntax for this member. They are not a functional examples. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Gets the percentage of records completed
4 var scriptObj = runtime.getCurrentScript();
5 if (scriptObj.executionContext === ContextType.SCHEDULED) {
6     log.debug({
7         details: 'Script percent complete: ' + scriptObj.percentComplete
8     });
9 }
10 ...
11 // Add additional code

```

```

1 // Add additional code
2 ...
3 // Sets the percent complete

```

```

4 var script = runtime.getCurrentScript();
5 for (x = 0; x < 500; x++) {
6     var rec = record.create({
7         type: record.Type.SALES_ORDER
8     });
9     script.percentComplete = (x * 100)/500;
10    log.debug({
11        title: 'New Sales Orders',
12        details: 'Record creation progress: ' + script.percentComplete + '%'
13    });
14 }
15 ...
16 // Add additional code

```

runtime.Session

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the user session for the currently executing script. Use this object to set and get user-defined objects for the current user session. Use the objects to track user-related session data. For example, you can gather information about the user scope, budget, or business problems. Use Session.set(options) to set session object values. Use Session.get(options) to retrieve session object values.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/runtime Module
Methods and Properties	Session Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sessionObj = runtime.getCurrentSession(); //sessionObj is a runtime.Session object
4 sessionObj.set({
5     name: 'myKey',
6     value: 'myValue'
7 });
8 log.debug('Session object myKey value: ' + sessionObj.get({name: 'myKey'}));
9 ...
10 // Add additional code

```

Session.get(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the user-defined session object value associated with a session object key.
---------------------------	---

	Both the session object value and associated key are defined using Session.set(options) (supported in server scripts only). If the key does not exist, this method returns null.
Returns	string null
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	Key used to store the session object.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.name parameter is not specified.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sessionObj = runtime.getCurrentSession();
4 sessionObj.set({
5     name: 'myKey',
6     value: 'myValue'
7 });
8 log.debug('Session object myKey value: ' + sessionObj.get({name: 'myKey'}));
9 ...
10 // Add additional code

```

Session.set(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets a key and value for a user-defined session object. Use Session.get(options) to retrieve the object value after you set it.
Returns	void

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	Key used to store the session object.	2015.2
options.value	string	required	Value to associate with the key in the user session.	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.name or options.value parameter is not specified.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sessionObj = runtime.getCurrentSession();
4 sessionObj.set({
5     name: 'myKey',
6     value: 'myValue'
7 });
8 log.debug('Session object myKey value: ' + sessionObj.get({name: 'myKey'}));
9 ...
10 // Add additional code

```

runtime.User

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the properties and preferences of the user currently executing the script.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/runtime Module

Methods and Properties	User Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var userObj = runtime.getCurrentUser();
4 ...
5 // Add additional code

```

User.getPermission(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a runtime.Permission user permission level for the specified permission.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	Internal ID of a permission. For a list of permission IDs, see the help topic Permission Names and IDs .	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var userObj = runtime.getCurrentUser();
4 var userPermission = userObj.getPermission({
5     name: 'ADMI_ACCOUNTING'
6 });

```



```

7 log.debug('User permission of ADMI_ACCOUNTING: ' +
8   (userPermission === runtime.Permission.FULL ? 'FULL' : userPermission);
9 ...
10 // Add additional code

```

User.getPreference(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value set for a NetSuite preference. Currently only General Preferences and Accounting Preferences are exposed in SuiteScript. You can also view General Preferences by going to Setup > Company > General Preferences. View Accounting Preferences by going to Setup > Accounting > Accounting Preferences. If you want to change the value of a General or Accounting preference using SuiteScript 2.x, you must load each preference page using config.load(options), where options.name is COMPANY_PREFERENCES or ACCOUNTING_PREFERENCES. The config.load(options) method returns a record.Record. You can use the Record.setValue(options) method to set the preference. For more information about these preferences, see the help topics General Preferences and Accounting Preferences. i Note: The permission level will be Permission.FULL if the script is configured to execute as administrator. You can configure a script to execute as administrator by selecting “administrator” from the Execute as Role field on Script Deployment page.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/runtime Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	Internal ID of the preference. For a list of preference IDs, see the help topic Preference Names and IDs .	2015.2

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
```

```

2  ...
3  var userObj = runtime.getCurrentUser();
4  var userPref = userObj.getPreference ({
5      name: 'emailemployeeonapproval'
6  });
7  log.debug('User preference for emailemployeeonapproval: ' + userPref);
8  ...
9  // Add additional code

```

User.contact

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the currently logged-in contact. If no logged-in entity or other entity than contact is logged in, then 0 is returned as value.
Type	number (read-only)
Module	N/runtime Module
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var userObj = runtime.getCurrentUser();
4  log.debug('Internal ID of current contact: ' + userObj.contact);
5  ...
6  // Add additional code

```

User.department

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the department for the current user.
Type	number (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var userObj = runtime.getCurrentUser();

```

```

4 log.debug('Internal ID of current user department: ' + userObj.department);
5 ...
6 // Add additional code

```

User.email

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The email address of the current user. To use this property, the email field on the user employee record must contain an email address.  Note: In a shopping context where the shopper is recognized but not logged in, this method can be used to return the shopper's email, instead of getting it from the customer record.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var userObj = runtime.getCurrentUser();
4 log.debug('Current user email: ' + userObj.email);
5 ...
6 // Add additional code

```

User.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the current user.
Type	number (read-only)
Module	N/runtime Module
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var userObj = runtime.getCurrentUser();
4  log.debug('Internal ID of current user: ' + userObj.id);
5  ...
6  // Add additional code

```

User.location

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the location of the current user.
Type	number (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var userObj = runtime.getCurrentUser();
4  log.debug('Internal ID of current user location: ' + userObj.location);
5  ...
6  // Add additional code

```

User.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the current user.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var userObj = runtime.getCurrentUser();
4  log.debug('Name of current user: ' + userObj.name);
5  ...

```

```
6 // Add additional code
```

User.role

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the role for the current user.
Type	number (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var userObj = runtime.getCurrentUser();
4 log.debug('Internal ID of current user role: ' + userObj.role);
5 ...
6 // Add additional code
```

User.roleCenter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The string value of the center type, or role center, for the current user. The NetSuite UI adjusts automatically to different users' business needs. For each user, NetSuite displays a variable set of tabbed pages, called a center, based on the user's assigned role. Each NetSuite center provides, for users with related roles, the pages and links they need to do their jobs. For more information about NetSuite centers, see the help topic Centers Overview.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
```

```

3 | var userObj = runtime.getCurrentUser();
4 | log.debug('String value of current user center type (role center): ' + userObj.roleCenter);
5 | ...
6 | // Add additional code

```

User.roleId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The custom scriptId of the role for the current user. You can use this value instead of User.role . When bundling a custom role, the internal ID number of the role in the target account can change after the bundle is installed. Therefore, in the target account you can use this property to access the unique/custom scriptId assigned to the role.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | var userObj = runtime.getCurrentUser();
4 | log.debug('Custom script ID of current user role: ' + userObj.roleId);
5 | ...
6 | // Add additional code

```

User.subsidiary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the subsidiary for the current user.
Type	number (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 | // Add additional code

```

```

2  ...
3  var userObj = runtime.getCurrentUser();
4  log.debug('Internal ID of current user subsidiary: ' + userObj.subsidiary);
5  ...
6  // Add additional code

```

runtime.getCurrentScript()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a runtime.Script object that represents the currently executing script. Use this method to get properties and parameters of the currently executing script and script deployment. If you want to get properties for the session or user, use runtime.getCurrentSession() or runtime.getCurrentUser() instead.
Returns	runtime.Script
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var scriptObj = runtime.getCurrentScript();
4  ...
5  // Add additional code

```

runtime.getCurrentSession()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a runtime.Session object that represents the user session for the currently executing script. Use this method to get session objects for the current user session. Note: In a client script, session information can change in some situations, such as due to a restart, HTTPS session timeout, or restart of a server script running in the same session which may have reset the value. If you want to get properties for the script or user, use runtime.getCurrentScript() or runtime.getCurrentUser() instead.
---------------------------	--

Returns	runtime.Session
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var sessionObj = runtime.getCurrentSession();
4 ...
5 // Add additional code

```

runtime.getCurrentUser()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a runtime.User object that represents the properties and preferences for the user currently executing the script. Use this method to get session objects for the current user session. If you want to get properties for the script or session, use runtime.getCurrentScript() or runtime.getCurrentSession() instead.
Returns	runtime.User
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var userObj = runtime.getCurrentUser();

```



```

4 | ...
5 | // Add additional code

```

runtime.isFeatureInEffect(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to determine if a particular feature is enabled in a NetSuite account. These are the features that appear on the Enable Features page at Setup > Company > Enable Features.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/runtime Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.feature	string	required	The internal ID of the feature to check. For a list of feature internal IDs, see the help topic Feature Names and IDs .	2015.2

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	The options.feature parameter is not specified.
WRONG_PARAMETER_TYPE	The value for the options.feature parameter is not a string.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | var featureInEffect = runtime.isFeatureInEffect({
4 |     feature: 'ADVBILLING'

```

```

5  });
6  log.debug('Advanced Billing feature is enabled: ' + featureInEffect);
7  ...
8  // Add additional code

```

runtime.accountId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The account ID for the current user.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  log.debug('Account ID for the current user: ' + runtime.accountId);
4  ...
5  // Add additional code

```

runtime.country

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The country for the current company.
Type	string (read-only)
Module	N/runtime Module
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  // Add additional code
2  ...
3  log.debug('Country for the current company: ' + runtime.country);
4  ...
5  // Add additional code

```

runtime.envType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The current environment in which the script is executing. This property uses values from the runtime.EnvType enum.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug('Environment for current user: ' + JSON.stringify(runtime.envType));
4 ...
5 // Add additional code

```

runtime.executionContext

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The execution context trigger of the current script. Execution contexts provide information about how a script is triggered to execute. For example, a script can be triggered in response to an action in the NetSuite application, or an action occurring in another context, such as a web services integration. You can use execution context filtering to ensure that your scripts are triggered only when necessary. For more information, see the help topic Execution Contexts. You can use the <code>runtime.executionContext</code> property to determine the execution context for a script and choose different logic depending on the context. Consider a script that applies to customer records and uses the beforeLoad(context) entry point. You may not want to execute this script when the entry point is triggered in response to a REST web services request. To prevent the script from executing in this context, you can do the following: <pre> 1 if (runtime.executionContext === runtime.ContextType.REST_WEBSERVICES) { 2 return; 3 } </pre> This property uses values from the runtime.ContextType enum.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 if (runtime.executionContext !== runtime.ContextType.USEREVENT)
4     return;
5 ...
6 // Add additional code
```

runtime.processorCount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of processors available to the current account. SuiteCloud Processors is the current system used to execute (process) scheduled scripts and map/reduce scripts. This property is helpful if you are a SuiteApp developer and your script needs to know the total number of processors available to a deployment. For scheduled script deployments that continue to use queues, use runtime.queueCount. With the introduction of SuiteCloud Processors, map/reduce script deployments and new scheduled script deployments no longer use queues, but pre-existing scheduled script deployments continue to use queues until the queues are removed (see the help topic SuiteCloud Processors Supported Task Types). Be aware that the number of processors available may not be the same as the number of queues available. For more information, see the help topic SuiteCloud Plus Settings. Note: The <code>runtime.processorCount</code> property reflects the number of processors available to an account. It is not impacted by changes to deployments. The value is the same regardless of whether deployments continue to use queues. For more information, see the help topic SuiteCloud Processors Supported Task Types. For more information about scheduled scripts, see the help topic SuiteScript 2.x Scheduled Script Type. For more information about map/reduce scripts, see the help topic SuiteScript 2.x Map/Reduce Script Type.
Type	number (read-only)
Module	N/runtime Module
Since	2018.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 log.debug('Number of processors available: ' + runtime.processorCount);
4 ...
5 // Add additional code
```

runtime.queueCount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of scheduled script queues available to the current account. SuiteCloud Processors is the current system used to execute (process) scheduled scripts and map/reduce scripts. This property is helpful if you are a SuiteApp developer and your script needs to know the total number of queues available to a deployment. For map/reduce script deployments, use runtime.processorCount. With the introduction of SuiteCloud Processors, no map/reduce script deployments use queues (see the help topic SuiteCloud Processors Supported Task Types). Be aware that the number of queues available may not be the same as the number of processors available (see the help topic SuiteCloud Plus Settings). Note: If all scheduled script deployments in an account are configured to no longer use queues (see the help topic SuiteCloud Processors Supported Task Types), the value of <code>runtime.queueCount</code> is unchanged. This property reflects the number of queues available to an account. It is not impacted by changes to deployments. For more information about scheduled scripts, see the help topic SuiteScript 2.x Scheduled Script Type.
Type	number (read-only)
Module	N/runtime Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 // Add additional code
2 ...
3 log.debug('Number of queues available: ' + runtime.queueCount);
4 ...
5 // Add additional code

```

runtime.version

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The version of NetSuite that the method is called in. For example, the <code>runtime.version</code> property in an account running NetSuite 2023.2 is 2023.2. For example, you can use this method when installing a bundle in another NetSuite accounts to find out the version number before installing the bundle.
Type	string (read-only)
Module	N/runtime Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 // Add additional code
2 ...
3 log.debug('Current NetSuite version: ' + runtime.version);
4 ...
5 // Add additional code
```

runtime.ContextType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the execution context values for script triggers. This is the type for the <code>runtime.executionContext</code> property.
Module	N/runtime Module
Sibling Module Members	N/runtime Module Members
Since	2015.2

Values

For a description of each execution context value, see the help topic [Execution Context Types](#).

Value	Sets <code>runtime.executionContext</code> Property To
ACTION	ACTION
ADVANCEDREVREC	ADVANCEDREVREC
BANKCONNECTIVITY	BANKCONNECTIVITY
BANKSTATEMENTPARSER	BANKSTATEMENTPARSER
BUNDLE_INSTALLATION	BUNDLEINSTALLATION
CLIENT	CLIENT
CONSOLRATEADJUSTOR	CONSOLRATEADJUSTOR
CSV_IMPORT	CSVIMPORT
CUSTOMGLLINES	CUSTOMGLLINES
CUSTOM_MASSUPDATE	CUSTOMMASSUPDATE
DATASETBUILDER	DATASETBUILDER

Value	Sets runtime.executionContext Property To
DEBUGGER	DEBUGGER
EMAIL_CAPTURE	EMAILCAPTURE
FICONNECTIVITY	FICONNECTIVITY
FIPARSER	FIPARSER
MAP_REDUCE	MAPREDUCE
NONE	NONE
OCRPLUGIN	OCRPLUGIN
PAYMENTGATEWAY	PAYMENTGATEWAY
PAYMENTPOSTBACK	PAYMENTPOSTBACK
PLATFORMEXTENSION	PLATFORMEXTENSION
PORTLET	PORTLET
PROMOTIONS	PROMOTIONS
RECORDACTION	RECORDACTION
RESTLET	RESTLET
REST_WEBSERVICES	RESTWEBSERVICES
SCHEDULED	SCHEDULED
SDF_INSTALLATION	SDFINSTALLATION
SHIPPING_PARTNERS	SHIPPINGPARTNERS
SUITELET	SUITELET
TAX_CALCULATION	TAXCALCULATION
USEREVENT	USEREVENT
USER_INTERFACE	USERINTERFACE
WEBAPPLICATION	WEBAPPLICATION
WEBSERVICES	WEBSERVICES
WEBSTORE	WEBSTORE
WORKBOOKBUILDER	WORKBOOKBUILDER
WORKFLOW	WORKFLOW

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```
1 | //Add additional code
```

```

2  ...
3  if (runtime.executionContext !== runtime.ContextType.USEREVENT)
4      return;
5  ...
6  //Add additional code

```

runtime.EnvType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds all possible environment types that the current script can execute in. This is the type for the runtime.envType property.
Module	N/runtime Module
Sibling Module Members	N/runtime Module Members
Since	2015.2

Values

Value
BETA
INTERNAL
PRODUCTION
SANDBOX

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var myEnvType = JSON.stringify(runtime.envType);
4
5  if (myEnvType === runtime.EnvType.SANDBOX) {
6      log.debug('You are using your sandbox!');
7  }
8  ...
9  //Add additional code

```


runtime.Permission

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the user permission level for a specific permission ID. This is the type returned by the User.getPermission(options) method. For information about working with NetSuite permissions, see the help topics NetSuite Permissions Overview and Permission Names and IDs .
Module	N/runtime Module
Sibling Module Members	N/runtime Module Members
Since	2015.2

Values

Value
FULL
EDIT
CREATE
VIEW
NONE

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/runtime Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var userObj = runtime.getCurrentUser();
4 var userPermission = userObj.getPermission({
5     name: 'ADMI_ACCOUNTING'
6 });
7 log.debug('User permission of ADMI_ACCOUNTING: ' +
8     (userPermission === runtime.Permission.FULL ? 'FULL' : userPermission);
9 ...
10 //Add additional code

```

N/scriptTypes/restlet Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/scriptTypes/restlet module to create custom HTTP responses for your RESTlet script.

Note: This module is available only to the RESTlet script type.

[Go To Samples {...}](#)

In This Help Topic

- [N/scriptTypes/restlet Module Members](#)
- [Response Object Members](#)

N/scriptTypes/restlet Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	restlet.Response	Object (read-only)	RESTlet script	An HTTP response of a RESTlet script.
Method	restlet.createResponse (options)	restlet.Response	RESTlet script	Creates a custom HTTP response for a RESTlet script.

Response Object Members

The following members are called on the [restlet.Response](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Response.content	string (read-only)	RESTlet script	The content of the RESTlet HTTP response.
	Response.contentType	string (read-only)	RESTlet script	The Content-Type header of the RESTlet HTTP response.

N/scriptTypes/restlet Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/scriptTypes/restlet module.

Return a JSON Object Based on Query Results

The following sample queries employee supervisors, and then sets the RESTlet response Content-Type header to application/json.

Note: This sample script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType RESTlet
4   */
5   define(['N/scriptTypes/restlet', 'N/query'],
6     function(restlet, query) {
7
8       // RESTlet entry point
9       const get = function (requestParams) {
10         // Create Employee query
11         const qObj = query.create({
12           type: query.Type.EMPLOYEE
13         });
14         // Set query condition for the Supervisor field on the employee record, it should not be empty
15         qObj.condition = qObj.createCondition({
16           fieldId: 'supervisor',
17           operator: query.Operator.EMPTY_NOT
18         });
19         // Add supervisor value in the query results
20         qObj.columns = [qObj.createColumn({
21           fieldId: 'supervisor'
22         })]
23
24         const returnArray = [];
25
26         // Retrieve the results
27         const results = qObj.run().results;
28         for (var i = results.length - 1; i >= 0; i--) {
29           const obj = results[i];
30           returnArray.push(obj.values[0]);
31         }
32
33         // Create a RESTlet custom response to set the Content-Type header to JSON object
34         const response = restlet.createResponse({
35           content: JSON.stringify(returnArray),
36           contentType: 'application/json'
37         });
38
39         log.debug('response', JSON.stringify(response))
40
41         return response;
42       }
43
44       return { get };
45     });

```

restlet.Response

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An HTTP response for a RESTlet script. This object is read-only. Use <code>restlet.createResponse(options)</code> to create and return this object.
Supported Script Types	RESTlet scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/scriptTypes/restlet Module
Methods and Properties	Response Object Members
Since	2024.1

Syntax



Important: The following code sample shows a basic implementation for this member. For a complete script example, see [N/scriptTypes/restlet Module Script Sample](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType RESTlet
4   */
5   define(['N/scriptTypes/restlet', 'N/query'],
6     function(restlet, query) {
7
8       const get = function (requestParams) {
9         const htmlResponse = restlet.createResponse({
10           content: '<h1>Hello World</h1>',
11           contentType: 'text/html'
12         });
13
14         log.debug({
15           title: 'Response Content-Type header: ',
16           details: htmlResponse.contentType
17         });
18
19         return htmlResponse;
20       };
21       return { get: get };
22     });

```

Response.content

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The content of the RESTlet HTTP response.
Type	string (read-only)
Module	N/scriptTypes/restlet Module
Parent Object	restlet.Response
Sibling Object Members	Response Object Members
Since	2024.1

Syntax



Important: The following code sample shows a basic implementation for this member. For a complete script example, see [N/scriptTypes/restlet Module Script Sample](#).

```

1  /**
2   * @NApiVersion 2.1

```

```

3  * @NScriptType RESTlet
4  */
5  define(['N/scriptTypes/restlet', 'N/query'],
6      function(restlet, query) {
7
8      const get = function (requestParams) {
9          const htmlResponse = restlet.createResponse({
10              content: '<h1>Hello World</h1>',
11              contentType: 'text/html'
12          });
13
14          log.debug({
15              title: 'Response Content: ',
16              details: htmlResponse.content
17          });
18
19          return htmlResponse;
20      };
21      return { get: get };
22  });

```

Response.contentType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The Content-Type header of the RESTlet HTTP response.
Type	string (read-only)
Module	N/scriptTypes/restlet Module
Parent Object	restlet.Response
Sibling Object Members	Response Object Members
Since	2024.1

Syntax



Important: The following code sample shows a basic implementation for this member. For a complete script example, see [N/scriptTypes/restlet Module Script Sample](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType RESTlet
4   */
5  define(['N/scriptTypes/restlet', 'N/query'],
6      function(restlet, query) {
7
8      const get = function (requestParams) {
9          const htmlResponse = restlet.createResponse({
10              content: '<h1>Hello World</h1>',
11              contentType: 'text/html'
12          });
13
14          log.debug({
15              title: 'Response Content-Type header: ',
16              details: htmlResponse.contentType
17          });
18
19          return htmlResponse;
20      };
21      return { get: get };

```

22 | });

restlet.createResponse(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a custom HTTP response for a RESTlet.
Returns	restlet.Response
Supported Script Types	RESTlet scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/scriptTypes/restlet Module
Sibling Module Members	N/scriptTypes/restlet Module Members
Since	2024.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.content	string	required	The content of the response.
options.contentType	string	required	The Content-Type header of the response. This value overrides the default Content-Type header, which is the same as the Content-Type header of the RESTlet HTTP request.

Errors

Error Code	Error Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	{method name}: Missing a required argument: {param name}	A required parameter is missing.
SSS_INVALID_TYPE_ARG	You have entered an invalid type argument: {param name}	A parameter has an invalid type.

Syntax

Important: The following code sample shows a basic implementation for this member. For a complete script example, see [N/scriptTypes/restlet Module Script Sample](#).

1 | /**

```

2  * @NApiVersion 2.1
3  * @NScriptType RESTlet
4  */
5  define(['N/scriptTypes/restlet', 'N/query'],
6      function(restlet, query) {
7
8      const get = function (requestParams) {
9          return restlet.createResponse({
10              content: '<h1>Hello World</h1>',
11              contentType: 'text/html'
12          });
13      };
14      return { get: get };
15  });

```

N/search Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/search module to create and run on-demand or saved searches, then analyze and work through the results. With this module, you can do things like:

- Search for a single record using keywords
- Create and save searches
- Load and run saved searches
- Search for duplicate records
- Return a set of records that match your defined filter criteria

You can also paginate results and add navigation to jump between pages. This makes it great for handling large result sets.



Important: The N/search module doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers [Outbound HTTPs in an unauthenticated client-side context](#).

Go To Samples

In This Help Topic

- [N/search Module Members](#)
- [Column Object Members](#)
- [Filter Object Members](#)
- [Page Object Members](#)
- [PagedData Object Members](#)
- [PageRange Object Members](#)
- [Result Object Members](#)
- [ResultSet Object Members](#)
- [Search Object Members](#)

■ Setting Object Members

N/search Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	search.Column	Object	Client and server scripts	Encapsulates a single search column in a search.Search object. Use the methods and properties available to the Column object to get or set Column properties.
	search.Filter	Object	Client and server scripts	Encapsulates a search filter used in a search. Use the properties for the Filter object to get and set the filter properties.
	search.Page	Object	Client and server scripts	Encapsulates a set of search results for a single search page.
	search.PagedData	Object	Client and server scripts	Holds metadata about a paginated query.
	search.PageRange	Object	Client and server scripts	Defines the page range to bound the result set for a paginated query.
	search.Result	Object	Client and server scripts	Encapsulate a single search result row. Use the methods and properties for the Result object to get the column values for the result row.
	search.ResultSet	Object	Client and server scripts	Encapsulates a set of search results returned by Search.run() .
	search.Search	Object	Client and server scripts	Encapsulates a NetSuite search. Use the methods available to the Search object to create a search, run a search, or save a search.
	search.Setting	Object	Client and server scripts	Encapsulates a search setting. Search settings let you specify search parameters that are typically available only in the UI.
Method	search.create(options)	search.Search	Client and server scripts	Creates a new search and returns it as a search.Search object.
	search.create.promise(options)	search.Search	Client and server scripts	Creates a new search asynchronously and returns it as a search.Search object.
	search.createColumn(options)	search.Column	Client and server scripts	Creates a new search column as a search.Column object.
	search.createFilter(options)	search.Filter	Client and server scripts	Creates a new search filter as a search.Filter object.
	search.createSetting(options)	search.Setting	Client and server scripts	Creates a new search setting and returns it as a search.Setting object.
	search.delete(options)	void	Client and server scripts	Deletes an existing saved search asynchronously and returns it as a search.Search object.
	search.delete.promise(options)	void	Client and server scripts	Deletes an existing saved search and returns it as a search.Search object.
	search.duplicates(options)	search.Result []	Client and server scripts	Performs a search for duplicate records based on the duplicate detection

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				configuration for the account. Returns an array of search.Result objects.
	search.duplicates.promise(options)	search.Result []	Client and server scripts	Performs a search for duplicate records asynchronously based on the duplicate detection configuration for the account. Returns an array of search.Result objects.
	search.global(options)	search.Result []	Client and server scripts	Performs a global search against a single keyword or multiple keywords.
	search.global.promise(options)	search.Result []	Client and server scripts	Performs a global search asynchronously against a single keyword or multiple keywords.
	search.load(options)	search.Search	Client and server scripts	Loads an existing saved search and returns it as a search.Search object.
	search.load.promise(options)	search.Search	Client and server scripts	Loads an existing saved search asynchronously and returns it as a search.Search object.
	search.lookupFields(options)	Object array	Client and server scripts	Performs a search for one or more body fields on a record. Returns select fields as an object with value and text properties. Returns multiselect fields as an object with value:text pairs.
	search.lookupFields.promise(options)	Object array	Client and server scripts	Performs a search asynchronously for one or more body fields on a record. Returns select fields as an object with value and text properties. Returns multiselect fields as an object with value:text pairs.
Enum	search.Operator	enum	Client and server scripts	Holds the values for search operators to use with the search.Filter .
	search.Sort	enum	Client and server scripts	Holds the values for supported sorting directions used with search.createColumn(options) .
	search.Summary	enum	Client and server scripts	Holds the values for summary types used by the Column.summary object.
	search.Type	enum	Client and server scripts	Holds the string values for search types supported in the N/search Module . Use this enum to set the value for the <code>options.type</code> parameter of the search.create(options) method.

Column Object Members

The following members are available for a [search.Column](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Column.setWhenOrderedBy(options)	search.Column	Client and server scripts	Returns the search column for which the minimal or maximal value should be found when returning the search.Column value.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Column.formula	string	Client and server scripts	Formula used for a search column as a string.
	Column.function	string	Client and server scripts	Special function used in the search column as a string.
	Column.join	string (read-only)	Client and server scripts	Join ID for a search column as a string.
	Column.label	string	Client and server scripts	Label used for the search column. You can only get or set custom labels with this property.
	Column.name	string (read-only)	Client and server scripts	Name of a search column as a string.
	Column.summary	string (read-only)	Client and server scripts	Returns the summary type for a search column.

Filter Object Members

The following members are available for a [search.Filter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Filter.formula	string	Client and server scripts	Formula used by the search filter.
	Filter.join	string (read-only)	Client and server scripts	Join ID for the search filter.
	Filter.name	string (read-only)	Client and server scripts	Name or internal ID of the search field.
	Filter.operator	string (read-only)	Client and server scripts	Operator used for the search filter.
	Filter.summary	search.Summary	Client and server scripts	Summary type for the search filter.

Page Object Members

The following members are available for a [search.Page](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Page.next()	void	Client and server scripts	Gets the next segment of data from a paginated search
	Page.next.promise()	void	Client scripts	Asynchronously gets the next segment of data from a paginated search

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Page.prev()	void	Client and server scripts	Gets the previous segment of data from a paginated search
	Page.prev.promise()	void	Client scripts	Asynchronously gets the previous segment of data from a paginated search
Property	Page.data	search.Result[]	Client and server scripts	The results from a paginated search.
	Page.isFirst	boolean (read-only)	Client and server scripts	Indicates whether a page is the first page of data for a result set
	Page.isLast	boolean (read-only)	Client and server scripts	Indicates whether a page is the last page of data for a result set.
	Page.pagedData	search.PagedData (read-only)	Client and server scripts	The PagedData Object used to fetch this Page Object.
	Page.pageRange	search.PageRange (read-only)	Client and server scripts	The PageRange Object used to fetch this Page Object.

PagedData Object Members

The following members are available for a [search.PagedData](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	PagedData.fetch(options)	search.Page	Client and server scripts	Retrieves the data within the specified page range.
	PagedData.fetch.promise()	search.Page	Client scripts	Asynchronously retrieves the data within the specified page range.
Property	PagedData.count	number (read-only)	Client and server scripts	The total number of results when Search.runPaged(options) was executed.
	PagedData.pageRanges	search.PageRange[] (read-only)	Client and server scripts	The collection of PageRange objects that divide the entire result set into smaller groups.
	PagedData.pageSize	number (read-only)	Client and server scripts	The maximum number of entries per page
	PagedData.searchDefinition	search.Search (read-only)	Client and server scripts	The search criteria used when Search.runPaged(options) was executed.

PageRange Object Members

The following members are available for a [search.PageRange](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PageRange.compoundLabel	string (read-only)	Client and server scripts	Human-readable label with beginning and ending range identifiers
	PageRange.index	number (read-only)	Client and server scripts	The index of this page range.

Result Object Members

The following members are available for a [search.Result](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Result.getText(column)	string	Client and server scripts	The text value for a search.Column if it is a stored select field.
	Result.getText(options)	string	Client and server scripts	The UI display name, or text value, for a search result column. This method is supported only for non-stored select, image, and document fields.
	Result.getValue(options)	string	Client and server scripts	Used on formula fields and non-formula (standard) fields to get the value of a specified search return column.
	Result.getValue(column)	string	Client and server scripts	Used on formula and non-formula (standard) fields. Returns the string value of a specified search result column. For convenience, this method takes a single search.Column Object.
Property	Result.columns	search.Column []	Client and server scripts	Array of search.Column objects that encapsulate the columns returned in the search result row.
	Result.id	string (read-only)	Client and server scripts	The internal ID for the record returned in a search result row.
	Result.recordType	string (read-only)	Client and server scripts	The type of record returned in a search result row.

ResultSet Object Members

The following members are available for a [search.ResultSet](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ResultSet.each(callback)	void	Client and server scripts	Use a developer-defined function to invoke on each row

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				in the search results, up to 4000 results at a time.
	ResultSet.each.promise(callback)	void	Client scripts	Asynchronously use a developer-defined function to invoke on each row in the search results, up to 4000 results at a time.
	ResultSet.getRange(options)	search.Result[]	Client and server scripts	Retrieve a slice of the search result as an array of search.Result objects.
	ResultSet.getRange.promise(options)	search.Result[]	Client scripts	Asynchronously retrieve a slice of the search result as an array of search.Result objects.
Property	ResultSet.columns	search.Column[]	Client and server scripts	An array of search.Column objects that represent the columns returned in the search results.

Search Object Members

The following members are available for a [search.Search](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Search.run()	search.ResultSet	Client and server scripts	Runs an on demand search created with search.create(options) or a search loaded with search.load(options) , returning the results as a search.ResultSet .
	Search.runPaged(options)	search.PagedData	Client and server scripts	Runs the current search and returns a search.PagedData Object.
	Search.runPaged.promise(options)	search.PagedData	Client and server scripts	Asynchronously runs the current search and returns a search.PagedData Object.
	Search.save()	number	Client and server scripts	Saves a search created by search.create(options) or loaded with search.load(options) . Returns the internal ID of the saved search.
	Search.save.promise()	number	Client and server scripts	Asynchronously saves a search created by search.create(options) or loaded with search.load(options) . Returns the internal ID of the saved search.
Property	Search.columns	search.Column[] string[]	Client and server scripts	Columns to return for this search as an array of search.Column objects or a string array of column names.
	Search.filterExpression	Object[]	Client and server scripts	Search filter expression for the search as an array of expression objects.
	Search.filters	search.Filter[]	Client and server scripts	Filters for the search as an array of search.Filter objects.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Search.id	string	Client and server scripts	Script ID for a saved search, starting with customsearch.
	Search.isPublic	boolean	Client and server scripts	Value is true if the search is public, or false if it is not.
	Search.packageId	string	Client and server scripts	The application ID for the search.
	Search.searchId	number	Client and server scripts	Internal ID of a search.
	Search.searchType	string	Client and server scripts	Search type on which a search is based.
	Search.settings	search.Setting[] string[]	Client and server scripts	Search settings for this search as an array of search.Setting objects or a string array of column names.
	Search.title	string	Client and server scripts	Title for a saved search. Use this property to set the title for a search before you save it for the first time.

Setting Object Members

The following members are available for a [search.Setting](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Setting.name	string (read-only)	Client and server scripts	The name of the search parameter.
	Setting.value	string (read-only)	Client and server scripts	The value of the search parameter.

N/search Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/search module:

- [Search for Customer Records and Log First 50 Results](#)
- [Search for Sales Order Records](#)
- [Load a Search for Sales Order Records and Use a Callback Function to Process Results](#)
- [Load a Search for Sales Order Records and Return the First 100 Search Results](#)
- [Load and Run a Paginated Search and Process the Results](#)
- [Create a Search for a Custom Record Type](#)
- [Search for Items in a Custom List](#)
- [Delete a Saved Search](#)
- [Search Using a Specific Record Field](#)



Important: These samples are designed to run in a NetSuite OneWorld account, so the OneWorld feature must be enabled in your NetSuite account for the samples to work.

Search for Customer Records and Log First 50 Results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a search for customer records. The sample specifies several result columns and one filter, and it logs the first 50 search results.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      var mySearch = search.create({
7          type: search.Type.CUSTOMER,
8          columns: ['entityid', 'firstname', 'lastname', 'salesrep'],
9          filters: ['entityid', 'contains', 'Adam']
10     });
11
12     var myResultSet = mySearch.run();
13
14     var resultRange = myResultSet.getRange({
15         start: 0,
16         end: 50
17     });
18
19     for (var i = 0; i < resultRange.length; i++) {
20         log.debug(resultRange[i]);
21     }
22 });

```

Search for Sales Order Records

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a search for sales order records and saves it. The sample specifies several result columns and two filters.



Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      function createSearch() {
7          var mySalesOrderSearch = search.create({
8              type: search.Type.SALES_ORDER,

```

```

9      title: 'My SalesOrder Search',
10     id: 'customsearch_my_so_search',
11     columns: ['entity', 'subsidiary', 'name', 'currency'],
12     filters: [
13         ['mainline', 'is', 'T'],
14         'and', ['subsidiary.name', 'contains', 'CAD']
15     ]
16 });
17
18 mySalesOrderSearch.save();
19 }
20
21 createSearch();
22 });

```

Load a Search for Sales Order Records and Use a Callback Function to Process Results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads and runs a saved search for sales order records. The sample uses the each callback function to process the results.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      function loadAndRunSearch() {
7          var mySearch = search.load({
8              id: 'customsearch_my_so_search'
9          });
10
11          mySearch.run().each(function(result) {
12              var entity = result.getValue({
13                  name: 'entity'
14              });
15              var subsidiary = result.getValue({
16                  name: 'subsidiary'
17              });
18
19              return true;
20          });
21      }
22
23      loadAndRunSearch();
24  });

```

Load a Search for Sales Order Records and Return the First 100 Search Results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads and runs a saved search for sales order records. The sample obtains the first 100 rows of search results.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      function runSearchAndFetchResult() {
7          var mySearch = search.load({
8              id: 'customsearch_my_so_search'
9          });
10
11         var searchResult = mySearch.run().getRange({
12             start: 0,
13             end: 100
14         });
15         for (var i = 0; i < searchResult.length; i++) {
16             var entity = searchResult[i].getValue({
17                 name: 'entity'
18             });
19             var subsidiary = searchResult[i].getValue({
20                 name: 'subsidiary'
21             });
22         }
23     }
24
25     runSearchAndFetchResult();
26 });

```

Load and Run a Paginated Search and Process the Results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads and runs a saved search for sales order records. The sample uses the `forEach` callback function to process the paginated results.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      function loadAndRunSearch() {
7          var mySearch = search.load({
8              id: 'customsearch_my_so_search'
9          });
10
11         var myPagedData = mySearch.runPaged();
12         myPagedData.pageRanges.forEach(function(pageRange){
13             var myPage = myPagedData.fetch({index: pageRange.index});
14             myPage.data.forEach(function(result){
15                 var entity = result.getValue({
16                     name: 'entity'
17                 });
18                 var subsidiary = result.getValue({
19                     name: 'subsidiary'
20                 });
21             });
22         });
23     }
24
25     loadAndRunSearch();
26 });

```

```

21         });
22     });
23 }
24
25 loadAndRunSearch();
26 });

```

Create a Search for a Custom Record Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

The following sample creates a search for a custom record type. To search for a custom record type, you must specify a type of `search.Type.CUSTOM_RECORD` and add the ID of the custom record type (as a string). In this sample, the ID of the custom record type is 6. The custom record also includes a custom field named `custrecord1`.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      var myCustomRecordSearch = search.create({
7          type: search.Type.CUSTOM_RECORD + '6',
8          title: 'My Search Title',
9          columns: ['custrecord1']
10     }).run().each(function(result) {
11         // Process each result
12         return true;
13     });
14 });

```

Search for Items in a Custom List

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a search for items in a custom list. It searches for the internal ID value of an abbreviation in a custom list named `customlist_mylist`.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      var internalId = -1;
7      var myCustomListSearch = search.create({
8          type: 'customlist_mylist',
9          columns: [
10             { name: 'internalId' },
11             { name: 'abbreviation' }
12         ]
13     });

```

```

14
15     myCustomListSearch.filters = [
16         search.createFilter({
17             name: 'formulatext',
18             formula: '{abbreviation}',
19             operator: search.Operator.IS,
20             values: abbreviation
21         });
22     ]
23
24     var resultSet = myCustomListSearch.run();
25     var results = resultSet.getRange({
26         start: 0,
27         end: 1
28     });
29     for(var i in results) {
30         // log.debug('Found custom list record', results[i]);
31         internalId = results[i].getValue({
32             name: 'internalid'
33         });
34     };
35 });

```

Delete a Saved Search

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to delete a saved search.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/search'], function(search) {
6      function deleteSearch() {
7          search.delete({
8              id: 'customsearch_my_so_search'
9          });
10     }
11
12     deleteSearch();
13 });

```

Search Using a Specific Record Field

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to search for records using the value of the `operationdisplaytext` field. If you run this sample in your account, be sure to replace the placeholder value `<transactionId>` with a valid value from your account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**

```

```

2  * @NApiVersion 2.x
3  */
4  require(['N/search'], function(search) {
5      var mySearch = search.create({
6          type: search.Type.TRANSACTION,
7          columns: ['item', 'operationdisplaytext'],
8          filters: [
9              ['internalid', search.Operator.ANYOF, '<transactionId>'],
10             'and',
11             ['operationdisplaytext', search.Operator.EQUALTO, 20]]
12      });
13
14      var myResultSet = mySearch.run();
15      var resultRange = myResultSet.getRange({
16          start: 0,
17          end: 20
18      });
19  });

```

search.Column

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a single search column in a search.Search. Use the methods and properties available to the Column object to get or set Column properties. You create a search column object with search.createColumn(options) and add it to a search.Search object that you create with search.create(options) or load with search.load(options). You can pass a Column object as a parameter to the Result.getValue(column) or Result.getText(column) methods. In addition, search.ResultSet contains an array of Column objects returned in the results of a search.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/search Module
Methods and Properties	Column Object Members
Since	2015.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1  //Add additional code
2  ...
3  search.create({
4      type: search.Type.TRANSACTION,
5      columns: [
6          'trandate',
7          'amount',
8          'entity',
9          'entity.firstname',
10         'entity.email',
11         search.createColumn({

```

```

12         name: 'formulatext',
13         formula: "{lastname}||', '||{firstname}"
14     })
15 },
16
17     // When the search is executed, the corresponding column in the result will then contain a value in the form: Last Name, First Name
18     ...
19     //Add additional code

```

Column.setWhenOrderedBy(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the search column for which the minimal or maximal value should be found when returning the search.Column value. For example, can be set to find the most recent or earliest date, or the largest or smallest amount for a record, and then the search.Column value for that record is returned.  Note: You can only use this method if you use MIN or MAX as the summary type on a search column with the Result.getValue(options) method.
Returns	search.Column
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/search Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The name of the search column for which the minimal or maximal value should be found.
options.join	string	required	The join id for the search column.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Execute a customer search that returns the amount of the most recent sales order per customer
4
5 var filters = [];
6 var columns = [];
7

```

```

8 filters[0] = search.createFilter({
9   name: 'recordtype',
10  join: 'transaction',
11  operator: search.Operator.IS,
12  values: 'salesorder'
13 });
14 filters[1] = search.createFilter({
15   name: 'mainline',
16   join: 'transaction',
17   operator: search.Operator.IS,
18   values: true
19 });
20
21 columns[0] = search.createColumn({
22   name: 'entityid',
23   summary: search.Summary.GROUP
24 });
25 columns[1] = search.createColumn({
26   name: 'totalamount',
27   join: 'transaction',
28   summary: search.Summary.MAX
29 });
30
31 columns[1].setWhenOrderedBy({
32   name: 'trandate',
33   join: 'transaction'
34 });
35
36 var mySearch = search.create({
37   type: 'customer',
38   filters: filters,
39   columns: columns
40 });
41 var resultsArray = mySearch.run().getRange({
42   start: 0,
43   end: 100
44 });
45 ...
46 // Add additional code

```

Column.formula

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Formula used for a search column as a string. To set this value, you must use <code>formulatext</code> , <code>formulanumeric</code> , <code>formuladatetime</code> , <code>formulapercent</code> , or <code>formulacurrency</code> .
Type	string
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

For example, in the UI, a field with a custom UI label named **Customer Name** is set by a formula of type **Formula (Text)** and the formula is defined with the following formula:

```
1 //Add additional code
```

```

2  ...
3  var columnObj = search.createColumn({
4      name: 'formulatext',
5      formula: "{firstname} || ', ' || {lastname}"
6  });
7  ...
8  //Add additional code

```

In the above formula, `firstname` and `lastname` are script IDs for the fields on the Customer record form.

Column.function

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Special function applied to values in a search column. See Supported Functions .
Type	string
Module	N/search Module
Since	2015.2

Supported Functions

The following table lists the supported functions and their internal IDs:

Internal ID	Name	Date Function	Output
percentOfTotal	% of Total	No	percent
absoluteValue	Absolute Value	No	integer
ageInDays	Age In Days	Yes	integer
ageInHours	Age In Hours	Yes	integer
ageInMonths	Age In Months	Yes	integer
ageInWeeks	Age In Weeks	Yes	integer
ageInYears	Age In Years	Yes	integer
calendarWeek	Calendar Week	Yes	date
day	Day	Yes	date
month	Month	Yes	text
negate	Negate	No	integer
numberAsTime	Number as Time	No	text
quarter	Quarter	Yes	text
rank	Rank	No	integer
round	Round	No	float
roundToHundredths	Round to Hundredths	No	float
roundToTenths	Round to Tenths	No	float

Internal ID	Name	Date Function	Output
weekOfYear	Week of Year	Yes	text
year	Year	Yes	text

Errors

Error Code	Message	Thrown If
INVALID_SRCH_FUNCN	A search.Column contains an invalid function: {1}.	Unknown function is set.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var columnObj = search.createColumn({ // the age of the sales order in days
4   name: 'trandate',
5   function: 'ageInDays'
6 });
7 ...
8 //Add additional code

```

Column.join

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Join ID for a search column as a string.
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Create a joined column. The name parameter is a field name on
4 // the joined record type, and the join parameter is a field name
5 // on the base record type that references the joined record type.
6 //
7 // In this example, a search is created for customer records.
8 // Customer records include a 'salesrep' field, which references an
9 // employee record. This joined column represents the value of the
10 // 'firstname' field on the referenced employee record.
11 var myColumn = search.createColumn({
12   name: 'firstname',
13   join: 'salesrep'

```



```

14 });
15
16 var mySearch = search.create({
17     type: search.Type.CUSTOMER,
18     columns: [
19         'entityid',
20         'email',
21         myColumn
22     ]
23 });
24
25 var theJoin = myColumn.join;
26 ...
27 // Add additional code

```

Column.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Label used for the search column. You can only get or set custom labels with this property.  Note: This label can only be set as a custom label or summary label. For columns that use the GROUP summary type, you must use a Custom label. For columns with non group summary types, you must use the Summary label. In SuiteScript, it is not possible to set both labels simultaneously.
Type	string
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var columnObj = search.createColumn({
4     name: 'formulanumeric',
5     label: 'Numeric Formula'
6 });
7 ...
8 //Add additional code

```

Column.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Name of a search column as a string.
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // Create a search definition that includes search columns
4 var mySearch = search.create({
5   type: search.Type.CUSTOMER,
6   columns: [
7     search.createColumn({
8       name: 'entityid'
9     }),
10    search.createColumn({
11      name: 'email'
12    })
13  ]
14 });
15
16 // Retrieve the first search column and log its name
17 var myColumn = mySearch.columns[0];
18 log.debug(myColumn.name);
19
20 // Run the search
21 var results = mySearch.run();
22 ...
23 // Add additional code
```

Column.sort

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort order of the column. Use search.createColumn(options) and a value from the search.Sort enum to set the value of this property. If <code>Column.sort</code> is not set, the column is not sorted in any particular order. After you create a column, you cannot change the sort order of the column. If you use the same column in another search and specify a new sort order, the previous sort order is still used.
Type	search.Sort enum
Module	N/search Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var columnObj = search.createColumn({
4   name: 'invoice',
5   sort: search.Sort.DESC
6 });
7 ...
```

8 //Add additional code

Column.summary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the summary type for a search column.
Type	search.Summary enum
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4     details: 'Summary Type for Search Column: '
5         + columnObj.summary
6 });
7 ...
8 //Add additional code

```

search.Filter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a search filter used in a search. Use the properties for the Filter object to get and set the filter properties. You create a search filter object with search.createFilter(options) and add it to a search.Search object that you create with search.create(options) or load with search.load(options). Note: NetSuite uses an implicit AND operator with search filters, as opposed to filter expressions which explicitly use either AND and OR operators. Use the following guidelines with the Filter object: ■ To search for a "none of null" value, meaning do not show results without a value for the specified field, use a value of @NONE@ in the Filter.formula property. ■ To search on checkbox fields, use the IS operator with a value of T or F to search for checked or unchecked fields, respectively.
Module	N/search Module
Methods and Properties	Filter Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearchFilter = search.createFilter({
4     name: 'entity',
5     operator: search.Operator.ISEMPY,
6 });
7 ...
8 //Add additional code

```

Filter.formula

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Formula used by the search filter. Use this property to get or set the formula used by the search filter. For more information about the formula property, see search.createFilter(options). For more information about using formulas in searches, see the help topic Formulas in Searches.
Type	string
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

For more examples of search formulas, see the help topic [Search Formula Examples and Tips](#).

```

1 // Add additional code
2 ...
3 var result = search.create({
4     type: 'employee',
5     columns: ['firstname', 'lastname', 'role'],
6     filters: [
7         search.createFilter({
8             name: 'formulanumeric',
9             formula: '{today} - {hiredate}',
10            operator: search.Operator.LESSTHAN,
11            values: 30
12        })
13    ]
14 })
15 result.run().getRange({
16     start: 0,
17     end: 100
18 });
19 ...
20 // Add additional code

```

Filter.join

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Join ID for the search filter as a string.
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Create a filter joined to another record type. When you create a joined filter:
4 // - The name property is the field ID of the field in the joined record that you are filtering on
5 // - The join property is the field ID of the field in the current record that contains the record
6 //   type you want to join to
7 // - The operator property is the operator to use to filter the results
8 // - The values property contains the values to use to filter the results
9 search.createFilter({
10   name: 'joined_record_field_id'
11   join: 'current_record_field_id',
12   operator: search.Operator.IS,
13   values: ['valueToFilter']
14 });
15 ...
16 // Add additional code

```

Filter.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Name or internal ID of the search field as a string. For more information, see search.createFilter(options) .
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4   details: 'Filter Name: '
5     + filterObj.name

```

```

6     });
7     ...
8     //Add additional code

```

Filter.operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Operator used for the search filter. This value is set with the search.Operator enum. The <code>search.Operator</code> enum contains the valid operator values for this property.
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearchFilter = search.createFilter({
4     name: 'entity',
5     operator: search.Operator.ISEMPY
6 });
7 log.debug({
8     details: 'Operator Used: '
9         + mySearchFilter.operator
10 });
11 ...
12 //Add additional code

```

Filter.summary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Summary type for the search filter. Use this property to get or set the value of the summary type. See search.Summary .
Type	search.Summary
Module	N/search Module
Since	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_SRCH_FILTER_SUM	A search.Filter contains an invalid summary type: {1}.	Unknown summary type is set.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var mySearchFilter = search.createFilter({
4   name: 'entity',
5   operator: search.Operator.ISNOTEMPTY,
6   summary: search.Summary.GROUP
7 });
8 ...
9
10 //Add additional code
```

search.Page

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates an individual search page containing a result set for a paginated search. Use the PagedData.fetch(options) method to create this object.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/search Module
Methods and Properties	Page Object Members
Since	Version 2015 Release 1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var page = pagedData.fetch({
4   index: lastPageRange.index
5 }); // creates a search.Page object
6 ...
7 //Add additional code
```

Page.next()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to fetch the next segment of data (bounded by search.PageRange). Moves the current page to next range.
---------------------------	--

Returns	Void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2016.1

Errors

Error Code	Message	Thrown If
INVALID_PAGE_RANGE	Invalid page range.	The page range is invalid, or when the page is the last page.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 while (!page.isFirst){
4   page = page.next();
5   ...
6 //Add additional code

```

Page.next.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to asynchronously fetch the next segment of data (bounded by search.PageRange). Moves the current page to another range. The promise is complete when the data for this range is loaded or rejected. Note: The parameters and errors thrown for this method are the same as those for Page.next(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Page.next()
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module

Since	2016.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 // In this sample, myPage is a Page object that encapsulates a page of search results,
4 // and processPage is the name of a callback function to execute when the promise
5 // method returns
6 return myPage.next.promise().then(processPage);
7 ...
8 // Add additional code

```

Page.prev()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to fetch the previous segment of data (bounded by search.PageRange). Moves the current page to previous range.
Returns	Void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2016.1

Errors

Error Code	Message	Thrown If
INVALID_PAGE_RANGE	Invalid page range.	The page range is invalid, or when the page is the first page.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 while (!page.isLast){
4   page = page.prev();
5   ...

```

6 | //Add additional code

Page.prev.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to asynchronously fetch the previous segment of data (bounded by search.PageRange). Moves the current page to another range. The promise is complete when the data for this range is loaded or rejected.  Note: The parameters and errors thrown for this method are the same as those for Page.prev(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Page.prev()
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	5 units
Module	N/search Module
Since	2016.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 return mypage.prev.promise().then(processPage);
4 ...
5 //Add additional code

```

Page.data

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The results from a paginated search.
Type	search.Result[] This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 function processPage(page){
4     page.data.forEach(function(value){
5         log.debug({
6             details: "data: " + page.data
7         });
8     });
9 //Add additional code

```

Page.isFirst

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the page is within the first range of the result set. Flags the start of the data collection.
Type	boolean This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 while (!page.isFirst){
4     page = page.next();
5     ...
6 //Add additional code

```

Page.isLast

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether a page is within the last range of the result set. Flags the end of the data collection.
Type	boolean This property is read-only.
Module	N/search Module

Since	2016.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 while (!page.isLast){
4   page = page.prev();
5   ...
6 //Add additional code

```

Page.pagedData

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The PagedData Object used to fetch this Page Object.
Type	search.PagedData This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var lastPageRange = pagedData.pageRanges[pagedData.pageRanges.length - 1];
4 ...
5 //Add additional code

```

Page.pageRange

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The PageRange Object used to fetch this Page Object. Page boundary information with the key and label.
Type	search.PageRange This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 //Add additional code
2 ...
3 log.debug({
4     details: "Page Range: " + mySearchPage.pageRange
5 });
6 ...
7
8 //Add additional code
```

search.PagedData

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Holds metadata for a paginated query. This object provides a high-level view of a search result, giving the total count of records, a list of pages ranges, and page size. For paged searches, the maximum number of result rows per page is 1000. The minimum number of result rows per page is 5, except for the last page in the result set (because the last page may include fewer than 5 results). Important: Search result sets are not cached. If records applicable to your search are created, modified, or deleted at the same time you are traversing your result set, your result set may change.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/search Module
Methods and Properties	PagedData Object Members
Since	Version 2015 Release 1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 //Add additional code
2 ...
3 // Run the paged search
4 var pagedData = mySearch.runPaged({
5     pageSize: 1000
6 });
7 ...
8 // Use the count property to count the
9 // search results easily
10 var resultCount = mySearch.runPaged({
11     pageSize: 1000
12 }).count;
```

```

13 ...
14 //Add additional code

```

PagedData.fetch(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	This method retrieves the data within the specified page range. This method also includes a promise version, PagedData.fetch.promise() . For more information about promises, see Promise Object .
Returns	search.Page
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
pageRange.index	number	required	The index of the page range that bounds the desired data. Page range indexes start at 0.

Errors

Error Code	Message	Thrown If
INVALID_PAGE_RANGE	Invalid page range.	The page range is not valid.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySearch = search.create({
4   type: search.Type.CUSTOMER,
5   columns: [
6     'entityid',
7     'email'
8   ]
9 });
10

```

```

11 var myPagedResults = mySearch.runPaged({
12     pageSize: 10
13 });
14
15 var thePageRanges = myPagedResults.pageRanges;
16
17 // Use PagedData.fetch(options) to retrieve a page of results (as a
18 // search.Page object) by index
19 var theData = myPagedResults.fetch({
20     index: 5
21 });
22 ...
23 // Add additional code

```

PagedData.fetch.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	This method asynchronously retrieves the data bounded by the pageRange parameter.  Note: The parameters and errors thrown for this method are the same as those for PagedData.fetch(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	PagedData.fetch(options)
Supported Script Types	Client scripts For more information see, SuiteScript 2.x Script Types.
Governance	5 units
Module	N/search Module
Since	2016.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 // Add additional code
2 ...
3 return pagedData.fetch.promise().then(processPage);
4 ...
5 // Add additional code

```

PagedData.count

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The total number of results when Search.runPaged(options) was executed.
Type	number

	This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySearch = search.create({
4   type: search.Type.CUSTOMER,
5   columns: [
6     'entityid',
7     'email'
8   ]
9 });
10
11 var myPagedResults = mySearch.runPaged({
12   pageSize: 10
13 });
14
15 var theCount = myPagedResults.count;
16 ...
17 // Add additional code

```

PagedData.pageRanges

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The collection of search.PageRange objects that divide the entire result set into smaller groups. This property includes page range information with the key and label for rendering. Page range indexes start at 0.
Type	search.PageRange[] This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySearch = search.create({
4   type: search.Type.CUSTOMER,
5   columns: [
6     'entityid',

```



```

7         'email'
8     ]
9 });
10
11 var myPagedResults = mySearch.runPaged({
12     pageSize: 10
13 });
14
15 var thePageRanges = myPagedResults.pageRanges;
16
17 // Use PagedData.fetch(options) to retrieve a page of results (as a
18 // search.Page object) by index
19 var theData = myPagedResults.fetch({
20     index: 5
21 });
22 ...
23 // Add additional code

```

PagedData.pageSize

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Maximum number of entries per page Possible values are 5 - 1000 entries per page.
Type	number This property is read-only.
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySearch = search.create({
4     type: search.Type.CUSTOMER,
5     columns: [
6         'entityid',
7         'email'
8     ]
9 });
10
11 var myPagedResults = mySearch.runPaged({
12     pageSize: 10
13 });
14
15 // In this example, the value of myPagedResults.pageSize is 10
16 var thePageSize = myPagedResults.pageSize;
17
18 // Use PagedData.fetch(options) to retrieve a page of results (as a
19 // search.Page object) by index
20 var theData = myPagedResults.fetch({
21     index: 5
22 });
23 ...
24 // Add additional code

```

PagedData.searchDefinition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The search criteria used to execute the result set for this PagedData Object.
Type	search.Search (read-only)
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4     details: "Search Details: " + myPagedData.searchDefinition
5 });
6 ...
7 //Add additional code

```

search.PageRange

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Defines the page range to contain the result set
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/search Module
Methods and Properties	PageRange Object Members
Since	Version 2015 Release 1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var page = pagedData.fetch({
4     index: lastPageRange
5 });
6 ...
7 //Add additional code

```

PageRange.compoundLabel

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Human-readable label with beginning and ending range identifiers
Type	string (read-only)
Module	N/search Module
Since	2016.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4   details: "Page Range Description: " + myPageRange.compoundLabel
5 });
6 ...
7 //Add additional code

```

PageRange.index

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The index of the page range. Page range indexes start at 0.
Type	number This property is read-only.
Module	N/search Module
Since	2016.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.debug({
4   details: "Page Range Index: " + myPageRange.index
5 });
6 ...
7 //Add additional code
8

```

search.Result

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulate a single search result row. Use the methods and properties for search. Result to get the column values for the result row.  Note: Use search.ResultSet for the set of results from a search. For more information about executing NetSuite searches using SuiteScript, see <i>Searching Overview</i>.  Important: Text columns in search results are limited to 4000 bytes, which is equivalent to 4000 characters in the English language. However, this limit can decrease significantly when texts are stored using intricate character sets, which can require more bytes to represent each character.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/search Module
Methods and Properties	Result Object Members
Since	2015.2

 **Important:** In search/lookup operations, custom multiselect fields of type "long text" are truncated at 4,000 characters. In accounts with multiple languages enabled, the returned value is truncated at 1,300 characters. In accounts that don't use multiple languages, the field return truncates at 3,900 characters. You can use the [record.load\(options\)](#) method as an alternative for retrieving desired results.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6
7 var searchResult = mySearch.run().getRange({
8   start: 0,
9   end: 100
10 });
11 for (var i = 0; i < searchResult.length; i++) {
12   var entity = searchResult[i].getValue({
13     name: 'entity'
14   });
15   var subsidiary = searchResult[i].getValue({
16     name: 'subsidiary'
17   });
18 ...

```

19 //Add additional code

Result.getText(column)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Used on select, image, and document fields. Returns the text value of a specified search result column. For convenience, this method takes a single search.Column Object.  Note: This method is overloaded. You can also use Result.getText(options) to get column text value based on the name, join and summary values for a column.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/search Module
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description
column	search.Column	required	Name of the search result column.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6
7 var resultSet = mySearch.run();
8 var firstResult = resultSet.getRange({
9   start: 0,
10  end: 1
11 })[0];
12
13 // get the text value of the second column (zero-based index)
14 var value = firstResult.getText(resultSet.columns[1]);
15
16 log.debug({
17   title: 'Value: ',
18   details: value
19 });
20 ...
21 //Add additional code

```

Result.getText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Used on select, image, and document fields. Returns the text value of a specified search result column.  Note: This method is overloaded. You can also use Result.getText(column) to get a column value. This method takes in a single search.Column.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/search Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The name of the search column.
options.join	string	optional	The join internal ID for the search column.
options.summary	string	optional	The summary type used for the search column. See search.Summary .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var searchResults = mySearch.run().getRange({
4     start: 0,
5     end: 100
6 });
7 for (var i = 0; i < searchResults.length; i++) {
8     var amount = searchResults[i].getText({
9         name: 'amount'
10    });
11    var entity = searchResults[i].getText({
12        name: 'name',
13        join: 'location'
14    });
15    ...
16 //Add additional code

```

Result.getValue(column)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Used on formula and non-formula (standard) fields. Returns the value of a specified search result column. For convenience, this method takes a single search.Column Object.  Note: This method is overloaded. You can also use Result.getValue(options) to get column values based on the name, join, and summary values for a column.
Returns	The return type depends on the type of search result column that was specified: - boolean if the column is a box field - number if the column is a record, list, decimal number, or image field, with the following considerations: - For image fields, the returned number represents the ID of the image file. - string for all other column types, with the following considerations: - For multiselect fields, the returned string represents a comma-separated list of IDs. Each ID represents a selectable option in the field. - For date/time fields, the returned string represents the formatted string value of the date. You can use methods in the N/format module to work with this string (for example, converting it to a Date object). For more information, see N/format Module.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/search Module
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description
column	search.Column	required	The search result column from which to return a value.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6
7 var resultSet = mySearch.run();
8 var firstResult = resultSet.getRange({
9   start: 0,
10  end: 1
11 });[0];

```

```

12
13 // get the value of the second column (zero-based index)
14 var value = firstResult.getValue(resultSet.columns[1]);
15
16 log.debug({
17     title: 'Value:',
18     details: value
19 });
20 ...
21 //Add additional code

```

Result.getValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Used on formula and non-formula (standard) fields. Returns the value of a specified search result column. Takes in arguments for name, join, and summary. Note: This method is overloaded. You can also use Result.getValue(column) to get column values. This method takes in a single search.Column. Important: If you have multiple search return columns and you apply grouping, all columns must include a summary property.
Returns	The return type depends on the type of search result column that was specified: - boolean if the column is a box field - number if the column is a record, list, decimal number, or image field, with the following considerations: - For image fields, the returned number represents the ID of the image file. - string for all other column types, with the following considerations: - For multiselect fields, the returned string represents a comma-separated list of IDs. Each ID represents a selectable option in the field. - For date/time fields, the returned string represents the formatted string value of the date. You can use methods in the N/format module to work with this string (for example, converting it to a Date object). For more information, see N/format Module.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/search Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The search return column name.

Parameter	Type	Required / Optional	Description
options.join	string	optional	The join id for this search return column. Join IDs are listed in the Records Browser. For more information, see the help topic Working with the SuiteScript Records Browser .
options.summary	string	optional	The summary type for this column. See search.Summary .
options.func	string	optional	Special function for the search column. See Column.function .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var searchResults = mySearch.run().getRange({
4     start: 0,
5     end: 100
6 });
7 for (var i = 0; i < searchResults.length; i++) {
8     var amount = searchResults[i].getValue({
9         name: 'amount'
10    });
11    var entity = searchResults[i].getValue({
12        name: 'name',
13        join: 'location'
14    });
15    ...
16 //Add additional code

```

Result.columns

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Array of search.Column objects that encapsulate the columns returned in the search result row.
Type	search.Column[] (read-only)
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4     id: 'customsearch_my_so_search'
5 });

```

```

6
7   var firstResult = mySearch.run().getRange({
8       start: 0,
9       end: 1
10    })[0];
11   log.debug({
12       details: "There are " + firstResult.columns.length + " columns in the result."
13   });
14
15   firstResult.columns.forEach(function(col) { // log each column
16       log.debug({
17           details: col
18       });
19   });
20   ...
21   //Add additional code

```

Result.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID for the record returned in a search result row. This ID is a number, but it is stored in this property as a string (for example, '237').
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1   // Add additional code
2   ...
3   var mySearch = search.create({
4       type: search.Type.CUSTOMER
5   });
6
7   var resultSet = mySearch.run();
8
9   resultSet.each(function(result) {
10       log.debug({
11           title: 'Record Internal ID: ',
12           details: result.id
13       });
14
15       return true;
16   });
17   ...
18   // Add additional code

```

Result.recordType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of record returned in a search result row.
-----------------------------	---

Type	search.Type enum
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6
7 var searchResult = mySearch.run();
8 log.debug({
9   title: 'Record Type: ',
10  details: searchResult.recordType
11 });
12 ...
13 //Add additional code

```

search.ResultSet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a set of search results returned by Search.run(). Use the methods and properties for the ResultSet object to iterate through each result returned by the search or access an arbitrary slice of results. The maximum number of results in a ResultSet object is 4000. If a search matches more than 4000 results, you must use Search.runPaged(options) or Search.runPaged.promise(options) to retrieve the full set of results. Important: Search result sets are not cached. If records applicable to your search are created, modified, or deleted at the same time you are traversing your result set, your result set may change. Important: Text columns in search results are limited to 4000 bytes, which is equivalent to 4000 characters in the English language. However, this limit can decrease significantly when texts are stored using intricate character sets, which can require more bytes to represent each character.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/search Module
Methods and Properties	ResultSet Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Load a saved search. Alternatively, you can create a search using search.create(options)
4 // and other methods in the N/search module.
5 var mySearch = search.load({
6   id: 'customsearch_my_cs_search'
7 });
8
9 // Run the search, and use ResultSet.each(callback) to define a callback function to
10 // execute on each search result.
11 //
12 // In this example, the saved search that was loaded above searches for Customer records.
13 // The Result.getValue(options) method obtains the search result value of one of the search
14 // columns that was specified in the search definition. Both 'entityid' and 'email' are valid
15 // search column names for a Customer record.
16 mySearch.run().each(function(result) {
17   var entity = result.getValue({
18     name: 'entityid'
19   });
20   log.debug(entity);
21
22   var email = result.getValue({
23     name: 'email'
24   });
25   log.debug(email);
26
27   return true;
28 });
29 ...
30 // Add additional code

```

ResultSet.each(callback)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use a developer-defined function to invoke on each row in the search results, up to 4000 results at a time. The callback function must use the following signature: <pre>boolean callback(result.Result result);</pre> The callback function takes a search.Result object as an input parameter and returns a boolean which can be used to stop the iteration with a value of false, or continue the iteration with a value of true. Important: The work done in the context of the callback function counts toward the governance of the script that called it. For example, if the callback function is running in the context of a scheduled script, which has a 10,000 unit governance limit, make sure the amount of processing within the callback function does not place the entire script at risk of exceeding scheduled script governance limits.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	10 units
Module	N/search Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
callback	function	required	Named JavaScript function or anonymous inline function that contains the logic to process a search.Result object.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 mySearch.run().each(function(result) {
4     var entity = result.getValue({
5         name: 'entity'
6     });
7     var subsidiary = result.getValue({
8         name: 'subsidiary'
9     });
10    return true;
11 });
12 ...
13 //Add additional code

```

ResultSet.each.promise(callback)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously uses a developer-defined function to invoke on each row in the search results, up to 4000 results at a time. The callback function must use the following signature: boolean callback(result.Result result);  Note: The parameters and errors thrown for this method are the same as those for ResultSet.each(callback). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	ResultSet.each(callback)
Supported Script Types	Client scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 mySearch.run().each.promise(function(result) {
4     var entity = result.getValue({
5         name: 'entity'
6     });
7     var subsidiary = result.getValue({
8         name: 'subsidiary'
9     });
10    return true;
11 })
12 .then(function(response){
13     log.debug({
14         title: 'Completed',
15         details: response
16     });
17 })
18 .catch(function onRejected(reason) {
19     log.debug({
20         title: 'Failed: ',
21         details: reason
22     });
23 })
24 ...
25 //Add additional code

```

ResultSet.getRange(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieve a slice of the search result as an array of search.Result objects. The start parameter is the inclusive index of the first result to return. The end parameter is the exclusive index of the last result to return. For example, <code>getRange(0, 10)</code> retrieves 10 search results, at index 0 through index 9. Unlimited rows in the result are supported, however you can only return 1,000 at a time based on the index values. If there are fewer results available than requested, then the array will contain fewer than end - start entries. For example, if there are only 25 search results, then <code>getRange(20, 30)</code> will return an array of 5 search.Result objects. If you specify a range for which there are no results, an empty array is returned. For example, if there are 25 search results, then <code>getRange(30, 40)</code> will return an empty array.
Returns	search.Result []
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/search Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.start	number	required	Index number of the first result to return, inclusive.
options.end	number	required	Index number of the last result to return, exclusive.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var results = rs.getRange({
4     start: 0,
5     end: 1000
6 });
7 ...
8 //Add additional code

```

ResultSet.getRange.promise(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Method used to asynchronously retrieve a slice of the search result as an array of search.Result objects.  Note: For information about the parameters and errors thrown for this method, see ResultSet.getRange(options) . For additional information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	ResultSet.getRange(options)
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 var results = rs.getRange.promise({
4     start: 0,
5     end: 1000
6 })
7 .then(function(response){
8     log.debug({
9         title: 'Completed',
10        details: response
11    });
12 })
13 .catch(function onRejected(reason) {
14     log.debug({
15         title: 'Failed: ',
16         details: reason
17     });
18 })
19 ...
20
21 //Add additional code

```

ResultSet.columns

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of search.Column objects that represent the columns returned in the search results.
Type	search.Column[] This property is read-only
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4     id: 'customsearch_my_so_search'
5 });

```



```

6
7   var resultSet = mySearch.run();
8   log.debug({
9       details: "There are " + resultSet.columns.length + " columns in the result set:"
10    });
11
12   resultSet.columns.forEach(function(col){ //log each column
13       log.debug({
14           details: col
15       });
16   });
17   ...
18   //Add additional code

```

search.Search

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a NetSuite search. Use the methods available to search.Search to create a search, run a search, or save a search. Note: You do not need to save the search to run it.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/search Module
Methods and Properties	Search Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4     id: 'customsearch_my_so_search'
5 });
6 ...
7 //Add additional code

```

Search.run()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Runs an on-demand search created with search.create(options) or a search loaded with search.load(options) , returning the results as a search.ResultSet . Calling this method does not save the search.
---------------------------	---

	Use this method with search.create(options) to create and run on-demand searches that are never saved to the database. After you run a search, you can use ResultSet.each(callback) to iterate through the result set and process each result.  Important: When you call this method, consider the following: - Search result sets are not cached. If records applicable to your search are created, modified, or deleted at the same time you are traversing your result set, your result set may change. - For better performance, consider creating a saved search in the UI and loading it in your script using search.load(options) instead of creating the search directly in your script using search.create(options).
Returns	search.ResultSet
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/search Module
Since	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SRCH_SETTING	An unknown search parameter name is provided.
SSS_INVALID_SRCH_SETTING_VALUE	An unsupported value is set for the provided search parameter name.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 function loadAndRunSearch() {
4     var mySearch = search.load({
5         id: 'customsearch_my_so_search'
6     });
7     mySearch.run().each(function(result) {
8         var entity = result.getValue({
9             name: 'entity'
10        });
11        var subsidiary = result.getValue({
12            name: 'subsidiary'
13        });
14        return true;

```

```

15 | });
16 | }

```

Search.runPaged(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Important: You must specify a sorting order in the search definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Runs the current search and returns summary information about paginated results. Calling this method does not give you the result set or save the search. To retrieve data, use PagedData.fetch(options). Important: When you use this method to run a paged search, consider the following: - Search result sets are not cached. If records applicable to your search are created, modified, or deleted at the same time you are traversing your result set, your result set may change. - This method can return a maximum of 1000 pages of search results.
Returns	search.PagedData
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	5 units
Module	N/search Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.pageSize	number	optional	Maximum number of entries per page There is an upper limit, a lower limit, and a default setting: - The maximum number allowed is 1000. - The minimum number allowed is 5. - By default, the page size is set to 50 entries per page.

Errors

Error Code	Thrown If
SSS_INVALID_SRCH_SETTING	An unknown search parameter name is provided.
SSS_INVALID_SRCH_SETTING_VALUE	An unsupported value is set for the provided search parameter name.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.create({
4     type: search.Type.CUSTOMER
5 });
6
7 // Run the paged search
8 var pagedData = mySearch.runPaged({
9     pageSize: 50
10 });
11 ...
12 // Use the count property to count the
13 // search results easily
14 var resultCount = mySearch.runPaged({
15     pageSize: 50
16 }).count;
17 ...
18 //Add additional code

```

Search.runPaged.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: You must specify a sorting order in the search definition when using this method to avoid duplicate or missing results. The query definition must provide a unique and unambiguous sorting order with a specified precedence.

Method Description	Runs the current search asynchronously and returns a search.PagedData Object.
	Note: The parameters and errors thrown for this method are the same as those for Search.runPaged(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Search.runPaged(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	5 units
Module	N/search Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 mySearch.runPaged.promise().then(getPageRangesPromiseChain);
4 ...
5 //Add additional code

```

Search.save()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Saves a search created by search.create(options) or loaded with search.load(options). Returns the internal ID of the saved search. You must set the title and id properties for a new saved search before you save it, either when you create it with search.create(options) or by setting the Search.title and Search.id properties. If you do not set the saved search ID, NetSuite generates one for you. See Search.id. Note: You do not need to set these properties if you load a previously saved search with search.load(options) and then save it. This method also includes a promise version, Search.save.promise(). For more information about promises, see Promise Object.
Returns	the internal search ID of the saved search as a number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	5 units
Module	N/search Module
Since	2015.2

Errors

Error Code	Message	Thrown If
NAME_ALREADY_IN_USE	A search has already been saved with that name. Please use a different name.	The Search.title property on search.Search is not unique.

Error Code	Message	Thrown If
SSS_DUPLICATE_SEARCH_SCRIPT_ID	Saved search script IDs must be unique. Please choose another script ID. If you are trying to modify an existing saved search, use <code>search.load()</code> .	The Search.id property on search.Search is not unique.
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required Search.title property not set on search.Search .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 const mySalesOrderSearch = search.create({
4   type: search.Type.SALES_ORDER,
5   title: 'My SalesOrder Search',
6   id: 'customsearch_my_so_search',
7   columns: ['entity', 'subsidiary', 'name', 'currency'],
8   filters: [
9     ['mainline', 'is', 'T'], 'and', ['subsidiary.name', 'contains', 'CAD']
10  ]
11 })
12
13 mySalesOrderSearch.save() // returns search ID
14
15 ...
16 //Add additional code

```

Search.save.promise()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Asynchronously saves a search created by search.create(options) or loaded with search.load(options) . Returns the internal ID of the saved search.  Note: The parameters and errors thrown for this method are the same as those for Search.save(). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	Search.save()
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.create.promise({
4     type: search.Type.SALES_ORDER
5 })
6 .then(function(searchObj) {
7     return searchObj.save.promise()
8 })
9 .then(function(result) {
10    log.debug({
11        details: "Completed: " + result
12    });
13    // do something after completion
14 })
15 .catch(function onRejected(reason) {
16    // do something on rejection
17 });
18 ...
19 //Add additional code

```

Search.columns

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Columns to return for this search as an array of search.Column objects or a string array of column names. You set this value with an array of search.Column objects or a single search.Column to overwrite any prior return columns for the search. Use null to set an empty array and remove any existing columns on this search.
Type	search.Column[] string[]
Module	N/search Module
Since	2015.2

Errors

Error Code	Thrown If
SSS_INVALID_SRCH_COLUMN	The value passed in was not a string or search.Column Object

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 function createSearch() {
4     var mySalesOrderSearch = search.create({
5         type: search.Type.SALES_ORDER,
6         columns: ['entity', 'subsidiary', 'name', 'currency'],

```

```

7 |      });
8 |      ...
9 |      //Add additional code

```

Search.filterExpression

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Use filter expressions as a shortcut to create filters (search.Filter). A search filter expression is a JavaScript string array of zero or more elements. Each element is one of the following: - Operator – For a list of supported operators, see search.Operator. - Filter term - Two or more filter expressions combined logically with 'and', 'or', or 'not'. Logical operators are evaluated according to JavaScript operator precedence. For more information, see Operator precedence. This property accepts nested arrays in which any element of the nested array can be an Object, a string, a number, or a boolean. Use null to set an empty array and remove any existing filter expressions on this search.  Note: If you want to get or set search filters, use the Search.filters property.
Type	Array[]
Module	N/search Module
Since	2015.2

Note: A filter expression is an array of filter elements defined with the `options.filters` parameter when creating a search using the [search.create\(options\)](#) method. This property is considered a shorthand way of defining filters, instead of using the [search.createFilter\(options\)](#) method.

Errors

Error Code	Message	Thrown If
SSS_INVALID_SRCH_FILTER_EXPR	Malformed search filter expression. This is a general error raised when a filter expression cannot be parsed. For example: <pre>[f1, 'and', 'and', f2]</pre>	The <code>options.filters</code> parameter is not a valid search filter, filter array, or filter expression.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

In the script sample below, each array of objects is considered to be a search filter expression. Filter expressions are automatically recognized and defined with the `options.filters` parameter.


```

1 //Add additional code
2 ...
3 search.create({
4   type: search.Type.CUSTOMER,
5   filters: [
6     ['email', search.Operator.STARTSWITH, 'kwolff'],
7     'and',
8     [
9       ['id', search.Operator.EQUALTO, 107], 'or',
10      ['id', search.Operator.EQUALTO, 2508]
11    ]
12  ]
13 });
14 ...
15 //Add additional code

```

Search.filters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Filters for the search as an array of search.Filter objects. Value is null if the search has no defined filters. You set this value with an array or single search. Filter objects to overwrite any prior filters. Use null to set an empty array and remove any existing filters on this search. Use search.createFilter(options) to create a filter. Filter expressions are an alternative way to define filters of a search. The expressions are an array of JavaScript objects that are passed through this property. For more information about filter expressions, see the Search.filterExpression property.
Type	search.Filter []
Module	N/search Module
Since	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_SRCH_FILTER	An search filter contains invalid search criteria	Invalid value for search filter type.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 var myFilter = search.createFilter({
5   name: 'entity',
6   operator: search.Operator.ISEMPY,
7 });
8
9 function createSearch() {
10   var mySalesOrderSearch = search.create({
11     type: search.Type.SALES_ORDER,
12     filters: myFilter

```

```

13     });
14     ...
15     //Add additional code

```

Search.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Script ID for a saved search, starting with customsearch. If you do not set this property and then save the search, NetSuite generates a script ID for you.  Note: This is not the internal NetSuite ID for the saved search. See Search.searchId.
Type	string
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 function createSearch() {
4     var mySalesOrderSearch = search.create({
5         type: search.Type.SALES_ORDER,
6         title: 'My SalesOrder Search',
7         id: 'customsearch_my_so_search',
8     });
9     ...
10 //Add additional code

```

Search.isPublic

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Value is true if the search is public, or false if it is not. By default, all searches created through search.create(options) are private.
Type	boolean
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code

```

```

2  ...
3  var mySearch = search.load({
4      id: 'customsearch_my_so_search'
5  });
6  mySearch.isPublic = true;
7  ...
8  //Add additional code

```

Search.packageId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The application ID for this search. An application ID identifies a SuiteApp project and is a fully qualified name with the following notation: <pre>1 <publisher_id>.<project_id></pre> For example, com.netsuite.mysuiteapp and org.mycompany.helloworld are application IDs. To use this feature, the Show App ID Field preference must be enabled in your NetSuite account. For more information, see the help topic SuiteCloud Development Framework Account Preferences (SDF Developers Only).
Type	string
Module	N/search Module
Since	2019.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var mySalesOrderSearch = search.create({
4      type: search.Type.SALES_ORDER,
5      packageId: 'com.example',
6      columns: ['entity', 'subsidiary', 'name', 'currency']
7  });
8
9  var thePackageId = mySalesOrderSearch.packageId;
10 ...
11 // Add additional code

```

Search.searchId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID of the search. The internal ID is available only when the search is either loaded with search.load(options) or after it has been saved with Search.save().
-----------------------------	--

	Typical values are 55 or 234 or 87, not a value like customsearch_mysearch. Any ID prefixed with customsearch is a script ID, not the internal system ID for a search.
Type	number (read-only)
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6 log.debug({
7   title: 'search id #:',
8   details: mySearch.searchId
9 });
10 ...
11 //Add additional code

```

Search.searchType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID name of the record type on which a search is based. Use this if you have the internal ID of the search, but do not know the record type the search was based on. For example, if the search was on a Customer record, this property is customer; if the search was on the Sales Order record type, this property is salesorder.
Type	string (read-only)
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6 log.debug({
7   title: 'record type:',
8   details: mySearch.searchType
9 });

```

```

10 ...
11 //Add additional code

```

Search.settings

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Search settings for this search as an array of search.Setting objects or a string array of column names. Search settings let you specify search parameters that are typically available only in the UI. You set this value with an array of search.Setting objects or a single search.Setting object. You can create a search.Setting object by calling search.createSetting(options). You can also set this value with an array of column names, each of which is a string. The supported values for a search.Setting object differ depending on the search parameter that you set. For more information, see Setting.name and Setting.value.
Type	search.Setting[] string[]
Module	N/search Module
Since	2018.2

Errors

Error Code	Thrown If
SSS_INVALID_SRCH_SETTING	An unknown search parameter name is provided.
SSS_INVALID_SRCH_SETTING_VALUE	An unsupported value is set for the provided search parameter name.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.create({
4   type: 'transaction',
5   columns: [ 'trandate', 'amount', 'entity' ],
6   filters: [
7     search.createFilter({
8       name: 'internalid',
9       operator: search.Operator.ANYOF,
10      values: [13, 12356]
11    })
12  ],
13   settings: [
14     search.createSetting({
15       name: 'consolidationtype',
16       value: 'NONE'
17     })
18  ]
19 });
20 ...
21 //Add additional code

```

Search.title

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Title for a saved search. Use this property to set the title for a search before you save it for the first time. You can also set the title for a search when you create it with search.create(options). The Search.title property is required to save a search with Search.save().
Type	string
Module	N/search Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 function createSearch() {
4     var mySalesOrderSearch = search.create({
5         type: search.Type.SALES_ORDER,
6         title: 'My SalesOrder Search',
7         id: 'customsearch_my_so_search',
8     });
9     mySalesOrderSearch.save();
10 ...
11 //Add additional code

```

search.Setting

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Defines a search setting. Search settings let you specify search parameters that are typically available only in the UI. The following settings are supported: ■ Consolidated Exchange Rate: This setting affects how consolidation is performed (for example, consolidation using the Average rate type, consolidation using the Historical rate type, and so on). This setting applies to transaction searches, and it is applicable only to OneWorld accounts. ■ Show Period End Transactions: This setting indicates whether period end transactions are included in search results. This setting applies to transaction searches, and it is applicable only to OneWorld accounts. It also requires the Show Period End transactions feature to be enabled. Use search.createSetting(options) to create a setting. After you create your settings, assign them as array values to Search.settings.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/search Module

Methods and Properties	Setting Object Members
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.create({
4   type: 'transaction',
5   columns: [ 'trandate', 'amount', 'entity' ],
6   filters: [
7     search.createFilter({
8       name: 'internalid',
9       operator: search.Operator.ANYOF,
10      values: [13, 12356]
11    })
12   ],
13   settings: [
14     search.createSetting({
15       name: 'consolidationtype',
16       value: 'NONE'
17     })
18   ]
19 });
20 ...
21 //Add additional code

```

Setting.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the search parameter. This property is set when you call search.createSetting(options). The following values are supported for this property: - consolidationtype: This value corresponds to the Consolidated Exchange Rate setting. - includeperiodendtransactions: This value corresponds to the Show Period End Transactions setting.
Type	string This property is read-only.
Module	N/search Module
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code

```

```

2 ...
3 var mySearch = search.create({
4   type: 'transaction',
5   columns: [ 'trandate', 'amount', 'entity' ],
6   filters: [
7     search.createFilter({
8       name: 'internalid',
9       operator: search.Operator.ANYOF,
10      values: [13, 12356]
11    })
12  ],
13   settings: [
14     search.createSetting({
15       name: 'consolidationtype',
16       value: 'NONE'
17     })
18  ]
19 });
20 ...
21 //Add additional code

```

Setting.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The value of the search parameter. This property is set when you call search.createSetting(options). If you specify <code>consolidationtype</code> as the search parameter name (Setting.name), the following values are supported for this parameter: - ACCTTYPE - AVERAGE - CURRENT - HISTORICAL - NONE If you specify <code>includeperiodendtransactions</code> as the search parameter name (Setting.name), the following values are supported for this parameter: - F - FALSE - T - TRUE These values are not case sensitive.
Type	string This property is read-only.
Module	N/search Module
Since	2018.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 //Add additional code
```



```

2  ...
3  var mySearch = search.create({
4    type: 'transaction',
5    columns: [ 'trandate', 'amount', 'entity' ],
6    filters: [
7      search.createFilter({
8        name: 'internalid',
9        operator: search.Operator.ANYOF,
10       values: [13, 12356]
11      }
12    ],
13    settings: [
14      search.createSetting({
15        name: 'consolidationtype',
16        value: 'NONE'
17      })
18    ]
19  });
20  ...
21  //Add additional code

```

search.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new search and returns it as a search.Search object. The search can be modified and run as an on demand search with Search.run(), without saving it. Alternatively, calling Search.save() will save the search to the database, so it can be reused later in the UI or loaded with search.load(options). Note: This method is agnostic in terms of its options.filters argument. It can accept input of a single search.Filter object, an array of search.Filter objects, or a search filter expression. The search.create(options) method also includes a promise version, search.create.promise(options). For more information about promises, see Promise Object. Important: When you use this method to create a search, consider the following: ■ When you define the search, make sure you sort using the field with the most unique values, or sort using multiple fields. Sorting with a single field that has multiple identical values can cause the result rows to be in a different order each time the search is run. ■ You cannot directly create a filter or column for a list/record type field in SuiteScript by passing in its text value. You must use the field's internal ID. If you must use the field's text value, you can create a filter or column with a formula using name: 'formulatext'.
Returns	search.Search
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/search Module

Since	2015.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	search.Type	required	The search type that you want to base the search on.	2015.2
options.filters	search.Filter search.Filter[] Object Object[] string string[]	optional	A single search.Filter object, an array of search.Filter objects, a search filter expression, or an array of search filter expressions. A search filter expression can be passed in as an Object with the following properties: - name (required) - join - operator (required) - summary - formula - values For more information about these properties, see Filter Object Members. If a provided filter value has an incorrect type (for example, a string instead of a number), the filter value is ignored. For server scripts, a log entry is created when an incorrect type is provided. Note: You can further filter the returned search.Search object by adding additional filters with Search.filters or Search.filterExpression.	2015.2
options.filterExpression	Object[]	optional	Search filter expression for the search as an array of expression objects. A search filter expression is created when an array of JavaScript objects are passed through the options.filters parameter. This value is set with an array of expression objects or single filter expression objects to overwrite any prior filter expressions. Note: A filter expression is automatically recognized by NetSuite. Filter expressions should not be explicitly denoted when scripting. A filter expression should be used with the options.filters parameter. For more information about how to set the values of this parameter, see Search.filterExpression.	2016.2
options.columns	search.Column search.Column[]	optional	A single search.Column object or array of search.Column objects.	2015.2

Parameter	Type	Required / Optional	Description	Since
	Object Object[] string string[]		You can optionally pass in an Object or array of Objects with the following properties to represent a Column: - name (required) - formula - function - join - label - sort - summary For more information about these properties, see Column Object Members.	
options.packageId	string	optional	The application ID for this search.	2019.2
options.settings	search.Setting search.Setting[] Object Object[] string string[]	optional	Search settings for this search as a single search.Setting object or an array of search.Setting objects. Search settings let you specify search parameters that are typically available only in the UI. See Search.settings. You can optionally pass in an Object or array of Objects with the following properties to represent a setting: - name - value For more information about these properties, see Setting Object Members.	2018.2
options.title	string	optional	The name for a saved search. The title property is required to save a search with Search.save() .	2015.2
options.id	string	optional	Script ID for a saved search. If you do not set the saved search ID, NetSuite generates one for you. See Search.id .	2015.2
options.isPublic	boolean	optional	Set to true to make the search public. Otherwise, set to false. If you do not set this parameter, it defaults to false. This parameter sets the value for the Search.isPublic property.	2016.2

Errors

Error Code	Thrown If
SSS_INVALID_SRCH_COL	The options.columns parameter is not a valid column, string, or column or string array.
SSS_INVALID_SRCH_FILTER_EXPR	The options.filters parameter is not a valid search filter, filter array, or filter expression.
SSS_MISSING_REQD_ARGUMENT	Required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySalesOrderSearch = search.create({
4   type: search.Type.SALES_ORDER,
5   title: 'My Second SalesOrder Search',
6   id: 'customsearch_my_second_so_search',
7   columns: [{
8     name: 'entity'
9   }, {
10    name: 'subsidiary'
11  }, {
12    name: 'name'
13  }, {
14    name: 'currency'
15  }],
16   filters: [{
17     name: 'mainline',
18     operator: 'is',
19     values: ['T']
20   }],
21   settings: [{
22     name: 'consolidationtype',
23     value: 'AVERAGE'
24   }]
25 });
26 ...
27 //Add additional code

```

search.create.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new search asynchronously and returns it as a search.Search object. Note: The parameters and errors thrown for this method are the same as those for search.create(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	search.create(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.create.promise({
4     type: search.Type.SALES_ORDER
5 })
6 .then(function(result) {
7     log.debug({
8         details: "Completed: " + result
9     });
10    // do something after completion
11 })
12 .catch(function(reason) {
13     log.debug({
14         details: "Failed: " + reason
15     });
16    // do something on failure
17 });
18 ...
19 //Add additional code

```

search.createColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new search column as a search.Column object. A search column represents a field on a record. You create a column for each field that you want to include in the search results. For example, if you create a column for the Sales Rep field on a customer record and specify that column when you create a search, the value of the Sales Rep field is included in the search results. Important: As you create search columns, consider the following: - You cannot directly create a filter or column for a list/record type field in SuiteScript by passing in its text value. You must use the field's internal ID. If you must use the field's text value, you can create a filter or column with a formula using name: 'formula:text'. - After you create a column, you cannot change the sort order of the column. If you use the same column in another search and specify a new sort order, the previous sort order is still used (the sort order that you specified using the options.sort parameter).
Returns	search.Column
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/search Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	Name of the search column. See Column.name .	2015.2
options.join	string	optional	Join ID for the search column. See Column.join .	2015.2
options.summary	string	optional	Summary type for the column. See search.Summary and Column.summary .	2015.2
options.formula	string	optional	Formula for the search column. See Column.formula .	2015.2
options.function	string	optional	Special function for the search column. See Column.function .	2015.2
options.label	string	optional	Label for the search column. See Column.label .	2015.2
options.sort	string	optional	The sort order of the column. Use the search.Sort enum for this argument. Also see Column.sort .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_SRCH_COLUMN_SUM	A search.Column object contains an invalid column summary type, or is not in proper syntax: {1}.	The options.summary parameter is not a valid search summary type. See search.Summary .
INVALID_SRCH_FUNCN	—	An unknown function is provided.
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```
1 | //Add additional code
```

```

2  ...
3  var currencyColumn = search.createColumn({
4      name: 'currency',
5      sort: search.Sort.ASC
6  });
7  ...
8  //Add additional code

```

search.createFilter(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new search filter as a search.Filter object.
	 Important: You cannot directly create a filter or column for a list/record type field in SuiteScript by passing in its text value. You must use the field's internal ID. If you must use the field's text value, you can create a filter or column with a formula using name: 'formulaText'.
Returns	search.Filter
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/search Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	Name or internal ID of the search field.	2015.2
options.join	string	optional	Join ID for the search filter.	2015.2
options.operator	string	required	Operator used for the search filter. Use the search.Operator enum.	2015.2
options.values	string Date number boolean string[] Date[] number[]	optional	Values to be used as filter parameters.	2015.2
options.formula	string	optional	Formula used by the search filter.	2015.2
options.summary	string	optional	Summary type for the search filter. See search.Summary .	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_SRCH_OPERATOR	An search.Filter object contains an invalid operator, or is not in proper syntax: {1}.	options.operator parameter is not a valid operator type. See search.Operator .
SSS_INVALID_SRCH_SUMMARY_TYP	A search.Column object contains an invalid column summary type, or is not in proper syntax: {1}.	options.summary parameter is not a valid search summary type. See search.Summary .
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

In the code sample below, the options.filters parameter creates [search.createFilter\(options\)](#) objects. For an example of filter expressions created using the options.filters parameter, see [Search.filterExpression](#).

```

1 // Add additional code
2 ...
3 // Create a filter joined to another record type. When you create a joined filter:
4 // - The name property is the field ID of the field in the joined record that you are filtering on
5 // - The join property is the field ID of the field in the current record that contains the record
6 // - type you want to join to
7 // - The operator property is the operator to use to filter the results
8 // - The values property contains the values to use to filter the results
9 //
10 // For example, the following search definition lists the first 100 employees found
11 // who have a custom role. The search definition specifies that the search applies to
12 // Employee records. The filter definition joins the Role record type to the search
13 // and returns results where the iscustom field (a field on the Role record) is true.
14 var result = search.create({
15     type: 'employee',
16     columns: ['firstname', 'lastname', 'role'],
17     filters: [
18         search.createFilter({
19             name: 'iscustom',
20             join: 'role',
21             operator: search.Operator.IS,
22             values: true
23         })
24     ]
25 }).run().getRange({
26     start: 0,
27     end: 100
28 });
29 log.debug({
30     title: 'Result',
31     details: result
32 });
33 ...
34 // Add additional code

```


Note: If you set **contains** search filters for fields with Long Text and Rich Text text types, the matching string length has a limit of approximately 500 characters. This limit may vary depending on the character encoding you use.

search.createSetting(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new search setting and returns it as a search.Setting object. Search settings let you specify search parameters that are typically available only in the UI. The following settings are supported: ■ Consolidated Exchange Rate: This setting affects how consolidation is performed (for example, consolidation using the Average rate type, consolidation using the Historical rate type, and so on). This setting applies to transaction searches, and it is applicable only to OneWorld accounts. ■ Show Period End Transactions: This setting indicates whether period end transactions are included in search results. This setting applies to transaction searches, and it is applicable only to OneWorld accounts. It also requires the Period End Journal Entries feature to be enabled. After you create your settings, assign them as array values to Search.settings.
Returns	search.Setting
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/search Module
Since	2018.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.name	string	required	The name of the search parameter to set. This value sets the Setting.name property. Use one of the following values for this parameter: ■ consolidationtype: This value corresponds to the Consolidated Exchange Rate setting. ■ includeperiodendtransactions: This value corresponds to the Show Period End Transactions setting.	2018.2
options.value	string	required	The value of the search parameter. If you are executing a joined search, this value is the join ID used for the search field specified by the	2018.2

Parameter	Type	Required / Optional	Description	Since
			<code>options.name</code> parameter. This value sets the Setting.value property. If you specify <code>consolidationtype</code> as the search parameter name, use one of the following values for this parameter: - ACCTTYPE - AVERAGE - CURRENT - HISTORICAL - NONE The default value is ACCTTYPE, which represents the type of consolidation associated with the account. If you specify <code>includeperiodendtransactions</code> as the search parameter name, use one of the following values for this parameter: - F - FALSE - T - TRUE The default value is false. These values are not case sensitive.	

Errors

Error Code	Thrown If
SSS_INVALID_SRCH_SETTING	An unknown search parameter name is provided.
SSS_INVALID_SRCH_SETTING_VALUE	An unsupported value is set for the provided search parameter name.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.create({
4   type: 'transaction',
5   columns: [ 'trandate', 'amount', 'entity' ],
6   filters: [
7     search.createFilter({
8       name: 'internalid',
9       operator: search.Operator.ANYOF,
10      values: [13, 12356]
11     })],
12   settings: [

```

```

13     search.createSetting({
14         name: 'consolidationtype',
15         value: 'NONE'
16     })
17 });
18 ...
19 //Add additional code

```

search.delete(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes an existing saved search. The saved search could have been created using the UI or created with search.create(options) and Search.save() . The search.delete(options) method also includes a promise version, search.delete.promise(options) . For more information about promises, see Promise Object .
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string number	required	Internal ID or script ID of a saved search. The script ID starts with customsearch. See Search.id for script ID. See Search.searchId for internal ID.	2015.2
options.type	string	Required if the saved search to delete uses a standalone search type, optional otherwise	The search type of the saved search to delete. Use a value from the search.Type enum for this parameter. To successfully delete a standalone search, this parameter is required to avoid an error being thrown. For more information about standalone search types, see search.load(options) .	2015.2

Errors

Error Code	Message	Thrown If
INVALID_ID_SEARCH	That search or mass update does not exist.	Cannot find saved search with the saved search ID from options.id parameter.

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 search.delete({
4   id: 'customsearch_my_so_search'
5 });
6 ...
7 //Add additional code

```

search.delete.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Deletes an existing saved search asynchronously and returns it as a search.Search object. The saved search can be created using the UI or created with search.create(options) and Search.save(). Note: The parameters and errors thrown for this method are the same as those for search.delete(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	search.delete(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	5 units
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.delete.promise({id: 'customsearch_txn_search_salesorder'})
4   .then(function(){

```

```

5      search.load({
6          id: 'customsearch_txn_search_salesorder'
7      });
8  })
9  .catch(function onRejected(reason) {
10     log.debug({
11         details: 'Invalid search: ' + reason.name
12     });
13 });
14 ...
15 //Add additional code

```

search.duplicates(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a search for duplicate records based on the account's duplicate detection configuration. This method also includes a promise version, search.duplicates.promise(options). For more information about promises, see Promise Object.  Important: This API works only for records that support duplicate record detection (for example, customer, lead, prospect, contact, partner, and vendor records). For more information about duplicate record detection, see the help topic Duplicate Record Detection.
Returns	search.Result[] that contains the duplicate records Results are limited to 1000 rows. If there are no search results, this method returns null.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/search Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	enum	required	The search type that you want to check for duplicates. Use the search.Type enum for this parameter. The type you specify must correspond to a record type that supports duplicate record detection (for example, customer, lead, prospect, contact, partner, and vendor records).	2015.2

Parameter	Type	Required / Optional	Description	Since
options.fields	Object	optional	A set of key-value pairs used to detect duplicates. The keys are internal ID names of the fields used to detect duplicates. You can specify fields such as companyname, email, name, phone, address1, city, state, and zipcode. For example, to detect duplicates based on the value of the email field, use 'email' : 'sample@test.com'. Use this parameter to specify the fields (and their values) to use to detect duplicates. If you are searching for duplicates based on fields that appear on a certain record type, this parameter is required. If you are searching for duplicates of a specific record (of the specified type), use the options.id parameter instead.	2015.2
options.id	number	optional	Internal ID of an existing record. Use this parameter to specify a record to detect duplicates of. The duplicate record detection settings in the account determine which fields are used to detect duplicates of the specified record. If you are searching for duplicates of a specific record (of the specified type), this parameter is required. If you are searching for duplicates based on fields that appear on a certain record type, use the options.fields parameter instead.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // Search for duplicates of a specific record using the options.id
4 // parameter
5 var duplicatesOfRecord = search.duplicates({
6     type: search.Type.CONTACT,
7     id: 425
8 });
9
10 // Search for duplicates based on specific fields on a record type
11 // using the options.fields parameter
12 var duplicatesUsingFields = search.duplicates({
13     type: search.Type.CONTACT,
14     fields: {
15         'email' : 'sample@test.com'
16     }
17 });
18 ...
19 //Add additional code

```

search.duplicates.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a search for duplicate records asynchronously based on the Duplicate Detection configuration for the account. Returns an array of search.Result objects. This method only applies to records that support duplicate record detection. These records include customer lead prospect partner vendor contact.  Note: The parameters and errors thrown for this method are the same as those for search.duplicates(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	search.duplicates(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/search Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.duplicates.promise({
4     type: search.Type.CUSTOMER,
5     id: 28
6 })
7 .then(function (result) {
8     log.debug({
9         details: "Completed: " + result
10    });
11    // do something after completion
12 })
13 .catch(function onRejected(reason) {
14    // do something on rejection
15 });
16 ...
17 //Add additional code

```

search.global(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a global search against a single keyword or multiple keywords.
---------------------------	---

	Similar to the global search functionality in the UI, you can programmatically filter the global search results that are returned. For example, you can use the following filter to limit the returned records to Customer records: <pre>'cu: simpson'</pre> The <code>search.global(options)</code> method also includes a promise version, search.global.promise(options). For more information about promises, see Promise Object. For more information about global search, see the help topic Global Search.
Returns	search.Result[] as an array of result objects containing these columns: name, type, info1, and info2 Results are limited to 1000 records. If there are no search results, this method returns null.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/search Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.keywords	string	required	Global search keywords string or expression.	2015.2

Errors

Error Code	Message	Thrown If
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var customerSearch = search.global({
4     keywords: 'cu: simpson'
5 });
6 ...
7 //Add additional code

```


search.global.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a global search asynchronously against a single keyword or multiple keywords. Returns an array of search.Result objects with four columns: name, type, info1, and info2.  Note: The parameters and errors thrown for this method are the same as those for search.global(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	search.global(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.global.promise({
4     keywords: 'Alan Rath'
5 })
6 .then(function (result) {
7     log.debug({
8         details: "Completed: " + result
9     });
10    // do something after completion
11 })
12 .catch(function onRejected(reason) {
13    // do something on rejection
14 });
15 ...
16 //Add additional code

```

search.load(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing saved search and returns it as a search.Search. The saved search could have been created using the UI or created with search.create(options) and Search.save(). The search.load(options) method also includes a promise version, search.load.promise(options). For more information about promises, see Promise Object.
---------------------------	---

Returns	search.Search
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	Internal ID or script ID of a saved search. The script ID starts with customsearch. See Search.id .	2015.2
options.type	string	Required if the saved search to load uses a standalone search type, optional otherwise	The search type of the saved search to load. Use a value from the search.Type enum for this parameter. This parameter is required if the saved search to load uses a standalone search type. A standalone search type is a search type that does not have a corresponding record type. Typically, the search type of the saved search can be determined automatically based on the corresponding record type. In this case, this parameter is not required. For standalone search types, you must specify the search type explicitly using this parameter. The following is a list of standalone search types: ■ DeletedRecord ■ EndToEndTime ■ ExpenseAmortPlanAndSchedule ■ RevRecPlanAndSchedule ■ GILinesAuditLog ■ Crosschargeable ■ FinRptAggregateFR ■ BillingAccountBillCycle ■ BillingAccountBillRequest ■ BinItemBalance ■ PaymentEvent ■ Permission ■ GatewayNotification ■ TimeApproval ■ RecentRecord ■ Role ■ SavedSearch ■ ShoppingCart	2015.2

Parameter	Type	Required / Optional	Description	Since
			■ SubscriptionRenewalHistory ■ SuiteScriptDetail ■ SupplyChainSnapshotDetails ■ SystemNote ■ TaxDetail ■ TimesheetApproval ■ Uber ■ ResAllocationTimeOffConflict ■ ComSearchOneWaySyn ■ ComSearchGroupSyn ■ Installment ■ InventoryBalance ■ InventoryNumberBin ■ InventoryNumberItem ■ InventoryStatusLocation ■ InvtNumberItemBalance ■ ItemBinNumber	

Errors

Error Code	Message	Thrown If
INVALID_SEARCH	That search or mass update does not exist.	Cannot find saved search with the saved search ID from options.id parameter.
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.load({
4   id: 'customsearch_my_so_search'
5 });
6 ...
7 //Add additional code

```

search.load.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing saved search asynchronously and returns it as a search.Search object. The saved search could have been created using the UI or created with search.create(options) and Search.save() .
---------------------------	--

	 Note: The parameters and errors thrown for this method are the same as those for search.load(options) . For more information about promises, see Promise Object .
Returns	Promise Object
Synchronous Version	search.load(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.load.promise({
4     type : search.Type.SALES_ORDER,
5     id : 'customsearch_txn_search_salesorder'
6 })
7 .then(function (result) {
8     log.debug({
9         details: "Completed: " + result
10    });
11    // do something after completion
12 })
13 .catch(function onRejected(reason) {
14     // do something on rejection
15 });
16 ...
17 //Add additional code

```

search.lookupFields(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a search for one or more body fields on a record. You can use joined-field lookups with this method, with the following syntax: <pre>join_id.field_name</pre> The <code>search.lookupFields(options)</code> method also includes a promise version, search.lookupFields.promise(options). For more information about promises, see Promise Object. Note that the return contains either an object or a scalar value, depending on whether
---------------------------	--

	the looked-up field holds a single value, or a collection of values. Single select fields are returned as an object with value and text properties. Multi-select fields are returned as an object with value:text pairs. In the following example, a select field like my_select would return an array of objects containing a value and text property. This select field contains multiple entries to select from, so each entry would have a numerical id (the value) and a text display (the text). For "internalid" in this particular code sample, the sample returns 1234. The internal id of a record is a single value, so a scalar is returned. <pre>1 { 2 internalid: 1234, 3 firstname: 'Joe', 4 my_select: [{ 5 value: 1, 6 text: 'US Sub' 7 }], 8 my_multiselect: [9 { 10 value: 1, 11 text: 'US Sub' 12 }, 13 { 14 value: 2, 15 text: 'EU Sub' 16 } 17] 18 }</pre> If you try to look up a field that does not exist on the specified record, an SSS_INVALID_SRCH_COL error is thrown.
Returns	Object Object[] ■ Returns select fields as an object with value and text properties. ■ Returns multiselect fields as an array of object with value:text pairs.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	1 unit
Module	N/search Module
Since	2015.2



Important: In search/lookup operations, custom multiselect fields of type "long text" are truncated at 4,000 characters. In accounts with multiple languages enabled, the returned value is truncated at 1,300 characters. In accounts that don't use multiple languages, the field return truncates at 3,900 characters. You can use the [record.load\(options\)](#) method as an alternative for retrieving desired results.

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	enum	required	The search type for which you want to look up fields. Use the search.Type enum for this argument.	2015.2
options.id	string	required	Internal ID for the record, for example 777 or 87.	2015.2
options.columns	string string[]	required	Array of column/field names to look up, or a single column/field name. The columns parameter can also be set to reference joined fields.	2015.2

Errors

Error Code	Message	Thrown If
SSS_INVALID_SRCH_COL	An nlobjSearchColumn contains an invalid column, or is not in proper syntax: {1}.	The options.columns parameter includes invalid columns for the specified record.
SSS_MISSING_REQD_ARGUMENT	{1}: Missing a required argument: {2}	Required parameter is missing.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var fieldLookup = search.lookupFields({
4   type: search.Type.SALES_ORDER,
5   id: '87',
6   columns: ['entity', 'subsidiary', 'name', 'currency']
7 });
8 ...
9 //Add additional code

```

search.lookupFields.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Performs a search asynchronously for one or more body fields on a record. Returns select fields as an object with value and text properties. Returns multiselect fields as an object with value:text pairs.  Note: The parameters and errors thrown for this method are the same as those for search.lookupFields(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	search.lookupFields(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	1 unit
Module	N/search Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 search.lookupFields.promise({
4     type: search.Type.EMPLOYEE,
5     id: -5,
6     columns : 'email'
7 })
8 .then(function (result) {
9     log.debug({
10         details: "Completed: " + result
11     });
12     // do something after completion
13 })
14 .catch(function onRejected(reason) {
15     // do something on rejection
16     });
17 ...
18 //Add additional code

```

search.Operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for search operators to use with the search.Filter . See SuiteScript 2.x Search Operators for more information about the field types supported for each search operator type.
Module	N/search Module
Sibling Module Members	N/search Module Members
Since	2015.2

Values

Value	Value	Value
- AFTER - ALLOF - ANY - ANYOF - BEFORE - BETWEEN - CONTAINS - DOESNOTCONTAIN - DOESNOTSTARTWITH - EQUALTO - GREATERTHAN - GREATERTHANOREQUALTO - HASKEYWORDS	- IS - ISEMPTY - ISNOT - ISNOTEMPTY - LESSTHAN - LESSTHANOREQUALTO - NONEOF - NOTAFTER - NOTALLOF - NOTBEFORE - NOTBETWEEN - NOTEQUALTO - NOTGREATERTHAN	- NOTGREATERTHANOREQUALTO - NOTLESSTHAN - NOTLESSTHANOREQUALTO - NOTON - NOTONORAFTER - NOTONORBEFORE - NOTWITHIN - ON - ONORAFTER - ONORBEFORE - STARTSWITH - WITHIN

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code...
2 var mySearchFilter = search.createFilter({
3   name: 'entity',
4   operator: search.Operator.ISEMPTY
5 });
6 ...
7
8 //Add additional code

```


SuiteScript 2.x Search Operators

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following table lists each search type and the fields that support it.

Search Operator	List/Record	Currency Decimal Number Time of Day	Date	Box	Document, Image	Email Address Free-Form Text, Long Text Password Percent Phone Number Rich Text, Text Area	Multi Select
after	—	—	✓	—	—	—	—
allof	—	—	—	—	—	—	✓
any	—	✓	—	—	—	✓	—
anyof	✓	—	—	—	✓	—	✓
before	—	—	✓	—	—	—	—
between	—	✓	—	—	—	—	—
contains	—	—	—	—	—	✓	—
doesnotcontain	—	—	—	—	—	✓	—
doesnotstartwith	—	—	—	—	—	✓	—
equalto	—	✓	—	✓	—	—	—
greaterthan	—	✓	—	—	—	—	—
greaterthano norequalto	—	✓	—	—	—	—	—
haskeywords	—	—	—	—	—	✓	—
is	—	—	—	✓	— — —	✓	—
isempty	—	✓	✓	—	—	✓	—
isnot	—	—	—	—	—	✓	—
isnotempty	—	✓	✓	—	—	✓	—
lessthan	—	✓	—	—	—	—	—
lessthano requalto	—	✓	—	—	—	—	—

Search Operator	List/Record	Currency Decimal Number Time of Day	Date	Box	Document, Image	Email Address Free-Form Text, Long Text Password Percent Phone Number Rich Text, Text Area	Multi Select
noneof	✓	—	—	—	✓	—	✓
notafter	—	—	✓	—	—	—	—
notallof	—	—	—	—	—	—	✓
notbefore	—	—	✓	—	—	—	—
notbetween	—	✓	—	—	—	—	—
notequalto	—	✓	—	—	—	—	—
notgreaterthan	—	✓	—	—	—	—	—
notgreaterthanorequalto	—	✓	—	—	—	—	—
notlessthan	—	✓	—	—	—	—	—
notlessthanorequalto	—	✓	—	—	—	—	—
noton	—	—	✓	—	—	—	—
notonorafter	—	—	✓	—	—	—	—
notonorbefore	—	—	✓	—	—	—	—
notwithin	—	—	✓	—	—	—	—
on	—	—	✓	—	—	—	—
onorafter	—	—	✓	—	—	—	—
onorbefore	—	—	✓	—	—	—	—
startswith	—	—	—	—	—	✓	—
within	—	—	✓	—	—	—	—

search.Sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for supported sorting directions used with search.createColumn(options) .
Module	N/search Module
Sibling Module Members	N/search Module Members
Since	2015.2

Values

Value
ASC
DESC
NONE

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var currencyColumn = search.createColumn({
4   name: 'currency',
5   sort: search.Sort.ASC
6 });
7 ...
8 //Add additional code

```

search.Summary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for summary types used by the Column.summary or Filter.summary properties. For more information about each search summary type, see the help topic SuiteScript 1.0 Documentation .
-------------------------	---

Module	N/search Module
Sibling Module Members	N/search Module Members
Since	2015.2

Values

Value
GROUP
COUNT
SUM
AVG
MIN
MAX

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearchFilter = search.createFilter({
4     name: 'entity',
5     summary: search.Summary.GROUP
6 });
7 ...
8 //Add additional code

```

search.Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for search types supported in the N/search Module . Use this enum to set the value for the options.type parameter of the search.create(options) method.
Module	N/search Module
Sibling Module Members	N/search Module Members
Since	2015.2

Note: You must adjust your account settings according to each search type being used in your scripts to avoid potential errors.

Values

Note: A search type is not a synonym for a record type. The supported search types listed below do not necessarily correspond with the supported record types listed in the [N/record Module](#).

Value	Value	Value
Value	Value	Value
■ ACCOUNT ■ ACCOUNTING_BOOK ■ ACCOUNTING_CONTEXT ■ ACCOUNTING_PERIOD ■ ACTIVITY ■ ADV_INTER_COMPANY_JOURNAL_ENTRY ■ AGGR_FIN_DAT ■ ALLOC_RECOMMENDATION_DEMAND ■ ALLOC_RECOMMENDATION_DETAIL ■ AMORTIZATION_SCHEDULE ■ AMORTIZATION_TEMPLATE ■ ASSEMBLY_BUILD ■ ASSEMBLY_ITEM ■ ASSEMBLY_UNBUILD ■ AUTHENTICATE_DEVICE_INPUT ■ BALANCE_TRX_BY_SEGMENTS ■ BALANCING_DETAIL ■ BALANCING_RESULT ■ BALANCING_TRANSACTION ■ BILLING_ACCOUNT ■ BILLING_ACCOUNT_BILL_CYCLE ■ BILLING_ACCOUNT_BILL_REQUEST ■ BILLING_CLASS ■ BILLING_RATE_CARD ■ BILLING_REVENUE_EVENT ■ BILLING_SCHEDULE ■ BIN ■ BIN_ITEM_BALANCE ■ BIN_TRANSFER ■ BIN_WORKSHEET ■ BLANKET_PURCHASE_ORDER ■ BOM ■ BOM_REVISION ■ BONUS ■ BONUS_TYPE ■ BUDGET_EXCHANGE_RATE	■ FOLDER ■ FULFILLMENT_REQUEST ■ GATEWAY_NOTIFICATION ■ GENERIC_RESOURCE ■ GIFT_CERTIFICATE ■ GIFT_CERTIFICATE_ITEM ■ GLOBAL_ACCOUNT_MAPPING ■ GLOBAL_INVENTORY_RELATIONSHIP ■ GL_LINES_AUDIT_LOG ■ GL_NUMBERING_SEQUENCE ■ GOAL ■ IMPORTED_EMPLOYEE_EXPENSE ■ INBOUND_SHIPMENT ■ INSTALLMENT ■ INTER_COMPANY_JOURNAL_ENTRY ■ INTER_COMPANY_TRANSFER_ORDER ■ INVENTORY_ADJUSTMENT ■ INVENTORY_BALANCE ■ INVENTORY_COST_REVALUATION ■ INVENTORY_COUNT ■ INVENTORY_DEMAND ■ INVENTORY_DETAIL ■ INVENTORY_ITEM ■ INVENTORY_NUMBER ■ INVENTORY_NUMBER_BIN ■ INVENTORY_NUMBER_ITEM ■ INVENTORY_STATUS ■ INVENTORY_STATUS_CHANGE ■ INVENTORY_STATUS_LOCATION ■ INVENTORY_TRANSFER ■ INVOICE ■ INVOICE_GROUP ■ INVT_NUMBER_ITEM_BALANCE ■ ISSUE ■ ITEM ■ ITEM_ACCOUNT_MAPPING	■ PLANNING_REPOSITORY_ITEM_LOCATION ■ PLANNING_REPOSITORY_SOURCE ■ PLANNING_RULE_GROUP ■ PLANNING_VIEW ■ PORTLET ■ PRICE_BOOK ■ PRICE_LEVEL ■ PRICE_PLAN ■ PRICING ■ PRICING_GROUP ■ PROJECT_EXPENSE_TYPE ■ PROJECT_IC_CHARGE_REQUEST ■ PROJECT_TASK ■ PROJECT_TEMPLATE ■ PROMISING_SETUP ■ PROMOTION_CODE ■ PROSPECT ■ PURCHASE_CONTRACT ■ PURCHASE_ORDER ■ PURCHASE_REQUISITION ■ RECENT_RECORD ■ RECEIVED_VENDOR_BILL ■ RES_ALLOCATION_TIME_OFF_CONFLICT ■ RESOURCE_ALLOCATION ■ RESTLET ■ RETURN_AUTHORIZATION ■ REVENUE_ARRANGEMENT ■ REVENUE_COMMITMENT ■ REVENUE_COMMITMENT_REVERSAL ■ REVENUE_PLAN ■ REV_REC_PLAN_AND_SCHEDULE ■ REV_REC_SCHEDULE ■ REV_REC_TEMPLATE ■ ROLE

Value	Value	Value
■ BULK_OWNERSHIP_TRANSFER	■ ITEM_BIN_NUMBER	■ S_C_M_PREDICTED_RISKS
■ BUNDLE_INSTALLATION_SCRIPT	■ ITEM_COLLECTION	■ S_C_M_PREDICTION_TRAIN_HISTORY
■ CALENDAR_EVENT	■ ITEM_COLLECTION_ITEM_MAP	■ S_C_M_PREDICTION_TRAIN_W_Q_STATUS
■ CAMPAIGN	■ ITEM_DEMAND_PLAN	■ SALES_CHANNEL
■ CARDHOLDER_AUTHENTICATION	■ ITEM_FULFILLMENT	■ SALES_ORDER
■ CARDHOLDER_AUTHENTICATION_EVENT	■ ITEM_GROUP	■ SALES_ROLE
■ CASH_REFUND	■ ITEM_LOCATION_CONFIGURATION	■ SALES_TAX_ITEM
■ CASH_SALE	■ ITEM_PROCESS_FAMILY	■ SAVED_SEARCH
■ CHALLENGE_SHOPPER_INPUT	■ ITEM_PROCESS_GROUP	■ SCHEDULED_SCRIPT
■ CHARGE	■ ITEM_RECEIPT	■ SCHEDULED_SCRIPT_INSTANCE
■ CHARGE_RULE	■ ITEM_REVISION	■ SCRIPT_DEPLOYMENT
■ CHECK	■ ITEM_SUPPLY_PLAN	■ SERIALIZED_ASSEMBLY_ITEM
■ CLASSIFICATION	■ JOB	■ SERIALIZED_INVENTORY_ITEM
■ CLIENT_SCRIPT	■ JOB_STATUS	■ SERVICE_ITEM
■ CMS_CONTENT	■ JOB_TYPE	■ SHIP_ITEM
■ CMS_CONTENT_TYPE	■ JOURNAL_ENTRY	■ SHOPPING_CART
■ CMS_PAGE	■ KIT_ITEM	■ SOLUTION
■ COM_SEARCH_BOOST	■ LABOR_BASED_PROJECT_REVENUE_RULE	■ STATE
■ COM_SEARCH_BOOST_TYPE	■ LABOR_CATEGORY	■ STATISTICAL_JOURNAL_ENTRY
■ COM_SEARCH_GROUP_SYN	■ LABOR_COST_CARD	■ STORE_PICKUP_FULFILLMENT
■ COM_SEARCH_ONE_WAY_SYN	■ LABOR_COST_CARD_ITEM	■ SUBSCRIPTION
■ COMM_AN_SESSION	■ LABOR_COST_CARD_SEGMENT	■ SUBSCRIPTION_CHANGE_ORDER
■ COMMERCE_CATEGORY	■ LABOR_COST_ELEMENT	■ SUBSCRIPTION_LINE
■ COMMERCE_SEARCH_ACTIVITY_DATA	■ LEAD	■ SUBSCRIPTION_LINE_REVISION
■ COMPETITOR	■ LOCATION	■ SUBSCRIPTION_PLAN
■ CONSOLIDATED_EXCHANGE_RATE	■ LOT_NUMBERED_ASSEMBLY_ITEM	■ SUBSCRIPTION_RENEWAL_HISTORY
■ CONTACT	■ LOT_NUMBERED_INVENTORY_ITEM	■ SUBSCRIPTION_TERM
■ CONTACT_CATEGORY	■ MANUFACTURING_COST_TEMPLATE	■ SUBSIDIARY
■ CONTACT_ROLE	■ MANUFACTURING_OPERATION_TASK	■ SUBTOTAL_ITEM
■ COST_CATEGORY	■ MANUFACTURING_ROUTING	■ SUITELET
■ COUPON_CODE	■ MAP_REDUCE_SCRIPT	■ SUITE_SCRIPT_DETAIL
■ CREDIT_CARD_CHARGE	■ MARKUP_ITEM	■ SUPPLY_CHAIN_SNAPSHOT
■ CREDIT_CARD_REFUND	■ MASSUPDATE_SCRIPT	■ SUPPLY_CHAIN_SNAPSHOT_DETAILS
■ CREDIT_MEMO	■ MEM_DOC	■ SUPPLY_CHANGE_ORDER
■ CURRENCY	■ MERCHANDISE _HIERARCHY_LEVEL	■ SUPPLY_PLAN_DEFINITION
■ CURRENCY_EXCHANGE_RATE	■ MERCHANDISE_HIERARCHY_NODE	■ SUPPORT_CASE
■ CUSTOMER	■ MERCHANDISE_HIERARCHY_VERSION	■ SYSTEM_NOTE
■ CUSTOMER_CATEGORY	■ MESSAGE	■ TASK
■ CUSTOMER_DEPOSIT	■ MFG_PLANNED_TIME	■ TAX_DETAIL
■ CUSTOMER_MESSAGE	■ NEXUS	■ TAX_GROUP
■ CUSTOMER_PAYMENT	■ NON_INVENTORY_ITEM	■ TAX_PERIOD
■ CUSTOMER_PAYMENT_AUTHORIZATION	■ NOTE	■ TAX_TYPE
■ CUSTOMER_REFUND	■ NOTE_TYPE	■ TERM
■ CUSTOMER_STATUS	■ OPPORTUNITY	■ TIMESHEET_APPROVAL
■ CUSTOMER_SUBSIDIARY_RELATIONSHIP	■ ORDER_RESERVATION	■ TIME_APPROVAL

Value	Value	Value
- CUSTOM_PURCHASE - CUSTOM_RECORD - CUSTOM_SALE - CUSTOM_TRANSACTION - DELETED_RECORD - DEPARTMENT - DEPOSIT - DEPOSIT_APPLICATION - DESCRIPTION_ITEM - DISCOUNT_ITEM - DOWNLOAD_ITEM - EMAIL_TEMPLATE - EMPLOYEE - EMPLOYEE_CHANGE_REQUEST - EMPLOYEE_CHANGE_REQUEST_TYPE - EMPLOYEE_CHANGE_TYPE - EMPLOYEE_PAYROLL_ITEM - EMPLOYEE_STATUS - EMPLOYEE_TYPE - END_TO_END_TIME - ENTITY - ENTITY_ACCOUNT_MAPPING - ESTIMATE - EXPENSE_AMORTIZATION_EVENT - EXPENSE_AMORT_PLAN_AND_SCHEDULE - EXPENSE_CATEGORY - EXPENSE_PLAN - EXPENSE_REPORT - EXPENSE_REPORT_POLICY - FAIR_VALUE_PRICE - FINANCIAL_INSTITUTION - FIXED_AMOUNT_PROJECT_REVENUE_RULE	- ORDER_TYPE - OTHER_CHARGE_ITEM - OTHER_NAME - OTHER_NAME_CATEGORY - PARTNER - PARTNER_CATEGORY - PAYCHECK - PAYCHECK_JOURNAL - PAYMENT_EVENT - PAYMENT_INSTRUMENT - PAYMENT_ITEM - PAYMENT_METHOD - PAYMENT_OPTION - PAYMENT_RESULT_PREVIEW - PAYROLL_SETUP - PAYROLL_ITEM - PCT_COMPLETE_PROJECT_REVENUE_RULE - PERFORMANCE_METRIC - PERFORMANCE_REVIEW - PERFORMANCE_REVIEW_SCHEDULE - PERIOD_END_JOURNAL - PERMISSION - PHONE_CALL - PICK_STRATEGY - PICK_TASK - PLANNED_ORDER - PLANNING_ENGINE_MESSAGE - PLANNING_ENGINE_PEGGING - PLANNING_ENGINE_RESULT - PLANNING_ITEM_CATEGORY - PLANNING_ITEM_GROUP - PLANNING_REPOSITORY_ALLOCATION - PLANNING_REPOSITORY_BOM_EDGE	- TIME_BILL - TIME_ENTRY - TIME_OFF_CHANGE - TIME_OFF_PLAN - TIME_OFF_REQUEST - TIME_OFF_RULE - TIME_OFF_TYPE - TIME_SHEET - TOPIC - TRANSACTION - TRANSFER_ORDER - UBER - UNITS_TYPE - UNLOCKED_TIME_PERIOD - USAGE - USEREVENT_SCRIPT - VENDOR - VENDOR_BILL - VENDOR_CATEGORY - VENDOR_CREDIT - VENDOR_PAYMENT - VENDOR_PREPAYMENT - VENDOR_PREPAYMENT_APPLICATION - VENDOR_RETURN_AUTHORIZATION - VENDOR_SUBSIDIARY_RELATIONSHIP - WAVE - WBS - WEBSITE - WORKFLOW_ACTION_SCRIPT - WORK_ORDER - WORK_ORDER_CLOSE - WORK_ORDER_COMPLETION - WORK_ORDER_ISSUE - WORKPLACE - ZONE

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/search Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mySearch = search.create({
4     type: search.Type.CUSTOMER,
5     filters: filters,
6     columns: columns
7 });

```

```
8 | ...
9 | //Add additional code
```

N/sftp Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/sftp module to manage folders and upload or download files from external SSH file transfer (SFTP) servers. You can perform the following SFTP functions using the N/sftp module:

SFTP servers can be hosted by your organization or by a third party. NetSuite does not provide SFTP server functionality. All SFTP transfers to or from NetSuite must originate from SuiteScript. It is not possible for external clients to initiate file transfers using SFTP.

Use SSH keys to establish an SFTP connection. By using the keys, you can manage files and directories by using the SFTP protocol. For more information, see the help topic [SSH Keys for SFTP](#). For more information about working with keys in SuiteScript, see the N/keyControl Module, see [N/keyControl Module](#).

Go To Samples 



Important: All paths, directories, and filenames that contain wildcards such as ? and * must have those characters escaped, unless these characters are specifically intended to work as wildcards.



Note: To use an external server to initiate a NetSuite file transfer that doesn't use SFTP, you can use RESTlets or SOAP web services. In SuiteScript, RESTlets can respond to requests containing file data and save them in the File Cabinet. RESTlets can also respond to requests for file data by loading the contents from the File Cabinet and returning them in the response. Note that binary file content must be received or sent as Base64 encoded Strings. See the help topic [SuiteScript 2.x RESTlet Script Type](#) for more information.

In SOAP web services, applications can invoke CRUD operations on the file record to populate or change the contents of the File Cabinet. See the help topics [SuiteTalk SOAP Web Services Platform Guide](#) and [File](#) for more information.

In This Help Topic

- [N/sftp Module Members](#)
- [Connection Object Members](#)
- [Setting up an SFTP Transfer](#)
- [SFTP Authentication](#)
- [Supported Cipher Suites and Host Key Types](#)
- [Supported SuiteScript File Types](#)
- [N/keyControl Module](#)

N/sftp Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	sftp.Connection	Object	Server scripts	Represents a connection to the account on the remote FTP server.
Method	sftp.createConnection(options)	sftp.Connection	Server scripts	Establishes a connection to a remote FTP server.
Enum	sftp.MAX_CONNECT_TIMEOUT	enum	Server scripts	Holds the values for maximum connection timeout.
	sftp.MIN_CONNECT_TIMEOUT	enum	Server scripts	Holds the values for minimum connection timeout.
	sftp.MAX_PORT_NUMBER	enum	Server scripts	Holds the values for the maximum port number.
	sftp.MIN_PORT_NUMBER	enum	Server scripts	Holds the values for the minimum port number.
	sftp.DEFAULT_PORT_NUMBER	enum	Server scripts	Holds the values for the default port number.
	sftp.Sort	enum	Server scripts	Holds the values to be used to sort the listed directory. Use this enum to set the value of the <code>options.sort</code> parameter of the Connection.list(options) method.

Connection Object Members

The following members are called on the [sftp.Connection](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Connection.download(options)	file.File	Server scripts	Downloads a file from the remote FTP server.
	Connection.upload(options)	void	Server scripts	Uploads a file to the remote FTP server.
	Connection.makeDirectory(options)	string	Server scripts	Creates an empty directory.
	Connection.removeDirectory(options)	void	Server scripts	Removes an empty directory.
	Connection.removeFile(options)	void	Server scripts	Removes a file in a directory.
	Connection.move(options)	void	Server scripts	Moves a file or directory from one location to another.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Connection.list(options)	Array<Object>	Server scripts	Lists the remote directory.
Enum	Connection.MAX_FILE_SIZE	enum	Server scripts	Holds the values for the maximum file size.
	Connection.MAX_TRANSFER_TIMEOUT	enum	Server scripts	Holds the values for the maximum transfer timeout.

N/sftp Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/sftp module:

- [Upload and Download a File](#)
- [Manage Files and Directories](#)
- [Set Conditional Default Settings Using N/sftp Enums](#)

Upload and Download a File

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to upload and download a file.

To obtain a real host key, use `ssh-keyscan <domain>`.

To create a real password GUID, obtain a password value from a credential field on a form. For more information, see [Form.addCredentialField\(options\)](#). Also see [N/https Module Script Samples](#) for a Suitelet sample that shows creating a form field that generates a GUID.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/sftp', 'N/file'], (sftp, file) => {
5       const myPwdGuid = "B34672495064525E5D65032D63852301";
6       const myHostKey = "AAA1234567890Q=";
7
8       // Establish a connection to a remote FTP server
9       let connection = sftp.createConnection({
10         username: 'myuser',
11         passwordGuid: myPwdGuid,
12         url: 'host.somewhere.com',
13         directory: 'myuser/wheres/my/file',
14         hostKey: myHostKey
15       });
16
17       // Create a file to upload using the N/file module

```

```

18 let myFileToUpload = file.create({
19   name: 'originalname.js',
20   fileType: file.Type.PLAINTEXT,
21   contents: 'I am a test file.'
22 });
23
24 // Upload the file to the remote server
25 connection.upload({
26   directory: 'relative/path/to/remote/dir',
27   filename: 'newFileNameOnServer.js',
28   file: myFileToUpload,
29   replaceExisting: true
30 });
31
32 // Download the file from the remote server
33 let downloadedFile = connection.download({
34   directory: 'relative/path/to/file',
35   filename: 'downloadMe.js'
36 });
37 });

```

Manage Files and Directories

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how you can manage files and directories.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4  require(['N/sftp', 'N/file'], (sftp, file) => {
5   // Establish a connection
6   log.debug('Establishing SFTP connection...');
7   let connection = sftp.createConnection({
8     username: 'sftpuser',
9     keyId: 'custkeysftp_nft_demo_key',
10    url: 'myurl',
11    port: 22,
12    directory: 'inbound',
13    hostKey: 'myhostkey'
14  });
15  log.debug('Connection established!');
16
17  // List the directory and log the number of elements there
18  let list = connection.list({
19    path: 'yyy/test'
20  });
21  log.debug('Items in directory "test" at the beginning: ' + list.length);
22
23  // Generate the test file
24  log.debug('Generating test file...');
25  let myFileToUpload = file.create({
26    name: 'asdf.txt',
27    fileType: file.Type.PLAINTEXT,
28    contents: 'I am a test file.'
29  });
30  log.debug('Test file generated, uploading to "test" directory...');
31
32  // Upload the test file

```

```

33 connection.upload({
34     directory: 'yyy/test',
35     filename: 'af.txt',
36     file: myFileToUpload,
37     replaceExisting: true
38 });
39 log.debug('Upload complete!');
40
41 // List the directory to confirm there is one more file than before
42 list = connection.list({
43     path: 'yyy/test'
44 });
45 log.debug('Items in directory "test" after the upload: ' + list.length);
46
47 // Create a new directory
48 log.debug('Creating directory "test2"...');
49 try {
50     connection.makeDirectory({
51         path: 'yyy/test2'
52     });
53     log.debug('Directory created. ');
54 } catch (e) {
55     log.debug('Directory not created. ');
56     log.error(e.message);
57 }
58 list = connection.list({
59     path: 'yyy/test2'
60 });
61 log.debug('Items in directory "test2": ' + list.length);
62
63 // Move the test file to the new directory
64 log.debug('Moving the test file from "test" to "test2"...');
65 connection.move({
66     from: 'yyy/test/af.txt',
67     to: 'yyy/test2/af.txt'
68 })
69 log.debug('File moved!');
70
71 // List the original directory again to see the file is moved
72 list = connection.list({
73     path: 'yyy/test'
74 });
75 log.debug('Items in directory "test" after the upload: ' + list.length);
76
77 // List the new directory for the file
78 list = connection.list({path: 'yyy/test2'});
79 log.debug('Items in directory "test2" after the upload: ' + list.length);
80 log.debug(JSON.stringify(list));
81
82 // Try to remove the directory
83 log.debug('Removing directory "test2"...');
84 try {
85     connection.removeDirectory({
86         path: 'yyy/test2'
87     });
88     log.debug('Directory removed! ');
89 } catch (e) {
90     log.debug('Directory not removed! ');
91     log.error(e.message);
92 }
93
94 // The directory is not empty so delete the file first
95 log.debug('Removing test file from "test2" directory...');
96 connection.removeFile({
97     path: 'yyy/test2/af.txt'
98 });
99 log.debug('Test file removed!');
100
101 list = connection.list({
102     path: 'yyy/test2'
103 });
104 log.debug('Items in directory "test2": ' + list.length);
105

```

```

106 // Try to remove the directory again
107 log.debug('Removing directory "test2"...');
108 try {
109     connection.removeDirectory({
110         path: 'yyy/test2'
111     });
112     log.debug('Directory removed!');
113 } catch (e) {
114     log.debug('Directory not removed!');
115     log.error(e.message);
116 }
117
118 // Try to list the removed directory
119 log.debug('Trying to list directory "test2"...');
120 try {
121     list = connection.list({
122         path: 'yyy/test2'
123     });
124 } catch (e) {
125     log.error(e.message);
126 }
127 });

```

Set Conditional Default Settings Using N/sftp Enums

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use the N/sftp module enums to set conditional default settings. The sample creates a secure connection and attempts to upload and add a large file.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/file', 'N/sftp', 'N/error'],
8  function(file, sftp, error) {
9      return {
10         beforeLoad: function(){
11             var portNumber = -1;
12             var connectTimeout = -1;
13             var transferTimeout = -1;
14             //these variables can be taken as parameters of the script instead
15
16             if (portNumber < sftp.MIN_PORT_NUMBER || portNumber > sftp.MAX_PORT_NUMBER)
17                 portNumber = sftp.DEFAULT_PORT_NUMBER;
18             if (connectTimeout < sftp.MIN_CONNECT_TIMEOUT) connectTimeout = sftp.MIN_CONNECT_TIMEOUT; else if (connectTimeout >
19 sftp.MAX_CONNECT_TIMEOUT)
20                 connectTimeout = sftp.MAX_CONNECT_TIMEOUT;
21
22             var connection = sftp.createConnection({
23                 username: 'sftpuser',
24                 keyId: 'custkey1',
25                 url: '192.168.0.100',
26                 port: portNumber,
27                 directory: 'inbound',
28                 timeout: connectTimeout,
29                 hostKey: "AAAAB3NzaC1yc2EAAAADAQABAAQDMiFkH2vTxditype8nem7+1S3x7dTQR/A67KdsR/5C2WUcDipBzYhHb
nG6Am12Nd2t1M01LnaBZA6/8P4Y9x/sGTxtsdE/MzeGDUBn6HB1QvgIrhX62wgoKGQ+P21EA01+Vz8y3/MB1NmD7Fc62cJ9Mu88YA6jwJOIPZeHYNNVyIm90rY6VyzYyvSJh

```

```

H0x75XyvGnijJQF4G8C4c8u/UVpF/sE16xKZtly2Rx0aDL2FsDRtpyPmM602/R6ISbsmgab3MzzAEIu+zLDMdIBJn3cDhNt1F7Rar6Tu0u18KCKk8GPxbnxDuG4sCN0nX
PYkDXSMubM/ocRjYGtqdZUMmeTf3"
29     });
30
31     // can also be a big file (created for example by async search)
32     var myFileToUpload = file.create({
33         name: 'originalname.txt',
34         fileType: file.Type.PLAINTEXT,
35         contents: 'I am a test file.'
36     });
37
38     if (myFileToUpload.size > connection.MAX_FILE_SIZE)
39         throw error.create({name: "FILE_IS_TOO_BIG", message: "The file you are trying to upload is too big"});
40
41     var minTransferTimeout = 10;
42     if (transferTimeout > connection.MAX_TRANSFER_TIMEOUT)
43         transferTimeout = connection.MAX_TRANSFER_TIMEOUT;
44     else if (transferTimeout < minTransferTimeout)
45         transferTimeout = minTransferTimeout;
46
47     connection.upload({
48         directory: 'files',
49         filename: 'test.txt',
50         file: myFileToUpload,
51         replaceExisting: true,
52         timeout: transferTimeout
53     });
54 }
55 };
56 });

```

Setting up an SFTP Transfer

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

- [Development Preparation for SFTP transfers](#)
- [Execution of an SFTP transfer](#)

Development Preparation for SFTP transfers

Follow these steps to successfully connect to your SFTP server with SuiteScript:

1. Talk to your SFTP service provider about your plans.
 - Determine the connection properties required to connect with your external SFTP server. For example:
 - username
 - password/key
 - url
 - port
 - upload/download directories
 - host key
 - host key type
 - Make sure that you know your provider's practices around host key changes, maintenance and failover. For example, find out if there are multiple URLs or ports to try.
 - Check compatibility with the SFTP ciphers supported by NetSuite. See [Supported Cipher Suites and Host Key Types](#).

- Determine if your provider requires at-rest file encryption (in addition to what the SFTP protocol provides during transfer). Decide if you need to add file encryption.
- 2. Build a credential management Suitelet to capture username and password token. Then, test the connection.
 - Create custom fields to store the user's SFTP username and password token
 - Implement the Suitelet.
 - a. Draw a form on a GET request.
 - b. Save the username and password token on a POST request.
 - c. Test the connection.

See [Creating a Suitelet Form that Contains a Credential Field](#).
- 3. Build a server script to handle operations such as:
 - Load a File Cabinet file and upload it to the SFTP server.
 - Download an on demand file from the SFTP server and save it in File Cabinet.

Execution of an SFTP transfer

The following steps occur during a successful SFTP transfer using SuiteScript:

1. The user submits their SFTP credentials using a Suitelet.
2. The Suitelet captures and stores the credential token.
3. A server script is triggered.
4. The script identifies the appropriate credential token and other connection attributes, and establishes the SFTP connection.
5. The script requests the transfer.

SFTP Authentication

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Please review the following sections for an overview of SFTP authentication when using SuiteScript.

- [Two-Factor Authentication](#)
- [Credential Tokenization](#)
- [Creating a Suitelet Form that Contains a Credential Field](#)
- [Reading the Credential Token in a Suitelet](#)
- [Credential Management](#)
- [Credential GUID Persistence](#)
- [Protocols](#)
- [Host Key Verification](#)
- [Retrieving the Host Key of an External SFTP Server](#)

Two-Factor Authentication

You can use two-factor authentication to connect to an SFTP server. A combination of password and key is supported. For more information, see [sftp.createConnection\(options\)](#).

Credential Tokenization

SuiteScript provides the ability for users to securely store authentication credentials in such a way that scripts are able to use encrypted saved credentials without being able to see their contents. The script author must specify which scripts and domains are permitted for use with the credential. To restrict the credential for use by SuiteScript automation triggered by the same user who originally saved the credential, the script author can set the `restrictToCurrentUser` parameter.

Creating a Suitelet Form that Contains a Credential Field

Note: Credential fields have a default maximum length of 32 characters. If needed, use the `Field.maxLength` property to change this value

```

1  ...
2
3  if (request.method === context.Method.GET){
4      var form = serverWidget.createForm({title: 'Enter SFTP Credentials'});
5      var credField = form.addCredentialField({
6          id: 'custfield_sftp_password_token',
7          label: 'SFTP Password',
8          restrictToScriptIds: ['customscript_sftp_script'],
9          restrictToDomains: ['acmebank.com'],
10         restrictToCurrentUser: true //Depends on use case
11     });
12
13     credField.maxLength = 64;
14     form.addSubmitButton();
15     response.writePage(form);
16 }
17
18 ...

```

Reading the Credential Token in a Suitelet

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1  ...
2
3  var request = context.request;
4  if (request.method === context.Method.POST){
5      // Read the request parameter matching the field ID we specified in the form
6      var passwordToken = request.parameters.custfield_sftp_password_token;
7      log.debug({
8          title: 'New password token',
9          details: passwordToken
10     });
11
12     // In a real-world script, "passwordToken" is saved into a custom field here...
13 }
14 ...

```

Credential Management

User passwords can be stored using secure Credential Fields. This type of field is available on the `serverWidget.Form` Object in the [N/ui/serverWidget Module](#).

Encrypted *custom* fields do not support tokenization and are not compatible with the SFTP module. Instead, you can add a credential field using [Form.addCredentialField\(options\)](#).

Credential GUID Persistence

Scripts may store credential tokens as convenient for the script author. Credential tokens are not related to the password in its original or encrypted form within NetSuite. These tokens are unique identifiers which allow a script to refer to an encrypted secret securely stored within the SuiteCloud platform. Automatic password expiration is not currently provided, nor is it possible to view an inventory of saved credentials in the user interface.

Protocols

The SFTP module allows scripts to transfer files using the SSH File Transfer Protocol only. Other file-based protocols such as FTP, FTPS, SCP are not supported by this module.

Host Key Verification

An SFTP server identifies itself using a host key when a client attempts to establish a connection. Host keys are unique keys that the underlying SSH protocol uses to allow the server to provide a fingerprint. Clients can verify that the expected server has responded to the connection request for a particular URL and port number.

SuiteScript requires that the host key is provided by the script attempting to connect so that the SFTP module can check the identity of the SFTP server. This security best practice is commonly referred to as "Strict Host Key Checking".

Host keys are used to verify the identity of the server, not the client. For more information, see the help topic [SSH Keys for SFTP](#).

There is no SuiteScript API call for checking the host key of a remote SFTP server, or an option to disable strict host key checking. The script must always know the host key ahead of time.

You can use the OpenSSH `ssh-keyscan` tool to check the host key of an external SFTP site. See [Retrieving the Host Key of an External SFTP Server](#) and [The OpenBSD's ssh-keyscan page](#).

Retrieving the Host Key of an External SFTP Server

An example usage checking the RSA host key of URL: `example.com` at port: `1234` from a `*nix` shell follows:

```
1 $ ssh-keyscan -t rsa -p 1234 example.com
2
3 AATpn1P9jB+cQx9Jq9UeZjA1245X7SBDcRiKh+Sok56VzSw==
```

You should always pass the key type and port number. This practice helps to avoid ambiguity in the response from the external SFTP server.

Supported Cipher Suites and Host Key Types

📌 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SFTP connections are encrypted. For security reasons, NetSuite requires that the server to which a connection request is being made supports at least one of the following ciphers: `aes128-ctr`, `aes192-ctr` or

aes256-ctr. These cipher specs refer to the AES cipher in Counter stream cipher mode using 128,192 or 256 bit key sizes.

To check interoperability of your SFTP server or service provider, refer to the following table:

Communication protocol	SFTP (SSH + FTP) is supported. Only CTR (and not CBC) ciphers are allowed. Your SFTP server can use the following encryption algorithms: ■ AES 128-CTR ■ AES 192-CTR ■ AES 256-CTR ■ RSA ■ DSA ■ ECDSA Files are not additionally encrypted during transfer. The entire transmission is encrypted by the SSH protocol.
Authentication mechanism	Username Password Password/SSH key with or without passphrase
SSH host key	With each connection request, you must supply the host key. Any host key changes need to be managed manually.
GUID	The password GUID should be a value generated by a credential field from a Suitelet using Form.addCredentialField(options). The password GUID field's originating credential field must include the SFTP domain on the restrictToDomains parameter. The password GUID field's originating credential field must include the script utilizing the password GUID on the restrictToScriptIds parameter.
Signature algorithms	RSA: ■ ssh-rsa ■ rsa-sha2-512-cert-v01@openssh.com ■ rsa-sha2-256-cert-v01@openssh.com ■ rsa-sha2-512 ■ rsa-sha2-256 DSA: ■ ssh-dss-cert-v01@openssh.com ■ ssh-dss EC: ■ ecdsa-sha2-nistp256-cert-v01@openssh.com ■ ecdsa-sha2-nistp384-cert-v01@openssh.com ■ ecdsa-sha2-nistp521-cert-v01@openssh.com ■ ecdsa-sha2-nistp256 ■ ecdsa-sha2-nistp384 ■ ecdsa-sha2-nistp521 EDDSA:

- ssh-ed25519-cert-v01@openssh.com
- ssh-ed25519

Firewall policy is at the discretion of your SFTP service provider.

Supported SuiteScript File Types

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript has two types of file objects: previously existing files in the NetSuite File Cabinet, and on demand files created using SuiteScript API calls such as [file.create\(options\)](#) or [Connection.download\(options\)](#).

File Cabinet and on demand files are supported by [Connection.upload\(options\)](#).

Note that [Connection.download\(options\)](#) returns an on demand file object. For an on demand file to be saved into the File Cabinet, it must receive a folder ID and be explicitly saved.

```

1  ...
2
3  var downloadedFile = sftp.download({...});
4  downloadedFile.folder = 1234;
5  downloadedFile.save();
6
7  ...

```



Important: It's possible that a file you are downloading may be encrypted, or your SFTP provider may expect an uploaded file in an encrypted format in accordance with that provider's security practices. Make sure that you understand your provider's expectations and the cryptographic capabilities in SuiteScript (see [N/crypto Module](#)).

You can also create and remove directories. For more information, see [N/sftp Module Members](#).

Syntax

```

1  require(['N/sftp', 'N/file'], function (sftp, file) {
2      var connection = sftp.createConnection({
3          // Username supplied by the administrator of the external SFTP server.
4
5          username: 'myuser',
6          // Refers to the Password supplied by the administrator of the external SFTP server.
7          // The Password Token/GUID obtained by reading the form POST parameter associated
8          // with user submission of a form containing a Credential Field.
9          // Value would typically be read from a custom field.
10         passwordGUID: "B34672495064525E5D65032D63B52301",
11
12         // The URL supplied by the administrator of the external SFTP server.
13         url: 'host.somewhere.com',
14
15         // The SFTP Port number supplied by the administrator of the external SFTP server
16         // defaults to 22).
17         port: 22,
18
19         // The transfer directory supplied by the administrator of the external SFTP server
20         // (optional).
21         directory: 'transferfiles',
22
23         /* RSA Host Key obtained using the ssh-keyscan tool.
24
25         $ ssh-keyscan -t rsa -p 22 host.somewhere.com
26

```

```

27      AATpn1P9jB+cQx9Jq9UeZjA1245X7SBDcRiKh+Sok56VzSw==
28
29      */
30
31      hostKey: "AATpn1P9jB+cQx9Jq9UeZjA1245X7SBDcRiKh+Sok56VzSw=="
32    });
33
34    // Create a simple file.
35    var myFileToUpload = file.create({
36      name: 'originalname.js',
37      fileType: file.Type.PLAINTEXT,
38      contents: 'I am a test file. Hear me roar.'
39    });
40
41    // Upload the file to the external SFTP server.
42    connection.upload({
43      // Subdirectory within the transfer directory specified when connecting (optional).
44      directory: 'relative/path/to/remote/dir',
45
46      // Alternate file name to use instead of the one given to the file object (optional).
47      filename: 'newFileNameOnServer.js',
48
49      // File to upload.
50      file: myFileToUpload,
51
52      // If a file already exists with that name, replace it instead of failing the upload.
53      replaceExisting: true
54    });
55
56    var downloadedFile = connection.download({
57      // Subdirectory within the transfer directory specified when connecting (optional).
58      directory: 'relative/path/to/file',
59
60      // Name of the file within the above directory on the external SFTP server which
61      // to download.
62      filename: 'downloadMe.js'
63    });
64  });

```

sftp.Connection

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Represents a connection to the account on the remote FTP server.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/sftp Module
Methods and Properties	Connection Object Members
Since	2016.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1  //Add additional code
2  ...
3

```

```

4 var objConnection = sftp.createConnection({ //establish connection to the FTP server
5   username: 'username',
6   keyId: 'custkey1',
7   url: 'host.somewhere.com',
8   directory: 'username/wheres/my/file'
9   hostKey: myHostKey
10 });
11 ...
12 //Add additional code

```

Connection.download(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Downloads a file from the remote FTP server.
Returns	file.File Object
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100 units
Module	N/sftp Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.filename	string	required	The name of the file to download.	2016.2
options.directory	string	optional	The relative path to the directory that contains the file to download. By default, the path is set to the current directory.  Important: This input must take the form of a relative path.	2016.2
options.timeout	number	optional	The number of seconds to allow for the file to download. By default, this value is set to 300 seconds.	2016.2

Errors

Error Code	Thrown If
FTP_MAXIMUM_FILE_SIZE_EXCEEDED	The file size is greater than the maximum file size allowed by NetSuite.

Error Code	Thrown If
FTP_NO_SUCH_FILE_OR_DIRECTORY	The file or directory does not exist.
FTP_TRANSFER_TIMEOUT_EXCEEDED	The transfer is taking longer than the specified options.timeout value.
FTP_INVALID_TRANSFER_TIMEOUT	The options.timeout value is either a negative value, zero or greater than 300 seconds.
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.
CONNECTION_RESET	The connection was reset.
THE_REMOTE_PATH_FOR_FILE_IS_NOT_VALID	The file's remote path is invalid.
CONNECTION_CLOSED_BY_HOST	The connection was closed by the host.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 var downloadedFile = objConnection.download({
5     directory: 'relative/path/to/file',
6     filename: 'downloadMe.js'
7 });
8 });
9 ...
10 //Add additional code

```

Connection.list(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Lists the remote directory. This method returns an array of Objects. Each Object includes the following properties with associated values: - directory- A flag whether the entry corresponds to a directory or a file. If true, it is a directory. If false, it is a file. - name- The name of the file - size- The size of the file - lastModified- The last modification date
Returns	Array<Object>
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units

Module	N/ftp Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.path	String	required	The relative path to directory of file that will be downloaded.	2019.2
options.sort	String	required	The sort options. Use values from sftp.Sort .	2019.2

Errors

Error Code	Thrown If
FTP_INVALID_DIRECTORY	The directory does not exist on the remote FTP server.
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ftp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 let objConnection = connection.list({
5     path: 'yyy/test'
6 });
7 ...
8 // Add additional code

```

 **Note:** Wildcards are accepted. The ? symbol can represent any character. The * symbol can represent any number of characters.

Connection.makeDirectory(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an empty directory.
Returns	void
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/sftp Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.path	string	required	The relative path of a directory to be created.	2019.2

Errors

Error Code	Thrown If
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.
FTP_DIRECTORY_NOT_FOUND	Creating a directory in a non-existent directory.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 objConnection.makeDirectory({
5     path: 'relative/path/to/remote/dir'
6 });
7 ...
8 //Add additional code

```

Connection.move(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Moves a file or directory from one location to another.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/sftp Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.from	String	required	The relative path of the file to be moved from.	2019.2
options.to	String	required	The relative path of the file to be moved to.	2019.2

Errors

Error Code	Thrown If
FTP_INVALID_MOVE	Source is not readable or the target is not writable. Source or target does not exist.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 objConnection.move({
5     from: 'relative/path/to/remote/dir',
6     to: 'relative/path/to/remote/dir/new'
7 });
8 ...
9 // Add additional code

```

Connection.removeDirectory(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes an empty directory.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/sftp Module
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.path	String	required	The relative path of a directory to be deleted.	2019.2

Errors

Error Code	Thrown If
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.
FTP_DIRECTORY_NOT_FOUND	Deleting a directory that does not exist.
FTP_DIRECTORY_NOT_EMPTY	Deleting a directory that is not empty.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 // Add additional code
2 ...
3
4 objConnection.removeDirectory({
5     path: 'relative/path/to/remote/dir'
6 });
7 ...
8 //Add additional code

```

Connection.removeFile(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes a file.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/sftp Module

Since	2019.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.path	String	required	The relative path of the directory location where this file should be removed.	2019.2

Errors

Error Code	Thrown If
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.
FTP_NO_SUCH_FILE_OR_DIRECTORY	The file or directory does not exist.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 objConnection.removeFile({
5     path: 'relative/path/test
6 });
7 ...
8 //Add additional code

```

Connection.upload(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Uploads a file to the remote FTP server. The maximum file size that can be uploaded to is 100 MB.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100 units
Module	N/ftp Module
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.file	file.File	required	The file to upload.	2016.2
options.filename	string	optional	The name to give the uploaded file on server. By default, the filename is the same specified by options.file.  Note: Illegal characters are automatically escaped.	2016.2
options.directory	string	optional	The relative path to the directory where the file should be upload to. By default, the path is set to the current directory.  Important: This input must take the form of a relative path.	2016.2
options.timeout	number	optional	The number of seconds to allow for the file to upload. By default, this value is set to 300 seconds.  Important: This parameter does not specify the overall timeout limit. The value is only applied when no data is received within the specified period.	2016.2
options.replaceExisting	boolean	optional	Indicates whether the file being uploaded should overwrite any file with the name options.filename that already exists in options.directory. If false, the FTP_FILE_ALREADY_EXISTS exception is thrown when a file with the same name already exists in the options.directory. By default, this value is false.	2016.2

Errors

Error Code	Thrown If
FTP_NO_SUCH_FILE_OR_DIRECTORY	The file or directory does not exist.
FTP_TRANSFER_TIMEOUT_EXCEEDED	The transfer is taking longer than the specified options.timeout value.
FTP_INVALID_TRANSFER_TIMEOUT	The options.timeout value is either a negative value, zero or greater than 300 seconds.

Error Code	Thrown If
FTP_FILE_ALREADY_EXISTS	The options.replaceExisting value is false and a file with the same name exists in the remote directory. The error is also thrown when the SFTP server doesn't support the STAT command and instead requires the LIST command. To enable the LIST command, see SFTP: Use LIST to Test That a File Exists preference in Setting General Account Preferences .
CONNECTION_RESET	The connection was reset.
THE_REMOTE_PATH_FOR_FILE_IS_NOT_VALID	The file's remote path is invalid.
CONNECTION_CLOSED_BY_HOST	The connection was closed by the host.
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 objConnection.upload({
5     directory: 'relative/path/to/remote/dir',
6     filename: 'newFileNameOnServer',
7     file: myFileToUpload,
8     replaceExisting: true
9 });
10 ...
11 //Add additional code

```

Connection.MAX_FILE_SIZE

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the maximum file size.
Module	N/sftp Module
Sibling Module Members	N/sftp Module Members
Since	2019.2

Values

Value
100000000

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 var myFileToUpload = file.create({
5     name: 'originalname.txt',
6     fileType: file.Type.PLAINTEXT,
7     contents: 'I am a test file.'
8 });
9
10 if (myFileToUpload.size > connection.MAX_FILE_SIZE)
11     throw error.create({
12         name: "FILE_IS_TOO_BIG",
13         message: "The file you are trying to upload is too big"
14     });
15 ...
16 //Add additional code

```

Connection.MAX_TRANSFER_TIMEOUT

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the maximum transfer timeout.
Module	N/sftp Module
Sibling Module Members	N/sftp Module Members
Since	2019.2

Values

Value
300

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 var minTransferTimeout = 10;
5
6 if (transferTimeout > connection.MAX_TRANSFER_TIMEOUT)
7     transferTimeout = connection.MAX_TRANSFER_TIMEOUT;

```

```

8 | else if (transferTimeout < connection.minTransferTimeout)
9 |     transferTimeout = connection.minTransferTimeout;
10 | ...
11 | //Add additional code

```

sftp.createConnection(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Establishes a connection to a remote FTP server. For more information, see Setting up an SFTP Transfer and Supported Cipher Suites and Host Key Types .
Returns	sftp.Connection which represents the connection
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/sftp Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The host of the remote account.	2016.2
options.passwordGuid	string	required if the keyId or secret parameter is not specified	The password GUID for the remote account. You can create a GUID using Form.addCredentialField(options). For more information, see SFTP Authentication. Or you can use the N/https Module to fetch the GUID value returned from a Suitelet's credential field. You can use this in combination with key ID for two-factor authentication. You cannot use this in combination with a secret ID.	2016.2
options.keyId	string	required if the passwordGuid or secret parameter is not specified	The script ID of the key to be used for authentication. You can upload SSH keys in PEM format at Setup > Company > Keys. For more information, see N/keyControl Module and SSH Keys for SFTP. You can use this in combination with a GUID or secret for two-factor authentication.	2019.2

Parameter	Type	Required / Optional	Description	Since
options.secret	string	required if the passwordGuid or keyId parameter is not specified	The script ID of the secret used for authentication. You can store secrets at Setup > Company > API Secrets. For more information, see the help topic Secrets Management. You can use the secret parameter in combination with the key parameter for two-factor authentication. You cannot use the secret parameter in combination with the passwordGuid parameter.	2021.1
options.hostKey	string	required	The host key for the trusted fingerprint on the server.	2016.2
options.username	string	required	The username of the remote account. The script ID of a secret can be accepted for this parameter. You can store secrets at Setup > Company > API Secrets. For more information, see the help topic Secrets Management. By default, the login is anonymous.	2016.2
options.port	number	optional	The port used to connect to the remote account. By default, port 22 is used.	2016.2
options.directory	string	optional	The remote directory of the connection.  Note: The directory parameter is required if you use a remote server that cannot resolve relative paths.	2016.2
options.timeout	number	optional	The number of seconds to allow for an established connection. This value should be between 1 and 20 seconds. By default, this value is set to 20 seconds.	2016.2
options.hostKeyType	string	optional	The type of host key specified by the options.hostKey parameter. This value can be set to one of the following options: ■ dsa ■ ecdsa ■ rsa	2016.2

Errors

Error Code	Thrown If
FTP_UNKNOWN_HOST	The host could not be found.

Error Code	Thrown If
FTP_CONNECT_TIMEOUT_EXCEEDED	A connection could not be established within the amount of time specified in the options.timeout parameter.
FTP_CANNOT_ESTABLISH_CONNECTION	The password/username was invalid or permission to access the directory was denied.
FTP_INVALID_PORT_NUMBER	The port number is invalid.
FTP_INVALID_CONNECTION_TIMEOUT	The options.timeout value is either a negative value, zero, or greater than 20 seconds.
FTP_INVALID_DIRECTORY	The directory does not exist on the remote FTP server.
FTP_INCORRECT_HOST_KEY	The options.hostKey parameter does not match the presented host key on the remote FTP server.
FTP_INCORRECT_HOST_KEY_TYPE	The options.hostKeyType parameter and provided host key type do not match.
FTP_MALFORMED_HOST_KEY	The options.hostKey parameter is not in the correct format. (for example, base 64, 96+ bytes)
FTP_PERMISSION_DENIED	Access to the file or directory on the remote FTP server was denied.
FTP_UNSUPPORTED_ENCRYPTION_ALGORITHM	The remote FTP server does not support one of NetSuite's approved algorithms. (for example, aes256-ctr, es192-ctr, es128-ctr)
AUTHENTICATION_FAIL_TOO_MANY_INCORRECT_AUTHENTICATION_ATTEMPTS	There are too many incorrect authentication attempts.
NO_ROUTE_TO_HOST_FOUND	No route to the host can be found.
CONNECTION_RESET	The connect was reset.
CONNECTION_CLOSED_BY_HOST	The connection was closed by the host.
THE_REMOTE_PATH_FOR_FILE_IS_NOT_VALID	The file's remote path is invalid.
SFTPCREDENTIAL_ENCODING_ERROR	There is an SFTP credential encoding error.
UNABLE_TO_GET_SFTP_SERVER_ADDRESS	The SFTP server address is unavailable.
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both secret and passwordGuid are defined. The passwordGuid can be used in combination with a key but not with a secret.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 var objConnection = sftp.createConnection({ // establish connection to the FTP server
5     username: 'username',
6     keyId: 'custkey1',
7     secret: 'custsecret1',
8     url: 'host.somewhere.com',
9     directory: 'username/wheres/my/file',
10    hostKey: myHostKey // references var myHostKey
11 });
12 ...

```

13 | //Add additional code

sftp.MAX_CONNECT_TIMEOUT

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the maximum connection timeout.
Module	N/ftp Module
Sibling Module Members	N/ftp Module Members
Since	2016.2

Values

Value
20

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 if (connectTimeout < sftp.MIN_CONNECT_TIMEOUT) connectTimeout = sftp.MIN_CONNECT_TIMEOUT; else if (connectTimeout > sftp.MAX_CONNEC
T_TIMEOUT)
5     connectTimeout = sftp.MAX_CONNECT_TIMEOUT;
6 ...
7 //Add additional code

```

sftp.MIN_CONNECT_TIMEOUT

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the minimum connection timeout.
Module	N/ftp Module
Sibling Module Members	N/ftp Module Members

Since	2016.2
-------	--------

Values

Value
1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 if (connectTimeout <sftp.MIN_CONNECT_TIMEOUT) connectTimeout = sftp.MIN_CONNECT_TIMEOUT; else if (connectTimeout > sftp.MAX_CONNEC
T_TIMEOUT)
5     connectTimeout = sftp.MAX_CONNECT_TIMEOUT;
6 ...
7 //Add additional code

```

sftp.MAX_PORT_NUMBER

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the maximum port number.
Module	N/ftp Module
Sibling Module Members	N/ftp Module Members
Since	2019.2

Values

Value
65535

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ftp Module Script Samples](#).

```

1 //Add additional code
2 ...

```

```

3
4 if (portNumber <sftp.MIN_PORT_NUMBER || portNumber > sftp.MAX_PORT_NUMBER)
5   portNumber = sftp.DEFAULT_PORT_NUMBER;
6 ...
7 //Add additional code

```

sftp.MIN_PORT_NUMBER

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for the minimum port number.
Module	N/sftp Module
Sibling Module Members	N/sftp Module Members
Since	2019.2

Values

Value
0

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 if (portNumber <sftp.MIN_PORT_NUMBER || portNumber > sftp.MAX_PORT_NUMBER)
5   portNumber = sftp.DEFAULT_PORT_NUMBER;
6 ...
7 //Add additional code

```

sftp.DEFAULT_PORT_NUMBER

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for default port number.
Module	N/sftp Module

Sibling Module Members	N/sftp Module Members
Since	2019.2

Values

Value
22

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 if (portNumber <sftp.MIN_PORT_NUMBER || portNumber > sftp.MAX_PORT_NUMBER)
5     portNumber = sftp.DEFAULT_PORT_NUMBER;
6 ...
7 //Add additional code

```

sftp.Sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values to be used to sort the listed directory.
Module	N/sftp Module
Sibling Module Members	N/sftp Module Members
Since	2019.2

Values

Value
DATE
DATE_DESC
SIZE
SIZE_DESC
NAME
NAME_DESC

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/sftp Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 var connection = sftp.createConnection({
5     username: 'sftpuser',
6     keyId: 'custkey1',
7     url: '192.168.0.100',
8     port: 22,
9     directory: 'inbound',
10    hostKey: "AAAAB3NzaC1yc2EAAAADAQABAAQDMi fKH2vTxditye8nem7+1S3x7dTQR/A67KdsR/5C2WUcDipBzYhHb
nG6Am12Nd2t1M01LnaBZA6/8P4Y9x/sGTxtsdE/MzeGDUBn6HB1QvgIrhX62wgoKGQ+P21EA01+Vz8y3/MB1NmD7Fc62cJ9Mu88YA6jwJOIPZeHYNNVyIm90rY6VyzYyvSJh
H0x7SYyvGnijJQF4G8C4c8u/UVpF/sE16xKZt1y2Rx0aDL2FsDRtpyPmM602/R6ISbsmgab3MzzAEIu+zLDMdIBJn3cDhNt1F7Rar6Tu0u18KCKk8GPxbnxDuG4sCN0nX
PYkDXSMubM/ocRjYGtqdZUMmeTf3"
11 });
12
13 connection.list({
14     path: "?path*",
15     sort: sftp.Sort.SIZE
16 });
17 ...
18 //Add additional code

```

N/sso Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

As of NetSuite 2025.1, the support ended for the SuiteSignIn feature, which means the N/sso module is also no longer supported. You should edit your scripts to remove the use of the N/sso module. If scripts execute that include the N/sso module, an error will occur and the script may not complete successfully.

N/suiteAppInfo Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/suiteAppInfo module to access information related to SuiteApps and Bundles. This module is available for all script types.

[Go To Samples](#)

In This Help Topic

- [N/suiteAppInfo Module Members](#)

N/suiteAppInfo Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	suiteAppInfo.isBundleInstalled (options)	boolean	Client and server scripts	Returns true if the specific bundle is installed.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	suiteAppInfo.isSuiteAppInstalled(options)	boolean	Client and server scripts	Returns true if the specified SDF SuiteApp is installed.
	suiteAppInfo.listBundlesContainingScripts(options)	Object	Client and server scripts	Returns the IDs for bundles that contain the specified script, for each script specified.
	suiteAppInfo.listInstalledBundles()	Object[]	Client and server scripts	Returns a list of bundles that are installed.
	suiteAppInfo.listInstalledSuiteApps()	Object[]	Client and server scripts	Returns a list of SDF SuiteApps that are installed.
	suiteAppInfo.listSuiteAppsContainingScripts(options)	Object	Client and server scripts	Returns the ID for the SDF SuiteApp that contains the specified script, for each script specified.

N/suiteAppInfo Module Script Sample

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/suiteAppInfo module.

Retrieve Information for a SuiteApp

The following script sample performs these tasks:

- Checks to see if a specific bundle is installed
- Checks to see if a specific SDF SuiteApp is installed
- Retrieves a list of successfully installed bundles
- Retrieves a list of successfully installed SDF SuiteApps
- Retrieves a list of bundles in the current account that include a specific script
- Retrieves a list of SDF SuiteApps in the current account that include a specific script

Some of the values in this sample are placeholders, such as the `bundleId` and `suiteAppId` values. Before using this sample, replace all hard-coded values, including IDs, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /*
2   * @NApiVersion 2.x
3   *
4   */
5
6  // This script sample uses each method available in the N/suiteAppInfo module to retrieve information about installed bundles and SuiteApps installed in
   the current account.
7
8  require(['N/suiteAppInfo'], function (suiteAppInfo){
9      var isBundleInstalled = suiteAppInfo.isBundleInstalled({
10         bundleId: '1234'
11     });
12

```

```

13  var isSuiteAppInstalled = suiteAppInfo.isSuiteAppInstalled({
14      suiteAppId: '789'
15  });
16
17  var allMyBundlesInstalled = suiteAppInfo.listInstalledBundles();
18
19  var allMySuiteAppsInstalled = suiteAppInfo.listInstalledSuiteApps();
20
21  var myScripts = {"scriptA", "scriptB", "scriptC"};
22  var scriptInBundles = suiteAppInfo.listBundlesContainingScripts({
23      scriptIds: myScripts
24  });
25
26  var scriptInSuiteApps = suiteAppInfo.listSuiteAppsContainingScripts({
27      scriptIds: myScripts
28  });
29  })

```

suiteAppInfo.isBundleInstalled(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to determine if a specified bundle is installed.
Returns	Returns true if the specified bundle is installed. Returns false if the specified bundle is not installed.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/suiteAppInfo Module
Sibling Module Members	N/suiteAppInfo Module Members
Since	2021.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.bundleId	integer	required	The id of the bundle.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/suiteAppInfo Module Script Sample](#).

```

1  // Add additional code
2  ...
3  var isBundleInstalled = suiteAppInfo.isBundleInstalled({
4      bundleId: 123
5  });
6  ...
7  // Add additional code

```


suiteAppInfo.isSuiteAppInstalled(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to determine if a specified SDF SuiteApp is installed.
Returns	Returns true if the specified SuiteApp is installed. Returns false if the specified SuiteApp is not installed.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Module	N/suiteAppInfo Module
Sibling Module Members	N/suiteAppInfo Module Members
Since	2021.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.suiteAppId	string	required	The id of the SuiteApp.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/suiteAppInfo Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var isSuiteAppInstalled = suiteAppInfo.isSuiteAppInstalled({
4   suiteAppId: 123
5 });
6 ...
7 // Add additional code

```

suiteAppInfo.listBundlesContainingScripts(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to retrieve the IDs for bundles that contain the specified script, for each script specified.
Returns	Object.<String, [int]>, which is a {scriptId: arrayOfBundleIds} mapping. See Syntax for an example.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	10 units
Module	N/suiteAppInfo Module
Sibling Module Members	N/suiteAppInfo Module Members
Since	2021.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptsIds	string[]	required	The list of script ids.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/suiteAppInfo Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var myScriptIds = {"scriptA", "scriptB", "scriptC"};
4 var bundlesWithScripts = suiteAppInfo.listBundlesContainingScripts({
5     scriptIds: myScriptIds
6 });
7 // For example, bundlesWithScripts would be:
8 // {
9 //     "scriptA": [], // there is no bundle that contains this script
10 //     "scriptB": [12], // bundle with bundle ID 12 contains this script
11 //     "scriptC": [12, 123] // bundles with bundle ID 12 and bundle ID 123 contain this script
12 // }
13 ...
14 // Add additional code

```

suiteAppInfo.listInstalledBundles()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to retrieve a list of successfully installed bundles, as an array of objects.
Returns	Object[] Each object includes the following properties: ■ id (integer) ■ name (string) ■ version (string) ■ description (string) ■ installedFrom (string) ■ isManaged (boolean) ■ dateInstalled (Date) ■ dateLastUpdated (Date)

	■ publisher (Publisher object, which includes an integer id and a string name) ■ installedBy (Installer object, which includes an integer id and a string name)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/suiteAppInfo Module
Sibling Module Members	N/suiteAppInfo Module Members
Since	2021.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/suiteAppInfo Module Script Sample](#).

```

1 // Add additional code
2 ...
3 var installedBundles = suiteAppInfo.listInstalledBundles();
4 ...
5 // Add additional code

```

suiteAppInfo.listInstalledSuiteApps()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to retrieve a list of successfully installed SDF SuiteApps, as an array of objects.
Returns	Object[] Each object includes the following properties: ■ appId (string) ■ publisherId (string) ■ name (string) ■ version (string) ■ description (string) ■ dateInstalled (Date) ■ dateLastUpdated (Date) ■ installedBy (Installer object, which includes an integer id and a string name)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/suiteAppInfo Module
Sibling Module Members	N/suiteAppInfo Module Members
Since	2021.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/suiteAppInfo Module Script Sample](#).

```
1 // Add additional code
2 ...
3 var installedSuiteApps = suiteAppInfo.listInstalledSuiteApps();
4 ...
5 // Add additional code
```

suiteAppInfo.listSuiteAppsContainingScripts(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Use this method to retrieve the ID for the SDF SuiteApp that contains the specified script, for each script specified. Only one ID will be returned for each specified script.
Returns	Object.<String, null String>, which is a {scriptId: suiteAppId null} mapping. See Syntax for an example.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/suiteAppInfo Module
Sibling Module Members	N/suiteAppInfo Module Members
Since	2021.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptsIds	string[]	required	The list of script ids.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/suiteAppInfo Module Script Sample](#).

```
1 // Add additional code
2 ...
3 var myScripts = {"scriptA", "scriptB", "scriptC"};
4 var suiteAppsWithScripts = suiteAppInfo.listSuiteAppsContainingScripts({
5   scriptIds: myScripts
6 });
7 // For example, suiteAppsWithScripts would be:
8 // {
```

```

9 // "scriptA": null, // there is no SuiteApp that contains this script
10 // "scriptB": "SuiteAppX", // SuiteAppX contains this script
11 // "scriptC": "SuiteAppY" // SuiteAppY contains this script
12 // }
13
14 ...
15 // Add additional code

```

N/task Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/task module to create tasks and place them in the internal NetSuite scheduling or task queue. You can use this module to create tasks for the following:

- To submit a scheduled script
- To run a map/reduce script
- To import CSV files
- To merge duplicate records
- To execute asynchronous searches, asynchronous document capture tasks, constructed queries, SuiteQL queries, and workflows

Each task is a specific task type ([task.TaskType](#)) and each task type has its own corresponding object types. Use the methods available to each object type to configure, submit, and monitor the tasks.

Go To Samples 

i Note: Regardless of task type, tasks are always triggered asynchronously.

In This Help Topic

- [N/task Module Members](#)
- [CsvImportTask Object Members](#)
- [CsvImportTaskStatus Object Members](#)
- [DocumentCaptureTask Object Members](#)
- [DocumentCaptureTaskStatus Object Members](#)
- [EntityDeduplicationTask Object Members](#)
- [EntityDeduplicationTaskStatus Object Members](#)
- [MapReduceScriptTask Object Members](#)
- [MapReduceScriptTaskStatus Object Members](#)
- [QueryTask Object Members](#)
- [QueryTaskStatus Object Members](#)
- [RecordActionTask Object Members](#)
- [RecordActionTaskStatus Object Members](#)
- [ScheduledScriptTask Object Members](#)
- [ScheduledScriptTaskStatus Object Members](#)
- [SearchTask Object Members](#)
- [SearchTaskStatus Object Members](#)
- [SuiteQLTask Object Members](#)

- [SuiteQLTaskStatus Object Members](#)
- [WorkflowTriggerTask Object Members](#)
- [WorkflowTriggerTaskStatus Object Members](#)

N/task Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	task.CsvImportTask	Object	Server scripts	The properties of a CSV import task. Use the methods and properties for this object to submit a CSV import task into the task queue and asynchronously import record data into NetSuite.
	task.CsvImportTaskStatus	Object	Server scripts	The status of a CSV import task placed into the NetSuite scheduling queue.
	task.DocumentCaptureTask	Object	Server scripts	The properties of a document capture task. Use the methods and properties of this object to submit a document capture task into the NetSuite task queue.
	task.DocumentCaptureTaskStatus	Object	Server scripts	The status of a document capture task placed into the NetSuite task queue.
	task.EntityDeduplicationTask	Object	Server scripts	All the properties of a merge duplicate records task request. Use the methods and properties of this object to submit a merge duplicate record job task into the NetSuite task queue.
	task.EntityDeduplicationTaskStatus	Object	Server scripts	The status of a merge duplicate record task placed into the NetSuite task queue.
	task.MapReduceScriptTask	Object	Server scripts	A map/reduce script deployment.
	task.MapReduceScriptTaskStatus	Object	Server scripts	The status of a map/reduce script deployment that has been submitted for processing.
	task.QueryTask	Object	Server scripts	The properties of a query task. Use the methods and properties of this object to submit a query task into the NetSuite task queue.
	task.QueryTaskStatus	Object	Server scripts	The status of a query task placed into the NetSuite task queue.
	task.RecordActionTask	Object	Server scripts	The properties of a record action task. Use this object to place a record action task into the NetSuite scheduling queue.
	task.RecordActionTaskStatus	Object	Server scripts	The status of a record action task in the NetSuite scheduling queue.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	task.ScheduledScriptTask	Object	Server scripts	All the properties of a scheduled script task in SuiteScript. Use this object to place a scheduled script deployment into the NetSuite scheduling queue.
	task.ScheduledScriptTaskStatus	Object	Server scripts	The status of a scheduled script placed into the NetSuite scheduling queue.
	task.SearchTask	Object	Server scripts	The properties required to initiate an asynchronous search.
	task.SearchTaskStatus	Object	Server scripts	The status of an asynchronous search initiation task that is placed into the NetSuite task queue.
	task.SuiteQLTask	Object	Server scripts	The properties of a SuiteQL task. Use the methods and properties of this object to submit a query task into the NetSuite task queue.
	task.SuiteQLTaskStatus	Object	Server scripts	The status of a SuiteQL task placed into the NetSuite task queue.
	task.WorkflowTriggerTask	Object	Server scripts	All the properties required to asynchronously initiate a workflow. Use WorkflowTriggerTask to create a task that initiates an instance of a specific workflow.
	task.WorkflowTriggerTaskStatus	Object	Server scripts	The status of an asynchronous workflow initiation task placed into the NetSuite task queue.
Method	task.checkStatus(options)	task.CsvImportTaskStatus task.DocumentCaptureTaskStatus task.EntityDeduplicationTaskStatus task.MapReduceScriptTaskStatus task.QueryTaskStatus task.RecordActionTaskStatus task.ScheduledScriptTaskStatus task.SearchTaskStatus task.SuiteQLTaskStatus task.WorkflowTriggerTaskStatus	Server scripts	Returns a task status object associated with a specific task ID.
	task.create(options)	task.CsvImportTask task.DocumentCaptureTask task.EntityDeduplicationTask task.MapReduceScriptTask task.QueryTask task.RecordActionTask task.ScheduledScriptTask task.SearchTask task.SuiteQLTask task.WorkflowTriggerTask	Server scripts	Creates an object for a specific task type and returns the task object.
Enum	task.ActionCondition	enum	Server scripts	Holds the string values for the possible record action conditions. This enum is returned by RecordActionTask.condition .
	task.DedupeEntityType	enum	Server scripts	Holds the string values for entity types for which you can merge duplicate records with task.EntityDeduplicationTask .

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	task.DedupeMode	enum	Server scripts	Holds the string values for available deduplication modes when merging duplicate records with task.EntityDeduplicationTask. Use this enum to set the EntityDeduplicationTask.entityType.
	task.MapReduceStage	enum	Server scripts	Holds the string values for the stages of a map/reduce script deployment, which is encapsulated by the task.MapReduceScriptTask object. This enum is returned by MapReduceScriptTaskStatus.stage.
	task.MasterSelectionMode	enum	Server scripts	Holds the string values for supported master selection modes when merging duplicate records with task.EntityDeduplicationTask. Use this enum to set the EntityDeduplicationTask.masterSelectionMode property.
	task.TaskStatus	enum	Server scripts	Holds the string values for the possible status of tasks created and submitted with the N/task Module.
	task.TaskType	enum	Server scripts	Holds the string values for the types of task objects you can create using task.create(options). Use this enum to set the value for the options.taskType parameter of the task.create(options) method.

CsvImportTask Object Members

The following members are available for a [task.CsvImportTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	CsvImportTask.submit()	string	Server scripts	Directs NetSuite to place a CSV import task into the NetSuite task queue and returns a unique ID for the task. You can use this method only in bundle installation scripts, scheduled scripts, and RESTlets.
Property	CsvImportTask.id	string	Server scripts	The ID of the task.
	CsvImportTask.importFile	file.File string	Server scripts	CSV file to import. Use a file.File object or a string that represents the CSV text to be imported.
	CsvImportTask.linkedFiles	Object	Server scripts	A map of key-value pairs that sets the data to be imported in a linked file for a multi-file import job, by referencing

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				a file in the File Cabinet or the raw CSV data to import.
	CsvImportTask.mappingId	number string	Server scripts	Script ID or internal ID of the saved import map that you created when you ran the Import Assistant.
	CsvImportTask.name	string	Server scripts	Name for the CSV import task.
	CsvImportTask.queueId	number	Server scripts	Overrides the Queue Number property under Advanced Options on the Import Options page of the Import Assistant.

CsvImportTaskStatus Object Members

The following members are available for a [task.CsvImportTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	CsvImportTaskStatus.status	string (read-only)	Server scripts	Status for a CSV import task. Returns a task.TaskStatus enum value.
	CsvImportTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

DocumentCaptureTask Object Members

The following members are available for a [task.DocumentCaptureTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	DocumentCaptureTask.addInboundDependency(options)	void	Server scripts	Adds a scheduled script task to the document capture task as a dependent task.
	DocumentCaptureTask.submit()	string	Server scripts	Submits the document capture task for asynchronous processing and returns the task ID.
Property	DocumentCaptureTask.documentType	string	Server scripts	The document type.
	DocumentCaptureTask.features	string[]	Server scripts	The features to extract from the document (such as fields, tables, or text).
	DocumentCaptureTask.id	string	Server scripts	The ID of the task.
	DocumentCaptureTask.inboundDependencies	Object[]	Server scripts	Key-value pairs that contain information about the dependent tasks added to the document capture task.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	DocumentCaptureTask.inputFile	file.File	Server scripts	The document to extract content from.
	DocumentCaptureTask.language	string	Server scripts	The language of the document.
	DocumentCaptureTask.outputFilePath	string	Server scripts	The path of the JSON file to export document capture results to.
	DocumentCaptureTask.ociConfig	Object	Server scripts	Oracle Cloud Infrastructure (OCI) credentials when using unlimited usage mode.

DocumentCaptureTaskStatus Object Members

The following members are available for a [task.DocumentCaptureTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DocumentCaptureTaskStatus.status	string (read-only)	Server scripts	The status for a document capture task (as a task.TaskStatus enum value).
	DocumentCaptureTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

EntityDeduplicationTask Object Members

The following members are available for a [task.EntityDeduplicationTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	EntityDeduplicationTask.submit()	string	Server scripts	Directs NetSuite to place the merge duplicate records task into the NetSuite task queue and returns a unique ID for the task.
Property	EntityDeduplicationTask.dedupeMode	string	Server scripts	The mode in which to merge or delete duplicate records. Use values from the task.DedupeMode enum.
	EntityDeduplicationTask.entityType	string	Server scripts	The type of entity on which you want to merge duplicate records. Use a task.DedupeEntityType enum to set the value.
	EntityDeduplicationTask.id	string	Server scripts	The ID of the task.
	EntityDeduplicationTask.masterRecordId	number	Server scripts	Master record ID. When you merge duplicate records, you can delete all duplicates for a record or merge information from the duplicate records into the master record.
	EntityDeduplicationTask.masterSelectionMode	string	Server scripts	Master selection mode. Use values from the task.MasterSelectionMode enum.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	EntityDeduplicationTask.recordIds	number[]	Server scripts	Number array of record internal IDs to perform the merge or delete operation on.

EntityDeduplicationTaskStatus Object Members

The following members are available for a [task.EntityDeduplicationTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	EntityDeduplicationTaskStatus.status	string (read-only)	Server scripts	Status for a merge duplicate record task. Returns a task.TaskStatus enum value.
	EntityDeduplicationTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

MapReduceScriptTask Object Members

The following members are available for a [task.MapReduceScriptTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	MapReduceScriptTask.submit()	string	Server scripts	Submits a map/reduce script deployment for processing.
Property	MapReduceScriptTask.deploymentId	string	Server scripts	Script ID (as a string), for the script deployment record for a map/reduce script.
	MapReduceScriptTask.id	string	Server scripts	The ID of the task.
	MapReduceScriptTask.params	Object	Server scripts	Object that represents key-value pairs that override static script parameter field values on the script deployment record.
	MapReduceScriptTask.scriptId	number string	Server scripts	Internal ID (as a number), or script ID (as a string), for the map/reduce script record.

MapReduceScriptTaskStatus Object Members

The following members are available for a [task.MapReduceScriptTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	MapReduceScriptTaskStatus.getCurrentTotalSize()	number	Server scripts	Returns the total size in bytes of all stored work in progress by a task.MapReduceScriptTask .

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	MapReduceScriptTaskStatus.getPendingMapCount()	number	Server scripts	Returns the total number of records or rows not yet processed by the map stage of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getPendingMapSize()	number	Server scripts	Returns the total number of bytes not yet processed by the map stage, as a component of total size, of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getPendingOutputCount()	number	Server scripts	Returns the total number of records or rows not yet processed by a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getPendingOutputSize()	number	Server scripts	Returns the total size in bytes of all key-value pairs written as output, as a component of total size, by a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getPendingReduceCount()	number	Server scripts	Returns the total number of records or rows not yet processed by the reduce stage of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getPendingReduceSize()	number	Server scripts	Returns the total number of bytes not yet processed by the reduce stage, as a component of total size, of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getPercentageCompleted()	number	Server scripts	Returns the current percentage complete for the current stage of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getTotalMapCount()	number	Server scripts	Returns the total number of records or rows passed as input to the map stage of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getTotalOutputCount()	number	Server scripts	Returns the total number of records or rows passed as inputs to the output phase of a task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.getTotalReduceCount()	number	Server scripts	Returns the total number of record or row inputs to the reduce stage of a task.MapReduceScriptTask .
Property	MapReduceScriptTaskStatus.deploymentId	string (read-only)	Server scripts	Script ID for a script deployment record associated with a specific task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.scriptId	number (read-only)	Server scripts	Internal ID for a map/reduce script record associated with a specific task.MapReduceScriptTask .
	MapReduceScriptTaskStatus.stage	string (read-only)	Server scripts	The current stage of a map/reduce script deployment that is being processed. See task.MapReduceStage for supported values.
	MapReduceScriptTaskStatus.status	string (read-only)	Server scripts	Status for a map/reduce script task. Returns a task.TaskStatus enum value.
	MapReduceScriptTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

QueryTask Object Members

The following members are available for a [task.QueryTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	QueryTask.addInboundDependency(options)	void	Server scripts	Adds a scheduled script task or map/reduce script task to the query task as a dependent task.
	QueryTask.submit()	string	Server scripts	Submits the query task for asynchronous processing and returns the task ID.
Property	QueryTask.fileId	number	Server scripts	Internal ID of the CSV file to export query results to. This property is mutually exclusive with the QueryTask.filePath parameter.
	QueryTask.filePath	string	Server scripts	Path of the CSV file to export query results to. This property is mutually exclusive with the QueryTask.fileId property.
	QueryTask.id	string	Server scripts	The ID of the task.
	QueryTask.inboundDependencies	Object[]	Server scripts	Key-value pairs that contain information about the dependent tasks added to the query task.
	QueryTask.query	string	Server scripts	Query definition for the query task.

QueryTaskStatus Object Members

The following members are available for a [task.QueryTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	QueryTaskStatus.fileId	number (read-only)	Server scripts	Internal ID of the CSV file that query results are exported to.
	QueryTaskStatus.query	query.Query (read-only)	Server scripts	Query definition for the submitted query task.
	QueryTaskStatus.status	string (read-only)	Server scripts	Status of the submitted query task.
	QueryTaskStatus.taskId	string (read-only)	Server scripts	ID of the submitted query task.

RecordActionTask Object Members

The following members are available for a [task.RecordActionTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	RecordActionTask.submit()	string	Server scripts	Submits a record action task for processing and returns its task ID.
Property	RecordActionTask.action	string	Server scripts	The ID of the action to be invoked.
	RecordActionTask.condition	Object	Server scripts	The condition used to select record IDs of records for which the action is to be executed. Only the action.ALL_QUALIFIED_INSTANCES constant is currently supported.
	RecordActionTask.id	string	Server scripts	The ID of the task.
	RecordActionTask.paramCallback	Object	Server scripts	Function that takes record ID and returns the parameter object for the specified record ID.
	RecordActionTask.params	Object[]	Server scripts	An array of parameter objects. Each object corresponds to one record ID of the record for which the action is to be executed. The object has the following form: {recordId: 1, someParam: 'example1', otherParam: 'example2'}
	RecordActionTask.recordType	string	Server scripts	The record type on which the action is to be performed. For a list of record types, see record.Type .

RecordActionTaskStatus Object Members

The following members are available for a [task.RecordActionTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	RecordActionTaskStatus.complete	number (read-only)	Server scripts	The number of record action tasks with a completed status.
	RecordActionTaskStatus.errors	Object (read-only)	Server scripts	The error details of failed action executions. The value of the property is the record instance ID and the corresponding error details. The error details are returned in an unnamed object with two properties: code and message.
	RecordActionTaskStatus.failed	number (read-only)	Server scripts	The number of record action tasks with a failed status.
	RecordActionTaskStatus.pending	number (read-only)	Server scripts	The number of record action tasks with a pending status.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	RecordActionTaskStatus.results	Object (read-only)	Server scripts	The results of successfully executed record action tasks. The value of the property is the task instance ID and the corresponding action result.
	RecordActionTaskStatus.status	string (read-only)	Server scripts	Represents the record action task status. Returns a value from the task.TaskStatus enum.
	RecordActionTaskStatus.succeeded	number (read-only)	Server scripts	The number of record action tasks with a succeeded status.
	RecordActionTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

ScheduledScriptTask Object Members

The following members are available for a [task.ScheduledScriptTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ScheduledScriptTask.submit()	string	Server scripts	Directs NetSuite to place a scheduled script deployment into the NetSuite scheduling queue and returns a unique ID for the task.
Property	ScheduledScriptTask.deploymentId	string	Server scripts	Script ID (as a string), for the script deployment record associated with a task.ScheduledScriptTask object.
	ScheduledScriptTask.id	string	Server scripts	The ID of the task.
	ScheduledScriptTask.params	Object	Server scripts	Object with key-value pairs that override the static script parameter field values on the script deployment.
	ScheduledScriptTask.scriptId	number string	Server scripts	Internal ID (as a number), or script ID (as a string) for the script record associated with a task.ScheduledScriptTask object.

ScheduledScriptTaskStatus Object Members

The following members are available for a [task.ScheduledScriptTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ScheduledScriptTaskStatus.deploymentId	string (read-only)	Server scripts	Script ID for a script deployment record associated with a specific task.ScheduledScriptTask object.
	ScheduledScriptTaskStatus.scriptId	number (read-only)	Server scripts	Internal ID for a script record associated with a specific task.ScheduledScriptTask object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	ScheduledScriptTaskStatus.status	string (read-only)	Server scripts	Status for a scheduled script task. Returns a task.TaskStatus enum value.
	ScheduledScriptTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

SearchTask Object Members

The following members are available for a [task.SearchTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	SearchTask.addInboundDependency()	void	Server scripts	Adds a scheduled script task or map/reduce script task to the search task as a dependent script. Dependent scripts are processed automatically when the search task is complete. For more information, see the help topic SuiteCloud Processors .
	SearchTask.submit()	string	Server scripts	Places the asynchronous search initiation task into the SuiteScript task queue, and returns a unique ID for the task.
Property	SearchTask.fileId	number	Server scripts	ID of the CSV file to export search results into.
	SearchTask.filePath	string	Server scripts	Path of the CSV file to export search results into.
	SearchTask.id	string	Server scripts	The ID of the task.
	SearchTask.inboundDependencies	Object[] (read-only)	Server scripts	Key-value pairs to describe the dependent scripts added to the search task.
	SearchTask.savedSearchId	number	Server scripts	ID of the saved search to be executed during the task.

SearchTaskStatus Object Members

The following members are available for a [task.SearchTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SearchTaskStatus.fileId	number (read-only)	Server scripts	ID of the CSV file into which search results are exported.
	SearchTaskStatus.savedSearchId	number (read-only)	Server scripts	ID of the saved search executed during the task.
	SearchTaskStatus.status	string (read-only)	Server scripts	Status of an asynchronous search task placed in the NetSuite

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				task queue. Returns one of the task.TaskStatus enum values.
	SearchTaskStatus.taskId	string (read-only)	Server scripts	ID of the asynchronous task.

SuiteQLTask Object Members

The following members are available for a [task.SuiteQLTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	SuiteQLTask.addInboundDependency(options)	void	Server scripts	Adds a scheduled script task or map/reduce script task to the SuiteQL task as a dependent task.
	SuiteQLTask.submit()	string	Server scripts	Submits the SuiteQL task for asynchronous processing and returns the task ID.
Property	SuiteQLTask.fileId	number	Server scripts	Internal ID of the CSV file to export SuiteQL query results to. This property is mutually exclusive with the SuiteQLTask.filePath parameter.
	SuiteQLTask.filePath	string	Server scripts	Path of the CSV file to export SuiteQL query results to. This property is mutually exclusive with the SuiteQLTask.fileId property.
	SuiteQLTask.id	string	Server scripts	The ID of the task.
	SuiteQLTask.inboundDependencies	Object[]	Server scripts	Key-value pairs that contain information about the dependent tasks added to the SuiteQL task.
	SuiteQLTask.params	Array<string boolean number>	Server scripts	Parameters for the SuiteQL query.
	SuiteQLTask.query	string	Server scripts	SuiteQL query definition for the SuiteQL task.

SuiteQLTaskStatus Object Members

The following members are available for a [task.SuiteQLTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SuiteQLTaskStatus.fileId	number (read-only)	Server scripts	Internal ID of the CSV file that SuiteQL query results are exported to.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	SuiteQLTaskStatus.params	Array<string boolean number> (read-only)	Server scripts	Parameters for the SuiteQL query.
	SuiteQLTaskStatus.query	string (read-only)	Server scripts	SuiteQL query definition for the SuiteQL task.
	SuiteQLTaskStatus.status	string (read-only)	Server scripts	Status of the SuiteQL task.
	SuiteQLTaskStatus.taskId	string (read-only)	Server scripts	ID of the submitted SuiteQL task.

WorkflowTriggerTask Object Members

The following members are available for a [task.WorkflowTriggerTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	WorkflowTriggerTask.submit()	string	Server scripts	Directs NetSuite to place the asynchronous workflow initiation task into the NetSuite scheduling queue and returns a unique ID for the task.
Property	WorkflowTriggerTask.id	string	Server scripts	The ID of the task.
	WorkflowTriggerTask.params	Object	Server scripts	Object that contains key-value pairs to set default values on fields specific to the workflow.
	WorkflowTriggerTask.recordId	number	Server scripts	Internal ID of the workflow definition base record. For example, 55 or 124.
	WorkflowTriggerTask.recordType	string	Server scripts	Record type of the workflow base record. For example, customer, salesorder, or lead.
	WorkflowTriggerTask.workflowId	number string	Server scripts	Internal ID (as a number), or script ID (as a string), for the workflow definition.

WorkflowTriggerTaskStatus Object Members

The following members are available for a [task.WorkflowTriggerTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	WorkflowTriggerTaskStatus.status	string (read-only)	Server scripts	Status for a asynchronous workflow placed in the NetSuite task queue. Returns a value from the task.TaskStatus enum.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	WorkflowTriggerTask Status.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

N/task Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/task module:

- [Create and Submit a Map/Reduce Script Task](#)
- [Create and Submit an Asynchronous Search Task and Export the Results into a CSV File](#)
- [Create and Submit a Task with Dependent Scripts](#)
- [Submit a Record Action Task and Check Status](#)

Create and Submit a Map/Reduce Script Task

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample submits a map/reduce script task for processing. Before you use this sample, you must create a map/reduce script file, upload the file to your NetSuite account, and create a script record and script deployment record for it. For help working with map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#). You must also edit the sample and replace all hard-coded IDs with values that are valid in your NetSuite account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4  require(['N/task', 'N/runtime', 'N/email'], (task, runtime, email) => {
5      // Store the script ID of the script to submit
6      //
7      // Update the following statement so it uses the script ID
8      // of the map/reduce script record you want to submit
9      const mapReduceScriptId = 'customscript_test_mapreduce_script';
10
11     // Create a map/reduce task
12     //
13     // Update the deploymentId parameter to use the script ID of
14     // the deployment record for your map/reduce script
15     let mrTask = task.create({
16         taskType: task.TaskType.MAP_REDUCE,
17         scriptId: mapReduceScriptId,
18         deploymentId: 'customdeploy_test_mapreduce_script'

```

```

19     });
20
21     // Submit the map/reduce task
22     let mrTaskId = mrTask.submit();
23
24     // Check the status of the task, and send an email if the
25     // task has a status of FAILED
26     //
27     // Update the authorId value with the internal ID of the user
28     // who is the email sender. Update the recipientEmail value
29     // with the email address of the recipient.
30     let taskStatus = task.checkStatus(mrTaskId);
31     if (taskStatus.status === 'FAILED') {
32         const authorId = -5;
33         const recipientEmail = 'notify@myCompany.com';
34         email.send({
35             author: authorId,
36             recipients: recipientEmail,
37             subject: 'Failure executing map/reduce job!',
38             body: 'Map reduce task: ' + mapReduceScriptId + ' has failed.'
39         });
40     }
41 });

```

Create and Submit an Asynchronous Search Task and Export the Results into a CSV File

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates an asynchronous search task to execute a saved search and export the results of the search into a CSV file stored in the File Cabinet. After the search task is submitted, the sample retrieves the task status using the task ID. Some of the values in this sample are placeholders. Before using this sample, replace all hard-coded values, such as IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4   require(['N/task'], task => {
5       // Do one of the following:
6       //
7       // - Create a saved search and capture its ID. To do this, you can use
8       //   the following code snippet (replacing the type, id, filters, and
9       //   columns values as appropriate):
10      //
11      // let mySearch = search.create({
12      //     type: search.Type.SALES_ORDER,
13      //     id: 'customsearch_my_search',
14      //     filters: [...],
15      //     columns: [...]
16      // });
17      // mySearch.save();
18      // let savedSearchId = mySearch.searchId;

```

```

19 //
20 // - Use the ID of an existing saved search. This is the approach that
21 // this script sample uses. Update the following statement with the
22 // internal ID of the search you want to use.
23 const savedSearchId = 669;
24
25 // Create the search task
26 let myTask = task.create({
27     taskType: task.TaskType.SEARCH
28 });
29 myTask.savedSearchId = savedSearchId;
30
31 // Specify the ID of the file that search results will be exported into
32 //
33 // Update the following statement so it uses the internal ID of the file
34 // you want to use
35 myTask.fileId = 448;
36
37 // Submit the search task
38 let myTaskId = myTask.submit();
39
40 // Retrieve the status of the search task
41 let taskStatus = task.checkStatus({
42     taskId: myTaskId
43 });
44
45 // Optionally, add logic that executes when the task is complete
46 if (taskStatus.status === task.TaskStatus.COMPLETE) {
47     // Add any code that is appropriate. For example, if this script created
48     // a saved search, you may want to delete it.
49 }
50 });

```

Create and Submit a Task with Dependent Scripts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a map/reduce script task. It then creates an asynchronous search task and adds the map/reduce script task to the search task as dependent scripts. The script is processed when the search task is complete. For more information, see the help topic [SuiteCloud Processors](#).

This sample refers to two script parameters: `custscript_ss_as_srch_res` for the scheduled script, and `custscript_mr_as_srch_res` for the map/reduce script. These parameters are used to pass the location of the CSV file to the dependent scripts, which is shown in the second and third code samples below. Before using this sample, create these parameters in the script record. For more information, see the help topic [Creating Script Parameters](#).

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5   require(['N/task'], (task) => {
6       // Specify a file for the search results
7       const asyncSearchResultFile = 'SuiteScripts/ExportFile.csv';
8
9       // Create a map/reduce script task
10      const mapReduceScript = task.create({
11          taskType: task.TaskType.MAP_REDUCE

```

```

12     });
13     mapReduceScript.scriptId = 'customscript_mr_create_submit';
14     mapReduceScript.deploymentId = 'customdeploy_mr_create_submit';
15     mapReduceScript.params = {
16         'custscript_mr_as_srch_res' : asyncSearchResultFile
17     };
18
19     // Create the search task
20     const asyncTask = task.create({
21         taskType: task.TaskType.SEARCH
22     });
23     asyncTask.savedSearchId = 'customsearch35';
24     asyncTask.filePath = asyncSearchResultFile;
25
26     // Add dependent scripts to the search task before it is submitted
27     asyncTask.addInboundDependency(mapReduceScript);
28
29     // Submit the search task
30     const asyncTaskId = asyncTask.submit();
31 });

```

To read the contents of the search results file in a dependent map/reduce script, consider the following script sample:

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType MapReduceScript
4   * @NModuleScope SameAccount
5   */
6
7  define(['N/runtime', 'N/file', 'N/log', 'N/email'], (runtime, file, log, email) => {
8      // Load the search results file, count the number of letters in the file, and
9      // store this count in another file
10
11      function getInputData() {
12          // Retrieve the ID of the search results file
13          //
14          // Update the completionScriptParameterName value to use the script
15          // ID of the original search task
16          const fileId = runtime.getCurrentScript().getParameter({
17              name: 'custscript_mr_new_submit_task_field'
18          });
19
20          if (!fileId) {
21              log.error({
22                  details: 'fileId is not valid. Please check the script parameter stored in the completionScriptParameterName
23                  variable in getInputData().'
24              });
25          }
26
27          return {
28              type: 'file',
29              id: fileId
30          };
31      }
32
33      function map(context) {
34          log.debug('map context: ', context);
35          context.write({
36              key: context.key,
37              value: 1
38          });
39      }
40
41      function reduce(context) {
42          log.debug('reduce context: ', context);
43          log.debug('context.values.length', context.values.length);
44
45          // Count the number of rows
46          let rowCount = 0;
47          rowCount = context.values.length - 1 // Subtracting 1 to remove header row from the count.
48          log.debug('rowCount: ', rowCount);

```

```

48 // Send an email to the user who ran the script, and attach the
49 // CSV file with the search results
50 const completionScriptParameterName = 'custscript_mr_new_submit_task_field';
51 const resFileId = runtime.getCurrentScript().getParameter({
52     name: completionScriptParameterName
53 });
54 const fileObj = file.load({
55     id: resFileId
56 });
57 const userId = runtime.getCurrentUser().id;
58 email.send({
59     author: userId,
60     recipients: userId,
61     subject: 'Search completed',
62     body: 'CSV file attached, ' + rowCount + ' record(s) found.',
63     attachments: [fileObj]
64 });
65 }
66
67 function summarize(summary) {
68     const type = summary.toString();
69     log.audit({ title: type + ' Usage Consumed ', details: summary.usage });
70     log.audit({ title: type + ' Concurrency Number ', details: summary.concurrency });
71     log.audit({ title: type + ' Number of Yields ', details: summary.yields });
72 }
73 return {
74     getInputData: getInputData,
75     map: map,
76     reduce: reduce,
77     summarize: summarize
78 };
79
80 });

```

Submit a Record Action Task and Check Status

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to submit a record action task and then check its status. For details about record action tasks, see [task.RecordActionTask](#) and [task.RecordActionTaskStatus](#).

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/task'], function(task) {
6      var recordActionTask = task.create({
7          taskType: task.TaskType.RECORD_ACTION
8      });
9      recordActionTask.recordType = 'timebill';
10     recordActionTask.action = 'approve';
11     recordActionTask.params = [
12         {recordId: 1, note: 'This is a note for 1'},
13         {recordId: 5, note: 'This is a note for 5'},
14         {recordId: 23, note: 'This is a note for 23'}
15     ];
16
17     var handle = recordActionTask.submit();
18
19     var res = task.checkStatus({

```

```

20     taskId: handle
21   }); // Returns a RecordActionTaskStatus object
22   log.debug('Initial status: ' + res.status);
23 });

```

task.CsvImportTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The properties of a CSV import task. Use the methods and properties for this object to submit a CSV import task into the task queue and asynchronously import record data into NetSuite. Use the CsvImportTask Object to perform the following types of tasks: - Automate standard record data import for SuiteApp installations, demo environments, and testing environments. - Import data on a schedule using a scheduled script. - Build integrated CSV imports with RESTlets. Use the following process to import CSV data with CsvImportTask: - In the NetSuite UI, run the Import Assistant to set up the CSV mapping and import options. You must run the Import Assistant to set up the necessary mapping for the CSV import. You can use a sample file or files to set up the mapping. Note the following information: - Script ID for import map. - Any required linked files. For more information, see the help topic Importing CSV Files with the Import Assistant. - Use task.create(options) to create the CsvImportTask object. - Use the CsvImportTask object properties to set the script and deployment properties. - Use CsvImportTask.submit() to submit the import task to the NetSuite task queue. - Use the properties for the task.CsvImportTaskStatus object to get the status of the import process. Use the following guidelines with the CsvImportTask Object: - CSV imports performed within scripts are subject to the existing application limit of 25,000 records. - You cannot import data that is imported by (2-step) assistants in the UI, because these import types do not support saved import maps. This limitation applies to budget, single journal entry, single inventory worksheet, project tasks, and website redirects imports. - This object has access only to the field mappings of a saved import map; it does not have access to advanced import options defined in the Import Assistant, such as multi-threading and multiple queues. Even if you set options to use multiple threads or queues for an import job and then save the import map, these settings are not available to CsvImportTask. When this object submits a CSV import job based on the saved import map, a single thread and single queue are used.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	CsvImportTask Object Members

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var scriptTask = task.create({
4     taskType: task.TaskType.CSV_IMPORT
5 });
6 scriptTask.mappingId = 51;
7 var f = file.load('SuiteScripts/custjoblist.csv');
8 scriptTask.importFile = f;
9 scriptTask.linkedFiles = {'addressbook': 'street,city\nval1,val2',
10     'purchases': file.load('SuiteScripts/other.csv')};
11 var csvImportTaskId = scriptTask.submit();
12 ...
13 //Add additional code

```

CsvImportTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Directs NetSuite to place a CSV import task into the NetSuite task queue and returns a unique ID for the task. Use CsvImportTaskStatus.status to view the status of a submitted task. This method throws errors resulting from inline validation of CSV file data before the import of data begins (the same validation that is performed between the mapping step and the save step in the Import Assistant). Any errors that occur during the import job are recorded in the CSV response file, as they are for imports initiated through the Import Assistant. Important: You can use this method only in bundle installation scripts, scheduled scripts, and RESTlets.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100 units
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	Failed to submit job request: {reason}	Task cannot be submitted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var csvImportTaskId = csvTask.submit();
4 ...
5 //Add additional code
```

CsvImportTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.CsvImportTask
Sibling Object Members	CsvImportTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task.
4
5 var csvImportTaskId = csvTask.submit(); //csvImportTaskId is the unique task id
6 ...
7 //Add additional code
```

CsvImportTask.importFile

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	CSV file to import. Use a file.File object or a string that represents the CSV text to be imported.
Type	file.File string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var f = file.load('SuiteScripts/custjoblist.csv');
4 scriptTask.importFile = f;
5 ...
6 //Add additional code
```

CsvImportTask.linkedFiles

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A map of key-value pairs that sets the data to be imported in a linked file for a multi-file import job, by referencing a file in the File Cabinet or the raw CSV data to import. The key is the internal ID of the record sublist for which data is being imported and the value is either a file.File object or the raw CSV data to import. You can assign multiple types of values to the <code>linkedFiles</code> property.
Type	Object
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 scriptTask.linkedFiles = {'addressbook': 'street,city\nval1,val2',
4   'purchases': file.load('SuiteScripts/other.csv')};
5 ...
6 //Add additional code
```

CsvImportTask.mappingId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Script ID or internal ID of the saved import map that you created when you ran the Import Assistant. See task.CsvImportTask .
Type	number string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 scriptTask.mappingId = 51;
4 ...
5 //Add additional code
```

CsvImportTask.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Name for the CSV import task. You can optionally set a different name for a scripted import task. In the UI, this name appears on the CSV Import Job Status page.
Type	string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 csvTask.name = 'Import Entities'
4 ...
5 //Add additional code
```

CsvImportTask.queueId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Overrides the Queue Number property under Advanced Options on the Import Options page of the Import Assistant. Use this property to programmatically select an import queue and improve performance during the import. Note: This property is only available if you have a SuiteCloud Plus license. For more information about using multiple queues when importing CSV files, see the help topics Queue Number and Use Multiple Threads and Multiple Queues to Run CSV Import Jobs.
Type	number
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 scriptTask.queueId = 2;
4 ...
5 //Add additional code
```

task.CsvImportTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The status of a CSV import task placed into the NetSuite scheduling queue. Use task.checkStatus(options) with the unique ID for the CSV import task to get the <code>CsvImportTaskStatus</code> object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	CsvImportTaskStatus Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code ...
2 var csvTaskStatus = task.checkStatus({
3     taskId: csvTaskId
4 });
5 if (csvTaskStatus.status === task.TaskStatus.FAILED)
6     log.debug('task failed');
7 ...
8 //Add additional code
```

CsvImportTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status for a CSV import task. Returns a task.TaskStatus enum value.
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus({
3     taskId: scriptTaskId
4 });
5
6 log.audit({
7     title: 'Status',
8     details: summary.status
9 });
10 ...
11 //Add additional code

```

CsvImportTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The task ID associated with the specified task.
Type	string (read-only)
Module	N/task Module
Parent Object	task.CsvImportTaskStatus
Sibling Object Members	CsvImportTaskStatus Object Members
Since	—

Errors

Error Code	Error Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myCsvImportTask is an existing task.CsvImportTask object
4 var myTaskId = myCsvImportTask.submit();
5 var myTaskStatus = task.checkStatus({

```

```

6      taskId: myTaskId
7    });
8    var theTaskStatusId = myTaskStatus.taskId;
9    ...
10   // Add additional code

```

task.DocumentCaptureTask

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An asynchronous document capture task. Use the methods and properties for this object to submit a document capture task to the NetSuite task queue and execute it asynchronously. You can create a <code>task.DocumentCaptureTask</code> object using task.create(options). You can populate the properties of this object as you create it. With a <code>task.DocumentCaptureTask</code> object, you can do the following: - Specify the following information about the document capture task: - The document to extract content from using the DocumentCaptureTask.inputFile property. This property accepts a file.File object that represents a supported document in the NetSuite File Cabinet. - The document type using the DocumentCaptureTask.documentType property. Use a value from the documentCapture.DocumentType enum to set the value of this property. - The feature content to extract from the document (such as fields, tables, or text) using the DocumentCaptureTask.features property. Use values from the documentCapture.Feature to set the value of this property. If you don't specify any features, the <code>TEXT_EXTRACTION</code> and <code>TABLE_EXTRACTION</code> features are used by default. - The language of the document using the DocumentCaptureTask.language property. Use a value from the documentCapture.Language enum to set the value of this property. If you don't specify a language, English is used by default. - Oracle Cloud Infrastructure (OCI) credentials for unlimited usage mode using the DocumentCaptureTask.ociConfig property. If you don't specify these credentials, the document capture task consumes usage from the free usage pool provided in NetSuite by default. For more information, see Using OCI Credentials to Obtain Additional Usage. - Set the file path of a JSON file in the NetSuite File Cabinet using the DocumentCaptureTask.outputFilePath property. Document capture results are exported to this file. - Add dependent tasks to the document capture task using DocumentCaptureTask.addInboundDependency(options). Dependent tasks are processed automatically when the document capture task is complete. - Submit the document capture task to the task queue using DocumentCaptureTask.submit(). - Get the status of a document capture task using the properties of an associated task.DocumentCaptureTaskStatus object. The task.checkStatus(options) method returns a task.DocumentCaptureTaskStatus object when you specify a task ID that represents a document capture task. For an example of submitting an asynchronous document capture task, see Extract Content from a Document Asynchronously.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	DocumentCaptureTask Object Members

Since	2025.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myDocumentCaptureTask = task.create({
4     taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6 ...
7 // Add additional code

```

DocumentCaptureTask.addInboundDependency(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a scheduled script task as a dependent task to the document capture task. Dependent tasks are processed automatically using SuiteCloud Processors when the document capture task is complete. For more information, see the help topic SuiteCloud Processors. You can add a dependent task in the following ways: ■ Provide a task.ScheduledScriptTask object that represents the dependent task as the only parameter to this method. ■ Provide an Object as the only parameter that includes values for the following properties: □ taskType □ scriptId □ deploymentId (optional) □ params (optional) For descriptions of these properties, see Parameters. Important: You can add only scheduled scripts as dependent tasks to asynchronous document capture tasks. Other script types are not supported. You can add only one dependent task per call to <code>DocumentCaptureTask.addInboundDependency(options)</code>. When you use this method to add a dependent task, the added task is considered an inbound dependency of the document capture task. For example, if you add a scheduled script task to a document capture task as a dependent task, the scheduled script depends on the document capture task. Because <code>DocumentCaptureTask.addInboundDependency(options)</code> is called on the document capture task, any dependent task that you add is considered an inbound dependency.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/task Module

Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	The script ID of the scheduled script record for the dependent task.
options.taskType	string	required	The type of dependent task. This property uses the task.TaskType.SCHEDULED_SCRIPT value in the task.TaskType enum.
options.deploymentId	string	optional	The script ID of the script deployment record for the dependent task. To use this property, the scheduled script for the dependent task must be deployed. If you do not specify a value for this property, an available deployment is selected automatically.
options.params	Object	optional	The parameters for the scheduled script. To use this property, you must first create script parameters that are required for the dependent task. For example, if your dependent task requires a reference to the JSON result file, you must create a script parameter that maps the JSON result file ID to the file name, as in the following example: <pre> 1 { 'custscript_docCapture_res' : 'resultFile.json' 2 } </pre> For more information about script parameters, see the help topic Creating Script Parameters (Custom Fields) .

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The specified dependent task is not a scheduled script.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | // Be sure to use a script ID that represents a scheduled script in your account

```

```

4 var myScheduledScript = task.create({
5   taskType: task.TaskType.SCHEDULED_SCRIPT,
6   scriptId: 'customscript_as_ftr_ss'
7 });
8
9 // myDocCaptureTask is an existing task.DocumentCaptureTask object
10 myDocCaptureTask.addInboundDependency(myScheduledScript);
11 ...
12 // Add additional code

```

DocumentCaptureTask.submit()

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits the document capture task for asynchronous processing. This method returns a unique ID for the document capture task. When the submission is successful, this method adds the script IDs of any dependent tasks (added using DocumentCaptureTask.addInboundDependency(options)) to the DocumentCaptureTask.inboundDependencies property. Use the task.DocumentCaptureTaskStatus object to view the status of a submitted document capture task. The task.checkStatus(options) method returns a task.DocumentCaptureTaskStatus object when you specify a task ID that represents a document capture task.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100 units
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Errors

Error Code	Thrown If
CANNOT_RESUBMIT_SUBMITTED_DOCUMENT_CAPTURE_TASK	The document capture task was already submitted and completed successfully.
DOCUMENT_CAPTURE_DEPENDENCY_SS_ALREADY_SUBMITTED	A dependent scheduled script task is already submitted and is not complete.
DOCUMENT_CAPTURE_DEPENDENCY_SS_INCORRECT_STATUS	The status of the script deployment record for the specified dependent scheduled script task has a value other than "Not Scheduled".
DOCUMENT_CAPTURE_DEPLOYMENT_FOR_DEPENDENCY	A script deployment record for the specified dependent script task is not available for one of the following reasons: ■ A script deployment record was not specified when the dependent task was created, and automatic lookup for an available script deployment record failed.

Error Code	Thrown If
	The script deployment record specified when the dependent task was created is not found.
DOCUMENT_CAPTURE_MULTIPLE_DEPENDENCIES	The same dependent task is passed to this method more than one time.
DOCUMENT_CAPTURE_SCRIPT_ID_NOT_FOUND	The specified dependent task is not found.
FAILED_TO_SUBMIT_JOB_REQUEST_1	The document capture task cannot be submitted due to an unexpected error.
MUST_IDENTIFY_A_FILE	The DocumentCaptureTask.outputFilePath property specifies a folder and not a file.
SSS_MISSING_REQD_ARGUMENT	A required property is not specified.
THAT_RECORD_DOES_NOT_EXIST	The DocumentCaptureTask.inputFile property references a file that does not exist.
YOU_DO_NOT_HAVE_ACCESS_TO_THE_MEDIA_ITEM_YOU_SELECTED	You do not have permission to access the file specified by the DocumentCaptureTask.inputFile property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myDocCaptureTask = task.create({
4     taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6 // Submit the task
7 var myTaskId = myDocCaptureTask.submit();
8
9 // Check the task status
10 var myStatus = task.checkStatus({
11     taskId: myTaskId
12 });
13 ...
14 // Add additional code

```

DocumentCaptureTask.documentType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The document type. Use a value from the documentCapture.DocumentType enum to set the value of this property. For more information, see the description of the <code>options.documentType</code> parameter in documentCapture.documentToStructure(options).
Type	string
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Errors

Error Code	Thrown If
INVALID_DOCUMENT_TYPE	The value of this property is not included in the documentCapture.DocumentType enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myDocCaptureTask = task.create({
4   taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6 myDocCaptureTask.documentType = documentCapture.DocumentType.INVOICE;
7 ...
8 // Add additional code

```

DocumentCaptureTask.features

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The features to extract from the document (such as fields, tables, or text). Use values from the documentCapture.Feature enum to set the value of this property. If you don't specify any features, the TEXT_EXTRACTION and TABLE_EXTRACTION features are used by default. For more information, see the description of the options.features parameter in documentCapture.documentToStructure(options) .
Type	string[]
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	This property has values that are not included in the documentCapture.Feature enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var myDocCaptureTask = task.create({
4      taskType: task.TaskType.DOCUMENT_CAPTURE
5  });
6  myDocCaptureTask.features = [
7      documentCapture.Feature.FIELD_EXTRACTION,
8      documentCapture.Feature.TABLE_EXTRACTION
9  ];
10 ...
11 // Add additional code

```

DocumentCaptureTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  // Add additional code
2  ...
3  // A unique task ID for the task is returned when you submit the task (docCaptureTask is a previously created task.DocumentCaptureTask object)
4
5  var docCaptureTaskId = docCaptureTask.submit(); // docCaptureTaskId is the unique task ID
6  ...
7  // Add additional code

```

DocumentCaptureTask.inboundDependencies

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Key-value pairs that contain information about the dependent tasks added to the document capture task. Use this property to verify the properties of dependent tasks after you add the tasks using DocumentCaptureTask.addInboundDependency(options). This property uses nested objects to store information about each dependent task. A nested object is included for each dependent task added to the document capture task, and these objects are referenced by their index (starting at 0). Dependent tasks are indexed in the order they are added to the document capture task. Each nested object contains the task type, script ID, script deployment ID, and script parameters. For example, consider a situation in which you add a scheduled script task to a document capture task as a dependent task. After you add the dependent task but before you
-----------------------------	--

	submit the document capture task using <code>DocumentCaptureTask.submit()</code>, the value of the <code>DocumentCaptureTask.inboundDependencies</code> property is similar to the following: <pre>1 {"0": {"type": "task.ScheduledScriptTask", "scriptId": "customscript_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": {"custscript_ss_as_srch_res": "SuiteScripts/ExportFile.csv"} }}</pre> After you submit the document capture task, the IDs of the dependent scripts are added to the <code>DocumentCaptureTask.inboundDependencies</code> property: <pre>1 {"0": {"type": "task.ScheduledScriptTask", "id": "SCHEDSCRIP T_0168697b126d1705061d0d690a787755500b046a1912686b10_349d94266564827c739a2ba0a5b9d476f4097217", "scriptId": "custom script_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": {"custscript_ss_as_srch_res": "SuiteScripts/Export File.csv"} }}</pre>
Type	Object[] (read-only)
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.DocumentCaptureTask object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 | // Add additional code
2 | ...
3 | // Be sure to use a script ID that represents a scheduled script in your account
4 | var myScheduledScript = task.create({
5 |     taskType: task.TaskType.SCHEDULED_SCRIPT,
6 |     scriptId: 'customscript_as_ftr_ss'
7 | });
8 |
9 | // myDocCaptureTask is an existing task.DocumentCaptureTask object
10 | myDocCaptureTask.addInboundDependency(myScheduledScript);
11 | ...
12 | // Add additional code
```

DocumentCaptureTask.inputFile

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The document to extract content from. The value of this property is a file.File object that represents a file in the NetSuite File Cabinet. For more information, see the description of the options.<code>file</code> parameter in documentCapture.documentToStructure(options).
----------------------	---

Type	file.File
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Errors

Error Code	Thrown If
UNSUPPORTED_FILE_TYPE	The specified document is not in PDF, TIFF, JPG, or PNG format.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myDocCaptureTask = task.create({
4     taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6 myDocCaptureTask.inputFile = file.load("542");
7 ...
8 // Add additional code

```

DocumentCaptureTask.language

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The language of the document. Use values from the documentCapture.Language enum to set the value of this property. If you don't specify a language, English is used by default. For more information, see the description of the options. <code>language</code> parameter in documentCapture.documentToStructure(options) .
Type	string
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Errors

Error Code	Thrown If
INVALID_LANGUAGE	The value of this property is not included in the documentCapture.Language enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var myDocCaptureTask = task.create({
4     taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6 myDocCaptureTask.language = documentCapture.Language.CES;
7 ...
8 // Add additional code
```

DocumentCaptureTask.outputFilePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The path of the JSON file to export document capture results to. The JSON file must be located in the NetSuite File Cabinet. For an example, see Extract Content from a Document Asynchronously .
Type	string
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var myDocCaptureTask = task.create({
4     taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6 myDocCaptureTask.outputFilePath = "SuiteScripts/result.json";
7 ...
8 // Add additional code
```

DocumentCaptureTask.ociConfig

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Oracle Cloud Infrastructure (OCI) credentials when using unlimited usage mode. For more information, see the description of the <code>options.ociConfig</code> parameter in documentCapture.documentToStructure(options) . For an example, see Extract Content from a Document Asynchronously .
-----------------------------	--

Type	Object
Module	N/task Module
Parent Object	task.DocumentCaptureTask
Sibling Object Members	DocumentCaptureTask Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myDocCaptureTask = task.create({
4     taskType: task.TaskType.DOCUMENT_CAPTURE
5 });
6
7 // Make sure you use the credentials from your OCI account with access to the
8 // OCI Document Understanding service
9 myDocCaptureTask.ociConfig = {
10     userId: 'user-ocid',
11     tenancyId: 'user-tenancy',
12     compartmentId: 'user-compartment',
13     fingerprint: 'custsecret_secret_fingerprint_id',
14     privateKey: 'custsecret_secret_privatekey_id',
15     objectStorageNamespace: 'oraclenetsuite',
16     outputBucketName: 'in-bucket-name',
17     inputBucketName: 'out-bucket-name'
18 };
19 ...
20 // Add additional code

```

task.DocumentCaptureTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Status of a document capture task (task.DocumentCaptureTask) in the NetSuite task queue. You create a <code>task.DocumentCaptureTaskStatus</code> object using task.checkStatus(options). To check the status of a document capture task, you must provide the task ID of the document capture task. To submit a document capture task and obtain the task ID, use DocumentCaptureTask.submit().
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	DocumentCaptureTaskStatus Object Members
Since	2025.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // myDocCaptureTask is an existing task.DocumentCaptureTask object
4 var myTaskId = myDocCaptureTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 ...
9 // Add additional code
```

DocumentCaptureTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status of the submitted document capture task. A document capture task can have the following statuses, which are represented by values in the task.TaskStatus enum: - task.TaskStatus.COMPLETE - task.TaskStatus.FAILED - task.TaskStatus.PENDING - task.TaskStatus.PROCESSING
Type	string (read-only)
Module	N/task Module
Parent Object	task.DocumentCaptureTaskStatus
Sibling Object Members	DocumentCaptureTaskStatus Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.DocumentCaptureTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // myDocCaptureTask is an existing task.DocumentCaptureTask object
4 var myTaskId = myDocCaptureTask.submit();
```

```

5  var myTaskStatus = task.checkStatus({
6      taskId: myTaskId
7  });
8  var theStatus = myTaskStatus.status;
9  ...
10 // Add additional code

```

DocumentCaptureTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of the submitted document capture task. This property references the same task ID that is returned by DocumentCaptureTask.submit() when you submit the task for asynchronous processing.
Type	string (read-only)
Module	N/task Module
Parent Object	task.DocumentCaptureTaskStatus
Sibling Object Members	DocumentCaptureTaskStatus Object Members
Since	2025.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.DocumentCaptureTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  // Add additional code
2  ...
3  // myDocCaptureTask is an existing task.DocumentCaptureTask object
4  var myTaskId = myDocCaptureTask.submit();
5  var myTaskStatus = task.checkStatus({
6      taskId: myTaskId
7  });
8  var theTaskId = myTaskStatus.taskId;
9  ...
10 // Add additional code

```

task.EntityDeduplicationTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	All the properties of a merge duplicate records task request. Use the methods and properties of this object to submit a merge duplicate record job task into the NetSuite task queue.
---------------------------	---

	When you submit a merge duplicate record task to NetSuite, SuiteScript enables you to use all of the same functionality available through the UI. Use SuiteScript to use the predefined duplicate detection rules, or you can define your own. After the records are merged or deleted, in the UI, the records no longer appear as duplicates at Lists > Mass Update > Entity Duplicate Resolution . For more information about merging duplicate records in NetSuite, see the help topic Merging or Deleting Duplicate Records. To use the EntityDeduplicationTask Object: ■ Use task.create(options) to create the EntityDeduplicationTask object. ■ Use EntityDeduplicationTask.entityType to select the entity type on which you want to merge duplicate records. ■ Use EntityDeduplicationTask.dedupeMode to select the action to take for the duplicate records. ■ Use a EntityDeduplicationTask.masterSelectionMode enum value to identify which record to use as the master record in the merge. ■ If you use MasterSelectionMode.SELECT_BY_ID for the master selection mode, set the ID of the master record with EntityDeduplicationTask.masterRecordId. ■ Identify the duplicate records. Use the search.duplicates(options) method in the N/search Module to find the duplicate records. ■ Use EntityDeduplicationTask.submit() to submit the merge duplicate record task to the NetSuite task queue. ■ Use the properties for the task.EntityDeduplicationTaskStatus object to get the status of the merge duplicate record task. Use the following guidelines with the EntityDeduplicationTask Object: ■ You can only submit 200 records in a single merge duplicate records task. ■ The merge duplicate functionality on non-entity records is not supported in SuiteScript. ■ You must have full permissions to the Duplicate Record Management permission to merge duplicates.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	EntityDeduplicationTask Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var dedupeTask = task.create({
4     taskType: task.TaskType.ENTITY_DEDUPLICATION
5 });
6 dedupeTask.entityType = task.DedupeEntityType.CUSTOMER;
7 dedupeTask.dedupeMode = task.DedupeMode.MERGE;
8 dedupeTask.masterSelectionMode = task.MasterSelectionMode.MOST_RECENT_ACTIVITY;
9 dedupeTask.recordIds = ['107', '110'];
10 var dedupeTaskId = dedupeTask.submit();

```

```

11 ...
12 //Add additional code

```

EntityDeduplicationTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Directs NetSuite to place the merge duplicate records task into the NetSuite task queue and returns a unique ID for the task. Use EntityDeduplicationTaskStatus.status to view the status of a submitted task.
Returns	string The task ID
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100 units
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	Failed to submit job request: {reason}	Task cannot be submitted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var dedupeTaskId = dedupeTask.submit();
4 ...
5 //Add additional code

```

EntityDeduplicationTask.dedupeMode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The mode in which to merge or delete duplicate records. Use a task.DedupeMode enum value to set this property.
Type	string

Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 dedupeTask.dedupeMode = task.DedupeMode.MERGE;
4 ...
5 //Add additional code

```

EntityDeduplicationTask.entityType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of entity on which you want to merge duplicate records. Use a task.DedupeEntityType enum value to set the value. Note: If you set entityType to CUSTOMER, the system will automatically include prospects and leads in the task request.
Type	string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 dedupeTask.entityType = task.DedupeEntityType.CUSTOMER;
4 ...
5 //Add additional code

```

EntityDeduplicationTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
-----------------------------	---------------------

Type	string
Module	N/task Module
Parent Object	task.EntityDeduplicationTask
Sibling Object Members	EntityDeduplicationTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task. (dedupeTask is a previously created task.EntityDeduplicationTask object)
4
5 var dedupeTaskId = dedupeTask.submit(); //dedupeTaskId is the unique task id
6 ...
7 //Add additional code

```

EntityDeduplicationTask.masterRecordId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	When you merge duplicate records, you can delete all duplicates for a record or merge information from the duplicate records into the master record. Use this property to set the ID of the master record that you want to use as the master record in the merge. Important: You must also select <code>SELECT_BY_ID</code> for the EntityDeduplicationTask.masterSelectionMode property, or NetSuite ignores this setting.
Type	number
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 dedupeTask.masterSelectionMode = task.MasterSelectionMode.SELECT_BY_ID;
4 dedupeTask.masterRecordId = 107;
5 ...
6 //Add additional code

```

EntityDeduplicationTask.masterSelectionMode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	When you merge duplicate records, you can delete all duplicates for a record or merge information from the duplicate records into the master record. Set this property to determine which of the duplicate records to keep or select the master record to use by ID. Use the task.MasterSelectionMode enum to set the value.
Type	string
Module	N/task Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 dedupeTask.masterSelectionMode = task.MasterSelectionMode.MOST_RECENT_ACTIVITY;
4 ...
5 //Add additional code

```

EntityDeduplicationTask.recordIds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Number array of record internal IDs to perform the merge or delete operation on. You can use the search.duplicates(options) method to identify duplicate records or create an array with record internal IDs.
Type	number[]
Module	N/task Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 dedupeTask.recordIds = ['107', '110'];
4 ...

```



```
5 //Add additional code
```

task.EntityDeduplicationTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The status of a merge duplicate record task placed into the NetSuite task queue by EntityDeduplicationTask.submit() . Use task.checkStatus(options) with the unique ID for the merge duplicate records task to get this Object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	EntityDeduplicationTaskStatus Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code ...
2 var dedupeTaskStatus = task.checkStatus({
3     taskId: dedupeTaskId
4 });
5 if (dedupeTaskStatus.status === task.TaskStatus.FAILED)
6     log.debug('task failed');
7 ...
8 //Add additional code
```

EntityDeduplicationTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status for a merge duplicate record task. Returns a task.TaskStatus enum value.
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

EntityDeduplicationTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The task ID associated with the specified task.
Type	string (read-only)
Module	N/task Module
Parent Object	task.EntityDeduplicationTaskStatus
Sibling Object Members	EntityDeduplicationTaskStatus Object Members
Since	—

Errors

Error Code	Error Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myEntityDeduplicationTask is an existing task.EntityDeduplicationTask object
4 var myTaskId = myEntityDeduplicationTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskStatusId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.MapReduceScriptTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The properties of a map/reduce script deployment. You can use this object to programmatically submit a script deployment for processing. To use the MapReduceScriptTask object:
---------------------------	---

	■ In the NetSuite UI, create the script record and script deployment records. ■ Use task.create(options) to create the MapReduceScriptTask object. ■ Use the MapReduceScriptTask object properties to set the script and deployment properties. ■ Use MapReduceScriptTask.submit() to submit the deployment for processing. ■ Use the properties for the task.MapReduceScriptTaskStatus object to get the status of the map/reduce script. For general information about map/reduce scripts, see the help topic Map/Reduce Key Concepts.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	MapReduceScriptTask Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mrTask = task.create({
4     taskType: task.TaskType.MAP_REDUCE
5 });
6 mrTask.scriptId = mapReduceScriptId;
7 mrTask.deploymentId = custdeploy1;
8 var mrTaskId = mrTask.submit();
9 ...
10 //Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits a map/reduce script deployment for processing. For more information, see task.MapReduceScriptTask. Additionally, note that a map/reduce script can be submitted for processing only if there is no unfinished map/reduce script task for the same script ID and script deployment ID. For this reason, if a map/reduce script resubmits itself, the resubmit does not occur until the current execution completes. This delay is necessary to avoid the existence of two unfinished tasks for the same deployment of the same script. Therefore, if a map/reduce script uses the submit() method to resubmit itself, then at runtime, no task ID is returned when the map/reduce script is submitted. If submitted scheduled scripts are requested at the same time or rapidly in
---------------------------	--

	succession, the scheduled tasks may not proceed to the Queued status and must resubmit if fails.  Note: This method will ad-hoc start the Map/Reduce deployment with the identity/role being inherited from the script that called the method. Executing a map/reduce script from a server script requires either the Administrator role or a role with the SuiteScript and SuiteScript Scheduling permissions. For more information, see the help topic Permissions Documentation. For general information about the execution of map/reduce scripts, see the help topic Map/Reduce Script Submission.
Returns	string The task ID, except as noted above.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	20 units
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	Failed to submit job request: {reason}	Task cannot be submitted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mxTaskId = mxTask.submit();
4 ...
5 //Add additional code

```

 **Note:** For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTask.deploymentId

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Script ID (as a string), for the script deployment record for a map/reduce script.
Type	string
Module	N/task Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 mrTask.deploymentId = custdeploy1;
4 ...
5 //Add additional code
```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.MapReduceScriptTask
Sibling Object Members	MapReduceScriptTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task. (mrTask is a previously created task.MapReduceScriptTask object)
4
5 var mrTaskId = mrTask.submit(); //mrTaskId is the unique task id
6 ...
7 //Add additional code
```

MapReduceScriptTask.params

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The key-value pairs that override static script parameter field values on the script deployment record.
----------------------	---

	Use these parameters on a task.MapReduceScriptTask object to programmatically pass values to the script deployment. For example, if you create a script parameter called <code>custscript_my_script_param</code>, you can set the value of that parameter as follows: <pre> 1 // mrTask is a MapReduceScriptTask object 2 mrTask.params = { 'custscript_my_script_param': 'value' 3 }; </pre> For more information about script parameters, see the help topic Creating Script Parameters Overview.
Type	Object
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 // mrTask is a MapReduceScriptTask object
3 mrTask.params = { 'custscript_boolean_param': true };
4 ...
5 // Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTask.scriptId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID (as a number), or script ID (as a string), for the map/reduce script record.
Type	number string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var mapReduceScriptId = 34;
4 ...
5 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

task.MapReduceScriptTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The status of a map/reduce script deployment that was submitted for processing. Use task.checkStatus(options) with the unique ID for the map/reduce script task to get the MapReduceScriptTaskStatus object. For general information about the execution of map/reduce scripts, see the help topic Map/Reduce Script Submission.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	MapReduceScriptTaskStatus Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 if (summary.stage === task.MapReduceStage.SUMMARIZE)
4     log.audit('Almost done...');
5 ...
6 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getCurrentTotalSize()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total size in bytes of all stored work in progress by a task.MapReduceScriptTask .
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	25 units

Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 var total = summary.getCurrentTotalSize()
4 log.audit({
5     title: 'Size of Remaining Data to Process',
6     details: total
7 });
8 ...
9 //Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPendingMapCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of records or rows not yet processed by the map stage of a task.MapReduceScriptTask. Use the MapReduceScriptTaskStatus.stage property to get the current stage. For general information about map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var summary = taskStatus.getPendingMapCount();
4 log.audit('Pending Map Count: ' + summary);
5 ...
6 //Add additional code

```


Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPendingMapSize()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of bytes not yet processed by the map stage, as a component of total size, of a task.MapReduceScriptTask . Use the MapReduceScriptTaskStatus.stage property to get the current stage. For general information about map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages .
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	25 units
Module	N/task Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var summary = taskStatus.getPendingMapSize();
4 log.audit('Pending Map Size: ' + summary);
5 ...
6 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPendingOutputCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of records or rows not yet processed by a task.MapReduceScriptTask .
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units

Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 var total = summary.getPendingOutputCount()
4 log.audit({
5     title: 'Count',
6     details: total
7 });
8 ...
9 //Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPendingOutputSize()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total size in bytes of all key-value pairs written as output, as a component of total size, by a task.MapReduceScriptTask .
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	25 units
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 var total = summary.getPendingOutputSize()
4 log.audit({
5     title: 'Size',
6     details: total
7 });
8 ...
9 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPendingReduceCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of records or rows not yet processed by the reduce stage of a task.MapReduceScriptTask. Use the MapReduceScriptTaskStatus.stage property to get the current stage. For general information about the reduce stage and other map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/task Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = taskStatus.getPendingReduceCount();
3 log.audit({
4     details: 'Pending Reduce Count: ' + summary
5 });
6 ...
7 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPendingReduceSize()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of bytes not yet processed by the reduce stage, as a component of total size, of a task.MapReduceScriptTask. Use the MapReduceScriptTaskStatus.stage property to get the current stage.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	25 units
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = taskStatus.getPendingReduceSize();
3 log.audit({
4     details: 'Pending Reduce Size: ' + summary
5 });
6 ...
7 //Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getPercentageCompleted()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the current percentage complete for the current stage of a task.MapReduceScriptTask. Use the MapReduceScriptTaskStatus.stage property to get the current stage. For general information about map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages. Note: The input and summarize stages are either 0% or 100% complete at any time.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...

```

```

3 var completion = taskStatus.getPercentageCompleted();
4 log.audit('Percentage Completed: ' + completion);
5 ...
6 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getTotalMapCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of records or rows passed as input to the map stage of a task.MapReduceScriptTask. Use the MapReduceScriptTaskStatus.stage property to get the current stage. For general information about map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/task Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var summary = taskStatus.getTotalMapCount();
4 log.audit('Total Map Count: ' + summary);
5 ...
6 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getTotalOutputCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of key-value pairs passed as inputs to the SUMMARIZE phase of a task.MapReduceScriptTask. Use the MapReduceScriptTaskStatus.stage property to get the current stage.
Returns	number

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 var total = summary.getTotalOutputCount()
4 log.audit({
5   title: 'Total Entries Passed to Output',
6   details: total
7 });
8 ...
9 //Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.getTotalReduceCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the total number of record or row inputs to the reduce stage of a task.MapReduceScriptTask . Use the MapReduceScriptTaskStatus.stage property to get the current stage. For general information about map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages .
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...

```

```

2 | var summary = taskStatus.getTotalReduceCount();
3 | log.audit('Reduce Count: ' + summary);
4 | ...
5 | //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.deploymentId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Script ID for a script deployment record associated with a specific task.MapReduceScriptTask .
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code ...
2 | var summary = task.checkStatus(scriptTaskId);
3 | log.audit({
4 |     title: 'Deployment ID',
5 |     details: summary.deploymentId
6 | }); ...
7 | //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.scriptId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID for a map/reduce script record associated with a specific task.MapReduceScriptTask .
Type	number (read-only)

Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 log.audit({
4     title: 'Script ID',
5     details: summary.scriptId
6 }); ...
7 //Add additional code

```



Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.stage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Current stage of processing for a map/reduce script deployment instance. Returns a task.MapReduceStage enum value. For general information about map/reduce stages, see the help topic Map/Reduce Key Concepts . For information about the execution of map/reduce scripts, see the help topic Map/Reduce Script Submission .
Type	string (read-only)
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 if (summary.stage === task.MapReduceStage.SUMMARIZE)
4     log.audit('Almost done...');
5 ...
6 //Add additional code

```


Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status of a map/reduce script deployment that was submitted for processing. Returns a task.TaskStatus enum value. For general details about the execution of map/reduce scripts, see the help topic Map/Reduce Script Submission .
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 log.audit({
4     title: 'Status',
5     details: summary.status
6 }); ...
7 //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

MapReduceScriptTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The task ID associated with the specified task.
Type	string (read-only)
Module	N/task Module
Parent Object	task.MapReduceScriptTaskStatus
Sibling Object Members	MapReduceScriptTaskStatus Object Members

Since	—
-------	---

Errors

Error Code	Error Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myMapReduceScriptTask is an existing task.MapReduceScriptTask object
4 var myTaskId = myMapReduceScriptTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskStatusId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.QueryTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An asynchronous query task. Use the methods and properties for this object to submit a query task to the NetSuite task queue and execute it asynchronously. You can create a <code>task.QueryTask</code> object using task.create(options). You can populate the properties of this object as you create it. With a <code>task.QueryTask</code> object, you can do the following: - Specify the query to submit using the QueryTask.query property. This property accepts a query.Query object that was constructed using the N/query module. For more information, see N/query Module. - Set the file ID or file path of a CSV file in the File Cabinet. Query results are exported to this file. Use the QueryTask.fileId property to specify a file ID, or use QueryTask.filePath to specify a file path. Only one of these properties can be set at the same time. If both are set, an error occurs. - Add dependent tasks to the query task using QueryTask.addInboundDependency(options). Dependent tasks are processed automatically when the query task is complete. - Submit the query task to the task queue using QueryTask.submit(). - Get the status of a query task using the properties of an associated task.QueryTaskStatus object. The task.checkStatus(options) method returns a task.QueryTaskStatus object when you specify a task ID that represents a query task.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	QueryTask Object Members

Since	2020.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myQueryTask = task.create({
4     taskType: task.TaskType.QUERY
5 });
6 ...
7 // Add additional code

```

QueryTask.addInboundDependency(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a scheduled script task or map/reduce script task as a dependent task to the query task. Dependent tasks are processed automatically using SuiteCloud Processors when the query task is complete. For more information, see the help topic SuiteCloud Processors. You can add a dependent task in the following ways: ■ Provide a task.ScheduledScriptTask object or task.MapReduceScriptTask object that represents the dependent task as the only parameter to this method. ■ Provide an Object as the only parameter that includes values for the following properties: □ taskType □ scriptId □ deploymentId (optional) □ params (optional) For descriptions of these properties, see Parameters. Important: You can add only scheduled scripts or map/reduce scripts as dependent tasks to asynchronous query tasks. Other script types are not supported. You can add only one dependent task per call to <code>QueryTask.addInboundDependency(options)</code>. When you use this method to add a dependent task, the added task is considered an inbound dependency of the query task. For example, if you add a scheduled script task to a query task as a dependent task, the scheduled script depends on the query task. Because <code>QueryTask.addInboundDependency(options)</code> is called on the query task, any dependent task that you add is considered an inbound dependency.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/task Module

Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	The script ID of the scheduled script record or map/reduce script record for the dependent task.
options.taskType	string	required	The type of dependent task. This property uses one of the following values in the task.TaskType enum: - task.TaskType.SCHEDULED_SCRIPT - task.TaskType.MAP_REDUCE
options.deploymentId	string	optional	The script ID of the script deployment record for the dependent task. To use this property, the scheduled script or map/reduce script for the dependent task must be deployed. If you do not specify a value for this property, an available deployment is selected automatically.
options.params	Object	optional	The parameters for the scheduled script or map/reduce script. To use this property, you must first create script parameters that are required for the dependent task. For example, if your dependent task requires a reference to the CSV result file, you must create a script parameter that maps the CSV result file ID to the file name, as in the following example: <pre>1 { 'custscript_query_res' : 'resultFile.csv' 2 }</pre> For more information about script parameters, see the help topic Creating Script Parameters (Custom Fields).

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // Be sure to use a script ID that represents a scheduled script in your account
4 var myScheduledScript = task.create({
5     taskType: task.TaskType.SCHEDULED_SCRIPT,
6     scriptId: 'customscript_as_ftr_ss'
7 });
```

```

8
9 // myQueryTask is an existing task.QueryTask object
10 myQueryTask.addInboundDependency(myScheduledScript);
11 ...
12 // Add additional code

```

QueryTask.submit()

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits the query task for asynchronous processing. This method returns a unique ID for the query task. When the submission is successful, this method adds the script IDs of any dependent tasks (added using QueryTask.addInboundDependency(options)) to the QueryTask.inboundDependencies property. Use the task.QueryTaskStatus object to view the status of a submitted query task. The task.checkStatus(options) method returns a task.QueryTaskStatus object when you specify a task ID that represents a query task.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100 units
Module	N/task Module
Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
ASYNC_QUERY_DEPENDENCY_MR_ALREADY_SUBMITTED	A dependent map/reduce script task is already submitted and is not complete.
ASYNC_QUERY_DEPENDENCY_MR_INCORRECT_STATUS	The status of the script deployment record for the specified dependent map/reduce script task has a value other than "Not Scheduled".
ASYNC_QUERY_DEPENDENCY_SS_ALREADY_SUBMITTED	A dependent scheduled script task is already submitted and is not complete.
ASYNC_QUERY_DEPENDENCY_SS_INCORRECT_STATUS	The status of the script deployment record for the specified dependent scheduled script task has a value other than "Not Scheduled".
ASYNC_QUERY_DEPLOYMENT_FOR_DEPENDENCY	A script deployment record for the specified dependent script task is not available for one of the following reasons: ■ A script deployment record was not specified when the dependent task was created, and

Error Code	Thrown If
	automatic lookup for an available script deployment record failed. The script deployment record specified when the dependent task was created is not found.
ASYNC_QUERY_MULTIPLE_DEPENDENCIES	The same dependent task is passed to this method more than one time.
ASYNC_QUERY_QUERY_ID_NOT_FOUND	A query task with the specified script ID is not found.
ASYNC_QUERY_SCRIPT_ID_NOT_FOUND	The specified dependent task is not found.
CANNOT_RESUBMIT_SUBMITTED_ASYNC_QUERY_TASK	The query task was already submitted and completed successfully.
FAILED_TO_SUBMIT_JOB_REQUEST_1	The query task cannot be submitted due to an unexpected error.
MUST_IDENTIFY_A_FILE	The QueryTask.filePath property specifies a folder and not a file.
SSS_MISSING_REQD_ARGUMENT	A required property is not specified.
THAT_RECORD_DOES_NOT_EXIST	The QueryTask.fileId property or QueryTask.filePath property references a file that does not exist.
YOU_DO_NOT_HAVE_ACCESS_TO_THE_MEDIA_ITEM_YOU_SELECTED	You do not have permission to access the file specified by the QueryTask.fileId property or QueryTask.filePath property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myQueryTask = task.create({
4     taskType: task.TaskType.QUERY
5 });
6 // Submit the task
7 var myTaskId = myQueryTask.submit();
8
9 // Check the task status
10 var myStatus = task.checkStatus({
11     taskId: myTaskId
12 });
13 ...
14 // Add additional code

```

QueryTask.fileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID of the CSV file to export query results to. The CSV file must be located in the File Cabinet. You can provide a value for this property or the QueryTask.filePath property. Only one of these properties can be set at the same time. If both are set, an error occurs.
-----------------------------	---

Type	number
Module	N/task Module
Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You specified values for both QueryTask.fileId and QueryTask.filePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myQueryTask = task.create({
4     taskType: task.TaskType.QUERY
5 });
6 myQueryTask.fileId = 18;
7 ...
8 // Add additional code

```

QueryTask.filePath

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Path of the CSV file to export query results to. The CSV file must be located in the File Cabinet. You can provide a value for this property or the QueryTask.fileId property. Only one of these properties can be set at the same time. If both are set, an error occurs.
Type	string
Module	N/task Module
Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You specified values for both QueryTask.fileId and QueryTask.filePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var myQueryTask = task.create({
4     taskType: task.TaskType.QUERY
5 });
6 myQueryTask.filePath = 'ExportFolder/export.csv';
7 ...
8 // Add additional code
```

QueryTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task. (queryTask is a previously created task.QueryTask object)
4
5 var queryTaskId = queryTask.submit(); //queryTaskId is the unique task id
6 ...
7 //Add additional code
```

QueryTask.inboundDependencies

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Key-value pairs that contain information about the dependent tasks added to the query task. Use this property to verify the properties of dependent tasks after you add the tasks using QueryTask.addInboundDependency(options). This property uses nested objects to store information about each dependent task. A nested object is included for each dependent task added to the query task, and these objects are
-----------------------------	--

	referenced by their index (starting at 0). Dependent tasks are indexed in the order they are added to the query task. Each nested object contains the task type, script ID, script deployment ID, and script parameters. For example, consider a situation in which you add a scheduled script task and a map/reduce script task to a query task as dependent tasks. After you add the dependent tasks but before you submit the query task using QueryTask.submit(), the value of the <code>QueryTask.inboundDependencies</code> property is similar to the following: <pre> 1 { "0": { "type": "task.ScheduledScriptTask", "scriptId": "customscript_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": { "custscript_ss_as_srch_res": "SuiteScripts/ExportFile.csv" } }, 2 "1": { "type": "task.MapReduceScriptTask", "scriptId": "customscript_as_ftr_mr", "deploymentId": "customdeploy_mr_dpl", "params": { "custscript_mr_as_srch_res": "SuiteScripts/ExportFile.csv" } } 3 } </pre> After you submit the query task, the IDs of the dependent scripts are added to the <code>QueryTask.inboundDependencies</code> property: <pre> 1 { "0": { "type": "task.ScheduledScriptTask", "id": "SCHEDSCRIP T_0168697b126d1705061d0d690a787755500b046a1912686b10_349d94266564827c739a2ba05b9d476f4097217", "scriptId": "customscript_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": { "custscript_ss_as_srch_res": "SuiteScripts/ExportFile.csv" } }, 2 "1": { "type": "task.MapReduceScriptTask", "id": "MAPREDUCETASK_0268697b126d1705061d0d69027f7b39560f01001c_7a02acb4bdeb0103120b09302170720aa57bca4", "scriptId": "customscript_as_ftr_mr", "deploymentId": "customdeploy_mr_dpl", "params": { "custscript_mr_as_srch_res": "SuiteScripts/ExportFile.csv" } } 3 } </pre>
Type	Object[] (read-only)
Module	N/task Module
Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.QueryTask object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  // Add additional code
2  ...
3  // Be sure to use a script ID that represents a scheduled script in your account
4  var myScheduledScript = task.create({
5      taskType: task.TaskType.SCHEDULED_SCRIPT,
6      scriptId: 'customscript_as_ftr_ss'
7  });
8
9  // myQueryTask is an existing task.QueryTask object
10 myQueryTask.addInboundDependency(myScheduledScript);
11 ...
12 // Add additional code

```

QueryTask.query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Query definition for the query task. The query definition is a query.Query object. To use this object, you must load the N/query module in your script and create a query using query.create(options) or load a query using query.load(options) . For more information, see N/query Module .
Type	query.Query
Module	N/task Module
Parent Object	task.QueryTask
Sibling Object Members	QueryTask Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var myQueryTask = task.create({
4     taskType: task.TaskType.QUERY
5 });
6
7 // myQuery is an existing query.Query object created using the N/query module
8 myQueryTask.query = myQuery;
9 ...
10 // Add additional code

```

task.QueryTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Status of a query task (task.QueryTask) in the NetSuite task queue. You create a task.QueryTaskStatus object using task.checkStatus(options). To check the status of a query task, you must provide the task ID of the query task. To submit a query task and obtain the task ID, use QueryTask.submit(). Important: Ensure that you are familiar with script execution time limits when creating tasks. Each server script type is limited to an amount of time it can run on a single execution. For more information, see the time limit chart on Script Execution Time Limits.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/task Module
Methods and Properties	QueryTaskStatus Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQueryTask is an existing task.QueryTask object
4 var myTaskId = myQueryTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 ...
9 // Add additional code

```

QueryTaskStatus.fileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID of the CSV file that query results are exported to. This property references the same CSV file that is specified by the QueryTask.fileId property or QueryTask.filePath property in the corresponding task.QueryTask object.
Type	number (read-only)
Module	N/task Module
Parent Object	task.QueryTaskStatus
Sibling Object Members	QueryTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.QueryTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  // myQueryTask is an existing task.QueryTask object
4  var myTaskId = myQueryTask.submit();
5  var myTaskStatus = task.checkStatus({
6      taskId: myTaskId
7  });
8  var theFileId = myTaskStatus.fileId;
9  ...
10 // Add additional code

```

QueryTaskStatus.query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Query definition for the submitted query task. The query definition is a query.Query object. To use this object, you must load the N/query module in your script and create a query using query.create(options) or load a query using query.load(options). For more information, see N/query Module. This property references the same query that is specified by the QueryTask.query property in the corresponding task.QueryTask object.
Type	query.Query (read-only)
Module	N/task Module
Parent Object	task.QueryTaskStatus
Sibling Object Members	QueryTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.QueryTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  // Add additional code
2  ...
3  // myQueryTask is an existing task.QueryTask object
4  var myTaskId = myQueryTask.submit();
5  var myTaskStatus = task.checkStatus({
6      taskId: myTaskId
7  });
8  var theQuery = myTaskStatus.query;
9  ...
10 // Add additional code

```

QueryTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status of the submitted query task. A query task can have the following statuses, which are represented by values in the task.TaskStatus enum: ■ <code>task.TaskStatus.COMPLETE</code> ■ <code>task.TaskStatus.FAILED</code> ■ <code>task.TaskStatus.PENDING</code> ■ <code>task.TaskStatus.PROCESSING</code>
Type	string (read-only)
Module	N/task Module
Parent Object	task.QueryTaskStatus
Sibling Object Members	QueryTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.QueryTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQueryTask is an existing task.QueryTask object
4 var myTaskId = myQueryTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theStatus = myTaskStatus.status;
9 ...
10 // Add additional code

```

QueryTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of the submitted query task. This property references the same task ID that is returned by QueryTask.submit() when you submit the task for asynchronous processing.
-----------------------------	--

Type	string (read-only)
Module	N/task Module
Parent Object	task.QueryTaskStatus
Sibling Object Members	QueryTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.QueryTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myQueryTask is an existing task.QueryTask object
4 var myTaskId = myQueryTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.RecordActionTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The properties of a record action task. Use the methods and properties for this object to submit a record action task into the task queue and to execute it asynchronously.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	RecordActionTask Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...

```

```

2  var recordActionTask = task.create({
3      taskType: task.TaskType.RECORD_ACTION
4  });
5  recordActionTask.recordType = 'timebill';
6  recordActionTask.action = 'approve';
7
8  recordActionTask.params = [{recordId: 1, note: "this is a note for 1"},
9      {recordId: 5, note: "this is a note for 5"},
10     {recordId: 23, note: "this is a note for 23"}];
11
12  var handle = recordActionTask.submit();
13  var res = task.checkStatus({
14      taskId: handle
15  }); // returns a RecordActionTaskStatus object
16
17  log.debug('Initial status: ' + res.status);
18  ...
19  //Add additional code

```

RecordActionTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits a record action task script deployment for processing and returns its task ID. The record action task is processed by a background process which executes the specified record action for each record ID provided in the parameters. The overall task status as well as individual action results can be queried using the <code>task.checkStatus()</code> method.
Returns	string The task ID.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	50 units
Module	N/task Module
Sibling Object Members	RecordActionTask Object Members
Since	2019.1

Errors

Error Code	Message	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	Failed to submit job request: {reason}	The task cannot be submitted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code ...
```

```

2  var recordActionTask = task.create({
3      taskType: task.TaskType.RECORD_ACTION
4  });
5  recordActionTask.recordType = 'timebill';
6  recordActionTask.action = 'approve';
7  recordActionTask.params = [{recordId: 1, note: "this is a note for 1"},
8      {recordId: 5, note: "this is a note for 5"},
9      {recordId: 23, note: "this is a note for 23"}];
10
11  var handle = recordActionTask.submit();
12  var res = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
13
14  log.debug('Initial status: ' + res.status);
15  }); ...
16  //Add additional code

```

RecordActionTask.action

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the action to be invoked.
Type	string
Module	N/task Module
Sibling Object Members	RecordActionTask Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  //Add additional code ...
2  var recordActionTask = task.create({
3      taskType: task.TaskType.RECORD_ACTION
4  });
5  recordActionTask.recordType = 'timebill';
6  recordActionTask.action = 'approve';
7  recordActionTask.params = [{recordId: 1, note: "this is a note for 1"},
8      {recordId: 5, note: "this is a note for 5"},
9      {recordId: 23, note: "this is a note for 23"}];
10
11  var handle = recordActionTask.submit();
12  var res = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
13
14  log.debug('Initial status: ' + res.status);
15  }); ...
16  //Add additional code

```

RecordActionTask.condition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The condition used to select record IDs of records for which the action is to be executed.
-----------------------------	--

	This parameter is specified with the task.ActionCondition enum. This is used in conjunction with RecordActionTask.paramCallback. If RecordActionTask.paramCallback is not specified, this default callback is used: <code>function(v) { return { recordId: v }; }</code>.
Type	Object
Module	N/task Module
Sibling Object Members	RecordActionTask Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var recordActionTask = task.create({
3     taskType: task.TaskType.RECORD_ACTION
4 });
5 recordActionTask.recordType = 'timebill';
6 recordActionTask.action = 'approve';
7 recordActionTask.condition = task.ActionCondition.ALL_QUALIFIED_INSTANCES; recordActionTask.paramCallback = function(v){
8     return { recordId: v, note: "this is a note for " + v };
9 };
10 var handle = recordActionTask.submit();
11 ...
12 // Add additional code

```

RecordActionTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.RecordActionTaskStatus
Sibling Object Members	RecordActionTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task. (recordActionTask is a previously created task.RecordActionTask object)

```

```

4
5 var recordActionTaskId = recordActionTask.submit(); //recordActionTaskId is the unique task id
6 ...
7 //Add additional code

```

RecordActionTask.paramCallback

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Property of type function that takes record ID and returns the parameter object for the specified record ID. Is to be used in conjunction with task.ActionCondition . This parameter cannot be specified when RecordActionTask.params is specified.
Type	function
Governance	None
Module	N/task Module
Sibling Object Members	RecordActionTask Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var recordActionTask = task.create({
3   taskType: task.TaskType.RECORD_ACTION
4 });
5 recordActionTask.recordType = 'timebill';
6 recordActionTask.action = 'approve';
7 recordActionTask.condition = task.ActionCondition.ALL_QUALIFIED_INSTANCES; recordActionTask.paramCallback = function(v) {
8   return { recordId: v, note: "this is a note for " + v };
9 };
10
11 var handle = recordActionTask.submit();
12 ...
13 // Add additional code

```

RecordActionTask.params

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An array of parameter objects. Each object corresponds to one record ID of the record for which the action is to be executed. The object has the following form: {recordId: 1, someParam: 'example1', otherParam: 'example2'}
Type	Object[]
Module	N/task Module
Sibling Object Members	RecordActionTask Object Members

Since	2019.1
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var recordActionTask = task.create({
3     taskType: task.TaskType.RECORD_ACTION
4 });
5 recordActionTask.recordType = 'timebill';
6 recordActionTask.action = 'approve';
7 recordActionTask.params = [{recordId: 1, note: "this is a note for 1"},
8     {recordId: 5, note: "this is a note for 5"},
9     {recordId: 23, note: "this is a note for 23"}];
10
11 var handle = recordActionTask.submit();
12 var res = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object log.debug('Initial status: ' + res.status);
13 }); ...
14 //Add additional code

```

RecordActionTask.recordType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The record type on which the action is to be performed. For a list of record types, see record.Type .
Type	string
Module	N/task Module
Sibling Object Members	RecordActionTask Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var recordActionTask = task.create({
3     taskType: task.TaskType.RECORD_ACTION
4 });
5 recordActionTask.recordType = 'timebill';
6 recordActionTask.action = 'approve';
7 recordActionTask.params = [{recordId: 1, note: "this is a note for 1"},
8     {recordId: 5, note: "this is a note for 5"},
9     {recordId: 23, note: "this is a note for 23"}];
10
11 var handle = recordActionTask.submit();
12 var res = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object log.debug('Initial status: ' + res.status);
13 }); ...
14 //Add additional code

```

task.RecordActionTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the status of a record action task.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	RecordActionTaskStatus Object Members
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var recordActionTask = task.create({
3     taskType: task.TaskType.RECORD_ACTION
4 });
5 recordActionTask.recordType = 'timebill';
6 recordActionTask.action = 'approve';
7 recordActionTask.params = [{recordId: 1}, {recordId: 5}, {recordId: 23}];
8 var handle = recordActionTask.submit(); // Add any additional processing here
9 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object log.debug('Initial status: ' + taskStatus.status);
10 ...
11 //Add additional code
12
13 /* Example contents of a RecordActionTaskStatus object at different stages of bulk action task execution:
14 // initial status immediately after submitting the task
15 { status: 'PENDING', results: {}, errors: {}, complete: 0, succeeded: 0, failed: 0, pending: 3
16 }
17 // in the middle of processing, two records processed, one to go
18 { status: 'PROCESSING', results: { 1: { response: { approvedId: 1 }, notifications: [] }, 5: { response: { approvedId: 5 }, notifications: [{ title: 'Title', message: 'Message', severity: { value: 2, label: 'Warning' } } ] }, errors: {}, complete: 2, succeeded: 2, failed: 0, pending: 1
19 }
20 // complete, all successful
21 { status: 'COMPLETE', results: { 1: { response: { approvedId: 1 }, notifications: [] }, 5: { response: { approvedId: 5 }, notifications: [{ title: 'Title', message: 'Message', severity: { value: 2, label: 'Warning' } } ] }, errors: {}, complete: 3, succeeded: 3, failed: 0, pending: 0
22 }
23 // complete, one action returned an error
24 { status: 'COMPLETE', results: { 1: { response: { approvedId: 1 }, notifications: [] }, 5: { response: { approvedId: 5 }, notifications: [{ title: 'Title', message: 'Message', severity: { value: 2, label: 'Warning' } } ] }, errors: { 23: { name: 'SSS_RECORD_DOES_NOT_SATISFY_CONDITION', message: ... } }, complete: 3, succeeded: 2, failed: 1, pending: 0
25 }
26 */ ...
27 //Add additional code

```

RecordActionTaskStatus.complete

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of record actions that are already executed, either failed or successful.
-----------------------------	--

Type	number (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();
10 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
11
12 log.debug('Actions already complete: ' + taskStatus.complete); // will log, for example, the
13     following: // Actions already complete: 2
14 ...
15 // Add additional code

```

RecordActionTaskStatus.errors

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The error details of failed action executions. The value of the property is the record instance ID and the corresponding error details. The error details are returned in an unnamed object with two properties: code and message.
Type	Object (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();
10 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
11
12 log.debug(taskStatus.errors); // will log, for example, the following: // { 2: { name:
13     'SSS_RECORD_DOES_NOT_SATISFY_CONDITION', message: '...' } }
14 ...
15 // Add additional code

```

RecordActionTaskStatus.failed

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of record actions with a failed status.
Type	number (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();

```

```

10 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
11
12 log.debug('Actions failed: ' + taskStatus.failed); // will log, for example, the following: Actions failed: 0
13 ...
14 // Add additional code

```

RecordActionTaskStatus.pending

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of record actions with a pending status.
Type	number (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();
10 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
11
12 log.debug('Actions pending: ' + taskStatus.pending); // will log, for example, the following:
13 // Actions pending: 2
14 ...
15 // Add additional code

```

RecordActionTaskStatus.results

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The results of successfully executed record action tasks. The value of the property is the task instance ID and the corresponding action result.
-----------------------------	--

Type	Object (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();
10 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
11
12 log.debug(taskStatus.results); // will log, for example, the following: [{ 1: { response: {
13     // approved: true }, notifications: []
14     ...
15 // Add additional code

```

RecordActionTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Represents the record action task status. Returns a value from the task.TaskStatus enum.
Type	string (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();
10 var taskStatus = task.checkStatus({taskId: handle}); // returns a RecordActionTaskStatus object
11
12 log.debug('Current task status: ' + taskStatus.status); // will log, for example, the following:
13     // Current task status: PENDING
14 ...
15 // Add additional code

```

RecordActionTaskStatus.succeeded

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of record actions with a successful status.
Type	number (read-only)
Module	N/task Module
Sibling Object Members	RecordActionTaskStatus Object Members
Since	2019.1

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var recordActionTask = task.create({
4     taskType: task.TaskType.RECORD_ACTION
5 });
6 recordActionTask.recordType = 'timebill';
7 recordActionTask.action = 'approve';
8 recordActionTask.params = [{recordId: 1}, {recordId: 2}];
9 var handle = recordActionTask.submit();
10 var taskStatus = task.checkStatus({
11     taskId: handle
12 }); // returns a RecordActionTaskStatus object
13

```

```

14 log.debug('Actions executed successfully: ' + taskStatus.succeeded); // will log, for example, the
15 // following: Actions executed successfully: 1
16 ...
17 // Add additional code

```

RecordActionTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The task ID associated with the specified task.
Type	string (read-only)
Module	N/task Module
Parent Object	task.RecordActionTaskStatus
Sibling Object Members	RecordActionTaskStatus Object Members
Since	

Errors

Error Code	Error Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myRecordActionTask is an existing task.RecordActionTask object
4 var myTaskId = myRecordActionTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskStatusId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.ScheduledScriptTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	All the properties of scheduled script task in SuiteScript. Use this object to place a scheduled script deployment into the NetSuite scheduling queue. To use the ScheduledScriptTask Object: 1. In the NetSuite UI, create the script record and script deployment record. 2. Use task.create(options) to create the ScheduledScriptTask object. 3. Use the ScheduledScriptTask object properties to set the script and deployment properties.
---------------------------	---

	4. Use ScheduledScriptTask.submit() to deploy the scheduled script to the NetSuite scheduling queue. 5. Use the properties for the task.ScheduledScriptTaskStatus object to get the status of the scheduled script. For more information about scheduled scripts in NetSuite, see the help topic SuiteScript 2.x Scheduled Script Type.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	ScheduledScriptTask Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var scriptTask = task.create({
4     taskType: task.TaskType.SCHEDULED_SCRIPT
5 });
6 scriptTask.scriptId = 1234;
7 scriptTask.deploymentId = 'customdeploy1';
8 scriptTask.params = {searchId: 'custsearch_456'};
9 var scriptTaskId = scriptTask.submit();
10 ...
11 //Add additional code

```

ScheduledScriptTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Directs NetSuite to place a scheduled script deployment into the NetSuite scheduling queue and returns a unique ID for the task. Additionally, note the following: ■ The scheduled script must have a status of Not Scheduled on the Script Deployment page. If the script status is set to Testing on the Script Deployment page, this method will not place the script into the scheduling queue. ■ If the deployment status on the Script Deployment page is set to Scheduled, the script will be placed into the queue according to the time(s) specified on the Script Deployment page. If submitted scheduled scripts are requested at the same time or rapidly in succession, the scheduled tasks may not proceed to the Queued status. ■ Only roles with the SuiteScript Scheduling permission or Administrators can run scheduled scripts executed by API. If a user event script calls <code>ScheduledScriptTask.submit()</code>, the user event script must be initiated by a role with permission. ■ A scheduled script can be submitted for processing only if there is no unfinished scheduled script task for the same script ID and script deployment ID. Therefore, if a scheduled script resubmits itself, the resubmit does not occur until the current execution completes. This delay is necessary to avoid the existence of two unfinished tasks for the same deployment of
---------------------------	---

	the same script. For this reason, if a scheduled script uses the submit() method to resubmit itself, then at runtime, no task ID is returned when the scheduled script is submitted.
Returns	string The task ID as a string, except as noted above.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	Failed to submit job request: {reason}	Task cannot be submitted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var scheduledScriptTaskId = scriptTask.submit();
4 ...
5 //Add additional code

```

ScheduledScriptTask.deploymentId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Script ID (as a string), for the script deployment record associated with a task.ScheduledScriptTask Object.
Type	string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code .
2 ..
3 scheduledTask.deploymentId = custdeploy1;

```

```

4 | ...
5 | //Add additional code

```

ScheduledScriptTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.ScheduledScriptTask
Sibling Object Members	ScheduledScriptTask Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | // A unique task id for the task is returned when you submit the task. (scheduledScriptTask is a previously created task.ScheduledScriptTask object)
4 |
5 | var scheduledScriptTaskId = scheduledScriptTask.submit(); //scheduledScriptTaskId is the unique task id
6 | ...
7 | //Add additional code

```

ScheduledScriptTask.params

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Object with key-value pairs that override static script parameter field values on the script deployment. Use these parameters for the task.ScheduledScriptTask object to programmatically pass values to the script deployment. For more information about script parameters, see the help topic Creating Script Parameters Overview .
Type	Object
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code

```

```

2  ...
3  scriptTask.params = {searchId: 'custsearch_456'};
4  ...
5  //Add additional code

```

ScheduledScriptTask.scriptId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID (as a number), or script ID (as a string), for the script record associated with a task.ScheduledScriptTask object.
Type	number string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var scheduledScriptId = 34;
4  ...
5  //Add additional code

```

task.ScheduledScriptTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The properties and status of a scheduled script placed into the NetSuite scheduling queue. Use task.checkStatus(options) with the unique ID for the scheduled script task to get the ScheduledScriptTaskStatus Object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	ScheduledScriptTaskStatus Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  //Add additional code

```

```

2 | ...
3 | var res = task.checkStatus(scriptTaskId);
4 | log.debug('Initial status: ' + res.status);
5 | ...
6 | //Add additional code

```

ScheduledScriptTaskStatus.deploymentId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Script ID for a script deployment record associated with a specific task.ScheduledScriptTask Object. Use this ID to get more details about the script deployment record for the scheduled task.
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code ...
2 | log.audit('Deployment ID: ' + status.deploymentId)
3 | ...
4 | //Add additional code

```

ScheduledScriptTaskStatus.scriptId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID for a script record associated with a specific task.ScheduledScriptTask Object. Use this ID to get more details about the script record for the scheduled task.
Type	number (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 log.audit('Initial status: ' + status.scriptId);
4 ...
5 //Add additional code

```

ScheduledScriptTaskStatus.status

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status for a scheduled script task. Returns a task.TaskStatus enum value.
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 log.audit({
3     details: 'Status: ' + summary.status
4 });
5 ...
6 //Add additional code

```

ScheduledScriptTaskStatus.taskId

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The task ID associated with the specified task.
-----------------------------	---

Type	string (read-only)
Module	N/task Module
Parent Object	task.ScheduledScriptTaskStatus
Sibling Object Members	ScheduledScriptTaskStatus Object Members
Since	—

Errors

Error Code	Error Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myScheduledScriptTask is an existing task.ScheduledScriptTask object
4 var myTaskId = myScheduledScriptTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskStatusId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.SearchTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The properties of a search task. Use the methods and properties for this object to submit a search task into the task queue, execute it asynchronously, and persist search results. Similar to SuiteAnalytics persisted search functionality, this capability is useful for searches across high volumes of data. You can create a <code>task.SearchTask</code> object using task.create(options). Use the <code>task.SearchTask</code> object to do the following: - Set the search ID using the SearchTask.savedSearchId property. - Set the file ID or file path of a CSV file in the File Cabinet. Search results are exported to this file. Use the SearchTask.fileId property or the SearchTask.filePath property. Exactly one of these properties must be set. If both are set, an error occurs. - Add dependent scripts to the search task using SearchTask.addInboundDependency(). Dependent scripts are processed automatically when the search task is complete. - Submit the search task to the NetSuite task queue using SearchTask.submit(). - Get the status of a search task using the properties of the task.SearchTaskStatus object.
---------------------------	---

	 Note: There is a limit to the number of asynchronous searches running at the same time. The limit is set to be the same as the limit for CSV import. The file size limit is based on File Cabinet limits.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	SearchTask Object Members
Since	2017.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var searchTask = task.create({
3     taskType: task.TaskType.SEARCH
4 });
5 searchTask.savedSearchId = 51;
6 var path = 'ExportFolder/export.csv';
7 searchTask.filePath = path;
8 var searchTaskId = searchTask.submit();
9 ...
10 //Add additional code

```

SearchTask.addInboundDependency()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a scheduled script task (task.ScheduledScriptTask) or map/reduce script task (task.MapReduceScriptTask) to the search task as a dependent script. Dependent scripts are processed automatically when the search task is complete. For more information, see the help topic SuiteCloud Processors.  Note: You can add only scheduled scripts or map/reduce scripts as dependent scripts to asynchronous search tasks. Other script types are not supported. When you use this method to add a dependent script, the script is considered an inbound dependency of the search task. The added script depends on the search task. For example, if you add a scheduled script task to a search task as a dependent script, the scheduled script depends on the search task. Because <code>addInboundDependency()</code> is called on the search task, any dependent scripts that you add are considered inbound dependencies.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/task Module

Since	2018.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
dependentScript	task.ScheduledScriptTask task.MapReduceScriptTask	required	The script to add as a dependent script to the search task. Use task.create(options) and the task.TaskType enum to create a script task with a type of SCHEDULED_SCRIPT or MAP_REDUCE. This script task is a task.ScheduledScriptTask object or a task.MapReduceScriptTask, and you can add this script task as a dependent script to the search task. The dependent script is processed when the search task is complete. You can add only one dependent script per call to SearchTask.addInboundDependency().	2018.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var scheduledScript = task.create({
4     taskType: task.TaskType.SCHEDULED_SCRIPT
5 });
6 // Set the properties of the scheduled script task
7 scheduledScript.scriptId = 'customscript_as_ftr_ss';
8
9 var asyncTask = task.create({
10     taskType: task.TaskType.SEARCH
11 });
12 // Set the properties of the search task
13 asyncTask.savedSearchId = 'customsearch35';
14
15 asyncTask.addInboundDependency(scheduledScript);
16 var asyncTaskId = asyncTask.submit();
17 ...
18 // Add additional code

```

SearchTask.submit()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Directs NetSuite to initiate the asynchronous search task and return a unique ID for the task. When the submission is successful, this method adds the internal IDs of any dependent scripts (added using SearchTask.addInboundDependency()) to the SearchTask.inboundDependencies property. Use task.SearchTaskStatus to view the status of a submitted task.
---------------------------	--

Returns	string The task ID
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	100 units
Module	N/task Module
Since	2017.1

Errors

Error Code	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	The task cannot be submitted.
SSS_MISSING_REQD_ARGUMENT	A required parameter is missing.
YOU_DO_NOT_HAVE_ACCESS_TO_THE_MEDIA_ITEM_YOU_SELECTED	You do not have permission to access the file.
THAT_RECORD_DOES_NOT_EXIST	The file Object references a file that doesn't exist.
MUST_IDENTIFY_A_FILE	The path specifies a folder and not a file.
CANNOT_RESUBMIT_SUBMITTED_ASYNC_SEARCH_TASK	The search task was already submitted and completed successfully.
ASYNC_SEARCH_DEPENDENCY_MR_ALREADY_SUBMITTED	A dependent map/reduce script is already submitted and is not complete.
ASYNC_SEARCH_DEPENDENCY_MR_INCORRECT_STATUS	The status of the deployment record for the specified dependent map/reduce script has a value other than "Not Scheduled".
ASYNC_SEARCH_DEPENDENCY_SS_ALREADY_SUBMITTED	A dependent scheduled script is already submitted and is not complete.
ASYNC_SEARCH_DEPENDENCY_SS_INCORRECT_STATUS	The status of the deployment record for the specified dependent scheduled script has a value other than "Not Scheduled".
ASYNC_SEARCH_DEPLOYMENT_FOR_DEPENDENCY	A deployment record for the specified dependent script is not available for one of the following reasons: ■ A deployment record was not specified when the dependent script was created, and automatic lookup for an available deployment record failed. ■ The deployment record specified when the dependent script was created is not found.
ASYNC_SEARCH_MULTIPLE_DEPENDENCIES	The same dependent script is passed to this method more than one time.
ASYNC_SEARCH_SCRIPT_ID_NOT_FOUND	The specified dependent script is not found.
ASYNC_SEARCH_SEARCH_ID_NOT_FOUND	The search task with the specified search ID is not found.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var searchTriggerTask = searchTask.submit();
4 ...
5 //Add additional code
```

SearchTask.fileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of the CSV file to export search results into. Note: Either this property or the SearchTask.filePath property must be set. If both are set, an error occurs.
Type	number
Module	N/task Module
Since	2017.1

Errors

Error Code	Thrown If
UNSUPPORTED_COMBINATION_OF_PARAMETERS	Both this property and the SearchTask.filePath property are set at the same time.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 searchTask.fileId = 18;
4 ...
5 //Add additional code
```

SearchTask.filePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Path of the CSV file to export search results into.
-----------------------------	---

	 Note: Either this property or the SearchTask.fileId property must be set. If both are set, an error occurs.
Type	string
Module	N/task Module
Since	2017.1

Errors

Error Code	Thrown If
UNSUPPORTED_COMBINATION_OF_PARAMETERS	Both this property and the SearchTask.fileId property are set at the same time.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 searchTask.filePath= 'ExportFolder/export.csv'
4 ...
5 //Add additional code

```

SearchTask.id

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.SearchTask
Sibling Object Members	SearchTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task. (searchTask is a previously created task.SearchTask object)
4
5 var searchTaskId = searchTask.submit(); //searchTaskId is the unique task id
6 ...
7 //Add additional code

```

SearchTask.inboundDependencies

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Key-value pairs that contain information about the dependent scripts added to the search task. Use this property to verify the properties of dependent scripts after you add the scripts using SearchTask.addInboundDependency(). This property uses nested objects to store information about each dependent script. A nested object is included for each dependent script added to the search task. The nested object contains information such as the task type, script ID, and deployment ID. It also includes the index of the script (starting at 0). Dependent scripts are indexed in the order they are added to the search task. For example, consider a situation in which you add a scheduled script task and a map/reduce script task to a search task as dependent scripts. After you add the dependent scripts, but before you submit the search task using SearchTask.submit(), the value of the <code>SearchTask.inboundDependencies</code> property is similar to the following: <pre> 1 { "0": { "type": "task.ScheduledScriptTask", "scriptId": "customscript_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": { "custscript_ss_as_srch_res": "SuiteScripts/ExportFile.csv" } }, 2 "1": { "type": "task.MapReduceScriptTask", "scriptId": "customscript_as_ftr_mr", "deploymentId": "customdeploy_mr_dpl", "params": { "custscript_mr_as_srch_res": "SuiteScripts/ExportFile.csv" } } 3 } </pre> After you submit the search task, the internal IDs of the dependent scripts are added to the <code>SearchTask.inboundDependencies</code> property: <pre> 1 { "0": { "type": "task.ScheduledScriptTask", "id": "SCHEDSCRIPT_0168697b126d1705061d0d690a78775500b046a1912686b10_349d94266564827c739a2ba0a5b9d476f4097217", "scriptId": "customscript_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": { "custscript_ss_as_srch_res": "SuiteScripts/ExportFile.csv" } }, 2 "1": { "type": "task.MapReduceScriptTask", "id": "MAPREDUCETASK_0268697b126d1705061d0d69027f7b39560f01001c_7a02acb4bdebf0103120b09302170720aa57bca4", "scriptId": "customscript_as_ftr_mr", "deploymentId": "customdeploy_mr_dpl", "params": { "custscript_mr_as_srch_res": "SuiteScripts/ExportFile.csv" } } 3 } </pre>
Type	Object[] (read-only)
Module	N/task Module
Since	2018.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | // Add additional code
2 | ...
3 | var scheduledScript = task.create({
4 |     taskType: task.TaskType.SCHEDULED_SCRIPT
5 | });
6 | // Set the properties of the scheduled script task

```

```

7  scheduledScript.scriptId = 'customscript_as_ftr_ss';
8  var mapReduceScript = task.create({
9      taskType: task.TaskType.MAP_REDUCE
10 });
11 // Set the properties of the map/reduce script task
12 mapReduceScript.scriptId = 'customscript_as_ftr_mr';
13
14 asyncTask.addInboundDependency(scheduledScript);
15 asyncTask.addInboundDependency(mapReduceScript);
16
17 var asyncTaskId = asyncTask.submit();
18
19 // Iterate over the dependent scripts
20 var p = asyncTask.inboundDependencies;
21 for (var key in p) { log.debug(key + ' > ' + p[key]);
22 ...
23 // Add additional code

```

SearchTask.savedSearchId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of the saved search to be executed during the task.
Type	number
Module	N/task Module
Since	2017.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 searchTask.savedSearchId = 51;
4 ...
5 //Add additional code

```

task.SearchTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The status of an asynchronous search task (task.SearchTask) placed into the NetSuite task queue. To initiate the task and retrieve the task ID, use SearchTask.submit().  Important: Ensure that you are familiar with script execution time limits when creating tasks. Each server script type is limited to an amount of time it can run on a single execution. For more information, see the time limit chart on Script Execution Time Limits.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/task Module
Methods and Properties	SearchTaskStatus Object Members
Since	2017.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var searchTaskStatus = task.checkStatus({
3     taskId: 51
4 });
5
6 if (searchTaskStatus.status === task.TaskStatus.FAILED) { // Handle the task failure }
7 ...
8 // Add additional code

```

SearchTaskStatus.fileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of CSV file into which search results are exported.
Type	number (read-only)
Module	N/task Module
Since	2017.1

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var status = task.checkStatus({
3     searchTaskId: 81
4 });
5
6 log.audit({
7     title: 'File ID',
8     details: status.fileId
9 });
10 ...
11 //Add additional code

```

SearchTaskStatus.savedSearchId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the saved search executed during the task.
Type	number (read-only)
Module	N/task Module
Since	2017.1

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code ...
2 var status = task.checkStatus({
3     searchTaskId: 81
4 });
5
6 log.audit({
7     title: 'Saved Search ID',
8     details: status.savedSearchId
9 });
10 ...
11 //Add additional code

```

SearchTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status for an asynchronous search placed in the NetSuite task queue by SearchTask.submit() . Returns a task.TaskStatus enum value.
Type	string (read-only)
Module	N/task Module
Since	2017.1

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code ...
2 var summary = task.checkStatus(scriptTaskId);
3 log.audit({
4     title: 'Status',
5     details: summary.status
6 }); ...
7 //Add additional code
```

SearchTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of the task.SearchTask Object. Use SearchTask.submit() to return this ID.
Type	string (read-only)
Module	N/task Module
Since	2017.1

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // mySearchTask is an existing task.SearchTask object
4 var myTaskId = mySearchTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskStatusId = myTaskStatus.taskId;
9 ...
10 // Add additional code
```

task.SuiteQLTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An asynchronous SuiteQL task. Use the methods and properties for this object to submit a SuiteQL task to the NetSuite task queue and execute it asynchronously.
---------------------------	---

	You can create a task.SuiteQLTask object using task.create(options). You can populate the properties of this object as you create it. With a task.SuiteQLTask object, you can do the following: ■ Specify the SuiteQL query to submit using the SuiteQLTask.query property. This property accepts a string that represents a SuiteQL query. For more information, see the help topic SuiteQL. ■ Provide parameters to use in the SuiteQL query using the SuiteQLTask.params property. ■ Set the file ID or file path of a CSV file in the File Cabinet. SuiteQL query results are exported to this file. Use the SuiteQLTask.fileId property to specify a file ID, or use the SuiteQLTask.filePath property to specify a file path. Only one of these properties can be set at the same time. If both are set, an error occurs. ■ Add dependent tasks to the SuiteQL task using SuiteQLTask.addInboundDependency(options). Dependent tasks are processed automatically when the SuiteQL task is complete. ■ Submit the SuiteQL task to the task queue using SuiteQLTask.submit(). ■ Get the status of a SuiteQL task using the properties of an associated task.SuiteQLTaskStatus object. The task.checkStatus(options) method returns a task.SuiteQLTaskStatus object when you specify a task ID that represents a SuiteQL task.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	SuiteQLTask Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySuiteQLTask = task.create({
4     taskType: task.TaskType.SUITE_QL
5 });
6 ...
7 // Add additional code

```

SuiteQLTask.addInboundDependency(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a scheduled script task or map/reduce script task as a dependent task to the SuiteQL task. Dependent tasks are processed automatically using SuiteCloud Processors when the SuiteQL task is complete. For more information, see the help topic SuiteCloud Processors. You can add a dependent task in the following ways: ■ Provide a task.ScheduledScriptTask object or task.MapReduceScriptTask object that represents the dependent task as the only parameter to this method. ■ Provide an Object as the only parameter that includes values for the following properties:
---------------------------	--

	- taskType - scriptId - deploymentId (optional) - params (optional) For descriptions of these properties, see Parameters.  Important: You can add only scheduled scripts or map/reduce scripts as dependent tasks to asynchronous SuiteQL tasks. Other script types are not supported. You can add only one dependent task per call to <code>SuiteQLTask.addInboundDependency(options)</code>. When you use this method to add a dependent task, the added task is considered an inbound dependency of the SuiteQL task. For example, if you add a scheduled script task to a SuiteQL task as a dependent task, the scheduled script depends on the SuiteQL task. Because <code>SuiteQLTask.addInboundDependency(options)</code> is called on the SuiteQL task, any dependent task that you add is considered an inbound dependency.
Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/task Module
Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.scriptId	string	required	The script ID of the scheduled script record or map/reduce script record for the dependent task.
options.taskType	string	required	The type of dependent task. This property uses one of the following values in the task.TaskType enum: - task.TaskType.SCHEDULED_SCRIPT - task.TaskType.MAP_REDUCE
options.deploymentId	string	optional	The script ID of the script deployment record for the dependent task. To use this property, the scheduled script or map/reduce script for the dependent task must be deployed. If you do not specify a value for this property, an available deployment is selected automatically.

Parameter	Type	Required / Optional	Description
options.params	Object	optional	The parameters for the scheduled script or map/reduce script. To use this property, you must first create script parameters that are required for the dependent task. For example, if your dependent task requires a reference to the CSV result file, you must create a script parameter that maps the CSV result file ID to the file name, as in the following example: <pre>1 { 'custscript_query_res' : 'resultFile.csv' 2 }</pre> For more information about script parameters, see the help topic Creating Script Parameters (Custom Fields).

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // Be sure to use a script ID that represents a scheduled script in your account
4 var myScheduledScript = task.create({
5     taskType: task.TaskType.SCHEDULED_SCRIPT,
6     scriptId: 'customscript_as_ftr_ss'
7 });
8
9 // mySuiteQLTask is an existing task.SuiteQLTask object
10 mySuiteQLTask.addInboundDependency(myScheduledScript);
11 ...
12 // Add additional code
```

SuiteQLTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits the SuiteQL task for asynchronous processing. This method returns a unique ID for the SuiteQL task. When the submission is successful, this method adds the script IDs of any dependent tasks (added using SuiteQLTask.addInboundDependency(options)) to the SuiteQLTask.inboundDependencies property. Use the task.SuiteQLTaskStatus object to view the status of a submitted SuiteQL task. The task.checkStatus(options) method returns a task.SuiteQLTaskStatus object when you specify a task ID that represents a SuiteQL task.
Returns	string
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	100 units
Module	N/task Module

Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
ASYNC_SUITEQL_DEPENDENCY_MR_ALREADY_SUBMITTED	A dependent map/reduce script task is already submitted and is not complete.
ASYNC_SUITEQL_DEPENDENCY_MR_INCORRECT_STATUS	The status of the script deployment record for the specified dependent map/reduce script task has a value other than "Not Scheduled".
ASYNC_SUITEQL_DEPENDENCY_SS_ALREADY_SUBMITTED	A dependent scheduled script task is already submitted and is not complete.
ASYNC_SUITEQL_DEPENDENCY_SS_INCORRECT_STATUS	The status of the script deployment record for the specified dependent scheduled script task has a value other than "Not Scheduled".
ASYNC_SUITEQL_DEPLOYMENT_FOR_DEPENDENCY	A script deployment record for the specified dependent script task is not available for one of the following reasons: ■ A script deployment record was not specified when the dependent task was created, and automatic lookup for an available script deployment record failed. ■ The script deployment record specified when the dependent task was created is not found.
ASYNC_SUITEQL_MULTIPLE_DEPENDENCIES	The same dependent task is passed to this method more than one time.
ASYNC_SUITEQL_SCRIPT_ID_NOT_FOUND	The specified dependent task is not found.
ASYNC_SUITEQL_SUITEQL_ID_NOT_FOUND	A SuiteQL task with the specified script ID is not found.
CANNOT_RESUBMIT_SUBMITTED_ASYNC_SUITEQL_TASK	The SuiteQL task was already submitted and completed successfully.
FAILED_TO_SUBMIT_JOB_REQUEST_1	The SuiteQL task cannot be submitted due to an unexpected error.
MUST_IDENTIFY_A_FILE	The SuiteQLTask.filePath property specifies a folder and not a file.
SSS_MISSING_REQD_ARGUMENT	A required property is not specified.
THAT_RECORD_DOES_NOT_EXIST	The SuiteQLTask.fileId property or SuiteQLTask.filePath property references a file that does not exist.
YOU_DO_NOT_HAVE_ACCESS_TO_THE_MEDIA_ITEM_YOU_SELECTED	You do not have permission to access the file specified by the SuiteQLTask.fileId property or SuiteQLTask.filePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySuiteQLTask = task.create({
4     taskType: task.TaskType.SUITE_QL
5 });
6
7 // Submit the task
8 var myTaskId = mySuiteQLTask.submit();
9
10 // Check the task status
11 var myStatus = task.checkStatus({
12     taskId: myTaskId
13 });
14 ...
15 // Add additional code

```

SuiteQLTask.fileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID of the CSV file to export SuiteQL query results to. The CSV file must be located in the File Cabinet. When you create a SuiteQL task using task.create(options) , you can provide a value for this property or the SuiteQLTask.filePath property. Only one of these properties can be set at the same time. If both are set, an error occurs.
Type	number
Module	N/task Module
Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You specify values for both SuiteQLTask.fileId and SuiteQLTask.filePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var mySuiteQLTask = task.create({
4     taskType: task.TaskType.SUITE_QL,

```



```

5     fileId: 18
6   });
7   ...
8   // Add additional code

```

SuiteQLTask.filePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Path of the CSV file to export SuiteQL query results to. The CSV file must be located in the File Cabinet. When you create a SuiteQL task using task.create(options) , you can provide a value for this property or the SuiteQLTask.fileId property. Only one of these properties can be set at the same time. If both are set, an error occurs.
Type	string
Module	N/task Module
Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You specify values for both SuiteQLTask.fileId and SuiteQLTask.filePath .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1   // Add additional code
2   ...
3   var mySuiteQLTask = task.create({
4     taskType: task.TaskType.SUITE_QL,
5     filePath: 'ExportFolder/export.csv'
6   });
7   ...
8   // Add additional code

```

SuiteQLTask.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module

Parent Object	task.SuiteQLTask
Sibling Object Members	QueryTask Object Members
Since	—

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 // A unique task id for the task is returned when you submit the task. (suiteQLTask is a previously created task.SuiteQLTask object)
4
5 var suiteQLTaskId = suiteQLTask.submit(); //suiteQLTaskId is the unique task id
6 ...
7 //Add additional code

```

SuiteQLTask.inboundDependencies

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Key-value pairs that contain information about the dependent tasks added to the SuiteQL task. Use this property to verify the properties of dependent tasks after you add the tasks using SuiteQLTask.addInboundDependency(options). This property uses nested objects to store information about each dependent task. A nested object is included for each dependent task added to the SuiteQL task, and these objects are referenced by their index (starting at 0). Dependent tasks are indexed in the order they are added to the SuiteQL task. Each nested object contains the task type, script ID, script deployment ID, and script parameters. For example, consider a situation in which you add a scheduled script task and a map/reduce script task to a SuiteQL task as dependent tasks. After you add the dependent tasks but before you submit the SuiteQL task using QueryTask.submit(), the value of the <code>SuiteQLTask.inboundDependencies</code> property is similar to the following: <pre> 1 { "0": { "type": "task.ScheduledScriptTask", "scriptId": "customscript_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": { "custscript_ss_as_srch_res": "SuiteScripts/ExportFile.csv" } }, 2 "1": { "type": "task.MapReduceScriptTask", "scriptId": "customscript_as_ftr_mr", "deploymentId": "customdeploy_mr_dpl", "params": { "custscript_mr_as_srch_res": "SuiteScripts/ExportFile.csv" } } 3 } </pre> After you submit the SuiteQL task, the IDs of the dependent scripts are added to the <code>SuiteQLTask.inboundDependencies</code> property: <pre> 1 { "0": { "type": "task.ScheduledScriptTask", "id": "SCHEDSCRIP T_0168697b126d1705061d0d690a787755500b046a1912686b10_349d94266564827c739a2ba0a5b9d476f4097217", "scriptId": "custom script_as_ftr_ss", "deploymentId": "customdeploy_ss_dpl", "params": { "custscript_ss_as_srch_res": "SuiteScripts/Export File.csv" } }, 2 "1": { "type": "task.MapReduceScriptTask", "id": "MAPREDUCETASK_0268697b126d1705061d0d69027f7b39560f01001c_7a02acb4b debf0103120b09302170720aa57bca4", "scriptId": "customscript_as_ftr_mr", "deploymentId": "customdeploy_mr_dpl", "params": { "custscript_mr_as_srch_res": "SuiteScripts/ExportFile.csv" } } 3 } </pre>
Type	Object[] (read-only)
Module	N/task Module

Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.SuiteQLTask object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // Be sure to use a script ID that represents a scheduled script in your account
4 var myScheduledScript = task.create({
5     taskType: task.TaskType.SCHEDULED_SCRIPT,
6     scriptId: 'customscript_as_ftr_ss'
7 });
8
9 // mySuiteQLTask is an existing task.SuiteQLTask object
10 mySuiteQLTask.addInboundDependency(myScheduledScript);
11 ...
12 // Add additional code

```

SuiteQLTask.params

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Parameters for the SuiteQL query. The specified parameters are substituted into the SuiteQL query string (represented by SuiteQLTask.query) when you submit the SuiteQL task using SuiteQLTask.submit() .
Type	Array<string boolean number>
Module	N/task Module
Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code

```

```

2  ...
3  var mySuiteQLTask = task.create({
4      taskType: task.TaskType.SUITE_QL
5  });
6  mySuiteQLTask.params = ['myStringParameter', true];
7  ...
8  // Add additional code

```

SuiteQLTask.query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	SuiteQL query definition for the SuiteQL task. The SuiteQL query definition is a string that represents a SuiteQL query. For more information, see the help topic SuiteQL .
Type	string
Module	N/task Module
Parent Object	task.SuiteQLTask
Sibling Object Members	SuiteQLTask Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  // Add additional code
2  ...
3  var mySuiteQLTask = task.create({
4      taskType: task.TaskType.SUITE_QL
5  });
6  mySuiteQLTask.query = 'SELECT customer.entityid, customer.email FROM customer';
7  ...
8  // Add additional code

```

task.SuiteQLTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The status of an asynchronous SuiteQL task (task.SuiteQLTask) in the NetSuite task queue. You create a task.SuiteQLTaskStatus object using task.checkStatus(options) . To check the status of a SuiteQL task, you must provide the task ID of the SuiteQL task. To submit a SuiteQL task and obtain the task ID, use SuiteQLTask.submit() .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module

Methods and Properties	SuiteQLTaskStatus Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // mySuiteQLTask is an existing task.SuiteQLTask object
4 var myTaskId = mySuiteQLTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 ...
9 // Add additional code

```

SuiteQLTaskStatus.fileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID of the CSV file that SuiteQL query results are exported to. This property references the same CSV file that is specified by the SuiteQLTask.fileId property or SuiteQLTask.filePath property in the corresponding task.SuiteQLTask object.
Type	number (read-only)
Module	N/task Module
Parent Object	task.SuiteQLTaskStatus
Sibling Object Members	SuiteQLTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.SuiteQLTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // mySuiteQLTask is an existing task.SuiteQLTask object
4 var myTaskId = mySuiteQLTask.submit();
5 var myTaskStatus = task.checkStatus({

```

```

6      taskId: myTaskId
7    });
8    var theFileId = myTaskStatus.fileId;
9    ...
10   // Add additional code

```

SuiteQLTaskStatus.params

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Parameters for the SuiteQL query. The specified parameters are substituted into the SuiteQL query string (represented by SuiteQLTask.query) when you submit the SuiteQL task using SuiteQLTask.submit(). This property references the same set of parameters that are specified by the SuiteQLTask.params property in the corresponding task.SuiteQLTask object.
Type	Array<string boolean number> (read-only)
Module	N/task Module
Parent Object	task.SuiteQLTaskStatus
Sibling Object Members	SuiteQLTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.SuiteQLTaskStatus object is created.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1   // Add additional code
2   ...
3   // mySuiteQLTask is an existing task.SuiteQLTask object
4   var myTaskId = mySuiteQLTask.submit();
5   var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7   });
8   var theParams = myTaskStatus.params;
9   ...
10  // Add additional code

```

SuiteQLTaskStatus.query

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	SuiteQL query definition for the SuiteQL task.
-----------------------------	--

	The SuiteQL query definition is a string that represents a SuiteQL query. For more information, see the help topic SuiteQL. This property references the same query that is specified by the SuiteQLTask.query property in the corresponding task.SuiteQLTask object.
Type	string (read-only)
Module	N/task Module
Parent Object	task.SuiteQLTaskStatus
Sibling Object Members	SuiteQLTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.SuiteQLTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // mySuiteQLTask is an existing task.SuiteQLTask object
4 var myTaskId = mySuiteQLTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theQuery = myTaskStatus.query;
9 ...
10 // Add additional code

```

SuiteQLTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status of the SuiteQL task. A SuiteQL task can have the following statuses, which are represented by values in the task.TaskStatus enum: ■ <code>task.TaskStatus.COMPLETE</code> ■ <code>task.TaskStatus.FAILED</code> ■ <code>task.TaskStatus.PENDING</code> ■ <code>task.TaskStatus.PROCESSING</code>
Type	string (read-only)
Module	N/task Module

Parent Object	task.SuiteQLTaskStatus
Sibling Object Members	SuiteQLTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.SuiteQLTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // mySuiteQLTask is an existing task.SuiteQLTask object
4 var myTaskId = mySuiteQLTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theStatus = myTaskStatus.status;
9 ...
10 // Add additional code

```

SuiteQLTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	ID of the submitted SuiteQL task. This property references the same task ID that is returned by SuiteQLTask.submit() when you submit the task for asynchronous processing.
Type	string (read-only)
Module	N/task Module
Parent Object	task.SuiteQLTaskStatus
Sibling Object Members	SuiteQLTaskStatus Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the task.SuiteQLTaskStatus object is created.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // mySuiteQLTask is an existing task.SuiteQLTask object
4 var myTaskId = mySuiteQLTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.WorkflowTriggerTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	All the properties required to asynchronously initiate a workflow. Use the WorkflowTriggerTask Object to create a task that initiates an instance of the specified workflow. The task is placed in the scheduling queue, and the workflow instance is initiated after the task reaches the top of the queue. task.WorkflowTriggerTask does not successfully place a workflow job in queue if an identical instance of that workflow (with the same recordType, id, and workflowId) is currently executing or already in the scheduling queue. To use the WorkflowTriggerTask Object: - Use task.create(options) to create the WorkflowTriggerTask Object. - Use WorkflowTriggerTask.recordType to set the record type of the workflow base record. - Use WorkflowTriggerTask.recordId to set the internal ID of the base record for the workflow. - Use WorkflowTriggerTask.workflowId to set the internal ID of the workflow that you want to run on the record specified by the recordId. - Optionally, use WorkflowTriggerTask.params to specify default values for workflow fields. - Use WorkflowTriggerTask.submit() to submit the asynchronous workflow initiation task to the NetSuite task queue. - Use the properties for the WorkflowTriggerTaskStatus.status object to get the status of the workflow execution. Use the following guidelines with the WorkflowTriggerTask Object: WorkflowTriggerTask.submit() does not successfully place a workflow task in the scheduling queue if an identical instance of that workflow, with the same recordType, recordId, and workflowId, is currently executing or already in the scheduling queue. To initiate a workflow on demand, see workflow.initiate(options).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/task Module
Methods and Properties	WorkflowTriggerTask Object Members

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var workflowTask = task.create({
4     taskType: task.TaskType.WORKFLOW_TRIGGER
5 });
6 workflowTask.recordType = 'customer';
7 workflowTask.recordId = 107;
8 workflowTask.workflowId = 3;
9 var taskId = workflowTask.submit();
10 ...
11 //Add additional code

```

WorkflowTriggerTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Directs NetSuite to place the asynchronous workflow initiation task into the NetSuite scheduling queue and returns a unique ID for the task. Use WorkflowTriggerTaskStatus.status to view the status of a submitted task.
Returns	string The task ID
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
FAILED_TO_SUBMIT_JOB_REQUEST_1	Failed to submit job request: {reason}	Task cannot be submitted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code

```

```

2 | ...
3 | var workflowTriggerTask = workflowTask.submit();
4 | ...
5 | //Add additional code

```

WorkflowTriggerTask.id

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the task.
Type	string
Module	N/task Module
Parent Object	task.WorkflowTriggerTask
Sibling Object Members	WorkflowTriggerTask Object Members
Since	—

Syntax

! **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | // A unique task id for the task is returned when you submit the task. (workflowTriggerTask is a previously created task.WorkflowTriggerTask object)
4 |
5 | var workflowTriggerTaskId = workflowTriggerTask.submit(); //workflowTriggerTaskId is the unique task id
6 | ...
7 | //Add additional code

```

WorkflowTriggerTask.params

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Key-value pairs to set default values on fields specific to the workflow. These can include fields on the Workflow Definition Page or workflow and state Workflow Custom Fields .
Type	Object
Module	N/task Module
Since	2015.2

Syntax

! **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code

```

```

2  ...
3  task.params = {context: portlet}
4  ...
5  //Add additional code

```

WorkflowTriggerTask.recordId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID of the base record. For example, 55 or 124.
Type	number
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  //Add additional code
2  ...
3  workflowTask.recordId = 107;
4  ...
5  //Add additional code

```

WorkflowTriggerTask.recordType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Record type of the workflow definition base record. For example, customer, salesorder, or lead. In the Workflow Manager, this is the record type that is specified in the Record Type field.
Type	string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  //Add additional code
2  ...
3  workflowTask.recordType = 'customer';
4  ...

```

5 | //Add additional code

WorkflowTriggerTask.workflowId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Internal ID (as a number), or script ID (as a string), for the workflow definition. This is the ID that appears in the ID field on the Workflow Definition Page .
Type	number string
Module	N/task Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 workflowTask.workflowId = 3;
4 ...
5 //Add additional code
```

task.WorkflowTriggerTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The status of an asynchronous workflow initiation task placed into the NetSuite task queue by WorkflowTriggerTask.submit() . Use task.checkStatus(options) with the unique ID for the asynchronous workflow initiation task to get the WorkflowTriggerTaskStatus object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task Module
Methods and Properties	WorkflowTriggerTaskStatus Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

1 | //Add additional code

```

2 | ...
3 | var workflowTaskStatus = task.checkStatus(taskId);
4 | if (workflowTaskStatus.status === task.TaskStatus.FAILED) { // Handle the failure }
5 | ...
6 | //Add additional code

```

WorkflowTriggerTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Status for an asynchronous workflow placed in the NetSuite task queue by WorkflowTriggerTask.submit() . Returns a task.TaskStatus enum value.
Type	string (read-only)
Module	N/task Module
Since	2015.2

Errors

Error Code	Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var summary = task.checkStatus(scriptTaskId);
4 | log.audit('Status', summary.status);
5 | ...
6 | //Add additional code

```

WorkflowTriggerTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The task ID associated with the specified task.
Type	string (read-only)
Module	N/task Module
Parent Object	task.WorkflowTriggerTaskStatus
Sibling Object Members	WorkflowTriggerTaskStatus Object Members
Since	—

Errors

Error Code	Error Message	Thrown If
READ_ONLY_PROPERTY	—	Setting the property is attempted.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code
2 ...
3 // myWorkflowTriggerTask is an existing task.WorkflowTriggerTask object
4 var myTaskId = myWorkflowTriggerTask.submit();
5 var myTaskStatus = task.checkStatus({
6     taskId: myTaskId
7 });
8 var theTaskStatusId = myTaskStatus.taskId;
9 ...
10 // Add additional code

```

task.checkStatus(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a task status object associated with a specific task ID. You can check the status of the following task types: - CSV import - Entity deduplication - Map/reduce script - Query - Record action - Scheduled script - Search - SuiteQL - Workflow trigger
Returns	task.CsvImportTaskStatus task.EntityDeduplicationTaskStatus task.MapReduceScriptTaskStatus task.QueryTaskStatus task.RecordActionTaskStatus task.ScheduledScriptTaskStatus task.SearchTaskStatus task.SuiteQLTaskStatus task.WorkflowTriggerTaskStatus
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/task Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.taskId	string	required	Unique ID of the task to check the status of.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var myTask = task.create({
3     taskType: task.TaskType.QUERY
4 });
5
6 // Submit the task
7 var myTaskId = myTask.submit();
8
9 // Check the status
10 var myTaskStatus = task.checkStatus({
11     taskId: myTaskId
12 });
13 ...
14 // Add additional code

```

task.create(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a task object for the specified task type and returns the object. Use this method to create tasks to do the following: - Schedule scripts - Run map/reduce scripts - Import CSV files - Merge duplicate records - Execute asynchronous searches, constructed queries, SuiteQL queries, and workflows
Returns	task.CsvImportTask task.EntityDeduplicationTask task.MapReduceScriptTask task.QueryTask task.RecordActionTask task.ScheduledScriptTask task.SearchTask task.SuiteQLTask task.WorkflowTriggerTask
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/task Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.taskType	string	required	The type of task object to create. Use the task.TaskType enum to set the value.
options.deploymentId	string	optional	The script ID (as a string) of the script deployment record. This parameter sets the value for the ScheduledScriptTask.deploymentId or MapReduceScriptTask.deploymentId property. This parameter is applicable only when options.taskType is set to task.TaskType.SCHEDULED_SCRIPT or task.TaskType.MAP_REDUCE.
options.params	Object Array<string boolean number>	optional	An object that represents additional parameters and values for the task. This parameter accepts different values based on the value of the options.taskType parameter: ■ If options.taskType is set to task.TaskType.SCHEDULED_SCRIPT, task.TaskType.MAP_REDUCE or task.TaskType.WORKFLOW_TRIGGER, this parameter accepts key-value pairs that override static script parameter field values on the script deployment record. For more information, see the help topic Creating Script Parameters Overview. For workflow tasks, keys can include fields on the Workflow Definition Page, or workflow and state custom fields. For more information, see the help topic Workflow Custom Fields. ■ If options.taskType is set to task.TaskType.SUITE_QL, this parameter accepts an array of parameters for the associated SuiteQL query. The specified parameters are substituted into the SuiteQL query string when you submit the task. This parameter sets the value for the ScheduledScriptTask.params , MapReduceScriptTask.params , WorkflowTriggerTask.params , or SuiteQLTask.params properties. This parameter is applicable only when options.taskType is set to task.TaskType.SCHEDULED_SCRIPT, task.TaskType.MAP_REDUCE, task.TaskType.WORKFLOW_TRIGGER, or task.TaskType.SUITE_QL.
options.scriptId	number string	optional	The internal ID (as a number) or script ID (as a string) for the script record. This parameter sets the value for the ScheduledScriptTask.scriptId or MapReduceScriptTask.scriptId property.

Parameter	Type	Required / Optional	Description
			This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.SCHEDULED_SCRIPT</code> or <code>task.TaskType.MAP_REDUCE</code> .
<code>options.importFile</code>	file.File string	optional	A CSV file to import. Use a file.File object or a string that represents the CSV text to be imported. This parameter sets the value for the CsvImportTask.importFile property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.CSV_IMPORT</code>.
<code>options.linkedFiles</code>	Object	optional	A map of key-value pairs that sets the data to be imported in a linked file for a multi-file import job, by referencing a file in the File Cabinet or the raw CSV data to import. The key is the internal ID of the record sublist for which data is being imported and the value is either a file.File object or the raw CSV data to import. You can assign multiple types of values to this property. This parameter sets the value for the CsvImportTask.linkedFiles property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.CSV_IMPORT</code>.
<code>options.mappingId</code>	number string	optional	The internal ID (as a number) or script ID (as a string) of a saved import map created using the Import Assistant. See task.CsvImportTask. This parameter sets the value for the CsvImportTask.mappingId property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.CSV_IMPORT</code>.
<code>options.name</code>	string	optional	The name for the CSV import task. You can optionally set a different name for a scripted import task. In the UI, this name appears on the CSV Import Job Status page. This parameter sets the value for the CsvImportTask.name property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.CSV_IMPORT</code>.
<code>options.queueId</code>	number	optional	Overrides the Queue Number property under Advanced Options on the Import Options page of the Import Assistant. Use this property to programmatically select an import queue and improve performance during the import.  Note: This property is only available if you have a SuiteCloud Plus license. For more information about using multiple queues when importing CSV files, see the help topics Queue Number and Use Multiple Threads and Multiple Queues to Run CSV Import Jobs. This parameter sets the value for the CsvImportTask.queueId property.

Parameter	Type	Required / Optional	Description
			This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.CSV_IMPORT</code> .
<code>options.dedupeMode</code>	string	optional	Sets the mode for merging or deleting duplicate records. This parameter sets the value for the EntityDeduplicationTask.dedupeMode property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.ENTITY_DEDUPLICATION</code>. Use the task.DedupeMode enum to set the value.
<code>options.entityType</code>	string	optional	Sets the type of entity on which you want to merge duplicate records. This parameter sets the value for the EntityDeduplicationTask.entityType property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.ENTITY_DEDUPLICATION</code>. Use the task.DedupeEntityType enum to set the value.  Note: If you specify a value of CUSTOMER for this parameter, NetSuite automatically includes prospects and leads in the task request.
<code>options.masterRecordId</code>	number	optional	When you merge duplicate records, you can delete all duplicates for a record or merge information from the duplicate records into the master record. Use this property to set the ID of the master record that you want to use as the master record in the merge.  Important: You must also select <code>SELECT_BY_ID</code> for the EntityDeduplicationTask.masterSelectionMode property, or NetSuite ignores this setting. This parameter sets the value for the EntityDeduplicationTask.masterRecordId property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.ENTITY_DEDUPLICATION</code>.
<code>options.masterSelectionMode</code>	string	optional	When you merge duplicate records, you can delete all duplicates for a record or merge information from the duplicate records into the master record. Set this property to determine which of the duplicate records to keep or to select the master record to use by ID. This parameter sets the value for the EntityDeduplicationTask.masterSelectionMode property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.ENTITY_DEDUPLICATION</code>. Use the task.MasterSelectionMode enum to set the value.
<code>options.recordIds</code>	number[]	optional	The internal IDs of the records to perform the merge or delete operation on. You can use the search.duplicates(options) method to identify duplicate records or create an array with record internal IDs.

Parameter	Type	Required / Optional	Description
			This parameter sets the value for the EntityDeduplicationTask.recordIds property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.ENTITY_DEDUPLICATION</code>.
<code>options.recordId</code>	number	optional	The internal ID of the base record. This parameter sets the value for the WorkflowTriggerTask.recordId property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.WORKFLOW_TRIGGER</code>.
<code>options.recordType</code>	string	optional	The record type of the workflow definition base record, such as customer, salesorder, or lead. In the Workflow Manager, this is the record type that is specified in the Record Type field. This parameter sets the value for the WorkflowTriggerTask.recordType property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.WORKFLOW_TRIGGER</code>.
<code>options.workflowId</code>	number string	optional	The internal ID (as a number) or script ID (as a string) for the workflow definition. This is the ID that appears in the ID field on the Workflow Definition Page. This parameter sets the value for the WorkflowTriggerTask.workflowId property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.WORKFLOW_TRIGGER</code>.
<code>options.fileId</code>	number	optional	The internal ID of the CSV file to export search results to. For more information about working with files, see N/file Module. This parameter is mutually exclusive with the <code>options.filePath</code> parameter. If you specify values for both parameters, an error occurs. This parameter sets the value for the SearchTask.fileId, QueryTask.fileId, or SuiteQLTask.fileId properties. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.SEARCH</code>, <code>task.TaskType.QUERY</code>, or <code>task.TaskType.SUITE_QL</code>.
<code>options.filePath</code>	string	optional	The path of the CSV file to export search results to. For more information about working with files, see N/file Module. This parameter is mutually exclusive with the <code>options.fileId</code> parameter. If you specify values for both parameters, an error occurs. This parameter sets the value for the SearchTask.filePath, QueryTask.filePath, or SuiteQLTask.filePath properties. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.SEARCH</code>, <code>task.TaskType.QUERY</code>, or <code>task.TaskType.SUITE_QL</code>.
<code>options.query</code>	query.Query string	optional	The query definition for a query task or SuiteQL task.

Parameter	Type	Required / Optional	Description
			This parameter accepts different values based on the value of the <code>options.taskType</code> parameter: ■ If <code>options.taskType</code> is set to <code>task.TaskType.QUERY</code>, this parameter accepts a query.Query object that represents the query to run. ■ If <code>options.taskType</code> is set to <code>task.TaskType.SUITE_QL</code>, this parameter accepts a string that represents the SuiteQL query to run. This parameter sets the value for the QueryTask.query or SuiteQLTask.query properties. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.QUERY</code> or <code>task.TaskType.SUITE_QL</code>.
<code>options.savedSearchId</code>	string	optional	The internal ID of the saved search to be executed during the task. This parameter sets the value for the SearchTask.savedSearchId property. This parameter is applicable only when <code>options.taskType</code> is set to <code>task.TaskType.SEARCH</code>.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var myTask = task.create({
3   taskType: task.TaskType.MAP_REDUCE,
4   scriptId: 34,
5   deploymentId: 'custdeploy1',
6   params: { doSomething: true }
7 });
8 ...
9 // Add additional code

```

task.ActionCondition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the possible record action conditions. This enum is returned by RecordActionTask.condition.
Module	N/task Module
Sibling Module Members	N/task Module Members

Since	2019.1
-------	--------

Values

Value
ALL_QUALIFIED_INSTANCES

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var recordActionTask = task.create({
3   taskType: task.TaskType.RECORD_ACTION
4 });
5 recordActionTask.recordType = 'timebill';
6 recordActionTask.action = 'approve';
7 recordActionTask.condition = task.ActionCondition.ALL_QUALIFIED_INSTANCES; recordActionTask.paramCallback = function(v) {
8   return {recordId: v, note: "this is a note for " + v };
9 };
10
11 var handle = recordActionTask.submit();
12 ...
13 // Add additional code

```

task.DedupeEntityType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for entity types for which you can merge duplicate records with task.EntityDeduplicationTask . Use this enum to set the EntityDeduplicationTask.entityType .
Module	N/task Module
Sibling Module Members	N/task Module Members
Since	2015.2

Values

Value
CUSTOMER

Value
CONTACT
VENDOR
PARTNER
LEAD
PROSPECT

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 //Add additional code
2 ...
3 dedupeTask.entityType = task.DedupeEntityType.CUSTOMER;
4 ...
5 //Add additional code

```

task.DedupeMode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the available deduplication modes when merging duplicate records with task.EntityDeduplicationTask . Use this enum to set the EntityDeduplicationTask.dedupeMode property.
Module	N/task Module
Sibling Module Members	N/task Module Members
Since	2015.2

Values

Value
MERGE
DELETE
MAKE_MASTER_PARENT
MARK_AS_NOT_DUPES

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
2 ...
3 dedupeTask.dedupeMode = task.DedupeMode.MERGE;
4 ...
5 //Add additional code
```

task.MapReduceStage

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for possible stages in task.MapReduceScriptTask for a map/reduce script. This enum is returned by MapReduceScriptTaskStatus.stage . For general information about map/reduce stages, see the help topics Map/Reduce Key Concepts and Map/Reduce Script Stages .
Module	N/task Module
Sibling Module Members	N/task Module Members
Since	2015.2

Values

Value
GET_INPUT
MAP
SHUFFLE
REDUCE
SUMMARIZE

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```
1 //Add additional code
```



```

2  ...
3  if (summary.stage === task.MapReduceStage.SUMMARIZE)
4  ...
5  //Add additional code

```

Note: For general information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

task.MasterSelectionMode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported master selection modes when merging duplicate records with task.EntityDeduplicationTask. Use this enum to set the EntityDeduplicationTask.masterSelectionMode property. For more information about these values, see the help topic Merging or Deleting Duplicate Records.
Module	N/task Module
Sibling Module Members	N/task Module Members
Since	2015.2

Values

Value
CREATED_EARLIEST
MOST_RECENT_ACTIVITY
MOST_POPULATED_FIELDS
SELECT_BY_ID

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1  //Add additional code
2  ...
3  dedupeTask.masterSelectionMode = task.MasterSelectionMode.MOST_RECENT_ACTIVITY;
4  ...
5  //Add additional code

```

task.TaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for possible task statuses. The following properties use values from this enum: ■ CsvImportTaskStatus.status ■ EntityDeduplicationTaskStatus.status ■ MapReduceScriptTaskStatus.status ■ RecordActionTaskStatus.status ■ QueryTaskStatus.status ■ ScheduledScriptTaskStatus.status ■ SearchTaskStatus.status ■ SuiteQLTaskStatus.status ■ WorkflowTriggerTaskStatus.status
Module	N/task Module
Sibling Module Members	N/task Module Members
Since	2015.2

Values

Value
COMPLETE
FAILED
PENDING
PROCESSING

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 if (status === task.TaskStatus.COMPLETE || status === task.TaskStatus.FAILED) {
3     // Handle the status
4 }
5 ...
6 // Add additional code

```

task.TaskType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the types of task objects you can create. Use this enum to set the value for the options.taskType parameter of the task.create(options) method.
Module	N/task Module
Sibling Module Members	N/task Module Members
Since	2015.2

Values

Value
CSV_IMPORT
DOCUMENT_CAPTURE
ENTITY_DEDUPLICATION
MAP_REDUCE
QUERY
RECORD_ACTION
SCHEDULED_SCRIPT
SEARCH
SUITE_QL
WORKFLOW_TRIGGER

 **Important:** Ensure that you are familiar with script execution time limits when creating tasks. Each server script type is limited to an amount of time it can run on a single execution. For more information, see the time limit chart on [Script Execution Time Limits](#).

Script Type	Time Limit (in Seconds)
Bundle Installation Scripts (SuiteScript 1.0) SuiteScript 2.x Bundle Installation Script Type	3,600
Client Scripts (SuiteScript 1.0) SuiteScript 2.x Client Script Type	300

Script Type	Time Limit (in Seconds)
Custom record action	300
Map/reduce (input stage) SuiteScript 2.x Map/Reduce Script Type	3,600
Map/reduce (map stage) SuiteScript 2.x Map/Reduce Script Type	300
Map/reduce (reduce stage) SuiteScript 2.x Map/Reduce Script Type	900
Map/reduce (summarize stage) SuiteScript 2.x Map/Reduce Script Type	3,600
Mass Update Scripts (SuiteScript 1.0) SuiteScript 2.x Mass Update Script Type	300
Portlet Scripts (SuiteScript 1.0) SuiteScript 2.x Portlet Script Type	300
RESTlets (SuiteScript 1.0) SuiteScript 2.x RESTlet Script Type	300
SuiteScript 2.x SDF Installation Script Type	3,600
Single-page application (SPA)	300
Scheduled Scripts (SuiteScript 1.0) SuiteScript 2.x Scheduled Script Type	3,600
Suitelets (SuiteScript 1.0) SuiteScript 2.x Suitelet Script Type	300
SuiteScript Server Pages (SSP) application	300
User Event Scripts (SuiteScript 1.0) SuiteScript 2.x User Event Script Type	300
Workflow Action Scripts (SuiteScript 1.0) SuiteScript 2.x Workflow Action Script Type	300

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task Module Script Samples](#).

```

1 // Add additional code ...
2 var myTask = task.create({
3     taskType: task.TaskType.MAP_REDUCE
4 });
5 ...
6 // Add additional code

```

N/task/accounting/recognition Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/task/accounting/recognition module to merge revenue arrangements or revenue elements. A revenue arrangement is a transaction that records the details of a sale for the purposes of revenue allocation and recognition. The N/task/accounting/recognition module lets you combine revenue arrangements or revenue elements from multiple sources to represent a single contract obligation for revenue allocation and recognition.

Go To Samples 

You can use the [recognition.create\(options\)](#) method to create a merge task that combines entire revenue arrangements or individual revenue elements. This method returns a [recognition.MergeArrangementsTask](#) object (when merging revenue arrangements) or [recognition.MergeElementsTask](#) object (when merging revenue elements). After you obtain one of these objects, you can set its properties, such as the list of arrangements or elements to merge, the date on the merged revenue arrangement, whether to prospectively merge arrangements, and so on. You can use these properties to specify the same input data that you can specify when you merge revenue arrangements using the NetSuite UI. After you set its properties, you can submit the task for processing. Merge tasks are processed asynchronously.

You cannot merge more than 10,000 revenue elements at one time. An error is thrown in the script if you attempt to merge more than 10,000 revenue elements. For UI related limitations, see the help topic [Limitations for Creating Transactions](#).

You can use the [recognition.checkStatus\(options\)](#) method to check the status of a submitted merge task. This method returns a [recognition.MergeArrangementsTaskStatus](#) object that describes the current status of the merge task (pending, processing, complete, or failed). This object represents the current status for either a [recognition.MergeArrangementsTask](#) or a [recognition.MergeElementsTask](#). If the task completes successfully, this object includes the ID of the merged revenue arrangement record that was created. If the task fails, this object includes an error message that describes the failure.

To merge revenue arrangements or revenue elements using the N/task/accounting/recognition module, the following requirements must be met:

- The Advanced Revenue Management feature must be enabled in your account. For more information, see the help topic [Enabling the Advanced Revenue Management \(Essentials\) Feature](#).
- Your role must have the (Transactions) Revenue Arrangement permission assigned at a level of Create or higher. For more information, see the help topic [NetSuite Permissions Overview](#).

For more information about revenue arrangements, see the following help topics:

- [Revenue Arrangement Management](#) — This topic describes revenue arrangements in general.
- [Combination and Modification of Performance Obligations](#) — This topic describes the different types of merge results (combined revenue arrangements and prospective change orders).
- [Revenue Arrangement](#) — This topic describes the revenue arrangement record type, including scripting considerations, supported script types, and sublist fields.

In This Help Topic

- [N/task/accounting/recognition Module Members](#)
- [MergeArrangementsTask Object Members](#)

- [MergeArrangementsTaskStatus Object Members](#)
- [MergeElementsTask Object Members](#)

N/task/accounting/recognition Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	recognition.MergeArrangementsTask	Object	Server scripts	Encapsulates a task to merge all of the revenue elements from a specified list of revenue arrangements. Use recognition.create(options) to create this object.
	recognition.MergeArrangementsTaskStatus	Object	Server scripts	Encapsulates the current status of a submitted merge task. Use recognition.checkStatus(options) to create this object.
	recognition.MergeElementsTask	Object	Server scripts	Encapsulates a task to merge all of the specified revenue elements. Use recognition.create(options) to create this object.
Method	recognition.checkStatus(options)	recognition.MergeArrangementsTaskStatus	Server scripts	Checks the status of a submitted merge task.
	recognition.create(options)	recognition.MergeArrangementsTask recognition.MergeElementsTask	Server scripts	Creates a merge task that combines entire revenue arrangements or individual revenue elements. Use values in the recognition.TaskType enum to specify the type of merge task to create.
Enum	recognition.TaskStatus	enum	Server scripts	Holds the string values for supported merge task statuses. This enum is used to represent the task status in a recognition.MergeArrangementsTaskStatus object.
	recognition.TaskType	enum	Server scripts	Holds the string values for supported merge task types. This enum is used to pass the task type argument to recognition.create(options) .

MergeArrangementsTask Object Members

The following members are available for a [recognition.MergeArrangementsTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	MergeArrangementsTask.submit()	number (read-only)	Server scripts	Submits the merge task for processing. This method returns a task ID that uniquely identifies the merge task.
Property	MergeArrangementsTask.arrangements	Array<number string> (read-only)	Server scripts	Holds an array of internal IDs of the revenue arrangement records to merge.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	MergeArrangementsTask.contractAcquisitionDeferredExpenseAccount	number string (read-only)	Server scripts	References the contract acquisition deferred expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . The default value is the account specified by the accounting preference Contract Acquisition Deferred Expense Account in your account.
	MergeArrangementsTask.contractAcquisitionExpenseAccount	number string (read-only)	Server scripts	References the contract acquisition expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . The default value is the account specified by the accounting preference Contract Acquisition Expense Account in your account.
	MergeArrangementsTask.contractCostAccrualDate	JavaScript Date (read-only)	Server scripts	Describes the contract cost accrual date to use for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . The default value is today's date.
	MergeArrangementsTask.mergeResidualRevenueAmounts	boolean (read-only)	Server scripts	Indicates whether the revenue arrangements are merged prospectively. For more information about prospective merges, see the help topic Prospective Merges . The default value is false.
	MergeArrangementsTask.recalculateResidualFairValue	boolean (read-only)	Server scripts	Indicates whether to recalculate the fair value on residual elements when revenue arrangements are prospectively merged. For more information about prospective merges, see the help topic Prospective Merges . This property can be set to true only if the MergeArrangementsTask.mergeResidualRevenueAmounts property is also set to true. The default value is false.
	MergeArrangementsTask.revenueArrangementDate	JavaScript Date (read-only)	Server scripts	Describes the date of the new revenue arrangement. The default value is today's date.

MergeArrangementsTaskStatus Object Members

The following members are available for a [recognition.MergeArrangementsTaskStatus](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	MergeArrangementsTaskStatus.errorMessage	string (read-only)	Server scripts	Holds an error message that describes the failure of the merge task. This property is valid only if the value of the status property is TaskStatus.FAILED.
	MergeArrangementsTaskStatus.inputArrangements	number[] (read-only)	Server scripts	Holds an array of internal IDs of the revenue arrangement records to merge. This property is valid only if the merge

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				task was created using a task type of <code>TaskType.MERGE_ARRANGEMENTS_TASK</code> .
	MergeArrangementsTaskStatus.inputElements	number[] (read-only)	Server scripts	Holds an array of internal IDs of the revenue elements to merge. This property is valid only if the merge task was created using a task type of <code>TaskType.MERGE_ELEMENTS_TASK</code> .
	MergeArrangementsTaskStatus.resultingArrangement	number string (read-only)	Server scripts	References the internal ID of the new revenue arrangement that was created. This property is valid only if the value of the status property is <code>TaskStatus.COMPLETE</code> .
	MergeArrangementsTaskStatus.status	string (read-only)	Server scripts	Represents the current status of the merge task. This property uses values in the recognition.TaskStatus enum.
	MergeArrangementsTaskStatus.submissionId	number string (read-only)	Server scripts	References the submission ID of the merge arrangements bulk process.
	MergeArrangementsTaskStatus.taskId	number string (read-only)	Server scripts	Holds the task ID of the merge task. The task ID is assigned to the merge task when you call recognition.create(options) .

MergeElementsTask Object Members

The following members are available for a [recognition.MergeElementsTask](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	MergeElementsTask.submit()	number (read-only)	Server scripts	Submits the merge task for processing. This method returns a task ID that uniquely identifies the merge task.
Property	MergeElementsTask.contractAcquisitionDeferredExpenseAccount	number string (read-only)	Server scripts	References the contract acquisition deferred expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . The default value is the account specified by the accounting preference Contract Acquisition Deferred Expense Account in your account.
	MergeElementsTask.contractAcquisitionExpenseAccount	number string (read-only)	Server scripts	References the contract acquisition expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . The default value is the account specified by the accounting preference Contract Acquisition Expense Account in your account.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	MergeElementsTask.contractCostAccrualDate	JavaScript Date (read-only)	Server scripts	Describes the contract cost accrual date to use for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . The default value is today's date.
	MergeElementsTask.elements	Array<number string> (read-only)	Server scripts	Holds an array of internal IDs of the revenue element records to merge.
	MergeElementsTask.revenueArrangementDate	JavaScript Date (read-only)	Server scripts	Describes the date of the new revenue arrangement. The default value is today's date.

N/task/accounting/recognition Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/task/accounting/recognition module:

- [Merge Revenue Elements Using Internal IDs](#)
- [Merge Revenue Arrangements Using a Saved Search](#)
- [Merge Revenue Arrangements Using an Ad-Hoc Search](#)

Merge Revenue Elements Using Internal IDs

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample adds the internal IDs of several revenue element records to an array. It calls [recognition.create\(options\)](#) to create a merge task for revenue element records, uses the array as the list of revenue element records to merge, and submits the merge task. The sample also checks the status of the merge task.

If you run this sample code in your account, be sure to use the internal IDs of valid revenue element records in your account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/task/accounting/recognition'], function(recognition){
6      var elementsList = [];
7      elementsList.push(401);
8      elementsList.push(402);
9
10     var recognitionTask = recognition.create({

```

```

11     taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
12 });
13 recognitionTask.elements = elementsList;
14 var taskStatusId = recognitionTask.submit();
15
16 var mergeTaskState = recognition.checkStatus({
17     taskId: taskStatusId
18 });
19
20 log.debug('Submission ID = ' + mergeTaskState.submissionId);
21 log.debug('Resulting Arrangement ID = ' + mergeTaskState.resultingArrangement);
22 log.debug('status = ' + mergeTaskState.status);
23 });

```

Merge Revenue Arrangements Using a Saved Search

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads a saved search for revenue arrangement records. It obtains the value of the `internalid` field from each record in the result set, and it adds the values to an array. It calls [recognition.create\(options\)](#) to create a merge task for revenue arrangement records, uses the array as the list of revenue arrangement records to merge, and submits the merge task. The sample also checks the status of the merge task and logs status information.

If you run this sample code in your account, be sure to use a saved search for valid revenue arrangement records in your account.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/task/accounting/recognition', 'N/search'], function(recognition, search){
6      var mySearch = search.load({
7          id: 'customsearch22'
8      });
9
10     var elementsList = [];
11     mySearch.run().each(function(result) {
12         var id = result.getValue({
13             name: 'internalid'
14         });
15         elementsList.push(id);
16     });
17
18     var recognitionTask = recognition.create({
19         taskType: recognition.TaskType.MERGE_ARRANGEMENTS_TASK
20     });
21
22     recognitionTask.arrangements = elementsList;
23     recognitionTask.revenueArrangementDate = new Date(2019, 2, 10);
24
25     var taskStatusId = recognitionTask.submit();
26     log.debug('taskId = ' + taskStatusId);
27
28     var mergeTaskState = recognition.checkStatus({
29         taskId: taskStatusId
30     });
31
32     log.debug('Submission ID = ' + mergeTaskState.submissionId);
33     log.debug('Resulting Arrangement ID = ' + mergeTaskState.resultingArrangement);

```

```

34 |     log.debug('status = ' + mergeTaskState.status);
35 |     log.debug('Error message = ' + mergeTaskState.errorMessage);
36 | });

```

Merge Revenue Arrangements Using an Ad-Hoc Search

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates an ad-hoc search for revenue element records. It obtains the first 50 results, obtains the value of the `elementsList` field from each record in the result set, and adds the values to an array. It calls [recognition.create\(options\)](#) to create a merge task for revenue element records, uses the array as the list of revenue elements records to merge, and submits the merge task. This sample also checks the status of the merge task and logs status information.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1 |  /**
2 |   * @NApiVersion 2.x
3 |   */
4 |
5 |   require(['N/task/accounting/recognition', 'N/search'], function(recognition, search) {
6 |       var elementsList = [];
7 |       var rs = search.create({
8 |           type: 'revenueelement',
9 |           columns: [
10 |               'internalid'
11 |           ]
12 |       }).run();
13 |
14 |       var results = rs.getRange(0, 50);
15 |       for (var i = 0; i < results.length; i++) {
16 |           var id = result.getValue('elementsList');
17 |           elementsList.push(id);
18 |       }
19 |
20 |       var t = recognition.create({
21 |           taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
22 |       });
23 |       t.elements = elementsList;
24 |       t.revenueArrangementDate = new Date(2019, 1, 1);
25 |
26 |       var taskId = t.submit();
27 |       log.debug('Initial status: ' + res.status);
28 |   });

```

recognition.MergeArrangementsTask

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a task to merge all of the revenue elements from a specified list of revenue arrangements. Use recognition.create(options) to create this object. After you create the object, you can populate its properties and submit the task for processing. The MergeArrangementsTask.arrangements property is required, and all other properties are optional.
---------------------------	---

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task/accounting/recognition Module
Methods and Properties	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var recognitionTask = recognition.create({
2   taskType: recognition.TaskType.MERGE_ARRANGEMENTS_TASK
3 });
```

MergeArrangementsTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits the merge task for processing. This method returns a task ID that uniquely identifies the merge task. This task ID also represents the submission ID of the internal bulk process that performs the merge. Before you call this method to submit a merge task, you must populate the properties of the recognition.MergeArrangementsTask object (such as MergeArrangementsTask.arrangements , MergeArrangementsTask.contractAcquisitionExpenseAccount , and so on).
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Errors

Error Code	Thrown If
NO_REVENUE_ARRANGEMENT_IDS_ARE_INCLUDED_IN_YOUR_INPUT	The MergeArrangementsTask.arrangements property is empty.

Error Code	Thrown If
NO_REVENUE_ELEMENTS_WERE_FOUND_FOR_THE_REVENUE_ARRANGEMENT_IDS_YOU_INPUT	No revenue elements were found for the revenue arrangements specified in the MergeArrangementsTask.arrangements property.
WRONG_PARAMETER_TYPE	An invalid date format is specified in the MergeArrangementsTask.contractCostAccrualDate property or the MergeArrangementsTask.revenueArrangementDate property.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 var recognitionTask = recognition.create({
2   taskType: recognition.TaskType.MERGE_ARRANGEMENTS_TASK
3 });
4
5 var taskStatusId = recognitionTask.submit();

```

MergeArrangementsTask.arrangements

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds an array of internal IDs of the revenue arrangement records to merge. This property is required. You must specify a value for this property before you can submit the task for processing using MergeArrangementsTask.submit(). If you do not specify any revenue arrangement record IDs, a NO_REVENUE_ARRANGEMENT_IDS_ARE_INCLUDED_IN_YOUR_INPUT error is thrown when you call MergeArrangementsTask.submit() for the task. Invalid IDs are ignored. If no revenue elements were found for the specified revenue arrangement record IDs, a NO_REVENUE_ELEMENTS_WERE_FOUND_FOR_THE_REVENUE_ARRANGEMENT_IDS_YOU_INPUT error is thrown when you call MergeArrangementsTask.submit().
Type	Array<number string>
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 var orderIDs = [1, 2, 3];

```

```
2 | recognitionTask.arrangements = orderIDs;
```

MergeArrangementsTask.contractAcquisitionDeferredExpenseAccount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	References the contract acquisition deferred expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . If this preference is not enabled, this property is ignored. This property is optional. The default value is the account specified by the accounting preference Contract Acquisition Deferred Expense Account in your account.
Type	number string
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | var deferredExpenseAccountId = 1; //Set to ID
2 | mergeTask.contractAcquisitionDeferredExpenseAccount = deferredExpenseAccountId;
```

MergeArrangementsTask.contractAcquisitionExpenseAccount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	References the contract acquisition expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . If this preference is not enabled, this property is ignored. This property is optional. The default value is the account specified by the accounting preference Contract Acquisition Expense Account in your account.
Type	number string
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members

Since	2019.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var contractExpenseAccountId = 1;
2 mergeTask.contractAcquisitionExpenseAccount = contractExpenseAccountId;
```

MergeArrangementsTask.contractCostAccrualDate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Describes the contract cost accrual date to use for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . If this preference is not enabled, this property is ignored. This property is optional. The default value is today's date. If you specify an invalid date format, a <code>WRONG_PARAMETER_TYPE</code> error is thrown when you call MergeArrangementsTask.submit() for the task.
Type	JavaScript Date
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var accrualDate = new Date('2023-05-01');
2 recognitionTask.contractCostAccrualDate = accrualDate;
```

MergeArrangementsTask.mergeResidualRevenueAmounts

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the revenue arrangements are merged prospectively. For more information about prospective merges, see the help topic Prospective Merges .
-----------------------------	---

	This property is optional. The default value is false.
Type	boolean
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | recognitionTask.mergeResidualRevenueAmounts = true;
```

MergeArrangementsTask.recalculateResidualFairValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether to recalculate the fair value on residual elements when revenue arrangements are prospectively merged. For more information about prospective merges, see the help topic Prospective Merges. This property is optional. This property can be set to true only if the MergeArrangementsTask.mergeResidualRevenueAmounts property is also set to true. If the MergeArrangementsTask.mergeResidualRevenueAmounts property is false, this property is ignored. The default value of this property is false.
Type	boolean
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTask
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | if (recognitionTask.mergeResidualRevenueAmounts == true) {
2 |     recognitionTask.recalculateResidualFairValue = true;
```


3 | }

MergeArrangementsTask.revenueArrangementDate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Describes the date of the new revenue arrangement. This property is optional. The default value is today's date. If you specify an invalid date format, a <code>WRONG_PARAMETER_TYPE</code> error is thrown when you call <code>MergeArrangementsTask.submit()</code> for the task.
Type	JavaScript Date
Module	N/task/accounting/recognition Module
Parent Object	<code>recognition.MergeArrangementsTask</code>
Sibling Object Members	MergeArrangementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var accrualDate = new Date('2023-05-01');
2 recognitionTask.revenueArrangementDate = accrualDate;
```

recognition.MergeArrangementsTaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the current status of a submitted merge task. Use <code>recognition.checkStatus(options)</code> to create this object. The current status corresponds to one of the values in the <code>recognition.TaskStatus</code> enum: <code>PENDING</code> , <code>PROCESSING</code> , <code>COMPLETE</code> , or <code>FAILED</code> .
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task/accounting/recognition Module
Methods and Properties	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var mergeTaskState = recognition.checkStatus({
2   taskId: taskStatusId
3 });
4
5 if (mergeTaskState.status === 'PENDING') { //Add additional code }
```

MergeArrangementsTaskStatus.errorMessage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds an error message that describes the failure of the merge task. This property is valid only if the value of the MergeArrangementsTaskStatus.status property is TaskStatus.FAILED.
Type	string (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var mergeTaskState = recognition.checkStatus({
2   taskId: taskStatusId
3 });
4
5 if (mergeTaskState.status === 'ERROR') {
6   log.debug('Error Message: ' + mergeTaskState.errorMessage);
7 }
```

MergeArrangementsTaskStatus.inputArrangements

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds an array of internal IDs of the revenue arrangement records to merge. This property is valid only if the merge task was created using a task type of TaskType.MERGE_ARRANGEMENTS_TASK.
-----------------------------	--

	 Note: This property has a potential governance value of 10 units. For more information about governance, see the help topic SuiteScript Governance and Limits .
Type	number[] (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 | var mergeTaskState = recognition.checkStatus({
2 |     taskId: taskStatusId
3 | });
4 |
5 | if (mergeTaskState.status === 'COMPLETE') {
6 |     log.debug('Input Arrangement IDs: ' + mergeTaskState.inputArrangements);
7 | }

```

MergeArrangementsTaskStatus.inputElements

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds an array of internal IDs of the revenue elements to merge. This property is valid only if the merge task was created using a task type of <code>TaskType.MERGE_ELEMENTS_TASK</code> .
Type	number[] (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 | var mergeTaskState = recognition.checkStatus({

```

```

2 |     taskId: taskStatusId
3 |   });
4 |
5 |   if (mergeTaskState.status === 'COMPLETE') {
6 |     log.debug('Input Element IDs: ' + mergeTaskState.inputElements);
7 |   }

```

MergeArrangementsTaskStatus.resultingArrangement

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	References the internal ID of the new revenue arrangement that was created. This property is valid only if the value of the MergeArrangementsTaskStatus.status property is <code>TaskStatus.COMPLETE</code> .
Type	number (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 | if (mergeTaskState.status === 'COMPLETE') {
2 |   log.debug('Resulting Arrangement ID = ' + mergeTaskState.resultingArrangement);
3 | }

```

MergeArrangementsTaskStatus.status

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Represents the current status of the merge task. This property uses values in the recognition.TaskStatus enum.
Type	string (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var mergeTaskState = recognition.checkStatus({
2   taskId: taskStatusId
3 });
4
5 log.audit({
6   title: 'Status',
7   details: mergeTaskState.status
8 });
```

MergeArrangementsTaskStatus.submissionId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	References the submission ID of the merge arrangements bulk process. This ID is the same as the task ID that is returned by MergeArrangementsTask.submit() or MergeElementsTask.submit() .
Type	number (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var mergeTaskState = recognition.checkStatus({
2   taskId: taskStatusId
3 });
4
5 log.audit({
6   title: 'Submission ID',
7   details: mergeTaskState.submissionId
8 });
```

MergeArrangementsTaskStatus.taskId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds the task ID of the merge task. The task ID is assigned to the merge task when you call MergeArrangementsTask.submit() or MergeElementsTask.submit() for the task.
-----------------------------	---

Type	number string (read-only)
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeArrangementsTaskStatus
Sibling Object Members	MergeArrangementsTaskStatus Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 | var mergeTaskState = recognition.checkStatus({
2 |     taskId: taskStatusId
3 | });
4 |
5 | log.audit({
6 |     title: 'Task ID',
7 |     details: mergeTaskState.taskId
8 | });

```

recognition.MergeElementsTask

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a task to merge all of the specified revenue elements. Use recognition.create(options) to create this object. After you create the object, you can populate its properties and submit the task for processing. The MergeElementsTask.elements property is required, and all other properties are optional.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/task/accounting/recognition Module
Methods and Properties	MergeElementsTask Object Members
Since	2019.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 | var recognitionTask = recognition.create({
2 |     taskType: recognition.TaskType.MERGE_ELEMENTS_TASK

```

```
3 | });
```

MergeElementsTask.submit()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Submits the merge task for processing. This method returns a task ID that uniquely identifies the merge task. This task ID also represents the submission ID of the internal bulk process that performs the merge.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeElementsTask
Sibling Object Members	MergeElementsTask Object Members
Since	2019.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	An invalid date format is specified in the MergeElementsTask.contractCostAccrualDate property or the MergeElementsTask.revenueArrangementDate property.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | var recognitionTask = recognition.create({
2 |     taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
3 | });
4 |
5 | recognitionTask.elements = elementsList;
6 | var taskStatusId = recognitionTask.submit()
```

MergeElementsTask.contractAcquisitionDeferredExpenseAccount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	References the contract acquisition deferred expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost
-----------------------------	---

	Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . If this preference is not enabled, this property is ignored. This property is optional. The default value is the account specified by the accounting preference Contract Acquisition Deferred Expense Account in your account.
Type	number string
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeElementsTask
Sibling Object Members	MergeElementsTask Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | var deferredExpenseAccountId = 1; //Set to ID
2 | mergeTask.contractAcquisitionDeferredExpenseAccount = deferredExpenseAccountId;
```

MergeElementsTask.contractAcquisitionExpenseAccount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	References the contract acquisition expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization . If this preference is not enabled, this property is ignored. This property is optional. The default value is the account specified by the accounting preference Contract Acquisition Expense Account in your account.
Type	number string
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeElementsTask
Sibling Object Members	MergeElementsTask Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | var contractExpenseAccountId = 1;
```



```
2 | mergeTask.contractAcquisitionExpenseAccount = contractExpenseAccountId;
```

MergeElementsTask.contractCostAccrualDate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Describes the contract cost accrual date to use for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see the help topic Advanced Cost Amortization. This property is optional. The default value is today's date. If you specify an invalid date format, a <code>WRONG_PARAMETER_TYPE</code> error is thrown when you call MergeElementsTask.submit() for the task.
Type	JavaScript Date
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeElementsTask
Sibling Object Members	MergeElementsTask Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 | var accrualDate = new Date('2023-05-01');
2 | recognitionTask.contractCostAccrualDate = accrualDate;
```

MergeElementsTask.elements

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds an array of internal IDs of the revenue element records to merge. This property is required. You must specify a value for this property before you can submit the task for processing using MergeElementsTask.submit().
Type	Array<number string>
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeElementsTask
Sibling Object Members	MergeElementsTask Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var recognitionTask = recognition.create({
2     taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
3 });
4
5 recognitionTask.elements = elementsList;
```

MergeElementsTask.revenueArrangementDate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Describes the date of the new revenue arrangement. This property is optional. The default value is today's date. If you specify an invalid date format, a <code>WRONG_PARAMETER_TYPE</code> error is thrown when you call MergeElementsTask.submit() for the task.
Type	JavaScript Date
Module	N/task/accounting/recognition Module
Parent Object	recognition.MergeElementsTask
Sibling Object Members	MergeElementsTask Object Members
Since	2019.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var accrualDate = new Date('2023-05-01');
2 recognitionTask.revenueArrangementDate = accrualDate;
```

recognition.checkStatus(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Checks the status of a submitted merge task.
Returns	recognition.MergeArrangementsTaskStatus
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	50 units
Module	N/task/accounting/recognition Module
Sibling Module Members	N/task/accounting/recognition Module Members
Since	2019.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.taskId	number string	required	The task ID of the merge task to check. The task ID is assigned to the merge task when you call recognition.create(options) .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var mergeTaskState = recognition.checkStatus({
2   taskId: taskStatusId
3 });
```

recognition.create(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a merge task that combines entire revenue arrangements or individual revenue elements. Use values in the recognition.TaskType enum to specify the type of merge task to create. After you call this method to create a merge task, you must specify the properties of the merge task (such as MergeArrangementsTask.arrangements or MergeElementsTask.elements) before you submit the task using MergeArrangementsTask.submit() or MergeElementsTask.submit() .
Returns	recognition.MergeArrangementsTask recognition.MergeElementsTask
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/task/accounting/recognition Module
Sibling Module Members	N/task/accounting/recognition Module Members

Since	2019.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.taskType	string	required	The type of merge task to create. Use values from the recognition.TaskType enum for this parameter.

Errors

Error Code	Thrown If
INVALID_TASK_TYPE	The options.taskType parameter represents an invalid task type.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var t = recognition.create({
4     taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
5 });
6 ...
7 // Add additional code

```

recognition.TaskStatus

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported merge task statuses. This enum is used to represent the task status in a recognition.MergeArrangementsTaskStatus object.
Module	N/task/accounting/recognition Module
Sibling Module Members	N/task/accounting/recognition Module Members
Since	2019.2

Values

Value
COMPLETE
FAILED
PENDING
PROCESSING

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 var mergeTaskState = recognition.checkStatus({
2   taskId: taskStatusId
3 });
4
5 log.audit({
6   title: 'Status',
7   details: mergeTaskState.status
8 });
```

recognition.TaskType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported merge task types. This enum is used to pass the task type argument to recognition.create(options) .
Module	N/task/accounting/recognition Module
Sibling Module Members	N/task/accounting/recognition Module Members
Since	2019.2

Values

Value
MERGE_ARRANGEMENTS_TASK
MERGE_ELEMENTS_TASK

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/task/accounting/recognition Module Script Samples](#).

```
1 // Add additional code
2 ...
3 var t = recognition.create({
4     taskType: recognition.TaskType.MERGE_ARRANGEMENTS_TASK
5 });
6 ...
7 // Add additional code
```

N/transaction Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/transaction module to void transactions.

When you void a transaction, the total and all the line items for the transaction are set to zero. The transaction is not removed from the system. NetSuite supports two types of voids: direct voids and voids by reversing journal. For additional information, see the help topic [Voiding, Deleting, or Closing Transactions](#).

The type of void performed with your script depends on the targeted account's preference settings:

- If the Using Reversing Journals preference is **disabled**, a **direct void** is performed.
- If the Using Reversing Journals preference is **enabled**, a **void by reversing journal** is performed.

**Important:** After you successfully void a transaction, you can no longer make changes to the transaction that impact the general ledger.

Go To Samples 

In This Help Topic

- [N/transaction Module Members](#)

N/transaction Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	transaction.void(options)	number	Client and server scripts	voids a transaction record.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	transaction.void.promise(options)	number	Client and server scripts	voids a transaction record asynchronously.
Enum	transaction.Type	enum	Client and server scripts	Holds the string values for supported record types. This enum is used for the <code>options.type</code> parameter of the transaction.void(options) method.

N/transaction Module Script Sample

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/transaction module.

i Note: For transaction types that use direct void, you must disable the **Void Transactions Using Reversing Journals** feature in your account to avoid an error being thrown. To disable this feature, go to Setup > Accounting > Preferences > Accounting Preferences.

Void a Sales Order Transaction

The following sample creates a sales order record, saves it, then voids the sales order. Before creating the sales order record, the sample loads a set of accounting preferences from the current NetSuite account, specifies that the REVERSALVOIDING preference should be disabled (set to false), and saves the preferences. This sample works only in NetSuite OneWorld accounts. Be sure to replace hard-coded values (such as record IDs) with valid values from your NetSuite account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/transaction', 'N/config', 'N/record'], function(transaction, config, record) {
6      function voidSalesOrder() {
7          var accountingConfig = config.load({
8              type: config.Type.ACCOUNTING_PREFERENCES
9          });
10         accountingConfig.setValue({
11             fieldId: 'REVERSALVOIDING',
12             value: false
13         });
14
15         accountingConfig.save();
16
17         var salesOrderObj = record.create({
18             type: 'salesorder',
19             isDynamic: false
20         });
21         salesOrderObj.setValue({

```

```

22     fieldId: 'entity',
23     value: 107
24   });
25   salesOrderObj.setSublistValue({
26     sublistId: 'item',
27     fieldId: 'item',
28     value: 233,
29     line: 0
30   });
31   salesOrderObj.setSublistValue({
32     sublistId: 'item',
33     fieldId: 'amount',
34     value: 1,
35     line: 0
36   });
37
38   var salesOrderId = salesOrderObj.save();
39
40   var voidSalesOrderId = transaction.void({
41     type: record.Type.SALES_ORDER,
42     id: salesOrderId
43   });
44
45   var salesOrder = record.load({
46     type: 'salesorder',
47     id: voidSalesOrderId
48   });
49
50   // The value of the memo field is 'VOID'
51   var memo = salesOrder.getValue({
52     fieldId: 'memo'
53   });
54 }
55
56 voidSalesOrder();
57 });

```

transaction.void(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Voids a transaction record object and return an id that indicates the type of void performed. The type of void performed depends on the targeted account's preference settings.  Important: After you void a transaction, you cannot make changes to the transaction that impact the general ledger.
Returns	An ID returned as a number. ■ If a direct void is performed, returns the ID of the record voided. ■ If a void by reversing journal is performed, returns the ID of the newly created voiding journal.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/transaction Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	number string	required	Internal ID of the specific transaction record instance to void.	2015.2
options.type	transaction.Type	required	Internal ID of the type of transaction record to void	2015.2

Errors

Error Code	Message	Thrown If
INVALID_RECORD_TYPE	—	The type argument passed is not valid or the record type is not voidable.
THAT_RECORD_DOES_NOT_EXIST	—	The id argument passed is not valid.
SSS_MISSING_REQD_ARGUMENT	—	The type or id argument is missing.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/transaction Module Script Sample](#).

In the code sample below, a sales order is voided. You must disable the **Void Transactions Using Reversing Journals** feature in your NetSuite account by navigating to Setup > Accounting > Preferences > Accounting Preferences to avoid an error being thrown. To see which transactions support this feature, refer to the chart in [transaction.Type](#). For more information about voiding transactions, see the help topic [Voiding, Deleting, or Closing Transactions](#).

```

1 //Add additional code
2 ...
3 var voidSalesOrderId = transaction.void({
4   type: transaction.Type.SALES_ORDER, //disable Void Transactions Using Reversing Journals in Account Pref
5   id: salesOrderId
6 });
7 ...
8 //Add additional code

```

transaction.void.promise(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	voids a transaction record object asynchronously and return an ID that indicates the type of void performed: If a direct void is performed, this method returns the ID of the record that was voided.
---------------------------	--

	■ If a void by reversing journal is performed, this method returns the ID of the newly created voiding journal. The type of void performed depends on the targeted account's preference settings.  Important: After you void a transaction, you cannot make changes to the transaction that impact the general ledger.  Note: The parameters and errors thrown for this method are the same as those for transaction.void(options). For more information about promises, see Promise Object.
Returns	Promise Object
Synchronous Version	transaction.void(options)
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/transaction Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete promise script example, see [Promise Object](#).

```

1 //Add additional code
2 ...
3 var voidSalesOrderId = transaction.void.promise({
4     type: record.Type.SALES_ORDER,
5     id: salesOrderId
6 });
7 ...
8 //Add additional code

```

transaction.Type

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported transaction record types. This enum is used for the options.type parameter of the transaction.void(options) method.
Module	N/transaction Module
Sibling Module Members	N/transaction Module Members
Since	2015.2

Note: For transaction types that use direct void, you must disable the **Void Transactions Using Reversing Journals** feature in your account to avoid an error being thrown. To disable this feature, go to Setup > Accounting > Preferences > Accounting Preferences.

Values

Value (Transaction Record)	Supported Void Type
ASSEMBLY_BUILD	None
ASSEMBLY_UNBUILD	None
BIN_TRANSFER	None
BIN_WORKSHEET	None
BLANKET_PURCHASE_ORDER	None
CASH_REFUND	Direct Void
CASH_SALE	Direct Void
CHECK	Void by Reversing Journal
CREDIT_CARD_CHARGE	None
CREDIT_CARD_REFUND	None
CREDIT_MEMO	Direct Void
CUSTOMER_DEPOSIT	Direct Void
CUSTOMER_PAYMENT	Direct Void
CUSTOMER_PAYMENT_AUTHORIZATION	None
CUSTOMER_REFUND	Direct Void and Void by Reversing Journal
CUSTOM_TRANSACTION	Void by Reversing Journal
DEPOSIT	None
DEPOSIT_APPLICATION	None
ESTIMATE	Direct Void
EXPENSE_REPORT	Direct Void
FULFILLMENT_REQUEST	None
INBOUND_SHIPMENT	None
INVENTORY_ADJUSTMENT	None
INVENTORY_COST_REVALUATION	None
INVENTORY_COUNT	None
INVENTORY_STATUS_CHANGE	None
INVENTORY_TRANSFER	None

Value (Transaction Record)	Supported Void Type
INVOICE	Direct Void
ITEM_FULFILLMENT	None
ITEM_RECEIPT	None
JOURNAL_ENTRY	Direct Void
OPPORTUNITY	None
ORDER_RESERVATION	None
PAYCHECK	None
PAYCHECK_JOURNAL	Direct Void
PERIOD_END_JOURNAL	None
PURCHASE_CONTRACT	None
PURCHASE_ORDER	None
PURCHASE_REQUISITION	None
RETURN_AUTHORIZATION	Direct Void
REVENUE_ARRANGEMENT	None
REVENUE_COMMITMENT	None
REVENUE_COMMITMENT_REVERSAL	None
SALES_ORDER	Direct Void
STORE_PICKUP_FULFILLMENT	None
TRANSFER_ORDER	Direct Void
VENDOR_BILL	Direct Void
VENDOR_CREDIT	Direct Void
VENDOR_PAYMENT	Direct Void and Void by Reversing Journal
VENDOR_PREPAYMENT	Direct Void and Void by Reversing Journal
VENDOR_PREPAYMENT_APPLICATION	Direct Void and Void by Reversing Journal
VENDOR_RETURN_AUTHORIZATION	Direct Void
WAVE	Direct Void
WORK_ORDER	Direct Void
WORK_ORDER_CLOSE	Direct Void
WORK_ORDER_COMPLETION	Direct Void
WORK_ORDER_ISSUE	Direct Void

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/transaction Module Script Sample](#).

```
1 //Add additional code
2 ...
3 var voidSalesOrderId = transaction.void({
4     type: transaction.Type.SALES_ORDER,
5     id: salesOrderId
6 });
7 ...
8 //Add additional code
```

N/translation Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/translation module to allow SuiteScript developers to interact with NetSuite Translation Collections programmatically. For more information about Translation Collections, see the help topic [Translation Collections Overview](#).

Go To Samples

You can watch a video that demonstrates how to use the N/translation module to work with Translation Collections.

[Using the N/translation Module for Translation Collections](#)



Note: The N/translation module provides read-only access. If you want to create or modify Translation Collections, you can do so in the NetSuite UI at Customization > Translations > Manage Translations

For more information about managing Translation Collections in the UI, see the help topic [Manage Translations](#).

A Translation Collection is encapsulated in the [translation.Handle](#) object. The translation.Handle object is a hierarchical object, which means that each node in the object is either another translation.Handle object or a [translation.Translator](#) function. Translator functions combine strings with parameters. When you create a Translation Collection in the NetSuite UI, you can include parameter placeholders in your translation strings. The translator function injects the specified parameter values into the placeholders in the returned translation string.

In your scripts, use [translation.get\(options\)](#) to get a translation. Translator function you can use to obtain specific translated strings in a collection. Consider the following code sample:

```
1 // key HELLO_1 = 'Hello, {1}'
2 ...
3 message: translation.get({
4     collection: 'custcollection_my_strings',
5     key: 'HELLO_1'
6 })({
7     params: ['NetSuite']
8 })
```

In this sample, if the string value of the HELLO_1 key is "Hello, {1}", the `translation.Translator` function combines the string with the `params` parameter value and returns "Hello, NetSuite". You can also use [translation.load\(options\)](#) to load translation strings from one or more Translation Collections. For information about the way strings are added to and formatted in collections, see the help topic [Working with Translation Collection Strings](#).

You can load collections in different language locales by using the `locales` parameter of [translation.load\(options\)](#). You can also use [translation.selectLocale\(options\)](#) to create a `translation.Handle` object in a specific locale from an existing `translation.Handle` object.

In This Help Topic

- [N/translation Module Members](#)

N/translation Module Members

Member Type	Name	Return Type / Value Type	Supported Script Type	Description
Object	translation.Handle	Object	Client and server scripts	Encapsulates a Translation Collection for a locale.
	translation.Translator	Object / Function	Client and server scripts	Represents a translator function that returns translated strings. The translated strings include variables that are passed as parameters to the translator function.
Method	translation.get(options)	translation.Translator	Client and server scripts	Creates a translator function for a key in the specified Translation Collection and locale.
	translation.load(options)	translation.Handle	Client and server scripts	Creates a translation.Handle object with translations for the specified Translation Collections and locales.
	translation.selectLocale(options)	translation.Handle	Client and server scripts	Creates a translation.Handle object in the specified locale from an existing <code>translation.Handle</code> object.
Enum	translation.Locale	enum	Client and server scripts	Holds the supported locales for Translation Collections. Use this enum to pass the locale argument to translation.get(options) and translation.selectLocale(options) .

N/translation Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/translation module.

- [Access Translation Strings](#)
- [Access Translation Strings Using a Non-Default Locale](#)
- [Access Parameterized Translation Strings](#)
- [Load Specific Translation Strings from a Collection](#)

■ Load Translation Strings By Key from Multiple Translation Collections

Access Translation Strings

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample accesses translation strings one at a time using [translation.get\(options\)](#). This method returns a translator function, which is subsequently called with any specified parameters. The translator function returns the string in the user's session locale by default.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/ui/message', 'N/translation'],
6    function(message, translation) {
7
8      // Create a message with translated strings
9      var myMsg = message.create({
10         title: translation.get({
11             collection: 'custcollection_my_strings',
12             key: 'MY_TITLE'
13         })(),
14         message: translation.get({
15             collection: 'custcollection_my_strings',
16             key: 'MY_MESSAGE'
17         })(),
18         type: message.Type.CONFIRMATION
19     });
20
21     // Show the message for 5 seconds
22     myMsg.show({
23         duration: 5000
24     });
25 });

```

Access Translation Strings Using a Non-Default Locale

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample accesses translation strings using a locale other than the default locale. When you call [translation.get\(options\)](#) and do not specify a locale, the method uses the current user's session locale. You can use the `options.locale` parameter to specify another locale. The [translation.Locale](#) enum lists all locales that are enabled for a company, and you can use these locales in [translation.get\(options\)](#). The [translation.Locale](#) enum also includes two special values: `CURRENT` and `COMPANY_DEFAULT`. The `CURRENT` value represents the current user's locale, and the `COMPANY_DEFAULT` value represents the default locale for the company.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**

```

```

2  /**
3   * @NApiVersion 2.x
4   */
5   require(['N/ui/message', 'N/translation'],
6     function(message, translation) {
7
8       // Create a message with translated strings
9       var myMsg = message.create({
10         title: translation.get({
11           collection: 'custcollection_my_strings',
12           key: 'MY_TITLE',
13           locale: translation.Locale.COMPANY_DEFAULT
14         })(),
15         message: translation.get({
16           collection: 'custcollection_my_strings',
17           key: 'MY_MESSAGE',
18           locale: translation.Locale.COMPANY_DEFAULT
19         })(),
20         type: message.Type.CONFIRMATION
21       });
22
23       // Show the message for 5 seconds
24       myMsg.show({
25         duration: 5000
26       });
27     });

```

Access Parameterized Translation Strings

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample accesses parametrized translation strings. When you create a Translation Collection in the NetSuite UI, you can include parameter placeholders in your translation strings. Placeholders use braces and a number (starting from 1). The translator function injects the specified parameter values into the placeholders in the translation string. For example, "Hello, {1}!" is a valid translation string, where {1} is a placeholder for a parameter. In this sample, the parameter "NetSuite" is provided to the translator function returned from [translation.get\(options\)](#), and the translator function returns a translated string of "Hello, NetSuite!"

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   require(['N/ui/message', 'N/translation'],
6     function(message, translation) {
7
8       // Create a message with translated strings
9       var myMsg = message.create({
10         title: translation.get({
11           collection: 'custcollection_my_strings',
12           key: 'MY_TITLE'
13         })(),
14         message: translation.get({
15           collection: 'custcollection_my_strings',
16           key: 'HELLO_1'
17         })({
18           params: ['NetSuite']
19         }),
20         type: message.Type.CONFIRMATION

```



```

21     });
22
23     // Show the message for 5 seconds
24     myMsg.show({
25         duration: 5000
26     });
27 });

```

Load Specific Translation Strings from a Collection

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads specific translation strings from a collection. The [translation.load\(options\)](#) method can load a maximum of 1,000 translation strings. If you need only a few of the translation strings in a collection, you can load only the strings you need instead of loading the entire collection.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   require(['N/ui/message', 'N/translation'],
6     function(message, translation) {
7
8       // Load translation strings by key
9       var localizedStrings = translation.load({
10         collections: [{
11           alias: 'myCollection',
12           collection: 'custcollection_my_strings',
13           keys: ['MY_TITLE', 'MY_MESSAGE']
14         }]
15       });
16
17       // Create a message with translated strings
18       var myMsg = message.create({
19         title: localizedStrings.myCollection.MY_TITLE(),
20         message: localizedStrings.myCollection.MY_MESSAGE(),
21         type: message.Type.CONFIRMATION
22       });
23
24       // Show the message for 5 seconds
25       myMsg.show({
26         duration: 5000
27       });
28 });

```

Load Translation Strings By Key from Multiple Translation Collections

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads translation strings by key from multiple Translation Collections in a single call of [translation.load\(options\)](#). This method can load a maximum of 1,000 translation strings, regardless of whether the strings are loaded from one collection or multiple collections.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @ApiVersion 2.x
3   */
4
5  require(['N/ui/message', 'N/translation'],
6    function(message, translation) {
7
8      // Load two Translation Collections
9      var localizedStrings = translation.load({
10         collections: [{
11             alias: 'myCollection',
12             collection: 'custcollection_my_strings',
13             keys: ['MY_TITLE']
14         }, {
15             alias: 'myOtherCollection',
16             collection: 'custcollection_other_strings',
17             keys: ['MY_OTHER_MESSAGE']
18         }]
19     });
20
21     // Create a message with translated strings
22     var myMsg = message.create({
23         title: localizedStrings.myCollection.MY_TITLE(),
24         message: localizedStrings.myOtherCollection.MY_OTHER_MESSAGE(),
25         type: message.Type.CONFIRMATION
26     });
27
28     // Show the message for 5 seconds
29     myMsg.show({
30         duration: 5000
31     });
32 });

```

translation.Handle

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates a Translation Collection for a locale. Use translation.load(options) to create a translation.Handle object with translations for the specified Translation Collections and locales. Use translation.selectLocale(options) to create a translation.Handle object in the specified locale from an existing translation.Handle object. The translation.Handle object is a hierarchical object, which means that each node in the object is either another translation.Handle object or a translation.Translator function. Translator functions combine strings with parameters. When you create a Translation Collection in the NetSuite UI, you can include parameter placeholders in your translation strings. The translator function injects the specified parameter values into the placeholders in the returned translation string.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Module	N/translation Module
Since	2019.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/translation Module Script Samples](#).

```

1 // Add additional code
2 ...
3 var localizedStrings = translation.load({
4     collections: [{
5         alias: 'myCollection',
6         collection: 'custcollection_my_strings',
7         keys: ['MY_TITLE', 'MY_MESSAGE']
8     }]
9 });
10
11 let myMsg = message.create({
12     title: localizedStrings.myCollection.MY_TITLE(),
13     message: localizedStrings.myCollection.MY_MESSAGE(),
14     type: message.Type.CONFIRMATION
15 });
16 ...
17 // Add additional code

```

```

1 /**
2  * @NApiVersion 2.1
3  * @NScriptType clientscript
4  */
5 define(['N/translation'], (translation) => {
6     return {
7         pageInit: function(context) {
8             let handle = translation.load({
9                 collections: [
10                     {alias: "phrases", collection: "CUSTCOLLECTION_PHRASES", keys: ["HELLO"]},
11                     {alias: "specialstrings", collection: "CUSTCOLLECTION_SPECIALSTRINGS", keys: ["HELLO_1"]}
12                 ],
13                 locales: ["fr_FR", 'en_US']
14             });
15
16             // Logs hello in company default language - Hello (if default is English)
17             console.log(handle.phrases.HELLO());
18             let frenchHandle = translation.selectLocale({handle: result, locale: "fr_FR"});
19
20             // Logs hello in french - Bonjour
21             console.log(frenchHandle.phrases.HELLO());
22             //logs hello in french - Bonjour
23         }
24     };
25     ...
26 // Add additional code

```

translation.Translator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object / Function Description	Represents a translator function that returns translated strings. Use translation.get(options) to obtain this function for the specified Translation Collection and locale. The translator function is called with any parameters that you specify, and the translator function returns the appropriate translated string. When you create a Translation Collection in the NetSuite UI, you can include parameter placeholders in your translation strings. Translation strings that include placeholders are
--------------------------------------	--

	called parametrized translation strings. Placeholders use braces and a number (starting from 1). The translator function injects the specified parameter values into the placeholders in the translation string. For example, “Hello, {1}!” is a valid translation string, where {1} is a placeholder for a parameter. If you call <code>translation.get(options)</code> and specify a parameter of “NetSuite”, the translator function returns “Hello, NetSuite!” in the appropriate locale.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Module	N/translation Module
Since	2019.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.params</code>	<code>string[]</code>	optional	The parameters to pass to the translator function. The parameter values are used in parametrized translation strings.

Errors

Error Code	Thrown If
<code>WRONG_PARAMETER_TYPE</code>	The function parameters were not passed as an array.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/translation Module Script Samples](#).

```

1 // Add additional code
2 ...
3 let myMsg = message.create({
4   title: translation.get({
5     collection: 'custcollection_my_strings',
6     key: 'MY_TITLE'
7   })(),
8   message: translation.get({
9     collection: 'custcollection_my_strings',
10    key: 'HELLO_1'
11  })({
12    params: ['NetSuite']
13  }),
14   type: message.Type.CONFIRMATION
15 });
16 ...
17 // Add additional code

```

translation.get(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a translator function for a key in the specified Translation Collection and locale. This method returns a translator function, which is subsequently called with any specified parameters. When you call <code>translation.get(options)</code> and do not specify a locale, the method uses the current user's session locale. You can use the <code>options.locale</code> parameter to specify another locale. The translation.Locale enum lists all locales that are enabled for a company, and you can use these locales in <code>translation.get(options)</code> .
Returns	translation.Translator
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	1 unit
Module	N/translation Module
Sibling Object Members	N/translation Module Members
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
<code>options.collection</code>	string	required	The script ID of the collection.
<code>options.key</code>	string	required	A valid key from the collection.
<code>options.locale</code>	string	optional	A valid locale from the translation.Locale enum. If a locale is not specified, the locale from the current session is used as the default locale.

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A collection or key parameter is missing.
INVALID_TRANSLATION_KEY	The format of a specified key is invalid.
INVALID_TRANSLATION_COLLECTION	The format of a specified collection is invalid.
INVALID_LOCALE	The format of a specified locale is invalid.
TRANSLATION_KEY_NOT_FOUND	A specified translation key was not found.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/translation Module Script Samples](#).

```

1 // Add additional code
2 ...
3 let myMsg = message.create({
4   title: translation.get({
5     collection: 'custcollection_my_strings',
6     key: 'MY_TITLE'
7   })(),
8   message: translation.get({
9     collection: 'custcollection_my_strings',
10    key: 'HELLO_1'
11  })({
12    params: ['NetSuite']
13  }),
14   type: message.Type.CONFIRMATION
15 });
16 ...
17 // Add additional code

```

translation.load(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a translation.Handle object with translations for the specified Translation Collections and locales. This method returns a translation.Handle object with translation strings organized by collection and ID. Every node in a translation.Handle object is either another translation.Handle object or a translation.Translator function. You can load translation strings from multiple Translation Collections in a single call of translation.load(options). You must specify the keys of individual translation strings that you want to load. You cannot load all of the terms in a Translation Collection at one time. The translation.load(options) method can load a maximum of 1,000 translation strings, regardless of whether the strings are loaded from one collection or multiple collections. When you load translation strings using translation.load(options), you can specify a list of valid locales for the strings. You can use these locales when you select a locale using translation.selectLocale(options). If you specify more than one locale when you call translation.load(options), the first specified locale in the list is used for the created translation.Handle object. If you want to use a different locale from the list, use translation.selectLocale(options), which returns a translation.Handle object in the specified locale. You must load a locale using translation.load(options) before you can select it using translation.selectLocale(options).
Returns	translation.Handle
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	10 units
Module	N/translation Module
Sibling Object Members	N/translation Module Members

Since	2019.1
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.collections	Object[]	required	A list of translation.Handle objects to load.
options.collections.alias	string	required	An alias to identify the collection. This alias is used by the script to determine the collection to load.
options.collections.collection	string	required	The script ID of the collection to load.
options.collections.keys	string[]	required	A list of translation keys from the collection to load.
options.locales	string[]	optional	A list of locales to load the collection in. Use the values in the translation.Locale enum to set this value.

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	One of the array parameters (options.collections, options.collections.keys, or options.locales) is not an array.
SSS_MISSING_REQD_ARGUMENT	A collection or key parameter is missing.
INVALID_TRANSLATION_KEY	The format of a specified key is invalid.
INVALID_TRANSLATION_COLLECTION	The format of a specified collection is invalid.
INVALID_LOCALE	The format of a specified locale is invalid.
INVALID_ALIAS	The format of a specified alias is invalid.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/translation Module Script Samples](#).

```

1 // Add additional code
2 ...
3 let localizedStrings = translation.load({
4   collections: [{
5     alias: 'myCollection',
6     collection: 'custcollection_my_strings',
7     keys: ['MY_TITLE', 'MY_MESSAGE']
8   }]
9 });

```

```

10
11 let myMsg = message.create({
12     title: localizedStrings.myCollection.MY_TITLE(),
13     message: localizedStrings.myCollection.MY_MESSAGE(),
14     type: message.Type.CONFIRMATION
15 });
16 ...
17 // Add additional code

```

translation.selectLocale(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a translation.Handle object in the specified locale from an existing translation.Handle object. This method returns a translation.Handle object that contains the same translation strings as the options.handle object, and the strings are in the options.locale locale. Before you can use this method to select a locale, the locale must be loaded using the locales parameter of translation.load(options) .
Returns	translation.Handle
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/translation Module
Sibling Object Members	N/translation Module Members
Since	2019.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.handle	translation.Handle	required	The translation.Handle object to select a locale for.
options.locale	string	required	The locale to select. Use the values in the translation.Locale enum to set this value.

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A handle or locale parameter is missing.

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The options.handle parameter is not a translation.Handle object.
INVALID_LOCALE	The specified translation.Handle object uses an unknown or unsupported locale.
TRANSLATION_HANDLE_IS_IN_AN_ILLEGAL_STATE	The specified translation.Handle object is in an illegal state.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/translation Module Script Samples](#).

```

1 // Add additional code
2 ...
3 let germanStrings = translation.load({
4   collections: [{
5     alias: 'myCollection',
6     collection: 'custcollection_my_strings',
7     keys: ['MY_TITLE', 'MY_MESSAGE'],
8   }],
9   locales: [translation.Locale.de_DE, translation.Locale.es_ES]
10 });
11
12 let spanishStrings = translation.selectLocale({
13   handle: germanStrings,
14   locale: translation.Locale.es_ES
15 });
16 ...
17 // Add additional code

```

translation.Locale

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported locales for Translation Collections. Use this enum to set the locale argument to translation.get(options), translation.selectLocale(options), and translation.load(options). This enum lists all locales that are enabled for a company. This enum also includes two special values: CURRENT and COMPANY_DEFAULT. The CURRENT value represents the current user's locale, and the COMPANY_DEFAULT value represents the default locale for the company.
Module	N/translation Module
Sibling Object Members	N/translation Module Members
Since	2019.1

Values

The following table lists all possible locale values and the respective languages. Typically, only some of these locales will be enabled for a company and available to use with Translation Collections.

Value (Locale)	Language
CURRENT	Language currently set by the user
COMPANY_DEFAULT	Default company language
af_ZA	Afrikaans (South Africa)
ar	Arabic
bg_BG	Bulgarian (Bulgaria)
bn_BD	Bengali (Bangladesh)
bs_BA	Bosnian (Bosnia and Herzegovina)
cs_CZ	Czech (Czechia)
da_DK	Danish (Denmark)
de_DE	German (Germany)
el_GR	Greek (Greece)
en	English
en_AU	English (Australia)
en_CA	English (Canada)
en_GB	English (United Kingdom)
en_US	English (United States)
es_AR	Spanish (Argentina)
es_ES	Spanish (Spain)
et_EE	Estonian (Estonia)
fa_IR	Farsi (Iran)
fi_FI	Finnish (Finland)
fr_CA	French (Canada)
fr_FR	French (France)
gu_IN	Gujarati (India)
he_IL	Hebrew (Israel)
hi_IN	Hindi (India)
hr_HR	Croatian (Croatia)
hu_HU	Hungarian (Hungary)
hy_AM	Armenian (Armenia)

Value (Locale)	Language
id_ID	Indonesian (Indonesia)
is_IS	Icelandic (Iceland)
it_IT	Italian (Italy)
ja_JP	Japanese (Japan)
kn_IN	Kannada (India)
ko_KR	Korean (Korea)
lb_LU	Luxembourgish (Luxembourg)
lt_LT	Lithuanian (Lithuania)
lv_LV	Latvian (Latvia)
mr_IN	Marathi (India)
ms_MY	Malay (Malaysia)
n1_NL	Dutch (Netherlands)
no_NO	Norwegian (Norway)
pa_IN	Punjabi (India)
pl_PL	Polish (Poland)
pt_BR	Portuguese (Brazil)
pt_PT	Portuguese (Portugal)
ro_RO	Romanian (Romania)
ru_RU	Russian (Russia)
sh_RS	Serbian (Latin)
sk_SK	Slovakian (Slovakia)
sl_SI	Slovenian (Slovenia)
sq_AL	Albanian (Albania)
sr_RS	Serbian (Serbia)
sv_SE	Swedish (Sweden)
ta_IN	Tamil (India)
te_IN	Telugu (India)
th_TH	Thai (Thailand)
t1_PH	Tagalog (Philippines)
tr_TR	Turkish (Türkiye)
uk_UA	Ukrainian (Ukraine)
vi_VN	Vietnamese (Viet Nam)

Value (Locale)	Language
zh_CN	Chinese (Simplified)
zh_TW	Chinese (Traditional)

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/translation Module Script Samples](#).

```

1 // Add additional code
2 ...
3 let myMsg = message.create({
4     title: translation.get({
5         collection: 'custcollection_my_strings',
6         key: 'MY_TITLE',
7         locale: translation.Locale.COMPANY_DEFAULT
8     })(),
9     message: translation.get({
10        collection: 'custcollection_my_strings',
11        key: 'MY_MESSAGE',
12        locale: translation.Locale.COMPANY_DEFAULT
13    })(),
14    type: message.Type.CONFIRMATION
15 });
16 ...
17 // Add additional code

```

N/ui Modules

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Modules within the N/ui namespace allow you to build a custom UI using SuiteScript 2.x. Use ['N/ui'] as the first argument of the define function as a shortcut to load the three modules (N/ui/dialog, N/ui/message, and N/ui/serverWidget) on server scripts, or the N/ui/dialog and N/ui/message on client scripts.

- **N/ui/dialog Module** — Used to create modal dialog boxes that can be used to present additional options or alerts. This module uses JavaScript promises to manage dialogs asynchronously.
- **N/ui/message Module** — Used to display and manage messages at the top of your User Interface.
- **N/ui/serverWidget Module** — Contains the UI components used to work with user interfaces in NetSuite. This module is used to create custom pages, forms, lists and widgets that have the NetSuite look-and-feel.

N/ui/dialog Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/ui/dialog module to create a modal dialog that persists until a button on the dialog is pressed.



Important: SuiteScript does not support direct access to the NetSuite UI through the Document Object Model (DOM). The NetSuite UI should only be accessed using SuiteScript APIs.

Go To Samples {...}

In This Help Topic

- [N/ui/dialog Module Members](#)

N/ui/dialog Module Members

Member Type	Name	Property Type / Method Return Type	Supported Script Types	Description
Method	dialog.alert(options)	Promise	Client scripts	Creates an Alert dialog with an OK button.
	dialog.confirm(options)	Promise	Client scripts	Creates a Confirm dialog with OK and Cancel buttons.
	dialog.create(options)	Promise	Client scripts	Creates a dialog with specified buttons.

N/ui/dialog Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/ui/dialog module:

- [Create an Alert Dialog](#)
- [Create a Confirmation Dialog](#)
- [Create a Dialog with Buttons](#)
- [Create a Dialog that Includes a Default Button](#)

Create an Alert Dialog

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create an alert dialog.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Note: To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```
1 /**
2  * @NApiVersion 2.1
3  * @NScriptType ClientScript
```

```

4  */
5  define(['N/ui/dialog'], (dialog) => {
6      function pageInit() {
7          let options = {
8              title: 'I am an Alert',
9              message: 'Click OK to continue.'
10         };
11
12         function success(result) {
13             console.log('Success with value ' + result);
14         }
15
16         function failure(reason) {
17             console.log('Failure: ' + reason);
18         }
19
20         dialog.alert(options).then(success).catch(failure);
21     }
22
23     return {
24         pageInit: pageInit
25     };
26
27 });

```

The following screenshot shows the Alert dialog created in this example:



Create a Confirmation Dialog

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create a confirmation dialog.

i Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

i Note: To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   */
5  define(['N/ui/dialog'], (dialog) => {
6      function pageInit() {
7          let options = {
8              title: 'I am a Confirmation',
9              message: 'Press OK or Cancel'

```

```

10     };
11
12     function success(result) {
13         console.log('Success with value ' + result);
14     }
15
16     function failure(reason) {
17         console.log('Failure: ' + reason);
18     }
19
20     dialog.confirm(options).then(success).catch(failure);
21 }
22
23 return {
24     pageInit: pageInit
25 };
26
27 });

```

The following screenshot shows the Confirmation dialog created in this example:



Create a Dialog with Buttons

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create a dialog with three buttons.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Note: To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   */
5  define(['N/ui/dialog'], (dialog) => {
6      function pageInit() {
7          let button1 = {
8              label: 'I am A',
9              value: 1
10         };
11         let button2 = {
12             label: 'I am B',
13             value: 2
14         };
15         let button3 = {
16             label: 'I am C',

```

```

17     value: 3
18   };
19   let options = {
20     title: 'Alphabet Test',
21     message: 'Which One?',
22     buttons: [button1, button2, button3]
23   };
24
25   function success(result) {
26     console.log('Success with value ' + result);
27   }
28
29   function failure(reason) {
30     console.log('Failure: ' + reason);
31   }
32
33   dialog.create(options).then(success).catch(failure);
34 }
35
36 return {
37   pageInit: pageInit
38 };
39
40 });

```

The following screenshot shows the dialog with buttons created in this example:



Create a Dialog that Includes a Default Button

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to create a dialog without specifying any buttons. The default behavior is to create a dialog that includes a single button with the label OK.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

i Note: To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

! Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

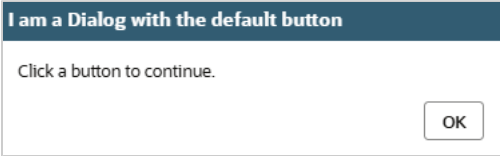
1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   */
5  define(['N/ui/dialog'], (dialog) => {
6    function pageInit() {
7      let options = {
8        title: 'I am a Dialog with the default button',

```



```
9      message: 'Click a button to continue.',
10    };
11
12    function success(result) {
13      console.log('Success with value ' + result);
14    }
15
16    function failure(reason) {
17      console.log('Failure: ' + reason);
18    }
19
20    dialog.create(options).then(success).catch(failure);
21  }
22
23  return {
24    pageInit: pageInit
25  };
26
27  });
```

The following screenshot shows the dialog with the default button created in this example:



dialog.alert(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an Alert dialog with an OK button.
Returns	Promise Object . To run a callback function when the OK button is clicked, pass a function to the then portion of the Promise object. When the OK button is clicked, true is passed to the callback. You do not have to use the Promise object unless there is an action you want performed after the user clicks the OK button.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/dialog Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

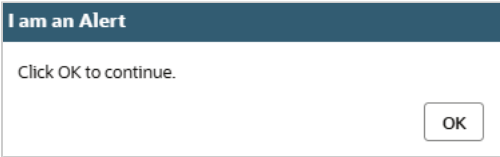
Parameter	Type	Required / Optional	Description	Since
options.title	string	optional	The alert dialog title. This value defaults to an empty string.	2016.1
options.message	string	optional	The content of the alert dialog. This value defaults to an empty string.	2016.1

Syntax

**Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/dialog Module Script Samples](#).

```
1 //Add additional code
2 ...
3 function success(result) { console.log('Success with value: ' + result) }
4 function failure(reason) { console.log('Failure: ' + reason) }
5
6 dialog.alert({
7     title: 'I am an Alert',
8     message: 'Click OK to continue.'
9 }).then(success).catch(failure);
10 ...
11 //Add additional code
```

The following screenshot shows an example of an Alert dialog:



dialog.confirm(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a Confirm dialog with OK and Cancel buttons.
Returns	Promise Object . To run a callback function when the OK button is pressed, pass a function to the then portion of the Promise object. The value of the pressed button, where OK is true and Cancel is false , is passed to the callback.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/dialog Module
Since	2016.1

Parameters

**Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	optional	The confirmation dialog title. This value defaults to an empty string.	2016.1

Parameter	Type	Required / Optional	Description	Since
options.message	string	optional	The content of the confirmation dialog. This value defaults to an empty string.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/dialog Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var options = {
4     title: "I am a Confirmation",
5     message: "Press OK or Cancel"
6 };
7
8 function success(result) {
9     console.log("Success with value " + result);
10 }
11
12 function failure(reason) {
13     console.log("Failure: " + reason);
14 }
15
16 dialog.confirm(options).then(success).catch(failure);
17
18 ...
19 //Add additional code

```

The following screenshot shows an example of a Confirmation dialog:



dialog.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a dialog with specified buttons.
Returns	Promise Object . To run a callback function when a button is pressed, pass a function to the then portion of the Promise object. The value of the button pressed is passed to the callback.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/dialog Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.buttons	string[]	optional	A list of buttons to include in the dialog. Each item in the button list must be a Javascript Object that contains a label and a value property. By default, a single button with the label OK and the value true is used.	2016.1
options.title	string	optional	The dialog title. This value defaults to an empty string.	2016.1
options.message	string	optional	The content of the dialog. This value defaults to an empty string.	2016.1

Errors

Error Code	Error Message	Thrown If
WRONG_PARAMETER_TYPE	Wrong parameter type: {1} is expected as {2}.	The options.buttons parameter is specified and is not an array.
BUTTONS_MUST_INCLUDE_BOTH_A_LABEL_AND_VALUE	Buttons must include both a label and value.	The options.buttons parameter is specified and one or more items do not have a label, a value, or both.

Syntax

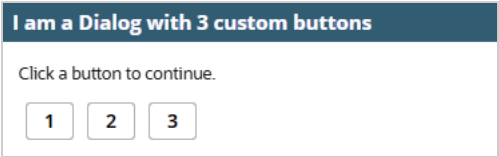
Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/dialog Module Script Samples](#).

```

1      //Add additional code
2      ...
3      var options = {
4          title: 'I am a Dialog with 3 custom buttons',
5          message: 'Click a button to continue.',
6          buttons: [
7              { label: '1', value: 1 },
8              { label: '2', value: 2 },
9              { label: '3', value: 3 }
10         ];
11
12         function success(result) { console.log('Success with value: ' + result) }
13         function failure(reason) { console.log('Failure: ' + reason) }
14
15         dialog.create(options).then(success).catch(failure);
16         ...
17         //Add additional code

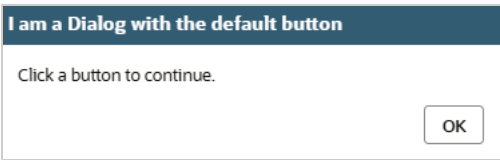
```

The following screenshot shows an example of a Custom dialog with three buttons:



If no buttons are specified, a single value with the label OK is used:

```
1 //Add additional code
2 ...
3 dialog.create({
4     title: 'I am a Dialog with the default button',
5     message: 'Click a button to continue.'
6 });
7 ...
8 //Add additional code
```



N/ui/message Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/ui/message module to display a message at the top of the screen under the menu bar.

 **Important:** SuiteScript does not support direct access to the NetSuite UI through the Document Object Model (DOM). The NetSuite UI should only be accessed using SuiteScript APIs.

Go To Samples {...}

In This Help Topic

- N/ui/message Members
- Message Object Members

N/ui/message Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	message.Message	void	Client scripts	Encapsulates the Message object that gets created when calling the message.create(options) method.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	message.create(options)	message.Message	Client scripts	Creates a message that can be displayed or hidden near the top of the page.
Enum	message.Type	enum	Client scripts	Indicates the type of message to create and display, which specifies the background color of the message and other message indicators. Use this enum to set the value of the <code>options.type</code> parameter of the message.create(options) method.

Message Object Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Message.hide()	void	Client scripts	Hides the message.
	Message.show(options)	void	Client scripts	Shows the message.

N/ui/message Module Script Sample

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/ui/message module.

Create Confirmation, Information, Warning, and Error Messages

The following sample shows how to create messages for the four available types (confirmation, information, warning, and error).

 **Note:** This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

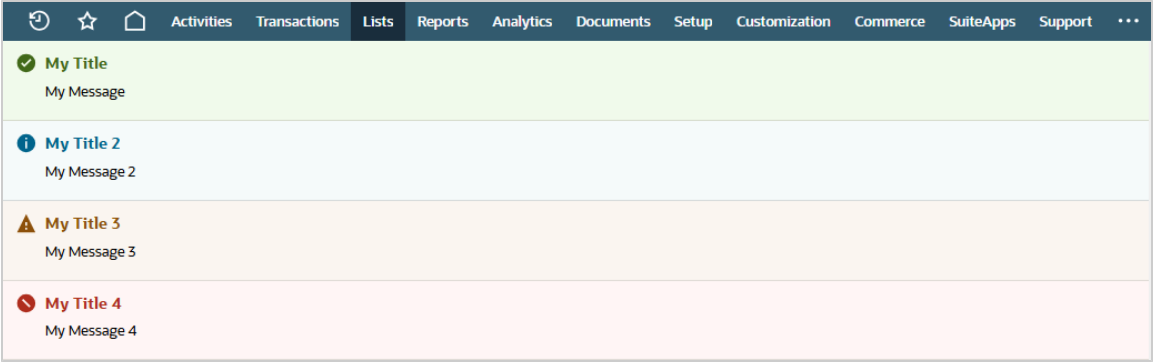
 **Note:** To debug client scripts like the following, you should use Chrome DevTools for Chrome, Firebug debugger for Firefox, or Microsoft Script Debugger for Internet Explorer. For information about these tools, see the documentation provided with each browser. For more information about debugging SuiteScript client scripts, see the help topic [Debugging Client Scripts](#).

 **Important:** This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

1 | /**

```
2  * @NApiVersion 2.1
3  */
4  define(['N/ui/message'], (message) => {
5    let myMsg = message.create({
6      title: 'My Title',
7      message: 'My Message',
8      type: message.Type.CONFIRMATION
9    });
10   myMsg.show({
11     duration: 5000 // will disappear after 5s
12   });
13
14   let myMsg2 = message.create({
15     title: 'My Title 2',
16     message: 'My Message 2',
17     type: message.Type.INFORMATION
18   });
19   myMsg2.show();
20   setTimeout(myMsg2.hide, 15000); // will disappear after 15s
21
22   let myMsg3 = message.create({
23     title: 'My Title 3',
24     message: 'My Message 3',
25     type: message.Type.WARNING,
26     duration: 20000
27   });
28   myMsg3.show(); // will disappear after 20s
29
30   let myMsg4 = message.create({
31     title: 'My Title 4',
32     message: 'My Message 4',
33     type: message.Type.ERROR
34   });
35   myMsg4.show(); // will stay up until hide is called.
36 });
```

You can see the outcome in the following screenshot:



message.Message

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The Message object that gets created when calling the message.create(options) method.
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/message Module

Methods and properties	Message Object Members
Since	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/message Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var newMsg = message.create({
4   title: "New Message Title",
5   message: "This is an new informational message",
6   type: message.Type.INFORMATION
7 });
8
9 //newMsg is the message.Message object created
10 ...
11 //Add additional code

```

Message.hide()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Hides the message.
Returns	void
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/message Module
Since	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/message Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var myMsg = message.create({
4   title: "My Title 2",
5   message: "My Message 2",
6   type: message.Type.INFORMATION
7 });
8 myMsg.show();
9 setTimeout(myMsg.hide(), 15000); // hide the message after 15s
10 ...
11 //Add additional code

```


Message.show(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Shows the message.
Returns	void
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/message Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.duration	int string	optional	The amount of time, in milliseconds, to show the message. The default is 0, which shows the message until Message.hide() is called. If you use a string, it will be parsed to a number. If you specify a duration both when you create the message and when you show it, the value when you show it (this parameter) takes precedence.	2016.1

Errors

Error Code	Error Message	Thrown If
WRONG_PARAMETER_TYPE	Wrong parameter type: {1} is expected as {2}.	The options.duration is specified with a non-numerical value or non-string value.

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/message Module Script Sample](#).

```

1      //Add additional code
2      ...
3      var myMsg = message.create({
4          title: "My Title 2",
5          message: "My Message 2",
6          type: message.Type.INFORMATION
7      });
8      myMsg.show({ duration : 1500 });
9      ...
10     //Add additional code

```

message.create(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a message that can be displayed or hidden near the top of the page.
Returns	message.Message .
Supported Script Types	Client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/message Module
Since	2016.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.type	message.Type	required	The type of message to create. Set this value using the message.Type enum.	2016.1
options.title	string	optional	The message title. This value defaults to an empty string.	2016.1
options.message	string	optional	The content of the message. This value defaults to an empty string.	2016.1
options.duration	int string	optional	The amount of time, in milliseconds, to show the message. The default is 0, which shows the message until Message.hide() is called. If you use a string, it will be parsed to a number. If you specify a duration both when you create the message and when you show it using Message.show(options) , the value when you show it takes precedence.	2018.2

Syntax

 **Important:** The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/message Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var myMsg = message.create({
4   title: "My Title",
5   message: "My Message",
6   type: message.Type.CONFIRMATION

```

```
7   });  
8   ...  
9   //Add additional code
```

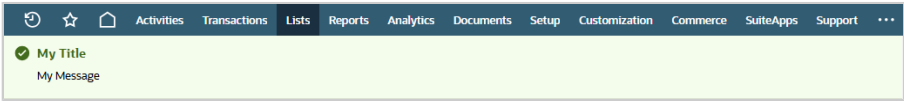
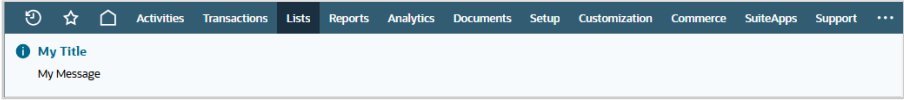
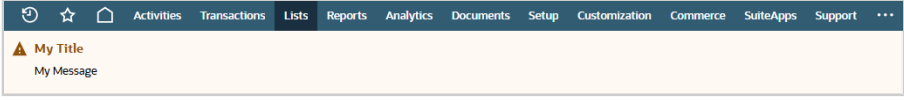
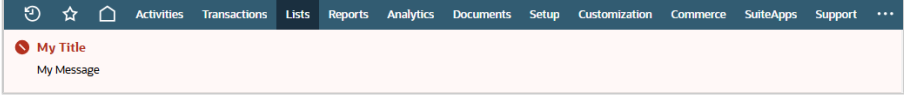
message.Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Indicates the type of message to create, which specifies the background color of the message and other message indicators when the message is displayed. Use this enum to set the value of the options.type parameter of the message.create(options) method.
Module	N/ui/message Module
Sibling Module Members	N/ui/message Members
Since	2016.1

Values

Value	Display characteristics
CONFIRMATION	A green background with a checkmark icon.  A screenshot of a NetSuite interface showing a confirmation message. The message bar has a green background and a checkmark icon. It contains the text "My Title" and "My Message". The interface includes a top navigation bar with tabs like Activities, Transactions, Lists, Reports, Analytics, Documents, Setup, Customization, Commerce, SuiteApps, and Support.
INFORMATION	A blue background with an Information icon.  A screenshot of a NetSuite interface showing an information message. The message bar has a blue background and an information icon. It contains the text "My Title" and "My Message". The interface includes a top navigation bar with tabs like Activities, Transactions, Lists, Reports, Analytics, Documents, Setup, Customization, Commerce, SuiteApps, and Support.
WARNING	A yellow background with a Warning icon.  A screenshot of a NetSuite interface showing a warning message. The message bar has a yellow background and a warning icon. It contains the text "My Title" and "My Message". The interface includes a top navigation bar with tabs like Activities, Transactions, Lists, Reports, Analytics, Documents, Setup, Customization, Commerce, SuiteApps, and Support.
ERROR	A red background with an X icon.  A screenshot of a NetSuite interface showing an error message. The message bar has a red background and an X icon. It contains the text "My Title" and "My Message". The interface includes a top navigation bar with tabs like Activities, Transactions, Lists, Reports, Analytics, Documents, Setup, Customization, Commerce, SuiteApps, and Support.

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/message Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var myMsg = message.create({
4   title: "My Title",
5   message: "My Message",
6   type: message.Type.CONFIRMATION
7 });
8 myMsg.show();
9 ...
10 //Add additional code

```

N/ui/serverWidget Module

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/ui/serverWidget module to work with the user interface within NetSuite. You can use Suitelets to build custom pages and wizards that have a NetSuite look-and-feel. You can also create various components of the NetSuite UI (for example, forms, fields, sublists, tabs).



Important: SuiteScript does not support direct access to the NetSuite UI through the Document Object Model (DOM). The NetSuite UI should only be accessed using SuiteScript APIs.



Important: When you add a UI object to an existing NetSuite page, to minimize the occurrence of field/object name conflicts, the internal ID that references the object must be prefixed with custpage.

Go To Samples

In This Help Topic

- [N/ui/serverWidget Module Members](#)
- [Assistant Object Members](#)
- [AssistantStep Object Members](#)
- [Button Object Members](#)
- [Field Object Members](#)
- [FieldGroup Object Members](#)
- [Form Object Members](#)
- [List Object Members](#)
- [ListColumn Object Members](#)
- [Sublist Object Members](#)

■ Tab Object Members

N/ui/serverWidget Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	serverWidget.Assistant	Object	Suitelets	A scriptable, multi-step NetSuite assistant.
	serverWidget.AssistantStep	Object	Suitelets	A step within a custom NetSuite assistant.
	serverWidget.Button	Object	Suitelets and beforeLoad user events	A button that appears in a UI object.
	serverWidget.Field	Object	Suitelets and beforeLoad user events	A NetSuite field.
	serverWidget.FieldGroup	Object	Suitelets and beforeLoad user events	A field group.
	serverWidget.Form	Object	Suitelets and beforeLoad user events	A NetSuite form.
	serverWidget.List	Object	Suitelets	A list.
	serverWidget.ListColumn	Object	Suitelets	A list column.
	serverWidget.Sublist	Object	Suitelets and beforeLoad user events	A NetSuite sublist.
	serverWidget.Tab	Object	Suitelets and beforeLoad user events	A NetSuite tab and subtabs.
Method	serverWidget.createAssistant(options)	serverWidget.Assistant	Suitelets	Creates and returns a new assistant object.
	serverWidget.createForm(options)	serverWidget.Form	Suitelets	Creates and returns a new form object.
	serverWidget.createList(options)	serverWidget.List	Suitelets	Creates a List object (specifying the title, and whether to hide the navigation bar).
Enum	serverWidget.AssistantSubmitAction	string (read-only)	Suitelets	Holds the string values for submit actions performed by the user. This enum is used to set the value of the Assistant.getLastAction() .
	serverWidget.FieldBreakType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported field break types. This enum is used to set the value of the Field.updateBreakType(options) property.
	serverWidget.FieldDisplayType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported field display types. This enum is used to set the value of the Field.updateDisplayType(options) property.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	serverWidget.FieldLayoutType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for the supported types of field layouts. This enum is used to set the value of the Field.updateLayoutType(options) property.
	serverWidget.FieldType	string (read-only)	Suitelets and beforeLoad user events	Holds the values for supported field types. This enum is used to set the value of the type parameter when Form.addField(options) is called.
	serverWidget.FormPageLinkType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported page link types on a form. This enum is used to set the value of the type parameter for Form.addPageLink(options) .
	serverWidget.LayoutJustification	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported justification layouts. This enum is used to set the value of the align parameter when List.addColumn(options) is called.
	serverWidget.ListStyle	string (read-only)	Suitelets	Holds the string values for supported list styles. This enum is used to set the value of the List.style property.
	serverWidget.SublistDisplayType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported sublist display types. This enum is used to set the value of the Sublist.displayType property.
	serverWidget.SublistType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for valid sublist types. This enum is used to define the type parameter when Form.addSublist(options) is called.

Assistant Object Members

The following members are called on the [serverWidget.Assistant](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Assistant.addField(options)	serverWidget.Field	Suitelets	Adds a field to an assistant.
	Assistant.addFieldGroup(options)	serverWidget.FieldGroup	Suitelets	Adds a field group to an assistant.
	Assistant.addStep(options)	serverWidget.AssistantStep	Suitelets	Adds a step to an assistant.
	Assistant.addSublist(options)	serverWidget.Sublist	Suitelets	Adds a sublist to an assistant.
	Assistant.getField(options)	serverWidget.Field	Suitelets	Gets a field object.
	Assistant.getFieldGroup(options)	serverWidget.FieldGroup	Suitelets	Gets a field group object.
	Assistant.getFieldGroupIds()	string[]	Suitelets	Gets all the field group IDs in an assistant.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Assistant.getFieldIds()	string[]	Suitelets	Gets all the field IDs in an assistant.
	Assistant.getFieldIdsByFieldGroup(fieldGroup)	string[]	Suitelets	Gets all field IDs in the assistant field group.
	Assistant.getLastAction()	string	Suitelets	Gets the last action submitted by the user.
	Assistant.getLastStep()	serverWidget.AssistantStep	Suitelets	Gets the step that the last submitted action came from.
	Assistant.getNextStep()	serverWidget.AssistantStep	Suitelets	Gets the next step prompted by the assistant.
	Assistant.getStep(options)	serverWidget.AssistantStep	Suitelets	Returns a step in an assistant.
	Assistant.getStepCount()	number	Suitelets	Gets the total count of steps in the assistant.
	Assistant.getSteps()	serverWidget.AssistantStep[]	Suitelets	Gets all the steps in the assistant.
	Assistant.getSublist(options)	serverWidget.Sublist	Suitelets	Get a Sublist object from its ID.
	Assistant.getSublistIds()	string[]	Suitelets	Gets all the sublist IDs in an assistant.
	Assistant.hasErrorHtml()	Boolean	Suitelets	Indicates whether the assistant has an error message to display.
	Assistant.isFinished()	Boolean	Suitelets	Indicates the status of the assistant. If set to true, the assistant is finished.
	Assistant.sendRedirect(options)	void	Suitelets	Manages redirects in an assistant.
	Assistant.setSplash(options)	void	Suitelets	Define a splash message.
	Assistant.updateDefaultValues(values)	void	Suitelets	Sets the default values of an array of fields that are specific to the assistant.
Property	Assistant.clientScriptFileId	number	Suitelets	The File Cabinet ID of client script file to be used in this assistant.
	Assistant.clientScriptModulePath	string	Suitelets	The relative path to the client script file to be used in this assistant.
	Assistant.currentStep	serverWidget.AssistantStep (read-only)	Suitelets	The current step.
	Assistant.errorHtml	string	Suitelets	The error message text.
	Assistant.finishedHtml	string	Suitelets	The text displayed after an assistant is finished.
	Assistant.hideAddToShortcutsLink	Boolean	Suitelets	Whether the Add to Shortcuts Link is displayed in the UI.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Assistant.hideStepNumber	Boolean	Suitelets	Whether the current and total step numbers are displayed in the UI.
	Assistant.isNotOrdered	Boolean	Suitelets	Whether assistant steps are ordered or unordered.
	Assistant.title	string	Suitelets	The title of the assistant.

AssistantStep Object Members

The following members are called on the [serverWidget.AssistantStep](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	AssistantStep.getFieldIds()	string[]	Suitelets	Gets all the field IDs in an assistant step.
	AssistantStep.getLineCount(options)	number	Suitelets	Gets the number of lines previously entered by a user in a step.
	AssistantStep.getLineCount(options)	string[]	Suitelets	Gets all the field IDs in a list.
	AssistantStep.getSublistValue(options)	string	Suitelets	Gets the current value of a sublist field (line item) in a step.
	AssistantStep.getSubmittedSublistIds()	string[]	Suitelets	Gets the IDs for all the sublist fields (line items) in a step.
	AssistantStep.getValue(options)	string string[]	Suitelets	Gets the current value of a field.
Property	AssistantStep.helpText	string	Suitelets	The help text for a step.
	AssistantStep.id	string (read-only)	Suitelets	The internal ID of the step.
	AssistantStep.label	string	Suitelets	The label for a step.
	AssistantStep.stepNumber	number	Suitelets	Indicates where this step appears sequentially in an assistant.

Button Object Members

The following members are called on the [serverWidget.Button](#) object.

Member Type	Name	Property Type	Supported Script Types	Description
Property	Button.isDisabled	Boolean	Suitelets and beforeLoad user events	Determines whether a button is dimmed.

Member Type	Name	Property Type	Supported Script Types	Description
	Button.isHidden	Boolean	Suitelets and beforeLoad user events	Determines whether the button is hidden in the UI.
	Button.label	string	Suitelets and beforeLoad user events	The label for the button.

Field Object Members

The following members are called on the [serverWidget.Field](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Field.addSelectOption(options)	void	Suitelets and beforeLoad user events	Adds a select option to a dropdown list for a selectable field.
	Field.getSelectOptions(options)	Object[]	Suitelets and beforeLoad user events	Returns the internal ID and label of the options for a select field as name/value pairs.
	Field.setHelpText(options)	serverWidget.Field	Suitelets and beforeLoad user events	Sets the help text that appears in the field help popup.
	Field.updateBreakType(options)	serverWidget.Field	Suitelets and beforeLoad user events	Updates the break type used to add a break in flow layout for the field.
	Field.updateDisplaySize(options)	serverWidget.Field	Suitelets and beforeLoad user events	Updates the height and width for the field.
	Field.updateDisplayType(options)	serverWidget.Field	Suitelets and beforeLoad user events	Updates the type of display for the field.
	Field.updateLayoutType(options)	serverWidget.Field	Suitelets and beforeLoad user events	Updates the layout type for the field.
Property	Field.alias	string	Suitelets and beforeLoad user events	The alias used to set the field value.
	Field.defaultValue	string	Suitelets and beforeLoad user events	The default value for the field.
	Field.helpText	string (read-only)	Suitelets and beforeLoad user events	The help text for the field.
	Field.id	string (read-only)	Suitelets and beforeLoad user events	The internal ID for the field.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Field.isMandatory	Boolean	Suitelets and beforeLoad user events	Whether the field is required.
	Field.label	string	Suitelets and beforeLoad user events	The label for the field.
	Field.linkText	string	Suitelets and beforeLoad user events	The text displayed for a link in place of the URL.
	Field.maxLength	number	Suitelets and beforeLoad user events	The maximum length, in characters, for the field.
	Field.padding	number	Suitelets and beforeLoad user events	The number of empty vertical character spaces above the field.
	Field.richTextHeight	number	Suitelets and beforeLoad user events	The height of a rich text field, in pixels.
	Field.richTextWidth	number	Suitelets and beforeLoad user events	The width of a rich text field, in pixels.
	Field.type	string (read-only)	Suitelets and beforeLoad user events	The type of field.

FieldGroup Object Members

The following members are called on the [serverWidget.FieldGroup](#) object.

Member Type	Name	Property Type	Supported Script Types	Description
Property	FieldGroup.isBorderHidden	Boolean	Suitelets and beforeLoad user events	Whether a border appears around the field group.
	FieldGroup.isCollapsible	Boolean	Suitelets and beforeLoad user events	Whether the field group is collapsible.
	FieldGroup.isCollapsed	Boolean	Suitelets and beforeLoad user events	Whether the field group is initially collapsed or expanded in the default view.
	FieldGroup.isSingleColumn	Boolean	Suitelets and beforeLoad user events	Whether the field group is displayed in a single column.
	FieldGroup.label	string	Suitelets and beforeLoad user events	The label for the field group.

Form Object Members

The following members are called on the [serverWidget.Form](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Form.addButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a button to the form.
	Form.addCredentialField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Adds a field that store credentials in NetSuite for invoking services provided by third parties.
	Form.addField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Adds a field to the form.
	Form.addFieldGroup(options)	serverWidget.FieldGroup	Suitelets and beforeLoad user events	Adds a group of fields to the form.
	Form.addPageInitMessage(options)	void	Suitelets and beforeLoad user events	Shows a message on a form in view mode. You can use this method to show a message on a form based on its user event script context.
	Form.addPageLink(options)	void	Suitelets and beforeLoad user events	Adds a link to a form.
	Form.addResetButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a Reset button to a form that clears the values of all fields.
	Form.addSecretKeyField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Add a secret key field to the form.
	Form.addSublist(options)	serverWidget.Sublist	Suitelets and beforeLoad user events	Adds a sublist to the form.
	Form.addSubmitButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a submit button to a form that saves user inputs.
	Form.addSubtab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Adds a subtab to a form.
	Form.addTab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Adds a tab to a form.
	Form.getButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Returns a button by internal ID.
	Form.getField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Returns a field by internal ID.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Form.getSublist(options)	serverWidget.Sublist	Suitelets and beforeLoad user events	Returns a sublist by internal ID.
	Form.getSubtab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Returns a subtab by internal ID.
	Form.getTab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Returns a tab object from its internal ID.
	Form.getTabs()	string[]	Suitelets and beforeLoad user events	Returns the internal IDs of all tabs.
	Form.insertField(options)	void	Suitelets and beforeLoad user events	Inserts a field before another field within a form.
	Form.insertSublist(options)	void	Suitelets and beforeLoad user events	Inserts a sublist before another sublist on a form.
	Form.insertSubtab(options)	void	Suitelets and beforeLoad user events	Inserts a subtab before another subtab on a form.
	Form.insertTab(options)	void	Suitelets and beforeLoad user events	Inserts a tab before another tab on a form.
	Form.removeButton(options)	void	Suitelets and beforeLoad user events	Removes a button from a form.
Property	Form.updateDefaultValues(options)	void	Suitelets and beforeLoad user events	Sets the default values of many fields on a form.
	Form.clientScriptFileId	number	Suitelets and beforeLoad user events	The File Cabinet ID of client script file to be used in this form.
	Form.clientScriptModulePath	string	Suitelets and beforeLoad user events	The relative path to the client script file to be used in this form.
	Form.title	string	Suitelets and beforeLoad user events	The title used for the form.

List Object Members

The following members are called on the [serverWidget.List](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	List.addButton(options)	serverWidget.Button	Suitelets	Adds a button to a list.
	List.addColumn(options)	serverWidget.ListColumn	Suitelets	Adds a column to a list.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	List.addEditColumn(options)	serverWidget.ListColumn	Suitelets	Adds a column containing Edit or Edit/View links to a Suitelet or Portlet list.
	List.addPageLink(options)	serverWidget.List	Suitelets	Adds a link to a list.
	List.addRow(options)	serverWidget.List	Suitelets	Adds a single row to a list.
	List.addRows(options)	serverWidget.List	Suitelets	Adds multiple rows to a list.
Property	List.clientScriptFileId	number	Suitelets	The File Cabinet ID of client script file to be used in this list.
	List.clientScriptModulePath	string	Suitelets	The relative path to the client script file to be used in this list.
	List.style	string	Suitelets	The display style for this list.
	List.title	string	Suitelets	The List title.

ListColumn Object Members

The following members are called on the [serverWidget.ListColumn](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	ListColumn.addParamToURL(options)	serverWidget.ListColumn	Suitelets	Adds a URL parameter (optionally defined per row) to the list column's URL.
	ListColumn.setURL(options)	serverWidget.ListColumn	Suitelets	Sets the base URL for the list column.
Property	ListColumn.label	string	Suitelets	The label of this list column.

Sublist Object Members

The following members are called on the [serverWidget.Sublist](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Sublist.addButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a button to a sublist.
	Sublist.addField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Add a field to a sublist.
	Sublist.addMarkAllButtons()	serverWidget.Button[]	Suitelets and beforeLoad user events	Adds a Mark All or Unmark All button.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Sublist.addRefreshButton()	serverWidget.Button	Suitelets and beforeLoad user events	Adds a Reset button.
	Sublist.getField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Returns a Field object on a specified sublist.
	Sublist.getSublistValue(options)	string	Suitelets and beforeLoad user events	Gets a field value on a sublist.
	Sublist.setSublistValue(options)	void	Suitelets and beforeLoad user events	Sets the value of a sublist field.
	Sublist.updateTotallingFieldId(options)	serverWidget.Sublist	Suitelets and beforeLoad user events	Updates the ID of a field designated as a totalling column, which is used to calculate and display a running total for the sublist.
	Sublist.updateUniqueFieldId(options)	serverWidget.Sublist	Suitelets and beforeLoad user events	Updates a field ID that is to have unique values across the rows in the sublist.
Property	Sublist.displayType	string	Suitelets and beforeLoad user events	The display style for a sublist.
	Sublist.helpText	string	Suitelets and beforeLoad user events	The inline help text for a sublist.
	Sublist.label	string	Suitelets and beforeLoad user events	The label for a sublist.
	Sublist.lineCount	number (read-only)	Suitelets and beforeLoad user events	The number of line items in a sublist.

Tab Object Members

The following members are called on the [serverWidget.Tab](#) object.

Member Type	Name	Value Type	Supported Script Types	Description
Property	Tab.helpText	string	Suitelets and beforeLoad user events	The inline help text for a tab or subtab.
	Tab.label	string	Suitelets and beforeLoad user events	The label for a tab or subtab.

N/ui/serverWidget Module Script Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/ui/serverWidget module:

- [Create a Custom Form with a Submit Button, Fields, and an Inline Editor Sublist](#)

- [Create a Custom Survey Form](#)
- [Create a Form with a Credential Field](#)

See the help topic [Complete Custom List Page Sample Script](#) for a sample of a List page, and [Sample Custom Assistant Script](#) for a sample of an Assistant.

Create a Custom Form with a Submit Button, Fields, and an Inline Editor Sublist

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a Suitelet that generates a sample form with a submit button, fields, and an inline editor sublist.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget'], (serverWidget) => {
6       const onRequest = (scriptContext) => {
7           if (scriptContext.request.method === 'GET') {
8               let form = serverWidget.createForm({
9                   title: 'Simple Form'
10              });
11
12              let field = form.addField({
13                  id: 'textfield',
14                  type: serverWidget.FieldType.TEXT,
15                  label: 'Text'
16              });
17              field.layoutType = serverWidget.FieldLayoutType.NORMAL;
18              field.updateBreakType({
19                  breakType: serverWidget.FieldBreakType.STARTCOL
20              });
21
22              form.addField({
23                  id: 'datefield',
24                  type: serverWidget.FieldType.DATE,
25                  label: 'Date'
26              });
27              form.addField({
28                  id: 'currencyfield',
29                  type: serverWidget.FieldType.CURRENCY,
30                  label: 'Currency'
31              });
32
33              let select = form.addField({
34                  id: 'selectfield',
35                  type: serverWidget.FieldType.SELECT,
36                  label: 'Select'
37              });
38              select.addSelectOption({
39                  value: 'a',
40                  text: 'Albert'
41              });
42              select.addSelectOption({
43                  value: 'b',

```

```

44         text: 'Baron'
45     });
46
47     let sublist = form.addSublist({
48         id: 'sublist',
49         type: serverWidget.SublistType.INLINEEDITOR,
50         label: 'Inline Editor Sublist'
51     });
52     sublist.addField({
53         id: 'sublist1',
54         type: serverWidget.FieldType.DATE,
55         label: 'Date'
56     });
57     sublist.addField({
58         id: 'sublist2',
59         type: serverWidget.FieldType.TEXT,
60         label: 'Text'
61     });
62
63     form.addSubmitButton({
64         label: 'Submit Button'
65     });
66
67     scriptContext.response.writePage(form);
68 } else {
69     const delimiter = /\u0001/;
70     const textField = scriptContext.request.parameters.textfield;
71     const dateField = scriptContext.request.parameters.datefield;
72     const currencyField = scriptContext.request.parameters.currencyfield;
73     const selectField = scriptContext.request.parameters.selectfield;
74     const sublistData = scriptContext.request.parameters.sublistdata.split(delimiter);
75     const sublistField1 = sublistData[0];
76     const sublistField2 = sublistData[1];
77
78     scriptContext.response.write(`You have entered: ${textField} ${dateField} ${currencyField} ${selectField} ${sublist
Field1} ${sublistField2}`);
79 }
80 }
81
82 return {onRequest}
83 });

```

Create a Custom Survey Form

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a Suitelet that generates a customer survey form with inline HTML fields, radio fields, and a submit button.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'], function(serverWidget) {
6      function onRequest(context) {
7          var form = serverWidget.createForm({
8              title: 'Thank you for your interest in Wolfe Electronics',
9              hideNavBar: true
10         });
11
12         var htmlHeader = form.addField({
13             id: 'custpage_header',
14             type: serverWidget.FieldType.INLINEHTML,

```



```

15     label: ' '
16   }).updateLayoutType({
17     layoutType: serverWidget.FieldLayoutType.OUTSIDEABOVE
18   }).updateBreakType({
19     breakType: serverWidget.FieldBreakType.STARTROW
20   }).defaulttValue = '<p style=\font-size:20px\>We pride ourselves on providing the best' +
21     ' services and customer satisfaction. Please take a moment to fill out our survey.</p><br><br>';
22
23   var htmlInstruct = form.addField({
24     id: 'custpage_p1',
25     type: serverWidget.FieldType.INLINEHTML,
26     label: ' '
27   }).updateLayoutType({
28     layoutType: serverWidget.FieldLayoutType.OUTSIDEABOVE
29   }).updateBreakType({
30     breakType: serverWidget.FieldBreakType.STARTROW
31   }).defaulttValue = '<p style=\font-size:14px\>When answering questions on a scale of 1 to 10,' +
32     ' 1 = Greatly Unsatisfied and 10 = Greatly Satisfied.</p><br><br>';
33
34   var productRating = form.addField({
35     id: 'custpage_lblproductrating',
36     type: serverWidget.FieldType.INLINEHTML,
37     label: ' '
38   }).updateLayoutType({
39     layoutType: serverWidget.FieldLayoutType.NORMAL
40   }).updateBreakType({
41     breakType: serverWidget.FieldBreakType.STARTROW
42   }).defaulttValue = '<p style=\font-size:14px\>How would you rate your satisfaction with our products?</p>';
43
44   form.addField({
45     id: 'custpage_rdoproducrating',
46     type: serverWidget.FieldType.RADIO,
47     label: '1',
48     source: 'p1'
49   }).updateLayoutType({
50     layoutType: serverWidget.FieldLayoutType.STARTROW
51   });
52   form.addField({
53     id: 'custpage_rdoproducrating',
54     type: serverWidget.FieldType.RADIO,
55     label: '2',
56     source: 'p2'
57   }).updateLayoutType({
58     layoutType: serverWidget.FieldLayoutType.MIDROW
59   });
60   form.addField({
61     id: 'custpage_rdoproducrating',
62     type: serverWidget.FieldType.RADIO,
63     label: '3',
64     source: 'p3'
65   }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
66   form.addField({
67     id: 'custpage_rdoproducrating',
68     type: serverWidget.FieldType.RADIO,
69     label: '4',
70     source: 'p4'
71   }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
72   form.addField({
73     id: 'custpage_rdoproducrating',
74     type: serverWidget.FieldType.RADIO,
75     label: '5',
76     source: 'p5'
77   }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
78   form.addField({
79     id: 'custpage_rdoproducrating',
80     type: serverWidget.FieldType.RADIO,
81     label: '6',
82     source: 'p6'
83   }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
84   form.addField({
85     id: 'custpage_rdoproducrating',
86     type: serverWidget.FieldType.RADIO,
87     label: '7',

```

```

88     source: 'p7'
89   }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
90   form.addField({
91     id: 'custpage_rdoproducrating',
92     type: serverWidget.FieldType.RADIO,
93     label: '8',
94     source: 'p8'
95   }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
96   form.addField({
97     id: 'custpage_rdoproducrating',
98     type: serverWidget.FieldType.RADIO,
99     label: '9',
100    source: 'p9'
101  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
102  form.addField({
103    id: 'custpage_rdoproducrating',
104    type: serverWidget.FieldType.RADIO,
105    label: '10',
106    source: 'p10'
107  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.ENDROW});
108
109  var serviceRating = form.addField({
110    id: 'custpage_lblservicerating',
111    type: serverWidget.FieldType.INLINEHTML,
112    label: ' '
113  }).updateLayoutType({
114    layoutType: serverWidget.FieldLayoutType.NORMAL
115  }).updateBreakType({
116    breakType: serverWidget.FieldBreakType.STARTROW
117  }).defaultVaue = '<p style=\''font-size:14px\''>How would you rate your satisfaction with our services?</p>';
118
119  form.addField({
120    id: 'custpage_rdoservicerating',
121    type: serverWidget.FieldType.RADIO,
122    label: '1',
123    source: 'p1'
124  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.STARTROW});
125  form.addField({
126    id: 'custpage_rdoservicerating',
127    type: serverWidget.FieldType.RADIO,
128    label: '2',
129    source: 'p2'
130  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
131  form.addField({
132    id: 'custpage_rdoservicerating',
133    type: serverWidget.FieldType.RADIO,
134    label: '3',
135    source: 'p3'
136  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
137  form.addField({
138    id: 'custpage_rdoservicerating',
139    type: serverWidget.FieldType.RADIO,
140    label: '4',
141    source: 'p4'
142  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
143  form.addField({
144    id: 'custpage_rdoservicerating',
145    type: serverWidget.FieldType.RADIO,
146    label: '5',
147    source: 'p5'
148  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
149  form.addField({
150    id: 'custpage_rdoservicerating',
151    type: serverWidget.FieldType.RADIO,
152    label: '6',
153    source: 'p6'
154  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
155  form.addField({
156    id: 'custpage_rdoservicerating',
157    type: serverWidget.FieldType.RADIO,
158    label: '7',
159    source: 'p7'
160  }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});

```

```

161     form.addField({
162         id: 'custpage_rdoservicerating',
163         type: serverWidget.FieldType.RADIO,
164         label: '8',
165         source: 'p8'
166     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
167     form.addField({
168         id: 'custpage_rdoservicerating',
169         type: serverWidget.FieldType.RADIO,
170         label: '9',
171         source: 'p9'
172     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
173     form.addField({
174         id: 'custpage_rdoservicerating',
175         type: serverWidget.FieldType.RADIO,
176         label: '10',
177         source: 'p10'
178     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.ENDROW});
179
180     form.addSubmitButton({
181         label: 'Submit'
182     });
183
184     context.response.writePage(form);
185 }
186
187 return {
188     onRequest: onRequest
189 };
190 });

```

Create a Form with a Credential Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to create a form with a credential field. For more information about credential fields, see [Form.addCredentialField\(options\)](#).

Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

Note: The default maximum length for a credential field is 32 characters. If needed, use the `Field.maxLength` property to change this value.

The values for `restrictToDomains`, `restrictToScriptIds`, and `baseUrl` in this sample are placeholders. You must replace them with valid values from your NetSuite account.

Important: This sample uses SuiteScript 2.1. For more information, see the help topic [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6   // This script creates a form with a credential field.
7   define(['N/ui/serverWidget', 'N/https', 'N/url'], (serverWidget, https, url) => {
8       function onRequest(context) {
9           if (context.request.method === 'GET') {
10               const form = serverWidget.createForm({
11                   title: 'Password Form'

```

```

12     });
13
14     const credField = form.addCredentialField({
15         id: 'password',
16         label: 'Password',
17         restrictToDomains: ['<accountID>.app.netsuite.com'],
18         restrictToCurrentUser: false,
19         restrictToScriptIds: 'customscript_my_script'
20     });
21
22     credField.maxLength = 32;
23
24     form.addSubmitButton();
25
26     context.response.writePage({
27         pageObject: form
28     });
29 }
30 else {
31     // Request to an existing Suitelet with credentials
32     let passwordGuid = context.request.parameters.password;
33
34     // Replace SCRIPTID and DEPLOYMENTID with the internal ID of the suitelet script and deployment in your account
35     let baseUrl = url.resolveScript({
36         scriptId: SCRIPTID,
37         deploymentId: DEPLOYMENTID,
38         returnExternalURL: true
39     });
40
41     let authUrl = baseUrl + '&pwd={' + passwordGuid + '}';
42
43     let secureStringUrl = https.createSecureString({
44         input: authUrl
45     });
46
47     let headers = ({
48         'pwd': passwordGuid
49     });
50
51     let response = https.post({
52         credentials: [passwordGuid],
53         url: secureStringUrl,
54         body: {authorization: ' ' + passwordGuid + ' ', data: 'anything can be here'},
55         headers: headers
56     });
57 }
58 }
59 return {
60     onRequest: onRequest
61 };
62 });

```

serverWidget.Assistant

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A scriptable, multi-step NetSuite assistant. An assistant contains a series of step that a user must complete to accomplish a larger goal. An assistant can be sequential, or non-sequential and include optional steps. Each page of the assistant is defined by a step. All data and states for an assistant are tracked automatically throughout the user's session until completion of the assistant. You can create a new assistant with the serverWidget.createAssistant(options) method. After you create an Assistant object, you can: ■ Build and run an assistant in your NetSuite account.
---------------------------	---

	■ Add a variety of scriptable elements to the assistant including fields, steps, buttons, tabs, and sublists.  Note: The assistant cannot be used on externally available Suitelets.
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Methods and Properties	Assistant Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 ...
7 //Add additional code

```

Assistant.addField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a field to an assistant. Use fields to record or display information specific to your needs.
Returns	serverWidget.Field object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID for this field.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.label	string	required	The label for this field.	2015.2
options.type	string	required	The field type. Use the serverWidget.FieldType enum to set this value.  Note: If you have set the type parameter to SELECT or MULTISELECT, and you want to add custom options to the select field, you must set source to NULL. Then, when a value is specified, the value will populate the options from the source.  Important: Long text fields created with SuiteScript have a character limit of 100,000. Long text fields created with Suitebuilder have a character limit of 1,000,000.	2015.2
options.source	string	optional	The internalId or scriptId of the source list for this field. Use this parameter if you are adding a select (List/Record) or multi-select type of field. For radio fields only, the source parameter is not an optional parameter, it must contain the radio button's unique internal ID. The id parameter contains the ID that identifies all the radio buttons of the same group.  Note: If you want to add custom options on a select or multi-select field, you must set the source parameter to NULL , and then add the custom options using Field.addSelectOption(options).  Important: After you create a select or multi-select field that is sourced from a record or list, you cannot add additional values with Field.addSelectOption(options). The select values are determined by the source record or list.	2015.2
options.container	string	optional	The internal ID of the field group to place this field in.	2016.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addField({
7     id : 'idname',

```

```

8      type : serverWidget.FieldType.TEXT,
9      label : 'Sample label'
10    });
11    ...
12    //Add additional code

```

Assistant.addFieldGroup(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a field group to the assistant. A field group is a collection of fields that can be displayed in a one or two column format. Assign a field to a field group to label, hide or collapse a group of fields. By default, the field group is collapsible and appears expanded on the assistant page. To change this behavior, set the FieldGroup.isCollapsed and FieldGroup.isCollapsible properties.
Returns	serverWidget.FieldGroup object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID for the field group.	2015.2
options.label	string	required	The label for the field group.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1    //Add additional code
2    ...
3    var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5    });
6    assistant.addFieldGroup({
7      id : 'idname',
8      label : 'Sample label'
9    });
10   ...
11   //Add additional code

```

Assistant.addStep(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a step to an assistant. Steps define each page of the assistant. Use Assistant.isNotOrdered to control if the steps must be completed sequentially or in no specific order. If you want to create help text for the step, you can use AssistantStep.helpText on the object returned.
Returns	serverWidget.AssistantStep object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID for this step (for example, 'entercontacts').	2015.2
options.label	string	required	The label for this step (for example, 'Enter Contacts'). By default, the step appears vertically in the left panel of the assistant.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4   title : 'Simple Assistant'
5 });
6 assistant.addStep({
7   id : 'idname',
8   label : 'Sample label'
9 });
10 ...
11 //Add additional code

```

Assistant.addSublist(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a sublist to an assistant.
---------------------------	---------------------------------

	 Note: Only inline editor sublists are added. Other sublist types are not supported.
Returns	serverWidget.Sublist object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID for the sublist.	2015.2
options.label	string	required	The label for the sublist.	2015.2
options.type	string	required	The type of sublist to add. Currently, only the sublist type of <code>INLINEEDITOR</code> can be added. For more information about this type of sublist, see serverWidget.SublistType .	2016.1

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/ui/serverWidget Module Script Samples .
--

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addSublist({
7     id : 'idname',
8     label : 'Sample label',
9     type : serverWidget.SublistType.INLINEEDITOR
10 });
11 ...
12 //Add additional code

```

Assistant.getField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a field object on an assistant page.
Returns	serverWidget.Field

Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the field.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addField({
7     id : 'idname',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Sample label'
10 });
11 var field = assistant.getField({
12     id: 'idname'
13 });
14 ...
15 //Add additional code

```

Assistant.getFieldGroup(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a field group on an assistant page.
Returns	serverWidget.FieldGroup object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the field group.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addFieldGroup({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var fieldgroup = assistant.getFieldGroup({
11     id: 'idname'
12 });
13 ...
14 //Add additional code

```

Assistant.getFieldGroupIds()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves all the internal IDs for field groups in an assistant.
Returns	string[]
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addFieldGroup({
7     id : 'idname',

```

```

8      label : 'Sample label'
9    });
10    var fieldgroupid = assistant.getFieldGroupIds();
11    ...
12    //Add additional code

```

Assistant.getFieldIds()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets all the internal IDs for fields in an assistant.
Returns	string[]
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1    //Add additional code
2    ...
3    var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5    });
6    assistant.addField({
7      id : 'idname',
8      label : 'Sample label'
9    });
10   var fieldid = assistant.getFieldIds();
11   ...
12   //Add additional code

```

Assistant.getFieldIdsByFieldGroup(fieldGroup)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets all field IDs in the assistant field group.
Returns	string[]
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
fieldGroup	string	required	The internal ID of the field group.	2016.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addFieldGroup({
7     id : 'fieldgroupid',
8     label : 'Sample label'
9 });
10 assistant.addField({
11     id : 'idname',
12     label : 'Sample label'
13 });
14 var fieldid = assistant.getFieldIdsByFieldGroup({
15     fieldGroup : 'fieldgroupid'
16 });
17 ...
18 //Add additional code

```

Assistant.getLastAction()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the last action taken by the user. To identify the step that the last action came from, use Assistant.getLastStep() .
Returns	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({

```

```

4     title : 'Simple Assistant'
5   });
6   ...
7   if (assistant.getLastAction() === serverWidget.AssistantSubmitAction.CANCEL) {
8     ...
9   }
10  ...
11
12  //Add additional code

```

Assistant.getLastStep()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the step that the last submitted action came from.
Returns	A serverWidget.AssistantStep object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4    title : 'Simple Assistant'
5  });
6  ...
7  var lastStep = assistant.getLastStep();
8  ...
9  //Add additional code

```

Assistant.getNextStep()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the next step corresponding to the user's last submitted action in the assistant. If you need information about the last step, use Assistant.getLastStep() before you use this method.
Returns	serverWidget.AssistantStep object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 ...
7 var nextStep = assistant.getNextStep();
8 ...
9 //Add additional code

```

Assistant.getStep(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a step in an assistant.
Returns	serverWidget.AssistantStep object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the step.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });

```

```

6  assistant.addStep({
7      id : 'idname',
8      label : 'Sample label'
9  });
10 var firststep = assistant.getStep({
11     id: 'idname'
12 });
13 ...
14 //Add additional code

```

Assistant.getStepCount()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the total number of steps in an assistant.
Returns	number
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  assistant.addStep({
7      id : 'idname',
8      label : 'Sample label'
9  });
10 var numSteps = assistant.getStepCount();
11 ...
12 //Add additional code

```

Assistant.getSteps()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets all the steps in an assistant.
Returns	serverWidget.AssistantStep[] object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var steps = assistant.getSteps();
11 ...
12 //Add additional code

```

Assistant.getSublist(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a sublist in an assistant.
Returns	serverWidget.Sublist object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the sublist.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code

```

```

2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  assistant.addSublist({
7      id : 'idname',
8      label : 'Sample label',
9      type : serverWidget.SublistType.LIST
10 });
11 var sublist = assistant.getSublist({
12     id : 'idname'
13 });
14 ...
15 //Add additional code

```

Assistant.getSublistIds()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the IDs for all the sublists in an assistant.
Returns	string[]
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  assistant.addSublist({
7      id : 'idname',
8      label : 'Sample label',
9      type : serverWidget.SublistType.LIST
10 });
11 var sublistid = assistant.getSublistIds();
12 ...
13 //Add additional code

```

Assistant.hasErrorHtml()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Indicates whether an assistant has an error message set.
---------------------------	--

	true if Assistant.errorHtml contains a value.
Returns	Boolean
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  ...
7  if (assistant.hasErrorHtml()) {
8      ...
9  }
10 ...
11 //Add additional code

```

Assistant.isFinished()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Indicates whether all steps in an assistant are completed. If true, the assistant is finished and a completion message is displayed. To set the text for the completion message, use the Assistant.finishedHtml property.
Returns	Boolean
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code

```

```

2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  ...
7  if (assistant.isFinished()) {
8      ...
9  }
10 ...
11
12 //Add additional code

```

Assistant.sendRedirect(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Manages redirects in an assistant. This method also addresses the case in which one assistant redirects to another assistant. In this scenario, the second assistant must return to the first assistant if the user Cancels or Finishes. This method, when used in the second assistant, ensures that users are redirected back to the first assistant.
Returns	void
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.response	response	required	The response that redirects the user.	2015.2

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title: 'Small Business Setup Assistant',
5     hideNavBar: true
6 });
7 if (request.method === 'POST') {
8     if (assistant.getLastAction() === 'finish') {

```

```

9      assistant.finishedHtml = 'Completed!';
10     assistant.sendRedirect({
11         response: response
12     });
13 }
14 }
15 ...
16 //Add additional code

```

Assistant.setSplash(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Defines a splash message.
Returns	void
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The title of the splash screen.	2015.2
options.text1	string	required	Text for the splash screen	2015.2
options.text2	string	optional	Text for a second column on the splash screen, if desired.	2015.2

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  ...
2  var assistant = serverWidget.createAssistant({
3      title : 'Simple Assistant'
4  });
5  assistant.setSplash({
6      title: "Welcome Title!",
7      text1: "An explanation of what this assistant accomplishes.",
8      text2: "Some parting words."
9  });
10 ...
11 //Add additional code

```

Assistant.updateDefaultValues(values)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the default values of an array of fields that are specific to the assistant.
Returns	void
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
values	Object[]	required	An object containing an array of name/value pairs that map field names to field values.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.addField({
7     id : 'idname',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Sample label'
10 });
11 assistant.updateDefaultValues({
12     idname : 'New Default Value'
13 });
14 ...
15 //Add additional code

```

Assistant.clientScriptFileId

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The File Cabinet ID of client script file to be used in this assistant.
Type	number
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module

Since	2015.2
--------------	--------

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You attempted to set this value when the <code>Assistant.clientScriptModulePath</code> property value has already been specified. For more information, see Assistant.clientScriptModulePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.clientScriptFileId = 32;
7 ...
8 //Add additional code

```

Assistant.clientScriptModulePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The relative path to the client script file to be used in this assistant.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2016.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You attempted to set this value when the <code>Assistant.clientScriptFileId</code> property value has already been specified. For more information, see Assistant.clientScriptFileId .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code

```

```

2  ...
3  objAssistant.clientScriptModulePath = 'SuiteScripts/assistantBehavior.js';
4  ...
5  //Add additional code

```

Assistant.currentStep

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The current step. You can set any step as the current step.
Type	serverWidget.AssistantStep (read-only)
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  assistant.addStep({
7      id : 'idname',
8      label : 'Sample label'
9  });
10 var step = assistant.currentStep;
11 ...
12 //Add additional code

```

Assistant.errorHtml

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Error message text for the current step. Optionally, you can use HTML tags to format the message.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.errorHtml = "You have <b>not</b> filled out the required fields. Please go back.";
7 ...
8 //Add additional code
```

Assistant.finishedHtml

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text to display after the assistant finishes. For example “You have completed the Small Business Setup Assistant. Take the rest of the day off”. To trigger display of the completion message, call Assistant.isFinished() .
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.finishedHtml = "Congratulations! You have successfully set up your account.";
7 ...
8 //Add additional code
```

Assistant.hideAddToShortcutsLink

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether to show or hide the Add to Shortcuts link that appears in the top-right corner of an assistant page. By default, the value is false, which means the Add to Shortcuts link is visible in the UI.
----------------------	---

	If set to true, the Add To Shortcuts link is not visible on an Assistant page.
Type	Boolean
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 assistant.hideAddToShortcutsLink = true;
7 ...
8 //Add additional code

```

Assistant.hideStepNumber

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether assistant steps are displayed with numbers. By default, the value is false, which means that steps are numbered. If set to true, the assistant does not use step numbers. To change step ordering, set Assistant.isNotOrdered .
Type	Boolean
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });

```

```

6  assistant.addStep({
7      id : 'idname',
8      label : 'Sample label'
9  });
10 assistant.hideStepNumber = true;
11 ...
12 //Add additional code

```

Assistant.isNotOrdered

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether steps must be completed in a particular sequence. If steps are ordered, users must complete the current step before proceeding to the next step. The default value is <code>false</code>, which means the steps are ordered. Ordered steps appear vertically in the left panel of the assistant If set to <code>true</code>, steps can be completed in any order. In the UI, unordered steps appear horizontally and below the assistant title
Type	Boolean
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4      title : 'Simple Assistant'
5  });
6  assistant.addStep({
7      id : 'idname',
8      label : 'Sample label'
9  });
10 assistant.addStep({
11     id : 'idname2',
12     label : 'Sample label 2'
13 });
14 assistant.isNotOrdered = false;
15 ...
16 //Add additional code

```

Assistant.title

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The title for the assistant. The title appears at the top of all assistant pages.
-----------------------------	---

	This value overrides the title specified in serverWidget.createAssistant(options) .
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var title = assistant.title;
7 ...
8 //Add additional code

```

serverWidget.AssistantStep

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A step within a custom NetSuite assistant. Create a step by calling Assistant.addStep(options) .
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Methods and Properties	AssistantStep Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7     id : 'idname',

```

```
8      label : 'Sample label'
9    });
10    ...
11    //Add additional code
```

AssistantStep.getFieldIds()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the IDs for all the fields in a step.
Returns	string[]
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4   title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7   id : 'idname',
8   label : 'Sample label'
9 });
10 assistant.addField({
11   id : 'fieldid',
12   type : serverWidget.FieldType.TEXT,
13   label : 'Field'
14 });
15 var fieldIds = assistantStep.getFieldIds();
16 ...
17 //Add additional code
```

AssistantStep.getLineCount(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the number of lines on a sublist in a step.  Note: The first line number on a sublist is 0 (not 1).
Returns	number (the count of line items on a sublist)

	 Note: if the sublist does not exist, -1 is returned.
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The sublist internal ID.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var sublist = assistant.addSublist({
11     id : 'sublistid',
12     type : serverWidget.SublistType.INLINEEDITOR,
13     label : 'Editor'
14 });
15 var numLines = assistantStep.getLineCount({
16     group : 'sublistid'
17 });
18 ...
19 //Add additional code

```

AssistantStep.getSublistFieldIds(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the IDs for all the sublist fields (line items) in a step.
Returns	string[]
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The sublist internal ID.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var sublist = assistant.addSublist({
11     id : 'sublistid',
12     type : serverWidget.SublistType.INLINEEDITOR,
13     label : 'Editor'
14 });
15 var sublistFieldIds = assistantStep.getSublistFieldIds({
16     group : 'sublistid'
17 });
18 ...
19 //Add additional code

```

AssistantStep.getSublistValue(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the current value of a sublist field (line item) in a step.
Returns	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.group	string	required	The internal ID of the sublist.	2015.2
options.id	string	required	The internal ID of the sublist field.	2015.2
options.line	number	required	The line number for the sublist field.	2015.2

 **Note:** The first line number on a sublist is 0 (not 1).

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var sublist = assistant.addSublist({
11     id : 'sublistid',
12     type : serverWidget.SublistType.INLINEEDITOR,
13     label : 'Editor'
14 });
15 var sublistfield = sublist.addField({
16     id : 'fieldid',
17     type : serverWidget.FieldType.TEXT,
18     label : 'Text'
19 });
20 var sublistvalue = assistantStep.getSublistValue({
21     group: 'sublistid',
22     id: 'fieldid',
23     line: 0
24 });
25 ...
26 //Add additional code

```

AssistantStep.getSubmittedSublistIds()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the IDs for all the sublists submitted in a step.
Returns	string[]
Supported Script Types	Suitelets

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var sublist = assistant.addSublist({
11     id : 'sublistid',
12     type : serverWidget.SublistType.INLINEEDITOR,
13     label : 'Editor'
14 });
15 var submittedSublistId = assistantStep.getSubmittedSublistIds();
16 ...
17 //Add additional code

```

AssistantStep.getValue(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets the current value(s) of a field or multi-select field.
Returns	string (for standard fields) string[] (for multi-select fields)
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of a field.	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4   title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7   id : 'idname',
8   label : 'Sample label'
9 });
10 var field = assistant.addField({
11   id : 'fieldid',
12   type : serverWidget.FieldType.TEXT,
13   label : 'Text'
14 });
15 var value = assistantStep.getValue({
16   id: 'fieldid'
17 });
18 ...
19 //Add additional code
```

AssistantStep.helpText

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The help text for a step.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code ...
2 assistantStep.helpText = 'Help Text Goes Here.';
3 ...
4 //Add additional code
```

AssistantStep.id

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID of the step.
----------------------	------------------------------

Type	string (read-only)
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 var assistantStep = assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9 });
10 var id = assistantStep.id;
11 ...
12 //Add additional code

```

AssistantStep.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for the step. Note: To create a label when the step is first added to the assistant, you can use the Assistant.addStep(options) method.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({

```

```

4     title : 'Simple Assistant'
5   });
6   var assistantStep = assistant.addStep({
7     id : 'idname',
8     label : 'Sample label'
9   });
10  var label = assistantStep.label;
11  ...
12  //Add additional code

```

AssistantStep.stepNumber

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The numerical order of the step.
Type	number Note: A sequence of assistant steps starts at 1.
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var assistant = serverWidget.createAssistant({
4    title : 'Simple Assistant'
5  });
6  var assistantStep = assistant.addStep({
7    id : 'idname',
8    label : 'Sample label'
9  });
10 var stepNum = assistantStep.stepNumber;
11 ...
12 //Add additional code

```

serverWidget.Button

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A button that appears in a UI object. To add a button, use Form.addButton(options) or Sublist.addButton(options). When adding a button to a record or form, consider using a beforeLoad user event script. On records, custom buttons appear to the left of the printer icon.
---------------------------	--

	 Note: Currently, you cannot use SuiteScript to add or remove a custom button to or from the Actions menu. However, you can do this using point-and-click customization. For more information, see the help topic Configuring Buttons and Actions .
Supported Script Types	Suitelets User event scripts – beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Methods and Properties	Button Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var button = form.addButton({
7     id : 'buttonid',
8     label : 'Test'
9 });
10 ...
11
12 //Add additional code

```

Button.isDisabled

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Determines whether a button is dimmed. The default value is false. If set to true, the button appears dimmed and cannot be clicked.  Note: This method is not supported for standard NetSuite buttons. This method can be used with custom buttons only.
Type	Boolean
Supported Script Types	Suitelets User event scripts – beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var button = form.addButton({
7   id : 'buttonid',
8   label : 'Test'
9 });
10 button.isDisabled = true;
11 ...
12 //Add additional code
```

Button.isHidden

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Determines whether the button is hidden in the UI. The default value is false, which means the button is visible. If set to true, the button is not visible in the UI.  Note: This property is supported on custom buttons and on some standard NetSuite buttons. For a list of supported standard buttons, see the help topic Button IDs.
Type	boolean
Supported Script Types	Suitelets User event scripts – beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var button = form.addButton({
7   id : 'buttonid',
8   label : 'Test'
9 });
10 button.isHidden = true;
11 ...
```

12 //Add additional code

Button.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for the button. You can use this property to rename a button based on context, for example to re-label a button for particular users that are viewing a page.  Note: This property is supported on custom buttons and most standard buttons. For a list of supported standard buttons, see the help topic Button IDs.
Type	string
Supported Script Types	Suitelets User event scripts – beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var button = form.addButton({
7   id : 'buttonid',
8   label : 'Test'
9 });
10 var label = button.label;
11 ...
12 //Add additional code

```

serverWidget.Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A body or sublist field. Use fields to record or display information specific to your needs. To add a Field object, use Assistant.addField(options), Form.addField(options), or Sublist.addField(options).
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/ui/serverWidget Module
Methods and Properties	Field Object Members
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_text',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 ...
12 //Add additional code

```

Field.addSelectOption(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds the select options that appears in the dropdown of a field.  Important: After you create a select or multi-select field that is sourced from a record or list, you cannot add additional values with <code>Field.addSelectOption(options)</code>. The select values are determined by the source record or list.
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.value	string	required	The internal ID of this select option.	2015.2

Parameter	Type	Required / Optional	Description	Since
options.text	string	required	The label for this select option.	2015.2
options.isSelected	Boolean	optional	If set to true, this option is selected by default in the UI. The default value for this parameter is false.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var selectField = form.addField({
7     id : 'custpage_selectfield',
8     type : serverWidget.FieldType.SELECT,
9     label : 'Select'
10 });
11
12 selectField.addSelectOption({
13     value : '',
14     text : ''
15 });
16
17 selectField.addSelectOption({
18     value : 'a',
19     text : 'Albert'
20 });
21 ...
22 //Add additional code

```

Field.getSelectOptions(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Obtains an array of available options on a dropdown select, multi-select, or radio field. The internal ID and label of the field options are returned as name/value pairs.
Returns	Object[] This function returns an array in the following format: <pre>1 [{value: 5, text: 'abc'}, {value: 6, text: '123'}]</pre> This function returns Type Error if the field is not a supported field for this method. If you attempt to get select options on a field that is not a dropdown select field, such as a popup select field or a field that does not exist on the form, null is returned. For example, if the number of options available for the field is greater than your setting for Maximum Entries in Dropdowns at Home > Set Preferences, the field becomes a popup field, and this method returns null. Because the maximum value for the Maximum Entries in Dropdowns settings is 500, the maximum number of options that can be returned by this method is 500.

	 Note: A call to this method may return different results for the same field for different roles.
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.filter	string	optional	A search string to filter the select options that are returned. For example, if there are 50 select options available, and 10 of the options contains 'John', for example, "John Smith" or "Shauna Johnson", only those 10 options will be returned. Filter values are case insensitive. The filters 'John' and 'john' will return the same select options.	2015.2
options.filteroperator	string	optional	Supported operators are contains is startswith. If not specified, defaults to the contains operator.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var selectField = form.addField({
7   id : 'custpage_selectfield',
8   type : serverWidget.FieldType.SELECT,
9   label : 'Select'
10 });
11 selectField.addSelectOption({
12   value : 'a',
13   text : 'Albert'
14 });
15 var options = field.getSelectOptions({
16   filter : 'a',
17   filteroperator: 'startswith'
18 });
19 ...
20 //Add additional code

```

Field.setHelpText(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the help text for the field. When the field label is clicked, a popup displays the help text defined using this method.
Returns	The serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.help	string	required	The text in the field help popup.	2015.2
options.showInlineFor Assistant	Boolean	optional	If set to true, the field help will display inline below the field on the assistant, and in a field help popup. The default value is false, which means the field help appears in a popup when the field label is clicked and does not appear inline. Note: The inline parameter is available only to serverWidget.Field objects that have been added to serverWidget.Assistant objects.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_textfield',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text'

```

```

10  });
11  field.setHelpText({
12      help : "Help Text Goes Here."
13  });
14  ...
15  //Add additional code

```

Field.updateBreakType(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the break type of the field. Set this value using the FieldBreakType enum The break type is used to add a break in flow layout for the field.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.breakType	string	required	The break type of the field. Set this value using the serverWidget.FieldBreakType enum	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  var field = form.addField({
7      id : 'custpage_textfield',
8      type : serverWidget.FieldType.TEXT,
9      label : 'Text'
10 });
11 field.updateBreakType({
12     breakType : serverWidget.FieldBreakType.STARTCOL
13 });
14 ...
15 //Add additional code

```

Field.updateDisplaySize(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the width and height of the field. Only supported on multi-selects, long text, and fields that get rendered as INPUT (type=text) fields. This function is not supported on list/record fields or rich text fields. To set height and width for rich text fields, use Field.richTextWidth and Field.richTextHeight.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.height	number	required	The new height of the field (rows for textarea and multiselect fields).	2015.2
options.width	number	required	The new width of the field (cols for textarea, characters for all others).	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_textfield',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text'
10 });
11 field.updateDisplaySize({
12   height : 60,
13   width : 100
14 });
15 ...
16 //Add additional code

```

Field.updateDisplayType(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the display type for the field.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.displayType	string	required	The new display type of the field. For more information about possible values, see serverWidget.FieldDisplayType .	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_textfield',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text'
10 });
11 field.updateDisplayType({
12   displayType : serverWidget.FieldDisplayType.HIDDEN
13 });
14 ...
15 //Add additional code

```

Field.updateLayoutType(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the layout type for the field.
---------------------------	--

Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.layoutType	string	required	The new layout type of the field. Set this value using the serverWidget.FieldLayoutType enum.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_textfield',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 field.updateLayoutType({
12     layoutType : serverWidget.FieldLayoutType.NORMAL
13 });
14 ...
15 //Add additional code

```

Field.alias

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	An alternate name that you can assign to a serverWidget.Field object. By default, the alias is equal to the field's internal ID. This property is only supported on scripted fields created using the N/ui/serverWidget Module .
Type	string
Supported Script Types	Suitelets

	User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_textfield',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 field.alias = 'fieldid';
12 ...
13 //Add additional code

```

Field.defaultValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The default value for this field. If you pass an empty string or any value that is not a number, such as undefined, the field defaults to a blank field in the UI. This property is supported only on scripted fields created using the . N/ui/serverWidget Module .
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });

```



```

6  var field = form.addField({
7      id : 'custpage_textfield',
8      type : serverWidget.FieldType.TEXT,
9      label : 'Text'
10 });
11 field.defaultValue = 'Insert Text Here.';
12 ...
13 //Add additional code

```

Field.helpText

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The help text for the field.
Type	string (read-only)
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	–

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  var field = form.addField({
7      id : 'custpage_textfield',
8      type : serverWidget.FieldType.TEXT,
9      label : 'Text'
10 });
11 field.setHelpText({
12     help : "Help Text Goes Here."
13 });
14 var fieldHelpText = field.helpText;
15 ...
16 //Add additional code

```

Field.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The field internal ID.
Type	string (read-only)
Supported Script Types	Suitelets User event scripts - beforeLoad entry point

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_textfield',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 var fieldId = field.id;
12 ...
13 //Add additional code
```

Field.isMandatory

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Whether the field is required or optional. If set to true, then the field is defined as required. The default value is false. This property is supported only on scripted fields created using the . N/ui/serverWidget Module .
Type	Boolean
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
```

```

7      id : 'custpage_textfield',
8      type : serverWidget.FieldType.TEXT,
9      label : 'Text'
10   });
11   field.isMandatory = true;
12   ...
13   //Add additional code

```

Field.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The field label. There is a 40-character limit for custom field labels.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1   //Add additional code
2   ...
3   var form = serverWidget.createForm({
4       title : 'Simple Form'
5   });
6   var field = form.addField({
7       id : 'custpage_textfield',
8       type : serverWidget.FieldType.TEXT,
9       label : 'Text'
10  });
11  var label = field.label;
12  ...
13  //Add additional code

```

Field.linkText

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text displayed for a link in place of the URL.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .

Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_textfield',
8     type : serverWidget.FieldType.URL,
9     label : 'URL'
10 });
11 field.linkText = 'NetSuite';
12 ...
13 //Add additional code

```

Field.maxLength

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The maximum length, in characters, of the field (only valid for text, rich text, long text, and textarea fields). This property is supported only on scripted fields created using the N/ui/serverWidget Module .
Type	number
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_textfield',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });

```

```

11 field.maxLength = 64;
12 ...
13 //Add additional code

```

Field.padding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of empty vertical character spaces above the field. This property is supported only on scripted fields created using the N/ui/serverWidget Module .
Type	number
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_textfield',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text'
10 });
11 field.padding = 50;
12 ...
13 //Add additional code

```

Field.richTextHeight

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The height of a rich text field, in pixels. The minimum value is 100 pixels and the maximum value is 500 pixels. Note: Rich Text Editing must be enabled.
Type	number
Supported Script Types	Suitelets User event scripts - beforeLoad entry point

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_textfield',
8   type : serverWidget.FieldType.RICHTEXT,
9   label : 'Rich Text'
10 });
11 field.richTextHeight = 50;
12 ...
13 //Add additional code
```

Field.richTextWidth

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The width of a rich text field, in pixels. The minimum value is 250 pixels and the maximum value is 800 pixels.  Note: Rich Text Editing must be enabled.
Type	number
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_textfield',
```

```

8      type : serverWidget.FieldType.RICHTEXT,
9      label : 'Rich Text'
10   });
11   field.richTextWidth = 100;
12   ...
13   //Add additional code

```

Field.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns the type of a body or sublist field. For example, the value can return text, date, currency, select, checkbox, etc. For more information about possible return values, see format.Type. The maximum character limit for select field types is 801.
Type	string (read-only)
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1   //Add additional code
2   ...
3   var form = serverWidget.createForm({
4       title : 'Simple Form'
5   });
6   var field = form.addField({
7       id : 'custpage_textfield',
8       type : serverWidget.FieldType.TEXT,
9       label : 'Text'
10  });
11  var fieldtype = field.type;
12  ...
13  //Add additional code

```

serverWidget.FieldGroup

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A field group on serverWidget.Assistant objects and on serverWidget.Form objects. A field group is a collection of fields that can be displayed in a one or two column format. Assign a field to a field group to label, hide or collapse a group of fields.
Supported Script Types	Suitelets User event scripts - beforeLoad entry point

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Methods and Properties	FieldGroup Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var fieldgroup = form.addFieldGroup({
7     id : 'fieldgroupid',
8     label : 'Field Group'
9 });
10 var field = form.addField({
11     id : 'custpage_textfield',
12     type : serverWidget.FieldType.TEXT,
13     label : 'Text',
14     container : 'fieldgroupid'
15 });
16 ...
17 //Add additional code

```

FieldGroup.isBorderHidden

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the field group border is hidden. If set to false, a border around the field group is displayed. The default value is false.
Type	Boolean
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...

```



```

3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var fieldgroup = form.addFieldGroup({
7   id : 'fieldgroupid',
8   label : 'Field Group'
9 });
10 var field = form.addField({
11   id : 'custpage_textfield',
12   type : serverWidget.FieldType.TEXT,
13   label : 'Text',
14   container : 'fieldgroupid'
15 });
16 fieldgroup.isBorderHidden = true;
17 ...
18 //Add additional code

```

FieldGroup.isCollapsible

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the field group can be collapsed. The default value is <code>false</code>, which means the field group displays as a static group that cannot be opened or closed. If set to <code>true</code>, the field group can be collapsed. Only supported for fields on serverWidget.Assistant objects
Type	Boolean
Supported Script Types	Suitelets User event scripts - <code>beforeLoad</code> entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var fieldgroup = form.addFieldGroup({
7   id : 'fieldgroupid',
8   label : 'Field Group'
9 });
10 var field = form.addField({
11   id : 'custpage_textfield',
12   type : serverWidget.FieldType.TEXT,
13   label : 'Text',
14   container : 'fieldgroupid'
15 });
16 fieldgroup.isCollapsible = true;
17 ...
18 //Add additional code

```

FieldGroup.isCollapsed

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether field group is collapsed or expanded. The default value is <code>false</code>, which means that when the page loads, the field group will not appear collapsed. If set to <code>true</code>, the field group is collapsed. Only supported for fields on serverWidget.Assistant objects
Type	Boolean
Supported Script Types	Suitelets User event scripts - <code>beforeLoad</code> entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var fieldgroup = form.addFieldGroup({
7   id : 'fieldgroupid',
8   label : 'Field Group'
9 });
10 var field = form.addField({
11   id : 'custpage_textfield',
12   type : serverWidget.FieldType.TEXT,
13   label : 'Text',
14   container : 'fieldgroupid'
15 });
16 var collapsed = fieldgroup.isCollapsed;
17 ...
18 //Add additional code

```

FieldGroup.isSingleColumn

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the field group is displayed into a single column. The default value is <code>false</code>.
Type	Boolean
Supported Script Types	Suitelets User event scripts - <code>beforeLoad</code> entry point

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var fieldgroup = form.addFieldGroup({
7     id : 'fieldgroupid',
8     label : 'Field Group',
9     isSingleColumn : true
10 });
11 var field = form.addField({
12     id : 'custpage_textfield',
13     type : serverWidget.FieldType.TEXT,
14     label : 'Text',
15     container : 'fieldgroupid'
16 });
17 var anotherField = form.addField({
18     id : 'custpage_another_textfield',
19     type : serverWidget.FieldType.TEXT,
20     label : 'Another Text',
21     container : 'fieldgroupid'
22 });
23 ...
24 //Add additional code

```

FieldGroup.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for the field group.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code

```

```
2  ...
3  var form = serverWidget.createForm({
4    title : 'Simple Form'
5  });
6  var fieldgroup = form.addFieldGroup({
7    id : 'fieldgroupid',
8    label : 'Field Group'
9  });
10 var field = form.addField({
11   id : 'custpage_textfield',
12   type : serverWidget.FieldType.TEXT,
13   label : 'Text',
14   container : 'fieldgroupid'
15 });
16 var label = fieldgroup.label;
17 ...
18 //Add additional code
```

serverWidget.Form

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A NetSuite-looking form After you create a Form object, you can: ■ Add a variety of scriptable elements to the form including fields, links, buttons, tabs, and sublists.
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Methods and Properties	Form Object Members
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 ...
7 //Add additional code
```

Form.addButton(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a button to a form.
--------------------	--------------------------

Returns	serverWidget.Button object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the button. If you are adding the button to an existing page, the internal ID must be in lowercase, contain no spaces, and include the prefix <code>custpage</code> . For example, if you add a button that appears as Update Order , the button internal ID should be something similar to custpage_updateorder .	2015.2
options.label	string	required	The label for this button.	2015.2
options.functionName	string	optional	The function name to call when clicking this button. The function name must be the name of the method defined in a custom module client script. For an example, see the help topic Update Fields on Current Record using a Custom Module Script and a User Event Script .  Important: Custom module client scripts with the specified function must be attached to the form. To learn more, see Form.clientScriptModulePath and Form.clientScriptFileId .	2016.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6
7 form.addButton({
8     id : 'buttonid',
9     label : 'Test'

```

```

10  });
11  ...
12  //Add additional code

```

Form.addCredentialField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a text field that lets you store credentials in NetSuite to be used when invoking services provided by third parties. The GUID generated by this field can be reused multiple times until the script executes again. For example, when executing credit card transactions, merchants need to store credentials in NetSuite that are used to communicate with Payment Gateway providers. The credentials added with this method can be used with the N/sftp Module and the N/https Module. Note the following about this method: ■ Credentials associated with this field are stored in encrypted form. ■ SuiteScript does not hold a credential in clear text mode. ■ NetSuite reports or forms will never provide the clear text form of a credential to the end user. ■ Any exchange of the clear text version of a credential with a third party must occur over an encrypted network transport of TLS 1.2 or SFTP. For more information about SFTP encryption, see Supported Cipher Suites and Host Key Types. ■ For no reason will NetSuite ever log the clear text value of a credential (for example, errors, debug message, alerts, system notes, and so on). ■ Decryption occurs though the scripts listed in the restrictToScriptIds parameter. These scripts can call https.createSecureString(options) to decrypt the GUID and create a SecureString instance.  Important: The default maximum length for a credential field is 32 characters. If needed, use the Field.maxLength property to change this value.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the credential field.	2015.2

Parameter	Type	Required / Optional	Description	Since
			The internal ID must be in lowercase, contain no spaces, and include the prefix <code>custpage</code> if you are adding the field to an existing page.	
<code>options.label</code>	string	required	The label for the credential field.	2015.2
<code>options.restrictToDomains</code>	string string[]	required	The domains that the credentials can be sent to, such as 'www.mysite.com'. Credentials cannot be sent to a domain that is not specified in this parameter. This value can be a single domain or a list of domains.  Note: Duplicate domain names in this parameter are not supported.	2015.2
<code>options.restrictToScriptIds</code>	string string[]	required	The IDs of the scripts that are allowed to use this credential field. For example, 'customscript_my_script'. Scripts defined in this parameter can call https.createSecureString(options) to decrypt the GUID.  Note: Duplicate script ids in this parameter are not supported.	2015.2
<code>options.restrictToCurrentUser</code>	Boolean	optional	Controls whether use of this credential is restricted to the same user that originally entered the credential. By default, the value is false, which means that multiple users can use the credential. For example, multiple clerks at a store making secure calls to a credit processor using a credential that represents the company they work for. If set to true, the credentials apply to a single user.	2015.2
<code>options.container</code>	string	optional	The internal ID of the tab or field group to add the credential field to. By default, the field is added to the main section of the form.	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#). For a complete script sample that uses `Form.addCredentialField`, see [Create a Form with a Credential Field](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var credField = form.addCredentialField({
7     id : 'username',
8     label : 'Username',
9     restrictToDomains : 'www.mysite.com',
10    restrictToScriptIds : 'customscript_my_script',

```

```

11     restrictToCurrentUser : false,
12   });
13   credField.maxLength = 64;
14   ...
15   //Add additional code

```

Form.addField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a field to a form.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the field. The internal ID must be in lowercase, contain no spaces, and include the prefix custpage if you are adding the field to an existing page. For example, if you add a field that appears as Purchase Details , the field internal ID should be something similar to custpage_purchasedetails or custpage_purchase_details.	2015.2
options.label	string	required	The label for this field.	2015.2
options.type	string	required	The field type for the field. Use the serverWidget.FieldType enum to define the field type.  Important: Long text fields created with SuiteScript have a character limit of 100,000. Long text fields created with Suitebuilder have a character limit of 1,000,000.	2015.2
options.source	string	optional	The internalId or scriptId of the source list or record type for this field if it is a select (List/Record) or multi-select field. Source list is a category containing standard lists and custom lists. Record types include standard record	2015.2

Parameter	Type	Required / Optional	Description	Since
			types like employee, sales order, and transaction, as well as custom record types. Examples of IDs include: standard record (salesorder, customer, employee); custom record (customrecord_car_model); standard list: (employeetype, ordertype); custom list (customlist_color).  Important: After you create a select or multi-select field that is sourced from a record or list, you cannot add additional values with Field.addSelectOption(options). The select values are determined by the source record or list.  Note: For radio fields only, the source parameter must contain the internal ID for the field. For more information about working with radio buttons, see the help topic SuiteScript 1.0 Documentation.	
options.container	string	optional	The internal ID of the tab or field group to add the field to. By default, the field is added to the main section of the form.	2016.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_abc_text',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 ...
12 //Add additional code

```

Form.addFieldGroup(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a group of fields to a form.
Returns	serverWidget.FieldGroup object
Supported Script Types	Suitelets

	User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	An internal ID for the field group.	2015.2
options.label	string	required	The label for this field group.	2015.2
options.tab	string	optional	The internal ID of the tab to add the field group to. By default, the field group is added to the main section of the form.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var fieldgroup = form.addFieldGroup({
7     id : 'fieldgroupid',
8     label : 'Field Group'
9 });
10 var field = form.addField({
11     id : 'custpage_text',
12     type : serverWidget.FieldType.TEXT,
13     label : 'Text',
14     container : 'fieldgroupid'
15 });
16 ...
17 //Add additional code

```

Form.addPageInitMessage(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Shows a message when users view a record or Suitelet. User event context can be used to control whether the message is shown on records in view, create, or edit mode (not applicable for Suitelets). See the help topic context.UserEventType . This method is called during a beforeLoad user event or in a Suitelet, and the message is later displayed on the client, after the pageInit script is completed.
Returns	void

Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2018.2

Parameters

The options object passed to the `Form.addPageInitMessage(options)` method takes a single property; either a [message.Message](#) object, or the same options object that can be passed to the [message.create\(options\)](#) method. The following tables list the parameters for the previously mentioned object property possibilities.

The [message.Message](#) object and the [message.Type](#) enum require loading the [N/ui/message Module](#).

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>options.message</code>	message.Message	required	Encapsulates the message to be shown on the form.	2018.2
<code>options.type</code>	message.Type	required	The message type.	2016.1
<code>options.title</code>	string	optional	The message title. This value defaults to an empty string.	2016.1
<code>options.message</code>	string	optional	The content of the message. This value defaults to an empty string.	2016.1
<code>options.duration</code>	int	optional	The amount of time, in milliseconds, to show the message. The default is 0, which shows the message until <code>Message.hide()</code> is called. If you specify a duration for <code>message.create()</code> and <code>message.show()</code> , the value from the <code>message.show()</code> method call takes precedence.	2018.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3
4 // Options object as parameter
5 form.addPageInitMessage({type: message.Type.INFORMATION, message: 'Hello world!', duration: 5000});
6
7 // Message object as parameter
8 var messageObj = message.create({type: message.Type.INFORMATION, message: 'Hello world!', duration: 5000}); form.addPageInitMes
  sage({message: messageObj});

```

```
9
10 // Show message when the record is in view mode
11 function beforeLoad(context) {
12     if(context.type === 'view')
13         context.form.addPageInitMessage(messageOptions);
14 }
15 ...
16 //Add additional code
```

Form.addPageLink(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a link to a form.
	 Note: You cannot choose where the page link appears.
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The text label for the link.	2015.2
options.type	string	required	The type of page link to add. Use the serverWidget.FormPageLinkType enum to set the value.	2015.2
options.url	string	required	The URL for the link.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 form.addPageLink({
7     type : serverWidget.FormPageLinkType.CROSSLINK,
8     title : 'NetSuite',
```

```

9      url : 'http://www.netsuite.com'
10    })
11    ...
12    //Add additional code

```

Form.addResetButton(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a Reset button to a form. The Reset button enables a user to clear the values from all fields on a form
Returns	serverWidget.Button object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.label	string	optional	The label used for this button. If no label is provided, the label defaults to Reset .	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1    //Add additional code
2    ...
3    var form = serverWidget.createForm({
4      title : 'Simple Form'
5    });
6    form.addResetButton({
7      label : 'Reset Button'
8    });
9    ...
10   //Add additional code

```

Form.addSecretKeyField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a secret key field to the form.
---------------------------	--------------------------------------

	This key can be used in crypto modules to perform encryption or hashing.  Important: The default maximum length for a secret key field is 32 characters. If needed, use the Field.maxLength property to change this value.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

 Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the secret key field. The internal ID must be in lowercase, contain no spaces, and include the prefix custpage if you are adding the field to an existing page.	2016.1
options.restrictToScriptIds	string string[]	required	The ID or list of IDs of the scripts where the key can be used.	2016.1
options.label	string	required	The UI label for the field.	2016.1
options.restrictToCurrentUser	Boolean	optional	Controls whether use of this secret key is restricted to the same user that originally entered the key. By default, the value is false, which means that multiple users can use the key. If set to true, the secret key applies to a single user.	2016.1
options.container	string	optional	The internal ID of the tab or field group to add the field to. By default, the field is added to the main section of the form.	2016.1

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/ui/serverWidget Module Script Samples . For a complete script example that uses <code>Form.addSecretKeyField(options)</code> , see N/crypto Module Script Samples .
--

```
1 | //Add additional code
```

```

2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  skField = form.addSecretKeyField({
7      id : 'password',
8      restrictToScriptIds : 'customscript_my_script',
9      label : 'Password',
10     restrictToCurrentUser : false
11 });
12 skField.maxLength = 64;
13 ...
14 //Add additional code

```

Form.addSublist(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Add a sublist to a form.  Note: If the row count exceeds 25, sorting is not supported on static sublists created using this method.
Returns	A serverWidget.Sublist object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the sublist. The internal ID must be in lowercase, contain no spaces, and include the prefix custpage if you are adding the sublist to an existing page. For example, if you add a sublist that appears as Purchase Details , the sublist internal ID should be something equivalent to custpage_purchasedetails or custpage_purchase_details.	2015.2
options.label	string	required	The label for this sublist.	2015.2
options.tab	string	optional	The tab under which to display this sublist. If empty, the sublist is added to the main tab.	2015.2
options.type	string	required	The sublist type. Use the serverWidget.SublistType enum to set the value.	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7   id : 'sublistid',
8   type : serverWidget.SublistType.INLINEEDITOR,
9   label : 'Inline Editor Sublist'
10 });
11 ...
12 //Add additional code
```

Form.addSubmitButton(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a submit button to a form.  Note: If the row count exceeds 25, sorting is not supported on static sublists created using this method.
Returns	serverWidget.Button object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.label	string	optional	The label for this button. If no label is provided, the label defaults to “Save”.	2016.1

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
```



```

2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  form.addSubmitButton({
7      label : 'Submit Button'
8  });
9  ...
10 //Add additional code

```

Form.addSubtab(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a subtab to a form. Note: To enable your subtab to appear on your form, there must be at least one object assigned to the subtab. Otherwise, the subtab will not appear. Note: If you have less than two subtabs on your form, the subtab will not appear. Instead the fields assigned to the tab will appear at the bottom of the form.
Returns	<code>serverWidget.Tab</code> object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the subtab. The internal ID must be in lowercase, contain no spaces. If you are adding the subtab to an existing page, include the prefix <code>custpage</code>. For example, if you add a subtab that appears as Purchase Details, the subtab internal ID should be something similar to <code>custpage_purchasedetails</code> or <code>custpage_purchase_details</code>.	2015.2
options.label	string	required	The label for this subtab.	2015.2
options.tab	string	optional	The tab under which to display this subtab. If empty, the subtab is added to the main tab.	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 form.addTab({
7     id : 'subtabid',
8     label : 'Subtab'
9 });
10 ...
11 //Add additional code
```

Form.addTab(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a tab to a form.
	 Note: To enable your tab to appear on your form, there must be at least one object assigned to the tab. Otherwise, the tab will not appear.
	 Note: If you have less than two tabs on your form, the tab will not appear. Instead the fields assigned to the tab will appear at the bottom of the form.
Returns	serverWidget.Tab object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

**Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the tab. The internal ID must be in lowercase and contain no spaces. If you are adding the tab to an	2015.2

Parameter	Type	Required / Optional	Description	Since
			existing page, include the prefix custpage. For example, if you add a subtab that appears as Purchase Details , the subtab internal ID should be something similar to custpage_purchasedetails or custpage_purchase_details.	
options.label	string	required	The label for this tab.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var tab = form.addTab({
7     id : 'tabid',
8     label : 'Tab'
9 });
10 ...
11 //Add additional code

```

Form.getButton(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a Button object by internal ID.
Returns	serverWidget.Button object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the button.	2015.2

Parameter	Type	Required / Optional	Description	Since
			Internal IDs must be in lowercase and contain no spaces.	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var button = form.addButton({
7     id : 'buttonid',
8     label : 'Test'
9 });
10 var button = form.getButton({
11     id : 'buttonid'
12 });
13 ...
14 //Add additional code

```

Form.getField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a Field object by internal ID.
Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the field. Internal IDs must be in lowercase and contain no spaces.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 form.addField({
7     id : 'custpage_text',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 var field = form.getField({
12     id : 'textfield'
13 });
14 ...
15 //Add additional code
```

Form.getSublist(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a Sublist object by internal ID.
Returns	serverWidget.Sublist object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the sublist. Internal IDs must be in lowercase and contain no spaces.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
```

```

2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  form.addSublist({
7      id : 'sublistid',
8      type : serverWidget.SublistType.INLINEEDITOR,
9      label : 'Inline Editor Sublist'
10 });
11 var sublist = form.getSublist({
12     id : 'sublistid'
13 });
14 ...
15 //Add additional code

```

Form.getSubtab(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a subtab by internal ID.
Returns	serverWidget.Tab object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the subtab. Internal IDs must be in lowercase and contain no spaces.	2015.2

Syntax

! **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  form.addSubtab({

```

```
7     id : 'subtabid',
8     label : 'Subtab'
9   });
10   var subtab = form.getSubtab({
11     id : 'subtabid'
12   });
13   ...
14   //Add additional code
```

Form.getTab(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a tab object from its internal ID.
Returns	serverWidget.Tab object.
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the tab to retrieve.	2016.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 form.addTab({
7   id : 'tabid',
8   label : 'Tab'
9 });
10 var tab = form.getTab({
11   id : 'tabid'
12 });
13 ...
14 //Add additional code
```

Form.getTabs()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an array that contains the internal IDs of all tabs in a form.
Returns	string[]
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 form.addTab({
7     id : 'tabid',
8     label : 'Tab'
9 });
10 var tabs = form.getTabs();
11 ...
12 //Add additional code

```

Form.insertField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a field in front of another field.
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

**Note:** The options parameter is a JavaScript object.

insertField is intended for custom fields created within beforeLoad. Moving fields between different tabs is not supported.

Parameter	Type	Required / Optional	Description	Since
options.field	serverWidget.Field	required	The Field object to insert.	2016.1
options.isBefore	boolean	optional	Used to specify whether the field is inserted before or after the next field (options.nextfield).	2024.2
options.nextfield	string	required	The internal ID name of the field you are inserting a field in front of.	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field1 = form.addField({
7   id : 'custpage_text',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text 1'
10 });
11 var field2 = form.addField({
12   id : 'custpage_text2',
13   type : serverWidget.FieldType.TEXT,
14   label : 'Text 2'
15 });
16 var field3 = form.addField({
17   id : 'custpage_text3',
18   type : serverWidget.FieldType.TEXT,
19   label : 'Text 3'
20 });
21 form.insertField({
22   field : field3,
23   isBefore: true,
24   nextfield : 'custpage_text'
25 });
26 ...
27 //Add additional code
```

Form.insertSublist(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a sublist in front of another sublist.
--------------------	--

Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.sublist	serverWidget.Sublist	required	The Sublist object to insert.	2016.1
options.nextsublist	string	required	The internal ID name of the sublist you are inserting a sublist in front of.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var sublist1 = form.addSublist({
7   id : 'sublistid1',
8   type : serverWidget.SublistType.INLINEEDITOR,
9   label : 'Inline Editor Sublist'
10 });
11 var sublist2 = form.addSublist({
12   id : 'sublistid2',
13   type : serverWidget.SublistType.INLINEEDITOR,
14   label : 'Inline Editor Sublist'
15 });
16 form.insertSublist({
17   sublist : sublist2,
18   nextsublist : 'sublistid1'
19 });
20 ...
21 //Add additional code

```

Form.insertSubtab(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a subtab before another subtab.
---------------------------	---

Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.subtab	serverWidget.Tab	required	The Subtab object to insert.	2016.1
options.nextsub	string	required	The internal ID name of the subtab before which you are inserting the subtab.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var subtab1 = form.addSubtab({
7   id : 'subtabid1',
8   label : 'Subtab'
9 });
10 var subtab2 = form.addSubtab({
11   id : 'subtabid2',
12   label : 'Subtab'
13 });
14 form.insertSubtab({
15   subtab : subtab2,
16   nextsub : 'subtabid1'
17 });
18 ...
19 //Add additional code

```

Form.insertTab(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a tab in front of another tab.
---------------------------	--

Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.tab	serverWidget.Tab	required	The Tab object to insert.	2016.1
options.nexttab	string	required	The internal ID name of the tab you are inserting a tab in front of.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var tab1 = form.addTab({
7   id : 'tabid1',
8   label : 'Tab'
9 });
10 var tab2 = form.addTab({
11   id : 'tabid2',
12   label : 'Tab'
13 });
14 form.insertTab({
15   tab: tab2,
16   nexttab: 'tabid1'
17 });
18 ...
19 //Add additional code

```

Form.removeButton(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes a button.
---------------------------	-------------------

	This method can be used on custom buttons and certain built-in NetSuite buttons. For more information about built-in NetSuite buttons, see the help topic Button IDs .
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID name of the button to remove. See the help topic Button IDs . The internal ID must be in lowercase and contain no spaces.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 function beforeLoad(context) {
4     var yourForm = context.form;
5     yourForm.removeButton('refund');
6 }
7 ...
8 //Add additional code

```

Form.updateDefaultValues(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Updates the default values of multiple fields on the form.
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/ui/serverWidget Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
values	Object[]	required	An object containing an array of name/value pairs that map field names to field values.	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var field = form.addField({
7   id : 'custpage_text',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text'
10 });
11 form.updateDefaultValues({
12   custpage_text : 'Text Goes Here'
13 })
14 ...
15 //Add additional code

```

Form.clientScriptFileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal file ID of client script file to be used in this form. Use this property when attaching an on demand client script to a server script.  Note: If you deploy a client script to a form using Form.clientScriptFileId or Form.clientScriptModulePath, using the N/log Module adds the logs to the deployment of the parent script. The parent script can be either a beforeLoad user event script or a SuiteScript 2.x Suitelet Script Type.
Type	number
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2015.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You attempted to set this value when the <code>Form.clientScriptModulePath</code> property value has already been specified. For more information, see Form.clientScriptModulePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 form.clientScriptFileId = 32;
7 ...
8 //Add additional code

```

Form.clientScriptModulePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The relative path to the client script file to be used in this form. Use this property when attaching an on demand client script to a server script. Note: If you deploy a client script to a form using <code>Form.clientScriptFileId</code> or <code>Form.clientScriptModulePath</code>, using the N/log Module adds the logs to the deployment of the parent script. The parent script can be either a <code>beforeLoad</code> user event script or a SuiteScript 2.x Suitelet Script Type.
Type	string
Supported Script Types	Suitelets User event scripts - <code>beforeLoad</code> entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Since	2016.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You attempted to set this value when the <code>Form.clientScriptFileId</code> property value has already been specified. For more information, see Form.clientScriptFileId .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 objForm.clientScriptModulePath = 'SuiteScripts/formBehavior.js';
4 ...
5 //Add additional code
```

Form.title

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The title used for the form.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var title = form.title;
7 ...
8 //Add additional code
```

serverWidget.List

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A list page.
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Methods and Properties	List Object Members

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 ...
7 //Add additional code

```

List.addButton(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a button to a list.
Returns	serverWidget.Button object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the button. The internal ID must be in lowercase, contain no spaces, and include the prefix custpage if you are adding the button to an existing page. For example, if you add a button that appears as Update Order , the button internal ID should be something similar to custpage_updateorder.	2015.2
options.label	string	required	The label for this button.	2015.2
options.functionName	string	optional	The function name to call when clicking this button. The function name must be the name of the method defined in a custom module client script. For examples, see the help topic N/currentRecord Samples .	2015.2

Parameter	Type	Required / Optional	Description	Since
			The function name must not be a client script entry point. For the list of client script entry points, see the help topic SuiteScript 2.x Client Script Entry Points and API.  Important: Custom module client scripts with the specified function must be attached to the list. To learn more, see List.clientScriptModulePath and List.clientScriptFileId.	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 list.addButton({
7     id : 'buttonid',
8     label : 'Test'
9 });
10 ...
11 //Add additional code

```

List.addColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a column to a list.
Returns	serverWidget.ListColumn object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of this column.	2015.2

Parameter	Type	Required / Optional	Description	Since
			The internal ID must be in lowercase, contain no spaces.	
options.label	string	required	The label for this column.	2015.2
options.type	string	required	The field type for this column. Set this value using the serverWidget.FieldType enum.  Note: Not all field types are supported. Check supported values in serverWidget.FieldType	2015.2
options.align	string	optional	The layout justification for this column. Set this value using the serverWidget.LayoutJustification enum. The default value is LEFT.	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4   title : 'Simple List'
5 });
6 list.addColumn({
7   id : 'column1',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text',
10  align : serverWidget.LayoutJustification.RIGHT
11 });
12 ...
13 //Add additional code

```

List.addEditColumn(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a column containing Edit or Edit/View links to a Suitelet or Portlet list. These Edit or Edit/View links appear to the left of a previously existing column.
Returns	serverWidget.ListColumn object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.column	serverWidget.ListColumn	required	Column that acts as a reference. The Edit/View column is added to the left of the column specified here.	2015.2
options.showHrefCol	Boolean	optional	If set to true, the URL for the link is clickable.	2015.2
options.showView	Boolean	optional	If true then an Edit/View column will be added. Otherwise only an Edit column will be added.	2015.2
options.link	string	optional	The Edit/View base link. (For example: /app/common/entity/employee.nl) The complete link is formed like this: <link>?<linkParamName>=<row data from linkParam>. (For example: /app/common/entity/employee.nl?id=123)	2019.2
options.linkParam	string	optional	The internal ID of the field in the row data where to take the parameter from. The default value is the value set in the options.column parameter.  Tip: In most cases, the value to use here is internalid	2019.2
options.linkParamName	string	optional	The name of the parameter. The default value is id.	2019.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6
7 list.addColumn({
8     id: 'internalid',
9     type: 'text',
10    label: 'Number'
11 });
12 list.addColumn({
13     id: 'entityid',
14     type: 'text',
15     label: 'Name'
16 });
17 list.addEditColumn({
18     column : 'entityid',
19     showHrefCol: true,
20     showView : true,

```

```

21     link: '/app/common/entity/employee.nl',
22     linkParam: 'internalid',
23     linkParamName: 'id',
24   });
25   ...
26   //Add additional code

```

List.addPageLink(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a link to a list.
Returns	serverWidget.List (it return itself)
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The text label for the link.	2015.2
options.type	string	required	The type of page link to add. Set this value using the serverWidget.FormPageLinkType enum.	2015.2
options.url	string	required	The URL for the link.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1   //Add additional code
2   ...
3   var list = serverWidget.createList({
4     title : 'Simple List'
5   });
6   list.addPageLink({
7     title : 'NetSuite',
8     type : serverWidget.FormPageLinkType.CROSSLINK,
9     url : 'http://www.netsuite.com'
10  });
11  ...
12  //Add additional code

```

List.addRow(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a single row to a list.
Returns	serverWidget.List
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.row	Object[]	required	A row that consists of either a search.Result array, or name/value pairs. Each pair should contain the value for the corresponding Column object in the list.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 list.addRow({
7     row : { columnid1 : 'value1', columnid2 : 'value2' }
8 });
9 ...
10 //Add additional code

```

List.addRows(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds multiple rows to a list.
Returns	serverWidget.List
Supported Script Types	SuiteScript 2.x Suitelet Script Type

Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.rows	Object[]	required	An array of rows that consist of either a search.Result array, or an array of name/value pairs. Each pair should contain the value for the corresponding Column object in the list.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 list.addRow({
4     rows : [{columnid1 : 'value1', columnid2 : 'value2'},
5             {columnid1 : 'value2', columnid2 : 'value3'}]
6 });
7 ...
8 //Add additional code

```

List.clientScriptFileId

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The File Cabinet ID of client script file to be used in this list.
Type	number
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2016.1

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You attempted to set this value when the List.clientScriptModulePath property value has already been specified. For more information, see List.clientScriptModulePath .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 list.clientScriptFileId = 123;
7 ...
8 //Add additional code
```

List.clientScriptModulePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The relative path to the client script file to be used in this list.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2016.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You attempted to set this value when the List.clientScriptFileId property value has already been specified. For more information, see List.clientScriptFileId .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 objList.clientScriptModulePath = 'SuiteScripts/listBehavior.js';
4 ...
5 //Add additional code
```

List.style

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The display style for this list. For more information about possible values, see serverWidget.ListStyle .
Type	string

Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 list.style = serverWidget.ListStyle.REPORT;
7 ...
8 //Add additional code

```

List.title

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The list title.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 var title = list.title;
7 ...
8 //Add additional code

```

serverWidget.ListColumn

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A list column
---------------------------	---------------

Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Methods and Properties	ListColumn Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 var listcolumn = list.addColumn({
7     id : 'column1',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text',
10    align : serverWidget.LayoutJustification.RIGHT
11 });
12 ...
13 //Add additional code

```

ListColumn.addParamToURL(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a URL parameter (optionally defined per row) to the list column's URL.
Returns	serverWidget.ListColumn object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.param	string	required	The name for the parameter.

Parameter	Type	Required / Optional	Description
options.value	string	required	The value for the parameter.
options.dynamic	Boolean	optional	If true, then the parameter value is an alias that is calculated per row.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 var listcolumn = list.addColumn({
7     id : 'column1',
8     type : serverWidget.FieldType.URL,
9     label : 'URL',
10 });
11 listcolumn.addParamToURL({
12     param : 'index',
13     value : '3'
14 })
15 ...
16 //Add additional code

```

ListColumn.setURL(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the base URL for the list column.
Returns	serverWidget.ListColumn
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.1

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.url	string	required	The base URL or a column in the data source that returns the base URL for each row

Parameter	Type	Required / Optional	Description
options.dynamic	Boolean	optional	If true, then the URL is an alias that is calculated per row.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 var listcolumn = list.addColumn({
7     id : 'column1',
8     type : serverWidget.FieldType.URL,
9     label : 'URL',
10 });
11 listcolumn.setURL({
12     url : 'http://www.netsuite.com'
13 })
14 ...
15 //Add additional code

```

ListColumn.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	This list column label.
Type	string
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 var listcolumn = list.addColumn({
7     id : 'column1',
8     type : serverWidget.FieldType.URL,
9     label : 'URL',
10 });
11 var label = listcolumn.label;
12 ...

```

13 | //Add additional code

serverWidget.Sublist

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A sublist on a serverWidget.Form or an serverWidget.Assistant object. To add a sublist, use Assistant.addSublist(options) or Form.addSublist(options).  Note: This object is read-only except for instances created using the serverWidget module using Suitelets or beforeLoad user event scripts.
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Methods and Properties	Sublist Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7   id : 'sublist',
8   type : serverWidget.SublistType.INLINEEDITOR,
9   label : 'Inline Editor Sublist'
10 });
11 ...
12 //Add additional code

```

Sublist.addButton(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a button to a sublist.
Returns	serverWidget.Button
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The internal ID name of the button. The internal ID must be in lowercase and without spaces.
options.label	string	required	The label for the button.
options.functionName	string	optional	The function name to call when clicking this button. The function name must be the name of the method defined in a custom module client script. For examples, see the help topic N/currentRecord Samples .  Important: Custom module client scripts with the specified function must be attached to the form. To learn more, see Form.clientScriptModulePath and Form.clientScriptFileId .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'
10 });
11 sublist.addButton({
12     id : 'buttonid',
13     label : 'Test'
14 });
15 ...
16 //Add additional code

```

Sublist.addField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a field to a sublist.
---------------------------	----------------------------

Returns	serverWidget.Field object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID for this field. The internal ID must be in lowercase and without spaces.	2015.2
options.label	string	required	The UI label for this field.	2015.2
options.type	string	required	The field type. Use the serverWidget.FieldType enum to set this value. The DATETIME, FILE, HELP, INLINEHTML, LABEL, LONGTEXT, MULTISELECT, RADIO and RICHTEXT values are not supported with this method.  Note: If you have set the type parameter to SELECT, and you want to add custom options to the select field, you must set source to NULL.	2015.2
options.source	string	optional	The internalId or scriptId of the source list for this field. Use this parameter if you are adding a select (List/Record) type of field.  Note: If you want to add custom options on a select field, you must set the source parameter to NULL.  Important: After you create a select or multi-select field that is sourced from a record or list, you cannot add additional values with Field.addSelectOption(options) . The select values are determined by the source record or list.	2015.2
options.container	string	optional	The internal ID of the tab or field group to add the field to.	2015.2

Parameter	Type	Required / Optional	Description	Since
			Use this parameter to specify either a tab or a field group to place the field in. By default, the field is added to the main section of the form.	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 /**
3  * @NApiVersion 2.x
4  * @NScriptType suitelet
5  */
6 define(['N/ui/serverWidget'], function(serverWidget) {
7     return Object.freeze({
8         onRequest: function(context) {
9             var form = serverWidget.createForm({
10                 title: 'Simple Form'
11             });
12             var sublist = form.addSublist({
13                 id: 'sublist',
14                 type: serverWidget.SublistType.INLINEEDITOR,
15                 label: 'Inline Editor Sublist'
16             });
17             sublist.addField({
18                 id: 'fieldid',
19                 type: serverWidget.FieldType.DATE,
20                 label: 'Date'
21             });
22             context.response.writePage(form);
23         }
24     });
25 }); //Add additional code
26

```

Sublist.addMarkAllButtons()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a Mark All and an Unmark All button to a LIST type of sublist. See the help topic Buttons and Menus in NetSuite for information about how Mark All/Unmark All Buttons works in NetSuite.
Returns	A serverWidget.Button[] object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module

Since	2015.2
-------	--------

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7   id : 'sublist',
8   type : serverWidget.SublistType.LIST,
9   label : 'List Sublist'
10 });
11 sublist.addMarkAllButtons();
12 ...
13 //Add additional code
```

Sublist.addRefreshButton()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a Refresh button to a LIST type of sublist.
Returns	serverWidget.Button object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7   id : 'sublist',
8   type : serverWidget.SublistType.LIST,
9   label : 'List Sublist'
10 });
11 sublist.addRefreshButton();
12 ...
13 //Add additional code
```

Sublist.getField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a Field object on a sublist.
Returns	serverWidget.Field
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The field internal ID (for example, use item as the ID for the Item field). For more information about supported sublists, internal IDs, and field IDs, see the SuiteScript Records Browser .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var itemField = form.getSublist({id: 'item'}).getField({id: 'item'});
4 ...
5 //Add additional code

```

Sublist.getSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Gets a field value on a sublist.
Returns	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The internal ID of a field.
options.line	number	required	The line number for this field.

 **Note:** The first line number on a sublist is 0 (not 1).

Errors

Error Code	Thrown If
SSS_MISSING_REQD_ARGUMENT	A required parameter is not passed.
YOU_CANNOT_CALL_1_METHOD_ON_SUBRECORD_FIELD_SUBLIST_2_FIELD_3	You called {1} method on subrecord field. Sublist: {2}, field: {3}.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var sublistvalue = sublist.getSublistValue({
4     id : 'quantity',
5     line: 1
6 })
7 ...
8 //Add additional code

```

Sublist.insertField(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a field in front of another field.
Returns	void
Supported Script Types	Suitelets

	User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2024.2

Parameters

 **Note:** The options parameter is a JavaScript object.

insertField is intended for custom fields created within beforeLoad. Moving fields between different tabs is not supported.

Parameter	Type	Required / Optional	Description	Since
options.field	serverWidget.Field	required	The Field object to insert.	2024.2
options.isBefore	boolean	optional	Used to specify whether the field is inserted before or after the next field (options.nextfield).	2024.2
options.nextfield	string	required	The internal ID name of the field you are inserting a field in front of.	2024.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title : 'Simple Form'
5 });
6 var sublist = form.getSublist({
7   id : 'custsublist_custom'
8 });
9 var field1 = sublist.addField({
10  id : 'custpage_text',
11  type : serverWidget.FieldType.TEXT,
12  label : 'Text 1'
13 });
14 var field2 = sublist.addField({
15  id : 'custpage_text2',
16  type : serverWidget.FieldType.TEXT,
17  label : 'Text 2'
18 });
19 var field3 = sublist.addField({
20  id : 'custpage_text3',
21  type : serverWidget.FieldType.TEXT,
22  label : 'Text 3'
23 });
24 sublist.insertField({
25   field : field3,
26   isBefore: true,
27   nextfield : 'custpage_text'
28 });
```

```

29 ...
30 //Add additional code

```

Sublist.setSublistValue(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets the value of a sublist field.
Returns	void
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The internal ID name of the line item field being set.
options.line	number	required	The line number for this field. Note: The first line number on a sublist is 0 (not 1).
options.value	string	required	The value for the field being set. Note: Checkbox fields accept string ('T'/'F') values, not Boolean.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title: 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id: 'sublist',
8     type: serverWidget.SublistType.INLINEEDITOR,
9     label: 'Inline Editor Sublist'

```

```
10  });
11
12  sublist.addField({
13    id: 'sublist',
14    type: ui.FieldType.TEXT,
15    label: 'Text Field'
16  });
17
18  sublist.setSublistValue({
19    id: 'sublist',
20    line: 2,
21    value: "Text"
22  });
23  ...
24  //Add additional code
```

Sublist.updateTotallingFieldId(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Sets a field as a totalling column, which is used to calculate and display a running total for the sublist.
Returns	serverWidget.Sublist object
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The internal ID of the field used to calculate the total field.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

 **Note:** This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript 2.x Global Objects](#).

```
1  /**
2  * @NApiVersion 2.x
```

```

3  * @NScriptType Suitelet
4  */
5  define(['N/ui/serverWidget', 'N/record'], function(serverWidget, record){
6      return {
7          onRequest: function(params) {
8              var form = serverWidget.createForm({
9                  title: 'Simple Form'
10             });
11             var sublistObj2 = form.addSublist({
12                 id: 'mylist',
13                 type: serverWidget.SublistType.INLINEEDITOR,
14                 label: 'List'
15             });
16             sublistObj2.addField({
17                 id: 'description',
18                 type: serverWidget.FieldType.TEXT,
19                 label: 'Description'
20             });
21             sublistObj2.addField({
22                 id: 'amount',
23                 type: serverWidget.FieldType.CURRENCY,
24                 label: 'Amount'
25             });
26             sublistObj2.updateTotallingFieldId({
27                 id: 'amount'
28             });
29             sublistObj2.setSublistValue({
30                 id: 'description',
31                 line: 0,
32                 value: 'foo'
33             });
34             sublistObj2.setSublistValue({
35                 id: 'amount',
36                 line: 0,
37                 value: '10'
38             });
39             sublistObj2.setSublistValue({
40                 id: 'description',
41                 line: 1,
42                 value: 'bar'
43             });
44             sublistObj2.setSublistValue({
45                 id: 'amount',
46                 line: 1,
47                 value: '15'
48             });
49             form.addSublist({
50                 id: 'dummy',
51                 type: serverWidget.SublistType.STATICLIST,
52                 label: 'Dummy'
53             });
54             params.response.writePage(form);
55         }
56     };
57 });

```

Sublist.updateUniqueFieldId(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Indicates a field that must have unique values across the rows in the sublist.
	 Note: This method is available on inlineeditor and editor sublists only.
Returns	serverWidget.Sublist object
Supported Script Types	Suitelets

	User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The internal ID of the field that you want to contain unique values

Syntax

The following code sample uses the `Sublist.updateUniqueFieldId(options)` method to set the **Date** field as unique. This means that a user will not be able to enter two rows with the same date on the sublist.

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'
10 });
11 sublist.addField({
12     id : 'fieldid',
13     type : serverWidget.FieldType.DATE,
14     label : 'Date'
15 });
16 sublist.updateUniqueFieldId({
17     id : 'fieldid'
18 })
19 ...
20 //Add additional code

```

Sublist.displayType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The display style for a sublist. Use the serverWidget.SublistDisplayType enum to set this value.
Type	string
Supported Script Types	Suitelets

	User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'
10 });
11 sublist.displayType = serverWidget.SublistDisplayType.HIDDEN;
12 ...
13 //Add additional code

```

Sublist.helpText

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The inline help text for a sublist.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'

```

```

10  });
11  sublist.helpText = "Help Text Goes Here.";
12  ...
13  //Add additional code

```

Sublist.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for this sublist.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  var sublist = form.addSublist({
7      id : 'sublist',
8      type : serverWidget.SublistType.INLINEEDITOR,
9      label : 'Inline Editor Sublist'
10 });
11 var label = sublist.label;
12 ...
13 //Add additional code

```

Sublist.lineCount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The number of line items on a sublist. Note: The first line number on a sublist is 0 (not 1).
Type	number (read-only)
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .

Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'
10 });
11 var numLines = sublist.lineCount;
12 ...
13 //Add additional code

```

serverWidget.Tab

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A tab or subtab on a serverWidget.Form object. You can add a new tab or subtab to a form using one of the following methods: - Form.addSubtab(options) - Form.addTab(options) - Form.insertSubtab(options) - Form.insertTab(options) The internal ID must be in lowercase, contain no spaces, and include the prefix custpage if you are adding the field to an existing page. Note: To enable your tab to appear on your form, there must be at least one object assigned to the tab. Otherwise, the tab will not appear. Note: If you have less than two tabs on your form, the tab will not appear. Instead the fields assigned to the tab will appear at the bottom of the form.
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/ui/serverWidget Module
Methods and Properties	Tab Object Members
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var tab = form.addTab({
7     id : 'tabid1',
8     label : 'Tab 1'
9 });
10
11 var tab = form.addTab({
12     id : 'tabid2',
13     label : 'Tab 2'
14 });
15
16 form.addField({
17     id : 'custpage_tabid1',
18     type: ui.FieldType.TEXT,
19     label: 'Tab 1 Field'
20 });
21
22 form.addField({
23     id : 'custpage_tabid2',
24     type: ui.FieldType.TEXT,
25     label: 'Tab 2 Field'
26 });...
27 //Add additional code
```

Tab.helpText

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The inline help text for a tab or subtab.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var tab = form.addTab({
```

```

7      id : 'tabid',
8      label : 'Tab'
9    });
10    tab.helpText = 'Help Text Goes Here';
11    ...
12    //Add additional code

```

Tab.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for a tab or subtab.
Type	string
Supported Script Types	Suitelets User event scripts - beforeLoad entry point For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/ui/serverWidget Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1    //Add additional code
2    ...
3    var form = serverWidget.createForm({
4      title : 'Simple Form'
5    });
6    var tab = form.addTab({
7      id : 'tabid',
8      label : 'Tab'
9    });
10   var label = tab.label;
11   ...
12   //Add additional code

```

serverWidget.createAssistant(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an assistant object. Note: The assistant cannot be used on externally available Suitelets.
Returns	serverWidget.Assistant object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/ui/serverWidget Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The title of the assistant. This title appears at the top of all assistant pages.	2015.2
options.hideNavBar	Boolean	optional	Indicates whether to hide the navigation bar menu. By default, set to false. The header appears in the top-right corner on the assistant. If set to true, the header on the assistant is hidden from view.	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 ...
7 //Add additional code

```

For more information, see the help topic [Sample Custom Assistant Script](#).

serverWidget.createForm(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a form object.
Returns	serverWidget.Form object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The title of the form.	2015.2
options.hideNavBar	Boolean	optional	Indicates whether to hide the navigation bar menu. By default, set to false. The header appears in the top-right corner on the form. If set to true, the header on the assistant is hidden from view.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code ...
2 var form = serverWidget.createForm({
3   title : 'Simple Form'
4 });
5 ...
6 //Add additional code

```

serverWidget.createList(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a standalone list.
Returns	serverWidget.List object
Supported Script Types	Suitelets For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/ui/serverWidget Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.title	string	required	The title of the list.	2016.1

Parameter	Type	Required / Optional	Description	Since
options.hideNavBar	Boolean	optional	Indicates whether to hide the navigation bar menu. By default, set to false. The header appears in the top-right corner on the form. If set to true, the header on the assistant is hidden from view.	2016.1

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 ...
7 //Add additional code

```

serverWidget.AssistantSubmitAction

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for submit actions performed by the user. This enum is used to set the value of the Assistant.getLastAction(). After a finish action is submitted, by default, the text “Congratulations! You have completed the <assistant title>” appears on the finish page. In a non-sequential process (steps are unordered), jump is used to move to the user’s last action.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value
BACK
CANCEL

Value
FINISH
JUMP
NEXT

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var assistant = serverWidget.createAssistant({
4     title : 'Simple Assistant'
5 });
6 ...
7 if (assistant.getLastAction() === serverWidget.AssistantSubmitAction.CANCEL) {
8     ...
9 }
10 ...
11 //Add additional code
```

serverWidget.FieldBreakType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported field break types. This enum is used to set the value of the breakType parameter when Field.updateBreakType(options) is called.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value	Description
NONE	This is the default value for field break type.
STARTCOL	This value moves the field into a new column. Additionally, it disables automatic field balancing if set on any field.
STARTROW	This value places a field located outside of a field group on a new row. This value only works on fields with a Field Layout Type set to OUTSIDE, OUTSIDEABOVE or OUTSIDEBELOW. For more information, see serverWidget.FieldLayoutType and Field.updateLayoutType(options) .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_text',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11
12 field.updateLayoutType({
13     layoutType: serverWidget.FieldLayoutType.OUTSIDE
14 });
15
16 field.updateBreakType({
17     breakType : serverWidget.FieldBreakType.STARTROW
18 });
19 ...
20 //Add additional code
```

serverWidget.FieldDisplayType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported field display types. This enum is used to set the value of the displayType parameter when Field.updateDisplayType(options) is called.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value	Description of Field Type
DISABLED	Prevents a user from changing the field
ENTRY	The sublist field appears as a data entry input field (for a select field without a checkbox)
HIDDEN	The field on the form is hidden
INLINE	The field appears as inline text
NODISPLAY	The field is not displayed on the form but can be made visible (for example, by an attached client script upon a field change)

Value	Description of Field Type
NORMAL	The field appears as a normal input field (for non-sublist fields)
READONLY	The field is disabled but it is still selectable and scrollable (for textarea fields)

Syntax



Important: The following code samples show the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var field = form.addField({
7     id : 'custpage_text',
8     type : serverWidget.FieldType.TEXT,
9     label : 'Text'
10 });
11 field.updateDisplayType({
12     displayType: serverWidget.FieldDisplayType.HIDDEN
13 });
14 ...
15 //Add additional code

```

serverWidget.FieldLayoutType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for the supported types of field layouts. This enum is used to set the value of the <code>layoutType</code> parameter when Field.updateLayoutType(options) is called.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value	Description
STARTROW	This value makes the field appear first in a horizontally aligned field group in the normal field layout.
MIDROW	This value makes the field appear in the middle of a horizontally aligned field group in the normal field layout.

Value	Description
ENDROW	This value makes the field appear last in a horizontally aligned field group in the normal field layout.
OUTSIDE	This value makes the field appear outside (above or below based on form default) the normal field layout area.
OUTSIDEBELOW	This value makes the field appear below the normal field layout area. Using this enables you to position a field below a field group.
OUTSIDEABOVE	This value makes the field appear above the normal field layout area. Using this enables you to position a field above a field group.
NORMAL	This value makes the fields appear in its default position.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4   title: 'Simple Form'
5 });
6 var field = form.addField({
7   id: 'custpage_text',
8   type: serverWidget.FieldType.TEXT,
9   label: 'Text'
10 });
11 field.updateLayoutType({
12   layoutType: serverWidget.FieldLayoutType.OUTSIDEBELOW
13 });
14 ...
15 //Add additional code
```

serverWidget.FieldType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the values for supported field types. This enum is used to set the value of the type parameter when Form.addField(options) is called.  Important: Long text fields created with SuiteScript have a character limit of 100,000. Long text fields created with SuiteBuilder have a character limit of 1,000,000.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members

Since	2015.2
-------	--------

Values

■ CHECKBOX	■ LONGTEXT
■ CURRENCY	■ MULTISELECT
■ DATE	■ PASSWORD
■ DATETIME	■ PERCENT
■ DATETIMETZ	■ PHONE
■ EMAIL	■ SELECT
■ FILE	■ RADIO
■ FLOAT	■ RICHTEXT
■ HELP	■ TEXT
■ INLINEHTML	■ TEXTAREA
■ INTEGER	■ TIMEOFDAY
■ IMAGE	■ URL
■ LABEL	

Consider the following as you work with these field types:

- The DATETIME field type is not supported with `addField` methods, you must specify DATETIMETZ.
- The FILE field type is available only for Suitelets and will appear on the main tab of the Suitelet page. FILE fields cannot be added to tabs, subtabs, sublists, or field groups and are not allowed on existing pages. It is not supported with [List.addColumn\(options\)](#).
- The IMAGE field type is available only for fields that appear on lists, static list sublists or forms.
- The INLINEHTML field type should be considered as a 'write-only' type of field used to add a field on a form. The INLINEHTML field type does not support labels. The `label` parameter is required for all field types, but the UI does not display the labels specified for INLINEHTML fields. If you want to display an INLINEHTML field with a label, add a separate custom LABEL field.
- All field types that work with text handle input as plain text, except for the INLINEHTML field type. HTML input is supported only in the INLINEHTML field type.
- Radio buttons that are inside one container are exclusive. The method `addField` on form has an optional parameter `container`. For an example, see [FieldGroup.label](#).
- The DATETIME, FILE, HELP, INLINEHTML, LABEL, LONGTEXT, MULTISELECT, RADIO and RICHTEXT field types are not supported with [Sublist.addField\(options\)](#).
- The CHECKBOX, DATETIME, DATETIMETZ, FILE, HELP, INLINEHTML, LABEL, MULTISELECT, SELECT, and RADIO field types are not supported with [List.addColumn\(options\)](#).

For a description of these field types, and additional information regarding character limits and format restrictions, see the help topic [Field Type Descriptions for Custom Fields](#).

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 | //Add additional code
```

```

2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  var field = form.addField({
7      id : 'custpage_text',
8      type : serverWidget.FieldType.TEXT,
9      label : 'Text'
10 });
11 ...
12 //Add additional code

```

serverWidget.FormPageLinkType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported page link types on a form. This enum is used to set the value of the type parameter when Form.addPageLink(options) is called.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value	Description
BREADCRUMB	Link appears on the top-left corner after the system bread crumbs
CROSSLINK	Link appears on the top-right corner

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var form = serverWidget.createForm({
4      title : 'Simple Form'
5  });
6  form.addPageLink({
7      type : serverWidget.FormPageLinkType.CROSSLINK,
8      title : 'NetSuite',
9      url : 'http://www.netsuite.com'
10 })
11 ...
12 //Add additional code

```

serverWidget.LayoutJustification

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported justification layouts. This enum is used to set the value of the <code>align</code> parameter when List.addColumn(options) is called.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value
CENTER
LEFT
RIGHT

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4   title : 'Simple List'
5 });
6 list.addColumn({
7   id : 'column1',
8   type : serverWidget.FieldType.TEXT,
9   label : 'Text',
10  align : serverWidget.LayoutJustification.RIGHT
11 });
12 ...
13 //Add additional code

```

serverWidget.ListStyle

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported list styles. This enum is used to set the value of the List.style property.
-------------------------	---

Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value
GRID
NORMAL
PLAIN
REPORT

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var list = serverWidget.createList({
4     title : 'Simple List'
5 });
6 list.style = serverWidget.ListStyle.REPORT;
7 ...
8 //Add additional code
```

serverWidget.SublistDisplayType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for supported sublist display types. This enum is used to set the value of the Sublist.displayType property.
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value
HIDDEN

Value
NODISPLAY
NORMAL

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'
10 });
11 sublist.displayType = serverWidget.SublistDisplayType.HIDDEN;
12 ...
13 //Add additional code

```

serverWidget.SublistType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for valid sublist types. This enum is used to define the type parameter when Form.addSublist(options) is called
Module	N/ui/serverWidget Module
Sibling Module Members	N/ui/serverWidget Module Members
Since	2015.2

Values

Value	Description
INLINEEDITOR	These types of sublists are both fully editable. The only difference between these types is their appearance in the UI: - With an inline editor sublist, a new line is displayed at the bottom of the list after existing lines. To add a line, a user working in the UI clicks inside the new line and adds a value to each column as appropriate. Examples of this style include the Item sublist on the sales order record and the Expense sublist on the expense report record. - With an editor sublist, a user in the UI adds a new line by working with fields that are displayed above the existing sublist lines. This style is not common on standard NetSuite record types.
EDITOR	

Value	Description
LIST	This type of sublist has a fixed number of lines. You can update an existing line, but you cannot add lines to it.  Note: To make a field within a LIST type sublist editable, use <code>Field.updateDisplayType(options)</code> and the enum <code>serverWidget.FieldDisplayType</code> to update the field display type to ENTRY.
STATICLIST	This type of sublist is read-only. It cannot be edited in the UI, and it is not available for scripting.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/ui/serverWidget Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form with Inline Editor type Sublist'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.INLINEEDITOR,
9     label : 'Inline Editor Sublist'
10 });
11 ...
12 //Add additional code

```

The following code sample shows how to make a field within a LIST type sublist editable by updating the `fieldDisplayType` to ENTRY.

```

1 //Add additional code
2 ...
3 var form = serverWidget.createForm({
4     title : 'Simple Form with List type Sublist'
5 });
6 var sublist = form.addSublist({
7     id : 'sublist',
8     type : serverWidget.SublistType.LIST,
9     label : 'List Type Sublist'
10 });
11 var internalId = sublist.addField({
12     id : 'id',
13     label : 'Internal ID',
14     type : serverWidget.FieldType.TEXT
15 });
16 internalId.updateDisplayType({displayType: serverWidget.FieldDisplayType.ENTRY});
17 ...
18 //Add additional code

```

N/url Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/url module to determine URL navigation paths within NetSuite and format URL strings.

[Go To Samples {..}](#)

In This Help Topic

- [N/url Module Members](#)

N/url Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	url.format(options)	string	Client and server scripts	Converts (serializes) URL query parameters into a string.
	url.resolveDomain(options)	string	Client and server scripts	Returns a domain name for a NetSuite account.
	url.resolveRecord(options)	string	Client and server scripts	Returns an internal URL to a NetSuite record.
	url.resolveScript(options)	string	Client and server scripts	Returns an external or internal URL to a script.
	url.resolveTaskLink(options)	string	Client and server scripts	Returns an internal URL for a tasklink.
Enum	url.HostType	enum	Client and server scripts	Holds the string values that describe a category of domain name. Use this enum to set the value of the hostType parameter of the url.resolveDomain(options) method.

N/url Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/url module:

- [Retrieve the Relative URL of a Record](#)
- [Generate an Absolute URL to a Specific Resource](#)
- [Retrieve the Domain for Calling a RESTlet](#)
- [Create a URL and Send a Secure HTTPS Post Request to the URL](#)

Retrieve the Relative URL of a Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to retrieve the relative URL of a record. With the internal ID value used in this sample, the returned output is /app/accounting/transactions/salesord.nl?id=6&e=T&compid=', followed by the NetSuite account ID.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: The value used in this sample for the `recordId` field is a placeholder. Before using this sample, replace the `recordId` field value with a valid value from your NetSuite account. If you run a script with an invalid value, an error may occur.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This script retrieves the relative URL of a record.
6   require(['N/url'], function(url) {
7       var output = url.resolveRecord({
8           recordType: 'salesorder',
9           recordId: 6,
10          isEditMode: true
11      });
12  });

```

Generate an Absolute URL to a Specific Resource

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to generate an absolute URL to a specific resource.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: The value used in this sample for the `recordId` field is a placeholder. Before using this sample, replace the `recordId` field value with a valid value from your NetSuite account. If you run a script with an invalid value, an error may occur.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This script generates an absolute URL to a specific resource.
6   require(['N/url', 'N/record'], function(url, record) {
7       function resolveRecordUrl() {
8           var scheme = 'https://';
9           var host = url.resolveDomain({
10              hostType: url.HostType.APPLICATION
11          });
12           var relativePath = url.resolveRecord({
13              recordType: record.Type.SALES_ORDER,
14              recordId: 6,
15              isEditMode: true
16          });
17           var myURL = scheme + host + relativePath;
18       }
19       resolveRecordUrl();
20  });

```

Retrieve the Domain for Calling a RESTlet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to retrieve the domain for calling a RESTlet.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: The value used in this sample for the `accountId` field is a placeholder. Before using this sample, replace the `accountId` field value with a valid value from your NetSuite account. If you run a script with an invalid value, an error may occur.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This script retrieves the domain for calling a RESTlet.
6   require(['N/url'], function(url) {
7       function resolveDomainUrl() {
8           var sCompId = 'MSTRWLF';
9           var output = url.resolveDomain({
10              hostType: url.HostType.RESTLET,
11              accountId: sCompId
12            });
13       }
14       resolveDomainUrl();
15   });

```

Create a URL and Send a Secure HTTPS Post Request to the URL

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to get the URL to a Suitelet and send a secure HTTPS POST request to that URL with an empty body. The server's response is also logged.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: The value used in this sample for the `scriptId` and `deploymentId` fields are placeholders. Before using this sample, replace the `scriptId` and `deploymentId` values with valid values from your NetSuite account. If you run a script with an invalid value, an error may occur.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5   // This script creates a URL, sends a secure HTTPS POST request to that URL, and logs the server's response.
6   require(['N/url', 'N/https'], function(url, https) {
7       var script = 'customscript1';
8       var deployment = 'customdeploy1';
9       var parameters = '';
10      try {
11          var suiteletURL = url.resolveScript({
12              scriptId: script,

```

```

13         deploymentId: deployment
14     });
15     var response = https.post({
16         url: suiteletURL,
17         body: parameters
18     });
19     log.debug(response.body.toString());
20 }
21 catch(e) {
22     log.error(e.toString());
23 }
24 });

```

url.format(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a serialized representation of an object containing query parameters. Use the returned value to build a URL query string.
Returns	URL as a string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/url Module
Since	2015.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.domain	string	required	The domain name.	2015.1
options.params	Object	required	Additional URL parameters as name/value pairs.	2015.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/url Module Script Samples](#).

For a script that uses the following code sample, the returned output is `http://fruitland.com?fruit=grape&seedless=true&variety=Concord+Giant&PLU=4272`, expressed as a string.

```

1 //Add additional code
2 ...
3 var output = url.format({
4     domain: 'http://fruitland.com',
5     params: {
6         fruit: 'grape',

```

```

7      seedless: true,
8      variety: 'Concord Giant',
9      PLU: 4272
10   }
11   });
12   ...
13   //Add additional code

```

url.resolveDomain(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a domain name for a NetSuite account.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/url Module
Since	2017.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.hostType	string	required	The type of domain name you want to retrieve. Set this value using the url.HostType enum.	2017.1
options.accountId	string	optional	The NetSuite account ID for which you want to retrieve data. If no account is specified, the system returns data on the account that is running the script. You can find the account ID at Setup > Company > Company Information in the Account ID field.	2017.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/url Module Script Samples](#).

```

1   //Add additional code
2   ...
3   var output = url.resolveDomain({
4     hostType: url.HostType.APPLICATION,
5     accountId: '012345'
6   });
7   ...
8   //Add additional code

```

url.resolveRecord(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the URL to a NetSuite record.  Important: This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/url Module
Since	2015.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.isEditMode	boolean	required	If set to true, returns a URL for the record in Edit mode. If set to false, returns a URL for the record in View mode The default value is View.	2015.1
options.recordId	string number	required	The record ID of the target record instance.	2015.1
options.recordType	string	required	The type of record. For example, 'purchaseorder'.	2015.1
options.params	Object	optional	Object used to add parameters for a custom URL. For example, a query to a database or to a search engine.	2015.1

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/url Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var output = url.resolveRecord({
4   recordType: 'salesorder',
5   recordId: 6,

```



```

6      isEditMode: true
7    });
8    ...
9    //Add additional code

```

url.resolveScript(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an external or internal URL to a RESTlet or Suitelet.  Important: This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/url Module
Since	2015.1

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.deploymentId	string number	required	The script ID (string) or internal ID (number) of the deployment script.	2015.1
options.scriptId	string number	required	The script ID (string) or internal ID (number) of the script. The ID must identify a RESTlet or a Suitelet.	2015.1
options.params	Object	optional	The object containing name/value pairs to describe the query.	2015.1
options.return ExternalUrl	boolean	optional	Indicates whether to return the external URL. Setting this value to true requires that the script is run in a trusted context for authenticated users. By default, the internal URL is returned (that is, the default value is false). You can only use the internal URL for this method in client scripts. To call a Suitelet with its internal URL from a server script, use https.requestSuitelet(options). Note the following URLformats: <pre> 1 Restlet internal: /app/site/hosting/restlet.nl?scrip t=123&deploy=123 </pre>	2015.1

Parameter	Type	Required / Optional	Description	Since
			<pre> 2 Suitelet internal: /app/site/hosting/scriptlet.nl?scrip t=123&deploy=123 3 4 Restlet external: [compid].restlets.api.netsuite.com/app/ site/hosting/restlet.nl?script=123&deploy=123 5 Suitelet external: [compid].extforms.netsuite.com/app/ site/hosting/scriptlet.nl?script=123&deploy=123&ns- at=[...anti-tampering...] </pre>	

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/url Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var output = url.resolveScript({
4     scriptId: 'custom_script',
5     deploymentId: 'custom_script_deployment',
6     returnExternalUrl: false
7 });
8 ...
9 //Add additional code

```

url.resolveTaskLink(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the internal URL to a NetSuite tasklink. Important: This method doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.
Returns	The URL as a string
Supported Script Types	Server and client scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/url Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The task ID for the tasklink.	2015.1

Parameter	Type	Required / Optional	Description	Since
			 Note: Each page in NetSuite has a unique task ID associated with it for a specific record type. For more information about finding the task ID, see the help topic Task IDs .	
options.params	Map	optional	The Map object containing name/value pairs to describe the query.	2015.1

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/url Module Script Samples](#).

The following code sample returns the mapping url to create a mass update for employees of a company. The optional parameters are used to provide more detail to the path of this url.

```

1 //Add additional code
2 ...
3 var output = url.resolveTaskLink({
4   id: 'EDIT_BULKOP',
5   params: {
6     searchtype: 'Employee',
7     opcode: 'GENERAL',
8     _entryformparams: null
9   }
10 });
11 ...
12 //Add additional code
13
```

The returned output expressed as a string is: /app/common/bulk/bulkop.nl?compid=TSTDRV2484080&searchtype=Employee&opcode=GENERAL&_entryformparams=.

The compid is relative to your NetSuite account.

url.HostType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values that describe a category of domain name. Use this enum to set the value of the hostType parameter of the url.resolveDomain(options) method.
Module	N/url Module
Sibling Module Members	N/url Module Members
Since	2017.1

Values

Value	Description	Sample Result
APPLICATION	The domain for UI access.	<accountID>.app.netsuite.com <accountID> is replaced with your NetSuite account number.
CUSTOMER_CENTER	The customer center.	<accountID>.app.netsuite.com <accountID> is replaced with your NetSuite account number.
FORM	The domain for forms hosted online, usually in Suitelets.	<accountID>.extforms.netsuite.com <accountID> is replaced with your NetSuite account number.
RESTLET	The domain for calling a RESTlet from an external source.	<accountID>.restlets.api.netsuite.com <accountID> is replaced with your NetSuite account number.
SUITETALK	The domain for SOAP web services requests.	<accountID>.suitetalk.api.netsuite.com <accountID> is replaced with your NetSuite account number.

For more information about NetSuite domains, see the help topic [Understanding NetSuite URLs](#).



Warning: The results returned, as shown in the sample results column, may change without notice. Because these values can change, your scripts must dynamically discover domain names. For more details, see the help topic [Dynamic Discovery of URLs](#).

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/url Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var output = url.resolveDomain({
4     hostType: url.HostType.APPLICATION,
5     accountId: '012345'
6 });
7 ...
8 //Add additional code

```

N/util Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/util module to manually access methods that verify object and primitive types in a SuiteScript 2.x script. These methods can also be accessed using the global util object. For more information about the global util object, see [SuiteScript 2.x Global Objects](#).

[Go To Samples {..}](#)

Use the N/util module to access methods that verify object and primitive types in a SuiteScript 2.x script.

Each method (for example, [util.isArray\(obj\)](#)) returns a boolean value, based on evaluation of the obj parameter.

If you need to identify a type specific to SuiteScript 2.x, use the [toString\(\)](#) global method.

In This Help Topic

■ [N/util Module Members](#)

N/util Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	util.each(iterable, callback)	Object or Array	Client and server scripts	Iterates over each member in an Object or Array.
	util.extend(receiver, contributor)	Object	Client and server scripts	Copies the properties in a source object to a destination object.
	util.isArray(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Array object and false otherwise.
	util.isAsyncFunction(obj)	boolean	Client and server scripts	Returns true if the obj parameter is JavaScript Async Function and false otherwise.
	util.isBoolean(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Boolean and false otherwise.
	util.isDate(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Date object and false otherwise.
	util.isFunction(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Function or Async Function and false otherwise.
	util.isNumber(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Number object or a value that evaluates to a Number object, and false otherwise.
	util.isObject(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a plain JavaScript object (new Object() or {} for example), and false otherwise.
	util.isRegExp(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript RegExp object or a

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				value that evaluates to a RegExp object, and false otherwise.
	<code>util.isString(obj)</code>	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript String object or a value that evaluates to a String object, and false otherwise.

N/util Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/util module.

Set Fields on a Sales Order Record Using the util.each Iterator

The following sample creates a sales order record. It uses the `util.each(iterable, callback)` method to set record fields based on the values in an iterable object. Be sure to replace hard-coded values (such as record IDs) with valid values from your NetSuite account.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/record'], function(record){
6      // Create a sales order
7      var rec = record.create({
8          type: 'salesorder',
9          isDynamic: true
10     });
11     rec.setValue({
12         fieldId: 'entity',
13         value: 107
14     });
15
16     // Set up an object containing an item's internal ID and the corresponding quantity
17     var itemList = {
18         39: 5,
19         38: 1
20     }
21
22     // Iterate through the object and set the key-value pairs on the record
23     util.each(itemList, function(quantity, itemId){ // (5, 39) and (1, 38)
24         rec.selectNewLine('item');
25         rec.setCurrentSublistValue('item', 'item', itemId);
26         rec.setCurrentSublistValue('item', 'quantity', quantity);
27         rec.commitLine('item');
28     });
29
30     var id = rec.save();
31 });

```

util.each(iterable, callback)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Iterates over each member in an Object or Array. This method calls the callback function on each member of the iterable.
Returns	The original collection as an Object Array
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
iterable	Object Array	required	The data collection to iterate on.	2016.1
callback	Function	required	Takes the custom logic that you want to execute on each member of your collection of data.	2016.1

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 // Iterate through the object and set the key-value pairs on the record
4 util.each(itemList, function(quantity, itemId){
5     rec.selectNewLine('item');
6     rec.setCurrentSublistValue('item','item',itemId);
7     rec.setCurrentSublistValue('item','quantity',quantity);
8     rec.commitLine('item');
9 });
10 ...
11 //Add additional code

```

util.extend(receiver, contributor)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Copies the properties in a source object to a destination object. You can use this method to merge two objects.
---------------------------	--

Returns	The destination object.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Syntax



Important: The following code snippets shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

This snippet shows combining two objects without the same keys:

```

1 //Add additional code
2 ...
3 var colors = {};
4 var firstSet = {'color1':'red',
5               'color2':'yellow',
6               'color3':'blue'};
7 var secondSet = {'color4':'green',
8                 'color5':'orange',
9                 'color6':'violet'
10                };
11
12 // Extends colors object with the information in firstSet
13 // Colors will get {'color1':'red','color2':'yellow','color3':'blue'}
14 util.extend(colors, firstSet);
15
16 // Extends colors object with the information in secondSet
17 // Colors will get {'color1':'red','color2':'yellow','color3':'blue','color4':'green','color5':'orange','color6':'violet'}
18 util.extend(colors, secondSet);
19 });
20 ...
21 //Add additional code

```

The following snippet shows overriding two objects with a few similar keys:

```

1 //Add additional code
2 ...
3 var colors = {};
4 var firstSet = {'color1':'red',
5               'color2':'yellow',
6               'color3':'blue'
7              };
8 var secondSet = {'color2':'green',
9                 'color3':'orange',
10                 'color4':'violet'
11                };
12
13 // Extends colors object with the information in firstSet
14 // Colors will get {'color1':'red','color2':'yellow','color3':'blue'}
15 util.extend(colors, firstSet);
16
17 // Extends colors object with the information in secondSet and overrides the value if there are similar keys
18 // Colors will get {'color1':'red','color2':'green','color3':'orange','color4':'violet'}
19 util.extend(colors, secondSet);
20
21 var x = 0;
22 });

```



```

23 | ...
24 | //Add additional code

```

util.isArray(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript Array object and false otherwise.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Global object	util Object
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 | //Add additional code
2 | ...
3 | var records = ["Sales Order", "Invoice", "Item Fulfillment"];
4 | util.isArray(records); // returns true
5 |
6 | var record = "Sales Order";
7 | util.isArray(record); // returns false
8 | ...
9 | //Add additional code

```

util.isAsyncFunction(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript AsyncFunction and false otherwise.
---------------------------	--

Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2020.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2020.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

Also note that the `async` keyword is used in this sample, which is only supported in SuiteScript 2.1.

```

1 //Add additional code
2 ...
3 function func1(){
4     var x = 1;
5 }
6 var func2 = function(){};
7
8 async function async1(){
9     var x = 2;
10 }
11
12 var async2 = async function(){};
13
14 var a = util.isAsyncFunction(func1); // function returns false (a = false)
15 var b = util.isAsyncFunction(func2); // function returns false (b = false)
16 var c = util.isAsyncFunction(async1); // function returns true (c = true)
17 var d = util.isAsyncFunction(async2) // function returns true (d = true)
18 ...
19 //Add additional code

```

util.isBoolean(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the <code>obj</code> parameter is a JavaScript Boolean and false otherwise.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var flag = true;
4 util.isBoolean(flag); // returns true
5 util.isBoolean(true); // returns true
6
7 util.isBoolean(1);    // returns false
8 ...
9 //Add additional code

```

util.isDate(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript Date object and false otherwise.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var todaysDate = new Date();
4 util.isDate(todaysDate); // returns true
5 util.isDate(new Date()); // returns true
6
7 var today = "September 28, 2015";
8 util.isDate(today); // returns false
9 ...
10 //Add additional code

```

util.isFunction(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript Function or AsyncFunction and false otherwise.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 function test() {};
4 var test2 = function() {};
5
6 util.isFunction(test); // returns true
7 util.isFunction(test2); // returns true
8 ...
9 //Add additional code

```

util.isNumber(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript Number object or primitive, and false otherwise.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 util.isNumber(112);           // returns true
4 util.isNumber("112");        // returns false
5 util.isNumber(NaN);          // returns true
6
7 var testNum = 112;
8 util.isNumber(testNum.valueOf()); // returns true
9 ...
10 //Add additional code

```

util.isObject(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a plain JavaScript object (new Object() or {} for example), and false otherwise. Use this method, for example, to verify that a variable is a JavaScript object and not a JavaScript Function.
Returns	boolean

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 util.isObject({});           // returns true
4 util.isObject(function() {}); // returns false
5 ...
6 //Add additional code

```

util.isRegExp(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript RegExp object, and false otherwise.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 util.isRegExp(/this is a regexp/); // returns true
4 util.isRegExp(new RegExp('this is another regexp')); // returns true
5 ...
6 //Add additional code

```

util.isString(obj)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the obj parameter is a JavaScript String object or primitive, and false otherwise
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/util Module
Since	2016.1

Parameters

Parameter	Type	Required / Optional	Description	Since
obj	Object Primitive	required	Object for which you want to verify the type.	2016.1

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a full script sample, see [N/util Module Script Sample](#).

```

1 //Add additional code
2 ...
3 util.isString(''); // returns true
4 util.isString('a string'); // returns true
5 ...
6 var myString = new String('another string');
7 util.isString(myString); // returns true
8 ...
9 util.isString(null); // returns false
10 ...
11 //Add additional code

```

N/workbook Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/workbook module to create new workbooks, load existing ones, or list all available workbooks.

A workbook can contain:

- Pivots
- Tables with columns, filters, and field contexts
- Charts with axes and legends
- Selectors
- Sections
- Data dimensions
- Sorts
- Conditional and limiting filters
- Expressions
- Data measures and calculated measures

Workbooks help you analyze your dataset query results using different components, like table views. Every workbook is based on a dataset. For more, see the [N/dataset Module](#). To learn more about SuiteAnalytics workbooks and datasets, check out:

- [SuiteAnalytics Workbook Overview](#)
- [Getting Started with SuiteAnalytics Workbook](#)
- [Custom Workbooks and Datasets](#)



Important: The N/workbook module doesn't work in unauthenticated client-side contexts. For details, see the SuiteAnswers [Outbound HTTPS in an unauthenticated client-side context](#).

Go To Samples 

In This Help Topic

- [N/workbook Module Members](#)
- [Aspect Object Members](#)
- [CalculatedMeasure Object Members](#)
- [Category Object Members](#)
- [ChartAxis Object Members](#)
- [Chart Object Members](#)
- [workbook.ChildNodesSelector](#)
- [Color Object Members](#)
- [ConditionalFilter Object Members](#)
- [ConditionalFormat Object Members](#)
- [ConditionalFormatRule Object Members](#)
- [Currency Object Members](#)

- [DataDimension Object Members](#)
- [DataDimensionItem Object Members](#)
- [DataDimensionItemValue Object Members](#)
- [DataDimensionValue Object Members](#)
- [DataMeasure Object Members](#)
- [workbook.DescendantorSelfNodesSelector](#)
- [DimensionSelector Object Members](#)
- [Duration Object Members](#)
- [Expression Object Members](#)
- [FieldContext Object Members](#)
- [FontSize Object Members](#)
- [Legend Object Members](#)
- [LimitingFilter Object Members](#)
- [MeasureSelector Object Members](#)
- [MeasureValue Object Members](#)
- [MeasureValueSelector Object Members](#)
- [PathSelector Object Members](#)
- [PivotAxis Object Members](#)
- [Pivot Object Members](#)
- [PivotIntersection Object Members](#)
- [PositionPercent Object Members](#)
- [PositionUnits Object Members](#)
- [PositionValues Object Members](#)
- [Range Object Members](#)
- [Record Object Members](#)
- [RecordKey Object Members](#)
- [ReportStyle Object Members](#)
- [ReportStyleRule Object Members](#)
- [Section Object Members](#)
- [SectionValue Object Members](#)
- [Series Object Members](#)
- [Sort Object Members](#)
- [SortByDataDimensionItem Object Members](#)
- [SortByMeasure Object Members](#)
- [SortDefinition Object Members](#)
- [Style Object Members](#)
- [Table Object Members](#)
- [TableColumn Object Members](#)
- [TableColumnCondition Object Members](#)
- [TableColumnFilter Object Members](#)
- [Workbook Object Members](#)

N/workbook Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	workbook.Aspect	Object	Server scripts	An aspect.
	workbook.Calculated Measure	Object	Server scripts	A calculated measure.
	workbook.Category	Object	Server scripts	A chart category.
	workbook.Chart	Object	Server scripts	A chart.
	workbook.ChartAxis	Object	Server scripts	A chart axis object which is used when you create a category or a legend.
	workbook.ChildNodes Selector	Object	Server scripts	A selector for child nodes.
	workbook.Color	Object	Server scripts	A color.
	workbook.ConditionalFilter	Object	Server scripts	A conditional filter.
	workbook.Conditional Format	Object	Server scripts	A conditional format.
	workbook.Conditional FormatRule	Object	Server scripts	A conditional format rule.
	workbook.Currency	Object	Server scripts	A currency amount and currency type.
	workbook.DataDimension	Object	Server scripts	A data dimension.
	workbook.Data DimensionItem	Object	Server scripts	A data dimension item.
	workbook.Data DimensionItemValue	Object	Server scripts	The value of a data dimension item.
	workbook.Data DimensionValue	Object	Server scripts	The value of a data dimension.
	workbook.DataMeasure	Object	Server scripts	A data measure.
	workbook.Descendantor SelfNodesSelector	Object	Server scripts	A selector for descendant or self nodes.
	workbook.Dimension Selector	Object	Server scripts	A dimension selector.
	workbook.Duration	Object	Server scripts	A duration.
	workbook.Expression	Object	Server scripts	An expression.
	workbook.FieldContext	Object	Server scripts	A field context.
	workbook.FontSize	Object	Server scripts	A font size.
	workbook.Legend	Object	Server scripts	A chart legend.
	workbook.LimitingFilter	Object	Server scripts	A limiting filter.
	workbook.MeasureSelector	Object	Server scripts	A measure selector.
	workbook.MeasureValue	Object	Server scripts	A measure value.
	workbook.MeasureValue Selector	Object	Server scripts	A measure value selector.
	workbook.PathSelector	Object	Server scripts	A path selector.
	workbook.Pivot	Object	Server scripts	A pivot definition. A pivot is a workbook component that enables you to pivot your

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				dataset query results by defining measures and dimensions, so that you can analyze different subsets of data.
	workbook.PivotAxis	Object	Server scripts	A pivot axis.
	workbook.PivotIntersection	Object	Server scripts	A pivot intersection.
	workbook.PositionPercent	Object	Server scripts	A position defined by percentages of the x and y axes.
	workbook.PositionUnits	Object	Server scripts	A position defined by units.
	workbook.PositionValues	Object	Server scripts	A position defined by horizontal and vertical position values.
	workbook.Range	Object	Server scripts	A date or date-time range.
	workbook.Record	Object	Server scripts	A record.
	workbook.RecordKey	Object	Server scripts	A record key.
	workbook.ReportStyle	Object	Server scripts	A report style.
	workbook.ReportStyleRule	Object	Server scripts	A report style rule.
	workbook.Section	Object	Server scripts	A workbook section.
	workbook.SectionValue	Object	Server scripts	A section value.
	workbook.Series	Object	Server scripts	A series in a workbook. A series is used when you create a chart definition.
	workbook.Sort	Object	Server scripts	A sort.
	workbook.SortByDataDimensionItem	Object	Server scripts	A sort based on a data dimension item.
	workbook.SortByMeasure	Object	Server scripts	A sort based on a measure.
	workbook.SortDefinition	Object	Server scripts	A sort definition.
	workbook.Style	Object	Server scripts	A style.
	workbook.Table	Object	Server scripts	A table.
	workbook.TableColumn	Object	Server scripts	A table column.
	workbook.TableColumnCondition	Object	Server scripts	Condition for a table view column.
	workbook.TableColumnFilter	Object	Server scripts	A table column filter.
	workbook.Workbook	Object	Server scripts	A workbook. Workbooks are where you analyze the results of your dataset queries using different components, such as table views and pivots. All workbooks are based on a dataset, and a single dataset can be used as the basis for multiple workbooks.
Method	workbook.create(options)	workbook.Workbook	Server scripts	Creates a workbook. Workbooks are where you analyze the results of your dataset queries using different components, such as table views and pivots. All workbooks are based on a dataset, and a single dataset can be used as the basis for multiple workbooks.
	workbook.createAspect(options)	workbook.Aspect	server scripts	Creates an aspect for a chart series. An aspect includes a measure and an aspect type.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	workbook.createCalculatedMeasure(options)	workbook.CalculatedMeasure	Server scripts	Creates a calculated measure.
	workbook.createCategory(options)	workbook.Category	Server scripts	Creates a chart category, which includes an axis, a data root, and a sort definition. A chart category is used in a workbook.Chart .
	workbook.createChart(options)	workbook.Chart	Server scripts	Creates a chart.
	workbook.createChartAxis(options)	workbook.ChartAxis	Server scripts	Creates an X-axis or a Y-axis for the chart.
	workbook.createColor(options)	workbook.Color	Server scripts	Creates a color.
	workbook.createComplexRecordKey	workbook.RecordKey	Server scripts	Creates a complex RecordKey object from another object.
	workbook.createConditionalFilter(options)	workbook.ConditionalFilter	Server scripts	Creates a conditional filter.
	workbook.createConditionalFormat(options)	workbook.ConditionalFormat	Server scripts	Creates a conditional format.
	workbook.createConditionalFormatRule(options)	workbook.ConditionalFormatRule	Server scripts	Creates a conditional format rule.
	workbook.createConstant(options)	workbook.Expression	Server scripts	Creates a constant expression.
	workbook.createCurrency(options)	workbook.Currency	Server scripts	Creates a currency.
	workbook.createDataDimension(options)	workbook.DataDimension	Server scripts	Creates a data dimension.
	workbook.createDataDimensionItem(options)	workbook.DataDimensionItem	Server scripts	Creates a data dimension item.
	workbook.createDataMeasure(options)	workbook.DataMeasure	Server scripts	Creates a data measure.
	workbook.createDimensionSelector(options)	workbook.DimensionSelector	Server scripts	Creates a dimension selector.
	workbook.createDuration(options)	workbook.Duration	Server scripts	Creates a duration.
	workbook.createExpression(options)	workbook.Expression	Server scripts	Creates an expression.
	workbook.createFieldContext(options)	workbook.FieldContext	Server scripts	Creates a field context for table column.
	workbook.createFontSize(options)	workbook.FontSize	Server scripts	Creates a font size.
	workbook.createLegend(options)	workbook.Legend	Server scripts	Creates a chart legend.
	workbook.createLimitingFilter(options)	workbook.LimitingFilter	Server scripts	Creates a limiting filter.
	workbook.createMeasureSelector(options)	workbook.MeasureSelector	Server scripts	Creates a measure selector.
	workbook.createMeasureValueSelector(options)	workbook.MeasureValueSelector	Server scripts	Creates a measure value selector.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	workbook.createPathSelector(options)	workbook.PathSelector	Server scripts	Creates a path selector.
	workbook.createPivot(options)	workbook.Pivot	Server scripts	Creates a pivot definition. A pivot is a workbook component that enables you to pivot your dataset query results by defining measures and dimensions, so that you can analyze different subsets of data.
	workbook.createPivotAxis(options)	workbook.PivotAxis	Server scripts	Creates a pivot axis, which includes a data root and a sort definition.
	workbook.createPositionPercent(options)	workbook.PositionPercent	Server scripts	Creates a position defined by percentages of the x and y axes.
	workbook.createPositionUnits(options)	workbook.PositionUnits	Server scripts	Creates a position defined by units.
	workbook.createPositionValues(options)	workbook.PositionValues	Server scripts	Creates a position defined by horizontal and vertical position values.
	workbook.createRange(options)	workbook.Range	Server scripts	Creates a range.
	workbook.createReportStyle(options)	workbook.ReportStyle	Server scripts	Creates a report style.
	workbook.createReportStyleRule(options)	workbook.ReportStyleRule	Server scripts	Creates a report style rule.
	workbook.createSection(options)	workbook.Section	Server scripts	Creates a section.
	workbook.createSeries(options)	workbook.Series	Server scripts	Creates a chart series, which is a set of aspects.
	workbook.createSimpleRecordKey	workbook.RecordKey	Server scripts	Creates a record key.
	workbook.createSort(options)	workbook.Sort	Server scripts	Creates a sort.
	workbook.createSortByDataDimensionItem(options)	workbook.SortByDataDimensionItem	Server scripts	Creates a sort based on a data dimension item.
	workbook.createSortByMeasure(options)	workbook.SortByMeasure	Server scripts	Creates a sort based on a measure.
	workbook.createSortDefinition(options)	workbook.SortDefinition	Server scripts	Creates a sort definition.
	workbook.createStyle(options)	workbook.Style	Server scripts	Creates a style.
	workbook.createTable(options)	workbook.Table	Server scripts	Creates a table view.
	workbook.createTableColumn(options)	workbook.TableColumn	Server scripts	Creates a table column.
	workbook.createTableColumnCondition(options)	workbook.TableColumnCondition	Server scripts	Creates a table column condition.
	workbook.createTableColumnFilter(options)	workbook.TableColumnFilter	Server scripts	Creates a table filter.
	workbook.createTranslation(options)	workbook.Expression	Server scripts	Creates a translation (a translation expression).
	workbook.list()	Object[]	Server scripts	Lists all existing workbooks.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	workbook.listPaged(options)	PagedInfoData	Server scripts	Retrieves a set of pages with metadata about workbooks.
	workbook.loadWorkbook(options)	workbook.Workbook	Server scripts	Loads an existing workbook.
Enum	workbook.Aggregation	enum	Server scripts	Holds string values for aggregation types. Used to set the value of the <code>options.aggregation</code> parameter of the workbook.createDataMeasure(options) method.
	workbook.AspectType	enum	Server scripts	Holds string values for aspect types. Used to set the <code>options.type</code> parameter of the workbook.createAspect(options) method.
	workbook.ChartType	enum	Server scripts	Holds string values for chart types. Used to pass the type value to workbook.createChart(options)
	workbook.Color	enum	Server scripts	Holds string values for colors. Used to set the <code>options.backgroundColor</code> , <code>options.color</code> , and <code>options.textDecorationColor</code> parameters of the workbook.createStyle(options) method.
	workbook.ConstantType	enum	Server scripts	Holds string values for constant types. Used to set the value of the <code>options.type</code> parameter of the workbook.createConstant(options) method.
	workbook.DateTimeHierarchy	enum	Server scripts	Holds string values for workbook date-time hierarchy types.
	workbook.DateTimeProperty	enum	Server scripts	Holds string values for workbook date-time property types. Used to set the value of the <code>DATE_TIME_PROPERTY</code> in workbook.ExpressionType .
	workbook.ExpressionType	enum	Server scripts	Holds string values for workbook expression types. Use these values for the <code>options.functionId</code> parameter when creating an expression using workbook.createExpression(options)
	workbook.FontSize	enum	Server scripts	Holds string values for font sizes. Used to set the value for the <code>options.fontSize</code> parameter of the workbook.createStyle(options) method.
	workbook.FontStyle	enum	Server scripts	Holds string values for font sizes. Used to set the value for the <code>options.fontSize</code> parameter of the workbook.createStyle(options) method.
	workbook.FontWeight	enum	Server scripts	Holds string values for font weights. Used to set the value for the <code>options.fontWeight</code> parameter of the workbook.createStyle(options) method.
	workbook.Image	enum	Server scripts	Holds string values for images that you can use in workbooks. Used as a value for the Style.backgroundImage property.
	workbook.Position	enum	Server scripts	Holds string values for positions. Used to set the value for the PositionValues.horizontal and PositionValues.vertical properties.
	workbook.Stacking	enum	Server scripts	Holds stacking types. Used to pass the stacking value to workbook.createChart(options) .
	workbook.TemporalUnit	enum	Server scripts	Holds string values for temporal units, such as hours or minutes. Used to set the value of the <code>options.start</code> and <code>options.end</code> parameters of the workbook.createDuration(options) method.
	workbook.TextAlign	enum	Server scripts	Holds string values for text alignments. Used to set the value for the <code>options.textAlign</code> parameter of the workbook.createStyle(options) method.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	workbook.TextDecorationLine	enum	Server scripts	Holds string values for text decoration line types, such as underline and strikethrough. Used to set the value for the options.textDecorationLine parameter of the workbook.createStyle(options) method.
	workbook.TextDecorationStyle	enum	Server scripts	Holds string values for text decoration line styles, such as solid and dashed. Used to set the value for the options.textDecoractionStyle parameter of the workbook.createStyle(options) method.
	workbook.TotalLine	enum	Server scripts	Holds string values for predefined total line formats. Used to pass the totalLine value to workbook.createDataDimension(options) and to workbook.createSection(options) .
	workbook.Unit	enum	Server scripts	Holds string values for units of measurement. Used to set the options.unit parameter in the workbook.createPositionUnits(options) method.

Aspect Object Members

The following members are available for a [workbook.Aspect](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Aspect.measure	workbook.CalculatedMeasure workbook.DataMeasure	Server scripts	The measure of the aspect.
	Aspect.type	string	Server scripts	The type of the aspect. Set this value using workbook.AspectType .

CalculatedMeasure Object Members

The following members are available for a [workbook.CalculatedMeasure](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	CalculatedMeasure.expression	workbook.Expression	Server scripts	The expression for the calculated measure.
	CalculatedMeasure.label	string workbook.Expression	Server scripts	The label of the calculated measure.

Category Object Members

The following members are available for a object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Category.axis	workbook.ChartAxis	Server scripts	The axis of the category.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Category.root	workbook.DataDimension workbook.Section	Server scripts	The section or data dimension (that is, the fields for the x-axis).
	Category.sortDefinitions	workbook.SortDefinition []	Server scripts	The sort definitions of the category.

ChartAxis Object Members

The following members are available for a [workbook.ChartAxis](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ChartAxis.title	string	Server scripts	The title of the chart axis.

Chart Object Members

The following members are available for a [workbook.Chart](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Chart.aggregationFilters	Array< workbook.LimitingFilter workbook.ConditionalFilter >	Server scripts	Limiting and conditional filters for the chart.
	Chart.category	string	Server scripts	The category of the chart.
	Chart.dataset	dataset.Dataset	Server scripts	The underlying dataset for the chart.
	Chart.filterExpressions	workbook.Expression []	Server scripts	The filter expressions of the chart.
	Chart.id	string	Server scripts	The ID of the chart.
	Chart.legend	workbook.Legend	Server scripts	The legend of the chart.
	Chart.name	string	Server scripts	The name of chart.
	Chart.series	workbook.Series	Server scripts	The series of the chart.
	Chart.stacking	string	Server scripts	The stacking type for the chart.
	Chart.subTitle	string	Server scripts	The subtitle of the chart.
	Chart.title	string	Server scripts	The title of chart.
	Chart.type	workbook.ChartType	Server scripts	The type of the chart.

Color Object Members

The following members are available for a [workbook.Color](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Color.alpha	number	Server scripts	The opacity, or transparency, of the color.
	Color.blue	number	Server scripts	The blue portion of the color.
	Color.green	number	Server scripts	The green portion of the color.
	Color.red	number	Server scripts	The red portion of the color.

ConditionalFilter Object Members

The following members are available for a [workbook.ConditionalFilter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ConditionalFilter.columnSelector	workbook.DescendantorSelfNodesSelector workbook.PathSelector workbook.DimensionSelector workbook.ChildNodesSelector	Server scripts	The column selector.
	ConditionalFilter.filteredNodesSelector	workbook.PathSelector workbook.DimensionSelector	Server scripts	The selected filters.
	ConditionalFilter.measure	workbook.CalculatedMeasure workbook.DataMeasure	Server scripts	The measure of the filter.
	ConditionalFilter.otherAxisSelector	workbook.PathSelector workbook.DimensionSelector	Server scripts	The filter selector for the other axis.
	ConditionalFilter.predicate	workbook.Expression	Server scripts	The actual predicate which indicates if the condition is met.
	ConditionalFilter.row	boolean	Server scripts	The row axis indicator.
	ConditionalFilter.rowSelector	workbook.DescendantorSelfNodesSelector workbook.PathSelector workbook.DimensionSelector workbook.ChildNodesSelector	Server scripts	The row selector.

ConditionalFormat Object Members

The following members are available for a [workbook.ConditionalFormat](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ConditionalFormat.rules	workbook.ConditionalFormatRule []	Server scripts	The conditional formatting rules that are included in the conditional format.

ConditionalFormatRule Object Members

The following members are available for a [workbook.ConditionalFormatRule](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ConditionalFormatRule.filter	workbook.TableColumnFilter	Server scripts	The filter that determines which rows or cells to apply the conditional format to.
	ConditionalFormatRule.style	workbook.Style	Server scripts	The style to apply as the conditional format.

Currency Object Members

The following members are available for a [workbook.Currency](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Currency.amount	number	Server scripts	The amount of the currency.
	Currency.id	string	Server scripts	The ID of the currency (for example, USD, EUR, GBP, and so on).

DataDimension Object Members

The following members are available for a [workbook.DataDimension](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DataDimension.children	Array< workbook.DataDimension workbook.Section >	Server scripts	The children of the data dimension.
	DataDimension.items	workbook.DataDimensionItem []	Server scripts	The items of the data dimension.
	DataDimension.totalLine	string	Server scripts	The formatting option for the total line. Set this value using workbook.TotalLine .

DataDimensionItem Object Members

The following members are available for a [workbook.DataDimensionItem](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DataDimensionItem.expression	workbook.Expression	Server scripts	The expression of data dimension item.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	DataDimensionItem.label	string	Server scripts	The label of the data dimension item.

DataDimensionItemValue Object Members

The following members are available for a [workbook.DataDimensionItemValue](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DataDimensionItemValue.item	workbook.DataDimensionItem	Server scripts	The data dimension item.
	DataDimensionItemValue.value	string number boolean workbook.Record workbook.Currency workbook.Range workbook.Duration	Server scripts	The value of the data dimension item.

DataDimensionValue Object Members

The following members are available for a [workbook.DataDimensionValue](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DataDimensionValue.dataDimension	workbook.DataDimension	Server scripts	The data dimension.
	DataDimensionValue.itemValues	workbook.DataDimensionItemValue []	Server scripts	The item values for the data dimension.

DataMeasure Object Members

The following members are available for a [workbook.DataMeasure](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DataMeasure.aggregation	string	Server scripts	The aggregation of the data measure.
	DataMeasure.expression	workbook.Expression	Server scripts	The expression for the data measure. This property is used if the data measure is a single-expression measure.
	DataMeasure.expressions	workbook.Expression []	Server scripts	The expressions for the data measure. This property is used if the data measure is a multiple-expression measure.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	DataMeasure.label	string workbook.Expression	Server scripts	The label of the data measure.

DimensionSelector Object Members

The following members are available for a [workbook.DimensionSelector](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	DimensionSelector.dimension	workbook.DataDimension workbook.Section	Server scripts	The dimension of the dimension selector.

Duration Object Members

The following members are available for a [workbook.Duration](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Duration.amount	number	Server scripts	The amount of the duration.
	Duration.units	Object	Server scripts	The units of the duration.

Expression Object Members

The following members are available for a [workbook.Expression](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Expression.functionId	string	Server scripts	The ID of the function used in the expression.
	Expression.parameters	Object	Server scripts	The parameters of the expression.

FieldContext Object Members

The following members are available for a [workbook.FieldContext](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	FieldContext.name	string	Server scripts	The name of the field context (for example, DISPLAY or CONSOLIDATED)
	FieldContext.parameters	Object	Server scripts	The parameters of the field context.

FontSize Object Members

The following members are available for a [workbook.FontSize](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	FontSize.size	number	Server scripts	The numeric size of the font size.
	FontSize.unit	string	Server scripts	The unit of the font size.

Legend Object Members

The following members are available for a object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Legend.axes	workbook.ChartAxis[]	Server scripts	The axes of the legend.
	Legend.root	workbook.DataDimension workbook.Section	Server scripts	The section or data dimension (that is., the fields for the y-axis).
	Legend.sortDefinitions	workbook.SortDefinition[]	Server scripts	The sort definitions of the legend.

LimitingFilter Object Members

The following members are available for a [workbook.LimitingFilter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	LimitingFilter.filteredNodesSelector	workbook.PathSelector workbook.DimensionSelector	Server scripts	The selected filter.
	LimitingFilter.limit	number	Server scripts	The limit number for the filter.
	LimitingFilter.row	boolean	Server scripts	The row axis indicator for the filter.
	LimitingFilter.sortBys	Array<workbook.SortByDataDimensionItem workbook.SortByMeasure>	Server scripts	The ordering elements of the filter.

MeasureSelector Object Members

The following members are available for a [workbook.MeasureSelector](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	MeasureSelector.measures	workbook.CalculatedMeasure[] workbook.DataMeasure	Server scripts	The measures for the measure selector.

MeasureValue Object Members

The following members are available for a [workbook.MeasureValue](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	MeasureValue.measure	workbook.MeasureValue	Server scripts	The measure to use for the measure value.
	MeasureValue.value	string number boolean workbook.Record workbook.Currency workbook.Range workbook.Duration	Server scripts	The value to use for the measure value.

MeasureValueSelector Object Members

The following members are available for a [workbook.MeasureValueSelector](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	MeasureValueSelector.columnSelector	workbook.DimensionSelector workbook.PathSelector workbook.DescendantorSelfNodesSelector	Server scripts	The column selector.
	MeasureValueSelector.measureSelector	workbook.MeasureSelector []	Server scripts	The measure selectors.
	MeasureValueSelector.rowSelector	workbook.DimensionSelector workbook.PathSelector workbook.DescendantorSelfNodesSelector	Server scripts	The row selector.

PathSelector Object Members

The following members are available for a [workbook.PathSelector](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PathSelector.elements	workbook.DimensionSelector	Server scripts	The elements denoting 'xpath' of the selector.

PivotAxis Object Members

The following members are available for a [workbook.PivotAxis](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PivotAxis.root	workbook.DataDimension workbook.Section	Server scripts	The data for the pivot axis.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	PivotAxis.sortDefinitions	workbook.SortDefinition []	Server scripts	The sort definitions of the pivot axis.

Pivot Object Members

The following members are available for a [workbook.Pivot](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Pivot.aggregationFilters	Array < workbook.ConditionalFilter workbook.LimitingFilter >	Server scripts	The limiting and conditional filters of the pivot definition.
	Pivot.columnAxis	workbook.PivotAxis	Server scripts	The column axis of the pivot definition.
	Pivot.dataset	dataset.Dataset	Server scripts	The underlying dataset of the pivot definition.
	Pivot.datasetLink	datasetLink.DatasetLink	Server scripts	Underlying dataset for the pivot.
	Pivot.filterExpressions	workbook.Expression	Server scripts	The filter expressions of the pivot definition.
	Pivot.id	string	Server scripts	The ID of the pivot definition.
	Pivot.name	string	Server scripts	The name of the pivot definition.
	Pivot.portletName	string workbook.Expression	Server scripts	The name of the portlet for the pivot.
	Pivot.reportStyles	workbook.ReportStyle []	Server scripts	Report styles for the pivot.
	Pivot.rowAxis	workbook.PivotAxis	Server scripts	The row axis of the pivot definition.

PivotIntersection Object Members

The following members are available for a [workbook.PivotIntersection](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PivotIntersection.column	workbook.DataDimensionValue workbook.SectionValue	Server scripts	The column dimension value.
	PivotIntersection.measureValues	workbook.MeasureValue []	Server scripts	The measure values in the pivot intersection.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	PivotIntersection.row	workbook.DataDimensionValue workbook.SectionValue	Server scripts	The row dimension value.

PositionPercent Object Members

The following members are available for a [workbook.PositionPercent](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PositionPercent.percentX	number	Server scripts	The percentage of the x dimension.
	PositionPercent.percentY	number	Server scripts	The percentage of the y dimension.

PositionUnits Object Members

The following members are available for a [workbook.PositionUnits](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PositionUnits.unit	string	Server scripts	The units for the position.
	PositionUnits.x	number	Server scripts	The x value of the position.
	PositionUnits.y	number	Server scripts	The y value of the position.

PositionValues Object Members

The following members are available for a [workbook.PositionValues](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	PositionValues.horizontal	string	Server scripts	The horizontal value of the position.
	PositionValues.vertical	string	Server scripts	The vertical value of the position.

Range Object Members

The following members are available for a [workbook.Range](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Range.end	string	Server scripts	The end date or date-time of the range.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Range.start	string	Server scripts	The start date or date-time of the range.

Record Object Members

The following members are available for a [workbook.Record](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Record.name	string	Server scripts	The name of the record type for the record.
	Record.primaryKey	number	Server scripts	The primary key of the record.
	Record.properties	Object	Server scripts	The properties of the record.

RecordKey Object Members

The following members are available for a [workbook.RecordKey](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	RecordKey.properties	Object	Server scripts	The properties of the record key.

ReportStyle Object Members

The following members are available for a [workbook.ReportStyle](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ReportStyle.rules	workbook.ReportStyleRule []	Server scripts	The formatting rules for the report style.
	ReportStyle.selectors	workbook.MeasureValueSelector []	Server scripts	The selectors for the report style.

ReportStyleRule Object Members

The following members are available for a [workbook.ReportStyleRule](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	ReportStyleRule.expression	workbook.Expression	Server scripts	A boolean expression indicating whether the style should be applied.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	ReportStyleRule.style	workbook.Style	Server scripts	The style to be applied.

Section Object Members

The following members are available for a [workbook.Section](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Section.children	Array< workbook.CalculatedMeasure workbook.DataMeasure workbook.DataDimension workbook.DataDimensionItem >	Server scripts	The children of the section.
	Section.totalLine	string	Server scripts	The formatting option for the total line. Set this value using workbook.TotalLine .

SectionValue Object Members

The following members are available for a [workbook.SectionValue](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SectionValue.section	workbook.Section	Server scripts	The section of the section value.

Series Object Members

The following members are available for a [workbook.Series](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Series.aspects	string	Server scripts	The aspects for the series.

Sort Object Members

The following members are available for a [workbook.Sort](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Sort.ascending	boolean	Server scripts	When set to true, indicates the sort is in ascending order.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Sort.caseSensitive	boolean	Server scripts	When set to true, indicates the sort is case sensitive.
	Sort.locale	query.SortLocale (read-only)	Server scripts	The locale of the sort.
	Sort.nullsLast	boolean	Server scripts	When set to true, indicates that nulls are placed last in the sort.
	Sort.order	number	Server scripts	Sort order indicator.

SortDefinition Object Members

The following members are available for a [workbook.SortDefinition](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SortDefinition.selector	workbook.DimensionSelector workbook.PathSelector	Server scripts	The selector for the sort definition.
	SortDefinition.sortBy	Array< workbook.SortByDataDimensionItem workbook.SortByMeasure >	Server scripts	The sort order for the sort definition.

SortByDataDimensionItem Object Members

The following members are available for a [workbook.SortByDataDimensionItem](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SortByDataDimensionItem.item	workbook.DataDimensionItem	Server scripts	The data dimension item to use for the sort.
	SortByDataDimensionItem.sort	workbook.Sort	Server scripts	The sort to use.

SortByMeasure Object Members

The following members are available for a [workbook.SortByMeasure](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	SortByMeasure.measure	workbook.CalculatedMeasure workbook.DataMeasure	Server scripts	The measure for the sort.
	SortByMeasure.otherAxisSelector	workbook.DescendantorSelfNodesSelector workbook.	Server scripts	The selector for the axis that is not defined in the associated sort definition.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
		PathSelector workbook.DimensionSelector		
	SortByMeasure.sort	workbook.Sort	Server scripts	The sort to use.

Style Object Members

The following members are available for a [workbook.Style](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Style.backgroundColor	string workbook.Color	Server scripts	The background color of the style.
	Style.backgroundImage	string	Server scripts	The background image of the style.
	Style.backgroundPosition	workbook.PositionPercent workbook.PositionUnits workbook.PositionValues	Server scripts	The background position of the style.
	Style.color	string workbook.Color	Server scripts	The color of the style.
	Style.fontSize	string workbook.FontSize	Server scripts	The font size of the style.
	Style.fontStyle	string	Server scripts	The font style of the style.
	Style.fontWeight	string	Server scripts	The font weight of the style.
	Style.textAlign	string	Server scripts	The text alignment of the style.
	Style.textDecorationColor	string workbook.Color	Server scripts	The text decoration color of the style.
	Style.textDecorationLine	string	Server scripts	The text decoration line of the style.
	Style.textDecorationStyle	string	Server scripts	The text decoration style of the style.

Table Object Members

The following members are available for a [workbook.Table](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Table.columns	workbook.TableColumn	Server scripts	The columns in the table view.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Table.dataset	dataset.Dataset	Server scripts	The dataset for the table view.
	Table.id	string	Server scripts	The ID of the table view.
	Table.name	string workbook.Expression	Server scripts	The label of the table view.

TableColumn Object Members

The following members are available for a [workbook.TableColumn](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	TableColumn.alias	string	Server scripts	The alias for the table column.
	TableColumn.datasetColumnAlias	string	Server scripts	The alias of the dataset column from which the table column was created.
	TableColumn.fieldContext	workbook.FieldContext	Server scripts	The field context for the field used in the table column.
	TableColumn.filters	workbook.TableColumnFilter	Server scripts	The filters for the table column.
	TableColumn.label	string	Server scripts	The label for the table column.
	TableColumn.sort	workbook.Sort	Server scripts	The sort for the table column.
	TableColumn.width	number	Server scripts	The desired width of the table column when displayed in the UI.

TableColumnCondition Object Members

The following members are available for a [workbook.TableColumnCondition](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	TableColumnCondition.filters	workbook.TableColumnFilter []	Server scripts	The filters for the condition
	TableColumnCondition.operator	string	Server scripts	The operator for the condition.

TableColumnFilter Object Members

The following members are available for a [workbook.TableColumnFilter](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	TableColumnFilter.operator	string	Server scripts	The operator for the table column filter.
	TableColumnFilter.values	Array<null Object boolean number string Date>	Server scripts	The values for the table column filter.

Workbook Object Members

The following members are available for a [workbook.Workbook](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Workbook.runPivot(options)	workbook.PivotIntersection[]	Server scripts	Runs a pivot in the workbook and returns the results as a set of row-column intersections.
Property	Workbook.description	string	Server scripts	The description of the workbook.
	Workbook.id	string	Server scripts	The ID of the workbook.
	Workbook.name	string	Server scripts	The name of the workbook.
	Workbook.pivots	workbook.Pivot[]	Server scripts	The pivots in the workbook.
	Workbook.tables	workbook.Table[]	Server scripts	The tables in the workbook.

N/workbook Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/workbook module:

- [Create Datasets, Dataset Links, and a Workbook with a Pivot and Run the Workbook](#)
- [Create a Comprehensive Workbook](#)

Create Datasets, Dataset Links, and a Workbook with a Pivot and Run the Workbook

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates two datasets, links them, and creates a workbook with a pivot based on the data in the datasets.

Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   // The following script creates a simple workbook that contains a simple table
5   require(['N/workbook', 'N/dataset', 'N/datasetLink'], function(nWorkbook, nDataset, datasetLink) {
6       var join = nDataset.createJoin({
7           fieldId: "budgetmachine"
8       });
9
10      var period = nDataset.createColumn({
11          join: join,
12          fieldId: "period",
13          alias: "budgetmachineperiod",
14          label: "Accounting Period"
15      });
16
17      var department = nDataset.createColumn({
18          fieldId: "department",
19          alias: "department",
20          label: "Department"
21      });
22
23      var total = nDataset.createColumn({
24          fieldId: "total",
25          alias: "total",
26          label: "Amount (Total)"
27      });
28
29      var budget = nDataset.create({
30          type: 'budgets',
31          columns: [period, department, total]
32      });
33
34      var postingperiod = nDataset.createColumn({
35          fieldId: "postingperiod",
36          alias: "postingperiod",
37          label: "Posting Period"
38      });
39
40      var amount = nDataset.createColumn({
41          fieldId: "amount",
42          alias: "amount",
43          label: "Amount"
44      });
45
46      var sales = nDataset.create({
47          type: 'salesinvoiced',
48          columns: [postingperiod, department, amount],
49      });
50
51      var budgetmachineperiod = budget.getExpressionFromColumn({
52          alias: "budgetmachineperiod"
53      });
54
55      var postingperiodExpression = sales.getExpressionFromColumn({
56          alias: "postingperiod"
57      });
58
59      var link = datasetLink.create({
60          datasets: [budget, sales],
61          expressions: [[budgetmachineperiod, postingperiodExpression]],
62          id: "link"
63      });
64
65      var postingPeriodItem = nWorkbook.createDataDimensionItem({
66          expression: postingperiodExpression

```

```

66     });
67     var postingPeriodDimension = nWorkbook.createDataDimension({
68         items: [postingPeriodItem]
69     });
70     var rowSection = nWorkbook.createSection({
71         children: [postingPeriodDimension]
72     });
73
74     var departmentItem = nWorkbook.createDataDimensionItem({
75         expression: budget.getExpressionFromColumn({
76             alias: "department"
77         })
78     });
79     var departmentDimension = nWorkbook.createDataDimension({
80         items: [departmentItem]
81     });
82
83     var sumTotal = nWorkbook.createDataMeasure({
84         label: 'Sum Total',
85         expression: budget.getExpressionFromColumn({
86             alias: 'total'
87         }),
88         aggregation: 'SUM'
89     });
90
91     var sumAmountNet = nWorkbook.createDataMeasure({
92         label: 'Sum Amount',
93         expression: sales.getExpressionFromColumn({
94             alias: 'amount'
95         }),
96         aggregation: 'SUM'
97     });
98
99     var columnSection = nWorkbook.createSection({
100         children: [departmentDimension, sumTotal, sumAmountNet]
101     });
102
103     var pivot = nWorkbook.createPivot({
104         id: "pivot",
105         rowAxis: nWorkbook.createPivotAxis({
106             root: rowSection
107         }),
108         columnAxis: nWorkbook.createPivotAxis({
109             root: columnSection
110         }),
111         name: "Pivot",
112         datasetLink: link
113     });
114
115     var wb = nWorkbook.create({
116         pivots: [pivot]
117     });
118
119     wb.runPivot.promise("pivot").then(function(intersections){
120         for (var i in intersections)
121         {
122             var intersection = intersections[i];
123             if (intersection.row.itemValues) //skip header
124             {
125                 console.log("Period: " + intersection.row.itemValues[0].value.name);
126                 console.log(intersection.column.section.children[1].label + ":");
127                 console.log(intersection.measureValues[0] ? intersection.measureValues[0].value.amount : 0);
128                 console.log(intersection.column.section.children[2].label + ":");
129                 console.log(intersection.measureValues[1] ? intersection.measureValues[1].value.amount : 0);
130             }
131         }
132     })
133 });

```


Create a Comprehensive Workbook

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a workbook that includes a chart, a table, and a pivot. This sample uses a dataset that is not included in your account, so you will need to change the dataset id to a valid value from your account. This dataset needs to include columns for 'id', 'name', 'date', and 'total'.

Note: This sample script uses the `require` function so you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (a script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4   // The following script creates a workbook that includes a chart, a table, and a pivot. This sample uses a dataset that is not included in your account, so
   // you will need to change the dataset id to a valid value from your account. This dataset needs to include columns for 'id', 'name', 'date', and 'total'.
5
6   require(['N/workbook', 'N/dataset'], function(workbook, dataset){
7       var myDataset = dataset.load({
8           id: 'dataset_7'
9       });
10
11      var theIDExpression = myDataset.getExpressionFromColumn({
12          alias: 'id'
13      });
14      var sort = workbook.createSort({
15          ascending: false
16      });
17      var columnID = workbook.createTableColumn({
18          datasetColumnAlias: 'id'
19      });
20      var columnName = workbook.createTableColumn({
21          datasetColumnAlias: 'name'
22      });
23      var columnDate = workbook.createTableColumn({
24          datasetColumnAlias: 'date'
25      });
26      var columnTotal = myBasicWorkbook.createTableColumn({
27          datasetColumnAlias: 'total'
28      });
29      var tableView = workbook.createTableDefinition({
30          id: 'view',
31          name: 'View',
32          dataset: myDataset,
33          columns: [columnID, columnName, columnDate, columnTotal]
34      });
35      var theDateExpression = dataset.getExpressionFromColumn({
36          alias: 'date'
37      });
38      var rowItem = workbook.createDataDimensionItem({
39          label: 'A',
40          expression: theDateExpression
41      });
42      var rowDataDimension = workbook.createDataDimension({
43          items: [rowItem]
44      });
45      var rowSection = workbook.createSection({
46          children: [rowDataDimension]
47      });
48      var theTotalExpression = dataset.getExpressionFromColumn({
49          alias: 'total'
50      });
51      var columnItem = workbook.createDataDimensionItem({
52          label: 'B',
53          expression: theTotalExpression
54      });
55      var columnDataDimension = workbook.createDataDimension({

```

```

55     items: [columnItem]
56   });
57   var columnMeasure = workbook.createMeasure({
58     label: 'M',
59     expression: theIDExpression,
60     aggregation: workbook.Aggregation.MAX
61   });
62   var columnSection = workbook.createSection({
63     children: [columnDataDimension, columnMeasure]
64   });
65   var constExpr = workbook.createConstant({
66     constant: 1
67   });
68   var anyOfExpr = workbook.createExpression({
69     functionId: workbook.ExpressionType.AND,
70     parameters: {
71       expression: theIDExpression,
72       set: [constExpr]
73     }
74   });
75   var notExpr = workbook.createExpression({
76     functionId: workbook.ExpressionType.NOT,
77     parameters: {
78       a: anyOfExpr
79     }
80   });
81   var allSubNodesSelector = workbook.createAllSubNodesSelector();
82   var rowItemSelector = workbook.createDimensionSelector({
83     dimension: rowDataDimension
84   });
85   var columnItemSelector = workbook.createDimensionSelector({
86     dimension: columnDataDimension
87   });
88   var rowSelector = workbook.createPathSelector({
89     elements: [allSubNodesSelector, rowItemSelector]
90   });
91   var columnSelector = workbook.createPathSelector({
92     elements: [allSubNodesSelector, columnItemSelector]
93   });
94   var rowSort = workbook.createDimensionSort({
95     item: rowItem,
96     sort: sort
97   });
98   var columnSort = workbook.createMeasureSort({
99     measure: columnMeasure,
100    sort: sort,
101    otherAxisSelector: allSubNodesSelector
102  });
103  var rowSortDefinition = workbook.createSortDefinition({
104    sortBy: [rowSort],
105    selector: rowSelector
106  });
107  var columnSortDefinition = workbook.createSortDefinition({
108    sortBy: [columnSort],
109    selector: columnSelector
110  });
111  var rowAxis = workbook.createPivotAxis({
112    root: rowSection,
113    sortDefinitions: [rowSortDefinition]
114  });
115  var columnAxis = workbook.createPivotAxis({
116    root: columnSection,
117    sortDefinitions: [columnSortDefinition]
118  });
119  var limitingFilter = workbook.createLimitingFilter({
120    row: true,
121    filteredNodesSelector: rowSelector,
122    limit: 1,
123    sortBy: [rowSort]
124  });
125  var conditionalFilter = workbook.createConditionalFilter({
126    row: false,
127    filteredNodesSelector: rowSelector,

```

```

128     otherAxisSelector: columnSelector,
129     measure: columnMeasure,
130     predicate: notExpr
131   });
132   var pivot = workbook.createPivotDefinition({
133     id: 'pivot',
134     name: 'Pivot',
135     dataset: myDataset,
136     rowAxis: rowAxis,
137     columnAxis: columnAxis,
138     filterExpressions: [notExpr],
139     aggregationFilters: [limitingFilter, conditionalFilter]
140   });
141   var firstAxis = workbook.createChartAxis({
142     title: 'First axis'
143   });
144   var secondAxis = workbook.createChartAxis({
145     title: 'Second axis'
146   });
147   var category = workbook.createCategory({
148     axis: firstAxis,
149     root: rowSection
150   });
151   var legend = workbook.createLegend({
152     axes: [secondAxis],
153     root: columnSection
154   });
155   var aspect = workbook.createAspect({
156     measure: columnMeasure
157   });
158   var series = workbook.createSeries({
159     aspects: [aspect]
160   });
161   var chart = workbook.createChartDefinition({
162     id: 'chart',
163     name: 'Chart',
164     type: workbook.ChartType.AREA,
165     dataset: myDataset,
166     category: category,
167     legend: legend,
168     series: [series]
169   });
170   var myNewWorkbook = workbook.create({
171     description: 'My new updated workbook',
172     name: 'Workbook Updated',
173     tableDefinitions: [tableView],
174     pivotDefinitions: [pivot],
175     chartDefinitions: [chart]
176   });
177   var workbookList = workbook.list();
178
179   log.debug({
180     title: 'MyNewWorkbook',
181     details: myNewWorkbook
182   });
183 });

```

workbook.Aspect

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An aspect object which is used when you create a series. Use workbook.createAspect(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/workbook Module
Methods and Properties	Aspect Object Members
Since	2020.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Aspect
4 var myAspect = workbook.createAspect({
5     measure: myMeasure,
6 });
7
8 // Create a complete Aspect
9 var myAspect = workbook.createAspect({
10     measure: myMeasure,
11     type: workbook.AspectType.COLOR
12 });
13
14 // View a workbook.Aspect used in a Chart
15 var myWorkbook = workbook.load ({
16     id: myWorkbookId
17 });
18 var myAspect = myWorkbook.charts[0].series[0].aspect[0];
19
20 // Note that some Aspect properties may be empty/null based on the loaded workbook
21 log.audit({
22     title: 'Aspect type = ',
23     details: myAspect.type
24 });
25 log.audit({
26     title: 'Aspect measure = ',
27     details: myAspect.measure
28 });
29 ...
30 // Add additional code

```

Aspect.measure

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The measure of the aspect.
Type	workbook.CalculatedMeasure workbook.DataMeasure
Module	N/workbook Module
Parent Object	workbook.Aspect
Sibling Object Members	Aspect Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.CalculatedMeasure or a workbook.DataMeasure .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Aspect.measure in a Chart
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Aspect.measure = ',
10  details: myWorkbook.charts[0].series[0].aspect[0].measure
11 });
12 ...
13 // Add additional code

```

Aspect.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of aspect.
Type	workbook.AspectType
Module	N/workbook Module
Parent Object	workbook.Aspect
Sibling Object Members	Aspect Object Members
Since	2020.2

Errors

Error Code	Thrown If
INVALID_ASPECT_TYPE	The value specified for this property is invalid. Set this value using workbook.AspectType .
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.AspectType .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Aspect.measure in a Chart
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Aspect.type = ',
10  details: myWorkbook.charts[0].series[0].aspect[0].type
11 });
12 ...
13 // Add additional code
```

workbook.CalculatedMeasure

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A calculated measure object which are used to calculate a value based on the value of fields or other measures. Use workbook.createCalculatedMeasure(options) to create this object. A calculated measure object is used as a parameter in the workbook.createMeasureSelector(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	CalculatedMeasure Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#) and [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myCalculatedMeasure = workbook.createCalculatedMeasure({
4   label: 'Balance',
5   expression: workbook.createExpression({
6     functionId: workbook.ExpressionType.MINUS,
7     parameters: {
8       operand1: workbook.createExpression({
```

```
9         functionId: workbook.ExpressionType.MEASURE_VALUE,
10         parameters: {
11             measure: myFirstDataMeasure
12         }
13     }},
14     operand2: workbook.createExpression({
15         functionId: workbook.ExpressionType.MEASURE_VALUE,
16         parameters: {
17             measure: mySecondDataMeasure
18         }
19     })
20 }
21 })
22 });
23 ...
24 // Add additional code
```

CalculatedMeasure.expression

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The expression for the calculated measure.
Type	workbook.Expression
Module	N/workbook Module
Parent Object	workbook.CalculatedMeasure
Sibling Object Members	CalculatedMeasure Object Members
Since	2021.2

Errors

Error Code	Thrown If
MUTUALLY_EXCLUSIVE_ARGUMENTS	The expression is already defined.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression object.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myCalculatedMeasure = workbook.createCalculatedMeasure({
4     label: 'Balance',
5     expression: workbook.createExpression({
6         functionId: workbook.ExpressionType.MINUS,
7         parameters: {
8             operand1: workbook.createExpression({
9                 functionId: workbook.ExpressionType.MEASURE_VALUE,
```

```
10         parameters: {
11             measure: myFirstDataMeasure
12         },
13     },
14     operand2: workbook.createExpression({
15         functionId: workbook.ExpressionType.MEASURE_VALUE,
16         parameters: {
17             measure: mySecondDataMeasure
18         }
19     })
20 }
21 })
22 });
23
24 var theExpression = myCalculatedMeasure.expression;
25 ...
26 // Add additional code
```

CalculatedMeasure.label

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for the calculated measure.
Type	workbook.Expression string
Module	N/workbook Module
Parent Object	workbook.CalculatedMeasure
Sibling Object Members	CalculatedMeasure Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression object or a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myCalculatedMeasure = workbook.createCalculatedMeasure({
4     label: 'Balance',
5     expression: workbook.createExpression({
6         functionId: workbook.ExpressionType.MINUS,
7         parameters: {
8             operand1: workbook.createExpression({
9                 functionId: workbook.ExpressionType.MEASURE_VALUE,
10                 parameters: {
```



```
11         measure: myFirstDataMeasure
12     },
13 },
14 operand2: workbook.createExpression({
15     functionId: workbook.ExpressionType.MEASURE_VALUE,
16     parameters: {
17         measure: mySecondDataMeasure
18     }
19 })
20 }
21 })
22 });
23
24 var theLabel = myCalculatedMeasure.label;
25 ...
26 // Add additional code
```

workbook.Category

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A category object which is used when you create a chart definition. Use workbook.createCategory(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	Category Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // Create a Category based on a Section
4 var myCategory = workbook.createCategory({
5     axis: myChartAxis,
6     root: mySection,
7     sortDefinitions: [mySortDefinition]
8 });
9
10 // Create a Category based on a DataDimension
11 var myCategory = workbook.createCategory({
12     axis: myChartAxis,
13     root: myDataDimension,
14     sortDefinitions: [mySortDefinition]
15 });
16
17 // View a workbook.Category used in a Chart
18 var myWorkbook = workbook.load ({
19     id: myWorkbookId
20 });
```

```
21 var myCategory = myWorkbook.charts[0].category;
22
23 // Note that some Category properties may be empty/null based on the loaded workbook
24 log.audit({
25     title: 'Category axis = ',
26     details: myCategory.axis
27 });
28 log.audit({
29     title: 'Category root = ',
30     details: myCategory.root
31 });
32 log.audit({
33     title: 'Category sortDefinitions = ',
34     details: myCategory.sortDefinitions
35 });
36 ...
37 // Add additional code
```

Category.axis

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The axis for the category.
Type	workbook.ChartAxis
Module	N/workbook Module
Parent Object	workbook.Category
Sibling Object Members	Category Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.ChartAxis .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Category.axis in a Chart
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Category.axis = ',
10    details: myWorkbook.charts[0].category.axis
11 });
```

```

12 ...
13 // Add additional code

```

Category.root

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The root data (that is, fields) that defines the category.
Type	workbook.DataDimension workbook.Section
Module	N/workbook Module
Parent Object	workbook.Category
Sibling Object Members	Category Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimension or a workbook.Section .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Category.root in a Chart
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Category.root = ',
10    details: myWorkbook.charts[0].category.root
11 });
12 ...
13 // Add additional code

```

Category.sortDefinitions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort definitions for the category.
-----------------------------	--

Type	workbook.SortDefinition[]
Module	N/workbook Module
Parent Object	workbook.Category
Sibling Object Members	Category Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.SortDefinition[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Category.sortDefinitions in a Chart
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Category.sortDefinitions = ',
10    details: myWorkbook.charts[0].category.sortDefinitions
11 });
12 ...
13 // Add additional code

```

workbook.Chart

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A chart definition object. A chart is a workbook component that enables you to visualize your dataset query results using predefined chart and graph types, such as line graphs and bar charts. A chart definition is used when you create a workbook. Use workbook.createChart(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Chart Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Chart
4 var myChart = workbook.createChart({
5     name: 'myChart',
6     id: '_myChart',
7     type: workbook.ChartType.BAR,
8     category: myCategory,
9     legend: myLegend,
10    series: [mySeries],
11    dataset: myDataset
12 });
13
14 // Create a comprehensive Chart
15 var myChart = workbook.createChart({
16     name: 'myChart',
17     title: 'My Chart Title',
18     subTitle: 'My Chart Subtitle',
19     id: '_myChart',
20     type: workbook.ChartType.BAR,
21     stacking: workbook.Stacking.PERCENT,
22     category: myCategory,
23     legend: myLegend,
24     series: [mySeries],
25     filterExpressions: [myExpression],
26     aggregationFilters: [myLimitingFilter],
27     dataset: myDataset
28 });
29
30 // View a workbook.Chart
31 var myWorkbook = workbook.load({
32     id: myWorkbookId
33 });
34 var myChart = myWorkbook.charts[0];
35
36 // Note that some Chart properties may be empty/null based on the loaded workbook
37 log.audit({
38     title: 'Chart id = ',
39     details: myChart.id
40 });
41 log.audit({
42     title: 'Chart name = ',
43     details: myChart.name
44 });
45 log.audit({
46     title: 'Chart title = ',
47     details: myChart.title
48 });
49 log.audit({
50     title: 'Chart subTitle = ',
51     details: myChart.subTitle
52 });
53 log.audit({
54     title: 'Chart legend = ',
55     details: myChart.legend
56 });
57 log.audit({
58     title: 'Chart category = ',
59     details: myChart.category
60 });
61 log.audit({
62     title: 'Chart series = ',
63     details: myChart.series
64 });

```

```

65 log.audit({
66     title: 'Chart stacking = ',
67     details: myChart.stacking
68 });
69 log.audit({
70     title: 'Chart filterExpressions = ',
71     details: myChart.filterExpressions
72 });
73 log.audit({
74     title: 'Chart aggregationFilters = ',
75     details: myChart.aggregationFilters
76 });
77 log.audit({
78     title: 'Chart dataset = ',
79     details: myChart.dataset
80 });
81 log.audit({
82     title: 'Chart type = ',
83     details: myChart.type
84 });
85 ...
86 // Add additional code

```

Chart.aggregationFilters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The limiting and conditional filters of the chart .
Type	Array< workbook.LimitingFilter workbook.ConditionalFilter >
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.LimitingFilter[] or a workbook.ConditionalFilter[] .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Chart.aggregationFilters
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({

```

```

9      title: 'Chart.aggregationFilters = ',
10     details: myWorkbook.charts[0].aggregationFilters
11   });
12   ...
13   // Add additional code

```

Chart.category

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The category of the chart.
Type	string
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View Chart.category
4   var myWorkbook = workbook.load({
5     id: myWorkbookId
6   });
7
8   log.audit({
9     title: 'Chart.category = ',
10    details: myWorkbook.charts[0].category
11  });
12  ...
13  // Add additional code

```

Chart.dataset

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The underlying dataset for the chart.
Type	dataset.Dataset

Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a dataset.Dataset .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Chart.dataset
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Chart.dataset = ',
10    details: myWorkbook.charts[0].dataset
11 });
12 ...
13 // Add additional code

```

Chart.filterExpressions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The filter expressions for the chart.
Type	workbook.Expression[]
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Chart.filterExpressions
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Chart.filterExpressions = ',
10    details: myWorkbook.charts[0].filterExpressions
11 });
12 ...
13 // Add additional code
```

Chart.id

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of chart.
Type	string
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this parameter is a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Chart.id
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
```

```

9      title: 'Chart.id = ',
10     details: myWorkbook.charts[0].id
11   });
12   ...
13   // Add additional code

```

Chart.legend

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The legend of the chart.
Type	TBD
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View Chart.legend
4   var myWorkbook = workbook.load({
5     id: myWorkbookId
6   });
7
8   log.audit({
9     title: 'Chart.legend = ',
10    details: myWorkbook.charts[0].legend
11  });
12  ...
13  // Add additional code

```

Chart.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the chart.
Type	string

Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Chart.name
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Chart.name = ',
10    details: myWorkbook.charts[0].name
11 });
12 ...
13 // Add additional code

```

Chart.series

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The series of the chart.
Type	workbook.Series
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Series .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Chart.series
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Chart.series = ',
10  details: myWorkbook.charts[0].series[0]
11 });
12 ...
13 // Add additional code
```

Chart.stacking

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The stacking type for the chart.
Type	workbook.Stacking
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
INVALID_STACKING_TYPE	The value for this property is invalid. Set this parameter using workbook.Stacking .
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Stacking .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Chart.stacking
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
```

```

7
8 log.audit({
9     title: 'Chart.stacking = ',
10    details: myWorkbook.charts[0].stacking
11 });
12 ...
13 // Add additional code

```

Chart.subTitle

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The subtitle of the chart.
Type	string
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this parameter is a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Chart.subTitle
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Chart.subTitle = ',
10    details: myWorkbook.charts[0].subTitle
11 });
12 ...
13 // Add additional code

```

Chart.title

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The title of chart.
Type	string

Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Chart.title
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Chart.title = ',
10    details: myWorkbook.charts[0].title
11 });
12 ...
13 // Add additional code

```

Chart.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The chart type of the chart.
Type	workbook.ChartType
Module	N/workbook Module
Parent Object	workbook.Chart
Sibling Object Members	Chart Object Members
Since	2020.2

Errors

Error Code	Thrown If
INVALID_CHART_TYPE	The value for this property is invalid. Set this parameter using workbook.ChartType .

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.ChartType .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Chart.type
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Chart.type = ',
10    details: myWorkbook.charts[0].type
11 });
12 ...
13 // Add additional code

```

workbook.ChartAxis

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A chart axis object which is used when you create a category or a legend. Use workbook.createChartAxis(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	ChartAxis Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a ChartAxis
4 var myChartAxis = workbook.createChartAxis({
5     title: 'My ChartAxis'
6 });
7

```

```
8 // View a workbook.ChartAxis used in a Chart
9 var myWorkbook = workbook.load ({
10   id: myWorkbookId
11 });
12 var myCategoryChartAxis = myWorkbook.charts[0].category.axis;
13
14 log.audit({
15   title: 'Category ChartAxis title = ',
16   details: myCategoryChartAxis.title
17 });
18
19 var myLegendChartAxis = myWorkbook.charts[0].legend.axes[0];
20
21 log.audit({
22   title: 'Legend ChartAxis title = ',
23   details: myLegendChartAxis.title
24 });
25 ...
26 // Add additional code
```

ChartAxis.title

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The title of the chart axis.
Type	string
Module	N/workbook Module
Parent Object	workbook.ChartAxis
Sibling Object Members	ChartAxis Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View ChartAxis.title in a Chart
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Category ChartAxis.title = ',
10  details: myWorkbook.charts[0].category.axis.title
11 });
```



```

12 log.audit({
13     title: 'Legend ChartAxis.title = '
14     details: myWorkbook.charts[0].legend.axes[0].title
15 });
16 ...
17 // Add additional code

```

workbook.ChildNodesSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A selector for child nodes. There is no method that creates this object - you can use it directly in scripts.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Since	2021.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myChildNodesSelector = workbook.ChildNodesSelector;
4 ...
5 // Add additional code

```

workbook.Color

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A color object which is made up of red, green, blue, and alpha components. Use workbook.createColor(options) to create this object. A color object is used as a parameter in the workbook.createStyle(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Color Object Members
Since	2021.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myColor = workbook.createColor({
4   red: 255,
5   green: 192,
6   blue: 203,
7   alpha: 0.5
8 });
9 ...
10 // Add additional code
```

Color.alpha

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The opacity, or transparency, of the color.
Type	number
Module	N/workbook Module
Parent Object	workbook.Color
Sibling Object Members	Color Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_ALPHA_VALUE	The value of this property is not between 0 and 255.
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myColor = workbook.createColor({
4   red: 255,
5   green: 192,
6   blue: 203,
7   alpha: 0.5
8 });
9 ...
10 var theAlpha = myColor.alpha;
```

```

11 ...
12 // Add additional code

```

Color.blue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The blue portion of the color.
Type	number
Module	N/workbook Module
Parent Object	workbook.Color
Sibling Object Members	Color Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_COLOR_VALUE	The value of this property is not between 0 and 255.
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myColor = workbook.createColor({
4   red: 255,
5   green: 192,
6   blue: 203,
7   alpha: 0.5
8 });
9
10 var theBlue = myColor.blue;
11 ...
12 // Add additional code

```

Color.green

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The green portion of the color.
Type	number
Module	N/workbook Module

Parent Object	workbook.Color
Sibling Object Members	Color Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_COLOR_VALUE	The value of this property is not between 0 and 255.
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myColor = workbook.createColor({
4     red: 255,
5     green: 192,
6     blue: 203,
7     alpha: 0.5
8 });
9
10 var theGreen = myColor.green;
11 ...
12 // Add additional code

```

Color.red

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The red portion of the color.
Type	number
Module	N/workbook Module
Parent Object	workbook.Color
Sibling Object Members	Color Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_COLOR_VALUE	The value of this property is not between 0 and 255.
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myColor = workbook.createColor({
4   red: 255,
5   green: 192,
6   blue: 203,
7   alpha: 0.5
8 });
9
10 var theRed = myColor.red;
11 ...
12 // Add additional code
```

workbook.ConditionalFilter

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A conditional filter object. Use workbook.createConditionalFilter(options) to create this object. A conditional filter object is used as a parameter in the workbook.createPivot(options) and workbook.createTableColumn(options) methods.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	ConditionalFilter Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4   filteredNodesSelector: mySelector,
5   measure: myMeasure,
6   otherAxisSelector: myOtherSelector,
7   predicate: myPredicateExpression,
8   row: true
9 });
10 ...
11 // Add additional code
```

ConditionalFilter.columnSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The column selector.
Type	workbook.DescendantorSelfNodesSelector workbook.PathSelector workbook.DimensionSelector workbook.ChildNodesSelector
Module	N/workbook Module
Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DescendantorSelfNodesSelector , a workbook.PathSelector , a workbook.DimensionSelector , or a workbook.ChildNodesSelector .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4   filteredNodesSelector: mySelector,
5   measure: myMeasure,
6   otherAxisSelector: myOtherSelector,
7   predicate: myPredicateExpression,
8   row: true
9 });
10
11 var theColumnSelector = myConditionalFilter.columnSelector;
12 ...
13 // Add additional code

```

ConditionalFilter.filteredNodesSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The selected filters in the condition filter.
Type	workbook.PathSelector workbook.DimensionSelector
Module	N/workbook Module

Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.PathSelector or a workbook.DimensionSelector .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4     filteredNodesSelector: mySelector,
5     measure: myMeasure,
6     otherAxisSelector: myOtherSelector,
7     predicate: myPredicateExpression,
8     row: true
9 });
10
11 var theFilteredNodesSelector = myConditionalFilter.filteredNodesSelector;
12 ...
13 // Add additional code

```

ConditionalFilter.measure

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The measure of the conditional filter.
Type	workbook.CalculatedMeasure workbook.DataMeasure
Module	N/workbook Module
Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.CalculatedMeasure or a workbook.DataMeasure .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4     filteredNodesSelector: mySelector,
5     measure: myMeasure,
6     otherAxisSelector: myOtherSelector,
7     predicate: myPredicateExpression,
8     row: true
9 });
10
11 theMeasure = myConditionalFilter.measure;
12 ...
13 // Add additional code
```

ConditionalFilter.otherAxisSelector

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The selector for the other axis in the conditional filter.
Type	workbook.PathSelector workbook.DimensionSelector
Module	N/workbook Module
Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.PathSelector or a workbook.DimensionSelector .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4     filteredNodesSelector: mySelector,
5     measure: myMeasure,
6     otherAxisSelector: myOtherSelector,
7     predicate: myPredicateExpression,
8     row: true
9 });
10
```



```

11 | var theOtherAxisSelector = myConditionalFilter.otherAxisSelector;
12 | ...
13 | // Add additional code

```

ConditionalFilter.predicate

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The predicate expression for the conditional filter, which indicates whether the condition is met.
Type	workbook.Expression
Module	N/workbook Module
Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 | // Add additional code
2 | ...
3 | var myConditionalFilter = workbook.createConditionalFilter({
4 |     filteredNodesSelector: mySelector,
5 |     measure: myMeasure,
6 |     otherAxisSelector: myOtherSelector,
7 |     predicate: myPredicateExpression,
8 |     row: true
9 | });
10 |
11 | var thePredicate = myConditionalFilter.predicate;
12 | ...
13 | // Add additional code

```

ConditionalFilter.row

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the conditional filter applies to a row (in which case, the value is true) or a column (in which case, the value is false).
Type	boolean
Module	N/workbook Module

Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4     filteredNodesSelector: mySelector,
5     measure: myMeasure,
6     otherAxisSelector: myOtherSelector,
7     predicate: myPredicateExpression,
8     row: true
9 });
10
11 var theRow = myConditionalFilter.row;
12 ...
13 // Add additional code

```

ConditionalFilter.rowSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The row selector.
Type	workbook.D descendantorSelfNodesSelector workbook.PathSelector workbook.DimensionSelector workbook.ChildNodesSelector
Module	N/workbook Module
Parent Object	workbook.ConditionalFilter
Sibling Object Members	ConditionalFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.D descendantorSelfNodesSelector , a workbook.PathSelector , a workbook.DimensionSelector , or a workbook.ChildNodesSelector .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myConditionalFilter = workbook.createConditionalFilter({
4   filteredNodesSelector: mySelector,
5   measure: myMeasure,
6   otherAxisSelector: myOtherSelector,
7   predicate: myPredicateExpression,
8   row: true
9 });
10
11 var theRowSelector = myConditionalFilter.rowSelector;
12 ...
13 // Add additional code
```

workbook.ConditionalFormat

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A set of conditional formatting rules. Use workbook.createConditionalFormat(options) to create this object. A conditional format object is used as a parameter in the workbook.createPivot(options) and workbook.createTableColumn(options) methods.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	ConditionalFormat Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 //myConditionalFormat is a workbook.ConditionalFormat object
4
5 var myConditionalFormat = workbook.createConditionalFormat({
6   rules: [formatrules] // array of previously created workbook.ConditionalFormatRule
7 });
8 ...
9 // Add additional code
```

ConditionalFormat.rules

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The formatting rules for the conditional format.
Type	workbook.ConditionalFormatRule
Module	N/workbook Module
Parent Object	workbook.ConditionalFormat
Since	2021.2

Errors

Error Code	Thrown If
NO_RULE_DEFINED	This property is an empty array.
WRONG_PARAMETER_TYPE	The value specified for this property is not an array of workbook.ConditionalFormatRule objects.

workbook.ConditionalFormatRule

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A conditional format rule object. Use workbook.createConditionalFormatRule(options) to create this object. A conditional format rule object is used as a parameter in the workbook.createConditionalFormat(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	ConditionalFormatRule Object Members
Since	2021.2

ConditionalFormatRule.filter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The filter that determines which rows or cells to apply the conditional format to.
Type	workbook.TableColumnFilter
Module	N/workbook Module
Parent Object	workbook.ConditionalFormatRule
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.TableColumnFilter object.

ConditionalFormatRule.style

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The style to apply as the conditional format.
Type	workbook.Style
Module	N/workbook Module
Parent Object	workbook.ConditionalFormatRule
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Style object.

workbook.Currency

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An amount in a particular currency. This object can be returned from a pivot execution. You can also use this object as a constant parameter when creating constant expressions. Use workbook.createCurrency(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Currency Object Members
Since	2021.2

Currency.amount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The amount of the currency.
-----------------------------	-----------------------------

Type	number
Module	N/workbook Module
Parent Object	workbook.Currency
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Currency object has been created.

Currency.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the currency (for example, USD, EUR, GBP, and so on).
Type	string
Module	N/workbook Module
Parent Object	workbook.Currency
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Currency object has been created.

workbook.DataDimension

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A data dimension object which can be used when you create a category, a legend, a pivot axis, a dimension selector, or a section. Use workbook.createDataDimension(options) to create this object. A data dimension object is used as a parameter in the workbook.createDataDimension(options), workbook.createDimensionSelector(options), workbook.createPivotAxis(options), and workbook.createSection(options) methods.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module

Methods and Properties	DataDimension Object Members
Since	2020.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDataDimension = workbook.createDataDimension({
4     items: [myDataDimensionItem]
5 });
6 ...
7 // Add additional code

```

DataDimension.children

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The children of the data dimension.
Type	Array< workbook.Section > Array< workbook.DataDimension > Array < workbook.CalculatedMeasure > Array < workbook.DataMeasure >
Module	N/workbook Module
Parent Object	workbook.DataDimension
Sibling Object Members	DataDimension Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimension[] or a workbook.Section[] .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDataDimension = workbook.createDataDimension({

```

```

4     items: [myDataDimensionItem],
5     children: [myMeasure],
6     totalLine: workbook.TotalLine.FIRST_LINE
7   });
8
9   var theChildren = myDataDimension.children;
10  ...
11  // Add additional code

```

DataDimension.items

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The items of the data dimension.
Type	workbook.DataDimensionItem[]
Module	N/workbook Module
Parent Object	workbook.DataDimension
Sibling Object Members	DataDimension Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimensionItem[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  var myDataDimension = workbook.createDataDimension({
4    items: [myDataDimensionItem],
5    children: [myMeasure],
6    totalLine: workbook.TotalLine.FIRST_LINE
7  });
8
9  var theItems = myDataDimension.items;
10 ...
11 // Add additional code

```

DataDimension.totalLine

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The formatting specification for the total line of the data dimension.
-----------------------------	--

Type	workbook.TotalLine
Module	N/workbook Module
Parent Object	workbook.DataDimension
Sibling Object Members	DataDimension Object Members
Since	2020.2

Errors

Error Code	Thrown If
INVALID_TOTAL_LINE	The value for this property is invalid. Set this parameter using values from the workbook.TotalLine enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not included in the workbook.TotalLine enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDataDimension = workbook.createDataDimension({
4     items: [myDataDimensionItem],
5     children: [myMeasure],
6     totalLine: workbook.TotalLine.FIRST_LINE
7 });
8
9 var theTotalLine = myDataDimension.totalLine;
10 ...
11 // Add additional code

```

workbook.DataDimensionItem

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A data dimension item object which is used when you create a data dimension or a dimension sort. Use workbook.createDataDimensionItem(options) to create this object. A data dimension item object is used as a parameter in the workbook.createDataDimension(options), and workbook.createSortByDataDimensionItem(options) methods.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module

Methods and Properties	DataDimensionItem Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDataDimensionItem = workbook.createDataDimensionItem({
4     expression: myExpression,
5     label: 'My DD Item'
6 });
7 ...
8 // Add additional code

```

DataDimensionItem.expression

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The expression for data dimension item.
Type	workbook.Expression
Module	N/workbook Module
Parent Object	workbook.DataDimensionItem
Sibling Object Members	DataDimensionItem Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDataDimensionItem = workbook.createDataDimensionItem({

```

```
4     expression: myExpression,  
5     label: 'My DD Item'  
6   });  
7  
8   var theExpression = myDataDimensionItem.expression;  
9   ...  
10  // Add additional code
```

DataDimensionItem.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label for the data dimension item.
Type	string
Module	N/workbook Module
Parent Object	workbook.DataDimensionItem
Sibling Object Members	DataDimensionItem Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code  
2 ...  
3 var myDataDimensionItem = workbook.createDataDimensionItem({  
4     expression: myExpression,  
5     label: 'My DD Item'  
6   });  
7  
8 var theLabel = myDataDimensionItem.label;  
9 ...  
10 // Add additional code
```

workbook.DataDimensionItemValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	The value of a data dimension item.
---------------------------	-------------------------------------

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	DataDimensionItemValue Object Members
Since	2021.2

DataDimensionItemValue.item

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The data dimension item.
Type	workbook.DataDimension
Module	N/workbook Module
Parent Object	workbook.DataDimensionItemValue
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.DataDimensionItemValue object has been created.

DataDimensionItemValue.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The value of the data dimension item.
Type	string number boolean workbook.Record workbook.Currency workbook.Range workbook.Duration
Module	N/workbook Module
Parent Object	workbook.DataDimensionItemValue
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.DataDimensionItemValue object has been created.

workbook.DataDimensionValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A set of item values for a data dimension.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	DataDimensionValue Object Members
Since	2021.2

DataDimensionValue.dataDimension

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The data dimension.
Type	workbook.DataDimension
Module	N/workbook Module
Parent Object	workbook.DataDimensionValue
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.DataDimensionValue object has been created.

DataDimensionValue.itemValues

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The item values for the data dimension.
Type	workbook.DataDimensionItemValue[]
Module	N/workbook Module
Parent Object	workbook.CalculatedMeasure
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.DataDimensionValue object has been created.

workbook.DataMeasure

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A data measure object. Use workbook.createDataMeasure(options) to create this object. A data measure object is used as a parameter in the workbook.createMeasureSelector(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	DataMeasure Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // myDataMeasure is a workbook.DataMeasure object
4
5 var myDataMeasure = workbook.createDataMeasure({
6   aggregation: 'DataCount',
7   expression: singleExpression, // previously created workbook.Expression
8   label: 'MeasureLabel'
9 });
10 ...
11 // Add additional code

```

DataMeasure.aggregation

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The aggregation of the data measure.
Type	workbook.Aggregation

Module	N/workbook Module
Parent Object	workbook.DataMeasure
Since	2021.2

Errors

Error Code	Thrown If
INVALID_AGGREGATION	The value of this property is not included in the workbook.Aggregation enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

DataMeasure.expression

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The expression for this data measure. This property is used if the data measure is a single-expression measure.
Type	workbook.Expression
Module	N/workbook Module
Parent Object	workbook.DataMeasure
Since	2021.2

Errors

Error Code	Thrown If
MUTUALLY_EXCLUSIVE_ARGUMENTS	The DataMeasure.expressions property is also defined for this data measure.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression object.

DataMeasure.expressions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The expressions for this data measure. This property is used if the data measure is a multiple-expression measure.
Type	workbook.Expression[]
Module	N/workbook Module
Parent Object	workbook.DataMeasure

Since	2021.2
-------	--------

Errors

Error Code	Thrown If
MUTUALLY_EXCLUSIVE_ARGUMENTS	The DataMeasure.expression property is also defined for this data measure.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression object.

DataMeasure.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label of the data measure.
Type	workbook.Expression string
Module	N/workbook Module
Parent Object	workbook.DataMeasure
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression object or a string.

workbook.DescendantorSelfNodesSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A selector for descendant or self nodes object. A selector for descendant or self nodes object is used as a parameter in the workbook.createMeasureValueSelector(options), and workbook.createSortByMeasure(options) methods. For more information about selectors, see the help topic Selectors.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var conditionalRowSelector = nWorkbook.DescendantOrSelfNodesSelector;
4 var conditionalColumnSelector = nWorkbook.DescendantOrSelfNodesSelector;
5 var conditionalMeasureSelector = nWorkbook.createMeasureSelector({
6     measures: [calculatedMeasure]
7 });
8
9 var measureValueSelector = nWorkbook.createMeasureValueSelector({
10     rowSelector: conditionalRowSelector,
11     columnSelector: conditionalColumnSelector,
12     measureSelector: conditionalMeasureSelector
13 });
14 ...
15 // Add additional code
```

workbook.DimensionSelector

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A dimension selector object which is used when you create a path selector, a sort definition, a conditional filter, a limiting filter or a measure sort. Use workbook.createDimensionSelector(options) to create this object. A dimension selector object is used as a parameter in these methods: workbook.createConditionalFilter(options), workbook.createLimitingFilter(options), workbook.createMeasureValueSelector(options) , workbook.createPathSelector(options), workbook.createSortByMeasure(options), and workbook.createSortDefinition(options).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	DimensionSelector Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myDimensionSelector = workbook.createDimensionSelector({
```

```
4     dimension: myDataDimension
5   });
6   ...
7   // Add additional code
```

DimensionSelector.dimension

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The dimension of the dimension selector.
Type	workbook.DataDimension workbook.Section
Module	N/workbook Module
Parent Object	workbook.DimensionSelector
Sibling Object Members	DimensionSelector Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimension or a workbook.Section .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myDimensionSelector = workbook.createDimensionSelector({
4     dimension: myDataDimension
5 });
6
7 var theDimension = myDimensionSelector.dimension;
8 ...
9 // Add additional code
```

workbook.Duration

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A duration object.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	Duration Object Members
Since	2021.2

Duration.amount

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The amount of the duration.
Type	number
Module	N/workbook Module
Parent Object	workbook.Duration
Sibling Object Members	Duration Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Duration object has been created.

Duration.units

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The units of the duration.
Type	string
Module	N/workbook Module
Parent Object	workbook.Duration
Sibling Object Members	Duration Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Duration object has been created.

workbook.Expression

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	An expression object which can be used when you create a pivot definition, a data dimension item, a measure, a conditional filter, or a constant. Use workbook.createExpression(options) to create this object. An expression object is used as a parameter in these methods: workbook.createCalculatedMeasure(options), workbook.createConditionalFilter(options), workbook.createDataDimensionItem(options), workbook.createDataMeasure(options), workbook.createPivot(options), workbook.createReportStyleRule(options), workbook.createTable(options), and workbook.createTableColumn(options).
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Expression Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myExpression = workbook.createExpression({
4   functionId: workbook.ExpressionType.AND,
5   parameters: {
6     expressions: [expression1, expression2]
7   }
8 });
9 ...
10 // Add additional code

```

Expression.functionId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the function used in the expression. This property uses values from the workbook.ExpressionType enum.
Type	workbook.ExpressionType
Module	N/workbook Module
Parent Object	workbook.Expression

Sibling Object Members	Expression Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string (workbook.ExpressionType).

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myExpression = workbook.createExpression({
4     functionId: workbook.ExpressionType.AND,
5     parameters: {
6         expressions: [expression1, expression2]
7     }
8 });
9
10 var theFunctionId = myExpression.functionId;
11 ...
12 // Add additional code

```

Expression.parameters

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The parameters for the expression. To determine which parameter names to use, see workbook.ExpressionType .
Type	Object
Module	N/workbook Module
Parent Object	workbook.Expression
Sibling Object Members	Expression Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not an Object.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myExpression = workbook.createExpression({
4     functionId: workbook.ExpressionType.AND,
5     parameters: {
6         expressions: [expression1, expression2]
7     }
8 });
9
10 var theParameters = myExpression.parameters;
11 ...
12 // Add additional code
```

workbook.FieldContext

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A field context object which is used when you create a table column. Use workbook.createFieldContext(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	FieldContext Object Members
Since	2020.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // Create a basic FieldContext
4 var myFieldContext = workbook.createFieldContext({
5     name: 'My Field Context'
6 });
7
8 // Create a complete FieldContext
9 var myFieldContext = workbook.createFieldContext({
10     name: 'My Field Context',
11     parameters: {
12         parm1: 'ABC'
13     }
14 });
15 // View a workbook.FieldContext used in a Table definition
```

```
16 var myWorkbook = workbook.load({
17     id: myWorkbookId
18 });
19
20 // Note that some FieldContext properties may be empty/null based on the loaded workbook
21 var myFieldContext = myWorkbook.tables[0].columns[0].fieldContext;
22
23 log.audit({
24     title: 'FieldContext.name = ',
25     details: myFieldContext.name
26 });
27 log.audit({
28     title: 'FieldContext.parameters = ',
29     details: myFieldContext.parameters
30 });
31 ...
32 // Add additional code
```

FieldContext.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the field context (for example, DISPLAY or CONSOLIDATED)
Type	string
Module	N/workbook Module
Parent Object	workbook.FieldContext
Sibling Object Members	FieldContext Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View FieldContext.name in a d
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'FieldContext.name = ',
10    details: myWorkbook.tables[0].columns[0].fieldContext.name
11 });
12 ...
13 // Add additional code
```

FieldContext.parameters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The user-specified parameters of the field context, specified as key:value pairs.
Type	Object
Module	N/workbook Module
Parent Object	workbook.FieldContext
Sibling Object Members	FieldContext Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not an Object.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View FieldContext.parameters in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'FieldContext.parameters = ',
10  details: myWorkbook.tables[0].columns[0].fieldContext.parameters
11 });
12 ...
13 // Add additional code

```

workbook.FontSize

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A font size object. Use workbook.createFontSize(options) to create this object. A font size object is used as a parameter in the workbook.createStyle(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Module	N/workbook Module
Methods and Properties	FontSize Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySecondStyle = workbook.createStyle({
4     fontSize: workbook.FontSize.LARGE,
5     fontStyle: workbook.FontStyle.ITALIC,
6     fontWeight: workbook.FontWeight.BOLD
7 });
8 ...
9 // Add additional code

```

FontSize.size

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The numerical size of the font size.
Type	number
Module	N/workbook Module
Parent Object	workbook.FontSize
Sibling Object Members	FontSize Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myFontSize = workbook.createFontSize({ //myFontSize is a workbook.FontSize
4     size: 12,
5     unit: workbook.UNIT.PX

```

```
6  });
7
8  // myFontSize.size = 12 and is a workbook.FontSize.size
9  ...
10 // Add additional code
```

FontSize.unit

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The unit of the font size.
Type	string
Module	N/workbook Module
Parent Object	workbook.FontSize
Sibling Object Members	FontSize Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_UNIT	The value of this property is not included in the workbook.Unit enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1  // Add additional code
2  ...
3  var myFontSize = workbook.createFontSize({ //myFontSize is a workbook.FontSize
4    size: 12,
5    unit: workbook.UNIT.PX
6  });
7
8  // myFontSize.unit = workbook.UNIT.PX and is a workbook.FontSize.unit
9  ...
10 // Add additional code
```

workbook.Legend

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A legend object which can be used when you create a chart definition. Use workbook.createLegend(options) to create this object.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	Legend Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Legend based on a Section
4 var myLegend = workbook.createLegend({
5     axes: [myChartAxis],
6     root: mySection
7 });
8 // Create a basic Legend based on a DataDimension
9 var myLegend = workbook.createLegend({
10     axes: [myChartAxis],
11     root: myDataDimension
12 });
13 // Create a sorted Legend
14 var myLegend = workbook.createLegend({
15     axes: [myChartAxis],
16     root: mySection,
17     sortDefinitions: [mySortDefinition]
18 });
19
20 // View a workbook.Legend used in a Chart
21 var myWorkbook = workbook.load ({
22     id: myWorkbookId
23 });
24
25 // Note that some Legend properties may be empty/null based on the loaded workbook
26 var myLegend = myWorkbook.charts[0].legend;
27 log.audit({
28     title: 'Legend.axes = ',
29     details: myLegend.axes
30 });
31 log.audit({
32     title: 'Legend.root = ',
33     details: myLegend.root
34 });
35 log.audit({
36     title: 'Legend.sortDefinitions = ',
37     details: myLegend.sortDefinitions
38 });
39 ...
40 // Add additional code

```

Legend.axes

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The axes for the legend.
Type	workbook.ChartAxis[]

Module	N/workbook Module
Parent Object	workbook.Legend
Sibling Object Members	Legend Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.ChartAxis[] .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Legend.axes in a Chart
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Legend.axes = ',
10    details: myWorkbook.charts[0].legend.axes
11 });
12 ...
13 // Add additional code

```

Legend.root

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The root data (that is, fields) that defines the legend.
Type	workbook.DataDimension workbook.Section
Module	N/workbook Module
Parent Object	workbook.Legend
Sibling Object Members	Legend Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimension or a workbook.Section .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Legend.root in a Chart
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Legend.root = ',
10  details: myWorkbook.charts[0].legend.root
11 });
12 ...
13 // Add additional code
```

Legend.sortDefinitions

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort definitions of the legend.
Type	workbook.SortDefinition[]
Module	N/workbook Module
Parent Object	workbook.Legend
Sibling Object Members	Legend Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.SortDefinition[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Legend.sortDefinitions in a Chart
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
```

```
7 |
8 | log.audit({
9 |     title: 'Legend.sortDefinitions = ',
10 |     details: myWorkbook.charts[0].legend.sortDefinitions[0]
11 | });
12 | ...
13 | // Add additional code
```

workbook.LimitingFilter

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A limiting filter object which can be used when you create a pivot. Use workbook.createLimitingFilter(options) to create this object. A limiting filter object is used as a parameter in the workbook.createPivot(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	LimitingFilter Object Members
Since	2020.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 | // Add additional code
2 | ...
3 | var myLimitingFilter = workbook.createLimitingFilter({
4 |     row: true,
5 |     sortBy: [myMeasureSort],
6 |     limit: 12,
7 |     filteredNodesSelector: myDimensionSelector
8 | });
9 | ...
10 | // Add additional code
```

LimitingFilter.filteredNodesSelector

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The selections for the limiting filter.
Type	workbook.PathSelector workbook.DimensionSelector
Module	N/workbook Module

Parent Object	workbook.LimitingFilter
Sibling Object Members	LimitingFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.PathSelector or a workbook.DimensionSelector .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myLimitingFilter = workbook.createLimitingFilter({
4     row: true,
5     sortBy: [myMeasureSort],
6     limit: 12,
7     filteredNodesSelector: myDimensionSelector
8 });
9
10 var theFilteredNodesSelector = myLimitingFilter.filteredNodesSelector;
11 ...
12 // Add additional code

```

LimitingFilter.limit

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The limit number for the limiting filter.
Type	number
Module	N/workbook Module
Parent Object	workbook.LimitingFilter
Sibling Object Members	LimitingFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a number.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myLimitingFilter = workbook.createLimitingFilter({
4   row: true,
5   sortBy: [myMeasureSort],
6   limit: 12,
7   filteredNodesSelector: myDimensionSelector
8 });
9
10 var theLimit = myLimitingFilter.limit;
11 ...
12 // Add additional code
```

LimitingFilter.row

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The row axis indicator for the limiting filter.
Type	boolean
Module	N/workbook Module
Parent Object	workbook.LimitingFilter
Sibling Object Members	LimitingFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a boolean.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myLimitingFilter = workbook.createLimitingFilter({
4   row: true,
5   sortBy: [myMeasureSort],
6   limit: 12,
7   filteredNodesSelector: myDimensionSelector
8 });
```



```
9
10 var theRow = myLimitingFilter.row;
11 ...
12 // Add additional code
```

LimitingFilter.sortBys

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ordering elements of the limiting filter.
Type	Array< workbook.SortByDataDimensionItem workbook.SortByMeasure >
Module	N/workbook Module
Parent Object	workbook.LimitingFilter
Sibling Object Members	LimitingFilter Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.SortByDataDimensionItem or a workbook.SortByMeasure .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myLimitingFilter = workbook.createLimitingFilter({
4   row: true,
5   sortBy: [myMeasureSort],
6   limit: 12,
7   filteredNodesSelector: myDimensionSelector
8 });
9
10 var theSortBys = myLimitingFilter.sortBys;
11 ...
12 // Add additional code
```

workbook.MeasureSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A measure selector object.
--------------------	----------------------------

	Use workbook.createMeasureSelector(options) to create this object. A measure selector object is used as a parameter in the workbook.createMeasureValueSelector(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	MeasureSelector Object Members
Since	2021.2

MeasureSelector.measures

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The measures for the measure selector.
Type	workbook.CalculatedMeasure[] workbook.DataMeasure[]
Module	N/workbook Module
Parent Object	workbook.MeasureSelector
Sibling Object Members	MeasureSelector Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not an array of workbook.CalculatedMeasure objects or an array of workbook.DataMeasure objects, or the value specified is an empty array.

workbook.MeasureValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A measure and its value.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	MeasureValue Object Members
Since	2021.2

MeasureValue.measure

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The measure to use for the measure value.
Type	workbook.MeasureValue
Module	N/workbook Module
Parent Object	workbook.MeasureValue
Sibling Object Members	MeasureValue Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.MeasureValue object has been created.

MeasureValue.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The value to use for the measure value.
Type	string number boolean workbook.Record workbook.Currency workbook.Range workbook.Duration
Module	N/workbook Module
Parent Object	workbook.MeasureValue
Sibling Object Members	MeasureValue Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.MeasureValue object has been created.

workbook.MeasureValueSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A measure value selector.
---------------------------	---------------------------

	Use workbook.createMeasureValueSelector(options) to create this object. A measure value selector object is used as a parameter in the workbook.createReportStyle(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	MeasureValueSelector Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 //myMeasureValueSelector is a workbook.MeasureValueSelector object
4
5 var myMeasureValueSelector = nWorkbook.createMeasureValueSelector({
6   rowSelector: conditionalRowSelector, // previously created
7   workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector, or workbook.PathSelector
8   columnSelector: conditionalColumnSelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector,
9   or workbook.PathSelector
10  measureSelector: conditionalMeasureSelector // previously created workbook.MeasureSelector
11 });
12 ...
13 // Add additional code

```

MeasureValueSelector.columnSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The column selector.
Type	workbook.DimensionSelector workbook.PathSelector workbook.DescendantorSelfNodesSelector
Module	N/workbook Module
Parent Object	workbook.MeasureValueSelector
Sibling Object Members	MeasureValueSelector Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DimensionSelector , workbook.PathSelector , or workbook.DescendantorSelfNodesSelector object.

MeasureValueSelector.measureSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The measure selectors.
Type	workbook.MeasureSelector[]
Module	N/workbook Module
Parent Object	workbook.MeasureValueSelector
Sibling Object Members	MeasureValueSelector Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not an array of workbook.MeasureSelector objects.

MeasureValueSelector.rowSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The row selector.
Type	workbook.DimensionSelector workbook.PathSelector workbook.DependantorSelfNodesSelector
Module	N/workbook Module
Parent Object	workbook.MeasureValueSelector
Sibling Object Members	MeasureValueSelector Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DimensionSelector , workbook.PathSelector , or workbook.DependantorSelfNodesSelector object.

workbook.PathSelector

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A path selector object which can be used when you create a sort definition, a conditional filter, a limiting filter, or a measure sort.
---------------------------	---

	Use workbook.createPathSelector(options) to create this object. A path selector object is used as a parameter in the workbook.createConditionalFilter(options), workbook.createLimitingFilter(options), workbook.createSortDefinition(options), and workbook.createSortByMeasure(options) methods.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	PathSelector Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create an AllSubNodesSelector PathSelector
4 var myPathSelector = workbook.createPathSelector({
5     elements: [workbook.createAllSubNodesSelector()]
6 });
7 // Create a DimensionSelector PathSelector
8 var myPathSelector = workbook.createPathSelector({
9     elements: [myDimensionSelector]
10 });
11
12 // View a workbook.PathSelector used in a Pivotd definition
13 var myWorkbook = workbook.load ({
14     id: myWorkbookId
15 });
16
17 var myPathSelector = myWorkbook.pivots[0].aggregationFilters[0].filteredNodesSelector.elements;
18 log.audit({
19     title: 'PathSelector.elements = ',
20     details: myPathSelector.elements
21 });
22 ...
23 // Add additional code

```

PathSelector.elements

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The elements denoting 'xpath' of the path selector.
Type	workbook.DimensionSelector
Module	N/workbook Module
Parent Object	workbook.PathSelector
Sibling Object Members	PathSelector Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DimensionSelector .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View PathSelector.elements in a Pivot definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.debug ({
8   title: 'PathSelector.elements = ',
9   details: myWorkbook.pivots[0].aggregationFilters[0].filteredNodesSelector.elements
10 });
11 ...
12 // Add additional code

```

workbook.Pivot

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A pivot object which is a workbook component that enables you to pivot your dataset query results by defining measures and dimensions, so that you can analyze different subsets of data. Use workbook.createPivot(options) to create this object. A pivot object is used as a parameter in the workbook.create(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Pivot Object Members
Since	2020.2

Pivot.aggregationFilters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The limiting and conditional filters of the pivot definition.
Type	Array< workbook.LimitingFilter workbook.ConditionalFilter >

Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.LimitingFilter[] or a workbook.ConditionalFilter[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View the Pivot's aggregationFilters
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Pivot aggregationFilters = ',
10  details: myWorkbook.pivots[0].aggregationFilters
11 });
12 ...
13 // Add additional code

```

Pivot.columnAxis

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The column axis of the pivot definition.
Type	workbook.PivotAxis
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.PivotAxis .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View the Pivot's columnAxis
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Pivot columnAxis = ',
10    details: myWorkbook.pivots[0].columnAxis
11 });
12 ...
13 // Add additional code
```

Pivot.dataset

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The underlying dataset of the pivot definition.
Type	dataset.Dataset
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a dataset.Dataset .

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View the Pivot's dataset
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
```

```

9      title: 'Pivot dataset = ',
10     details: myWorkbook.pivots[0].dataset
11   });
12   ...
13   // Add additional code

```

Pivot.datasetLink

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Underlying dataset for the pivot.
Type	datasetLink.DatasetLink
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a datasetLink.DatasetLink .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View the Pivot's datasetLink
4   var myWorkbook = workbook.load({
5     id: myWorkbookId
6   });
7
8   log.audit({
9     title: 'Pivot.datasetLink = ',
10    details: myWorkbook.pivot[0].datasetLink
11  });
12  ...
13  // Add additional code

```

Pivot.filterExpressions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The filter expressions of the pivot definition.
Type	workbook.Expression[]

Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View the Pivot's filterExpressions
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Pivot.filterExpressions = ',
10    details: myWorkbook.pivots[0].filterExpressions[0]
11 });
12 ...
13 // Add additional code

```

Pivot.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the pivot definition.
Type	string
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View the Pivot's id
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Pivot id = ',
10    details: myWorkbook.pivots[0].id
11 });
12 ...
13 // Add additional code
```

Pivot.name

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of pivot definition.
Type	string
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View the Pivot's name
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
```

```

9      title: 'Pivot name = ',
10     details: myWorkbook.pivots[0].name
11   });
12   ...
13   // Add additional code

```

Pivot.portletName

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the portlet for the pivot.
Type	string workbook.Expression
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string or workbook.Expression

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View the Pivot's portlet name
4   var myWorkbook = workbook.load({
5     id: myWorkbookId
6   });
7
8   log.audit({
9     title: 'Pivot portlet name = ',
10    details: myWorkbook.pivot[0].portletName
11  });
12  ...
13  // Add additional code

```

Pivot.reportStyles

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Report styles for the pivot.
Type	workbook.ReportStyle[]

Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2021.1

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.ReportStyle .
NO_RULE_DEFINED	The workbook.ReportStyle array is empty.

Pivot.rowAxis

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The row axis of the pivot definition.
Type	workbook.PivotAxis
Module	N/workbook Module
Parent Object	workbook.Pivot
Sibling Object Members	Pivot Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.PivotAxis .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View the Pivot's rowAxis
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Pivot rowAxis = ',
10  details: myWorkbook.pivots[0].rowAxis

```

```

11  });
12  ...
13  // Add additional code

```

workbook.PivotAxis

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A pivot axis object which is used with you create a pivot definition. Use workbook.createPivotAxis(options) to create this object. A pivot axis object is used as a parameter in the workbook.createPivot(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	PivotAxis Object Members
Since	2020.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  // Create a basic Section-based PivotAxis
4  var myPivotAxis = workbook.createPivotAxis({
5      root: mySection
6  });
7
8  // Create a basic DataDimension-based PivotAxis
9  var myPivotAxis = workbook.createPivotAxis({
10     root: myDataDimension
11 });
12
13 // Create a sorted Section-based PivotAxis
14 var myPivotAxis = workbook.createPivotAxis({
15     root: mySection,
16     sortDefinitions: [mySortDefinition]
17 });
18
19 // Create a sorted DataDimension-based PivotAxis
20 var myPivotAxis = workbook.createPivotAxis({
21     root: myDataDimension,
22     sortDefinitions: [mySortDefinition]
23 });
24
25 // View a workbook.PivotAxis used in a Pivot definition
26 var myWorkbook = workbook.load ({
27     id: myWorkbookId
28 });
29
30 var myPivotAxis = pivots[0].rowAxis.root;
31

```

```
32 // Note that some PivotAxis properties may be empty/null based on the loaded workbook
33 log.audit({
34     title: 'PivotAxis root = ',
35     details: myPivotAxis.root
36 });
37 log.audit({
38     title: 'PivotAxis sortDefinitions = ',
39     details: myPivotAxis.sortDefinitions
40 });
41 ...
42 // Add additional code
```

PivotAxis.root

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The root data for the pivot axis.
Type	workbook.DataDimension workbook.Section
Module	N/workbook Module
Parent Object	workbook.PivotAxis
Sibling Object Members	PivotAxis Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimension or a workbook.Section .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View PivotAxis.root in a Pivot definition
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Pivot PivotAxis(rowAxis) root = ',
10    details: myWorkbook.pivots[0].rowAxis.root
11 });
12
13 log.audit({
14     title: 'Pivot PivotAxis (columnAxis) root = ',
15     details: myWorkbook.pivots[0].columnAxis.root
16 });
17 ...
18 // Add additional code
```


PivotAxis.sortDefinitions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort definitions of the pivot axis.
Type	workbook.SortDefinition[]
Module	N/workbook Module
Parent Object	workbook.PivotAxis
Sibling Object Members	PivotAxis Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.SortDefinition[] .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View PivotAxis.sortDefinitions in a Pivot definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Pivot PivotAxis(rowAxis) sortDefinitions = ',
10  details: myWorkbook.pivots[0].rowAxis.sortDefinitions
11 });
12
13 log.audit({
14   title: 'Pivot PivotAxis (columnAxis) sortDefinitions = ',
15   details: myWorkbook.pivots[0].columnAxis.sortDefinitions
16 });
17 ...
18 // Add additional code

```

workbook.PivotIntersection

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A row-column intersection in a pivot. An array of pivot intersections is returned from Workbook.runPivot(options) .
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	PivotIntersection Object Members
Since	2021.2

PivotIntersection.column

Applies to: [SuiteScript 2.x](#) | [APIs](#) | [SuiteCloud Developer](#)

Property Description	The column dimension value.
Type	workbook.DataDimensionValue workbook.SectionValue
Module	N/workbook Module
Parent Object	workbook.PivotIntersection
Sibling Object Members	PivotIntersection Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.PivotIntersection object has been created.

PivotIntersection.measureValues

Applies to: [SuiteScript 2.x](#) | [APIs](#) | [SuiteCloud Developer](#)

Property Description	The measure values in the pivot intersection.
Type	workbook.MeasureValue[]
Module	N/workbook Module
Parent Object	workbook.PivotIntersection
Sibling Object Members	PivotIntersection Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.PivotIntersection object has been created.

PivotIntersection.row

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The row dimension value.
Type	workbook.DataDimensionValue workbook.SectionValue
Module	N/workbook Module
Parent Object	workbook.PivotIntersection
Sibling Object Members	PivotIntersection Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.PivotIntersection object has been created.

workbook.PositionPercent

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A position that is defined using a percentage of x and y dimensions. Use workbook.createPositionPercent(options) to create this object. A position percent object is used as a parameter in the workbook.createStyle(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	PositionPercent Object Members
Since	2021.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // myPositionPercent is a workbook.PositionPercent object
4
5 var myPositionPercent = workbook.createPositionPercent({
6   percentX: 25,
7   percentY: 47

```

```

8  });
9  ...
10 // Add additional code

```

PositionPercent.percentX

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The percentage of the x dimension.
Type	number
Module	N/workbook Module
Parent Object	workbook.PositionPercent
Sibling Object Members	PositionPercent Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

PositionPercent.percentY

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The percentage of the y dimension.
Type	number
Module	N/workbook Module
Parent Object	workbook.PositionPercent
Sibling Object Members	PositionPercent Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

workbook.PositionUnits

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A position that is defined using numeric values and units.
---------------------------	--

	Use workbook.createPositionUnits(options) to create this object. A position units object is used as a parameter in the workbook.createStyle(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	PositionUnits Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // myPositionUnits is a workbook.PositionUnits object
4
5 var myPositionUnits = workbook.createPositionUnits ({
6     unit: workbook.Unit.CM,
7     x: 3,
8     y: 3
9 });
10 ...
11 // Add additional code

```

PositionUnits.unit

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The units for the position.
Type	string
Module	N/workbook Module
Parent Object	workbook.PositionUnits
Sibling Object Members	PositionUnits Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_UNIT	The value specified for this property is not included in the workbook.Unit enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

PositionUnits.x

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The x value of the position.
Type	number
Module	N/workbook Module
Parent Object	workbook.PositionUnits
Sibling Object Members	PositionUnits Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

PositionUnits.y

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The y value of the position.
Type	number
Module	N/workbook Module
Parent Object	workbook.PositionUnits
Sibling Object Members	PositionUnits Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a number.

workbook.PositionValues

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A position that is defined using position values, such as LEFT and CENTER.
---------------------------	--

	Use workbook.createPositionValues(options) to create this object. A position values object is used as a parameter in the workbook.createStyle(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	PositionValues Object Members
Since	2021.2

Syntax


Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 //myPositionValues is a workbook.PositionValues object
4
5 var myPositionValues = workbook.createPositionValues({
6     horizontal: workbook.Position.CENTER,
7     vertical: workbook.Position.LEFT
8 });
9 ...
10 // Add additional code

```

PositionValues.horizontal

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The horizontal value of the position.
Type	workbook.Position
Module	N/workbook Module
Parent Object	workbook.PositionValues
Sibling Object Members	PositionValues Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_POSITION	The value specified for this property is not included in the workbook.Position enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

PositionValues.vertical

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The vertical value of the position.
Type	workbook.Position
Module	N/workbook Module
Parent Object	workbook.PositionValues
Sibling Object Members	PositionValues Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_POSITION	The value specified for this property is not included in the workbook.Position enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

workbook.Range

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A date or date-time range. The dates in the range are formatted according to the user's preferences in their account. This object can be returned from a pivot execution.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	Range Object Members
Since	2021.2

Range.end

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The end date or date-time of the range. This date is formatted according to the user's preferences in their account.
Type	string

Module	N/workbook Module
Parent Object	workbook.Range
Sibling Object Members	Range Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Range object has been created.

Range.start

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The start date or date-time of the range. This date is formatted according to the user's preferences in their account.
Type	string
Module	N/workbook Module
Parent Object	workbook.Range
Sibling Object Members	Range Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Range object has been created.

workbook.Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A record. This object can be returned from a pivot execution.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Module	N/workbook Module
Methods and Properties	Record Object Members
Since	2021.2

Record.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the record type for the record.
Type	string
Module	N/workbook Module
Parent Object	workbook.Record
Sibling Object Members	Record Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Record object has been created.

Record.primaryKey

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The primary key of the record.
Type	number
Module	N/workbook Module
Parent Object	workbook.Record
Sibling Object Members	Record Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Record object has been created.

Record.properties

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The properties of the record.
Type	Object
Module	N/workbook Module
Parent Object	workbook.Record
Sibling Object Members	Record Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.Record object has been created.

workbook.RecordKey

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A record key. This object can be returned from a pivot execution. This object is used as a parameter in the workbook.createConstant(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	RecordKey Object Members
Since	2021.2

RecordKey.properties

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The properties of the record key.
Type	Object
Module	N/workbook Module
Parent Object	workbook.RecordKey

Sibling Object Members	RecordKey Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.RecordKey object has been created.

workbook.ReportStyle

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A report style object. Use workbook.createReportStyle(options) to create this object. A report style object is used as a parameter in the workbook.createPivot(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	ReportStyle Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 //myReportStyle is a workbook.ReportStyle object
4
5 var myReportStyle = workbook.createReportStyle({
6   selectors: [measureValueSelector], // previously created workbook.MeasureValueSelector objects
7   rules: [rule]                      // previously created workbook.ReportStyleRule objects
8 });
9 ...
10 // Add additional code

```

ReportStyle.rules

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The formatting rules for the report style.
-----------------------------	--

Type	workbook.ReportStyleRule[]
Module	N/workbook Module
Parent Object	workbook.ReportStyle
Sibling Object Members	ReportStyle Object Members
Since	2021.2

Errors

Error Code	Thrown If
NO_RULE_DEFINED	The value specified for this property is an empty array.
WRONG_PARAMETER_TYPE	The value specified for this property is not an array of workbook.ReportStyleRule objects.

ReportStyle.selectors

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The selectors for the report style.
Type	workbook.MeasureValueSelector[]
Module	N/workbook Module
Parent Object	workbook.ReportStyle
Sibling Object Members	ReportStyle Object Members
Since	2021.2

Errors

Error Code	Thrown If
NO_SELECTORS_DEFINED	The value specified for this property is an empty array.
WRONG_PARAMETER_TYPE	The value specified for this property is not an array of workbook.MeasureValueSelector object.

workbook.ReportStyleRule

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A rule for a report style. Use workbook.createReportStyleRule(options) to create this object. A report style rule object is used as a parameter in the workbook.createReportStyle(options) method.
--------------------	---

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	ReportStyleRule Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // myReportStyleRule is a workbook.ReportStyleRule object
4
5 var myReportStyleRule = workbook.createReportStyleRule({
6     expression: myExpression, // previously created workbook.Expression object
7     style: myReportStyle      // previously created workbook.Style object
8 });
9 ...
10 // Add additional code

```

ReportStyleRule.expression

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	A boolean expression indicating whether the style should be applied.
Type	workbook.Expression
Module	N/workbook Module
Parent Object	workbook.ReportStyleRule
Sibling Object Members	ReportStyleRule Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Expression object.

ReportStyleRule.style

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The style to be applied.
-----------------------------	--------------------------

Type	workbook.Style
Module	N/workbook Module
Parent Object	workbook.ReportStyleRule
Sibling Object Members	ReportStyleRule Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Style object.

workbook.Section

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A section object which can be used when you create a data dimension, a dimension selector, a pivot axis, or a pivot. Use workbook.createSection(options) to create this object. A section object is used as a parameter in the workbook.createDataDimension(options), workbook.createPivotAxis(options), workbook.createSection(options) methods.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Section Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySection = workbook.createSection({
4   children: [myDataDimension, myMeasure],
5   totalLine: workbook.TotalLine.HIDDEN
6 });
7 ...
8 // Add additional code

```

Section.children

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The children of the section.
Type	Array< workbook.CalculatedMeasure workbook.DataDimension workbook.DataMeasure workbook.Section >
Module	N/workbook Module
Parent Object	workbook.Section
Sibling Object Members	Section Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.CalculatedMeasure[] or a workbook.DataDimension[] or a workbook.DataMeasure[] or a workbook.Section[]

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySection = workbook.createSection({
4   children: [myDataDimension, myMeasure],
5   totalLine: workbook.TotalLine.HIDDEN
6 });
7
8 var theChildren = mySection.children;
9 ...
10 // Add additional code

```

Section.totalLine

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The format for the total line on a section.
Type	workbook.TotalLine
Module	N/workbook Module
Parent Object	workbook.Section
Sibling Object Members	Section Object Members

Since	2020.2
-------	--------

Errors

Error Code	Thrown If
INVALID_TOTAL_LINE	The value for this property is invalid. Set the value for this parameter using workbook.TotalLine .
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.TotalLine .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySection = workbook.createSection({
4     children: [myDataDimension, myMeasure],
5     totalLine: workbook.TotalLine.HIDDEN
6 });
7
8 var theTotalLine = mySection.totalLine;
9 ...
10 // Add additional code

```

workbook.SectionValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A section value object which wraps a workbook.Section object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	SectionValue Object Members
Since	2021.2

SectionValue.section

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The section of the section value.
Type	workbook.Section
Module	N/workbook Module

Parent Object	workbook.SectionValue
Sibling Object Members	SectionValue Object Members
Since	2021.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You try to set the value of this property after the workbook.SectionValue object has been created.

workbook.Series

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A series in a workbook. A series is used when you create a chart definition. Use workbook.createSeries(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	Series Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a Series
4 var mySeries = workbook.createSeries({
5     aspects: [myAspect]
6 });
7
8 // View a workbook.Series used in a Chart
9 var myWorkbook = workbook.load ({
10     id: myWorkbookId
11 });
12
13 var mySeries = myWorkbook.charts[0].series[0];
14
15 log.audit({
16     title: 'Series.aspect = ',
17     details: mySeries.aspects[0]
18 });
19 ...
20 // Add additional code

```

Series.aspects

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The aspects for the series.
Type	workbook.Aspect[]
Module	N/workbook Module
Parent Object	workbook.Series
Sibling Object Members	Series Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Aspect[] .

Syntax


Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Series.aspects in a Chart
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Series.aspects = ',
10    details: myWorkbook.charts[0].series[0].aspects[0]
11 });
12 ...
13 // Add additional code

```

workbook.Sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A sort object which is used when you create a table column, a dimension sort, or a measure sort. Use workbook.createSort(options) to create this object. A sort object is used as a parameter in the workbook.createSortByDataDimensionItem(options), and workbook.createTableColumn(options) methods.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	Sort Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a default Sort
4 var mySort = workbook.createSort({});
5
6 // Create a custom descending Sort
7 var mySort = workbook.createSort({
8     ascending: false,
9     caseSensitive: true,
10    nullsLast: true,
11    locale: query.SortLocale.US_EN
12 });
13
14 // View a workbook.Sort used in a Table definition
15 var myWorkbook = workbook.load ({
16     id: myWorkbookId
17 });
18
19 // Note that some Sort properties may be empty/null based on the loaded workbook
20 var mySort = myWorkbook.tables[0].columns[0].sort;
21
22 log.audit({
23     title: 'Sort.ascending = ',
24     details: mySort.ascending
25 });
26
27 log.audit({
28     title: 'Sort.caseSensitive = ',
29     details: mySort.caseSensitive
30 });
31
32 log.audit({
33     title: 'Sort.locale = ',
34     details: mySort.locale
35 });
36
37 log.audit({
38     title: 'Sort.nullsLast = ',
39     details: mySort.nullsLast
40 });
41 ...
42 // Add additional code

```

Sort.ascending

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ascending sort indicator of the sort.
-----------------------------	---

Type	boolean
Module	N/workbook Module
Parent Object	workbook.Sort
Sibling Object Members	Sort Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a boolean.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Sort.ascending in a Table definition
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7 log.audit({
8     title: 'Sort.ascending = ',
9     details: myWorkbook.tables[0].columns[0].sort.ascending
10 });
11 ...
12 // Add additional code

```

Sort.caseSensitive

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The indicator that determines if the sort is case sensitive.
Type	boolean
Module	N/workbook Module
Parent Object	workbook.Sort
Sibling Object Members	Sort Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a boolean.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Sort.caseSensitive in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.audit({
8   title: 'Sort.caseSensitive = ',
9   details: myWorkbook.tables[0].columns[0].sort.caseSensitive
10 });
11 ...
12 // Add additional code
```

Sort.locale

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The locale of the sort.
Type	query.Operator (read-only)
Module	N/workbook Module
Parent Object	workbook.Sort
Sibling Object Members	Sort Object Members
Since	2020.2

Errors

Error Code	Thrown If
READ_ONLY_PROPERTY	You attempted to set this property.
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Sort.locale in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.audit({
8   title: 'Sort.locale = ',
```

```

9      details: myWorkbook.tables[0].columns[0].sort.locale
10    });
11    ...
12    // Add additional code

```

Sort.nullsLast

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The indicator for placing nulls last of the sort.
Type	boolean
Module	N/workbook Module
Parent Object	workbook.Sort
Sibling Object Members	Sort Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a boolean.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1    // Add additional code
2    ...
3    // View Sort.nullsLast in a Table definition
4    var myWorkbook = workbook.load({
5      id: myWorkbookId
6    });
7    log.audit({
8      title: 'Sort.nullsLast = ',
9      details: myWorkbook.tables[0].columns[0].sort.nullsLast
10   });
11   ...
12   // Add additional code

```

Sort.order

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Sort order indicator.
Type	number
Module	N/workbook Module

Parent Object	workbook.Sort
Sibling Object Members	Sort Object Members
Since	2022.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a number.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Sort.nullsLast in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.audit({
8   title: 'Sort.order = ',
9   details: myWorkbook.tables[0].columns[0].sort.order
10 });
11 ...
12 // Add additional code

```

workbook.SortByDataDimensionItem

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A sort based on a data dimension item. Use workbook.createSortByDataDimensionItem(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	SortByDataDimensionItem Object Members
Since	2021.2

SortByDataDimensionItem.item

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The data dimension item to use for the sort.
----------------------	--

Type	workbook.DataDimensionItem
Module	N/workbook Module
Parent Object	workbook.SortByDataDimensionItem
Sibling Object Members	SortByDataDimensionItem Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DataDimensionItem object.

SortByDataDimensionItem.sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort to use.
Type	workbook.Sort
Module	N/workbook Module
Parent Object	workbook.SortByDataDimensionItem
Sibling Object Members	SortByDataDimensionItem Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Sort object.

workbook.SortByMeasure

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A sort based on a measure. Use workbook.createSortByMeasure(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	SortByMeasure Object Members

Since	2021.2
-------	--------

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 //mySortByMeasure is a workbook.SortByMeasure object
4
5 var mySortByMeasure = workbook.createSortByMeasure({
6     measure: myWorkbookMeasure, // previously created workbook.Measure object
7     otherAxisSelector: myAxisSelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector, or
      workbook.PathSelector object
8     selector: mySelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector, or
      workbook.PathSelector object
9
10     sort: mySort // previously created workbook.Sort object
11 });
12 ...
13 // Add additional code
```

SortByMeasure.measure

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The measure for the sort.
Type	workbook.CalculatedMeasure workbook.DataMeasure
Module	N/workbook Module
Parent Object	workbook.SortByMeasure
Sibling Object Members	SortByMeasure Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.CalculatedMeasure or workbook.DataMeasure object.

SortByMeasure.otherAxisSelector

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The selector for the axis that is not defined in the associated sort definition. In a sort definition, the sort is applied to a row or column. This property represents the selector for the other axis.
----------------------	---

Type	workbook.DescendantorSelfNodesSelector workbook.PathSelector workbook.DimensionSelector
Module	N/workbook Module
Parent Object	workbook.SortByMeasure
Sibling Object Members	SortByMeasure Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.DescendantorSelfNodesSelector , workbook.PathSelector , or workbook.DimensionSelector object.

SortByMeasure.sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort to use.
Type	workbook.Sort
Module	N/workbook Module
Parent Object	workbook.SortByMeasure
Sibling Object Members	SortByMeasure Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Sort object.

workbook.SortDefinition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A sort definition object which can be used with you create a pivot axis. Use workbook.createSortDefinition(options) to create this object. A sort definition object is used as a parameter in the workbook.createPivotAxis(options) method.
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	SortDefinition Object Members
Since	2020.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySortDefinition = workbook.createSortDefinition({
4     selector: myDimensionSelector,
5     sortBy: [myDimensionSort]
6 });
7 ...
8 // Add additional code

```

SortDefinition.selector

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The selector for the sort definition.
Type	workbook.DimensionSelector workbook.PathSelector
Module	N/workbook Module
Parent Object	workbook.SortDefinition
Sibling Object Members	SortDefinition Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a boolean.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySortDefinition = workbook.createSortDefinition({

```

```
4     selector: myDimensionSelector,  
5     sortBy: [myDimensionSort]  
6   });  
7  
8   var theSelector = mySortDefinition.selector;  
9   ...  
10  // Add additional code
```

SortDefinition.sortBys

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ordering elements for the sort definition.
Type	Array< workbook.SortByDataDimensionItem workbook.SortByMeasure >
Module	N/workbook Module
Parent Object	workbook.SortDefinition
Sibling Object Members	SortDefinition Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this parameter is not a workbook.SortByDataDimensionItem or a workbook.SortByMeasure .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1  // Add additional code  
2  ...  
3  var mySortDefinition = workbook.createSortDefinition({  
4    selector: myDimensionSelector,  
5    sortBy: [myDimensionSort]  
6  });  
7  
8  var theSortBys = mySortDefinition.sortBys;  
9  ...  
10 // Add additional code
```

workbook.Style

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A style object which defines attributes such as background color, font size, font style, text alignment, and more.
--------------------	--

	Use workbook.createStyle(options) to create this object. A style object is used as a parameter in the workbook.createConditionalFormatRule(options) and workbook.createReportStyle(options) methods. method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Style Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 //myNewStyle is a workbook.Style object
4
5 var myNewStyle = workbook.createStyle({
6     backgroundColor: workbook.Color.WHITE, // backgroundColor: 'WHITE' also works
7     backgroundImage: 'myStyleImage', // this value must be previously defined
8     color: workbook.Color.BLUE, // color: 'BLUE' also works
9 });
10 ...
11 // Add additional code

```

Style.backgroundColor

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The background color of the style.
Type	workbook.Color string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
UNSUPPORTED_COLOR	The value of this property is a string but it is not included in the workbook.Color enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Color object or a string.

Style.backgroundImage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The background image of the style.
Type	workbook.Image
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_ID_IMAGE	The value of this property is not included in the workbook.Image enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

Style.backgroundPosition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The background position of the style.
Type	workbook.PositionPercent workbook.PositionUnits workbook.PositionValues
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.PositionPercent , workbook.PositionUnits , or workbook.PositionValues object.

Style.color

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The color of the style.
-----------------------------	-------------------------

Type	workbook.Color string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
UNSUPPORTED_COLOR	The value of this property is a string but is not included in the workbook.Color enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Color object or a string.

Style.fontSize

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The font size of the style.
Type	workbook.FontSize string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_FONT_SIZE	The value of this property is a string but is not included in the workbook.FontSize enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.FontSize object or a string.

Style.fontStyle

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The font style of the style.
Type	string

Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_FONT_STYLE	The value of this property is not included in the workbook.FontStyle enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

Style.fontWeight

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The font weight of the style.
Type	string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_FONT_WEIGHT	The value of this property is not included in the workbook.FontWeight enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

Style.textAlign

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text alignment of the style.
Type	string
Module	N/workbook Module
Parent Object	workbook.Style

Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_TEXT_ALIGN	The value of this property is not included in the workbook.TextAlign enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

Style.textDecorationColor

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text decoration color of the style.
Type	workbook.Color string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
UNSUPPORTED_COLOR	The value of this property is a string but is not included in the workbook.Color enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Color object or a string.

Style.textDecorationLine

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text decoration line of the style.
Type	string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_TEXT_DECORATION_LINE	The value of this property is not included in the workbook.TextDecorationLine enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

Style.textDecorationStyle

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The text decoration style of the style.
Type	string
Module	N/workbook Module
Parent Object	workbook.Style
Sibling Object Members	Style Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_TEXT_DECORATION_STYLE	The value of this property is not included in the workbook.TextDecorationStyle enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

workbook.Table

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A table object which is a view of the table. Use workbook.createTable(options) to create this object. A table object is used as a parameter in the workbook.create(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Table Object Members
Since	2021.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 //myTable is a workbook.Table object
4
5 var myTable = nWorkbook.createTable({
6   columns: [column1, column2], // previously defined workbook.TableColumn objects
7   dataset: myDataset           // previously defined dataset
8   id: 'myTableId'
9   name: 'myTableName'
10 });
11 ...
12 // Add additional code
```

Table.columns

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The columns in the table view.
Type	workbook.TableColumn[]
Module	N/workbook Module
Parent Object	workbook.Table
Sibling Object Members	Table Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not an array of workbook.TableColumn objects.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Table.columns
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Table.columns = ',
```

```

10     details: myWorkbook.tables[0].columns[0]
11   });
12   ...
13   // Add additional code

```

Table.dataset

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The dataset for the table view.
Type	dataset.Dataset
Module	N/workbook Module
Parent Object	workbook.Table
Sibling Object Members	Table Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a dataset.Dataset object.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View Table.dataset
4   var myWorkbook = workbook.load({
5     id: myWorkbookId
6   });
7
8   log.audit({
9     title: 'Table.dataset = ',
10    details: myWorkbook.tables[0].dataset
11  });
12  ...
13  // Add additional code

```

Table.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the table view.
Type	string

Module	N/workbook Module
Parent Object	workbook.Table
Sibling Object Members	Table Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View Table.id
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'Table.id = ',
10    details: myWorkbook.tables[0].id
11 });
12 ...
13 // Add additional code

```

Table.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the table view.
Type	string workbook.Expression
Module	N/workbook Module
Parent Object	workbook.Table
Sibling Object Members	Table Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string or a workbook.Expression object.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View Table.name
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'Table.name = ',
10  details: myWorkbook.tables[0].name
11 });
12 ...
13 // Add additional code
```

workbook.TableColumn

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A table column object which is used when you create a table definition. Use workbook.createTableColumn(options) to create this object. A table column object is used as a parameter in the workbook.createTable(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	TableColumn Object Members
Since	2020.2

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#).

```
1 // Add additional code
2 ...
3 // Create a basic TableColumn
4 var myTableColumn = workbook.createTableColumn({
5   sort: mySort
6   datasetColumnId: myDatasetColumnId
7 });
8 // Create a TableColumn with filters
9 var myTableColumn = workbook.createTableColumn({
10  filters: [myTableFilter],
11  sort: mySort,
```

```
12     datasetColumnId: myDatasetColumnId
13   });
14   // Create a full TableColumn
15   var myTableColumn = workbook.createTableColumn({
16     filters: [myTableFilter],
17     width: 50,
18     datasetColumnAlias: 'Column7',
19     fieldContext: myFieldContext,
20     label: 'My Complex Table Column',
21     alias: 'myComplexTableColumn',
22     sort: mySort
23   });
24
25   // View a workbook.TableColumn used in a Table definition
26   var myWorkbook = workbook.load ({
27     id: myWorkbookId
28   });
29
30   var myTableColumn = myWorkbook.tables[0].columns[0];
31
32   // Note that some TableColumn properties may be empty/null based on the loaded workbook
33   log.audit({
34     title: 'TableColumn.datasetColumnAlias = ',
35     details: myTableColumn.datasetColumnAlias
36   });
37
38   log.audit({
39     title: 'TableColumn.sort = ',
40     details: myTableColumn.sort
41   });
42
43   log.audit({
44     title: 'TableColumn.filters = ',
45     details: myTableColumn.filters
46   });
47
48   log.audit({
49     title: 'TableColumn.label = ',
50     details: myTableColumn.label
51   });
52
53   log.audit({
54     title: 'TableColumn.width = ',
55     details: myTableColumn.width
56   });
57
58   log.audit({
59     title: 'TableColumn.fieldContext = ',
60     details: myTableColumn.fieldContext
61   });
62
63   log.audit({
64     title: 'TableColumn.alias = ',
65     details: myTableColumn.alias
66   });
67   ...
68   // Add additional code
```

TableColumn.alias

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The alias for the column.
Type	string
Module	N/workbook Module

Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View TableColumn.alias in a Table definition
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'TableColumn.alias = ',
10    details: myWorkbook.tables[0].columns[0].alias
11 });
12 ...
13 // Add additional code

```

TableColumn.datasetColumnAlias

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The alias of the dataset column from which the table column was created.
Type	string
Module	N/workbook Module
Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View TableColumn.datasetColumnAlias in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'TableColumn.datasetColumnAlias = ',
10  details: myWorkbook.tables[0].columns[0].datasetColumnAlias
11 });
12 ...
13 // Add additional code
```

TableColumn.fieldContext

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The field context specification for the field used in the table column.
Type	workbook.FieldContext
Module	N/workbook Module
Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.FieldContext .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View TableColumn.fieldContext in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
```

```

9      title: 'TableColumn.fieldContext = ',
10     details: myWorkbook.tables[0].columns[0].fieldContext
11   });
12   ...
13   // Add additional code

```

TableColumn.filters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The filters for the table column.
Type	workbook.TableColumnFilter
Module	N/workbook Module
Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.TableColumnFilter .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1   // Add additional code
2   ...
3   // View TableColumn.filters in a Table definition
4   var myWorkbook = workbook.load({
5     id: myWorkbookId
6   });
7
8   log.audit({
9     title: 'TableColumn.filters = ',
10    details: myWorkbook.tables[0].columns[0].filters[0]
11  });
12  ...
13  // Add additional code

```

TableColumn.label

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The label of table column.
Type	string

Module	N/workbook Module
Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View TableColumn.label in a Table definition
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7
8 log.audit({
9     title: 'TableColumn.label = ',
10    details: myWorkbook.tables[0].columns[0].label
11 });
12 ...
13 // Add additional code

```

TableColumn.sort

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The sort of the table column.
Type	workbook.Sort
Module	N/workbook Module
Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Sort .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View TableColumn.sort in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
8 log.audit({
9   title: 'TableColumn.sort = ',
10  details: myWorkbook.tables[0].columns[0].sort
11 });
12 ...
13 // Add additional code
```

TableColumn.width

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The width of the table column when displayed in the UI.
Type	number
Module	N/workbook Module
Parent Object	workbook.TableColumn
Sibling Object Members	TableColumn Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a number.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View TableColumn.width in a Table definition
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7
```

```

8 | log.audit({
9 |     title: 'TableColumn.width = ',
10 |     details: myWorkbook.tables[0].columns[0].width
11 | });
12 | ...
13 | // Add additional code

```

workbook.TableColumnCondition

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Condition for a table view column. Use workbook.createTableColumnCondition(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/workbook Module
Methods and Properties	TableColumnCondition Object Members
Since	2021.1

TableColumnCondition.filters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The filters for the condition.
Type	workbook.TableColumnFilter[]
Module	N/workbook Module
Parent Object	workbook.TableColumnFilter
Sibling Object Members	TableColumnCondition Object Members
Since	2021.1

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The parameter type is incorrect.

TableColumnCondition.operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The operator for the condition.
-----------------------------	---------------------------------

Type	string
Module	N/workbook Module
Parent Object	workbook.TableColumnCondition
Sibling Object Members	TableColumnCondition Object Members
Since	2021.1

Errors

Error Code	Thrown If
INVALID_OPERATOR	The operator is not valid.
WRONG_PARAMETER_TYPE	The parameter type is incorrect.

workbook.TableColumnFilter

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A table column filter object. Use workbook.createTableColumnFilter(options) to create this object. A table column filter object is used as a parameter in the workbook.createConditionalFormatRule(options) method.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	TableColumnFilter Object Members
Since	2021.2

TableColumnFilter.operator

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The operator for the table column filter.
Type	string
Module	N/workbook Module
Parent Object	workbook.TableColumnFilter
Sibling Object Members	TableColumnFilter Object Members
Since	2021.2

Errors

Error Code	Thrown If
INVALID_OPERATOR	The value of this property is not included in the query.Operator enum.
WRONG_PARAMETER_TYPE	The value specified for this property is not a string.

TableColumnFilter.values

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The values for the table column filter.
Type	Array<null Object boolean number string Date>
Module	N/workbook Module
Parent Object	workbook.TableColumnFilter
Sibling Object Members	TableColumnFilter Object Members
Since	2021.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not an array, or a value inside the array is not supported. Supported values are null, Object, boolean, number, string, and Date.

workbook.Workbook

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A workbook object. Workbooks are where you analyze the results of your dataset queries using different components, such as table views and pivots. All workbooks are based on a dataset, and a single dataset can be used as the basis for multiple workbooks. A workbook can include tables and pivots. Use workbook.create(options) to create this object.
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/workbook Module
Methods and Properties	Workbook Object Members

Since	2020.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1  / Add additional code
2  ...
3  var myWorkbook = workbook.create({
4      description: 'A sample workbook',
5      name: 'My Workbook',
6      pivots: [myPivot],
7      tables: [myTable]
8  });
9  ...
10 // Add additional code

```

Workbook.runPivot(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Runs the pivot and returns an array of pivot intersections.
Returns	workbook.PivotIntersection[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units for each intersection returned
Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2021.2

Parameters

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the pivot.

Errors

Error Code	Thrown If
PIVOT_DOES_NOT_EXIST	The pivot specified by the options.id parameter is not included in the workbook.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myWorkbook = workbook.load({
4   id: myWorkbookId,
5 });
6 var myResultSet = myWorkbook.runPivot({
7   id: myTableId,
8 });
9 log.debug({
10  title: 'My Run Pivot Result Set: ',
11  details: myResultSet
12 });
13 ...
14 // Add additional code
```

Workbook.charts

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Charts that can be included in a workbook when you create a new workbook.
Type	workbook.Chart[]
Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Chart[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View a workbook's chart
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.audit({
```

```

8     title: 'Workbook.chart = ',
9     details: myWorkbook.charts[0]
10  });
11  ...
12  // Add additional code

```

Workbook.description

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The description of the workbook. This is set when you create a workbook.
Type	string
Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  // View a workbook's description
4  var myWorkbook = workbook.load({
5      id: myWorkbookId
6  });
7  log.audit({
8      title: 'Workbook.description = ',
9      details: myWorkbook.description
10  });
11  ...
12  // Add additional code

```

Workbook.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The ID of the workbook, that is set when you create a workbook.
Type	string

Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // View a workbook's id
4 var myWorkbook = workbook.load({
5     id: myWorkbookId
6 });
7 log.audit({
8     title: 'Workbook.id = ',
9     details: myWorkbook.id
10 });
11 ...
12 // Add additional code

```

Workbook.name

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of the workbook.
Type	string
Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value for this property is not a string.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View a workbook's name
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.audit({
8   title: 'Workbook.name = ',
9   details: myWorkbook.name
10 });
11 ...
12 // Add additional code
```

Workbook.pivots

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Pivot definitions that can be included in a workbook when you create a new workbook.
Type	workbook.Pivot[]
Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2020.2

Errors

Error Code	Thrown If
WRONG_PARAMETER_TYPE	The value specified for this property is not a workbook.Pivot[] .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // View a workbook's pivot
4 var myWorkbook = workbook.load({
5   id: myWorkbookId
6 });
7 log.audit({
```

```

8     title: 'Workbook.pivot = ',
9     details: myWorkbook.pivots[0]
10  });
11  ...
12  // Add additional code

```

Workbook.tables

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Table definitions that can be included in a workbook when you create a new workbook.
Type	workbook.Table[]
Module	N/workbook Module
Parent Object	workbook.Workbook
Sibling Object Members	Workbook Object Members
Since	2020.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  // View a workbook's table definition
4  var myWorkbook = workbook.load({
5      id: myWorkbookId
6  });
7  log.audit({
8      title: 'Workbook.tables = ',
9      details: myWorkbook.tables[0]
10 });
11 ...
12 // Add additional code

```

workbook.create(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new workbook. Workbooks are where you analyze the results of your dataset queries using different components, such as table views and pivots. All workbooks are based on a dataset, and a single dataset can be used as the basis for multiple workbooks. A workbook can include an ID, a name, a description, pivots, and tables.
Returns	workbook.Workbook
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.charts	workbook.Chart[]	optional	The c
options.description	string	optional	The description of the workbook.
options.name	string	optional	The name of the workbook.
options.pivots	workbook.Pivot[]	optional	The pivots to include in the workbook.
options.tables	workbook.Table[]	optional	The tables to include in the workbook.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myWorkbook = workbook.create({
4     description: 'A sample workbook',
5     name: 'My Workbook',
6     pivots: [myPivot],
7     tables: [myTable]
8 });
9 ...
10 // Add additional code

```

workbook.createAspect(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an aspect for a chart series. An aspect includes a measure and an aspect type.
Returns	workbook.Aspect
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.measure	workbook.CalculatedMeasure workbook.DataMeasure	required	The measure of the chart series.
options.type	workbook.AspectType	optional	The aspect type of the chart series.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // Create a basic Aspect
4 var myAspect = workbook.createAspect({
5     measure: myMeasure,
6 });
7
8 // Create a complete Aspect
9 var myAspect = workbook.createAspect({
10     measure: myMeasure,
11     type: workbook.AspectType.COLOR
12 });
13 ...
14 // Add additional code
```

workbook.createCalculatedMeasure(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a calculated measure.
Returns	workbook.CalculatedMeasure
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.expression	workbook.Expression	required	The expression for the calculated measure.
options.label	string workbook.Expression	optional	The label for the calculated measure.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myCalculatedMeasure = workbook.createCalculatedMeasure({
4     expression: measureExpression, // previously created workbook.Expression
5     label: 'Measure Label'
6 });
7 ...
8 // Add additional code

```

workbook.createCategory(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a chart category, which includes an axis, a data root, and a sort definition. A chart category is used in a workbook.Chart .
Returns	workbook.Category
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members

Since	2020.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.axis	workbook.ChartAxis	required	The chart category axis definition.
options.root	workbook.DataDimension workbook.Section	required	The data that feeds the chart category.
options.sortDefinitions	workbook.SortDefinition []	optional	The sorting for the chart category.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a Category based on a Section
4 var myCategory = workbook.createCategory({
5     axis: myChartAxis,
6     root: mySection,
7     sortDefinitions: [mySortDefinition]
8 });
9
10 // Create a Category based on a DataDimension
11 var myCategory = workbook.createCategory({
12     axis: myChartAxis,
13     root: myDataDimension,
14     sortDefinitions: [mySortDefinition]
15 });
16 ...
17 // Add additional code

```

workbook.createChart(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a chart , A chart is a workbook component that enables you to visualize your dataset query results using predefined chart and graph types, such as line graphs and bar charts. A chart is built from an underlying dataset and can also include a category, a legend, series, a type, expressions, filters, stacking behavior indicators, along with an ID, a name, a title, and a subtitle. For more information about charts in SuiteAnalytics, see the help topic Workbook Charts.
Returns	workbook.Chart

Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.aggregationFilters	Array< workbook.ConditionalFilter workbook.LimitingFilter >	optional	The set of conditional filters or limiting filters that are applied to the chart. When multiple filters are defined, they are aggregated together to form a single compounded filter.
options.category	workbook.Category	required	The category of the chart.
options.dataset	dataset.Dataset	required	The underlying dataset that the chart is based on. A chart can include only the data (fields) that are included in the dataset.
options.datasetLink	datasetLink.DatasetLink	required	The datasetLink on which the chart is based
options.filterExpressions	workbook.Expression []	optional	The simple, non-aggregated value-based filters for the chart.
options.id	string	required	The ID of the chart.
options.legend	workbook.Legend	required	The legend for the chart.
options.name	string	required	The name of the chart.
options.portletName	string workbook.Expression	required	The name of the portlet for the chart.
options.series	workbook.Series []	required	The set of series for the chart. Each series includes information about how each data point is calculated. For example, a series can represent the sum of transaction amounts for a customer in a particular month.
options.stacking	workbook.Stacking	optional	The stacking type that describes how the chart data is stacked.
options.subTitle	string workbook.Expression	optional	The subtitle of the chart.
options.title	string workbook.Expression	optional	The title of the chart.

Parameter	Type	Required / Optional	Description
options.type	workbook.ChartType	required	The type of the chart.

Errors

Error Code	Thrown If
INVALID_CHART_TYPE	The value specified for the options.type parameter is invalid. Set this value using workbook.ChartType
INVALID_STACKING_TYPE	The value specified for the options.stacking parameter is invalid. Set this value using workbook.Stacking .

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Chart
4 var myChart = workbook.createChart({
5     name: 'myChart',
6     id: '_myChart',
7     type: workbook.ChartType.BAR,
8     category: myCategory,
9     legend: myLegend,
10    series: [mySeries],
11    dataset: myDataset
12 });
13
14 // Create a comprehensive Chart
15 var myChart = workbook.createChart({
16     name: 'myChart',
17     title: 'My Chart Title',
18     subTitle: 'My Chart Subtitle',
19     id: '_myChart',
20     type: workbook.ChartType.BAR,
21     stacking: workbook.Stacking.PERCENT,
22     category: myCategory,
23     legend: myLegend,
24     series: [mySeries],
25     filterExpressions: [myExpression],
26     aggregationFilters: [myLimitingFilter],
27     dataset: myDataset,
28     datasetLink: myDatasetLink,
29     portletName: myPortlet
30 });
31 ...
32 // Add additional code

```

workbook.createChartAxis(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an X-axis or a Y-axis for the chart.
--------------------	--

Returns	workbook.ChartAxis
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.title	string	required	The title of the chart axis.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myChartAxis = workbook.createChartAxis({
4     title: 'My Chart Axis'
5 });
6 ...
7 // Add additional code

```

workbook.createColor(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a color.
Returns	workbook.Color
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members

Since	2021.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.alpha	number	optional	The alpha portion (opacity or transparency) of the color.
options.blue	number	optional	The blue portion of the color.
options.green	number	optional	The green portion of the color.
options.red	number	optional	The red portion of the color.

Errors

Error Code	Thrown If
INVALID_COLOR_VALUE	The value of any of the options.blue, options.green, or options.red parameters is less than 0 or greater than 255.
INVALID_INVALID_ALPHA_VALUE	The value of the options.alpha parameter is less than 0 or greater than 1.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myColor = workbook.createColor({
4   alpha: 0.4,
5   blue: 255,
6   green: 192,
7   red: 203
8 });
9 ...
10 // Add additional code

```

workbook.createComplexRecordKey

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a complex RecordKey object from another object.
Returns	workbook.RecordKey workbook.RecordKey
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options	Object	required	Properties of the record key (e. g. {numberkey: 1, stringkey:"a"})

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a ConditionalFilter
4 var myComplexRecordKey = workbook.createComplexRecordKey ({
5     numberkey: '1',
6     stringkey: 'a'
7 });
8 ...
9 // Add additional code

```

workbook.createConditionalFilter(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a conditional filter, which includes a selector of what to filter, a row axis and other axis, a measure and a predicate. Conditional filters can be used in pivot definitions.
Returns	workbook.ConditionalFilter
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.filteredNodesSelector	workbook.DimensionSelector workbook.PathSelector	required	The selector for the conditional filter.
options.measure	workbook.CalculatedMeasure workbook.DataMeasure	required	The measure for the conditional filter.
options.otherAxisSelector	workbook.DimensionSelector workbook.PathSelector	required	The other axis selector for the conditional filter.
options.predicate	workbook.Expression	required	The predicate for the conditional filter.
options.row	boolean	required	Indicator for a row axis filter. If set to false, the filter is on a column axis.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a ConditionalFilter
4 var myConditionalFilter = workbook.createConditionalFilter({
5     row: true,
6     otherAxisSelector: myDimensionSelector, // previously created workbook.PathSelector or workbook.DimensionSelector object
7     filteredNodesSelector: myPathSelector, // previously created workbook.PathSelector or workbook.DimensionSelector object
8     measure: myMeasure, // previously created workbook.Measure object
9     predicate: myExpression // previously created workbook.Expression object
10 });
11 ...
12 // Add additional code

```

workbook.createConditionalFormat(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a conditional format.
Returns	workbook.ConditionalFormat
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module

Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.rules	workbook.ConditionalFormatRule []	required	The style rules to use for this conditional format.

Errors

Error Code	Thrown If
NO_RULE_DEFINED	The options.rules parameter is an empty array.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myConditionalFormat = workbook.createConditionalFormat({
4   rules: [formatrules] // array of previously created workbook.ConditionalFormatRule
5 });
6 ...
7 // Add additional code

```

workbook.createConditionalFormatRule(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a conditional format rule.
Returns	workbook.ConditionalFormatRule
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.filter	workbook.TableColumnFilter	required	A filter indicating when the conditional format rule should be applied.
options.style	workbook.Style	required	The style to use for the conditional format rule.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myConditionalFormatRule = workbook.createConditionalFormatRule({
4     filter: formatFilter, // previously created workbook.TableColumnFilter
5     style: formatStyle    // previously created workbook.Style
6 });
7 ...
8 // Add additional code

```

workbook.createConstant(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a constant expression.
Returns	workbook.Expression
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.constant	string number boolean Date	required	The ID of the function to use.

Parameter	Type	Required / Optional	Description
options.type	workbook.ConstantType	optional	The type of constant.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myBooleanConstant = workbook.createConstant({
4     constant: true,
5     type: workbook.ConstantType.BOOLEAN
6 });
7
8 var myCurrencyConstant = workbook.createConstant({
9     constant: 10,
10    type: workbook.ConstantType.CURRENCY
11 });
12
13 var myDateConstant = workbook.createConstant({
14     constant: new Date(),
15     type: workbook.ConstantType.DATE
16 });
17
18 var myDateTimeConstant = workbook.createConstant({
19     constant: new Date(),
20     type: workbook.ConstantType.DATE_TIME
21 });
22
23 var myDurationConstant = workbook.createConstant({
24     constant: 15,
25     type: workbook.ConstantType.DURATION
26 });
27
28 var myNumberConstant = workbook.createConstant({
29     constant: 77,
30     type: workbook.ConstantType.NUMBER
31 });
32
33 var myTextConstant = workbook.createConstant({
34     constant: 'Sample Text',
35     type: workbook.ConstantType.TEXT
36 });
37 ...
38 // Add additional code

```

workbook.createCurrency(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a currency.
Returns	workbook.Currency
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.amount	number	required	The amount of the currency.
options.id	string	required	The ID of the currency (for example, USD, EUR, GBP, and so on).

Errors

Error Code	Thrown If
INVALID_CURRENCY	The provided options.id is not valid.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myCurrency = workbook.createCurrency({
4     amount: 1,
5     id: "USD"
6 });
7 ...
8 // Add additional code

```

workbook.createDataDimension(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a data dimension, which includes items, child data items, and a total line. A data dimension is used in a workbook.PivotAxis , a workbook.DimensionSelector , and a workbook.Section .
Returns	workbook.DataDimension
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.children	Array< workbook.DataDimension workbook.Section workbook.CalculatedMeasure workbook.DataMeasure >	optional	The child data dimensions or sections of the data dimension.
options.items	workbook.DataDimensionItem []	required	The items of the data dimension.
options.totalLine	workbook.TotalLine	optional	The predefined formats for the total line.

Errors

Error Code	Thrown If
INVALID_TOTAL_LINE	The value specified for the options.totalLine parameter is invalid. Set this parameter value using workbook.TotalLine .
NO_DIMENSION_ITEM_DEFINED	The options.items parameter is empty.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic DataDimension
4 var myDataDimension = workbook.createDataDimension({
5     items: [myDataDimensionItem]
6 });
7
8 // Create a comprehensive DataDimension
9 var myDataDimension = workbook.createDataDimension({
10     totalLine: workbook.TotalLine.HIDDEN,
11     items: [myDataDimensionItem],
12     children: [mySection]
13 });
14 ...
15 // Add additional code

```

workbook.createDataDimensionItem(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a data dimension item, which includes an expression and a label.
Returns	workbook.DataDimensionItem
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.expression	workbook.Expression	required	The expression for the data dimension item.
options.label	string	optional	The label for the data dimension item.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a DataDimensionItem
4 var myDataDimensionItem = workbook.createDataDimensionItem({
5     label: 'my DataDimensionItem Label',
6     expression: myExpression
7 });
8 ...
9 // Add additional code

```

workbook.createDataMeasure(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a data measure.
---------------------------	-------------------------

Returns	workbook.DataMeasure
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.aggregation	workbook.Aggregation	required	The type of aggregation to use for the data measure.
options.expression	workbook.Expression	required for single-expression measures (in which options.expressions is not used)	The expression for the data measure (if the data measure is a single-expression measure).
options.expressions	workbook.Expression[]	required for multi-expression measures (in which options.expression is not used)	The expressions for the data measure (if the data measure is a multi-expression measure).
options.label	string workbook.Expression	optional	The label for the data measure.

Errors

Error Code	Thrown If
AT_LEAST_ONE_EXPRESSION_IS_NEEDED	The options.expression parameter is not provided and the options.expressions parameter is an empty array.
MUTUALLY_EXCLUSIVE_ARGUMENTS	Both options.expression and options.expressions are provided.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myDataMeasure = workbook.createDataMeasure({

```

```
4 aggregation: 'DataCount',
5 expression: singleExpression, // previously created workbook.Expression
6 label: 'MeasureLabel'
7 });
8 ...
9 // Add additional code
```

workbook.createDimensionSelector(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a dimension selector.
Returns	workbook.DimensionSelector
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.dimension	workbook.DataDimension workbook.Section	required	The dimension of the selector (the dimension to be selected).

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // Create a DataDimension DimensionSelector
4 var myDimensionSelector = workbook.createDimensionSelector({
5   dimension: myDataDimension
6 });
7
8 // Create a Section DimensionSelector
9 var myDimensionSelector = workbook.createDimensionSelector({
10   dimension: mySection
11 });
12 ...
13 // Add additional code
```


workbook.createDuration(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an duration.
Returns	workbook.Duration
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.start	string	required	Start date or date time. Use the workbook.TemporalUnit enum to set the value.
options.end	string	required	End date or date time. Use the workbook.TemporalUnit enum to set the value.

Errors

Error Code	Thrown If
INVALID_TEMPORAL_UNIT	If options.start or options.end values are not valid temporal unit values.

workbook.createExpression(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an expression, that includes a function ID and parameters. Expressions can be used to create a pivot definition, a data dimension item, a measure, a conditional filter, and a dimension sort.
Returns	workbook.Expression
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.functionId	workbook.ExpressionType	required	The function for the expression.
options.parameters	Object	optional	The parameters for the expression.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Expression
4 var myExpression = workbook.createExpression({
5     functionId: query.Operator.EMPTY
6 });
7
8 // Create a complete Expression
9 var myExpression = workbook.createExpression({
10     functionId: query.Operator.EQUAL,
11     parameters: {
12         parm1: 15
13     }
14 });
15 ...
16 // Add additional code

```

workbook.createFieldContext(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a field context for a table definition column.
Returns	workbook.FieldContext
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The name of the field context.
options.parameters	Object	optional	The parameters for the field context, as a key: value pair.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic FieldContext
4 var myFieldContext = workbook.createFieldContext({
5     name: 'My Field Context'
6 });
7
8 // Create a complete FieldContext
9 var myFieldContext = workbook.createFieldContext({
10     name: 'My Field Context',
11     parameters: {
12         parm1: 'ABC'
13     }
14 });
15 ...
16 // Add additional code

```

workbook.createFontSize(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a font size defined using units.
Returns	workbook.FontSize
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module

Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.size	number	required	The numeric value of the font size.
options.unit	workbook.Unit	required	The units of the font size.

Errors

Error Code	Thrown If
INVALID_UNIT	The value of the options.unit parameter is not included in the workbook.Unit enum.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myFontSize = workbook.createFontSize({
4     size: 12,
5     unit: workbook.UNIT.PX
6 });
7 ...
8 // Add additional code
```

workbook.createLegend(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a chart legend.
Returns	workbook.Legend
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members

Since	2020.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.axes	workbook.ChartAxis[]	required	The axes for the legend.
options.root	workbook.Section workbook.DataDimension	required	The data for the legend.
options.sortDefinitions	workbook.SortDefinition[]	optional	The sort definition for the corresponding legend axes.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Legend based on a Section
4 var myLegend = workbook.createLegend({
5     axes: [myChartAxis],
6     root: mySection
7 });
8 // Create a basic Legend based on a DataDimension
9 var myLegend = workbook.createLegend({
10    axes: [myChartAxis],
11    root: myDataDimension
12 });
13 // Create a sorted Legend
14 var myLegend = workbook.createLegend({
15    axes: [myChartAxis],
16    root: mySection,
17    sortDefinitions: [mySortDefinition]
18 });
19 ...
20 // Add additional code

```

workbook.createLimitingFilter(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a limiting filter, which includes a selector of what to filter, a row axis, a limit, and a sorting order. Limiting filters can be used in pivot definitions to limit the data shown on a pivot.
Returns	workbook.LimitingFilter
Supported Script Types	Server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.filteredNodesSelector	workbook.DimensionSelector workbook.PathSelector	required	The selector for the limiting filter.
options.limit	number	required	The limit for the limiting filter.
options.row	boolean	required	Indicator for a row axis filter. If set to false, the filter is on a column axis.
options.sortBys	Array< workbook.SortByDataDimensionItem workbook.SortByMeasure >	required	The sort order for the limiting filter.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a DimensionSelector LimitingFilter sorting by MeasureSort
4 var myLimitingFilter = workbook.createLimitingFilter({
5     row: true,
6     sortBy: [myMeasureSort],
7     limit: 12,
8     filteredNodesSelector: myDimensionSelector
9 });
10
11 // Create a PathSelector LimitingFilter sorting by DimensionSort
12 var myLimitingFilter = workbook.createLimitingFilter({
13     row: true,
14     sortBy: [myDimensionSort],
15     limit: 12,
16     filteredNodesSelector: myPathSelector
17 });
18
19 // Create a AllSubNodesSelector Limiting Filter
20 var myLimitingFilter = workbook.createLimitingFilter({
21     row: true,
22     sortBy: [myDimensionSort],
23     limit: 12,
24     filteredNodesSelector: mySelector
25 });
26 ...
27 // Add additional code

```

workbook.createMeasureSelector(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a measure selector.
Returns	workbook.MeasureSelector
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.measures	Array< workbook.CalculatedMeasure workbook.DataMeasure >	required	The measures for the measure selector.

Errors

Error Code	Thrown If
NO_MEASURE_DEFINED	The options.measures parameter is an empty array.

workbook.createMeasureValueSelector(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a measure value selector.
Returns	workbook.MeasureValueSelector
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members

Since	2021.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.columnSelector	workbook.DescendantorSelfNodesSelector workbook.DimensionSelector workbook.PathSelector	required	The column to style.
options.measureSelector	workbook.MeasureSelector	required	The measures to apply conditional formatting to.
options.rowSelector	workbook.DescendantorSelfNodesSelector workbook.DimensionSelector workbook.PathSelector	required	The row to style.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var measureValueSelector = nWorkbook.createMeasureValueSelector({
4   rowSelector: conditionalRowSelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector,
   or workbook.PathSelector
5   columnSelector: conditionalColumnSelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector,
   or workbook.PathSelector
6   measureSelector: conditionalMeasureSelector // previously created workbook.MeasureSelector
7 });
8 ...
9 // Add additional code
```

workbook.createPathSelector(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a path selector.
Returns	workbook.PathSelector
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members

Since	2020.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.elements	workbook.DimensionSelector[]	required	The elements in the path selector.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create an AllSubNodesSelector PathSelector
4 var myPathSelector = workbook.createPathSelector({
5     elements: [workbook.createAllSubNodesSelector()]
6 });
7 // Create a DimensionSelector PathSelector
8 var myPathSelector = workbook.createPathSelector({
9     elements: [myDimensionSelector]
10 });
11 ...
12 // Add additional code

```

workbook.createPivot(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a pivot. A pivot is a workbook component that enables you to pivot your dataset query results by defining measures and dimensions, so that you can analyze different subsets of data. A pivot definition is based on an underlying dataset and can include an ID, a name, a row axis, a column axis, conditional/limiting filters, and filter expressions.
Returns	workbook.Pivot
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.aggregationFilters	Array< workbook.ConditionalFilter workbook.LimitingFilter >	optional	The set of conditional filters or limiting filters that are applied to the pivot. When multiple filters are defined, they are aggregated together to form a single compounded filter.
options.columnAxis	workbook.PivotAxis	required	The column axis (X-axis) for the pivot. This includes the fields and measures that are used on the column axis, as well as sorting behavior that is applied to the axis fields and measures.
options.dataset	dataset.Dataset	required	The underlying dataset that the pivot is based on. A pivot can include only the data (fields) that are included in the dataset.
options.filterExpressions	workbook.Expression []	optional	The simple, non-aggregated value-based filters for the pivot.
options.id	string	required	The ID of the pivot.
options.name	string	required	The name of the pivot.
options.reportStyles	workbook.ReportStyle	required	The report styles for the pivot.
options.rowAxis	workbook.PivotAxis	required	The row axis (Y-axis) for the pivot. This includes the fields and measures that are used on the column axis, as well as sorting behavior that is applied to the axis fields and measures.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Pivot
4 var myPivot = workbook.createPivot({
5     name: 'My Pivot',
6     id: '_myPivot',
7     rowAxis: myRowPivotAxis,
8     columnAxis: myColumnPivotAxis,
9     dataset: myDataset
10 });
11
12 // Create a comprehensive Pivot with Expressions

```

```
13 var myPivot = workbook.createPivot({
14   name: 'My Pivot',
15   id: '_myPivot',
16   rowAxis: myRowPivotAxis,
17   columnAxis: myColumnPivotAxis,
18   dataset: myDataset,
19   filterExpressions: [myExpression]
20 });
21
22 // Create a comprehensive Pivot with LimitingFilters
23 var myPivot = workbook.createPivot({
24   name: 'My Pivot',
25   id: '_myPivot',
26   rowAxis: myRowPivotAxis,
27   columnAxis: myColumnPivotAxis,
28   dataset: myDataset,
29   filterExpressions: [myExpression],
30   aggregationFilters: [myLimitingFilter]
31 });
32
33 // Create a comprehensive Pivot with ConditionalFilters
34 var myPivot = workbook.createPivot({
35   name: 'My Pivot',
36   id: '_myPivot',
37   rowAxis: myRowPivotAxis,
38   columnAxis: myColumnPivotAxis,
39   dataset: myDataset,
40   filterExpressions: [myExpression],
41   aggregationFilters: [myConditionalFilter]
42 });
43 ...
44 // Add additional code
```

workbook.createPivotAxis(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a pivot axis, which includes a data root and a sort definition.
Returns	workbook.PivotAxis
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.root	workbook.DataDimension workbook.Section	required	The data for the pivot axis.

Parameter	Type	Required / Optional	Description
options.sortDefinitions	workbook.SortDefinition[]	optional	The sorting definition for the pivot axis.

Syntax


Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Section-based PivotAxis
4 var myPivotAxis = workbook.createPivotAxis({
5     root: mySection
6 });
7
8 // Create a basic DataDimension-based PivotAxis
9 var myPivotAxis = workbook.createPivotAxis({
10     root: myDataDimension
11 });
12
13 // Create a sorted Section-based PivotAxis
14 var myPivotAxis = workbook.createPivotAxis({
15     root: mySection,
16     sortDefinitions: [mySortDefinition]
17 });
18
19 // Create a sorted DataDimension-based PivotAxis
20 var myPivotAxis = workbook.createPivotAxis({
21     root: myDataDimension,
22     sortDefinitions: [mySortDefinition]
23 });
24 ...
25 // Add additional code

```

workbook.createPositionPercent(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a percent-defined background position.
Returns	workbook.PositionPercent
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.percentX	number	required	The percent of the X dimension.
options.percentY	number	required	The percent of the Y dimension.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myPositionPercent = workbook.createPositionPercent({
4     percentX: 25,
5     percentY: 47
6 });
7 ...
8 // Add additional code

```

workbook.createPositionUnits(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a background position defined using x-y coordinates and units.
Returns	workbook.PositionUnits
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.unit	workbook.Unit	required	The units to use for x-y coordinates.
options.x	number	required	The x coordinate.

Parameter	Type	Required / Optional	Description
options.y	number	required	The y coordinate.

Errors

Error Code	Thrown If
INVALID_POSITION	The resulting position is not included in the workbook.Position enum.
INVALID_UNIT	The value of the options.unit parameter is not included in the workbook.Unit enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myPositionUnits = workbook.createPositionUnits ({
4     unit: workbook.Unit.CM,
5     x: 3,
6     y: 3
7 });
8 ...
9 // Add additional code

```

workbook.createPositionValues(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a background position defined using position values.
Returns	workbook.PositionValues
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.horizontal	string	required	The units to use for x-y coordinates.

Parameter	Type	Required / Optional	Description
options.vertical	string	required	The x coordinate.

Errors

Error Code	Thrown If
INVALID_POSITION	The value of any of the options.horizontal or options.vertical parameters is not included in the workbook.Position enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myPositionValues = workbook.createPositionValues({
4     horizontal: workbook.Position.CENTER,
5     vertical: workbook.Position.LEFT
6 });
7 ...
8 // Add additional code

```

workbook.createRange(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a range
Returns	workbook.Range
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.end	string	required	End date or date time in the range.

Parameter	Type	Required / Optional	Description
options.start	string	required	Start date or date time in the range.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myRange = workbook.createRange({
4     start: '5/25/2025',
5     end: '5/26/2025'
6 });
7 ...
8 // Add additional code

```

workbook.createReportStyle(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a report style.
Returns	workbook.ReportStyle
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.rules	workbook.ReportStyleRule []	required	The report style rules to apply to the report style.
options.selectors	workbook.MeasureValue Selector []	required	Selectors indicating where to apply the report style.

Errors

Error Code	Thrown If
NO_RULE_DEFINED	The options.rules parameter is an empty array.
NO_SELECTORS_DEFINED	The options.selectors parameter is an empty array.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var reportStyle = workbook.createReportStyle({
4     selectors: [measureValueSelector], // previously created workbook.MeasureValueSelector objects
5     rules: [rule]                     // previously created workbook.ReportStyleRule objects
6 });
7 ...
8 // Add additional code

```

workbook.createReportStyleRule(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a report style formatting rule.
Returns	workbook.ReportStyleRule
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.expression	workbook.Expression	required	An expression indicating when the report style rule should be applied.

Parameter	Type	Required / Optional	Description
options.style	workbook.Style	required	The style to use for the report style rule.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // create the report style rule using previously created workbook objects
4 var myReportStyleRule = workbook.createReportStyleRule({
5     expression: myExpression, // previously created workbook.Expression object
6     style: myReportStyle // previously created workbook.Style object
7 });
8
9 // defining parameters as part of creating the style rule
10 var rule = workbook.createReportStyleRule({
11     expression: workbook.createExpression({
12         functionId: workbook.ExpressionType.COMPARE,
13         parameters: {
14             comparisonType: "GREATER_OR_EQUAL",
15             operand1: workbook.createExpression({
16                 functionId: workbook.ExpressionType.MEASURE_VALUE,
17                 parameters: {
18                     measure: calculatedMeasure // previously created workbook.CalculatedMeasure
19                 }
20             }),
21             operand2: workbook.createConstant(workbook.createCurrency({
22                 amount: 1,
23                 id: "USD"
24             }))
25         }
26     }),
27     style: workbook.createStyle({
28         backgroundColor: workbook.createColor({
29             red: 255,
30             green: 192,
31             blue: 203
32         })
33     })
34 });
35 ...
36 // Add additional code

```

workbook.createSection(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a section, which includes children and a total line.
Returns	workbook.Section
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.children	Array< workbook.DataDimension workbook.CalculatedMeasure workbook.DataMeasure workbook.Section >	required	The child data.
options.totalLine	workbook.TotalLine	optional	Predefined format values for the total line.

Errors

Error Code	Thrown If
INVALID_TOTAL_LINE	The value specified for the options.totalLine parameter is invalid. Set this parameter value using workbook.TotalLine .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a basic Section with DataDimension children
4 var mySection = workbook.createSection({
5     children: [myDataDimension]
6 });
7
8 // Create a basic Section with Measure children
9 var mySection = workbook.createSection({
10     children: [myMeasure]
11 });
12
13 // Create a basic Section with Section children
14 var mySection = workbook.createSection({
15     children: [mySectionChildren] // create this using a separate call to workbook.createSection
16 });
17
18 // Create a comprehensive Section
19 var mySection = workbook.createSection({
20     children: [myMeasure],
21     totalLine: workbook.TotalLine.HIDDEN
22 });
23 ...

```

24 | // Add additional code

workbook.createSeries(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a chart series, which is a set of aspects.
Returns	workbook.Series
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.aspects	workbook.Aspect[]	required	The aspects for the series.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a Series
4 var mySeries = workbook.createSeries({
5     aspects: [myAspect]
6 });
7 ...
8 // Add additional code

```

workbook.createSimpleRecordKey

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a RecordKey object.
---------------------------	-----------------------------

Returns	workbook.RecordKey
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Parameter	Type	Required / Optional	Description
options.key	string number	required	String or number key.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySimpleRecordKey = workbook.createSimpleRecordKey({
4     key: '1'
5 });
6 ...
7 // Add additional code

```

workbook.createSort(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a sort, which includes indicators for sorting in ascending order, case sensitivity, sort locale, whether nulls should be placed last, and the sort order.
Returns	workbook.Sort
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.ascending	boolean	optional	Indicator for sorting in ascending order. If set to false, sorting is done in descending order. The default value is true (sort in ascending order).
options.caseSensitive	boolean	optional	Indicator that the sort should consider case sensitivity.
options.locale	query.SortLocale	optional	The option for locale specific sorting.
options.nullsLast	boolean	optional	Indicator that the sort should place null items last. The default of this parameter is the value of the ascending parameter (for example, if options.ascending is set to false, the default for this parameter will be false).

Errors

Error Code	Thrown If
INVALID_SORT_LOCALE	The value specified for the options.locale parameter is invalid. Set this parameter value using query.SortLocale .

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create a default Sort (no options specified)
4 var mySort = workbook.createSort({});
5
6 // Create a custom descending Sort
7 var mySort = workbook.createSort({
8     ascending: false,
9     caseSensitive: true,
10    nullsLast: true,
11    locale: query.SortLocale.US_EN
12 });
13 ...
14 // Add additional code

```

workbook.createSortByDataDimensionItem(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a sort based on data dimension items.
---------------------------	---

Returns	workbook.SortByDataDimensionItem
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.item	workbook.DataDimensionItem	required	The data dimension items to sort by.
options.sort	workbook.Sort	required	The sort to use, which defines sort behavior such as whether to sort ascending or descending, whether the sort is case sensitive, and so on.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySortByDataDimensionItem = workbook.createSortByDataDimensionItem({
4     item: myDataDimensionItem, // previously created workbook.DataDimensionItem object
5     sort: SORT.ASCENDING
6 });
7 ...
8 // Add additional code

```

workbook.createSortByMeasure(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a sort based on a measure.
Returns	workbook.SortByMeasure
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .

Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.measure	workbook.CalculatedMeasure workbook.DataMeasure	required	The measure to sort by.
options.otherAxisSelector	Array< workbook.DescendantorSelfNodesSelector workbook.DimensionSelector workbook.PathSelector >	required	The selector for the other axis.
options.selector	Array< workbook.DescendantorSelfNodesSelector workbook.DimensionSelector workbook.PathSelector >	required	The selector for the sort.
options.sort	workbook.Sort	required	The sort used to create.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var mySortByMeasure = workbook.createSortByMeasure({
4     measure: myWorkbookMeasure, // previously created workbook.Measure object
5     otherAxisSelector: myAxisSelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector, or
    workbook.PathSelector object
6     selector: mySelector, // previously created workbook.DescendantorSelfNodesSelector, workbook.DimensionSelector, or
    workbook.PathSelector object
7
8     sort: mySort // previously created workbook.Sort object
9 });
10 ...
11 // Add additional code

```

workbook.createSortDefinition(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a sort definition. A sort definition is used to specify sorting for a pivot or pivot axis.
---------------------------	--

Returns	workbook.SortDefinition
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.selector	workbook.DimensionSelector workbook.PathSelector	required	The selector for the sort definition.
options.sortBys	Array < workbook.SortByDataDimensionItem workbook.SortByMeasure >	required	The sort order for the sort definition.

Errors

Error Code	Thrown If
NO_SORT_BY_DEFINED	The value set for the options.sortBys parameter is empty.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 // Create an AllSubNodesSelector SortDefinition sorted by Measure
4 var mySort = workbook.createSortDefinition({
5     selector: workbook.createAllSubNodesSelector(),
6     sortBys: [myMeasureSort]
7 });
8
9 // Create a DimensionSelector SortDefinition sorted by Measure
10 var mySort = workbook.createSortDefinition({
11     selector: myDimensionSelector,
12     sortBys: [myMeasureSort]
13 });
14
15 // Create a PathSelector SortDefinition sorted by Dimension

```

```
16 var mySort = workbook.createSortDefinition({
17     selector: myPathSelector,
18     sortBy: [myDimensionSort]
19 });
20 ...
21 // Add additional code
```

workbook.createStyle(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a style to be used for conditional formatting.
Returns	workbook.Style
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.backgroundColor	workbook.Color string	optional	The background color.
options.backgroundImage	string	optional	The background image.
options.backgroundPosition	workbook.PositionPercent workbook.PositionUnits workbook.PositionValues	optional	The background position.
options.color	workbook.Color string	optional	The font color.
options.fontSize	workbook.FontSize	optional	The font size.
options.fontStyle	workbook.FontStyle	optional	The font style.
options.fontWeight	workbook.FontWeight	optional	The font weight.
options.textAlign	workbook.TextAlign	optional	The alignment of text.
options.textDecorationColor	workbook.Color string	optional	The text decoration color.

Parameter	Type	Required / Optional	Description
options.textDecorationLine	workbook.TextDecorationLine	optional	The text decoration line.
options.textDecorationStyle	workbook.TextDecorationStyle string	optional	The text decoration style.

Errors

Error Code	Thrown If
INVALID_COLOR	The value of any of the options.backgroundColor, options.color, or options.textDecorationColor parameters is not included in the workbook.Color enum and is not a workbook.Color object.
INVALID_FONT_SIZE	The value of the options.fontSize parameter is not included in the workbook.FontSize enum.
INVALID_FONT_STYLE	The value of the options.fontStyle parameter is not included in the workbook.FontStyle enum.
INVALID_FONT_WEIGHT	The value of the options.fontWeight parameter is not included in the workbook.FontWeight enum.
INVALID_IMAGE	The value of the options.backgroundImage parameter is not included in the workbook.Image enum.
INVALID_TEXT_ALIGN	The value of the options.textAlign parameter is not included in the workbook.TextAlign enum.
INVALID_TEXT_DECORATION_LINE	The value of the options.textDecorationLine parameter is not included in the workbook.TextDecorationLine enum.
INVALID_TEXT_DECORATION_STYLE	The value of the options.textDecorationStyle parameter is not included in the workbook.TextDecorationStyle enum.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 //Add additional code
2 ...
3 var newStyle = workbook.createStyle({
4   backgroundColor: workbook.Color.WHITE, // backgroundColor: 'WHITE' also works
5   backgroundImage: 'myStyleImage', // this value must be previously defined
6   backgroundPosition: workbook.createPositionPercent ({
7     percentX: 25,
8     percentY: 45
9   }),
10  color: workbook.Color.BLUE, // color: 'BLUE' also works
11  fontSize: workbook.FontSize.MEDIUM,
12  fontStyle: workbook.FontStyle.NORMAL,
13  fontWeight: workbook.FontWeight.BOLD,
14  textAlign: workbook.TextAlign.CENTER,

```

```
15 |   textDecorationColor: workbook.Color.YELLOW,  
16 |   textDecorationLine: workbook.TextDecorationLine.UNDERLINE,  
17 |   textDecorationsStyle: workbook.TextDecorationStyle.WAVY  
18 | });  
19 | ...  
20 | // Add additional code
```

workbook.createTable(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a table view.
Returns	workbook.Table
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.columns	workbook.TableColumn[]	required	The columns to include in the table view.
options.dataset	dataset.Dataset	required	The dataset containing the columns to include in the table view.
options.id	string	required	The ID of the table view.
options.name	string workbook.Expression	required	The name of the table view.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 | // Add additional code  
2 | ...
```

```

3 var myTable = nWorkbook.createTable({
4   columns: [column1, column2], // previously defined workbook.TableColumn objects
5   dataset: myDataset          // previously defined dataset
6   id: 'myTableId'
7   name: 'myTableName'
8 });
9 ...
10 //Add additional code

```

workbook.createTableColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a table column. Table columns are used in table definitions, and include an alias, dataset column alias/ID, filters, a label, sorts, and a column width.
Returns	workbook.TableColumn
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.alias	string	optional	The alias for the table column. The alias can be used for mapping results.
options.condition	workbook.ConditionalFilter	optional	Additional conditions for the table column.
options.conditionalFormats	workbook.ConditionalFormat []	required	The conditional formatting to apply to the table column.
options.datasetColumnAlias	string	required	The alias of the underlying dataset column.
options.label	string workbook.Expression	optional	The label for the table column.
options.sort	workbook.Sort	optional	The sorting behavior for the table column.
options.width	number	optional	The width of the table column in the UI, in pixels.

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // Create a basic TableColumn
4 var myTableColumn = workbook.createTableColumn({
5     datasetColumnId: 1,
6     sort: mySort
7 });
8
9 // Create a TableColumn with filters
10 var myTableColumn = workbook.createTableColumn({
11     datasetColumnId: 1,
12     filters: [myTableFilter],
13     sort: mySort
14 });
15
16 // Create a complex TableColumn
17 var myTableColumn = workbook.createTableColumn({
18     filters: [myTableFilter],
19     width: 50,
20     fieldContext: myFieldContext,
21     label: 'My Complex Table Column',
22     alias: 'myComplexTableColumn',
23     sort: mySort
24 });
25 ...
26 // Add additional code
```

workbook.createTableColumnCondition(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a table column condition.
Returns	workbook.TableColumnFilter
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.1

Parameters

**Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.filters	workbook.TableColumnFilter []	optional	Filters for the condition.

Parameter	Type	Required / Optional	Description
options.operator	string	required	Operator of the condition.

Errors

Error Code	Thrown If
INVALID_OPERATOR	The options.operator parameter is invalid.

workbook.createTableColumnFilter(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a table column filter, which includes an operator and values.
Returns	workbook.TableColumnFilter
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.operator	string	required	The operator for the filter. Values correspond to those in query.Operator . For instance, you can use the string value 'EMPTY', which corresponds to query.Operator.EMPTY.
options.values	Array<null Object number string boolean Date>	optional	The values for the filter. The values are specified based on the table column the filter is being placed on and the options.operator value.

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 | // Add additional code
```

```

2  ...
3  // Create a TableFilter without values
4  var myTableFilter = workbook.createTableFilter({
5      operator: query.Operator.EMPTY
6  });
7
8  // Create a TableFilter with values
9  var myValues = ['value1', 'value2', 'value3'];
10 var myTableFilter = workbook.createTableFilter({
11     operator: query.Operator.ANY_OF,
12     values: [myValues]
13 });
14 ...
15 // Add additional code

```

workbook.createTranslation(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a translation (a translation expression)..
Returns	workbook.Expression
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.key	string	required	Key of translation.
options.collection	string	required	Collection of the translation.

Syntax

! Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1  // Add additional code
2  ...
3  var myTranslation = workbook.createTranslation({
4      key: '1',
5      collection: 'myCollection'
6  });
7
8  ...

```



```
9 // Add additional code
```

workbook.list()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Lists all existing workbooks.
Returns	Object[]
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myWorkbookList = workbook.list();
4
5 log.debug({
6     title: 'List of workbooks: ',
7     details: myWorkbookList
8 });
9 ...
10 // Add additional code
```

workbook.listPaged(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves a set of pages with metadata about workbooks.
Returns	PagedInfoData
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.pageSize	number	required	The page size.
options.category	string	optional	The category of datasets or workbooks to retrieve metadata for.

Errors

Error Code	Thrown If
INVALID_OWNER_CATEGORY	The value of the options.category parameter is not included in the workbook.OwnerCategory enum.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3
4 var myWorkbook = workbook.listPaged({
5     pageSize: 2
6 })
7
8 ...
9 // Add additional code

```

workbook.loadWorkbook(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Loads an existing workbook. After you load a workbook, you can execute a table and view the results.
Returns	workbook.Workbook
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	10 units
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.id	string	required	The ID of the existing workbook.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myWorkbook = workbook.load({
4     id: myWorkbookId
5 });
6 ...
7 // Add additional code
```

workbook.Aggregation

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for aggregation types. Used to set the value of the options.aggregation parameter of the workbook.createDataMeasure(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
AVG
COUNT
COUNT_DISTINCT
MAX
MEDIAN

Value
MIN
SUM

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myMeasure = workbook.createMeasure({
4   label: 'My Measure',
5   aggregation: workbook.Aggregation.SUM,
6   expressions: [myExpression]
7 });
8 ...
9 // Add additional code
```

workbook.AspectType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for aspect types. Used to set the options.type parameter of the workbook.createAspect(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value	Sets Aspect.type Property To
COLOR	color
VALUE	value

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
```

```
2  ...
3  var myColorAspect = workbook.createAspect({
4      type: workbook.AspectType.COLOR,
5      measure: myMeasure
6  });
7
8  var myValueAspect = workbook.createAspect({
9      type: workbook.AspectType.VALUE,
10     measure: myMeasure
11 });
12 ...
13 // Add additional code
```

workbook.ChartType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for chart types. Used to pass the type value to workbook.createChart(options) . For more information about charts in SuiteAnalytics, see the help topic Chart Types .
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
AREA
BAR
COLUMN
LINE

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1  // Add additional code
2  ...
3  var myAreaChart = workbook.createChart ({
4      id: '123',
5      name: 'Chart A',
6      type: workbook.ChartType.AREA,
7      category: myCategory,
```

```
8      legend: myLegend,
9      series: [mySeries],
10     dataset: myDataset
11   });
12
13   var myBarChart = workbook.createChart ({
14     id: '456',
15     name: 'Chart B',
16     type: workbook.ChartType.BAR,
17     category: myCategory,
18     legend: myLegend,
19     series: [mySeries],
20     dataset: myDataset
21   });
22
23   var myColumnChart = workbook.createChart ({
24     id: '789',
25     name: 'Chart C',
26     type: workbook.ChartType.COLUMN,
27     category: myCategory,
28     legend: myLegend,
29     series: [mySeries],
30     dataset: myDataset
31   });
32
33   var myLineChart = workbook.createChart ({
34     id: '1000',
35     name: 'Chart D',
36     type: workbook.ChartType.LINE,
37     category: myCategory,
38     legend: myLegend,
39     series: [mySeries],
40     dataset: myDataset
41   });
42   ...
43   // Add additional code
```

workbook.Color

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for colors. Used to set the options.backgroundColor, options.color, and options.textDecorationColor parameters of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
BLACK

Value
BLUE
BROWN
GRAY
GREEN
ORANGE
PINK
PURPLE
RED
WHITE
YELLOW

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 //Add additional code
2 ...
3 // use when creating a style
4 var newStyle = workbook.createStyle({
5   backgroundColor: workbook.Color.WHITE, // backgroundColor: 'WHITE' also works
6   color: workbook.Color.BLUE, // color: 'BLUE' also works
7   textAlign: workbook.TextAlign.CENTER,
8   textDecorationColor: workbook.Color.YELLOW, // textDecorationColor: 'YELLOW' alwo works
9   textDecorationLine: workbook.TextDecorationLine.UNDERLINE,
10  textDecorationStyle: workbook.TextDecorationStyle.WAVY
11 });
12 ...
13 // Add additional code
```

workbook.ConstantType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for constant types. Used to set the value of the options.type parameter of the workbook.createConstant(options) method.
Module	N/workbook Module

Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
BOOLEAN
CURRENCY
DATE
DATE_TIME
DECIMAL
DURATION
INTEGER
NUMBER
RANGE
RECORD
TEXT

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```

1 // Add additional code
2 ...
3 var myBooleanConstant = workbook.createConstant({
4     constant: true,
5     type: workbook.ConstantType.BOOLEAN
6 });
7
8 var myCurrencyConstant = workbook.createConstant({
9     constant: 'USD',
10    type: workbook.ConstantType.CURRENCY
11 });
12
13 var myDateConstant = workbook.createConstant({
14     constant: Date('7/1/2020'),
15     type: workbook.ConstantType.DATE
16 });
17
18 var myDateTimeConstant = workbook.createConstant({
19     constant: Date('2015-03-25T12:00:00Z'),
20     type: workbook.ConstantType.DATE_TIME
21 });
22
23 var myDurationConstant = workbook.createConstant({

```



```
24     constant: 15,  
25     type: workbook.ConstantType.DURATION  
26   });  
27  
28   var myNumberConstant = workbook.createConstant({  
29     constant: 77,  
30     type: workbook.ConstantType.NUMBER  
31   });  
32  
33   var myTextConstant = workbook.createConstant({  
34     constant: 'Sample Text',  
35     type: workbook.ConstantType.TEXT  
36   });  
37   ...  
38   // Add additional code
```

workbook.DateTimeHierarchy

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for date-time hierarchy types.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
MONTH_BASED
WEEK_BASED

workbook.DateTimeProperty

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds the string values for date-time property types.
------------------	---

	Used to set the value of the DATE_TIME_PROPERTY in workbook.ExpressionType .
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
DATE
DAY_OF_MONTH
DAY_OF_WEEK
MONTH
QUARTER
WEEK_OF_YEAR
YEAR

workbook.ExpressionType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for expression types. Use these values for the options.functionId parameter when creating an expression using workbook.createExpression(options). Each expression type uses a set of parameters, which you specify using the options.parameters parameter of workbook.createExpression(options). Make sure that you use the exact parameter names that are supported for each expression type. For example, if you create an EQUALS expression, the supported parameters are operand1 and operand2. You can specify these parameters as the value of the options.parameters parameter of workbook.createExpression(options): <pre>1 parameters: { 2 operand1: <first value to test>, 3 operand2: <second value to test> 4 }</pre>
Module	N/workbook Module

Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value	Supported Parameters	Description
AND	expressions	Returns true if all specified expressions evaluate to true. The specified expressions must be boolean expressions.
ANY_OF	expression set	Returns true if expression is equal to one of the expressions in set.
BETWEEN	expression min max	Returns true if expression is between min and max (inclusive). The specified expressions must be numeric expressions.
CHILD_OF	child parent hierarchy	Returns true if child is a child of parent, and they both belong to the hierarchy hierarchy.
COMPARE	comparisonType operand1 operand2	Returns true if operand1 and operand2 have a comparisonType relationship (such as equal to, greater than, less than, and so on). You can specify the following relationships (as strings for comparisonType): ■ EQUAL ■ GREATER ■ GREATER_OR_EQUAL ■ LESS ■ LESS_OR_EQUAL
CONSTANT	type value	Creates a constant with a type of type and a value of value. The type can often be inferred automatically, but not in all cases. For example, there is no way to distinguish between decimal values and integer values, so in this case, you must specify the type.
CURRENCY_CONVERSION	expression targetCurrency (optional) date (optional)	Applies currency conversion on a currency expression. The exchange rate is determined by the values for source currency, target currency, and date. If the target currency is not specified, the expression uses the currency of the user's subsidiary. If the date is not specified, the expression uses the exchange rate for the current date.

Value	Supported Parameters	Description
DATASET_COLUMN	—	—
DATE_RANGE_SELECTOR_ID	expression anchor (optional)	Converts a relative date expression (as a query.RelativeDateRange value) to its corresponding date range expression. The anchor lets you specify the starting point for the relative date. If an anchor is not specified, the current date is used.
DATE_SELECTOR_ID	expression anchor startExpected	Converts a relative date expression (as a query.RelativeDateRange value) to its corresponding date expression. The anchor lets you specify the starting point for the relative date. If an anchor is not specified, the current date is used. startExpected is a boolean expression. If true, the start date of the relative date is returned. If false, the end date of the relative date is returned.
DATE_TIME_PROPERTY	dateTime property hierarchy	Returns a property from the specified date/time expression. For property, use a value from the workbook.DateTimeProperty enum.
DIVIDE	operand1 operand2	Returns operand1 divided by operand2. Both operands must be numeric expressions.
EQUALS	operand1 operand2	Returns true if operand1 and operand2 are equal.
FIELD	type fieldId datasetId	References a field of a certain type in the datasetId dataset. Do not create the fieldId expression yourself. Instead, use Dataset.getExpressionFromColumn(options).
HIERARCHY	hierarchy expression	Enables hierarchy for an expression. hierarchy is an expression that resolves to a record. expression is the hierarchy expression. Typically, the hierarchy value is defaultHierarchy.
HIERARCHY_TO_TEXT	hierarchy levelToText delimiter	Converts individual values of a hierarchy to a single string value using concatenation and the specified delimiter character. For example, if a hierarchy includes the values L1, L1.1, and L1.1.1 and you specify a delimiter character of /, the converted value is L1/L1.1/L1.1.1. Typically, the hierarchy value is defaultHierarchy.
IF	—	—
IN_RANGE	value range	Returns true if value is inside range.

Value	Supported Parameters	Description
		range can be created using the DATE_RANGE_SELECTOR_ID function.
IS_NULL	expression	Returns true if expression evaluates to NULL.
LAMBDA	None	Returns the value of the current expression. You can use this function only as an argument in another expression. For example, the HIERARCHY_TO_TEXT expression type uses this type to transform a hierarchy level value to a string.
MEASURE_VALUE	measure	Returns an expression that holds the value of the measure measure.
MINUS	operand1 operand2	Returns operand1 minus operand2.
MULTIPLY	operand1 operand2	Returns operand1 multiplied by operand2.
NOT	expression	Returns true if expression evaluates to false.
OR	expressions	Returns true if at least one of the specified expressions evaluates to true. The specified expressions must be boolean expressions.
PLUS	operand1 operand2	Returns operand1 plus operand2.
RECORD_DISPLAY_VALUE	record	Returns a display value from the specified record. The returned value and value type are based on the type of record you specify.
RECORD_KEY	record	Returns the record key (sometimes called the internal ID) of the specified record.
SIMPLE_CONSOLIDATE	expression	Consolidates a currency expression based on the user's subsidiary and the current accounting period.
TRANSLATE	key collection	Returns the key string from the collection translation collection.
TRUNCATE_DATE_TIME	dateTime unit	Returns a date/time value, and if any fields have a value that is smaller than the specified unit of time, these fields are set to zero. For example, if you use this expression type with a unit of minutes, the second-of-minute and nano-of-second fields are set to zero. You can specify the following units of time: ■ hours ■ minutes ■ seconds

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myExpression = workbook.createExpression({
4     functionId: workbook.ExpressionType.EQUALS,
5     parameters: {
6         operand1: dataset.getExpressionFromColumn({
7             alias: 'salary'
8         }),
9         operand2: workbook.createConstant(10000)
10    }
11 });
12 // Add additional code
```

workbook.FontSize

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for font sizes. Used to set the value for the options.fontSize parameter of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
LARGE
LARGER
MEDIUM
SMALL
SMALLER
XX_LARGE
XX_SMALL

Value
X_LARGE
X_SMALL

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 //Add additional code
2 ...
3 // use when creating a style
4 var newStyle = workbook.createStyle({
5     fontSize: workbook.FontSize.MEDIUM,
6     fontStyle: workbook.FontStyle.NORMAL,
7     fontWeight: workbook.FontWeight.BOLD,
8 });
9 ...
10 // Add additional code
```

workbook.FontStyle

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for font styles. Used to set the value for the options.fontStyle parameter of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
ITALIC
NORMAL
OBLIQUE

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 //Add additional code
2 ...
3 // use when creating a style
4 var newStyle = workbook.createStyle({
5     fontSize: workbook.FontSize.MEDIUM,
6     fontStyle: workbook.FontStyle.NORMAL,
7     fontWeight: workbook.FontWeight.BOLD,
8 });
9 ...
10 // Add additional code
```

workbook.FontWeight

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

**Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for font weights. Used to set the value for the options.fontWeight parameter of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
BOLD
NORMAL

Syntax

**Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 //Add additional code
```



```
2  ...
3  // use when creating a style
4  var newStyle = workbook.createStyle({
5    fontSize: workbook.FontSize.MEDIUM,
6    fontStyle: workbook.FontStyle.NORMAL,
7    fontWeight: workbook.FontWeight.BOLD,
8  });
9  ...
10 // Add additional code
```

workbook.Image

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for images that you can use in workbooks. Used as a value for the Style.backgroundImage property.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
EXCLAMATION
QUESTION
SMILE

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1  //Add additional code
2  ...
3  // use when creating a style
4  var newStyle = workbook.createStyle({
5    backgroundColor: workbook.Color.WHITE,
6    backgroundImage: workbook.Image.SMILE,
7    color: workbook.Color.BLUE,
8    fontSize: workbook.FontSize.MEDIUM,
9    fontStyle: workbook.FontStyle.NORMAL,
```

```
10   fontWeight: workbook.FontWeight.BOLD,  
11   });  
12   ...  
13   // Add additional code
```

workbook.Position

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for positions. Used to set the value for the PositionValues.horizontal and PositionValues.vertical properties.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
BOTTOM
CENTER
LEFT
RIGHT
TOP

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1   // Add additional code  
2   ...  
3   var myPositionValues = workbook.createPositionValues({  
4     horizontal: workbook.Position.CENTER,  
5     vertical: workbook.Position.LEFT  
6   });  
7   ...  
8   // Add additional code
```

workbook.Stacking

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds stacking types. Used to pass the stacking value to workbook.createChart(options) .
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
DISABLED
NORMAL
PERCENT

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myChartA = workbook.createChart({
4   id: '123',
5   name: 'Chart A',
6   type: workbook.ChartType.LINE,
7   category: myCategory,
8   legend: myLegend,
9   series: [mySeries],
10  dataset: myDataset,
11  title: 'Chart A',
12  stacking: workbook.Stacking.DISABLED
13 });
14
15 var myChartB = workbook.createChart({
16   id: '456',
17   name: 'Chart B',
18   type: workbook.ChartType.BAR,
19   category: myCategory,
20   legend: myLegend,
21   series: [mySeries],
22   dataset: myDataset,
23   title: 'Chart B',
```

```
24     stacking: workbook.Stacking.NORMAL
25   });
26
27   var myChartC = workbook.createChart({
28     id: '789',
29     name: 'Chart C',
30     type: workbook.ChartType.AREA,
31     category: myCategory,
32     legend: myLegend,
33     series: [mySeries],
34     dataset: myDataset,
35     title: 'Chart C',
36     stacking: workbook.Stacking.PERCENT
37   });
38   ...
39   // Add additional code
```

workbook.TemporalUnit

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for temporal units, such as hours or minutes. Used to set the value of the options.start and options.end parameters of the workbook.createDuration(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
CENTURIES
DAYS
DECADES
ERA
HALF_DAYS
HOURS
MICROS
MILLENIA
MILLIS

Value
MINUTES
MONTHS
NANOS
SECONDS
WEEKS
YEARS

workbook.TextAlign

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for text alignments. Used to set the value for the options.textAlign parameter of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
CENTER
JUSTIFY
LEFT
RIGHT

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 | // Add additional code
```

```
2  ...
3  // use when creating a style
4  var newStyle = workbook.createStyle({
5      textAlign: workbook.TextAlign.CENTER,
6      textDecorationColor: workbook.Color.YELLOW,
7      textDecorationLine: workbook.TextDecorationLine.UNDERLINE,
8      textDecorationStyle: workbook.TextDecorationStyle.WAVY
9  });
10 ...
11 // Add additional code
```

workbook.TextDecorationLine

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for text decoration line types, such as underline and strikethrough. Used to set the value for the options.txtDecorationLine parameter of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
LINE_THROUGH
NONE
OVERLINE
UNDERLINE

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1  // Add additional code
2  ...
3  // use when creating a style
```

```
4 var newStyle = workbook.createStyle({
5   textAlign: workbook.TextAlign.CENTER,
6   textDecorationColor: workbook.Color.YELLOW,
7   textDecorationLine: workbook.TextDecorationLine.UNDERLINE,
8   textDecorationStyle: workbook.TextDecorationStyle.WAVY
9 });
10 ...
11 // Add additional code
```

workbook.TextDecorationStyle

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for text decoration line styles, such as solid and dashed. Used to set the value for the options.textDecoractionStyle parameter of the workbook.createStyle(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2021.2

Values

Value
DASHED
DOTTED
DOUBLE
SOLID
WAVY

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // use when creating a style
```

```
4 var newStyle = workbook.createStyle({
5   textAlign: workbook.TextAlign.CENTER,
6   textDecorationColor: workbook.Color.YELLOW,
7   textDecorationLine: workbook.TextDecorationLine.UNDERLINE,
8   textDecorationStyle: workbook.TextDecorationStyle.WAVY
9 });
10 ...
11 // Add additional code
```

workbook.TotalLine

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds formatting presets for the total line. Used to set the options.totalLine parameter of the workbook.createDataDimension(options) and to workbook.createSection(options) methods.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
FIRST_LINE
HIDDEN
LAST_LINE

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 var myDataDimension = workbook.createDataDimension({
4   totalLine: workbook.TotalLine.HIDDEN,
5   children: [mySection],
6   items: [myDataDimensionItem]
7 });
```



```
8
9  var myFirstSection = workbook.createSection({
10    totalLine: workbook.TotalLine.FIRST_LINE,
11    children: [mySectionChildren]
12  });
13  ...
14  // Add additional code
```

workbook.Unit

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: JavaScript does not include an enumeration type. The SuiteScript 2.x documentation uses the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. In this object, each key points to a read-only string value.

Enum Description	Holds string values for units of measurement. Used to set the options.unit parameter in the workbook.createPositionUnits(options) method.
Module	N/workbook Module
Sibling Module Members	N/workbook Module Members
Since	2020.2

Values

Value
CH
CM
EM
EX
IN
MM
PC
PT
PX
REM
VH
VMAX
VMIN

Value
VW

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workbook Module Script Samples](#). Also see the help topic [Full scripts](#) in the [Tutorial: Creating a Workbook Using the Workbook API](#) topic.

```
1 // Add additional code
2 ...
3 // used to create a font
4 var myFontSize = workbook.createFontSize({
5     size: 12,
6     unit: workbook.UNIT.PX
7 });
8
9 // used when creating a position
10 var myPositionUnits = workbook.createPositionUnits ({
11     unit: workbook.Unit.CM,
12     x: 3,
13     y: 3
14 });
15 ...
16 // Add additional code
```

N/workflow Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/workflow module to initiate new workflow instances or trigger existing workflow instances.

Go To Samples 

In This Help Topic

- [N/workflow Module Members](#)

N/workflow Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	workflow.initiate (options)	number	Server scripts	Initiates a workflow on-demand. This method is the programmatic equivalent of the Initiate Workflow Action action in SuiteFlow.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				Returns the internal ID of the workflow instance used to track the workflow against the record.
	<code>workflow.trigger</code> (options)	number	Server scripts	Triggers a workflow on a record. The actions and transitions of the workflow are evaluated for the record in the workflow instance, based on the current state for the workflow instance. Returns the internal ID of the workflow instance used to track the workflow against the record.

N/workflow Module Script Sample

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script sample demonstrates how to use the features of the N/workflow module.

Search For and Execute a Workflow Deployed on the Customer Record

The following sample searches for a specific workflow deployed on the customer record and then executes it.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

Important: This script sample uses placeholder values for the customer `recordId` and `workflowId`. Before using this sample, replace these IDs with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/workflow', 'N/search', 'N/error', 'N/record'],
6    function(workflow, search, error, record) {
7      function initiateWorkflow() {
8          var workflowInstanceId = workflow.initiate({
9              recordType: 'customer',
10             recordId: 24,
11             workflowId: 'customworkflow_myWorkflow'
12         });
13         var customerRecord = record.load({
14             type: record.Type.CUSTOMER,
15             id: 24
16         });
17     }
18     initiateWorkflow();
19 });

```

workflow.initiate(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Initiates a workflow on-demand. This method is the programmatic equivalent of the Initiate Workflow Action action in SuiteFlow. Returns the internal ID of the workflow instance used to track the workflow against the record. To asynchronously initiate a workflow, see task.WorkflowTriggerTask.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types
Governance	20 units
Module	N/workflow Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.defaultValues	Object	optional	The object that contains key-value pairs to set default values on fields specific to the workflow. These can include fields on the Workflow Definition Page or workflow and state Workflow Custom Fields.	2015.2
options.recordId	string number	required	The internal ID of the base record.	2015.2
options.recordType	string	required	The record type ID of the workflow base record. Use values from the record.Type enum. This is the Record Type field on the Workflow Definition Page .	2015.2
options.workflowId	string number	required	The internal ID (number) or script ID (string) for the workflow definition. This is the ID field on the Workflow Definition Page .	2015.2

Syntax

Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workflow Module Script Sample](#).

```

1 //Add additional code
2 ...
3 var workflowInstanceId = workflow.initiate({

```

```

4   recordType: 'customer',
5   recordId: 24,
6   workflowId: 'customworkflow_myWorkflow'
7 });
8 ...
9 //Add additional code

```

workflow.trigger(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Triggers a workflow on a record. The actions and transitions of the workflow are evaluated for the record in the workflow instance, based on the current state for the workflow instance. Returns the internal ID of the workflow instance used to track the workflow against the record.
Returns	number
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types
Governance	20 units
Module	N/workflow Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.actionId	string number	optional	The internal ID of a button that appears on the record in the workflow. Use this parameter to trigger the workflow as if the specified button were clicked.	2015.2
options.recordId	string	required	The record ID of the workflow base record.	2015.2
options.recordType	string	required	The record type ID of the workflow base record. Use values from the record.Type enum. This is the Record Type field on the Workflow Definition Page.	2015.2
options.stateId	string number	optional	The internal ID (number) or script ID (string) of the workflow state that contains the action.	2015.2
options.workflowId	string number	required	The internal ID (number) or script ID (string) for the workflow definition. This is the ID field on the Workflow Definition Page .	2015.2
options.workflowInstanceId	string number	optional	The internal ID of the workflow instance.	2015.2

Syntax



Important: The following code snippet shows the syntax for this member. It is not a functional example. For a complete script example, see [N/workflow Module Script Sample](#).

```
1 //Add additional code
2 ...
3 var workflowInstanceId = workflow.trigger({
4     recordId: '5', // replace with an actual record id
5     recordType: 'salesorder',
6     workflowId: 'custworkflow_name',
7     actionId: workflowaction25
8 });
9 ...
10 //Add additional code
```

N/xml Module

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Use the N/xml module to validate, parse, read, and modify XML documents.

Go To Samples 

In This Help Topic

- [N/xml Module Members](#)
- [Attr Object Members](#)
- [Document Object Members](#)
- [Element Object Members](#)
- [Node Object Members](#)
- [Parser Object Members](#)
- [XPath Object Members](#)

N/xml Module Members

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Object	xml.Attr	Object	Client and server scripts	Represents an attribute node of an xml.Element object.
	xml.Document	Object	Client and server scripts	Represents an entire XML document. The XML DOM presents a document as a hierarchy of node objects. Use the methods and properties available to the xml.Document object to manipulate the

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				XML document and the nodes in the document tree.
	xml.Element	Object	Client and server scripts	Represents an element in an XML document. Elements may contain attributes, other elements, or text. If an element contains text, the text is represented in a text node of type <code>TEXT_NODE</code> .
	xml.Node	Object	Client and server scripts	Represents a generic XML node in an XML document. A node can be a Document, Element, or Attribute.
	xml.Parser	Object	Client and server scripts	Encapsulates the functionality used by NetSuite to parse XML.
	xml.XPath	Object	Client and server scripts	Encapsulates the functionality used by NetSuite to run XPath expressions. XPath is a standard for enumerating paths in an XML document collection.
Method	xml.escape(options)	string	Client and server scripts	Prepares a string for use in XML by escaping XML markup, such as angle brackets, quotation marks, and ampersands.
	xml.validate(options)	void	Server scripts	Validates an XML document against an XML Schema (XSD).
Enum	xml.NodeType	string (read-only)	Client and server scripts	Holds the string values for the supported node types. Use this enum to set the Node.nodeType property.

Attr Object Members

The following members are called on the [xml.Attr](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Attr.ownerElement	xml.Element (read-only)	Client and server scripts	The xml.Element object that is the parent of the xml.Attr object.
	Attr.name	string (read-only)	Client and server scripts	The name of an attribute.
	Attr.specified	boolean	Client and server scripts	Returns <code>true</code> if the attribute value is set in the parsed XML document, and <code>false</code> if it is a default value in a DTD or Schema.
	Attr.value	string	Client and server scripts	Value of an attribute. The value of the attribute is returned as a string. Character and general entity references are replaced with their values.

Document Object Members

Note: In addition to the Document object members, Document objects inherit the members of the Node object. The methods and properties associated with a Node object can be used as members of a Document object. For more information, see [Node Object Members](#).

The following members are called on the [xml.Document](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Document.adoptNode(options)	xml.Node	Client and server scripts	Attempts to adopt a node from another document to this document.
	Document.createAttribute(options)	xml.Attr	Client and server scripts	Creates an attribute node of type <code>ATTRIBUTE_NODE</code> with the optional specified value.
	Document.createAttributeNS(options)	xml.Attr	Client and server scripts	Creates an attribute node of type <code>ATTRIBUTE_NODE</code> , with the specified namespace value and optional specified value.
	Document.createCDATASection(options)	xml.Node	Client and server scripts	Creates a CDATA section node of type <code>DOCUMENT_FRAGMENT_NODE</code> with the specified data.
	Document.createComment(options)	xml.Node	Client and server scripts	Creates a Comment node of type <code>COMMENT_NODE</code> with the specified string.
	Document.createDocumentFragment()	xml.Node	Client and server scripts	Creates a node of type <code>DOCUMENT_FRAGMENT_NODE</code> .
	Document.createElement(options)	xml.Element	Client and server scripts	Creates a new node of type <code>ELEMENT_NODE</code> with the specified name.
	Document.createElementNS(options)	xml.Element	Client and server scripts	Creates a new node of type <code>ELEMENT_NODE</code> with the specified namespace URI and name.
	Document.createProcessingInstruction(options)	xml.Node	Client and server scripts	Creates a new node of type <code>PROCESSING_INSTRUCTION_NODE</code> with the specified target and data.
	Document.createTextNode(options)	xml.Node	Client and server scripts	Creates a new node of type <code>TEXT_NODE</code> .
	Document.getElementById(options)	xml.Element	Client and server scripts	Returns the element that has an ID attribute with the specified value as an xml.Element object.
	Document.getElementsByTagName(TagName(options))	xml.Element[]	Client and server scripts	Returns an array of xml.Element objects with a specific tag name, in the order in which they appear in the XML document.
	Document.getElementsByTagNameNS(TagNameNS(options))	xml.Element[]	Client and server scripts	Returns an array of xml.Element objects with a specific tag name and namespace, in the order in which they appear in the XML document.
	Document.importNode(options)	xml.Node	Client and server scripts	Imports a node from another document to this document. Creates a new copy of the source node.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Property	Document.doctype	Object (read-only)	Client and server scripts	Returns a node of type DOCUMENT_TYPE_NODE that represents the doctype of the XML document.
	Document.documentElement	xml.Element (read-only)	Client and server scripts	Root node of the XML document.
	Document.documentURI	string (read-only)	Client and server scripts	Location of the document or null if undefined.
	Document.inputEncoding	string (read-only)	Client and server scripts	Encoding used for an XML document at the time the document was parsed.
	Document.xmlEncoding	string (read-only)	Client and server scripts	Part of the XML declaration, the XML encoding of the XML document.
	Document.xmlStandalone	boolean	Client and server scripts	Part of the XML declaration, returns true if the current XML document is standalone or returns false if it is not.
	Document.xmlVersion	string	Client and server scripts	Part of the XML declaration, the version number of the XML document.

Element Object Members

 **Note:** In addition to the Element object members, Element objects inherit the members of the Node object. The methods and properties associated with a Node object can be used as members of a Element object. For more information, see [Node Object Members](#).

The following members are called on the [xml.Element](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Element.getAttribute(options)	string	Client and server scripts	Returns the value of the specified attribute.
	Element.getAttributeNode(options)	xml.Attr	Client and server scripts	Retrieves an attribute node by name.
	Element.getAttributeNodeNS(options)	string	Client and server scripts	Returns an attribute node with the specified namespace URI and local name.
	Element.getAttributeNS(options)	xml.Attr	Client and server scripts	Returns an attribute value with the specified namespace URI and local name.
	Element.getElementsByTagName(options)	xml.Element []	Client and server scripts	Returns an array of descendant xml.Element objects with a specific tag name, in the order in which they appear in the XML document.
	Element.getElementsByTagNameNS(options)	xml.Element []	Client and server scripts	Returns an array of descendant xml.Element objects with a specific tag name and namespace, in the order in which they appear in the XML document.
	Element.hasAttribute(options)	boolean	Client and server scripts	Returns true if the current element has an attribute with the specified name or if that

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
				attribute has a default value. Otherwise, returns false.
	Element.hasAttributeNS(options)	boolean	Client and server scripts	Returns true if the current element has an attribute with the specified local name and namespace or if that attribute has a default value. Otherwise, returns false.
	Element.removeAttribute(options)	void	Client and server scripts	Removes the attribute with the specified name.
	Element.removeAttributeNode(options)	xml.Attr	Client and server scripts	Removes the attribute specified as a xml.Attr object.
	Element.removeAttributeNS(options)	void	Client and server scripts	Removes the attribute with the specified namespace URI and local name.
	Element.setAttribute(options)	void	Client and server scripts	Adds a new attribute with the specified name. If an attribute with that name is already present in the element, its value is changed to the value specified in method argument.
	Element.setAttributeNode(options)	xml.Attr	Client and server scripts	Adds the specified attribute node. If an attribute with the same name is already present in the element, it is replaced by the new one.
	Element.setAttributeNodeNS(options)	xml.Attr	Client and server scripts	Adds the specified attribute node. If an attribute with the same local name and namespace URI is already present in the element, it is replaced by the new one.
	Element.setAttributeNS(options)	void	Client and server scripts	Adds a new attribute with the specified name and namespace URI. If an attribute with the same name and namespace URI is already present in the element, its value is changed to the value specified in method argument.
Property	Element.tagName	string (read-only)	Client and server scripts	The tag name of this xml.Element object.

Node Object Members

The following members are called on the [xml.Node](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Node.appendChild(options)	xml.Node	Client and server scripts	Appends a node after the last child node of a specific element node. Returns the new child node.
	Node.cloneNode(options)	xml.Node	Client and server scripts	Creates a copy of a node. Returns the copied node.
	Node.compareDocumentPosition(options)	number	Client and server scripts	Returns a number that reflects where two nodes are located, compared to each other.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Node.hasAttributes()	boolean	Client and server scripts	Returns true if the current node has any attributes. Note that only element nodes can have attributes.
	Node.hasChildNodes()	boolean	Client and server scripts	Returns true if the current node has child nodes or returns false if the current node does not have child nodes.
	Node.insertBefore(options)	xml.Node	Client and server scripts	Inserts a new child node before an existing child node for the current node.
	Node.isDefaultNamespace(options)	boolean	Client and server scripts	Returns true if the specified namespace uniform resource identifier (URI) is the default namespace for the current node or returns false if the specified namespace is not the default namespace.
	Node.isEqualNode(options)	boolean	Client and server scripts	Returns true if two nodes are equal or returns false if two nodes are not equal.
	Node.isSameNode(options)	boolean	Client and server scripts	Returns true if two nodes reference the same object or returns false if two nodes do not reference the same object.
	Node.lookupNamespaceURI(options)	string	Client and server scripts	Returns the namespace uniform resource identifier (URI) that matches the specified namespace prefix.
	Node.lookupPrefix(options)	string	Client and server scripts	Returns the namespace prefix associated with the specified namespace uniform resource identifier (URI).
	Node.normalize()	void	Client and server scripts	Puts all text nodes underneath a node, including attribute nodes, into a normal form.
	Node.removeChild(options)	xml.Node	Client and server scripts	Removes the specified child node. Returns the removed child node.
	Node.replaceChild(options)	xml.Node	Client and server scripts	Replaces a specific child node with another child node in a list of child nodes.
Property	Node.attributes	Object (read-only)	Client and server scripts	Key-value pairs for all attributes for an xml.Element node. Returns null for all other node types.
	Node.baseURI	string (read-only)	Client and server scripts	Absolute base uniform resource identifier (URI) of a node or null if the URI cannot be determined.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
	Node.childNodes	xml.Node[] (read-only)	Client and server scripts	Array of all child nodes of a node or an empty array if there are no child nodes.
	Node.firstChild	xml.Node (read-only)	Client and server scripts	First child node for a specific node or null if there are no child nodes.
	Node.lastChild	xml.Node (read-only)	Client and server scripts	Last child node for a specific node or null if there is no last child node.
	Node.localName	string (read-only)	Client and server scripts	The local part of the qualified name of a node.
	Node.namespaceURI	string (read-only)	Client and server scripts	The namespace uniform resource identifier (URI) of a node or null if there is no namespace URI for the node.
	Node.nextSibling	xml.Node (read-only)	Client and server scripts	The next node in a node list or null if the current node is the last node.
	Node.nodeName	string (read-only)	Client and server scripts	Name of a node, depending on the type. For example, for a node of type xml.Element , the name is the name of the element.
	Node.nodeType	string	Client and server scripts	The type of node defined as a value from the xml.NodeType enum.
	Node.nodeValue	string	Client and server scripts	The value of a node, depending on its type.
	Node.ownerDocument	xml.Document (read-only)	Client and server scripts	The root element for a node as a xml.Document object.
	Node.parentNode	xml.Node (read-only)	Client and server scripts	The parent node of a node.
	Node.prefix	string	Client and server scripts	The namespace prefix of the node, or null if the node does not have a namespace.
	Node.previousSibling	xml.Node (read-only)	Client and server scripts	The previous node in a node list or null if the current node is the first node.
	Node.textContent	string	Client and server scripts	The textual content of a node and its descendants.

Parser Object Members

The following members are called on the [xml.Parser](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	Parser.fromString(options)	xml.Document	Client and server scripts	Parses a string into a W3C XML document object.
	Parser.toString(options)	string	Client and server scripts	Converts (serializes) an xml.Document object into a string.

XPath Object Members

The following members are called on the [xml.XPath](#) object.

Member Type	Name	Return Type / Value Type	Supported Script Types	Description
Method	XPath.select(options)	xml.Node[]	Client and server scripts	Selects an array of nodes from an XML document using an XPath expression.

N/xml Module Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script samples demonstrate how to use the features of the N/xml module:

- [Load an XML File and Obtain Child Element Values](#)
- [Parse an XML File and Append New Elements](#)
- [Parse an XML String and Log Element Values](#)

These samples reference the following XML file called BookSample.xml:

```

1 <bookstore xmlns:b="http://www.qualifiednamespace.com/book">
2   <b:book category="cooking">
3     <b:title lang="en">Everyday Italian</b:title>
4     <b:author>Giada De Laurentiis</b:author>
5     <b:year>2005</b:year>
6     <b:price>30.00</b:price>
7   </b:book>
8   <b:book category="children">
9     <b:title lang="en">Harry Potter</b:title>
10    <b:author>J K. Rowling</b:author>
11    <b:year>2005</b:year>
12    <b:price>29.99</b:price>
13  </b:book>
14  <b:book category="web">
15    <b:title lang="en">XQuery Kick Start</b:title>
16    <b:author>James McGovern</b:author>
17    <b:author>Per Bothner</b:author>
18    <b:author>Kurt Cagle</b:author>
19    <b:author>James Linn</b:author>
20    <b:author>Vaidyanathan Nagarajan</b:author>
21    <b:year>2003</b:year>
22    <b:price>49.99</b:price>
23  </b:book>
24  <b:book category="web" cover="paperback">
25    <b:title lang="en">Learning XML</b:title>
26    <b:author>Erik T. Ray</b:author>
27    <b:year>2003</b:year>
28    <b:price>39.95</b:price>

```

```

29 </b:book>
30 </bookstore>

```

Load an XML File and Obtain Child Element Values

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To see the content of the BookSample.xml file that's referenced in this sample, see [N/xml Module Script Samples](#).

The following sample loads the BookSample.xml file from the File Cabinet, iterates through the individual book nodes, and accesses the child node values.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   require(['N/xml', 'N/file'], function(xml, file) {
6       return {
7           onRequest: function(options) {
8               var sentence = '';
9               var xmlFileContent = file.load('SuiteScripts/BookSample.xml').getContents();
10              var xmlDocument = xml.Parser.fromString({
11                  text: xmlFileContent
12              });
13              var bookNode = xml.XPath.select({
14                  node: xmlDocument,
15                  xpath: '//b:book'
16              });
17
18              for (var i = 0; i < bookNode.length; i++) {
19                  var title = bookNode[i].firstChild.nextSibling.textContent;
20                  var author = bookNode[i].getElementsByName({
21                      tagName: 'b:author'
22                  })[0].textContent;
23                  sentence += 'Author: ' + author + ' wrote ' + title + '.\n';
24              }
25
26              options.response.write(sentence);
27          }
28      };
29  });

```

This script produces the following output when used with the BookSample.xml file:

```

1  Author: Giada De Laurentiis wrote Everyday Italian.
2  Author: J K. Rowling wrote Harry Potter.
3  Author: James McGovern wrote XQuery Kick Start.
4  Author: Erik T. Ray wrote Learning XML.

```

Parse an XML File and Append New Elements

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To see the content of the BookSample.xml file that's referenced in this sample, see [N/xml Module Script Samples](#).

The following sample shows how to parse an xml file and append new xml element nodes.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/xml', 'N/file'], function(xml, file) {
6      var xmlData = file.load('SuiteScripts/BookSample.xml').getContents();
7      var bookShelf = xml.Parser.fromString({
8          text: xmlData
9      });
10
11     var newBookNode = bookShelf.createElement("book");
12     var newTitleNode = bookShelf.createElement("title");
13     var newTitleNodeValue = bookShelf.createTextNode("");
14     var newAuthorNode = bookShelf.createElement("author");
15     var newAuthorNodeValue = bookShelf.createTextNode("");
16
17     newBookNode.appendChild(newTitleNode);
18     newBookNode.appendChild(newAuthorNode);
19     newTitleNode.appendChild(newTitleNodeValue);
20     newAuthorNode.appendChild(newAuthorNodeValue);
21
22 });

```

Parse an XML String and Log Element Values

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To see the content of the BookSample.xml file that's referenced in this sample, see [N/xml Module Script Samples](#).

The following sample parses the XML string stored in the `xmlString` variable. The sample selects all `config` elements in the `xmlDocument` node, loops through them, and logs their contents.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see the help topics [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5
6  require(['N/xml'], function(xml) {
7      return {
8          onRequest: function(options) {
9              var xmlString = '<?xml version="1.0" encoding="UTF-8"?><config date="1465467658668" transient="false">Some content</config>';
10
11              var xmlDocument = xml.Parser.fromString({
12                  text: xmlString
13              });
14

```

```

15         var bookNode = xml.XPath.select({
16             node: xmlDocument,
17             xpath: '//config'
18         });
19
20         for (var i = 0; i < bookNode.length; i++) {
21             log.debug('Config content', bookNode[i].textContent);
22         }
23     }
24 };
25 });

```

xml.Attr

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Represents an attribute node of an xml.Element object.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/xml Module
Methods and Properties	Attr Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attr = elem[0].getAttributeNode({
4     name : 'lang'
5 });
6 ...
7
8 //Add additional code

```

Attr.name

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The name of an attribute. This property is a qualified name if the Node.localName property for the parent xml.Element object is null.
Type	string (read-only)
Module	N/xml Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attrName = attr.name; \\ returns 'lang'.
4 ...
5 //Add additional code

```

Attr.ownerElement

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	xml.Element object that is the parent of the xml.Attr object. Value is null if the attribute is not used by an element. Important: This property is not supported on Internet Explorer.
Type	xml.Element (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attrElement = attr.ownerElement; \\ returns the title element.
4 ...
5 //Add additional code

```

Attr.specified

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Returns true if the attribute value is set in the parsed XML document, and false if it is a default value in a DTD or Schema.
Type	boolean
Module	N/xml Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attrSpecified = attr.specified;
4 ...
5 //Add additional code

```

Attr.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Value of an attribute. The value of the attribute is returned as a string. Character and general entity references are replaced with their values. For example, a character reference such as <code>&#160;</code> or an entity reference such as <code>&#nbsp;</code> is replaced with a non-breaking space. Note: If you set this value, it creates a text node with the unparsed contents of the string, for example, any characters that an XML processor would recognize as markup are instead treated as literal text.
Type	string
Module	N/xml Module
Since	2015.2

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: value	Cannot set the attribute value with the specified value.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attrValue = attr.value;
4 ...
5 //Add additional code

```

xml.Document

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Represents an entire XML document. The XML DOM presents a document as a hierarchy of node objects. Use the methods and properties available to the xml.Document object to manipulate the XML document and the nodes in the document tree. An XML document object is also a node of type DOCUMENT_NODE. In addition to the Document object members, Document objects inherit the members of the Node object. For a complete list of these methods and properties, see Node Object Members.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/xml Module
Methods and Properties	Document Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var xmlDocument = xml.Parser.fromString({
4     text : xmlFileContent
5 });
6 ...
7 //Add additional code

```

Document.adoptNode(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Attempts to adopt a node from another document to this document. If successful, this method changes the Node.ownerDocument property of the source node, its children, and any attribute nodes to the current document. If the source node has a parent node, the parent node is first removed from the child list of its own parent node. Important: This method is not supported on Internet Explorer.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.source	xml.Node	required	Source node to add as a child into the current node object.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	NOT_FOUND_ERR: An attempt is made to reference a node in a context where it does not exist.	Node cannot be adopted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var adoptedNode = xmlDocument1.adoptNode({
4     source : sourceNode,
5 });
6 ...
7 //Add additional code

```

Document.createAttribute(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an attribute node of type ATTRIBUTE_NODE with the optional specified value and returns the new xml.Attr object. The localName, prefix, and namespaceURI properties of the new node are set to null.
Returns	xml.Attr
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	Name of the new attribute node.
options.value	string	optional	Value for the attribute node. If unspecified, the value is an empty string.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Attribute with the specified name or value cannot be created.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attr = xmlDocument.createAttribute({
4     name : 'lang',
5     value : 'fr'
6 });
7 ...
8 //Add additional code

```

Document.createAttributeNS(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates an attribute node of type <code>ATTRIBUTE_NODE</code>, with the specified namespace value and optional specified value, and returns the new <code>xml.Attr</code> object. The <code>Node.localName</code>, <code>Node.prefix</code>, and <code>Node.namespaceURI</code> properties of the new node are set to <code>null</code>.  Important: This method is not supported on Internet Explorer.
Returns	<code>xml.Attr</code>
Supported Script Types	Client and server scripts

	For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the attribute to create. Value can be null.
options.qualifiedName	string	required	Qualified name of the new attribute node.
options.value	string	optional	Value for the attribute node. If unspecified, the value is an empty string.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Attribute with the specified value cannot be created.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attr = xmlDocument.createAttributeNS({
4   namespaceURI : '*',
5   qualifiedName : 'lang',
6   value : 'fr'
7 });
8 ...
9 //Add additional code

```

Document.createCDATASection(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a CDATA section node of type DOCUMENT_FRAGMENT_NODE with the specified data and returns the new xml.Node object.
Returns	xml.Node

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.data	string	required	Data for the new CDATA section node.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: data	Cannot create CDATA section node with the specified data.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var newNode = xmlDocument.createCDATASection({
4   data : 'Limited Edition.'
5 });
6 ...
7 //Add additional code

```

Document.createComment(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a Comment node of type COMMENT_NODE with the specified string.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.data	string	required	Data for the Comment node.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var newNode = xmlDocument.createComment({
4   data : 'This is a comment.'
5 });
6 ...
7 //Add additional code

```

Document.createDocumentFragment()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a node of type DOCUMENT_FRAGMENT_NODE and returns the new xml.Node object.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...

```



```

3 | var newNode = xmlDocument.createDocumentFragment();
4 | ...
5 | //Add additional code

```

Document.createElement(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new node of type ELEMENT_NODE with the specified name and returns the new xml.Element node. The Node.localName , Node.prefix , and Node.namespaceURI properties of the new node are set to null.
Returns	xml.Element
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.tagName	string	required	Name of the element to create.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Element cannot be created with the specified tagName value.

Syntax

⚠ Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var elem = xmlDocument.createElement({
4 |   tagName : 'book'
5 | });
6 | ...

```

7 //Add additional code

Document.createElementNS(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new node of type ELEMENT_NODE with the specified namespace URI and name and returns the new xml.Element object. The Node.localName , Node.prefix , and Node.namespaceURI properties of the new node are set to null.
Returns	xml.Element
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the element to create. Can be null.
options.qualifiedName	string	required	Qualified name of the element to create.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Element with the specified namespace cannot be created.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var elem = xmlDocument.createElementNS({
4     namespaceURI : '*',
5     qualifiedName : 'book'
6 });
7 ...
8 //Add additional code

```

Document.createProcessingInstruction(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new node of type PROCESSING_INSTRUCTION_NODE with the specified target and data and returns the new xml.Node object. The following example shows a sample processing instruction: <?xml version="1.0"?> Use a processing instruction node to keep processor-specific information in the text of the XML document.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.target	string	required	Target part of the processing instruction.
options.data	string	required	Data for the processing instruction.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Processing instruction node cannot be created with the specified target or data.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var newNode = xmlDocument.createProcessingInstruction({
4   target : 'xml'
5   data : 'version="1.0"'
6 });
7 ...
8 //Add additional code

```

Document.createTextNode(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a new text node and returns the new xml.Node object.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.data	string	required	Data for the text node.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var newNode = xmlDocument.createTextNode({
4   data : 'Sample Title'
5 });
6 ...
7 //Add additional code

```

Document.getElementById(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the element that has an ID attribute with the specified value as an xml.Element object. Returns null if no such element exists.
Returns	xml.Element
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.elementId	string	required	Unique ID value for an element.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var elem = xmlDocument.getElementById({
4     elementId : 'id12345'
5 });
6 ...
7 //Add additional code

```

Document.getElementsByTagName(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an array of xml.Element objects with a specific tag name, in the order in which they appear in the XML document.
Returns	xml.Element[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.tagName	string	required	Case-sensitive tag name of the element to match on. Use the * wildcard to match all elements.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var elem = xmlDocument.getElementsByTagName({
4     tagName : 'book'
5 });
6 ...
7 //Add additional code
```

Document.getElementsByTagNameNS(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an array of xml.Element objects with a specific tag name and namespace, in the order in which they appear in the XML document. Important: This method is not supported on Internet Explorer.
Returns	xml.Element[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI to match on. Use the * wildcard to match all namespaces.
options.localName	string	required	Localname property to match on. Use the * wildcard to match all local names.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
```

```

2 ...
3 var elem = xmlDocument.getElementsByTagNameNS({
4     namespaceURI : '*',
5     localName : 'book'
6 });
7 ...
8 //Add additional code

```

Document.importNode(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Imports a node from another document to this document. This method creates a new copy of the source node. If the deep parameter is set to true, it imports all children of the specified node. If set to false, it imports only the node itself. Method returns the imported xml.Node object.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.importedNode	xml.Node	required	Node from another XML document to import.
options.deep	boolean	required	Use true to import the node, its attributes, and all descendents. Use false to only import the node and its attributes. Important: This parameter is not supported on Internet Explorer.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	NOT_SUPPORTED_ERR: The implementation does not support the requested type of object or operation.	Node cannot be imported.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 ...
2 var importedNode = xmlDocument1.importNode({
3     importedNode : foreignNode,
4     deep : true
5 });
6 ...
```

Document.doctype

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The doctype of the XML document.  Important: This property is not supported on Internet Explorer.
Type	xml.Element (read-only)
Module	N/xml Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var doctype = xmlDocument.doctype;
4 ...
5 //Add additional code
```

Document.documentElement

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Root node of the XML document. Use this property to directly access the xml.Element object that represents the root node of an XML document.
Type	xml.Element (read-only)
Module	N/xml Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var root = xmlDocument.documentElement;
4 ...
5 //Add additional code

```

Document.documentURI

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Location of the document or null if undefined.
Type	string
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var documentURI = xmlDocument.documentURI;
4 ...
5 //Add additional code

```

Document.inputEncoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Encoding used for an XML document at the time the document was parsed. When parsing an XML document with the following declaration, the inputEncoding property is UTF-8: <pre><?xml version="1.0" encoding="UTF-8" standalone="yes"?></pre> Important: The value of this property is browser-specific.
Type	string (read-only)
Module	N/xml Module

Since	2015.2
-------	--------

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var encoding = xmlDocument.inputEncoding;
4 ...
5 //Add additional code

```

Document.xmlEncoding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Part of the XML declaration, the XML encoding of the XML document. In the following declaration, the xmlEncoding property is UTF-8: <pre><?xml version="1.0" encoding="UTF-8" standalone="yes"?></pre> Important: This property is not supported on Internet Explorer or Firefox.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var encoding = xmlDocument.xmlEncoding;
4 ...
5 //Add additional code

```

Document.xmlStandalone

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Part of the XML declaration, returns true if the current XML document is standalone or returns false if it is not. In the following declaration, the xmlStandalone property is true: <pre><?xml version="1.0" encoding="UTF-8" standalone="yes"?></pre>
-----------------------------	---

	 Important: This property is not supported on Internet Explorer or Firefox.
Type	boolean
Module	N/xml Module
Since	2015.2

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/xml Module Script Samples .
--

```

1 //Add additional code
2 ...
3 var isStandalone = xmlDocument.xmlStandalone;
4 ...
5 //Add additional code

```

Document.xmlVersion

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Part of the XML declaration, the version number of the XML document. In the following declaration, the xmlVersion property is 1.0: <pre><?xml version="1.0" encoding="UTF-8" standalone="yes"?></pre>
	 Important: This property is not supported on Internet Explorer or Firefox.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	The implementation does not support the requested type of object or operation.	Cannot edit the XML version for the document.

Syntax

 Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/xml Module Script Samples .
--

```

1 //Add additional code
2 ...
3 var version = xmlDocument.xmlVersion;

```

```

4 | ...
5 | //Add additional code

```

xml.Element

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Represents an element in an XML document. Elements may contain attributes, other elements, or text. If an element contains text, the text is represented in a text node of type TEXT_NODE. For example, the following element year contains a text node with the value of 2015: <pre><year>2015</year></pre> An XML element object is also a node of type ELEMENT_NODE. In addition to the Element object members, Element objects inherit the members of the Node object. For a complete list of these methods and properties, see Node Object Members.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/xml Module
Methods and Properties	Element Object Members
Since	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var elem = parentNode[0].getElementsByTagName({tagName : 'title'});
4 | ...
5 | //Add additional code

```

Element.getAttribute(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the value of the specified attribute.
Returns	xml.Attr
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	Name of the attribute for which to return the value.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attr = elem[0].getAttribute({
4   name : 'lang'
5 });
6 ...
7 //Add additional code

```

Element.getAttributeNode(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Retrieves an attribute node by name.  Important: This method is not supported on Internet Explorer.
Returns	xml.Attr
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	The name of the attribute to return.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var attr = elem[0].getAttributeNode({
4     name : 'lang'
5 });
6 ...
7 //Add additional code
```

Element.getAttributeNodeNS(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an attribute node with the specified namespace URI and local name. Important: This method is not supported on Internet Explorer.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the attribute to return. Value can be null.
options.localName	string	required	Local name of the attribute to return.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: namespaceURI	Attribute node with the specified namespace cannot be retrieved.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var attr = elem[0].getAttributeNodeNS({
4     namespaceURI : '*'
5     localName : 'lang'
6 });
7 ...
8 //Add additional code
```

Element.getAttributeNS(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an attribute value with the specified namespace URI and local name. Important: This method is not supported on Internet Explorer.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the attribute to return. Value can be null.
options.localName	string	required	Local name of the attribute to return.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: namespaceURI	Attribute with the specified namespace cannot be retrieved.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var attr = elem[0].getAttributeNS({
4     namespaceURI : '*'
5     localName : 'lang'
6 });
7 ...
8 //Add additional code
```

Element.getElementsByTagName(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an array of descendant xml.Element objects with a specific tag name, in the order in which they appear in the XML document.
Returns	xml.Element[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.tagName	string	required	Case-sensitive tag name of the element to match on. Use the * wildcard to match all elements.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var elem = parentNode[0].getElementsByTagName({tagName : 'title'});
4 ...
5 //Add additional code
```


Element.getElementsByTagNameNS(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns an array of descendant xml.Element objects with a specific tag name and namespace, in the order in which they appear in the XML document.  Important: This method is not supported on Internet Explorer.
Returns	xml.Element []
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI to match on. Use the * wildcard to match all namespaces.
options.localName	string	required	Localname property to match on. Use the * wildcard to match all local names.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: namespaceURI	Elements with the specified namespace cannot be retrieved.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var elem = parentNode[0].getElementsByTagNameNS({
4     namespaceURI : '*',
5     localName : 'lang'
6 });
7 ...

```

8 | //Add additional code

Element.hasAttribute(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the current element has an attribute with the specified name or if that attribute has a default value. Otherwise, returns false.  Important: This method is not supported on Internet Explorer.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	Name of the attribute to match on.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attrExists = elem[0].hasAttribute({
4   name : 'lang'
5 });
6 ...
7 //Add additional code

```

Element.hasAttributeNS(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the current element has an attribute with the specified local name and namespace or if that attribute has a default value. Otherwise, returns false.
---------------------------	--

	 Important: This method is not supported on Internet Explorer.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the attribute to match on.
options.localName	string	required	Local name of the attribute to match on.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: namespaceURI	The method is called with an illegal namespace value.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attrExists = elem[0].hasAttributeNS({
4   namespaceURI : '*',
5   localName : 'lang'
6 });
7 ...
8 //Add additional code

```

Element.removeAttribute(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the attribute with the specified name.
---------------------------	--

Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	Name of the attribute to remove.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: name	Attribute with the specified name cannot be removed.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 elem[0].removeAttribute({
4   name : 'lang'
5 });
6 ...
7 //Add additional code

```

Element.removeAttributeNode(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the attribute specified as a xml.Attr object.
Returns	xml.Attr
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None

Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.oldAttr	xml.Attr	required	xml.Attr object to remove.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	NOT_FOUND_ERR: An attempt is made to reference a node in a context where it does not exist.	Attribute node cannot be removed.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var removedAttr = elem[0].removeAttributeNode({
4     oldAttr : attr
5 });
6 ...
7 //Add additional code

```

Element.removeAttributeNS(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the attribute with the specified namespace URI and local name.  Important: This method is not supported on Internet Explorer.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module

Since	2015.2
-------	--------

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the attribute node to remove.
options.localName	string	required	Local name of the attribute node to remove.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected string: namespaceURI	Attribute with the specified namespace cannot be removed.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 elem[0].removeAttributeNS({
4     namespaceURI : '*',
5     localName : 'lang'
6 });
7 ...
8 //Add additional code

```

Element.setAttribute(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a new attribute with the specified name. If an attribute with that name is already present in the element, its value is changed to the value specified in method argument. If an attribute with the specified name already exists, the value of the attribute is changed to the value of the value parameter.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.name	string	required	Name of the attribute to add.
options.value	string	required	Value of the attribute to add.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Value for the attribute cannot be set.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 elem[0].setAttribute({
4   name : 'lang',
5   value : 'fr'
6 });
7 ...
8 //Add additional code

```

Element.setAttributeNode(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds the specified attribute node. If an attribute with the same name is already present in the element, it is replaced by the new one. If an attribute with the same nodeName property already exists, it is replaced with the object in the newAttr parameter. If the attribute node replaces an existing attribute node, the method returns the new xml.Attr object.
Returns	xml.Attr
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module

Since	2015.2
-------	--------

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.newAttr	xml.Attr	required	New xml.Attr object to add to the xml.Element object.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INUSE_ATTRIBUTE_ERR: An attempt is made to add an attribute that is already in use elsewhere.	Attribute node cannot be added.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 elem[0].setAttributeNode({
4   newAttr : attr
5 });
6 ...
7 //Add additional code

```

Element.setAttributeNodeNS(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds the specified attribute node. If an attribute with the same local name and namespace URI is already present in the element, it is replaced by the new one. If an attribute with the same namespaceURI and localName property already exist, it is replaced with the object in the newAttr parameter. If the attribute node replaces an existing attribute node, the method returns the new xml.Attr object.  Important: This method is not supported on Internet Explorer.
Returns	xml.Attr
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None

Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.newAttr	xml.Attr	required	New xml.Attr object to add to the xml.Element object.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INUSE_ATTRIBUTE_ERR: An attempt is made to add an attribute that is already in use elsewhere.	Attribute node cannot be added.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 elem[0].setAttributeNode({
4     newAttr : attr
5 });
6 ...
7 //Add additional code

```

Element.setAttributeNS(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a new attribute with the specified name and namespace URI. If an attribute with the same name and namespace URI is already present in the element, its value is changed to the value specified in method argument. If an attribute with the specified name already exists, the value of the attribute is changed to the value of the value parameter. If the attribute node replaces an existing attribute node, the method returns the new xml.Attr object.  Important: This method is not supported on Internet Explorer.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.

Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI of the attribute node to add.
options.qualifiedName	string	required	Fully qualified attribute name to add.
options.value	string	required	String value of the attribute to add.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	INVALID_CHARACTER_ERR: An invalid or illegal XML character is specified.	Attribute node with the specified value cannot be added.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 elem[0].setAttributeNS({
4   namespaceURI : '*',
5   qualifiedName : 'lang',
6   value : 'fr'
7 });
8 ...
9 //Add additional code

```

Element.tagName

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The tag name of this xml.Element object.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var tagName = elem[0].tagName; \\ returns 'title'.
4 ...
5 //Add additional code
```

xml.Node

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Represents a single node in an XML document tree. A node can be an element, attribute, or document. For a list of possible node types, see xml.NodeType. The XML Document Object Module (DOM) is a general way to access and manipulate XML documents. In an XML document, the DOM organizes nodes into a hierarchical tree structure. For general information about the XML DOM structure, see XML DOM. NetSuite supports a subset of W3C DOM methods to manipulate parent and nested nodes. For other code samples that use this object, see the syntax sample that follows, as well as Node.childNodes and N/xml Module Script Samples.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Module	N/xml Module
Methods and Properties	Node Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

In the following code sample, a single node is selected for access. The type of node being selected is an XML document. For more information about the context of the xpath parameter, see [xml.XPath](#) and [XPath.select\(options\)](#).

```
1 //Add additional code
2 ...
3 var bookNode = xml.XPath.select({
4     node : xml.Document,
5     xpath : '//book'
6 });
7 ...
8 //Add additional code
```

Node.appendChild(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Appends a node after the last child node of a specific element node. Returns the new child node.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.newChild	xml.Node	required	xml.Node object to append.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	HIERARCHY_REQUEST_ERR: An attempt was made to insert a node where it is not permitted.	Node cannot be appended.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var bookShelf = xml.Parser.fromString(file.load('SuiteScripts/books.xml').getContents());
4
5 var newBookNode = xmlData.createElement("book");
6 var newTitleNode = xmlData.createElement("title");
7 var newTitleNodeValue = xmlData.createTextNode("");
8 var newAuthorNode = xmlData.createElement("author");
9 var newAuthorNodeValue = xmlData.createTextNode("");
10 newTitleNode.appendChild(newTitleNodeValue);
11 newAuthorNode.appendChild(newAuthorNodeValue);
12 newBookNode.appendChild(newTitleNode);
13 newBookNode.appendChild(newAuthorNode);
14
15 var newbook = bookShelf.appendChild({
16   newChild : newBookNode
17 });
18 ...
19 //Add additional code

```

Node.cloneNode(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Creates a copy of a node. Returns the copied node.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.deep	boolean	optional	Use true to clone the node, attributes, and all descendents. Use false to only clone the node and attributes.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var copiednode = elem[0].cloneNode({
4   deep : true
5 });
6 ...
7 //Add additional code

```

Node.compareDocumentPosition(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns a number that reflects where two nodes are located, compared to each other. Returns one of the following numbers: ■ 1. The two nodes do not belong to the same document. ■ 2. The specified node comes before the current node. ■ 4. The specified node comes after the current node. ■ 8. The specified node contains the current node. ■ 16. The current node contains the specified node. ■ 32. The specified and current nodes do not have a common container node or the two nodes are different attributes of the same node.
---------------------------	--

	Note: The return value can be a combination of the above values. For example, a return value of 20 means the specified node is contained by the current node, a value of 16, and the specified node follows the current node, a value of 4. Important: This method is not supported on Internet Explorer.
Returns	number
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
options.other	xml.Node	required	The node to compare with the current node.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	Invalid argument type, expected xml.Node or subclass: other	The options.other is of type xml.Node .

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see N/xml Module Script Samples .
--

```

1 //Add additional code
2 ...
3 var posCode = elem[0].compareDocumentPosition({
4     other : parentNode[0]
5 });
6 ...
7 //Add additional code

```

Node.hasAttributes()

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the current node has attributes defined, or false otherwise.
---------------------------	--

	 Important: This method is not supported on Internet Explorer.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hasAttributes = parentNode[0].hasAttributes()
4 ...
5 //Add additional code

```

Node.hasChildNodes()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the current node has child nodes or returns false if the current node does not have child nodes.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var hasChildren = parentNode[0].hasChildNodes()
4 ...
5 //Add additional code

```

Node.insertBefore(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Inserts a new child node before an existing child node for the current node. If the new child node is already in the list of children, this method removes the new child node and inserts it again.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.newChild	xml.Node	required	The new child node to insert.
options.refChild	xml.Node	required	The node before which to insert the new child node. If refChild is , the method inserts the new node at the end of the list of children.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	HIERARCHY_REQUEST_ERR: An attempt was made to insert a node where it is not permitted.	Node cannot be inserted.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var insertednode = parentNode[0].insertBefore({
4     newChild : elemList1[0],
5     refChild : elemList2[0]
6 });
7 ...
8 //Add additional code

```


Node.isDefaultNamespace(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if the specified namespace uniform resource identifier (URI) is the default namespace for the current node or returns false if the specified namespace is not the default namespace. See also Node.namespaceURI.  Important: This method is not supported on Internet Explorer.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	The namespace URI to compare.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var isDefault = parentNode[0].isDefaultNamespace({
4     namespaceURI : '*'
5 });
6 ...
7 //Add additional code

```

Node.isEqualNode(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if two nodes are equal or returns false if two nodes are not equal.
---------------------------	--

	The two nodes are equal if they meet the following conditions: - Both nodes have the same type. - Both nodes have the same attributes and attribute values. The order of the attributes is not considered. - Both nodes have equal lists of child nodes and the child nodes appear in the same order.  Note: Two nodes may be equal, even if they are not the same. See Node.isSameNode(options).  Important: This method is not supported on Internet Explorer.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.other	xml.Node	required	The node to compare with the current node.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var isEqual = elem[0].isEqualNode({
4   other : node
5 });
6 ...
7 //Add additional code

```

Node.isSameNode(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns true if two nodes reference the same object or returns false if two nodes do not reference the same object.
---------------------------	---

	If two nodes are the same, all attributes have the same values and you can use methods on the two nodes interchangeably.  Note: Two nodes that are the same are also equal. See Node.isEqualNode(options).  Important: This method is not supported on Internet Explorer or Firefox.
Returns	boolean
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.other	xml.Node	required	The node to compare with the current node.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var isSame = elem[0].isSameNode({
4   other : node
5 });
6 ...
7 //Add additional code

```

Node.lookupNamespaceURI(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the namespace uniform resource identifier (URI) that matches the specified namespace prefix. Returns null if the specified prefix does not have an associated URI.  Important: This method is not supported on Internet Explorer.
Returns	string

Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.prefix	string	required	Namespace prefix associated with the namespace URI.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var uri = parentNode[0].lookupNamespaceURI({
4     prefix : '*'
5 });
6 ...
7 //Add additional code

```

Node.lookupPrefix(options)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Returns the namespace prefix associated with the specified namespace uniform resource identifier (URI). Returns null if the specified URI does not have an associated prefix. If more than one prefix is associated with the namespace prefix, the namespace returned by this method depends on the module implementation.  Important: This method is not supported on Internet Explorer.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.namespaceURI	string	required	Namespace URI associated the namespace prefix.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var prefix = parentNode[0].lookupPrefix({
4   namespaceURI : '*'
5 });
6 ...
7 //Add additional code

```

Node.normalize()

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Puts all text nodes underneath a node, including attribute nodes, into a normal form. In normal form, only structure (such as elements, comments, processing instructions, CDATA sections, and entity references) separates text nodes. After normalization, there are no adjacent or empty text nodes. Use this method if you require a particular document tree structure and want to make sure that the XML DOM view of a document is identical when you save and reload it.
Returns	void
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module
Since	2015.2

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 node.normalize();
4 ...
5 //Add additional code

```

Node.removeChild(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Removes the specified child node.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.oldChild	xml.Node	required	Node to remove.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	NOT_FOUND_ERR: An attempt is made to reference a node in a context where it does not exist.	Node cannot be removed.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var removednode = parentNode[0].removeChild({
4   oldChild : node
5 });
6 ...
7 //Add additional code

```

Node.replaceChild(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Replaces a specific child node with another child node in a list of child nodes.
---------------------------	--

	If the new child node to add already exists in the list of child nodes, the node is first removed.
Returns	xml.Node
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.newChild	xml.Node	required	New child node to add.
options.oldChild	xml.Node	required	Child node to replaced with the new node.

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	NOT_FOUND_ERR: An attempt is made to reference a node in a context where it does not exist.	Child node cannot be found.
SSS_XML_DOM_EXCEPTION	HIERARCHY_REQUEST_ERR: An attempt was made to insert a node where it is not permitted.	Child node cannot be replaced.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var replacednode = parentNode.replaceChild({
4   newChild : elem[2],
5   oldChild : elem[1]
6 });
7 ...
8 //Add additional code

```

Node.attributes

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Key-value pairs for all attributes for an xml.Element node. Returns null for all other node types.
-----------------------------	--

Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var attribs = elem[0].attributes;
4 ...
5 //Add additional code

```

Node.baseURI

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Absolute base uniform resource identifier (URI) of a node or null if the URI cannot be determined. For client scripts, this property always returns null. Note: The format of this value is browser-specific.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var baseuri = parentNode[0].baseURI;
4 ...
5 //Add additional code

```

Node.childNodes

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Array of all child nodes of a node or an empty array if there are no child nodes.
-----------------------------	---

Type	xml.Node[]
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var childnodes = parentNode[0].childNodes;
4 ...
5 //Add additional code

```

Node.firstChild

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The first child node of a node, or null if there are no child nodes.
Type	xml.Node
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var nodeValue1 = bookNode[0].firstChild.nextSibling.textContent;
4 ...
5 //Add additional code

```

Node.lastChild

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The last child node of a node, or null if there are no child nodes.
Type	xml.Node
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var nodeValue = parentNode[0].lastChild.previousSibling.textContent;
4 ...
5 //Add additional code
```

Node.localName

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The local part of the qualified name of a node.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var localname = parentNode[0].localName;
4 ...
5 //Add additional code
```

Node.namespaceURI

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The namespace uniform resource identifier (URI) of a node or null if there is no namespace URI for the node.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
```

```

2 | ...
3 | var uri = parentNode[0].namespaceURI;
4 | ...
5 | //Add additional code

```

Node.nextSibling

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The next node in a node list or null if the current node is the last node.
Type	xml.Node (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var nodeName = parentNode[0].firstChild.nextSibling.textContent;
4 | ...
5 | //Add additional code

```

Node.nodeName

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Name of a node, depending on the type. For example, for a node of type xml.Element , the name is the name of the element. Note: On Chrome, this property also includes the namespace or prefix.
Type	string (read-only)
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code

```

```

2  ...
3  var nodeName = parentNode[0].firstChild.nodeName;
4  ...
5  //Add additional code

```

Node.nodeType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The type of node as an enum. For all possible values of this property, see xml.NodeType .
Type	xml.NodeType
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var nodeType = parentNode[0].firstChild.nodeType;
4  ...
5  //Add additional code

```

Node.nodeValue

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The value of a node, depending on its type. If the value is null, setting this value has no effect.
Type	string
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1  //Add additional code
2  ...
3  var nodeValue = parentNode[0].firstChild.nodeValue;

```

```

4 | ...
5 | //Add additional code

```

Node.ownerDocument

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The root element for a node as a xml.Document object. Use this object to create new nodes with Document.createElement(options) or Document.createElementNS(options) .
Type	xml.Document
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var doc = parentNode[0].ownerDocument;
4 | ...
5 | //Add additional code

```

Node.parentNode

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The parent node of a node. All node types, except xml.Attr , xml.Document , DocumentFragment , Entity , and Notation can have a parent node. See xml.NodeType for possible node types.
Type	xml.Node
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code ...
2 | var nodevalue = parentNode[0].lastChild.parentNode.textContent;
3 | ...
4 | //Add additional code

```

Node.prefix

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The namespace prefix of the node, or null if the node does not have a namespace. If the value is null, setting it has no effect, including read-only node types.
Type	string
Module	N/xml Module
Since	2015.2

Errors

Error Code	Message	Thrown If
SSS_XML_DOM_EXCEPTION	NAMESPACE_ERR: An attempt is made to create or change an object in a way which is incorrect with regard to namespaces.	Cannot edit the node prefix.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var namespacePrefix = parentNode[0].firstChild.prefix;
4 ...
5 //Add additional code

```

Node.previousSibling

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The previous node in a node list or null if the current node is the first node.
Type	xml.Node
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var nodeValue = parentNode[0].lastChild.previousSibling.textContent;

```

```

4 | ...
5 | //Add additional code

```

Node.textContent

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The textual content of a node and its descendants. If the value is null, then setting it has no effect. If you set this value, any child nodes are removed and replaced by a single text node with this string as a value.
Type	string
Module	N/xml Module
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var nodeValue = parentNode[0].firstChild.textContent;
4 | ...
5 | //Add additional code

```

xml.Parser

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the functionality used by NetSuite to parse an XML document.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/xml Module
Methods and Properties	Parser Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var parserObj = xml.Parser;

```

```

4 | ...
5 | //Add additional code

```

Parser.fromString(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Parses a String into a W3C XML document object. This API is useful if you want to query a structured XML document more effectively using either the Document API or NetSuite built-in XPath functions.  Note: You can also use this method to validate your XML. If you pass a malformed string in as the options.text argument, Parser.fromString returns an SSS_XML_DOM_EXCEPTION error.
Returns	xml.Document
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types.
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.text	string	required	String being converted to an xml.Document .

Errors

Error Code	Thrown If
SSS_XML_DOM_EXCEPTION	The input XML string is malformed.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var xmlDocument = xml.Parser.fromString({
4 |     text : xmlStringContent
5 | });
6 | ...
7 | //Add additional code

```


Parser.toString(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Converts (serializes) an xml.Document object into a string. This API is useful, for example, if you want to serialize and store an xml.Document in a custom field.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.document	xml.Document	required	XML document being serialized.

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var xmlStringContent = xml.Parser.toString({
4     document : xmlDocument
5 });
6 ...
7 //Add additional code

```

xml.XPath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Encapsulates the functionality to run XPath expressions.
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Module	N/xml Module
Methods and Properties	XPath Object Members
Since	2015.2

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```
1 //Add additional code
2 ...
3 var xpath = xml.XPath;
4 ...
5 //Add additional code
```

XPath.select(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Selects an array of nodes from an XML that match an XPath expression.
Returns	xml.Node[]
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters



Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.node	xml.Node	required	XML node being queried.
options.xpath	string	required	XPath expression used to query node.

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

In the following code sample, a single node is selected based on the xpath and node parameters. The type of node being selected is an XML document. The xpath parameter matches to the `//book` expression to locate the node. For more general information about xpath, see [XPath Tutorial](#).

```
1 //Add additional code
2 ...
3 var bookNode = xml.XPath.select({
4   node : xml.Document,
5   xpath : '//book'
6 });
7 ...
8 //Add additional code
```

xml.escape(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Prepares a string for use in XML by escaping XML markup, such as angle brackets, quotation marks, and ampersands.
Returns	string
Supported Script Types	Client and server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.xmlText	string	required	String being escaped.	2015.2

Syntax

Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 var xmlEscapedDocument = xml.escape({
4     xmlText : xmlFileContent
5 });
6 ...
7 //Add additional code

```

xml.validate(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Validates an XML document against an XML Schema (XSD).  Important: This method only validates XML Schema (XSD); validation of other XML schema languages is not supported. The XML document must be passed as an xml.Document object. The location of the source XML Document does not matter; the validation is performed with the Document object stored in memory. The XSD must be stored in the File Cabinet.
---------------------------	--

Returns	void
Supported Script Types	Server scripts For more information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Module	N/xml Module
Since	2015.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.xml	xml.Document	required	The xml.Document object to validate.	2015.2
options.xsdFilePathOrId	number string	required	The file ID or path to the XSD in the File Cabinet to validate the XML document against.	2015.2
options.importFolderPathOrId	number string	optional	The folder ID or path to a folder in the File Cabinet containing additional XSD schemas which are imported by the parent XSD.	2015.2

Errors

Error Code	Thrown If
SSS_XML_DOES_NOT_CONFORM_TO_SCHEMA	The provided XML is invalid for the provided schema.
SSS_INVALID_XML_SCHEMA_OR_DEPENDENCY	Schema is an incorrectly structured XSD or the dependent schema cannot be found.
ILLEGAL_REQUEST_FOR_A_FILE_THAT_ISNT_DOWNLOADABLE	The logged in user doesn't have permission to access the file referenced by the options.xsdFilePathOrId property.

Syntax

 **Important:** The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 //Add additional code
2 ...
3 xml.validate({
4   xml : xmlDocument,
5   xsdFilePathOrId : 'SuiteScripts/schema_parent.xsd',
6   importFolderPathOrId : 'SuiteScripts/'
7 });
8 ...

```

9 | `//Add additional code`

xml.NodeType

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Enum Description	Holds the string values for the supported node types. Use this enum to determine the type of a node in an XML document and to set the Node.nodeType property.  Note: Enum values are constants and therefore read-only.
Module	N/xml Module
Sibling Module Members	N/xml Module Members
Since	2015.2

Values

Value
ATTRIBUTE_NODE
CDATA_SECTION_NODE
COMMENT_NODE
DOCUMENT_FRAGMENT_NODE
DOCUMENT_NODE
DOCUMENT_TYPE_NODE
ELEMENT_NODE
ENTITY_NODE
ENTITY_REFERENCE_NODE
NOTATION_NODE
PROCESSING_INSTRUCTION_NODE
TEXT_NODE

Syntax



Important: The following code sample shows the syntax for this member. It is not a functional example. For a complete script example, see [N/xml Module Script Samples](#).

```

1 | //Add additional code
2 | ...
3 | var DocType = xmlDocument.nodeType; \\ returns DOCUMENT_NODE
4 | ...

```

```
5 | //Add additional code
```